

计算机硬件从零到整机

从电、二极管、逻辑门和时钟到最小 CPU 与整机硬件

CPU Books

本书沿着器件、电平、逻辑、状态、时钟、算术、数据通路、控制器、最小 CPU、主存、总线和 I/O 的顺序，把一台计算机从最小可理解部件推导出来。

前言

这本书从硬件本身开始。全书首先建立底层约定：一根线上连续变化的电压怎样被稳定解释成 0 和 1；一个器件怎样让电流更容易沿某个方向通过；一个受控开关怎样让一个信号决定另一条电流路径是否导通。只有这些约定成立，后续的逻辑门、寄存器、ALU、CPU 和整机接口才具有可靠含义。

本书按照两部分六章展开。第一章把电平、器件、开关、逻辑门和组合电路放在同一条链路中讨论，说明 bit 如何从物理电平进入可计算的逻辑网络。第二章集中讨论状态、时钟与同步边界，说明硬件如何保存过去，并在清楚的时钟边界提交下一值。

第三章讨论数值解释、算术电路、ALU、数据通路和控制器。它把固定宽度 bit、补码、标志位、选择器、寄存器阵列和控制 trace（执行轨迹）统一起来，使读者能够从旧状态走读到新状态。第四章把这些部件合成为最小 CPU，集中说明 PC、IR、控制字、数据通路和指令 trace。第五章再把 CPU 接到寄存器、cache、TLB、DRAM、MMIO、DMA、SSD/硬盘、网络和其他 I/O 队列，说明 CPU 之外的硬件如何形成整机吞吐和可见性边界。第六章借助 6502、Z80、8051、8086、80386、80486/Pentium、PC 主板、显卡和现代 SoC 观察真实处理器与整机平台的演进。

这种组织方式刻意避免把 CPU 作为孤立名词提前给出。CPU 在本书中是前述硬件部件按同步边界组织后的结果：PC 保存下一条编码地址，译码逻辑产生控制线，数据通路执行计算和写回，外部接口把处理器连接到主存与设备。读完整本书后，读者应能把一台机器从电平、开关、门电路、组合逻辑、时序逻辑、运算部件、数据通路、控制器一路连到整机。

本书采用接近 CSAPP (Computer Systems: A Programmer's Perspective) 等经典系统教材的写法：每讲一个抽象，都同时说明它在机器层的对应实现。读逻辑门时同时对照真值表和开关路径；读组合逻辑时同时分析函数、延迟和毛刺；读寄存器时同时观察 bit 和采样边界；读 CPU 时同时跟踪指令、控制信号和数据通路 trace。章节末尾的“经典教材标注”不是附加材料，而是用来检验读者是否已经把抽象概念与机器行为联系起来。

这不是一本要求读者实际焊板、购买示波器或搭完整实验平台的书。文中的“实验”和“项目”主要指纸面推演、结构图、波形阅读、规格分析、故障定位和运行记录整理；如果读者有 FPGA、仿真器、逻辑分析仪或真实开发板，可以把这些任务落实到真实硬件上，但没有硬件也应能完成大部分训练。读者真正要掌握的不是某块板的接线，而是能把一次访问、一次计算、一次启动、一次中断或一次 DMA 写成可解释的状态边界。

扩写目录与路线图

当前书稿已经有一条从电平、逻辑门、状态、ALU、最小 CPU 到整机平台的主线，但它仍然更接近讲义，而不是完整教材。后续扩写不应继续在六章末尾堆专题，而应按经典教材的粒度拆成更多章节：每章有明确问题、结构图、案例、习题和纸面项目。下面是后续扩写采用的目标目录。

章	章节名	需要讲清楚的问题	章末项目
1	数字抽象与电气基础	电压、电流、噪声裕量、阈值、负载、电平标准、受控电流路径如何形成可靠 bit。	电平约定图、噪声裕量题
2	组合逻辑与布尔代数	真值表、布尔代数、NAND/NOR 完备性、译码器、编码器、mux、毛刺和传播延迟。	逻辑化简与毛刺案例
3	时序逻辑、寄存器和状态机	锁存器、触发器、setup/hold、复位、同步器、计数器、移位寄存器和 FSM。	状态机分析表
4	RTL 与硬件描述语言阅读	如何读 Verilog/SystemVerilog：组合块、时序块、非阻塞赋值、综合边界、testbench 思路。	RTL 走读题
5	数值表示与算术电路	无符号、补码、定点、浮点、BCD、加法器、减法器、乘除法、多精度和标志位。	边界向量表
6	ISA、汇编和机器码	指令格式、寄存器、寻址模式、立即数、条件码、调用约定、栈帧和机器码编码。	汇编到机器码 trace
7	数据通路与控制	寄存器堆、ALU、mux、控制字、硬连线控制、微程序控制和数据通路阶段。	控制信号表
8	单周期与多周期 CPU	IF/ID/EX/MEM/WB 的状态边界，单周期关键路径，多周期复用硬件和等待状态。	访存周期图
9	流水线、冒险和异常	结构冒险、数据冒险、控制冒险、转发、停顿、冲刷、精确异常和中断入口。	五级流水线题
10	cache 与存储层次	locality、cache line、tag/index/offset、替换、写策略、多级 cache 和一致性入门。	cache 命中/miss 推演
11	虚拟内存、TLB 和页表	虚拟地址、页表、TLB、权限、缺页、页错误、IOMMU 和地址空间隔离。	地址转换案例
12	DRAM/DDR 与内存控制器	SRAM 与 DRAM 差异、bank、row buffer、refresh、ECC、训练、调度和带宽。	DRAM 访问时序图
13	总线、互连和 PCIe	片选、valid/ready、AXI/APB 概念、PCIe 配置空间、BAR、TLP、MSI 和 AER。	PCIe 枚举路径

14	MMIO、中断、DMA 和 IOMMU	设备寄存器位语义、中断控制器、DMA 描述符、completion、cache 可见性和权限。	设备事务 trace
15	SSD、硬盘、网络与外设队列	块设备、NVMe、SATA、硬盘、网卡、描述符环、错误恢复、吞吐和尾延迟。	NVMe/网卡队列题
16	GPU、VRAM 和显示系统	GPU 执行模型、SIMD/SIMT、shader、VRAM、命令缓冲、fence、显示扫描和帧缓冲。	GPU 命令路径图
17	主板、固件、启动和平台管理	VRM、时钟、复位、UEFI/BIOS、PCH、DDR 训练、ACPI、EC、风扇和传感器。	启动 bring-up 日志
18	链接、装载和系统软件边界	目标文件、符号、重定位、ELF、动态链接、loader、系统调用和异常控制流。	ELF/loader 纸面项目
19	性能分析与定量方法	latency、throughput、CPI、带宽、Amdahl 定律、roofline、cache miss 代价和瓶颈定位。	性能分析题
20	综合纸面项目与故障案例集	从开机、读文件、执行代码、访问设备、GPU 显示到错误恢复的全链路 trace。	整机规格与排错报告

拆分原则

旧六章不会被简单复制成二十章。拆分时按问题边界移动内容：第一章和第二章保留电气与组合逻辑；当前状态机内容拆到第 3 章；数值和 ALU 拆到第 5、7 章；最小 CPU 拆到第 6、8、9 章；主存、cache、DMA、SSD、PCIe、GPU 和主板分别进入第 10 到第 17 章；目标文件、链接、装载和系统软件边界补成第 18 章；性能分析和综合项目补成第 19、20 章。

当前六章与规划章的对应关系如下，读者可以按右列判断某一主题会先在哪里出现、又会在哪里被完整展开：

当前章	拆入规划章
第 1 章电平、开关、逻辑门与组合电路	第 1、2 章（数字抽象与电气基础、组合逻辑与布尔代数）
第 2 章状态、时钟与同步边界	第 3 章（时序逻辑、寄存器和状态机）
第 3 章数值、ALU、数据通路与控制器	第 5、7 章（数值表示与算术电路、数据通路与控制器）
第 4 章最小 CPU 与控制 trace	第 6、8、9 章（ISA 与汇编、单周期与多周期、流水线与冒险）
第 5 章主存、总线、I/O 与 DMA	第 10-15 章（cache、虚拟内存、DRAM、总线互连、MMIO/DMA、外设队列）
第 6 章经典芯片、启动与平台演进	第 16、17 章（GPU 与显示、主板固件与平台管理），并吸收第 13 章 PCIe 内容
第 7 章动手搭一台 8-bit 教学计算机	综合实践章：把第 1-3、6-9 章的规划内容落到可搭建的最小系统
第 8 章电源、时钟与信号完整性	第 1、3 章（电气基础、时序逻辑）的工程延伸，补充电源分配与高速信号内容
第 9 章 RTL 与 Verilog 阅读入门	第 4 章（RTL 与硬件描述语言阅读）

第 10 章性能分析与定量方法	第 19 章（性能分析与定量方法）
第 11 章外设与队列：SSD、硬盘、网卡	第 15 章（SSD、硬盘、网络与外设队列）
第 12 章 GPU 与显示系统	第 16 章（GPU、VRAM 和显示系统）
第 13 章链接、装载和系统软件边界	第 18 章（链接、装载和系统软件边界）
当前尚未覆盖	第 20 章综合项目

每章最低质量标准

每一章至少包含五类内容。第一，开头给出本章要解决的硬件问题，避免堆名词。第二，给出一到三张能说明机制的图，图必须区分数据路径、控制路径、状态边界和错误路径。第三，至少包含一个真实或接近真实的案例，例如 8086 总线、RISC-V 指令、cache miss、NVMe 队列或 GPU fence。第四，章末习题要覆盖概念题、trace 题、边界题和故障定位题。第五，给出参考解答和答题要点，让读者知道答案需要落实到哪些机器细节。

扩写顺序

后续优先扩第 6 到第 15 章，因为这些内容决定本书能不能从“硬件入门讲义”变成“计算机底层教材”。建议顺序是：先补 ISA/汇编和数据通路，再补流水线和异常，然后补 cache/TLB/虚拟内存，接着补 DRAM、PCIe、MMIO/DMA、SSD/网络，最后补 GPU、主板启动、链接装载和性能分析。这样每一轮扩写都能增加一个完整知识块，而不是只增加页数。

目录

扩写目录与路线图	3
第一部分 从电平到数据通路	9
第 1 章 电平、开关、逻辑门与组合电路	11
一、从一根线得到可靠 bit	12
二、从器件到受控电流路径	22
三、从路径表得到逻辑门	33
四、门网络构成组合逻辑	40
五、延迟、负载、毛刺与检查	56
第 2 章 状态、时钟与同步边界	72
一、反馈、锁存器与保存一个 bit	72
二、触发器、时钟周期与采样窗口	77
三、关键路径、复位与跨边界输入	83
四、寄存器、计数器与移位结构	89
五、状态机把硬件动作切成阶段	98
第 3 章 数值、ALU、数据通路与控制	124
一、固定宽度 bit 的数值解释	124
二、从加法器到可复用 ALU	136
三、进位、移位、比较与乘法的硬件取舍	140
四、数据通路让值从旧状态走到新状态	147
五、控制器输出一组一致的控制线	151
第二部分 最小 CPU 与整机硬件	171
第 4 章 最小 CPU 与控制 trace	173
一、从动作编码到控制字组织	173
二、极小 ISA 与可接线数据通路	180
三、可见状态、条件码与调用约定	184
四、异常、中断与精确异常	191
五、整机实验、调试顺序与案例 trace	195
第 5 章 主存、总线、I/O 与 DMA	208
一、主存不是抽象数组	208
二、地址译码定义整机地图	216
三、MMIO 把设备寄存器放进地址空间	218
四、中断和 DMA 把外部事件接回 CPU	223
五、从教学总线到现代互连	226
第 6 章 经典芯片、启动与平台演进	249
一、早期微处理器：从 6502、Z80 到 8086	249
二、80386：32-bit、保护模式与地址转换	261
三、整机启动与经典芯片后的演进	267

四、从目标文件、固件到可启动机器	276
第 7 章 动手搭一台 8-bit 教学计算机	294
一、总体架构：总线、时钟与复位	295
二、寄存器与 ALU 模块	300
三、内存与输出模块	305
四、控制单元：指令译码与微码	315
五、编程、调试与扩展	325
 第三部分 工程边界：电源、时钟与信号	 340
第 8 章 电源、时钟与信号完整性	342
一、电源分配：稳压、去耦与电源完整性	342
二、时钟树：分配、偏斜与抖动	347
三、信号完整性：传输线、反射与端接	352
四、串扰、回流与电磁兼容入门	357
五、测量与工程实践	361
 第四部分 描述与度量	 371
第 9 章 RTL 与 Verilog 阅读入门	373
一、从电路图到 RTL：两种描述同一个硬件	373
二、组合逻辑的 Verilog 写法	381
三、时序逻辑的 Verilog 写法	389
四、从 Verilog 读回硬件：综合边界	397
五、testbench 与仿真阅读	405
第 10 章 性能分析与定量方法	415
一、时延、吞吐与利用率	415
二、CPI 与指令级性能	421
三、Amdahl 定律与加速极限	425
四、带宽、roofline 与瓶颈定位	432
五、性能测量与案例	440
 第五部分 外设与加速	 451
第 11 章 外设与队列：SSD、硬盘、网卡	453
一、块设备与文件系统的硬件接口	453
二、SSD 与闪存的硬件特性	458
三、机械硬盘与访问调度	462
四、网卡与网络队列	468
五、外设队列综合案例与尾延迟	474
第 12 章 GPU 与显示系统	482
一、GPU 为什么长得不像 CPU	482
二、SIMD/SIMT 执行模型	486
三、显存系统：VRAM、HBM 与数据进出	491
四、命令路径：命令环、fence 与显示扫描	497
五、从图形管线到通用计算	502

第 13 章 链接、装载和系统软件边界	510
一、目标文件与符号	510
二、链接：重定位与地址形成	516
三、装载与动态链接	520
四、系统调用与异常控制流	529
五、从源码到进程的完整案例	537

从电平到数据通路

本部分导读

本部分集中回答一个连续问题：怎样从可控电流路径得到稳定的 0 和 1，再把多个 0/1 组合成可计算的逻辑网络；怎样让硬件保存状态、按时钟边界推进；怎样把数值解释、ALU、寄存器阵列、数据通路和控制信号组织成可重复的硬件动作。

电平、开关、逻辑门与组合电路

先从一个会出问题的场景开始。一块控制板上有一个启动按钮：按下时，信号线被接到电源；松开时，信号线哪里都不接。程序读者会以为松开就是 0，但真实电路不是这样——那根悬空的线可能停在旧电压上，也可能被旁边的信号线、人体或漏电流慢慢推走，接收端今天读到 0，明天读到 1。本章就从这类事故出发，一层层回答：一根真实导线上的电压，怎样变成 CPU 可以信赖的 bit 计算。

这条链路分三步展开：连续变化的电压先被解释成稳定的 0/1；二极管和受控开关提供可预测的电流路径；开关网络形成 NOT、NAND、NOR、XOR 等门；门再组合成判断器、加法器、选择器和译码器。读者应始终沿着同一条线索阅读本章：一个逻辑值必须能追溯到电压区间，一个门必须能追溯到上拉/下拉路径，一个组合输出必须能追溯到完整真值表、延迟和负载边界。这些名词现在听不懂没有关系，本章会逐个把它们变成可以检查的东西。

本章不展开完整的半导体物理，也不把内容处理成布尔代数速查表。它关注一个核心问题：一根真实导线上的电压，怎样逐步成为 CPU 可使用的 bit 计算。后续所有寄存器、ALU、数据通路、控制器、内存和 I/O，最终都必须满足这里建立的底层约定。

为避免把内容讲成分散概念，本章使用一个贯穿案例：一块简化的安全启动控制板。它接收启动按钮、保护盖、急停、温度报警和电流报警，输出启动允许、电机使能和故障灯。这个小系统不涉及软件，也不需要处理器；它只需要把输入电平可靠地解释成 bit，再用门网络形成输出。正因为它足够小，读者可以逐项检查每根线的默认状态、每个门的路径、每个输出的电平和每条组合路径的延迟。

信号	含义	需要检查的硬件问题
start_button	人按下启动按钮	松开时是否悬空，按下时是否可靠为 1
cover_closed	保护盖已经闭合	断线或接触不良时应回到安全默认值
emergency_ok	急停没有被按下	信号名为正逻辑，但实物开关可能采用低有效接法
temp_alarm	温度报警	任一报警都应阻止启动并点亮故障灯
current_alarm	电流报警	报警输入不能依赖偶然电平或悬空节点
allow_start	允许启动	必须由所有安全条件共同决定
motor_enable_cmd	电机使能命令	只能由经过检查确认的组合结果产生
fault_lamp	故障灯	需要汇总多个故障信号

表中的每个信号都不是孤立变量，而是一项硬件约定。输入信号要说明来源、默认值、有效极性、断线后果和进入逻辑前的调理方式；输出信号要说明由哪些条件决定、是否允许毛刺、何时被后级使用，以及是否需要驱动执行机构。后文反复回到这张表，是为了让抽象逻辑始终落在可检查的电气边界上。

核心理想

数字硬件的第一层抽象不是门，而是**带余量的电平**；数字硬件的第一层实现不是公式，而是**受控电流路径**。



本章所有例子都沿这条链路逐步检查：先确认电平可靠，再确认路径不悬空、不冲突，最后确认组合输出在后级采样前已经稳定。

图示：从电平到组合逻辑的抽象链。箭头表示“后一个抽象必须由前一个物理事实支撑”。

本章还提前引入几个芯片参照。它们暂时不作为完整处理器来讲，而是作为本章概念的落点：7400 系列把门电路封装成标准逻辑芯片，4000 系列 CMOS 展示互补路径带来的低静态功耗；8086 把门、寄存器、选择器、译码器和总线控制组合成早期完整 CPU；80386 用更多晶体管换来更宽数据通路和更复杂的系统控制；现代 CPU 则靠器件、布局、体系结构和存储层次，把同样的逻辑函数做得更快、更密、更低时延。真正芯片的数据手册还会给出**输入阈值、输出电流、传播延迟和扇出限制**——门不是纸面符号，而是带电气约定的器件。把这些案例提前放在这里，是为了避免读者把第一章误解成“只讲离散门电路”；本章讲的是所有芯片共同依赖的底层约定。

因此，本章的阅读重点不在于快速记住某个门符号，而在于形成一种固定检查法：先确认信号能否成为可靠 bit，再确认器件路径能否稳定地产生这个 bit，最后确认多个 bit 组成的组合网络能否在后级需要之前稳定下来。这个检查法会直接延伸到第二章的触发器、寄存器和状态机。

本章的学习范围与目标 本章保留“从电压到组合网络”的完整链路，是为了让读者先看到底层各层之间的支撑关系，而不是把电气、门级和系统边界割裂为互不相干的名词。阅读时可以把本章看成三层基础：第一层回答一根线怎样成为可靠 bit；第二层回答器件怎样形成受控电流路径；第三层回答多个 bit 怎样组合成可靠的逻辑结果。

这意味着第一章不追求把所有细节一次讲完，而是要求每个抽象都能追溯到具体的物理事实。后面讨论 cache 命中、PCIe 中断、GPU 显存访问或 SSD 控制器状态机时，仍然会回到同一问题：这个信号在电气上是否可靠，在时序上是否被正确采样，在逻辑上是否有完整定义。

读完本章后，读者至少应能掌握四类能力。第一，能为一个板外输入写出默认电平、保护边界、阈值整形和内部语义信号，而不是直接把外部触点当作可靠 bit。第二，能把一个二输入门写成真值表和上拉/下拉路径表，而不是只画 $Y=A \text{ AND } B$ 这类公式。第三，能把一个组合需求分解为门、选择器、译码器、编码器或比较器，并说明最长路径、扇出、毛刺和危险输出处理，而不是只证明稳定真值表正确。第四，能把同一套概念映射到标准逻辑芯片、早期微处理器和现代 CPU 的低时延路径上，而不是只罗列芯片名称。

一、从一根线得到可靠 bit

电路里说“一根线是 1”，完整含义不是“线上写着数字 1”，而是“这个节点相对于参考地的电压落在接收端承认的高电平区间内”。节点可以理解成多条金属连接汇合成的同一个电气位置；参考地是整块电路共同约定的 0V 参考点；电压是两个点之间的差值。数字电路里说某根信号线是 0V、3.3V 或 5V，实际都是在说它相对于地的电压。

电流说明电荷正在沿通路流动。若输出节点被一条通路接向电源，它可能被拉到高电平；若被一条通路接向地，它可能被拉到低电平。若两条通路都断开，节点不会自动变成 0，而是可能暂时停在旧电压附近，再被漏电流、干扰、后级输入或测

量探针慢慢推走。若上拉和下拉两条强通路同时打开，电源到地之间形成冲突电流，电平不可靠，还会增加功耗和发热。

需要先区分导线和程序变量。变量没有被赋值时，语言可以规定默认值；导线没有被驱动时，只剩物理过程。它可能因为寄生电容保存旧电压，表现为保留上一次状态；也可能被旁边切换的信号线、人体接近、探针输入阻抗或后级漏电流轻微影响。调试数字硬件时，不能把这种偶然稳定误认为设计正确。

以按钮输入为例。假设按钮按下表示“请求发生”。一种不完整的接法是按钮一端接信号线，另一端接电源：

```
button pressed -> signal connected to VCC
button released -> signal connected to nothing
```

按下时，信号线被电源路径拉高，该部分行为符合预期。松开时，信号线并没有被明确拉低，它只是断开了。这个节点可能保留先前电荷，也可能被旁边导线耦合出一定电压，还可能被后级输入漏电流拖向某个不确定位置。于是接收端可能读到 1、0 或过渡电压。问题的根源不是程序错误，而是电路没有为节点规定明确归宿。

正确的第一层想法是：任何要被后级判断的输出节点，都必须在每一种输入情况下被明确地拉向高电平或低电平。按钮松开时需要一条弱下拉路径把信号线拉向地；按钮按下时，按钮路径把它拉向电源。弱下拉不是为了在按钮按下时和按钮争夺电源，而是为了在按钮松开时给节点一个默认归宿。它太弱，节点下降慢、抗干扰差；它太强，按钮按下时浪费电流，甚至让高电平不够高。



图示：同一个按钮，两种结局。左图松开按钮后节点没有任何归宿，只能听天由命；右图多了一条弱下拉电阻：松开时它把节点拉向地（可靠 0），按下时按钮的强路径“赢过”弱下拉，把节点拉向电源（可靠 1）。

场景	高电平路径	低电平路径	结论
按下	按钮把节点接向电源	弱下拉仍存在	可靠高
松开	高电平路径断开	弱下拉把节点接向地	可靠低
接触抖动	高电平路径快速断续	弱下拉持续存在	需要消抖
导线断裂	按钮路径永远断开	若下拉还在，则为低	有默认值
没有下拉	高电平路径断开	低电平路径也断开	悬空

该表给出硬件读法的雏形。它不只问逻辑上应该是 0 还是 1，还问路径在哪里、谁驱动谁、异常时会回到哪个默认状态。后面讲复位、总线仲裁和设备中断时，同样要用这种检查法：每个状态都要有明确来源，每个默认值都要有物理路径支持。

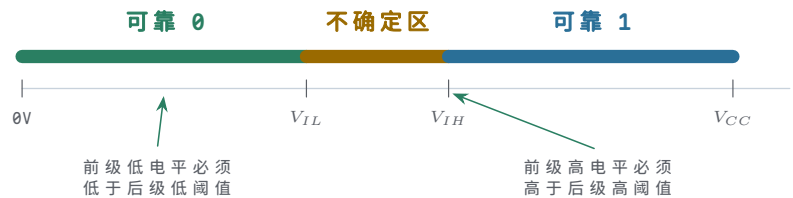
把按钮放回安全启动控制板中，可以得到更明确的安全要求。启动按钮松开时，系统不应因为线缆悬空而误认为有人按下；保护盖传感器断线时，系统不应默认保护盖闭合；报警线断开时，系统不应静默地继续允许运行。因此，“默认值”不是随意选择，而是安全策略的一部分。对人机输入而言，常见策略是让未动作、断线或未知状态对应不启动、不使能或报警一侧；对内部数据通路而言，策略可能不同，但仍必须明确。

输入线	不完整设计的风险	更稳妥的默认方向
启动按钮	松开时悬空，可能误触发	默认不启动，按下才变为启动请求

保护盖检测	断线时可能被误读为闭合	默认未闭合，需要传感器明确证明闭合
急停链路	接触不良时可能丢失急停信号	默认不允许运行，需要链路明确证明急停正常
报警输入	断线时可能丢失报警	根据安全需求选择默认报警或要求上层诊断

电平也不能理解成两个数学点。假设一块小电路用 5V 供电。线上的真实电压可能是 0V、0.17V、1.3V、2.6V、4.8V，也可能在切换过程中从 0V 慢慢升到 5V。硬件不能要求每一级都精确读出这个连续数值。更可靠的做法是规定两个稳定区间：低到足够接近地电位时解释为 0，高到足够接近电源电位时解释为 1，中间一段不作为长期稳定状态。

电压范围	逻辑解释	设计意义
接近地电位	逻辑 0	小噪声不会轻易把它推成 1
中间区域	不作为稳定状态	切换可以经过，但不能长期停留
接近电源电位	逻辑 1	小噪声不会轻易把它拉成 0



图示：电压不是两个数学点，而是带噪声余量的区间。中间区域允许切换经过，不允许作为稳定工作点。

可以用四个电压指标描述两级电路之间的约定。 V_{OH} 是输出端保证的高电平下限， V_{OL} 是输出端保证的低电平上限； V_{IH} 是输入端承认为高的最低电压， V_{IL} 是输入端承认为低的最高电压。若一个电路保证 V_{OH} 高于下一级需要的 V_{IH} ，中间差额就是高电平噪声余量；若 V_{OL} 低于下一级允许的 V_{IL} ，中间差额就是低电平噪声余量。

符号	含义	读法
V_{OH}	输出保证为高时的最差电压	前一级说：我给出的 1 至少高到这里
V_{IH}	输入承认为高的最低电压	后一级说：至少给到这里，我才认作 1
V_{OL}	输出保证为低时的最差电压	前一级说：我给出的 0 至多高到这里
V_{IL}	输入承认为低的最高电压	后一级说：低到这里以下，我才认作 0

例如，某一级输出保证高电平至少为 2.4V，后一级输入只要求达到 2.0V 就承认为高，那么高电平噪声余量为 0.4V。若同一输出在最坏负载、温度和工艺条件下只能到 1.9V，逻辑表达式仍然可能写着“输出为 1”，但后一级不会可靠承认这个 1。低电平也同理：前级输出低电平的最坏值必须低于后一级允许的低电平上限，并留出余量。

项目	前级保证	后级要求	结论
高电平	$V_{OH} \geq 2.4V$	$V_{IH} \geq 2.0V$	余量 0.4V
低电平	$V_{OL} \leq 0.4V$	$V_{IL} \leq 0.8V$	余量 0.4V

失败情形	实测高电平仅 1.9V	后级需要 2.0V	不能可靠读作 1
------	----------------	-----------	----------

把这个例子进一步写成定量算式，会更接近数据手册读法。假设某输出在最坏负载下保证 $V_{OH} = 3.0V$ 、 $V_{OL} = 0.25V$ ，后级输入要求 $V_{IH} = 2.1V$ 、 $V_{IL} = 0.7V$ ，则高电平噪声余量为 $3.0V - 2.1V = 0.9V$ ，低电平噪声余量为 $0.7V - 0.25V = 0.45V$ 。若板级串扰、地弹或线缆压降可能带来约 $0.3V$ 扰动，这个连接仍有余量；若某条长线在最坏情况下可能叠加 $0.6V$ 扰动，低电平余量已经不足，必须改变驱动、布线、阈值或输入调理方案。

检查项	算式	结果	解释
高电平余量	$3.0V - 2.1V$	0.9V	前级高电平高于后级高阈值
低电平余量	$0.7V - 0.25V$	0.45V	前级低电平低于后级低阈值
0.3V 干扰	余量仍为正	可接受	仍需用最坏条件确认
0.6V 干扰	低余量不足	不合格	可能把低电平推入不确定区

数字抽象并不是忽略模拟世界，而是把模拟世界约束在约定之内。只要所有连接都满足这些最差条件，后续讨论 bit、真值表和指令编码才有可靠基础。若这些条件不满足，系统层面可能出现间歇性现象：同一块板有时能启动、有时不能；按键有时触发两次；某条总线在高温下出错；更换更长排线后原本稳定的电路开始误判。这些现象不一定来自逻辑表达式错误，也可能来自电平约定被破坏。

输出还必须能驱动下一级。一个节点如果只能在空载时看起来像 1，一接上后级就被拖低，它不是可靠数字输出。后级输入、板级走线、封装和探针都有寄生电容。把一个节点从 0 拉到 1，本质上是在给这个电容充电；把它从 1 拉到 0，本质上是在放电。驱动路径越弱，等效电阻越大；负载越多，等效电容越大；上升和下降越慢。

```
larger load capacitance -> slower edge
weaker driver           -> slower edge
slower edge             -> less time margin before sampling
```

所以示波器上看到的真实信号不是理想直角。电压边沿有斜率，可能有过冲、振铃、平台和噪声。逻辑 0/1 的背后是一条随时间变化的电压曲线，接收端只是在某个时刻把它归类为 0 或 1。若这个时刻到来之前电压还没有进入可靠区间，最终会稳定也不够。

因此，一个 bit 被称为可靠，至少需要同时满足四项条件。第一，有明确驱动路径，不能悬空、保留旧值或被干扰改变；第二，电平进入接收端承认的稳定区间并留有噪声余量，最坏输出也要满足后级阈值；第三，驱动端能够带动实际负载，不能让边沿被电容拖慢、电压被拖低；第四，在后级采样或使用之前，信号已经稳定足够长时间——最终值正确但采样时刻错误，同样是失败。

外部输入还需要调理。按钮和机械触点会抖动，线缆会引入噪声，传感器输出可能是开漏或开集结构，板外信号还可能与板内电源域不同。第一章暂不展开完整接口电路，但必须建立原则：进入组合逻辑核心之前，外部输入应先变成带默认值、带驱动能力、带明确有效电平的内部信号。否则后面的真值表虽然正确，输入源本身却可能不可靠。

外部现象	进入组合逻辑前的处理	不处理的后果
按钮抖动	消抖或在后续同步边界过滤	一次按下可能产生多次请求

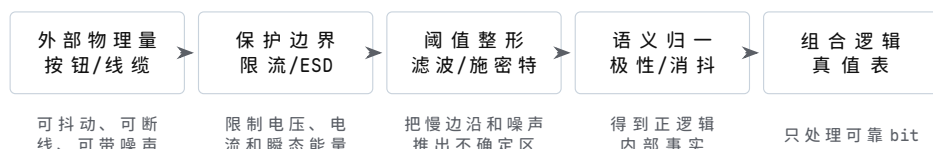
长线噪声	上拉/下拉、滤波、缓冲或屏蔽	误触发、边沿变慢、阈值附近摆动
开漏输出	外部上拉并限制总线负载	释放时没有可靠高电平
电源域不同	电平转换或隔离	后级阈值不匹配甚至损坏器件

板外输入还要有保护边界。按钮、传感器、线缆和连接器会把静电放电、误接电压、浪涌、长线耦合和地电位差带到芯片引脚附近。若把这些信号直接接入 CMOS 输入，逻辑上也许只是“读一个 0/1”，电气上却可能让输入保护二极管承受过大电流，或者让引脚在绝对最大额定值之外工作。保护电路的职责不是改变业务含义，而是在进入逻辑核心之前限制**电压、电流、带宽和边沿形态**。

保护措施	主要作用	设计时必须说明
串联电阻	限制异常电压下的输入电流，减小尖峰冲击	电阻不能大到让边沿过慢或阈值附近停留过久
RC 滤波	抑制低速按钮抖动和高频噪声	时间常数要匹配输入速度，不能吞掉有效脉冲
TVS 或 ESD 保护	为静电和瞬态浪涌提供旁路	器件选型要匹配工作电压、钳位电压和能量等级
输入钳位与限流	让引脚异常电压不直接伤害芯片内部结构	不能依赖片内保护长期承受外部错误连接
缓冲或施密特输入	恢复边沿、提供滞回、隔离前级负载	输出电平、传播延迟和驱动能力仍需逐项确认

因此，板外按钮或传感器不应被写成“直接连到门输入”。更严谨的写法是：连接器输入先经过默认上拉/下拉、限流、保护、滤波或施密特边界，再形成内部语义信号。这样即使后续表达式仍是 `allow_start = start_request AND covered AND emergency_ok AND no_fault`，读者也能看出 `start_request` 已经不是裸露引脚，而是经过电气边界整理后的可靠 bit。

从外部引脚到内部 bit 的五层边界 外部世界进入芯片内部之前，至少要经过五层边界。每一层都回答一个不同问题：连接器和线缆负责把物理事件带到板上；保护与限流负责让异常能量不直接伤害芯片；阈值整形负责把连续电压变成稳定 0/1；语义归一化负责把低有效、反相、断线默认和消抖结果整理成内部名字；后级逻辑只应依赖最后得到的内部语义信号。



图示：外部输入进入组合逻辑前的五层边界。图中的箭头不是简单连线，而是逐层收窄不确定性的过程。

这张图能避免两个常见错误。第一，不能把 `start_button_raw` 直接代入真值表，因为它还包含抖动、噪声、线缆和保护问题。第二，不能把保护电路当作逻辑表达式的一部分，因为保护电路的职责是限制物理风险，而不是决定业务语义。一个严谨的规格应分别写出 `start_button_raw`、`start_button_pin`、`start_button_clean` 和 `start_request`，并说明每个名字跨越了哪一层边界。

上拉电阻和 RC 滤波可以用一个数量级例题理解。若开漏信号采用 $4.7\text{k}\Omega$ 上拉到 3.3V ，低电平被拉下时的电流约为 $3.3\text{V}/4.7\text{k}\Omega \approx 0.7\text{mA}$ ，后级通常较容易承受；若同一节点总电容约 100pF ，则时间常数约为 $4.7\text{k}\Omega \times 100\text{pF} = 0.47\mu\text{s}$ ，低速按钮或报警线通常可以接受。若把上拉改成 $100\text{k}\Omega$ ，同样 100pF 下时间常数变成 $10\mu\text{s}$ ，边沿会明显变慢；若后级很快采样，或者输入没有施密特滞回，信号就可能在阈值附近停留太久。

参数选择	数量级结果	设计含义
4.7k Ω 上拉到 3.3V	拉低电流约 0.7mA	要确认开漏器件能可靠吸收该电流
4.7k Ω 与 100pF	时间常数约 0.47 μ s	适合低速输入，仍需看采样时刻
100k Ω 与 100pF	时间常数约 10 μ s	边沿显著变慢，抗噪声能力下降
RC 消抖 10ms	能过滤机械抖动	不适合需要微秒级响应的输入

这些数值不是固定推荐值，而是提醒读者把“能不能读到 1”拆成电流、边沿和时间三个问题。上拉过强会增加拉低电流，上拉过弱会让释放后的高电平上升太慢；RC 过小无法过滤抖动，RC 过大又会吞掉有效边沿。真正设计时应以器件数据手册、输入速度、阈值、功耗和安全响应时间共同决定。

上拉/下拉电阻的阻值权衡 “上拉取 10k Ω ”常被当作口诀背下来，但阻值其实是一笔可以算清的账。先看电阻太大的一端。后级输入并非完全不取电流，CMOS 输入存在微安量级以下的漏电流，而这股电流必须流过上拉电阻，在电阻两端形成压降 $V = IR$ ，于是高电平被拉低。以 5V 系统、10k Ω 上拉、输入漏电流 1 μ A 为例：压降为 $1\mu\text{A} \times 10\text{k}\Omega = 10\text{mV}$ ，实测高电平约 4.99V，几乎无损。若为了“省电”把上拉换成 1M Ω ，同样 1 μ A 就产生 1V 压降，高电平只剩 4.0V；若高温下漏电升到 10 μ A，压降将达到 10V 量级——节点根本到不了高电平。阻值越大，默认电平越怕漏电，噪声裕量越薄。

再看电阻太小的一端：按键或开漏器件把节点拉低时，上拉电阻直接从电源向地通电。若上拉只有 100 Ω ，按键按下期间持续流过 $5\text{V}/100\Omega = 50\text{mA}$ ，电阻上的功耗为 $I^2R = 0.05^2 \times 100 = 0.25\text{W}$ ，已超过常见 1/8W 贴片电阻的额定功率，按着不放就会发烫，拉低它的器件也必须灌得进这 50mA。换成 10k Ω ，同样动作只消耗 0.5mA，普通逻辑输出和开漏器件都能承受。

但阻值也不能无限大。默认电平靠电阻“拽住”，阻值越大节点越“轻”，同样的耦合电荷产生的电压扰动越大，抗干扰越差；同时电阻与节点电容构成 RC 充电回路，100k Ω 配 50pF 走线电容的时间常数已达 $100\text{k}\Omega \times 50\text{pF} = 5\mu\text{s}$ 。综合起来，阻值由灌/拉电流能力、噪声裕量、功耗三者权衡，常用范围在 1k Ω 到 100k Ω 之间：需要强抗干扰或较快边沿时偏向下端，电池供电、信号很慢时偏向上端。

情形	算式	结果	说明
10k Ω 上拉，漏电 1 μ A	$1\mu\text{A} \times 10\text{k}\Omega$	压降 10mV	5V 高电平约 4.99V，裕量几乎无损
1M Ω 上拉，漏电 1 μ A	$1\mu\text{A} \times 1\text{M}\Omega$	压降 1V	高电平只剩 4.0V，高温下更差
100 Ω 上拉，按键接地	$5\text{V}/100\Omega, I^2R$	50mA、0.25W	超过 1/8W 电阻额定，按着不放会发烫
10k Ω 上拉，按键接地	$5\text{V}/10\text{k}\Omega$	0.5mA	功耗可忽略，开漏器件也灌得起
100k Ω 上拉，节点 50pF	$100\text{k}\Omega \times 50\text{pF}$	$\tau = 5\mu\text{s}$	边沿明显变慢，抗扰动力下降

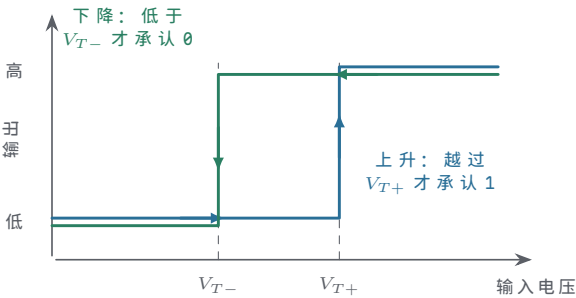
这张表也说明为什么“先看数据手册再定阻值”：输入漏电、开漏器件的灌电流能力和节点电容分别决定阻值的上限和下限，口诀只是在常见参数下的折中。

由此可见，书写硬件规格时应区分“原始外部信号”和“内部逻辑信号”。原始外部信号属于板外世界，可能带有抖动、噪声、线缆压降和接插件故障；内部逻辑信号属于组合逻辑世界，必须已经具有明确的 0/1 含义。若把二者混成一个名字，读者会误以为真值表直接约束了外部物理现象。

信号层次	典型名称	设计要求
------	------	------

外部触点	<code>start_button_raw</code>	允许抖动，但必须有默认路径和保护边界
输入调理后	<code>start_button_clean</code>	在逻辑核心看来只能是稳定 0 或稳定 1
语义信号	<code>start_request</code>	明确表示“本周期有启动请求”这一逻辑事实
安全汇总	<code>allow_start</code>	由全部安全条件共同决定，不能依赖原始触点

施密特触发输入是常见的边界处理方式。普通输入通常只有一个判断阈值附近的过渡区；当信号边沿很慢或带噪声时，电压可能在阈值附近反复穿越，导致接收端多次翻转。施密特触发器使用两个不同阈值：上升时必须超过较高阈值才承认为 1，下降时必须低于较低阈值才承认为 0。两个阈值之间的间隔称为滞回。滞回不是改变逻辑功能，而是让输入在噪声环境中更稳定地跨过边界。

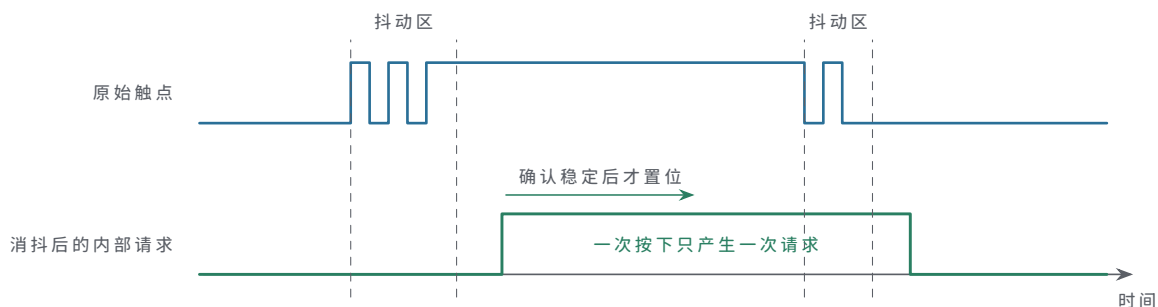


图示：施密特输入的传输特性曲线。普通输入只有一个模糊阈值，噪声会让输出在阈值附近反复横跳；施密特把“上升承认”和“下降承认”拉开成两个阈值，中间这段滞回区间就是噪声的容身之地——信号必须真正越过 V_{T+} 才算 1，真正跌回 V_{T-} 以下才算 0。

输入处理	适用情形	对可靠 bit 的贡献
上拉或下拉	按钮、开漏输出、断线默认值	给悬空节点明确归宿
限流和保护	板外连接、误插、瞬态干扰	避免异常电压直接进入逻辑核心
滤波或消抖	机械触点、低速传感器	把多次物理跳变收敛为一次逻辑事件
施密特触发	慢边沿、噪声叠加的输入	用滞回减少阈值附近的反复翻转
电平转换	1.8V、3.3V、5V 等混合系统	让输出约定匹配输入约定

按钮消抖尤其适合作为入门反例。人只按下一次，触点却可能在几毫秒内多次接通、断开。如果后级把每一次跳变都看成一次请求，一个启动按钮就可能产生多个启动脉冲。消抖电路或后续时序逻辑的职责，是在确认输入持续稳定后才更新内部请求信号。

时间段	原始触点	内部请求	解释
按下前	稳定断开	0	默认下拉给出明确低电平
刚按下	断续跳变	保持 0	尚未接受为一次稳定请求
稳定按下	持续接通	1	已满足消抖条件
刚松开	断续跳变	保持 1 或等待释放确认	不把抖动解释为多次按键
稳定松开	持续断开	0	回到默认状态



图示：人只按了一次，触点却在几毫秒内多次通断（抖动区）。消抖的逻辑是“连续稳定一段时间才算数”：内部请求比真实按下晚一点点，代价可忽略，换来的是它只跳变一次。

这组处理说明一个原则：逻辑核心应当面对已经被整理过的事实，而不是直接面对物理噪声。安全启动控制板中，`allow_start` 不应直接依赖 `start_button_raw`，而应依赖 `start_request`；故障汇总不应直接依赖线缆上的未定义电平，而应依赖经过默认值、阈值和驱动能力检查后的报警信号。后面进入触发器和时钟边界后，还会继续把这种“边界整理”推广到同步采样。

可靠 bit 还必须考虑上电和复位。电源从 0V 上升到额定电压需要时间，芯片内部阈值、外部上拉下拉、时钟源和输入调理电路不一定同时进入正常状态。在这段时间里，若某个输出直接控制电机、继电器或写使能，就不能假设它已经是安全值。正式硬件设计通常要求复位链路或电源监控电路在电源稳定之前保持关键输出禁止。

阶段	硬件状态	对关键输出的要求
断电	节点可能通过漏电或外部连接获得微弱电压	不应驱动执行机构
电源爬升	阈值、时钟和输入调理尚未完全有效	复位或禁止链路保持有效
复位保持	内部状态被强制到约定默认值	<code>motor_enable_cmd=0</code> ，写使能关闭
复位释放	输入和时钟达到有效条件	组合逻辑开始按规格产生输出
正常运行	所有电平约定和时序约定成立	后级只使用已经稳定的信号

这也是“默认安全”的电气含义。它不是在文档里写一句“默认不启动”，而是要能指出：电源未稳时谁拉住使能，复位未释放时谁关闭输出，输入断线时谁把条件解释为不满足，组合路径未稳定时谁阻止后级采样。若这些问题回答不出来，系统可能在最危险的上电瞬间出现短促误动作。

最后，可靠 bit 还要能被实际测量确认。万用表可以检查静态电压，却看不见窄脉冲和抖动；示波器能观察边沿、过冲、振铃和噪声；逻辑分析仪适合记录长时间数字事件，但它同样依据自己的阈值把模拟电压解释成 0/1。测量工具不是抽象之外的附属品，而是把电平约定落实到实物上的手段。

工具或手段	能说明什么	不能单独证明什么
原理图审查	是否有默认路径、保护和驱动边界	实物焊接和噪声环境是否合格
万用表	静态高低电平是否大致正确	边沿速度、毛刺和短时抖动
示波器	波形、阈值穿越、过冲和振铃	长时间低概率事件一定不会发生
逻辑分析仪	数字事件顺序和持续时间	模拟电平余量是否足够
数据手册参数	最坏阈值、延迟、负载能力	板级实现是否满足这些边界

同一根按钮输入线，可以用三类工具得到互补的测量结果。假设 `start_button`

`_raw` 采用上拉输入，按下时接地，内部逻辑再把它反相为 `start_request`。万用表能确认松开和按下时的静态电压方向；示波器能观察按下瞬间的触点抖动、边沿速度和阈值穿越；逻辑分析仪能记录内部请求是否只产生一次事件。三者关注的问题不同，不能互相替代。

测量手段	在按钮案例中应看到什么	若结果异常应怀疑什么
万用表	松开为稳定高电平，按下为稳定低电平	上拉/下拉错误、接线反向、触点失效
示波器	按下和松开有有限抖动，最终越过阈值并稳定	抖动过长、边沿太慢、噪声跨越阈值
逻辑分析仪	消抖后只出现一次 <code>start_request</code>	消抖不足、阈值设置不当、采样边界错误
数据手册	输入阈值、滞回、电流和最大额定值满足设计	电平约定只在实验样品上偶然成立

把这类测量结果写入设计文档后，第一章的抽象就不再停留在“按钮产生 0/1”。更完整的说法是：外部触点通过默认路径得到静态归宿，通过输入调理穿过阈值，通过消抖变成一次语义事件，并在后级使用前满足电平和时间条件。后面的内存读写、总线请求和中断输入，也应按同样方式处理边界。

万用表与示波器各自能看见什么 万用表直流电压档测出的是一段时间内的平均值。静态节点上，松开约 5V、按下约 0V，读数直接可信；但一个 0V/5V、占空比 50% 的时钟信号，直流档读数约为 2.5V——既不是 0 也不是 1，无法据此判断波形好坏。窄毛刺、触点几毫秒的抖动、一次几纳秒的过冲，在平均值里几乎不留痕迹。所以万用表适合回答“默认电平对不对、供电对不对”，不适合回答“信号干不干净”。

示波器把电压随时间的曲线显示出来，才能看到边沿时间、过冲、振铃和阈值穿越次数。读边沿时要看 10% 到 90% 电平之间的行程时间，而不是只看最终稳定值；按钮消抖是否成功、复位释放是否单调、开漏上拉是否太慢，都要在波形上确认。两类工具回答的是不同问题，不能互相替代。

还要记住：测量仪器接入后本身就是被测电路的一部分。万用表和示波器的输入电阻通常约为 $10M\Omega$ ，去测一个靠 $1M\Omega$ 上拉维持的节点，相当于给节点并联了一个 $10M\Omega$ 电阻，分压后读数约为 $5V \times 10M\Omega / (1M\Omega + 10M\Omega) \approx 4.55V$ ——比真实值低了约 0.45V，可能把一个勉强合格的高电平误判成故障。示波器探头还带有电容： $\times 10$ 探头约 10 到 15pF， $\times 1$ 探头可达上百 pF；把 15pF 加到 $100k\Omega$ 上拉的节点上，时间常数就多了 $100k\Omega \times 15pF = 1.5\mu s$ ，边沿会被明显拖慢。测量会改变被测对象，在高阻、高速节点上尤其明显。

工具	能读出的内容	读不出或会引入的问题
万用表直流档	静态高低电平、供电电压、慢变化平均值	看不到毛刺、抖动和边沿，快速波形只剩平均值
示波器	边沿时间、过冲、振铃、阈值穿越、瞬态过程	探头电容与输入电阻会改变高阻节点的读数
测量行为本身	$10M\Omega$ 级输入电阻、pF 级探头电容	高阻节点被分压，边沿被附加电容拖慢

因此读数之前要先问两个问题：仪器接入是否改变了这个节点，以及读数代表的是哪一段时间内的电压。

还要补上板级物理边界。数字信号不是只沿着“信号线”单独传播；每一次电流变化都需要回流路径，供电和地也有电阻、电感和寄生电容。若多个输出同时翻转，瞬时电流会让地线或电源线上出现小幅电压变化，表现为地弹、供电跌落或参考点移动。若两根长线靠得太近，快速边沿还可能通过寄生电容或电感耦合到邻线，表现为串扰。它们不会改变布尔表达式，却会改变接收端看到的真实电压。

板级现象	物理来源	对可靠 bit 的影响
回流路径过长	信号电流的返回路径绕远或不连续	边沿振铃、辐射增加、邻线耦合增强
地弹	多个输出同时切换，地线上产生瞬时压降	接收端参考地移动，噪声余量被压缩
供电跌落	瞬时电流超过局部供电支撑能力	输出驱动变弱，边沿变慢或阈值判断不稳
串扰	相邻走线之间存在寄生电容和互感	静止信号上出现错误脉冲或阈值附近扰动
长线振铃	走线具有传输线特性且终端不匹配	接收端可能多次穿越阈值
去耦不足	芯片附近缺少局部电荷支撑	切换时电源/地噪声增大

因此，电平约定至少隐含三类物理条件。第一，信号源和接收端必须共享可接受的参考地，或者通过隔离/差分/电平转换重新定义边界。第二，供电必须能在切换瞬间提供局部电流，去耦电容的职责就是在芯片附近提供短时间电荷支撑。第三，长线和外部连接器不能只按“理想导线”理解，必要时要考虑终端、屏蔽、滤波、缓冲和同步采样。第一章不展开 PCB 设计，但必须让读者知道：真值表正确并不自动保证板级物理边界正确。

输入模型和输出模型：引脚不是理想变量 为了把原理讲清楚，可以把每个数字引脚暂时看成一个等效模型。输入端不是完全不取电流的数字变量，它有输入漏电、输入电容、阈值和绝对最大额定值；输出端也不是理想电压源，它有等效导通电阻、最大源电流/灌电流、传播延迟和短路风险。把这些模型放进脑中，很多“逻辑式没错但板子不稳”的问题就能被提前解释。

引脚模型	关键参数	影响的问题
输入漏电	输入端即使不主动驱动，也可能有微小电流	上拉/下拉过弱时，默认电平可能被漏电和噪声改变
输入电容	输入门电容、封装和走线电容共同形成负载	边沿速度、RC 时间常数和阈值穿越时间
输入阈值	V_{IH} 、 V_{IL} 、滞回和电源域	电平余量、慢边沿误触发和电平转换需求
输出电阻	上拉/下拉晶体管导通后仍有等效电阻	负载越重，输出电平越偏离理想电源或地
输出电流	源电流、灌电流和总封装电流有限	扇出、LED/继电器驱动、总线冲突和发热
输出延迟	输入变化到输出越过阈值需要时间	组合路径最长延迟、毛刺宽度和采样窗口

因此，扇出不是“一个输出能画多少根线”的绘图问题，而是负载、电流和延迟问题。一个输出同时驱动十个 CMOS 输入，静态电流也许很小，但输入电容会叠加，边沿会变慢；同一个输出若还要驱动 LED、长线或继电器线圈，就已经超出普通逻辑门职责，通常需要缓冲器、晶体管、MOSFET、驱动芯片或隔离器。工程文档应明确区分“逻辑输出”和“功率驱动输出”。

连接方式	不能只看逻辑式的原因	更稳妥的检查问题
一个门驱动多个输入	负载电容叠加，边沿变慢	最坏负载下是否按时越过阈值
逻辑门直接点亮 LED	LED 电流可能超过输出额定值	是否需要限流、电流预算和专用驱动

普通输出接长线	长线可能振铃、串扰或引入地电位差	是否需要缓冲、终端、滤波或差分传输
两个输出并接	一高一低时会形成强冲突电流	是否改用开漏、三态仲裁或总线协议
跨电源域连接	阈值和最大额定值可能不匹配	是否需要电平转换或隔离

读数据手册时，也应把这些模型翻译成检查清单：输入端看阈值、漏电、电容和最大额定值；输出端看高低电平保证、驱动电流、传播延迟、上升下降时间和总功耗限制。只要这张清单没有填完，布尔式就还不能直接等同于可工作的硬件。

二、从器件到受控电流路径

二极管可以先理解成方向性很强的器件。在合适偏置下，它比较容易让电流沿一个方向通过；反向偏置时，它基本阻止电流通过。这个模型不需要一开始展开半导体物理，却已经足够说明硬件设计的一条基本路线：利用器件物理特性，制造可预测的电流路径。

二极管的“单向”不是理想数学开关。正向导通通常需要一定压差，导通后还有压降；反向并非绝对没有电流，而是漏电很小；电压过高时还可能击穿。初读数字硬件时可以先用理想模型建立方向感，但不能忘记真实器件会吃掉电压余量、引入延迟和功耗。

把数量级写下来，后面的讨论才有抓手。常见硅二极管的正向压降约为 0.6V 到 0.7V，锗二极管约为 0.2V 到 0.3V，肖特基二极管约为 0.2V 到 0.4V；反向漏电流通常在微安量级以下，普通小功率管的反向耐压从几十伏到几百伏不等。本章估算一律取硅管 0.7V：一路 5V 信号经过一只二极管后，高电平只剩约 4.3V；再串一级，就只剩约 3.6V。若后级输入高阈值是 2.0V，单看一级还剩 2.3V 余量，串到第三级时余量只剩 0.9V——这就是“级联多了电平余量越来越紧”的定量含义。

级数	输出高电平（输入 5V）	距 2.0V 阈值的余量
0 级（直连）	5.0V	3.0V
1 级二极管	$5.0 - 0.7 = 4.3V$	2.3V
2 级二极管	$4.3 - 0.7 = 3.6V$	1.6V
3 级二极管	$3.6 - 0.7 = 2.9V$	0.9V

二极管事实	对数字电路的影响	读书时要追问
有方向性	可以限制电流路径	这条路径在何种输入下导通
有正向压降	输出电平会损失一部分余量	级联后高电平还够不够高
有漏电和极限	不能把反向阻断看成无限好	极端温度、电压下是否仍可靠
不能主动放大	不能恢复变弱的电平	下一级是否需要缓冲或开关恢复

整流、钳位与续流：二极管的三个工程角色 在把二极管用进逻辑之前，先看它在电源与保护中的三个更常见的工程角色。这三种用法都只依赖“单向导通”这一条性质，解决的却是三类完全不同的事故。

第一个角色是整流：把交变或极性不定的电压整理成单向电流。接法上，可以单管串联做半波整流，也可以用四只二极管组成桥式全波整流，再配合电容和稳压得到直流。电源入口串一只二极管同时也是最低成本的防反接：电源装反时它不导通，电路只是不工作，而不是烧毁。失效后果是直接的一整流二极管接反或击穿短路后，反向电压或交流分量直接加到直流母线上，后级电解电容和稳压器可能损坏。

第二个角色是钳位保护：把引脚电压限制在电源轨附近。接法是用两只二极管从信号引脚分别“反并”到电源和地：引脚电压高于 $V_{CC} + 0.7V$ 时上管导通，把电流引

入电源轨；低于 $-0.7V$ 时下管导通。引脚因此被钳在电源轨上下各一个二极管压降的范围之内。芯片引脚内部的 ESD 保护正是这种结构。没有钳位时，人体静电动辄几千伏，一次触碰就可能击穿片内结构；但钳位也不是无限保险——它只为瞬态小能量设计，若把 12V 长期误接到 5V 引脚上，持续大电流会先烧掉钳位二极管，再烧掉芯片。

第三个角色是续流：给电感电流留一条退路。电感电流不能突变。继电器线圈由晶体管驱动，导通时电流 100mA；晶体管约在 $1\mu s$ 内关断，线圈仍要维持电流，两端感应电压可按 $V = L \cdot \Delta I / \Delta t$ 估算：10mH 电感在 $1\mu s$ 内切断 100mA，理想估算达 $10mH \times 100mA / 1\mu s = 1000V$ ，实际虽被寄生电容和器件击穿限制，仍常达几十到上百伏，远超小信号晶体管的耐压。接法是在线圈两端反并一只二极管：正常供电时它反偏不工作，关断瞬间线圈电流改经二极管环流、慢慢衰减，反峰被钳在约 0.7V。没有续流二极管的继电器驱动，常见现象是晶体管工作一段时间后“莫名其妙”损坏。

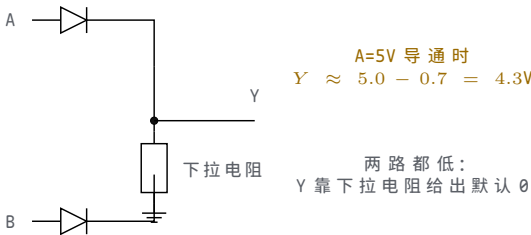
角色	典型接法	缺少它的失效后果
整流与防反接	单管串联或四管桥，串在电源入口	反向或交变电压直达直流母线，电容与稳压器受损
钳位保护	两只二极管把引脚反并到电源轨和地	静电与瞬态直接打入片内结构；长期过压同样会烧毁钳位
续流	与继电器或电感线圈反并联	关断反峰击穿驱动晶体管，表现为多次动作后失效

这三个角色都不参与逻辑判断，却决定了逻辑电路能否在真实电源和真实负载下长期存活。

二极管逻辑可以表达某些简单关系。若有两路输入想共同点亮一个指示节点，任意一路为高时都能通过二极管把输出节点拉高，输出就呈现 OR 行为：

```
if A can drive through a diode:
    Y may be pulled high
if B can drive through a diode:
    Y may be pulled high
```

该例能够说明方向性逻辑，但也暴露出局限。第一，二极管会带来压降，输出高电平可能比输入高电平低一截；级联多了，电平余量会越来越紧。第二，二极管只能提供方向性，不能主动把一个偏弱的高电平重新恢复成强高电平。第三，它不方便实现取反，而取反是构造通用逻辑必须具备的能力。



图示：二极管 OR 的真实样子：任意一路为高，该路二极管导通，把 Y 拉高——但 Y 比输入低一个正向压降（硅管约 0.7V）；两路都低时没有驱动源，靠下拉电阻给出默认 0。方向性由二极管保证，电平却不会被恢复：这就是它不能无限级联的原因。

以安全启动控制板的故障汇总为例，temp_alarm 和 current_alarm 都可能点亮 fault_lamp。二极管 OR 可以让任一报警输入把故障节点拉高，但如果这个故障节点还要继续驱动后面的禁止逻辑、记录逻辑和指示灯，问题就会扩大。每经过一级二极管，电平余量都可能被吃掉一部分；每增加一个负载，边沿都会变慢。最终出现的不是“公式错了”，而是高电平不够高、上升不够快、后级读不稳。

二极管 OR 用法	可以解决的问题	不能解决的问题
-----------	---------	---------

汇总多个报警源	任一报警能提供一条拉高路径	不能主动恢复被削弱的高电平
隔离多个输入源	一个输入不容易反向灌入另一个输入	正向压降会消耗电平余量
形成简单有线关系	结构直观，适合早期理解	不适合无限级联成通用逻辑网络

取反为什么重要？因为许多条件天然需要反向解释。传感器未触发时允许运行，触发时禁止运行；按钮未按下时保持，按下时改变。若电路只能表达“有一路导通”，却能把 1 变成 0、把 0 变成 1，那么可组合的逻辑很快就受限。要继续向前，需要一个信号能控制另一条电流路径是否导通。

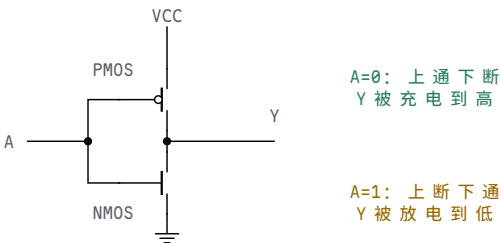
晶体管的细节很深，但作为第一层硬件模型，可以先把它当成受控开关。控制端不是直接给输出供电，而是决定输出节点到某条电源路径之间是否接通。受控开关和普通机械开关最大的差别，是控制信号本身也是电信号。按钮需要人按，晶体管可以由前一级逻辑输出控制。于是硬件有了“信号控制信号”的递归能力：一个门的输出可以控制下一批开关，下一批开关再形成新的输出。

在 CMOS 数字电路中，可以先用极简模型区分两类受控开关。NMOS 更适合把节点拉向地，输入控制为高时容易导通；PMOS 更适合把节点拉向电源，输入控制为低时容易导通。真实器件还有阈值电压、体效应、漏电、沟道电阻和寄生电容等细节，但第一章暂时只抓住一件事：上拉路径和下拉路径由互补器件协作，才能把输出恢复为强 0 或强 1。

器件视角	在门电路中的典型职责	需要避免的误解
NMOS 下拉	给输出提供到地的受控路径	不是“产生 0”这个符号，而是放电路径
PMOS 上拉	给输出提供到电源的受控路径	不是“产生 1”这个符号，而是充电路径
互补配对	一个负责拉低，另一个负责拉高	不能让两条强路径长期同时导通
输入过渡	两类器件可能都部分导通	过渡区只应短暂经过，不应长期停留

反相器可以作为第一种完整逻辑门来理解。输入低时，上拉路径导通、下拉路径断开，输出被拉高；输入高时，上拉路径断开、下拉路径导通，输出被拉低。注意这里不是“输入低所以输出自动高”，而是“输入低使某个高电平路径导通”。把每一句逻辑描述翻译成路径描述，能避免很多硬件误解。

输入	上拉路径	下拉路径	输出
低电平	导通	断开	被拉高
高电平	断开	导通	被拉低
过渡中	可能部分导通	可能部分导通	暂时不应采样



图示：反相器的开关模型。输入 A 同时接到两个控制端：PMOS 控制端带小圆圈，表示“输入低才导通”；NMOS 没有圆圈，表示“输入高才导通”。这就是 NOT 的全部——但它同时完成了二极管做不到的事：不管输入边沿多差，输出都被

重新驱动成强 0 或强 1，电平被恢复了。

把这张路径表压缩成真值表，就是 NOT 的全部定义：

A	Y	路径解释
0	1	上拉导通，输出被拉到高
1	0	下拉导通，输出被拉到低

第三行很重要。真实输入从低到高不是瞬间跳变，在过渡区间里，上拉和下拉可能都不处于理想开/关状态。短时间内出现电源到地的电流并不奇怪，这就是动态功耗和短路功耗的一部分。只要过渡足够快、采样避开中间区域，数字抽象仍然成立。若输入边沿太慢，电路可能在中间区域停留过久，功耗和误判风险都会上升。

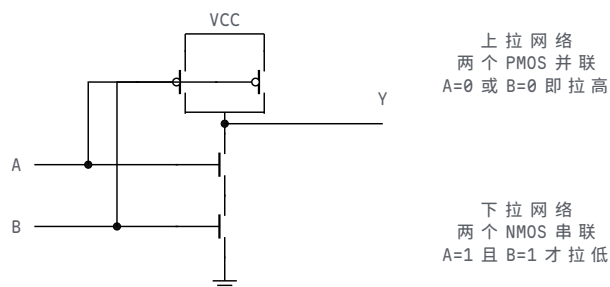
把开关串联，可以表达“两个条件都满足才有通路”；把开关并联，可以表达“任意一个条件满足就有通路”。以下拉网络为例，两个受控开关串联，只有 A 和 B 都让开关导通时，输出节点才会被拉向低电平；两个开关并联，只要 A 或 B 有一个导通，输出节点就会被拉低。上拉网络通常要和下拉网络互补，保证输出在稳定输入下要么被拉高，要么被拉低，而不是两边都断或两边都强通。这种“互补路径”思想，后面会自然走到 CMOS 逻辑。

互补路径可以用两条规则记住。第一，输出应该被明确驱动：下拉网络不导通时，上拉网络应当负责把输出拉高；下拉网络导通时，上拉网络应当关闭，避免电源到地的强冲突。第二，稳定输入下不应依赖中间电压：如果上拉和下拉都处于部分导通状态，输出可能落在不可靠区间，功耗也会上升。CMOS 逻辑之所以适合作为数字门的基础，关键就在于它把“拉高”和“拉低”组织成互补关系，并在稳定状态下尽量避免静态直通电流。

路径状态	输出后果	设计判断
上拉开、下拉关	输出被拉向高电平	合法稳定状态
上拉关、下拉开	输出被拉向低电平	合法稳定状态
上拉关、下拉关	输出悬空，可能保留旧电压	需要补默认路径或改逻辑
上拉开、下拉开	电源到地形成强通路	功耗高，电平不可靠
两边部分导通	输出可能处于过渡区	只允许短时间经过，不应长期停留

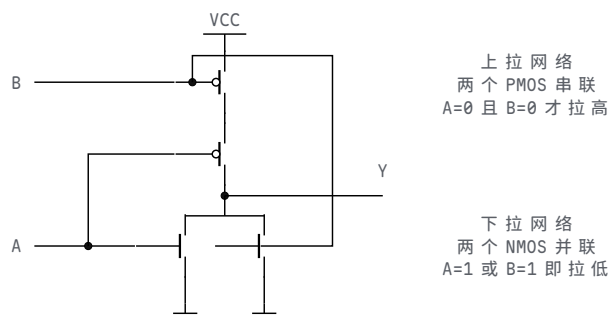
把这条规则应用到 NAND 和 NOR，可以看到 CMOS 门并不是把 AND、OR 符号直接画在硅片上，而是把上拉网络 and 下拉网络组织成互补关系。二输入 NAND 的下拉网络由两个 NMOS 串联：只有 A、B 都为 1 时，输出才被拉向地；它的上拉网络由两个 PMOS 并联：只要 A 或 B 有一个为 0，就有一条上拉路径把输出拉高。二输入 NOR 则相反：下拉网络由两个 NMOS 并联，只要 A 或 B 为 1 就能拉低；上拉网络由两个 PMOS 串联，只有 A、B 都为 0 时才拉高。

CMOS 门	下拉网络	上拉网络
NAND	NMOS 串联，A=1 且 B=1 时完整导通	PMOS 并联，A=0 或 B=0 时至少一路导通
NOR	NMOS 并联，A=1 或 B=1 时至少一路导通	PMOS 串联，A=0 且 B=0 时完整导通



图示：二输入 NAND 的晶体管级结构。输出为 0 的唯一情形是 $A=B=1$ ：两个串联 NMOS 同时导通，下拉路径完整；其余三行至少一只 PMOS 导通，上拉把输出充到高。真值表的每一行都能在这张图上找到对应的路径状态。（输入 B 的连线跨过 A 的线但不连接：没有连接点的交叉不算接通。）

NOR 用同一套符号画出来，两个网络的串并联关系正好与 NAND 对调：



图示：二输入 NOR 的晶体管级结构，与上一张 NAND 图成对：上拉网络换成两个 PMOS 串联，只有 $A=B=0$ 时上拉路径才完整、输出被拉到高；下拉网络换成两个 NMOS 并联，A 或 B 任一为 1 就有一条下拉路径把输出拉低。输出为 1 的唯一稳定条件对应“上拉串联路径完整”，与互补网络表逐行对应。（输入 B 的连线跨过输出线但不连接：没有连接点的交叉不算接通。）

这张表把逻辑表和器件路径接在一起。NAND 输出为 0 的唯一稳定条件，对应“下拉串联路径完整”；NAND 输出为 1 的三个稳定条件，对应“至少一条上拉路径存在”。NOR 输出为 1 的唯一稳定条件，对应“上拉串联路径完整”；NOR 输出为 0 的三个稳定条件，对应“至少一条下拉路径存在”。如果只背真值表，读者很难解释为什么 NAND/NOR 常成为门级实现的基础；如果只看器件路径，又容易失去逻辑函数的目的。两者必须同时保留。

CSAPP 一类系统教材常把“布尔函数”和“硬件实现”分层书写：先说明抽象函数，再说明门级和时序代价。这里的 CMOS NAND/NOR 正好提供一个小型范例。稳定逻辑值来自互补路径，传播延迟来自器件和连线，功耗来自节点切换和漏电；同一个门符号背后同时存在功能、时间和能量三个维度。

从功耗角度看，数字门并非“只有 0 和 1，没有模拟代价”。稳定状态下，如果只有上拉或只有下拉导通，静态电流可以很小；切换时，输出节点的寄生电容需要充电或放电，形成动态功耗；输入经过过渡区时，上拉和下拉可能短时间同时导通，形成短路功耗。后面讨论频率、负载和关键路径时，这些现象会继续出现。

这也是 CMOS 成为主流数字逻辑基础的重要原因。它不是没有功耗，而是在稳定 0 和稳定 1 时尽量避免持续直通电流，把主要能量消耗集中到节点切换和短暂过渡中。随着芯片集成度提高，设计者更关心哪些节点频繁翻转、哪些长线负载过大、哪些门需要更强驱动，以及哪些路径应由寄存器边界切分。由此可见，第一章的受控路径模型会直接延伸到后面的频率、功耗和时序预算。

功耗来源	物理原因	设计含义
静态功耗	漏电或稳定状态下仍存在电 流路径	器件越小越不能忽略漏电和待 机功耗

动态功耗	输出节点和连线电容反复充放电	高翻转率、长线和大扇出都会增加能耗
短路功耗	输入过渡时上拉和下拉短暂同时导通	慢边沿不仅影响时序，也会增加功耗
驱动损耗	输出级推动外部负载或长线	指示灯、总线和连接器常需要专用驱动级

BJT 与 MOSFET 开关对照 本章的受控开关主要以 MOS 管出现，但值得把它和另一类器件—双极型晶体管（BJT）—放在一起对照，因为两者的差别正好解释了现代数字电路的工艺选择。最核心的差别在控制端：BJT 是电流驱动器件，集电极电流要靠持续的基极电流维持，工程估算常取 $I_B \geq I_C/\beta$ ， β 约几十到几百；MOSFET 是电压驱动器件，栅极与沟道之间隔着绝缘氧化层，稳定状态下几乎没有栅极电流，能量只在翻转瞬间用于给栅电容充放电。

作开关使用时，两者的导通损耗来源也不同。BJT 在截止与饱和之间切换，导通时呈现近似恒定的饱和压降 $V_{CE(sat)}$ （约 0.2V），损耗为压降乘以电流；MOSFET 在夹断与导通之间切换，导通时表现为沟道电阻 $R_{DS(on)}$ ，损耗按 I^2R 计算。算一笔账：驱动一个 100mA 的继电器，BJT 取 $\beta = 100$ ，基极要持续供出约 1mA，导通损耗约 $0.2V \times 100mA = 20mW$ ；换用 $R_{DS(on)} = 50m\Omega$ 的 MOSFET，损耗只有 $0.1^2 \times 0.05 = 0.5mW$ ，且栅极在稳定期间几乎不取电流。

对比项	BJT	MOSFET
驱动方式	电流驱动，基极需持续电流	电压驱动，栅极只取充放电电流
开关状态	截止与饱和	夹断与导通
导通损耗来源	$V_{CE(sat)} \times I$ ，近似恒定压降	$I^2 \times R_{DS(on)}$ ，随电流平方上升
100mA 驱动例	基极约 1mA，损耗约 20mW	栅极静态近零，损耗约 0.5mW（50mΩ）
静态功耗 仍活跃の場合	基极电流贯穿整个导通期 大电流驱动、线性放大、分立高压	稳定状态下几乎为零 数字集成与功率开关的主流

由此可以理解为什么现代数字电路主流是 MOS。数字芯片里有数十亿个开关，若每个开关的控制端都要持续电流，静态功耗将完全无法接受；MOS 栅极不取静态电流，NMOS 与 PMOS 两种极性又容易在同一工艺中做成互补对，器件尺寸还能持续缩小—静态功耗和集成密度因此都站在 MOS 一边。BJT 并未消失：在大电流驱动、线性放大和某些高压分立场合，它的线性区特性和电流处理能力仍然合适，只是不再是构成通用数字逻辑的器件。

开漏或开集结构是理解共享线路的重要例子。一个输出器件只负责把线拉低，释放时并不主动拉高；高电平由公共上拉电阻提供。多个器件可以共享同一根线，只要任意一个器件拉低，整根线就是低电平；只有所有器件都释放，线上才被上拉为高电平。这种结构常用于中断请求、故障汇总或简单总线。

共享线状态	物理路径	逻辑后果
所有输出释放	上拉电阻把线拉高	线为 1，但上升速度取决于上拉和负载
任一输出拉低	该输出提供到地路径	线为 0，多个输出不会互相强冲突
上拉过弱	高电平上升慢	采样前可能尚未达到高阈值
上拉过强	拉低时电流大	功耗增加，输出器件负担加重

这说明“多个来源接到一根线”并非永远错误，关键在于是否有协议和电气结构保证同一时刻不会出现强驱动冲突。开漏共享线允许多个设备共同拉低，是因为它们只竞争“是否提供到地路径”，不竞争“谁主动拉高”。普通推挽输出若直接并在一起，一个输出拉高、另一个输出拉低，就会形成强冲突。

三态输出是另一类共享线路方案。普通数字输出只有主动拉高和主动拉低两种驱动状态；三态输出还可以进入高阻态。高阻态不是逻辑 0，也不是逻辑 1，而是输出驱动器基本断开，让这一路暂时不参与驱动总线。若多块设备共享数据总线，控制逻辑必须保证同一时刻最多只有一个设备打开输出驱动，其余设备都处于高阻态。否则，一个设备驱动 1、另一个设备驱动 0，仍会发生总线争用。

输出方式	驱动状态	适合表达的问题
推挽输出	主动拉高或主动拉低	单一来源驱动一条内部信号
开漏输出	只主动拉低，释放靠上拉	多个来源共同提出低有效请求
三态输出	拉高、拉低或高阻	多个来源按仲裁结果轮流占用总线

开漏和三态都要求协议。开漏协议通常是“任一设备可以拉低，所有设备释放后才为高”；三态协议通常是“仲裁者只允许一个设备驱动，其他设备必须释放”。这两类结构后来会在中断线、片选、数据总线和 I/O 控制中反复出现。现在先记住硬件判断标准：多来源共享一根线时，必须说明每个来源什么时候能驱动、什么时候必须释放，以及释放后由谁给出默认电平。

把这些器件观点收束起来，可以得到本章的第二层抽象：数字门不是画在纸上的符号，而是一组受输入控制的上拉、下拉、释放和恢复路径。二极管提供方向性，但不恢复电平；晶体管提供受控路径；互补开关网络提供明确的高低驱动；缓冲和三态驱动器提供负载隔离和共享边界。后面所有更大的组合电路，仍然要回到这些路径检查。

器件族之间还存在电平兼容问题。即使两个器件都叫“数字逻辑”，也不表示它们可以任意直连。一个 5V 供电器件的输出、一个 3.3V CMOS 输入、一个 TTL 兼容输入和一个开漏传感器，可能有不同的高低阈值、输入电流和输出驱动能力。设计者必须把数据手册中的最坏输出和最坏输入放在同一张表里比较，而不是只看“都是 0/1”。

连接检查	要问的问题	典型风险
电源电压	前级输出高电平是否超过后级输入上限	过压损坏或长期可靠性下降
输入阈值	前级最差高/低输出是否满足后级阈值	电平看似变化，后级不承认
输入电流	前级能否提供或吸收后级所需电流	电平被拖偏，余量不足
输出驱动	前级能否驱动全部负载和走线电容	边沿变慢，时序失败
上拉配置	开漏/开集输出是否有合适上拉	释放后没有可靠高电平

5V 与 3.3V 系统互连的电平转换 上表回答的是“怎么查”，本专题回答“查出不匹配之后怎么办”。先把两个方向各自卡在哪里算清楚。5V 输出进 3.3V 输入：5V 超过多数 3.3V 器件的最大输入额定值，长期直连可能损坏输入结构，除非该输入明确标注 5V 容忍。反方向的问题更隐蔽：3.3V 系统的最差高电平可能只有 2.4V 左右，而 74HC 类 CMOS 输入在 5V 供电时要求 $V_{IH} \geq 0.7V_{CC} = 3.5V$ ，电平看似翻转了，后级却不承认。同一 5V 供电下，74HCT 系列把输入阈值做成 TTL 兼容： $V_{IH} = 2.0V$ 、 $V_{IL} = 0.8V$ ，2.4V 的高电平仍有 0.4V 余量。74HC 与 74HCT 的分工由此而来：同一

电源域内部级联用 HC，需要承接 TTL 电平或 3.3V 输出的场合用 HCT。

真正需要转换时，常见做法有三种，代价各不相同。第一种是电阻分压：上臂 $1.7\text{k}\Omega$ 、下臂 $3.3\text{k}\Omega$ ，可把 5V 分成 $5\text{V} \times 3.3 / (1.7 + 3.3) = 3.3\text{V}$ 。它只适合从高到低、单向、低速的信号：分压网络的等效内阻（本例约 $1.1\text{k}\Omega$ ）会拖慢边沿， $5\text{V} / 5\text{k}\Omega = 1\text{mA}$ 的静态电流常流不断，而且电阻只会衰减不会放大，3.3V 到 5V 方向完全用不了。第二种是专用电平转换芯片：双向、多通道，阈值和速度在数据手册中有明确规格；代价是多一片器件、两组电源脚和相应的布板面积。第三种是开漏加目标电源上拉：输出级只做开漏，上拉电阻接到对方电源—器件只负责拉低，高电平天然由目标电源域提供，电平自动匹配；I2C 总线正是靠这个结构允许不同电压域挂在同一条线上。代价是速度受上拉 RC 限制，且只有开漏或开集结构的信号才能这样接。

做法	适用方向与速度	代价与限制
电阻分压 ($1.7\text{k}\Omega + 3.3\text{k}\Omega$)	仅 5V 到 3.3V、单向、低速	约 1mA 静态电流，约 $1.1\text{k}\Omega$ 等效内阻拖慢边沿，不能反向
电平转换芯片	双向、多通道，速度有手册规格	增加器件、两组电源和布板面积
开漏加目标域上拉	双向共享线，适合总线与中断请求	速度受上拉 RC 限制，仅开漏结构可用
选对输入阈值（74HC 与 74HCT）	3.3V 到 5V 单方向最省事	只是选型手段，不解决 5V 到 3.3V 过压

三种做法都不改变逻辑内容，只重新匹配两侧的电平约定。选型顺序是先定方向、速度和通道数，再决定用分压、专用芯片还是开漏结构。

数据手册里的参数要按“最坏组合”阅读。若前级在轻载、常温下能输出漂亮的 3.3V，不代表它在最大负载、高温、低电源电压下仍有同样余量；若后级典型阈值较低，也不代表所有批次、所有温度下都按典型值判断。经典教材强调抽象层次，但并不鼓励忽略边界条件。抽象的意义是把边界写清楚，然后在边界内使用更高层概念。

读取标准逻辑芯片数据手册时，可以采用固定顺序。先看供电范围，确认器件是否处在允许电源电压内；再看输入阈值，确认前级输出能被后级可靠承认；再看输出电流能力，确认它能驱动实际后级和上拉/下拉网络；再看传播延迟，确认多级组合路径能在采样前稳定；最后看封装、引脚、温度范围和绝对最大额定值，确认板级使用没有越界。这个顺序比直接查“某个门是什么功能”更接近硬件设计的实际检查顺序。

读取顺序	要确认的参数	与本章概念的关系
供电条件	工作电压范围、绝对最大额定值	电平约定必须先处在器件允许边界内
输入条件	V_{IH} 、 V_{IL} 、输入电流、滞回类型	后级如何把真实电压承认为 0/1
输出条件	V_{OH} 、 V_{OL} 、输出源/灌电流	前级能否给出足够强的高低电平
时间条件	传播延迟、上升/下降时间、使能/禁止延迟	组合路径多久后才可被使用
负载条件	扇出、输出电容、推荐上拉、总线负载	真实连接是否拖慢边沿或压缩余量
环境和封装	温度范围、封装引脚、ESD 和焊接约束	板级实现是否仍满足器件边界

例如同样是“用一个 NAND 门”，74LS、74HC、4000 CMOS 和高速 CMOS 的电气约定并不相同。某个系列可能适合 5V TTL 兼容输入，另一个系列可能要求 CMOS 阈

值，某些高速器件对输入边沿和去耦要求更严格。正式设计不能只写“接一个 NAND”，还要写清器件族、供电电压、输入输出阈值、负载和时序要求。否则，逻辑表达式相同，实物板卡仍可能在温度、批次或负载变化时失效。

从这个角度看，缓冲器、电平转换器和驱动器并不是“逻辑上多余”的器件。缓冲器可以恢复电平和增强驱动；电平转换器让两个电源域的约定重新匹配；驱动器让小逻辑信号控制更大的负载。它们是否改变真值表不是唯一问题；它们是否让真实节点满足电平、负载和时序约定，才是硬件设计要回答的问题。

标准逻辑芯片把这些约束以产品形式暴露出来。以常见的 7400 系列为例，一个封装中可以放入若干个 NAND 门。教材里的 NAND 真值表只说明稳定逻辑行为，而芯片数据手册还会说明每个输入能承认的高低电平、输出能提供或吸收的电流、传播延迟范围、工作电压范围和扇出建议。换言之，标准门芯片把“布尔函数”和“电气约定”绑定在一起出售。

从 7400 类门芯片读到的项目	对应本章概念	设计时的含义
输入高低阈值	可靠 bit	前级输出必须落入后级承认区间
输出电流能力	驱动能力和扇出	一个门不能无限驱动后级
传播延迟	组合路径时序	多级门相连后要累计最坏延迟
工作电压范围	电平约定边界	不同电源域不能随意直连
封装引脚	物理边界	逻辑门最终仍要通过真实引脚连接

4000 系列 CMOS 逻辑则更适合说明互补路径和功耗。CMOS 门在稳定 0 或稳定 1 时，理想情况下没有从电源到地的持续强通路，静态功耗可以很低；切换时，输出电容充放电，形成动态功耗。现代处理器虽然远比 4000 系列复杂，但仍然继承了这个核心事实：频率越高、切换节点越多、负载越大，动态功耗越高；门越小、线越短、负载越低，延迟越容易下降。

比较 7400 TTL 和 4000 CMOS 时，不应把它们只看成“都能实现 NAND/NOR”。TTL 器件常用于说明输入电流、扇出和标准逻辑电平；CMOS 器件更适合说明极高输入阻抗、低静态功耗和输入悬空风险。一个 CMOS 输入若没有明确上拉或下拉，可能因为极小漏电和耦合电荷保持在不确定电平；这与“输入阻抗高”并不矛盾，反而正是高阻输入必须定义默认值的原因。

芯片族视角	教材中应观察的事实	对本章的补充意义
7400 TTL	输入电流、输出灌/拉电流、传播延迟和扇出限制	门芯片是带电气约定的布尔函数
4000 CMOS	高输入阻抗、低静态功耗、宽电源范围和较慢边沿选型	默认路径和输入调理不能省略
高速 CMOS	更低延迟、更强驱动、更严格边沿和布局要求	速度提高后，负载、布线和毛刺更敏感

体二极管与栅极保护 高输入阻抗这条性质还有更深的器件根源，值得补两句。第一句关于体二极管。功率 MOSFET 的源极与衬底在制造时短接，衬底与漏极之间天然留下一个 PN 结，称为体二极管。对源极接地的 NMOS，它的方向是从源极指向漏极：正常工作时漏极电位高于源极，二极管反偏不导通；一旦漏极被拉到比源极低约 0.7V——例如走线电感上的负尖峰或电源接反——它就绕过栅极自行导通。栅极控制不了这只二极管，所以 MOSFET 开关挡不住反向电流；桥式驱动里它有时被有意用作续流路径，但长期依赖它导通会带来额外损耗和可靠性风险。

第二句关于栅氧击穿。MOS 栅极是一块被氧化层绝缘的电容极板，氧化层只有纳米量级厚，常见器件的栅源额定电压仅约 $\pm 20V$ ；而人体摩擦积累的静电轻松达到几

千伏。栅极又是高阻节点，电荷没有泄放路径，一次静电触碰就可能把氧化层击穿，器件当场失效或留下隐患。这正是 CMOS 输入必须配保护结构、未用输入必须接确定电平、插拔与焊接要防静电的物理原因。

两个廉价元件能化解其中大部分风险。其一是在驱动输出与栅极之间串一只电阻：它与栅电容构成 RC，既限制瞬态进入栅极的电流，也阻尼快速边沿引起的振铃；取值按速度定， $1k\Omega$ 串阻配 $5pF$ 小信号栅电容的时间常数约为 $1k\Omega \times 5pF = 5ns$ ，对低速输入毫无影响，驱动功率 MOS 时则常用几欧到几十欧换取边沿与开关损耗的平衡。其二是给栅极配一只到地或到电源的大阻值电阻，例如 $100k\Omega$ ：驱动断开、复位期间或连接器拔出时，栅极不再悬空，器件保持确定关断。这就是前面上拉/下拉原则在器件级的再次出现——高阻节点必须有默认归宿。

风险	物理来源	常用对策
体二极管误导通	漏源电压反向超过约 $0.7V$	抑制负压尖峰，必要时串肖特基二极管或按反峰设计
栅氧击穿	静电在无路可泄的高阻栅极上积累	输入保护结构、串联电阻、防静电操作
悬空误开通	驱动断开时栅极电荷随机	栅极上拉或下拉给出确定电平
边沿振铃	栅电容与走线电感形成谐振	串联栅电阻阻尼，按速度取值

这些措施都不改变真值表，却决定了开关器件在第一块板子上能否活着工作。

读一片标准 NAND 芯片，至少要同时读三层信息。第一层是逻辑：两个输入同时为 1 时输出为 0，其余情况输出为 1。第二层是电气：输入高低阈值、输出电流、工作电压和负载能力说明它能否与前后级可靠连接。第三层是时间：传播延迟说明输入变化后多久输出才可用。若只读第一层，就会把真实芯片误当成理想布尔符号；若只读第二、三层，又会失去门的功能目的。

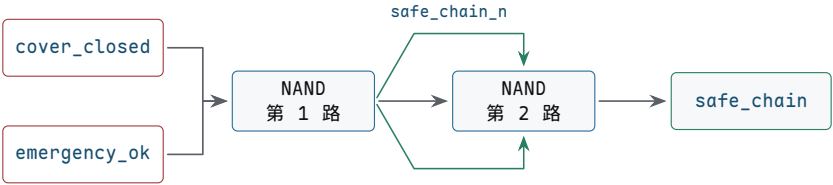
把逻辑式映射到芯片引脚时，还要完成一次**表达式到封装**的翻译。以 74HC00 或传统 7400 类四路二输入 NAND 芯片为例，一个封装中包含四个独立 NAND 门。设计者不能只写“使用一个 NAND”，还要指定哪两个信号接到某一路 NAND 的输入脚，该路输出脚驱动哪个后级，芯片的 VCC 和 GND 接到哪个电源域，电源脚附近是否放置去耦电容，未使用门的输入是否被接到确定电平。

落地项目	需要写清的内容	常见错误
电源脚	VCC、GND、工作电压和去耦电容位置	只接信号脚，忽略供电噪声和地参考
输入脚	A、B 两个输入分别来自哪些内部语义信号	把低有效或未调理信号直接接入门
输出脚	输出驱动哪个门、缓冲器或后级输入	未检查扇出、负载和传播延迟
未用门	未使用输入接到确定高或低，输出可悬空不用	CMOS 输入悬空导致随机翻转和额外功耗
封装方向	引脚编号、缺口方向、封装类型和焊接方向	原理图正确但实物引脚接反

例如要把 `safe_chain = cover_closed AND emergency_ok` 映射到 74HC00，可先用一路 NAND 得到反相信号，再用另一路 NAND 自反相恢复为 AND。文字规格可以写成：第 1 路 NAND 输入接 `cover_closed` 与 `emergency_ok`，输出命名为 `safe`

_chain_n；第 2 路 NAND 两个输入都接 safe_chain_n，输出命名为 safe_chain。这种写法让中间反相信号、芯片资源占用和调试测点都可审查。

```
safe_chain_n = cover_closed NAND emergency_ok
safe_chain   = safe_chain_n NAND safe_chain_n
```



第二路 NAND 的两个输入接同一个反相信号，等价于把第 1 路输出再反相。图中分成两条回路，是为了避免箭头穿过文字。

图示：用两路 NAND 实现 AND。图的命题是：功能等价必须能对应到具体芯片资源和中间测点。

芯片引脚级说明还必须保留物理约束。每片芯片电源脚附近通常需要本地去耦电容，以降低门切换时的电源压降；输入不能超过允许电压范围；输出不能超出源电流或灌电流能力；同一输出驱动多个输入时要重新计算扇出；跨连接器或长线时还要考虑保护、终端和边沿质量。把布尔式映射到芯片，不是把公式抄到封装上，而是把功能、电平、时间、负载和装配边界同时写进规格。

若读者希望把安全启动控制板搭成一块低速演示电路，可以采用标准 CMOS 逻辑芯片完成第一版。这个版本不追求工业安全认证，只用于验证本章的电平、路径、组合逻辑和测量方法。输入侧使用按键或拨码开关表示 start_request、cover_closed、emergency_ok、temp_alarm 和 current_alarm；每个输入都有明确上拉或下拉，必要时经施密特输入整形；组合核心用 74HC00、74HC04、74HC08、74HC32 或同类芯片实现；输出侧用缓冲器或三极管/MOSFET 驱动 LED，而不是让普通门输出承担全部指示灯电流。

模块	可选器件或结构	检查重点
输入默认值	10kΩ 量级上拉/下拉、电阻网络或拨码开关模块	松开、断线、未接时不悬空，默认方向符合安全策略
慢边沿整形	74HC14 或同类施密特反相器	慢按钮边沿和噪声不能在阈值附近反复翻转
基本门	74HC00、74HC08、74HC32、74HC04 或等价标准门	每个门的输入阈值、输出驱动和传播延迟满足连接要求
故障汇总	OR 门、NAND-only 网络或二极管加缓冲结构	任一故障都点亮故障灯并禁止启动，双故障有明确行为
输出指示	74HC244/74HC125 缓冲、三极管或小 MOSFET 驱动 LED	指示灯负载不拖垮内部逻辑电平
供电与去耦	5V 或 3.3V 稳定电源，每片芯片近旁 0.1μF 去耦	切换时电源和地不出现影响阈值判断的扰动

搭建顺序应从静态边界开始，而不是直接运行全部逻辑。第一步只接电源、地和去耦，确认芯片供电正确。第二步只接输入默认网络，用万用表确认每个输入在松开、按下、断线时的电平。第三步逐个验证单门行为，例如 NAND、反相器和 OR 的四行真值表。第四步再连接 fault_lamp 和 no_fault，确认任一报警都会禁止启动。第五步连接 safe_chain、request_ok 和 allow_start，遍历正常允许、单条件缺失、单故障、双故障和断线默认。最后才连接 LED 驱动或执行机构模拟负载，并确认负载加入后内部逻辑电平仍满足后级阈值。

阶段	操作	进入下一阶段的条件
----	----	-----------

1	只检查电源、地、去耦和芯片方向	供电电压正确，芯片不发热，电源脚附近无明显跌落
2	接入输入默认网络，不接组合核心	每个输入在所有机械状态下都有确定电平
3	单独验证每片门芯片的一路门	真值表正确，输出能驱动一个后级输入或测试点
4	接入故障汇总和无故障反相信号	报警输入任一为 1 时 <code>no_fault=0</code>
5	接入安全链和启动允许逻辑	只有所有允许条件同时成立时 <code>allow_start=1</code>
6	接入缓冲、LED 或模拟执行负载	负载不改变内部逻辑结果，边沿和电平仍可测量

这个低速演示电路也能暴露真实工程问题。若未用 CMOS 输入悬空，LED 可能随机闪烁；若按钮没有消抖，逻辑分析仪可能看到多个启动请求；若把 LED 直接接到弱输出上，门输出的高电平可能被拖低；若两片推挽输出误接到同一节点，一高一低时会产生冲突电流；若所有芯片共用长电源线而缺少去耦，多个输出同时翻转时可能出现误触发。它们都不是布尔代数错误，却都属于第一章必须掌握的硬件错误。

三、从路径表得到逻辑门

门电路的第一步不是画符号，而是把输入组合列完整。两个输入 A、B 只有四种组合。电路必须对每一种组合给出明确输出，而且每一行都要满足前面的要求：输出节点不能悬空，不能上下冲突，必须能在规定时间内进入可靠电平区间。

从需求到门电路，建议固定采用五步。第一，给每个输入和输出写清楚硬件含义，特别注意别把低有效信号误读成高有效。第二，列出完整真值表，不跳过“看起来不会发生”的行——只按自然语言想象少数情况，是最常见的漏项来源。第三，标出输出为 1 的行，或者标出输出为 0 的禁止行，不遗漏边界输入和异常组合。第四，把每一行翻译成开关路径：哪些条件使上拉导通，哪些条件使下拉导通，不写只有公式、不看电流路径。第五，检查每一行是否有明确驱动、是否存在冲突、是否满足电平和延迟约束——只证明逻辑等价，不算证明硬件可靠。

以安全允许灯为例。A 表示保护盖合上，B 表示急停未按下。允许灯亮的条件很严格：任一条件不满足，输出都必须为 0。

A	B	允许灯	为什么
0	0	0	盖子没合上，急停也不是可运行状态
0	1	0	急停条件满足，但盖子没合上
1	0	0	盖子合上了，但急停条件不满足
1	1	1	两个安全条件同时满足

该表对应 AND 的行为。它不是抽象数学定义，而是“两个条件都满足才允许”的硬件约束。若某个实现让 A=1、B=0 时灯也亮，那就不是 AND 近似得不够好，而是安全行为错了。真值表的价值在于防止“只凭自然语言写电路”。“两个条件都满足”看起来简单，但实现时很容易漏掉某一行。

再看故障灯。A 表示温度报警，B 表示电流报警。故障灯的规则反过来：只要有一个坏消息出现，就要亮。

A	B	故障灯	为什么
0	0	0	没有任何故障报警
0	1	1	电流报警已经足够触发故障
1	0	1	温度报警已经足够触发故障

1

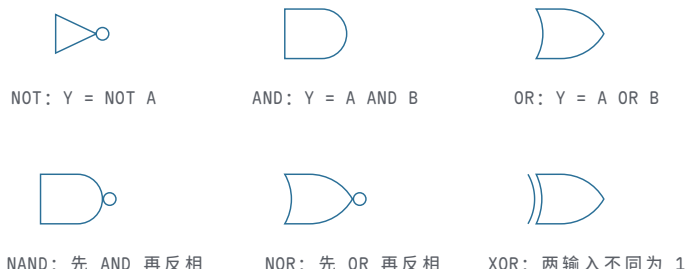
1

1

两个故障同时存在，仍应报警

该表对应 OR 的行为。AND 和 OR 的差别不在符号长什么样，而在需求不同：一个要求所有条件同时成立，一个要求任一条件成立即可。NOT 只有一个输入，但同样要列完整。若 A 表示“禁止信号”，输出 Y 表示“允许动作”，那么 $A=0$ 时 $Y=1$ ， $A=1$ 时 $Y=0$ 。它解决的是反向解释：输入出现时，输出反而关闭。

到这里六个基本门都已经出场，把它们的工程符号一次认全。前面刻意只用方框和路径表，是为了先建立“门 = 电流路径”的读法；从此刻起，本书改用标准符号：



图示：六个基本门的标准符号（美式画法）。注意输出端的小圆圈：它表示反相——NAND 就是 AND 加一个小圆圈，NOR 就是 OR 加一个小圆圈。这个小圆圈和晶体管开关模型里 PMOS 控制端的圆圈是同一个约定：有圈即取反。

不要把“0 是假、1 是真”机械套到每个硬件信号上。某些信号是低有效，例如 `reset_n`、`chip_select_n`，低电平才表示动作发生。读这类信号时，先写真值表，再谈门电路。

低有效信号值得单独处理。名字中的 `_n` 常表示 active low，即低电平有效。若信号名为 `emergency_ok_n`，那么 0 可能表示“急停链路正常允许运行”，1 反而表示“不允许”。这种命名并不适合作为入门案例的默认读法，但真实硬件中很常见，因为开漏输出、有线汇总、复位链路和片选信号都可能倾向于低有效形式。读低有效信号时，不能先把 1 写成“发生”、0 写成“未发生”，必须先读信号约定。

信号名	有效电平	读法
<code>reset_n</code>	0	低电平时复位动作发生
<code>chip_select_n</code>	0	低电平时芯片被选中
<code>output_enable_n</code>	0	低电平时输出驱动器打开
<code>fault_n</code>	0	低电平可能表示故障请求有效，需看约定

低有效并不只是命名习惯，它会改变推理方向。假设外部急停链路使用低有效输入 `emergency_stop_n`：链路正常释放时为 1，急停动作、断线或安全继电器打开时为 0。若内部逻辑需要的是正逻辑 `emergency_ok`，就必须先做一次语义转换：

```
emergency_ok = emergency_stop_n
stop_request = NOT emergency_stop_n
```

这里 `emergency_ok` 和 `stop_request` 都可以是正确内部信号，但它们表达的事实不同。前者适合放进 `allow_start` 的 AND 链；后者适合放进故障或停机请求的 OR 链。若在同一个表达式里混用低有效原始信号和正逻辑语义信号，很容易写出方向相反的网网络。正式硬件文档通常会在信号表里明确有效电平，就是为了防止这种错误。

真值表说清楚“应该怎样”，路径表说明“为什么会这样”。用受控开关看 AND，最自然的结构是串联。两个开关串在同一条路径上，只有 A 和 B 都让自己的开关闭合时，整条路径才通。OR 更像并联。A 控制一条路径，B 控制另一条路径。只要任意一路闭合，输出就能被拉向目标电平。

A	B	串联路径状态	AND 输出为什么成立或不成立
0	0	两个开关都断开	没有完整导通路径
0	1	A 断开, B 闭合	串联路径仍被 A 切断
1	0	A 闭合, B 断开	串联路径仍被 B 切断
1	1	两个开关都闭合	从源到输出的路径完整

如果只画一条“让输出为 1 的路径”，还没有完成门电路设计。输出节点不能靠悬空表示 0。因此每一行真值表还要补一张路径表，说明高电平路径和低电平路径的状态。一个合法门在稳定输入下，不应该让输出悬空，也不应该让上拉和下拉强路径同时导通。

从开关网络看，NAND 和 NOR 往往比 AND 和 OR 更像基础积木。原因是它们天然带反相输出，更容易形成互补的上拉和下拉结构：当下拉网络导通时输出被拉低；当下拉网络不导通时，上拉网络负责把输出拉高。

A	B	NAND 输出	开关直觉
0	0	1	下拉串联路径断开，上拉负责拉高
0	1	1	A 断开，下拉路径不完整
1	0	1	B 断开，下拉路径不完整
1	1	0	A、B 都闭合，下拉路径完整，输出被拉低

NOR 可以用同样方式读。NOR 的输出只有在 $A=0$ 且 $B=0$ 时为 1；只要 A 或 B 为 1，输出就为 0。若以下拉网络看，A、B 并联，只要任一路输入为 1，就能把输出拉低；只有两路都不导通时，上拉网络把输出拉高。

NAND 和 NOR 还可以帮助理解 De Morgan 定律。逻辑上， $\text{NOT}(A \text{ OR } B)$ 等价于 $(\text{NOT } A) \text{ AND } (\text{NOT } B)$ ；硬件上，这句话的含义是：只要 A 或 B 有一路故障报警，输出就应被拉低；只有 A 和 B 都没有故障报警，输出才允许被拉高。对于安全启动控制板，若 $\text{fault} = \text{temp_alarm OR current_alarm}$ ，那么“没有故障”就是 NOT fault ，也就是 $(\text{NOT temp_alarm}) \text{ AND } (\text{NOT current_alarm})$ 。这不是代数游戏，而是把“任一故障禁止启动”和“所有故障都不存在才允许启动”写成同一硬件约束的两种形式。

```
fault = temp_alarm OR current_alarm
no_fault = NOT fault
no_fault = (NOT temp_alarm) AND (NOT current_alarm)
```

只用 NAND 就能构造 NOT、AND、OR：

```
NOT A = A NAND A
A AND B = (A NAND B) NAND (A NAND B)
A OR B = (A NAND A) NAND (B NAND B)
```

这说明 NAND 在功能上完备，不说明所有实现都应该只用 NAND。真实设计会在功能正确的前提下，考虑门级数量、路径延迟、面积、功耗和负载。逻辑式等价，只说明稳定输出行为相同；它不保证延迟、功耗和驱动能力相同。

例如安全允许信号可以先写成：

```
allow_start = start_button AND cover_closed
              AND emergency_ok AND no_fault
```

若只用 NAND 实现，需要把多输入 AND 拆成若干 NAND 与反相结构。每拆一次，逻辑功能仍可保持，但门级层数、负载和延迟都会变化。因此“用 NAND 可以实现”

只是功能完备性结论；真正进入硬件设计时，还要继续检查最长路径和每一级扇出。

XOR 的输出在两个输入不同的时候为 1，相同的时候为 0。它解决的是“检测两个 bit 是否不同”这个具体问题。

A	B	A XOR B	解释
0	0	0	两个输入相同
0	1	1	两个输入不同
1	0	1	两个输入不同
1	1	0	两个输入相同

可以把 XOR 拆成两种会输出 1 的情况： $A=1$ 且 $B=0$ ，或者 $A=0$ 且 $B=1$ ：

$$A \text{ XOR } B = (A \text{ AND NOT } B) \text{ OR } (\text{NOT } A \text{ AND } B)$$

这条式子不要只当代数记忆。它对应两个乘积项：一项在“只有 A 为 1”时成立，另一项在“只有 B 为 1”时成立，最后的 OR 把两项合并。若两个输入都为 0，两项都不成立；若两个都为 1，两个“只有一个为 1”的项同样都不成立。XOR 后面会反复出现：两个 bit 相加时，和位就是“两个输入是否不同”；比较两个 bit 时，XOR 可以告诉我们它们是否不同；接一个 NOT，就能得到“两个输入是否相同”。

XOR 也说明了“常用门”和“便宜门”不一定一致。在纸面表达式里，XOR 是一个符号；在门级实现里，它往往比 AND、OR、NOT 更复杂，路径延迟也可能更长。因此，读到加法器、比较器、校验位或地址译码时，不能只数逻辑符号数量。若关键路径里连续经过多个 XOR，速度代价可能比同样数量的简单门更高。经典教材在讲门延迟时强调这一点，是为了让读者把布尔函数和实现代价同时放进脑中。

XOR 与 XNOR：门级构成与三个经典用途 XOR 既然比基本门贵，就值得先看清它贵在哪里。若器件库只有二输入 NAND，XOR 最少可以用 4 个 NAND 实现。办法不是凭空想的，而是把与或式逐步改写成 NAND 形式：

```
n = A NAND B
p = A NAND n      -- 等价于 (NOT A) OR B
q = B NAND n      -- 等价于 A OR (NOT B)
Y = p NAND q      -- 展开后正是 (A AND NOT B) OR (NOT A AND B)
```

按信号流向数级数： n 在第一级， p 、 q 在第二级， Y 在第三级——一个二输入 XOR 需要三级门延迟。若改用与或式直接搭建，需要 2 个反相器、2 个 AND、1 个 OR 共 5 个门，信号同样要依次经过反相、AND、OR 三级。两种搭法殊途同归：XOR 无论怎么实现，门数和级数都比 AND、OR、NAND 多。XNOR 只需在 XOR 之后再取反一次，门级成本再加一级反相。这就是前文“常用门不等于便宜门”的具体含义：表达式里一个清爽的符号，在门级就是三级延迟。

值得付出这个代价，是因为 XOR 回答的是算术与校验中最常问的问题：“两个 bit 是否不同”。三个经典用途几乎贯穿本书后续所有章节。

第一个用途是奇偶校验。8 位数据的偶校验位就是把 8 个 bit 全部异或起来：1 的个数为偶数时结果为 0，为奇数时结果为 1。8 个输入两两分组，用 7 个二输入 XOR 排成 3 层树（第一层 4 个，第二层 2 个，第三层 1 个）；若图省事排成一条链，则要 7 级延迟。树形结构门数不变、级数从 7 降到 3，这是“同一函数、不同拓扑、不同延迟”的又一个实例。接收端把收到的 8 位再算一次同一棵树，与随数据传来的校验位比较，任何单个 bit 出错都会让两边不一致——XOR 树因此是通信和存储系统里最小的检错硬件。

第二个用途是加法器的和位。后文全加器的 $S = A \text{ XOR } B \text{ XOR } C_{in}$ ，问的正是“A、B、 C_{in} 中是否有奇数个 1”；而进位 C_{out} 问的是“是否至少两个 1”，即三传感器判断器。一位全加器因此可以读成一个奇偶网络加一个多数网络：XOR 负责“奇数个”，AND/OR 负责“至少两个”。

第三个用途是相等比较。XOR 输出 1 表示“不同”，把它反过来：XNOR 输出 1 表

示“相同”。比较两个 4 位数是否相等，就是逐位 XNOR，再把四路结果用 AND 归并：

```
eq0 = A0 XNOR B0
eq1 = A1 XNOR B1
eq2 = A2 XNOR B2
eq3 = A3 XNOR B3
equal = eq0 AND eq1 AND eq2 AND eq3
```

归并的 AND 同样可以排成两级树而不是四输入单门，以控制扇入。后文数值比较器还会看到：相等信号 `eq_i` 不只用于判断全等，还是大小比较的裁决基础。

用途	回答的问题	关键结构	检查点
奇偶校验	1 的个数是奇还是偶	n bit 用 n-1 个 XOR 排成 $\lceil \log_2 n \rceil$ 级树	排成链则延迟随 位数线性增长
加法器和位	是否有奇数个 1	两级 XOR: A XOR B 再 XOR Cin	和位与进位是两 条不同延迟的路 径
相等比较	每一位是否都相同	XNOR 逐位加 AND 归并 树	归并树分级，避 免单门扇入过大

这张表也预告了一个反复使用的读图方法：看到 XOR，先问它在数“不同”还是数“奇偶”；看到 XOR 树，先数层数而不是门数。

真值表还可以用“最小项”来读。所谓最小项，就是与一行输入组合对应的完整乘积项。例如三输入 A、B、C 中，只有 A=1、B=0、C=1 这一行输出为 1，则对应的乘积项是 `A AND (NOT B) AND C`。把所有输出为 1 的行用 OR 合并，就得到一种直接实现。它不一定最简，但它有一个优点：每一项都能追溯到真值表中的具体一行。

A B C	输出 Y	对应乘积项
0 1 1	1	<code>(NOT A) AND B AND C</code>
1 0 1	1	<code>A AND (NOT B) AND C</code>
1 1 0	1	<code>A AND B AND (NOT C)</code>
1 1 1	1	<code>A AND B AND C</code>

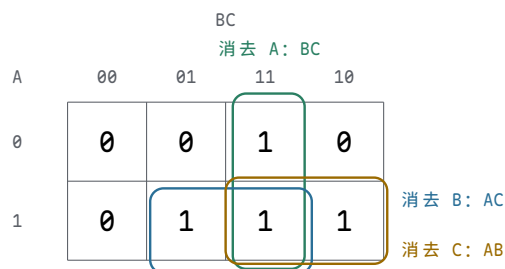
若把这些最小项直接相加，就能实现“三个输入至少两个为 1”的判断。但前文给出的表达式 `(A AND B) OR (A AND C) OR (B AND C)` 更短，因为它把若干行合并成“任意两项同时成立”的条件。这里体现出两个层次：最小项保证从真值表机械得到正确表达式，化简则减少门数和路径深度。化简以后仍要确认没有改变低有效含义、默认安全策略和毛刺风险。

卡诺图的核心可以用“相邻最小项”的文字规则理解。若两个最小项只在一个输入上不同，且其余输入完全相同，那么这个不同输入对输出为 1 并不是必要条件，可以被消去。例如：

$$(A \text{ AND } B \text{ AND } C) \text{ OR } (A \text{ AND } B \text{ AND NOT } C) = A \text{ AND } B$$

左边两项分别覆盖 C=1 和 C=0 两行；既然 C 的两种取值都让输出为 1，最终表达式就不必再依赖 C。卡诺图的格子相邻关系，本质上就是帮助设计者系统地寻找这种“只差一个输入”的合并机会。合并得当可以减少门数、缩短路径和降低负载；但合并之后必须重新检查低有效语义、非法状态和过渡行为。

文字规则必须落实到格子才算学会。以“三个输入至少两个为 1”为例，把 A 放在行、BC 放在列，每个格子填一行的输出：



图示：三变量卡诺图（注意列头顺序 00、01、11、10 是 Gray 码：相邻两列永远只差一个 bit，这样“相邻”才有消元意义）。三个圈各合并两个相邻的 1：每个圈里那个“两种取值都出现”的输入被消去，得到 $Y = AB \text{ OR } AC \text{ OR } BC$ ——与后文三传感器判断器用最小项硬推出的结果一致，但快得多。

化简还可能改变毛刺形态。某些冗余项对稳定真值表没有贡献，却能在输入翻转时提供连续路径。后文会看到， $(A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$ 在 $B=1$ 、 $C=1$ 且 A 翻转时可能短暂掉到 0；增加共识项 $B \text{ AND } C$ 不改变稳定真值表，却能减少这类静态毛刺风险。因此，安全相关组合逻辑不应把“项数最少”当作唯一目标。形式上最简的表达式，未必是危险输出最稳妥的实现。

有时真值表里会出现“不关心”项。所谓不关心，不是设计者懒得定义，而是某些输入组合在系统约定中不会被使用，或者该组合出现时输出 0/1 都不会影响可观察行为。例如一个 2-bit 故障编码只允许 00、01、10，保留 11 作为非法状态；若后级在 11 时一定进入停机诊断，那么某个普通指示灯在 11 下亮或灭可能都可以接受。化简逻辑时可以利用不关心项减少门数，但必须谨慎：一旦所谓“不可能”的输入来自外部连接器、未初始化状态或故障状态，它就不应轻易被当作不关心。

输入组合类别	可以怎样处理	必须确认的问题
正常组合	严格定义输出	是否覆盖了所有业务行为
保留组合	可定义为诊断、停机或不关心	后级是否真的不会误用该输出
非法组合	通常导向安全默认值	硬件上是否可能短暂出现
外部故障组合	不应随意当作不关心	断线、噪声、上电瞬间如何表现

对于安全启动控制板，保守做法是：任何未确认安全的组合，都不应让 `allow_start` 为 1。即使某个组合在正常操作流程中不会出现，只要它可能由断线、上电初态、输入调理失败或编码错误产生，就应进入“不启动”或“故障”一侧。组合逻辑的化简必须服从系统约定，而不是反过来让系统约定迁就最短表达式。

逻辑化简还要保留可审查性。对教学和安全控制而言，表达式稍长但能清楚对应需求，往往比极度压缩的表达式更容易审查。若某个输出必须由四个安全条件共同允许，那么把表达式保留为四个语义项，有助于后续检查每个条件来自哪里、默认值是什么、是否经过输入调理。化简可以减少门数和延迟，但不能让规格意图变得不可见。

四种常见表达方式各有取舍。逐项语义表达式容易对应需求和安全审查，但门数和层数通常不是最少；最小项展开可以机械追溯到真值表行，表达式却可能很长、延迟不一定好；代数化简式可能减少门数和路径层数，却可能隐藏低有效、默认值和故障策略；器件库映射式最接近实际可实现结构，但必须重新检查负载、扇入和时序。选择哪一种，取决于这段逻辑更需要可审查性还是更少的门级资源。

因此，正式设计常保留多个层次的描述：需求层写“所有安全条件满足才允许启动”；逻辑层写 `allow_start = start_request AND cover_closed AND emergency_ok AND no_fault`；实现层再说明由哪些门、缓冲和输出驱动器组成；验证层记录电平、延迟、毛刺和异常输入的检查结果。四个层次互相对应，才能避免“公式正确但系统含义错误”。

处理器控制逻辑中常见的 PLA，可以提前理解为“可规则化实现的组合逻辑”。PLA 的基本思想是先用一组 AND 项识别输入条件，再用 OR 项合成输出控制信号。例如

指令译码可以把操作码、状态位和时序阶段转成若干控制线：选择哪一路进入 ALU、是否写寄存器、是否读内存、下一步进入哪个控制状态。第一章不需要展开 PLA 版图，但读者应看出它和最小项方法的关系：先识别条件，再汇总结果。

PLA 视角	本章对应概念	在处理器中的用途
输入条件	真值表行、最小项、状态位	操作码、标志位、时序阶段
AND 平面	乘积项	某条指令或某类条件成立
OR 平面	乘积项汇总	产生写寄存器、读内存、选择 ALU 等控制线
输出控制	组合逻辑输出	驱动数据通路选择器和使能信号

8086 这类早期微处理器内部就包含大量译码和控制逻辑。它不是一堆孤立门的随意堆叠，而是把操作码解释、微操作顺序、总线周期和算术逻辑组合成可重复执行的机器动作。后面讲整机时会展开这些结构；在第一章，只需要先建立一个判断：处理器内部那些看似高级的控制信号，本质上也必须由可靠电平、门网络、组合路径和时序边界支撑。

8086 常被放在“完整 CPU”章节讲，但在第一章也能提前借用它的局部结构。它把取指相关的总线接口工作和执行相关的内部运算工作分开，并通过指令预取队列尝试让取指与执行重叠。这个案例说明：低时延并不只来自某个门更快，也来自**把等待路径重叠**。不过预取队列不能消除所有等待；分支改变控制流、总线访问受阻或指令消费速度超过预取速度时，执行仍会遇到空洞。

80386 的案例则说明另一件事：晶体管增加以后，芯片不只是把原有电路做得更快，还会把更多系统功能搬到片内。32-bit 数据通路、保护机制、地址转换和更复杂控制逻辑，都意味着更多组合判断、更多寄存器边界和更复杂的关键路径管理。处理器越复杂，越不能把“时延”只理解成单个门的传播时间；控制路径、数据路径、存储路径和外部接口路径都要分别分析。

还可以把几个经典芯片作为后续整机章节的预告。它们不需要在第一章展开完整微架构，但可以帮助读者把“门网络、组合控制、寄存器边界和片上集成”放到真实产品谱系中理解。

芯片	先观察的结构特征	与本章概念的关系
MOS 6502	早期微处理器，外部总线与内部控制紧密配合	指令执行依赖译码、数据通路选择和 控制时序
Zilog Z80	经典 8-bit CPU，寄存器组和总线控制较丰富	组合译码与状态机共同组织内存和 I/O 访问
Intel 8051	单片机把 CPU、存储器、I/O 和定时器放到同一芯片	片上集成减少板级连接，使控制信号 更局部
Intel 80486	在处理器内加入更强流水线、片上 cache 和浮点方向的集成	常用路径靠近执行核心，外部等待被 部分削减
Pentium	双流水线、分支预测和更复杂执行控制成为重点	低时延不仅靠门快，还靠预测、并行 和路径重叠

这些案例覆盖了从“CPU 依赖外部存储器和总线”到“更多功能进入片内”的演进。6502 和 Z80 让读者看到早期微处理器怎样把控制、数据通路和外部总线组织成机器周期；8051 说明 CPU 不一定孤立存在，片上 I/O 与定时器会把整机控制的一部

分直接收进芯片；80486 与 Pentium 则预告 cache、流水线和预测将成为降低常见等待的重要工具。后面讲整机时会展开这些芯片；第一章只要求先建立共识：无论芯片多经典，内部仍由可靠电平、门网络、组合路径和时序边界支撑。

四、门网络构成组合逻辑

组合逻辑的特点很直接：输出只由当前输入决定，不保存过去。它像一个硬件函数，但这个函数不是一步一步执行的过程，而是一张由门和连线组成的网络。输入变化后，信号沿多条路径同时传播；等延迟过去，输出稳定到当前输入对应的值。

设计组合逻辑时，可以把前一节的五步具体化为一条工作顺序。先列出输入输出，让每个 0/1 的含义明确；再列出行为，用真值表或条件表覆盖所有输入组合；然后写出逻辑表达式，让输出为 1 或 0 的每个条件都对应一条可实现路径；接着合成网络，用基本门、选择器和译码器明确每个输出的驱动者和选择者；最后检查物理限制——延迟、扇出、毛刺、电平余量，并说明后级在什么时候使用这个输出。这个顺序不要求一开始就写出最简表达式，而是先把需求完整展开，再逐步收缩为硬件网络。

先做一个三传感器判断器。输入 A、B、C 表示三个传感器，要求至少两个传感器为 1 时输出 1。这个需求不能只凭直觉画线，先列出行为：

A	B	C	Y	原因
0	0	0	0	一个条件都没有
1	0	0	0	只有一个条件
0	1	0	0	只有一个条件
0	0	1	0	只有一个条件
1	1	0	1	A 和 B 同时满足
1	0	1	1	A 和 C 同时满足
0	1	1	1	B 和 C 同时满足
1	1	1	1	三个都满足

从表中可以直接得到一个门网络：

$$Y = (A \text{ AND } B) \text{ OR } (A \text{ AND } C) \text{ OR } (B \text{ AND } C)$$

这条表达式包含三个乘积项。第一项在 A、B 同时为 1 时成立；第二项在 A、C 同时为 1 时成立；第三项在 B、C 同时为 1 时成立。最后的 OR 不是任意合并，而是在检查三个条件中是否至少有一个成立。按这种方式阅读，门网络就不只是公式变形，而是把真值表中每一种输出为 1 的原因接成硬件。

两个 bit 相加是第二个基本组合逻辑例子。输出不止一个 bit，因为 1+1 得到二进制 10，需要一个和位 S 和一个进位 C。

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

观察表格可以看到，S 在 A、B 不同时为 1，C 在 A、B 同时为 1：

$$\begin{aligned} S &= A \text{ XOR } B \\ C &= A \text{ AND } B \end{aligned}$$

这就是半加器。它已经完成了一位相加，但它还缺一个输入：来自低位的进位。只要要做多位加法，最低位之外的每一位都不能只看 A 和 B，还要看低一位是否进上来。

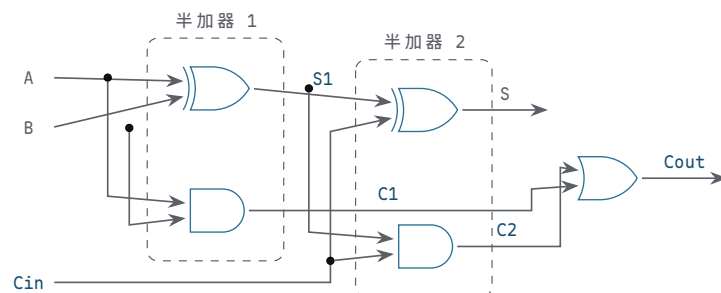
全加器输入为 A 、 B 、 C_{in} ，输出为 S 、 C_{out} 。可以先把 A 和 B 用半加器相加，得到临时和 $S1$ 和进位 $C1$ ；再把 $S1$ 和 C_{in} 用第二个半加器相加，得到最终和 S 和第二个进位 $C2$ ；最后把两个进位合并。

```
S1 = A XOR B
C1 = A AND B
S  = S1 XOR Cin
C2 = S1 AND Cin
Cout = C1 OR C2
```

先别急着看电路，把三输入八行真值表列全。加法本质是“数 1 的个数”：一个 1 都没有，和为 0；恰好一个 1，和为 1 不进位；两个 1，和位变 0、产生进位；三个 1，和位为 1 且进位：

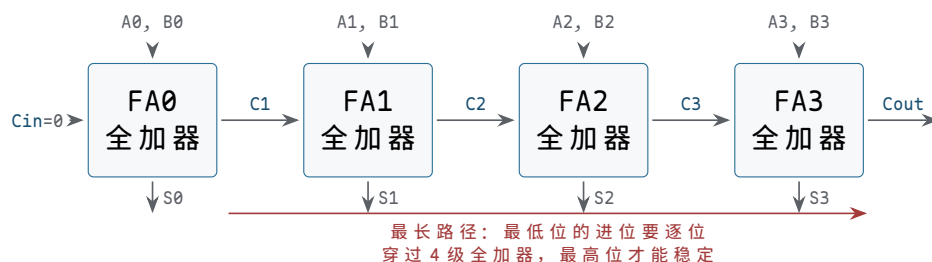
A	B	Cin	S	Cout	解释
0	0	0	0	0	没有 1
0	0	1	1	0	只有一个 1（来自进位输入）
0	1	0	1	0	只有一个 1
0	1	1	0	1	两个 1：本位写 0，向高位进 1
1	0	0	1	0	只有一个 1
1	0	1	0	1	两个 1
1	1	0	0	1	两个 1
1	1	1	1	1	三个 1：和位 1，再进 1

对照表格验证上面的式子： S 在“恰好一个 1”或“三个 1”时为 1，正是 $A \oplus B \oplus C_{in}$ ； C_{out} 在“至少两个 1”时为 1，正是三传感器判断器的结果。再把式子落成门，就是两个半加器加一个 OR：



图示：全加器 = 两个半加器 + 一个 OR。第一个半加器算 $A+B$ ，第二个半加器把临时和 $S1$ 再与进位输入 C_{in} 相加；两处的进位输出经 OR 合并——只要有一处产生进位，就向高位进 1。把这张图的每个输出和上面八行真值表逐行对一遍，比背公式可靠得多。

把全加器一位一位串起来，就得到串行进位加法器。它的逻辑很清楚：第 0 位算出进位，传给第 1 位；第 1 位再把进位传给第 2 位。问题也很清楚：若最低位产生的进位一路传到最高位，最高位必须等前面所有进位稳定后才能稳定。这是组合逻辑里第一次真正遇到“逻辑正确”和“速度足够”的差别。真值表保证最后结果正确，传播延迟决定结果多久之后才可靠。



图示：4 位串行进位加法器。每位的和位 S 只依赖本位输入，很快；但 $C4$ 依赖 $C3$ 、 $C3$ 依赖 $C2$ ……进位像接力一样从低位向高位逐级传播，红色线段就是决定全机速度那条最长路径。加法器位数越多，这条链越长——这正是第三章

要解决的矛盾。

全加器也可以用生成与传播来阅读。若 A 和 B 同时为 1，本位一定产生进位，称为生成；若 A 和 B 恰好一个为 1，那么本位不会自己产生进位，但会把输入进位传到输出，称为传播。

```
generate = A AND B
propagate = A XOR B
Cout = generate OR (propagate AND Cin)
S = propagate XOR Cin
```

这种写法为后面的加法器优化留下接口。串行进位把进位逐位传递，结构简单但最长路径随位数增长；更快的加法器会提前计算若干位的生成和传播关系，减少等待层数。本章只要求读者看清一个事实：组合电路不仅有“算什么”，还有“最慢路径从哪里穿过”。算术电路尤其如此，因为进位路经常常决定 ALU 的速度上限。

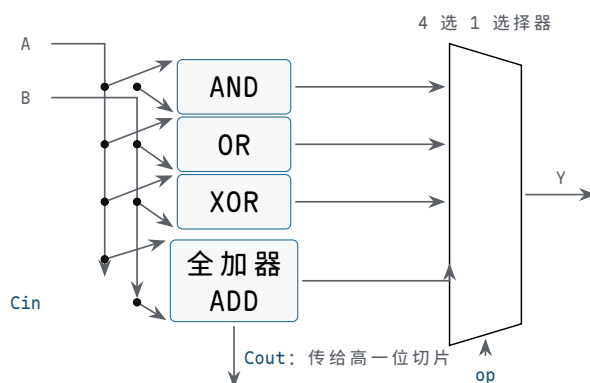
把全加器再向前推进一步，就得到 1-bit ALU 切片的雏形。ALU 不是不可分解的黑箱，而是许多位宽相同的切片并排工作：每一位接收本位的 A 、 B 、来自低位的进位，以及操作选择信号；它同时准备若干候选结果，例如 AND、OR、XOR 和 ADD；最后由选择器决定哪一种候选结果成为本位输出（选择器是由控制信号从多路输入中选一路输出的组合电路，下一小节马上讲到）。进位输出则继续传给高一位或交给更快的进位逻辑处理。

```
and_result = A AND B
or_result = A OR B
xor_result = A XOR B

sum_result = A XOR B XOR Cin
Cout = (A AND B) OR ((A XOR B) AND Cin)

Y = mux(op, and_result, or_result, xor_result, sum_result)
```

这段行为式只是说明逻辑关系，不表示硬件按行执行。真实电路中，AND、OR、XOR 和加法候选路径可以同时传播； op 控制选择器，让其中一路成为输出。于是，一个 1-bit ALU 切片至少包含三类组合结构：基本逻辑门、全加器、选择器。多个切片并排以后，AND/OR/XOR 这类逐位运算的延迟主要取决于本位门和选择器；ADD 则还受进位路径影响。



图示：1-bit ALU 切片：四路候选同时传播， op 控制选择器让其中一路成为本位输出 Y ；进位不进选择器，沿进位链直接传给高位切片。把这种切片并排 16 个就是 8086 的加法/逻辑单元雏形，并排 32 个、64 个，就逼近 80386 和现代执行单元——结构没变，只是宽度和进位优化在变。

减法通常不需要另造一套独立减法器。二进制补码中， $A - B$ 可以写成 $A + (NOT B) + 1$ 。硬件上常见做法是在 B 输入前放置一组可控反相器或 XOR 门：做加法时让 B 原样进入全加器，做减法时让 B 翻转，同时把最低位进位输入置为 1。这样同一条加法进位链既可服务 ADD，也可服务 SUB。

```
B_eff = B XOR B_invert
```

```
sum_result = A XOR B_eff XOR Cin
Cout = (A AND B_eff) OR ((A XOR B_eff) AND Cin)
```

操作	B_invert	最低位 Cin	本位输出含义
ADD	0	0	A + B 的和位
SUB	1	1	A + (NOT B) + 1, 即补码减法
AND	不使用或置 0	不使用	输出 A AND B, 进位链不作为结果来源
OR	不使用或置 0	不使用	输出 A OR B, 进位链不作为结果来源
XOR	不使用或置 0	不使用	输出 A XOR B, 进位链不作为结果来源

这张表说明**控制信号本身也是硬件约定**。若 op 选择 SUB, 但 B_invert 或最低位 Cin 没有同步改变, 结果会变成错误的加法变体; 若多位减法要生成借位、进位、零标志、符号标志或溢出标志, 还要明确每个标志按哪一种数值解释计算。第一章不展开完整标志逻辑, 只强调复用加法器并不意味着控制可以随意; 控制位、数据路径和标志解释必须一起确定。

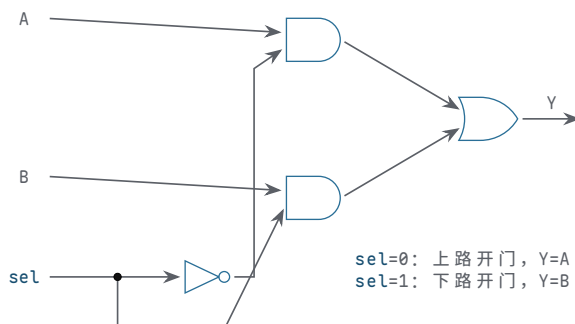
逐位逻辑操作路径最短: AND、OR、XOR 的本位输出只经过一两个门, 主要受负载和选择器影响; 它们适合逐位屏蔽、合并标志位、差异检测和翻转控制。加法路径则由全加器和进位链组成, 最长路经常由进位传播决定。选择器一层看似透明, 却是最容易被忽视的约定: op 必须稳定, 不能让错误候选短暂输出到危险后级。这个切片模型提前解释了 CPU 数据通路里的三个常见事实。第一, 位宽增加不只是“多写几个 bit”, 而是复制更多切片, 并重新处理进位、布线和负载。第二, 控制信号不是附属注释; op、进位选择、结果写回选择都会直接改变哪条硬件路径有效。第三, ALU 的延迟不是单个门延迟, 而是由候选路径、进位路径、选择器和输出负载共同决定。8086 的 16-bit 数据通路、80386 的 32-bit 数据通路, 以及现代 64-bit 执行单元, 都可以从这个切片观点继续放大的理解。

多路选择器解决“多个输入选一个输出”。2 选 1 选择器的行为是:

```
if sel = 0, Y = A
if sel = 1, Y = B
```

上述 if 只是描述行为, 电路本身没有顺序执行。它的门级表达式是:

```
Y = (NOT sel AND A) OR (sel AND B)
```



图示: 2 选 1 选择器的门级实现: sel 直接控制下路 AND, 反相后控制上路 AND——任何瞬间两路不会同时开门, 输出永远有明确来源。选择器的“选择”不是程序分支, 而是两扇由互补信号控制的门。

选择器的作用很基础。若一条输出线可能来自加法器, 也可能来自传感器, 也可能来自某个寄存器, 那么必须有一个明确的选择信号决定哪一路接到输出。没有选择器, 多路来源同时驱动同一条线就会冲突。

选择器的控制信号本身也需要检查。若 `sel` 悬空，输出可能在 A 和 B 之间随机变化；若选择信号来自毛刺严重的组合网络，输出可能短暂选择错误来源；若 A、B 属于不同电源域或不同稳定时间，选择器输出还会继承这些边界问题。因此，选择器不是“安全地把两根线接在一起”，而是“在一个可靠控制信号约束下，只让一路值成为输出”。

选择器检查项	问题	失败后果
选择信号默认值	上电或复位前选哪一路	输出来源不确定
数据输入稳定性	被选中的输入是否已稳定	选择了正确来源但值仍不可靠
未选输入行为	未选输入是否允许变化	不应通过内部耦合影响输出
输出负载	选择器是否能驱动后级	边沿变慢或电平不足

译码器做相反方向的事：把一个二进制编号展开成 one-hot 信号。2-bit 输入可以选中 4 条输出中的一条。

输入	输出
00	$Y_0=1, Y_1/Y_2/Y_3=0$
01	$Y_1=1$, 其他为 0
10	$Y_2=1$, 其他为 0
11	$Y_3=1$, 其他为 0

选择器把多路收成一路，译码器把编号展开成多路选择。后面无论是寄存器阵列、数据通路，还是存储器地址选择，都会反复看到这两个结构。

选择器还可以直接实现布尔函数。若一个函数有两个输入 A、B，可以用 A 作为选择信号，选择器的两个数据输入分别接“当 $A=0$ 时 Y 应该等于什么”和“当 $A=1$ 时 Y 应该等于什么”。例如 XOR 中，当 $A=0$ 时， $Y=B$ ；当 $A=1$ 时， $Y=\text{NOT } B$ 。因此一个 2 选 1 选择器加一个反相器即可实现 XOR：

```
if A = 0: Y = B
if A = 1: Y = NOT B
Y = mux(A, B, NOT B)
```

这种读法对后面的数据通路很重要。ALU 输入选择、写回选择、下一 PC 选择，本质上都可以看成“控制信号选择哪一路值成为输出”。选择器不是附属小部件，而是把多种候选硬件动作收束成一个当前动作的基本结构。

多路复用器：树形扩展与任意函数实现 一个 2 选 1 选择器只是一粒种子。回看它的门级结构：一个反相器、两个 AND、一个 OR——成本固定。把 2 选 1 当积木按树形堆叠，就能得到任意规模的选择器。4 选 1 用 3 个 2 选 1 排成两级：第一级两个，由选择信号低位 `S0` 分别在 `D0/D1`、`D2/D3` 中各选一路；第二级一个，由高位 `S1` 在两路胜者中选最终输出：

```
w0 = mux(S0, D0, D1)
w1 = mux(S0, D2, D3)
Y = mux(S1, w0, w1)
```

逐一代入可验证：`S1S0=00` 时 $Y=D_0$ ，`01` 时 $Y=D_1$ ，`10` 时 $Y=D_2$ ，`11` 时 $Y=D_3$ 。同一规律继续放大：8 选 1 用 7 个 2 选 1 排成三级，16 选 1 用 15 个排成四级。一般地， 2^n 选 1 需要 $2^n - 1$ 个 2 选 1，共 n 级：

规模	2 选 1 个数	级数	结构读法
2 选 1	1	1	最小单元
4 选 1	3	2	第一级两个并行，第二级一个汇总
8 选 1	7	3	第一级四个并行，逐级减半

16 选 1

15

4

数据输入指数增长，级数只线性增长

这张表藏着一个工程读法：选择器规模翻倍，数据输入数量翻倍，但延迟只增加一级——树形结构把指数增长的数据宽度换成了线性增长的级数。布线时还有一个讲究：最晚稳定的那一路选择信号应放在最后一级，让先到的数据先在低级门里传播。这与后文把晚到信号放在最后一级的做法是同一个原则。

多路复用器的第二个身份是通用函数发生器。一个 n 变量函数，可以把全部 n 个变量接到 2^n 选 1 的选择端，再把真值表逐行抄到数据端：第 k 个数据端接第 k 行的输出值。这样实现函数不需要任何化简——选择器本身就是一张接死的真值表。更省的办法是只把 $n-1$ 个变量接选择端，剩下的一个变量以四种形态之一出现在数据端：0、1、变量本身、变量的反。以三变量奇校验函数 $F(A, B, C)$ (A 、 B 、 C 中有奇数个 1 时输出 1) 为例，先用 8 选 1 实现，选择端 $S_2S_1S_0$ 接 A 、 B 、 C ：

A B C	最小项	F	数据端接法
0 0 0	m_0	0	D0 接 0
0 0 1	m_1	1	D1 接 1
0 1 0	m_2	1	D2 接 1
0 1 1	m_3	0	D3 接 0
1 0 0	m_4	1	D4 接 1
1 0 1	m_5	0	D5 接 0
1 1 0	m_6	0	D6 接 0
1 1 1	m_7	1	D7 接 1

再省一级：只把 A 、 B 接 4 选 1 的选择端，数据端改成 C 的函数。逐行检查每个 A 、 B 组合下 F 随 C 的变化：

A B	$C=0$ 时 F	$C=1$ 时 F	数据端接法
0 0	0	1	D0 接 C
0 1	1	0	D1 接 NOT C
1 0	1	0	D2 接 NOT C
1 1	0	1	D3 接 C

例如 $A=0$ 、 $B=1$ 时： $C=0$ 对应最小项 m_2 输出 1， $C=1$ 对应 m_3 输出 0， F 恰好等于 NOT C ，于是 D1 接反相后的 C ——其余三行同理。一片 4 选 1 加一个反相器，就装下了整张八行真值表。

这个身份的意义远超节省门数。可编程逻辑器件中的查找表，本质就是把数据端换成可写存储单元的多路复用器：改写存储单元，就改写了函数。读法上也要注意代价的另一面：用选择器实现函数是机械的，它不利用函数内部结构——八行真值表和八个随机值，对 8 选 1 来说是同一个价格。化简的机会被换成了实现的整齐。

选择器用集中的逻辑裁决“谁成为输出”，前提是所有候选来源都汇聚到一处。当候选来源分散在不同芯片、不同板卡上时，还有另一种组织方式：让每个来源挂在同一条线上，各自决定何时驱动、何时松手。接下来的两个专题分别考察这两种方式的两种输出级：三态与开漏。

三态输出与总线争用 到目前为止，每个输出级都遵循同一约定：任何时刻要么强驱动高电平，要么强驱动低电平，绝不允许悬空，也绝不允许上下同时导通。三态输出在这个约定上增加第三个合法状态：高阻态。使能端无效时，输出级的上拉管和下拉管同时关断，输出端既不供出电流也不吸收电流，对外表现为“像没接一样”——它不是 0，也不是 1，而是退出。使能端有效时，它就是一个普通的推挽输出。按惯例，使能常做成低有效，记为 OE_n ，与前文低有效信号约定一致：

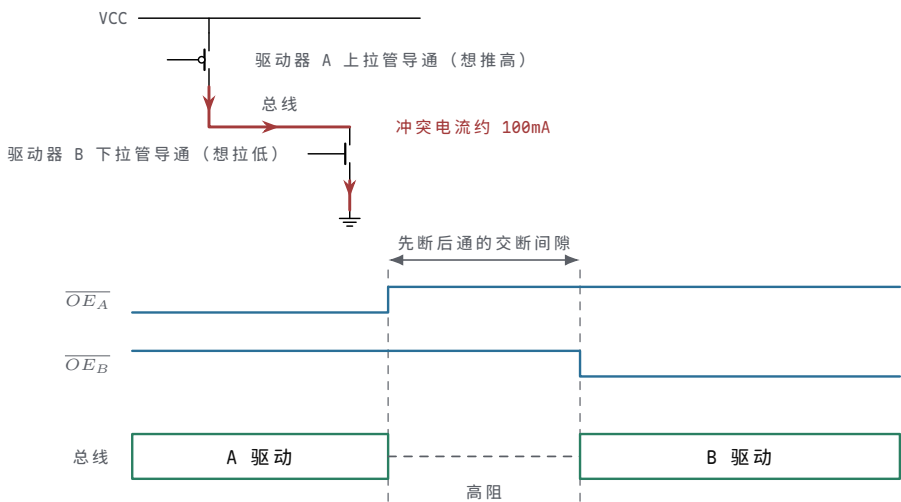
OE_n	D	Y	输出级状态
--------	---	---	-------

0	0	0	下拉管导通，强驱动低电平
0	1	1	上拉管导通，强驱动高电平
1	任意	Z	两管都关断，高阻，退出驱动

高阻态的价值在“共享”。若加法器、寄存器、外部接口都要把结果送到同一条数据线上，集中式选择器要求所有来源汇聚到一点；而给每个来源配一个三态输出，让它们平时处于高阻、轮到自己时才使能，同一条线就能被多个模块分时使用——这就是总线。总线能成立，靠的是一条硬约定：同一时刻最多一个驱动者。使能信号必须互斥，通常由译码器产生的 one-hot 或专门的仲裁逻辑保证。

违反这条约定的后果值得用电流算一遍。设两个三态驱动者同时使能，一个想推高、一个想拉低：电流从电源出发，经前者的上拉管、总线、后者的下拉管直到地，形成一条低阻通路。以每个输出级导通电阻约 25Ω 估算，这条通路上的电流约为 $5V/(25\Omega+25\Omega)=100\text{mA}$ ——比正常信号电流大一个数量级以上。此时总线电压停在中点附近，接收端读到的既不是合法高电平也不是合法低电平；大电流持续流过输出级，带来发热，反复或长时间争用会损坏器件。即使两个驱动者想驱动同一电平，超出额定的并联驱动也会使电平和边沿偏离数据手册的保证值。这就是总线争用：它不是逻辑错误——逻辑式里两个驱动者的值可以都正确；它是电气事故。

把这条冲突通路和正确的使能切换顺序画在一起，事故与预防各是什么模样就一目了然：



图示：上：两个三态驱动者同时使能时的冲突通路——电流从 VCC 经 A 的上拉管、总线、B 的下拉管直到 GND（图中只画出导通的管子，另一只方向的管子均关断），按每个输出级导通电阻约 25Ω 估算约 100mA ，总线电平停在中点附近，既不是合法高也不是合法低。下：正确的驱动权交接——A 的使能先撤、输出进入高阻，留出交断间隙后 B 才使能，任何时刻总线上最多一个驱动者。

争用是一个方向的故障，另一个方向的故障是无人驱动。若某一时刻所有使能都无效，总线悬空，电压会被漏电和耦合慢慢推走——这正是本章开篇悬空按钮问题的总线版本。工程上常用总线保持器兜底：它是挂在总线上的一个弱反馈锁存，最后一个驱动者留下什么电平，它就以微弱电流维持什么电平；新驱动者接管时，其强驱动轻易盖过这个弱保持，总线正常翻转。注意分工：总线保持器解决“无人驱动时漂移”，完全不解决“多人驱动时争用”——使能互斥仍是设计必须证明的性质，不能交给器件去兜底。

关键问题	预期结果
使能是否互斥	任何时刻至多一个 OE_n 有效，由译码器 one-hot 或仲裁逻辑保证
驱动器切换间隙	旧驱动器先进入高阻，新驱动器再使能，留出交断时间
无人驱动时	总线保持器或上拉/下拉电阻给出确定电平，不悬空

上电与复位期间 所有 OE_n 默认无效，总线默认空闲，不误导后级

后文讲整机时会看到，存储器、外设和 CPU 的数据引脚几乎都是三态的；一张原理图上几十根数据线的和平共处，靠的就是这里写下的互斥约定。

开漏输出与线与 三态输出的思路是“要么全力驱动，要么彻底退出”。开漏输出走另一条路：它只保留下拉管，干脆没有上拉管。输出管导通时，线被强拉到 0；输出管关断时，驱动器松手——注意不是输出 1，而是什么都不做。高电平由线路上唯一的公共上拉电阻提供：所有驱动器都松手后，电流经电阻向线路电容充电，电压回升到电源。

这个结构天然允许一条线挂多个驱动者。任何一个驱动者拉低，全线就是 0；只有全部松手，线才回到 1。把“松手”读成赞成、“拉低”读成否决，线上的电平就是全体的 AND——没有门的实体，连线本身完成了与运算，所以叫线与。与三态方案对比，差别很清楚：三态要求使能互斥，违反就是电源到地的争用事故；开漏不存在这个事故，最坏情况只是多个驱动者同时拉低，线依然可靠为 0。代价集中在高电平一侧：上升沿靠电阻慢慢充电，快不起来；拉低期间电阻上持续流过电流，白白耗电。

I2C 总线的 SDA 和 SCL 就是这个模型。每个设备都只能拉低或松手，起始、停止、应答全部是“谁在何时拉低”的约定；从机处理不过来时按住 SCL 不放，主机发现时钟线升不回去，就知道必须等待——这就是时钟延展。这类总线的动作电平因此都是低有效，与前文低有效信号约定出自同一个电气原因：低电平是有源强驱动，可靠且谁都能发起；高电平是电阻慢慢充出来的，只配当默认态。

上拉电阻的取值是开漏设计的主要权衡，它同时卡在两个约束之间。上升沿一侧：时间常数 $\tau = R \cdot C_{\text{bus}}$ ，电阻越大，回到高电平越慢。拉低一侧：电流 $I = V_{\text{DD}}/R$ ，电阻越小，拉低时耗电越多，且驱动器的吸收电流能力有限，电流超限会把低电平抬高、侵蚀低电平余量。以 $V_{\text{DD}} = 3.3\text{V}$ 、总线电容 100pF 为例：

上拉电阻	拉低电流 V_{DD}/R	时间常数 $\tau = R \cdot C$	适用场合
10 k Ω	0.33 mA	1000 ns	低速、省电优先的场合
4.7 k Ω	0.70 mA	470 ns	I2C 标准模式 (100 kHz) 常见取值
2.2 k Ω	1.5 mA	220 ns	I2C 快速模式 (400 kHz) 常见取值
1.0 k Ω	3.3 mA	100 ns	边沿更快，但已接近常见的 3 mA 拉低限值

电压升到约 70% 需要 1.2τ ：4.7 k Ω 时约 560 ns，尚在标准模式对上升沿的容限之内；总线电容更大或速率更高，就必须减小电阻，或者改用有源上拉电路。读表的方向不要反：不是“电阻越小越好”，而是在上升沿速度和拉低电流之间取交点。

到这里，三种输出级形态可以放在一起读：推挽输出任何时候都独占线路；三态输出靠互斥使能分时共享；开漏输出靠“只能拉低”让多驱动者和平共存。它们回答的是同一个问题——谁驱动这条线、何时驱动、没人驱动时谁来兜底。后文译码器产生的互斥 one-hot 信号，正是三态总线最常见的值班表；而开漏线把“兜底”直接做进了电气结构里。

译码器则常用于“编号变成单独使能”。例如 2-bit 编号选择 4 个寄存器中的一个写入，译码器输出 one-hot 写使能：同一时刻只允许一个目标被写。若译码器错误地产生两个 1，就可能同时写两个寄存器；若所有输出都是 0，本周期写操作会消失。后面寄存器阵列和控制器都会反复依赖这个性质。

结构	典型用途	必须保证
选择器	多个来源选一个输出	同一时刻只有一路被选中成为结果
译码器	编号展开成 one-hot 使能	合法编号只激活一个目标

编码器	多个请求压缩成编号	多请求同时有效时要有优先级或报错
比较器	判断相等、大小或条件成立	输入解释方式必须明确

编码器和比较器也是常见组合电路。编码器把 one-hot 或优先级请求压缩成编号；比较器判断两个值是否相等，或者按某种解释比较大小。安全启动控制板可以用简单编码器把故障来源压缩成故障码：没有故障为 00，温度故障为 01，电流故障为 10，若两者同时故障则输出优先级最高的一项或输出“多故障”码。关键在于规则必须写清楚，不能让两个输入同时为 1 时输出处于未定义状态。

temp_alarm	current_alarm	故障码	说明
0	0	00	无故障
1	0	01	温度故障
0	1	10	电流故障
1	1	11	多故障或最高优先级故障

若系统选择优先级编码，就要把优先级写进规格。例如温度故障比电流故障优先，则两个报警同时为 1 时输出温度故障码；若系统更重视诊断完整性，则两个报警同时为 1 时输出多故障码。二者没有绝对优劣，但不能在电路里含糊。

编码策略	规则	适合场景
优先级编码	多个请求同时有效时选择最高优先级	中断控制、快速响应最重要的请求
多故障编码	多个请求同时有效时输出专用多故障码	故障诊断和维护记录
非法状态报警	非 one-hot 输入触发错误输出	希望尽早暴露编码器前级错误

译码器、编码器与优先编码器 前文把译码器和编码器各用一段话带过，这两种结构值得展开，因为它们是“编号”与“选择”之间往返转换的标准件。

译码方向先一般化： n 位输入、 2^n 条输出，第 k 条输出正好是最小项 m_k ——它只在输入等于 k 时为 1。前文的 2 到 4 译码器是 $n=2$ 的特例；常见的 3 到 8 译码器（74HC138 一类）还带使能端，使能无效时所有输出一律无效。使能端给了译码器两个能力：级联，以及给片选加一个总开关。译码器的关键性质是输出互斥：合法输入下恰好一条输出有效。这个 one-hot 性质正好满足三态总线和寄存器写使能要求的“唯一驱动器”——译码器因此是“地址变片选”的通用件，存储器选体、外设选择、寄存器堆写入都靠它。

编码器走反方向： 2^n 条 one-hot 输入，输出 n 位编号，即“第几条线有效”。普通编码器要求输入严格 one-hot；若两条输入同时有效，输出没有定义。这不是电路的缺陷，而是输入约定被打破——编码器只承诺在约定之内给出正确编号。

优先编码器把这条约定放宽：允许多个输入同时有效，优先级最高者胜出，并额外给出一个标志 G ，表示“当前至少有一个请求有效”。以 4 到 2 优先编码器为例，约定 I_3 优先级最高、 I_0 最低：

I_3	I_2	I_1	I_0	Y_1	Y_0	G	说明
0	0	0	0	0	0	0	无请求， $G=0$
0	0	0	1	0	0	1	只有 I_0 请求
0	0	1	X	0	1	1	I_1 在场， I_0 被压住
0	1	X	X	1	0	1	I_2 在场，低两位被压住
1	X	X	X	1	1	1	I_3 一票定音

表中 X 表示该输入不影响本行输出。逐列读出布尔式：

```
Y1 = I3 OR I2
Y0 = I3 OR (NOT I2 AND I1)
G = I3 OR I2 OR I1 OR I0
```

验算两行：I2=1、I3=0 时，Y1=1、Y0=0，编号 10，且无论 I1、I0 如何都不变——这正是“压住”的含义；只有 I1=1 时，Y1=0、Y0=1，编号 01。G 与编号相互独立：全 0 输入时编号也是 00，全靠 G 区分“有请求、编号为 00”和“根本没有请求”。

优先编码器是中断请求排队的雏形。多个设备共用一路请求通道时，同时请求是常态而不是异常；固定优先级的编码器先报出最高优先级设备的编号，G 则直接充当“有待处理请求”的总信号。后文中断控制器会在此基础上增加屏蔽、嵌套和可编程优先级，但裁决的第一步就是这里这几行式子。

两种结构的级联也值得对照。译码器靠使能端扩展：两个 3 到 8 译码器，地址最高位 A3 直接接高位片的使能、反相后接低位片的使能，就得到 4 到 16 译码，输出仍然互斥。优先编码器靠 G 串联扩展：高位片的 G 决定低位片的编号是否被采用，全局优先级仍然唯一。选择器的树形扩展、译码器的使能级联、优先编码器的 G 串联，是同一个思想的三种形态：用少量控制端，把“同一时刻只有一个胜者”的性质扩展到大网络。

比较器也必须说明数值解释。同样的 bit 模式，用无符号数解释和用补码有符号数解释，大小关系可能不同。以 4-bit 为例，1111 作为无符号数是 15；作为补码有符号数是 -1。若比较器输出 less_than，规格必须说明它比较的是无符号大小、有符号大小，还是只比较 bit 模式是否相等。

比较类型	示例	说明
相等比较	<code>A == B</code>	只关心每一位是否相同，通常不涉及数值解释
无符号大小	<code>1111 > 0001</code>	1111 表示 15
有符号大小	<code>1111 < 0001</code>	1111 表示 -1，0001 表示 1
条件比较	报警级别是否超过阈值	必须说明编码范围和非法值处理

数值比较器与四变量卡诺图案例 相等比较只问“每位是否相同”，数值比较器还要多问一句“谁大”。裁决规则与字典序相同：从最高位开始逐位比较，第一对不同的位决定全局结果，更低的位不再有机会发言。以 4 位无符号数为例，先逐位求相等信号 `eq_i = A_i XNOR B_i`，再让高位的一票定音权逐级传递：

```
gt = (A3 AND NOT B3)
    OR (eq3 AND A2 AND NOT B2)
    OR (eq3 AND eq2 AND A1 AND NOT B1)
    OR (eq3 AND eq2 AND eq1 AND A0 AND NOT B0)

eq = eq3 AND eq2 AND eq1 AND eq0
lt = 与 gt 对称，A、B 互换即可
```

读 gt 的每一项：第一项说“最高位就分出胜负”；第二项说“最高位打平，则由次高位裁决”，依此类推。`eq_i` 在这里身兼两职——既是相等比较的输出，又是大小比较中“把裁决权交给下一位”的通行证。这与逐位进位链在结构上同构：都是高位优先、逐级传递，级数同样随位宽线性增长。

单片 4 位比较器（74HC85 一类）还带三个级联输入：`I(A>B)`、`I(A=B)`、`I(A<B)`。它们的作用是：当本片四位全部打平时，比较结果不由本片决定，而是透传级联输入。把低位片的三个输出接到高位片的级联输入，两片就串成 8 位比较器，四片串成 16 位——裁决权从最低位一路传递到最高位，与式子里的 eq 链严格对应。有符号与无符号的差别集中在最高有效位的解释上（补码的最高位是负权），级联结构本身不变。

比较器用到的“相等信号加传递链”是一种标准手法。下面换一个完整演算，把前文只在文字层面讲过的卡诺图，在一个含无关项的四变量函数上走完全程。题目：检测 8421 BCD 码是否大于等于 5。输入 A、B、C、D（A 为最高位），按约定只会出现 0 到 9；10 到 15 是无关项，记为 d。

先列完整真值表，一行都不跳：

十进制	A B C D	F	说明
0	0 0 0 0	0	小于 5
1	0 0 0 1	0	小于 5
2	0 0 1 0	0	小于 5
3	0 0 1 1	0	小于 5
4	0 1 0 0	0	小于 5
5	0 1 0 1	1	大于等于 5
6	0 1 1 0	1	大于等于 5
7	0 1 1 1	1	大于等于 5
8	1 0 0 0	1	大于等于 5
9	1 0 0 1	1	大于等于 5
10	1 0 1 0	d	无关项：约定不会出现
11	1 0 1 1	d	无关项
12	1 1 0 0	d	无关项
13	1 1 0 1	d	无关项
14	1 1 1 0	d	无关项
15	1 1 1 1	d	无关项

再填四变量卡诺图。A、B 沿行方向按 Gray 码排列，C、D 沿列方向按 Gray 码排列，每个格子填入对应最小项的函数值：

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	1	1	1
11	d	d	d	d
10	1	1	d	d

圈组的规则只有两条：每个圈必须是 2^k 个相邻格子组成的矩形，且每个 1 至少被一个圈覆盖；无关项可以当 1 圈进去，也可以当 0 留在圈外，取舍的唯一标准是能否把圈撑得更大。按这个标准可以圈出三组。第一组：A=1 的下面两行共 8 格（m8 到 m15），六个无关项全部圈入，得到单文字项 A—B、C、D 在圈内部取遍两种值，全部消去。第二组：B=1 且 C=1 的四格（m6、m7、m14、m15），其中两格是无关项，得到 BC。第三组：B=1 且 D=1 的四格（m5、m7、m13、m15），同样借两个无关项撑大，得到 BD。三个圈合起来：

$$F = A \text{ OR } (B \text{ AND } C) \text{ OR } (B \text{ AND } D) = A \text{ OR } (B \text{ AND } (C \text{ OR } D))$$

逐行验算：5=0101 由 BD 覆盖，6=0110 由 BC 覆盖，7 同时落在三个圈里（重复覆盖不改变函数），8、9 由 A 覆盖；0 到 4 各行 A=0，且 B、C、D 不满足任何一项，输出 0。全部符合真值表。

为什么这样选圈，而不是逐对合并？对照不用无关项的化简就清楚了。若把 d 全部当 0，m8、m9 只能两两合并成 $A \text{ AND } (\text{NOT } B) \text{ AND } (\text{NOT } C)$ ，m5、m7 合并成 $(\text{NOT } A) \text{ AND } B \text{ AND } D$ ，m6、m7 合并成 $(\text{NOT } A) \text{ AND } B \text{ AND } C$ ——同样是三个乘积项，但每项有三个文字，合计 9 个文字；借入无关项后，三项合计只有 5 个文字，门更小、扇入更低。无关项的价值就在这里：它不增加任何需要覆盖的 1，却允许把已有的 1 圈进更大的组里。

最后必须重申前文对不关心项的警告。这个化简把 10 到 15 实现成了 1——在

“输入只会是合法 BCD 码”的约定下无害；一旦输入可能来自外部连接器或故障状态，这个约定就可能被打破，届时 d 必须改回确定值重新化简。化简服从系统约定，而不是相反。

现在把贯穿案例写成一个完整组合网络。安全启动控制板至少需要两个输出：`fault_lamp` 和 `allow_start`。故障灯由报警信号决定；启动允许由启动按钮、安全条件和无故障共同决定。

```
fault_lamp = temp_alarm OR current_alarm
no_fault = NOT fault_lamp
allow_start = start_button AND cover_closed
               AND emergency_ok AND no_fault
```

这组表达式可以继续展开成门网络。第一层 OR 汇总故障信号；第二层 NOT 得到无故障条件；第三层多输入 AND 汇总启动按钮和安全条件。若硬件库里没有四输入 AND，可以用二输入 AND 逐级合成：

```
safe_chain = cover_closed AND emergency_ok
request_ok = start_button AND no_fault
allow_start = safe_chain AND request_ok
```

这种重写在逻辑上等价，但硬件上改变了门级分组。若 `cover_closed` 和 `emergency_ok` 来自同一个连接器，先合成 `safe_chain` 可能便于布线；若 `fault_lamp` 驱动多个后级，`no_fault` 前后可能需要缓冲。组合逻辑的表达式只是第一层结果，门级分组、负载和时序仍要继续检查。

若器件库只允许二输入 NAND，还可以把同一逻辑展开为 NAND-only 网络。每个 AND 由“一次 NAND 得到反相信号，再用 NAND 自反相恢复”为正逻辑。中间的反相信号必须命名清楚，否则很容易在后续维护中把 `safe_chain_n` 当作 `safe_chain` 使用。

```
n1 = cover_closed NAND emergency_ok
safe_chain = n1 NAND n1

n2 = start_request NAND no_fault
request_ok = n2 NAND n2

n3 = safe_chain NAND request_ok
allow_start = n3 NAND n3
```

NAND-only 写法证明了功能可实现，但不自动证明它就是最好的实现。线性串接、平衡二输入树和 NAND-only 分解在稳定真值表上可以等价，门级层数、局部负载、中间节点数量和毛刺形态却可能不同。平衡树通常缩短最长逻辑层数，线性结构可能更便于让某个晚到信号放在最后一级，NAND-only 结构可能更符合某些标准门芯片资源。正式规格应保留可审查的安全表达式，再单独说明实际门级映射，避免把实现技巧误写成需求本身。

顺这条链再检查一遍内部信号：`fault_lamp` 表示任一报警成立，OR 路径要能驱动指示灯和后级逻辑；`no_fault` 是它的反相，必须由反相器恢复为强 0/1；`safe_chain` 要求保护盖和急停都满足，任一断开时默认禁止；`request_ok` 要求有启动请求且无故障，按钮抖动不能传到后级；`allow_start` 是全部条件的汇合，后级何时采样、有无毛刺风险都要写明；`motor_enable_cmd` 是发给执行机构的命令，必须经过时序边界或专用安全链路，不能直接把组合输出接出去。

组合逻辑设计到这里仍然没有引入“步骤执行”。所有输入同时作用于网络，所有中间信号同时传播。若 `temp_alarm` 从 0 变 1，`fault_lamp` 会在 OR 路径延迟后变为 1，`no_fault` 会在反相器延迟后变为 0，`allow_start` 会在 AND 路径延迟后关闭。这个传播链条不是程序调用栈，而是多条电气路径的延迟叠加。

`motor_enable_cmd` 需要比 `allow_start` 更谨慎。前者是给外部执行机构的命

令，后者只是内部组合判断。直接令 `motor_enable_cmd = allow_start` 在最小示例里看似合理，但正式系统通常会增加时序边界、复位保持、驱动级和独立停机链路。原因很简单：组合判断可以短暂过渡，执行机构不应响应这种过渡。

```
allow_start = start_request AND cover_closed
              AND emergency_ok AND no_fault

motor_enable_cmd = registered_allow_start AND power_stage_ready
```

上式中的 `registered_allow_start` 表示已经在时钟边界保存过的允许结果，`power_stage_ready` 表示功率级自身已准备好并未报告故障。第一章还没有正式引入触发器，所以这里只把它作为边界预告：组合逻辑负责给出当前条件下的判断，执行机构命令通常还需要由时序逻辑和驱动电路接管。这样组织，能把“判断是否允许”和“真正驱动负载”分成两个可独立检查的层次。

可以把安全启动控制板整理成一份组合规格。规格不必一开始就写门级细节，但必须把输入语义、输出语义和异常策略写清楚：输入前提是板外触点已经完成默认值、电平和消抖处理，原始触点不直接进入安全组合式；允许条件是启动请求、保护盖闭合、急停正常、无故障四者缺一不可；故障输出要求温度或电流报警都点亮故障灯，同时报警要有明确编码策略；默认安全要求未知、断线、非法编码一律倒向禁止启动，化简逻辑不得破坏这条策略；使用时刻要求后级只在信号稳定后使用，异步使能必须额外做毛刺检查。这样做的好处是，之后无论用分立门、可编程逻辑，还是把同一逻辑放进更大的控制器，行为边界都保持一致。

下面给出这类小组合模块的完整书写顺序。它看起来比直接写一个公式更长，但每一步都对应一种常见错误：信号没定义、非法状态没处理、公式和需求不一致、实现后电气条件不满足。第一步，列出原始输入、内部语义信号和输出，不把物理触点直接当作可靠 bit。第二步，说明每个信号的有效电平和默认值，避免低有效、断线和上电默认错误。第三步，写出正常行为和异常行为，不让非法组合默默进入允许状态。第四步，写逻辑表达式或真值表，让需求可以机械检查。第五步，映射到门、选择器、译码器或编码器，明确驱动来源和选择关系。第六步，检查延迟、负载、毛刺、电平余量，证明实物能按时产生可靠输出。第七步，记录测量或仿真结果，让设计从文字规格变成可验证的对象。

把这七步用于 `allow_start`，可以得到更严格的文字规格：输入必须已经完成调理；`start_request` 为 1 表示一次经过消抖或同步确认的启动请求，缺失时不启动；`cover_closed` 为 1 表示保护盖闭合且传感器链路有效，缺失时不启动；`emergency_ok` 为 1 表示急停链路处于允许运行状态，缺失时不启动或停机；`no_fault` 为 1 表示没有温度或电流报警，缺失时不启动并报警；组合路径已越过最坏延迟，输出才算稳定，此前后级暂不采样、也不驱动执行机构。

若某个条件缺失却仍能启动，就是需求错误或实现错误；若输出尚未稳定就驱动执行机构，就是时序边界错误。组合逻辑的严肃性正在于此：它没有软件中的异常处理栈，只有电路在每一种输入情况下实际产生的电平。

为了让贯穿案例形成闭环，可以把安全启动控制板从外部物理输入一路写到执行命令。该表不是新增功能，而是把前面分散的原则合成一条可审查链路：原始线缆先进入电气边界，调理后得到内部语义信号，组合网络只处理已经可靠的 bit，危险输出再经过时序和驱动边界。

层次	典型对象	约定要求
原始输入	<code>start_button_raw</code> 、传感器触点、报警线	允许抖动和噪声，但必须有默认路径、保护和极性说明
输入调理	上拉/下拉、滤波、施密特触发、电平转换	把板外现象整理成稳定内部电平，不让悬空和慢边沿直接进入核心
语义信号	<code>start_request</code> 、 <code>cover_closed</code> 、 <code>emergency_ok</code>	每个 0/1 含义清楚，低有效信号先转换为可审查语义

组合判断	<code>fault_lamp</code> 、 <code>no_fault</code> 、 <code>allow_start</code>	真值表覆盖正常、异常、非法和默认状态，化简不破坏安全策略
时序边界	<code>registered_allow_start</code> 、复位保持、采样时刻	只在组合路径稳定后保存或使用结果，避免毛刺进入危险动作
输出驱动	<code>motor_enable_cmd</code> 、指示灯驱动、功率级使能	具备足够驱动能力，上电和复位期间保持禁止，故障时进入安全状态
验证记录	原理图、数据手册、示波器、逻辑分析仪、测试矩阵	说明电平、延迟、负载、毛刺和异常输入处理都满足约定

这张闭环表也说明为什么第一章要同时讲电平、器件、门和组合逻辑。如果只看组合表达式，会把 `start_request` 当作天然可靠的变量；如果只看器件，又难以说明为何这些输入最终必须合成为一个 `allow_start`。正式硬件说明应当让两者互相校验：每个语义信号都能追溯到电气边界，每个组合输出都能追溯到真值表和电流路径，每个执行命令都能追溯到时序和驱动电路。

端到端示例：用两片 74HC00 实现安全启动控制板 前面所有片段—真值表、NAND-only 展开、芯片引脚、噪声余量—现在可以合成一次完整演练。目标是让 `allow_start` 从自然语言需求一路走到可焊接的引脚分配。

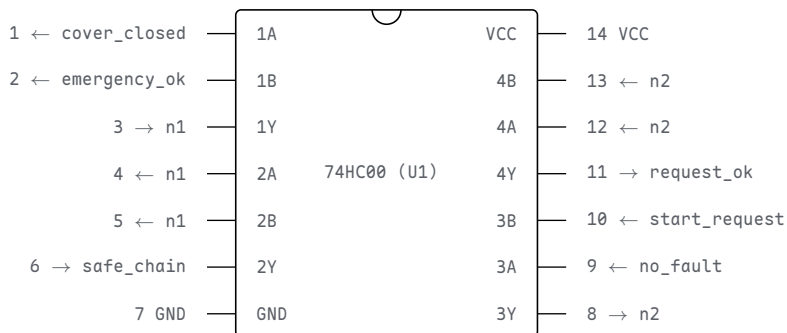
第一步，需求表达式（安全语义层，保持可审查）：

```
fault_lamp = temp_alarm OR current_alarm
no_fault = NOT fault_lamp
allow_start = start_request AND cover_closed
              AND emergency_ok AND no_fault
```

第二步，NAND-only 展开（实现层，每个反相中间信号命名清楚）。`fault_lamp` 与 `no_fault` 用一路 OR 芯片（如 74HC32）或二极管 OR 加缓冲实现即可，这里聚焦 AND 链：

```
n1 = cover_closed NAND emergency_ok
safe_chain = n1 NAND n1
n2 = start_request NAND no_fault
request_ok = n2 NAND n2
n3 = safe_chain NAND request_ok
allow_start = n3 NAND n3
```

第三步，数门。共 6 个二输入 NAND；一片 74HC00 有 4 路，需要两片。74HC00 的封装是 14 脚双列直插：左边 1-7 脚，右边 14-8 脚，缺口朝上时左上角是 1 脚。



图示：第一片 74HC00 的引脚分配：门 1 产生 `n1`，门 2 把 `n1` 自反相得到 `safe_chain`，门 3 产生 `n2`，门 4 把 `n2` 自反相得到 `request_ok`。缺口（上方半圆）决定引脚编号方向—原理图和实物靠它对应。

第二片（U2）完成最后两级：门 1 计算 `n3 = safe_chain NAND request_ok`，门 2 自反相得到 `allow_start`；门 3、门 4 未使用。未使用的 CMOS 输入必须接

确定电平（例如都接 VCC），否则悬空输入会让输出级随机翻转、白白耗电。

资源	分配	检查点
U1 门 1-2	<code>cover_closed</code> 、 <code>emergency_ok</code> 合成 <code>safe_chain</code>	中间信号 <code>n1</code> 是反相电平，不可误用
U1 门 3-4	<code>start_request</code> 、 <code>no_fault</code> 合成 <code>request_ok</code>	<code>no_fault</code> 来自故障汇总级
U2 门 1-2	<code>safe_chain</code> 、 <code>request_ok</code> 合成 <code>allow_start</code>	输出送时序边界，不直接驱动执行机构
U2 门 3-4	未使用	输入接确定电平，输出悬空不理
电源	14 脚 VCC、7 脚 GND，两片各自就近 $0.1\mu\text{F}$ 去耦	去耦电容贴近电源脚

第四步，电气参数核算。以 74HC00 数据手册在 4.5V 供电下的保证值为例：输出高电平最差约 $V_{OH} = 3.84\text{V}$ （4mA 负载），输出低电平最差 $V_{OL} = 0.33\text{V}$ ；输入高阈值 $V_{IH} = 3.15\text{V}$ ，输入低阈值 $V_{IL} = 1.35\text{V}$ 。逐级核算：

检查项	算式	结论
高电平余量	$3.84\text{V} - 3.15\text{V}$	约 0.69V，合格
低电平余量	$1.35\text{V} - 0.33\text{V}$	约 1.0V，合格
级联三级后	每级输出都重新驱动到 $3.84\text{V}/0.33\text{V}$	CMOS 恢复电平，余量不逐段衰减

注意第三行与二极管 OR 的本质差别：二极管每级吃掉 0.7V，三级后余量所剩无几；CMOS 门每级都把电平重新驱动到接近电源和地，余量被恢复——这就是为什么通用逻辑网络必须用有源门，而不是二极管逻辑。

至此，`allow_start` 可以回答本章开头的全部追问：需求来自真值表，实现落在两片具体芯片的 12 个引脚上，电气有余量核算，时序上它只送到时序边界而不是执行机构。这个从自然语言到引脚的完整链条，就是本章要达到的目标。

端到端示例二：三级报警优先权电路 上一例回答的是“所有安全条件同时成立才允许动作”，还有另一类同样常见的需求：多个请求可能同时到来，必须按优先级裁决，高优先级抢占输出并屏蔽低优先级。本例用一片 74HC04、一片 74HC00 和一片 74HC32，把一台设备的三级报警面板从需求走到芯片分配，流程与上一例完全相同。

需求如下。面板有三个请求源：火警 `fire`（最高优先级）、故障 `fault`、提示 `info`（最低优先级），对应三盏指示灯 `fire_lamp`、`fault_lamp`、`info_lamp`。任一时刻只允许点亮当前最高优先级的那一盏：高优先级请求出现时立即抢占，并把低优先级输出强制为 0。注意屏蔽不是“盖住显示”，而是让低优先级输出在逻辑上就是 0——这样无论后级接的是灯、蜂鸣器还是记录电路，看到的都是被裁决过的结果。全部请求消失时三盏都熄灭。

先列完整真值表。三个请求共八种组合，裁决结果如下：

请求 (fire, fault, info)	输出 (fire/fault/info_lamp)	裁决结果
0, 0, 0	0, 0, 0	无请求，全部熄灭
0, 0, 1	0, 0, 1	只有提示
0, 1, 0	0, 1, 0	故障点亮，提示未请求
0, 1, 1	0, 1, 0	故障屏蔽提示
1, 0, 0	1, 0, 0	火警独占
1, 0, 1	1, 0, 0	火警屏蔽提示

1, 1, 0	1, 0, 0	火警屏蔽故障
1, 1, 1	1, 0, 0	火警屏蔽全部

从真值表直接读出化简结果。`fire_lamp` 在 `fire=1` 的四行全为 1，与另两个输入无关，所以 `fire_lamp = fire`——最高优先级输出不经过任何门，路径最短，这正符合它的安全地位。`fault_lamp` 只在 (0,1,0) 与 (0,1,1) 两行为 1，化简得 `fault AND (NOT fire)`：火警的反相信号作为一个输入项，它一来，故障灯就被逻辑本身关断。`info_lamp` 只在 (0,0,1) 一行为 1，即 `info AND (NOT fire) AND (NOT fault)`；用德摩根律改写成 `info AND NOT(fire OR fault)`，可以省下一级三输入 AND：

```

fire_n      = NOT fire
m1          = fault NAND fire_n      -- 74HC00 门1
fault_lamp  = NOT m1                 -- 74HC04
any_hi      = fire OR fault          -- 74HC32 门1
any_hi_n    = NOT any_hi             -- 74HC04
m2          = info NAND any_hi_n     -- 74HC00 门2
info_lamp   = NOT m2                 -- 74HC04
any_alarm   = any_hi OR info         -- 74HC32 门2，校验输出
fire_lamp   = fire                   -- 直连，不经门

```

芯片分配如下。74HC04 (U1) 用四路反相器，产生 `fire_n`、`fault_lamp`、`any_hi_n`、`info_lamp`；74HC00 (U2) 用两路 NAND，产生 `m1`、`m2`；74HC32 (U3) 用两路 OR，产生 `any_hi`、`any_alarm`。U1 门 5-6、U2 门 3-4、U3 门 3-4 未使用，输入一律接确定电平。其中 `any_alarm` 是故意多算的校验输出：它在逻辑上应恒等于 `fire OR fault OR info`，也应恒等于三盏灯输出的 OR——同一个事实沿两条独立路径各算一遍，上电测试时逐行比对，就是一种廉价的一致性检查。

资源	分配	检查点
U1 (74HC04) 门 1-4	<code>fire_n</code> 、 <code>fault_lamp</code> 、 <code>any_hi_n</code> 、 <code>info_lamp</code>	门 5-6 未用，输入接确定电平
U2 (74HC00) 门 1-2	产生 <code>m1</code> 、 <code>m2</code>	门 3-4 未用，输入接确定电平
U3 (74HC32) 门 1-2	产生 <code>any_hi</code> 、 <code>any_alarm</code>	门 3-4 未用，输入接确定电平
<code>fire</code> 直连	<code>fire_lamp</code> 不经门	最高优先级路径最短；灯的电流由驱动级承担，不经过逻辑门
裁决延迟	<code>fire</code> 到 <code>fault_lamp</code> 三级门、到 <code>info_lamp</code> 四级门	延迟沿路径逐级累加，定量核算见第五节
电源	每片 14 脚 VCC、7 脚 GND，就近 0.1μF 去耦	去耦电容贴近电源脚，理由见第五节专题
一致性	<code>any_alarm</code> 与三灯输出逐行比对	两条独立路径的结果必须一致

与上一例对照可以看到两点。第一，优先级不是额外的器件，而是写在真值表里的屏蔽关系：高优先级输入以反相形式出现在低优先级的乘积项里，它一来，低优先级输出就被逻辑本身关断。第二，优先级越高的输出经过的门越少——`fire_lamp` 零级、`fault_lamp` 最多三级、`info_lamp` 最多四级——“越重要的信号越快”在这里是自然结果，不需要刻意安排。至于三级门到底慢到什么程度，正是第五节要算的账。

把同一思路放大到 8086 和 80386，就能看出早期处理器为什么需要大量组合模块。以 Intel 历史速查表所列参数为例，8086 于 1978 年引入，约 29,000 个晶体管，采用约 3 微米制造技术；Intel386 DX 于 1985 年引入，约 275,000 个晶体管，制造技术从约 1.5 微米进入后续 1 微米级别。这些数字本身不是本章重

点，重点在于它们对应的结构变化：晶体管数量增加以后，可以容纳更宽的数据通路、更复杂的译码、更丰富的寄存器和更强的总线控制；工艺尺寸缩小以后，门和连线的寄生电容下降，单位距离连接更短，许多路径的延迟随之降低。

芯片	年代与规模	本章可观察的硬件变化	对时延的意义
8086	1978 年，约 29,000 晶体管	16-bit 数据通路、指令译码、总线接口	已经需要大量组合控制与寄存器边界配合
80386	1985 年，约 275,000 晶体管	32-bit 数据通路、更复杂保护和地址机制	更多片上逻辑减少外部协作等待，关键路径需重新组织
现代 CPU	数十亿级晶体管	多级缓存、乱序执行、预测、片上互连	不只让门更快，还减少取数和等待的次数

这里要避免一个误解：现代芯片低时延不是因为“逻辑门不再有延迟”。门仍然有延迟，连线仍然有电容，存储器仍然需要访问时间。现代芯片做的是把延迟压低、隐藏、分层和局部化——这四个动词的具体含义放在本章第五节末尾展开，这里先记住结论：低时延来自让常见情况少走远路，而不是让所有路径同样快。

五、延迟、负载、毛刺与检查

门电路在纸上像瞬间给出输出的函数，真实硬件却至少有四个限制。第一，输入变化后，输出需要传播延迟才稳定，组合逻辑层数越多，稳定越慢。第二，一个输出能驱动的后级输入数量有限，后级太多会让切换变慢，甚至让电平余量变差，大网络需要缓冲器和分层布线。第三，门的输出必须重新形成清楚的 0/1，不能只靠一个偏弱路径勉强改变节点电压。第四，多条路径延迟不同，输入变化过程中可能出现短暂毛刺，后级不能在毛刺期间采样。

考虑一个反例。若把十几个门的输入全接到同一个弱输出上，真值表仍然可以保持正确，但物理电路可能切换缓慢。输出从 0 变 1 时，要给所有后级输入电容充电；如果下一级在它尚未进入可靠高电平区间时读取，就可能看到旧值或不确定值。因此，分析组合逻辑时不能只数门的个数，还要考察最长路径、负载和稳定时间。

扇出问题可以放在安全启动控制板里理解。若 `fault_lamp` 只驱动一个小逻辑门，边沿可能很快；若它同时驱动指示灯缓冲、记录电路、禁止启动网络和外部连接器，等效负载明显增加。正确做法通常不是让一个弱节点直接驱动所有后级，而是分层缓冲：一个内部故障信号先驱动若干缓冲器，再由缓冲器分别驱动不同区域。缓冲器不改变逻辑含义，却改变驱动能力和负载隔离。同理，长线不能直接接到敏感输入上，容易耦合噪声和振铃，需要加强驱动、终端处理或同步采样；普通逻辑输出也不能直接承担指示灯负载，要用驱动级把指示灯的电流需求隔在逻辑电平之外。

短暂毛刺也很常见。假设一个输出由多条逻辑路径合成，输入同时变化时，每条路径稳定的时间可能不同。真值表只告诉我们变化前和变化后的稳定结果，却不保证中间不会短暂出现错误电平。如果后级只是另一个组合门，毛刺可能很快消失；如果后级在毛刺期间被时钟采样，错误就可能被保存下来。后面讲触发器和时钟时，会把这个问题放大讨论。

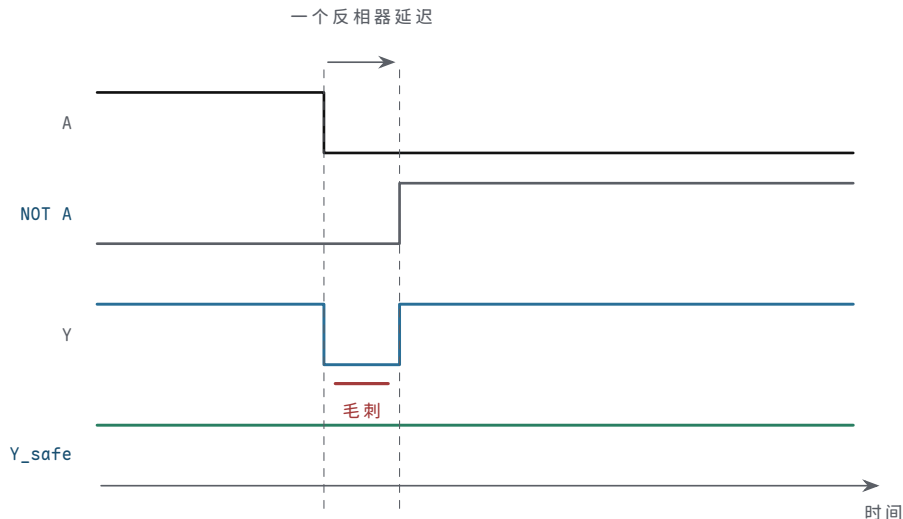
一个典型毛刺表达式是：

$$Y = (A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$$

假设 $B=1$ 、 $C=1$ 。稳定状态下，不论 $A=1$ 还是 $A=0$ ， Y 都应为 1： $A=1$ 时第一项成立， $A=0$ 时第二项成立。问题出现在 A 翻转的瞬间。若 A 从 1 变 0，第一条路径可能先关断，而 `NOT A` 经过反相器后才让第二条路径打开，中间可能出现一个短暂间隙，使 Y 从 1 短暂掉到 0。真值表看不见这个瞬间，因为真值表只描述稳定

输入和稳定输出。

时刻	路径状态	Y 的风险
A=1 稳定	A AND B 成立	Y=1
A 正在翻转	第一条路径可能已关， 第二条路径尚未开	Y 可能短暂为 0
A=0 稳定	NOT A AND C 成立	Y=1



图示：B=C=1 时，Y 的稳定值本应从 1 到 1 不变。但 A 翻转到 NOT A 生效之间隔着一个反相器延迟：上路先关、下路未开，Y 出现一个真值表看不见的窄 0 脉冲。加入共识项 B AND C 后 (Y_safe)，有一条不依赖 A 的路径全程把输出钉在 1。

若这个 Y 只是驱动另一个组合网络，并且后级只在更晚的时钟边界采样，毛刺可能不会造成状态错误；若 Y 直接作为写使能、复位、片选或异步控制信号，短暂毛刺就可能触发真实动作。安全启动控制板里的 allow_start 尤其需要谨慎：即使最终会回到正确值，也不能让一个窄脉冲误开电机使能。

上面的毛刺可以用增加共识项来消除。表达式 $Y = (A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$ 在 B=1、C=1 且 A 翻转时有静态 1 毛刺风险。加入第三项 B AND C 后，A 翻转期间仍有一条不依赖 A 的路径保持 Y 为 1：

$$Y_{\text{safe}} = (A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C) \text{ OR } (B \text{ AND } C)$$

这不是为了改变稳定真值表。因为当 B AND C 为 1 时，原表达式在 A=0 或 A=1 下本来也为 1；新增项只是在过渡期间提供连续路径。代价是多一个门和更多负载。是否值得加入，取决于这个输出是否会直接驱动危险控制线，或者是否会在毛刺窗口内被采样。

危险信号需要分级处理。并非所有组合输出都必须做成完全无毛刺：普通数据位通常可以容忍短暂过渡，只要后级在稳定的时钟边界采样；状态指示灯容忍度更高，短暂闪动人眼看不见，主要关心驱动能力；写使能和片选容忍度低，应避免异步毛刺，必要时寄存后再使用；复位、停机和电机使能容忍度极低，需要安全默认值、路径冗余和严格的时序检查。对这些信号，常见策略包括：把组合结果先送入触发器，在时钟边界后再使用；避免用毛刺风险高的组合式直接产生异步控制；必要时加入共识项或专用使能门；对外部执行机构再增加独立保护链路。

组合路径还需要时间预算。假设某个输出要在 20ns 内稳定，而路径包含输入缓冲、两级门、一级选择器和输出缓冲，就要把最坏延迟相加。下面的数字只是说明这种预算方法，不代表某个工艺或芯片规格。

路径段	最坏延迟	说明
输入缓冲	2ns	外部信号进入内部逻辑

第一层门	3ns	例如报警 OR 或安全条件 AND
第二层门	4ns	例如 no_fault 与请求合并
选择器或缓冲	5ns	输出选择或驱动增强
布线和负载	3ns	走线、电容、扇出影响
合计	17ns	若要求 20ns 内稳定，则余量 3ns

负载也应纳入同一份时间预算。假设一个输出驱动 8 个 CMOS 输入，每个输入电容按 5pF 估算，走线和封装再贡献约 20pF，则总负载约为 60pF。若输出级等效电阻按 100Ω 粗估，时间常数约为 6ns；若后级要求在 10ns 内越过阈值，这条连接已经没有什么余量。若通过缓冲把 8 个后级分成两组，每个缓冲只承担较小负载，边沿和电平余量通常更容易满足。这个例子不替代仿真和数据手册，但它说明扇出不是“能接几个门”的静态数字，而是电流、电容、边沿和采样时刻共同形成的约束。

估算项	数值	结论
输入电容	$8 \times 5pF$	后级越多，总电容越大
走线和封装	约 20pF	板级和封装不是零负载
总负载	约 60pF	边沿速度需要重新估算
等效 RC	$100\Omega \times 60pF \approx 6ns$	采样窗口紧时需要缓冲或分层驱动

扇出估算：每个后级输入都是一只小电容 上表把负载当成一个给定的 60pF，这个数字从哪里来？答案是一条经验估算规则：每个 CMOS 输入贡献约 3–5pF（栅电容加引脚封装），每厘米走线对地平面再贡献约 1pF。一个输出驱动 10 个输入、走线 15cm，负载约为 $10 \times 4pF + 15 \times 1pF \approx 55pF$ 。驱动这样一个输出，本质上就是给这只总电容充电和放电，扇出问题的全部后果都从这一只电容展开。

边沿快慢可以用一阶 RC 粗估。从 74HC 数据手册的输出压降可以反推输出级等效电阻：额定 4mA 负载下 V_{OL} 约 0.2V，对应约 50Ω 。于是时间常数 $\tau = R \cdot C = 50\Omega \times 55pF \approx 2.8ns$ ，10%–90% 边沿约为 $2.2\tau \approx 6ns$ 。若扇出加倍到 20 个输入、走线 20cm，负载约 100pF，边沿拖到约 11ns——已经与一级门自身的传播延迟相当，等于白白多串了一级门，而这级“门”不贡献任何逻辑功能。

估算项	规则与算式	结果
后级输入电容	每输入 3–5pF, $10 \times 4pF$	约 40pF
走线电容	约 1pF/cm, $15cm \times 1pF$	约 15pF
合计负载	$40 + 15$	约 55pF
边沿时间	$2.2 \times 50\Omega \times 55pF$	约 6ns
扇出加倍后	$2.2 \times 50\Omega \times 100pF$	约 11ns，堪比一级门延迟

高扇出还直接推高功耗。每次翻转搬运的电荷是 $Q = C \cdot V$ ，动态功耗为 $P = C \cdot V^2 \cdot f$ ：55pF 在 5V、1MHz 下约 1.4mW，一块板上有几十根这样的重负载信号线，就是几十毫瓦白白发热。更隐蔽的是第二重代价：边沿变慢以后，后级输入在阈值附近停留的时间变长，接收门内部上下管同时微导通的窗口随之拉长，产生额外的短路功耗——慢边沿不仅自己慢，还让接收它的每一扇门更耗电。

缓冲器解决的正是这个物理问题。它不改变逻辑值，却把一个重负载拆成几个轻负载：先由普通输出驱动缓冲器，再由缓冲器分组驱动各片后级，每条分支电容小、边沿快，局部延迟反而更可控。代价是缓冲器自身的传播延迟要计入路径，所以缓冲层级不是越多越好——这又是一次功能、负载与延迟之间的取舍，与“逻辑上保持原值”并不矛盾。

若后续系统在 15ns 时就使用这个输出，稳定真值表仍然不够；输出可能还在传播或毛刺阶段。第二章要引入的触发器和时钟边界，正是为了规定“什么时候保存”。组合逻辑在边界之间准备结果，时钟边界只应采样已经稳定、满足建立时间要求的信号。

时间预算要使用最坏情况，而不是典型情况。最小延迟说明某条路径最快可能多久变化，用来检查保持时间、窄脉冲和过早到达；典型延迟是常见条件下的大致延迟，便于估算，但不能作为设计边界；最大延迟是不利条件下最晚多久稳定，用来检查时钟周期和采样时刻是否足够；偏斜是不同路径或不同信号到达时间的差，用来解释毛刺、竞争和采样风险。若一条路径在实验室常温下 12ns 稳定，并不能证明它在低电压、高温、最大负载下也能在 15ns 内稳定。正式规格通常同时给出最小、典型、最大值；做时序分析时应以满足系统要求的最坏组合为准。

为了衔接下一章，还需要先建立几个时序词汇。建立时间指数据在采样时钟边沿到来前必须提前稳定的最短时间——结果不能刚到边沿才变化；保持时间指数据在采样时钟边沿之后仍必须继续稳定的最短时间——结果不能过早被下一条路径改写；时钟到输出指触发器被时钟采样后输出变化所需的时间，它是新数据进入下一段组合逻辑的起点；时钟偏斜指不同触发器看到同一时钟边沿的时间差，它会压缩某些路径的有效时序余量。

完整同步时序检查将在第二章展开。本章只使用最基本的判断：组合路径的最大延迟不能侵占建立时间，组合路径的最小延迟不能破坏保持时间，时钟偏斜不能被假定为零。换言之，组合逻辑不是“算完就行”，而是必须在规定的时钟边界之前算完，并且在边界附近保持稳定。

传播延迟沿路径累加：一条五级门的核算 前面时间预算表里的数字是示意性的，现在用真实器件把同一条账重算一遍。回到安全启动控制板：从板级连接器到 `allow_start` 的最长路径，经过输入调理反相（一片 74HC04）和四级 NAND（两片 74HC00 依次产生 `n1`、`safe_chain`、`n3`、`allow_start`），共五级门。链式结构有一个简单却关键的性质：前一级的输出越过阈值后，后一级的延迟才开始计时，所以总延迟近似等于各级传播延迟之和。

查 74HC 系列数据手册（4.5V–5V 供电、50pF 负载电容、25°C 附近的标称条件），这一档门的单级传播延迟典型值约 9ns；手册同时给出保证最大值约 20ns，覆盖工艺离散、全温度范围和电压波动的最不利组合。不同厂商、不同温度档次的具体数字略有出入，正式设计必须以手头器件的手册为准，这里取代表值是为了说明算法。

路径级	器件与作用	典型	最大
第 1 级	74HC04，输入调理反相	9ns	20ns
第 2 级	74HC00，产生 <code>n1</code>	9ns	20ns
第 3 级	74HC00，自反相得 <code>safe_chain</code>	9ns	20ns
第 4 级	74HC00，汇合得 <code>n3</code>	9ns	20ns
第 5 级	74HC00，自反相得 <code>allow_start</code>	9ns	20ns
合计	五级串联	45ns	100ns

两个合计数回答两个不同的问题。45ns 回答“台架上大概会看到什么”：实验台恰好常温、额定 5V、轻载，手头的芯片又只是少量样本，所以示波器上的实测值通常落在典型值附近——测量常见典型值，是因为实验室条件恰好全部偏有利。100ns 回答“设计必须容忍什么”：任何一片合格芯片都允许慢到最大值，换一批料、盛夏高温、电源跌到下限、负载加重，都可能把延迟推向 100ns。若采样边界按典型值设计，比如 60ns 就读取，台架上一切正常，量产后最不利的那批板子却会读进旧值。这就是为什么建立时间一类的检查必须用最大值核算，而典型值只能用来估量级。

还有两条细则。第一，数据手册把上升翻转（ t_{PLH} ）和下降翻转（ t_{PHL} ）分开给出，两者一般不相等，链式相加时每一级都应取两者中较大的那个。第二，50pF 是手册规定的测试条件；若实际负载更重，每级还要按扇出估算中的 RC 法则追加延迟——门的固有延迟和负载造成的延迟从来不是两个无关的量。

现在可以更系统地回答“现代芯片为什么时延低”。第一层原因来自器件和工艺。更小的晶体管通常意味着更小的栅电容、更短的沟道、更短的局部连接和更低的单个门负载；同样的逻辑函数，在更小、更近的器件中传播，很多局部路径的绝对时间会下降。但这不是无条件成立。长连线、全芯片时钟、供电网络和散热会逐渐成

为新的限制，所以现代芯片必须同时优化**晶体管**和**连线**。

第二层原因来自片上集成。8086 时代，处理器与内存、外设和许多支撑逻辑之间存在大量片外通信；片外总线距离长、电容大、引脚数量有限，等待代价高。8086 已经用总线接口单元和执行单元的分工，以及指令预取队列，尝试把取指和执行重叠起来。到 80386，更多地址转换、保护和控制逻辑进入处理器内部，32-bit 数据通路和更丰富的片上控制减少了对外部协作的依赖。再往后，cache、浮点单元、内存控制器、图形或 I/O 控制器逐步进入同一颗芯片或同一封装，许多原本必须跨板级总线的路径被缩短为片上路径。

第三层原因来自存储层次。主存容量大，但距离核心远、访问路径长；寄存器和 L1 cache 容量小，却离执行单元近。现代芯片通过多级 cache 把最常用的数据和指令放在更近的位置。一次 L1 命中不是因为 DRAM 变得同样快，而是因为本次访问根本没有走到 DRAM。时延降低来自“常见情况走短路径”，不是“所有路径都同样短”。

第四层原因来自流水线和并行。把一条很长的组合路径拆成多个较短阶段，每个阶段之间放入寄存器，就能提高时钟频率。这样做降低了每个时钟周期需要容纳的组合延迟，但并不等于每条指令的总周期数一定减少。现代处理器还会用分支预测、乱序执行、多个执行单元和内存预取，把将来可能需要的工作提前准备，把等待主存或分支结果的空洞尽量填满。它们降低的是常见执行路径上的可见等待。

因此，现代芯片的低时延是一组工程选择的结果。器件缩小降低局部门延迟；门级库、缓冲和布局缩短关键路径；寄存器边界把长组合逻辑切成阶段；cache 把常用数据放近；预测和乱序把等待隐藏；片上集成减少跨芯片通信。若某次访问落在 **DRAM**、分支预测失败、cache 未命中或跨核心通信，时延仍会显著上升。严肃的硬件分析必须同时说明**最快常见路径**和**最坏慢路径**。这些机制在本书后续章节（ISA、流水线、cache 与虚拟内存，见扩写目录）和第六章经典芯片里会逐个展开成可检验的结构；第一章只需要先建立判断框架：低时延来自让常见情况少走远路。

现代 CPU 的一个典型低时延路径是：操作数已经在寄存器或 L1 cache 中，分支预测正确，所需执行单元空闲，结果能在相邻流水线阶段之间传递。此时，芯片用局部电路、短路径和寄存器边界把等待压到很低。相反，若数据不在 cache 中、分支预测错误或需要跨核心同步，同一条指令流会进入慢路径。现代芯片的工程目标不是让所有路径同样快，而是让高频路径短、让慢路径少发生，并在慢路径发生时尽量隐藏等待。

还要区分**时延**和**吞吐量**。流水线可以让每个周期都有不同指令处在不同阶段，从而提高吞吐量；但一条具体指令从进入流水线到完成，仍然要经过多个阶段。多执行单元可以同时处理多条互不依赖的操作，但无法让一条有严格依赖链的操作跳过必要计算。cache 可以显著降低常见访问时延，但不能让未命中的访问变成命中。理解现代芯片时，必须同时问两个问题：单次操作最快多久返回，连续大量操作每单位时间能完成多少。

还要区分反相器和缓冲器。反相器改变逻辑含义，缓冲器通常保持逻辑含义但增强驱动能力或隔离负载。很多时候，插入缓冲器不是为了“多一个门”，而是为了让一个信号能推得动更长的线或更多的后级。软件读者容易觉得保持值不变的缓冲器没有意义，但在硬件里，驱动能力本身就是功能的一部分。

最后还要检查扇入。扇出关心一个输出驱动多少后级，扇入关心一个门接收多少输入。四输入、八输入甚至更多输入的门在逻辑表达式里写起来很自然，但在硬件实现中可能被拆成多级结构；输入越多，门内部路径、电容和延迟也可能越大。把 `allow_start` 写成一个四输入 AND 并不意味着实际芯片里一定存在一个理想四输入 AND。实现可能把它拆成树形结构，也可能先合成若干中间信号。拆分方式会改变最长路径和毛刺形态。

flat idea:

```

allow_start = A AND B AND C AND D

tree implementation:
  left  = A AND B
  right = C AND D
  allow_start = left AND right

```

树形实现通常比线性串接更平衡，但会引入中间节点和不同路径长度。若某一路输入很晚才稳定，把它放在最后一级汇合可能有利于减少无效切换；若 A、B 来自同一连接器，先合成 `left` 可能便于局部布线。这里没有脱离功能的“最佳写法”，只有在功能、负载、延迟、布局和安全策略之间作出的实现选择。

对组合逻辑的最终检查，不应只停留在“真值表正确”。功能上，稳定输入下输出要符合真值表，否则是逻辑行为错误；默认状态上，悬空、断线、复位前的输入解释要有定义，否则异常时会误启动或误使能；驱动能力上，每个输出要能带动实际负载，否则电平被拖低、边沿过慢；路径延迟上，最长组合路径要能在后级使用前稳定，否则后级采到旧值或过渡值；毛刺风险上，多路径汇合不能产生危险窄脉冲，否则写使能、复位、片选会被误触发；低有效信号上，信号命名和真值表要一致，否则使能和禁止方向会接反；物理边界上，长线、外部输入和按钮抖动都要处理，否则干扰和抖动会进入逻辑核心。

对安全启动控制板，可以把这些检查整理成矩阵。矩阵的价值在于把“正常输入”“边界输入”“异常输入”和“时间条件”放在一起，防止只验证最容易通过的几行真值表。

检查类别	要施加的条件	期望结果
正常允许	请求有效、盖闭合、急停正常、无故障	<code>allow_start=1</code> ，执行命令等待时序边界
正常禁止	任一安全条件为 0	<code>allow_start=0</code>
故障禁止	温度或电流报警为 1	<code>fault_lamp=1</code> ， <code>allow_start=0</code>
断线默认	保护盖、急停或报警链路断开	进入禁止或报警一侧
上电过程	电源未稳或复位保持	<code>motor_enable_cmd=0</code>
按钮抖动	原始按钮快速断续	内部请求不产生多次启动事件
最坏负载	故障灯、后级逻辑和连接器同时连接	电平仍满足后级阈值
最坏时序	输入在不利延迟组合下变化	后级只在稳定后采样或使用

常见失败也应明确记录。许多硬件问题并不是因为设计者不知道 AND、OR、NOT，而是因为把抽象层次接错：悬空输入让系统偶尔误启动或误报警，根源是没有默认上拉/下拉或输入调理；低有效误读让复位、片选或急停方向相反，根源是信号约定未先于公式建立；电平不兼容让单独测试正常的电路接入后失效，根源是前后级阈值和驱动能力不匹配；毛刺误触发让最终值正确的电路出现短促动作，根源是危险输出直接使用了组合信号；扇出过大让边沿变慢、高电平不够高，根源是一个输出承担了过多负载；时序乐观让常温可用的电路在高温或低压下失效，根源是用典型值代替最坏值做核算；不关心误用让异常状态进入允许输出，根源是把可能发生的故障组合当作无关项。

若要把本章内容带入后续 CPU 数据通路，读者应保留两种同时成立的视角。第一，抽象视角：逻辑门实现布尔函数，组合网络实现加法、选择、译码和比较。第二，物理实现视角：每个输出都来自具体电流路径，每条路径都有延迟、负载和阈值，每个危险输出都需要明确的使用时刻。缺少前者，无法构造大系统；缺少后者，系统只是在纸上正确。

本章结束时，读者应能整理出一份小型硬件设计包。它不要求购买昂贵仪器，也不要求完成工业级认证；它要求把抽象、实现和验证记录放在同一份文档里：一张信

号表，列出原始输入、内部语义信号、组合输出和危险输出及其有效电平；一张电平表，列出前级 V_{OH}/V_{OL} 、后级 V_{IH}/V_{IL} 、噪声余量和负载条件，全部用最坏值；一张路径表，写出关键门在每行输入下的上拉、下拉、悬空和冲突状态；一张实现表，写明使用的门芯片、缓冲器、上拉/下拉、保护和输出驱动；一张时序表，写明最长路径、最小路径、毛刺风险、后级使用时刻和危险输出策略；一张测试表，写明施加条件、期望结果、测量工具、测点和波形记录；外加几张案例卡，把 7400/4000/8086/80386/现代 CPU 与本章概念真正关联起来。若后续章节要把这个组合逻辑接入寄存器、CPU 或总线，这份设计包就是接口约定。

对安全启动控制板的检查也应按执行顺序展开，而不是只看最终结果。第一步在断电和上电爬升阶段确认 `motor_enable_cmd` 被保持为 0；第二步测量每个原始输入在松开、按下、断线和报警状态下的默认电平；第三步用示波器确认按钮抖动、传感器边沿和故障输入不会在阈值附近长期停留；第四步用逻辑分析仪或等效测试记录内部语义信号是否只产生规定事件；第五步施加正常允许、单故障、双故障和断线条件，确认组合输出满足真值表；第六步在最坏负载和最坏时序假设下确认后级只在稳定后使用结果。

实验记录应当能让第三方复查。只写“测试通过”不够；至少要记录条件、预期结果、测量数据和结论。下面的模板适合本章的小组合模块，也适合后续寄存器和总线实验继续扩展。

测试项	施加条件	期望结果	实测电压或波形	结论
上电默认	电源爬升、复位保持	<code>motor_enable_cmd=0</code>	记录关键节点电压与复位释放时间	合格或失败
按钮输入	松开、按下、抖动、断线	内部请求只产生规定事件	记录阈值穿越和消抖后信号	合格或失败
故障禁止	温度或电流报警有效	<code>fault_lamp=1</code> , <code>allow_start=0</code>	记录报警输入与组合输出延迟	合格或失败
最坏负载	连接全部后级和指示灯驱动	电平余量和边沿仍满足规格	记录高低电平、上升/下降时间	合格或失败
毛刺检查	输入按不利时序翻转	危险输出不响应窄脉冲	记录示波器或逻辑分析仪波形	合格或失败

若使用逻辑分析仪，还应记录采样率、触发条件和被测信号名称；若使用示波器，还应记录探头倍率、带宽限制、地线接法和触发边沿。测量工具本身也会影响电路：长地线可能引入额外振铃，高电容探头可能拖慢边沿。因此，实验记录不仅证明电路行为，也证明测量方法没有把问题掩盖或放大。

实际测量时，探头应尽量接在接收端附近，而不只接在驱动端。驱动端波形漂亮，不代表穿过连接器、长线和负载后的接收端仍有足够余量。示波器观察快速边沿时应优先使用短地弹簧或低感抗接地方式，避免探头地线把额外振铃带入测量结果；逻辑分析仪只给出阈值后的数字结果，不能证明边沿是否干净；万用表只能证明低速或静态电压，不能证明毛刺和建立时间。正式记录应注明测点、探头、阈值、触发条件和环境条件——只证明常温空载，不等于证明最坏条件。

最后预告一个第二章会正式处理的问题：异步输入进入时钟边界时可能出现亚稳态。亚稳态指触发器在采样边沿附近遇到变化中的输入，内部节点可能暂时停在不能立刻判定为 0 或 1 的状态。第一章只要求记住边界原则：外部按钮、传感器、中断线和跨时钟域信号，不应未经同步就直接控制内部状态或危险输出。组合逻辑可以整理当前事实，但“何时保存”必须由后续时序电路给出约定。

到这里，本章完成了从电平到组合逻辑的闭环：节点要有明确驱动路径；电平要有噪声余量；二极管提供方向性但不能恢复电平；受控开关让信号控制电流路径；路径表和真值表共同定义门；门网络构成加法器、选择器、译码器等组合电路；组合电路虽然没有记忆，却有延迟、负载和毛刺。下一章开始讨论另一件事：怎样让

硬件保存状态，并按时钟边界一步一步推进。

专题：把电平特性从“能亮”推进到“有裕量” 前面已经把电平、开关和门电路连成一条主线。真正做硬件时，还必须把“能看到 0/1”推进到“在误差、噪声、负载和温度变化下仍然可靠”。一个输入脚不只是在某个理想电压上突然改变含义，而是由输入低阈值、输入高阈值、输出低电平、输出高电平共同定义可用范围。输出端能给出的高电平必须高于下一级认可的高阈值，输出端能拉到的低电平必须低于下一级认可的低阈值，中间差额就是噪声裕量。

参数	含义	设计时要回答的问题
VOH	输出为 1 时保证达到的最低高电平	在最大负载下是否仍高于下一级 VIH
VOL	输出为 0 时保证不超过的最高低电平	在灌电流时是否仍低于下一级 VIL
VIH	输入被识别为 1 的最低电压	外部信号、上拉和噪声是否能跨过阈值
VIL	输入被识别为 0 的最高电压	慢边沿和串扰是否会停在不确定区
噪声裕量	输出保证值到输入阈值之间的余量	板级噪声、地弹和电源纹波是否小于余量

这张表比“LED 亮不亮”更接近工程真实。LED 亮只能说明某个输出能驱动肉眼可见负载，不能证明它满足数字输入阈值；示波器上看到高电平接近电源，也不等于在最大扇出、最坏温度和最长连线下仍可靠。教材应让读者从第一章就建立习惯：每个数字信号都要同时看电平范围、驱动能力、边沿速度和负载。

```
noise margin check:
high_margin = VOH_min - VIH_min
low_margin  = VIL_max - VOL_max
pass only if expected_noise < both margins
```

CMOS 反相器的关键不是“一个管上、一个管下” CMOS 反相器常被画成上拉 PMOS 和下拉 NMOS。这个图很有用，但只说“输入 0 时上管开、下管关”还不够。实际电路要处理三个非理想问题：输出节点有电容—栅电容、扩散电容、连线电容和下级输入电容累加，翻转需要充放电时间，这就是传播延迟的来源；输入在阈值附近缓慢变化时，上下管可能同时部分导通形成短路电流，慢边沿因此既增加功耗也增加噪声；驱动多个输入或长线时等效电容更大，边沿更慢，延迟和裕量都被压缩。同一张清单上还有两条板级事实：多个输出同时翻转产生的瞬态电流会让地弹或电源跌落，输入阈值附近可能被误触发；高阻输入容易被漏电和耦合影响，必须用上拉、下拉或明确驱动源给出归宿。

因此，第一章的“开关”要升级为“受控电流路径”。当输出从 0 变 1 时，上拉网络不是瞬间把节点变成电源，而是向负载电容充电；当输出从 1 变 0 时，下拉网络把负载电容放电。电容越大、电阻越大，边沿越慢。延迟并不是后面时序章节才出现的概念，它从第一颗门的输出节点就已经存在。

扇出、上拉和开漏输出要按电流读 许多初学错误来自只看电压不看电流。一个输出脚能否驱动多个输入，要看它在保持合法 VOH/VOL 时能提供或吸收多少电流。TTL、CMOS、开漏、三态输出的驱动方式不同，不能只按“都是 5V 逻辑”混接。

输出形式	电流路径	常见用途和风险
推挽输出	上拉和下拉都由有源器件驱动	边沿快，但两个推挽输出不能直接并联
开漏/开集	只能主动拉低，高电平靠上拉电阻	适合线与、总线仲裁，但上升沿较慢

三态输出	可驱动高、低或断开	总线共享，但必须保证同一时刻唯一驱动者
弱上拉/下拉	用较大电阻给默认电平	适合防悬空，不适合驱动大负载

开漏输出尤其适合解释“电压是结果，电流路径才是原因”。当任意设备拉低总线时，电流从电源经上拉电阻流入该设备到地，总线电压变为低；当所有设备都断开时，上拉电阻慢慢给总线电容充电，电压回到高。上拉电阻太大，上升沿慢；太小，拉低时电流大。I2C、按键输入和许多中断线都能用这个模型理解。

```
open-drain line:
any driver pulls low  -> line = 0
all drivers released  -> pull-up charges line to 1
rise time depends on pull-up R and bus capacitance C
```

去耦电容是本地瞬态电源 门翻转的电流从哪里来？不是从稳压器“立刻”流过来——稳压器的响应以微秒计，而门的边沿以纳秒计。输出翻转时对负载电容充放电的电荷，加上内部上下管瞬间同时微导通的贯穿电流，都只能先从电源脚附近的储能元件取得。如果附近没有储能，电荷只能沿电源走线从远处赶来，而走线不是理想导线：每厘米约贡献 5–10nH 电感（含回流路径），电感上的电压满足 $v = L \cdot di/dt$ ，电流变化越快，走线上感应出的压降越大。

代入一组典型数字。8 路输出同时翻转，负载各 50pF，搬运的总电荷为 $Q = 8 \times 50\text{pF} \times 5\text{V} = 2\text{nC}$ ；在 5ns 内完成，意味着瞬态电流约 $2\text{nC}/5\text{ns} = 0.4\text{A}$ 。若这 0.4A 要穿过约 20nH 的走线电感，感应电压为 $20\text{nH} \times 0.4\text{A}/5\text{ns} = 1.6\text{V}$ ——电源脚瞬时跌落、地脚瞬时抬高（后者俗称地弹），整片芯片看到的“5V”在剧烈抖动，输入阈值跟着晃，邻近信号可能被误判。地弹不是器件坏了，而是瞬态电流与走线电感的必然乘积。

每颗芯片就近放一只 0.1μF 瓷片电容，就是给瞬态电流修一座本地水库：尖峰电流先由它供给，走线电感只需在事后把电荷慢慢补回。同样 2nC 改由 0.1μF 供给，电压跌落只有 $\Delta V = Q/C = 2\text{nC}/0.1\mu\text{F} = 20\text{mV}$ ，比 1.6V 低约两个数量级。“贴近电源脚”是电气要求而非工艺美观：电容与芯片之间每多一毫米走线，水库与芯片之间就多串一段电感，本地储能的效果就打一分折扣。所以布局规则是紧贴电源脚、短焊盘、短回流路径。

0.1μF 只管得住快而小的口袋，整板还需要分层：电源入口的 10–100μF 大电容应付更大、更慢的电流波动（继电器吸合、数码管扫描、背光开启），稳压器负责直流供给。按时间尺度分层供电，与后面存储层次按速度分层是同一个思想：快的层级小而近，慢的层级大而远，各自接住上一层来不及响应的那一段。

走线多短才算“短”：信号完整性初步 信号在走线上的传播速度是有限的：FR4 板材上约为 15cm/ns，大约是光速的一半。只要走线延迟远小于信号边沿时间，线上各点就来不及显现分布特性，可以把整条走线当作一只集中电容——这就是“短”线。常用的经验法则是：走线延迟小于边沿时间的六分之一（有的教材取四分之一），即可按集中参数处理。前面的扇出估算把走线只算作每厘米 1pF，依据正是这个近似。

74HC 的边沿约 5–8ns，代入法则：临界长度 $\approx 5\text{ns} \times 15\text{cm/ns} \div 6 \approx 12\text{cm}$ 。本章的面板板和实验连线都在这个量级之下，所以把走线当集中电容是站得住的——一本书低速实验大多安全地待在“短”线区。反过来，一旦边沿变快（现代器件可低于 1ns）或距离变长（跨板、背板、电缆），短线近似就会失效，同一根线必须改按传输线看待。

长线上的核心现象是反射。阻抗不连续的地方，行波会部分折返：无端接的长线末端近似开路，入射波几乎全反射回来，与入射波叠加后，接收端出现过冲、振铃甚至回沟，一个边沿可能被看成多次穿越阈值——接收端误以为信号翻转了好几回。端接的思路是为行波安排能量归宿：末端并联电阻到地或电源，或源端串联电阻，使反射系数接近零，波形一次到位。反射不是噪声玄学，它的幅度和极性都可以从阻抗差算出。

串扰则来自相邻走线之间的互容与互感。两条长平行线中，一条快速翻转会在另

一条上感应出窄毛刺；平行段越长、间距越近、边沿越快，耦合越强。处理办法是拉开间距、缩短平行段、在敏感信号之间夹一根地线。这些现象在 HC 实验里只会以“边沿有点圆、偶尔带振铃”的形式露面，但同一条物理规律决定了高速主板为什么讲究受控阻抗与端接电阻——先在这里建立直觉，后续章节再定量。

毛刺不是玄学，而是路径延迟不一致 组合逻辑里不同输入路径有不同延迟。当多个输入几乎同时变化时，中间节点可能短暂经过错误组合，输出形成窄脉冲，这就是毛刺。若毛刺只被后级组合逻辑看到，可能不影响最终稳定值；若毛刺被锁存器、计数器、异步复位或外部设备采样，就会变成真实错误。

毛刺最常见的来源有四种。重收敛路径：同一输入经不同延迟路径再汇合，处理办法是改写逻辑、增加共识项，或让后级只在时钟边界采样。译码器切换：地址多位同时变化，片选可能短暂重叠或空洞，处理办法是地址先锁存、片选加使能窗口。机械按键：触点弹跳导致多次高低变化，处理办法是 RC、施密特、同步去抖或状态机去抖。异步输入：外部事件不按本机时钟变化，处理办法是同步器和边沿检测分开设计。

这也解释了为什么本书反复强调“边界必须可检查”。组合逻辑允许在稳定前经历暂态，时序元件只应在规定窗口采样稳定结果。若把组合毛刺直接接到计数器时钟、异步清零或设备写使能，系统会表现得像随机错误。第一章的门延迟和毛刺，直接通向第二章的时钟边界和后面整机的片选安全。

第一章实验应记录波形而不只记录现象 一块面包板、一个反相器、一只按键和一台示波器，就能做出高质量的第一章实验。目标不是炫耀器件数量，而是把电平、阈值、边沿、负载和毛刺变成可以直接观察的量。

实验	操作	必须记录
输入阈值扫描	用电位器缓慢改变输入电压	输出翻转点、滞回是否存在
负载电容实验	在输出加不同电容负载	上升/下降时间和传播延迟变化
上拉电阻实验	改变开漏线的上拉电阻	上升沿、低电平电流和噪声裕量
按键抖动观察	直接观察按键输入波形	抖动持续时间和次数
译码毛刺观察	让多位地址同时变化	片选是否短暂重叠或误触发

这些实验报告应写出测量条件：电源电压、器件型号、负载、电阻电容值、示波器探头倍率、时基和触发方式。没有测量条件的波形很难复现，也很难判断结论是否能迁移到另一块板。

章末练习

本节练习要求使用文字、真值表、路径表和检查表完成。重点不是记忆门符号，而是能说明某个 bit、某条路径、某个组合输出为什么在真实硬件条件下可靠成立。

练习按三层完成。基础层要求掌握电平、路径和真值表，能从每根线的 0/1 语义追溯到物理路径；设计层要求把组合逻辑实现为门、芯片和实验，能把表达式变成可工作的实现；扩展层要求把第一章概念映射到经典芯片和现代低时延机制，能把概念放进真实产品和测量环境。读者不应只做会写公式的题，也要能留下可复查的实验记录。

任务	要完成的产物	检查要点
按钮输入	为 <code>start_button_raw</code> 写出上拉或下拉方案、有效电平、断线默认值和消抖策略	松开、按下、抖动、断线四种状态都有明确内部语义

电平约定	给出一组前级 V_{OH}/V_{OL} 与后级 V_{IH}/V_{IL} ，计算高低噪声余量	说明余量为正、为零、为负时分别意味着什么
路径表	任选 NAND 或 NOR，列出四行真值表，并为每一行写出上拉与下拉路径状态	不出现稳定输入下的悬空或强冲突
低有效信号	将 <code>reset_n</code> 、 <code>chip_select_n</code> 或急停链路转换为正逻辑语义信号	说明原始电平、内部语义和默认安全值之间的关系
故障汇总	为 <code>temp_alarm</code> 与 <code>current_alarm</code> 设计故障灯和故障码逻辑	单故障、双故障、无故障和非法输入都有定义
1-bit ALU	写出 AND、OR、XOR、ADD、SUB 的候选结果、 <code>op</code> 、 <code>B_invert</code> 和最低位 <code>Cin</code> 规则	区分逐位逻辑路径、加减法进位路径和标志解释
NAND-only 实现	把 <code>allow_start</code> 的安全表达式改写成二输入 NAND 网络，并命名所有反相中间信号	稳定真值表等价，同时说明门级层数、负载和毛刺风险
芯片引脚映射	任选 74HC00 或 7400 类芯片，把一个逻辑式映射到电源脚、输入脚、输出脚和未用门处理	说明去耦、未用输入默认值、扇出和封装方向
输入保护	为一个板外按钮或传感器写出限流、滤波、ESD/TVS、施密特输入或缓冲方案	说明保护不改变语义，但限制异常电压、电流和噪声
五层边界图	任选 <code>start_button</code> 、 <code>cover_closed</code> 或 <code>temp_alarm</code> ，画出外部物理量、保护边界、阈值整形、语义归一和组合逻辑五层	每层都要写出输入、输出、风险和验证方法，不能把原始引脚直接当作内部 bit
引脚等效模型	为一个输入和一个输出分别列出漏电、电容、阈值、驱动电流、传播延迟或绝对最大额定值	能解释为什么逻辑变量模型不足以判断扇出、长线、LED 驱动和跨电源域连接
毛刺分析	分析 $Y=(A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$ 在 $B=1$ 、 $C=1$ 、 A 翻转时的风险	能说明共识项为什么不改变稳定真值表，却改变过渡行为
安全闭环	按“原始输入、输入调理、语义信号、组合判断、时序边界、输出驱动、验证记录”重写安全启动控制板规格	每一层都能追溯到上一层，危险输出不能直接依赖未经检查的组合结果
芯片案例	分别用 7400、4000 CMOS、8086、80386 和现代 CPU 解释本章一个概念；6502、Z80、8051、80486、Pentium 作为选做，等学完第六章经典芯片后回看	每个案例都要指出对应的电平、路径、组合逻辑、片上集成或低时延机制
测量计划	为按钮输入或故障输出写出万用表、示波器、逻辑分析仪各自要证明的内容	不把静态电压测量误认为完整的时序和毛刺检查
时序词汇	用一条组合路径解释建立时间、保持时间、时钟到输出和时钟偏斜	能说明最大延迟、最小延迟和采样边界的关系
实验记录	按测试项、条件、期望结果、实测波形和结论整理一次安全启动控制板实验	记录足以让第三方复查，不只写“通过”

板级边界	说明参考地、回流路径、去耦、串扰或长线振铃如何影响某个输入	能把布尔错误和物理边界错误区分开
快慢路径	为现代 CPU 写出一个快路径和一个慢路径案例	能说明低时延来自常见路径短，而不是所有路径同样短
上拉阻值	为一条 5V 开漏总线（总线电容 100pF，驱动器灌电流上限 4mA）选择上拉电阻，要求从低电平上升到 $V_{IH} = 3.5V$ 的时间不超过 $1\mu s$	同时算出阻值上界和下界，并验证所选标称值两头都满足
三态总线互斥	为共享一条数据总线的两个三态驱动器写出使能逻辑，并分析选择信号切换瞬间的风险	任何时刻至多一个驱动器使能，切换要先断后通
选择器实现函数	用 8 选 1 选择器实现 $F(A, B, C) = \sum m(1, 3, 5, 6)$ ，再改用 4 选 1 选择器、以 A 作数据输入实现同一函数	每个数据输入的常量或函数都要与真值表逐行一致
延迟链核算	一条 4 级 74HC 路径，每级典型 9ns、最大 22ns，要求信号在采样前 90ns 稳定	分别用典型值和最大值核算，说明哪一个才能作为设计依据
去耦电容选择	为一块 8 路输出可能同时翻转的 74HC 小板选择去耦方案，估算本地电容上的电压跌落	说明就近原则与板级大电容的分工，跌落估算要有数值
电平转换	3.3V CMOS 输出（最差 $V_{OH} = 3.0V$ ）驱动 5V 74HC 输入（ $V_{IH} = 3.15V$ ），给出可行方案	先核算直接连接的余量，再给出器件级方案并重算余量

完成这些练习后，读者应能把一个组合电路从自然语言需求推进到可检查的规格：先定义信号含义，再列真值表和路径表，然后检查电平、负载、延迟、毛刺、默认状态和测量记录。这个顺序将直接用于后续寄存器、数据通路、控制器和整机接口。

参考解答与检查要点

本节给出参考答案。实际工程方案允许存在不同器件和参数选择，但结论必须有电平、路径、时序和测量结果支撑。若答案只给出布尔式而没有默认值、负载、毛刺或测量边界，不能视为完整解答。

任务	参考解答	必须保留的检查点
按钮输入	可采用上拉输入：松开时 <code>start_button_raw=1</code> ，按下接地为 0，再经反相、消抖和同步得到 <code>start_request=1</code> ；也可采用下拉输入，关键是松开、按下、断线都要有定义。	不允许松开或断线时悬空；按钮抖动不能形成多次启动事件。
电平约定	例如 $V_{OH} = 3.0V$ 、 $V_{IH} = 2.1V$ ，高余量 0.9V； $V_{OL} = 0.25V$ 、 $V_{IL} = 0.7V$ ，低余量 0.45V。余量为正表示最坏条件仍可识别，余量为零表示没有抗扰空间，余量为负表示电平约定失败。	使用最坏值，不能用典型值替代核算。

路径表	以 NAND 为例, A=B=1 时下拉串联路径完整、输出为 0; 其余三行至少一个下拉开关断开, 上拉网络提供高电平。NOR 则对应并联下拉和串联上拉。	每一行都要说明上拉、下拉、悬空和冲突。
低有效信号	<code>reset_n=0</code> 表示复位有效, 可转换为 <code>reset_active = NOT reset_n</code> ; <code>chip_select_n=0</code> 表示选中, 可转换为 <code>chip_selected = NOT chip_select_n</code> 。	名称必须反映极性, 默认安全状态要明确。
故障汇总	<code>fault_lamp = temp_alarm OR current_alarm</code> ; <code>no_fault = NOT fault_lamp</code> ; 双故障可输出专用多故障码 11。	任一故障都禁止启动, 非法或未知组合不应进入允许状态。
1-bit ALU	<code>AND=A AND B</code> , <code>OR=A OR B</code> , <code>XOR=A XOR B</code> ; ADD 使用 <code>B_invert=0, Cin=0</code> ; SUB 使用 <code>B_invert=1, Cin=1</code> , 即 <code>A+(NOT B)+1</code> 。	<code>op</code> 、 <code>B_invert</code> 、最低位 <code>Cin</code> 必须一致, 标志位要说明无符号或补码解释。
NAND-only 实现	可写为 <code>n1=cover_closed NAND emergency_ok</code> 、 <code>safe_chain=n1 NAND n1</code> 、 <code>n2=start_request NAND no_fault</code> 、 <code>request_ok=n2 NAND n2</code> 、 <code>n3=safe_chain NAND request_ok</code> 、 <code>allow_start=n3 NAND n3</code> 。	反相中间信号必须命名清楚, NAND-only 等价不代表延迟和毛刺也相同。
芯片引脚映射	以 74HC00 为例, 一路 NAND 接 <code>cover_closed/emergency_ok</code> 得到 <code>safe_chain_n</code> , 第二路 NAND 两输入并接 <code>safe_chain_n</code> 得到 <code>safe_chain</code> ; <code>VCC/GND</code> 接正确电源并就近去耦, 未用 CMOS 输入接确定电平。	检查封装方向、引脚编号、未用门、扇出、输出驱动和电源去耦。
输入保护	板外按钮可采用串联电阻、上拉/下拉、RC 滤波、TVS/ESD 保护和施密特输入; 高速输入不能使用过大 RC。	保护电路限制异常电压和电流, 但不能破坏有效边沿和响应时间。
五层边界图	以 <code>start_button</code> 为例, <code>raw</code> 表示触点和线缆; 保护层包含限流、ESD 和默认上拉/下拉; 整形层包含 RC 或施密特输入; 语义层把低有效、消抖和反相整理成 <code>start_request</code> ; 组合层只使用 <code>start_request</code> 。	每层输出都要比上一层更接近可靠 bit, 不能跳过保护和阈值整形环节。
引脚等效模型	输入端可写成阈值比较器加输入电容、漏电和保护结构; 输出端可写成受控上拉/下拉开关加等效电阻、电流限制和延迟。这个模型能解释上拉太弱、负载过重、长线振铃、输出并接和跨电源域损坏。	数据手册最坏值优先; 不要用“仿真里是 1”替代引脚电气检查。

毛刺分析	对 $Y=(A \text{ AND } B) \text{ OR } ((\text{NOT } A) \text{ AND } C)$, 当 $B=C=1$ 且 A 翻转时, 两条路径延迟不同, Y 可能短暂掉到 0; 加入共识项 $B \text{ AND } C$ 可保持静态 1。	稳定真值表不变, 但过渡行为改变; 危险输出要寄存或专门处理。
安全闭环	规格应写成: 原始输入先经保护、默认值和消抖, 得到语义信号; 组合逻辑形成 <code>fault_lamp/no_fault/allow_start</code> ; 危险输出经时序边界和驱动级形成 <code>motor_enable_cmd</code> 。	原始触点不能直接驱动执行机构; 复位和断线默认必须安全。
芯片案例	7400 展示门芯片电气约定; 4000 CMOS 展示高输入阻抗和低静态功耗; 6502/Z80 展示外部总线微处理器; 8051 展示片上 I/O; 8086 展示预取和复用总线; 80386 展示 32-bit 和保护; 80486/Pentium 展示 cache、流水线和预测。	每个案例都要绑定到本章概念, 不能只列名称。
测量计划	万用表确认静态高低电平; 示波器观察抖动、边沿、振铃和毛刺; 逻辑分析仪记录内部语义事件和时序关系。	三类工具不能互相替代, 测点应靠近接收端。
时序词汇	建立时间是采样沿前必须稳定的时间; 保持时间是采样沿后必须继续稳定的时间; 时钟到输出是触发器输出延迟; 时钟偏斜是不同触发器看到时钟沿的差异。	最大延迟影响建立, 最小延迟影响保持。
实验记录	记录测试项、条件、期望结果、实测电压或波形、逻辑分析仪结果和结论; 注明探头倍率、触发条件、采样率、电源电压和负载。	只写“测试通过”不算结论, 记录必须能让第三方复查。
板级边界	地弹会压缩噪声余量; 回流路径不连续会增加环路面积; 去耦不足会造成供电跌落; 串扰和振铃会让接收端误判。	区分布尔函数错误和物理边界错误。
快慢路径	快路径: 操作数在寄存器或 L1, 分支预测正确, 旁路可用; 慢路径: cache miss、TLB miss、分支错误、跨核心同步或 DRAM 访问。	低时延来自常见路径短, 不代表所有路径都短。
上拉阻值	上界由上升时间决定: $t = R \cdot C \cdot \ln \frac{V_{CC}}{V_{CC}-V_{IH}}$, 代入得 $t \approx R \times 100\text{pF} \times 1.20$, 由 $t \leq 1\mu\text{s}$ 得 $R \leq 8.3\text{k}\Omega$; 下界由低电平电流决定: 拉低时电阻电流 $(5-0.4)\text{V}/R \leq 4\text{mA}$, 得 $R \geq 1.15\text{k}\Omega$ 。取标称 $4.7\text{k}\Omega$: 上升时间约 $4.7\text{k}\Omega \times 100\text{pF} \times 1.20 \approx 0.57\mu\text{s}$, 低电平电流约 0.98mA , 两关都过。	只算上升时间会漏掉拉低电流这一关; $\ln$ 因子来自电容充电曲线, 不是凑出来的系数。

三态总线互斥	使能由同一选择信号经独热译码产生，并在时序边界上先全部断开、下一拍再接通新驱动者（先断后通）。若用 $EN_B = NOT\ EN_A$ 组合派生，反相器延迟窗口内会出现两者同通或同断；同通时两个推挽输出并联，可能一个拉高一个拉低，形成冲突电流。	使能信号属于危险输出，要按毛刺与时序边界管理；分析切换瞬间必须写出电流路径。
选择器实现函数	8 选 1：选择端接 A、B、C，数据端 $D1=D3=D5=D6=1$ ，其余接 0。4 选 1：选择端接 B、C，逐行对照得 $D0=0$ ($m0$ 、 $m4$ 均为 0)、 $D1=1$ ($m1$ 、 $m5$ 均为 1)、 $D2=A$ ($m2=0$ 、 $m6=1$)、 $D3=NOT\ A$ ($m3=1$ 、 $m7=0$)。	每个数据输入都要能追溯到真值表的两行；选择端变量顺序一旦接反，整张映射全错。
延迟链核算	典型合计 $4 \times 9ns = 36ns$ ；最大合计 $4 \times 22ns = 88ns$ ，仍满足 $90ns$ 的要求，但余量只有 $2ns$ 。若误用典型值，会误以为余量 $54ns$ 。设计依据只能是最大值： $88ns \leq 90ns$ 。	建立类检查用最大值；典型值只用于估量级，不能写进设计边界。
去耦电容选择	8 路同时翻转搬运电荷 $Q = 8 \times 50pF \times 5V = 2nC$ ；由就近的 $0.1\mu F$ 供给，跌落 $\Delta V = Q/C = 20mV$ 。若无本地电容，约 $0.4A$ ($2nC/5ns$) 的瞬态电流穿过约 $20nH$ 走线电感，将引起约 $L \cdot di/dt = 1.6V$ 的扰动。方案：每片 $0.1\mu F$ 瓷片紧贴电源脚，电源入口再放 $10\sim 47\mu F$ 。	就近是电气要求而非工艺美观；只写“加 $0.1\mu F$ ”而没有位置与数值估算不算完整。
电平转换	直接连接的高电平余量为 $3.0 - 3.15 = -0.15V$ ，必然失败。方案之一：用一片 74HCT (5V 供电、TTL 输入阈值 $V_{IH} = 2.0V$) 作第一级整形，新余量 $3.0 - 2.0 = 1.0V$ ，输出已是标准 5V CMOS 电平；也可用开漏输出加上拉到 5V，或专用电平转换芯片。反向 (5V 驱动 3.3V) 可用串联电阻加钳位二极管，或选用耐 5V 输入的器件。	先证明原方案余量为负，再谈转换；不能只写“中间加一级”而不重算新余量。

本章的掌握标准不是“会背 AND、OR、NOT、XOR 的真值表”，而是能把每个抽象还原为具体的硬件行为。

- 电平：前一级的最差高/低输出，必须落在后一级输入阈值允许的范围，并留下噪声余量。
- 路径：任何输出节点都要能写成“高电平路径是否导通、低电平路径是否导通、是否存在冲突电流、是否存在悬空”四项。
- 门级：任选一个二输入门，列出 4 行真值表，再为每一行写出上拉网络和下拉网络状态。
- 组合：半加器的 Sum 是 XOR，Carry 是 AND；全加器是在同一张真值表中

纳入进位输入后的组合电路。

- 延迟：同一个逻辑函数可以有不同门级网络，最长路径不同，稳定时间也不同。
- 负载：一个门的输出不是无限共享变量，后级数量、线长和缓冲策略会改变电气行为。
- 边界：外部输入、上电复位、开漏共享线、三态总线和电源域转换都必须先满足边界约定，才能进入普通组合逻辑。
- 安全：内部允许判断不等同于执行机构命令，危险输出必须说明默认状态、使用时刻和毛刺处理策略。
- 芯片案例：7400/4000 系列展示标准门芯片的电气约定；8086/80386 展示组合控制 and 数据通路如何扩展成处理器；现代 CPU 展示 cache、流水线、预测和片上集成如何降低常见路径的可见等待。
- 检查题：若组合网络输出短暂从 0 跳到 1 又回到 0，而最终值正确，它是否一定构成设计错误？答案取决于后级是否在毛刺期间采样。

经典系统教材常把抽象的位函数和它的物理实现放在同一页上看。只看位函数会漏掉电平、驱动和时序；只看器件又会失去逻辑行为。两者合在一起，才是硬件设计对象。

状态、时钟与同步边界

第一章讲的是组合逻辑：输入稳定后，输出由当前输入唯一决定。它能计算，却不能保存过去。真正的机器必须能把某个结果留下来，等下一个步骤继续使用；也必须能规定“什么时候更新状态”，否则反馈路径会让硬件在一个周期内连续变化，机器无法分步骤推进。

本章将反馈、锁存器、触发器、时钟、复位、寄存器、计数器和状态机合并讨论。读完后，应当能把“状态”理解成真实硬件边界：一些 bit 在时钟边界被提交，边界之间由组合逻辑准备下一值。后续 CPU 的 PC、指令寄存器、控制状态、流水线寄存器和中断入口，都将反复使用这一章的模型。

核心思想

组合逻辑负责准备下一值，状态元件负责在边界保存下一值。同步硬件就是把这两件事反复组织起来。

第一章已经把建立时间、保持时间、时钟到输出和时钟偏斜作为词汇预告。本章从这里正式接上：组合逻辑的输出并不是“最终正确即可”，它必须在目标触发器的采样窗口之前稳定；状态元件也不是理想变量，它采样后需要时间把新值送到 Q 端。同步设计的基本任务，就是把这些时间约定整理成一组可检查的时序关系：时钟到输出说明触发器被时钟触发后 Q 端多久开始并完成变化，它决定下一段组合逻辑最早什么时候开始传播；组合最大延迟说明 Q 端变化经过门和连线到达 D 端的最晚时间，它决定下一个时钟沿到来时结果是否已稳定；建立时间要求 D 端在采样沿之前提前稳定，否则目标触发器会采到过过渡值；保持时间要求 D 端在采样沿之后继续稳定，否则新值会过早冲进采样窗口；时钟偏斜说明源触发器和目标触发器看到时钟沿的时间差，它会压缩有效周期、甚至破坏保持窗口。

本章的版面组织遵循同一顺序：先说明反馈为什么能保存一个 bit，再说明锁存器和触发器如何定义写入边界，随后把多个触发器扩展成寄存器、计数器、移位寄存器和状态机，最后把复位、跨边界输入、实验板接线和数据手册参数整合成完整的工程方法。读者不应把这些小节看成独立名词表，而应把它们看成“状态如何被创建、提交、传播和验证”的连续链条。

一、反馈、锁存器与保存一个 bit

保存一个 bit 的起点是反馈。两个反相器首尾相接，会形成两个稳定状态。若左边节点为 0，右边经过反相后为 1；右边的 1 再反相回来，正好维持左边为 0。反过来，左边为 1、右边为 0 也能稳定。只要没有外部强制改变，这个状态会保持。

该结构说明，“保存一个 bit”对应电路中某些节点在反馈作用下停留于稳定物理状态。一个稳定状态解释为 0，另一个稳定状态解释为 1。状态的本质，是电路有能力在输入不再变化时仍然维持某个内部结果。

从电路角度看，反馈不是抽象箭头，而是输出重新影响输入。两个反相器互相连接时，每一侧都在“支持”另一侧的当前电平。若某个扰动让左侧电压略有上升，右侧反相器会把右侧电压压低，右侧的低电平又通过另一个反相器继续支持左侧高电平。这个正反馈过程让电路迅速落入两个稳定点之一。数字电路把这两个稳定点命名为 0 和 1，但底层仍然是晶体管、电容、电阻和电源共同决定的物理平衡。



两个反相器首尾相接后，任意一个稳定输出都会成为另一侧的输入；反馈线绕到下方，避免穿过门符号和标签。

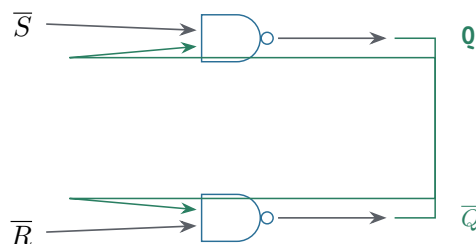
图示：两个反相器首尾相接形成双稳态： $X=0, Y=1$ 稳定； $X=1, Y=0$ 也稳定。

真实锁存结构通常不会直接暴露为两个反相器，而是用交叉耦合的 NAND 门或 NOR 门构成 SR 锁存器。SR 的含义是 set 与 reset: set 请求把状态置为 1, reset 请求把状态置为 0。它已经能表达“外部命令改变内部状态”，但也引入一个重要约束：不能同时给出相互冲突的命令。若 set 和 reset 同时有效，输出可能进入不符合设计语义的状态；当冲突解除时，最终稳定在 0 还是 1 取决于门延迟和释放顺序。这就是第一类状态设计约定：不仅要说明哪个状态合法，还要说明哪些输入组合禁止出现。

把 SR 锁存器的四种输入组合逐行推一遍。两个输入 $\bar{S}$ 、 $\bar{R}$ 都是低有效（为 0 才表示请求），以 NAND 实现为例：

$\bar{S}$	$\bar{R}$	Q	逐行推导
0	0	禁用	两个 NAND 的输入都有 0, Q 和 $\bar{Q}$ 都被迫为 1, 互补关系被破坏；这是非法输入，不是“存了 1”
0	1	1	上侧 NAND 因 $\bar{S}=0$ 输出 1；下侧 NAND 两输入全 1 输出 0, 反馈回去维持 $Q=1$ 。set 生效
1	0	0	与上一行对称： $\bar{R}=0$ 把 $\bar{Q}$ 拉成 1, 反馈把 Q 拉成 0。reset 生效
1	1	保持	无外部请求：若 $Q=1$, 下侧输出 0 支持上侧输出 1；若 $Q=0$ 则反过来。反馈环自己维持旧值

禁行还要单独说一句“释放之后发生什么”。若 $\bar{S}$ 、 $\bar{R}$ 从 0,0 同时回到 1,1, 两个 NAND 的旧输出都是 1, 于是两个门都打算把输出改成 0——谁先翻转，反馈就支持谁，最终状态取决于两个门的实际延迟差。这就是为什么正文说“稳定在 0 还是 1 取决于门延迟和释放顺序”：它不是玄学，是禁行退出瞬间的一场不可预测的竞争。可靠设计要么不产生 0,0, 要么让两个输入不同时释放。



交叉反馈只连接到另一只 NAND 的第二输入，输入线、输出线和反馈线各走独立通道。

图示：由两个交叉耦合的 NAND 门构成的 SR 锁存器。 $\bar{S}$ 与 $\bar{R}$ 低有效；当两者都为高时，反馈维持原状态。

专题：NOR 版 SR 锁存器——同一结构，另一套输入约定

上面用 NAND 搭出的 SR 锁存器不是唯一的搭法。把两个 NAND 换成两个 NOR，得到的仍是交叉耦合反馈环，存储机制完全一样；变化只发生在输入约定上。NAND 版的 $\bar{S}$ 、 $\bar{R}$ 是低有效：输入为 0 才表示请求，无请求时两者都停在 1。NOR 版的 S、R 是高有效：输入为 1 才表示请求，无请求时两者都停在 0。极性相反不是设计风格问题，而是门本身的函数决定的：NAND 只要有一个输入为 0 输出就是 1，所以

“请求”只能靠 0 来表达；NOR 只要有一个输入为 1 输出就是 0，所以“请求”只能靠 1 来表达。

把 NOR 版逐行推一遍。设上侧 NOR 输出 Q ，下侧 NOR 输出 $\bar{Q}$ ，S 接下侧、R 接上侧。S=1, R=0 时，下侧 NOR 因 S=1 输出 0，上侧 NOR 两个输入全为 0 输出 1， Q 被写成 1，置位生效；S=0, R=1 时对称， Q 被写成 0，复位生效；S=0, R=0 时，两个 NOR 各有一个输入来自对方输出：若 $Q=1$ ，下侧输出 0 正好支持上侧输出 1；若 $Q=0$ 则反过来，反馈环维持旧值；S=1, R=1 时，两个 NOR 都被迫输出 0， Q 与 $\bar{Q}$ 的互补关系被破坏，这就是 NOR 版的禁用组合。两版放在一起对照：

S (NOR)	R (NOR)	$\bar{S}$ (NAND)	$\bar{R}$ (NAND)	锁存器行为
0	0	1	1	无请求：反馈环保持旧值
1	0	0	1	置位： Q 被写成 1
0	1	1	0	复位： Q 被写成 0
1	1	0	0	禁用：set 与 reset 同时请求，互补输出被破坏

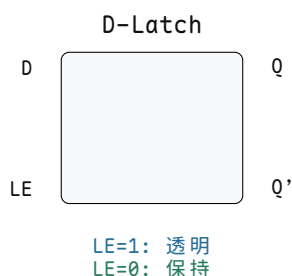
表面看，两版的禁用组合一个是 11、一个是 00，似乎完全不同；实际上它们表示同一件事——set 和 reset 被同时请求。禁用组合指向的从来不是某个二进制图案，而是“两条相互冲突的命令同时有效”这个语义。退出禁用组合时的竞争也一样：NOR 版从 11 同时回到 00，与 NAND 版从 00 同时回到 11 一样，两个门的旧输出都是 0，于是都打算把输出改成 1，谁先翻转反馈就支持谁，最终状态同样取决于门延迟差和释放顺序。

这个对照的意义在于把“结构”和“约定”分开。存储能力来自交叉耦合反馈，它在 NAND 版和 NOR 版里是同一个结构；低有效还是高有效、空闲电平停在 1 还是 0、禁用图案是 00 还是 11，只是套在结构外面的输入约定。数据手册里常见同一功能给出两种极性的型号，读表时应当先确认有效电平，再读功能行；顺序反了，就会把保持行误读成禁用行。

到这里，四种保存 1 bit 的结构可以排成一条线：双反相器反应用两个稳定点保存 1 bit，但没有独立写入控制；SR 锁存器允许用 set/reset 命令改变状态，代价是 set 与 reset 不能给出冲突请求；D 锁存器用一个数据输入和 enable 表达写入，代价是 enable 有效期间输入可以穿透到输出；D 触发器只在时钟边沿采样并保存，代价是必须满足建立时间、保持时间和复位要求。后面的讨论会逐个展开后三种。

单纯反馈还不足以形成可用存储单元。若无法可靠地把它从 0 改成 1，或从 1 改成 0，它只是一个稳定反馈环。因此需要写入控制。锁存器可以理解为带写使能的 1-bit 存储单元：写使能有效时，输入 D 可以影响输出 Q ；写使能无效时，反馈通路维持原来的 Q 。

写使能	数据输入	输出行为
有效	0 或 1	输出跟随输入变化
无效	任意	输出保持原值

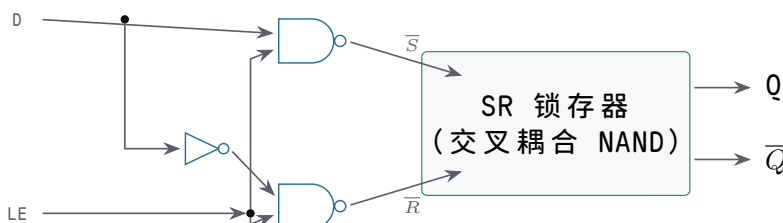


图示：D 锁存器方框符号：LE=1 时 Q 跟随 D（透明），LE=0 时反馈保持原值。

例如一个“按住才能设置”的电路中，enable 为 1 时，D=1 会把锁存器写成 1；enable 回到 0 后，即使 D 变回 0，Q 仍保持 1。这样，过去某一时刻的输入就被硬件保存下来了。

把 SR 锁存器变成 D 锁存器的意义，在于把两个可能冲突的命令合并为一个数据输入 D 和一个写使能 enable。enable 有效且 D=1 时，相当于请求 set；enable 有效且 D=0 时，相当于请求 reset；enable 无效时，set 和 reset 都不被触发，反馈保持旧值。这样，外部控制逻辑不再同时生成 set/reset 两条危险命令，而是生成“是否写入”和“写入什么值”两个更清晰的问题。

这个改造在门级只需要一个反相器和两个 NAND：

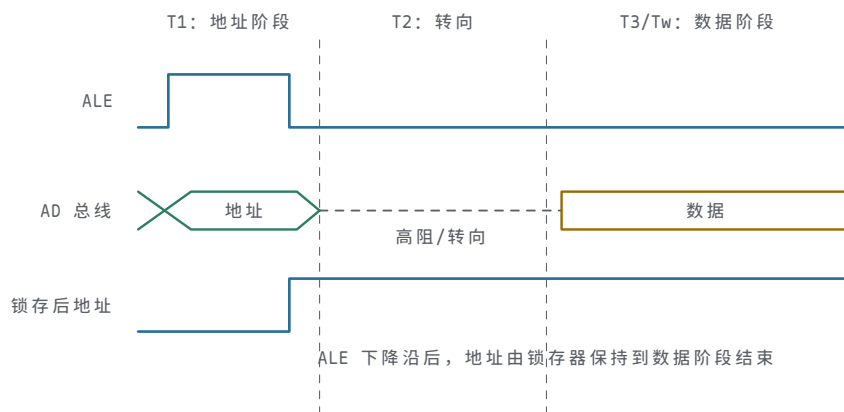


图示：D 锁存器的门级实现：LE=1 时，D 和它的反相信号分别推开两扇 NAND，一路 $\bar{S}$ 或 $\bar{R}$ 被拉低（写入）；LE=0 时两扇都关闭， $\bar{S}$ 、 $\bar{R}$ 同为高（保持）。0,0 禁行从结构上不可能再出现——这就是“把 set/reset 两条危险命令合并成 D 和 LE”在电路结构上的直接体现。

真实器件中的锁存器还会暴露输出使能、低有效清零、三态输出等控制脚。以 74HC373/74HC573 这类透明锁存器为例，锁存使能决定内部是否继续跟随输入，输出使能只决定引脚是否驱动外部总线；二者不能混淆。关闭输出并不等于内部状态消失，打开输出也不等于重新采样输入。后面讨论总线、寄存器堆和 I/O 芯片时，这个区分会反复出现：内部状态、外部驱动、写入边界是三件不同的事。

8086 是理解锁存器用途的经典案例。8086 的部分地址线与数据线复用，地址阶段先在总线上给出地址，随后同一组引脚又可能转为数据传输。外部存储器不能在数据阶段继续把这些引脚当作地址，因此系统板需要由 ALE 这类地址锁存允许信号驱动外部锁存器，把地址阶段的值保存下来。8086 时代常见的板级器件包括 8282/8283 或 74LS373；今天做低速教学实验时也常用 74HC373/74HC573 复现同一思想。这里的锁存器不是“多余缓冲器”，而是把一个总线相位中的地址变成后续相位仍然可用的状态。

总线阶段	锁存器动作	设计含义
地址有效	锁存窗口打开，地址进入锁存器	允许复用总线先表达地址
窗口关闭	内部反馈保持地址	外部存储器继续看到稳定地址
数据传输	总线引脚改作数据或状态用途	地址不会随复用引脚变化而丢失
输出使能控制	决定是否驱动外部总线或后级	不改变锁存器内部保存的值



图示：8086 复用总线的一个读周期：T1 时 ALE 打开锁存窗口，地址随 AD 总线进入锁存器；ALE 落下后窗口关闭，AD 总线转向传数据，而锁存器把地址保持到 T3 采样完成。锁存器把一个总线相位的值变成后续相位仍然可用的状态。

从这个案例可以看出，锁存器适合用在协议已经提供清晰电平窗口的地方，例如地址锁存、外部总线相位分离、显示输出暂存等。若协议没有给出可靠窗口，或者多个状态级之间存在组合反馈，透明锁存器反而会让调试变难。教材后面讲最小 CPU 时，PC、指令寄存器和控制状态一般优先使用边沿触发器；讲总线和 I/O 时，再把锁存器作为相位保持和输出隔离工具使用。

锁存器的问题在于透明窗口。只要 enable 有效，输入变化就可能一路穿透到输出。若多个锁存器共用同一个 enable，并且它们之间还接着组合逻辑，状态可能在一个打开窗口内穿过多级，边界变得模糊。对小电路来说，锁存器易于分析；对大规模同步机器来说，更清晰的做法是把写入集中到离散时钟边界。

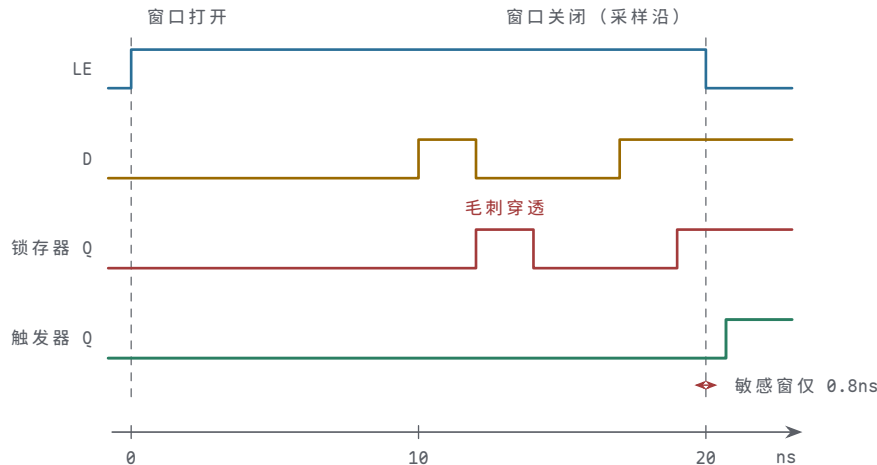
透明窗口不是“锁存器不能用”的理由，而是要求设计者明确时间边界。两相锁存器（主、从两级各用半个时钟周期的锁存器）、流水线透明锁存、片内时钟门控单元都可能利用锁存器，但它们依赖严格的相位、非重叠时钟（两相高低时间不重叠的时钟）和时序分析。初学同步机器时，先使用边沿触发器更稳妥，因为每一级状态只在时钟沿提交，调试时可以把每个周期看成独立的一步：上一步的 Q 经过组合逻辑形成本步的 D，本步边界再把 D 提交为下一步的 Q。

专题：门控 D 锁存器的透明窗口——为什么计数器和状态机不用它

门控 D 锁存器在 LE 有效期间是透明的：D 的任何变化经过几级门延迟就出现在 Q 上。这不是缺陷描述，而是工作机制描述——锁存器保存的是窗口关闭那一刻的 D，窗口之内它只是一条带延迟的组合通路。由此推出两个直接后果。第一，晚到的数据会穿透：只要 D 在窗口关闭之前变化，新值就来得及进入并被保存。第二，毛刺也会穿透：D 上的短暂干扰会经过同样的延迟出现在 Q 上，下游看到的是一条和输入几乎一样乱的波形，只是晚了一点。

用一组具体数字感受两种元件的差别。设 LE 窗口宽 20ns ($t=0$ 打开， $t=20\text{ns}$ 关闭)，锁存器传播延迟约 2ns。D 在 $t=17\text{ns}$ 变化， $t=19\text{ns}$ 新值已经出现在 Q 上，窗口关闭时保存的就是这个新值——离关闭只剩 3ns 的变化仍然穿透。D 上一个 2ns 宽的毛刺出现在 $t=10\text{ns}$ ，约 $t=12\text{ns}$ 起 Q 上跟着出现一个毛刺。换成 D 触发器看同样的输入：触发器只关心采样沿前后那一小段窗口，设建立时间 0.5ns、保持时间 0.3ns，那么 $t=10\text{ns}$ 的毛刺和 $t=17\text{ns}$ 的变化只要不落在沿前后共 0.8ns 的窗口里，就不会被采进去。锁存器的敏感区是整个 20ns，触发器的敏感区是 0.8ns，相差 25 倍；这个倍数就是“透明”二字的代价。

把这段过程按时间轴画出来，两种元件的敏感区差别就一目了然：



图示：同一个 D 送到两种元件：LE 窗口（t=0 到 20ns）内，D 上 2ns 宽的毛刺（t=10ns）和晚到的变化（t=17ns）都经过约 2ns 延迟穿透到锁存器 Q；D 触发器只认 t=20ns 采样沿前后共 0.8ns 的窗口，毛刺完全不可见，晚到变化被正常采入，沿后 Q 一步更新。敏感区 20ns 对 0.8ns，相差 25 倍——这就是透明窗口的代价。

输入事件	门控 D 锁存器（LE 窗口 20ns）	D 触发器（沿采样）
D 在窗口打开前已稳定	Q 早已跟随，窗口关闭时保存	沿到来时采样，沿后 Q 更新
D 在窗口中段才变化（晚到数据）	Q 随之更新，保存的是晚到的值	距沿超过建立时间则正常采样，但 Q 只在沿后才变
D 在窗口内出现毛刺	毛刺经传播延迟出现在 Q 上	毛刺不落在沿附近窗口内则完全不可见
D 在窗口外任意变化	无影响	无影响

为什么计数器和状态机必须用触发器，而不能用共用一个 LE 的透明锁存器？关键在于这两类电路存在经过组合逻辑回到自身输入的反馈。计数器的下一值是当前值加一，状态机的下一状态由当前状态和输入共同译码。若状态元件是透明锁存器，LE 打开的整个窗口里，新状态会穿过组合逻辑再次改变 D，D 再穿透到 Q，一个窗口内状态可能连续前进好几次，前进步数由窗口宽度和路径延迟共同决定——这等于把时序问题换成了竞态问题。触发器把这个回路切成离散步：一个沿只提交一次，本沿采到的 D 要等下一个沿之后才可能再次影响 D，于是每台状态机每个周期恰好前进一步。这就是同步设计可分析、可调试的根本原因。

这并不是说透明锁存器一无是处。前面 8086 地址锁存的案例已经说明，协议给出清晰电平窗口的地方正是锁存器的用武之地；下一节还会看到，主从结构正是用两个窗口错开的锁存器拼出一个触发器。工程结论因此是分层而不是否决：跨相位保持用锁存器，状态回路的提交边界用触发器。

二、触发器、时钟周期与采样窗口

触发器解决的是“什么时候写入”的问题。它不像锁存器那样在一段时间内透明，而是在时钟沿附近采样输入，把采样值保存到输出。

```
on clock edge:
    Q = D
between clock edges:
    Q holds
```

这让大电路有了清楚节奏。一个时钟沿之后，上一批触发器输出新状态；这些状态经过组合逻辑传播，形成下一批输入；到下一个时钟沿，目标触发器再统一采样。

于是“计算”和“保存”被分开了。

以 74HC74 这类双 D 触发器为例，D 是待采样数据，CP 或 CLK 是采样边界，Q 是保存后的输出，异步置位和异步清零用于把状态强制带到已知值。它的引脚命名迫使设计者区分三件事：数据是否已经准备好，时钟边沿是否合格，异步控制脚是否处于安全状态。若 D 在边沿附近变化，触发器不保证采样结果；若清零脚悬空，触发器可能在电源噪声中被误复位；若时钟来自未消抖按钮，单次按压可能被解释成多个边沿。

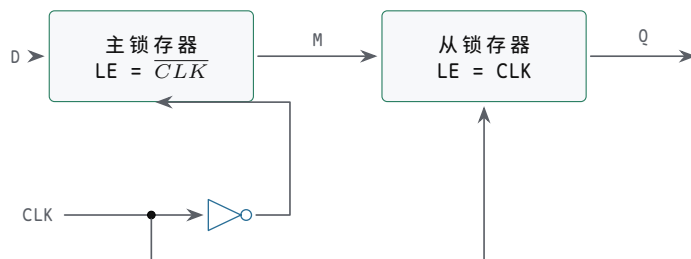
把 74HC74 接成最小实验电路时，应把异步置位和异步清零接到确定无效电平或受控复位，D 端由开关、组合逻辑或前级触发器驱动，CP 端只能接经过整形的时钟。若用按钮作为单步时钟，按钮必须先经过上拉/下拉、消抖和施密特整形；否则一次机械按压可能产生多个有效边沿，使 Q 连续翻转或连续采样。这个案例说明，**时钟边沿质量**与逻辑功能同等重要。

把这只触发器拆开看，每个对象都有明确的读法。D 输入是下一个边界要保存的候选值，但“只要最终正确、何时稳定都无所谓”是误解——它必须在采样窗口前后都稳定。CLK/CP 是状态提交的边界，不是任意跳变都能当时钟，边沿质量和电平窗口都有要求。Q 输出是已提交的当前状态，它和 D 只在边界之后才相等，不能认为随时相等。异步复位/置位不等时钟即可强制状态，但不能悬空或随意接按钮。数据手册时序（建立、保持、脉宽、传播延迟）不是高频设计才需要读的东西，低速实验同样会因为不满足它而失败。



图示：D 触发器方框符号：只在时钟上升沿采样 D 并保存到 Q。

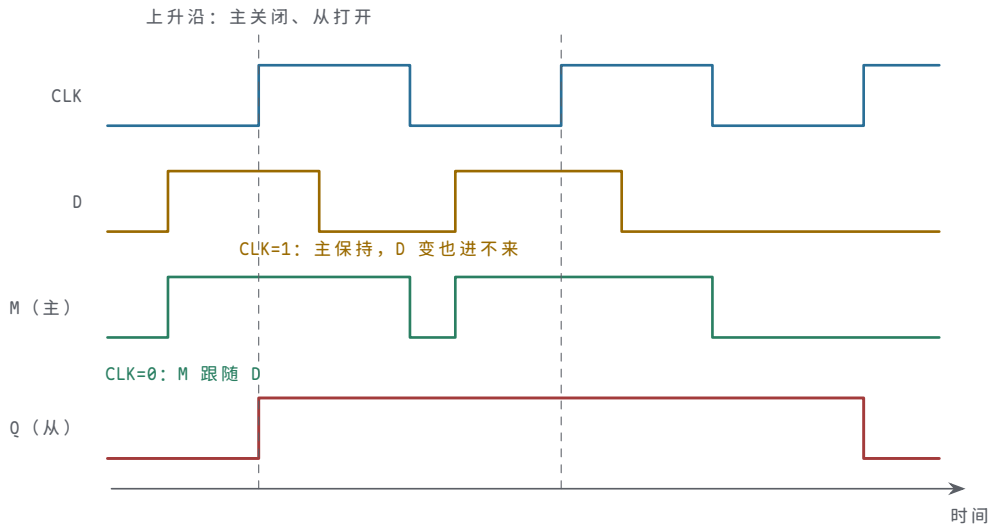
触发器为什么能做到“不像锁存器那样透明”？最常见的实现是主从结构：两个 D 锁存器串联，主锁存器的使能接反相后的时钟，从锁存器的使能接原时钟。CLK=0 时，主锁存器透明、从锁存器保持——D 的变化可以进入主，但到不了 Q；CLK 从 0 变 1 的瞬间，主锁存器关闭、从锁存器打开，主此刻保存的值被交给 Q；CLK=1 时，主保持、从透明——Q 跟随主保存的值，但主不再受 D 影响。所以 Q 只在上升沿那一刻获得新值。边沿触发不是一种新的存储元件，而是两个透明窗口错开半个周期的锁存器组合。



主锁存器在 CLK=0 时透明，从锁存器在 CLK=1 时透明：两个窗口永远错开

图示：主从 D 触发器：D 只能在 CLK=0 期间进入主锁存器；CLK 上升沿把主保存的值移交从锁存器，Q 才更新。

把主从两级放进同一张波形，“边沿”就不再是断言：



图示：主从两级的工作波形。M 在 CLK=0 期间跟随 D，CLK=1 期间保持——所以 D 在 CLK=1 里的翻转（本例 2.8 处）进不了主；Q 只在上升沿复制 M 当时的值。所谓“上升沿采样”，全部秘密就在这两条错开的透明窗口里。

专题：JK 与 T 触发器——从两条命令到一个翻转请求

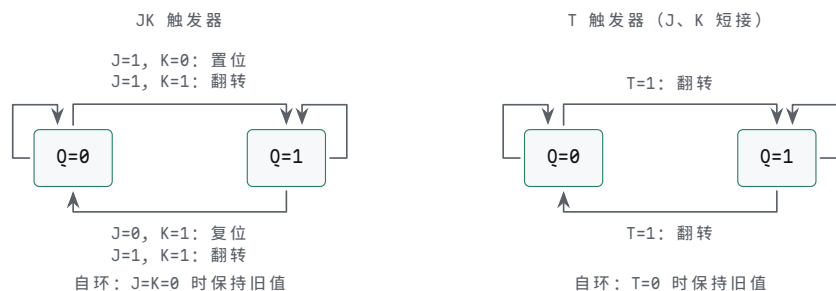
D 触发器不是唯一的边沿触发器。中小规模器件里长期流行的还有 JK 触发器：它把 SR 的两条命令搬到时钟沿上，J 对应置位请求，K 对应复位请求，并且顺手解决了 SR 的禁用组合问题——J 和 K 同时有效不再是非法输入，而是表示“翻转”。逐行看它的约定：J=1，K=0 时，下一个有效沿把 Q 写成 1，与当前值无关；J=0，K=1 时写成 0；J=K=0 时没有请求，Q 保持；J=K=1 时，下一值被定义成当前值的反，于是每个有效沿 Q 翻转一次。

J	K	下一 Q	含义
0	0	Q	无请求：保持旧值
1	0	1	置位请求：沿后 Q=1
0	1	0	复位请求：沿后 Q=0
1	1	$\bar{Q}$	翻转请求：沿后 Q 取反

对照 SR 锁存器就能看出 JK 的价值：SR 的第四行是禁用组合，JK 的第四行是有明确定义的功能。同一个“两条命令同时有效”的情形，前者只能交给门延迟竞争，后者被约定成确定行为。这也说明触发器家族的差异不在存储机制——它们都在边沿保存 1 bit——而在输入命令集如何约定。

把 J 和 K 短接在一起，JK 触发器就退化成 T 触发器：T=0 相当于 J=K=0，保持；T=1 相当于 J=K=1，每个有效沿翻转一次。T 触发器几乎不单独出现，但它是计数与分频的基本单元。把 T 固定接 1，每个时钟沿 Q 翻转一次：输入每过两个完整周期，Q 才完成一次高低循环，所以 Q 的频率恰好是输入的一半——一只 T=1 的触发器就是一个二分频器。

把 JK 的四行命令画成两个状态之间的转移，四种输入组合各就各位：



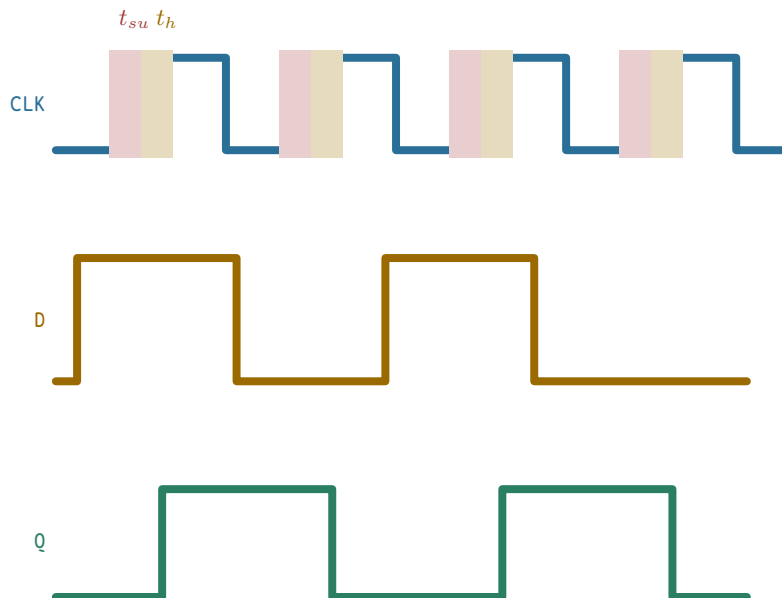
图示：JK 触发器（左）与 T 触发器（右）的状态转移。JK 的两条跨边覆盖三种组合：J=1, K=0 把 Q 写 1, J=0, K=1 把 Q 写 0, J=K=1 在两条跨边上都出现——每个有效沿必换边，这就是翻转环；J=K=0 时两个状态都走自环保留旧值。T 触发器是 J、K 短接的退化：T=0 保持，T=1 每沿翻转一次。

把三级这样的触发器串起来，前一级的 Q 作为后一级的时钟，就得到一条纹波分频链：

级	时钟来源	输出翻转频率	相对输入时钟
第 1 级 Q_0	输入时钟 CLK	$f/2$	二分频
第 2 级 Q_1	Q_0	$f/4$	四分频
第 3 级 Q_2	Q_1	$f/8$	八分频

代入数字：输入时钟 1MHz 时， Q_0 为 500kHz， Q_1 为 250kHz， Q_2 为 125kHz； Q_2 的周期是 8μs，恰好等于 8 个输入周期。换一个角度看同一条链：每经过 8 个输入沿，三级输出（ Q_2, Q_1, Q_0 ）的组合就遍历 8 种编码回到起点，所以三级 T 链同时就是一个模 8 的 3-bit 计数器。分频和计数是同一个电路的两种读法：前者看单路输出的频率，后者看多路输出的编码。本章后面讲计数器时还会看到，纹波结构的代价是各级时钟到输出延迟逐级累加， Q_2 要等三级延迟之后才稳定；位数一多，这条链本身就成为关键路径，这正是同步计数器存在的理由。

有了触发器的采样模型，建立/保持时间窗口就可以对照波形图逐项检查：



图示：建立/保持时间窗口都相对于上升采样沿：D 必须在上升沿之前 t_{su} 已经稳定，并在上升沿之后 t_h 继续保持；Q 在上升沿后经过 t_{cq} 才更新。

最常见的同步路径可以写成：

```
source register -> combinational logic -> destination register
```

时钟沿到来后，源寄存器输出旧状态对应的新值。这个值进入组合逻辑，经过门延迟和布线延迟，最终到达目标寄存器输入。下一个时钟沿到来时，目标寄存器采样这个结果。因此频率不是越高越好。频率提高意味着周期缩短，留给组合逻辑稳定的时间变少。若周期短到最慢路径来不及稳定，目标寄存器就可能采到旧值、过渡值或错误值。

同步设计的关键，是区分**当前状态**和**下一状态**。当前状态存在于源寄存器 Q 端，下一状态存在于目标寄存器 D 端。组合逻辑可以在整个周期内反复变化，直到输入稳定、路径传播完成、D 端满足建立时间；但只有时钟沿到来，D 才会成为新的 Q。用软件赋值类比硬件时，最容易犯的错误是把 D 端的临时变化当成已经写入。硬件

没有“变量立即更新”的全局瞬间，只有被状态元件定义出来的提交边界。

阅读触发器数据手册时，至少要把四类时间参数分开。第一，时钟到输出延迟说明源状态何时对后级可见。第二，建立时间和保持时间说明目标触发器允许 D 端怎样靠近采样沿。第三，时钟高电平、低电平和脉冲宽度说明怎样的波形才算合格时钟。第四，异步复位或置位的脉宽、恢复时间和释放时间说明复位不能随意撤销。即使低速面包板实验看起来“肉眼很慢”，按钮抖动、慢边沿和悬空输入仍然可能违反这些时间约定。

数据手册项目	应回答的问题	板上故障表现
时钟到输出延迟	Q 何时能作为后级输入使用	后级过早采样旧值或过渡值
建立/保持时间	D 在边沿前后要稳定多久	偶发采错、亚稳态或重复触发
最小时钟脉宽	CP 高低电平是否足够长	窄脉冲被忽略或产生异常采样
复位恢复时间	复位释放离时钟沿是否过近	复位后状态机随机进入非入口状态

触发器采样也不是数学瞬间。输入 D 必须在时钟沿到来前稳定一小段时间，这称为建立时间，违反它会采到错误值或不稳定值；时钟沿之后还要继续稳定一小段时间，这称为保持时间，违反它会让新旧值混入采样窗口。



图示：时钟周期预算：各段时间之和决定最小周期。

一条路径的周期预算可以写成：

```
clock period >= source clock-to-Q
                + combinational delay
                + wire delay
                + destination setup time
```

更严谨地写，还要把时钟偏斜和余量纳入公式。若目标触发器的时钟沿比源触发器更早到达，有效可用时间会变少；若目标时钟更晚到达，建立时间压力可能减轻，但保持时间压力可能增加。为了保持第一轮理解清晰，可以先把建立约束写成：

```
Tclk >= tCQ_max + tcomb_max + twire_max + tsetup + tskew + margin
```

其中 `tCQ_max` 是源触发器时钟到输出的最大时间，`tcomb_max` 是组合逻辑最大延迟，`twire_max` 是布线最大延迟，`tsetup` 是目标触发器建立时间，`tskew` 表示不利时钟偏斜，`margin` 是留给电压、温度、工艺、建模误差和老化的余量。建立约束回答“最慢结果能否赶上下一个采样沿”。

项目	示例数值	读法
<code>tCQ_max</code>	1.2ns	源寄存器输出最晚 1.2ns 后有效
<code>tcomb_max</code>	5.8ns	组合逻辑最坏传播时间
<code>twire_max</code>	0.9ns	远距离连线和负载引入延迟
<code>tsetup</code>	0.6ns	目标寄存器采样前要求

$t_{\text{skew}} + \text{margin}$	0.5ns	偏斜和工程余量
合计	9.0ns	时钟周期必须不小于 9.0ns



图示：异步拉入、同步释放的复位电路。

若目标频率要求周期为 8ns，上表路径就不合格。修复方式不是在公式里删除余量，而是缩短组合逻辑、增加寄存器边界、改善布局布线、换用更快单元、降低频率，或重新安排微操作。同步设计的严肃性在于：功能真值表正确，仅能说明最终会算对；时序约束也满足，才说明能在规定边界前算对。

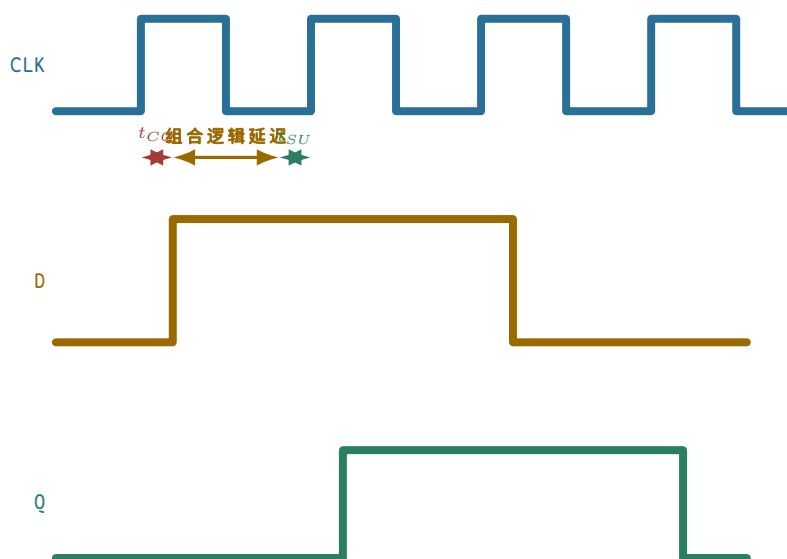
保持时间看起来更小，却同样危险。若源寄存器输出太快、路径太短，目标寄存器可能在同一个时钟沿之后立刻看到新值，从而破坏它本应采旧值的窗口。长路径会限制最高频率，短路径也可能破坏保持约束。

保持约束可以按最小延迟理解：

$$t_{CQ_min} + t_{comb_min} + t_{wire_min} \geq t_{hold} + \text{unfavorable_skew}$$

这条式子回答“新值是否来得太早”。例如源触发器最快 0.1ns 就改变 Q，路径中几乎没有组合逻辑，布线也很短，而目标触发器要求保持 0.3ns；若时钟偏斜又让目标采样沿相对更晚，那么新值可能在目标仍需要旧值稳定时抵达。修复保持问题的常见手段是插入小延迟、调整布线、改变时钟树或重新放置寄存器。它和建立问题方向相反：建立嫌路径太慢，保持嫌路径太快。

把这几类约束放在一起看，修复手段完全不同：建立约束失败说明最慢路径赶不上采样沿，要缩短组合逻辑、流水线化、降频或优化布局；保持约束失败说明最快路径太早冲进采样窗口，要增加最小延迟、调整时钟偏斜或改变布线；复位释放失败是状态元件退出复位不一致，要异步拉入、同步释放并限定复位域；跨时钟输入失败是采样沿附近输入不受控，要用两级同步、握手、异步 FIFO 或协议化采样。用建立的修法去修保持，或者用降频去解决复位和跨边界问题，都会把故障掩盖成偶发错误。



图示：同一个上升沿既是源寄存器的更新沿，也是目标寄存器的采样沿：源在 t_{CQ} 后改变 D，组合逻辑在下一上升沿 t_{SU} 前让 D 稳定，目标在该上升沿后 t_{CQ} 更新 Q。 $t_{CQ} + \text{组合逻辑延迟} + t_{SU} = \text{最小时钟周期}$ 。

三、关键路径、复位与跨边界输入

一块同步电路里会有很多寄存器到寄存器路径。最长、最慢、最限制周期的那条叫关键路径。它可能是一串加法器，也可能是多级选择器，也可能是远距离布线加上复杂译码。

例如一个 32-bit 串行进位加法器接在两个寄存器之间。最低位进位要一位一位传到最高位，最高位结果最后才稳定。如果把这个加法器放在一个周期里完成，那么周期至少要等最坏进位链走完。若把运算拆成更短的几段，中间插入寄存器，单个周期可以变短，但完成一次完整运算需要更多周期。

关键路径不只来自“算术看起来复杂”的地方。一个很宽的 mux、远距离控制线、解码后再进入选择器的路径、寄存器堆读口到 ALU 再到写回选择器的路径，都可能成为限制频率的对象。真实 CPU 中，很多优化不是改变逻辑功能，而是重新安排边界：把一次长动作拆成取指、译码、执行、访存、写回；把全局信号改成本地译码；把长总线切成更短的局部通路。时序分析让这些选择可以相互比较，而不是靠直觉判断“这条线应该很快”。

在整机视角下，8086、80386 和现代处理器都在反复处理同一个问题：哪些动作放在一个边界之间完成，哪些动作必须切成多个边界。8086 的外部总线周期把地址、控制 and 数据分成若干相位，用总线相位和等待状态分步完成长访问；80386 的 32-bit 数据通路、分页地址转换和总线接口让路径更宽、更复杂，因此必须更严格地区分内部状态、外部总线状态和等待状态；现代 CPU 则进一步把取指、译码、重命名、执行、访存和提交拆成流水阶段，用局部化、cache 层次和旁路网络缩短常见路径。FPGA/ASIC 数据通路里，宽 mux、长连线、加法器链和译码扇出也按同样思路处理：插入寄存器、重定时、局部译码和布局优化。它们规模不同，但工程判断相同：若一条路径太长，就要缩短组合逻辑、增加状态边界或改变协议等待方式。

这也解释了“一周期完成”和“拆段插入寄存器”的取舍：一周期完成长运算，控制简单、单条动作需要的边界少，代价是关键路径长、频率低；拆段插入寄存器让单周期路径变短、频率可能更高，代价是完成一次动作需要更多边界和更复杂的控制。



图示：两级触发器同步器结构。

触发器通电后的值不一定符合设计预期。若一组状态寄存器上电后取值各不相同，后面的组合逻辑可能立刻进入非法状态。复位信号的作用，是把关键状态单元带到已知点，例如计数器清零、状态机回到 idle、控制寄存器进入安全默认值。

并不是所有触发器都必须复位。控制状态、外部使能、写存储器信号、片选、错误标志和协议状态通常需要复位，因为它们会决定机器是否对外产生动作；纯数据缓冲如果有可靠写使能保护，未必需要每一位都在复位时清零。过度复位会增加布线、负载和时序压力，尤其在大规模芯片中会让复位网络本身变成困难对象。因此，复位设计不是“全清零”，而是列出哪些状态必须已知、哪些输出必须安全、哪些数据可以等第一次有效写入后再使用。

按这个原则分类：控制状态机必须复位，它要有唯一入口和安全输出；外部写使能和片选必须复位，上电和复位期间不能误写外设或存储器；计数器和超时器通常要复位，初始计数会影响协议等待和超时判断；流水数据寄存器视情况而定，若有 valid 位保护，数据位可以等首次有效写入；同步器和事件 pending 位必须复位，复位释放不能制造伪事件。复位本身也要讲边界。

若复位释放时，有的触发器先退出复位，有的后退出复位，状态机可能看到一半新状态、一半旧状态。常见处理是异步拉入复位以便快速进入安全状态，再同步

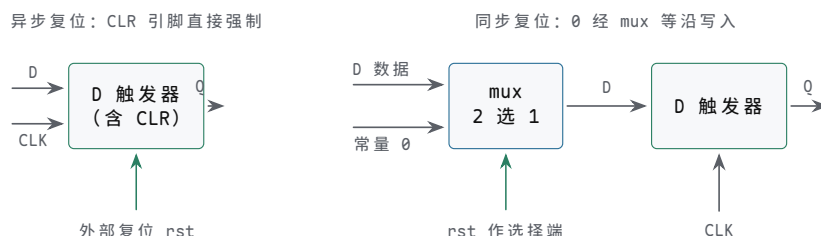
释放，让所有相关触发器在清楚的时钟边界恢复工作。复位不能简化为“清零所有东西”。哪些状态需要复位，哪些数据寄存器可以不复位，哪些输出在复位期间必须安全，都要由硬件需求决定。

专题：异步复位与同步复位—立即生效与沿上生效的对照

触发器的复位有两条实现路径。异步复位使用触发器专用的 CLR 引脚，直接强制内部反馈环进入 0 状态，不问时钟：引脚一有效，Q 立刻被拉成 0，时钟是否在运行都无所谓。同步复位只是数据路径上的一个约定：复位有效时，D 端之前的选择逻辑把 0 送到触发器输入，Q 要等下一个有效沿才变成 0。前者改变存储元件本身的状态，后者改变准备提交给存储元件的下一值——这是两种复位最本质的区别。

两条路径各有代价。异步复位的优势是上电即有效：时钟还没起振、时钟被门控关闭、系统卡在未知状态时，只有不问时钟的复位才能保证把状态拉到已知点。它的代价在释放端：撤销复位同样是一个异步事件，若释放时刻落在时钟沿附近的恢复时间窗口内，触发器可能这一拍就响应新的 D，也可能仍停留在复位值，结果不可预知。同步复位的优势恰好相反：生效和释放都以时钟沿为界，普通的建立/保持分析就能覆盖，不会制造半个周期的歧义；它的代价是必须有时钟才起作用，并且复位选择逻辑串在 D 路径上，给关键路径增加一级延迟。

把两条路径画成结构对照，“改存储元件本身”与“改下一值”的区别就写在连线上：



图示：异步复位（左）：rst 走触发器专用 CLR 引脚（虚线控制），不问时钟，引脚一有效 Q 立即清零，时钟停振时也成立。同步复位（右）：rst 只是 D 路径上 mux 的选择端，有效时把常量 0 送到 D，Q 要等下一个有效沿才变 0；生效与释放都由时钟沿界定，代价是选择逻辑串在数据路径上，给关键路径加一级延迟。

关键问题	异步复位	同步复位
何时生效	引脚有效立即生效，不问时钟	下一个有效沿才生效
无时钟时能否工作	能：上电、停振时都有效	不能：沿不来状态不变
数据路径代价	不占 D 路径，使用专用引脚	选择逻辑串入 D 路径，增加一级延迟
释放时刻	仍是异步事件，可能违反恢复时间	由建立/保持分析覆盖，时刻确定
引脚毛刺影响	毛刺直接清掉状态	只有沿附近的毛刺才可能被采入

用一个数量例子看释放风险。设触发器要求的恢复时间是 1.0ns，异步复位在某个沿之前 0.3ns 才撤销——离沿太近，这个触发器本次沿是否退出复位不可预知。若状态寄存器的两个 bit 因复位走线长度不同，一个看到释放离沿 0.3ns，另一个看到 1.2ns，前者行为不定、后者满足恢复时间正常退出，两个 bit 就可能错开一拍退出复位，状态机出现一个周期的混合状态，可能走进任何编码。这就是“异步拉入、同步释放”成为标准做法的原因：拉入用异步路径，保证任何时刻都能进入安全状态；释放经过本节前面的两级触发器同步到本地时钟边界，让所有状态元件在同一拍恢复工作。两种复位不是互相替代的竞争方案，而是分别负责“进得去”和“出得来”两道工序。

跨时钟或来自外部世界的输入还会带来亚稳态。若输入 D 正好在时钟沿附近变化，触发器内部节点可能短时间落在不稳定区域，既不像明确 0，也不像明确 1。多

数情况下，它会在一小段时间后恢复到明确的 0 或 1，但恢复到哪一边、需要多久，不能按普通组合逻辑精确保证。因此来自另一个时钟节奏或外部世界的信号，不能直接拿来控制大面积电路，通常要先经过同步结构，降低亚稳态扩散概率。

最常见的单 bit 同步结构是两级触发器串联。第一级直接采样异步输入，可能进入亚稳态；第二级在下一个时钟沿再采样第一级输出，给第一级更多时间恢复到明确 0 或 1。两级同步不能从数学上消灭亚稳态，但能显著降低亚稳态传播到后级控制逻辑的概率。多 bit 数据不能简单地每一位各接两级同步后直接使用，因为各位可能在不同周期被采到，形成混合值；多 bit 跨边界通常要使用握手、有效位、灰码计数器或异步 FIFO。

工程上常用 **平均无故障时间** (MTBF, Mean Time Between Failures) 描述亚稳态风险是否可接受。它不是功能真值表，而是可靠性指标：异步输入变化越频繁、目标时钟越快，触发器遇到采样窗口的机会越多；留给第一级同步器恢复的时间越长，亚稳态传播到第二级之后的概率越低。因此，提高同步级数、降低异步事件频率、给同步器留出完整周期、避免在同步器后面接复杂组合逻辑，都能改善可靠性。严肃设计不会声称“两级同步一定安全”，而是说明在目标频率、输入事件频率和器件参数下，失效率是否满足系统要求。

放一个数量级感受“恢复时间买可靠性”的杠杆有多大。亚稳态未恢复概率随恢复时间按指数 $e^{-t/\tau}$ 下降：恢复窗口每多一个器件时间常数 τ ，失败概率约下降 2.7 倍；每多约 2.3τ ，失败概率大约下降一个数量级。假设某触发器 $\tau = 0.1\text{ns}$ ，第一级留约 2ns (约 20τ) 恢复窗口，单次采样的失败概率约 10^{-10} 量级；异步输入平均每秒变化 1000 次，则大约 10^7 秒 (近 4 个月) 才出现一次可见失败。若只留 0.3ns (约 3τ) ——比如同步器后面紧跟一条长组合路径再吃一级触发器——恢复窗口少了约 17 个 τ ，失败概率抬升约 e^{17} (约 7 个数量级)，达到约 10^{-3} ，同样输入下每天失败数万次。这些数字是示意量级，真实参数必须查器件手册；但结论形态是普适的：**可靠性按指数购买，时钟越快、事件越频繁、恢复窗口越短，买到的越少。**

MTBF 影响因素	对可靠性的影响	设计含义
目标时钟频率	采样次数越多，遇到危险窗口机会越多	高频域更需要规范同步结构
异步事件频率	输入变化越频繁，触发亚稳态机会越多	高频事件应使用握手或 FIFO
恢复时间	两级触发器之间时间越充足，传播概率越低	同步器后不应接长组合路径再采样
器件特性	触发器恢复参数决定概率下降速度	应使用目标工艺或芯片数据手册参数

跨边界对象	推荐处理	不当处理的风险
单 bit 按钮/中断	两级同步后再进入状态机	亚稳态扩散或一次事件被识别多次
多 bit 数据总线	使用握手、valid/ready 或 FIFO	各 bit 来自不同采样时刻，形成错误编码
复位释放脉冲信号	异步拉入、同步释放 转换为电平握手或拉宽后同步	状态机部分触发器先运行 窄脉冲被漏采或重复采样

专题：跨时钟域的两类基本做法——单 bit 同步与多 bit 协议

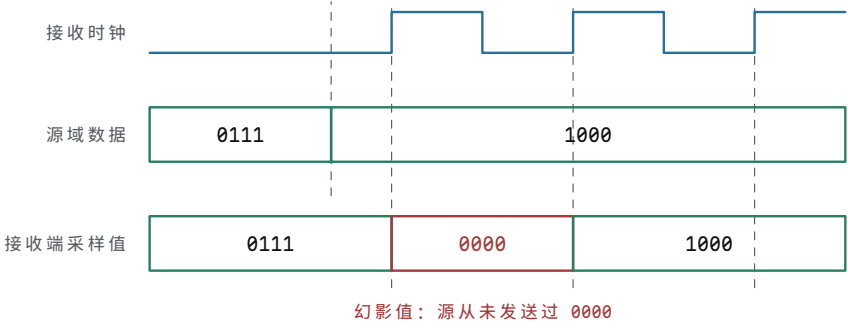
上表把跨边界对象分了类，这里把背后的两类做法讲透。分类的依据不是信号数量本身，而是“接收方能否容忍采样时刻的不确定”。单 bit 信号只有一个自由度，采早采晚最多差一个周期，两级同步器把亚稳态概率降到可忽略量级之后，剩下的

时机不确定性可以靠协议容忍。多 bit 数据的每一位各有自己的采样时刻，逐位同步可能让同一拍采到不同代的 bit，错的不只是时机，而是数值本身，所以必须引入协议。

单 bit 做法本章已经展开：两级触发器串联，第一级留给亚稳态恢复，第二级输出干净电平，前面也已经算过 MTBF 的账——恢复窗口从约 2ns 缩到 0.3ns，失败间隔就从近 4 个月缩成每天数万次。这里只补一条数量级推论：因为失败概率随恢复时间按指数 $e^{-t/\tau}$ 下降，MTBF 对余量极端敏感，余量每多一个器件时间常数 τ ，失败间隔大约拉长 2.7 倍；每多约 2.3τ ，大约拉长 10 倍。在两级之后再添第三级，恢复窗口近似翻倍，MTBF 随之再抬高若干个数量级，从“产品寿命内偶见”推到“远远超出产品寿命”；反过来，同步器后面紧接一条长组合路径再吃一级触发器，等于把恢复窗口削掉大半，MTBF 可能从年级跌回天级。结论很朴素：可靠性按指数购买，买多买少由恢复窗口决定，而恢复窗口几乎不花钱——只是触发器和布线。

多 bit 做法先看为什么不能逐位同步。设源域一个 4-bit 计数值从 0111 变成 1000，四个 bit 同时变化；若每位各接一条两级同步器，各路走线和器件延迟略有差异，接收域某一拍可能采到低三位的新值和最高位的旧值——于是接收方依次看到 0111、0000、1000，中间的 0000 是源从未发送过的幻影值。控制逻辑若拿幻影值做判断，机器就按一个不存在的状态行动。逐级加触发器也解决不了这个问题，因为它消除的是亚稳态，不是各位之间的采样时刻分歧。

把这次传送按接收时钟逐拍画出，幻影值的出现过程就清晰可见：

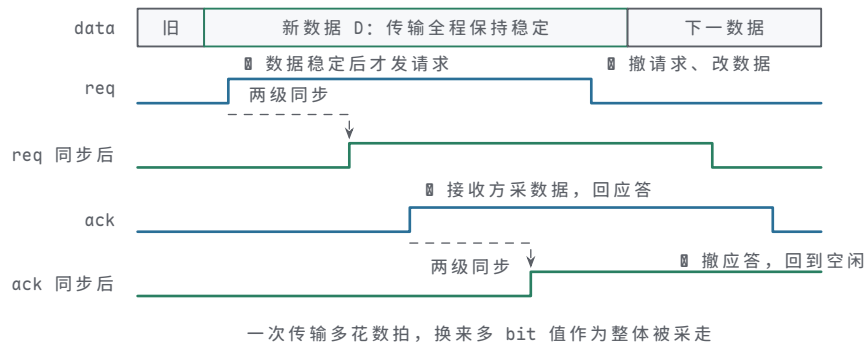


图示：4-bit 计数值从 0111 跳变到 1000，四个 bit 同时变化；各路两级同步器的走线和器件延迟略有差异，最高位的新值得比低三位晚。接收端在 $x=4$ 的采样沿采到低三位的新值和最高位的旧值，于是依次看到 0111、0000、1000——中间的 0000（红色）是源从未发送过的幻影值。逐级加触发器消除的是亚稳态，消除不了各位之间的采样时刻分歧；多 bit 跨域必须靠握手、格雷码或异步 FIFO 把数值作为整体传递。

跨域对象与做法	机制	代价与适用
单 bit 电平或事件：两级同步器	第一级吸收亚稳态，第二级给出稳定电平	延迟一两拍；适用按钮、中断、状态标志
多 bit 批量数据：请求/应答握手	源先放好数据再发请求，数据保持稳定直到应答返回	每次传输数拍；适用低速、非连续传输
多 bit 连续数据流：异步 FIFO	读写两侧各用自己的时钟，指针用格雷码跨域传递	结构复杂、占用资源；适用连续数据流

握手的要点是“数据稳定在先，请求在后，撤销在应答之后”。源域把数据总线放好并保持不变，然后发出 request；request 经两级同步进入接收域时，数据早已稳定了好几个周期，接收方采数据不存在时刻分歧；接收方采完后发 acknowledge，同样同步回源域；源域收到应答之后才允许改变数据。一次传输多花几拍，换来的是多 bit 值作为整体被采走。异步 FIFO 则把这套思想结构化和流水化：写指针在源域递增，读指针在接收域递增，两个指针必须跨域比较才能判断空满；指针采用格雷码，相邻计数值只差一个 bit，于是指针跨域时可以安全地逐位同步——最多把指针读旧一拍，不会读出幻影编码。格雷码在这里的角色值得单独记住：它不是检错手段，而是把“多 bit 同时变化”约束成“一次只变一个 bit”，让逐位同步重新合法化。

把一次四相握手画成波形，数据、请求、应答的先后次序就固定下来了：



图示：req/ack 四相握手的一次传输：源域先把 data 放稳，再发 req (1)；req 经两级同步晚约两拍进入接收域，此刻数据早已稳定数个周期，接收方采数据不存在时刻分歧，采完回 ack (1)；ack 同步回源域后，源域才撤 req 并允许改变数据 (0)；接收方看到 req 落下再撤 ack，双方回到空闲 (0)。下一次传输（右侧下一数据）必须在 0 之后才开始。

安全启动控制板中的 `start_request` 可以作为例子。按钮原始触点先经过默认电平、保护和消抖，得到板内稳定请求；若后级控制器是同步状态机，这个请求还要在控制器时钟域内同步，状态机只能使用同步后的 `start_request_sync`。这样第一章的“可靠 bit”与本章的“可靠采样边界”才真正接上：前者解决电平和路径，后者解决保存时刻。

用 74HC14 和 74HC74 可以搭出一个完整的单 bit 跨边界入口。

74HC14 负责把按钮或慢边沿信号整形成确定翻转的逻辑电平，74HC74 的两级触发器负责把该电平带入目标时钟域。若后级是计数器或状态机，只允许使用第二级 Q；第一级 Q 是给亚稳态恢复留时间的内部边界，不应直接参与控制。这个小电路比“按钮直接接计数器 CP”更接近真实机器的输入入口。

专题：从外部事件到同步状态机

许多同步电路的故障并不发生在核心状态机内部，而是发生在外部事件进入状态机之前。按钮、传感器、限位开关、中断线、另一片芯片的 `ready` 信号，都可能在本时钟域的任意时刻变化。若把这些信号直接接到状态机输入，设计等于默认外部世界服从本时钟的建立时间和保持时间；这个默认在真实硬件中通常不成立。

可以把外部事件进入同步状态机的链路写成六段：

```
raw pin
-> electrical default and protection
-> filtering or debounce
-> level normalization
-> clock-domain synchronization
-> event detection or handshake
-> FSM consumption
```

第一段处理电气边界。原始引脚不能悬空，不能让过压、静电或长线干扰直接进入芯片内部；上拉、下拉、限流、钳位、RC 滤波和施密特输入都属于这一层。第二段处理物理抖动或噪声。按钮按下不是一个理想阶跃，可能在数毫秒内多次跳变；传感器线缆也可能受到干扰。第三段把电平极性转成内部语义，例如把低有效的 `start_n` 转成高有效的 `start_pressed`。

第四段才进入本章的核心：同步。一个单 bit 电平在消抖后仍然是异步输入，因为它的变化时刻不由目标时钟决定。两级同步器给第一级触发器留下恢复时间，降低亚稳态传播概率。同步后的信号仍然只是“本时钟域看到的电平”，若状态机需要的是—次事件，还必须再做边沿检测、事件保持或握手确认。

这条链上的每一段都不能替代另一段：默认电平与保护解决悬空、过压、静电和异常电流，但不能保证采样时刻安全；滤波与消抖解决机械抖动、窄噪声和线缆干扰，但不能消除跨时钟亚稳态；语义归一化解决低有效、高有效和断线默认的含义

问题，但不能证明事件只发生一次；两级同步把单 bit 异步电平带进目标时钟域，但不能直接处理多 bit 一致性；边沿检测把同步电平转换成单周期事件，但不能保证目标状态机一定接受；握手或事件保持让事件持续到被消费并确认，但不能替代前面的电气保护。跳段的代价不是少一个步骤，而是某一层风险被悄悄漏过去。

以安全启动按钮为例，较完整的命名应当区分每一层信号：

```
start_button_raw      // 引脚上的原始电平
start_button_filtered // 经过默认电平、保护和消抖
start_pressed         // 归一化后的语义电平
start_pressed_meta    // 第一级同步器输出
start_pressed_sync    // 第二级同步器输出
start_event           // 本时钟域检测到的一次请求
start_pending         // 等待状态机消费的保持事件
```

这些名字不是形式主义，而是调试边界。若示波器看到 `start_button_raw` 抖动，问题属于电气或消抖；若 `start_pressed_sync` 偶发异常，重点检查同步器、时钟域和亚稳态扩散；若 `start_event` 出现两次，重点检查边沿检测和消抖窗口；若 `start_event` 只有一次但状态机没有启动，重点检查 `start_pending`、状态转移条件和消费确认。

事件保持尤其重要。假设同步后的 `start_event` 只有一个时钟周期，而状态机当前处在 `RESET_WAIT` 或 `FAULT`，这个事件可能被漏掉。更稳妥的做法是让事件进入 `start_pending` 寄存器，直到状态机在允许启动的状态下消费它，再由 `start_ack` 清除。

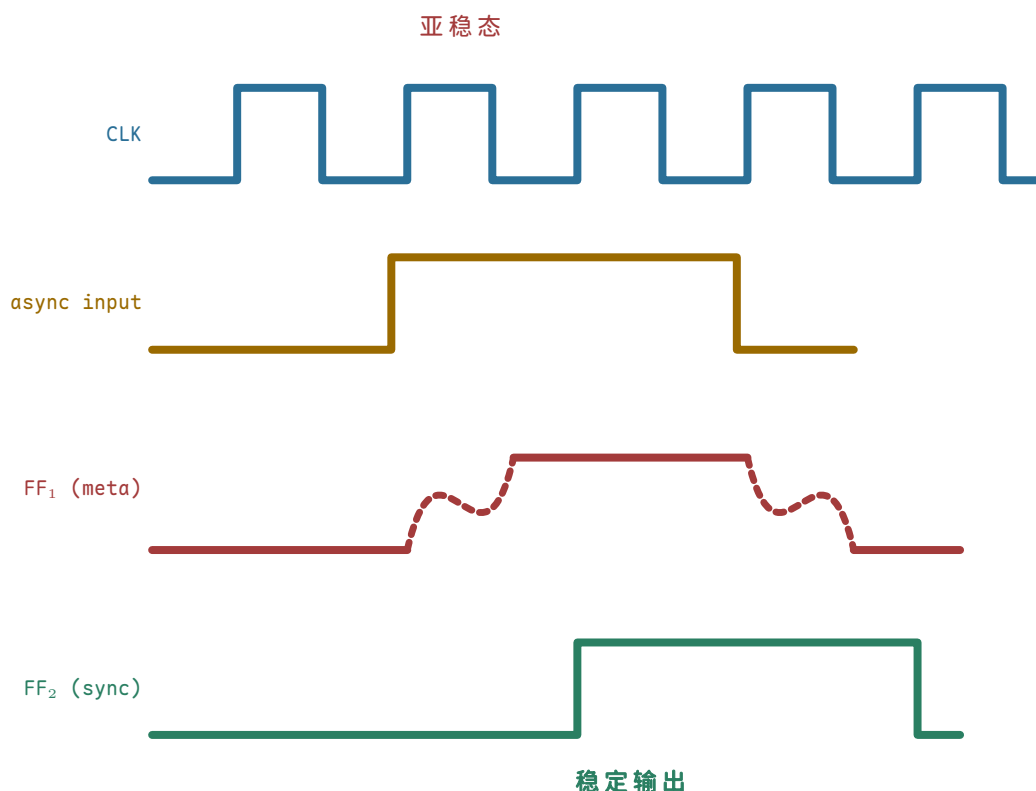
```
on clock edge:
  if reset:
    start_pending = 0
  else if start_event:
    start_pending = 1
  else if start_ack:
    start_pending = 0
```

这段逻辑说明，外部事件不是直接驱动危险输出，而是先转化为本时钟域内的状态。状态机可以在 `IDLE` 中消费 `start_pending`，在 `FAULT` 中忽略或保留它，在复位期间清除它。这样，事件、状态和安全策略都由时钟边界定义，而不是由一根不受控的外部线决定。

两级同步器只适合单 bit 控制电平或事件标志。若外部输入是 8-bit 数据、地址、计数值或编码状态，逐位接两级同步器不能保证各位来自同一个源时刻。多 bit 跨域必须使用握手、有效信号加稳定保持、格雷码计数器或异步 FIFO 等协议。

检查一条外部事件链路，可以分四个方面逐项确认。第一，电气处理：默认电平、保护、电压范围和抖动处理，缺失它就会出现悬空、抖动和多次触发。第二，同步处理：进入哪个时钟域、几级同步、复位后同步器处于什么默认状态，缺失它就会出现亚稳态扩散和复位后伪事件。第三，事件处理：电平如何变成一次事件，事件是否保持到被消费，缺失它就会出现窄脉冲漏采或重复消费。第四，状态机接口：哪些状态接受事件，哪些状态拒绝事件，拒绝时事件清除还是保留，缺失它就会出现故障状态误启动或请求丢失。

这个专题把第一章的电气可靠性、本章的同步边界和后续控制器的状态消费连接在一起。对硬件工程而言，“外部输入已经变成 0/1”不是终点；只有当它被目标时钟域可靠采样、被事件协议保持、被状态机按安全条件消费之后，它才成为可用于控制机器动作的内部事实。



图示：两级同步器波形：异步输入在上升沿附近变化；第一级在上升沿采到变化中的输入，可能短暂亚稳态（红色虚线），但在下一个上升沿前恢复到明确值；第二级在同一个上升沿采到的是已经稳定的值，输出干净。

四、寄存器、计数器与移位结构

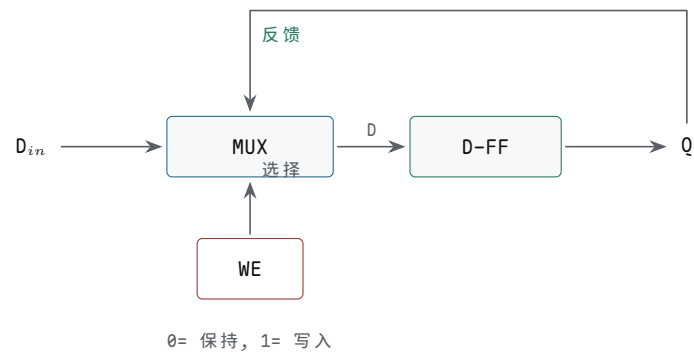
一个触发器保存 1 bit。把多个触发器并排，并让它们共享同一个时钟边界，就得到多 bit 寄存器。寄存器可以理解为共享时钟和写使能的触发器。若写使能有效，在时钟沿采样输入；若写使能无效，保持原值。

```
on clock edge:
    if write_enable:
        Q = D
    else:
        Q holds
```

寄存器的意义不只是“能存数”。它把连续传播的组合逻辑切成离散步骤。上一个周期的结果在寄存器输出端稳定存在，成为本周期组合逻辑的输入；本周期算出的结果，要到下一个时钟沿才成为新的状态。共享同一提交边界是关键：寄存器保存的是同一个边界上的一组 bit，不是一位一位随意变化。

多 bit 寄存器还承担“原子提交”的角色。若一个 8-bit 状态表示 01111111 到 10000000 的变化，多个 bit 会同时改变；后级逻辑应在同一个时钟边界之后看到新状态，而不是看到一部分旧 bit、一部分新 bit。若这些 bit 来自同一组触发器，时钟边界和传播延迟可以被统一分析；若它们来自不同异步来源，后级就可能看到不存在于设计语义中的混合值。因此，寄存器不只是若干触发器的并排复制，而是一个“这些 bit 同属一个提交时刻”的约定。

写使能寄存器通常有两种实现方式。第一种是在触发器前放数据选择器：使能为 1 时选择新数据，使能为 0 时选择旧 Q 反馈；第二种是使用器件或标准单元内部提供的 enable 触发器。两者的目标都是让时钟边界仍然存在，只是决定这个边界保存新值还是旧值。把 enable 直接拿去切时钟看起来也能保持状态，但会把 enable 的毛刺和传播延迟带进时钟树，风险远高于在数据路径上做选择。



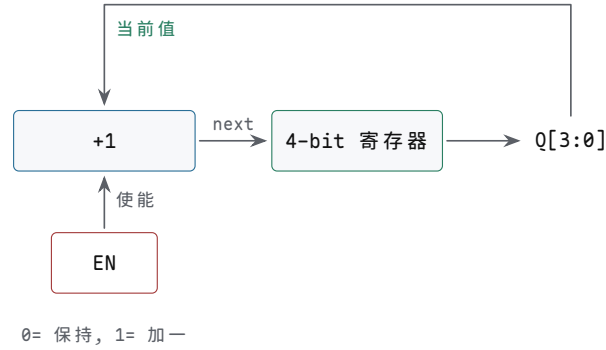
图示：写使能寄存器：数据路径决定写入或保持。WE=0 时 mux 选择旧 Q 反馈，时钟照常到达但状态不变；WE=1 时选择新 D_in。时钟树全程干净，这就是它优于门控时钟的地方。

寄存器动作	下一值表达	硬件含义
保持	<code>next = current</code>	写回旧 Q 或关闭写使能
装载	<code>next = input</code>	在边界保存外部或组合结果
清零	<code>next = 0</code>	同步清零或受控复位路径
置位	<code>next = all_ones</code>	标志位、掩码或诊断状态写入
条件更新	<code>next = cond ? input : current</code>	由控制线决定是否提交

计数器的下一值来自当前值加一：

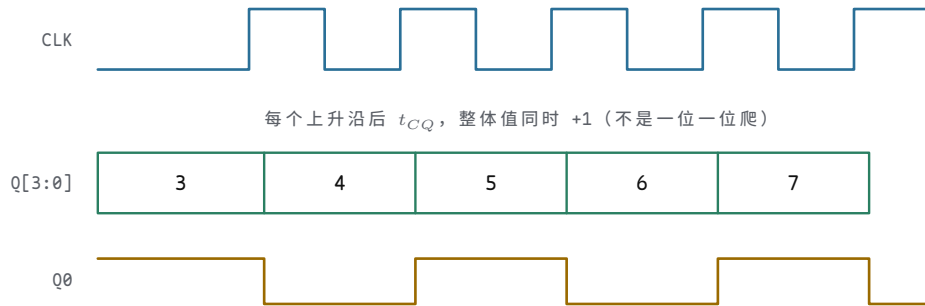
```
next = current + 1
on clock edge:
    current = next
```

该结构里有反馈，但反馈不是直接绕回去，而是经过加法器，并且被时钟边界切开。若 current 为 3，本周组合逻辑算出 next 为 4；只有下一个时钟沿到来，寄存器才保存 4。随后组合逻辑再从 4 算出 5。如果没有寄存器边界，current+1 的结果会立刻回到输入，再加一，再回到输入，电路无法表达“每个周期只增加一次”。



图示：4-bit 同步计数器。

把 3、4、5、6、7 的计数过程画成波形，”整体提交”和”逐位爬”的区别就一目了然：



图示：4-bit 计数器波形：每个上升沿，寄存器把“当前值 +1”整体提交一次，总线带上的值从 3 整齐跳到 4、5、6、7。注意 Q0——它每个周期翻转，正好是 CLK 的二分频；Q1 再慢一半。计数器的每个 bit 都是比它低一位的半速时钟，这是后面分频和定时器的基础。

74HC161/74HC163 这类 4-bit 计数器把这种结构做成了可直接使用的芯片。它们通常有时钟、计数使能、并行装载、清零、进位输出和 4 个 Q 输出。计数使能决定是否执行 `current+1`，并行装载允许在边界写入外部给定值，进位输出用于把多个计数器级联成更宽的计数器。读这类芯片的数据手册时，不能只看“它会加一”，还要查清楚清零是同步还是异步、装载优先级如何、使能脚无效时是否保持、进位输出何时有效。

计数器引脚/功能	对应的状态问题	典型用途
CP/CLK	哪个边界提交新计数值	单步时钟、系统时钟、分频后时钟
ENP/ENT 等使能	本周期是否执行加一	PC 加一、循环计数、定时等待
并行装载	是否用外部值覆盖加一结果	跳转地址、预置计数、状态恢复
清零/复位	初始状态如何确定	上电入口、错误恢复、实验复位
进位输出	更宽计数器何时进位	级联 8-bit、16-bit 或更宽计数器

把两个 4-bit 计数器级联成 8-bit 计数器时，低 4 位的进位不能被理解为普通数据输出。它是高 4 位是否在同一边界加一的控制条件，必须满足高位计数器的使能时序。若把低位 Q 经过随意组合逻辑再驱动高位时钟，就把同步计数器退化成脉动计数器，时序关系会变得难以分析。同步级联的要点是：所有位共享同一个时钟，进位只作为下一状态选择条件。

专题：纹波计数器与同步计数器对照

上一段提醒过：级联计数器时若把低位 Q 经过随意组合逻辑去驱动高位时钟，同步结构就退化成了脉动计数器。脉动计数器也叫**纹波计数器** (ripple counter)，它和同步计数器完成同一个功能，但时钟的组织方式完全不同，值得放在一起对照。

纹波计数器的做法极为节省：每个触发器接成翻转模式（把 D 触发器的 $\overline{Q}$ 接回 D），第一级由外部时钟驱动，之后每一级的时钟直接取自前一级的 Q 输出。第 0 位在时钟沿之后先翻转；第 1 位看到第 0 位变化后才跟着翻转；如此逐级传递，第 n 位要等前面各级依次翻完才轮到本级，延迟随位号一级一个 t_{CQ} 累加。变化像波浪一样从低位传到高位，这正是“纹波”二字的来历。

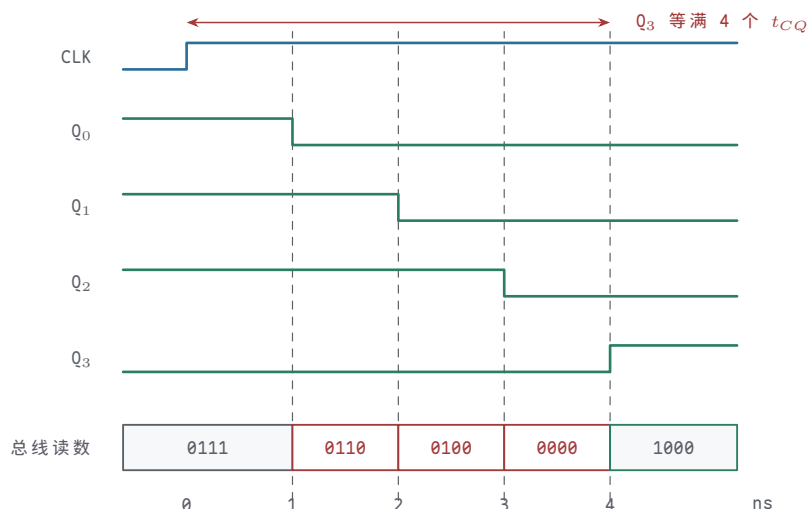
这个逐级延迟不是理论细节。以 4-bit 纹波计数器从 0111 计到 1000 为例，设每级 $t_{CQ} = 1\text{ns}$ ：

时钟沿后时刻	已完成的动作	总线上读到的值
--------	--------	---------

0ns	外部时钟沿到达，各级 尚未更新	0111（旧值 7）
1ns	第 0 位翻转完成	0110（读数 6，中间值）
2ns	第 1 位跟随翻转	0100（读数 4，中间值）
3ns	第 2 位跟随翻转	0000（读数 0，中间值）
4ns	第 3 位最后翻转	1000（读数 8，最终值）

在最终值稳定之前，总线上依次出现了 6、4、0 三个并不存在于计数序列中的中间值。若后级只是低速显示驱动，这几纳秒毫无影响；但若后级是译码逻辑——例如“计数值等于 8 时打开某写使能”——中间值 0 就可能让另一路译码输出意外有效，制造窄毛刺。纹波计数器的风险不在计数错误，而在读取窗口内的值不可信。

把表中的过程按纳秒画成波形，三个中间值各停留 1ns 的读取窗口就一目了然：



图示：4-bit 纹波计数器从 0111 计到 1000（每级 $t_{CQ} = 1\text{ns}$ ）：CLK 沿后 1ns Q_0 先翻成 0，它的变化再触发 Q_1 ，逐级传递到 Q_3 。已是沿后 4ns。底部总线行依次读出 0111、0110、0100、0000、1000——红色三格是计数序列中不存在的中值，后级译码逻辑在这个窗口读数就会看到幻影。同步计数器所有位共享时钟，沿后一步整体更新，没有这些格子。

同步计数器走另一条路：所有触发器共享同一个时钟，每一位的下一值由组合逻辑根据当前各位统一算出（ $\text{next} = \text{current} + 1$ ），在同一个时钟边界整体提交。前文 4-bit 计数器波形里“整体提交、不是一位一位爬”说的就是这件事。代价是每个触发器前面多了下一状态逻辑，位数越宽进位链越长；收益是时钟沿之后只有一级 t_{CQ} 加组合路径的延迟，各位同时稳定，任何时刻读到的都是真实计数值。

速度差异可以用数字算出来。设 $t_{CQ} = 1\text{ns}$ ：纹波结构 8 位时最高位要 $8 \times 1 = 8\text{ns}$ 才稳定，时钟周期必须大于 8ns，上限约 125MHz，还未计任何译码余量；同步结构同样 8 位，若下一状态组合逻辑最大延迟 2ns、目标建立时间 0.4ns，周期下限约为 $1 + 2 + 0.4 = 3.4\text{ns}$ ，5ns 周期（200MHz）仍有富余。位数越多，纹波结构的级数惩罚线性增长，同步结构的组合延迟只缓慢增加。

结构	时钟接法	稳定延迟	读数行为	适用场合
纹波计数器	每级时钟取自前一级 Q	低位 1 个 t_{CQ} 起，逐级累加	稳定前出现短暂中间值	低速分频链、定时链、读数不被同域逻辑消费的场景
同步计数器	所有触发器共享时钟	一级 t_{CQ} 加组合路径	边界后各位同时稳定	计数值要被同域译码、比较、控制的场合

选择原则因此很直接：只把计数器当分频器用、中间值无人读取，纹波结构简单省电；计数值要被同一时钟域的逻辑读取、译码或比较，就必须用同步计数器。这

解释了为什么 74HC161/163 这类通用计数芯片全部做成同步结构，而纹波结构更多出现在专用分频链和异步预分频器里。

专题：任意模数计数器—复位法与预置法

4-bit 计数器自然按模 16 循环，但十进制显示、BCD 运算和秒分计时需要的是模 10。把模 16 改成任意模 N ，工程上有两种标准做法，差别在“如何让第 N 个状态消失”。

第一种是复位法：让计数器照常加一，另用一个译码门监视计数值，一旦检出 N 就立刻异步清零，不等下一个时钟沿。以模 10 为例，译码门检出 1010（十进制 10），清零端把各级直接拉回 0000：

时钟沿序号	沿后稳定值	说明
0	0000	初始值
1	0001	正常加一
...	...	依次递增
9	1001	最后一个合法值（十进制 9）
10	0000	加一结果 1010 短暂出现，译码门检出后立即异步清零

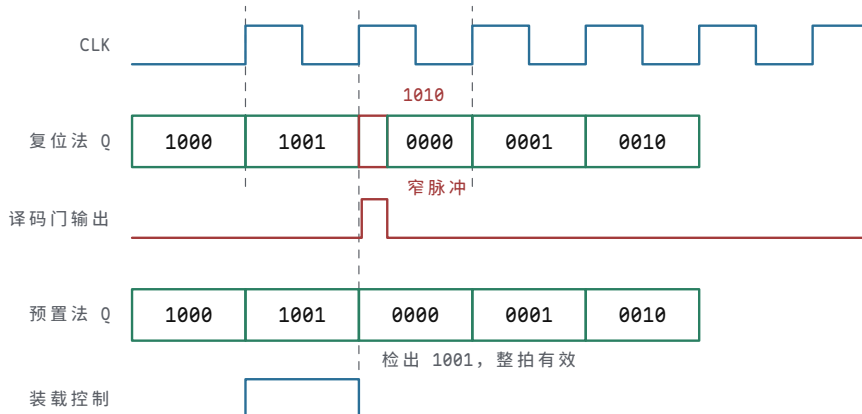
注意第 10 拍：1010 并非完全不出现，它作为瞬态存在几纳秒——正是这段瞬态触发了清零。这带来复位法的固有弱点：清零脉冲的宽度只有“译码门延迟加清零响应”那么长，若各级触发器清零速度不齐，最快的一级先清掉会使译码条件消失、清零脉冲提前撤除，最慢的一级可能还没清完，计数器从一个错误值重新起步。复位法的优点是只要一个译码门，缺点是把可靠性押在各级清零延迟一致上。

第二种是预置法：不消灭第 N 个状态，而是根本不让它出现。译码门检出的是最后一个合法值 $N-1$ （模 10 时为 1001），检出后拉起同步装载控制，下一个时钟沿把初值 0000 装入，覆盖掉本来的加一结果：

时钟沿序号	沿后稳定值	说明
0	0000	初始值
1	0001	正常加一
...	...	依次递增
9	1001	译码门检出末态，本拍内拉起装载控制
10	0000	时钟沿把初值装入，1010 从头到尾不出现

预置法的每一次变化都发生在时钟边界上，没有任何瞬态；装载控制有整整一个周期的时间稳定，只需像普通同步输入一样满足建立时间。代价是每级前面多一级装载选择逻辑，且译码门在整个 1001 周期内都必须稳定有效，不能带毛刺。

把两种做法在回卷附近的几拍画在同一时间轴上，瞬态有无一眼可见：



图示：复位法（上）：第 10 个时钟沿后，加一结果 1010 作为瞬态出现几纳秒（红色窄格），译码门检出后输出一条窄清零脉冲把各级拉回 0000——脉冲宽度只有“译码门延迟加清零响应”那么长，各级清零速度不齐时可能清不干净。预置法（下）：译码门在 1001 的整个周期内拉起装载控制，下一个时钟沿把 0000 装入，1010 从头到尾不出现，序列里没有任何瞬态。

做法	译码的状态	动作方式	有无瞬态	关键问题
复位法	检出 N (如 1010)	异步清零，不等时钟沿	有， N 短暂出现	清零脉冲仅数纳秒，各级清零不齐时可能清不干净
预置法	检出 $N-1$ (如 1001)	同步装载，等下一时钟沿	无	装载控制须整拍稳定并满足建立时间，多用一级选择逻辑

两种做法在芯片族里都有对应型号：74HC160/161 用异步清零，74HC162/163 用同步清零。同一族芯片同时提供两种清零方式，正说明这个差别是真实的设计选择，而不是教科书里的文字游戏。BCD 计数器（模 10）只是最常用的一例；秒针的模 60、地址回卷的模 2^k 分区，用的都是同一对方法。

专题：格雷码、环形与约翰逊计数器

二进制顺序不是唯一的计数方式。改变“下一状态是什么”的规则，可以得到三类各有专长的计数器，它们的取舍恰好说明：状态编码本身就是设计。

格雷码计数器让相邻状态只差一位。3-bit 格雷码的顺序是 000、001、011、010、110、111、101、100，随后回到 000。普通二进制从 0111 计到 1000 时四位同时变化，异步采样器在变化途中可能读到任意混合值；格雷码每步只变一位，采样器最坏读到旧值或新值，绝不可能读出第三种组合。本章前面讲两级同步器时提过“多 bit 跨域要用格雷码计数器”，原因就在这里：异步 FIFO 的读写指针用格雷码编码，指针跨域采样时的多值风险被结构性消除。代价是下一状态逻辑比二进制加一复杂，且状态顺序不再是数值顺序，不能直接当地址用。

环形计数器把移位寄存器的最后一位接回串行输入，形成一个环。4 级环以种子值 0001 起步，依次经过 0001、0010、0100、1000，再回到 0001。 N 个触发器只得到 N 个状态，状态利用率很低，但换来一个独特性质：每个触发器的输出本身就是状态标志，译码连一个门都不用——一想知道“是否处于第 2 态”，看第 2 根输出线即可。独热编码的状态机正是这个结构的推广：把带种子值的环形计数器当作状态寄存器，一个触发器对应一个状态；反过来看，环形计数器可以看成转移固定的独热状态机特例。后文状态编码专题里说的“one-hot 译码直接”，说的就是这件事。

约翰逊计数器（扭环计数器）只改一处：接回串行输入的不是最后一位本身，而是它的反相值。4 级约翰逊从 0000 起步，依次经过 0000、0001、0011、0111、1111、1110、1100、1000，再回到 0000—— N 个触发器得到 $2N$ 个状态，比环形多一倍。更妙的是译码：每一步仍只有一位变化，且每个状态都可以由唯一一对相邻位（含首尾相接的一对）识别，译码一个状态只需一个两输入门。例如状态 0000 由“ $Q_3 = 0$ 且 $Q_0 = 0$ ”唯一确定，0111 由“ $Q_3 = 0$ 且 $Q_2 = 1$ ”确定。译码门少、每步只变一位，使约翰逊计数器的译码输出干净无毛刺，常用于多相时钟生成、步进电机相序和简单分频。

结构	4 级状态数	每步变化位数	译码代价	典型用途
二进制	16	最多 4 位同时变	全译码，变化途中可能出中间值	通用计数、地址生成
格雷码	16	恒为 1 位	译码无中间态	跨时钟域指针、角度与位置编码
环形	4	2 位（一位退、一位进）	无需译码门， Q 直接当状态标志	独热状态机、简单轮转控制

约翰逊 8 恒为 1 位 每态一个两输入 多相时钟、步进电机相
门 序、分频

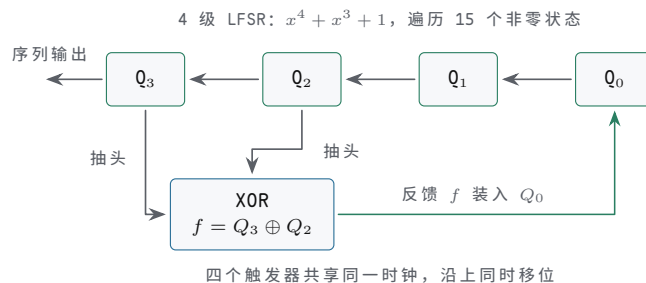
三类结构共用同一个提醒：未在序列里的编码必须有恢复路径。环形计数器若因上电扰动出现两个 1，就会在错误序列里一直循环；可靠设计要在上电时置入种子值，或加自检逻辑把非法编码拉回主循环——这和状态机“未使用编码要有默认去向”是同一条约定。

专题：LFSR 伪随机计数器

把移位寄存器的某几位抽出来做异或，把异或结果接回串行输入，就得到线性反馈移位寄存器（LFSR）。抽头位置用多项式记号表示： $x^4 + x^3 + 1$ 表示 4 级结构中取第 4 位和第 3 位（即 Q_3 、 Q_2 ）做异或反馈。设状态写作 $Q_3Q_2Q_1Q_0$ ，每个时钟沿各位左移一格，反馈值 $f = Q_3 \oplus Q_2$ 装入 Q_0 ：

```
f = Q3 XOR Q2
on clock edge:
  Q3 = Q2; Q2 = Q1; Q1 = Q0; Q0 = f
```

画成结构图，数据在四个触发器之间逐拍左移，唯一的组合逻辑是那只异或门：



图示：4 级 LFSR ($x^4 + x^3 + 1$)：每个时钟沿 $Q_0 \rightarrow Q_1 \rightarrow Q_2 \rightarrow Q_3$ 同时移位，抽头 Q_3 、 Q_2 经唯一一只异或门算出反馈 f 装入 Q_0 。这组抽头对应本原多项式，从非零种子出发遍历全部 15 个非零状态后精确重复；全零是死状态，上电必须置入非零种子。

这组抽头选得恰当（ $x^4 + x^3 + 1$ 是 4 级的本原多项式之一），LFSR 会遍历全部 $2^4 - 1 = 15$ 个非零状态才重复。从种子 0001 起步，前 8 拍为：

拍	状态 $Q_3Q_2Q_1Q_0$	反馈 $f = Q_3 \oplus Q_2$	说明
0	0001	0	种子值，必须非零
1	0010	0	
2	0100	1	
3	1001	1	
4	0011	0	
5	0110	1	
6	1101	0	
7	1010	1	之后继续遍历其余 7 态，第 15 拍回到 0001

把 15 个状态写成十进制是 1、2、4、9、3、6、13、10、5、11、7、15、14、12、8，随后精确重复。这个序列“伪”在完全确定、可由种子重现，“随机”在局部统计性质接近真随机序列：任一输出位上 0 和 1 的出现次数只差一次，游程分布也与随机序列一致，因此得名伪随机。

LFSR 的价值在于用极少的逻辑产生长序列。与二进制计数器相比，它没有进位链：每个触发器的下一状态只是一根移位线，唯一的一级组合逻辑是那个异或门，因此同样位宽下更快、更省。三类经典用途：一是伪随机测试，芯片内建自测（BIST）用 LFSR 给被测电路灌激励，再压缩响应做签名分析；二是定时器和模数计数，凡是只问“数满多少个周期”、不问数值顺序的场合，LFSR 都比二进制计数器划算；三是

扰码，通信链路把数据流与 LFSR 序列异或，打散长连 0 和长连 1，让频谱和时钟恢复更友好。

必须记住两个约束。第一，全零是死状态： $f = 0 \oplus 0 = 0$ ，一旦进入 0000 就永远停在那里，所以上电必须置入非零种子，或加全零检测逻辑把序列拉回。第二，状态顺序不是数值顺序，需要“第几号状态”的场合要查表，不能直接当地址计数器用。另外，LFSR 的内核正是一个移位寄存器—接下来就来正式看移位结构本身。

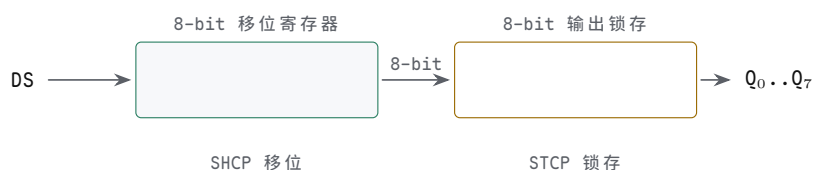
移位寄存器把一组 bit 向左或向右移动。它可以用于串行输入、串行输出、简单延迟线和位级变换。

```
next[3] = current[2]
next[2] = current[1]
next[1] = current[0]
next[0] = serial_in
```

假设 current 为 1010，serial_in 为 1。下一个时钟沿后，寄存器变成 0101。注意，四个位不是按顺序一个一个搬。组合逻辑同时准备好四个 next 输入，时钟沿到来时一起写入。该例说明状态更新不一定是加法，也可以是重新排列、选择、清零、保持或装载外部输入。

移位结构在整机里非常常见。串行外设把一个引脚上的 bit 流变成内部并行字节，显示驱动把内部字节逐位送到外部，流水线把阶段之间的状态向前推进，调试链把片内状态一位一位移出。74HC595 的移位寄存器与输出锁存器分开，正好说明两级状态边界：移位时钟改变内部移位状态，锁存时钟才改变外部可见输出。这个结构避免了外部 LED 或后级芯片在移位过程中看到中间状态乱跳。

74HC595 边界	发生的状态动作	读者应观察到的现象
SHCP 边沿	串行 bit 进入内部移位寄存器	外部输出可保持不变
连续 8 次移位	内部移位状态逐步形成一个字	中间状态不应直接驱动负载语义
STCP 边沿	内部字节提交到输出锁存器	Q0 到 Q7 一起更新
OE_n/MR_n	控制输出驱动和清零默认状态	低有效脚必须处于确定电平



图示：74HC595：移位寄存器与输出锁存器分开。串行数据经 DS 在 SHCP 边沿逐位移入，外部输出全程不动；STCP 边沿才把整字节一次性提交到 Q0..Q7。

这一类器件提前引出后续流水线寄存器的思想。流水线寄存器保存的不是单个数，而是一组需要一起前进的状态：数据、控制位、valid 位、异常标志和目的寄存器编号。若只有数据向前移动而控制位没有同步移动，后级就会把一个周期的数据解释成另一个周期的操作。移位寄存器是最简单的“状态随边界推进”模型；CPU 流水线是同一思想在更宽状态集合上的应用。

专题：串并转换与动态扫描显示

74HC595 的两级边界前面已经画过。把它真正用到板子上，还要把每个引脚的职责和一种典型负载—多位数码管—连起来看，才能理解为什么这颗芯片几乎成了“输出扩展”的代名词。

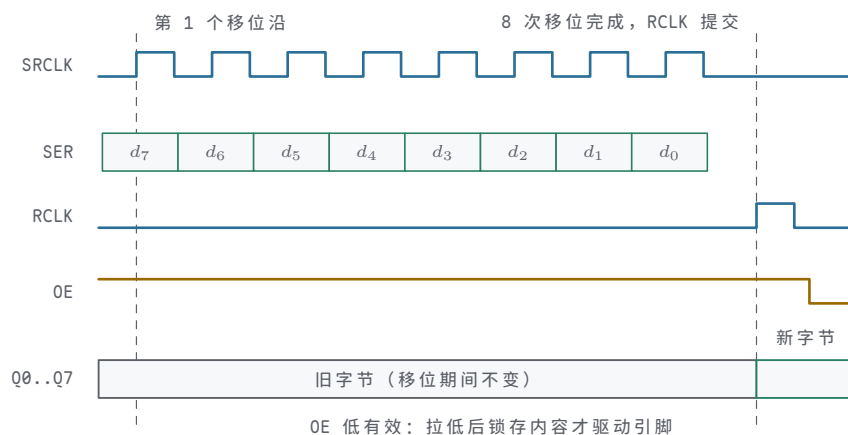
先看引脚职责。SER（部分手册记作 DS）是串行数据输入，给出当前要移入的那一位；SRCLK（SHCP）是移位时钟，每个有效沿把 SER 的位移入内部移位寄存器，内部状态前进一格，外部引脚纹丝不动；RCLK（STCP）是锁存时钟，有效沿把整个移

位寄存器的内容一次性抄进输出锁存器，8 个输出脚同时更新；OE（低有效）是输出使能，决定锁存器内容是否真正驱动引脚，无效时输出呈高阻。此外还有 SRCLR（MR，低有效）异步清空移位寄存器，以及 Q7' 级联输出，把移出的位交给下一片的 SER。

引脚	职责	关键问题
SER (DS)	串行数据输入，给出待移入的一位	该位必须在 SRCLK 有效沿前稳定
SRCLK (SHCP)	移位时钟，驱动内部移位寄存器	移位期间外部输出不变，这正是设计目的
RCLK (STCP)	锁存时钟，把整字节提交到输出	8 次移位完成后才给，过早会输出半成品字节
OE (低有效)	输出使能，无效时输出高阻	上电期间保持无效，避免显示乱闪
Q7'	最高位移出，级联下一片 SER	多片串联时移位次数按总位数计

两级寄存器为什么输出不抖，值得再说透一层。移位过程必然要经历 8 个中间状态，若这些中间状态直接驱动 LED 或数码管，用户会看到一片乱闪。595 把“内部状态推进”和“外部可见提交”拆到两个时钟沿上：SRCLK 管内部怎么动，RCLK 管外部什么时候看。外部负载在任何时刻看到的要么是完整的旧字节，要么是完整的新字节，绝不会看到移了一半的混合字节——这就是“寄存器是原子提交”在接口芯片上的一次应用。

把 8 次移位加一次锁存画到同一条时间轴上，两个时钟各管一段的边界就清楚了：



图示：74HC595 串并转换时序（括号内为部分手册别名：SER=DS、SRCLK=SHCP、RCLK=STCP）：SRCLK 的 8 个有效沿把 SER 上的 d_7 到 d_0 逐位移入内部移位寄存器，期间输出锁存器保持旧字节，外部引脚纹丝不动；8 次移位完成后给一个 RCLK 沿，整字节一次性提交到 Q0..Q7；OE 低有效，拉低之前引脚呈高阻。内部状态推进与外部可见提交分开，正是这颗芯片输出不抖的原因。

最典型的负载是多位数码管。N 位数码管通常段线并联、位线分开：所有位的 a 段接同一根线，每位的公共端各有位选。要显示不同的数字，只能分时复用：每一小段时间只选通一位，同时段码线送出这一位的图案；下一小段时间选通下一位，段码同步换成下一位的图案。只要整帧刷新率超过约 50Hz（工程上常取 100Hz 以上），人眼视觉暂留会把轮流点亮看成同时常亮。

数字算一遍：4 位数码管，每位点亮 2ms，一帧 8ms，帧率 125Hz，每位每秒被刷新 125 次，远在闪烁阈值之上。代价是每位占空比只有 1/4，亮度随之下降，需要在器件允许的脉冲电流范围内提高段电流来补偿。扫描顺序上有一条经验：先消隐、再换码、后点亮——若先开下一位再改段码，上一位的段码会短暂串到新位上，形成鬼影。用 595 驱动段码时这条天然成立：新段码在移位期间被锁存器挡在门内，移完后一拍 RCLK 原子切换，剩下的只是位选时序。

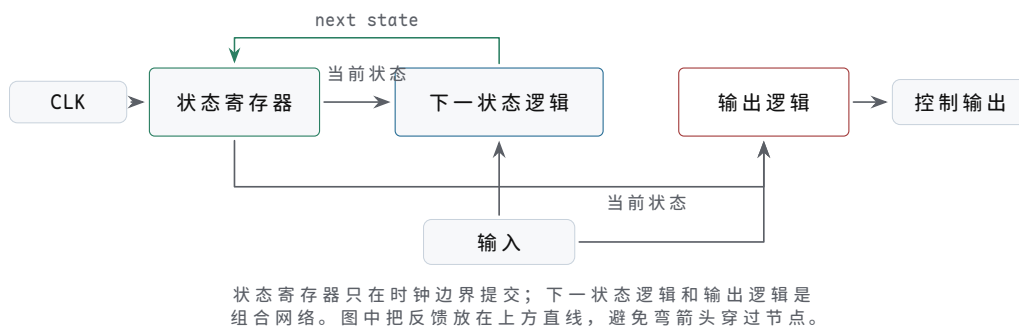
扫描步骤	动作	检查点
消隐	关闭当前位选（或令 OE 无效）	位选与段码不能同时切换
送段码	段码线（或 595）给出下一位图案	段码必须在位选打开前稳定
选通保持	打开下一位位选 维持点亮约 1 至 2ms	同一时刻只有一位被选通 整帧周期小于 20ms，即帧率大于 50Hz
循环	推进到下一位，周而复始	扫描由定时器驱动，不能被长任务卡住

串并转换和动态扫描合起来，说明同一个原则：外部接口的引脚永远比内部状态少，解决办法是把“哪些位先走、哪些位后走”变成明确的时间约定，并用寄存器边界保证每一拍对外都是完整的。

把本节几种结构放在一起看，各自的设计要点不同：普通寄存器保存 ALU 结果、地址和控制字，要写使能只在目标周期打开；计数器用于 PC 加一、循环计数和超时计数，要使能、清零、装载优先级明确；移位寄存器用于串并转换、延迟线和输出扩展，要移位边界和输出锁存边界分开；标志寄存器保存 zero/carry/error 等条件，要说明哪些指令或事件会覆盖标志。

五、状态机把硬件动作切成阶段

有限状态机由三部分组成：当前状态、输入条件、下一状态逻辑。当前状态保存在寄存器里，组合逻辑根据当前状态和输入条件决定下一状态。



图示：有限状态机结构：状态寄存器 + 下一状态逻辑 + 输出逻辑。

设计状态机时，第一步不是写状态名字，而是列出机器必须区分的阶段。若一个控制器要等待请求、装载数据、运行运算、等待外设完成、提交结果，那么这些阶段是否能合并，取决于它们是否需要不同的控制输出和不同的等待条件。第二步才是给阶段编码，并写出每个状态在每种关键输入下的下一状态。第三步是列出输出：哪些输出只由状态决定，哪些输出还依赖输入，哪些输出必须寄存后再送往外部。

一个完整状态机说明至少包含五张表。第一张是状态含义表，说明每个状态代表哪个硬件阶段——状态命名漂亮但动作不明确，等于没有状态。第二张是转移表，说明当前状态遇到关键输入后进入哪里，等待、错误和默认分支都不能遗漏。第三张是输出表，说明每个状态打开哪些控制线，写使能、片选、ALU 选择、驱动使能不能在错误周期打开。第四张是复位和非法状态表，说明上电、复位释放和异常编码如何处理，入口状态、默认安全输出和恢复路径缺一不可。第五张是波形检查表，说明哪些信号必须在同一周期出现——最终结果正确但中间周期错误，同样是失败。缺少任何一张，状态机都容易退化成“名字看起来对，但边界不清楚”的草率描述。

以一个三阶段硬件控制器为例：

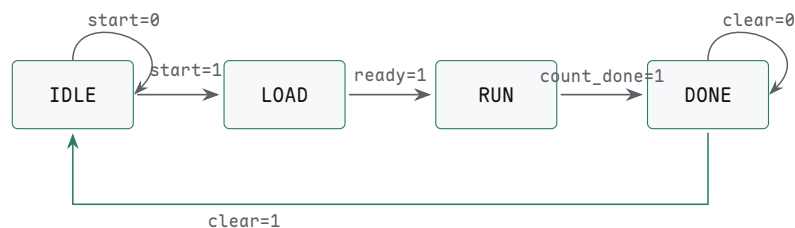
当前状态	条件	下一状态
------	----	------

IDLE	start=1	LOAD
LOAD	ready=1	RUN
RUN	count_done=1	DONE
DONE	clear=1	IDLE

若当前状态是 IDLE 且 start=0，状态保持 IDLE；若 start=1，下一状态逻辑准备 LOAD，等时钟沿到来后状态寄存器才真正变成 LOAD。状态机因此不会在一个周期里连续越过 IDLE、LOAD、RUN、DONE，它每次只跨过一个明确边界。

上表还不够完整，因为它只写了“去哪里”，没有写“做什么”。若 LOAD 状态要把外部数据装入寄存器，就必须说明 load_reg_we 在哪个周期为 1；若 RUN 状态要让计数器递减，就必须说明计数使能、ALU 选择和完成条件；若 DONE 状态要向外部报告结果，就必须说明输出是否寄存、ready 信号保持多久、clear 到来前是否允许覆盖结果。状态机设计的核心不是列出状态名称，而是让状态名称、控制输出和被控制的状态元件逐周期对应。

状态	本周动作	关键输出	离开条件
IDLE	等待同步后的启动请求	关闭写使能与外部驱动	start_pending=1
LOAD	装载输入寄存器或初始计数	load_we=1	输入 ready 或装载完成
RUN	执行运算、计数或移位	run_en=1	count_done=1
DONE	保持结果并等待清除	done=1，输出稳定	clear=1



主路径水平推进，返回路径独占下方通道，自环标签放在节点上方，避免和返回线相交。

图示：四状态硬件控制器状态图：IDLE 到 LOAD 到 RUN 到 DONE 再回到 IDLE。每个保持条件画成自环——状态机不是只会前进，更多时候是在等条件。

把这个状态机真正做完一次。第二步是编码：四个状态用两个 bit，IDLE=00、LOAD=01、RUN=10、DONE=11，编码刚好用完，没有非法空洞。第三步写下一状态逻辑和输出逻辑：

```

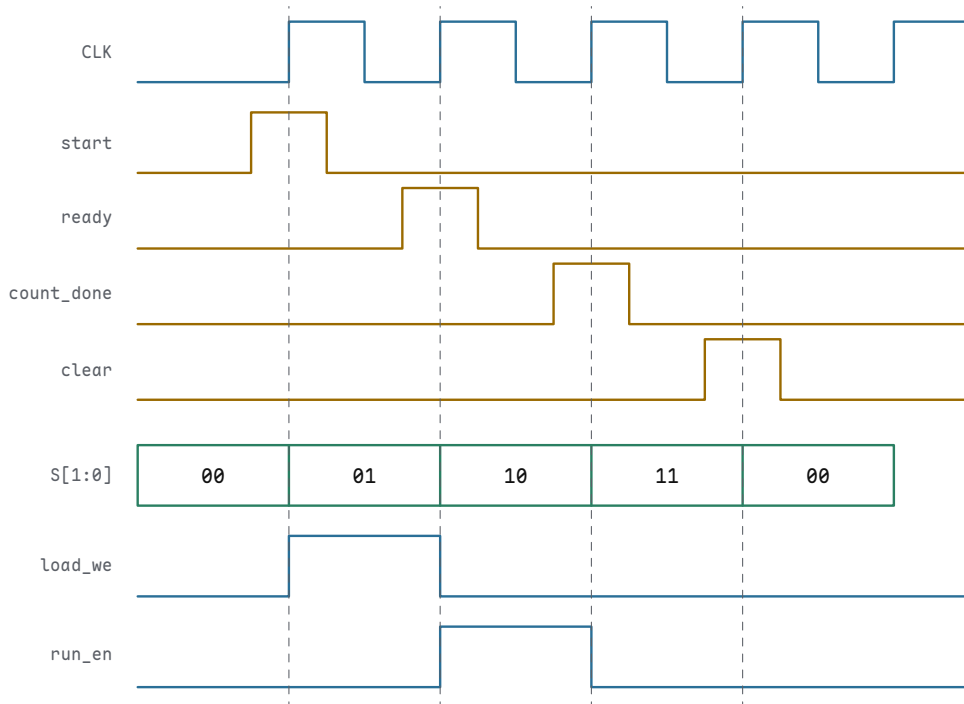
case (S):
  IDLE: next = start      ? LOAD : IDLE
  LOAD: next = ready      ? RUN  : LOAD
  RUN:  next = count_done ? DONE : RUN
  DONE: next = clear      ? IDLE : DONE
  default: next = IDLE

load_we = (S == LOAD)
run_en  = (S == RUN)
done    = (S == DONE)
  
```

写成布尔式也能机械推导。以 next_LOAD 为例：进入 LOAD 的条件是“当前 IDLE 且 start=1”，留在 LOAD 的条件是“当前 LOAD 且 ready=0”。

```
next_LOAD = (S == IDLE AND start) OR (S == LOAD AND NOT ready)
```


第四张表是复位：上电和复位期间状态寄存器强制为 `IDLE=00`，`load_we/run_en/done` 全为 0。第五张表是波形检查：把一次完整运行按时钟对齐画出来，检查每个控制线是否只在规定周期有效。



图示：一次完整运行的对齐波形：输入条件在采样沿前后保持稳定，状态只在上升沿跳转；`load_we` 只在 `LOAD` 的那一个周期为高，`run_en` 也只在 `RUN` 的那一个周期为高。检查一台状态机，就是对照这张图逐周期核对：控制线有没有早一拍、晚一拍或粘在错误的周期。

状态机输出可以只由当前状态决定，也可以同时依赖输入。前者输出更稳定，后者反应更快但更容易受输入毛刺影响。基本原则是：状态机不是若干状态名称的集合，而是“状态寄存器 + 下一状态组合逻辑 + 输出组合逻辑”的硬件结构。

Moore 型输出只由当前状态决定，常用于写使能、片选、锁存时钟、功率使能等需要稳定边界的控制线。Mealy 型输出同时依赖状态和输入，响应可能少等一个周期，但输入毛刺也可能被带到控制线。工程上并不是简单规定只能用哪一种，而是按后果分类：危险输出、外部驱动和写存储器信号倾向于寄存或由稳定状态产生；纯内部、已同步且后级还会采样的条件信号，可以在确认时序余量后使用组合依赖。

专题：Moore 与 Mealy 输出对照

上一段把输出分成“只由状态决定”和“同时依赖输入”两类，并给出了按后果分类的原则。这里用同一个功能把两类做实：检测已同步输入 `sig` 的上升沿，发现后输出一个脉冲 `pulse`。

Moore 型实现需要三个状态。`S_LOW` 表示上一拍 `sig` 为 0，`S_EDGE` 表示刚采到上升沿（`pulse` 只在此状态为 1），`S_HIGH` 表示 `sig` 已稳定为 1、等待回落：

当前状态	含义	pulse 输出	sig=1 去向	sig=0 去向
<code>S_LOW</code>	上一拍 <code>sig</code> 为 0	0	<code>S_EDGE</code>	<code>S_LOW</code>
<code>S_EDGE</code>	刚采到上升沿	1	<code>S_HIGH</code>	<code>S_LOW</code>
<code>S_HIGH</code>	<code>sig</code> 已稳定为 1	0	<code>S_HIGH</code>	<code>S_LOW</code>

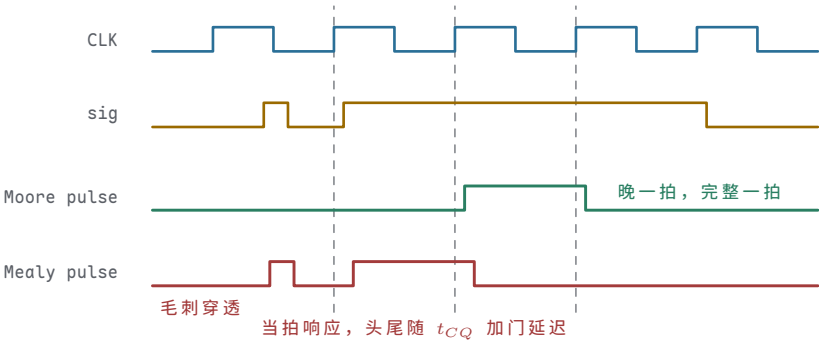
Mealy 型实现只用两个状态，把“是否刚采到边沿”交给输入参与判断：`pulse` 在“当前状态为 `S_LOW` 且 `sig=1`”的那个周期直接为 1。

当前状态	sig	pulse (本拍)	下一状态
<code>S_LOW</code>	0	0	<code>S_LOW</code>
<code>S_LOW</code>	1	1	<code>S_HIGH</code>

S_HIGH	0	0	S_LOW
S_HIGH	1	0	S_HIGH

对照两种实现的时序。Moore 版在采到边沿的下一拍才进入 S_EDGE，脉冲晚一个周期出现，但宽度恰为一个完整时钟周期，且输出只是状态译码，触发器输出之后最多一级门，干净稳定。Mealy 版的 `pulse = (state == S_LOW) AND sig`，在采到边沿的同一个周期内就升高，抢回了一个周期；但这条输出路径上有输入 sig 的组合分量：若 sig 未经同步、带有毛刺，毛刺会原样穿到 pulse 上；即使 sig 本身已是寄存器输出，pulse 也要等沿后 t_{CQ} 加一级门延迟才稳定，脉冲的头尾不像 Moore 版那样整齐。

把同一个 sig 下的两路输出画在一起，当拍与晚一拍、毛刺与干净的差别直接可见：



图示：同一 sig 下的两种输出：Mealy 在采到上升沿的当拍拉高，抢回一个周期，但 sig 上的窄毛刺原样穿透（左侧红色小脉冲），且脉冲头尾要等沿后 t_{CQ} 加一级门延迟，不如 Moore 整齐；Moore 晚一个周期进入 S_EDGE，输出恰为整一拍宽的状态译码，毛刺无从穿透。驱动外部器件与写使能选 Moore 或再寄存一拍，同域内部条件信号可用 Mealy。

维度	Moore	Mealy
本例状态数	3	2
输出决定因素	仅当前状态	当前状态加输入
响应时刻	采到条件后下一拍	采到条件的当拍
输出稳定度	整拍稳定，沿后至多一级门	受输入路径影响，沿后 t_{CQ} 加组合延迟
毛刺风险	输出无输入直接路径	输入毛刺可穿透到输出
适用输出	写使能、片选、外部驱动	同域内部、后级会再采样的条件信号

选择原则于是回到前文的按后果分类。输出要驱动外部器件、写存储器、控制功率，选 Moore，或者在 Mealy 输出后再加一级寄存器——后者把 Mealy 的早判断和 Moore 的干净边界拼在一起，代价是又回到晚一拍。输出只在同一时钟域内部被后级再次采样、且功能上必须抢那一拍，Mealy 可以放心用，前提是输入已经同步、组合路径有时序余量。状态数上 Mealy 常常更省，因为它把一部分状态信息转交给了输入条件；但省下的触发器换来的是输出上的组合路径，这笔交换必须用上面这张对照表逐项想清楚。

一个状态表必须覆盖默认情况。若状态是 IDLE、LOAD、RUN、DONE，而编码使用两个 bit，则四个编码刚好用完；若状态更多或编码留有空洞，未使用编码不能留给综合器或门延迟“自己决定”。安全写法是给下一状态逻辑设置默认恢复路径，例如进入 FAULT、RESET_WAIT 或 IDLE，同时让写使能、外部驱动、片选和功率使能默认为关闭。这样，即使上电扰动、复位释放问题或外部干扰让状态寄存器进入非法编码，电路也不会非法状态下驱动危险动作。

把状态机连接到后续 CPU 时，可以把每条控制线都问成一个状态问题：PC 什么时候写入，指令寄存器什么时候装载，寄存器堆写使能在哪个周期打开，ALU 选择

哪种运算，存储器读写请求何时有效，外设未 ready 时机器停在哪个状态。若这些问题只用“连线会变成正确值”回答，就没有真正形成控制器；只有把它们逐一对应到状态、条件、下一状态和输出时序上，CPU 才能按周期推进。

以最小多周期 CPU 为例，状态机通常至少要区分取指、译码、执行、访存和写回。取指状态打开存储器读请求 `mem_read`，选择 `pc_to_addr`，并在存储器 ready 后装载指令寄存器 (`ir_we`)；译码状态读取寄存器堆并准备立即数，但不写回任何目的寄存器；执行状态选择 ALU 运算 `alu_op` 或计算地址，分支条件和 ALU 结果必须在采样前稳定；访存状态等待数据存储器 ready，期间保持地址和控制线不变；写回状态打开寄存器堆写使能 `rf_we`，且只在目标指令的目标寄存器上打开。若存储器没有 ready，状态机不能假设数据已经存在，而应保持在等待状态并关闭不该打开的写使能。这样，CPU 的每个周期都遵循本章模型：当前状态和寄存器输出经过组合控制，形成下一状态和下一批写入。

这种写法也能解释为什么现代芯片会引入更复杂的控制状态和流水阶段。它们不是为了把教材概念复杂化，而是为了缩短关键路径、隐藏等待、保持高吞吐，并让异常、中断、cache miss、分支错误等事件有可恢复边界。无论是 8086 的总线等待状态、80386 的系统控制状态，还是现代 CPU 的提交队列和异常入口，本质上都要回答同一个问题：某个动作在哪个状态被允许，结果在哪个边界对外可见，失败时回到哪个安全状态。

现在硬件已经能够保存一组 bit、按周期加一、按周期移动、按条件切换阶段。下一步要把这些能力用于更大的动作组织：哪些寄存器在这个周期写入，ALU 做哪种运算，哪一路数据被选择，下一阶段去哪里。这就是控制器的基础。

专题：时钟、复位、状态编码与验证闭环

前面已经把同步边界讲成寄存器到寄存器的路径，但真实工程中还必须处理四个经常被低估的问题：时钟如何送达，复位如何跨越边界，状态编码如何影响安全性，验证如何证明边界确实成立。这四件事不改变“组合逻辑准备下一值、状态元件保存下一值”的基本模型，却决定一个设计能否从纸面状态表进入可靠机器。

首先区分写使能和门控时钟。写使能的做法是让时钟继续到达触发器，但在时钟沿到来时决定是否接收新 D 值；若写使能为 0，寄存器保持原值。门控时钟则是让某段时钟停止翻转，以降低功耗或隔离无用活动。两者在功能描述上都可能表现为“保持不变”，但硬件风险完全不同。普通数据通路优先使用写使能；真正需要门控时钟时，应使用专用 clock gating 单元，并保证 enable 在安全窗口内被锁存。不能随意用一个 AND 门把 `clk` 和 `enable` 相与后送给触发器，因为 `enable` 的毛刺或变化时刻可能制造短脉冲、窄时钟或额外时钟沿。

把几种保持状态的写法排开看：写使能让时钟照常到达，触发器按使能选择写入或保持，使能信号必须满足目标触发器时序；数据 mux 保持用 `D=mux(we,new,Q)` 写入旧值实现保持，代价是 mux 延迟进入数据关键路径；专用时钟门控用门控单元生成局部时钟，要保证 `enable` 被锁存、无毛刺、满足时钟树约束；普通门电路直接切时钟最容易产生短脉冲和不可控偏斜，通常禁止使用。

其次，复位也有“域”。

一个时钟域内部可以规定复位异步拉入、同步释放；两个时钟域之间则不能假设复位释放同时发生。若控制状态机已经退出复位，而它依赖的外设状态机、同步器或计数器仍在复位中，就可能读到无效状态。复位域设计应回答三个问题：哪些状态元件属于同一复位域，哪些跨域信号在复位期间必须保持安全，复位释放后第一个有效事件如何产生。

按对象写复位策略：控制状态要复位到 `RESET_WAIT` 或 `IDLE`，上电后进入未定义状态等于没有控制器；数据寄存器只复位影响控制或外部可见安全的那些，大面积无意义复位只会增加布线和时序压力；跨域复位释放要在每个时钟域内部同步释放，并等待依赖域 ready，否则一个域先运行、另一个域仍无效；复位后的第一个事件要特别小心，同步器和 pending 位要进入不会伪触发的默认值，否则复位释放本身就会产生假启动或假中断。

第三，状态机编码不是纯粹的排版选择。二进制编码使用较少触发器，但下一状

态译码可能较复杂；one-hot 编码为每个状态使用一个触发器，译码直接、常适合较高速度或 FPGA 结构，但触发器数量更多；格雷编码让相邻状态尽量只改变一位，常用于跨域计数指针或减少多位同时变化带来的采样风险。无论使用哪种编码，都要定义未使用编码的下一状态和输出默认值。安全相关控制器还应考虑显式安全编码：为非法编码、错误记录和恢复路径预留设计，而不是指望编码本身永远不出错。

第四，验证必须看波形上的边界。只看最终寄存器值，可能错过“某个周期写使能提前打开”“复位释放产生伪事件”“同步器第一级短暂不稳定”“pending 位被错误清除”等问题。一个同步小系统的波形检查至少要同时观察 `clk`、`reset`、源寄存器 `Q`、目标寄存器 `D`、目标写使能、状态编码、同步器两级输出、事件 pending 位和 `ack` 信号。

每个对象要检查的问题不同：`reset` 的释放是否在本时钟域边界生效，否则状态机会半周期启动或产生伪事件；`Q -> D` 路径上 `D` 是否在采样沿前稳定，否则目标寄存器采到过渡值；`write_enable` 是否只在允许写入的周期有效，提前一拍就是寄存器被提前覆盖；同步器的第二级输出是否作为唯一后级输入，后级直接使用 `raw` 或 `meta` 信号等于没有同步；`pending/ack` 的事件是否保持到被消费，窄事件丢失、重复消费和错误清除都是真实失败；非法状态编码是否关闭危险输出并恢复，未使用状态不能驱动写使能或外部输出。

这些检查会直接连接到后续 CPU 章节。PC 是寄存器，指令寄存器是状态，流水线寄存器是阶段边界，控制器是状态机，异常和中断入口依赖同步事件，写回使能必须在正确周期打开。若第二章的边界概念不清楚，后面的数据通路和最小 CPU 就会退化成“线连起来就能跑”的误解。真正的 CPU 不是一张组合逻辑图，而是一组由时钟、复位、状态编码、控制使能和波形验证共同约束的同步系统。

经典案例：交通灯控制器的完整设计

前面的状态机例子只有四五个抽象状态，现在用一个带真实时间需求的案例，把“需求、状态列表、转移表、输出表”完整走一遍。设计一个十字路口的交通灯控制器：东西向绿灯亮 30 秒，转黄灯 5 秒，然后双向全红 2 秒；接着南北向绿灯 30 秒、黄灯 5 秒，再全红 2 秒，随后回到东西向绿灯，如此循环。全红段不是装饰：在一个方向放行之前，两个方向都必须先看到红灯，给已经进入路口的车辆留出清空时间。这 2 秒是用一个专门状态换来的安全余量，删掉它，状态机更简单，路口更危险。

设计的第一步是把时间从状态机里剥离出去。30、5、2 都是计数值，而状态机不应该自己数数。让一个独立的定时器——一个带装载值的递减计数器——负责计数：状态机进入每个状态时，把该阶段的秒数装载进去并启动定时器；定时器减到 0，发出一个周期宽的 `timer_done` 信号；状态机看到它就走向下一阶段，并装载下一阶段的秒数。分工之后，状态机只回答“现在在哪个阶段、下一阶段是什么”，定时器只回答“这个阶段结束了没有”。将来把 30 秒改成 40 秒，只动装载值，状态机结构一行不改。若系统时钟是 1Hz，定时器就是一个最大装载 30 的 5-bit 递减计数器；若系统时钟更快，先分频得到 1Hz 使能，定时器每秒钟才减一次。

状态列表按灯色阶段划分，共六个状态。注意两个全红状态不能合并：它们的灯色输出完全相同，但下一状态不同——一个之后放行南北向，一个之后放行东西向。状态划分由“未来行为是否相同”决定，而不是由“当前输出是否相同”决定；输出相同而去向不同的两个阶段，必须是两个状态。

状态	含义	装载值
<code>EW_GREEN</code>	东西向绿灯，南北向红灯	30s
<code>EW_YELLOW</code>	东西向黄灯，南北向红灯	5s
<code>ALL_RED_EW</code>	东西向刚结束，双向全红	2s
<code>NS_GREEN</code>	南北向绿灯，东西向红灯	30s
<code>NS_YELLOW</code>	南北向黄灯，东西向红灯	5s
<code>ALL_RED_NS</code>	南北向刚结束，双向全红	2s

转移表只有一类条件：`timer_done` 是否为 1。为 0 时保持原状态，下表只列出为 1 时的转移行。

当前状态	条件	下一状态	动作
EW_GREEN	<code>timer_done=1</code>	EW_YELLOW	装载 5s，重启定时器
EW_YELLOW	<code>timer_done=1</code>	ALL_RED_EW	装载 2s，重启定时器
ALL_RED_EW	<code>timer_done=1</code>	NS_GREEN	装载 30s，重启定时器
NS_GREEN	<code>timer_done=1</code>	NS_YELLOW	装载 5s，重启定时器
NS_YELLOW	<code>timer_done=1</code>	ALL_RED_NS	装载 2s，重启定时器
ALL_RED_NS	<code>timer_done=1</code>	EW_GREEN	装载 30s，重启定时器

输出是典型 Moore 型：灯色只由当前状态决定，与 `timer_done` 无关，因此在整个状态期间稳定，输入信号上的毛刺不可能让灯闪一下。

状态	东西向	南北向
EW_GREEN	绿	红
EW_YELLOW	黄	红
ALL_RED_EW	红	红
NS_GREEN	红	绿
NS_YELLOW	红	黄
ALL_RED_NS	红	红

还有两处工程细节值得写明。第一，复位状态应选全红状态而不是某个绿灯状态：上电后的第一件事是让所有方向看到红灯，而不是让某个方向侥幸先走。第二，一个完整循环是 74 秒，但状态机的硬件成本与秒数无关——计时全部由定时器承担。这个案例再次印证本章模型：定时器是“寄存器加递减组合逻辑的反馈”，状态机是“寄存器加下一状态逻辑和输出逻辑”，两者靠 `timer_done` 和装载、启动三条线连接，所有变化都发生在时钟边界上。

专题：状态编码的三种选择

交通灯控制器有六个状态，状态寄存器该用几个触发器、每个状态编成什么二进制值，并不是排版偏好。编码直接决定三件事：触发器用掉多少、下一状态和输出译码的组合逻辑有多深、状态翻转瞬间译码输出会不会出毛刺。三种经典选择是二进制编码、格雷编码和独热编码，下表给出交通灯六状态的三种编法。

状态	二进制 (3 位)	格雷 (3 位)	独热 (6 位)
EW_GREEN	000	000	000001
EW_YELLOW	001	001	000010
ALL_RED_EW	010	011	000100
NS_GREEN	011	010	001000
NS_YELLOW	100	110	010000
ALL_RED_NS	101	100	100000

二进制编码按状态顺序编号，用 $\lceil \log_2 6 \rceil = 3$ 个触发器，最省存储元件；代价是下一状态逻辑和输出译码都要看全部 3 位，逻辑级数最深。格雷编码让循环中相邻的两个状态只差一位：本例刻意选取 000、001、011、010、110、100 这条六状态环，连从未态 100 回到首态 000 的闭环边也只翻最高一位。独热编码为每个状态分配一个触发器，某一位为 1 就表示“正处于这个状态”，译码常常只需看一位，最浅最直接；代价是触发器从 3 个变成 6 个。

编码	触发器数	译码逻辑	适用场合
二进制	3	最深，需综合全部编码位	状态很多、触发器昂贵的 ASIC

格雷	3	深度与二进制相当	相邻转移无中间码，跨时钟域计数指针常用
独热	6	最浅，常常一级	FPGA 触发器丰富，常是默认选择

格雷码的价值要放在“多位同时翻转”的背景下看。状态从 011 变到 100 时三位都要翻，而物理上三位不可能精确同时到达，译码器在翻转窗口里可能瞬间看到 000 或 111；若这个中间码恰好对应某个有效输出，输出线上就冒出一条毛刺。格雷码每次转移只翻一位，不存在中间码，译码窗口天然干净——这也是异步 FIFO 的读写指针跨时钟域传递时一律用格雷码的原因。

独热编码在 FPGA 上流行有结构上的理由：FPGA 的每个逻辑单元都自带触发器，触发器几乎随用随有，而宽输入的译码逻辑要占用查找表层级、拉长关键路径。用 6 个“免费”的触发器换最浅的译码，往往反而更容易满足时序。ASIC 的情形相反，触发器占面积，状态一多就倾向二进制编码。无论选哪种，未使用编码都必须有去处：二进制和格雷各有 110、111 或 101、111 两个空洞，独热更有 $2^6 - 6 = 58$ 个非法编码（多位同时为 1 或全为 0），下一状态逻辑应把它们一律导向全红状态并关闭所有绿灯——非法状态恢复是编码设计的收尾，不是附加题。

经典案例：1101 序列检测器（允许重叠）

第二类经典状态机不控制外部设备，而是识别数据流：每个时钟沿采样 1 位输入 x ，当最近收到的 4 位恰好是 1101 时，输出 y 为 1。要求允许重叠：一次匹配的末尾几位如果同时是下一次匹配的开头，必须被复用，不能匹配一次就从头再来。

设计的关键是给每个状态一个明确含义：“到目前为止，输入串的尾巴与模式 1101 的前缀匹配了多长”。状态不需要记住整段历史，只需记住这个长度——未来能否完成匹配，只取决于它。

状态	含义	已匹配前缀
S0	没有可用前缀（起始或刚失配）	（空）
S1	尾巴是 1	1
S2	尾巴是 11	11
S3	尾巴是 110	110

采用 Mealy 型，输出标在转移上（写法为“下一状态/输出”）：

当前状态	$x=0$	$x=1$
S0	S0/0	S1/0
S1	S0/0	S2/0
S2	S3/0	S2/0
S3	S0/0	S1/1

逐行核对这张表，能看出构造方法是完全机械的。在 S2 收到 1：已匹配串变成 111，它不是 1101 的前缀，但其尾巴 11 仍是最长可用前缀，所以留在 S2，不退回起点。在 S3 收到 1：恰好完成 1101，输出 1；同时这一位 1 是下一个匹配的开头，所以退到 S1 而不是 S0。在 S3 收到 0：得到 1100，其尾巴 0、00、100、1100 没有一个是 1101 的前缀，只能回 S0。一般规则是：失配时把“已匹配前缀加上新输入”的尾巴从长到短逐个试，第一个仍是模式前缀的，就是下一状态。

```
input x : 1 1 0 1 1 0 1
state  : S0 S1 S2 S3 S1 S2 S3 S1
output y: 0 0 0 1 0 0 1
```

重叠检测正是靠“匹配后退回 S1 而非 S0”实现的。以输入 11011 为例：第 4 位到达时检测到第一个 1101，机器停在 S1；第 5 位的 1 随即把它推进 S2——第二

个匹配的前缀 11 已经备好。若输入继续 01，即完整序列 1101101，第二个 1101 在第 7 位被认出，它与第一个匹配共享了第 4 位的那个 1。倘若错误地让机器匹配后回 S0，这个共享位就被白白丢弃，1101101 中的第二个 1101 将被漏检。

最后说明为什么用 Mealy 型：检测输出依赖“状态加当前输入”，比 Moore 型少一个 DONE 状态，同拍给出结果，响应快一个周期。代价是 y 直接跟着 x 走， x 上的毛刺会原样出现在 y 上。若 y 只被后级在时钟沿采样，提前一拍是净收益；若 y 要驱动写使能这类危险控制线，应当再寄存一拍，用稳定性换掉这一拍的提前量。四个状态用 2 位二进制编码恰好用完，没有空洞编码。

六、从时序约束到检查方法

同步设计不能仅以“这个状态机功能正确”作为结论。功能正确说明下一状态逻辑在稳定输入下给出正确答案；时序正确说明这些答案能在规定边界前到达，并且不会过早冲入保持窗口。二者缺一不可。分析一条关键路径，应当把它写成四个对象：源状态元件，即哪个触发器在发射沿后改变 Q ，不能只写信号名而不写状态边界；数据路径，即结果经过哪些门、mux、加法器、译码和连线，不能只看逻辑级数而忽略布线和负载；目标状态元件，即哪个触发器在捕获沿采样 D ，不能没有采样窗口；时钟关系，即周期、偏斜、抖动和工程余量，不能用理想同相时钟代替真实时钟树。

建立约束和保持约束检查的是同一条路径的两个方向。建立约束关心最慢数据是否赶得上下一个采样沿；保持约束关心最快数据是否过早进入同一个采样沿后的保持窗口。工程上常见的错误，是只盯着最高频率而忽略保持时间。事实上，降低频率可以缓解建立问题，却不一定修复保持问题；保持问题往往需要改变最小延迟、时钟偏斜或布局。

可以用一个寄存器到寄存器路径作为分析模板。若源寄存器 R_{src} 的 $tCQ_{max}=0.8ns$ ，组合逻辑最大延迟为 $3.6ns$ ，连线最大延迟为 $0.7ns$ ，目标寄存器建立时间为 $0.4ns$ ，不利偏斜和余量合计 $0.5ns$ ，则周期至少为 $6.0ns$ 。若系统要求 $200MHz$ ，即周期 $5.0ns$ ，该路径就不满足建立约束。正确处理方式是把组合逻辑拆段、优化 mux 和加法路径、调整布局或降低频率，而不是把余量从时序预算中删除。

```
required period = 0.8ns + 3.6ns + 0.7ns + 0.4ns + 0.5ns
                = 6.0ns
target period   = 5.0ns
result          = setup violation
```

保持检查则使用最小延迟。若 $tCQ_{min}=0.08ns$ ，组合逻辑最小延迟为 $0.02ns$ ，连线最小延迟为 $0.03ns$ ，目标保持时间为 $0.20ns$ ，再叠加不利偏斜 $0.08ns$ ，则数据最早 $0.13ns$ 到达，而目标要求至少 $0.28ns$ 后才允许变化。这条路径存在保持违例。修复方法通常是在数据路径上增加受控延迟、调整寄存器相对位置、修改时钟树或让工具插入保持缓冲。它不是“频率太高”的问题。

现象	优先检查	不应直接假设
高频下偶发错误	建立约束、关键路径、温度和电压角落	逻辑表达式一定写错
任何频率都偶发错误	保持约束、异步输入、复位释放、亚稳态	降低频率一定有效
复位后偶发失控	复位释放同步、非法状态恢复、安全默认输出	只要复位电平正确就足够
外部按钮偶发多次触发	消抖、同步、脉冲宽度、状态机边界	按钮已经是数字信号

复位要求同样要写成明确的约定。首先列出哪些寄存器必须复位：控制状态、计数器、使能输出、错误状态、协议状态通常需要复位；纯数据缓冲如果写使能受控，未必每一位都需要复位。其次说明复位期间输出是否安全，例如电机使能、写存储

器使能、片选信号和外部驱动必须进入禁止态。最后说明复位如何释放：异步拉入可以快速进入安全状态，同步释放可以避免同一状态机内不同触发器在不同边界开始运行。

按对象核对默认值：控制状态机复位到 `IDLE` 或 `RESET_WAIT`，只能有一个合法入口状态；写使能复位为 0，复位期间不能写任何寄存器或存储器；外部驱动进入禁止输出或高阻安全态，不能误启动执行机构；错误状态被清除或进入诊断态，要能区分复位清除和真实故障；跨域同步器进入不会触发事件的状态，释放后不能产生伪脉冲。

跨时钟边界要按对象分类。单 bit 电平可以使用两级同步；窄脉冲应先转换为持续足够长的电平事件，或使用握手协议；多 bit 数据不能逐位同步后直接合并，因为各位可能来自不同采样边界；计数器跨域常用格雷码，异步 FIFO 则用读写指针、同步后的指针和空满判断来建立跨域约定。这里的关键不是背某一种结构，而是承认两个时钟域之间没有共同的“同时”概念，必须让协议定义何时有效、何时可采样、何时完成。

74HC74 这类双 D 触发器可以作为本章的门槛案例：它把 D、CLK、Q、复位和置位引脚明确暴露出来，迫使读者区分数据输入、采样边界和异步控制。74HC161 这类同步计数器则展示“当前状态经加法路径反馈到下一状态”的标准结构。74HC164、74HC595 等移位寄存器进一步说明，多 bit 状态可以按固定连线和时钟边界逐步移动。使用这些芯片时，必须检查电源去耦、输入默认电平、时钟边沿质量、复位极性和输出负载；否则纸面上的状态表无法保证板上行为。

专题：把本章改写成纸面设计审查

本章概念可以用常见 74HC/74HCT 逻辑芯片搭成低速实验板，但本书不要求读者实际搭建。更适合本书目标的训练，是把这些器件当作可审查的真实案例：根据数据手册写出电源、输入默认值、复位极性、时钟边界、输出负载和故障表现。若读者没有面包板和仪器，也可以完成同样的纸面设计审查；若读者选择实际搭建，则必须限制在低压直流逻辑范围内，例如 3.3V 或 5V 数字电路，不得把这些小信号电路直接接到市电、电机、继电器线圈或其他大功率负载。LED 必须串联限流电阻，CMOS 输入不能悬空，每片芯片的 VCC 与 GND 之间应靠近芯片放置约 0.1μF 去耦电容，实验板和信号源必须共地。

数据手册是搭电路时的依据。正文用信号名说明连接关系，而不依赖某一种封装的脚号；实际搭建时必须以手中芯片和封装的数据手册为准。低有效输入常写作 `reset_n`、`MR_n`、`OE_n` 或 `RD_n`，含义是拉低有效、拉高无效。若把低有效复位误当成高有效复位，电路会长期处于复位或完全失去复位保护。

器件	本章对应概念	搭建时的重点
74HC14	施密特输入、慢边沿整形、按钮消抖前后边界	输出反相；语义校正信号取自第一级，需要与输入同相的低有效信号时再串联第二级
74HC74	正沿 D 触发器、两级同步器、事件保持位	异步置位/复位脚必须接到确定无效电平或受控复位
74HC161/74HC163	4-bit 同步计数器、当前状态加一、进位级联	区分异步复位和同步复位，计数使能与并行装载不能悬空
74HC157	数据选择器、写使能保持、 <code>D=mux(we,new,Q)</code>	让时钟连续运行，用数据路径决定写入或保持
74HC595	串入并出移位寄存器、输出锁存、三态输出	区分移位时钟和锁存时钟，输出负载要受限

NE555 或晶振模块 低速实验时钟

时钟边沿应再经过施密特整形，按钮时钟必须消抖

如果只做纸面训练，不需要准备物料；需要准备的是三张表：器件功能表、引脚电平表和故障定位表。器件功能表说明每颗芯片在本章概念中扮演什么角色；引脚电平表说明每个输入脚在复位、空闲和动作时处于什么电平；故障定位表说明某个现象优先怀疑哪条边界。下面保留实际搭建所需物料，是为了让这些纸面表格有真实约束来源，而不是暗示读者必须做实物实验。

具体到本章实验，常用芯片包括 74HC14、74HC74、74HC161 或 74HC163、74HC157、74HC595；常用阻容范围包括 330R 到 1k 的 LED 限流电阻、4.7k 到 10k 的上拉或下拉电阻、100nF 去耦电容，以及 100nF 到 1uF 的按钮滤波电容。逻辑分析仪或示波器不是第一步必需品，但一旦要判断“按一次为什么计了两次”“复位释放为什么产生假事件”“锁存时钟为什么没有更新输出”，它们就比肉眼观察 LED 更可靠。

物料	用途	检查要点
稳定直流电源	给所有逻辑芯片提供同一电源域	电压在芯片允许范围内，实验板各处共地
0.1uF 去耦电容	抑制芯片开关瞬间的局部电源扰动	每片芯片靠近 VCC/GND 放置一颗
上拉/下拉电阻	给按钮、复位和未驱动输入确定默认值	松开按钮或开关断开时节点不悬空
LED 与限流电阻	低速观察 Q、同步级和计数输出	LED 电流不超过芯片输出能力
按钮与 RC 滤波	产生人工事件和复位请求	按钮边沿进入 74HC14 前已有默认电平
逻辑分析仪或示波器	观察边沿、抖动、短脉冲和时序关系	能同时看到 raw/pressed/sync/clk/reset_n

上电前应先按连接检查表逐项确认。许多低速实验失败不是因为状态机概念错误，而是因为 VCC/GND 接反、低有效输入悬空、输出直接短接、不同电压域硬连或 LED 没有限流。对 CMOS 逻辑而言，“未使用输入不接”不是中性状态，而是让输入节点暴露给噪声和漏电流；它可能让芯片耗电增大、输出随机变化，甚至把后级状态机带入错误状态。

检查项	必须确认	常见后果
电源脚	每片芯片的 VCC、GND 与数据手册一致	芯片发热、无输出或永久损坏
去耦	每片芯片电源脚旁有 0.1uF 电容	时钟沿或输出翻转时产生偶发错误
低有效脚	reset_n/MR_n/OE_n 等处于确定有效或无效电平	电路长期复位、输出高阻或随机启动
未用输入	接到确定 0 或 1，不能悬空	LED 随机闪烁，计数器误跳变
输出负载	LED 串电阻，不能多个推挽输出硬短接	输出电流过大或总线冲突
电压域	3.3V 与 5V 芯片互连满足输入阈值和绝对最大额定值	高电平不被识别，或 5V 损伤 3.3V 输入

搭建顺序也应分层进行。第一步只接电源、去耦、一个 LED 和限流电阻，确认电源轨、极性和 LED 方向。第二步只接 74HC14 与按钮，验证按钮松开和按下时输出有确定变化。第三步接 74HC74 两级同步器，用低速时钟观察 button_meta 和 button_sync 的周期差。第四步再接 74HC161/74HC163 或 74HC595。若一次把所有

芯片都插上，故障会混在一起，读者很难判断问题属于电源、按钮、时钟、复位还是状态逻辑。

第一个可搭电路是“按钮进入同步状态机”的前半段。按钮一端接 GND，另一端通过约 10k 电阻上拉到 VCC，形成低有效原始输入 `button_raw_n`。在按钮节点到 GND 之间可并联 100nF 左右电容作为低速消抖起点；随后进入 74HC14。由于 74HC14 是反相施密特触发器，按钮按下时 `button_raw_n=0`，经过第一级反相后即得到极性校正后的 `button_pressed`（1 表示按下）；需要保留与输入同相的低有效信号时，再取第二级输出。这个电路对应外部事件链路中的“默认电平、滤波、语义归一化”。

然后用 74HC74 搭两级同步器。第一触发器的 D 接 `button_pressed`，CP 接系统时钟，异步置位保持无效，异步复位接受控的 `reset_n`；第一触发器的 Q 记为 `button_meta`。第二触发器的 D 接 `button_meta`，CP 接同一个系统时钟，Q 记为 `button_sync`。状态机、计数器或后级逻辑只能使用 `button_sync`，不能直接使用 `button_raw_n` 或 `button_meta`。若要观察现象，可把 `button_pressed`、`button_meta`、`button_sync` 分别通过较大限流电阻接到 LED；在 1Hz 到 5Hz 的慢时钟下，读者能看到同步输出只在时钟边界变化。

```
button_raw_n -> 74HC14 -> button_pressed
```

```
74HC74 stage 1:
```

```
  D = button_pressed
```

```
  CP = clk
```

```
  Q = button_meta
```

```
74HC74 stage 2:
```

```
  D = button_meta
```

```
  CP = clk
```

```
  Q = button_sync
```

该实验的观察重点不是“按键能点亮 LED”这么简单，而是要记录四个事实。第一，按钮松开时原始节点有确定默认电平。第二，施密特输入之后的边沿不再长时间停留在中间电压。第三，`button_meta` 可能比 `button_sync` 早一个周期变化，后级不得使用它。第四，复位拉低后两个触发器进入不会伪触发的默认值，复位释放后输出只能在时钟沿重新进入有效状态。

第二个可搭电路是 74HC161 或 74HC163 计数器。把 CP 接低速时钟，计数使能接有效电平，并行装载保持无效，复位接到受控 `reset_n`，Q0 到 Q3 分别通过限流电阻接 LED。此时四个 LED 显示的是同一个时钟边界保存的 4-bit 状态，而不是四个彼此独立的灯。每来一个有效时钟沿，内部寄存器一起更新，组合加一逻辑准备下一个值。若使用 74HC161，应特别注意其复位口是异步复位；若教材实验要强调“同步释放”，可以让外部复位先进入本时钟域同步器，再驱动后级控制逻辑，或选用同步复位方式更清楚的计数器。

这个计数器实验可以验证三件事。第一，把时钟停住后，Q0 到 Q3 保持不变，说明状态由触发器保存。第二，把计数使能关掉后，时钟仍然可以继续进入芯片，但状态保持，说明写使能和门控时钟不是一回事。第三，若把并行装载打开并在 D0 到 D3 上给出固定值，计数器会在规定边界装载该值，说明状态元件可以执行“保持、加一、装载、清零”等不同下一状态选择。

第三个可搭电路是 74HC595 输出链路。它内部有移位寄存器和输出锁存器，通常使用 DS 作为串行数据，SHCP 作为移位时钟，STCP 作为输出锁存时钟，MR_n 作为低有效清零，OE_n 作为低有效输出使能。每给 SHCP 一个有效边沿，串行 bit 进入移位寄存器；连续移入 8 bit 后，再给 STCP 一个边沿，八个输出 Q0 到 Q7 才一起更新。这个器件非常适合说明“内部状态移动”和“外部可见输出提交”是两个边界。

```
for each bit:
  set DS stable
```



```
pulse SHCP

after 8 bits:
  pulse STCP
  Q0..Q7 update together
```

若用按钮手动提供 DS、SHCP 或 STCP，这些按钮信号仍然要先经过上拉/下拉、消抖和施密特整形。未经消抖的手动时钟可能一次按压产生多个边沿，使移位寄存器多移几位。若用微控制器驱动 74HC595，要保证电压域兼容；5V 逻辑输出不能随意接到只允许 3.3V 的输入。Q0 到 Q7 可以驱动小电流 LED 作为观察负载，若要控制继电器、电机或更大负载，必须增加合适的驱动器件、续流保护和隔离策略，不能把 74HC595 输出脚当作功率开关。

第四个可搭电路用 74HC157 和 74HC74 说明写使能。把 74HC74 的 Q 反馈到 74HC157 的一个输入，把新数据 new_bit 接到另一个输入，用 write_enable 选择哪一路送到 74HC74 的 D。时钟始终送到 74HC74；当 write_enable=0 时，D 得到旧 Q，下一个边界写回旧值，表现为保持；当 write_enable=1 时，D 得到 new_bit，下一个边界写入新值。这个实验应当与“用 74HC08 把时钟和 enable 相与”明确区分。后者即使在低速实验中似乎能工作，也会把 enable 的毛刺、传播延迟和不对称边沿带进时钟路径；它适合用逻辑分析仪作为反例观察，不适合作为后级状态机的正常时钟。

实验	观察信号	预期现象
按钮同步器	raw/pressed/meta/sync/reset_n/clk	后级只在时钟边界看到同步后的事件
4-bit 计数器	clk/reset_n/enable/Q0..Q3	关使能时状态保持，开使能时每周期加一
74HC595 输出链 路	DS/SHCP/STCP/MR_n/OE_n/Q0.. Q7	移位期间输出不乱跳，锁存边界后一起更新
写使能寄存器	new_bit/write_enable/D/Q/c lk	时钟连续存在，是否更新由数据路径决定

故障定位应从可观察事实开始，而不是先猜逻辑表达式。若计数器按一次按钮跳多格，优先检查按钮抖动、手动时钟是否经过 74HC14、是否把按钮直接当 CP 使用。若 LED 随机闪烁，优先检查 CMOS 输入是否悬空、复位或输出使能是否处于确定电平、面包板电源轨是否断开。若 74HC595 的 Q0 到 Q7 始终不变，应检查 OE_n 是否允许输出、MR_n 是否保持无效、STCP 是否真的收到锁存边沿，而不是只检查 SHCP。若复位后出现一次假启动，应检查同步器和 pending 位的复位默认值，以及复位释放是否在本时钟域边界发生。

失败现象	优先检查	修复方向
按一次计多次	按钮抖动、手动时钟、74HC14 整形	增加 RC 消抖，整形后再进时钟域
LED 随机闪烁	悬空输入、复位脚、输出使能脚、电源轨	给所有输入确定默认电平并检查共地
复位后假事件	同步器复位值、pending 位默认值、释放边界	异步拉入、同步释放，事件位复位为 0
74HC595 输出不变	OE_n/MR_n/STCP/SHCP 与数据稳定时间	区分移位时钟和锁存时钟，固定控制脚电平
写使能保持失败	mux 选择脚、Q 反馈路径、D 采样边界	观察 write_enable/D/Q/clk 的周期关系

芯片明显发热	电源极性、输出短路、总线冲突、负载电流	立即断电，按数据手册逐脚复查
--------	---------------------	----------------

专题：从数据手册到可靠接线

真实芯片不能只按逻辑符号连接，还要按数据手册给出的电气边界连接。数据手册通常有两类容易混淆的表。**绝对最大额定值**（absolute maximum ratings）是器件不应超过的损伤边界，不是推荐工作点；长期按这个边界运行并不可靠。**推荐工作条件**（recommended operating conditions）才是设计应采用的电源、电压、温度、输入上升下降时间和时钟条件范围。实验报告应记录使用的是哪一种条件，而不能把“没有立刻烧毁”当成合格。

数据手册项目	读法	设计用途
绝对最大额定值	超过可能损伤器件	用来划红线，不用来定正常工作点
推荐工作条件	厂商承诺正常工作的范围	决定 VCC、电压域、温度和输入边沿要求
VIH/VIL	输入被承认为 1 或 0 的电压门限	判断 3.3V 输出能否驱动 5V 输入
VOH/VOL	在给定输出电流下仍能保证的输出电平	判断下一级是否能可靠识别高低电平
输出电流	单脚和整片芯片的源/灌电流限制	决定 LED 电阻、负载数量和是否需要驱动器
传播延迟	输入变化到输出有效的最坏时间	计入时序预算，不能只看典型值
最小时钟脉宽	CP、SHCP、STCP 等高/低电平至少持续多久	判断按钮、NE555 或逻辑脉冲是否太窄
最高工作频率	在规定电压、负载和温度下的频率上限	面包板实验应留出明显余量

电压兼容要用最坏条件判断。若一个 3.3V 微控制器输出接到 5V 供电的 74HC 输入，不能只看示波器上有 3.3V 高电平，还要查 5V 条件下该输入要求的 `VIH_min`。如果 `VIH_min` 高于 3.3V 输出在负载下能保证的 `VOH_min`，这个连接就没有静态高电平余量，即使短时间看似能工作，也可能随温度、电源和芯片批次变化而失败。74HCT 系列常用于 5V 系统接收 TTL 电平标准的输入，是因为它的输入门限更适合较低高电平；但 74HCT 输出仍由自身 VCC 决定，不能把 5V 输出直接送入不耐 5V 的 3.3V 输入。

```
static high-level margin:
    driver VOH_min >= receiver VIH_min + margin

static low-level margin:
    driver VOL_max <= receiver VIL_max - margin
```

扇出与负载也要按电流和电平一起看。**扇出**（fan-out）指一个输出能可靠驱动多少个输入。CMOS 输入静态电流很小，所以低速实验中一个 74HC 输出常能驱动多个 CMOS 输入；但这不等于它能随意驱动多个 LED、长线或电容负载。LED 会消耗直流输出电流，电流过大时 `VOH` 会下降或 `VOL` 会升高，后级输入余量随之变小。长杜邦线和多个输入会增加电容负载，使边沿变慢；边沿太慢又可能让普通 CMOS 输入在门限附近停留过久，造成多次翻转或额外功耗。因此，观察 LED 应使用较大限流电阻，真实负载应使用专门驱动器或晶体管/MOSFET 级。

负载类型	风险	正确处理
------	----	------

多个 CMOS 输入	电容变大，边沿变慢	降低频率、缩短连线，必要时加缓冲器
LED 观察负载	输出电流过大拉低高电平或抬高电平	串较大限流电阻，优先低电流观察
长杜邦线	串扰、反射、接触不良和地线回路	低速实验、短线、共地、必要时示波器确认
继电器或电机	电流和反灌电压远超逻辑芯片能力	使用驱动管、续流二极管、独立供电和隔离策略
多个推挽输出相连	一个拉高、一个拉低会形成短路	只用三态总线或开漏/开集加上拉结构

时钟质量决定状态边界是否真实存在。一个合法时钟不仅要有频率，还要满足最小高电平时间、最小低电平时间、上升/下降时间和边沿单调性。NE555 可以产生低速时钟，但输出边沿、占空比和电源扰动仍需检查；按钮可以产生人工事件，但未经消抖不应直接作为计数器 CP；晶振模块边沿质量较好，但频率可能高到面包板布线、LED 观察和手工调试都跟不上。手工实验建议先用 1Hz 到 5Hz 的可观察时钟验证状态，再逐步提高频率，并在提高频率前去掉重负载 LED 或改用缓冲观察点。

时钟来源	适合用途	额外检查
按钮单步	教学观察、单周期推进	必须消抖并经施密特整形，不直接驱动 CP
NE555	低速自动运行	检查占空比、电源去耦、边沿和最小脉宽
晶振模块	稳定周期时钟	频率、电压域、输出负载和面包板布线余量
微控制器 GPIO	可编程时钟或测试脉冲	电压兼容、脉宽、地线和软件抖动

面包板适合低速验证概念，不等同于可量产硬件。面包板触点有接触电阻和寄生电容，长杜邦线会带来额外电感和串扰，电源轨可能在中间断开，地线回路会让几个芯片看到不同的瞬时参考电位。这些问题在 1Hz LED 实验里可能不明显，在更快边沿和更多芯片同时翻转时就会变成偶发故障。将来进入 CPU 数据通路、总线和存储器章节时，读者应记住：面包板证明“逻辑关系能被观察”，不能证明“任意频率和任意规模都可靠”。

安全边界也要明确。本章实验只验证低压逻辑信号，不验证功率驱动。若要让逻辑电路控制继电器、电机、灯带或其它外部负载，必须增加驱动晶体管或 MOSFET、栅极/基极电阻、续流二极管或 TVS、独立电源退耦、共地或隔离策略，并重新评估电流、热、反灌电压和失效状态。把 74HC595、74HC74 或 74HC161 的输出脚直接当作功率开关，是把逻辑输出误用为能量输出。

下面是一份完整故障报告样例。现象：用按钮给 74HC161 提供单步时钟，按一次按钮时计数器从 3 跳到 5。定位步骤一，逻辑分析仪同时观察按钮原始节点 `button_raw_n`、74HC14 输出 `button_pressed` 和计数器 CP；发现一次按压期间 CP 出现两个上升沿。定位步骤二，断开计数器，只观察按钮链路；发现 RC 时间常数过小且按钮释放也产生窄脉冲。修复动作：增大消抖电容，保证按钮先进入 74HC14，再由整形后的信号产生单步请求；若该请求进入状态机，则继续经过 74HC74 同步。复测结果：一次按压期间 CP 只有一个有效边沿，计数器从 3 变为 4。结论：故障不是 74HC161 的加一逻辑错误，而是手动时钟没有形成唯一边界。

实验报告应像一份小型工程记录，而不是只写“现象正确”。每个实验至少记录芯片型号和系列、电源电压、时钟来源、复位极性、每个低有效控制脚的默认电平、异步输入如何进入同步器、观察到的信号、失败现象和修复动作。若使用 74HCT 与 74HC 混合，还要记录电平兼容依据。一般而言，5V 输出不应直接接入只允许 3.3V 的输入；3.3V 输出能否可靠驱动 5V 供电的 74HC 输入，也不能凭“看起来能亮”判断，

而应查数据手册中的输入高电平阈值。74HCT 的输入阈值更接近 TTL 电平标准，常用于 5V 系统中接收较低高电平，但输出仍受自身供电影响。

报告项	应记录的内容	判断依据
物料与版本	芯片完整型号、逻辑系列、电源电压	可回到数据手册复查电气条件
连接约定	VCC/GND、复位、置位、输出使能、未用输入	没有悬空控制脚和危险负载
时钟与复位	时钟来源、频率范围、复位拉入和释放方式	状态只在边界变化，复位后无伪事件
异步输入	原始引脚、消抖、施密特整形、同步级数	后级只使用同步后的信号
观测结果	LED、逻辑分析仪或示波器看到的关键节点	现象能解释到具体边界和信号名
故障处理	失败现象、定位步骤、修复前后对比	修复不是偶然重插线，而有明确原因

这些实验把“状态、时钟与同步边界”的概念应用到真实芯片上。完成实验后，读者应能为每个小电路写出一页完整的实验记录：使用了哪些芯片，电源和默认电平如何保证，哪些输入是异步的，进入了几级同步器，复位默认状态是什么，哪些输出只是观察 LED，哪些信号绝不能直接驱动危险负载。这样的记录比单纯画出逻辑关系更接近真实工程。

因此，本章的最终目标不是让读者背出触发器、寄存器、计数器和状态机的定义，而是能为一个同步小系统留下**可复查的设计记录**：状态表、复位表、时序预算、跨边界处理方式和异常恢复策略。后续 CPU 的 PC、寄存器堆、控制器、异常入口和设备中断，都将重复使用这套记录格式。

专题：setup/hold 不是公式，而是采样窗口 触发器的 setup 和 hold 约束经常被背成两个时间参数，但它们本质上是在说明采样窗口。时钟沿到来前，D 输入要提前稳定一段时间；时钟沿之后，D 输入还要继续保持一小段时间。若输入在窗口内变化，触发器内部再生过程可能无法快速决定 0 或 1，输出延迟变长，甚至进入亚稳态。

时序参数	原理含义	违反后果
t_{setup}	时钟沿前 D 必须稳定的最短时间	采样值可能错误或输出延迟异常
t_{hold}	时钟沿后 D 必须保持的最短时间	新数据可能穿透到本周采样
t_{clkq}	时钟沿后 Q 输出稳定所需时间	后级组合逻辑可用时间减少
t_{pd}	组合逻辑最坏传播延迟	决定下一级 setup 是否满足
skew	不同触发器实际看到时钟沿的差异	可能同时影响 setup 和 hold 裕量

单周期路径的基本时序关系可以写成：

```
clk_period >= t_clkq_max + t_logic_max + t_setup + skew + margin
hold_margin = t_clkq_min + t_logic_min - skew - t_hold
```

第一条约束决定频率上限，第二条约束决定短路径是否太快。很多初学者只关心“逻辑太慢”，却忽略“逻辑太快”也可能破坏 hold。若同一时钟沿从前级触发器出来的数据太快到达后级，而后级仍处在 hold 窗口内，就可能把本该下一拍才采样的

数据提前采进去。解决 hold 的方法通常不是降低频率，而是调整时钟树、增加最小延迟或改变寄存器布局。

专题：时序预算的一个完整数值例

把采样窗口的两条不等式套到一组具体数字上。某设计目标频率 50MHz，即时钟周期 20ns。关键路径参数：源触发器 $t_{CQ_max}=3ns$ 、 $t_{CQ_min}=1.5ns$ ，组合逻辑最坏延迟 9ns，目标触发器建立时间 $t_{SU}=2ns$ 、保持时间 $t_H=0.5ns$ ，时钟偏斜按最不利方向取 1ns。时序分析没有更多公式，只有一条纪律：建立检查用“最坏组合”，所有延迟取最大；保持检查用“最好组合”，所有延迟取最小——因为两个问题的敌人方向相反，一个怕数据来得太晚，一个怕数据来得太早。

先做建立检查。数据最晚在时钟沿后 3ns 离开源触发器，再花 9ns 穿过组合逻辑，第 12ns 才到达目标 D 端；目标要求它比下一个时钟沿（第 20ns）至少早 t_{SU} 加偏斜共 3ns 就绪，即第 17ns 之前。12ns 早于 17ns，检查通过，余量 5ns。

```
setup check (worst corner):
arrival   = tCQ_max + t_comb_max = 3 + 9       = 12ns
required = T - tSU - skew        = 20 - 2 - 1 = 17ns
slack     = 17 - 12              = +5ns (pass)
```

再做保持检查。数据最快在时钟沿后 1.5ns 就可能离开源触发器；若这条路径几乎没有组合逻辑（例如移位寄存器的直接相邻位），新数据第 1.5ns 就到达目标 D 端。而目标的保持窗口要求旧数据至少维持到时钟沿后 t_H 加偏斜共 1.5ns。1.5ns 对 1.5ns，余量恰好是 0：纸面上不违例，工程上叫边缘。

```
hold check (best corner):
earliest = tCQ_min + t_comb_min = 1.5 + 0     = 1.5ns
required = tH + skew            = 0.5 + 1      = 1.5ns
margin    = 1.5 - 1.5           = 0ns (marginal)
```

余量为 0 意味着任何一点工艺波动、温度漂移，或实际时钟树偏斜略大于估计值，都会把它推成违例。更要紧的是，降频救不了它：把 50MHz 降到 25MHz，建立余量从 5ns 变成 25ns，保持余量却仍是 0ns——保持窗口跟着时钟沿走，不跟着周期走。修复只有三条路：在数据路径上增加受控的最小延迟（插缓冲器）、减小时钟偏斜（修时钟树）、或换用 t_{CQ_min} 更大的触发器。这就是“降频治不了保持问题”的数值含义。

检查项	代入组合	计算	结论与余量
建立时间	最坏组合： t_{CQ_max} 、组合最大延迟、 t_{SU} 、不利偏斜	$3+9+2+1=15ns$ ，周期 20ns	通过，余量 +5ns
保持时间	最好组合： t_{CQ_min} 、组合最小延迟约 0、 t_H 、不利偏斜	1.5ns 对 $0.5+1=1.5ns$	边缘，余量 0ns，建议加最小延迟
降频效果	周期从 20ns 改为 40ns	建立余量升至 25ns，保持余量仍 0ns	只治建立，不治保持

静态时序分析工具对设计中每一条寄存器到寄存器路径都做这两遍计算，报告里的 setup slack 与 hold slack 就是上面的 +5ns 和 0ns。手工算一遍的价值在于：看到报告时知道每个数字对应哪一段物理延迟，也知道该改逻辑、改时钟树还是改目标频率。

亚稳态只能降低概率，不能被完全消灭 异步输入、跨时钟信号和机械按键都可能在触发器采样窗口内变化。触发器内部的再生电路可能暂时停在中间电平，随后随机解析为 0 或 1，这就是亚稳态。同步器不能保证“永不亚稳”，只能让亚稳态有更多时间衰减，使错误传播到后级的概率低到工程可接受。

设计选择	作用	仍需注意
两级同步器	给第一级亚稳态多一个周期衰减	只能同步单 bit 电平，不能同步多 bit 总线值
握手协议	用 valid/ready 或请求/应答跨边界	吞吐降低，但语义清楚
异步 FIFO	用双端口存储和格雷码指针跨时钟	指针同步和空满判断必须严格验证
脉冲拉宽	让短事件至少覆盖目标时钟周期	过窄脉冲可能完全漏采

多 bit 总线不能简单给每一位各接两级同步器后直接使用，因为各位解析时间可能不同，接收端会看到不存在的混合值。正确做法通常是先在源时钟域保持数据稳定，再用握手同步“数据可用”事件；或者使用异步 FIFO 让写指针和读指针通过受控编码跨域。

复位要区分上电初始化、错误恢复和运行控制 复位不只是“把所有寄存器清零”。有些寄存器必须复位到安全值，有些数据寄存器可以不复位，有些外设需要按电源和时钟顺序释放，有些跨时钟域需要各自同步释放。把复位当成普通控制信号乱用，会造成时钟域之间状态不一致、异步释放毛刺或组合路径被复位扇出拖慢。

复位对象	建议策略	检查要点
控制状态机	复位到明确安全状态	复位释放后第一拍输出是否安全
PC/入口状态 数据寄存器	装入复位向量或启动状态 可按需要不复位，由控制路径保证无效时不使用	第一条取指地址是否确定 是否有 valid 位防止读旧值
外设接口	先复位设备内部，再开放总线响应	复位期间是否会误写 MMIO
跨域复位	异步断言、同步释放或每域独立处理	释放边界是否满足目标时钟域时序

良好的复位规格应写清楚：复位信号有效电平、最短保持时间、时钟是否必须运行、复位释放后第一个可接受输入的时间、输出是否需要保持安全值。没有这些边界，系统 bring-up 时会把电源、时钟、复位、状态机和总线问题混成一团。

状态机验证要覆盖非法状态和输出时序 状态机图只画正常转移是不够的。真实触发器可能因复位不完整、毛刺、单粒子翻转或设计错误进入未列出的编码。若非法状态没有恢复路径，控制器可能卡死或输出危险控制线。状态机输出还要区分 Moore 型和 Mealy 型：前者只依赖当前状态，后者还依赖输入，可能更快但更容易受输入毛刺影响。

验证项	要检查什么	失败风险
复位覆盖 非法状态恢复	所有关键状态寄存器进入已知状态 未使用编码能回到安全状态或错误入口	上电随机进入危险路径 状态机锁死或误写设备
输出默认值	每个状态未显式赋值的控制线有安全默认	锁存器推断或保持旧控制线
输入同步	Mealy 输入是否来自同步边界	毛刺直接影响输出控制线
进度性	等待状态是否有超时或退出条件	外设无响应时永久等待

这套检查表会在 CPU 控制器、内存等待、DMA 描述符、总线仲裁和中断控制器中反复出现。第二章的状态机不是孤立数字逻辑练习，而是后面所有“硬件会按阶段

推进”的基础。

去抖和边沿检测要分成两件事 机械按键输入常常同时暴露三个问题：异步、抖动和事件检测。异步意味着它不按本机时钟变化，需要同步；抖动意味着一次按下会产生多个跳变，需要滤除；事件检测意味着电平稳定后还要决定什么时候产生一个周期宽的“按下事件”。把三者混在一起，常会导致一次按键被计数多次，或按住不放时不断触发。

```
button pipeline:
  raw_pin -> synchronizer -> debounce counter -> stable_level
  stable_level + previous_level -> one_cycle_press_event
```

阶段	原理	预期现象
同步	两级触发器降低亚稳态传播概率	raw 改变时内部不会产生多 bit 混合状态
去抖	输入持续稳定 N 个周期才接受新电平	抖动期间 <code>stable_level</code> 不变化
边沿检测	当前稳定电平与上一稳定电平比较	一次按下只产生一个事件脉冲
事件消费	状态机或计数器只看事件脉冲	长按不会重复触发，除非规格要求连发

这个小例子把第二章与第一章连接起来：第一章能用示波器看到按键抖动，第二章用同步状态机把抖动变成稳定事件，后面整机章再把该事件映射成 GPIO 状态位或中断请求。

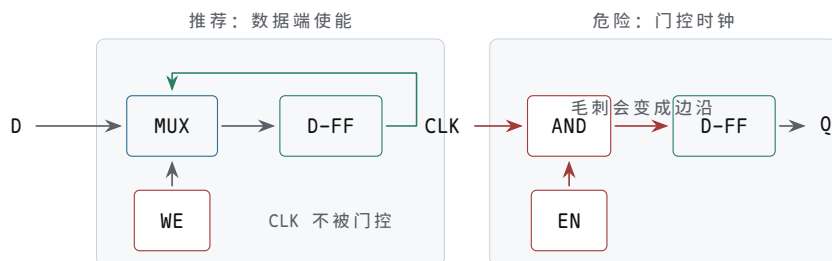
章末练习

本节练习要求把状态边界写清楚。答案必须说明哪个值在组合路径上传播，哪个值在时钟沿保存；若涉及外部输入、复位或跨时钟边界，还必须说明同步和默认安全状态。

任务	要完成的产物	预期结果
入门：画 Q 波形	给定一段 D 和 CLK 波形，分别画出 D 锁存器（LE=CLK）和 D 触发器（上升沿）的 Q	锁存器 Q 在 CLK=1 全程跟随 D，触发器 Q 只在上升沿变化
入门：填 SR 真值表	补全 $\bar{S}/\bar{R}$ 四种组合下 Q 的值，并标出禁行	禁行说明互补被破坏，不是“存了 1”；释放顺序决定恢复结果
入门：画计数波形	从初值 9 开始画 6 个周期的 Q[3:0] 总线带和 Q0 波形	每个上升沿整体 +1，Q0 恰好是 CLK 二分频
入门：主从结构	给定 D 与 CLK，填出主锁存器 M 和输出 Q 在各阶段的值	M 只在 CLK=0 跟随 D，Q 只在上升沿复制 M
触发器路径时序分析	为一条寄存器到寄存器路径列出 t_{CQ} 、组合延迟、连线延迟、建立时间、偏斜和余量	能判断目标周期是否通过建立约束
保持违例	构造一条最小延迟过短的路径，并写出修复方法	能区分建立问题和保持问题
锁存器透明窗口	说明两个锁存器共用 enable 时可能出现的穿透风险	能解释为什么大系统更偏好清晰时钟边界

复位释放	为一个三状态控制器写复位表和释放策略	复位期间危险输出关闭，释放在同步边界发生
两级同步	说明外部按钮或中断线进入时钟域的处理流程	原始异步信号不能直接驱动状态机
MTBF 指标	说明为什么同步器只能降低亚稳态风险，并列影响 MTBF 的因素	能把时钟频率、事件频率、恢复时间和器件特性联系起来
外部事件链路	为一个按钮或传感器写出从原始引脚到状态机消费的完整信号链	能区分电气处理、同步、边沿检测和事件保持
事件保持	设计 <code>pending/ack</code> 结构，说明何时置位、何时清除	窄事件不能因状态机暂时不可接收而丢失
多 bit 跨域	说明为什么 8-bit 总线不能逐位两级同步后直接使用	给出握手、FIFO 或格雷码方案
脉冲同步	设计一个不会漏采窄脉冲的跨域事件方案	能说明事件保持到被确认
时钟门控	比较写使能、数据 mux 保持、专用时钟门控和普通门控时钟	能说明为什么普通门电路不应直接切时钟
复位域	为两个相关时钟域写复位释放和 <code>ready</code> 等待策略	能说明复位跨域也需要同步和安全默认值
状态编码	比较二进制、one-hot 和格雷编码的用途与风险	能为未使用编码写安全恢复规则
波形验证	列出验证一个同步小系统必须观察的信号	能从波形判断写使能、复位、 <code>pending/ack</code> 是否正确
CPU 连接	说明 PC、流水线寄存器、控制器和中断入口分别对应本章哪个概念	能把第二章边界模型映射到后续 CPU
按钮同步器实作	用 74HC14 和 74HC74 写出按钮到同步电平的连接方案	能区分原始引脚、消抖整形、第一级同步和第二级同步
计数器实作	用 74HC161 或 74HC163 设计 4-bit LED 计数器	能说明时钟、复位、计数使能、并行装载和输出负载
移位输出实作	用 74HC595 说明串行输入、移位时钟、锁存时钟和输出使能	能解释为什么八个输出在锁存边界一起更新
写使能实作	用 74HC157 和 74HC74 搭一个可保持的 1-bit 寄存器	能说明为什么数据 mux 保持优于普通门控时钟
实验板物料	为本章低速逻辑实验列出最小 BOM 和用途	能覆盖电源、去耦、默认电平、观察负载和测量工具
接线检查	写出上电前检查表，覆盖低有效脚、未用输入、电压域和输出负载	能防止悬空、误复位、输出短路和电平不兼容
故障定位报告	针对一个失败现象写出定位步骤和修复验证	能从现象追到具体信号边界，而不是只重插线

数据手册读取	给出 74HC 实验前必须查的参数表和用途	能区分绝对最大额定值、推荐工作条件、输入输出电平和时序参数
电压与扇出	判断一个输出能否可靠驱动后级输入、LED 或多路负载	能用 $V_{OH}/V_{OL}/V_{IH}/V_{IL}$ 和输出电流说明余量
时钟与面包板限制	说明按钮、NE555、晶振模块和面包板布线的边界	能解释最小脉宽、边沿质量、寄生效应和低速验证范围
非法状态恢复	为状态机未使用编码指定下一状态和输出	非法状态不产生危险使能
状态机输出	比较只依赖状态的输出和依赖输入的输出	能说明稳定性与响应速度的取舍
芯片案例	用 74HC74、74HC161 或移位寄存器说明一个本章概念	绑定到真实引脚、复位、时钟和输出负载
纹波与同步计数器	比较 4-bit 纹波计数器与 4-bit 同步计数器的时钟接法、延迟来源和译码风险	能说明纹波逐级延迟累积、译码有毛刺窗口，同步计数器把代价移到组合进位链
LFSR 前八态	4 位 LFSR 每拍 Q_3 取旧 Q_2 、 Q_2 取旧 Q_1 、 Q_1 取旧 Q_0 、 Q_0 取旧 Q_3 与旧 Q_2 的异或，从 0001 起列出前 8 个状态	序列正确，并能说明 0000 为何必须排除
模 12 计数器	用 4-bit 寄存器与加一逻辑设计模 12 计数器（0 到 11 循环）	归零发生在同步边界，无瞬时非法态
Moore 与 Mealy 区分	判断交通灯控制器与 1101 序列检测器各属哪类，并说明输出稳定性差异	分类依据写成“输出是否依赖当前输入”
交通灯转移表	按东西绿 30s、黄 5s、全红 2s、南北绿 30s、黄 5s、全红 2s 的需求补全状态转移表与输出表	六状态闭环， <code>timer_done</code> 为唯一转移条件，输出只依赖状态
格雷码相邻性	核对交通灯六状态格雷编码 000、001、011、010、110、100 的每对相邻状态（含首尾闭环）是否只差一位	六次转移逐一核对，全部只差一位



左边只改变 D 端选择，时钟树保持干净；右边把 EN 放进时钟路径，EN 的毛刺可能被触发器当作真实时钟沿。

图示：写使能 vs 门控时钟：写使能在数据路径上选择；门控时钟可能把 EN 的毛刺变成额外时钟沿。

参考解答与要点

任务	参考解答	必须保留的要点
入门：画 Q 波形	锁存器 Q 在 CLK=1 期间完全复制 D 的每次变化；触发器 Q 只在每个上升沿复制当时刻的 D，其余时间保持。	触发器不是“更快的锁存器”，采样时刻完全不同。
入门：填 SR 真值表	0,0 禁用；0,1 置 1；1,0 置 0；1,1 保持。从 0,0 同时释放到 1,1 时结果不可预测。	禁行是非法输入；可靠设计不产生 0,0 或不同时释放。
入门：画计数波形	Q[3:0] 总线带：9、10、11、12、13、14 逐沿跳变；Q0：1、0、1、0、1、0。	总线带整体跳变，不是逐位爬；Q0 每周翻转。
入门：主从结构	M：CLK=0 期间等于 D，CLK=1 期间保持上升沿时刻的 D；Q：每个上升沿等于 M 当时值。	D 在 CLK=1 期间的变化只影响 M 保持的旧值，不影响 Q。
触发器路径时序分析	示例： $T_{clk} \geq 0.8ns + 3.6ns + 0.7ns + 0.4ns + 0.5ns = 6.0ns$ 。若目标周期为 5.0ns，则建立约束失败。	使用最大延迟和不利偏斜，不得只用典型门延迟。
保持违例	示例：最快路径 0.13ns 到达，而目标保持窗口要求 0.28ns 后才允许变化，存在保持违例。修复可插入保持缓冲、调整布局或修正时钟树。	降频不能可靠修复保持违例。
锁存器透明窗口	enable 有效期间，前一级输入变化可能穿过第一级锁存器、组合逻辑和第二级锁存器，使一次更新跨越多个预期阶段。	必须指出透明期间没有清晰提交边界。
复位释放	控制状态复位到 IDLE 或 RESET_WAIT，写使能和危险输出为 0；复位可异步拉入，但在本时钟域用触发器同步释放。	复位值、安全输出、释放边界三者都要写明。
两级同步	外部按钮先经过电气默认值和消抖，再进入两级触发器，状态机只使用 button_sync。	两级同步降低亚稳态扩散概率，但不替代消抖。
MTBF 指标	MTBF 表示亚稳态导致可见失败的平均时间尺度。目标时钟越快、异步事件越频繁，风险越高；第一级到第二级之间恢复时间越长，风险越低；器件恢复参数也会影响结果。	同步器是可靠性设计，不是绝对功能保证。
外部事件链路	示例链路：raw -> filtered -> pressed -> meta -> sync -> event -> pending -> FSM。其中 raw/filtered 属于电气与消抖，meta/sync 属于时钟域同步，event/pending 属于目标时钟域事件协议。	每一层信号名应对应一个可测量、可观察的边界。
事件保持	start_event 置位 start_pending，状态机在 IDLE 且安全条件满足时产生 start_ack 清除 pending；复位或故障可按安全策略清除或拒绝事件。	单周期事件不应直接作为危险动作使能。

多 bit 跨域	8-bit 总线逐位同步会采到混合时刻的 bit。若是数据传输，应使用 valid/ready 握手或异步 FIFO；若是计数指针，可用格雷码减少一次变化的 bit 数。	多 bit 数据必须有整体有效性约定。
脉冲同步	可把源域窄脉冲转换为保持电平或翻转事件位，目标域同步后检测变化，再用确认信号让源域清除事件。	事件必须持续到目标域可见，不应仅依赖单周期脉冲宽度。
时钟门控	写使能让时钟继续到达触发器，只控制是否保存新值；专用时钟门控用于功耗控制并要求 enable 无毛刺和安全锁存；普通 AND 门切时钟会产生短脉冲、额外边沿和不可控偏斜。	普通数据通路优先用写使能，不应随意组合门控时钟。
复位域	每个时钟域内部同步释放复位；跨域依赖通过 ready、同步器或状态握手确认。同步器、pending 位和危险输出在复位期间应进入不会伪触发的默认值。	复位释放顺序属于设计约定，不能假设所有域同时恢复。
状态编码	二进制编码节省触发器但译码较复杂；one-hot 译码直接但触发器多；格雷编码适合相邻状态或跨域指针。无论哪种编码，未使用编码都应导向安全状态并关闭危险输出。	状态编码选择要绑定资源、速度、跨域和安全要求。
波形验证	至少观察 <code>clk/reset/source_q/dest_d/write_enable/state/sync1/sync2/pending/ack</code> ，检查 D 是否在采样沿前稳定、写使能是否只在目标周期有效、复位释放是否产生伪事件。	只看最终寄存器值不足以证明同步边界正确。
CPU 连接	PC 是寄存器，流水线寄存器是阶段边界，控制器是状态机，写回使能是状态控制线，中断入口来自同步后的外部或内部事件。	后续 CPU 仍遵守“组合准备、边界提交”的模型。
按钮同步器实作	按钮节点用上拉或下拉给出默认电平，经过 RC 和 74HC14 得到整形后的 <code>button_pressed</code> ；再接入 74HC74 两级触发器，后级只使用第二级 Q。复位脚进入不会伪触发的默认状态。	必须写清楚每一级信号名，不能把 raw 引脚直接送入状态机。
计数器实作	74HC161/74HC163 的 CP 接低速时钟，计数使能接受控有效电平，并行装载保持无效或由实验开关控制，Q0 到 Q3 通过限流电阻接 LED。复位极性按数据手册连接。	计数器体现同一时钟边界上的 4-bit 状态更新。
移位输出实作	74HC595 的 DS 在 SHCP 边沿前稳定，8 次移位后再给 STCP 边沿，Q0 到 Q7 一起更新；MR_n 和 OE_n 保持在确定电平。	必须区分移位寄存器内部状态和输出锁存边界。
写使能实作	74HC157 在旧 Q 和 <code>new_bit</code> 之间选择，输出送入 74HC74 的 D；时钟持续送入触发器， <code>write_enable=0</code> 时写回旧 Q， <code>write_enable=1</code> 时写入新值。	不应使用普通与门直接门控时钟作为正式方案。

实验板物料	最小 BOM 应包含稳定 3.3V 或 5V 电源、面包板、目标 74HC/74HCT 芯片、0.1uF 去耦电容、上拉/下拉电阻、LED 与限流电阻、按钮、RC 滤波电容、杜邦线和万用表；逻辑分析仪或示波器用于观察边沿和短脉冲。	物料清单必须对应实验中的电源、输入、状态输出和测量需求。
接线检查	上电前检查 VCC/GND、低有效复位和输出使能、未用输入默认值、LED 限流、输出是否短接、电压域是否兼容。发现芯片发热或输出异常时应立即断电复查。	检查表应能解释常见硬件失败，而不是只列器件名称。
故障定位报告	示例：按一次计数器跳多格时，先观察按钮 raw，再看 74HC14 输出，最后看计数器 CP；若 raw 抖动而整形后仍有多个边沿，应增加消抖或避免按钮直接作为时钟。报告要写出修复前后观测差异。	失败分析必须对应到可观察信号和修复后的波形。
数据手册读取	必须先查推荐工作条件确认 VCC、电压和输入边沿，再查 $V_{IH}/V_{IL}/V_{OH}/V_{OL}$ 判断电平兼容，查输出电流判断 LED 和负载，查传播延迟、最高频率与最小脉宽判断时钟是否合格；绝对最大额定值只用于损伤边界。	不得把绝对最大额定值当作正常工作条件。
电压与扇出	若驱动器 V_{OH_min} 小于接收器 V_{IH_min} 加余量，高电平不可靠；若 LED 电流过大，输出电平会被拉坏；一个输出驱动多个 CMOS 输入时还要考虑电容负载和边沿变慢。	判断必须同时看电压余量、电流限制和负载电容。
时钟与面包板限制	按钮单步必须消抖和施密特整形，NE555 要检查占空比和脉宽，晶振模块要检查频率和电压域；面包板只适合低速概念验证，不能证明高频 CPU 级可靠性。	状态边界必须来自合格边沿，不来自慢爬升或多次抖动。
非法状态恢复	未使用编码的下一状态导向 IDLE、FAULT 或复位等待态，输出逻辑默认关闭写使能、驱动使能和片选。	非法状态的输出也要安全，不应只写下一状态。
状态机输出	Moore 输出只依赖状态，通常更稳定；Mealy 输出还依赖输入，响应更快但可能把输入毛刺传到控制线。危险输出宜寄存或只由稳定状态产生。	取舍必须结合后级是否采样或驱动危险对象。
芯片案例	以 74HC74 为例，D 是待保存数据，CLK 是采样边界，Q 是保存结果，异步复位/置位必须按数据手册极性连接；未用输入不能悬空。	真实芯片要检查引脚、电源、复位极性、时钟边沿和负载。

纹波与同步计数器	纹波计数器用低位 Q 作高位时钟，第 k 位比第 0 位晚 k 个 t_{CQ} 翻转，最高位在第 4 个 t_{CQ} 才稳定，与第 0 位相差 3 个 t_{CQ} ；翻转窗口内译码器会看到瞬时错值，输出毛刺真实存在。同步计数器所有位共用同一时钟沿、同时更新，代价是进位链（高位使能等于低位全 1）随位数变长，成为组合关键路径。	纹波的风险在时钟不同步，同步的代价在进位逻辑深度。
LFSR 前八态	0001、0010、0100、1001、0011、0110、1101、1010。逐拍按规则计算：高三位左移接收邻位， Q_0 取旧 Q_3 与旧 Q_2 的异或。全 0 态的反馈恒为 0，一旦进入便永远锁死，故初态必须非零。	每拍只由旧态决定新态；0000 是陷阱态。
模 12 计数器	现态为 1011（即 11）时，下一状态逻辑选 0000 而非加一：用比较器译码 1011 控制数据 mux 即可。若改用异步清零计数器（如 74HC161），需译码 1100 去拉清零脚，会产生短暂的 1100 毛刺态，且各位清零时刻不一致；同步清零（如 74HC163）译码 1011 后在时钟沿干净归零。	归零必须发生在时钟边界；异步清零方案要指出毛刺态。
Moore 与 Mealy 区分	交通灯的灯色只由当前状态决定，属 Moore 型，输出在整个状态期间稳定；1101 检测器在 S_3 且输入为 1 的转移上给出输出，属 Mealy 型，同拍响应但输入毛刺可直达输出。	分类看输出函数的自变量，与状态个数无关。
交通灯转移表	EW_GREEN 到 EW_YELLOW 到 ALL_RED_EW 到 NS_GREEN 到 NS_YELLOW 到 ALL_RED_NS 再回到 EW_GREEN，转移条件均为 $timer_done=1$ ，装载值依次为 5、2、30、5、2、30 秒；输出依次为绿/红、黄/红、红/红、红/绿、红/黄、红/红。 $timer_done=0$ 时保持原状态。	状态数、转移条件、输出必须与需求逐条对应。
格雷码相邻性	000 到 001 差最低位；001 到 011 差中间位；011 到 010 差最低位；010 到 110 差最高位；110 到 100 差中间位；100 到 000 差最高位。六次转移均只翻转一位。	闭环的最后一边（100 回到 000）也必须核对，不能只看前五次。

本章的标注对象是“边界”和“状态转移表”。仅说明某个值会变化并不充分，必须说明哪个时钟边界提交，提交前后的组合路径是什么。

- 纸上实验：画出一个 D 触发器前后各接一段组合逻辑，标出当前周期的旧 Q、组合路径上的 D、下一个时钟沿后的新 Q。
- 时序分析：为一条路径写出 $T_{clk} \geq t_{CQ} + t_{comb} + t_{setup} + t_{skew}$ ，再解释每一项对应哪个硬件现象。
- 复位设计：复位要让控制状态进入已知安全点，但释放复位也要有同步边

界，不能让状态机半边先跑。

- 状态结构：寄存器是一组并排触发器；计数器是寄存器输出经过加法器反馈到输入；移位寄存器是固定连线选择了相邻 bit。
- 控制结构：状态机输出可以只依赖状态，也可以依赖状态和输入；后者反应快，但更容易把输入毛刺传给控制线。
- 检查题：若状态编码里出现未使用编码，电路上电或受干扰进入它时，下一状态逻辑应该怎样让机器回到安全状态？参考答案：未使用编码不应保持未定义，也不应产生危险输出；下一状态逻辑应把它导向复位、IDLE、FAULT 或其他安全诊断状态，同时关闭写使能、功率使能等危险控制线。若安全要求更高，还应记录错误或触发复位流程。

经典系统教材通常会把组合逻辑和状态逻辑分开建模：组合逻辑计算下一值，状态元件在边界保存下一值。后面 CPU 的 PC、寄存器、状态机和控制器都遵守这个模型。

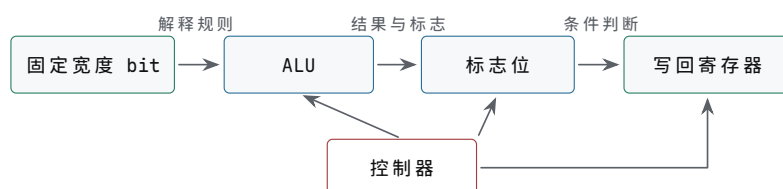
第 3 章

数值、ALU、数据通路与控制 器

前两章已经建立了两条基础事实：bit 不是抽象符号，而是带噪声余量的电平；状态不是文字变量，而是在时钟边界被触发器保存下来的电荷和电压。本章讨论第一部分的第三块内容，集中回答 CPU 出现之前必须解决的一组连续问题：固定宽度 bit 如何解释成数，算术逻辑单元如何产生结果和标志，寄存器、选择器、总线和写回路径如何组成数据通路，控制器又如何在每个周期输出一组一致的控制信号。

本章的写法遵循一个具体任务：让硬件完成 $R3 = R1 + R2$ ，再根据结果是否为 0 决定是否改变下一步动作。这个任务看似简单，却已经包含了计算机硬件的核心约定。首先， $R1$ 和 $R2$ 里保存的只是固定宽度 bit，必须说明它们按无符号、补码、定点还是其他格式解释；其次，ALU 要选择加法路径并产生结果、进位、零标志和溢出标志；再次，数据通路要把两个旧寄存器值送入 ALU，再把新结果写回目标寄存器；最后，控制器要保证读、算、写和下一状态选择发生在正确周期，不能把尚未稳定的组合输出提前提交。

经典体系结构教材讲 ALU 和数据通路时，很少把它们当作孤立名词处理，而是反复追问三件事：输入 bit 的解释是什么，组合路径的输出是什么，状态在什么边界提交。本章也按这个顺序阅读。若能为一条动作写出“源寄存器、选择器、ALU 操作、标志位、写回来源、写使能、下一状态”的 trace（执行轨迹），就已经具备走读最小 CPU 的基本能力。



本章不是先讲“加法公式”，而是把 bit 解释、组合计算、标志生成和时钟提交连成同一条连贯路径。

图示：ALU 章节的总图。箭头表示信息流向：固定 bit 先被解释，再进入组合计算，最后由控制器决定是否提交。

一、固定宽度 bit 的数值解释

硬件只保存 bit。所谓整数、正负号、小数点、十进制数字、溢出、大小比较，都是对同一组 bit 模式的解释规则。若不先讲清楚表示，后面的加法器、减法器、比较器和 ALU 就会像在做抽象数学；实际上它们处理的始终是固定宽度的一组电线。位宽一旦确定，硬件能保存的模式数量也就确定：4-bit 有 16 种模式，8-bit 有 256 种模式，32-bit 有 2^{32} 种模式。显示成十进制、十六进制还是二进制只是记法，不能改变硬件位宽。

4-bit 无符号数 $b_3 b_2 b_1 b_0$ 的值是：

$$\text{value} = b_3 \times 8 + b_2 \times 4 + b_1 \times 2 + b_0$$

因此 1011 表示 11，因为它等于 $8+0+2+1$ 。无符号数没有负数概念，所有 bit 都贡献非负权重。4-bit 能表达的范围是 0 到 15，8-bit 能表达的范围是 0 到 255。一个 4-bit 加法器只能输出 4 个结果 bit，再加上可能的最高位外进位。若

15 加 1，低 4 位回到 0000，最高位外产生进位。这个进位不是“可有可无的第 5 位”，而是无符号运算超出 4-bit 范围时硬件给出的明确信号。

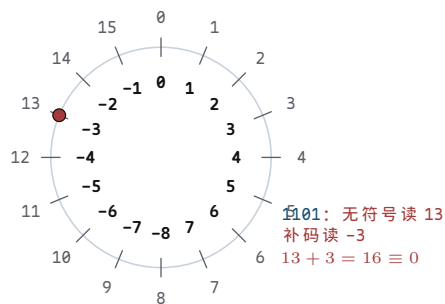
位宽	模式数量	无符号范围	典型硬件含义
4-bit	16	0 到 15	一片 4-bit 加法器、一个十六进制数字
8-bit	256	0 到 255	早期微处理器常见数据宽度、一个字节
16-bit	65536	0 到 65535	8086 通用寄存器和 ALU 的核心宽度
32-bit	2^{32}	0 到 $2^{32} - 1$	80386 扩展后的通用寄存器与地址计算宽度

补码表示让负数也能走同一个加法器。以 4-bit 为例，最高位不再只是 8，而是解释成负权重 -8：

$$\text{value} = b_3 * (-8) + b_2 * 4 + b_1 * 2 + b_0$$

于是 1111 的值是 $-8+4+2+1=-1$ ，1110 的值是 -2，1000 的值是 -8。4-bit 补码范围是 -8 到 7。补码范围不对称，因为 0000 已经表示 0，负数一侧多出一个最小值。这个细节很重要：在固定宽度内，最小负数没有对应的正数。例如 4-bit 的 -8 取相反数仍会得到 1000，这不是数学规则失效，而是硬件位宽不足。

为什么负权重成立？补码不是人为规定“最高位算负数”，而是模运算的自然结果。4-bit 加法器算的是模 16 加法：结果只保留低 4 位，超出 16 的部分被丢弃。在模 16 的世界里，-3 和 13 是同一个数，因为 $13 + 3 = 16 \equiv 0$ 。所以“存 -3”就是“存 13”，即 1101。同一根线上的 bit 模式于是有了两种读法：无符号读作 13，有符号读作 -3。把最高位解释成 -8 而不是 +8，两种读法就对上了： $8 + 4 + 1 = 13$ ， $-8 + 4 + 1 = -3$ ，而 $13 \equiv -3 \pmod{16}$ 。负权重不是新算术，只是给同一个模运算换了一个名字区间：0 到 15 换成 -8 到 7。



4-bit 模 16 数环：外圈无符号，内圈补码

图示：补码的数环读法：同一个 bit 模式，外圈是无符号读数，内圈是补码读数。加法器只按模 16 进位，两种读法共享同一条进位链——这就是为什么加法和减法能共用一套硬件。

“取反加一”也不再是口诀，而是一行证明：对任意 x ，都有 $x + \sim x = 1111 = -1$ （模 16 下各位互补，相加必得全 1），所以 $-x = \sim x + 1$ 。硬件只需要一排 NOT 门和一条加法进位链，减法就被归并成了加法。

求一个补码数的相反数，可以按“取反加一”理解：

```
x      = 0011 // 3
~x     = 1100
~x + 1 = 1101 // -3 in 4-bit two's complement
```

这个规则适合硬件，因为取反可以用一排 NOT 或 XOR 门完成，加一可以交给加法器。于是减法可以变成加法：

$$a - b = a + (\text{NOT } b) + 1$$

这条公式是 ALU 设计的关键。硬件不必为加法和减法各放一套完全独立的算术电路，只要让控制信号同时决定 B 操作数是否反相，以及最低位进位输入是否为 1，就能在同一条进位链上完成 ADD 和 SUB。第一章已经讲过受控电流路径，本章把这个思想推进到算术层面：**控制信号改变的是电路连接和输入选择，不是纸面公式。**

原码、反码、补码是三种被真实采用过的负数表示 补码不是唯一被试过的方案。早期计算机还认真使用过两种更直觉的表示：原码把最高位专门用作符号位，其余位保存绝对值；反码把正数逐位取反得到对应负数；补码则在反码基础上再加一。三种方案用 8-bit 都能表示 -5，但硬件代价差别很大。先把同一个负数的三种写法并排摆开：

表示法	负数规则	-5 的写法	零的写法
原码	最高位记符号，其余位记绝对值	1000 0101	0000 0000 与 1000 0000 两个零
反码	正数逐位取反	1111 1010	0000 0000 与 1111 1111 两个零
补码	逐位取反再加一	1111 1011	唯一的 0000 0000

补码最终胜出，三条原因都直接对应门电路的开销。第一，加法会自然处理符号。原码做加减前必须先比较两数符号和绝对值大小，决定实际执行加法还是减法，再单独确定结果符号——符号位不能参与进位，需要一条独立的逻辑路径。补码把符号位当成普通位一起进位： $1111\ 1011 + 0000\ 0101 = 1\ 0000\ 0000$ ，最高位进位被位宽丢弃，低 8 位就是 0， $(-5)+5$ 在普通加法器上一次算对，不需要任何符号特例。第二，零唯一。原码和反码都有 +0 和 -0 两种模式，“结果是否为零”要检测两种 bit 模式，判零电路多一级组合逻辑；补码只有一种零，Zero 标志就是对所有结果 bit 做一次 NOR。第三，取负便宜且单向。反码取负确实只要一排 NOT 门，但它的加法在产生最高位进位时必须把进位绕回最低位再加一次，这叫循环进位 (end-around carry)，进位链因此多绕一圈；补码取负是“取反加一”，NOT 加已有进位链即可完成，进位始终朝一个方向走。

循环进位的差别用 $(-5)+7$ 演算一次。反码： $1111\ 1010 + 0000\ 0111 = 1\ 000\ 0\ 0001$ ，最高位的进位 1 必须加回最低位， $0000\ 0001 + 1 = 0000\ 0010$ ，才得到 2。补码： $1111\ 1011 + 0000\ 0111 = 1\ 0000\ 0010$ ，进位直接丢弃，低 8 位 $0000\ 0010$ 就是 2。一次加法里反码要等进位回绕，补码只走一趟。

这三条性质的意义超出表示法本身：补码把符号处理、判零和取负全部归并到同一条进位链上，加减法、比较和取负共用一套硬件。历史上 UNIVAC 1100 系列等机器沿用反码，而 x86、ARM、RISC-V、MIPS 的有符号整数一律是补码。前面“减法复用加法器”的全部便利，本质上都来自这张表最后一行。

同一组 bit 可以按无符号解释，也可以按补码解释。因此溢出判断也不同。

解释方式	关注点	例子
无符号	是否产生最高位外的进位	15 + 1 超出 4-bit 无符号范围
补码有符号	正正得负或负负得正	7 + 1 得到 1000，被解释为 -8

以下 4-bit 加法给出同一结果在两种解释下的差异：

$$0111 + 0001 = 1000$$

若按无符号解释，这是 $7+1=8$ ，仍在表示范围内。若按补码解释，0111 是 7，0001 是 1，结果 1000 是 -8，正数加正数却得到负数，所以发生有符号溢出。硬件不会替使用者决定哪种解释成立；它只提供结果 bit、最高位外进位、符号位变化和溢出标志。程序、指令语义和控制器必须约定当前采用哪一种解释。

溢出判定演算：四个 8-bit 例子覆盖全部标志组合 把“正正得负、负负得正”推进到 8-bit，用四个例子走完整数值过程。8-bit 补码范围是 -128 到 127，无符号范围是 0 到 255：

```
+100 + +50: 0110 0100 + 0011 0010 = 1001 0110
              signed: +150 read as -106   WRONG (overflow)
              Cout = 0, Overflow = 1

-100 + -50: 1001 1100 + 1100 1110 = 1 0110 1010
              signed: -150 read as +106   WRONG (overflow)
              Cout = 1, Overflow = 1

+100 + -50: 0110 0100 + 1100 1110 = 1 0011 0010
              signed: +50                  correct
              Cout = 1, Overflow = 0

-100 + +50: 1001 1100 + 0011 0010 = 1100 1110
              signed: -50                  correct
              Cout = 0, Overflow = 0
```

判定规则可以总结成一句：**两个同号数相加，得到异号结果，即补码溢出**。写成布尔式，若 a_7 、 b_7 、 r_7 是两个输入和结果的符号位，则 $V = a_7 b_7 \bar{r}_7 + \bar{a}_7 \bar{b}_7 r_7$ 。硬件里还有一个更便宜的等价判法：进入符号位的进位 C_7 与最高位向外的进位 C_{out} 不一致就是溢出， $V = C_7 \oplus C_{out}$ 。用第一个例子验证：逐位进位 (c_1 到 c_8) 是 0,0,0,0,0,1,1,0，进入符号位的进位是 1、向外进位是 0，两者不同， $V = 1$ ，与符号判法一致。

为什么异号相加永不溢出？一个正数加一个负数，结果必然落在两个加数之间：和的绝对值不超过两数绝对值中较大的那个，而那个加数本身就在 -128 到 127 之内。范围内加范围内得到范围外，只有在两个加数同号、绝对值同向叠加时才可能发生。减法的判定也归并到同一条规则：把它看成加上取负后的数，于是“异号相减且结果符号与被减数不同”就是溢出。

Carry 和 Overflow 是两个相互独立的判断，上面四个例子恰好把 (C_{out}, V) 的四种组合 (0,1)、(1,1)、(1,0)、(0,0) 都走了出来。Carry 报告的是“bit 模式超出无符号范围”，即模 2^8 回绕发生；Overflow 报告的是“补码真值超出有符号范围”。第二个例子里两者同时为 1：无符号读是 $156 + 206 = 362$ 超界，补码读是 $-100 + (-50) = -150$ 超界。第三个例子里 Carry=1 但 $V=0$ ：无符号读 $100 + 206 = 306$ 回绕，补码读 +50 完全正确。硬件不替程序选择哪种读法，所以两个标志都必须产生。

演算	输入符号	结果符号	Cout	V	读法结论
+100 + +50	同号 (正)	负	0	1	补码溢出；无符号 150 仍正确
-100 + -50	同号 (负)	正	1	1	补码溢出；无符号 362 也超界
+100 + -50	异号	正	1	0	补码正确；无符号发生回绕
-100 + +50	异号	负	0	0	两种读法都正确

两个标志的用途也因此分开：多字长加法把低字节的 Carry 送进高字节的 C_{in} ，有符号范围检查看 Overflow。x86-64 把两者分别放在 CF 和 OF，条件转移各走各

的：

```
; x86-64, Intel syntax
mov al, 100
add al, 50      ; AL = 0x96, CF = 0, OF = 1
jo  too_big    ; signed path checks OF

mov al, 200
add al, 100     ; AL = 0x2C, CF = 1, OF = 0
jc  carry_in   ; unsigned path checks CF
```

这段分工的意义在于：标志位不是“结果好或坏”的笼统指示，而是两种解释规则各自的边界信号。控制器和编译器按当前约定取其中一个，另一个照常产生但可以被忽略。

位宽变化也要有明确规则。把 4-bit 值送入 8-bit 数据通路时，若它是无符号数，应做零扩展；若它是补码有符号数，应做符号扩展。把 8-bit 值截断成 4-bit 时，高位被丢弃，剩余低 4 位不一定仍表示原值。

动作	输入解释	输出规则	示例
零扩展	无符号	高位补 0	1011 -> 00001011
符号扩展	补码有符号	高位复制符号位	1011 -> 11111011
截断	固定低位	只保留低位	10011011 -> 1011
饱和	特定 DSP/多媒体语义	超出范围钳到最大/最小	需要额外比较和选择

符号扩展不是显示格式，而是真实硬件路径。若 4-bit 的 1011 按补码解释为 -5，扩展到 8-bit 后必须成为 11111011，仍然表示 -5；若错误地零扩展为 00001011，它就变成 11。这个错误在地址偏移、短立即数、分支位移和加载有符号字节时都可能出现。8086、80386 这类处理器的指令语义中会明确区分零扩展、符号扩展和普通截断，因为这些规则直接决定 ALU 输入。

字节、字和字节序把数值解释放进内存 体系结构教材常从“一个对象在内存中占哪些字节”开始训练机器级思维。硬件寄存器里可以一次保存 16-bit、32-bit 或 64-bit 值，但主存通常按字节编址。于是一个 32-bit 数值放进内存时，必须规定四个字节按什么顺序排列。若数值为 0x12345678，大端序把最高有效字节放在最低地址，小端序把最低有效字节放在最低地址。两者保存的是同一个 32-bit 模式，只是地址到字节车道的映射不同。

字节序	低地址到高地址	硬件含义
大端序	12 34 56 78	低地址保存最高有效字节，适合按书写顺序观察十六进制
小端序	78 56 34 12	低地址保存最低有效字节，低位字节扩展和多精度运算方便

字节序不是“显示问题”，而是 LOAD/STORE、字节使能和总线车道的硬件约定。CPU 执行 32-bit LOAD 时，内存系统返回四个相邻字节，处理器内部再按本机字节序把它们拼成寄存器值。CPU 执行 8-bit LOAD 时，只取其中一个字节；若目标寄存器更宽，还要按指令语义做零扩展或符号扩展。若把这三件事混在一起，调试时会出现非常典型的错误：内存窗口看见的字节顺序和寄存器中显示的十六进制值不一致，看起来像“读错了”，实际可能只是字节序解释不同。

```
memory bytes at increasing addresses:
```



```

0x1000: 78
0x1001: 56
0x1002: 34
0x1003: 12

little-endian 32-bit load from 0x1000 -> 0x12345678
8-bit load from 0x1000                -> 0x78

```

不同宽度的访问有各自必须写清的规则。字节 LOAD 选择一个字节车道，再扩展到寄存器宽度——零扩展还是符号扩展必须由指令规定；半字 LOAD 选择两个相邻字节并按字节序拼接，地址低位和对齐规则必须满足；字 LOAD 选择四个字节拼成 32-bit 值，小端/大端、字节使能和未对齐处理都要写清；字节 STORE 只打开一个字节写使能，绝不能破坏同一字中的其他字节。

内存对象和指针本质上也是位模式 从硬件角度看，指针不是神秘类型，而是一个被解释为地址的固定宽度 bit 模式。数组、结构体、栈帧和缓冲区最终都归结为“起始地址 + 偏移”的地址形成路径。若数组元素是 4 字节整数，访问 $a[i]$ 时地址形成部件通常要计算 $base + i*4$ ；若结构体字段在对象内偏移 12 字节，LOAD/STORE 的地址就是 $base + 12$ 。这与前面的定点数类似：bit 本身不变，解释约定决定它是整数、地址、偏移还是控制字段。

把几类对象的硬件路径分开看：标量整数是固定位宽 bit 模式，存在寄存器或内存字节里，经 ALU 解释；指针是地址宽度 bit 模式，会进入地址形成、译码、cache/TLB 或总线；数组元素的地址是基址加缩放下标，由加法器、移位器或地址生成单元算出；结构体字段的地址是基址加固定偏移，通常由立即数扩展和 ALU 地址加法完成；栈对象的地址是栈指针加偏移，依赖 SP/RSP 类寄存器和调用约定。

这个视角能解释为什么越界访问危险。硬件不会自动知道“这个地址还属于数组”。若没有保护机制， $a[10]$ 只是另一次地址加法和一次 LOAD/STORE，它可能指向相邻变量、返回地址、MMIO 窗口或未映射区域。后续章节讨论 80386 分页、IOMMU 和异常时，本质上就是把“哪些地址可访问、按什么权限访问”提升为硬件可检查的约定。

小数不是新电路，而是新的位权解释 整数位的权重是 1, 2, 4, 8, ...，小数位只是把二进制小数点右侧的权重继续向下分成 $1/2, 1/4, 1/8, \dots$ 。例如：

```

10.101_2 = 1*2 + 0*1 + 1*(1/2)
          + 0*(1/4) + 1*(1/8)
          = 2.625

```

这里的“小数点”不是一根额外电线。若寄存器里只保存 10101，硬件本身看见的仍然是五个 bit；把它解释成 21 、 10.101_2 ，还是某种编码字段，取决于电路和指令约定。这个事实会直接影响硬件设计：二进制小数的加法仍可以复用整数加法器，但必须保证两个操作数的小数点位置相同；若小数点位置不同，必须先对齐。

二进制小数和十进制小数并不总能互相有限表示。 0.5 可以写成 0.1_2 ，因为它正好是 $1/2$ ； 0.25 可以写成 0.01_2 ，因为它正好是 $1/4$ 。十进制的 0.1 则不能用有限个二进制小数位精确表示，只能形成无限循环展开。计算器、编译器和 FPU 不是“不认真”，而是受位宽限制：寄存器只有固定数量的 bit，必须在某个位置舍入。

十进制值	二进制表示	是否有限	硬件含义
0.5	0.1_2	有限	一位小数即可精确保存
0.25	0.01_2	有限	权重为 $1/4$ 的 bit 置 1
0.625	0.101_2	有限	$1/2 + 1/8$

0.1	无限循环	不能有限表示	固定位宽内必须近似和舍入
-----	------	--------	--------------

定点数把小数点位置变成硬件约定 定点数把“小数点在第几位”固定下来。常见写法是 Q 格式（沿用早期 DSP 手册用字母 Q 标记小数位数的习惯）：Q4.4 表示 8-bit 总宽度中有 4 个整数相关位和 4 个小数位，实际值等于原始整数除以 2^4 。例如 00011000 的无符号整数值是 24；若按 Q4.4 解释，它的实际值是 $24/16 = 1.5$ 。同一串 bit 并没有改变，改变的是缩放约定。

原始 bit	原始整数	定点解释	说明
00011000	24	Q4.4 中为 1.5	小数位数为 4，除以 16
00000110	6	Q1.7 中为 0.046875	小数位数为 7，除以 128
11110000	-16	有符号 Q4.4 中为 -1.0	先按补码得 -16，再除以 16
01111111	127	有符号 Q1.7 中接近 0.9921875	最大正值小于 1

定点加减最适合用现有整数 ALU。只要两个数采用相同 Q 格式，硬件可以直接把原始 bit 送入加法器；结果仍按同一个缩放因子解释。例如 Q4.4 中的 $1.5 + 0.25$ 可以看成原始整数 $24 + 4 = 28$ ，结果 00011100 解释为 $28/16 = 1.75$ 。因此许多音频、控制、电机、传感器和早期图形场景会优先使用定点数：整数加法器、移位器和乘法器即可承担主要工作。

定点乘法需要多一步处理。若两个 Q4.4 数相乘，原始整数相乘后缩放因子变成 2^8 ，因为两个 2^4 相乘得到 2^8 。要把结果放回 Q4.4，通常要右移 4 位，并决定被丢弃的低位是截断、四舍五入还是参与饱和判断。

```
Q4.4 value 1.5  -> raw 24
Q4.4 value 0.5  -> raw 8
raw product      -> 24 * 8 = 192
rescale to Q4.4 -> 192 >> 4 = 12
Q4.4 result      -> 12 / 16 = 0.75
```

这个例子说明定点不是“比浮点低级”的临时方案，而是一种明确的硬件选择。它牺牲动态范围，换来较低延迟、较小面积和更容易预测的执行时间。若电路要实时控制电机电流、滤波音频采样或处理传感器数据，固定缩放因子往往比复杂浮点更合适。

许多微控制器并不一定带硬件 FPU，或者即使带 FPU，也会在中断、功耗和实时性严格的路径上使用整数和定点运算。DSP 芯片则常见乘加单元，也就是 MAC 路径，用于反复执行“乘法、对齐、累加、饱和”。这些芯片的手册通常会明确给出 Q 格式、饱和模式、舍入模式和累加器宽度，因为它们决定滤波器、控制环和音频处理是否稳定。

定点乘法先做整数乘，再把小数点移回约定位置 两个 Q4.4 数相乘时，硬件看见的仍然只是两个 8-bit 整数。完整走一遍 1.5×2.25 ：

```
1.5   -> raw 24 = 0001 1000   (24 / 16 = 1.5)
2.25  -> raw 36 = 0010 0100   (36 / 16 = 2.25)

integer product: 24 * 36 = 864
16-bit accumulator: 0000 0011 0110 0000
read as Q8.8: 864 / 256 = 3.375
```

```
rescale to Q4.4: 864 >> 4 = 54 = 0011 0110
check: 54 / 16 = 3.375 = 1.5 * 2.25
```

为什么乘完必须移动小数点？每个操作数的 raw 值都是“真值 $\times 2^4$ ”，整数乘积天然携带 $2^4 \times 2^4 = 2^8$ 的缩放。若把 864 直接按 Q4.4 读，得到的是 54，恰好是真值的 16 倍。小数点在电路里不是实体，而是“缩放因子是多少”的约定；乘法把两个约定相乘，所以必须右移 4 位把约定改回 Q4.4。若目标是更宽的累加器，也可以不立即移位，按 Q8.8 继续参与后续乘加，等写回存储前再统一缩放——缩放时机是设计选择，缩放动作不可省略。

被右移丢弃的低位是真实的信息损失，处理方式必须写清。直接 $>> 4$ 是截断： 1.1875×1.1875 的 raw 乘积是 $19 \times 19 = 361$ ， $361 >> 4 = 22$ ，即 1.375，而真值是 1.41015625。若要在 Q4.4 内就近舍入，右移前先加半个最低位的权重，即加 1000₂： $(361 + 8) >> 4 = 23$ ，即 1.4375，距真值更近。两种做法不是对错问题，而是误差分布的约定：控制环在意单向误差累积，音频在意舍入噪声，芯片手册会把所选模式写明。

溢出检查同样在移位这一步完成。两个 8-bit 数相乘必须用 16-bit 暂存；右移 4 位后，若只取低 8 位作为 Q4.4 结果，则 bit 15 到 bit 8 必须全是 bit 7（新符号位）的复制，否则值已经超出 8-bit Q4.4 的范围。例如 7.9375×7.9375 ：raw 乘积 $127 \times 127 = 16129 = 0011\ 1111\ 0000\ 0001$ ，右移 4 位得 $0000\ 0011\ 1111\ 0000$ ；此时 bit 7 为 1，而 bit 15 到 bit 8 是 $0000\ 0011$ ，不是全 1，判定溢出——真值约 63.004 远超有符号 Q4.4 的上限 7.9375。此时要么置溢出标志交给软件，要么按饱和约定钳到 $0111\ 1111$ (7.9375)。

步骤	数值或位模式	缩放约定与检查点
解释输入	raw 24 与 raw 36	各带 2^4 缩放，两数小数位数必须一致
整数相乘	$24 \times 36 = 864$	缩放变为 2^8 ，需要双倍位宽暂存
右移还原	$864 >> 4 = 54$	回到 Q4.4；右移前加 8 可就近舍入
溢出检查	高位必须只是符号复制	超界时置标志，或饱和到最大/最小值

真实 DSP 把这条路径做成了专门部件。MAC（乘加）单元内部保留比输出格式宽得多的累加器，例如 32-bit 或 40-bit，让一连串“乘、加、乘、加”在中间过程不右移、不丢位，只在最终写回时做一次缩放、舍入和溢出检查。累加器多出来的高位常叫保护位（guard bits），作用正是推迟溢出判定：中间和允许暂时超出输出范围，只要最终结果能装回目标格式。这再次说明定点不是“简化版浮点”，而是一套把缩放时机、舍入模式和溢出策略全部显式写出来的工程约定。

十进制与 BCD 解决的是输入输出和精确十进制表示 二进制适合硬件算术，但人们日常习惯使用十进制。BCD，即 binary-coded decimal，用 4 个 bit 保存一个十进制数字。最常见的 8421 BCD 中，0000 到 1001 表示 0 到 9，1010 到 1111 不是合法十进制数字。两位十进制 59 可以保存为 0101 1001，而不是普通二进制整数 59 的 00111011。

表示	示例	适用问题
普通二进制	59 为 00111011	ALU 运算紧凑，位宽利用率高
8421 BCD	59 为 0101 1001	每个十进制位可直接显示和校正
非法 BCD	1010 到 1111	需要检测，不能当作十进制数字

BCD 加法的关键在于十进制校正。若一个半字节相加结果超过 9，或者低半字节产生了十进制意义上的进位，需要加 0110 把结果调整回合法 BCD。例如 $8 + 7 = 15$ ，普通二进制低 4 位是 1111，这不是合法 BCD 数字；加 0110 后得到 1 0101，表

示十进制进位 1 和低位数字 5。早期处理器保留辅助进位、十进制调整等机制，往往就是为了兼容十进制输入输出、计算器和商业数据处理。

8086 继承了面向 BCD 的十进制调整指令和辅助进位标志，原因不在于 BCD 比二进制 ALU 更快，而在于早期商业计算、显示和键盘输入大量使用十进制数字。现代通用 CPU 的主路径通常不会为 BCD 牺牲整数流水线，但十进制浮点、金融数据库和显示驱动仍会在特定边界上关心十进制精确性。

浮点数把范围和精度分开 定点数的小数点位置固定，动态范围有限。浮点数采用另一种约定：用一个符号位表示正负，用指数字段表示尺度，用有效数字字段表示精度。可以把它理解成硬件版科学计数法。一个常见规格化二进制浮点数可写成：

$$\text{value} = (-1)^{\text{sign}} * 1.\text{fraction} * 2^{(\text{exponent} - \text{bias})}$$

其中 **bias** 是阶码偏置，用来让指数字段以无符号 bit 保存正负指数。有效数字前面的 1 通常不显式保存，这叫隐藏位。浮点格式还要为特殊值保留编码：正负零、非规格化数、正负无穷大和 NaN。非规格化数用于靠近 0 的渐进下溢；无穷大用于表示超出范围或除以 0 一类结果；NaN 表示“不是一个数”的异常传播，例如 0/0 或无效运算。

把这条公式落实到最常见的格式上。IEEE 754 单精度用 32 个 bit，分成三段：

$$\text{值} = (-1)^s \times 1.f \times 2^{e-127}$$

s	阶码 e (8 位, bit 30-23)	尾数 f (23 位, bit 22-0)
---	-----------------------	-----------------------

按无符号保存, bias=127: e=127 表示 2^0

规格化时前导 1 隐藏, 不占 bit

图示：IEEE 754 单精度的 32-bit 布局：1 位符号、8 位阶码、23 位尾数。阶码偏置 127，所以 e=1 表示 2^{-126} ，e=254 表示 2^{127} ；e=0 和 e=255 被保留给非规格化数、零、无穷大和 NaN。

两个完整的解码演算，胜过十遍公式：

```
0x3F800000:
s = 0
e = 0111 1111 = 127 -> exponent = 127 - 127 = 0
f = 000...0 -> significand = 1.0
value = +1.0 * 2^0 = 1.0
```

```
0x40490FDB:
s = 0
e = 1000 0000 = 128 -> exponent = 1
f = 100 1001 0000 1111 1101 1011
    significand = 1.5707963...
value = 1.5707963 * 2^1 = 3.1415926...
```

对阶加法也用一个真实位宽走一遍。计算 $2^{10} + 2^{-14}$ ：两个数指数相差 24，较小数的尾数必须右移 24 位才能对齐；但单精度尾数只有 23 位，被移出的最低那个 1 落在保留位宽之外，只能按舍入规则处理——结果舍入回 2^{10} 。 2^{-14} 这次加法在硬件里真实发生了，却被格式位宽“吃掉”了。

```
2^10 + 2^-14 in binary32:
align: shift small significand right by 24
23-bit field cannot keep the shifted-out bit
result rounds back to 2^10
```

四个字段各自的硬件动作也要分清：符号位进入符号处理和比较路径，不能直接等同整数的最高位；指数字段决定数值尺度，加减前要对阶、乘除时参与指数加减，

风险是范围溢出或下溢；有效数字是精度来源，进入尾数加法、乘法和规格化，风险是位数有限导致舍入；特殊值（零、非规格化、Inf、NaN）需要旁路和异常规则，比较、传播和转换时都要查清语义。

阶码的两个端点被保留给零、次正规数、Inf 和 NaN 单精度阶码 1 到 254 留给规格化数，全 0 和全 1 两个端点另有约定： $e = 0$ 表示零和次正规数， $e = 255$ 表示无穷大和 NaN。于是一个 32-bit 模式先按阶码分五类，再看尾数细分：

类别	阶码 e	尾数 f	值	十六进制例
+0	全 0	全 0	+0	0x00000000
-0	全 0	全 0	-0 ($s=1$)	0x80000000
次正规数	全 0	非 0	$(-1)^s \times 0.f \times 2^{-126}$	0x00000001 为 2^{-149}
$\pm\text{Inf}$	全 1	全 0	$(-1)^s \times \infty$	0x7F800000 为 $+\infty$
NaN	全 1	非 0	不是一个数	0x7FC00000

次正规数 (subnormal, 也叫非规格化数) 处理下溢一侧的边界。规格化数依赖隐含的前导 1, 能表示的最小正数是 $e = 1, f = 0$ 的 1.0×2^{-126} (0x00800000); 比这更小的数装不下隐含位。于是标准约定 $e = 0$ 时隐含位变成 0, 指数固定为 -126, 值为 $0.f \times 2^{-126}$ 。f 的 23 个 bit 里最小非零值是末位的 1, 对应 $2^{-23} \times 2^{-126} = 2^{-149}$, 这就是单精度最小正数。关键在于过渡方式: 从 2^{-126} 继续向下, 表示能力不是悬崖式掉到 0, 而是有效位一位一位减少、精度逐渐丢失——这称为渐进下溢 (gradual underflow)。典型产生场景是两个接近的极小规格化数相减, 差落在规格化范围之下。

无穷大覆盖另一端的边界, 产生路径有两条。一条是真除零: IEEE 语义下 $1.0/0.0$ 得 $+\infty$ 、 $-1.0/0.0$ 得 $-\infty$, 同时置除零标志, 默认不停机。另一条是溢出: 结果的指数超过单精度上限 (约 3.4×10^{38}), 例如 $10^{38} \times 10$, 按当前舍入模式变成 Inf 或最大有限值。Inf 参与运算有定义好的结果: $1/\infty = 0$, $\infty + x = \infty$, 比较时大于一切有限值。流水线因此不必为超界结果插入特例停顿。

NaN 表示“没有定义”的运算结果: $0/0$ 、 $\infty - \infty$ 、 $0 \times \infty$ 、对负数开平方都产生 NaN。硬件必须实现它的两条语义。一是传播: NaN 参与几乎所有运算, 结果仍是 NaN (尾数字段通常原样传下去), 异常不会在长算式中途中被静默吞掉。二是比较: NaN 与任何值比较都为不相等, 包括它自己, 所以 $x == x$ 为假即可检出 NaN——这条性质被编译器和数学库直接利用。还要注意零的符号: $+0$ 与 -0 比较相等, 但 $1/(+0) = +\infty$ 、 $1/(-0) = -\infty$, 符号位在除法中仍然可观察; 负数下溢向零舍入时也会产生 -0 。

特殊值的意义在于让浮点成为封闭系统: 任意 32-bit 输入、任意运算, 都有定义好的 32-bit 输出, 控制逻辑无需为异常组合停下流水线。代价是 FPU 必须为阶码全 0 和全 1 各准备一条旁路, 译码、比较、类型转换和舍入都要先查这两类模式——这正是前面“特殊值需要旁路和异常规则”的具体内容。

浮点加法比整数加法复杂得多。两个整数相加, 只需对齐同名 bit 并沿进位链传播; 两个浮点数相加, 通常要先比较指数, 把较小数的有效数字右移对齐, 再做加减, 随后规格化结果、舍入, 并检查异常标志。若一个很大的数和一个很小的数相加, 小数可能在对阶右移时被移出保留位宽之外, 结果看起来像“小数被吃掉”。这不是软件显示问题, 而是 FPU 内部位宽和舍入规则的结果。

```
large = 1.0 * 2^30
small = 1.0
large + small may round back to large
when the significand has too few bits.
```

一次浮点加法要走完对阶、相加、规格化、舍入四步 FPU 的加法器不是一条进位链, 而是一条固定次序的小流水线。用 $1.5 + 0.078125$ 把四步完整走一遍。先把两

个十进制数写成规格化二进制，再写成 binary32:

```
1.5      = 1.1_2 * 2^0
          s=0  e=0111 1111 (127)  f=100 0000 ... 0000
          -> 0x3FC00000
0.078125 = 1.01_2 * 2^-4
          s=0  e=0111 1011 (123)  f=010 0000 ... 0000
          -> 0x3DA00000
```

第一步，对阶。两数指数不同，尾数不能直接相加：小数点位置不同的两个值必须先对齐，这与前面定点加法的前提完全一样。指数差是 $127 - 123 = 4$ ，规则是小阶向大阶对齐：把 0.078125 的有效数字右移 4 位，它的指数随之补到 127。为什么不许大阶向小阶对齐？那需要把大数尾数左移，最高的隐含位会移出保留位宽，丢的是最高有效位；右移小数丢的是最低有效位，损失最小。对阶后的 bit:

```
1.5:      1.1000 0000 0000 0000 0000 0000 * 2^0
0.078125: 0.0001 0100 0000 0000 0000 0000 * 2^0
          (significand shifted right by 4)
```

实际硬件在尾数右侧还会多留 guard、round、sticky 三种附加位，接住被移出的 bit，供最后一步舍入查询。

第二步，尾数相加。对齐后的两个 24-bit 有效数字（含隐含位）走普通整数进位链：

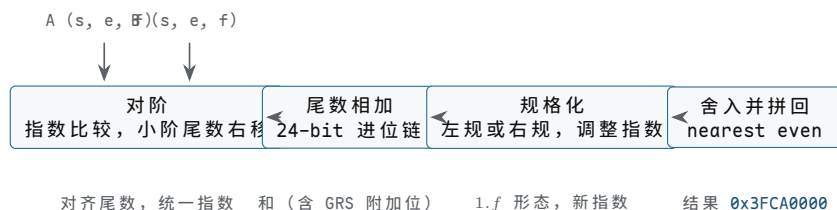
```
1.1000 0000 0000 0000 0000 0000
+ 0.0001 0100 0000 0000 0000 0000
= 1.1001 0100 0000 0000 0000 0000
```

第三步，规格化。和已经是 $1.x$ 形态，前导 1 在位，无需移动。（若和大于等于 2，例如 $1.5 + 1.5 = 11.0_2$ ，要右移 1 位并把指数加 1；若和小于 1，例如两个相近数相减，要左移直到前导 1 重新出现，指数相应减小。）

第四步，舍入。本例 23 位尾数恰好装下 1001 0100 ...，被舍弃部分为 0，不发生舍入，结果直接拼回三段：

```
s=0  e=0111 1111  f=100 1010 0000 ... 0000
-> 0x3FCA0000 = 1.578125 = 1.5 + 0.078125
```

把四步连成一张数据流图，每一步的输入和输出都是明确的字段，次序固定。



图示：浮点加法四步的输入输出：对阶按指数差把小阶尾数右移，移出位进入 guard、round、sticky 附加位；尾数相加走普通整数进位链；规格化把结果移回 $1.f$ 形态并同步加减指数；舍入按 round to nearest even 处理附加位后拼回 s/e/f 三段。四步次序固定，真实 FPU 把它们分到不同流水级。

舍入规则换一个必然触发它的例子看： $2^{24} + 1$ 。 2^{24} 的尾数是 1.000 ... 000，指数 24；1 对阶时要右移 24 位，它唯一的那个 1 被移到 23 位尾数之外。此时和的真值是 $2^{24} + 1$ ，恰好落在两个可表示值 2^{24} 和 $2^{24} + 2$ 的正中间——这个指数下尾数末位的权重 (ulp) 是 2。默认舍入模式是 round to nearest even：就近舍入，平局时取尾数末位为偶数的那个。 2^{24} 的尾数末位是 0，为偶，于是结果舍回 2^{24} ：这次加法真实执行了，加上的 1 却被格式位宽吃掉。若加的是 3， $2^{24} + 3$ 恰好落在 $2^{24} + 2$ 和 $2^{24} + 4$ 的正中间，到两边距离都是 1，又是一个平局：按取偶规则， $2^{24} + 4$ 的尾数末位是 0，为偶，于是结果舍到 $2^{24} + 4$ 。

这个机制解释了为什么大数加小数容易“吃掉”小数：尾数只有 23 位，ulp 随指数放大，小数的全部有效位可能都在 ulp 之下，对阶右移后被舍入规则抹平。这也意味着浮点加法的每一步都可能按当时的指数重新舍入。

由此也能理解浮点加法通常不满足严格结合律。 $(a+b)+c$ 和 $a+(b+c)$ 可能在不同中间步骤舍入，最终得到不同的最低几位。工程上不能把浮点数当成实数域的无限精确元素；必须知道格式位宽、舍入模式、异常处理和比较策略。

8087 x87 协处理器把浮点执行从 8086 主整数路径旁边分离出来，说明浮点硬件从一开始就有明显的复杂度边界。后来 x86 通过 SSE、AVX 等 SIMD 浮点/向量扩展，把多个浮点操作组织成并行数据通路；现代高性能 CPU 还会用流水线、转发、乱序调度和宽向量单元降低吞吐延迟。相反，许多小型 MCU 为了面积、功耗和确定性，仍然选择没有 FPU 或只提供较窄 FPU，由软件库、整数或定点路径承担一部分数值任务。

数值格式决定硬件代价和时延 同样是“加法”，整数、定点、BCD 和浮点的硬件路径并不相同。整数加法的核心是位对齐后的进位链；定点加法在格式一致时复用整数加法器；BCD 加法需要半字节合法性检测和十进制校正；浮点加法则增加对阶、尾数运算、规格化、舍入和特殊值处理。格式越灵活，硬件要承担的解释工作越多。

四种格式的取舍也因此不同。无符号/补码整数靠加法器、移位器和比较标志工作，简洁、快速、面积低，代价是范围固定、不能直接表示小数；定点用整数 ALU 加缩放、舍入和饱和，实时性好、适合 DSP 和控制场景，代价是尺度要人工管理；BCD 用二进制加法加十进制校正，十进制输入输出直接，代价是位宽利用率低、校正逻辑额外增加；浮点靠对阶、尾数、规格化、舍入和异常处理工作，动态范围大、通用科学计算方便，代价是面积、功耗、验证和延迟全面上升。

现代芯片能把很多数值运算做得很快，不是因为物理定律允许“没有延迟”，而是因为工程上同时用了多种手段。第一，常见路径被专门做成短关键路径，例如整数加法器采用超前进位或前缀结构，避免所有进位逐位等待。第二，复杂运算被流水线化，单次结果仍有若干周期延迟，但每个周期都能接收新操作，提高吞吐。第三，SIMD/向量单元把多个相同格式的数据并排处理，摊薄取指、译码和控制成本。第四，缓存、寄存器重命名、旁路转发和调度逻辑尽量让数据在核心附近流动，减少等待内存和长线传输的时间。

因此，“低时延”必须分清两种说法：**单次操作延迟**指一个结果从输入到可用需要多久，**吞吐率**指稳定状态下每个周期能完成多少个操作。现代浮点乘加单元可能有多个周期的单次延迟，但由于流水线很深，可以在每个周期发射新的乘加；整数 ALU 可能单次延迟更短，但若数据依赖形成长链，仍会受到前一条结果可用时间限制。读芯片资料时，应同时看 latency、throughput、数据格式、向量宽度、异常模式和是否支持饱和/舍入。

移位器也是算术部件。左移一位通常相当于乘以 2，但前提是移出去的 bit 没有携带需要保留的信息。右移更容易出差别。逻辑右移在高位补 0；算术右移保留符号位，用于补码有符号数。

```
logical right shift: 1000 >> 1 = 0100
arithmetic right shift: 1000 >> 1 = 1100
```

若 1000 按 4-bit 补码解释为 -8，算术右移得到 1100，也就是 -4；逻辑右移得到 0100，也就是 4。两个结果差异巨大。原因不是移位器“出错”，而是控制信号选择了不同硬件规则。循环移位又是第三种规则：移出的 bit 从另一端回填，并且常常与进位标志一起参与多字长位操作。

8086 的 FLAGS 寄存器把 Carry、Zero、Sign、Overflow 等标志暴露给分支、条件转移和多字长运算。80386 把通用寄存器和地址计算扩展到 32-bit 后，仍保留同类标志语义。这个延续说明：位宽可以扩大，执行单元可以复杂化，但 CPU 仍需要一组可复查的**结果标志**，把 ALU 的组合输出交给控制流和后续指令使用。

若想把本节内容搭到面包板上，可以先做一个 4-bit 数值解释实验：用 4 个拨码开关给出 $b_3..b_0$ ，用 4 个 LED 显示原始 bit，再用纸面表格分别写出无符号值和补码值。此时不需要任何复杂芯片，关键是训练一个习惯：LED 只显示 bit，数值来自解释规则。后面接入 74HC283 加法器、74HC86 反相控制和 74HC157 选择器时，仍然要保留这张解释表，否则很容易把结果 bit 与数学整数混为一谈。

二、从加法器到可复用 ALU

ALU 是算术逻辑单元。它不是孤立出现的大型部件，而是因为加法、减法、按位逻辑、移位和比较反复出现，硬件需要把这些能力集中到一个受控制的组合逻辑块中。ALU 的本质是：多个候选计算电路并排存在，控制信号选择本周期采用的结果，并同时产生可供控制器判断的标志位。

最小 ALU 可以写成接口：

```
ALU(a, b, op) -> result, flags
```

a 和 b 是两个操作数， op 是选择信号， $result$ 是结果， $flags$ 是结果附带的标志信息。这里的接口不是软件函数，而是一组并行电线：两个操作数总线进入 ALU，控制线进入选择网络，结果总线和标志线从 ALU 输出。

op	运算	硬件来源
ADD	$a + b$	加法器
SUB	$a - b$	加法器、B 反相控制、最低位进位
AND	$a \& b$	按位 AND 门阵列
OR	$a b$	按位 OR 门阵列
XOR	$a \text{ xor } b$	按位 XOR 门阵列
SHIFT	逻辑/算术/循环移位	移位器和补位选择逻辑
CMP	比较	通常复用减法路径和标志位

加法器从半加器开始。半加器接受两个 1-bit 输入，输出和位 sum 与进位 $carry$ ：

```
sum   = a xor b
carry = a and b
```

完整加法器还要接受来自低位的进位 cin ：

```
sum   = a xor b xor cin
cout  = (a and b) or (cin and (a xor b))
```

把多个完整加法器串起来，就得到串行进位加法器。最低位先计算，再把进位送到下一位。这个结构规则、容易画出、容易搭电路，也最能说明关键路径：最高位结果必须等待低位进位逐级传播。位宽越宽，最坏延迟越大。

这条层次链值得记住：半加器用 XOR 和 AND 解释一位相加的和位与进位；完整加法器在半加器上加入进位合成，构成多位串行进位链；四个完整加法器级联就是一片 4-bit 加法器，对应 74HC283/74LS283 一类芯片；ALU 加法路径则再加上输

入选择和标志生成，支持 ADD、SUB、地址计算和比较。第一章的门级实验、本章的 ALU 和后面 CPU 的数据通路，走的都是这一条路。

ALU 里最值得细看的是 SUB。若单独做减法器，会增加面积和连线。补码让它复用 ADD 的加法器：

```
if op = ADD:
    b_to_adder = b
    carry_in   = 0

if op = SUB:
    b_to_adder = NOT b
    carry_in   = 1

result = a + b_to_adder + carry_in
```

实际电路里，B 的每一位前面可以接 XOR 门：当 $sub=0$ ， $b \text{ xor } sub = b$ ；当 $sub=1$ ， $b \text{ xor } sub = \text{NOT } b$ 。最低位进位输入也由 sub 控制。这样 ADD 和 SUB 共用同一条进位链。

sub	B 输入处理	Cin0	实际运算
0	B 原样通过	0	$A + B$
1	B 每位取反	1	$A + (\sim B) + 1 = A - B$

这个小结构可以直接搭成可观察实验。实验不需要先做完整 CPU，只要把几个连接点分开搭建、逐一观察，就已经包含了 ALU 复用的核心：A/B 输入用拨码开关给出两个 4-bit 操作数，原始 bit 要和纸面数值解释一致；B 反相用 74HC86 的四个 XOR 门处理 B 的四位， $sub=0$ 时原样通过、 $sub=1$ 时逐位取反；进位输入把同一个 sub 接到最低位 Cin，加法时 $Cin=0$ 、减法时 $Cin=1$ ；结果输出让 74HC283 的输出接 LED 或逻辑探针， sub 切换时加法和减法结果都要可复现。

74HC283/74LS283 是常见 4-bit 二进制全加器，适合观察多位进位链；74HC86 提供四个二输入 XOR 门，适合做 B 输入的可控反相；74HC157 提供四路二选一选择器，可用于在逻辑结果和加法结果之间选择。若再接上 74HC173 或 74HC574 保存操作数和结果，就能组成一个可单步观察的 4-bit ALU 实验台。实验重点不是追求速度，而是把 sub 、Cin、结果 LED 和标志 LED 之间的关系看清楚。

ALU 内部可以并行产生多个候选结果，再用多路选择器按 op 选出最终 $result$ 。

```
add_result   = adder(a, b_to_adder, carry_in)
and_result   = a and b
or_result    = a or b
xor_result   = a xor b
shift_result = shifter(a, shift_amount, shift_kind)

result = mux(op, add_result, and_result,
             or_result, xor_result, shift_result)
```

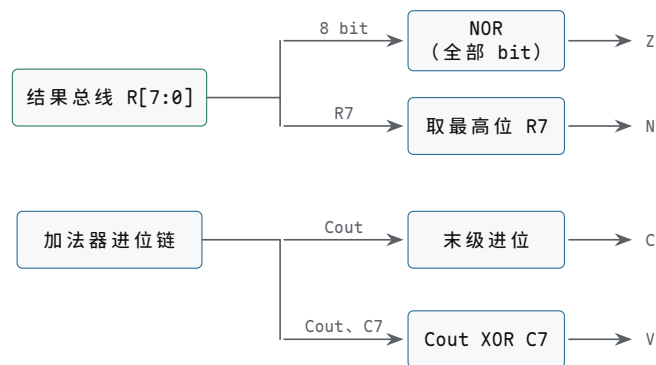
这不表示所有运算按软件顺序“算一遍”。组合逻辑是并行传播的：加法器、按位逻辑、移位器都可能同时产生自己的候选输出。最终哪一路被后级看见，由 mux 决定。代价也随之出现。候选电路越多，ALU 面积越大；mux 输入越多，选择延迟越大；若某个候选结果来自长进位链，它可能成为 ALU 的关键路径。

ALU 常见标志位包括：

标志位	来源	用途
Zero	结果所有 bit 做 OR 后再取反	相等判断、循环结束、条件选择
Carry	最高位外进位或借位约定	无符号溢出、多字长加减、无符号比较
Overflow	补码有符号运算的符号异常	有符号范围检查、有符号比较
Negative/Sign	结果最高位	补码符号判断、条件分支输入
Parity/辅助标志	特定低位统计或半字节进位	某些 ISA、BCD 或兼容性需求

Zero 可以由所有结果 bit 做 OR 后再取反得到。Negative 通常直接来自结果最高位。Carry 来自加法器最高位外的进位。Overflow 则要看有符号加减法中符号是否出现不可能的变化。对于加法，若两个输入符号相同而结果符号不同，则发生补码溢出；对于减法，若被减数和减数符号不同而结果符号与被减数不同，也发生补码溢出。标志位不是额外装饰，而是后续控制的重要输入。

四个标志的生成电路各不相同，但都与结果同时产生：它们并联在结果总线和进位链上，互不等待。



图示：四个标志并联生成，与结果同时有效：Z 是结果全部 bit 的 NOR；N 直接取结果最高位 R7；C 是进位链末级输出 Cout；V 用末两级进位 Cout XOR C7——最高位与次高位进位不一致即补码溢出。四根标志线随后送进标志寄存器，供条件分支查询。

动作	Carry/Borrow 解释	Overflow 解释	常见用途
ADD	最高位外进位，表示无符号超界	同号相加得到异号	无符号加法、多字长加法、有符号溢出检查
SUB	由 ISA 约定表示无借位或借位	异号相减且结果符号异常	无符号大小比较、有符号大小比较
CMP	通常执行减法但不写回结果	与 SUB 相同，只保留比较标志	条件分支、条件选择、循环判断
INC/DEC	有些 ISA 不更新 Carry，或按专门规则更新	仍按补码边界判断	地址递增、循环计数，必须查清 ISA 约定

这张表强调一个容易被忽略的事实：标志位不是全局数学真理，而是**指令语义与硬件标志的约定**。同样执行 $A-B$ ，相等判断只关心 Zero，无符号小于关心借位或 Carry 约定，有符号小于要结合 Sign 和 Overflow；判断 A 是否为 0，直接检查 A 或执行传递操作，看 Zero 标志即可。早期 x86、6502、Z80 等处理器都把这些标

志放进状态寄存器，但具体命名、是否更新、条件码如何解释并不完全相同。读芯片手册或写控制器时，必须先弄清“本条动作会更新哪些标志”。

多字长运算展示了 Carry 的实用价值。若只有 8-bit ALU，却要做 16-bit 加法，可以先加低 8 位，保存低字节结果和进位；再加高 8 位，并把低字节产生的 Carry 作为高字节加法的 Cin。早期 8-bit 处理器经常依靠这种方式完成更宽整数运算。软件看到的是 16-bit 加法，硬件实际执行的是多个周期内的低字节、高字节和进位链传递。

```
low_sum, carry0 = add8(A_low, B_low, 0)
high_sum, carry1 = add8(A_high, B_high, carry0)
result = high_sum:low_sum
```

把这个观察放到经典芯片中，可以看到同一问题的不同规模。6502、Z80、8086、80386 都必须解决 ALU 如何支持加法、减法、逻辑运算、移位和标志位。早期 8-bit 处理器更强调用较小数据通路和多周期控制完成动作；8086 把 16-bit 数据通路、段偏移地址形成和标志控制组织在更复杂的执行部件里，ALU 不只做算术，也服务地址计算和条件判断；80386 扩展到 32-bit 后，位宽、标志、地址计算和控制路径都随之加重，进位、布线、译码和时序压力同时上升；现代核心则把 ALU 复制到多个执行单元，用旁路、重命名和乱序调度喂养它们，但每条执行路径仍要满足时序约束。不同芯片的实现细节不同，但抽象不变：候选计算、选择器、进位路径、标志生成和时钟边界共同形成 ALU 行为。

超前进位按层级展开，进位选择用复制换等待 串行进位的瓶颈前面已经看过：16-bit 加法器的 C16 要等 16 段进位接力。超前进位 (carry-lookahead) 把每一位的进位方程 $C_{i+1} = G_i + P_i C_i$ 逐位代入，直到右边只剩 G、P 和 C0。4-bit 完整展开：

```
G_i = a_i AND b_i      (generate)
P_i = a_i XOR b_i      (propagate)

C1 = G0 + P0*C0
C2 = G1 + P1*G0 + P1*P0*C0
C3 = G2 + P2*G1 + P2*P1*G0 + P2*P1*P0*C0
C4 = G3 + P3*G2 + P3*P2*G1 + P3*P2*P1*G0 + P3*P2*P1*P0*C0
```

右边只依赖输入位和 C0，四个进位因此可以同时开算：一串代入被换成一片与-或两级逻辑。代价也写在式子里——项数和扇入随位宽增长，C4 的最后一项要 5 输入 AND，进位 OR 要收 4 项；位宽到 8 以上，单级展开的门扇入就不现实。所以更宽的加法器把同一条递推分层使用。

16-bit 的做法是 4 组 4-bit CLA 再加一层组间逻辑。每组内部按上面的展开并行产生组内进位；每组再输出两个组级信号：组产生 GG 表示“本组不管低位进位是什么，自己一定向外产生进位”，组传播 GP 表示“低位来什么进位，本组原样传出去”：

```
GG = G3 + P3*G2 + P3*P2*G1 + P3*P2*P1*G0
GP = P3*P2*P1*P0
```

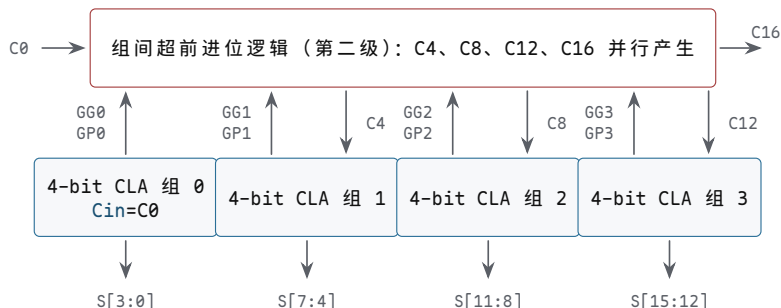
第二级用完全相同的递推处理四个组的 GG/GP，一次算出全部组间进位：

```
C4 = GG0 + GP0*C0
C8 = GG1 + GP1*GG0 + GP1*GP0*C0
C12 = GG2 + GP2*GG1 + GP2*GP1*GG0 + GP2*GP1*GP0*C0
C16 = GG3 + GP3*GG2 + GP3*GP2*GG1 + GP3*GP2*GP1*GG0
      + GP3*GP2*GP1*GP0*C0
```

延迟由此重排。串行 16-bit 是 16 段进位接力；两级超前的进位路径只剩约 5 段：位级 G/P 一级门，组级 GG/GP 生成与-或两级，组间进位与-或两级，组内进

位与-或两级，末位求和 XOR 一级。按级门延迟估算，串行约 $1 + 2 \times 16 = 33$ 级，两级超前约 $1 + 2 + 2 + 2 + 1 = 8$ 级—接力段数从 16 降到约 5。层级化的含义就是： $C = G + PC$ 这条递推在组内用一次、在组间再用一次；经典板级方案里，74182 超前进位发生器承担的正是组间这一层。

把 16-bit 的两级结构画成方块图，每个组对外的接口只剩 GG、GP 和组间进位三根线。



图示：16-bit 两级超前进位：四个 4-bit CLA 组各自生成组产生 GG 与组传播 GP 上报第二级；第二级用同一条 $C = G + PC$ 递推并行算出 C4、C8、C12、C16 发回各组（经典 74182 承担的正是这一层）。进位接力从串行 16 段降到约 5 段，约 8 级门。

进位选择加法器 (carry-select) 走另一条路。把 16-bit 分成 4 段，每段并排摆两套 4-bit 加法器：一套假设段进位输入为 0，另一套假设为 1，两份和同时算完。真实的段间进位到达时，不再穿过段内进位链，只驱动一层 mux，从两份候选里选一份。等待时间变成“第一段完整加法”加“其后每段一级 mux”，约等于 4-bit 加法加三级 mux，远小于 16 段接力。

代价同样直接：段内加法器接近两倍，再加一层与位宽成正比的 mux 阵列，面积显著上升。为什么仍值得？因为串行结构里“等进位”的那段时间被利用起来，把两种可能的分支都提前算完—选择代替了等待。**提前算出所有候选，再用真实信号选择**是硬件里反复出现的换时手段：超前进位用逻辑深度换时间，进位选择用电路复制换时间；位宽继续加大时两者常常组合使用（组内超前、组间选择），32-bit 和 64-bit ALU 的主加法器很少是单一结构。

16-bit 结构	进位等待	额外代价	适用场景
串行进位	16 段接力，约 33 级门	几乎无额外面积	教学实验、窄位宽
两级超前	约 5 段，约 8 级门	大扇入与-或逻辑、组间 PG 网络	ALU 主加法路径
进位选择	一段加法加每段一级 mux	段内加法器近两倍加 mux 阵列	高主频宽位加法

这三种结构后面还会与移位、比较、乘法一起再比较一次；这里先记住一点：加法器的位宽不是简单复制全加器，进位组织方式本身就是设计变量。

三、进位、移位、比较与乘法的硬件取舍

加法器是 ALU 的核心路径之一，因为减法、地址计算、数组下标、栈指针更新和分支目标计算都会反复使用它。最直观的结构是串行进位加法器：第 0 位产生的进位送到第 1 位，第 1 位再送到第 2 位，直到最高位。它面积小、结构规则，但最坏延迟随位宽增加而增长。4-bit 时容易接受，32-bit 或 64-bit 时就可能成为关键路径。

可以把每一位的进位逻辑归结为两类信号：

```
generate_i = a_i and b_i
propagate_i = a_i xor b_i
```

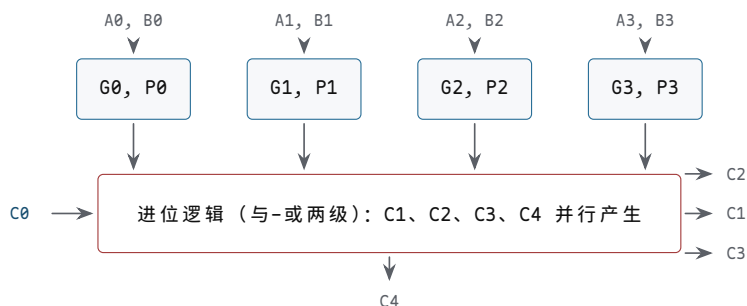
```
carry_out_i = generate_i or (propagate_i and carry_in_i)
```

若本位的 A 和 B 都为 1，本位会**产生**进位；若 A 和 B 有且仅有一个为 1，本位会**传递**低位进位。快速加法器的思想，就是用额外组合逻辑提前计算这些产生和传递关系，减少“等低位一位一位传上来”的时间。这说明速度不是免费获得的：更短的延迟通常换来更多门、更复杂连线 and 更严格布局。

把“提前计算”真正展开一次。以 4-bit 为例，把 $C_{i+1} = G_i \vee (P_i C_i)$ 逐位代入，每个进位都改写成只含 G、P 和 **C0** 的形式：

```
C1 = G0 + P0*C0
C2 = G1 + P1*G0 + P1*P0*C0
C3 = G2 + P2*G1 + P2*P1*G0 + P2*P1*P0*C0
C4 = G3 + P3*G2 + P3*P2*G1 + P3*P2*P1*G0 + P3*P2*P1*P0*C0
```

注意右边只依赖 A、B 和 **C0**：C4 不再等 C3，四个进位可以同时开算。这就是“超前进位”的全部含义——不是让进位消失，而是把进位链的串行代入改成并行的与或逻辑。



图示：4-bit 超前进位结构：四个 G/P 单元同时算出产生/传播信号，进位逻辑用与-或两级把 C1-C4 一次性算出来，不再串行等待。

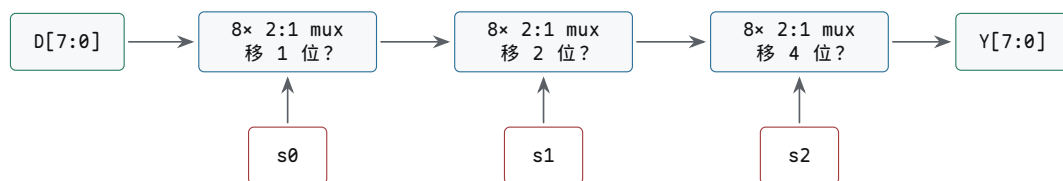
延迟对比按“级门延迟”估算（数量级示意，不代表具体工艺）：4-bit 串行进位，G/P 1 级加每位约 2 级进位逻辑再乘 4 级接力，最坏约 9 级；4-bit 超前进位，G/P 1 级、进位逻辑与-或 2 级、求和 XOR 1 级，最坏约 4 级。位宽越大差距越夸张：64-bit 串行要约 $1 + 2 \times 64 = 129$ 级，而树形前缀结构能把进位逻辑压到 $\log_2 64 = 6$ 层左右。这就是 64-bit ALU 不能只按“64 个全加器排队”估算性能的原因——真实高性能加法器都在用前缀或分组结构买这几级延迟。

几种结构的取舍由此而来：串行进位面积小、布线规则、容易扩展，代价是进位逐位传播、最坏延迟随位宽增长；超前进位预先计算产生/传递关系、缩短进位等待，代价是组合逻辑和布线更复杂；分组进位在面积和速度之间折中，代价是要设计分组边界和跨组进位；前缀加法器适合高性能宽位加法，代价是布线密集、功耗和布局压力较大。教学机可以先用串行进位把 trace 跑通，追求主频时再换超前或前缀结构。

现代芯片为什么能降低时延，不能简单归结为“晶体管更快”。在真实处理器里，低时延来自一组共同措施：把常用路径做短，把宽路径拆分成前缀或分组结构，把长组合逻辑放进流水线，把结果通过旁路网络尽快送给后续执行单元，把缓存和寄存器堆放在物理上更近的位置，并让版图工具控制连线长度、扇出和时钟偏斜。晶体管速度、门级结构、物理布局、流水线边界和调度策略共同决定最终周期时间。每项措施都有代价：更快的标准单元可能增加面积、功耗和漏电；分组和前缀进位让布线更密、布局更难；流水线切分把延迟折成周期数，控制和旁路更复杂；结果旁路要增加选择网络和冲突判断；物理邻近要求模块复制和更强的布局约束；专用单元则要求常见运算有足够高的利用率才值得定制。

移位器也有类似取舍。若每次只移动一位，硬件简单，但一次移动 13 位就要多周期完成。桶形移位器允许在一个组合路径中按控制位选择移 1、2、4、8 等距离，多个阶段叠加得到任意移位量。以 8-bit 为例，移位量的低三位可以控制三层

mux：第一层决定是否移 1 位，第二层决定是否再移 2 位，第三层决定是否再移 4 位。这样能在一个周期内完成多位移位，但 mux 层数和连线负载会进入关键路径。



例：移位量为 5 (1 0 1) 时，s2=1 先移 4 位，s1=0 不动，s0=1 再移 1 位，合计 5 位，一次组合传播完成。

图示：8-bit 桶形移位器：三层 mux 分别由移位量的三个 bit 控制，每一层选择“直送”或“移 2^k 位”。任意 0~7 位移位量都能在一个组合路径内完成，代价是三层 mux 的延迟和大量交叉连线。

几种移位的硬件规则也要分清：逻辑移位在一端补 0，控制线选错会改变数值解释；算术右移在高位补符号位，必须区分有符号和无符号解释；桶形移位用多层 mux 按 1/2/4/8 位选择，mux 延迟和长连线容易成为关键路径；循环移位把移出位从另一端回填，标志位和数据位的关系必须写清楚。

比较器通常可以复用减法路径。判断相等时，执行 $A-B$ 后检查 Zero；判断无符号大小时，关注借位或 Carry 的解释；判断补码有符号大小时，不应仅看最高位，还要结合 Overflow。例如 1000 和 0111 在补码中分别是 -8 和 7，在无符号中分别是 8 和 7。同一组 bit 的大小关系随解释变化，因此比较器必须和控制器约定当前比较类型。

```
eq_unsigned_or_signed = (A - B).Zero
lt_unsigned            = borrow_from_subtract
lt_signed              = result.Sign xor result.Overflow
```

这里的公式表达的是常见标志组合，具体处理器对 Carry 与 Borrow 的命名可能不同。重要的是不要把“最高位为 1”误当成所有比较场景里的“小于”。在补码有符号比较中，结果符号需要用 Overflow 修正；在无符号比较中，符号位没有负数含义。

乘法器展示了另一类工程折中。最朴素的二进制乘法可以生成部分积，再把部分积相加。若一次性展开成组合阵列，延迟和面积都较大，但吞吐可以高；若用移位加法多周期完成，面积小，但一次乘法需要多个周期。早期处理器常常用多周期或微码方式处理复杂运算，现代处理器则可能提供专用乘法单元、流水化乘法器或多个执行单元。抽象上看，乘法仍然是“部分积生成、对齐、求和、保存结果”，只是硬件把这些步骤压缩、展开或流水线化。

几种实现的取舍：移位加法每周期检查乘数一位、必要时累加被乘数移位值，面积小、周期数多，适合简单控制器；组合阵列同时生成多个部分积并通过加法网络求和，面积大、延迟大，但吞吐高；流水化乘法器在部分积压缩和求和之间插入寄存器，吞吐高但单次结果跨多个周期；Booth 编码和压缩树减少部分积数量或加快求和，代价是控制和布局复杂度增加。

阵列乘法器把乘法展开成部分积矩阵

二进制乘法和十进制竖式乘法结构相同：用乘数的每一位去乘被乘数，把得到的行按位权错开，再全部相加。差别只在于二进制里“一位乘一个数”格外简单——乘数位是 1，部分积就是被乘数本身；乘数位是 0，部分积就是全 0。所以“生成一个部分积位”在硬件上就是一个 AND 门： $p_{ij} = a_i \wedge b_j$ 。

以 4x4 位无符号乘法 $A \times B$ 为例， $A = a_3a_2a_1a_0$ ， $B = b_3b_2b_1b_0$ ，先把 16 个部分积位排成矩阵，每一行是被乘数与乘数一位的逐位与，行与行之间按乘数位的权重错开一位：

	a3	a2	a1	a0
x	b3	b2	b1	b0

a3b0	a2b0	a1b0	a0b0	<- 第 0 行, 权 2^0				
a3b1	a2b1	a1b1	a0b1	<- 第 1 行, 权 2^1				
a3b2	a2b2	a1b2	a0b2	<- 第 2 行, 权 2^2				
a3b3	a2b3	a1b3	a0b3	<- 第 3 行, 权 2^3				

p7	p6	p5	p4	p3	p2	p1	p0	<- 乘积, 共 8 位

代入具体数值 0111×0101 ($7 \times 5 = 35$), 逐行移位累加:

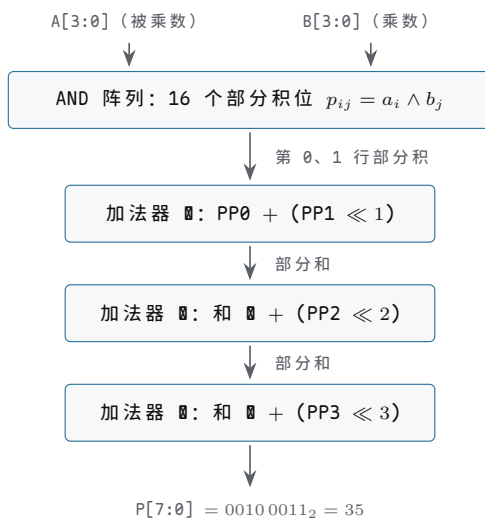
0 1 1 1	
x 0 1 0 1	

0 1 1 1	(b0=1, 部分积 = 被乘数)
0 0 0 0	(b1=0, 部分积 = 0)
0 1 1 1	(b2=1, 左移两位)
0 0 0 0	(b3=0)

0 0 1 0 0 0 1 1	= 35

阵列乘法器就是把这张竖式直接铺成硬件: n^2 个 AND 门一次生成全部部分积位, 再用 $n-1$ 行 n 位加法器逐行累加——第 0 行与第 1 行相加, 结果再与第 2 行相加, 依此类推。4×4 位乘法因此需要 16 个 AND 门和 3 行加法器, 共约 12 个全加器。

这张竖式对应到硬件, 就是下面的结构: AND 阵列一次生成 16 个部分积位, 三行 4-bit 加法器逐行移位累加。



图示: 4×4 阵列乘法器结构: 16 个 AND 门一次生成全部部分积位, 三行 4-bit 加法器逐行累加, 每行按乘数位权重错开一位 ($\ll 1$ 、 $\ll 2$ 、 $\ll 3$)。4×4 需要 16 个 AND 门和约 12 个全加器; 0111×0101 经三行累加得 $0010 0011_2 = 35$ 。

延迟来自“逐行累加”这条链。每行加法本身有一次进位传播, 行与行之间又是串行的, 所以阵列乘法器的最坏延迟大致随位宽线性增长, 且每一级都比同位宽加法器多出一行进位链。面积的增长更快: AND 门和全加器的数量都按 n^2 走, 连线规模也是 n^2 量级。位宽翻一倍, 加法器只长大一倍, 乘法器却大约长大三倍 (变为四倍)。

位宽 n	AND 门 (部分积位)	全加器 ($n(n-1)$, 逐行累加)	关键路径
4	16	12	3 行加法链
8	64	56	7 行加法链
16	256	240	15 行加法链

这就解释了为什么硬件乘法器比加法器贵得多：加法器是 $O(n)$ 个全加器排成一条链，乘法器是 $O(n^2)$ 个门铺成一片阵列，面积、延迟、功耗全面吃亏。所以小型控制器常用“每周期加一行”的移位加法方案，用时间换面积；只有乘法频繁出现的处理器才值得放一块完整阵列。现代高性能处理器进一步用 Booth 编码把部分积行数减半、用压缩树（如 Wallace 树）把逐行串行累加改成树形并行压缩，但“部分积矩阵加求和网络”这个基本结构没有改变。

Booth 编码用“一减一加”代替逐位累加

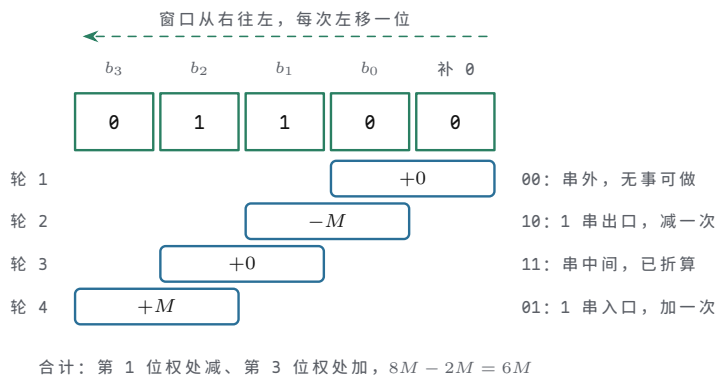
逐位移位加法的累加次数等于乘数中 1 的个数，最坏情况下 n 位乘法要做 n 次。Booth 编码的观察是：一串连续的 1 可以整体改写。例如乘数中出现 011110，按位权展开是 $2^4 + 2^3 + 2^2 + 2^1 = 30$ ；但它也等于 $2^5 - 2^1 = 32 - 2$ ，也就是 $100000 - 000010$ 。原本 4 次加法的区段，改写成“高端加一次、低端减一次”就够了。二进制减法同样由加法器完成（取补相加），所以这种改写不引入新硬件，只减少累加次数。

两位 Booth 编码把这个想法系统化：从最低位开始，每次看乘数的当前位 b_i 和它右边的相邻位 b_{i-1} （最低位右边补一个虚拟的 0），两位组合决定本轮对被乘数 M 做什么：

$b_i b_{i-1}$	本轮动作	含义
0 0	+0	一串 0 的中间，无事可做
0 1	+ M	一串 1 的开始（低端），把被乘数加上
1 0	- M	一串 1 的结束（高端），把被乘数减掉
1 1	+0	一串 1 的中间，已经折算过，跳过

直观记忆：从右往左扫乘数，0→1 是“1 串的入口”，做一次加；1→0 是“1 串的出口”，做一次减；00 和 11 都在串的内部，不累加。每轮做完后乘数右移一位，把下一对 bit 送上来。

以乘数 0110 为例把滑动窗口画出来：右侧补一个虚拟 0，窗口从最低位起每次左移一位，四轮考察各做一次判断。



图示：两位 Booth 编码的滑动窗口（乘数 0110，右侧补虚拟 0）。窗口每次只看相邻两位 $b_i b_{i-1}$ ：0→1 是 1 串的入口，对应 + M ；1→0 是出口，对应 - M ；00 与 11 都在串的内部或串外，不累加。四轮中只有轮 2、轮 4 有动作，合计 $+M \cdot 2^3 - M \cdot 2^1 = 8M - 2M = 6M$ ，与乘数值 6 一致——连串 1 被折算成串尾之后的一次减和串首的一次加。

以 4-bit 例子 0011×0110 ($3 \times 6 = 18$) 走一遍。被乘数 $M = 0011$ ，乘数 $B = 0110$ ，右侧补 0 后逐对考察：

轮次	考察位 $b_i b_{i-1}$	动作	对累加结果的贡献
1	0 0	+0	0
2	1 0	- M	$-0011 \times 2^1 = -6$
3	1 1	+0	0 (1 串中间)

$$4 \qquad 01 \qquad +M \qquad +0011 \times 2^3 = +24$$

合计 $-6 + 24 = 18$ ，结果正确。普通逐位法对乘数 0110 要做 2 次加法，这个例子里 Booth 也是一加一减，看似没有节省。差别在 1 串变长时才显现：乘数若是 011110（4 个连续 1），逐位法要 4 次加法，Booth 仍只要“低端一加、高端一减”共 2 次动作；乘数若是全 1 的 1111，逐位法要 4 次加法，Booth 只在最低位遇到一次 $1 \rightarrow 0$ （实际是考察位 $(1,0)$ ），做一次减就结束。

Booth 编码的意义不在于硬件更精巧，而在于把“乘法要做几次累加”从“取决于乘数里有几个 1”变成“取决于乘数里有几段 1”。它还附带一个好处：这套规则天然兼容补码有符号乘数，不需要像原码乘法那样单独处理符号位。现代乘法器普遍采用按两位、三位分组的改进 Booth (radix-4、radix-8)，把部分积行数减到 $n/2$ 或 $n/3$ 行，再配合压缩树求和——这就是乘法器能在一个流水段里算完的起点。

恢复余数除法：一次一位地试出商

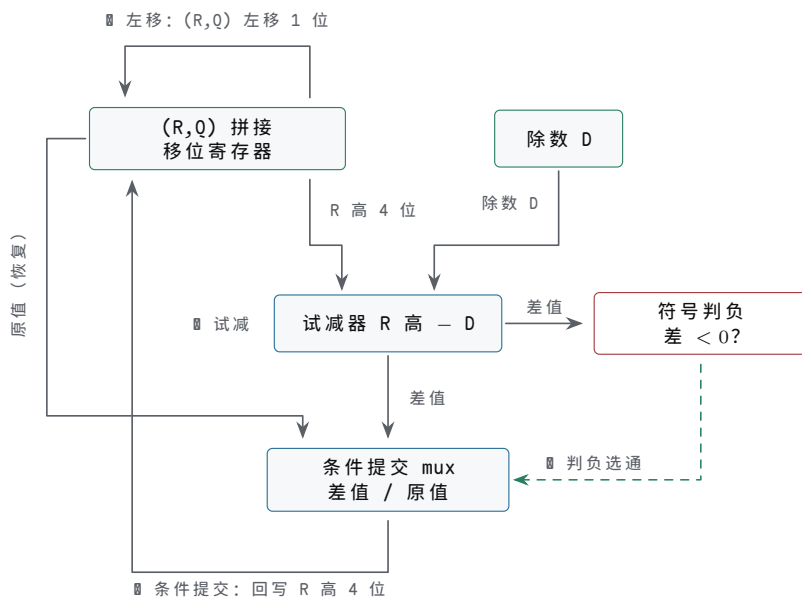
除法是乘法的逆过程，但硬件上它比乘法更难安排：乘法每一步做什么由乘数位直接决定，除法每一步要先“试一试”才知道。恢复余数除法 (restoring division) 是最直白的方案，和笔算长除法同构：每轮把部分余数左移一位，试减除数；够减，这一位商就是 1，余数保留差值；不够减，这一位商是 0，把余数恢复成试减前的值，进入下一轮。

以 4-bit 无符号除法 $13 \div 3$ 为例：被除数 1101，除数 0011。用一个 8-bit 寄存器 R 存放部分余数，高 4 位初始为 0，低 4 位初始为被除数，即 $R = 00001101$ ；除数始终与高 4 位对齐比较。每轮三步： R 整体左移一位；高 4 位试减 0011；差不为负则保留差并把最低位置 1，否则恢复高 4 位、最低位置 0。

轮次	左移后的 R	高 4 位试减 0011	商位	本轮结束的 R
初始	—	—	—	0000 1101
1	0001 1010	0001 - 0011，不够减， 恢复	0	0001 1010
2	0011 0100	0011 - 0011 = 0000，够减	1	0000 0101
3	0000 1010	0000 - 0011，不够减， 恢复	0	0000 1010
4	0001 0100	0001 - 0011，不够减， 恢复	0	0001 0100

四轮结束后，低 4 位 0100 是商，高 4 位 0001 是余数： $13 = 3 \times 4 + 1$ ，与笔算一致。注意第 2 轮左移后，原先在高位的余数和新进来的被除数位拼在一起，恰好第一次够减——这和笔算长除法里“逐位落数，直到够减”是同一个过程。

把每轮的三个动作画成数据通路，硬件只剩三块：一个 (R,Q) 拼接移位寄存器、一个试减器、一个按符号选择的写回 mux。



图示：恢复余数除法的数据通路：每轮三条路径。■ (R,Q) 整体左移一位，空出的 Q 低位留给本轮商位；■ R 高 4 位与除数试减；■ 按差的符号条件提交—差非负则写回差值、商位记 1，差为负则沿左侧恢复路径写回原值、商位记 0。“不够减要恢复”在图上就是 mux 多出来的那路原值输入：恢复不需要再做一次加法，只是把试减前的旧值选回来。

为什么除法非要 n 轮不可：商有 n 位，每一位都要独立判断一次“够不够减”，而这个判断的输入是上一轮算完的余数，无法并行预知。所以除法本质上是串行的， n 位商就要 n 轮，每轮一次减法（同时就是比较）加一次条件提交。恢复余数法的代价是那一次“白做的减法”：不够减时要恢复，硬件上体现为每轮多一个二选一（减的结果与原值），或者多一步把原值加回来。

改进方向值得记一句：不恢复余数法（non-restoring）发现“本轮恢复、下轮再减”可以合并成“下轮直接加”，省掉恢复动作；SRT 除法用商数字集合 $\{-1, 0, +1\}$ 加查找表，一轮产生多位商。但无论哪种方案，“每轮一次比较、一次条件更新余数”的串行骨架不变——这就是现代处理器里整数除法仍要十几到几十个周期、而乘法只要几个周期的根本原因。

74181 是经典 4-bit ALU 芯片，常被用来观察算术逻辑单元如何把多种函数、进位输入、进位输出和模式控制组合到一个器件中。它不是现代 CPU 的内部实现模板，却非常适合作为教学标本：控制引脚选择函数，数据引脚提供操作数，输出引脚给出结果和进位信息。把 74181 与 8086、80386 对照，可以看到同一问题的放大版本：位宽增加、控制复杂度上升、标志位更多地参与分支、地址和异常判断，ALU 不再只是一个孤立芯片，而是数据通路和控制器中的高频核心。

用 74181 做实验时，不应只把它当作“会算很多函数的黑盒”。更好的读法是把每一类引脚对应到本章讲过的控制对象：A/B 是操作数输入，F 是结果输出，S 选择具体函数，M 选择逻辑模式或算术模式， C_n 是低位进位输入， C_{n+4} 是高位进位输出，若要扩展到更宽位宽，还要用跨片进位或专门的超前进位芯片处理片间等待。

74181 观察点	接线含义	对应本章概念
A/B 输入	两个 4-bit 操作数进入芯片	ALU 操作数总线
S 控制	选择具体逻辑或算术函数	alu_op 的一部分
M 控制	区分逻辑类函数和算术类函数	ALU 模式选择
C_n	低位进位输入，可接前一级进位或控制信号	加法、减法、多字长运算入口

Cn+4	4-bit 组的高位进位输出	Carry 标志或级联输入
F 输出	4-bit 结果输出	写回候选结果

若用两片 74181 组成 8-bit ALU，低 4 位芯片处理 bit0 到 bit3，高 4 位芯片处理 bit4 到 bit7。最低位 Cn 由本次动作决定：普通加法为 0，带进位加法接 Carry，减法按芯片函数和控制约定接入相应初始进位。低片的 Cn+4 送到高片 Cn，即可得到串行跨片进位。若继续扩展到 16-bit，这种片间进位会增加等待；因此经典板级设计常配合 74182 一类超前进位发生器，减少多片 74181 之间逐片等待的时间。这里再次回到本章第三节的主题：更宽的数据通路不是简单复制芯片，还要重新处理进位、控制和时序。

若用分立芯片搭一个 4-bit ALU 实验台，可以按以下顺序推进。第一步只搭 74HC283 加法；第二步加入 74HC86 实现加/减复用；第三步用 74HC08、74HC32、74HC86 分别产生 AND、OR、XOR 候选结果；第四步用 74HC157 选择加法结果或逻辑结果；第五步用 LED 显示 Zero、Carry、Negative 和 Overflow。每一步都要确保前一步的结果仍可观察、可复现，避免在复杂电路里同时排查过多变量。

实验阶段	新增器件或连线	预期现象
4-bit 加法	74HC283、拨码开关、LED	0011+0101=1000, Carry 正确
加/减复用	74HC86 处理 B, sub 接 Cin	sub=1 时 A-B 正确
逻辑候选	AND/OR/XOR 门阵列	候选结果与真值表一致
结果选择	74HC157 多路选择器	op 改变时只有被选结果进入 LED
标志输出	OR 合成 Zero，最高位和进位生成标志	结果为 0 时 Zero=1，溢出案例可复现

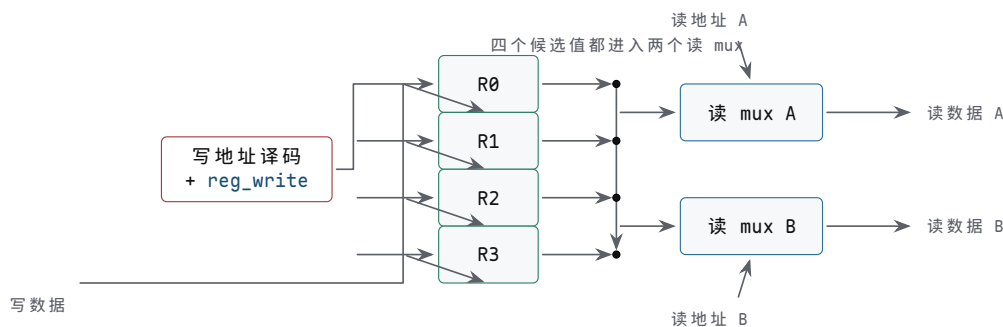
四、数据通路让值从旧状态走到新状态

数据通路回答一个问题：值从哪里来，经过哪里，最后保存到哪里。孤立的寄存器和 ALU 还不能完成连续动作。必须把寄存器阵列、选择器、ALU、内部连线和写回路接成一个可控制的闭环。若 ALU 是“能算什么”，数据通路就是“值能走哪条路”。

一个寄存器只能保存一个值。若硬件需要同时保存多个中间值，就需要寄存器阵列。寄存器阵列可以理解为一组编号寄存器，加上读写选择电路。

端口	作用	类型
读地址 A/B	选择两个源寄存器	输入
读数据 A/B	输出两个源值	输出
写地址	选择目标寄存器	输入
写数据	提供要保存的新值	输入
写使能	决定是否在时钟沿写入	输入

读地址进入译码和选择网络，决定哪两个寄存器的值出现在读数据端口。写地址进入译码器，写使能有效时，只允许被选中的那个寄存器在时钟沿接收写数据。这样，一组寄存器就能被编号访问，而不是每个寄存器单独拉一套控制线。真实处理器中的寄存器堆常常有多个读端口和写端口，原因也很直接：ALU 常常需要同时读两个源操作数，并在周期末写回一个目标。



图示：4 寄存器堆的内部结构：写地址经译码（并和 `reg_write` 相与）只打开一个目标的写入；写数据广播到所有寄存器的 D 端，但只有被选者提交；两个读 mux 按读地址各选一路输出。读是组合选择，写是时钟沿提交——这条不对称是寄存器堆和“一组变量”的本质区别。

ALU 的两个输入不一定总来自同一处。一个输入可能来自寄存器阵列，也可能来自 PC、计数器或固定 0；另一个输入可能来自寄存器、常量或扩展后的字段。因此 ALU 输入前通常有 mux。

```
alu_a = mux(a_sel, reg_a, pc, zero, latch_a)
alu_b = mux(b_sel, reg_b, immediate, one, offset)
```

以寄存器 R1 和 R2 相加、结果写入 R3 为例。数据通路需要让读地址 A 选择 R1，读地址 B 选择 R2；`a_sel` 选择 `reg_a`，`b_sel` 选择 `reg_b`；`alu_op` 选择 ADD。此时 ALU 输出就是 $R1+R2$ 。

写回数据也不一定只来自 ALU。它可能来自 ALU，也可能来自存储器读口、外部输入、PC+4 或专用状态寄存器。写回前也需要 mux。

```
write_data = mux(wb_sel, alu_result,
                 memory_data, input_data, pc_plus_4)
```

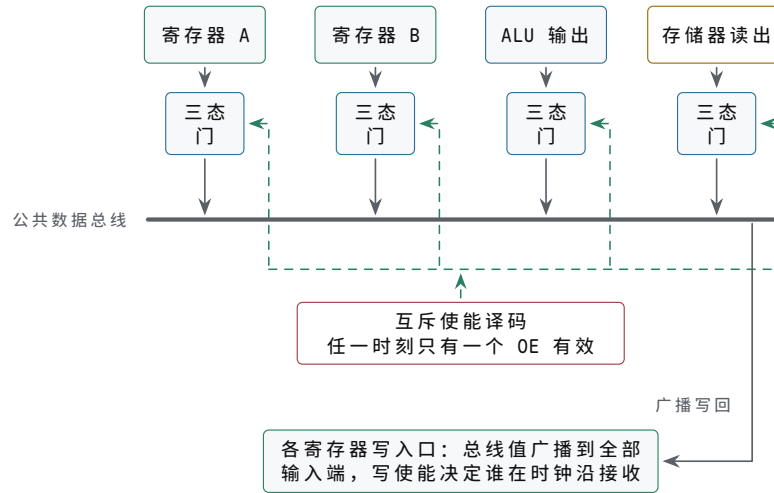
在 $R3=R1+R2$ 的动作中，`wb_sel` 选择 `alu_result`，写地址选择 R3，`reg_write` 为 1。等时钟沿到来，R3 才真正保存结果。若 `reg_write` 为 0，即使 ALU 算出了正确结果，寄存器阵列也不会改变。这个边界来自第二章的同步设计原则：组合逻辑可以在周期内传播和抖动，状态只在受控时钟沿提交。

当多个部件都可能把值送到同一个目的地时，不能让它们同时强行驱动同一条线。要么用 mux 明确选择一路，要么用总线协议规定同一时刻只有一个驱动者。ALU 输出、存储器输出和外部输入如果直接接在一起，一旦两个源同时输出不同电平，就会形成冲突。mux 让这件事变成明确选择：当前周期只允许一路成为 `write_data`，适合芯片内部数据通路和 FPGA 逻辑，代价是输入多时选择器延迟和面积增加。三态总线也能共享连线，适合板级总线和部分分立芯片实验，但必须保证同一时刻只有一个输出使能有效，多个驱动同时打开就是冲突。点到点连线适合高频、固定方向的数据路径，代价是连线数量增加、灵活性降低。在初学实验和 FPGA 设计中，优先使用 mux 更容易检查和调试。

三态总线型数据通路：一条线上轮流说话

第一章讲三态门时说过：三态输出除了 0 和 1，还有高阻态 Z——输出端像被“拔掉”一样，既不驱高也不驱低。三态总线正是利用这一点组织数据通路：把多个寄存器的输出、ALU 的输出、存储器读出都接到同一组公共线上，每个源前面放一个三态缓冲器，各自由输出使能控制。任何时刻最多一个使能有效，总线上就只有这一家在“说话”，其余源都处在高阻态，对总线电平没有影响。写入侧则反过来：总线上的值广播到所有寄存器的输入端，写使能决定谁在时钟沿接收。

把这段话画成数据通路，就是下面的结构：所有源挂到同一条总线上，读和写各由一组使能控制。



图示：三态总线型数据通路：寄存器 A/B、ALU、存储器的输出各经一个三态门挂到同一条公共总线上，使能译码保证任一时刻只有一个 OE 有效，其余源处于高阻态；总线值广播到全部寄存器写入口，写使能决定谁在时钟沿接收。“谁在驱动”由控制逻辑负责，两个使能同时有效就是总线冲突。

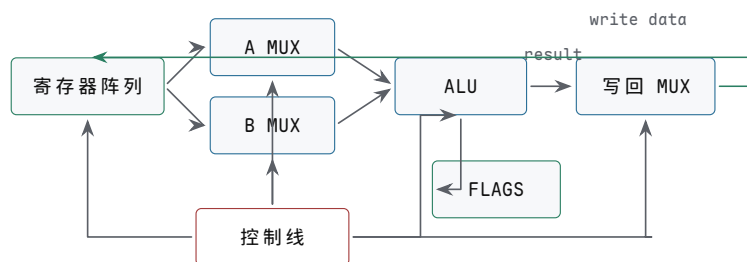
与 mux 型数据通路对照，差别在“选择发生在哪里”。mux 方案里每条数据通路独立布线，由接收端的选择器挑一路——选择是集中的、点对点的，几路输入同时驱动各自的线互不相干。总线方案里，选择分散在每个驱动者的输出使能上，依赖一条全局约定：同一时刻只有一个驱动者。这条约定一旦被破坏，两个源同时驱动总线，一个驱高一个驱低，线上就是不可预期的电平，还可能流过过大的电流。

对照点	三态总线型	mux 型
连线数量	每个位宽只有一组公共线，源再多也不加线	每个源到每个接收点独立走线，随源数增长
面积代价	省布线、省选择器，适合布线紧张的情况	多路 mux 本身占面积，输入越多级数越深
出错模式	使能冲突（多驱）或全高阻（无驱），影响是全局性的	选择信号错即选错路，故障局限在单点
时序分析	总线电平依赖各使能的时序，要分析所有潜在驱动者	路径固定，逐条路径算延迟即可
调试入口	先查“此刻谁在驱动”	直接观察选择信号和对应路径

早期小型机和微处理器爱用总线型，原因很实际：当时芯片内部布线层数少、布线资源紧张，分立搭建时电路板上的走线和插接件更是寸土寸金；一条公共总线让寄存器、ALU、存储器、I/O 都挂上来，扩展一个新部件只需再挂一个三态驱动器，不必为每个新源重铺一组线。PDP-11 的 Unibus、各类 8-bit 微处理器的地址/数据总线，都是这个思路。代价是把“谁在驱动”的正确性推给了控制设计：使能信号必须由译码逻辑保证互斥，切换驱动者时最好留一拍全高阻的间隔，避免旧驱动尚未关闭、新驱动已经打开的瞬间冲突。正因为这条约定难以形式化检查，现代芯片内部数据通路大多回到 mux 型；三态总线更多留在板级互连和存储器接口这类“真正共享一条物理线”的地方。

一个最小数据通路闭环可以写成：

```
register file -> mux -> ALU -> mux -> register file
```



数据通路图要区分“值走哪条路”和“控制线选择哪条路”。写回箭头绕到上方，避免穿过 ALU 和标签。

图示：最小数据通路闭环。源值从寄存器阵列读出，经过输入选择和 ALU，最后由写回选择器在时钟边界写回。

它能在一个周期内读取旧值、选择输入、完成组合计算，并在时钟沿保存结果。这个闭环已经能表达很多动作，但它还不会自己决定“当前周期做什么”。哪些地址送入寄存器阵列，ALU 做加法还是减法，写回选择哪一路，写使能是否打开，都需要控制器统一产生。

如果把这个闭环搭成分立电路，器件可以按功能分组选择。74HC574 或 74HC173 可以保存寄存器值；74HC138 可以把写地址译码成某个寄存器的写使能；74HC157 可以选择 ALU 输入或写回来源；74HC283/74HC86 组合成加减核心；LED 或逻辑分析仪用于观察寄存器输出、ALU 输出和写使能。这个实验的关键不是一次做完整指令集，而是让 `R1`、`R2`、`R3` 的值在时钟沿前后清晰可见。

这个 4 寄存器堆有一个重要限制：若用普通 74HC574，它虽有三态输出使能 `/OE`，但没有专门的写使能逻辑，就要额外设计时钟使能或 D 端保持路径；若用 74HC173 这类带使能和输出控制的寄存器，实验更容易分阶段观察。无论选哪种器件，原则相同：写地址只决定“谁有资格写”，全局 `reg_write` 决定“本周期是否写”，时钟沿决定“何时真正写”。读 mux 是组合选择，写入是时序提交，两者不能混在一起。

把各连接点的器件和检查要点再落实一层：`R0` 到 `R3` 的存储用 74HC173 (4-bit 寄存器) 或 74HC574 (8-bit 成组保存)，复位后初值确定、时钟沿前后值可观察；写地址译码用 74HC138 把两位 `write_addr` 展开成四路写使能候选，同一周期只允许一个目标写入；全局 `reg_write` 与译码输出相与后送入各寄存器，`reg_write=0` 时所有寄存器保持；读端口 A/B 各用一组 4-bit mux 按 `read_addr_a`、`read_addr_b` 独立选择，A 端口变化不应改写寄存器内容，A/B 可同时读同一寄存器或不同寄存器；写回数据经写回 mux 进入所有寄存器 D 端，未选寄存器即使 D 端变化也不能写入。

真实芯片把这些部件压缩进同一颗硅片。8051 把 CPU、片内 RAM、特殊功能寄存器、定时器、串口和 I/O 口组织在一个单片机内部；对程序员来说，某些外设像“寄存器地址”一样被读写；对硬件来说，这是数据通路、地址译码、外设寄存器和控制器共同工作的结果。8086 把总线接口单元和执行单元分工，执行单元内部仍要围绕寄存器、ALU、标志和控制路径组织动作。80386 继续扩大寄存器和地址计算规模，并引入更复杂的保护与寻址硬件。规模不同，问题相同：值必须从旧状态沿受控路径走到新状态。

从 8051 看，单片机不能简化为“小型软件平台”，而是 CPU 数据通路、片内存储器、I/O 口、定时器和中断控制的硬件组合。第一章的输入调理和逻辑门，第二章的状态边界，本章的寄存器与控制线，都在单片机内部或引脚边界重新出现。后面整机章讨论 MMIO、设备和中断时，会继续使用这条线索。

数据通路与时序分析应至少包含四项：源状态何时稳定，组合路径最坏延迟是多少，目标触发器建立/保持时间是否满足，哪些写使能在时钟沿有效。若一个单周期 `R3=R1+R2` 路径太长，可以把动作拆成多周期：第一周期读 `R1/R2` 并锁存到 A/B，第二周期 ALU 计算并锁存到 `ALUOut`，第三周期把 `ALUOut` 写回 `R3`。这样周期时间可

以变短，但一次动作需要更多周期。速度、面积和控制复杂度再次形成取舍。

三种方案的取舍：单周期让读寄存器、ALU 计算、写回都在一个周期内完成，控制简单，但周期必须容纳最长路径；多周期把读、算、写分到多个状态和多个时钟沿，周期可短，代价是控制状态增加；流水线让多条动作在不同阶段重叠推进，吞吐提高，代价是冒险、旁路和冲刷更复杂。本章先用单周期和多周期把控制 trace 跑通，流水线留到最小 CPU 之后再展开。

五、控制器输出一组一致的控制线

数据通路提供很多可能路径，但同一周期不能所有路径都随意打开。控制器的任务是根据当前状态、指令字段和条件输入，产生一组控制信号，让数据通路执行一个明确动作。控制器不是写在纸上的命令列表，而是一块输出控制电线的硬件。它可以由组合逻辑、状态机、ROM、PLA（可编程逻辑阵列：把若干 AND 项和 OR 项做成规则阵列的组合逻辑，适合把译码表直接烧进硬件）、微码表或这些方法的混合实现。

控制信号通常是一组 0/1 或多 bit 选择值：

```
alu_op
a_sel
b_sel
wb_sel
read_addr_a
read_addr_b
write_addr
reg_write
flags_write
mem_read
mem_write
next_state_sel
```

这些信号直接接到 ALU、mux、寄存器阵列、标志寄存器和存储器接口。控制信号给错，数据通路就会走错。例如 `reg_write` 本应为 0 却变成 1，某个寄存器会在时钟沿被意外覆盖；`wb_sel` 本应选择 ALU，却选了外部输入，写回值就会错；`flags_write` 本应关闭却打开，旧标志会被无关结果污染，后续条件分支会误判。

若动作规则简单，可以用组合逻辑直接从输入生成控制信号。例如输入 `mode` 为 0 时做加法，为 1 时做按位 AND：

```
mode = 0 -> alu_op = ADD
mode = 1 -> alu_op = AND
```

硬连线控制速度快，结构也直接。缺点是规则复杂后，组合逻辑会变大，修改一个动作可能影响多条控制线。若一个动作无法在一个周期内完成，就需要状态机。状态机保存“当前走到哪一步”，每个状态输出一组控制信号，并决定下一状态。

控制器可以分成两个层次理解。第一层是**译码**：把 opcode、功能位、条件位和当前状态翻译成控制线。第二层是**时序安排**：决定这些控制线在哪个周期有效，哪些状态元件在该周期写入。若指令简单、动作少，硬连线译码可以直接生成控制信号；若指令复杂、步骤多，控制器可能使用微操作表或微码 ROM，把一条指令拆成多行控制字逐步执行。

几种控制方式的取舍：硬连线控制适合动作规则固定、追求低延迟的路径，代价是逻辑复杂后修改困难、一个错误影响多条控制线；状态机控制适合多周期动作和明确阶段推进，代价是状态编码、非法状态和输出毛刺都要处理；微码控制适合指令复杂、步骤可表格化的场景，代价是微码存储、入口选择和异常中断点更复杂；混合控制让常见简单路径走硬连线、复杂路径走微操作，代价是必须统一提交边界和异常恢复规则。

一行控制字可以像下面这样理解：

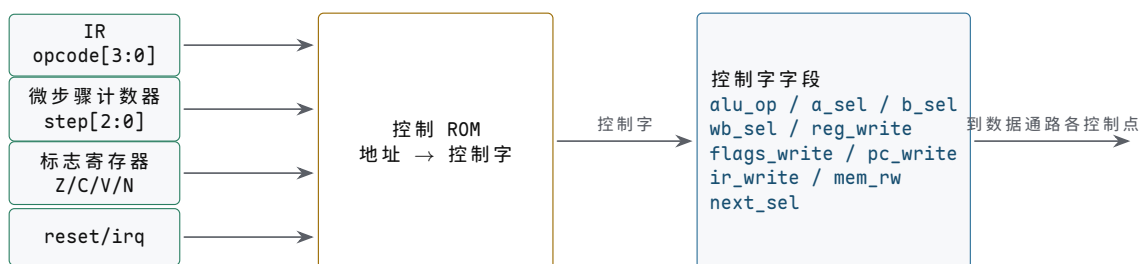
```

state EXEC_ADD:
    read_addr_a = rs1
    read_addr_b = rs2
    a_sel       = REG_A
    b_sel       = REG_B
    alu_op      = ADD
    wb_sel      = ALU_RESULT
    write_addr  = rd
    reg_write   = 1
    flags_write = 1
    next_state  = FETCH

```

这并非软件语句，而是对硬件控制线的命名描述。控制字中的每一位最终都要连接到 mux、ALU、寄存器写使能、标志寄存器、存储器接口或下一状态逻辑。调试控制器时，不应只看“当前指令是什么”，还要看当前状态、控制字、条件输入和本周期会提交哪些状态。

控制器也可以用分立芯片做出教学模型。一个 74HC161 计数器可以充当微步骤计数器，74HC138 可以把微步骤译码成若干状态脉冲，小容量 EEPROM 或 ROM 可以存放控制字。地址线接 opcode、微步骤和标志位，数据线输出 `alu_op`、`a_sel`、`b_sel`、`reg_write` 等控制信号。这种做法的好处是非常直观：修改控制表就能改变机器动作；代价是控制存储器访问延迟、控制字宽度和异常入口都要进入时序设计。



图示：控制 ROM 的结构：指令 opcode、微步骤、标志位和复位/中断请求拼成 ROM 地址，地址内容就是本周期全组控制线。改机器动作等于改 ROM 内容表——这就是微码控制”指令是可编程表格”的含义。

控制 ROM 地址位	来源	作用
<code>opcode[3:0]</code>	指令寄存器中的操作码	选择当前指令类型，如 ADD、SUB、LOAD、BRANCH
<code>step[2:0]</code>	微步骤计数器或状态寄存器	区分取指、译码、执行、写回等阶段
<code>Z/C/V/N</code>	标志寄存器输出	条件分支或条件写回的输入
<code>reset/irq</code>	复位或外部请求同步后的状态	强制进入复位入口或异常入口

控制 ROM 的数据输出就是控制字。为了便于搭实验机，可以先把控制字设计成固定宽度字段，而不是零散电线。

控制字段	连接对象	含义
<code>alu_op[2:0]</code>	ALU 函数选择	指定 ADD、SUB、AND、OR、XOR、PASS 等动作
<code>a_sel[1:0]</code>	ALU A 输入 mux	选择寄存器 A、PC、0 或临时寄存器
<code>b_sel[1:0]</code>	ALU B 输入 mux	选择寄存器 B、立即数、1 或偏移

<code>wb_sel[1:0]</code>	写回 mux	选择 ALUOut、存储器数据、PC+1 或外部输入
<code>reg_write</code>	寄存器堆写使能	决定目标寄存器是否在时钟沿写入
<code>flags_write</code>	标志寄存器写使能	决定 ALU 标志是否提交
<code>pc_write</code>	PC 写使能	决定下一 PC 是否提交
<code>ir_write</code>	指令寄存器写使能	决定本周期是否锁存取到的指令
<code>mem_read/write</code>	存储器接口	发起读周期或写周期
<code>next_sel[1:0]</code>	下一状态逻辑	选择顺序微步骤、取指、分支目标或异常入口

这个表可以直接变成 EEPROM 内容表。例如 `opcode=ADD, step=EXEC` 的控制字会选择寄存器 A/B 作为 ALU 输入, `alu_op=ADD`, 打开 `flags_write`, 并把下一状态设为写回; `opcode=BRANCH, step=EXEC` 的控制字则可能关闭 `reg_write`, 读取 Zero 标志, 通过 `next_sel` 选择顺序 PC 或分支目标。判断控制 ROM 方案是否可行, 关键不在“表看起来合理”, 而在每一位输出都能追到一个实际器件引脚。

微程序控制器：把控制字存进一块存储器

前面控制 ROM 的做法其实已经触到了微程序控制 (microprogrammed control) 的核心: 与其用组合逻辑从 `opcode` 现场译出控制线, 不如把“每条指令在每个周期该输出什么”事先写成一张表, 存进一块专门的控制存储器。表里的每一行叫一条微指令, 一台机器全部微指令合起来叫微程序。一条机器指令不再对应一段组合逻辑, 而对应控制存储器里的一段微指令序列——取指之后跳到这段微程序, 逐条执行, 走完这段微程序, 这条机器指令的所有周期也就走完了。

一条微指令通常包含三类字段:

字段	作用	例子
控制线字段	本周期送往数据通路的全部控制位, 每位接一条控制线	<code>alu_op</code> 、 <code>a_sel</code> 、 <code>reg_write</code> 、 <code>mem_read</code>
下一微地址字段	指出正常情况下下一条微指令的地址	<code>next = 微地址 17</code>
条件选择字段	指定本周期是否查看条件、查看哪个条件, 条件成立时改走哪个地址	看 Zero: 成立去分支微地址, 不成立用下一微地址

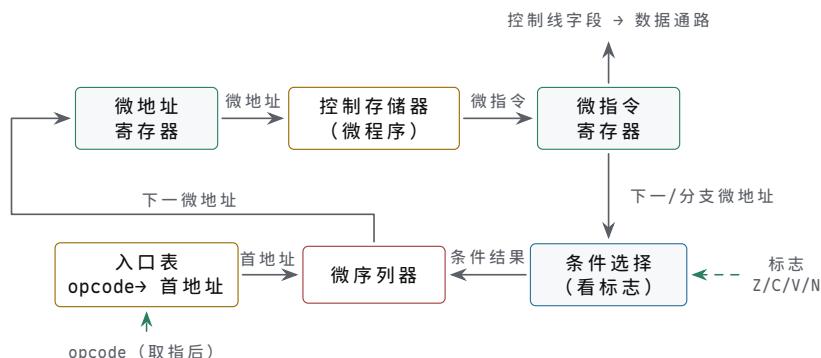
负责读这张表的小硬件叫微序列器 (microsequencer)。它每个周期做同一件事: 按当前微地址取出一条微指令, 把控制线字段直接送往数据通路, 同时按条件选择字段决定下一个微地址——顺序、跳到指定地址、或按条件二选一。机器指令译码在这里退化成一次跳转: 取指结束后, 用 `opcode` (必要时拼上功能位) 查一张入口表, 得到这条机器指令微程序的首地址, 交给微序列器接着走。条件分叉也用同一机制: 条件跳转指令的微程序里安排一条指定“看 Zero 标志”的微指令, Zero 为 1 时下一微地址指向“更新 PC”的微指令, 为 0 时指向“直接回取指”的微指令。

```

; 微程序示意: ADD 与条件跳转 BEQ 各对应一段微指令
FETCH:   mem_read=1, ir_write=1, next=DISPATCH
DISPATCH: 按 opcode 查入口表 -> ADD_U0 / BEQ_U0 / ...
ADD_U0:   a_sel=REG_A, b_sel=REG_B, alu_op=ADD,
          reg_write=1, flags_write=1, next=FETCH
BEQ_U0:   a_sel=PC, b_sel=OFFSET, alu_op=ADD,
          条件选择=Zero: Z=1 -> BEQ_U1, Z=0 -> FETCH
BEQ_U1:   wb_sel=ALU_RESULT, pc_write=1, next=FETCH

```

把微程序控制器本身画成结构图, 就是五个部件构成的闭环。



图示：微程序控制器的五个部件构成闭环：微地址寄存器给出当前微地址；控制存储器读出该行的微指令；微指令寄存器的控制线字段直接驱动数据通路，下一微地址和条件选择字段交给微序列器；条件选择按标志二选一；opcode 经入口表提供各段微程序的首地址。改指令行为只改控制存储器内容，硬件不动。

与普通组合逻辑译码（硬连线控制）对照，差别不在“控制线从哪来”——最终每根控制线都要在正确的周期输出正确的电平——而在“规则写在哪里”。硬连线方案把规则固化在与或门阵列里，改一条指令的行为就要改逻辑、重新布线；微程序方案把规则写成存储器内容，改指令行为只改微码，硬件本身一根线不动。这个差别在工程上影响深远：指令集复杂、寻址方式多的机器，比如 8086 这类变长指令、多周期动作的处理器，用硬连线译码会让组合逻辑大到难以设计和检查，微程序把复杂度转移成“填表”；机器出厂后发现指令行为有问题，有时只更新微码就能修正，不必更换芯片。代价同样明确：每条微指令多一次控制存储器访问，周期下限受存储器延迟约束；微地址、条件选择、入口表又是一层要调试的状态。所以规则简单的现代核心回到硬连线快速路径，把微码留给复杂指令和罕见情况——两层控制器并存，正是这一节两种方案在真实芯片里的最终分工。

在分立实验中，每个部件都有对应实现和各自的检查点：状态寄存器用 74HC161 计数器或 D 触发器组，检查每个时钟沿状态是否按预期推进；状态译码用 74HC138 或门阵列，检查是否只有当前状态对应的控制线有效；控制存储用 EEPROM、ROM 或拨码开关矩阵，检查同一地址是否输出稳定控制字；条件选择让 Zero/Carry/Overflow 进入下一状态逻辑，检查条件成立和不成立是否走向不同状态；复位逻辑用复位按钮加同步释放或明确初始化，检查上电后是否进入确定状态。

控制错误往往比 ALU 错误更隐蔽，因为 ALU 本身可能算对了，但错误选择线把结果送错地方，或者错误写使能在错误时钟沿提交状态。

排查要按控制线分类：`alu_op` 错会算错函数、标志位也可能错，先查 opcode 译码、功能位和状态选择；`a_sel/b_sel` 错让 ALU 输入不是预期来源，先查操作数字段、mux 控制和临时寄存器；`wb_sel` 错让正确结果没有被写回，先查写回 mux 和控制状态；`reg_write` 错让目标寄存器未写或被意外覆盖，先查写使能门控和时钟边界；`flags_write` 错让条件分支使用错误标志，先查标志寄存器写入条件和指令语义；`mem_write` 错让存储器或设备被误写，先查地址译码、写周期和保护条件。

8086 常被用来说明控制复杂度。它的指令长度不固定，寻址方式丰富，许多动作需要拆成多个内部步骤；因此不能把控制器想象成“opcode 直接连到 ALU op”这么简单。80386 进一步加入 32-bit 模式、保护机制和更复杂的地址转换相关检查，控制路径需要处理更多条件。现代处理器又在硬连线快速路径、微操作、预测、乱序调度和异常恢复之间做更复杂的工程分解。无论规模如何，控制器最终仍要回答同一个问题：本周期哪些电线有效，哪个状态会被提交。

六、用控制 trace 走读一次动作

复杂动作可以拆成微操作：读、选择、计算、保存、更新状态。本节所称微操作，指一个周期内同时打开的一组硬件控制信号。以下先以 `R1` 和 `R2` 相加、结果写入 `R3` 为例。为了让时序更容易观察，采用三周期版本：`S0` 读源寄存器并锁存，`S1` 计算并锁存 ALU 输出，`S2` 写回目标寄存器。

```
S0: read_sel_a=R1, read_sel_b=R2
```

```

A_write=1, B_write=1
reg_write=0, ALU_out_write=0

S1: a_sel=A, b_sel=B, alu_op=ADD
    ALU_out_write=1
    flags_write=1
    reg_write=0, A_write=0, B_write=0

S2: wb_sel=ALU_out, write_sel=R3
    reg_write=1
    A_write=0, B_write=0, ALU_out_write=0

```

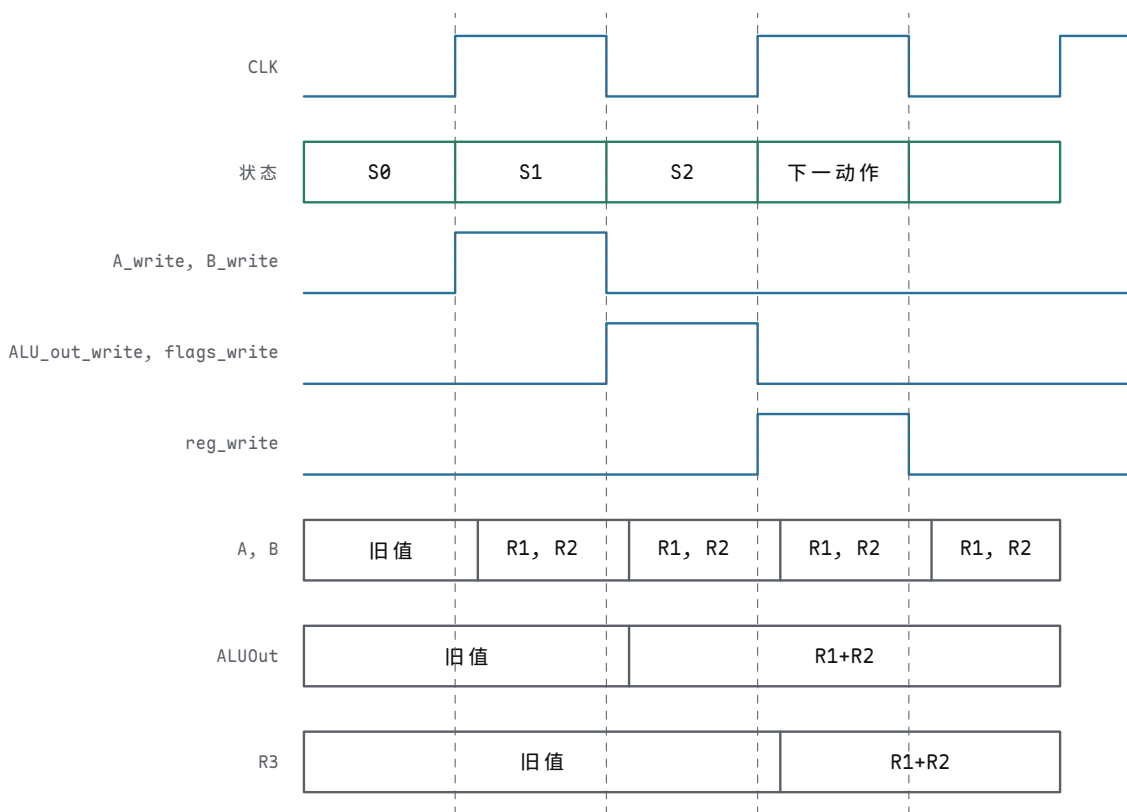
第一步 S0 需要落实为具体控制信号。`read_sel_a` 选中 R1 后，寄存器阵列的第一个读端口会把 R1 当前保存的 bit 组合驱动到读数据线上；`read_sel_b` 选中 R2 后，第二个读端口同理输出 R2。等这些组合路径稳定后，时钟沿到来，`A_write` 和 `B_write` 让 A、B 锁存器保存这两个值。这个周期里 `reg_write` 必须为 0，因为此时写回 mux 上的值并不代表最终结果。

第二步 S1 负责 ALU 计算。A、B 锁存器的输出作为 ALU 输入，`alu_op=ADD` 让 ALU 内部选择加法路径。加法电路不是瞬间得到稳定结果，低位进位会影响高位，输出线上可能经历一小段变化过程。控制器在这个周期打开 `ALU_out_write`，让时钟沿把已经稳定的 ALU 输出锁存下来；同时 `reg_write` 仍然保持 0。若本动作需要后续条件判断，`flags_write` 也在这个周期打开，把 Zero、Carry、Overflow 和 Negative 保存到标志寄存器。

第三步 S2 负责写回。`wb_sel` 选择 ALUOut 锁存器，`write_sel` 选择 R3，`reg_write` 在这个周期置 1。到时钟沿时，寄存器阵列把写数据端口上的值保存到 R3。至此，R1 和 R2 没有被改动，R3 得到 R1+R2 的结果，控制器回到空闲、取指或下一动作状态。

状态	打开的路径	周期末提交	必须关闭
S0	R1/R2 到 A/B 锁存器	A、B	<code>reg_write</code> 、 <code>ALU_out_write</code>
S1	A/B 到 ALU，ALU 到 ALUOut	ALUOut、Flags	<code>reg_write</code>
S2	ALUOut 到写回端口，写地址 R3	R3	A/B 写使能、 ALUOut 写使能

把这张表画成对齐波形后，“打开/提交/关闭”的关系就能逐项检查：



图示：三周期 $R3=R1+R2$ 的对齐波形：控制线只在规定周期为高；状态元件只在上升沿提交。注意三条控制线的互斥关系——任何一拍 `reg_write` 提前变高，R3 都会被未成熟的值污染；任何一拍 `ALU_out_write` 忘记关闭，下一次计算边界就会混入旧值。

该例也说明控制错误的风险。若 S2 的 `wb_sel` 错选成 B 锁存器，R3 写入的就是 R2，而不是相加结果；若 S1 提前打开 `reg_write`，R3 可能在 ALU 输出尚未稳定时被覆盖；若 S0 忘记关闭 `ALU_out_write`，旧的计算边界会被无关读数污染。微操作表的价值在于：每一步都能检查哪些线打开、哪些线关闭、哪个状态元件会在时钟沿改变。

再看一个条件动作：若 $R1-R2$ 的结果为 0，则下一状态跳到 `TARGET`，否则继续 `NEXT`。硬件上这仍然是数据通路和控制器配合：ALU 执行 `SUB`，Zero 标志保存比较结果，下一状态逻辑读取 Zero 并选择目标。

```
S0: read_sel_a=R1, read_sel_b=R2
    A_write=1, B_write=1

S1: a_sel=A, b_sel=B, alu_op=SUB
    flags_write=1

S2: if Zero then next_state=TARGET
    else next_state=NEXT
    reg_write=0
```

这个 trace 说明比较不是神秘动作。相等比较可以由减法和 Zero 标志实现；条件跳转不是 ALU 输出直接“跳走”，而是控制器根据标志选择下一状态或下一 PC。后面进入最小 CPU 章节时，分支、跳转、访存、I/O 和中断都可以用类似 trace 分解。

七、把第三章接到最小 CPU

到这里，CPU 的几个必要部件已经在概念上齐备。寄存器堆保存通用值，ALU 产生算术逻辑结果和标志位，数据通路用 mux 和总线把源、结果和写回连起来，控制器在每个周期输出一组控制线。最小 CPU 还需要 PC、指令寄存器、指令译码、存储器接口和复位入口，但这些部件并不是凭空出现的：PC 是带加法和选择的寄存

器，下一章要给它加上顺序取指、分支目标和异常入口；指令寄存器是保存当前指令 bit 的状态元件，下一章要解释字段和 opcode 译码；译码器是把 bit 字段翻译成控制线的组合逻辑；存储器接口是另一条受控数据通路，下一章要处理总线等待、对齐、MMIO 和异常。

把这些部件真正拼成最小 CPU 时，接口清单比概念名称更重要。下面这张表可以作为下一章的硬件检查单：每个部件不只要有名称，还要有输入、输出和写入边界。

部件	关键输入/输出	必要控制线
PC	输出取指地址，输入顺序 PC 或分支目标	pc_write、pc_sel
指令寄存器 IR	输入存储器返回的指令 bit，输出 opcode 和寄存器字段	ir_write
寄存器堆	输出两个源操作数，输入写回数据	read_addr_a/b、write_addr、reg_write
立即数扩展器	输入指令字段，输出零扩展或符号扩展值	imm_kind 或译码规则
ALU	输入两个操作数，输出结果和标志	a_sel、b_sel、alu_op、flags_write
存储器接口	地址、写数据、读数据、等待信号	mem_read、mem_write、addr_sel
控制器	输入 opcode、状态、标志和等待信号，输出控制字	next_sel、复位入口、异常入口

若这张接口表无法填写完整，最小 CPU 就还只是部件堆叠，而不是可执行机器。特别要注意两条线：第一，PC、IR、寄存器堆和标志寄存器都是状态元件，必须有明确写使能；第二，存储器接口可能比 CPU 内部组合路径慢，因此控制器必须能等待、重试或保持状态。后面讨论整机硬件时，设备寄存器、总线等待和中断入口都会沿用同一套接口分析方法。

本章也给出了调试整机硬件的基本方法。不要先问“CPU 为什么不工作”，而要按 trace 缩小范围：PC 是否在时钟沿更新，指令寄存器是否保存了正确 bit，译码是否输出了预期控制字，ALU 输入是否来自正确源，ALU 输出和标志是否正确，写回 mux 是否选对，写使能是否只在目标周期打开。这个方法从 4-bit 面包板实验到 8086、80386 这样的经典处理器分析，再到现代核心的流水线调试，层次不同，思路一致。

八、从机器级表达式读回硬件动作 到这一章末尾，读者已经能看见一条机器级表达式背后的硬件路径。高级语言里的 `x+y`、`a[i]`、`p->field`、`x & mask` 看起来属于软件语法，但映射到硬件上，它们都要变成寄存器读、立即数扩展、移位、加法、逻辑运算、LOAD/STORE、标志位和写回。若教材只讲 ALU 真值表，不讲这些表达式如何变成地址和控制线，读者会知道“加法器能加”，却不知道程序为什么会变成一连串硬件动作。

把常见表达式逐个拆开：`x+y` 读两个寄存器，ALU 加法，结果写回，对应 `alu_op=ADD`，并产生 Carry、Overflow、Zero 标志；`x-y` 让 B 取反加一、复用加法器，`sub` 同时控制 XOR 和最低位进位；`x<<k` 由移位器按移位量选择各位来源，逻辑移位、算术移位和溢出规则要分开；`x & mask` 走 ALU 逻辑路径保留部分 bit，常用于字段提取、权限位和状态位；`a[i]` 由基址加缩放下标形成地址再 LOAD/STORE，经过移位器或乘法路径、地址加法和字节使能；`p->field` 是指针基址加固定字段偏移，依赖结构体布局、对齐和访问宽度。

以 `a[i] = a[i] + 1` 为例，若 `a` 是 32-bit 整数数组，机器动作至少包含五步。第一步下标缩放：`i<<2` 或地址生成单元计算 `i*4`，元素大小必须由类型布局决

定。第二步地址形成：基址与缩放下标相加，要说明加法按地址宽度截断还是产生异常。第三步读取旧值：发起 32-bit LOAD，字节序、对齐、cache/MMIO 属性都要清楚。第四步加一：ALU 执行 `old+1`，要说明是否更新条件码、溢出如何解释。第五步写回内存：发起 32-bit STORE，字节使能必须正确，不能破坏相邻元素。每一步都能追到本章已有部件。

```
addr = base(a) + i * 4
old  = MEM[addr]
new  = old + 1
MEM[addr] = new
```

如果把 `a[i]` 错看成“数组对象知道自己的长度”，就会误解越界访问。硬件只看到基址、下标、缩放和加法。除非后续保护机制或编译器插入检查，否则 `i` 超出范围仍然会形成一个地址，并继续发起 LOAD 或 STORE。这个地址可能指向同一栈帧中的另一个变量、堆对象的元数据、返回地址、MMIO 区域，或未映射页。后面讨论异常和分页时，正是把“地址是否允许访问”提升为硬件可检查的约定。

结构体布局把字段名变成偏移 结构体字段名不是硬件对象。编译后，字段访问会变成“对象起始地址加字段偏移”。字段偏移由字段顺序、字段宽度和对齐规则决定。对齐不是排版习惯，而是为了让硬件访问落在更自然的字节边界上，减少拆分传输、简化字节使能和提高 cache 行利用率。

```
struct Packet {
    uint8_t type;
    uint8_t flags;
    uint16_t length;
    uint32_t address;
};

p->address -> LOAD32 [base(p) + 4]
```

字段	宽度	可能偏移	硬件访问
<code>type</code>	1 字节	0	字节 LOAD，通常零扩展
<code>flags</code>	1 字节	1	字节 LOAD，可与掩码配合
<code>length</code>	2 字节	2	半字 LOAD，受字节序影响
<code>address</code>	4 字节	4	字 LOAD，要求字节使能覆盖四个车道

若为了节省空间强行取消对齐，某些字段可能跨越自然边界。硬件仍可通过多个字节车道或多个总线周期完成访问，但代价会增加；某些架构会直接规定未对齐访问异常，要求软件拆成多个字节访问。教学阶段要把这件事写清楚：类型布局不是只影响内存大小，它会进入地址形成、字节选择、cache line 命中和总线传输。

位掩码是最小的机器级抽象工具 许多硬件状态寄存器不是把每个状态放进独立地址，而是把多个 1-bit 字段压在同一个字中。程序读出一个状态寄存器后，常用掩码提取某些位；写控制寄存器时，也常用掩码只改指定字段。掩码操作本质上是 ALU 的 AND、OR、XOR 和移位路径。

```
status = MMIO[UART_STAT]
ready  = (status & 0x01) != 0
error  = (status & 0x0E) >> 1

ctrl = ctrl | 0x04    // set bit 2
ctrl = ctrl & ~0x04   // clear bit 2
```



```
ctrl = ctrl ^ 0x04 // toggle bit 2
```

四种位操作各走一条 ALU 路径：AND mask 保留掩码为 1 的位、清除其他位，用于提取状态位、对齐地址和清除字段；OR mask 把掩码为 1 的位置 1，用于设置控制位和构造权限字段；XOR mask 翻转掩码为 1 的位，用于切换状态和构造取反路径；SHIFT 改变字段位置或完成缩放，用于字段对齐和数组下标乘元素宽度。

位掩码也能解释为什么设备寄存器规格必须写清楚读写语义。有些位是只读状态位，有些位写 1 清除，有些位写 0 无效，有些位一写就启动设备动作。若用普通“读-改-写”序列处理一个带副作用寄存器，可能在读时清掉事件，或在写回时误改其他字段。硬件书在这里要提醒读者：同样是 bit，普通 RAM 字节和设备寄存器字段的约定完全不同。

整数溢出要同时给出数学结果和机器结果 固定宽度整数运算的结果总是某个位宽内的 bit 模式。数学上的结果可能超出范围，此时硬件仍然给出低位截断结果和若干标志位。若程序把 `INT_MAX+1` 当作普通正数继续使用，后续比较、数组下标和地址形成都会被污染。硬件不会自动替程序选择“报错、饱和、回绕、陷入”哪一种语义，必须由 ISA、编译器、语言规则或显式指令规定。

这个表能把第三章和后面 CPU 章节连接起来。若 ISA 提供普通 ADD、带陷入 ADD、饱和 ADD 或宽乘法指令，控制器和 ALU 就必须输出不同的控制线和标志处理路径。若 ISA 只提供普通 ADD，软件就需要用比较和条件分支组合出额外检查。所谓“语言整数语义”，最后仍要体现为硬件结果、标志位、异常入口和控制流。

四种溢出语义各适合一类场景：回绕只保留低 N 位并由标志位记录越界，适合地址计数器、哈希和无符号模运算；陷入在溢出时进入异常或错误路径，适合安全语言运行时和调试检查；饱和把超范围结果钳到最大或最小值，适合 DSP、音频、图像和控制系统；扩大位宽用更宽临时结果保存数学值，适合乘法、累加和编译器优化的中间值。

案例：解析一个二进制包头 系统程序经常要解析来自网络、磁盘或外设 FIFO 的二进制包头。这个任务看起来属于软件格式处理，实际上集中使用了第三章的全部机器级工具：字节序、字段偏移、位掩码、整数扩展、对齐访问和溢出检查。假设某设备返回如下 8 字节包头：

```
offset  size  field
0       1    type
1       1    flags
2       2    length // little-endian
4       4    address // little-endian
```

解析过程可以写成：

```
type  = LOAD8 [base + 0]
flags = LOAD8 [base + 1]
len_lo = LOAD8 [base + 2]
len_hi = LOAD8 [base + 3]
length = len_lo | (len_hi << 8)
addr   = LOAD32 [base + 4]
ready  = (flags & 0x01) != 0
error  = (flags & 0x0E) >> 1
```

解析动作	硬件路径	需要确认的问题
读取 <code>type</code>	字节 LOAD 后零扩展	目标寄存器高位是否清零
读取 <code>length</code>	两个字节按小端拼接	字节序是否和设备规格一致
读取 <code>address</code>	32-bit LOAD 或四次字节 LOAD 组合	对齐、字节使能和异常规则是否清楚
提取 <code>ready</code>	AND 掩码后比较零	状态位是否被读操作清除

提取 <code>error</code>	AND 后右移	字段位置和移位方向是否正确
检查长度	与缓冲区容量比较	无符号比较和溢出检查不能混用

若 `length` 来自外部设备，不能直接用它做 `base + length`。地址加法可能回绕，长度可能超过缓冲区，字段可能被设备错误写入。可靠代码需要先做范围检查，再执行后续 LOAD/STORE。硬件层面，这些检查仍然是比较器、标志位和分支。

```
if length > buffer_capacity:
    reject packet
if base + length < base:
    reject packet      // unsigned wraparound check
payload_end = base + length
```

检查项	机器级机制	漏掉后的后果
长度上限	无符号比较 <code>length <= capacity</code>	后续 STORE 越过缓冲区
地址回绕	<code>base+length < base</code> 说明无符号回绕	访问低地址或错误对象
字段合法性	掩码后剩余位应为 0 或规定值	未定义设备状态被当成合法状态
对齐要求	<code>address & (align-1) == 0</code>	未对齐异常或拆分传输代价
权限属性	地址不能落入 MMIO 或只读区域	普通数据访问变成设备副作用或保护异常

这个案例是 CSAPP 式机器级思维和本书硬件 trace 的交汇点。读者不只要会写 C 代码，还要能指出每一步需要哪些 LOAD、移位、掩码、比较、标志和分支；更要能说明这些动作失败时，会表现为第五章讨论的总线错误、DMA 可见性问题，或第六章讨论的页错误和保护异常。

案例：多字长整数为什么要保存 Carry 很多低成本 CPU 或教学 CPU 的 ALU 宽度较小，但程序需要处理更宽整数。例如用 16-bit ALU 实现 32-bit 加法，必须先加低 16 位，再把低位加法产生的 Carry 加入高 16 位。这里 Carry 不是附属显示灯，而是下一步运算的真实输入。

```
// 32-bit A+B on a 16-bit ALU
lo = A_lo + B_lo
carry = carry_out(lo)
hi = A_hi + B_hi + carry
result = [hi:lo]
```

减法、多精度计数器、地址加法和校验和也会用到类似链路。若控制器在低位加法后错误关闭 `flags_write`，或在高位加法前让另一条指令覆盖 Carry，多字长结果就会错。这个例子帮助读者理解为什么真实 ISA 会提供 ADC、SBB 这类“带进位加/减”指令，也解释了为什么标志位是程序员可见状态的一部分。

把这条链路的控制线逐拍列清：低位加法用 `alu_op=ADD`、`carry_in=0`，预期低位结果和 CarryOut 正确；保存 Carry 用 `flags_write=1`，预期标志寄存器在下一周期仍可读；高位带进位加法用 `carry_in=Carry`，预期高位结果包含低位进位；写回组合结果时写回低位和高位目标寄存器，预期两个半字都在正确的时钟边界提交。

案例：32-bit 加法拆进 8-bit ALU 上一个案例用 16-bit ALU 分两次完成 32-bit 加法。若数据通路只有 8-bit 宽——早期微处理器和许多单片机正是如此——同一个 32-bit 加法就要拆成四步，从最低字节开始逐字节推进。每步的差别只有两处：第一步的最低字节没有进位输入，用普通 ADD；后面三步都必须把上一步的

Carry 当作本次加法的第三个输入，用带进位加 ADC。拆字长不是把同一个动作简单重复四遍，而是一条穿过四个字节的进位接力。

以 $A = 0x1FF34A80$ 、 $B = 0x0E21C590$ 为例，逐字节 trace 如下：

步骤	本字节计算	结果字节	Carry 输出与去向
1	$0x80+0x90$ (ADD, 进位输入为 0)	0x10	Carry=1, 送入步骤 2
2	$0x4A+0xC5+1$ (ADC, 加上一步 Carry)	0x10	Carry=1, 送入步骤 3
3	$0xF3+0x21+1$ (ADC)	0x15	Carry=1, 送入步骤 4
4	$0x1F+0x0E+1$ (ADC)	0x2E	Carry=0, 全和无第 33 位

四步之后从高到低拼出 $0x2E151010$ 。逐步核对数值：步骤 1 的 $128 + 144 = 272$ ，结果字节只保留 $0x10$ ，超出的 256 变成 Carry；步骤 2 的 $74 + 197 + 1 = 272$ ，同样保留 $0x10$ 并把 1 交给下一步；步骤 3 的 $243 + 33 + 1 = 277$ ，保留 $0x15$ ；步骤 4 的 $31 + 14 + 1 = 46$ ，恰好不再越界。若步骤 3 漏掉来自步骤 2 的 Carry，本字节就差 1，错误还会沿进位链继续向高字节放大。这条链路上的每个 Carry 都不是显示信息，而是下一步的真实数据输入：任何一步算完后 Carry 被意外覆盖，后面所有字节的和都不可信。

这就是 ADC 指令存在的理由。普通 ADD 的进位输入恒为 0，无法表达“把上一步的进位加进来”；ISA 若只提供 ADD，软件就要用额外的比较和条件分支手工重建进位，代价远高于一条直接读取 Carry 标志的加法指令。因此从 8080、8086 到 x86-64，指令集都把 ADD 与 ADC 成对提供：做超出寄存器宽度的加法时，最低位用 add，更高位依次用 adc，且两条指令之间不能插入会改写标志的第三条指令。硬件层面，ADD 与 ADC 往往共享同一条进位链，差别只在 carry_in 这一根线接 0 还是接标志寄存器的 Carry 输出：

```
byte0: alu_op=ADD, carry_in=0, flags_write=1
byte1: alu_op=ADC, carry_in=CF, flags_write=1
byte2: alu_op=ADC, carry_in=CF, flags_write=1
byte3: alu_op=ADC, carry_in=CF, flags_write=1
```

后续章节处理更宽的数据和地址计算时，还会反复用到这条链：宽数据被切成机器字长的若干片，片与片之间靠标志寄存器里的一位 Carry 衔接。多字长运算的正确性，最终就取决于这一位能否在两次 ALU 动作之间原样存活。

案例：把十进制字符串转成二进制整数 程序从串口、文件或命令行读到的数字是一串字符，不是数值。字符 '0' 到 '9' 在 ASCII 中的编码是 $0x30$ 到 $0x39$ ，连续排列，所以“字符减 $0x30$ ”就得到该数字的数值 0 到 9。剩下的事情是按十进制位权组装：每读到一个新数字，已累积的结果扩大十倍，再加上新数值，即 $result = result * 10 + digit$ 。以字符串 "1234" 为例：

读到字符	减 $0x30$ 得数值	累积结果 $result * 10 + digit$
'1' = $0x31$	1	$0 \times 10 + 1 = 1$
'2' = $0x32$	2	$1 \times 10 + 2 = 12$
'3' = $0x33$	3	$12 \times 10 + 3 = 123$
'4' = $0x34$	4	$123 \times 10 + 4 = 1234$

四步之后得到二进制整数 $1234 = 0x04D2$ 。循环里唯一的算术是“乘十加”，而乘十不必动用乘法器： $10x = 8x + 2x$ ，乘 8 是左移 3 位，乘 2 是左移 1 位，两次移位加两次加法即可。移位器和加法器是 ALU 里已有的路径，乘法器则是面积大、延迟长的独立部件；编译器把常数乘法改写成移位加组合，正是强度削减优化的典型例子。对应的 x86-64 汇编 (Intel 语法) 如下，设 rsi 指向字符串、遇到第一个

非数字字符结束：

```

    xor    eax, eax           ; result = 0
.next:
    movzx  ecx, byte [rsi]    ; 读当前字符并零扩展
    sub    ecx, 0x30          ; '0' 的编码是 0x30, 得数值 0..9
    cmp    ecx, 9
    ja     .done              ; 无符号大于 9: 非数字, 结束
    mov    rdx, rax
    shl    rdx, 3             ; rdx = result * 8
    shl    rax, 1             ; rax = result * 2
    add    rax, rdx            ; rax = result * 10
    add    rax, rcx            ; rax = result * 10 + digit
    inc    rsi
    jmp    .next
.done:

```

这段代码值得用本章的眼光再读一遍。`sub` 与 `cmp` 共用 ALU 的减法路径并更新标志；`ja` 是无符号条件跳转，依据 Carry 标志——字符小于 '0' 时减法产生借位、结果成为很大的无符号数，与大于 9 的情形被同一次比较一并拦下，所以一次无符号比较就完成了上下两个边界的检查。`shl` 走桶形移位器路径，`add` 走进位链，整个循环没有任何一条指令需要乘法器。字符串解析听起来是软件话题，拆到底仍然是本章的加法器、移位器和标志位。

专题：从门级代价看 ALU 为什么这样组织 ALU 表面上像一个会做很多运算的黑盒，内部通常更像若干条专用组合路径加一个结果选择器。加法/减法依赖进位链，逻辑运算依赖逐位门，移位依赖多级选择网络，比较往往复用减法路径和标志。理解这一点之后，读者就不会把 `ADD`、`AND`、`SLL`、`SLT` 当成同等代价的“软件操作”。

各项功能的来源和代价也不同：加法来自全加器链、超前进位或前缀进位网络，进位传播决定关键路径，位宽越大越明显；减法让 `B` 逐位取反并令最低位 `carry_in=1`，Carry 和 Overflow 的解释必须按减法规则重新定义；按位逻辑是每一位独立的 `AND/OR/XOR/NOT`，延迟短，但结果选择 mux 会增加负载；无符号小于看减法后的借位或 Carry 标志，不能直接看符号位；有符号小于由减法结果符号与 Overflow 共同决定，`0x8000-1` 这类边界最容易出错；移位用桶形移位器或多周期逐位移位，单周期桶形移位器面积和 mux 级数较大。

如果教学 CPU 的目标是先跑通控制 trace，可以先用纹波进位加法器和多周期移位；如果目标是提高主频，就要把加法器换成 carry-lookahead、carry-select 或 prefix 结构。这里的选择没有绝对优劣，只有约束不同：面积、功耗、最高频率、验证复杂度和 ISA 对单周期完成的承诺不同。

进位优化的核心是把“等待前一位进位”变成“并行计算哪些位会产生或传播进位”。对第 i 位，常用：

$$G_i = A_i B_i, \quad P_i = A_i \oplus B_i, \quad C_{i+1} = G_i \vee (P_i C_i)$$

其中 G 表示本位必定产生进位， P 表示本位会传播输入进位。前缀加法器会把这些关系按树形组合，缩短最长逻辑深度，但布线和门数会上升。书中不要求读者手推完整 Kogge-Stone 网络，但要求能说明：为什么一个 64-bit ALU 不能只按“64 个全加器排队”等比例粗略估计性能。

专题：ALU 状态标志的生成电路 Zero、Carry、Overflow、Negative 四个标志容易被初学者想象成“算完结果之后，再拿结果去分析一遍”的后置步骤。硬件上恰恰相反：四个标志都是同一次运算的副产物，与结果共用同一批内部信号，在同一个周期内与结果一起稳定。它们不是事后的评论，而是进位链和结果线上早已存在的消息，各自由一小段专用组合电路引出。

四个标志的来源各不相同。Negative 最简单，就是结果最高位那根线本身，补码解释下它天然指示正负，不经过任何门。Zero 是结果全部位经 OR 树归并后再取反：

任何一位为 1 都会让 OR 树输出 1，只有结果全 0 时 Zero 才为 1，其树深随位宽按对数增长。Carry 来自进位链的最后一位——加法时是最高位向外的进位，减法时按约定改读为借位；它通常是标志中稳定最晚的一个，因为进位链本来就是 ALU 的关键路径。Overflow 是符号位两处进位的异或：记最高位的进位输入为 C_{n-1} 、进位输出（即 Carry）为 C_n ，则 $V = C_{n-1} \oplus C_n$ ——两个正数相加时，若数值部分向符号位进了位而符号位没有向外进位，符号就被异常翻转，异或正好检出这种“两处进位不一致”。

标志	生成电路	延迟特点
Negative	结果最高位直接引出，不经过任何门	与结果线同时稳定
Zero	全部结果位的 OR 树加一级取反	树深随位宽对数增长
Carry	进位链末级的进位输出	与进位链同长，常是关键路径
Overflow	最高位进位输入与进位输出相异或	在 Carry 之后只多一级 XOR

理解“同周期产生”对控制器设计有直接影响。前面三周期 trace 的 S1 拍里，`flags_write` 与 `ALU_out_write` 在同一个时钟沿把标志和结果分别锁存进标志寄存器和 ALUOut，二者谁也不等谁；S2 拍的条件分支读到的标志，与写回的数据严格属于同一次运算。若把标志想象成下一周期才慢慢算出来的东西，就会错误地在控制器里多加等待拍，或者反过来在标志尚未稳定的周期提前采样。真实时序里要照顾的只是各标志延迟不同：Carry 和 Overflow 沿进位链走，稳定最晚，标志寄存器的建立时间预算必须按它们来定。

专题：乘法、除法和规格化为什么常常多周期 乘法和除法不是“更大的加法按钮”。一个 n 位乘法可以看成 n 个部分积的移位求和；一个除法可以看成反复试减、恢复或不恢复余数；浮点运算还要经历对阶、尾数运算、规格化和舍入。若所有步骤都做成单周期组合逻辑，面积和延迟会迅速失控。

每种运算的拆分方式和控制方式也不同：无符号乘法判断乘数最低位、条件加被乘数、移位、计数，用多周期移位加法或阵列/树形乘法器实现；补码乘法要处理符号扩展和负权部分积，Booth 编码能减少连续 1 的部分积数量；无符号除法左移余数、试减除数、决定商位、必要时恢复余数，用多周期状态机逐位产生商；定点乘法在原始整数相乘后调整缩放位数，乘积位宽翻倍，截断和舍入必须规定；浮点加法要对阶、尾数加减、规格化、舍入、异常标志，分流水级或由微码/协处理器完成。

以 8-bit 乘法为例，若采用移位加法，硬件只需要一个加法器、若干寄存器和一个计数器。每拍检查乘数最低位：若为 1，把当前被乘数加到累加器；然后被乘数左移，乘数右移。8 拍后得到 16-bit 乘积。这个实现慢，但面积小，特别适合早期处理器或低成本控制器。

```
product = 0
for i in 0..7:
    if (multiplier[0] == 1):
        product = product + multiplicand
        multiplicand = multiplicand << 1
        multiplier = multiplier >> 1
```

除法值得给一个完整例子。以 4-bit 恢复余数除法计算 $7 \div 2$ ：余数寄存器 R 初值为 0，被除数寄存器 Q 初值为 0111 (7)。每一拍先把 (R,Q) 整体左移一位，再试减除数；试减结果为负说明“这一位商不上”，商位记 0 并把余数加回去（恢复），否则商位记 1：

拍	(R,Q) 左移后	试减 R-2	商位	动作
1	0000,1110	$0 - 2 < 0$	0	恢复 R=0000
2	0001,1100	$1 - 2 < 0$	0	恢复 R=0001

3	0011,1000	$3 - 2 = 1$	1	保留 R=0001
4	0011,0010	$3 - 2 = 1$	1	保留 R=0001

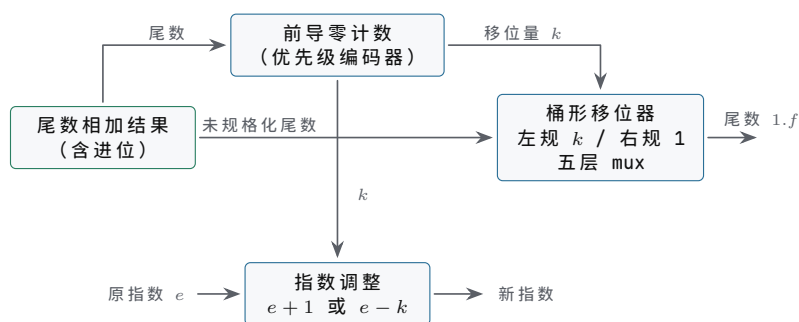
四拍后 $Q=0011=3$, $R=0001=1$: $7 \div 2 = 3$ 余 1。每一步只用移位、减法和一次条件恢复——这就是为什么除法天然是多周期状态机：每拍都有“移位、试减、判负、恢复”四个微操作，商逐位产生，没有一条能把所有位一次算完的组合捷径。

浮点的难点更隐蔽。两个指数不同的浮点数相加时，较小数的尾数要右移对齐；右移时丢掉的位会进入 guard、round、sticky 三个附加位——guard 是被移出的最高一位，round 是次一位，sticky 是其余被移出位按 OR 合并出的“是否还有 1”，舍入电路用这三个附加位决定末位该不该进 1。尾数相加后可能需要左规或右规；最后按舍入模式提交。如果读者只把浮点看成“整数加法多一个小数点”，就无法解释为什么 $(a+b)+c$ 和 $a+(b+c)$ 可能不同，也无法解释 NaN、Inf、非规格化数和异常标志。

专题：规格化左移右移的代价 浮点加减法把尾数对齐并相加之后，工作还没有结束：和或差不一定仍保持规格化形式 $1.f \times 2^e$ 。尾数相加可能产生向整数位的进位，例如 $1.110 + 1.011 = 11.001$ ，结果大于等于 2；尾数相减可能让前导 1 消失，例如 $1.001 - 1.000 = 0.001$ ，整数位变成 0。两种情况都必须把结果移回“小数点前恰好一个 1”的形式，并同步调整指数，这一步叫规格化。前一种情形右移一位、指数加一，称为右规；后一种情形左移若干位、指数减去相应位数，称为左规。

两种规格化对应两条不同的硬件路径。右规的情形简单：进位只可能多出一位，硬件只需要一条固定右移一位的通路，把多出的进位放回最高位、把末位挤进 guard 位，同时给指数加一。左规的情形麻烦得多：相减造成的抵消可能抹掉任意多个前导位，最坏情况下（两个几乎相等的数相减）需要左移几十位。因此左规路径必须先在尾数上做前导零计数——用优先级编码器找出第一个 1 的位置 k ——再按 k 左移，并从指数中减去 k 。这条“先探测移位量、再移位”的链是浮点加法器特有的组合逻辑，定点 ALU 里并不存在。

把这两条路径画出来，左规比右规多出的那级前导零计数就清楚了。



图示：规格化的两条硬件路径：尾数和送入桶形移位器的同时，也送入前导零计数（优先级编码器）找出第一个 1 的位置 k ；左规按 k 左移、指数减 k ，右规只需固定右移一位、指数加一。单精度 24 位尾数用移 1、2、4、8、16 五层 mux，任意 0 到 23 位左移在一轮组合逻辑内完成。

移位量不固定正是需要桶形移位器的原因。若用逐位移位寄存器，左规最坏要花费与尾数位宽相等的周期数，单精度是 24 位、双精度是 53 位，而且延迟随数据变化，流水级无法按固定节拍安排。桶形移位器用 $\log_2 n$ 层 mux 按二进制分解移位量：24 位尾数用移 1、2、4、8、16 五层，每层由移位量的一个 bit 控制，任意 0 到 23 位的左移都在一轮组合逻辑内完成，延迟固定且可预测。代价是面积和布线：每层都要把每个输出位接到多个候选输入之一， n 位 $\log_2 n$ 层共需约 $n \log_2 n$ 个二选一开关；对阶右移和左规两条路径还各占一套。

规格化还反过来影响舍入。右规把一位挤进 guard，左规把 guard、round、sticky 里的信息带回尾数低位，因此对阶时算好的舍入依据在规格化之后必须重新评估，舍入电路要拿规格化之后的三个附加位再做一次判断。把对阶移位、尾数相加、前导零计数、规格化移位和舍入全部塞进一个周期，关键路径会相当长；真实浮点加法器把它们分到不同流水级，这正是上一个专题说浮点运算“常常多周期”的具体原因。

专题：ALU 测试向量要覆盖边界值 ALU 的测试最怕只使用随机正常值。随机测试当然有用，但边界值才会逼出进位、溢出、符号扩展、移位宽度和比较解释错误。一张合格的测试向量表至少应覆盖零、最大正数、最小负数、全 1、单 bit、交替 bit、进位跨字节、符号位翻转和移位距离边界。

向量类型	示例	主要检查点
零值	0+0, 0-0	Zero 标志、无额外进位、写回不遗漏
无符号进位	0xFF+0x01	结果回绕和 CarryOut 同时正确
有符号溢出	0x7F+0x01	Sign 为 1 不等于“合法负数”，Overflow 必须置位
最小负数	0x80-0x01, -128/-1	补码非对称范围和除法溢出
比较混用	0xFF < 0x01	无符号为假，有符号为真
移位边界	1<<0, 1<<7, 1<<8	移位距离截断或非法规则必须明确
多字进位	0x00FF+0x0001	低字节进位进入高字节

调试时建议把每个向量写成五列：输入、期望结果、期望标志、经过的 ALU 路径、提交时钟沿。这样才能区分“组合逻辑算错”“标志解释错”“控制线选错路径”和“写回时机错”。这也是后续 CPU trace 的基础。

章末练习

本节练习要求把 bit 解释、硬件路径和控制信号联系起来思考。答案不应仅写算术结果，还要说明该结果来自哪条组合路径，在哪个时钟边界被保存，哪些控制线必须同时成立。

任务	要完成的产物	完成要求
补码解释	分别按无符号和补码解释 1000、1111、0111+0001	能区分结果 bit 与解释规则
位宽扩展	说明 1011 从 4-bit 扩展到 8-bit 时零扩展和符号扩展的差异	能说明扩展规则属于硬件路径
二进制小数	计算 10.101_2 的十进制值，并说明十进制 0.1 为什么不能有限二进制表示	能把小数位权和固定位宽联系起来
定点 Q 格式	解释 00011000 在 Q4.4 中的值，并说明两个相同 Q 格式数相加为何可复用整数加法器	能说明小数点位置是格式约定
有符号定点	解释 11110000 在有符号 Q4.4 中的值，并指出符号扩展是否仍然必要	能先按补码解释，再应用缩放因子
定点乘法	用 Q4.4 计算 1.5×0.5 的原始整数过程，说明为什么结果要右移 4 位	能说明乘法后缩放因子变化
BCD 表示	写出十进制 59 的 8421 BCD 表示，并说明 1010 为什么不是合法 BCD 数字	能区分普通二进制和十进制编码
BCD 校正	解释 BCD 中 $8+7$ 为什么需要加 0110 校正	能说明辅助进位和十进制调整的硬件意义
浮点字段	说明浮点数的符号位、指数字段、有效数字字段分别解决什么问题	能区分范围、精度和符号处理
浮点特殊值	说明零、非规格化数、Inf、NaN 为什么需要保留编码	能说明浮点不是普通整数拼接
浮点舍入	说明“大数加小数”为什么可能舍入回大数，以及这对结合律有什么影响	能把对阶、位宽和舍入联系起来

格式选型	在 MCU 传感器滤波、金融十进制显示、科学计算三种场景中选择整数/定点/BCD/浮点方案	能根据芯片约束和数值约定选型
减法复用	写出 $A-B$ 如何由加法器、XOR 反相和最低位进位输入完成	控制信号与数据路径一致
标志位	为一次加法和一次减法说明 Carry、Overflow、Zero、Negative 的来源	不把无符号进位和有符号溢出混用
进位路径	比较串行进位、超前进位和前缀加法的速度、面积、布线取舍	能指出关键路径来自哪里
桶形移位	说明 8-bit 桶形移位器怎样用 1/2/4 位阶段完成任意移位	能解释 mux 层数和延迟代价
比较器	用 $A-B$ 说明相等、无符号小于、有符号小于分别依据哪些标志	比较类型必须绑定解释规则
乘法器	比较组合阵列乘法和移位加法多周期乘法	能说明面积、延迟和周期数取舍
分立 ALU	列出用 74HC283、74HC86、74HC157 搭 4-bit 加/减/选择电路的连接要点	每个控制信号有可观察输出
74181 级联	说明两片 74181 如何组成 8-bit ALU, 以及片间进位如何连接	能区分操作数、函数选择、进位输入和进位输出
寄存器堆	说明读地址、写地址、写数据、写使能如何配合完成 $R3=R1+R2$	写回发生在时钟沿
4 寄存器实验	用 74HC173/574、74HC138、mux 设计 4 个 4-bit 寄存器的读写方案	同一周期只写一个目标, A/B 两个读端口可独立选择
总线冲突	解释为什么 ALU 输出和存储器输出不能随意直接并联	能说明 mux 或三态使能的必要性
控制器	比较硬连线控制、状态机控制和微码控制	能说明适用场景和风险
控制 ROM	给出一个控制 ROM 地址格式和控制字段, 说明 ADD 与条件分支各看哪些输入	每个控制字位都能追到实际控制线
控制 trace	为三周期 $R3=R1+R2$ 列出 S0/S1/S2 的打开路径和提交状态	能定位每个周期的写使能
最小 CPU 接口	列出 PC、IR、寄存器堆、ALU、存储器接口和控制器之间的关键输入输出	能说明哪些是组合路径, 哪些在时钟沿提交
错误定位	给出 alu_op、wb_sel、reg_write 错误各自导致的现象	能按 trace 排查, 而不是盲改 ALU
补码溢出判定	对 8-bit 运算 $01111111+00000001$ 与 $11111111+00000001$ 分别判定 Carry 和 Overflow	进位与溢出是两个独立结论, 不能互相替代
浮点加法四步	按对阶、相加、规格化、舍入计算 $1.101_2 \times 2^3 + 1.101_2 \times 2^3$, 尾数保留三位小数	能说明每步做什么、指数何时变化
CLA 展开	用 G_i 、 P_i 把 C_2 展开成只含输入位与 C_0 的表达式	能说明展开式为什么比逐级等待进位快
Booth 编码	对乘数 0110 做 Booth 重编码, 写出对应的部分积组合	能说明连续 1 串如何变成一加一减

除法演算	用 4-bit 恢复余数法演算 $13 \div 3$ ，逐拍列出 (R,Q)、试减与商位	每拍的移位、试减、判负、恢复完整
微程序读法	说明由 opcode、step 和标志拼成的地址如何读出控制字并驱动控制线	能把查表动作和实际电线对应起来

参考解答与答题要点

任务	参考解答	必须答到的要点
补码解释	1000 无符号为 8，4-bit 补码为 -8； 1111 无符号为 15，补码为 -1； 0111+0001=1000 无符号为 8，补码中是 7+1 得到 -8，发生有符号溢出。	同一 bit 模式可有不同解释，硬件只给出结果和标志。
位宽扩展	1011 零扩展为 00001011，按无符号仍为 11；符号扩展为 11111011，按 8-bit 补码为 -5。若把补码负数误做零扩展，数值语义会改变。	扩展规则要由指令语义和控制线决定。
二进制小数	$10.101_2 = 2 + 1/2 + 1/8 = 2.625$ 。十进制 0.1 不能由有限个 $1/2, 1/4, 1/8, \dots$ 权重精确相加得到，因此固定位宽二进制格式只能保存近似值。	小数点不是新电线，而是位权解释规则。
定点 Q 格式	00011000 的原始整数为 24；Q4.4 的小数位数为 4，所以实际值为 $24/16 = 1.5$ 。两个相同 Q 格式数相加时缩放因子一致，原始整数可直接送入加法器，结果仍按同一缩放因子解释。	定点加减复用整数 ALU，但格式必须一致。
有符号定点	11110000 按 8-bit 补码为 -16；在 Q4.4 中实际值为 $-16/16 = -1.0$ 。扩展到更宽路径时仍需符号扩展，否则负值会变成正的大数。	先补码解释，再除以缩放因子。
定点乘法	Q4.4 中 1.5 的原始值为 24，0.5 的原始值为 8，乘积原始值为 192。两个 Q4.4 相乘后缩放因子为 2^8 ，要回到 Q4.4 需右移 4 位得到 12，解释为 $12/16 = 0.75$ 。	乘法后必须重缩放，并决定截断、舍入或饱和。
BCD 表示	十进制 59 的 8421 BCD 为 0101 1001，每个半字节表示一个十进制位。1010 到 1111 不对应 0 到 9，属于非法 BCD 数字。	BCD 是十进制数字编码，不是紧凑二进制整数。
BCD 校正	8+7 的低半字节二进制和为 1111，不是合法 BCD。加 0110 后得到 1 0101，表示十进制进位 1 和低位数字 5。	十进制校正需要额外检测和加法路径。
浮点字段	符号位决定正负；指数字段决定尺度和动态范围；有效数字字段决定保留多少精度。规格化数通常解释为 $(-1)^s \times 1.f \times 2^{e-bias}$ 。	浮点把范围和精度分开管理。
浮点特殊值	零需要正负零编码；非规格化数让接近 0 的下溢更平滑；Inf 表示超出范围或特定除零结果；NaN 表示无效运算并可传播异常状态。	特殊编码必须由 FPU、比较器和转换路径共同遵守。

浮点舍入	浮点加法要先对阶，小数的有效数字可能在右移时被移出保留位宽之外，所以大数加小数可能舍入回大数。不同括号顺序会改变中间舍入位置，因此浮点加法通常不满足严格结合律。	对阶、规格化和舍入是硬件路径，不是显示问题。
格式选型	MCU 传感器滤波常选整数或定点，便于控制延迟、功耗和饱和；金融十进制显示可在边界使用 BCD 或十进制格式，避免十进制小数展示误差；科学计算通常使用浮点或向量浮点，换取大动态范围和通用数学库支持。	数值格式要跟芯片资源、实时性、精度约定和输入输出形式绑定。
减法复用	$A-B = A + (\text{NOT } B) + 1$ 。每个 B 位前接 XOR，sub=1 时反相；最低位 Cin=sub；sub=0 时执行普通加法。	sub 必须同时控制 B 反相和最低位进位。
标志位	Zero 来自结果各位 OR 后取反；Negative 来自最高位；Carry 来自最高位外进位或借位约定；Overflow 来自补码符号变化规则。	标志位要绑定运算类型和解释方式。
进位路径	串行进位面积小但最坏延迟随位宽增长；超前或前缀结构用更多门和布线提前计算进位，缩短关键路径；前缀结构适合宽位高性能路径但布局压力更大。	速度换来面积、功耗和布局复杂度。
桶形移位	8-bit 移位量三位可控制三层 mux：按需移 1、2、4 位，组合后得到 0 到 7 位移位。逻辑右移补 0，算术右移补符号位。	一周周期多位移位会增加 mux 延迟和连线负载。
比较器	相等看 A-B 的 Zero；无符号小于看借位或 Carry 约定；有符号小于要结合结果符号和 Overflow，例如常见组合为 Sign xor Overflow。	不能用同一标志解释所有比较。
乘法器	阵列乘法展开部分积并求和，速度和吞吐较好但面积大；移位加法每周周期处理一部分，面积小但周期数多；流水化乘法器提高吞吐但增加控制边界。	乘法仍由部分积、对齐、求和和保存构成。
分立 ALU	A/B 来自拨码开关或寄存器；B 先经 74HC86 与 sub XOR；74HC283 完成加/减；逻辑候选由门阵列产生；74HC157 选择最终输出；LED 显示结果和标志。	控制线、候选结果和最终输出要分开观察。
74181 级联	低 4 位 74181 接 bit0 到 bit3，高 4 位 74181 接 bit4 到 bit7；两片共享函数选择控制；低片 Cn+4 接高片 Cn 可形成串行跨片进位，若追求更短延迟可加入超前进位器。	级联后位宽扩大，但片间进位会占用时序预算。
寄存器堆	read_a=R1, read_b=R2, a_sel=reg_a, b_sel=reg_b, alu_op=ADD, wb_sel=ALU, write_addr=R3, reg_write=1。若采用多周期，还需 A/B 和 ALUOut 写使能。	写回必须等 ALU 输出稳定并在时钟沿提交。

4 寄存器实验	四个寄存器保存 R0 到 R3；写地址经译码产生四路候选写使能，再与 <code>reg_write</code> 相与；读端口 A/B 分别用 <code>mux</code> 按读地址选择一路输出；写回数据并接到所有寄存器 D 端但只有被选寄存器写入。	读是组合选择，写是时钟沿提交。
总线冲突	两个输出若同时驱动同一线且电平不同，会形成冲突和大电流。应使用 <code>mux</code> 明确选择一路，或用三态总线并保证同一时刻只有一个输出使能有效。	共享连线必须有唯一驱动者。
控制器	硬连线控制适合简单高频路径；状态机适合多周期阶段；微码适合复杂指令分解；混合方案让常见路径快、复杂路径可管理。	所有方案最终都要输出具体控制线。
控制 ROM	地址可由 <code>opcode</code> 、微步骤 <code>step</code> 、标志位和复位/中断请求组成；数据输出可包含 <code>alu_op</code> 、 <code>a_sel</code> 、 <code>b_sel</code> 、 <code>wb_sel</code> 、 <code>reg_write</code> 、 <code>flags_write</code> 、 <code>pc_write</code> 、 <code>ir_write</code> 、 <code>mem_read/write</code> 、 <code>next_sel</code> 。ADD 看 <code>opcode</code> 与 <code>step</code> ，条件分支还要看 Zero 等标志。	ROM 每个输出位都必须连接到一个真实控制对象。
控制 trace	S0 打开 R1/R2 到 A/B 的路径并提交 A/B；S1 打开 A/B 到 ALU 的路径并提交 ALUOut/Flags；S2 选择 ALUOut 写回 R3 并打开 <code>reg_write</code> 。	每周期要写清打开路径、提交状态和关闭信号。
最小 CPU 接口	PC 输出取指地址并由 <code>pc_write/pc_sel</code> 更新；IR 锁存指令并输出 <code>opcode</code> /寄存器字段；寄存器堆输出操作数并接收写回；ALU 产生结果和标志；存储器接口处理地址、读写数据和等待；控制器输入 <code>opcode</code> 、状态、标志和等待信号并输出控制字。	PC、IR、寄存器堆和标志是状态元件，必须有写使能。
错误定位	<code>alu_op</code> 错会算错函数； <code>wb_sel</code> 错会写回错误来源； <code>reg_write</code> 错会漏写或误写寄存器。应按控制 trace 检查状态、选择线和写使能。	先区分计算错误、选择错误和提交错误。
补码溢出判定	<code>01111111+00000001=10000000</code> ：最高位外无进位，Carry=0；补码读作 $127+1$ 得 -128 ，正加正得负，Overflow=1。 <code>11111111+00000001=00000000</code> ：最高位外有进位，Carry=1；补码读作 $-1+1=0$ ，结果合法，Overflow=0。两例各占一种组合，说明两个标志来源不同、结论独立。	进位看最高位外，溢出看符号异常，各管无符号与补码边界。
浮点加法四步	对阶：两数指数相同，无需移位。相加： $1.101 + 1.101 = 11.010$ 。规格化：和不少于 2，右规一位得 1.1010 ，指数加一变为 2^4 。舍入：保留三位小数，移出位为 0，直接舍去，结果 1.101×2^4 ，即十进制 26。	四步顺序固定，右规后指数必须同步加一。
CLA 展开	$C_1 = G_0 + P_0 C_0$ ；代入得 $C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_0$ 。展开式只含各位输入和最初进位，经两级与或门即可算出，不必等 C_1 稳定。	用额外门数换进位延迟与位宽脱钩。

Booth 编码	乘数 0110 从低位起看相邻两位，最低位右侧补 0：(0,0) → 0，(1,0) → -1，(1,1) → 0，(0,1) → +1。即第 1 位处减一次被乘数、第 3 位处加一次被乘数： $8M - 2M = 6M$ 。连续的两位 1 被编码成串尾之后 -1、串首 +1，部分积由逐位相加变成一加减组合。	部分积数量由 1 的个数变成 1 串的个数。
除法演算	13 = 1101 ，除数 3 = 0011 ，R 初值 0000 。 拍 1：(R,Q) 左移为 (0001,1010)， 1 - 3 < 0，商 0 并恢复 R；拍 2：左移为 (0011,0100)，3 - 3 = 0，商 1，得 R= 0000 、Q= 0101 ；拍 3：左移为 (0000,1010)，0 - 3 < 0，商 0 并恢复；拍 4：左移为 (0001,0100)，1 - 3 < 0，商 0 并恢复。最终 Q= 0100 =4，R= 0001 =1，验算 $3 \times 4 + 1 = 13$ 。	商逐位产生，每拍都是移位、试减、判负、条件恢复。
微程序读法	把 opcode、微步骤 step 和要检查的标志拼成 ROM 地址；该地址读出的数据行就是本周期控制字，各字段直接接 alu_op 、 a_sel 、 wb_sel 、 reg_write 、 next_sel 等控制线。例如条件分支在 EXEC 步且 Z=1 时，控制字让 next_sel 选择分支目标而非顺序状态。	微指令不是被执行的软件语句，而是被查出的电平表。

本章的经典读法是 trace：从 bit 解释，到 ALU 候选计算，到数据通路值流，再到控制信号时间表。

- 位级机制：补码取负可写成按位取反再加一；减法可复用加法器执行“B 取反、最低位进位为 1”。
- 标志规则：Carry 适合无符号边界，Overflow 适合补码有符号边界；不能把两者混成同一个“溢出”。
- ALU 层面：候选计算并行产生，选择器选出一个结果，标志位保存结果的特征。
- 数据通路层面：为 $R3 = R1 + R2$ 写读地址、ALU 输入选择、**alu_op**、写回选择、写地址、写使能。
- 控制层面：每个周期哪些写使能为 1，哪些 mux 选择哪一路，ALU 做什么，都要能列成 trace。
- 检查题：若 ALU 结果已经正确，但写回 mux 选择线接错，寄存器结果会怎样？参考答案：寄存器会保存被错误选择的来源，例如旧寄存器值、内存数据或外部输入，而不是 ALU 结果。若示波器、LED 或仿真已经证明 ALU 输出正确，排查重点应转向 **wb_sel**、写回 mux、写地址、**reg_write** 和控制器状态，而不是继续修改 ALU。

经典教材会把控制器和数据通路并排讲：数据通路说明“能走哪些路”，控制器说明“本周期允许哪条路”。两者合起来，才是可执行动作。

最小 CPU 与整机硬件

本部分导读

前面已经有了计算、状态、选择和控制。本部分先把这些部件合成一台可走读 trace 的最小 CPU，再把 CPU 接到寄存器、cache、TLB、DRAM、SSD/硬盘、MMIO、PCIe 设备、中断控制器和 DMA，通过 8086、80386、80486/Pentium、PC 主板、显卡和现代 SoC 观察真实硬件如何把这些边界逐步扩展到整机平台，最后用 74HC 器件把一台 8-bit 教学计算机从时钟到程序完整搭出来。

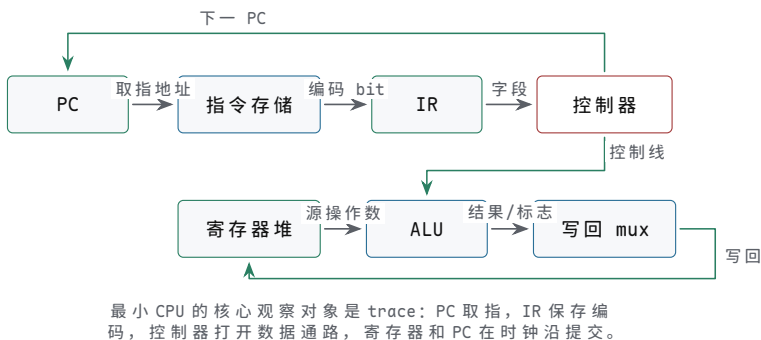
最小 CPU 与控制 trace

前面已经具备构造处理器的必要部件：组合逻辑、状态元件、时钟边界、ALU、数据通路和控制器。本章只做一件事：把这些部件合成为一台能够取指、译码、执行、写回和更新 PC 的最小 CPU。主存、总线、设备和经典芯片会在后续章节展开，本章先把 CPU 内部的状态边界和控制 trace（执行轨迹）写清楚。

CPU 可以视为一组同步硬件闭环：PC 指出下一条编码地址，取指路径取回指令 bit，译码逻辑生成控制信号，数据通路读取和计算，状态元件在时钟边界提交结果。只要能为一条指令写出 PC、IR、寄存器堆、ALU、写回 mux、标志位和控制器的完整 trace，就已经能判断这台最小 CPU 是否真的会执行程序。

核心思想

最小 CPU 不是“ALU 加寄存器”的粗线图，而是一组按周期提交的状态转换。每条指令都必须说明：本周期读哪些旧状态，组合路径经过哪些部件，哪些控制线有效，时钟沿会提交哪些新状态，异常或等待是否会阻止提交。



图示：最小 CPU 总图。箭头表示一条指令从取指到写回的完整路径，反馈线表示 PC 和寄存器堆在时钟边界更新。

一、从动作编码到控制字组织

若希望同一套数据通路在不同时间做不同动作，就需要把“这一次做什么”也表示成 bit。动作编码可以先写成最小形式：

```
opcode | dst | src_a | src_b
```

`opcode` 表示动作类型，例如 ADD、SUB、AND；`dst` 表示写入哪个寄存器；`src_a` 和 `src_b` 表示读取哪两个寄存器。控制器读取这些字段后，把它们译成 ALU op、读地址、写地址、写使能和写回选择。编码本身仍然是硬件输入，不是文字命令。

若有一组动作编码连续放在存储结构中，硬件还需要知道当前取哪一条。这个位置由 PC 保存。PC 本质上也是寄存器，只是它保存的是“下一条动作编码的地址”。

部件	作用
PC	保存下一条编码地址
指令存储器	根据 PC 输出当前编码
控制器	根据编码字段产生控制信号
寄存器阵列	保存通用数据
ALU	做整数和逻辑运算

选择器网络

选择 ALU 输入、写回来源和下一 PC

顺序执行时，下一 PC 可以由加法器产生：

```
pc_next = pc_current + instruction_size
```

如果每条编码占 4 个字节，`instruction_size` 就是 4；如果是可变长度指令，下一 PC 的计算会更复杂。关键事实是：顺序执行不是抽象的“下一行”，而是 PC 这个状态寄存器保存了新的地址。

最小单周期模型可以这样走：

阶段	硬件动作	边界
取指	PC 送入指令存储器，得到编码	组合路径
译码	控制器产生 mux、ALU、写使能信号	组合路径
执行	寄存器阵列读值，ALU 计算	组合路径
写回	结果在时钟沿写入寄存器或 PC	状态边界

用一条普通运算编码走读：当前位置 PC=100，指令存储器在地址 100 处保存的是：

```
ADD R1, R2 // R1 = R1 + R2
```

周期开始时，PC 输出稳定为 100。这个地址进入指令存储器，存储器经过地址译码后把这条编码输出。控制器看到 `opcode=ADD`，于是把 `alu_op` 设为 ADD，把寄存器阵列的两个读地址分别设为 R1 和 R2，把写地址设为 R1，把 `wb_sel` 设为 ALU，把 `reg_write` 设为 1，同时让下一 PC 选择 PC 加指令长度。

这些控制信号稳定后，寄存器阵列的两个读端口输出 R1 和 R2 当前的值。它们经过 ALU 输入选择器进入 ALU，加法路径产生结果。结果再经过写回选择器，到达寄存器阵列的写数据端口。另一边，PC 加法器准备下一条指令地址，PC 选择器把它送到 PC 的下一值输入。只有到时钟沿，两个状态变化才同时提交：R1 保存 ALU 结果，PC 保存下一条地址。

把这次走读填上具体数值，抽象的路径描述就变成了可逐项核对的数值关系。设 R1=5、R2=7、指令长 2 字节：

时刻	路径上的值	核对要点
周期开始	PC=100	取指地址稳定
取指后	IR = ADD R1,R2	编码来自 MEM[100]
译码后	alu_op=ADD，读地址 R1/R2，控制线与 opcode 一致 写地址 R1，wb_sel=ALU， reg_write=1	
读寄存器	read_a=5，read_b=7	寄存器堆当前值
执行后	ALU_out=12，写回 mux 输出 12	加法路径和写回选择正确
时钟沿	R1 <- 12，PC <- 102	两个状态同时提交，R2 不变

任何一格对不上，故障范围就锁死了：`read_a` 不是 5，查寄存器堆读地址；`ALU_out` 不是 12，查 `alu_op` 和进位链；12 没到写数据口，查 `wb_sel`；R1 没变或 PC 没变，查 `reg_write`、`pc_write` 和时钟沿本身。这就是 trace 比“检查一下”可靠的地方。

分支也可以归结为下一 PC 选择。顺序情况下，下一 PC 是 PC 加指令长度；条件分支则由 ALU 比较结果和选择器决定下一 PC 来源：

```
branch_taken = condition_met AND branch_op
next_pc = mux(branch_taken, pc_sequential, branch_target)
```

分支的硬件本质是改变 PC 这个寄存器的下一值。下一周期取到哪条编码，完全由这个新 PC 决定。

把指令周期写成逐阶段的执行过程 经典教材讲 CPU 时，常把一条指令拆成取指、译码、执行、访存、写回和下一 PC 更新。这个拆分不是为了背阶段名称，而是为了把每个阶段对应到真实硬件路径。最小单周期 CPU 可以在一个长周期内完成所有路径；多周期 CPU 可以把这些路径分到多个时钟；流水线 CPU 可以让多条指令在不同阶段重叠推进。三者的共同点是：每个阶段都要说明源状态、组合路径、目标状态和异常边界。

阶段	主要硬件动作	关键控制信号	提交边界
IF 取指	PC 送到指令存储器或 I-cache，取回指令 bit	pc_out、imem_read、ir_write	IR 或取指缓冲
ID 译码	opcode、寄存器号、立即数字段被解释成控制线	decode、imm_kind、read_addr	可组合，也可锁存
EX 执行	ALU 做加减、逻辑、比较或地址形成	alu_op、a_sel、b_sel	输出和标志
MEM 访存	对数据存储器、cache 或 MMIO 发起读写事务	mem_read、mem_write、byte_en	MDR 或外部响应
WB 写回	ALU 结果或读回数据写入寄存器堆	wb_sel、reg_write	目标寄存器
PC 更新	选择顺序 PC、分支目标、跳转目标或异常入口	pc_sel、pc_write	PC

这张表也可以直接用作调试时的排查清单。若某条加载指令读到了错误数据，不应只问“内存为什么错”，而要逐项检查：PC 是否取到正确编码，译码是否识别为加载，ALU 是否形成正确地址，MEM 阶段是否访问正确目标，读回数据是否进入 MDR (memory data register，存储器数据寄存器—访存返回数据在进入写回路径之前的暂存，写回 mux 的“存储器数据”候选就是从它引出)，WB 阶段是否选择了 MDR 而不是 ALUOut。这样的排查方式比凭感觉改控制器可靠得多。

控制字把“会执行指令”变成可接线信号 教学 CPU 最容易写成口号：取指、译码、执行、写回。真正能搭出来的版本必须有控制字。控制字不是高级软件结构，而是一组在某个周期稳定的控制线集合。它决定寄存器是否写入、ALU 选哪个操作、存储器是否读写、PC 从哪里取下一值。若控制字没有定义清楚，CPU 即使图上连通，也无法稳定执行程序。

控制对象	ADD	MOV（加载）	MOV（存储）	JZ
ALU 操作	加法或指定算术	基址加偏移	基址加偏移	比较或测试零
寄存器读	两个源寄存器	基址寄存器	基址和数据寄存器	条件源寄存器
存储器读	无	有	无	无
存储器写	无	无	有	无
写回选择	ALU 结果	存储器数据	无	无
PC 选择	顺序 PC	顺序 PC	顺序 PC	条件目标或顺序 PC

若把这些控制对象编码成 bit，可以得到类似下面的教学控制字。具体 bit 位不重要，重要的是每一位都能对应到实际连线或选择器输入。

```
control word fields:
pc_sel      // sequential, branch_target, jump_target, exception
```

```

reg_write    // enable register-file write port
wb_sel       // ALU, memory, pc_plus_size
alu_op       // add, sub, and, or, compare
alu_b_sel    // register or immediate
mem_read     // request data read
mem_write    // request data write
byte_en      // selected byte lanes
flags_write  // update Zero/Carry/Overflow/Sign

```

把控制字应用到这台具体 ISA 上，每条指令一行，全部 7 条覆盖：

指令	alu_op	a_sel	b_sel	wb_sel	reg_w	mem_r	mem_w	pc_sel
ADD	ADD	寄存器	寄存器	ALU	1	0	0	顺序
MOV (立即数)	直送	立即数	—	ALU	1	0	0	顺序
MOV (加载)	ADD	寄存器	off4 扩展	存储器	1	1	0	顺序
MOV (存储)	ADD	寄存器	off4 扩展	—	0	0	1	顺序
JZ	判零	寄存器	—	—	0	0	0	Z? 目标：顺序
JMP	—	—	—	—	0	0	0	目标
SHL	左移	寄存器	imm4	ALU	1	0	0	顺序

这张表还能进一步写成布尔式。以 `reg_write` 为例，它只在四条指令下为 1——`ADD`、立即数 `MOV`、加载 `MOV` 和 `SHL`；多条 `MOV` 在布尔式中用 `opcode` 值区分：

```

reg_write = (op == ADD) OR (op == 0010) OR (op == 0011) OR (op == SHL)
mem_read  = (op == 0011)    // MOV load
mem_write = (op == 0100)    // MOV store
pc_sel    = (op == JMP) OR ((op == JZ) AND Zero) ? target : sequential

```

译码器到此就没有黑箱了：输入 `opcode` 的 4 个 bit，输出这张表里的每一格，且每一格都能在这张真值表和数据通路图之间互相指认。若某条指令行为异常，先在这张表上查控制字哪一格错了，再回数据通路图查那条线——排查顺序永远是先控制字、后路径。

硬连线控制与微程序控制是两种组织控制字的方法 控制器可以直接由 `opcode`、状态位和当前周期译码出控制线，这称为硬连线控制。它速度快，结构适合简单指令集和高速流水线，但修改指令行为需要改组合逻辑。另一种方法是把控制字存在控制存储器中，由微程序计数器逐条取出微指令；每条机器指令对应一段微操作序列。这种方法更像“硬件内部的小程序”，适合组织复杂指令、多周期总线访问和异常入口，但控制存储器和微序列会增加延迟。

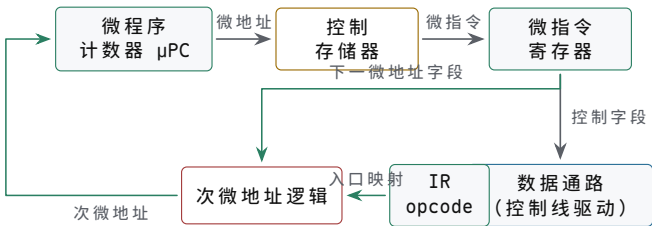
控制方式	适合场景	关键问题
硬连线控制	指令格式规整、周期少、目标频率高	<code>opcode</code> 、状态位和时序状态是否唯一决定控制线
微程序控制	指令复杂、多周期动作多、需要复用数据通路	微地址如何选择、分支微指令如何跳转、异常如何打断
混合方式	常见简单指令走快路径，复杂指令走微序列	快路径和微序列是否共享一致的提交边界

```
microinstruction sketch:
alu_op | src_a | src_b | dst | mem_cmd | pc_cmd | next_micro_pc

load sequence:
u0: IR <- MEM[PC],      next = u1
u1: A <- R[base],       next = u2
u2: ADR <- A + imm,      next = u3
u3: MDR <- MEM[ADR],     next = u4 or wait
u4: R[dst] <- MDR, PC <- PC+4, next = fetch
```

微程序写法有一个重要好处：它迫使作者说明每个周期保存什么中间状态。若某一步需要等待外部存储器，`next_micro_pc` 可以停留在当前微地址；若访问失败，微序列可以跳到异常入口。这样，等待、错误和普通写回不再混在一句“执行加载”里。

把这段微指令草图对应的硬件画出来，微程序控制器就是一台嵌在 CPU 内部的小取指机：



图示：微程序控制器的结构骨架。μPC 保存当前微地址，控制存储器按它取出整条微指令，微指令寄存器把字段分成两路：控制字段直接驱动数据通路的各条控制线；下一微地址字段与 IR 的 opcode 入口映射一起进入次微地址逻辑，决定 μPC 的下一值。访存等待时令次微地址等于当前微地址原地停留，异常时改指异常微程序入口——控制规则全部写在控制存储器的表格内容里，数据通路本身不变。

多周期 CPU 用微操作缩短最长组合路径 单周期 CPU 概念清楚，但周期必须长到容纳取指、译码、ALU、访存和写回的最坏组合路径。多周期 CPU 把同一条指令拆成若干微操作，让每个周期只完成一小段界限清楚的动作。例如加载指令可以拆为：PC 取指、译码读寄存器、ALU 形成地址、存储器读、写回寄存器。每一步之间用 IR、MDR、ALUOut 或临时寄存器保存中间结果。

周期	微操作	打开的硬件路径	提交对象
T0	IR = MEM[PC]	PC 到指令存储器，读响应到 IR	IR
T1	A = R[rs], B = R[rt]	指令字段到寄存器堆读地址	A/B 临时寄存器
T2	ALUOut = A + imm	ALU 做地址形成或比较	ALUOut/Flags
T3	MDR = MEM[ALUOut]	地址到数据存储器，读响应到 MDR	MDR
T4	R[rd] = MDR, PC = PC + 4	写回 mux 和 PC 加法器	寄存器、PC

这个 trace 说明，多周期设计不是“慢版单周期”，而是把过长的硬件路径切成可计时、可观察、可插入等待的边界。若数据存储器慢，可以只让 T3 等待；若分支比较已经在 T2 得到结果，可以在 T2 或 T3 改写 PC；若发生异常，可以把 PC 选择器改到异常入口，而不是继续提交错误写回。

微操作记法把每条指令写成可核对的寄存器传输序列 前面的微指令草图和多周期 trace 已经用过 `IR = MEM[PC]` 这类写法，现在把这套记法定死，称为寄存器传输级记法。核心约定只有三条：第一，箭头左侧是时钟沿要提交的状态元件，右侧读取的全部是本周期的旧值；第二，写在同一行里的多个传输在同一个时钟沿同时提交，行内没有先后；第三，行与行之间才是时间边界——多周期 CPU 里一行对应

一拍，单周期 CPU 里整条指令写进一行，表示这些传输在同一拍内相继稳定、同一沿提交。于是 $R[d] \leftarrow R[d] + R[b]$ 读作“时钟沿到来时，d 获得 d 与 b 旧值之和”， $MEM[R[a] + off] \leftarrow R[s]$ 读作“时钟沿把 s 的旧值写入地址 $R[a] + off$ 处”。

为什么值得为一层记法专门立约定：自然语言的“先取指、再执行”说不清哪些动作并发、哪些动作分拍，而硬件恰恰在这一点上不允许含糊。把指令写成传输序列之后，单周期与多周期之争就变成同一序列的两种排布——是一行写完，还是拆成五行、行间插入 IR、MDR、ALUOut 这些暂存元件——两种写法描述的是同一台机器。

写法	含义	核对要点
$R[x]$	寄存器堆中编号 x 的当前值	读到的是旧值
$MEM[a]$	地址 a 处的存储字	地址须先形成
$A \leftarrow B$	时钟沿把 B 提交给 A	右侧全是旧值
同一行多个传输	同一时钟沿同时提交	行内无先后
$cond ? x : y$	条件成立取 x，否则取 y	条件须提前稳定

用这套记法，本章 ISA 的四条代表指令各写成一行。注意本章 ROM 按字编址，顺序执行的下一 PC 是 $PC+1$ ：

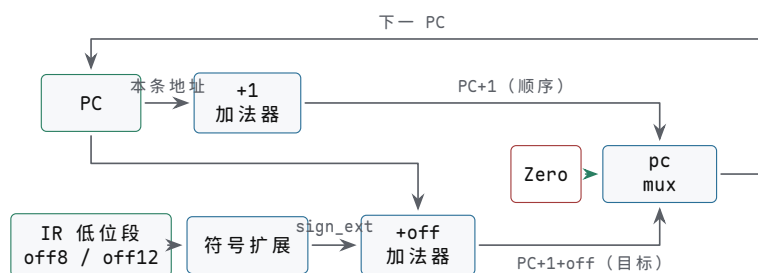
```

ADD Rd, Rs:      IR ← ROM[PC]; R[d] ← R[d] + R[b]; PC ← PC+1
MOV Rd, [Ra+off4]: IR ← ROM[PC]; R[d] ← MEM[R[a]+sign_ext(off4)]; PC ← PC+1
MOV [Ra+off4], Rs: IR ← ROM[PC]; MEM[R[a]+sign_ext(off4)] ← R[s]; PC ← PC+1
JZ Rr, +off:     IR ← ROM[PC]; PC ← (R[r]==0) ? PC+1+sign_ext(off8) : PC+1

```

每一行都能翻成数值核对。以 $PC=2$ 、 $R1=0xF000$ 执行 $MOV R2, [R1+4]$ (ROM 字 $0x3450$)：本沿读旧值 $R[a]=0xF000$ ，地址形成 $0xF000+4=0xF004$ ，提交 $R2 \leftarrow MEM[0xF004]$ 与 $PC \leftarrow 3$ 。以 $PC=3$ 、 $R2=0$ 执行 $JZ R2, +3$ ($0x5403$)：条件成立，提交 $PC \leftarrow 3+1+3=7$ ；若 $R2=1$ ，同一行记法给出 $PC \leftarrow 4$ 。多周期写法只是把这些传输拆到 $T0-T4$ 五行、行间用暂存元件保存中间值，与上一张多周期 trace 表逐行对应；单周期写法则把同一行内的传输理解为一长组合路径上相继稳定的各段。

分支目标的形成路径值得单独画一张图，两路候选 PC 和选择条件可以逐项核对：



图示：分支目标的地址形成。顺序值 $PC+1$ 与目标值 $PC+1+sign_ext(off)$ 同时算好，由 Zero 选择一路作为下一 PC 提交；目标相对下一条指令计算，偏移按补码解释、可正可负。对照 ROM 程序：地址 3 的 $JZ R2, +3$ 成立时目标是 $3+1+3=7$ ；地址 6 的 $JMP -5$ 目标是 $6+1-5=2$ ； JZ 不成立时选择 $PC+1$ ，顺序取地址 4。

同一程序在三种组织下的拍数对照 单周期、多周期和流水线不是三台不同的机器，而是同一组硬件路径的三种时间安排。比较它们必须先固定前提：三种组织使用同一批部件——同一个指令存储、同一个寄存器堆、同一个 ALU、同一个数据存储；设每段组合路径（取指、译码、执行、访存、写回）延迟相同，记为 1 个时间单位，并忽略流水寄存器自身的建立延迟。在此前提下，单周期机的时钟周期必须容纳最坏指令（加载：五段串行），即 5 个单位；多周期和流水线的周期由最长一段决定，即 1 个单位。

考察下面这个 6 条指令的小程序（取自下一节 ROM 程序的前 6 条，设按键已按下，GPIO_IN 读出 1，因此 JZ 条件不成立、顺序执行）：


```
0: MOV R1, 0xF0      // R1 = 0x00F0
1: SHL R1, 8         // R1 = 0xF000 (MMIO base)
2: MOV R2, [R1+4]    // R2 = MEM[0xF004], GPIO_IN reads 1
3: JZ R2, +3         // R2 != 0, not taken
4: MOV R3, 1         // R3 = 1
5: MOV [R1+0], R3    // MEM[0xF000] = 1, LED on
```

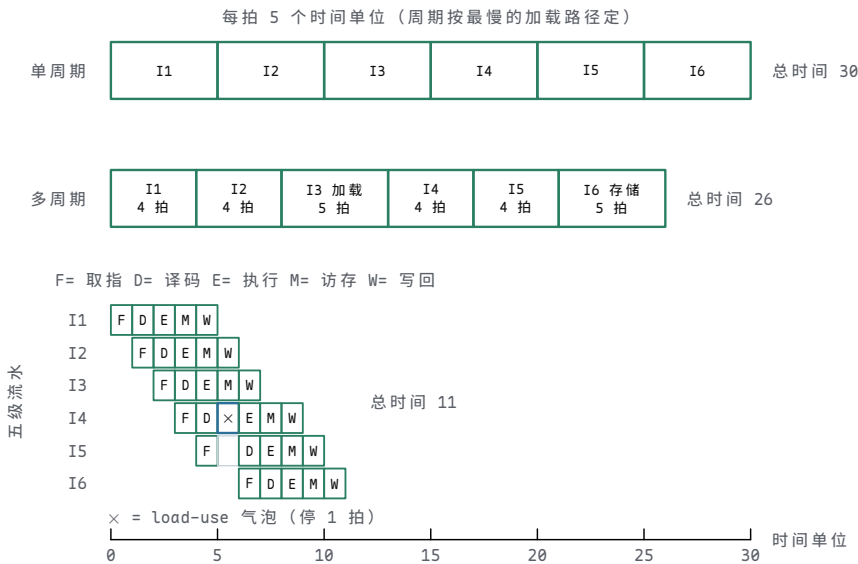
单周期组织最直截：每条 1 拍，共 $6 \times 1 = 6$ 拍，每拍 5 个时间单位，总时间 $6 \times 5 = 30$ 。

多周期组织按指令分流。公共的取指、译码各占 1 拍；立即数 MOV、SHL 与 JZ 这类非访存指令再走执行、写回，各 4 拍；加载与存储多一拍访存，各 5 拍。总拍数 $4 + 4 + 5 + 4 + 4 + 5 = 26$ ，总时间 26 个单位，平均每条约 4.3 拍。

理想 5 级流水(取指、译码、执行、访存、写回各一级)先看无冒险情形： $5 + 6 - 1 = 10$ 拍—前 4 拍是填充，此后稳态每拍完成 1 条。再逐对检查相关：SHL 读 R1 紧跟立即数 MOV，加载的基址又读 R1，这两对靠 ALU 出口到入口的转发解决，不停顿；JZ 读 R2，而 R2 来自上一条加载，数据要到访存拍末才就绪，这是 load-use 冒险，即使有转发也要停 1 拍；JZ 不跳且按“猜不跳”预取，猜中无冲刷—若 $R2=0$ 真跳，已在取指、译码级的两条预取作废，要再加 2 拍。本场景总拍数 $10 + 1 = 11$ ，总时间 11 个单位。

组织	总拍数	折合时间	关键问题
单周期	6 拍	$6 \times 5 = 30$	周期被最慢的加载路径拖长
多周期	26 拍	$26 \times 1 = 26$	周期短，但每条仍要排满 4-5 拍，平均每条约 4.3 拍
理想流水	11 拍	$11 \times 1 = 11$	填充 4 拍、load-use 停 1 拍；程序越长平均拍数越接近 1

把三种组织画到同一条时间轴上，拍数与周期长度的差别直接可见：



图示：同一程序在三种组织下的时间轴（横轴共用同一时间单位）。单周期每条 1 拍、每拍 5 个单位，共 $6 \times 5 = 30$ ；多周期非访存指令 4 拍、加载与存储 5 拍，共 $4 + 4 + 5 + 4 + 4 + 5 = 26$ ；五级流水理想情形 10 拍，JZ 读 R2 撞上上一条加载的结果，插入 1 拍 load-use 气泡，共 11。流水行里 I4 的 × 就是这个气泡：I3 的数据要到访存拍末才就绪，I4 的执行拍只能推迟一格，I5 第 6 拍的空心格是随之滞留的取指。

三个数字 30、26、11 的含义要说准确：它们不是说流水线永远快两倍以上，而是说明吞吐率来自哪里。单周期把整条路径拉长成一拍，多周期把拍切短但每条指令仍要排满几拍，流水线则让多条指令同时占据不同段。真实机器里各段延迟并不

相等、一次访存可能等几十拍、流水寄存器本身也有延迟，但只要比较的前提仍是同一组硬件路径，“切短周期、重叠执行”这个方向性结论就稳定成立。

二、极小 ISA 与可接线数据通路

先规定一个极小 ISA，程序才能进入 ROM。为了让整机实验真正运行，需要把“指令”编码成固定 bit。下面是一种 16-bit 教学编码：高 4 bit 是 opcode，中间字段选择寄存器，低位可解释为寄存器号、立即数或分支偏移。寄存器字段 3 bit，即 R0-R7，其中 R0 固定为 0。它不是任何真实 ISA 的复制品，但足以把编码、译码、控制字和 ROM 内容连接起来。

指令	编码格式	语义	需要的硬件路径
ADD R d, Rs	0001 d a b ---	$R[d]=R[d]+R[b]$	双读端口、ALU、写回
MOV R d, imm8	0010 d x imm8	$R[d]=\text{zero_extend}(\text{imm8})$	立即数扩展、写回
MOV R d, [Ra+off4]	0011 d a off4 --	$R[d]=\text{MEM}[R[a]+\text{off4}]$	地址形成、读存储、写回
MOV [Ra+off4], Rs	0100 s a off4 --	$\text{MEM}[R[a]+\text{off4}]=R[s]$	地址形成、写数据、字节使能
JZ	0101 r x off8	若 $R[r]==0$, $\text{PC}=\text{PC}+1+\text{sign_extend}(\text{off8})$	比较、符号扩展、PC 选择
JMP	0110 off12	$\text{PC}=\text{PC}+1+\text{sign_extend}(\text{off12})$	目标形成、PC 写入
SHL R d, imm4	0111 d a imm4 --	$R[d]=R[d]<<\text{imm4}$, 低位补 0	移位器、写回

两个语义细节必须先定死，后面的程序才跑得上。第一，跳转偏移都相对下一条指令的地址计算：跳转目标 = 本条地址 + 1 + 偏移，偏移按补码解释，可正可负。第二，立即数形式的 MOV 只能装 8 位立即数，高地址必须靠移位拼出来：MOV R1, 0xF0 得到 $R1=0x00F0$ ，再 SHL R1, 8 左移 8 位得到 $R1=0xF000$ 。SHL 的字段布局与 ADD 相同（d、a、低 4 位立即数），所以译码路径几乎免费。与 x86-64 的二操作数约定一致，ADD Rd, Rs 和 SHL Rd, imm4 的目的寄存器同时是源操作数，编码中 a 字段固定等于 d。另外，JZ R2, +3 把判零与跳转融合为一条指令，这是教学子集的简化；在 x86-64 中，同样的动作对应 test/cmp 与 jz 两条指令。

这个编码能直接写入 ROM。若 GPIO_IN=0xF004、GPIO_OUT=0xF000，程序先用 MOV+SHL 拼出 MMIO 基址 0xF000，再循环读取按键、条件跳转、写 LED。下面左栏是 ROM 中的真实 16-bit 字，右栏是对应汇编和语义：

地址	内容	汇编	语义
0	0x22F0	MOV R1, 0xF0	$R1 = 0x00F0$
1	0x7260	SHL R1, 8	$R1 = 0xF000$ (MMIO 基址)
2	0x3450	MOV R2, [R1+4]	$R2 = \text{MEM}[0xF004]$, 读 GPIO_IN
3	0x5403	JZ R2, +3	$R2==0$ 时跳到地址 7 ($3+1+3$)
4	0x2601	MOV R3, 1	$R3 = 1$
5	0x4640	MOV [R1+0], R3	$\text{MEM}[0xF000] = 1$, 点亮 LED
6	0x6FFB	JMP -5	跳回地址 2 ($6+1-5$)
7	0x2600	MOV R3, 0	$R3 = 0$
8	0x4640	MOV [R1+0], R3	$\text{MEM}[0xF000] = 0$, 熄灭 LED
9	0x6FF8	JMP -8	跳回地址 2 ($9+1-8$)

这个程序有三个可以直接核对的事实。第一，分支偏移逐项吻合：地址 3 的 `JZ +3` 跳到地址 7，地址 6 的 `JMP -5` 和地址 9 的 `JMP -8` 都跳回地址 2。第二，`0xF004` 这个地址在 ISA 里是可以形成的——不是注释里的愿望，而是 `0x00F0 << 8 + 4` 的真实计算。第三，ROM 里每个 16-bit 字都能被译码器逐字段读出：拿 `0x7260` 核对，高 4 位 `0111` 是 `SHL`，`d=001`、`a=001`、`imm4=1000=8`。

定长编码与变长编码的取舍 本章用的是定长编码：每条指令恰好占一个 16-bit 字，取指地址每次加 1，字段位置一成不变。代价直接可见——`ADD` 有 3 位空置，立即数 `MOV` 空 1 位且只能装 8 位立即数，`JMP` 的范围被 12 位偏移限制在 ± 2048 个字。把 ROM 程序十条的空位加总：3 条立即数 `MOV` 各空 1 位，`SHL` 与 3 条访存 `MOV` 各空 2 位，`JZ` 空 1 位，共 12 位，占全部 160 位的 7.5%。收益同样直接：译码器是对固定字段的纯组合查表，任何一条指令的边界无需计算，这为以后每拍同时取多条、译多条留了路。

x86-64 在另一极：指令长 1 到 15 字节，长度由前缀字节、opcode、ModRM 字段逐层决定——不先译出本条长度，就不知道下一条从哪个字节开始，边界本身构成串行依赖。变长换来编码密度：常用指令短，`push rbp` 仅 1 字节，`ret` 仅 1 字节；长格式只在需要时出现。一个函数序言 `push rbp; mov rbp, rsp; sub rsp, 32` 实际占 8 字节（1+3+4）；若按最长格式定长 8 字节排布，同样三条要占 24 字节。

为什么 RISC 传统坚持定长、x86 坚持变长，原因都在各自约束里。RISC 的目标是把译码做窄做快：定长让字段位置固定、多条指令的边界天然已知，译码逻辑短，才有可能每拍译出多条并维持高频率。x86 背负 8086 以来的兼容：历史二进制里充满变长编码，高密度编码也确实省指令存储和取指带宽。它的工程补偿是预取加预译码：8086 的总线接口单元带 6 字节预取队列，在执行单元忙碌时提前把后续字节取回排队，把“按字节块取指”和“按条译码执行”的速率差解耦；分支跳转时队列作废重填，这正是变长世界里控制冒险代价的一部分。现代 x86 走得更远：按 16 字节块取指，预译码逻辑标出每条指令的边界再送进指令 cache，译出的微操作还可以进微操作 cache，把变长译码的成本从“每条指令每次执行”摊薄到“每条指令一次”。

维度	本章 16-bit 定长	x86-64 变长（1-15 字节）
下一指令边界 译码逻辑	<code>PC+1</code> ，无需译码 固定字段查表，纯组合	须先译出本条长度 前缀、opcode、ModRM 逐层判定
编码密度	低，短指令也占满 16 位	高，常用指令 1-3 字节
取指方式	按字一次一条	按字节块预取，队列缓冲
每拍多条	边界已知，容易扩展	靠预译码标记和微操作 cache 补偿

指令字段放在哪里也是设计决策 本章编码里每个字段的位置都有一理由。把它们逐字段摆出来，就能看出编码格式不是随手排布，而是译码路径的形状。

opcode 固定占最高 4 位（bit 15-12）。理由是分级译码：第一级只看这 4 位就分出大类，决定低位如何解释——是寄存器号、立即数还是偏移。位置固定意味着这 4 位到译码器的连线与指令无关，取指一完成查表就开始。这个约定对人同样友好：ROM 十六进制字的第一个数字就是 opcode，`0x22F0` 的 2 是立即数 `MOV`，`0x7260` 的 7 是 `SHL`，人工核对与硬件译码走同一顺序。

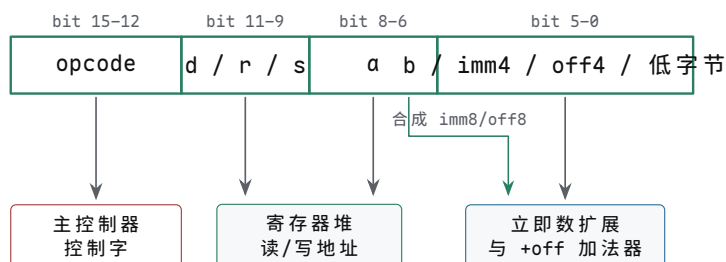
寄存器字段位置固定。`d`（以及 `JZ` 的 `r`、存储的 `s`）恒在 bit 11-9，`a` 恒在 bit 8-6。理由是寄存器堆的读地址可以直接由 IR 的固定位驱动，与 opcode 译码并行启动——读出来的值若用不上，丢弃即可，没有副作用。若字段位置随格式漂移，读地址之前就必须先插一级按格式选择的多路选择器，译码路径条数加倍，寄存器读还被推迟到译码完成之后，恰好撞上关键路径。`0x3450` 里 `d=010`（R2）、`a=001`（R1），与 `0x4640` 里 `s=011`（R3）、`a=001`，占的是完全相同的位置——这不是巧合，是让读

地址“免译码”的刻意安排。

立即数和偏移一律向低位对齐。imm4/off4 从 bit 5 起向下占 4 位，imm8/off8 占 bit 7-0，off12 占满 opcode 以外的 bit 11-0。理由是扩展逻辑单一：各种长度的立即数共享从 bit 0 起的低位段，零扩展或符号扩展只需按格式决定从哪一位开始复制符号、向上补多少位，低位部分原样旁路；指令字的 bit 0 就是立即数的 bit 0，数值意义与位置一致。JMP 把 opcode 之外全部 12 位留给偏移，符号位在 bit 11，范围 ± 2048 个字，0x6FFB 的低 12 位 0xFFB 即 -5——这是“低位全给数值、拿长度换范围”的极端情形。

字段	位置	这样放的理由	本章实例
opcode	bit 15-12	先出大类，第一级译码只接 4 位	0x22F0 首位 2 = 立即数 MOV
d/r/s	bit 11-9	第一个寄存器号位置恒定，读地址免选择	0x5403 的 r=010 即 R2
a	bit 8-6	基址/源读地址恒定，可与译码并行启动	0x3450 的 a=001 即 R1
b、 imm4/off4	bit 5 起向下	第二源与短立即数共用低位段	0x7260 的 imm4=1000=8
imm8/off8	bit 7-0	长立即数低字节对齐，扩展路径单一	0x5403 的 off8=+3
off12	bit 11-0	opcode 之外全给跳转范围	0x6FFB 低 12 位 = -5

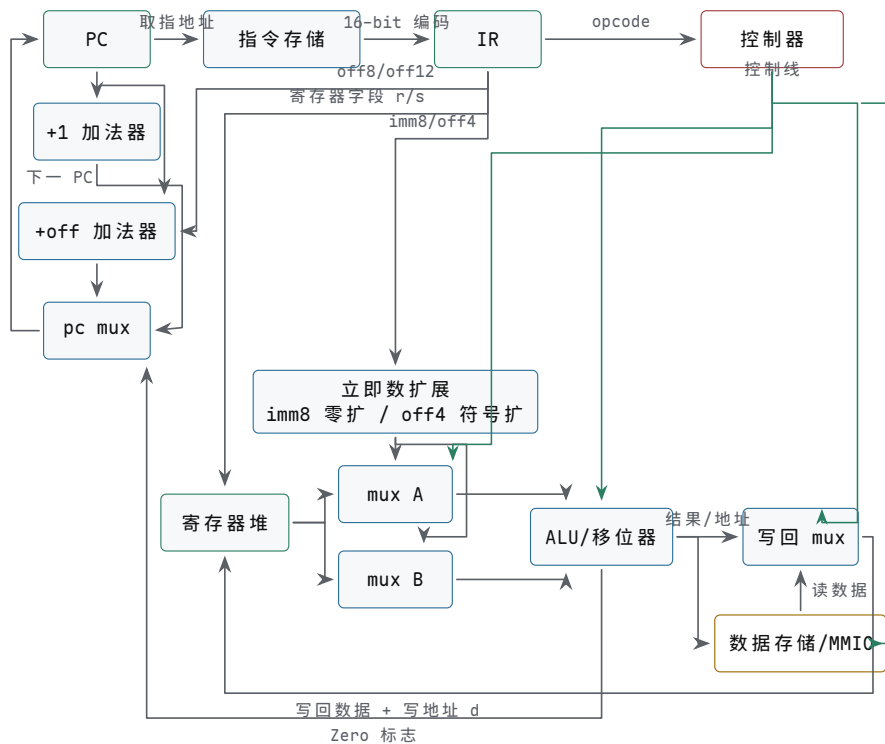
把本章 16-bit 编码按 bit 位置画成字段条，各字段的译码去向可以直接读出：



图示：16-bit 编码的字段条与各字段的译码去向。opcode 固定占 bit 15-12，第一级译码只接这 4 位；d/r/s 与 a 位置恒定，直接驱动寄存器堆的读/写地址，与 opcode 译码并行启动；低位段进入立即数扩展与偏移加法路径，长立即数 imm8/off8 由 a 字段与低位段合成（虚线）。以 0x7260 核对：高 4 位 0111 是 SHL，d=a=001，imm4=1000=8。

字段位置定下来之后，下面的数据通路图里那些从 IR 直接引出的线才有依据：寄存器字段到寄存器堆读地址的竖线、立即数扩展器的输入、偏移到 +off 加法器的路径，都对应这里的固定位置。

把这台 ISA 画成可接线的数据通路 有了确定指令集，数据通路就不能再停在“ALU 加寄存器”的粗框上。下面这张图把取指、执行、写回和 PC 更新全部画成可检查的路径，每条指令都可以在上面走一遍：



图示：这台最小 ISA 的完整数据通路。实线是数据路径，虚线是控制线。左侧一列是 PC 更新：顺序值和分支目标都先算好，由 pc mux 按 Zero 标志选择；中间是执行路径；右侧写回 mux 在 ALU 结果和存储器数据之间选择。任何一条指令的 trace，都应能在这张图上逐段指出来。

用六条指令走查这张图，可以覆盖全部指令类型：ADD 走“寄存器堆双读 → ALU → 写回 mux 选 ALU 结果”；立即数 MOV 走“imm8 零扩展 → mux A → ALU 直送 → 写回”；加载 MOV 走“基址寄存器加 off4 符号扩展 → ALU 形成地址 → 数据存储 → 写回 mux 选存储器数据”；存储 MOV 复用加载的地址路径，但数据从寄存器堆流向数据存储；JZ 的 Zero 决定 pc mux 选 +1 加法器还是 +off 加法器；SHL 走“mux A → ALU 内置移位路径 → 写回”。没有一条指令需要图外的部件——这是数据通路完整性的基本要求。

最后把 ROM 程序放进这台机器跑一整圈。设初态 PC=0、按键未按下（GPIO_IN 读 0），第 8 个时钟沿前按键被按下（GPIO_IN 读 1）：

时钟沿	PC	IR	本沿提交的状态
1	0	0x22F0	R1=0x00F0
2	1	0x7260	R1=0xF000
3	2	0x3450	R2=0（读 GPIO_IN）
4	3	0x5403	无（Zero=1，pc_sel 选目标：3+1+3=7）
5	7	0x2600	R3=0
6	8	0x4640	MEM[0xF000]=0，灯灭
7	9	0x6FF8	无（跳回 2）
8	2	0x3450	R2=1（按键已按下）
9	3	0x5403	无（Zero=0，顺序执行）
10	4	0x2601	R3=1
11	5	0x4640	MEM[0xF000]=1，灯亮
12	6	0x6FFB	无（跳回 2）

这一圈执行过程值得逐行核对：偏移算术（沿 4 的 3+1+3、沿 7 和 12 的回跳）、MMIO 地址形成（沿 3 和 8 的 0xF000+4）、条件跳转两次走向相反（沿 4 跳、沿 9 不跳）、以及程序效果本身——LED 跟随按键状态循环亮灭。能写出这张表，才说明编码、译码、数据通路、控制字和 PC 更新在同一个执行模型中互相吻合；任

何一格对不上，回到控制真值表和数据通路图逐格反查即可。

为这台 ISA 增加一条指令要走完一圈改动 以新增 `SUB Rd, Rs`（语义 `R[d] <- R[d]-R[b]`，按结果更新 Zero）为例，把“加一条指令”拆成必须同时完成的几处改动。漏掉任何一处，机器都会出现一种特征明确的故障。

第一步，定义语义与编码。语义必须连标志一起写清：结果写回 `d`，Zero 按结果更新，因为 `JZ` 依赖它。编码要找一个空 opcode：0001 到 0111 已被占用；0000 故意保留一空 ROM 字、越界取指都会读出全零，让它译成“非法指令”比译成某条真指令更容易暴露问题。于是取 1000，字段布局与 `ADD` 完全相同：1000 `d a b ---`，二操作数约定同样是 `a=d`。

第二步，改译码真值表。控制字表新增一行：`alu_op=SUB`，两个源都选寄存器，`wb_sel=ALU`，`reg_w=1`，`pc_sel=顺序`；布尔式里 `reg_write` 多析取一项（`op == 1000`）。

第三步，确认数据通路已有减法路径。减法是加补码： $R[d] - R[b] = R[d] + (\text{按位取反 } R[b]) + 1$ 。本机 ALU 的加法器与异或网络（整机实验中由 74HC283 与 74HC86 构成）天然带这条路径：`SUB` 控制线让 B 输入走异或取反、进位输入置 1 即可，不需要任何新部件。这一步的工作量是“确认”，不是“新建”。

第四步，更新写回与标志通路。写回 mux 对 `SUB` 选 ALU 结果，与 `ADD` 共用同一条写回线；Zero 在同一个时钟沿随结果提交，保证下一条 `JZ` 读到的是新值，而不是上一条指令的旧标志。

第五步，补 trace 案例。先在纸面上核对编码：`SUB R1, R2` 拼出 1000 001 0 01 010 000 = 0x8250；再放进一段小程序走数值：

时钟沿	PC	IR	本沿提交的状态
1	0	0x2209	R1=9 (MOV R1, 9)
2	1	0x2404	R2=4 (MOV R2, 4)
3	2	0x8250	R1=9-4=5, Zero=0 (SUB R1, R2)
4	3	0x8490	R2=0, Zero=1 (SUB R2, R2)

沿 4 之后 `Zero=1`，若下一条是 `JZ R2, +off` 将按新标志跳转——这一步同时检验了第四步的标志更新时序。把五步归并一下，ISA 扩展的本质是四处一起改：编码表、译码真值表、数据通路、控制字（含写回与标志）；前两处决定“新指令是谁”，后两处决定“新指令做什么”，trace 案例则把四处改动放进同一个执行模型互相印证。

改动处	关键问题	漏改时的故障
编码表	有没有空 opcode，字段布局能否复用	新编码无处安放，或与保留字冲突
译码真值表	新行与布尔式是否同步加上	新指令译成非法，或译成别的指令
数据通路	现有部件能否完成新运算	<code>alu_op=SUB</code> 没有对应连线
控制字与写回	写回选择、写使能、标志更新是否正确	结果写不回去，或 <code>JZ</code> 读到旧 Zero

三、可见状态、条件码与调用约定

程序员可见状态不是全部硬件状态 机器级程序通常只直接看见一小组状态：PC、通用寄存器、条件码或标志、可寻址内存，有些 ISA 还显式暴露栈指针、链接寄存器或状态寄存器。CPU 内部还会有 IR、MDR、ALUOut、流水线寄存器、旁路网络和微程序计数器，但这些通常不是普通程序可以直接读写的对象。区分这两类状态很重要：ISA 规定的是程序必须可依赖的边界，微结构内部状态则服务于实现速度、面积和功耗。

状态	程序员视角	硬件实现视角
PC	下一条或当前执行位置	取指地址、分支选择、异常入口和返回路径
通用寄存器	保存整数、地址、临时值和返回值	寄存器堆端口、旁路、写回优先级
条件码	最近运算留下的比较结果	ALU 标志生成、 <code>flags_write</code> 和分支条件译码
内存	按地址访问的字节对象	cache、TLB、总线、MMIO 和权限检查
栈指针	当前栈顶位置	普通寄存器加受约束的加减和加载/存储序列

这种划分能解释为什么同一套 ISA 可以有不同实现。一个简单教学 CPU 可以用多周期状态机执行加载指令；一个高性能 CPU 可以把它拆进流水线、cache 和乱序队列。只要最终提交到程序员可见状态的结果相同，内部路径可以不同。反过来，若内部优化改变了 PC、寄存器、内存或条件码的可见顺序，就不再只是实现细节，而是破坏了 ISA 约定。

条件码把 ALU 结果变成控制流 条件分支不是“看高级语言的 if”，而是看硬件已经保存的标志。ALU 做加法、减法或比较时，可以同时产生 Zero、Sign、Carry、Overflow 等标志。控制器把这些标志与指令中的条件字段组合，决定下一 PC 选择顺序地址还是分支目标。比较指令常常等价于一次不写回结果的减法：结果本身丢弃，但标志被保留下来供后续分支使用。

条件	常见硬件标志	容易犯错的解释
相等	A-B 的 Zero 为 1	只适合判断 bit 模式相等，不说明大小
无符号小于	减法借位或 Carry 约定	不能直接用结果最高位判断
有符号小于	Sign 与 Overflow 的组合	最高位受溢出影响，必须修正
结果为负	Sign 标志为 1	只在补码解释下表示负数
有符号溢出	同号相加得到异号，或减法符号规则触发	与无符号进位不是同一件事

```
CMP R1, R2      // flags = R1 - R2, result is not written
JE equal_path   // branch if Zero == 1
JL less_path    // signed branch uses Sign xor Overflow
JB below_path   // unsigned branch uses borrow/carry convention
```

这个例子说明，分支条件必须绑定解释规则。同一对寄存器 R1 和 R2 的 bit 模式，按无符号数、有符号补码数或地址偏移解释，可能得到不同的“小于”。硬件不会自动知道程序员想要哪一种比较，必须由 opcode 或条件字段告诉控制器使用哪组标志逻辑。

条件码的使用模式：比较只设标志，分支只读标志 x86-64 把一次“判断”拆成两条指令配合完成。`cmp` 与 `test` 做一次减法或按位与，运算结果本身被丢弃，不写回任何寄存器或内存，只留下 Zero、Sign、Carry、Overflow 等标志；随后的 `jz`、`jnz`、`jc`、`jnc` 等条件跳转不再访问通用寄存器，只读标志决定是否改写 PC。两条指令之间唯一的通信通道就是标志寄存器——这正是 x86-64 把标志归入程序员可见状态的原因：离开它，`cmp` 的结果无处可去。

三个最常用的配对覆盖绝大多数分支。第一，`cmp` 后接 `jz/jnz` 判断相等或不等：两操作数的 bit 模式相同则差为零，ZF 置 1。

```
cmp rax, rbx    // 计算 rax - rbx, 只写标志, 不写寄存器
jz equal_path   // ZF=1 时跳转: rax 与 rbx 相等
```

第二, `cmp` 后接 `jg/jl` 判断有符号大小: `jl` 的成立条件是 `SF` 与 `OF` 不同, `jg` 的成立条件是 `ZF=0` 且 `SF` 与 `OF` 相同。之所以要两个标志组合, 是因为减法一旦溢出, 结果最高位就不再可靠, 必须用溢出标志修正符号位。

```
cmp rax, rbx    // 同一对操作数, 同一组标志
jl less_path    // SF 与 OF 不同: 按补码解释 rax < rbx
jg greater_path // ZF=0 且 SF 与 OF 相同: rax > rbx
```

第三, `test` 后接 `jz` 判断一个寄存器是否为零: `test rcx, rcx` 计算 `rcx` 与自身的按位与, 结果必等于 `rcx` 本身, `ZF` 于是恰好表示“`rcx` 是否为 0”, 同时 `Carry` 与 `Overflow` 被清零。这是编译器生成“判零”时的惯用写法, 比 `cmp rcx, 0` 更短。

```
test rcx, rcx    // rcx AND rcx: ZF 恰好表示 rcx==0
jz loop_done     // ZF=1 时跳转: 计数器已减到 0
```

配对	判断依据	关键问题
<code>cmp + jz/jnz</code>	<code>ZF</code> , 差是否为零	只比较 <code>bit</code> 模式是否相同, 有符号与无符号通用
<code>cmp + jg/jl</code>	<code>ZF</code> 与 <code>SF</code> 、 <code>OF</code> 的组合	只适用于补码有符号比较; 无符号大小要换 <code>ja/jb</code> , 改读 <code>Carry</code>
<code>test + jz</code>	按位与结果的 <code>ZF</code>	操作数与自身相与, 等价于判零, 附带清 <code>Carry/Overflow</code>

回头看本章的教学 `ISA`, `JZ Rr, +off` 把“读寄存器、与零比较、条件改写 `PC`”三件事融合在一条指令里, 数据通路因此省掉独立的标志寄存器和 `cmp/test`, 分支可以在一个周期内完成。这是有意的教学简化, 代价同样清楚: 每条分支都要重新读一次寄存器, 而且只能判零。真实 `ISA` 把比较与跳转分开, 多付一条指令和一组必须在异常、中断时一并保护的标志状态, 换来的是一次比较可供多条后续分支复用, 判断条件也不限于零。

调用、返回和栈帧是受约束的地址事务 函数调用可以从软件语法降到三件硬件事: 保存返回地址, 跳到被调用函数入口, 为局部对象和保存寄存器预留内存。返回时再恢复必要状态, 把 `PC` 改回返回地址。很多 `ISA` 用专门的 `CALL/RET` 指令完成其中一部分; 精简教学 `CPU` 也可以用普通的存储、加载、加减栈指针和跳转指令组合出来。

```
CALL target, stack grows downward:
    RSP = RSP - word_size
    MEM[RSP] = PC + instruction_size    // push return address
    PC = target

RET:
    PC = MEM[RSP]                      // pop return address
    RSP = RSP + word_size
```

栈帧对象	机器级表示	硬件动作
返回地址	一个位宽等于 <code>PC</code> 的地址值	<code>push</code> 压栈保存, <code>RET</code> 时 <code>pop</code> 到 <code>PC</code>
保存寄存器	调用约定规定要保留的寄存器内容	进入时压栈, 返回前弹出恢复

局部变量 实参溢出区	RSP 或帧指针加固定偏移 寄存器放不下的参数位置	地址形成后普通加载/存储 调用方或被调用方按约定访问
对齐空洞	为总线宽度、ABI 或向量访问 保留的字节	调整 RSP，避免未对齐访问 代价或异常

把调用栈看成地址事务后，许多系统错误会变得具体。栈越界写不是抽象的“内存不安全”，而是一次存储指令形成了错误地址，可能覆盖相邻局部变量、保存寄存器、返回地址或未映射页。若返回地址被覆盖，RET 只是按约定把栈顶字节装进 PC；CPU 并不知道这个地址原本是否可信。后续章节讨论保护模式、页表、异常和执行权限时，就是在给这些地址事务加上硬件可检查的边界。

从一条语句贯通到硬件事务 为了避免本章变成若干概念并列，可以用一条简单语句贯通整机：`if (MMIO[GPIO_IN] != 0) MMIO[GPIO_OUT] = 1;`。在软件层面它像一次条件判断；在硬件层面，它至少包含取指、译码、地址形成、MMIO 读、条件分支、MMIO 写和 PC 更新。每一步都可以写成可逐项核对的硬件事务。

层次	发生的动作	需要观察的信号
取指	PC 指向 ROM 或 I-cache 中的 MOV/JZ 等指令编码	PC、指令存储片选、IR 内容
译码	opcode 被译成 <code>mem_read</code> 、 <code>pc_sel</code> 、 <code>mem_write</code> 等控制线	控制字和寄存器读地址
地址形成	基址寄存器与偏移形成 <code>GPIO_IN</code> 或 <code>GPIO_OUT</code> 地址	ALU 输入、ALUOut、地址总线
MMIO 读	GPIO 控制器返回已同步按键状 态	GPIO 片选、读使能、返回数 据稳定
条件判断	比较结果决定顺序 PC 或分支 目标	Zero 标志、分支控制、下一 PC
MMIO 写	输出锁存器接收写数据并驱动 LED	写使能、字节使能、锁存器输 出

这个例子也说明，编译后的指令顺序与外设副作用不能随意重排。普通内存访问可以被 cache、写缓冲和预取优化；但 `GPIO_OUT` 写入是外部可见动作，必须按设备属性和顺序规则执行。若系统把 MMIO 当作普通 cacheable 内存，LED 可能不会在预期时刻改变，甚至写入被合并或延迟到错误位置。

五类基本指令覆盖了 CPU 的主要通路 最小 CPU 不必一开始支持复杂指令，但至少要能解释五类动作：寄存器到寄存器运算、加载、存储、条件分支和无条件跳转。它们共同覆盖寄存器堆、ALU、数据存储器、PC 选择器和标志位。

指令类	典型路径	必须打开的控制	写回对象
ADD/SUB	寄存器读出，经 ALU，结果 写回寄存器	<code>alu_op</code> 、 <code>wb_sel=ALU</code> 、 <code>reg_write</code>	寄存器、 Flags
MOV（加载）	基址寄存器加立即数形成地 址，存储器读回数据	<code>alu_op=ADD</code> 、 <code>mem_read</code> 、 <code>wb_sel=MEM</code>	寄存器
MOV（存储）	基址寄存器加立即数形成地 址，源寄存器写入存储器	<code>alu_op=ADD</code> 、 <code>mem_write</code> 、 <code>store_data_sel</code>	存储器 或 MMIO
BRANCH	比较源操作数或标志位，条 件成立则选择分支目标	<code>alu_op=CMP</code> 、 <code>pc_sel</code> 、 <code>pc_write</code>	PC
JUMP	立即数、寄存器或链接地址 决定下一 PC	<code>target_sel</code> 、 <code>pc_sel</code> 、可 选 <code>reg_write</code>	PC，可 选链接 寄存器

以加载指令为例，`MOV R3, [R1 + 8]`（即 $R3 = \text{MEM}[R1 + 8]$ ）不是单纯“读内存”。硬件要先读出 `R1`，把立即数 8 符号扩展或零扩展到 ALU 输入宽度，ALU 做地址加法，地址进入数据存储路径，读响应回来后经过写回 mux，最后在时钟沿写入 `R3`。若目标地址落入普通 RAM，读回的是存储阵列内容；若目标地址落入 MMIO 区，读回的是设备寄存器当前状态。地址相同形式，语义由地址译码决定。

```
MOV R3, [R1 + 8]
EX:  addr = R1 + sign_extend(8)
MEM:  read target selected by addr
WB:   R3 = read_data
```

存储指令的风险更高，因为它有副作用。`MEM[R1 + 8] = R3` 若落在 RAM，表示保存数据；若落在设备命令寄存器，可能启动设备、清除中断或改变输出引脚。因此存储指令必须明确字节使能、写数据来源、地址空间属性和完成条件。普通内存写响应表示写事务被接受；设备命令写响应通常不表示设备动作完成。

```
MOV [R1 + 8], R3
EX:  addr = R1 + sign_extend(8)
MEM:  write_data = R3, byte_en = access_size
      target receives write command
WB:   no register writeback
```

BRANCH 和 JUMP 则说明 PC 是数据通路的一部分。条件分支一般要先得到比较标志，例如 Zero、Sign、Carry 或 Overflow，再由控制器决定 `pc_sel`。无条件跳转可以直接选择目标地址；带链接跳转还会把返回地址写入寄存器。此时同一条指令可能同时写 PC 和一个通用寄存器，控制器必须明确两个写边界是否在同一时钟沿提交。

基本块和循环把 PC 更新变成可读结构 机器级程序可以先按基本块理解。一个基本块内部只有顺序执行，入口只有一个，出口通常是分支、跳转、返回或异常边界。硬件不认识“基本块”这个软件概念，但基本块对读 trace 很有用：块内每条指令都让 PC 顺序前进，块尾那条指令才选择下一 PC。循环就是某个块尾分支把 PC 改回较低地址。

```
loop:
  mov eax, [rdi]
  add rsi, rax
  add rdi, 4
  sub rcx, 1
  jnz loop
```

程序结构	PC 动作	硬件表现
顺序指令	$PC = PC + \text{size}$	取指地址递增， <code>pc_sel=sequential</code>
条件分支	根据标志选择顺序 PC 或目标 PC	标志位、条件译码、目标加法器
循环回边	分支目标小于当前 PC 附近地址	同一组编码被重复取指
函数调用	保存返回地址并选择函数入口	链接寄存器或栈写入，PC 改为目标
返回	从寄存器或栈恢复 PC	<code>RET</code> 或间接跳转选择返回地址

用这个视角看循环，性能问题也会具体起来。循环体中的加载可能 cache miss，加法依赖加载结果，分支依赖计数器更新，取指路径又要处理回跳。简单 CPU 会逐

条完成；流水线 CPU 需要处理 load-use 冒险和分支控制冒险；带预测的 CPU 会猜测回跳是否成立。无论复杂度如何，循环仍然是 PC、寄存器、标志和内存状态反复经过同一批硬件路径。

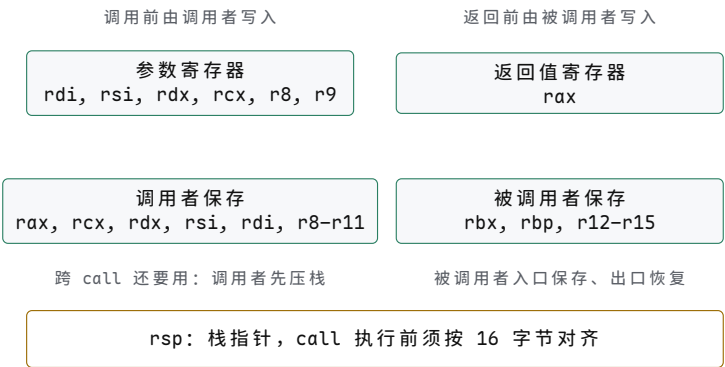
调用约定把寄存器和栈的责任写成明确约定 硬件只提供寄存器、加载/存储、PC 改写和少数专门指令；函数调用是否能可靠嵌套，还需要调用约定。调用约定规定参数放在哪些寄存器或栈位置，返回值放在哪里，哪些寄存器由调用者保存，哪些由被调用者保存，栈指针如何对齐。它不是纯软件礼节，因为每一条规则都会变成具体寄存器写回和内存访问。

调用约定对象	机器级作用	硬件动作
参数寄存器	前几个参数直接进入寄存器	调用前写寄存器，被调用函数读寄存器
返回值寄存器	函数结果放在约定寄存器	返回前写回，调用者随后读取
调用者保存寄存器	调用后不保证保持旧值	调用者必要时先压栈保存
被调用者保存寄存器	函数返回前必须恢复	被调用者入口压栈，出口弹出恢复
栈对齐	保证局部对象、返回地址和向量访问边界	调整 RSP，可能插入填充字节

```
caller:
    push r10                // caller saves if it needs r10 later
    mov edi, arg0
    mov esi, arg1
    call target
    pop r10
    use eax                 // return value

callee:
    push rbx                // callee-saved register
    ...
    pop rbx
    ret
```

这套约定在 System V AMD64 里的具体分工，可以画成一张寄存器分配图：



图示：System V AMD64 调用约定的寄存器分工。前 6 个整型参数依次进入 rdi、rsi、rdx、rcx、r8、r9，返回值进入 rax；调用者保存寄存器跨 call 不保证保持，还需要旧值时由调用者自己压栈；被调用者保存寄存器由被调用者在入口保存、出口恢复。本章 sum2 的参数 edi/esi、返回值 eax 和序言里的 push rbx，正是按这张图分工的实例。

这个 trace 能帮助读者理解为什么栈损坏常常表现成“返回后失控”。返回地址、保存寄存器和局部数组可能同在一段连续内存中。若局部数组越界写覆盖了保存寄存器，错误会在函数返回后才显现；若覆盖了返回地址，RET 会把被污染的字节装

入 PC。CPU 执行 RET 时并不知道这个地址是否来自真实 CALL，它只执行“从约定位置取一个地址并写入 PC”的硬件动作。

叶子函数与非叶子函数的保存策略不同 编译器和手写汇编都按同一条规则决定函数序言的规模：这个函数还会不会调用别人。不再调用任何函数的函数称为叶子函数。它不会触发新的 CALL，栈顶不会被再压入一层返回地址，参数寄存器、返回值寄存器这类调用者保存寄存器也可以放心使用——没有下游调用来破坏它们。本章的 `sum2` 正是叶子函数：函数体只用 `eax`、`edi`、`esi`，按 System V AMD64 约定它们都属于调用者保存（调用后不必保持原值），所以单就计算而言它连栈帧都不必建立，三条指令足够。后面案例 trace 中那对 `push rbx/pop rbx` 是为了展示序言与尾声而保留的，最小的 `sum2` 长这样：

```
sum2:           // 叶子函数：不调用别人
    mov eax, edi // 只用调用者保存寄存器
    add eax, esi
    ret          // 没有 push，也没有 sub rsp
```

一旦 `sum2` 改成自己也要调用别人——例如先调用 `sat_add` 对第二个参数做一元饱和加（约定 `sat_add(x)` 返回饱和意义下的 `x+x`），再把结果与第一个参数相加——它就不再是叶子，至少要补上三类动作。第一，保护会被下游调用破坏的现场：`edi/esi` 要腾出来给新调用传参，`call` 之后还需要的旧值必须先挪进被调用者保存寄存器或栈上。第二，凡是自己要用的被调用者保存寄存器，入口按约定保存、出口按相反顺序恢复。第三，维护栈对齐：x86-64 约定 `call` 执行前 `rsp` 是 16 的倍数，而函数入口因返回地址压栈变成 16 的倍数减 8，非叶子函数通常用一次 `push` 或 `sub rsp, 8` 把对齐调回来。

```
sum2:
    push rbx      // 保存被调用者保存寄存器，顺带对齐 rsp
    mov ebx, edi  // a 在 call 之后还要用，先进 rbx 保命
    mov edi, esi  // 为 sat_add 准备唯一参数 b
    call sat_add  // eax = 饱和的 b+b；栈顶再压一层返回地址
    add eax, ebx  // call 之后仍能取回 a，完成最后一步相加
    pop rbx       // 按相反顺序恢复
    ret
```

对比项	叶子函数	非叶子函数
可用寄存器	调用者保存寄存器可随意使用	跨过 <code>call</code> 还要用的值必须放被调用者保存寄存器或栈
栈帧	可以不建， <code>rsp</code> 全程不动	通常用 <code>push</code> 或 <code>sub</code> 建帧，出口反向拆除
返回地址	只在栈顶出现一层	每层内层 <code>call</code> 再压一层，自己的帧不能遮挡栈顶
对齐责任	无下游调用，无对齐负担	每次 <code>call</code> 前必须保证 <code>rsp</code> 按 16 字节对齐

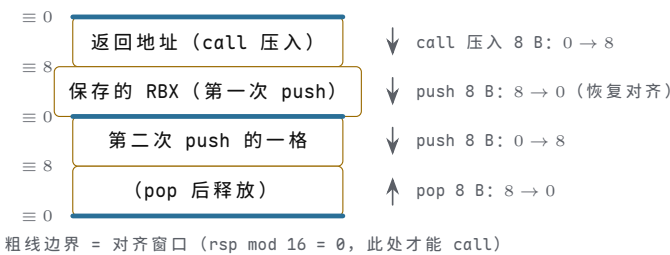
这个区分直接决定序言与尾声的指令数，也就决定小函数的调用开销。叶子函数省下的不只是几条 `push` 和 `pop`，还有对应的栈读写事务；非叶子函数少做一次保存，错误不会立刻显现，而会推迟到某次深层调用之后，以“寄存器值莫名改变”的形式暴露。调用约定的意义正在这里：保存责任按“谁会破坏、谁还需要旧值”预先分配，每个函数只管自己那一段，调用链才能任意嵌套。

栈对齐是约定的一部分，不是性能洁癖 x86-64 的 System V 约定要求：`call` 执行的一瞬间，`rsp` 必须是 16 的倍数；`call` 压入 8 字节返回地址后，被调用函数入口处的 `rsp` 就是 16 的倍数减 8。这条规则写进约定而不是留给各人随意，原因有三层。第一层是压栈粒度：一次 `push` 或 `pop` 固定移动 8 字节，栈上每一格

都是整字；若 `rsp` 不以 8 为边界，相邻两格就会错位重叠，读写互相覆盖。第二层是指令的硬性要求：`movaps` 这类对齐传送指令要求内存地址 16 字节对齐，编译器会把栈上某些局部对象按 16 字节放置，入口对齐一旦被破坏，这些指令直接触发通用保护异常。第三层是异常路径：中断与异常入口按约定位置保存现场，若干平台的异常栈帧同样假设栈指针对齐，对齐失守会让处理程序自身再出异常。

对齐的算术只有一条规律：每一次 8 字节的 `push` 或 `call`，都把“`rsp` 模 16”的余数翻转一次。入口时余数为 8，做奇数次 `push`（或一次 `sub rsp, 8`）后余数回到 0，可以安全 `call`；做偶数次则余数仍是 8，再 `call` 就把错位传给下游。上一条目里非叶子 `sum2` 用一条 `push rbx` 同时完成寄存器保存与对齐调整，正是这个算术的应用。

把每次压栈后的余数标在栈格边界上，翻转规律一眼可见：



图示：栈按 8 字节为一格，高地址在上。每次 `push` 或 `call` 压入一格， $rsp \bmod 16$ 的余数就翻转一次；`pop` 弹出一格，余数再翻转回来。`call` 要求执行瞬间余数为 0——图中粗线标出的对齐窗口；`call` 压入返回地址后，被调函数入口处的余数变为 8，序言用奇数次 `push`（或一条 `sub rsp, 8`）把余数调回 0，下游的 `call` 才落在对齐窗口上。

对齐规则	机制理由	失守后果
<code>rsp</code> 以 8 字节为粒度移动	<code>push/pop</code> 按整字压栈、读栈	相邻栈格错位重叠，数据互相覆盖
<code>call</code> 前 <code>rsp</code> 是 16 的倍数	给 <code>movaps</code> 等 16 字节对齐访问留边界	对齐传送指令触发通用保护异常
入口处余数为 8 由 <code>call</code> 压栈造成	序言用奇数次 <code>push</code> 或 <code>sub</code> 把余数调回 0	错位沿调用链向下游传播
普通访问允许未对齐	一次访问可能横跨两个 <code>cache</code> 行甚至两页	拆成两次总线事务，变慢且失去单次事务的原子性

未对齐的后果因此分两档：普通整数加载/存储未对齐时，x86-64 硬件大多照常执行，只是付出额外事务的代价；对齐传送指令、严格平台以及打开 `Alignment Check` 的 x86 则直接抛对齐异常。这与第五章总线事务的字节使能互为表里：总线按字传输时，字节使能标记本次事务实际写哪些字节，地址未对齐意味着同一数据要动用两个总线周期、两组字节使能。对齐约定把这种代价在编译期就消掉，而不是留给硬件每次补救。

四、异常、中断与精确异常

异常和中断是受控的非顺序 PC 更新 异常、中断和系统调用都可以视为特殊控制流。普通分支由指令显式请求；异常由当前指令执行过程中发现的条件触发，例如非法 `opcode`、除零、未对齐访问、页错误或总线错误；中断来自外部设备或定时器。三者最终都要让 CPU 保存足够的旧状态，并把 PC 改到一个规定入口。

事件	触发来源	入口动作
同步异常	当前指令导致，如非法指令、页错误、除零	保存故障 PC 和原因，跳异常向量

外部中断	设备、定时器或中断控制器请求	在可接收边界保存 PC/Flags，跳中断入口
系统调用	指令显式请求进入特权服务	切换权限和入口，保存返回位置
复位	电源或复位逻辑强制初始化	PC 装入复位向量，控制状态清零或设默认值

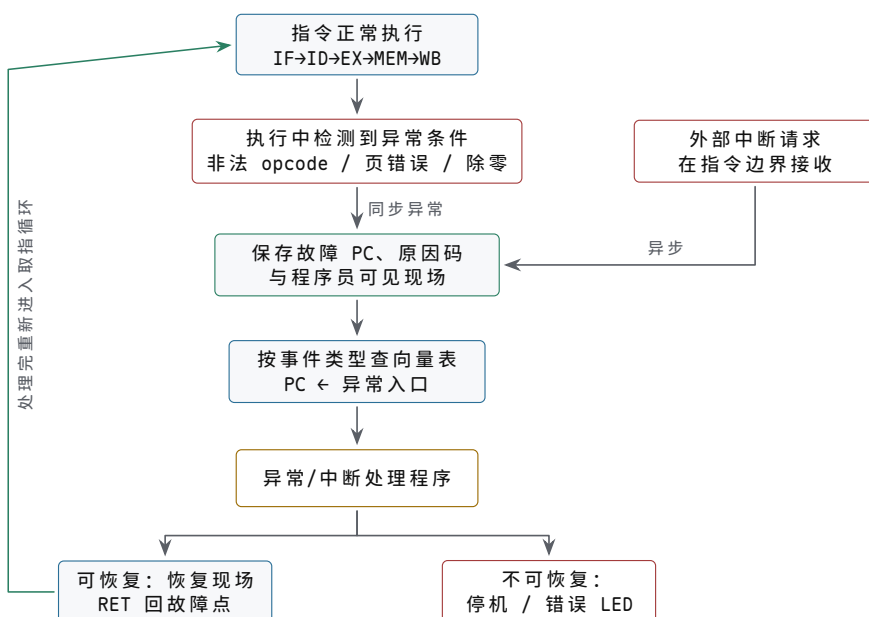
异常路径最重要的是提交边界。若一条加载指令因页错误失败，目标寄存器不应保存随机数据；若一条存储指令访问设备前发生权限错误，设备不应看到写命令；若一条除法发生除零异常，后续指令不应像已经成功执行那样提交。简单 CPU 可以在每条指令结束时才接收中断；复杂流水线 CPU 则需要记录哪些指令已经可以提交，哪些仍可能被冲刷。

```

faulting load:
IF:  fetch instruction
ID:  decode MOV
EX:  form address
MEM: address translation fails
WB:  suppressed
PC:  exception_vector

```

这个例子说明，异常不是“多跳一次分支”这么简单。它还要阻止错误写回，把失败原因带给入口，并保证重试或终止时系统能判断发生了什么。第六章讨论 80386 页错误时，会看到硬件如何保存故障线性地址和错误码；第四章这里先建立统一模型：异常是硬件检测到无法继续普通提交后，选择另一条受控 PC 更新路径。



图示：异常与中断共用同一条入口骨架：保存现场、查向量表、跳转处理程序。区别只在触发来源和接收时机——同步异常必须禁止当前指令的错误写回，异步中断只在指令边界接收。处理完后沿虚线回到取指循环；裸机程序若无法恢复，至少要把系统停在一个可诊断的状态，而不是带着错误现场继续跑。

中断向量把异常号变成入口地址 异常或中断被接收之后，硬件面对的问题很具体：PC 应该改成多少。普遍做法是把所有入口预先存成一张向量表——本质是一个“异常号映射入口地址”的数组。事件类型被编码成异常号（中断号、向量号），硬件拿它做下标查表：表项地址 = 表基址 + 异常号 × 表项宽度，一次访存就得到入口，CPU 不需要逐条询问设备。

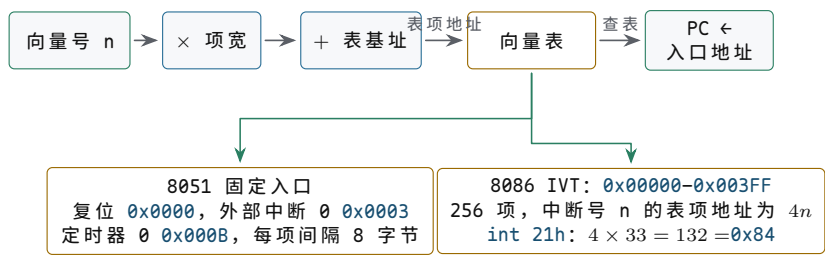
慢的对照方案是轮询式。多个设备共享一条中断请求线时，CPU 进入公共处理程序，逐个读各设备的状态寄存器，读到哪家的请求标志置位就处理哪家。若共享 5

个设备，读一次状态寄存器算一次 20 个周期的总线事务，最坏情况下来源排在最后，仅确认来源就要 $5 \times 20 = 100$ 个周期，真正的处理还没开始；设备数增加，这笔开销线性上升。向量式让设备在中断响应周期直接把向量号送上总线（8086 系统中由 8259 中断控制器完成这个动作），确认来源的开销与设备数无关。

8051 是固定向量的例子：向量表从程序存储器地址 0 开始，每个入口间隔 8 字节—复位用 0x0000，外部中断 0 用 0x0003，定时器 0 用 0x000B，外部中断 1 用 0x0013，定时器 1 用 0x001B，串行口用 0x0023。8 字节只够放一条跳转指令加一点收尾，装不下完整处理程序，所以表项里通常就是一条跳往真正处理程序的跳转指令。8086 则是数组式向量表的完整形态：内存最低端 0x00000-0x003FF 的 1 KB 放 256 项，每项 4 字节（2 字节偏移加 2 字节段地址），中断号为 n 的表项地址就是 $4n$ 。以 int 21h 为例，中断号 0x21 即十进制 33， $4 \times 33 = 132$ ，也就是 0x84，它的入口存在 0x00084 开始的 4 个字节里。

机器	表项定位	表项内容	组织特点
8051	固定地址，间隔 8 字节	通常是一条跳转指令	入口少；同一入口有多个来源时仍要软件再分辨
8086	$4n$ (n 为中断号)	偏移加段地址共 4 字节	256 项统一索引，软中断与硬件中断同表
教学模型	表基址 + 异常号 $\times$ 项宽	处理程序入口地址	本章异常入口沿用同一查表动作

把这条查表路径与两种真实组织画在同一张图上，查表动作就很具体了：

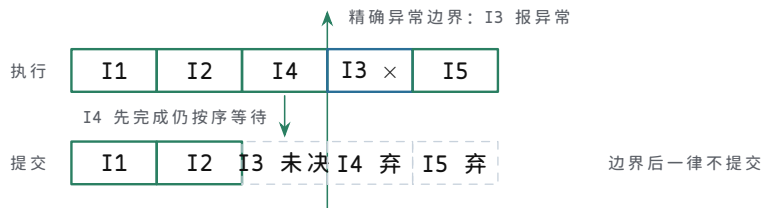


图示：向量表的查表路径：向量号乘以表项宽度、加上表基址得到表项地址，一次访存取出入口地址装入 PC。8051 把少数入口固定在程序存储器低端、每项 8 字节只够放一条跳转指令；8086 在内存最低 1 KB 放 256 项、每项 4 字节，软中断与硬件中断共用同一张表、同一个下标规则。

向量表把“谁请求”翻译成“去哪里”，此后的入口动作—保存现场、切换权限、阻止错误写回—对各类事件是公共的，这正是上一主题那张异常骨架图中“查向量表”一格的具体内容。还要留意一个边界：向量表本身也是内存，表项若被写坏，事件到来时 PC 会得到一个无意义地址。保护模式下的 x86 把向量表（IDT）的基址与界限交给特权寄存器管理，正是为这张表加上访问边界。

精确异常要求程序员可见状态像顺序执行 流水线和乱序执行会让多条指令同时处在不同阶段。若较晚指令已经写回，较早指令随后发现异常，程序员可见状态就会变得难以恢复。精确异常要求异常发生时，状态看起来像按程序顺序执行到某条指令：异常之前的指令都已提交，异常指令和之后的指令都没有提交。教学 CPU 可以通过按序完成天然满足；现代 CPU 需要提交队列、重排序缓冲或类似机制。

把执行与提交画成两条时间轴，精确异常的边界位置就具体了：



图示：执行与提交是两条不同的时间轴。乱序机器里 I4 可能比 I3 先执行完成，但提交严格按程序顺序排队，I4 的结果只能先留在内部寄存器里等待；I3 在边界处报异常时，I1、I2 已经提交，I3 自身不提交，I4、I5 一律丢弃——程序员可见状态就像顺序执行到 I3 为止，这正是精确异常要求的形态。

机制	解决的问题	代价
按序流水线	后一条指令不越过前一条提交	吞吐有限，长延迟指令会拖住后续
提交队列	执行可乱序，提交按程序顺序	需要记录目的寄存器、异常原因和结果
寄存器重命名	消除部分假依赖，允许更多并行	需要映射表、空闲表和恢复机制
冲刷流水线	分支预测错或异常时丢弃错误路径	浪费已经做的工作，需要恢复 PC 和状态

虽然本书不展开乱序核心设计，但提前理解“执行”和“提交”的区别很重要。ALU 算出结果只是执行完成，结果进入程序员可见寄存器才是提交。加载指令从 cache 取回数据只是访存完成，目标寄存器被写回才是提交。存储更复杂：它可能先进入 store buffer，等确认没有异常、顺序允许、地址属性允许后才对 cache、内存或设备可见。这个区分把第四章 CPU trace、第五章内存顺序和第六章平台异常连成一条线。

程序时间等于指令数乘 CPI 再乘时钟周期 评价一台 CPU 跑一个程序有多快，只需一个乘法：

$$\text{程序时间} = \text{指令数} \times \text{CPI} \times \text{时钟周期} = \text{指令数} \times \text{CPI} / \text{主频}$$

指令数由程序、编译器和 ISA 决定；CPI（每条指令的平均周期数）由微结构与访存行为决定；时钟周期由工艺和最长组合路径决定。三个因子各自独立地进入乘积，任何优化都必须先说清楚自己动的是哪一个。

把三个因子画成乘积结构，每个因子由什么决定就各有归属：



图示：程序时间的三因子分解。指令数由程序、编译器与 ISA 决定，CPI 由微结构与访存行为决定，时钟周期由工艺与最长组合路径决定；三个因子独立进入乘积，压低任何一个都可能抬高另一个。本节第一组演算中，同一程序 100 万条指令、周期同为 10 ns，CPI=1 的机器甲用 10 ms，CPI=4 的机器乙用 40 ms——时间差恰好等于 CPI 差。

第一组演算：同一个程序（同一份二进制，100 万条指令）分别跑在两台 100 MHz 的机器上，时钟周期都是 10 ns。机器甲是理想单周期，CPI=1；机器乙是多周期，CPI=4。

机器	CPI	周期	程序时间
甲（单周期）	1	10 ns	$10^6 \times 1 \times 10 \text{ ns} = 10 \text{ ms}$
乙（多周期）	4	10 ns	$10^6 \times 4 \times 10 \text{ ns} = 40 \text{ ms}$

周期相同的前提下，时间差恰好等于 CPI 差：乙比甲慢 4 倍，多花的 30 ms 全部来自每条指令多用的 3 个周期。但不能反推出“CPI 低就一定快”。多周期设计敢于把周期做短，正是因为每个周期只做一小段：若甲按单周期实现，最长组合路径要求 40 ns 的周期，甲的实际时间变成 $10^6 \times 1 \times 40 \text{ ns} = 40 \text{ ms}$ ，与乙持平。只看 CPI 或只看主频都会误判，三个因子必须一起算。

第二组演算：cache 命中率对有效 CPI 的影响。cache 全命中时基础 CPI 为 1；设平均每条指令引起 1.2 次存储器访问（取指 1 次，约 20% 的 load/store 指令再贡献 0.2 次），miss 率 2%，每次 miss 罚 50 个周期，则：

$$\begin{aligned} \text{有效 CPI} &= \text{基础 CPI} + \text{每条指令访存次数} \times \text{miss 率} \times \text{miss 代价} \\ &= 1 + 1.2 \times 0.02 \times 50 = 1 + 1.2 = 2.2 \end{aligned}$$

98% 的命中率听起来很高，却因为 50 周期的 miss 代价把有效 CPI 抬到 2.2，程序时间变成理想情形的 2.2 倍。miss 代价越大，命中率小数点后的每一位就越值钱，这也是存储层次要在本书后文独占整章的原因。

提速途径	典型手段	付出的代价
减少指令数	更好的算法、编译器优化、表达力更强的 ISA	单条指令更重，CPI 或周期可能变长
降低 CPI	流水线、cache、旁路、分支预测	控制变复杂，冒险与精确异常都要额外机制
缩短周期	更深流水、更短组合路径、更先进工艺	级间寄存器开销增加，CPI 可能回升

三条途径共用同一个乘积，所以彼此牵制：复杂指令集用一条指令做更多事来减少指令数，往往抬高 CPI；深流水缩短周期，却让分支预测错误的冲刷更昂贵。本章的最小 CPU 在三条路上都走得很保守—指令数不省、CPI 接近 1、周期却必须包容最长路径—它存在的意义是把乘积里的每一项都暴露出来，让后续每个性能相关的章节都能对号入座。

五、整机实验、调试顺序与案例 trace

最小整机实验要先有明确的观察点 若在一块低速实验板上实现本节内容，不必立即追求完整 ISA。可以先定义四条动作：ADD、MOV（加载）、MOV（存储）、JZ。指令 ROM 提供编码，4 个寄存器保存通用值，74HC283/74HC86 构成加减核心，SRAM 或寄存器阵列模拟数据存储器，若干 LED 和拨码开关映射成 MMIO。实现要点是每一步都可观察：PC LED 显示当前取指地址，IR LED 或逻辑分析仪显示当前指令，控制线可单步观察，写回和 PC 更新只在时钟沿发生。

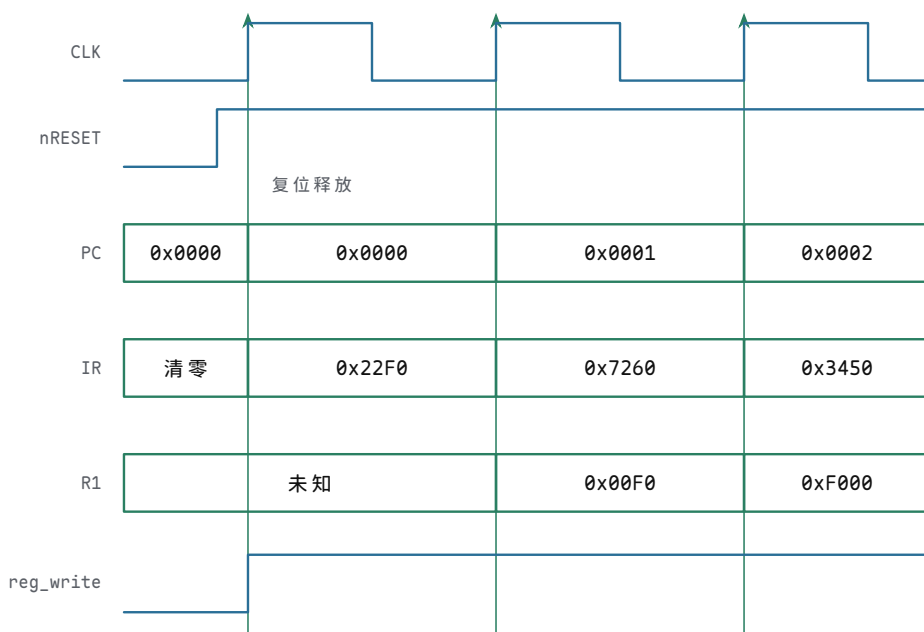
实验对象	可用器件或实现	可观察行为
指令 ROM	EEPROM、Flash 或预置拨码/仿真 ROM	复位后固定地址输出第一条编码
PC/IR	74HC161、74HC574 或触发器组	PC 每步按控制更新，IR 保存本周期指令
数据 RAM	SRAM、寄存器阵列或仿真存储器	加载/存储的地址、数据、写使能可观察
地址译码	74HC138、比较器或门阵列	RAM、LED、按键寄存器不会同时响应
MMIO LED	锁存器驱动 LED	写指定地址后 LED 状态在边界改变
MMIO 按键	上拉/下拉、施密特、同步器	读指定地址返回已同步状态，不直接用生按键

最小整机的单步调试顺序 搭建教学 CPU 时，最可靠的调试顺序不是一次性运行程序，而是逐个检查每个状态边界。先让复位把 PC 固定到入口地址，再单步确认 ROM 输出第一条指令；随后确认 IR 能锁存该指令，译码能产生正确控制线；再用一条 ADD 验证寄存器读、ALU 和写回；最后才加入加载、存储、分支和 MMIO。

调试阶段	操作	预期行为
复位	拉低/拉高复位并给一个时钟	PC 固定为入口，写使能均处于安全值

取指	单步一个取指周期	ROM 片选有效, IR 保存预期编码
ADD	预置 R1/R2, 执行 <code>ADD R1,R2</code>	R1 在时钟沿变为正确和, PC 前进
MOV (加载)	RAM 某地址预置数据, 执行加载	地址、读使能、写回来源均正确
MOV (存储)	执行写 RAM 或写 LED 寄存器	只有目标片选有效, 非目标不被改写
分支	改变 Zero 条件重复执行	PC 在顺序地址和分支目标之间正确选择

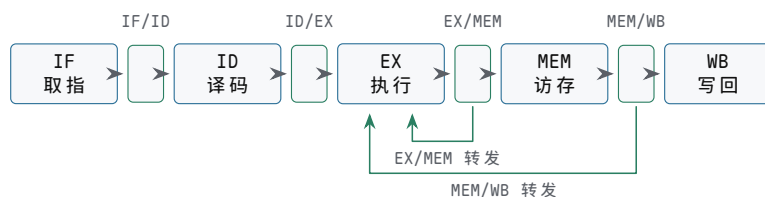
这种逐步观察的方式也适用于 FPGA。逻辑分析仪、仿真波形或片上 ILA 应至少抓取 PC、IR、控制字、ALU 输入输出、内存地址、读写使能、写回数据和寄存器写地址。若只看最终 LED 是否闪烁, 错误可能被循环掩盖; 若能看到每条指令的提交边界, 错误就能定位到具体周期。



图示：复位后前三个周期的时序，对照 § 二 ROM 程序开头三条编码（`0x22F0=MOV R1,0xF0`、`0x7260=SHL R1,8`、`0x3450=MOV R2,[R1+4]`）。复位把 PC 保持在 0x0000、写使能保持在安全值；释放后每个周期内组合路径完成取指、译码、执行，周期末上升沿才提交新状态—0x00F0 在第二个上升沿、0xF000 在第三个上升沿写进 R1。这张波形图直观展示了“状态只在时钟沿改变”，单步调试时应逐格核对。

流水线把吞吐率问题提前暴露出来 本节及其后的 scoreboard 专题属于扩写目录中“流水线、冒险和异常”一章的提前浓缩，标记为选读：第一遍读最小 CPU 时可以先跳过，等单周期 trace 走通后再回来。一旦把取指、译码、执行、访存和写回拆成流水级，CPU 的平均吞吐率可以提高，但控制复杂度也立刻上升。相邻指令可能争用同一个存储器端口，后一条指令可能读取前一条尚未写回的寄存器，分支可能让已经取到的指令变成错误路径。这些问题统称结构冒险、数据冒险和控制冒险。

先把五级流水的基本结构画出来，再讨论它引来的冒险：



图示：五级流水的基本结构。IF、ID、EX、MEM、WB 五级之间用级间寄存器隔开，每拍所有级并行推进不同指令，级间寄存器在时钟沿把本级的结果与控制权交给下一级。虚线是旁路转发：EX/MEM、MEM/WB 级间寄存器里的新结果不等写回，直接回送 EX 级的 ALU 输入，大多数寄存器依赖因此不必停顿；load-use 这种数据要到访存拍末才就绪的情形，仍要停一拍。

冒险类型	例子	硬件处理
结构冒险	取指和加载同时争用单端口存储器	分离指令/数据存储，或插入停顿
数据冒险	ADD R1,... 后立刻 SUB R2,R1,...	旁路/转发，或等待写回后再读
控制冒险	条件分支结果未出时已取后续指令	预测、延迟槽、停顿或冲刷流水线
异常冒险	较晚指令先完成，较早指令发生异常	提交队列或按序提交保证精确状态

因此，现代芯片低时延不只是“流水线更深”。流水线必须配套转发网络、停顿控制、冲刷控制和精确异常边界。否则，吞吐率提高会以错误状态提交为代价。学习最小 CPU 时提前看到这些冒险，可以防止把流水线误解成简单地把框图切成几段。

案例：从一个计数循环走到逐周期 trace 为了把本章从概念推进到可运行的工程，可以完整走读一个小程序。假设内存中从 `BASE` 开始有若干 32-bit 数，`rdi` 保存当前地址，`rcx` 保存剩余元素个数，`rsi` 保存累加和。程序每轮加载一个元素，加入累加和，地址加 4，计数减 1，若计数不为 0 就回到循环入口。

这个案例改用真实 x86-64 写法（Intel 语法，目的操作数在前）：32-bit 数据经 `eax` 中转，地址、计数与累加和使用 64-bit 寄存器，比前面的 16-bit 极小 ISA 宽一档，数值关系更醒目；两者的 trace 方法完全相同——读旧状态、经组合路径、在时钟沿提交。

```

loop:
  mov eax, [rdi]
  add rsi, rax
  add rdi, 4
  sub rcx, 1
  jnz loop
done:
  mov [RESULT], esi

```

这段程序覆盖了本章大部分通路：取指、译码、地址形成、数据加载、ALU 加法、立即数扩展、条件码或比较、条件分支、存储。若它能在单步下跑通，最小 CPU 的核心路径基本都被验证过。

指令	主要输入	组合路径	提交对象
MOV（加载）	rdi、偏移 0	ALU 地址加法，数据存储器读	eax, PC
ADD	rsi、rax	ALU 加法，写回 mux	rsi, Flags, PC
ADD	rdi、立即数 4	立即数扩展，ALU 加法	rdi, Flags, PC
SUB	rcx、立即数 1	B 取反加一或减法路径	rcx, Zero, PC
JNZ	Zero 标志	条件判断，PC mux	PC
MOV（存储）	rsi、RESULT 地址	地址形成，数据存储器写	内存或 MMIO, PC

若采用五阶段流水线，前两条指令立即暴露 load-use 数据冒险。`mov eax, [rd`

i] 读回的数据通常到 MEM 阶段末尾才可用，而下一条 `add rsi, rax` 在 EX 阶段就需要 `rax`。没有转发或停顿时，加法指令会读到 `rax` 的旧值。

周期	MOV	ADD	ADD	SUB
1	IF			
2	ID	IF		
3	EX	ID	IF	
4	MEM	stall 或 EX 等待	ID	IF
5	WB	EX 使用转发值	ID/EX	ID

这个表不是为了背流水线图，而是训练读者问三个问题：数据什么时候产生，下一条什么时候需要，是否有合法路径把新值送过去。若没有旁路网络，就必须插入停顿；若有从 MEM/WB 到 EX 的转发，控制器必须检测依赖并选择转发输入。否则程序在低速单周期模型中正确，在流水线模型中错误。

案例：条件分支和预测错误怎样恢复 同一个循环的 `JNZ` 还能展示控制冒险。取指单元通常希望每周周期都取下一条指令，但分支是否成立要等比较结果出来。若等到结果再取指，流水线会空等；若提前猜测，就可能取错路径。教学 CPU 可以先选择“遇到分支就停顿”，再讨论预测。

策略	硬件动作	代价
停顿直到分支结果	分支进入 EX 前不继续提交后续路径	控制简单，循环每次损失周期
总是假设不跳	继续取顺序 PC，若实际跳转则冲刷	适合少跳分支，循环回边常错
总是假设跳转	提前取目标 PC，若实际不跳则冲刷	循环体内常较好，退出时错一次
动态预测	根据历史选择方向和目标	需要预测表、目标缓存和恢复机制

预测错误恢复必须同时处理两类状态。第一，已经取到但不该执行的指令要被冲刷，不能写寄存器、内存或设备。第二，PC 要恢复到正确目标，取指从那里重新开始。若错误路径上有存储或 MMIO 写已经对外可见，就无法简单恢复；因此流水线通常要把真正对外可见的提交推迟到确认顺序正确之后。

```
branch predicted not taken:
  fetch sequential instruction
  branch resolves taken
  squash wrong-path instruction
  PC = branch_target
  restart fetch
```

案例：函数调用的完整栈帧 trace 再用一个更接近系统程序的例子。函数 `sum2(a,b)` 返回两个整数之和。调用者把参数放进 `edi/esi`，`call` 把返回地址压栈并跳转；被调用者用 `push` 保存需要保留的寄存器，计算结果放进 `eax`，`ret` 恢复 PC。

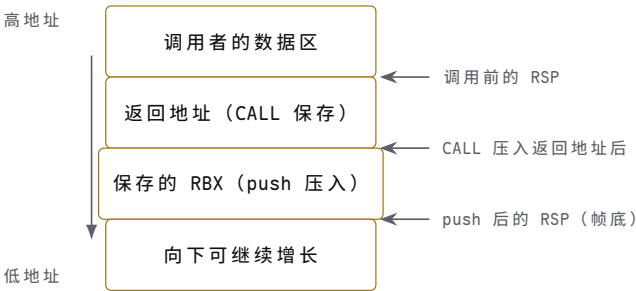
```
caller:
  mov edi, 7
  mov esi, 5
  call sum2
  mov [RESULT], eax

sum2:
```



```
push rbx
mov eax, edi
add eax, esi
pop rbx
ret
```

调用阶段	程序员可见状态变化	硬件边界
准备参数	edi=7, esi=5	MOV 写回寄存器，PC 顺序前进
CALL	返回地址压栈，PC=sum2 入口	栈指针减小，返回地址写入栈顶，再写 PC
建立栈帧	push 保存 RBX，RSP 减小	栈指针减小，写数据到栈地址
计算返回值	eax=edi+esi	ALU 加法，eax 写回
销毁栈帧	pop 恢复 RBX，RSP 加回	读栈、栈指针加回、寄存器写回
RET	PC= 返回地址	从栈顶读地址，写 PC



图示：函数调用案例对应的栈帧内存布局。CALL 先把返回地址压入，RSP 下移一格；sum2 的序言用 push 把 RBX 保存到新帧里；尾声 pop 按相反顺序恢复，RSP 加回。RET 硬件只认“返回地址”这一格：若某条越界写改写了它，RET 就把被污染的值装进 PC，程序表现为“返回后失控”。

这个例子把“调用约定”和“硬件状态”联系在一起。若 RSP 初始化错误，第一条 push 就可能写坏数据区；若 CALL 压入返回地址的位置错误，RET 会跳到不可预测地址；若被调用者忘记恢复 RBX，调用者看到的 RBX 就被破坏。所谓 ABI 兼容，最终表现为这些寄存器、栈位置和 PC 更新都按同一约定执行。

案例：异常插入到调用链中 若 sum2 内部执行加载时发生页错误或总线错误，异常入口必须知道故障发生在哪条指令、如何保存返回上下文、是否能恢复。简单裸机可能直接停机或点亮错误 LED；带操作系统的机器会把故障地址、错误码和当前 PC 保存到异常栈，再进入处理程序。

异常保存对象	为什么需要	恢复或诊断用途
故障 PC	知道哪条指令失败	修复后重试，或报告故障位置
状态/标志	保留中断允许、条件码、特权状态	返回时恢复执行环境
通用寄存器	处理程序可能使用寄存器	避免破坏被打断程序
错误原因	页不存在、权限错误、总线超时等	选择缺页、杀进程、复位设备或停机
栈指针/特权栈	异常处理本身需要可靠栈	防止在坏用户栈上继续保存状态

因此，异常处理不是在程序旁边加一条旁路，而是 CPU、栈、向量表、权限和内存映射共同定义的第二套控制流。第四章到这里可以停在一个结论：最小 CPU 只要

能顺序执行还不够，真正的机器还必须能在错误、等待、分支和外部事件中保持可恢复的状态边界。

案例：写回来源选错怎样定位到一根控制线 第一个调试案例发生在多周期整机第一次跑通 ROM 程序时。现象是：前两条指令完全正确——`0x22F0` 执行后 `R1=0x00F0`，`0x7260` 执行后 `R1=0xF000`；第三条 `0x3450=MOV R2, [R1+4]` 执行后，`R2` 应当是 `MEM[0xF004]` (GPIO_IN, 按键未按下时读 0)，实测却是 `0xF004`。随后 `JZ R2, +3` 因为 `R2` 非零总是顺序执行，表现为按键没有按下灯也亮。先把这个事实本身看清楚：`R2` 拿到的值恰好等于本条指令刚形成的访存地址。加载结果一般不会等于自己的地址，这个“巧合”就是第一根线索。

定位沿着写回数据通路反向走。写回 mux 只有两个候选：ALUOut 和 MDR。`R2` 拿到 ALUOut，说明 `wb_sel` 选错了来源；但在下结论之前，还要排除“访存本身就错了”的可能。单步重跑这条加载，逐拍观察：

检查步骤	观察结果	可以排除或锁定的范围
核对写回值与地址	<code>R2=0xF004</code> ，与 ALU 形成的地址相同	写回数据来自 ALU 通路，不是随机错误
单步查 MDR	<code>MDR=MEM[0xF004]=0</code> ，读使能、地址均正确	访存路径正常，故障在 MDR 之后
查 <code>wb_sel</code> (T4 访存拍)	期望已切到“存储器”，实测仍为 ALU	锁定到这一根选择线
回查控制字来源	T3 地址形成把 <code>wb_sel</code> 置为 ALU，T4 没有切换	控制字按拍切换缺失

根因是控制字在访存拍没有切换到存储器来源：`wb_sel` 在 T3 被“地址形成”这一拍置成 ALU (对地址加法本身无害，因为 T3 不写回)，但控制器没有在 T4 把它改成“存储器”，于是 T5 写回拍沿用旧值，写回 mux 输出 ALUOut 而非 MDR。修复办法是把 `wb_sel` 改为按 (状态, opcode) 译码：加载指令从 T4 起 `wb_sel=` 存储器，T5 提交时写回 mux 自然输出 MDR。这个案例的方法比结论更重要：症状 (寄存器值错) 先归到路径 (mux 输出错)，再归到控制线 (`wb_sel`)，最后归到控制字 (某一拍的某一格)——每一环都可观察、可单步复核。调试控制器时，永远先问“这个错误值是数据通路上哪一个 mux 能产生的”，再问“谁在这一拍驱动它的选择线”。

案例：PC 更新顺序错怎样让分支被撤销 第二个调试案例发生在同一台多周期整机上。正确设计约定 PC 只在一条指令的最后一个状态 (T5) 由 `pc_sel` 一次性提交；某次实现中，设计者让 `JZ` 在 T3 判零成立时就把目标写进 PC，理由是“早一拍跳转可以省时间”，同时又保留了所有指令在 T4 位置的“顺序 PC 提交”——`PC←PC_seq`，其中 `PC_seq` 在 T1 由 PC 加一锁存。两处改动各自看起来都合理，合在一起就出错了。

现象要对照正确行为看。按键未按下时 `R2=0`，地址 3 的 `JZ R2, +3` 成立，目标为 $3+1+3=7$ ，正确机器跳过地址 4、5、6，执行 7、8、9 的灭灯路径；实测地址 4 的 `MOV R3, 1` 仍然执行——连续两轮按键采样里它提交了两次，而正确行为下第一轮它根本不该出现，地址 7 的 `MOV R3, 0` 则一次也没有出现，灯不随按键熄灭。单步观察 PC，问题立刻显形：

时刻	PC	IR	说明
JZ 的 T1 末	3	0x5403	取指； <code>PC_seq</code> 锁存为 $3+1=4$
JZ 的 T3 末	7	0x5403	判零成立，PC 被提前改写为目标 $3+1+3=7$
JZ 的 T4 末	4	0x5403	顺序提交 <code>PC←PC_seq=4</code> ，目标被覆盖

下一拍 T1 4 0x2601 IR 装入地址 4 的 MOV R3, 1——本应被跳过

根因是 PC 写使能的时机与指令提交边界重叠：同一条指令的生存期内 PC 被写了两次，T3 的分支写先发生，T4 的顺序提交后发生，后者覆盖前者，分支于是被静默撤销。这类错误尤其危险，因为它不产生任何异常、不停机、不点错误灯，只是“跳转偶尔不生效”，很容易被当成程序逻辑错误。修复办法是取消 T3 的直接写：分支目标在 T3 只装入 `pc_next` 寄存器，PC 只在一条指令的最后一个状态写入一次，由 `pc_sel` 在提交拍统一选择顺序值或目标。PC 是全机共享的状态，它在每一拍至多有一个写入者——这条纪律一旦破坏，分支、调用返回和异常入口会一起失去保证。

本章到此只考察 CPU 内部 trace 最小 CPU 章节到这里应停在内部状态边界：PC、IR、寄存器堆、ALU、写回 mux、控制字、等待状态和流水线冒险。只要读者能把每条指令写成“读旧状态、经过组合路径、在时钟沿提交新状态”的 trace，本章目标就已经完成。

下一章再把这个 CPU 接到主存、总线、MMIO、DMA 和中断，讨论地址译码、等待、错误、cache 可见性和设备副作用。第六章再用 6502、Z80、8051、8086、80386、80486/Pentium 和现代 SoC 观察真实芯片如何把这些边界逐步扩展到整机平台。这样分章后，CPU 内部 trace、整机事务、经典芯片三类问题不会混在同一节里。

专题：译码器怎样从指令字段产生控制线 指令不是被 CPU “理解”后才执行，而是字段被译码成一组控制选择。操作码决定大类，寄存器字段连接寄存器堆读写地址，立即数字段进入符号扩展或零扩展路径，功能字段进一步选择 ALU 操作。最小 CPU 的译码器可以是组合逻辑，也可以是 ROM/PLA 或微码入口，但它的产物都应体现为可检查的控制线。

指令字段	进入的硬件路径	典型错误
opcode	主控制器，决定 mem_read/reg_write/pc_sel 等	未定义 opcode 被当成合法指令执行
rs/rt	寄存器堆读地址	字段位置接反导致读错源寄存器
rd	寄存器堆写地址或写回 mux 的目标选择	存储或分支指令错误写寄存器
imm	符号扩展、零扩展、左移、地址加法器	立即数扩展规则与 ISA 不一致
funct	ALU 控制的二级译码	ADD/SUB/SLT 等同 opcode 下混淆

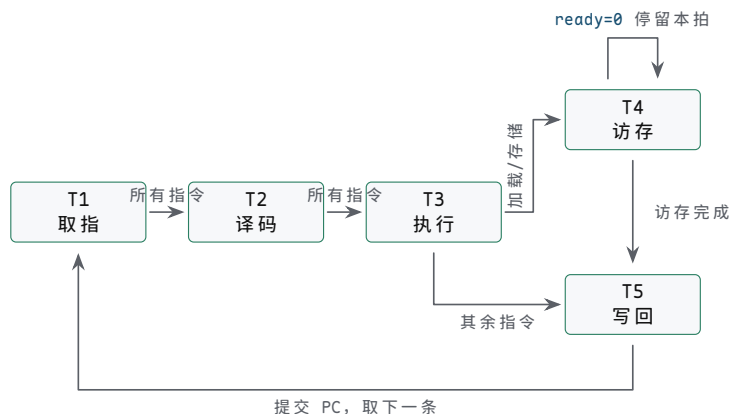
译码必须明确非法指令策略。教学机器可以停机并点亮错误码；带异常的机器可以保存 PC 并进入非法指令入口；极简硬件也可以规定未定义 opcode 为 NOP，但这种做法会掩盖程序损坏。无论选择哪种策略，都不能让非法组合悄悄打开 `mem_write` 或跳到不可预测 PC。

专题：多周期控制 FSM 的完整状态表 前面把一条指令拆成取指、译码、执行、访存、写回五拍，§ 一还用 T0-T4 的编号给出过加载指令的微操作序列。本专题把同一拆分改记为 T1-T5 (从 1 起编号)，并补齐所有指令的完整状态表。机制上只需记住三件事：控制器里有一个状态寄存器保存当前拍；每一拍输出的控制字由 (状态, opcode) 共同译出，而不是只随 opcode；每一拍末状态寄存器按“下一状态”规则前进，一条指令走完回到 T1。T4 只被加载、存储两条 MOV 使用，因为只有它们需要数据存储器事务；PC 只在 T5 提交一次，这正是前面 PC 更新顺序案例的修复原则写进状态表的样子。

状态	本拍动作	控制线	下一状态
T1 取指	$IR \leftarrow MEM[PC]$; PC 本拍不改变	$pc_out=1$ 、 $imem_read=1$ 、 $ir_write=1$	T2 (所有指令)
T2 译码	$A \leftarrow R[a]$ 、 $B \leftarrow R[b]$; imm8/off4/off8 按指令扩展; 按 opcode 预置后续控制字	读地址选择、 imm_kind ; 本拍无状态提交	T3 (所有指令)
T3 执行	ADD: $ALUOut \leftarrow A+B$; SHL: $ALUOut \leftarrow A \ll imm4$; MOV 立即数: $ALUOut$ 直送 imm8; 加载/存储: $ALUOut \leftarrow A+off4$; JZ: $Zero \leftarrow (R[r]==0)$; JMP: 备好目标	alu_op 、 a_sel 、 b_sel 按指令; JZ 时 $flags_write=1$	加载/存储 $\rightarrow T4$; 其余 $\rightarrow T5$
T4 访存	加载: $MDR \leftarrow MEM[ALUOut]$; 存储: $MEM[ALUOut] \leftarrow R[s]$; $ready=0$ 时停留本拍	加载: $mem_read=1$, wb_sel 预选存储器; 存储: $mem_write=1$ 、 $byte_en$	T5
T5 写回	寄存器提交 (仅 ADD、MOV 立即数、加载、SHL); $PC \leftarrow pc_sel$ 输出并提交	reg_write 按指令; $wb_sel=ALU$ 或存储器; $pc_write=1$	T1 (下一条)

从这张表可以直接读出哪些指令跳过 T4: ADD、MOV 立即数、SHL、JZ、JMP 五种非访存指令在 T3 之后由次态逻辑直接送往 T5, 每条只占 4 拍; 加载和存储占满 5 拍。同理, JZ、JMP 和存储在 T5 的 $reg_write=0$, 它们只提交 PC。次态逻辑因此有两个输入: 当前状态和 opcode—T3 之后去 T4 还是 T5, 正是靠 opcode 区分的唯一一处分叉。慢存储器也很好安放: $ready=0$ 时次态保持 T4 不变, 等待只拖住访存拍, 不影响其他指令的其他拍。

把这张表画成状态转移图, 结构就是一条主链加一处分叉:



图示: 多周期控制 FSM 的状态转移。T1→T2→T3 是所有指令的公共主链; T3 之后按 opcode 分叉是全机唯一一处分叉—加载、存储去 T4 访存, 其余五种指令直达 T5 写回, 各占 4 拍。T4 上方的自环是慢存储器的等待: $ready=0$ 时次态保持 T4 不变, 只拖住访存拍。T5 提交寄存器与 PC 后回到 T1, 开始下一条。

这张状态表同时是硬连线控制与微程序控制的交点。用硬连线实现, 它是“状态触发器加次态组合逻辑再加输出译码”; 用微程序实现, 每一行就是一条微指令, “本拍动作”是微指令的数据通路字段, “下一状态”就是下一微地址字段, T3 之后的分叉对应一条按 opcode 选择微地址的条件微指令。第三章讨论微程序控制器时说过, 微序列器每个周期只做一件事: 按微地址取微指令、送出控制线、再按条件选下一微地址—本表就是那台微序列器在这台教学 CPU 上的全部表格内容。两种实现的差别只在规则写在哪里, 每一拍该出现的控制线一根也不多、一根也不少。

专题：scoreboard 和冒险检测的最小逻辑（选读） 本专题与前面的流水线冒险同属“流水线、冒险和异常”一章的提前浓缩，选读。流水线把多条指令重叠执行后，控制器必须知道“某个结果现在是否已经可用”。最简单的办法不是先设计复杂乱序执行，而是在流水级寄存器里保留目标寄存器号、是否写回、结果来自 ALU 还是存储器等信息，再由冒险检测逻辑比较当前指令的源寄存器。

冒险	检测条件	处理策略
ALU-ALU RAW	前一条会写 <code>rd</code> ，后一条读取同一寄存器	转发 EX/MEM 或 MEM/WB 结果
load-use	加载的目标寄存器被下一条立即读取	停顿一拍，等待数据从存储阶段回来
存储数据	存储指令的写数据来自尚未写回的指令	转发到存储数据输入，或停顿
分支控制	分支结果尚未决定但后续指令已取入	冲刷流水线中错误路径指令
结构冲突	取指和访存争用同一存储端口	停顿，或分离指令/数据存储寄存器

最小冒险检测可写成硬件式条件，而不是口头描述。例如：

$$\text{stall} = \text{IDEX.memRead} \wedge ((\text{IDEX.rd} = \text{IFID.rs}) \vee (\text{IDEX.rd} = \text{IFID.rt}))$$

这表示当前处于执行级的加载指令还没有数据，而取指/译码级指令马上要读这个目标寄存器。停顿时 PC 和 IF/ID 不更新，ID/EX 注入 bubble，避免错误指令带着旧数据前进。

专题：目标寄存器 pending 表—scoreboard 检测冒险的最小逻辑（选读） 上一专题把冒险检测写成布尔式，工程上更常用的最小结构是一张“目标寄存器 pending 表”：每个可写寄存器配一个触发器，置 1 表示“已有一条在途指令以它为目标、结果尚未提交”。规则只有两条：译码拍末，凡 `reg_write=1` 的指令把自己的目标寄存器置位；写回拍末，对应位清除。译码拍内，把本指令的全部源寄存器号与 pending 表比对，任一为 1 就停顿—PC、IR 保持，指令滞留译码拍，且不做登记。比对时要按本章编码认清谁是源：`ADD`、`SHL` 是二操作数约定，`d` 字段既是目标也是源；存储 `MOV` 的 `s` 字段（写数据）是源；`JZ` 的 `r` 字段是源；`MOV` 立即数没有源寄存器。`R0` 固定为 0、永不作写目标，它的 pending 位恒为 0，比对可以直接省掉。

下面用四条指令把这张表走一遍。约定五级流水（IF/ID/EX/MEM/WB）逐拍推进、无转发、写回在 WB 拍末提交、译码用当拍沿前的 pending 值比对；序列为 `MOV R2, [R1+4]` (I1)、`ADD R3, R2` (I2)、`MOV R4, 1` (I3)、`SHL R4, 4` (I4)：

拍	译码拍内容	pending 表事件	判定与动作
2	I1 译码	源 R1 未置位	放行；拍末置 P2=1
3-5	I2 译码	源 R2 的 P2=1	停顿三拍，滞留译码拍，不登记
5 末	I1 写回 R2	清 P2=0	I2 本拍仍见旧值，下一拍再比对
6	I2 再次译码	P2=0	放行；拍末置 P3=1
7	I3 译码	无源寄存器	放行；拍末置 P4=1
8-10	I4 译码	源 R4 的 P4=1	停顿三拍，待 I3 写回
10 末	I3 写回 R4	清 P4=0	I4 本拍仍见旧值
11	I4 再次译码	P4=0	放行；拍末再置 P4=1 (SHL 的目标也是 R4)

这张表有两条边界约定必须写死。第一，置位与清除都发生在拍末（时钟沿），比对用拍内的当前值，所以写回当拍的目标寄存器仍算 pending—多停一拍是保守但

安全的约定。第二，pending 表只回答“能不能走”，不回答“值是多少”：它维护提交顺序，不维护数据本身；想减少停顿，需要上一专题的转发网络把未提交的值直接送往 ALU 输入。意义在于规模：8 个触发器加几组比较器，就把“某个结果现在是否可用”变成可查表的硬件事实；上一专题 `IDEX.memRead` 那条布尔式只是这张表“只看最近一条在途指令”的特例。理解了 pending 表，后面再看记分牌调度、寄存器重命名时，认出的都是同一思想的更细版本。

专题：同一条加载指令在三种 CPU 组织中的对照 为了避免“单周期、多周期、流水线”只停留在名词层面，可以用同一条 `MOV R1,[R2+8]` 对照三种组织。它们完成的语义相同：读 R2，加立即数形成地址，访问存储器，把数据写回 R1。区别在于资源是否复用、状态保存在哪里、关键路径由谁决定。

组织方式	加载指令的执行形态	主要代价
单周期	取指、译码、寄存器读、地址加法、时钟周期被最慢指令拉长 存储读、写回全部在一拍完成	
多周期	IF、ID、EX 地址形成、MEM、WB 分拍完成	控制状态机复杂，但硬件可复用
流水线	不同加载指令位于不同阶段，阶段间用寄存器隔开	需要处理冒险、冲刷和阶段平衡

单周期设计最适合教学入门，因为一条指令的 trace 很直观；多周期设计更像早期现实 CPU 的控制方式，因为它允许慢步骤独占多个周期；流水线设计提高吞吐量，但不会让单条加载的物理工作消失。读者应能把三种设计画成状态边界，而不是只背“流水线更快”。

专题：控制器检查要同时覆盖正常路径和错误路径 CPU 设计是否扎实，不能只跑一段加法程序来判断。控制器必须面对非法 opcode、未对齐访问、慢存储器、分支冲刷、中断屏蔽、异常返回和复位。否则机器在正常程序上看似正确，一旦接入真实总线或设备就会暴露状态边界不清的问题。

场景	必须观察的状态	合格表现
非法 opcode	PC、IR、错误码或 halt 状态	不写内存，不错误提交寄存器
慢速加载	地址、 <code>mem_read</code> 、 <code>ready</code> 、MDR	等待期间请求稳定，只提交一次
分支命中	IF/ID、ID/EX、PC 选择	错误路径指令被冲刷
异常入口	故障 PC、状态寄存器、异常向量	可诊断、可恢复或明确停机
复位	PC、寄存器堆、控制状态机	无旧控制线残留，第一条取指确定

这一类检查会迫使实现者回答：哪个周期算提交？哪些控制线在等待时保持？错误路径是否会产生副作用？这些答案比“程序最后算出 42”更接近 CPU 工程。

章末练习

本章练习不要求先追求完整 ISA，而要求把每条指令写成可检查的硬件 trace。答案必须同时说明控制线、数据路径和提交边界。

任务	要完成的产物	检查点
PC 取指	写出 PC、指令存储器、IR 和 <code>pc_write/ir_write</code> 的周期关系	能说明哪一拍保存指令
ADD trace	为 <code>ADD R1,R2</code> （即 <code>R1=R1+R2</code> ）列出读地址、ALU 操作、写回选择和写使能	写回只在目标时钟沿发生

加载 trace	把加载指令拆成地址形成、存储读、MDR 保存、寄存器写回	能处理存储器未 ready
存储 trace	说明地址、写数据、字节使能、写控制和无寄存器写回	能区分 RAM 写和 MMIO 副作用
JZ trace	说明 Zero 标志如何影响下一 PC	分支目标和顺序 PC 选择明确
条件码比较	分别解释有符号小于和无符号小于使用哪些 ALU 标志	不把 Carry、Sign 和 Overflow 混用
CALL/RET trace	把调用和返回拆成保存返回地址、更新栈指针、改写 PC 和恢复 PC	能指出栈访问本质上是加载/存储事务
控制字	设计一个包含 <code>alu_op/wb_sel/reg_write/mem_read/mem_write/pc_sel</code> 的控制字	每一位能追到实际电路
微程序	为加载指令写出 5 个微步骤和等待状态处理	慢存储器只拖住访存阶段
流水线冒险	说明结构、数据、控制三类冒险各破坏什么边界	能提出停顿、转发或冲刷策略
SHL 微操作	为 <code>0x7260=SHL R1, 8</code> 写出 T1-T5 各拍的微操作、控制线与提交对象	不经过 T4, 写回只在 T5 提交
三种组织拍数	统计 ROM 前三条 (<code>0x22F0</code> 、 <code>0x7260</code> 、 <code>0x3450</code>) 在单周期、五拍多周期、理想五级流水下各用多少拍	拍数与周期长度分开讨论
新增 SUB	为 <code>SUB Rd, Rs (R[d]=R[d]-R[b])</code> 写控制字行和逐拍流程, 说明数据通路是否要加新部件	复用既有 ALU 减法路径
cmp+jcc 配对	把 <code>JZ R2, +3</code> 改写成 x86-64 的 <code>test/jz</code> 两条指令, 说明标志位在两条指令之间的角色	标志是前写后读的约定状态
CPI 演算	某程序动态执行 100 条指令: 加载 20、存储 10、其余 70 条为 <code>ADD/MOV</code> 立即数/ <code>SHL/JZ/JMP</code> ; 按多周期拍数求总拍数、CPI 与 100 MHz 时钟下的执行时间	CPI 是动态混合的平均
pending 表读法	对 <code>MOV R2,[R1+4]</code> 、 <code>ADD R3,R2</code> 、 <code>MOV R4,1</code> 、 <code>SHL R4,4</code> 序列 (五级流水、无转发、写回在 WB 拍末提交) 写出 pending 表逐拍变化并标出停顿	置位在译码拍末、清除在写回拍末

参考解答与要点

任务	参考解答	必须保留的要点
PC 取指	PC 输出取指地址, 指令存储器返回编码, <code>ir_write</code> 在取指边界保存编码; 下一 PC 可由 <code>pc+size</code> 、分支目标或异常入口选择。	PC 和 IR 都是状态元件。

ADD trace	读端口选择 R1/R2, ALU 选择 ADD, 写回 mux 选择 ALUOut, 写地址为 R1, <code>reg_write=1</code> 只在写回边界有效。示例: <code>R1=5, R2=7</code> 时 <code>read_a=5</code> 、 <code>read_b=7</code> 、 <code>ALU_out=12</code> , 时钟沿 <code>R1<-12</code> 、 <code>PC=100->102</code> 。	算对不等于写对。
加载 trace	EX 阶段形成地址, MEM 阶段发起读, 目标未 ready 时保持地址和控制, 响应后保存 MDR, WB 阶段写回目标寄存器。	等待不能污染其他状态。
存储 trace	存储指令使用 ALU 形成地址, 源寄存器提供写数据, 打开字节使能和写控制, 不打开寄存器写回。若地址命中 MMIO, 副作用由设备规格决定。	存储是有副作用的事务。
JZ trace	比较或 ALU 结果产生 Zero, 控制器根据 Zero 选择顺序 PC 或分支目标; PC 只在提交边界更新。示例 (本章 ROM 程序): 地址 3 的 <code>JZ +3</code> 在 <code>R2=0</code> 时 <code>PC=3+1+3=7</code> ; 同一指令在 <code>R2=1</code> 时顺序取地址 4。	条件输入必须在采样前稳定。
条件码比较	无符号小于看减法后的借位/Carry 约定 (结果可能为正仍算小于); 有符号小于看 <code>Sign XOR Overflow</code> (如 <code>0x80-1</code> : $-128-1$ 的真值超出范围, 结果为正 <code>0x7F</code> 但溢出位置 1, 符号位经溢出修正后仍有符号小于成立)。相等判断只看 Zero。	标志必须绑定解释规则。
CALL/RET trace	CALL: 返回地址先压栈保存 (栈指针减小后写入栈顶), 再 <code>PC=目标</code> ; RET: 从栈读回返回地址, 恢复栈指针, <code>PC=返回地址</code> 。栈访问本质就是加载/存储事务, 栈指针是普通寄存器加约定。	调用链是事务序列, 不是语法。
控制字	控制字字段应覆盖 PC、IR、寄存器堆、ALU、存储器和下一状态逻辑; 任何字段没有连接对象都不是硬件控制。	控制字是电线集合, 不是注释。
微程序	加载指令可写为取指、读寄存器、地址形成、访存等待、写回; 访存未完成时微地址保持在等待状态, 错误时跳异常入口。	微步骤提供清晰的观察点。
流水线冒险	结构冒险来自资源争用, 数据冒险来自结果未写回, 控制冒险来自下一 PC 尚未确定。停顿保护边界, 转发减少等待, 冲刷丢弃错误路径。	提高吞吐不能破坏提交顺序。
SHL 微操作	T1: <code>IR<MEM[PC]</code> , 取回 <code>0x7260</code> , <code>imem_read/ir_write=1</code> ; T2: <code>A<R1=0x00F0</code> , <code>imm4</code> 读出 8; T3: <code>alu_op=左移</code> 、 <code>b_sel=imm4</code> , <code>ALUOut<0x00F0<<8=0xF000</code> ; 跳过 T4; T5: <code>wb_sel=ALU</code> 、 <code>reg_write=1</code> , 时钟沿 <code>R1<0xF000</code> 、 <code>PC<PC+1</code> 。	非访存指令不走 T4。

三种组织拍数	单周期 3 拍：每条一拍，但周期长度按最慢的加载路径定。五拍多周期：MOV 立即数 4 拍、SHL 4 拍、MOV 加载 5 拍，共 $4+4+5=13$ 拍。理想五级流水（无冒险、每拍流入一条）：第 5 拍完成第一条，共 $3+5-1=7$ 拍。	比较组织要同时看拍数和周期长度。
新增 SUB	控制字与 ADD 同行，仅 <code>alu_op=SUB</code> ： <code>a_sel/b_sel=</code> 寄存器、 <code>wb_sel=ALU</code> 、 <code>reg_w=1</code> 、 <code>mem_r/mem_w=0</code> 、 <code>pc_sel=</code> 顺序。流程同 ADD：T1 取指，T2 双读，T3 ALU 做减（B 取反加一），跳过 T4，T5 写回并 <code>PC+1</code> 。数据通路不加新部件，译码真值表加一行即可。	新指令优先复用既有路径。
cmp+jcc 配对	x86-64 写法： <code>test ecx, ecx</code> （或 <code>cmp ecx, 0</code> ）按结果置 ZF，再由 <code>jz target</code> 读 ZF 决定是否跳转。标志寄存器是两条指令之间“前一条写、后一条读”的约定状态；教学 JZ 把比较与跳转内部化为一条。两者的偏移都相对下一条指令计算。	标志位必须绑定解释规则。
CPI 演算	总拍数 $20 \times 5 + 10 \times 5 + 70 \times 4 = 100 + 50 + 280 = 430$ 拍； $CPI = 430/100 = 4.3$ ；100 MHz 时钟下执行时间为 $430/10^8$ 秒 = $4.3 \mu s$ 。	CPI 依赖指令混合，不是硬件常数。
pending 表读法	拍 2 I1 译码置 P2；拍 3-5 I2（读 R2）停顿三拍，拍 5 末 I1 写回清 P2，拍 6 I2 放行置 P3；拍 7 I3 译码置 P4；拍 8-10 I4（读 R4）停顿三拍，拍 10 末 I3 写回清 P4，拍 11 I4 放行并再置 P4。共停 6 拍，四条指令 14 拍完成；无冒险理想流水为 8 拍。	同拍比对用清除前的值。

最小 CPU 的经典读法是逐周期 trace。不要只画“PC 连到内存、ALU 连到寄存器”的框图；要写清楚每条指令在哪个周期读旧状态、在哪条组合路径上传播、在哪个时钟沿提交新状态。后续主存、总线、I/O 和经典芯片只是在这个 trace 外面继续增加事务边界。

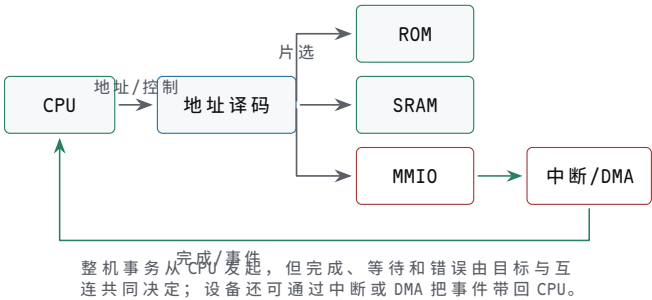
主存、总线、I/O 与 DMA

前面已经解释了最小 CPU 如何在内部取指、译码、执行和提交状态。若停在这里，读者只能得到一台“会在寄存器之间搬数”的小机器。真正的整机还需要大容量主存、可寻址设备、外部事件、等待、错误和批量数据搬运。本章专门补上这条桥：CPU 怎样把一个地址变成一次存储器或设备事务，设备怎样把完成事件通知 CPU，DMA 又怎样在不让 CPU 逐字搬运的情况下读写主存。

本章的目标不是背总线协议名字。无论是 8086 板级总线、微控制器片内总线、PCIe 设备，还是现代 SoC 互连，工程问题都相同：谁发起请求，谁响应地址，数据由谁驱动，什么时候采样，目标慢了怎样等待，目标不存在怎样失败，设备完成后怎样通知。把这些问题写清楚，才能从最小 CPU 走到可启动、可调试、可扩展的整机。

核心理念

整机硬件不是“CPU 加一块内存”的粗线图，而是由一组定义明确的事务组成。每次访问都必须说明地址、方向、字节选择、目标片选、握手、完成、错误和副作用。只有这些边界明确，CPU 才能可靠地取指、访存、访问设备和处理中断。



图示：从最小 CPU 到整机事务。地址译码决定目标，握手和事件决定本次访问何时真正完成。

一、主存不是抽象数组

软件把内存看成按地址编号的字节序列，硬件却必须用地线、译码器、存储阵列、数据线和控制信号实现这件事。一次读访问至少包含四个事实：发起者给出地址，译码逻辑选中目标，目标经过内部访问后驱动数据，发起者在规定边界采样。一次写访问则要求地址、写数据、字节使能和写控制在目标采样窗口内稳定。

对象	硬件含义	关键问题
地址	指出本次事务目标位置	高位译码是否只选中一个目标
数据	读时由目标返回，写时由发起者提供	同一时刻是否只有一个驱动器
读写控制	区分本次事务方向和类型	输出使能和写使能是否互斥
字节使能	指明哪些 8-bit 车道有效	字节写不会破坏相邻数据
ready/valid	指明请求或响应是否真正完成	慢目标能否插入等待而不丢地址和数据

ROM、SRAM 和 DRAM 的读写不是同一种动作 ROM 或 Flash 适合保存启动代码，掉电后内容仍在；SRAM 接口直观，地址稳定后在访问时间内返回数据，适合教学板、cache 和小容量片上 RAM；DRAM 密度高，但单元需要刷新，访问还要经过 bank、行、列、预充电和控制器调度。把这些对象都叫“内存”没有错，但在硬件设计中必须区分它们的时序和控制边界。

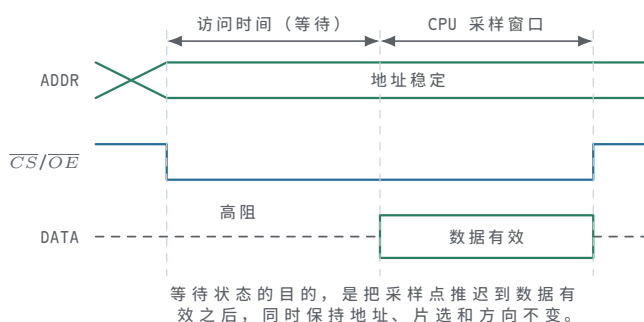
存储类型	典型用途	关键边界
ROM/Flash	复位入口、固件、常量表	复位地址必须映射到有效内容，读时序满足取指
SRAM	cache、实验板 RAM、小容量片上存储	CS/OE/WE 互斥，地址和数据满足建立时间
DRAM	大容量主存	控制器负责 activate、read/write、precharge、refresh
寄存器文件	CPU 内部近距离状态	多读端口、写端口、旁路和同周期读写规则明确

异步 SRAM 是最好的第一块主存 教学整机可以先使用低速异步 SRAM。读周期中，地址和片选稳定后，打开输出使能，SRAM 在访问时间后驱动数据线；写周期中，输出使能关闭，发起者驱动数据线，在写使能有效窗口内把数据写入目标字节。这个模型足够简单，也足以暴露硬件最常见错误：片选重叠、输出争用、写使能过早、字节使能错误。

```

SRAM read:
  address stable
  CS active, OE active, WE inactive
  wait access time
  sample data

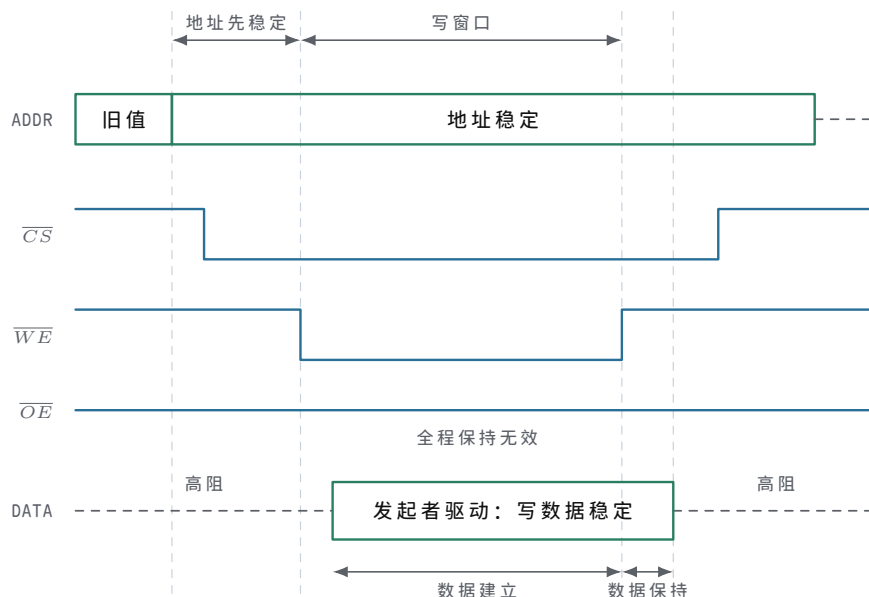
SRAM write:
  address and write_data stable
  CS active, OE inactive, WE active for write window
  selected byte lanes store data
  
```



图示：异步 SRAM 读周期：地址先稳定， $\overline{CS}/\overline{OE}$ 拉低后存储器需要一段访问时间才把数据驱动到总线上；CPU 只能在数据有效后采样。三条线各有明确的翻转时刻——读时序图读的就是这些边沿，而不是三条平直线。

SRAM 写周期的驱动权与写窗口 写周期与读周期使用同一组地址线、数据线和片选，差别在数据线的驱动方向和写使能窗口。读周期中 $\overline{OE}$ 有效，由 SRAM 驱动数据线，发起者只负责在数据有效后采样；写周期中 $\overline{OE}$ 必须保持无效，SRAM 的输出驱动器关闭，改由发起者把写数据驱动到数据线上——同一组数据线在任何时刻只能有一个驱动者。写使能 $\overline{WE}$ 拉低形成写窗口，规格书围绕它规定三条边界：地址必须在窗口打开前已经稳定，写数据必须在窗口关闭前满足建立时间，窗口关闭后还要继续保持一段保持时间，窗口本身也有最小宽度。这些要求来自同一个事实：

存储阵列在 $\overline{WE}$ 回升、窗口关闭的边沿附近把数据线上的值写入选中单元，窗口内任何一次地址或数据翻动，都可能把错误的值写进错误的单元。因此写周期的固定顺序是：先给地址和数据，再打开写窗口，先关窗口，最后才撤数据。

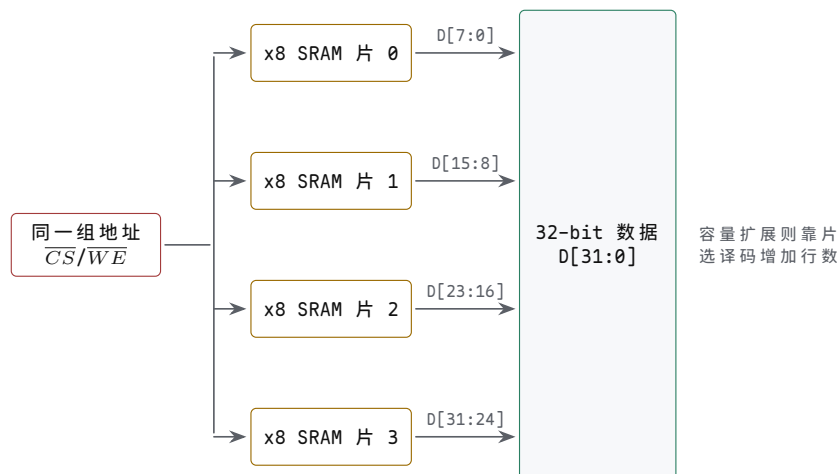


图示：异步 SRAM 写周期：地址与片选先稳定， $\overline{OE}$ 全程无效，发起者在写窗口内驱动数据线。存储阵列在 $\overline{WE}$ 回升、窗口关闭的边沿把数据线上的值写入选中单元，因此数据必须在窗口关闭前满足建立时间、关闭后满足保持时间，地址在整个窗口内不能翻动——与读周期对照，差别就在数据线由谁驱动、哪个边沿是真正的写入点。

DRAM 需要控制器而不是直接接地址线 DRAM 的容量来自更密集的存储单元，代价是接口复杂。控制器要先打开某个 bank 的某一行，再读写列；换行前需要预充电；即使没有普通访问，也要周期性刷新；高速 DRAM 还需要训练采样窗口。CPU 看到的是“主存读写”，但实际路径经过控制器、队列、bank 调度、行命中或行冲突。这也是为什么现代 CPU 必须靠 cache 把常用数据放近。

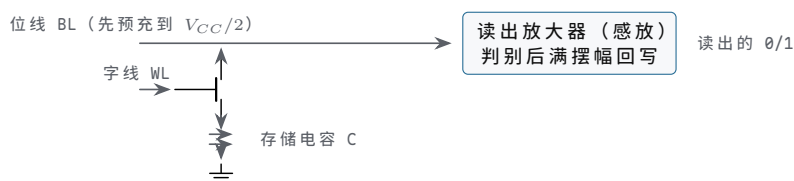
DRAM 动作	硬件含义	时延来源
Activate Read/Write	打开某个 bank 中的一行 在已打开行内选择列传输数据	行译码和感放建立 列访问、突发传输和总线排队
Precharge	关闭当前行，准备访问另一行	位线恢复到可再次访问状态
Refresh	周期性恢复存储单元电荷	控制器必须暂停或调度普通访问

存储芯片的位宽扩展与容量扩展 单片存储芯片的位宽和容量都有限，整机用两组方向相反的手段补齐。位宽扩展解决“字不够宽”：把若干片芯片并接同一组地址线、片选和读写控制，每片只接数据线的的一个字节车道——四片 x8 SRAM 并接后，一次 32-bit 访问四片同时动作，各自收发 D[7:0]、D[15:8]、D[23:16]、D[31:24]，对外表现为一片 32-bit 宽的 SRAM。容量扩展解决“行不够多”：数据线和低位地址全部并接，新增的高位地址经译码器产生多路互斥片选，任一地址最多选中一片，行数随片数倍增。两种扩展经常同时使用，但边界必须分清：位宽扩展中各片同时被选中、各管一段数据线；容量扩展中各片互斥选中、数据线全部共用。混淆二者的直接后果，是两片同时驱动同一个字节车道的总线争用。



图示：位宽扩展：四片 x8 SRAM 并接同一组地址、 $\overline{CS}$ 和 $\overline{WE}$ ，同时动作、各管一个字节车道，对外是一片 32-bit SRAM。容量扩展方向相反：数据线全部并接，高位地址译码出互斥片选来增加行数，任一地址最多一片被选中。

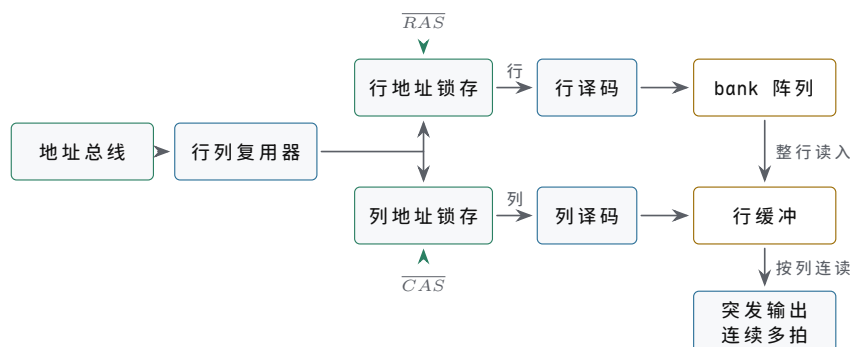
DRAM 单元用电容存电荷，读出后必须重写 DRAM 的一个 bit 由一个访问晶体管和一个存储电容构成 (1T1C)。位线先预充到 $V_{CC}/2$ ；字线打开晶体管后，电容与位线共享电荷，位线电压略升或略降，读出放大器（感放）判别方向并输出 0/1。两个后果决定了 DRAM 的全部特殊性：其一，读出是破坏性的，感放必须把满摆幅电平经位线写回单元，否则原值丢失；其二，电容持续漏电，电荷在几十毫秒内就衰减到无法判别，控制器必须周期性刷新——逐行打开并重写。规格书里的 Activate、Precharge、Refresh 和刷新周期（典型要求 64 ms 内把全部行刷新一遍），都直接来自这两件事。



打开字线后，电容与位线共享电荷，位线电压轻微上升或下降；感放判别方向输出 0/1，并立刻把满摆幅电平写回单元——读出即重写。

图示：1T1C 存储单元与读出放大器。刷新并不是额外功能，而是控制器替所有行周期性重复这个“读出-回写”过程；这也是 DRAM 不能像 SRAM 那样直接把地址线接上去用的原因。

DRAM 的行列地址复用与突发传输 DRAM 的地址引脚不随容量等比增长，靠的是行列地址复用：同一组地址引脚先送行地址、再送列地址。一个 2^{20} 字的阵列直接接地址需要 20 根地址线，复用后 10 根就够—— $\overline{RAS}$ 有效时把引脚上的值锁存为行地址， $\overline{CAS}$ 有效时把同一组引脚上的值锁存为列地址。省下的引脚直接降低封装成本，代价是接口从“给地址等数据”变成一串带时序要求的命令。行地址译码后打开整行，整行数据被感放读出并留在行缓冲中；此后的列访问只是从行缓冲里选一段送到数据总线。突发传输利用的正是这一点：给一个起始列地址，DRAM 按内部列计数器连续多拍输出相邻数据，后续各拍不再付出行打开的代价。行缓冲命中是突发便宜的根本原因，也是内存控制器尽量把相邻访问调度到同一行的理由。



同一组地址引脚在 $\overline{RAS}$ 、 $\overline{CAS}$ 两拍分别送出行、列地址；整行先读入行缓冲，列访问只是在缓冲里选段，行命中后的连续突发因此不必重开行。

图示：行列地址复用与突发传输。 $\overline{RAS}$ 拍锁存行地址并打开整行， $\overline{CAS}$ 拍锁存列地址；行缓冲命中后，连续列地址直接从行缓冲多拍输出，比重开一行便宜得多。

行打开、列传输和预充电之间的最小间隔由一组时序参数规定，规格书把它们写成以时钟拍为单位的数字。DDR3-1600 的内部时钟为 800 MHz，一拍 1.25 ns；1600 MT/s 的数据速率来自同一时钟的双沿传输，时序参数仍按时钟拍计。常见标注 11-11-11-28 就是下表四个数字。

参数	含义	DDR3-1600	纳秒换算
t_{RCD}	Activate 打开行之后，到第一个列读/写命令的最小间隔	11 拍	13.75 ns
t_{CL}	列读命令发出后，到第一个数据出现在总线上（CAS 延迟）	11 拍	13.75 ns
t_{RP}	Precharge 关闭当前行之后，到允许再次 Activate 的最小间隔	11 拍	13.75 ns
t_{RAS}	一次 Activate 到允许 Precharge 的最小行打开时间	28 拍	35 ns

换算很直接：拍数乘 1.25 ns。同一行内连续读，列延迟 13.75 ns 后就能紧跟突发数据；访问另一行则要先等 t_{RP} 再重新 Activate，一次行冲突至少多花 $t_{RP} + t_{RCD} = 22$ 拍，约 27.5 ns。这个数字就是内存调度器在乎行命中、软件在乎访问局部性的原因。

SDRAM 的突发长度与多 bank 交错 Activate 打开一行之后，控制器可以对同一行连发多个列读写命令；每个列命令传的也不是一个数据，而是按约定的突发长度 (burst length) 连续传一串——DDR3 固定为 8 (BL8)，也允许切成 4。DDR 的“双倍数据率”指时钟的上升沿和下降沿各传一次数据，而不是把时钟翻倍：DDR3-1600 的内部时钟是 800 MHz，数据速率 1600 MT/s，所以一次 BL8 突发占 4 个时钟拍、交 8 个数据，这期间数据总线被这一串独占。

bank 是 DRAM 内部可独立 Activate、独立 Precharge 的子阵列，各有自己的行缓冲。一个 bank 在等 t_{RCD} 、等数据的时候，命令总线可以对另一个 bank 发命令，把后者的行打开时间藏进前者的传输里，这就是多 bank 交错。沿用上一张表的 DDR3-1600 参数（1 拍 1.25 ns， $t_{RCD} = t_{CL} = t_{RP} = 11$ 拍， $t_{RAS} = 28$ 拍），比较“读两个不同行”在单 bank 与双 bank 下的时间轴：

单 bank，两次访问落在同一 bank 的不同行：

- 拍 0 ACT bank0 row0
- 拍 11 RD bank0 col0 (t_{RCD} 到期)
- 拍 22 数据出现在总线，BL8 占用拍 22-25
- 拍 28 PRE bank0 (t_{RAS} 到期才允许关行)
- 拍 39 ACT bank0 row1 (t_{RP} 到期)
- 拍 50 RD bank0 col1 (t_{RCD} 到期)
- 拍 61 第二组数据出现，共等 61 拍

双 bank 交错，两行分别在 bank0、bank1:

```

拍 0  ACT bank0
拍 1  ACT bank1      (不同 bank, 行打开并行进行)
拍 11 RD bank0       (数据占拍 22-25)
拍 15 RD bank1       (数据占拍 26-29, 恰好接上)
拍 26 第二组数据出现, 共等 26 拍

```

第二组数据从拍 61 提前到拍 26, 省 35 拍、约 43.75 ns。省下的不是任何一条时序参数—单 bank 必须串行等完 $t_{RP} + t_{RCD}$ 才能再读, 双 bank 让 bank1 的 Activate 与 bank0 的全过程并行, 这段等待被完全覆盖; 拍 15 才发第二条读命令, 是因为数据总线仍由两个 bank 共享, bank0 的突发占着拍 22-25, bank1 的数据最早只能接在拍 26。这就是内存条标注 bank 数、控制器把相邻地址散到不同 bank、调度器重排请求的原因: 行命中决定一次访问多快, bank 并行决定等待能不能被藏起来。这个例子是教学模型, 为突出 bank 并行的收益, 忽略了 t_{RRD} 、 t_{FAW} 等对连续 Activate 的命令间约束。

对照点	单 bank 顺序访问	双 bank 交错
第二组数据出现	拍 61	拍 26 (省 35 拍, 约 43.75 ns)
数据总线占用	拍 22-25、61-64, 中间大量气泡	拍 22-29 连续无气泡
关键等待	换行时 $t_{RP} + t_{RCD}$ 串行暴露	被另一 bank 的并行 Activate 覆盖
检查点	PRE 与 ACT 都卡在同一个 bank 上	命令错开, 数据在共享总线上排队衔接

组相联 cache 的替换策略是精度与代价的取舍 前面说过, 主存的时延是结构性的, 整机的应对是把热数据留在近处的 cache 里。本章后半部分会拆开 cache 的地址字段、写策略和一致性, 这里先回答放进 cache 之后必然面对的问题: 新行取回时组内已无空路, 牺牲哪一路? 直接映射没有这个问题, 每个地址只有唯一槽位; 组相联把它交给了替换策略。

精确 LRU (最久未使用) 记录组内各路的年龄顺序, 牺牲最老的一路, 命中率最好, 代价随路数急剧膨胀。2 路组相联时每组只需 1 位: 这一位指出哪一路更久未用, 每次命中取反即可。4 路要区分完整的年龄顺序, 共 $4! = 24$ 种排列, 每组需 $\lceil \log_2 24 \rceil = 5$ 位, 且每次命中都要重排年龄; 8 路则需 $\lceil \log_2 40320 \rceil = 16$ 位。伪 LRU 把 4 路组织成两层二叉树, 3 个内部节点各 1 位, 每组 3 位, 命中时沿路径翻转节点位; 它选出的不保证是最老路, 但命中率损失通常很小。FIFO 只记进入顺序, 命中时不更新任何状态; 随机替换连状态位都不要。

高组相联之所以普遍放弃精确 LRU, 原因有两头: 状态位按 $n!$ 增长还是小事, 更难办的是每次命中都要更新年龄, 路数越多更新逻辑的组合路径越长, 直接拖慢命中时间; 而 8 路、16 路的组里, “精确选最老”换来的命中率收益早已递减, 随机替换与 LRU 的差距常常小到测量噪声量级。工程上的结论因此很清楚: 2 路用 1 位精确 LRU, 4 路用 3 位伪 LRU, 更高组相联用伪 LRU 或随机。

策略	每组状态位 (4 路)	命中时是否更新	关键问题
精确 LRU	5 位 ($4! = 24$ 种年龄序)	每次命中重排年龄	路数增多后状态位和更新路径急剧膨胀
伪 LRU	3 位 (两层树节点)	沿树路径翻转节点位	选出的不保证是最老路, 命中率略降
FIFO	2 位 (轮流指针)	不更新	不看访问热度, 热行可能被轮到

随机

0 位

不更新

实现最简，大组相联下
命中率损失很小

硬件预取与软件预取都是拿带宽换命中 替换策略决定牺牲谁，预取则试图让牺牲更少发生：顺序扫描数组时，用完一行 cache 行，下一行几乎马上要用，空间局部性让“将来要访问什么”变得可预测。硬件预取器监视 miss 流的地址规律：next-line 预取在用到第 i 行时把第 $i+1$ 行取回；步长预取识别出固定步长后沿步长提前取回；预取流通常在 4 KiB 页边界停下——跨页地址可能未映射，为一次猜测触发缺页代价太大。

软件预取由指令显式给出提示。x86-64 提供 `prefetcht0`、`prefetchnta` 等预取指令：它们不改变任何架构可见状态，地址无效时也不触发缺页异常，只是提示硬件“这个地址很快会被读”。编译器或手写汇编常在循环里提前若干次迭代发出提示：

```
; rdi 指向当前元素，rsi 是数组结尾，每元素 8 字节
scan_loop:
    prefetcht0 [rdi + 512]    ; 提前 8 行(64 B 行)提示
    add     rax, [rdi]
    add     rdi, 8
    cmp     rdi, rsi
    jb      scan_loop
```

预取不是免费的：取错行会占住 cache 槽位、挤掉有用数据，每次预取本身也消耗内存带宽，过激的预取反而把真正需要的访问排在后面。可以粗算一笔账：顺序扫描大数组，行 64 B、元素 8 B，一行装 8 个元素；miss 时延按 200 拍、每个元素处理 4 拍计。无预取时每行一次 miss，每行花 $200 + 32 = 232$ 拍；预取把“取下一行”与“处理当前行”重叠，若提前量达到 $200 \div 32 \approx 7$ 行，处理完当前行时下一行已经在 cache 里，稳态每行只剩 32 拍，miss 被完全隐藏，吞吐提高约 $232/32 \approx 7.3$ 倍。若预取只能提前一行，32 拍的处理盖不住 200 拍的取回，每行仍暴露约 168 拍。提前量必须匹配“处理一行的耗时”与“取回一行的耗时”之比，这正是预取器设计的核心数字。

预取方式	触发时机	关键问题
next-line 硬件预取	用到第 i 行时取第 $i+1$ 行	顺序流效果好，随机访问纯属浪费
步长硬件预取	识别出固定地址步长后沿步长取	步长误判会持续污染 cache
软件预取指令	程序在用到数据之前显式提示	提示距离要折算成拍数，地址须确实会用
跨页停止	预取流在 4 KiB 页边界停下	猜测性访问不得触发缺页副作用

上电自检的走步与棋盘格各查一类故障 主存焊上板不等于可信：某位固定读成 0 或 1 (stuck-at)、相邻数据线或地址线短路、写入后状态维持不住（数据保持故障），都可能让“写一个值再读回来”这种最简单的测试侥幸通过——它只验证了大部分位在此刻、对这组值是可写的。上电自检用图案化的写入-读回序列把故障类别分开：走步 (walking 1/0) 每次只让一位与众不同，棋盘格 (checkerboard) 让相邻位恒取反值。

走步 1 先写全 0 背景，再对同一单元依次写 0x01、0x02、……、0x80，每步读回比对，并复查其余单元仍是背景值。某位固定 0 会在轮到它走 1 时露馅，固定 1 会在背景检查中露馅；两位短路时走步值会变形——写 0x01 可能读回 0x03，直接指认相邻两位被短接。走步 0 把背景换成全 1、逐位走 0，对称覆盖另一半故障方向。棋盘格把 0x55 (0101 0101) 写满读回，再换成 0xAA (1010 1010)：任何相邻两位恒取反，相邻短路或位间耦合会让图案当场变形；保持故障则靠“写入后停一段

再读”暴露。这些图案便宜、确定、能在无操作系统的裸机上运行，查不出所有故障，但恰好覆盖焊接与走线最常出的几类，因此是板级 bring-up 的固定节目。

以 4 字节小内存 (addr0-addr3, 8-bit 字节) 为例，一次走步 1 加棋盘格的完整步骤如下。

步骤	操作	预期结果	检查点
1	addr0-addr3 全写 0x00，逐字节读回	全部 0x00	无固定 1，背景可写可读
2	addr0 依次写 0x01、0x02、.....、0x80，每步读回	读回恒等于写入	每个数据位都能独立置 1
3	步骤 2 每步之后读 addr1-addr3	仍为 0x00	无地址串扰、无相邻线短路
4	addr1-addr3 重复步骤 2、3	同上	逐字节覆盖全部单元
5	全写 0x55 读回，再全写 0xAA 读回	图案不变形	相邻位恒反，查相邻短路
6	背景改 0xFF，逐位走 0 (0xFE、0xFD、.....)	读回恒等于写入	对称覆盖固定 0 故障

校验位与 ECC 把位错从灾难降级为事件 主存的位会错：宇宙射线、电压毛刺、保持时间失守都可能翻转一个 bit。最便宜的对策是奇偶校验—每字节加 1 个校验位，使连同校验位在内的 1 的个数恒为偶数（偶校验）。数据 1011 0001 已有 4 个 1，校验位记 0；读回时若某位翻转，1 的个数变奇，硬件立刻知道出错了。奇偶校验的边界同样明确：只能发现奇数个位错，两位同时翻转就漏检；即使发现，也不知道错在哪一位，只能报错，不能纠正。

汉明码用多个校验位换取定位能力。(7,4) 汉明码把 4 个数据位和 3 个校验位排成 7 位，位置从 1 编号；校验位放在编号为 2 的幂的位置 (1、2、4)，数据位占据 3、5、6、7。把位置编号写成 3 位二进制，第 i 个校验位负责所有编号第 i 位为 1 的位置。每个校验组各做一次偶校验，哪几组失败，其编号拼起来恰好就是出错位置的编号—这个把“哪些组错”翻译成“哪里错”的数称为校正子 (syndrome)。

位置编号	存放内容	参与校验的校验位	说明
1	p_1	p_1 自身	编号 001，最低位为 1
2	p_2	p_2 自身	编号 010
3	d_1	p_1 、 p_2	编号 011
4	p_4	p_4 自身	编号 100
5	d_2	p_1 、 p_4	编号 101
6	d_3	p_2 、 p_4	编号 110
7	d_4	p_1 、 p_2 、 p_4	编号 111

算一遍：数据位 (位 3、5、6、7) 依次为 1、0、1、1。 p_1 覆盖位 1、3、5、7，其中数据 1、0、1 共两个 1， $p_1 = 0$ ； p_2 覆盖位 2、3、6、7，数据 1、1、1 共三个 1， $p_2 = 1$ ； p_4 覆盖位 4、5、6、7，数据 0、1、1 共两个 1， $p_4 = 0$ 。发出码字 (位 1 至 7) 为 0 1 1 0 0 1 1。若位 6 在途中翻成 0： p_1 组 (0,1,0,1) 共两个 1，通过； p_2 组 (1,1,0,1) 共三个 1，失败； p_4 组 (0,0,0,1) 共一个 1，失败。校正子按 $p_4p_2p_1$ 读出 (1,1,0) = 6，直接指向位 6，翻转回来即完成纠正。

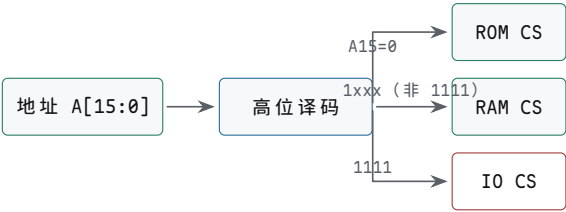
实用的 ECC 内存把同一思想做到 64 位数据加 8 位校验、72 位宽，实现单错纠正、双错检测 (SEC-DED)。贵在三处：阵列宽了 1/8，每条内存要多挂芯片；控制器对每次读做译码纠正，对部分写先读-改-写再重算校验，多一层逻辑和时延；平台还要为纠错路径做完整验证。对长时间运行、数据不能错的机器这是便宜的保险，桌面机则普遍省掉—这也是普通内存条与 ECC 内存条不能随意混插的原因。

二、地址译码定义整机地图

CPU 输出的是地址数值，整机必须规定这个数值由哪个目标响应。这个规定叫地址映射。一个简单教学板可以把低地址映射到 ROM，中间映射到 RAM，高地址小窗口映射到 GPIO、定时器和串口等设备寄存器。地址译码器把高位地址和访问类型变成片选信号。

example 16-bit address map:

0x0000-0x7FFF boot ROM
0x8000-0xEFFF SRAM
0xF000 GPIO_OUT
0xF004 GPIO_IN
0xF010 TIMER_COUNT
0xF014 TIMER_CTRL



设计目标：任意一个地址最多选中一个目标；
未映射区域也要有错误、固定值或超时规则。

图示：地址译码不是软件表格，而是片选电路。片选重叠会造成总线争用或误写设备。

映射对象	响应规则	典型错误
ROM	复位入口和只读代码地址命中	复位后 CPU 取不到第一条指令
RAM	普通数据读写地址命中	写使能误打到 ROM 或设备窗口
GPIO	设备寄存器地址命中并产生副作用	地址译码过宽，普通内存写误触发外设
未映射区	返回错误、超时或固定空值	访问悬空，CPU 永久等待或读到随机总线值

片选表达式就是硬件约定 若地址空间为 64 KiB，可以用高位地址产生粗粒度片选。例如 $A_{15}=0$ 选 ROM， $A_{15}=1$ 且 A_{14}, A_{13}, A_{12} 不全为 1 选 RAM (0x8000-0xEFFF，28 KiB)， $A_{15}=1, A_{14}=1, A_{13}=1, A_{12}=1$ 选 I/O 窗口。无论用门电路、译码器、CPLD 还是 FPGA 实现，都必须证明任意合法周期最多只有一个目标片选有效。

```
rom_cs = !A15  
ram_cs = A15 && !(A14 && A13 && A12)  
io_cs = A15 && A14 && A13 && A12
```

这三个表达式看起来像逻辑练习，实际上就是整机安全边界。若 ROM 和 RAM 可同时片选，读周期可能出现两个芯片同时驱动数据线；若 RAM 和 GPIO 片选重叠，普通 STORE 可能既改内存又改外设输出；若未映射地址没有超时或错误返回，CPU 可能等待一个永远不会响应的目标。

时序预算决定是否需要等待状态 地址译码需要时间，存储器访问需要时间，数据到达 CPU 输入端后还要满足建立时间。若这些时间总和超过时钟周期，就必须降低频率、换更快器件，或插入等待状态。等待状态不是软件延时，而是硬件事务延长：地址、片选、读写方向和数据驱动关系必须在等待期间保持可解释。

```
read_budget =
```

```

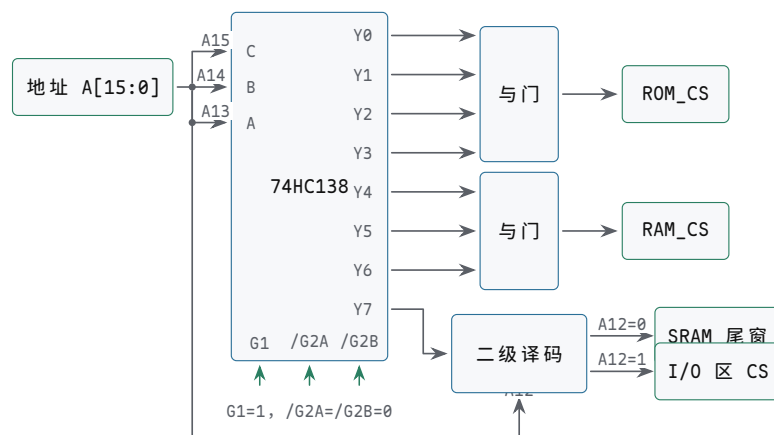
cpu_address_output_delay
+ address_decode_delay
+ memory_access_time
+ input_setup_time
+ board_margin

```

若 `read_budget` 大于一个周期，控制器就不能在下一拍采样数据。慢 ROM、慢外设和长板级走线都可能需要 READY/WAIT 机制。正式规格还应说明最大等待长度和超时行为，避免坏设备把整机永久挂住。

处理方式	适合场景	代价
降低时钟	全系统都慢，教学板容易观察	所有操作都变慢
更快器件	SRAM 或译码器成为瓶颈	成本、功耗或电平兼容性变化
插入等待	只有部分目标较慢	控制器必须保持事务状态
超时返回错误	目标可能不存在或失效	需要错误状态和恢复入口

案例：用 74HC138 译出整机片选 本节开头给出了 16-bit 地图：`0x0000-0x7FFF` 是 boot ROM (32 KiB, $A_{15}=0$)，`0x8000-0xEFFF` 是 SRAM (28 KiB)，`0xF000` 起是 I/O 区 (4 KiB)。现在给出这张地图的引脚级实现。最经典的器件是 3-8 译码器 74HC138：它有三个地址输入 C、B、A 和八个低有效输出 Y_0 – Y_7 ，任一时刻只有一个输出为低。把 $A[15:13]$ 依次接上 C、B、A，每个输出就恰好管辖一个 8 KiB 窗口，窗内偏移由 $A[12:0]$ 决定；使能端 G_1 接高电平、 $/G_2A$ 与 $/G_2B$ 接低电平，译码器才工作——这两个使能端也常改接总线读写选通，让片选只在存储周期内出现。输出低有效带来一个方便的性质：把若干相邻窗口合并成一个片选只需一个与门，任何一个被选中的输出变低，与门输出即为低。于是 Y_0 – Y_3 经四输入与门合成 ROM_CS ，正好覆盖 $A_{15}=0$ 的整个低半空间； Y_4 – Y_6 经三输入与门合成 RAM_CS ； Y_7 则送入二级译码，由 A_{12} 把 `0xE000-0xFFFF` 再分成 SRAM 尾窗和 I/O 区，合起来正好复现上面这张地图。



图示：引脚级地址译码。 $A[15:13]$ 经 74HC138 译成 8 个 8 KiB 窗口（输出低有效），与门把连续窗口合成 ROM_CS 与 RAM_CS ， Y_7 窗口由 A_{12} 二级译码再分；使能端决定译码器何时工作。

输出	C B A ($A[15:13]$)	地址区间	去向
Y_0	000	<code>0x0000-0x1FFF</code>	boot ROM 第 0 页
Y_1	001	<code>0x2000-0x3FFF</code>	boot ROM 第 1 页
Y_2	010	<code>0x4000-0x5FFF</code>	boot ROM 第 2 页
Y_3	011	<code>0x6000-0x7FFF</code>	boot ROM 第 3 页
Y_4	100	<code>0x8000-0x9FFF</code>	SRAM 第 0 页

Y5	101	0xA000-0xBFFF	SRAM 第 1 页
Y6	110	0xC000-0xDFFF	SRAM 第 2 页
Y7	111	0xE000-0xFFFF	二级译码：A12=0 归 SRAM 尾窗 (0xE000-0xEFFF)，A12=1 为 I/O 区 (0xF000-0xFFFF)

这套译码仍然是部分译码：I/O 区里只有 A12 和少数低位参与选择，A[11:5] 等地址位没有进译码器。结果是同一个设备寄存器会在整个 4 KiB 窗口里反复出现——0xF010 的定时器寄存器，在 0xF110、0xF210 等只改变未译码位的地址上同样可见，这些重复地址称为别名。SRAM 一侧同理：若实际芯片容量小于所属窗口，未译码的高位也会让同一个存储单元出现在多个地址上。部分译码用更少的门和更短的延迟换来足够大的窗口，在小系统里是合理的工程选择；但它把一道边界推给了规格：别名是否允许必须写明。允许别名，软件就不得用未译码位区分设备，调试时也不能从“访问了哪个地址”反推“选中了哪个设备”；不允许别名，就必须补足低位译码，让未映射组合返回错误或固定值，而不是把一次写错的访问悄悄变成对某个真实设备的读写。

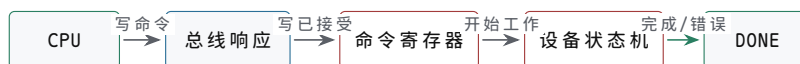
三、MMIO 把设备寄存器放进地址空间

设备寄存器可以像内存地址一样被 CPU 读写，但它们不是普通 RAM。读状态寄存器可能清除某些位，写命令寄存器可能启动外设动作，写数据寄存器可能进入 FIFO，读数据寄存器可能弹出一个元素。这类地址称为内存映射 I/O，简称 MMIO。MMIO 的关键不是地址长得像内存，而是每个寄存器都有副作用和顺序规则。

设备寄存器	读写语义	关键问题
DIR	设置 GPIO 引脚方向	输出引脚和输入引脚不会混用
OUT	写输出锁存器	写入只改变目标输出位
IN	读取同步后的外部输入	按键抖动和亚稳态不能直接进入 CPU
STAT	返回 ready、busy、error、irq 等状态	清除规则不会丢掉同时到来的新事件
DATA	数据收发寄存器或 FIFO 端口	空读、满写和溢出有明确定义

MMIO 写响应不等于设备已经完成 CPU 写一个命令寄存器，通常只表示设备已经接收命令。设备真正完成可能要等若干周期、若干微秒，甚至更久。完成可以通过状态位、轮询、DMA completion 或中断通知。若把 MMIO 写返回当作“设备动作完成”，软件会在设备尚未写完缓冲区、尚未刷新状态或尚未清除错误时继续执行，形成难复现的硬件错误。

```
device command sequence:
CPU writes COMMAND register
bus write response returns
device starts internal work
device sets DONE or ERROR
interrupt or polling tells CPU to inspect result
```



总线写响应只证明命令到达设备接口；真正完成必须看状态位、中断或 DMA completion。

图示：MMIO 写命令的两层完成边界。总线完成不等于设备动作完成。

端口 I/O 与 MMIO 是两种地址约定 8086 家族保留了独立 I/O 地址空间，IN 和 OUT 指令访问端口；许多 RISC 和单片机系统则把设备寄存器映射进普通地址空

间，用 LOAD/STORE 访问。两者都能控制设备，但硬件约定不同。端口 I/O 需要 I/O 周期和端口地址译码；MMIO 需要内存地址译码和内存属性规则，例如不可缓存、不可合并或必须保持顺序。

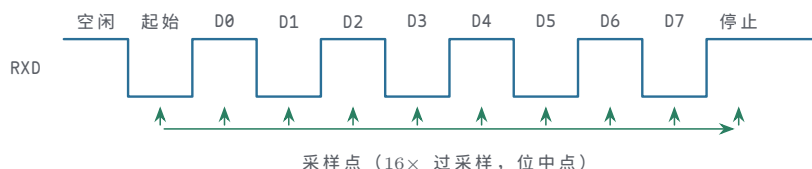
外部输入必须先调理和同步 按键、传感器、串口线和外部中断引脚都不按 CPU 时钟整齐变化。按键还会抖动，异步输入还可能让触发器进入亚稳态。教学板上的 GPIO 输入至少应经过电平保护、上拉/下拉、去抖策略和同步触发器，再进入设备寄存器。否则 CPU 读到的不是稳定硬件事实，而是未定义的边界行为。

不要把设备寄存器当作普通变量。普通 RAM 的读写目标是保存数据；MMIO 的读写目标是驱动硬件状态机。编译器、cache、写缓冲和乱序执行都可能改变普通内存访问的时机，但设备访问通常需要不可缓存属性、顺序约束和明确的完成检查。

从 GPIO、定时器和 UART 看设备寄存器 设备寄存器最好用具体布局理解。一个 GPIO 控制器可以有输出寄存器、输入寄存器、方向寄存器和中断状态寄存器；定时器可以有计数值、比较值、控制位和中断清除位；UART 可以有发送数据、接收数据、状态和波特率配置寄存器。CPU 访问这些寄存器时走的是地址事务，但设备内部看到的是控制状态机输入。

设备	寄存器边界	常见问题
GPIO	DIR/OUT/IN/IRQ_STAT	输入是否同步，输出是否只改目标位
定时器	COUNT/CMP/CTRL/IRQ_CLEAR	到期事件是否锁存，清除是否丢新事件
UART	TX/RX/STAT/BAUD	空读、满写、错误位和中断顺序是否清楚

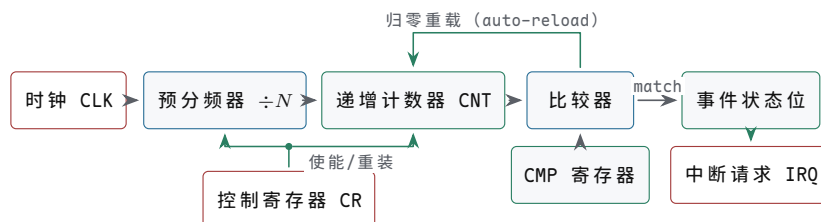
UART 帧格式与波特率约定 UART 没有时钟线，收发双方只靠“每一位持续同样的时间”这条约定保持对齐。线路空闲时停在高水平；发送方先用一个位时间的低电平打出起始位，再把 8 个数据位按 LSB 在前依次驱动上线，最后用至少一个位时间的高电平作为停止位。接收端用 16 倍于波特率的本地时钟对线路过采样：检测到下降沿后数 8 个采样周期，在起始位中点确认它仍是低电平，随后每隔 16 个采样周期在各位中点读取一次。中点采样让判决位置离前后边沿各半位，这正是异步串行能够容忍时钟误差的来源。



图示：UART 一帧（数据 0x55，LSB 在前）。空闲为高电平；起始位的下降沿给出帧起点，数据位在中点被逐位采样，停止位把线路带回高电平。

以 115200 baud 为例，每位宽度 $1/115200 \approx 8.68 \mu\text{s}$ 。接收端在起始位边沿对齐之后，最后一位（停止位）的采样点距该边沿 9.5 个位时间；若收发时钟的相对误差为 ϵ ，这个采样点会漂移 9.5ϵ 位，漂移超过半位即采到相邻位，所以理论上限约为 $0.5/9.5 \approx 5\%$ 。工程上还要从中扣除边沿检测的量化误差（16 倍过采样下最多 1/16 位）与电平翻转时间，留下的常用要求是每一侧波特率误差不超过约 2%，或者收发双方干脆从同一时钟源分频。误差超限时，出错的往往不是帧首而是帧尾——越早的位漂移越小，最后的停止位最先越界，表现为间歇性的帧错误或数据错位。

定时器的预分频、计数与比较匹配 定时器做的事只有一件：把固定频率的时钟变成软件可编程的周期间隔。它的数据路径很简单——预分频器先把输入时钟除以 N ，递增计数器 CNT 在每个分频后时钟上加一，比较器持续比较 CNT 与比较寄存器 CMP；当 CNT 数到 CMP，比较器输出 match。match 同时驱动两件事：置起事件状态位（若控制寄存器 CR 中的中断使能有效，同时拉起 IRQ），以及在 auto-reload 模式下把 CNT 清零，让计数立刻开始下一轮。于是中断周期严格等于（预分频 $\times$ (CMP+1)）个输入时钟——CMP 要加一，因为计数从 0 开始，数到 CMP 共经过 CMP+1 拍。以 1 MHz 输入、预分频 8 为例：分频后的计数时钟是 125 kHz，若 CMP=999，周期为 $8 \times (999 + 1) = 8000$ 个输入时钟，即 8 ms (125 Hz)；要得到 1 kHz 中断，应令 CMP=124，此时 $8 \times (124 + 1) = 1000$ 个时钟，正好 1 ms。事件状态位由软件按设备规则清除，清除顺序属于 § 四要讨论的中断约定。



图示：定时器数据路径：时钟经预分频驱动递增计数器，比较器在 CNT 等于 CMP 时产生 match，一路置事件状态位并按 CR 的中断使能拉起 IRQ，一路按 auto-reload 把 CNT 归零，开始下一周期。中断周期 = 预分频 $\times$ (CMP+1) 个输入时钟。

I2C 用两根开漏线支撑一条多设备总线 I2C 只有两根信号线：串行数据线 SDA 和串行时钟线 SCL。两根线都是开漏输出加上拉电阻——任何设备都只能把线拉低，不能主动驱动为高，线电平相当于所有设备输出的“线与”。空闲时两线靠上拉电阻停在高电平；主设备产生全部时钟，数据位只允许在 SCL 为低时改变，在 SCL 为高时被采样。事务以起始条件开头——SCL 保持高电平时 SDA 从高跳低；以停止条件结尾——SCL 保持高电平时 SDA 从低跳高。每个字节由 8 个数据位加 1 个应答位组成：第 9 个时钟里发送方释放 SDA，接收方把 SDA 拉低表示 ACK，保持高表示 NACK。寻址用 7 位设备地址，与 1 位读写方向合成地址阶段的一个字节。开漏结构正是这条总线能够共享的原因：多主同时发送时，发 0 的一方自然赢过发 1 的一方，竞争不需要额外仲裁线；从设备还可以在字节之间把 SCL 按住不放，强制主设备等待 (clock stretching)，慢设备因此不会丢数据。

UART 是点对点链路，没有地址概念；I2C 把多个设备挂在同一对线上，用起始条件和地址字节决定本次事务跟谁说话。下表对照二者的边界。

方面	I2C	UART
拓扑	一条共享总线挂多个主从设备	点对点，一发一收
时钟	主设备产生 SCL，收发同步于时钟沿	无时钟线，靠双方约定的波特率
寻址	起始条件后发送 7 位地址加读写位	无地址，线路上只有一个对端
确认	每字节后跟 ACK/NACK，主设备能发现无应答	无应答机制，只能靠上层协议
线数与速度	2 线；标准 100 kHz、快速 400 kHz	2 线收发交叉；常见 115200 baud

一次典型的“主发地址、寄存器号、再读回数据”事务如下，传感器和 EEPROM 的随机读几乎都是这个形状：

```

I2C register read transaction:
START
master sends [7-bit address + W] -> slave ACK
  
```

```

master sends [register number]    -> slave ACK
repeated START
master sends [7-bit address + R] -> slave ACK
slave sends [data byte]          -> master NACK
STOP

```

中间的重复起始 (repeated START) 把“先告诉对方读哪个寄存器”和“再开始读”合并成一次不间断事务，避免其他主设备在两段之间插入；读方向下最后一个字节由主设备回 NACK 再发停止条件，通知从设备释放 SDA、结束发送。这次事务的开销可以直接算：一帧 9 位，共 4 帧 36 位，未计起始和停止条件的少量时间；标准模式 100 kHz 下每位 $10\mu\text{s}$ ，合计约 $36 \times 10 = 360\mu\text{s}$ ，快速模式 400 kHz 下约 $90\mu\text{s}$ 。也就是说，主设备每 0.36 ms 才能完成一次这样的寄存器读取，这正是 I2C 传感器数据更新率的量级来源；I2C 的意义不在于快，而在于只用两根线就把一整板低速设备纳入同一个地址空间。

SPI 用片选换速度，用移位环换全双工 SPI 是四线同步串行接口：SCLK 时钟、MOSI 主出从入、MISO 主入从出、CS 片选（低有效）。总线上没有地址阶段——主设备想跟谁通信就把谁的片选拉低，同一时刻只允许被选中的从设备驱动 MISO。数据通路是一对首尾相接的移位寄存器：每来一个时钟沿，主设备从 MOSI 移出一位，同时从 MISO 移入一位，收发在物理上永远同时进行，全双工是这个结构的自然结果而不是协议选项。时钟行为由两个参数规定：CPOL 决定空闲时 SCLK 停在高还是低，CPHA 决定在第几个时钟沿采样数据，组合出四种模式。通信双方必须事先约定同一种模式——这条协商写在芯片手册里，不在总线上进行。

模式	CPOL	CPHA	空闲电平	数据采样沿
0	0	0	低	第一个沿（上升沿）
1	0	1	低	第二个沿（下降沿）
2	1	0	高	第一个沿（下降沿）
3	1	1	高	第二个沿（上升沿）

以最常用的模式 0 读一个字节为例：主设备先拉低 CS；从设备把数据的最高位驱动到 MISO（发送方在时钟下降沿换位）；主设备发出 8 个时钟，在每个上升沿采样 MISO，依次得到 D7 到 D0；与此同时主设备把 MOSI 上的 8 位移给从设备，纯读时通常发全 0 占位字节；8 拍结束后主设备拉高 CS，事务结束。

```

SPI mode-0 byte read (MSB first):
CS low
for bit = D7 downto D0:
    slave changes MISO on falling edge of SCLK
    master samples MISO on rising edge of SCLK
    master shifts one dummy bit out on MOSI
CS high

```

SPI 没有 ACK，也没有起始停止开销，8 个时钟就是 8 位；代价写在引脚数量上，每多一个从设备多一根片选。与 I2C 对照，两者在速度、引脚和协议复杂度上各站一端：

方面	SPI	I2C
速度	常见数 MHz 到数十 MHz	标准 100 kHz，快速 400 kHz
引脚	4 线，且每个从设备一根片选	2 线，设备数量不占引脚
寻址	无地址，靠片选	7 位地址加读写位
确认与等待	无 ACK，无从设备等待机制	每字节 ACK/NACK，从设备可拉住 SCL 等待

协议复杂度	几乎只有移位，模式须事先约定	起始、停止、重复起始、应答状态机
-------	----------------	------------------

速度差距可以直接算：SCLK 取 10 MHz 时位时间 100 ns，读一个字节恰需 8 拍 = 0.8 μs；I2C 快速模式 400 kHz 下位时间 2.5 μs，一个字节加应答共 9 位 = 22.5 μs，相差约 28 倍。Flash、显示屏、ADC 这类数据量较大的从设备多用 SPI，原因就在这里；而传感器这类低速、多节点、布线路径长的场合，I2C 的两线共享更省板级资源。可见两条总线没有优劣之分，只有“用引脚和约定换什么”的取舍不同。

USB 用枚举和描述符换来“通用”二字 UART、SPI、I2C 的共同前提是主设备事先知道对端是什么；USB 把这个前提取消了。总线严格以主机为中心，设备不能主动发起任何事务。插入检测靠电气约定：设备端在 D+ 或 D- 上接上拉电阻，全速设备拉 D+，低速设备拉 D-，集线器看到哪条线被拉高就知道来了什么速度的设备（高速设备先以全速现身，在复位期间再协商提速），并把端口状态变化上报主机。随后主机执行枚举：先复位该端口，设备以默认地址 0 应答；主机通过默认控制端点 0 读取设备描述符，再发 Set Address 给设备分配 1 到 127 之间的新地址；之后按新地址继续读取配置描述符、接口描述符、端点描述符和字符串描述符。描述符是一套标准格式的自描述数据：厂商 ID、产品 ID、设备类、需要多少端点、每个端点的类型和最大包长，操作系统据此匹配驱动。插拔能自动识别，不是因为总线聪明，而是因为每台设备都能用同一种格式回答“我是谁、我要什么”；“通用”二字，正来自枚举和描述符这套约定。

枚举完成后，数据按端点组织。端点是设备内部的单向通道，端点 0 固定留给控制传输，其余端点在描述符中声明自己的传输类型：

传输类型	语义	典型设备
控制	有应答、用于配置和命令，端点 0 专用	所有设备的枚举阶段
中断	主机按声明周期轮询，小数据量，出错重试	键盘、鼠标
批量	利用剩余带宽，无周期保证，出错重传	U 盘、打印机
等时	每帧预留带宽，不重传，容忍少量错误	音频设备、摄像头

把四种总线放在一起，复杂度阶梯很清楚：

总线	线数	寻址方式	设备自描述	典型定位
UART	2	无，点对点	无	调试口、模块间低速链路
SPI	4，每从设备加 1 片选	片选	无	板内高速从设备
I2C	2	7 位地址	无	板内多节点低速设备
USB	4（含电源和地）	枚举分配 7 位地址	标准格式描述符	机箱外可热插拔外设

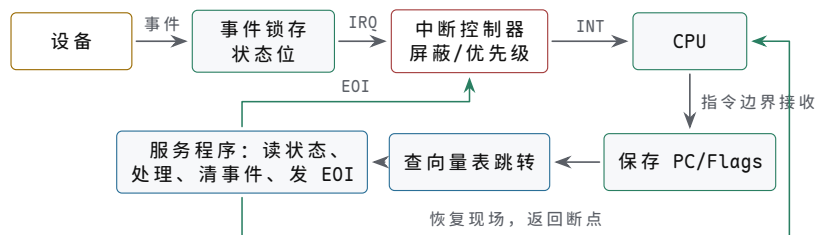
带宽数字能说明“通用”的另一面：高速模式 480 Mb/s 折合每微帧 125 μs 约 7500 B，批量端点最大包 512 B，一个微帧最多容纳 13 个整包，理论上限约 $13 \times 512 \text{ B} \div 125 \mu\text{s} \approx 53 \text{ MB/s}$ 。同一条总线既要服务每 10 ms 被轮询一次、一次只发几个字节的键盘，又要服务成批搬数据的 U 盘，靠的正是把调度规则写进四种传输类型，而不是为每类设备各做一种接口。

四、中断和 DMA 把外部事件接回 CPU

轮询可以让 CPU 一直读状态寄存器，直到设备 ready。但若设备很慢，CPU 会浪费大量周期。中断提供另一种路径：设备在状态变化后请求中断，中断控制器汇总、屏蔽和排序请求，CPU 在可接受边界保存当前执行上下文，跳到中断入口，服务程序读取设备状态并清除事件。

对象	硬件职责	关键问题
设备	产生完成、错误或数据到达事件	状态位和中断请求是否同步一致
中断控制器	汇总、屏蔽、优先级和投递	屏蔽位、优先级、EOI 顺序是否正确
CPU 入口	保存边界状态并跳到入口	PC、标志和特权边界是否可恢复
服务程序	读状态、搬数据、清除事件	不丢新事件，不重复进入同一旧事件

中断链路从设备事件一直到服务程序返回 中断把“CPU 主动轮询”反转为“设备主动请求”。完整链路是：事件先在设备内锁存成状态位并拉起 IRQ；中断控制器按屏蔽位和优先级决定是否投递；CPU 只在指令边界采样 INT，接收后保存 PC/Flags，查向量表跳到服务程序；服务程序读状态、处理数据、按设备规则清事件，最后发 EOI 并恢复现场返回。链路上任何一环顺序错误，都会表现为丢中断或同一事件重复进入。



图示：中断的完整链路。实线是请求与服务路径，虚线是返回路径：EOI 让中断控制器恢复投递，恢复现场让 CPU 回到被打断的位置。服务程序必须按设备规定的顺序清事件——先清设备还是先 EOI，是每种芯片数据手册都会写明的细节。

中断清除顺序是硬件约定 若服务程序先清设备状态，再清中断控制器，或先通知中断控制器再清设备，效果可能不同。某些设备要求先读状态后读数据，某些要求写 1 清位，某些要求读数据寄存器自动清除 ready。教材级规格必须写清楚事件锁存、状态读取、清除、EOI 和重新开放中断的顺序。否则系统在低速测试中正常，在高频事件下丢中断或重复中断。

中断控制器用三张寄存器决定谁先被服务 设备多起来之后，“谁先被服务”不能靠先到先得。以 8259 可编程中断控制器为例，它用三张 8 位寄存器管理 8 条中断线：IRR（中断请求寄存器）锁存已到但尚未投递的请求；IMR（中断屏蔽寄存器）按位禁止对应线路参与竞争；ISR（在服务寄存器）记录当前正在被服务的请求。优先级裁决器在未屏蔽的 IRR 位中选出最高优先级的一位，与 ISR 中已在服务的最高位比较，只有新请求优先级更高时才向 CPU 拉起 INT。默认是固定优先级—IR0 最高、IR7 最低；控制器也可配置成轮转优先级，刚被服务的线路自动排到队尾，避免某条热线路长期独占。

寄存器	记录内容	关键问题
IRR	已锁存、待投递的请求位	请求是否真正被锁存，边沿触发与电平触发是否分清

IMR	每条线路的屏蔽位	屏蔽期间到来的请求是挂起还是丢失
ISR	正在服务的请求位	EOI 之前同级和低级请求一律等待

EOI 是这套机制里最要紧的动作。服务程序发出 EOI，控制器才清除对应的 ISR 位；ISR 位不清，同级和更低优先级的请求即使进了 IRR 也只能等待——优先级秩序正是靠 ISR 位维持的。若希望高级中断能打断正在运行的低级服务程序，即允许嵌套，低级服务程序必须在保存好现场之后重新打开 CPU 的中断允许位。嵌套时保存的内容必须完整，因为返回时要逐层恢复：CPU 接收中断时已把返回地址和标志寄存器压栈，服务程序还要保存自己用到的全部通用寄存器。每一层嵌套占用一帧栈，层数越深栈消耗越大，这是嵌套中断容易忽略的隐性成本。

```
isr_low:
    push rax                ; save registers this handler uses
    push rbx
    sti                    ; allow higher-priority nesting only now
    ; ... read device, handle data, clear device event ...
    cli                    ; block nesting on the return path
    mov al, 0x20            ; non-specific EOI command
    out 0x20, al           ; write 8259 command port, clears top ISR bit
    pop rbx
    pop rax
    iretq                  ; restore flags and return address
```

优先级机制带来的风险是反转与饿死。反转的典型情形：低级服务程序长时间关中断执行，高级请求虽已进 IRR 却无法投递，实际等待时间反而更长。可以量化：115200 baud 下串口一个字节连起始、停止位共 10 位，到达间隔约 $86.8\mu\text{s}$ ；接收缓冲只有一字节深时，若某个低级服务程序关中断超过这个时间，串口数据就会被下一字节覆盖，表现为溢出错误——优先级更高的串口实际上输给了优先级更低的服务程序。饿死的典型情形则相反：固定优先级下高级中断密集到来时，低级请求在 IRR 里长期得不到服务，轮转优先级正是为这种情况准备的。工程上的通用对策有两条：服务程序尽量短，把重活推迟到中断返回之后；必须关中断的窗口要给出时间上界，而不是随手一关了事。

DMA 解决的是另一类浪费。若磁盘、网卡、显示控制器或音频设备需要搬运大量数据，CPU 不应逐字从设备读出再逐字写入内存。DMA 让设备或 DMA 控制器成为总线发起者，按描述符或寄存器配置直接读写主存。CPU 负责准备缓冲区和描述符，设备负责搬运，完成后用状态位或中断通知 CPU。

```
DMA receive sketch:
CPU allocates buffer
CPU writes descriptor address and length
CPU ensures descriptor is visible to device
device reads descriptor and writes buffer
device posts completion
CPU handles interrupt and inspects buffer
```

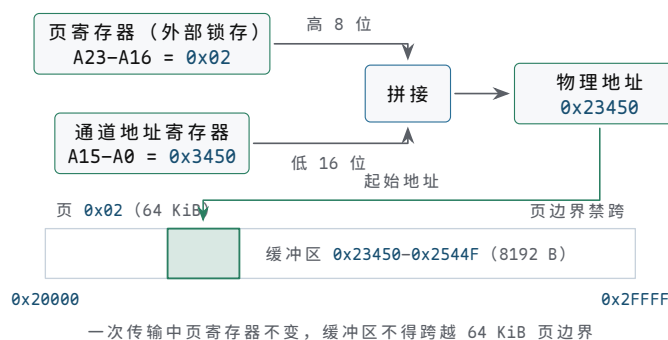
DMA 控制器按通道组织，每个通道就是地址加计数 8237 是理解 DMA 控制器的经典样本。它提供 4 个独立通道，每个通道对应一条设备请求线 DREQ；通道的核心状态只有两类 16 位寄存器——当前地址寄存器指出下一个要访问的内存地址，每传一字节自动加一，当前计数寄存器记录还剩多少字节，每传一字节自动减一。计数写入值比真实字节数少一：写 0 表示传 1 字节，写 `0xFFFF` 表示传 65536 字节，计数从 0 再减一回绕时产生终止计数 TC。模式寄存器决定传输方向（设备到内存或内存到设备）和传输方式。由于 16 位地址最多覆盖 64 KiB，PC 还给每个通道配了一个外部页寄存器，锁存物理地址的高 8 位，合成 24 位地址；由此带来一条

经典限制：一次传输中低 16 位地址在页内推进，页寄存器保持不变，缓冲区不得跨越 64 KiB 页边界。

寄存器组	记录内容	关键问题
地址寄存器	下一个内存单元的低 16 位地址	初值是否指向缓冲区起点，方向是否为递增
计数寄存器	剩余字节数，写入值等于实际字节数减 1	忘记减 1 会多传一个字节，覆盖缓冲区之后的内容
模式/命令	传输方向、单字/块/请求方式、通道号	方向写反会把设备数据当内存数据搬
页寄存器	物理地址高 8 位（外部锁存）	缓冲区是否跨 64 KiB 页边界
屏蔽/请求	通道使能与软件请求	配置期间通道必须屏蔽，配完再开放

传输方式回答“拿到总线后干多久”。单字 (single) 模式每传一个字节就把总线还给 CPU，等设备下一次 DREQ 再申请；块 (block) 模式一旦拿到总线就连传到计数结束，其间 CPU 无法使用总线；请求 (demand) 模式随 DREQ 电平走走停停。慢设备配单字模式时，DMA 只是零星借用本属于 CPU 的总线周期，这称为周期窃取 (cycle steal)：CPU 不被告知、不切换状态，只在恰好要用总线的那一拍多等一拍。块模式是另一个极端——它把 CPU 完全关在总线之外，换取最短的总搬运时间。

以软盘控制器常用的通道 2 为例，把 8192 字节从设备搬到物理地址 **0x23450** 的完整配置序列如下。页号 **0x02**、页内偏移 **0x3450**，结束地址 **0x2544F**，不跨页边界；计数写 $8192 - 1 = 8191 = \mathbf{0x1FFF}$ 。拼接关系与这个缓冲区在页内的位置如图所示。



图示：8237 的 24 位地址拼接：外部页寄存器锁存 A23-A16，通道地址寄存器提供 A15-A0。示例页号 **0x02**、页内偏移 **0x3450** 合成物理地址 **0x23450**；8192 字节传到 **0x2544F** 为止，仍在页 **0x02** (**0x20000-0x2FFFF**) 之内。一次传输中页寄存器保持不变，跨 64 KiB 页边界的缓冲区必须拆成两次传输。

```

; 8237 ch2 block transfer: device -> memory, 8192 B at phys 0x23450
mov al, 0x06      ; mask ch2 while programming
out 0x0A, al      ; single-channel mask register
mov al, 0x86      ; mode: block, write-to-memory, ch2
out 0x0B, al      ; mode register
out 0x0C, al      ; clear byte-pointer flip-flop (value ignored)
mov al, 0x50      ; address low byte (0x3450)
out 0x04, al
mov al, 0x34      ; address high byte
out 0x04, al
mov al, 0xFF      ; count low byte (0x1FFF = 8192-1)
out 0x05, al
mov al, 0x1F      ; count high byte
out 0x05, al
mov al, 0x02      ; page register -> A16..A23

```

```

out 0x81, al
mov al, 0x02    ; unmask ch2
out 0x0A, al
; device raises DREQ; 8237 requests the bus and moves bytes;
; TC (count wraps through 0) ends the transfer and raises EOP/IRQ

```

序列里有两处容易出错。其一，16 位寄存器经同一个 8 位端口分两次写入，内部字节指针触发器决定下一次写的是低字节还是高字节，配置前必须先发清除命令，否则高低字节会写反。其二，屏蔽必须在配置之前、开放必须在配置之后，否则设备可能在地址只写了一半时就开始搬运。周期窃取的比例可以估算：软盘数据率约 62.5 KB/s，即每 $16\mu\text{s}$ 来一个字节；设总线周期 250ns 、一次单字搬运占 4 拍共 $1\mu\text{s}$ ，DMA 占用总线的时间比例约 $1/16 \approx 6\%$ ，其余 94% 的总线时间仍归 CPU。上面的配置序列按块模式给出：8237 一旦拿到总线就传满 8192 字节，按一次搬运 $1\mu\text{s}$ 算是 $8192 \times 1\mu\text{s} \approx 8.2\text{ms}$ 内总线全部归 DMA。但这个数字只有在设备能持续供数时才成立——块模式传输的跨度实际由 DREQ 决定，软盘每 $16\mu\text{s}$ 才来一个字节，传完 8192 字节仍要 $8192 \times 16\mu\text{s} \approx 131\text{ms}$ ，并不比单字模式快，而这 131 ms 里总线被 DMA 占着，CPU 长时间取不到指令。正因如此，PC 的软盘 DMA 实际配成单字模式（模式字节 0x46）；块模式适合的是设备端能跟上总线速度的场合，如内存到内存传送或磁盘内部缓冲命中后的连续读出。选哪种模式，本质上是在搬运总时间与 CPU 停顿之间做权衡。

DMA 最容易错在可见性和所有权 DMA 让 CPU 和设备共享主存，也引入 cache 一致性、顺序和缓冲区所有权问题。CPU 写好的描述符必须先对设备可见；设备写入的缓冲区必须在 CPU 读取前变得可见；CPU 不能在设备仍拥有缓冲区时重用它；设备不能越过分配的地址范围。现代系统还会用 IOMMU 限制 DMA 能访问哪些物理页，避免坏设备或错误驱动破坏任意内存。



图示：DMA 缓冲区所有权。完成通知、cache 可见性和所有权回收必须按顺序发生。

边界	必须说明	失败后果
描述符可见	CPU 写描述符后如何让设备看到	设备读到旧地址、旧长度或旧标志
数据可见	设备写缓冲区后 CPU 如何看到新数据	CPU 读到 cache 中的旧内容
所有权	缓冲区何时归 CPU，何时归设备	重用正在 DMA 的缓冲区，数据被覆盖
地址权限	设备能访问哪些物理页或 I/O 虚拟地址	DMA 写坏无关内存或触发 IOMMU fault
完成顺序	completion 到达是否代表所有数据已可见	收到中断后仍读到未完成数据

五、从教学总线到现代互连

早期微机的并行总线比较直观：地址线、数据线、读写控制、片选和等待信号都能在逻辑分析仪上看到。现代芯片内部和 PCIe 这类外部互连更复杂，可能使用分层 packet、请求队列、completion、错误报告和流控。但抽象问题没有改变：事务由发起者提出，经互连路由到目标，目标返回数据或完成状态，错误路径必须被记录和上报。

互连对象	旧总线中的类比	现代形式
地址阶段	地址线和片选	请求 packet、BAR、路由 ID、地址译码
数据阶段	数据线和字节使能	payload、byte enable、burst 或 cache line
等待	READY/WAIT	credit、ready/valid、队列背压
完成	采样数据或写周期结束	completion packet、状态位、中断或 MSI
错误	总线错误或超时	poison、fault、AER、IOMMU fault、设备 error

总线仲裁：多个主设备谁先用总线 共享总线上可以有多个能发起事务的主设备：CPU、DMA 控制器、显示控制器都可能同时要占用地址线 and 数据线。同一时刻只能有一个主设备驱动总线，因此必须有一种决定“这一轮给谁”的机制，这就是总线仲裁（bus arbitration）。仲裁要同时回答三个问题：裁决花多少时间，谁拥有优先权，低优先级请求会不会永远等不到。经典做法有三种，它们在线数、延迟和公平性之间做出不同取舍。

集中式仲裁器为每个主设备拉一对独立的请求线（request）和授权线（grant），全部连到一个中央仲裁器。所有请求并行到达，仲裁器在一两个周期内按优先级逻辑选出获胜者并抬起对应的 grant。这是最快的做法，优先级还可以做成可编程寄存器；代价是线数随主设备数线性增长， N 个主设备需要 $2N$ 根专用线，仲裁器本身也成为单点。

菊花链（daisy chain）只拉一条授权线，从仲裁器出发依次穿过所有主设备。设备没有请求时把授权信号向下一级传递，有请求时把授权截留下来占用总线。线数与设备数量无关，结构最简单；但优先级由设备在链上的物理位置写死——离仲裁器越近永远越优先，链尾设备可能长期等不到授权，而且授权要逐级传播，链越长延迟越大。

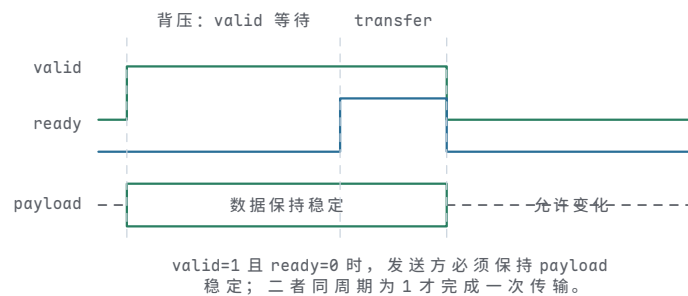
轮询计数（polling）用一小组设备号线代替一对一连线：仲裁器驱动 $\lceil \log_2 N \rceil$ 根线，轮流扫描设备编号，被扫到且有请求的设备抬起 busy 占用总线。线数最少，公平性也最灵活——计数器若每次从零开始，编号小的设备优先；若从上一次授权的设备之后继续扫描，就得到轮转公平。代价是平均要扫过半个编号表才能命中请求者，是三者中最慢的。

做法	专用线数	优先级	授权速度	关键问题
集中式仲裁器	$2N$ （每设备一对请求/授权）	由仲裁器逻辑决定，可编程	最快，并行比较	线多，仲裁器是单点
菊花链	与设备数无关（一条授权链）	由物理位置固定，链首最高	逐级传播，链长则慢	链尾可能等不到，优先级不可改
轮询计数	$\lceil \log_2 N \rceil$ 根扫描线	由计数起点决定，可轮转公平	最慢，平均扫半表	扫描本身占用总线时间

现代片上互连并没有消灭仲裁，而是把它从“整条共享线上的全局问题”拆小，做进了每一个交换节点。NoC 路由器的每个输入端口要仲裁来自不同方向的 flit，crossbar 要为每个输出端口在多个输入之间仲裁，PCIe switch 要在端口和虚通道之间仲裁。共享并行总线退化成点到点链路之后，“谁先用”的问题依然存在于每个交换机内部——只是裁决范围从整块板卡缩小到一个端口。

valid/ready 是理解现代片上互连的最小模型 许多片上接口都可以抽象成

valid/ready 握手。发送方在 `valid` 为 1 时保持请求或响应稳定,接收方在 `ready` 为 1 时表示可以接收,只有二者同周期为 1,事务片段才真正完成。请求通道和响应通道可以分开,读请求和读数据也可以隔很多周期。这个模型比“总线一拍完成”更接近现代硬件。

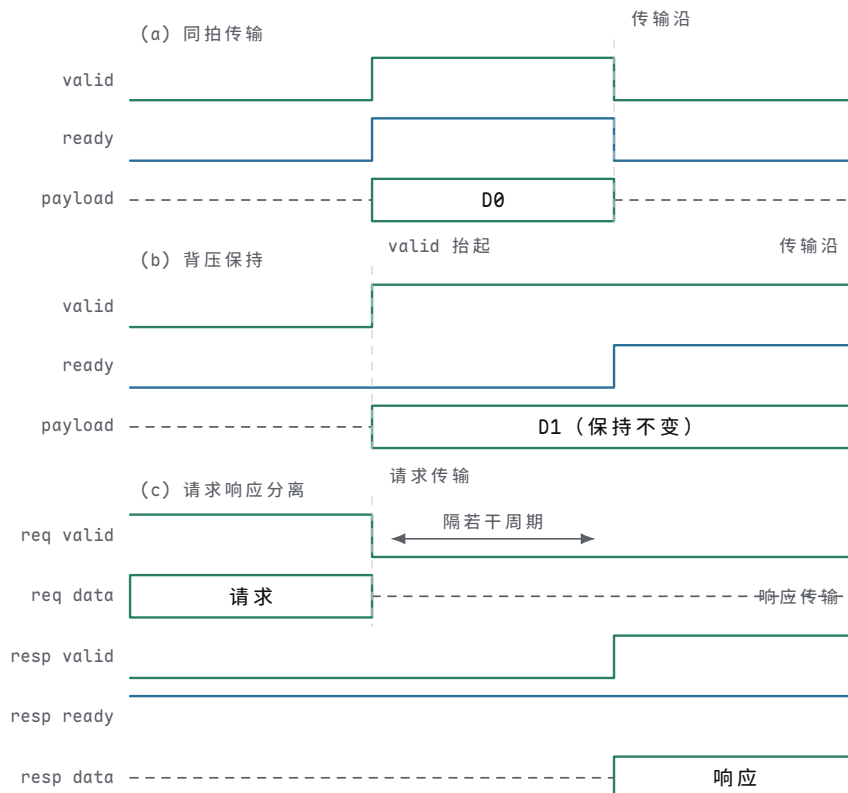


图示: `valid/ready` 握手。发送方先抬 `valid` 并保持数据稳定;接收方在背压期间保持 `ready=0`;两条线在 `transfer` 周期同时为 1,事务片段才完成。背压不是错误,而是接收方暂时不能接受。

handshake rule:

```
transfer happens only when valid == 1 and ready == 1
while valid == 1 and ready == 0:
    sender keeps payload stable
receiver may apply backpressure by deasserting ready
```

三种 `valid/ready` 时序情形 `valid/ready` 的全部行为可以归纳成一条采样规则和三种常见情形。采样规则是:只在某个采样沿上 `valid` 和 `ready` 同时为 1,该通道才完成一次传输。三种常见情形是:双方同拍就绪,当拍完成;接收方拉低 `ready` 施加背压,发送方必须保持 `valid` 和 `payload` 不变;请求通道和响应通道各自独立握手,两次传输之间可以隔任意多个周期。这三种情形都不是异常,而是流控协议的正常组成部分。



图示：三种情形一条规则：只在 valid 与 ready 同拍为 1 的沿上传输。(a) 双方同拍就绪，当拍完成；(b) ready 为 0 的期间 valid 与 D1 保持不变，这是接收方施加的背压，不是错误；(c) 请求在一个沿完成，响应通道隔若干周期后才完成自己的传输，两个通道互不等待。

写缓冲和内存顺序解释了现代芯片为什么看起来更快 写入比读取更容易被隐藏。CPU 可以先把写事务放进写缓冲，让执行流水线继续推进；写缓冲再把相邻写合并、排队发送到 cache 或互连。这样常见 STORE 的表面延迟很低，但代价是可见性变复杂：其他核心、DMA 或设备什么时候看到这次写，取决于 cache 一致性、内存类型和屏障规则。普通 RAM、MMIO 和 DMA 描述符不能使用同一套可见性假设。

总线错误和超时必须成为规格的一部分 未映射地址不能只写成“不要访问”。真实系统必须规定访问未映射区域时会发生什么：返回固定值、保持等待直到超时、产生总线错误，或进入异常入口。设备无响应、DRAM 控制器报错、ECC 检测失败、外设权限不允许，也都应该形成可观察错误状态。没有错误约定的整机，调试时会把硬件故障误判为程序死循环。

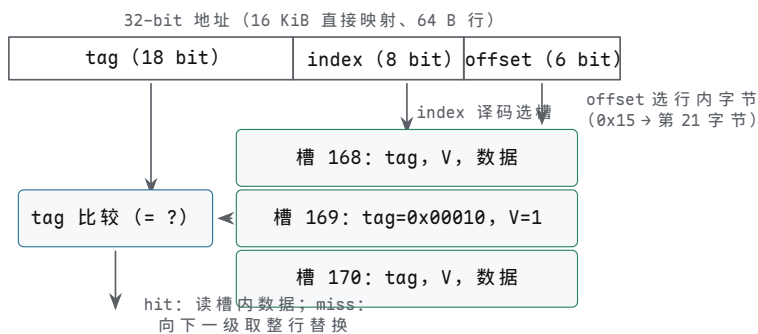
cache 行和局部性把常见访问留在近处 主存容量大但离 CPU 远，寄存器离 CPU 近但容量小，cache 处在二者之间。cache 不是“更快的内存副本”这么简单，而是按 cache line 搬运的一组近处 SRAM。CPU 读取某个地址时，cache 会用地址中的 tag 判断目标行是否已经在近处；若命中，数据直接从 cache 返回；若未命中，cache 控制器向下一级 cache 或 DRAM 请求整行，再把需要的字节送回 CPU。

对象	硬件含义	关键问题
cache line	一次填充和替换的最小数据块	行大小、字节偏移和未对齐访问是否清楚
tag	判断某个 cache 槽保存的是哪段地址	命中比较是否覆盖高位地址和有效位
index	选择 cache 中的候选槽或集合	映射冲突是否可能造成反复 miss
valid/dirty	表示行是否有效、是否被修改过	替换时是否需要写回下一级
miss	近处没有所需行，需要向下一级请求	miss 期间 CPU 停顿、继续执行或乱序等待的边界

局部性解释了 cache 为什么有效。时间局部性表示刚访问过的数据很可能再次访问，例如循环变量和栈顶对象；空间局部性表示访问某个地址后，很可能访问相邻地址，例如顺序扫描数组。cache line 正是利用空间局部性：一次 miss 不只取一个字节，而是取一整段相邻字节。代价也随之出现：若程序跨很大步长访问数组，或者两个热数据反复映射到同一 cache 集合，硬件仍会不停搬行，表面上只是 LOAD 很慢，实质上是事务层反复 miss 和替换。

```
32-byte cache line example:
address bits = [ tag | index | byte_offset ]
byte_offset selects one byte within the line
index selects a cache set
tag comparison decides hit or miss
```

案例：一次 cache 命中的完整分解 cache 的查找可以拆成三个字段各管一件事：offset 选行内字节，index 选槽，tag 判身份。以 32-bit 地址、16 KiB 直接映射、64 B 行为例：offset = $\log_2 64 = 6$ bit；行数 = $16 \text{ KiB} \div 64 \text{ B} = 256$ ，index = $\log_2 256 = 8$ bit；tag = $32 - 8 - 6 = 18$ bit。任何地址到来时，硬件先按 index 找到唯一的槽，再用 tag 和 valid 位判断“槽里这行是不是这段地址”。



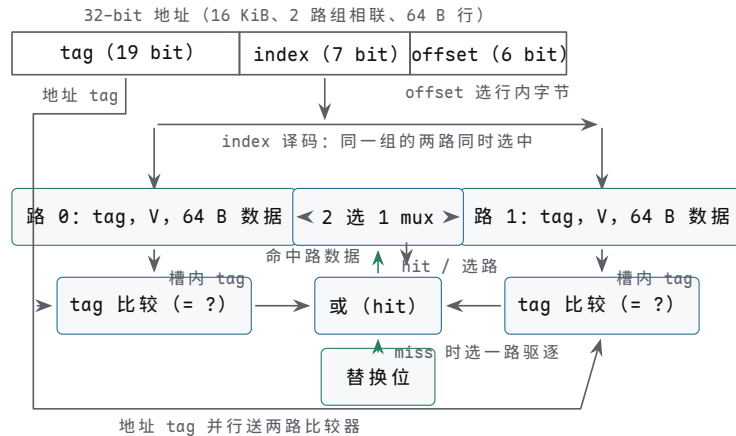
图示：直接映射 cache 的查找。地址拆成 tag/index/offset 三段：index 译码选出唯一的槽，槽内 tag 与地址 tag 比较决定 hit/miss，offset 再从 64 B 行内选出目标字节。

下表把四次访问的分解和结果走一遍。注意第 3、4 次展示了直接映射最典型的失效模式：

次序	地址	tag / index / offset	结果
1	0x00042A40	0x00010 / 0xA9 / 0x00	槽 169 的 V=0 → miss；从 DRAM 取回 64 B 整行填入，tag 写 0x00010、V 置 1，返回第 0 字节
2	0x00042A40	0x00010 / 0xA9 / 0x00	tag 相等且 V=1 → hit，直接从槽内返回数据
3	0x00082A55	0x00020 / 0xA9 / 0x15	槽 169 的 tag 是 0x00010，不等 → 冲突 miss；取回新行替换旧行（旧行若 dirty 先写回 DRAM），返回第 21 字节
4	0x00042A40	0x00010 / 0xA9 / 0x00	槽 169 刚被第 3 次访问换掉 → 又 miss

两个热地址映射到同一个槽时，直接映射会反复 miss——这正是组相联存在的理由：同一个槽放多路，几路 tag 并行比较，任何一路相等都算 hit，常用数据就不必互相驱逐。

两路组相联让热行各占一路 直接映射的冲突来自“一个槽只能放一行”。把同样 16 KiB、64 B 行的 cache 改成 128 组、每组 2 路：offset 仍是 6 bit，index 从 8 bit 变成 7 bit，tag 从 18 bit 变成 19 bit。重走前面的四次访问：0x00042A40 和 0x00082A55 的 index 都是 0x29，tag 分别是 0x00021 和 0x00041。第 1 次 miss，填入路 0；第 2 次 tag 相等，hit；第 3 次两路 tag 都不等（路 1 的 V 仍为 0），仍然 miss，但新行填入路 1，不再驱逐路 0；第 4 次再读 0x00042A40，路 0 的 tag 相等，变成 hit。直接映射下第 3、4 次的互相驱逐，在 2 路组相联下只剩第 3 次那次无法避免的首次 miss。代价是每组多一个 tag 比较器、一个数据选择 mux 和若干替换位。



图示：2 路组相联的查找。index 译码选中整个组，组内两路的 tag 同时与地址 tag 比较；任一路相等，或门即输出 hit，并用比较结果控制 mux 选出命中路的数据。miss 时按组内替换位选一路驱逐、填入新行。

写策略决定 STORE 什么时候真正离开 CPU 读 miss 的动作比较直观：向下级取回数据。写入更复杂，因为 CPU 可以先改 cache，再决定什么时候把修改传播到下一级。写穿策略每次写命中都继续写下一级，行为直接但带宽压力大；写回策略只在 cache 中标脏位，替换时再写回下一级，常见情况下更快，但可见性和错误恢复更复杂。无论哪种策略，字节写都必须尊重 byte enable，不能把同一 cache line 中无关字节改坏。

写策略	优点	风险和边界
写穿	下一级更快看见新值，调试直观	带宽压力大，常配合写缓冲隐藏延迟
写回	多次写同一行可合并，减少外部事务	dirty 行替换、掉电和 DMA 可见性更难处理
写分配	写 miss 时先把整行取入 cache 再修改	小写入可能触发整行读取
非写分配	写 miss 直接向下级写，不装入 cache	后续读同地址可能仍 miss

这也是 MMIO 不能随意缓存的根本原因。写 LED 寄存器、清中断状态或启动 DMA 命令时，程序需要的是一次外设可见动作，而不是某条 cache line 中的普通字节更新。若 MMIO 被写回 cache 吞掉，设备可能永远看不到命令；若读状态寄存器被 cache 命中，CPU 可能一直看到旧状态。因此地址属性必须区分普通可缓存内存、不可缓存设备内存、写合并 framebuffer、DMA coherent 区域等不同事务约定。

cache miss 是事务暂停，不是抽象的慢 当 LOAD miss 时，流水线不能凭空得到数据。简单 CPU 会停在访存阶段，保持地址和控制状态，直到下级返回数据；高性能 CPU 可能让独立指令继续执行，把这条 LOAD 放在未完成队列中，等数据回来再唤醒依赖指令。二者复杂度不同，但都必须维护同一个可见性约定：依赖这次 LOAD 的指令不能使用错误旧值，异常和中断也不能让程序看见半提交状态。

miss 类型	事务路径	典型处理
L1 miss/L2 hit	L1 请求 L2 返回 cache line	几到十几周期，流水线可能短暂停顿
最后级 cache miss	请求内存控制器和 DRAM	上百周期级别，常需乱序、预取或多线程隐藏
TLB miss	地址转换缓存缺失，要查页表或进异常	页表 walker、权限检查和可能的页错误

写回替换	脏行被换出，需先写下一级	替换队列、写缓冲和错误报告
设备/DMA 冲突	cache 中旧副本与设备写入不一致	coherent 互连、cache 维护或不可缓存映射

低时延来自近处、缓存、并行和卸载 现代整机降低时延，不是让所有路径都变成零等待。L1/L2 cache 让常见数据走近处 SRAM；TLB 让近期地址转换不必每次查页表；预取和分支预测提前准备可能需要的路径；流水线、旁路和乱序调度让独立工作重叠；DMA、队列和中断聚合把批量 I/O 从 CPU 指令流中卸载。失败路径仍然存在：cache miss、TLB miss、DRAM 排队、设备背压、DMA fault 和错误中断都会把访问拉回慢路径。

六、存储层次、地址转换和设备驱动 trace（执行轨迹）

前面已经讲了单次总线事务、MMIO、中断、DMA 和 cache。真正的系统程序运行时，这些对象不会孤立出现。一次普通 LOAD 可能先经过 TLB 查找，再查 L1 cache，未命中后查 L2/L3，最后进入内存控制器；一次设备接收数据可能由驱动准备描述符，设备 DMA 写缓冲区，cache 一致性或维护操作让 CPU 看见新数据，中断再把完成事件带回来。若教材只讲“内存比寄存器慢”，就漏掉了真正的硬件路径。

访问层次	常见命中路径	失败或慢路径
寄存器	直接从寄存器堆读写	端口冲突、旁路失败或流水线停顿
L1 cache	近处 SRAM 命中，几个周期内返回	L1 miss，向 L2 请求整条 cache line
L2/L3 cache	片上较大 cache 命中	最后级 miss，访问内存控制器和 DRAM
TLB	地址转换缓存命中	page walk、页错误或权限异常
DRAM	控制器调度行命中和突发传输	行冲突、刷新、排队、ECC 错误
设备/MMIO	设备寄存器响应读写	背压、超时、错误状态或中断完成

TLB 把地址转换也纳入近处缓存 虚拟地址或线性地址到物理地址的转换若每次都查页表，访存会变得很慢。TLB 可以理解为“地址转换的 cache”：它保存近期虚拟页到物理页框的映射及权限位。LOAD/STORE 形成虚拟地址后，先用虚拟页号查 TLB；命中时得到物理页框和权限信息，再与页内偏移组合成物理地址；未命中时，硬件 page walker 或软件异常处理会去内存里的页表结构查找。

```
virtual address = [ virtual_page_number | page_offset ]
TLB hit:
    physical_address = TLB[virtual_page_number].frame | page_offset
TLB miss:
    walk page tables or enter miss handler
```

TLB 字段	作用	关键问题
虚拟页号 tag	判断当前地址是否命中某个转换项	进程切换、地址空间标识和刷新规则是否清楚
物理页框	与页内偏移组合成物理地址	页大小、对齐和物理地址宽度是否一致
权限位	读写、执行、用户/特权等访问属性	访问失败是权限异常，不是普通 cache miss

valid/global 表示转换是否有效、是否跨进程保留 页表修改后如何让旧 TLB 项失效

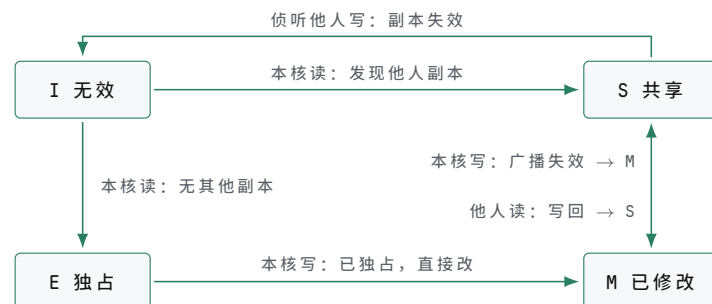
TLB miss 和 cache miss 都是“近处没有”，但语义不同。cache miss 只是数据不在近处，通常可以向下级取回；TLB miss 可能只是转换缓存缺失，也可能说明页不存在或权限不允许。若页表项不存在，硬件不能继续访问内存数据；若权限不允许，访问必须转入异常。这个差别解释了为什么地址转换、保护和异常在系统硬件中是同一条路径的不同分支。

多核和 DMA 让 cache 一致性成为硬件约定 单核教学机里，cache 可以先理解为 CPU 与主存之间的近处副本。多核或 DMA 加入后，问题变成“多个观察者如何看见同一地址的值”。一个核心的 cache 中可能有某条 line 的旧副本，另一个核心或设备刚刚写入了新值；如果没有一致性协议、cache 维护或不可缓存映射，CPU 读到的就可能不是系统真实最新值。

场景	风险	常见硬件/软件边界
核心 A 写，核心 B 读	B 的 cache 中可能有旧行	cache coherence、失效消息、共享/独占状态
CPU 写描述符，设备读	描述符可能还在 CPU cache 或写缓冲中	coherent DMA、flush、写屏障
设备写缓冲区，CPU 读	CPU cache 中可能有旧数据	invalidate、coherent 互连、不可缓存映射
MMIO 状态寄存器	cache 命中会隐藏设备新状态	设备内存属性、禁止普通缓存

一致性协议细节可以很深，但教学阶段先抓住三件事。第一，同一地址可能在多个近处缓存中有副本。第二，写入必须让其他观察者的旧副本失效或更新。第三，设备并不总是自动参与 CPU cache 一致性；若平台不支持 coherent DMA，驱动必须在交出缓冲区前后做 cache 维护。否则 DMA 完成中断到达后，CPU 仍可能读到旧 cache line。

MESI 用四个状态约定谁拿着最新值 这套状态存在的意义，是让任何时刻、任何地址在所有观察者眼里只有一份“最新值”。想写某一行，核必须先成为唯一持有者：从 S 升级要广播失效，把其他核的副本打成 I；只有处于 M 或 E 时才能直接改。想读可以共享：多个核同时持有 S，读到的都是同一个干净副本，但共享期间谁也不能直接写。于是两个核不会各自拿着不同的新值——要么一个核独占、其余无效，要么大家一起只读。参与一致性互连的 DMA 设备也被同一套状态约束；不参与的 platform，驱动就要用 cache clean/invalidate 手工扮演“写回”和“失效”的角色。



图示：MESI 一致性状态图（教学简化版）。本核读把 I 变成 E（无其他副本）或 S（他人有副本）；本核写必须先独占：S 到 M 要广播失效，E 到 M 直接改；总线侦听到他人读时 M 写回并退到 S；侦听到他人写时副本失效（E、M 同样失效，M 先写回）。

案例：MESI 在两个核之间的完整走查 状态图只有放进具体访问序列才有意义。下面用两个核、一个地址 X 把上一专题的状态机完整走一遍。设初始时核 A、核 B

的 cache 中 X 行均为 I (无效)，内存持有 X 的最新值。先说明一个细节：按严格 MESI，核 A 第一次读入 X 时若没有发现任何其他副本，应记为 E (独占)；本走查假设 X 此前已被第三个核读过，或采用“读入即共享”的教学约定，因此第一步记为 S。这不影响后续步骤的转换逻辑。

步骤	动作	核 A	核 B	互连上发生的事
0	初始	I	I	X 的最新值在内存中
1	核 A 读 X	S	I	A 的读 miss，经互连从内存取回整行，记为共享副本
2	核 B 读 X	S	S	B 的读 miss，从内存取行；A 侦听到这次读，保持 S，两核持有同一份干净副本
3	核 A 写 X	M	I	A 先广播失效消息，B 的副本作废 (S → I)；确认后 A 独占修改本行 (S → M)，内存中的 X 从此过期
4	核 B 读 X	S	S	B 的读 miss；A 侦听到读，把脏行写回内存（或直接转发给 B），自己降为 S；B 得到最新数据的共享副本

逐步看两处关键转换。第 3 步是“写之前先独占”：A 不能只改自己手里的 S 副本，否则 B 会继续读到旧值，同一地址就出现了两个“最新值”；广播失效把其他副本全部打成 I 之后，A 才升级成 M 并修改。第 4 步是“最新值跟着 M 走”：此时内存里的 X 是旧数据，B 的读不能由内存直接回答，必须由持有 M 行的 A 先把数据交出来—写回内存或 cache 之间直接转发—之后两核各持 S，内存恢复最新。

这张表就是一致性协议的 trace：每一行对应状态图上的一条边，每一步结束都满足同一条不变式—X 的最新值在全局唯一，要么在某核的 M 行里（其余全为 I，内存过期），要么在所有 S 副本与内存里保持一致。调试多核程序时若怀疑数据不一致，最有效的办法正是把可疑的访问序列按这种方式列成表，逐步检查哪一步破坏了唯一性。

store buffer 和屏障把“执行完成”与“可见”分开 STORE 对流水线来说可以很早“执行完成”：地址和数据已经形成，写事务进入 store buffer，后续独立指令可以继续执行。但这个 STORE 什么时候对其他核心、DMA 或设备可见，要看 store buffer 排队、cache 状态、内存类型和屏障规则。多核内存模型本身超出本书范围，这里只需要与 DMA 直接相关的一条规则：普通 RAM 写可以合并和延迟；MMIO 写通常要求更强顺序；DMA 描述符写在通知设备前必须已经对设备可见。

```
prepare DMA descriptor:
    descriptor.addr = buffer
    descriptor.len  = length
    memory_barrier()
    MMIO[DEVICE_DOORBELL] = descriptor_index
```

这里的屏障不是“让 CPU 慢一点”的软件仪式，而是要求硬件在门铃 MMIO 写对设备可见之前，先让描述符内容按平台规则可见。若顺序反了，设备可能看到门铃，却读到旧描述符或半写入描述符。许多难复现的设备错误，本质上就是这个可见性边界没有写清楚。

对象	可见性要求	失败表现
普通数据	由语言、编译器和内存模型共同约束	多核读到旧值或乱序观察
锁变量	获取/释放顺序必须约束临界区数据	进入临界区后仍看见旧数据

DMA 描述符	通知设备前必须对设备可见	设备搬错地址或长度
MMIO 门铃	前面的配置写必须先到设备接口	设备按旧配置启动
中断完成	completion 到达后数据必须已可读取或可同步	中断处理读到旧缓冲区

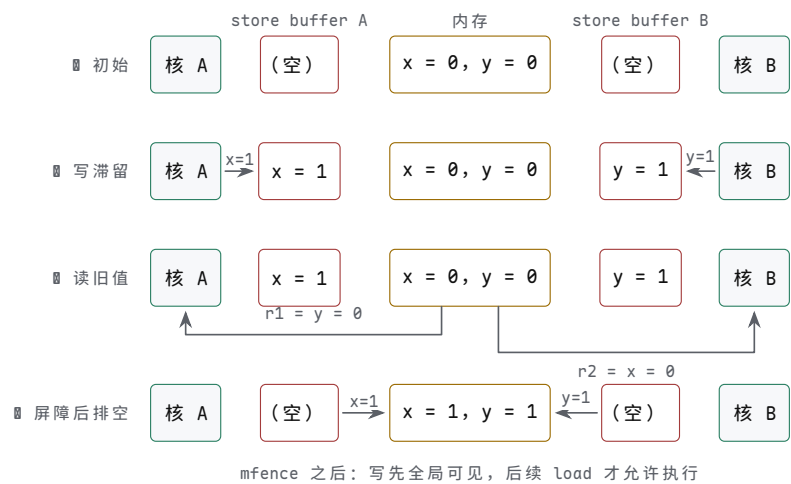
案例：一个内存屏障挡住什么 屏障的作用可以用一个经典的双核序列看清。两个核共享变量 x 和 y ，初值都为 0：核 A 先把 1 写入 x ，再读 y 到 $r1$ ；核 B 先把 1 写入 y ，再读 x 到 $r2$ 。直觉上，两次写和两次读无论怎样交错，总有一个读应该看到对方的新值 1。但在带 store buffer 的真实机器上， $r1=0$ 且 $r2=0$ 的结果确实可能出现。

```
core A      core B
mov qword ptr [x], 1  mov qword ptr [y], 1
mov rax, [y]          mov rbx, [x]
```

原因就在上一专题的可见性划分： $x=1$ 对核 A 来说“执行完成”只表示写事务进入了 A 的 store buffer，并不意味着其他核已经能看见它。两个核的 store 都可能还排在各自的 store buffer 里，两个 load 就已经从内存（或对方尚未更新的 cache 副本）里取回了 0。

次序	核 A	核 B	内存中可见的 x / y
1	$x=1$ 进入 A 的 store buffer	—	0 / 0
2	—	$y=1$ 进入 B 的 store buffer	0 / 0
3	$r1=y$ ，读到 0	—	0 / 0
4	—	$r2=x$ ，读到 0	0 / 0
5	store buffer 排空， $x=1$ 全局可见	store buffer 排空， $y=1$ 全局可见	1 / 1

把上表的关键时刻画成四张快照，写滞留与读旧值各处于什么位置就更直观。



图示：store buffer 双核序列的四张快照。 $x=1$ 、 $y=1$ 先滞留在各自核的 store buffer (❶)，两个 load 直接从内存读回旧值 0 (❷)，于是 $r1=0$ 且 $r2=0$ 成立；mfence 强制 store buffer 排空、写全局可见之后才放行后续 load (❸)，这个结果即被禁止。四张快照与上表第 1、2、5 步前后的状态一一对应。

这个结果不违反任何一条单核规则——每个核自己看来程序都按序执行了——但它违反了“所有核看到同一个全局顺序”的直觉。 $x86$ 的 `mfence` 指令正是为此存在：

它强制本核的 store buffer 排空之后，后续的 load 才允许执行，也就是说本核此前的写必须先对其他核可见。

core A	core B
mov qword ptr [x], 1	mov qword ptr [y], 1
mfence	mfence
mov rax, [y]	mov rbx, [x]

加入屏障后， $r1=0$ 且 $r2=0$ 被禁止。推理只需直觉：若 A 读到了 $y=0$ ，说明 B 的 $y=1$ 在那一刻尚未对 A 可见；而 B 的 `mfence` 保证 $y=1$ 全局可见之后 B 才会执行 $r2=x$ ，到那时 A 的 $x=1$ 早已因 A 自己的 `mfence` 而对所有核可见，所以 B 读 x 必得 1。屏障挡住的正是“写还留在 store buffer 里、读却已经出发”这段窗口；它没有让写变得更快，只是把可见性顺序钉死。

描述符环把 DMA 变成持续事务流 高速设备通常不会每次只处理一个缓冲区。网卡、磁盘控制器、USB 控制器常使用描述符环或队列：CPU 准备一批描述符，设备按头尾指针消费，完成后写回状态或 completion 队列，并用中断或轮询通知 CPU。这样可以减少 MMIO 次数，让 CPU 和设备并行工作。

```
receive ring:
CPU owns descriptor N
CPU fills buffer address and length
CPU marks descriptor ready for device
CPU updates tail pointer or rings doorbell
device writes packet into buffer
device writes completion status
CPU regains ownership and processes buffer
```

环队列字段	硬件含义	关键问题
描述符地址	设备要读写的主存位置	地址是否经过 IOMMU 或物理地址转换
长度	本次 DMA 允许访问的字节数	设备不能越界写出缓冲区
所有权位	当前由 CPU 还是设备拥有	所有权不重叠，状态切换有屏障
头尾指针	CPU 和设备约定的生产/消费位置	指针更新顺序和环绕规则清楚
completion	设备报告完成、长度、错误码	中断到达不等于所有 cache 可见性自动正确

这个模型也说明驱动程序为什么必须理解硬件。驱动不是“调用设备 API”的普通软件，它要写 MMIO 寄存器、分配对齐缓冲区、建立 DMA 映射、维护 cache 可见性、处理中断、回收描述符和处理错误。每一步都对应本章的硬件对象：地址、权限、字节数、所有权、完成和错误。

描述符环不动、指针动 描述符环的关键是“环不动、指针动”：描述符数组在内存中的位置固定，CPU 只推进 SW 指针，设备只推进 HW 指针，走到末尾就回绕；两个指针的差值就是环中未完成事务的数量。doorbell 不携带数据，只是告诉设备“指针又动了”，设备即使错过一次 doorbell，靠指针差也能补读。反过来，某个描述符是否完成，不能凭 doorbell 或中断猜测，必须以设备写回描述符状态位（或 completion 队列项）为准；CPU 回收缓冲区前，还要确认这次写回对自己可见。



图示：描述符环。设备按 HW 指针顺序消费，CPU 按 SW 指针顺序生产，描述符 7 之后回到 0；CPU 更新 SW 后写一次 doorbell 提示设备。完成与否以设备写回描述符状态位为准，doorbell 只是“有新描述符”的提示。

驱动 bring-up 应分层验证 调试设备时，最差的方法是直接运行完整驱动并等待它“能不能用”。更可靠的顺序是先验证配置空间或设备 ID，再验证 MMIO 映射，再验证单个寄存器读写，再验证中断，再验证一个最小 DMA 描述符，最后才跑持续队列。每一层都要有可观察的结果。

bring-up 阶段	操作	预期结果
枚举/识别	读取设备 ID、版本或能力寄存器	地址映射正确，读值稳定可复现
复位	写复位位并轮询 ready/busy	状态位按规格变化，不永久 busy
MMIO 回读	写可回读配置寄存器再读回	字节序、位域和写掩码正确
中断	人工触发或配置定时事件	中断线/MSI 到达，清除顺序正确
单次 DMA	一个描述符、一个缓冲区、一个 completion	地址、长度、cache 可见性和错误码正确
持续队列	多描述符环和批量 completion	所有权、环绕、背压和中断聚合正确

这张表也适用于教学板。即使没有 PCIe，只用一个 GPIO、定时器或简化 DMA 控制器，也应该按同样顺序验证：先确认地址译码，再确认寄存器语义，再确认事件通知，再确认数据搬运。这样本书的硬件 trace 就能从最小 CPU 直接延伸到现代设备驱动。

案例：一个接收包怎样进入内存 以网卡接收一个数据包为例。CPU 先分配若干缓冲区，把每个缓冲区的地址和长度写入接收描述符环；随后保证描述符对设备可见，再写 MMIO 门铃或尾指针告诉设备可以接收。设备收到外部数据后，作为 DMA 发起者把数据写入某个缓冲区，再写 completion 或描述符状态，最后触发中断。CPU 处理中断，确认 completion，按一致性规则让缓冲区数据对 CPU 可见，然后把缓冲区交给上层软件。

阶段	硬件事务	观察要点
准备缓冲区	CPU 在 RAM 中分配对齐缓冲区	物理地址或 I/O 虚拟地址有效，长度足够
写描述符	CPU STORE 地址、长度、所有权位	描述符 cache line 已按平台规则可见
通知设备	CPU 写 MMIO tail/doorbell	MMIO 不被普通 cache 合并或延迟越界
设备 DMA 写	设备发起内存写事务	IOMMU 权限、字节数、completion 顺序正确
投递完成	设备写状态并触发中断	completion 可读，中断不会早于数据可见

CPU 回收 中断处理读取状态和缓冲区

cache invalidate 或
coherent 规则完成

receive path:

```

CPU: fill rx_desc[N] with buffer address and length
CPU: barrier, then MMIO[RX_TAIL] = N
DEV: reads rx_desc[N]
DEV: writes packet bytes into buffer
DEV: writes completion status
DEV: raises interrupt
CPU: acknowledges interrupt and reads completion
CPU: synchronizes buffer visibility
CPU: consumes packet

```

这条路径里至少有三次“完成”。第一，CPU 写 MMIO 门铃完成，只表示设备接口看到了通知；第二，设备 DMA 写缓冲区完成，表示数据已经到达内存系统的某个可见点；第三，中断处理完成，表示 CPU 已经确认状态并回收所有权。把这三者混在一起，是驱动和硬件协同时最常见的错误来源。

案例：发送路径为什么要区分数据和描述符 发送数据包时方向相反。CPU 先把待发送数据写入缓冲区，再写发送描述符，最后通知设备。设备读描述符，再 DMA 读缓冲区，把数据发出。这里最关键的可见性顺序是：缓冲区内容必须先对设备可见，描述符必须描述正确地址和长度，门铃必须最后到达。否则设备可能发出旧数据、半写数据或错误长度的数据。

发送对象	可见性要求	失败表现
数据缓冲区	CPU 写入的数据先对设备可见	设备发送旧包或未初始化字节
描述符	地址、长度、标志位完整可见	设备读错缓冲区或长度
门铃寄存器	在数据和描述符之后对设备可见	设备提前启动，读到旧状态
completion	设备确认已不再读取缓冲区	CPU 过早重用缓冲区导致数据损坏

发送 completion 到达前，CPU 不应重用缓冲区。即使 MMIO 写门铃已经返回，设备可能还没有读描述符；即使设备已经读描述符，可能还没有读完数据；即使数据已经发出，completion 可能还在队列里等待 CPU 处理。因此所有权位和 completion 队列不是形式主义，而是让 CPU 和设备不同时修改同一块内存的硬件约定。

案例：块设备读请求的队列化 磁盘、NVMe、SD 控制器或简化块设备也可以用同一模型理解。CPU 准备命令描述符：逻辑块地址、目标缓冲区、长度、读写方向；设备读取命令后，在内部介质或控制器中完成实际读写，再通过 DMA 把数据写到主存，最后投递 completion。区别只在于设备内部工作可能更慢、队列更深、错误码更复杂。

块请求字段	含义	硬件问题
LBA 或块号	设备内部介质位置	越界、坏块、介质错误如何报告
缓冲区地址 长度	DMA 写入或读取的主存位置 传输字节数或块数	对齐、IOMMU、cache 可见性 不能越过缓冲区和描述符许可范围
方向	读设备到内存，或写内存到设备	DMA 方向决定 cache 维护方式

命令 ID	completion 对应哪个请求	队列乱序完成时仍能匹配请求
状态码	成功、超时、校验错误、权限错误	驱动恢复或上报错误

这个案例能把“设备很慢”讲得更具体。CPU 发出块读请求后，并不会逐字等待数据从设备口读出。它可以切换去做别的工作；设备和 DMA 在后台推进；中断或轮询 completion 时，CPU 再取回结果。硬件性能来自异步队列和批量搬运，而不是让单条 LOAD 直接等磁盘或网络。

案例：一个未映射访问如何成为可诊断错误 再看失败路径。CPU 执行 `MOV R1, [0xDEAD0000]`，地址译码没有任何目标响应。没有错误约定时，CPU 可能读到悬空总线值，或永久等待 ready。可靠平台应规定超时、总线错误或异常入口。若有 MMU，还可能在页表阶段就发现该地址未映射，不会走到外部总线。

失败位置	可观察现象	正确处理
页表不存在	地址转换阶段触发页错误	保存故障地址和错误码，进入页错误入口
权限不允许	转换存在但读写/特权不满足	触发保护异常，不发起外部访问
地址译码无目标	总线事务无人响应	超时或 bus error，而不是无限等待
目标返回错误	设备或内存控制器报告失败	错误状态进入异常或驱动恢复路径
ECC 校验失败	数据返回但校验错误	更正、重读、上报或机器检查

这张表对硬件调试尤其有用。现象“程序卡住”可能来自页表、总线译码、设备 ready、时钟、复位或错误入口任何一层。把失败位置写成层级，就能决定先看页表项、地址线、片选、ready、错误状态，还是异常向量。

读任何硬件平台，都可以用同一张事务表约束理解：发起者、目标、地址、方向、数据宽度、握手、完成、错误、副作用和可观察状态。早期 8086 总线、微控制器片内外设、PCIe 设备和现代 SoC 互连的规模不同，但这十项问题不变。

专题：DRAM 控制器为什么要排队和调度 SRAM 可以近似看成“地址稳定后读出数据”的存储器，DRAM 则必须先打开行、读写列、预充电、刷新。主存控制器面对的不是单个 LOAD，而是一串来自 CPU、DMA、显示控制器和其他主设备的请求。它要在正确性约束内排序请求，尽量利用行命中和 bank 并行，同时保证刷新不丢失数据。

DRAM 概念	含义	对系统的影响
row activate	把某一行搬到行缓冲中	同一行后续列访问较快
row conflict	下一个请求落在不同的行	需要预充电再打开新行，延迟增加
bank	多组可相对独立工作的阵列	交错访问可隐藏部分等待
refresh	周期性恢复电容电荷	刷新窗口内部分请求会被延后

ECC	用冗余位检测或纠正错误	增加位宽和延迟，但提高可靠性
-----	-------------	----------------

因此，主存延迟不是一个固定常数。一次 cache miss 到达内存控制器后，可能遇到行命中、行冲突、刷新、写队列排空或高优先级 DMA。系统软件看到的是“同样的 LOAD 有时快有时慢”，硬件工程师看到的是请求队列、调度策略和时序参数。教学时不必展开 DDR 全部命令，但必须让读者知道：主存不是无限宽、无限快、无排队的数据。

专题：cache 设计参数怎样改变命中率和时延 cache 的基本问题是把大而慢的主存切成较小的行，并保存最近使用的数据。地址会被拆成 tag、index、offset：offset 选择 cache 行内字节，index 选择组，tag 判断这一组里哪一路是目标行。不同组织改变冲突概率、比较器数量、替换策略和命中时延。

组织	优点	代价或风险
直接映射	查找简单、速度快、面积小	两个常用地址若映到同一行会反复冲突
组相联	多路候选降低冲突 miss	多个 tag 比较器和路选择 mux 增加延迟
全相联	任意行可放任意位置，冲突最少	硬件代价高，通常用于 TLB 或小 cache
写直达 写回	内存总是较新，恢复简单 多次写合并到 cache 行，带宽效率高	写流量大，常需写缓冲 dirty 行替换时必须写回，失电/一致性更复杂

评价 cache 不能只看“第二次访问变快”。还要看替换、脏行写回、字节写使能、未对齐访问、MMIO 不可 cache、DMA 可见性和异常路径。例如设备把数据 DMA 到主存后，CPU 若继续读旧 cache 行，就会看到过期数据；CPU 写好发送缓冲区后，若脏行没有写回，设备也可能读到旧内容。这就是为什么驱动要有 cache clean/invalidate 或一致性互连。

专题：MMIO 寄存器位语义要像数据手册一样写 MMIO 寄存器不是普通变量。每一位都可能副作用：读清、写 1 清、只读、保留位必须写 0、写后自动启动、硬件置位软件清除。若教材只写“把某地址当寄存器读写”，读者会在真实芯片上犯危险错误。

位语义	例子	软件访问要求
R0	输入电平、设备 ID	写入无效或保留，不应读改写破坏其他位
W0	FIFO 写口、启动命令	读值可能无意义，不能依赖回读
RW W1C	配置模式、使能位 中断状态位	修改前要明确复位默认值 写 1 清除对应事件，写 0 保持
RC reserved	读状态自动清除 未来扩展或工厂测试位	调试打印也可能改变设备状态 按手册写固定值，通常为 0

一个严谨的 GPIO 寄存器描述至少要写清：方向寄存器决定引脚驱动还是输入；输出寄存器只影响输出模式；输入寄存器来自同步后的外部电平；中断状态由边沿检测置位；清除中断采用 W1C；保留位读值未定义且写 0。这样驱动代码才能避免“读状态时把事件清掉”“读改写时误清中断”“把保留位写成 1 后芯片行为改变”等问题。

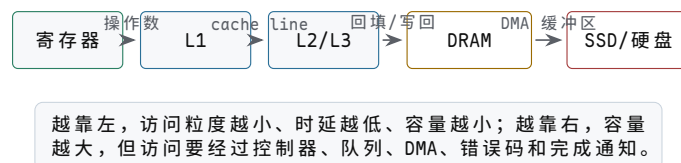
专题：总线错误要能定位到层级 当一次访问失败时，工程上最重要的问题不是

“错了”，而是错在地址转换、权限、互连、目标设备还是数据校验。若每层都只把错误折叠成一个笼统异常，调试会非常困难。更好的设计是保留故障地址、访问方向、主设备 ID、错误来源和目标返回码。

记录字段	作用	调试价值
fault address	失败访问的地址	区分空指针、越界和 MMIO 映射错误
access type	读、写、取指、原子操作	判断权限和副作用风险
master ID	CPU、DMA、GPU 或调试器	找到真正发起者
decode result	命中哪个目标或未命中	定位地址地图错误
response code	OKAY、SLVERR、DECERR、超时等	区分目标报错和无人响应

这也是为什么后续平台 bring-up 日志要记录总线层的事务信息。单看 C 代码中的一行 `*ptr` 无法知道失败来自页表、cache、IOMMU、互连还是设备时钟未开；完整错误记录能把问题缩小到可测量的硬件边界。

专题：内存层次要同时看容量、时延、带宽和所有权 计算机硬件不是只有 CPU。CPU 的每一次有用工作，几乎都要和某一级存储或 I/O 交换信息。寄存器最快但数量极少；L1/L2/L3 cache 容量逐级增大、时延逐级上升；DRAM 容量大但需要控制器、队列和刷新；SSD/硬盘容量更大，却只能通过控制器、队列、DMA 和中断间接参与执行。把这些对象都称为“存储”会掩盖关键差异：有些层按字节随机访问，有些层按 cache line 交换，有些层按页或块传输，有些层只能通过命令队列返回 completion。



图示：内存层次不是一条“更大的数组”，而是一组访问粒度和完成语义不同的层。

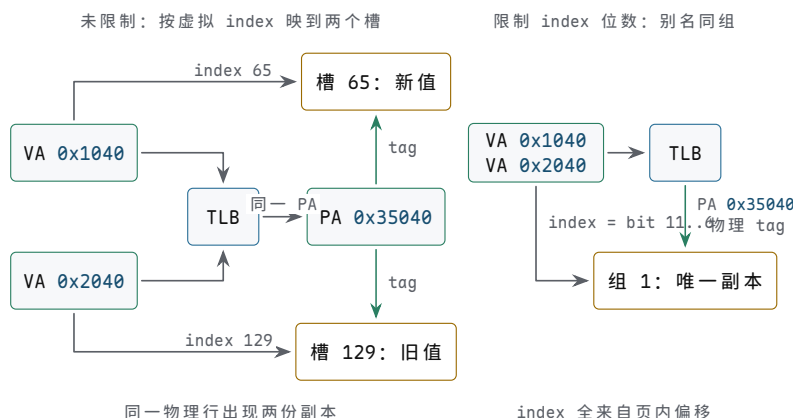
层次	常见粒度	谁直接访问	主要硬件问题
寄存器	word 或向量 lane	执行单元	端口数、旁路、写回冲突
L1 cache	cache line	当前核心 load/store 单元	命中时延、虚实地址、别名和写策略
L2/L3 cache	cache line	一个核心或多个核心	容量、共享、替换、一致性
DRAM	burst/cache line	内存控制器	bank、行命中、刷新、ECC、调度
SSD/硬盘	块、页或扇区	设备控制器和 DMA	命令队列、坏块、磨损、错误恢复
网络/显卡等设备	描述符、包、帧、buffer	DMA 引擎和设备内部队列	所有权、completion、IOMMU、cache 可见性

从程序角度看，`load x` 只是取一个变量；从硬件角度看，它可能命中寄存器旁路、命中 L1、命中 L2、命中共享 L3、访问 DRAM、触发页表遍历、发生缺页，甚至等待块设备把页面读回主存。CSAPP 强调的 *locality* 就是让常见访问停留在左侧短路径。硬件书不能只说“内存很快/很慢”，而要说明每一级的访问粒度、所有者、完成条件和失败路径。

专题：内存别名与同源不同名的 cache 问题 上一专题的层次表把“别名”列为 L1 cache 的关键问题之一，这里把它展开。TLB 的存在意味着同一个物理页可以被多个虚拟页映射：共享库、进程间共享内存、同一页在用户态和内核态各有别名，都是日常情形。这带来一个只在“用虚拟地址索引 cache”时才出现的问题：两个不同的虚拟地址指向同一物理行，若 cache 按虚拟地址的低位选槽，而两个别名的虚拟页号不同，index 就可能不同，同一物理行会被存进两个不同的槽。之后程序经别名一写入，经别名二读到的却是另一个槽里的旧副本——同一数据在 cache 里有了两份不一致的拷贝。

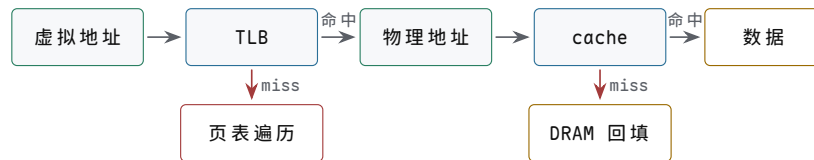
看一个小例子。页大小 4 KiB，cache 为 32 KiB 直接映射、64 B 行：offset 为 6 bit，行数 = $32\text{ KiB} \div 64\text{ B} = 512$ ，index 取地址的 bit 14..6 共 9 bit。页内偏移只有 12 bit (bit 11..0)，因此 index 的高 3 位 (bit 12..14) 来自虚拟页号——别名地址的差异正好在这几位里。设虚拟地址 0x1040 和 0x2040 都映射到物理地址 0x35040：前者的 index 是 65，后者的 index 是 129，同一物理行会被分别存进槽 65 和槽 129。经 0x1040 写入后，槽 65 是新值，槽 129 仍是旧值，两次读到同一物理地址却得到不同数据。

两种硬件解法基于同一条思路：让同一物理行的所有别名映到同一个槽，再由物理 tag 确认身份。第一种是物理索引 (PIPT)：index 直接取自物理地址，0x1040 和 0x2040 经 TLB 翻译后都是 0x35040，index 都是 321，必然同一个槽，tag 也比较物理地址，别名问题根本不会出现；代价是要先等 TLB 翻译完成才能开始查 cache，命中路径变长。第二种是限制 index 位数 (VIPT 的常见约定)：把 cache 设计成 index 加 offset 不超过页内偏移的 12 bit，例如 4 KiB 直接映射，或 32 KiB、8 路组相联——组数 = $32\text{ KiB} \div 64\text{ B} \div 8 = 64$ ，index 只需 6 bit，index 加 offset 正好 12 bit。此时 index 全部来自页内偏移，而任何别名的页内偏移都与物理地址相同：上例中两个别名与物理地址的 bit 11..6 都等于 1，必然映到同一组。查找与 TLB 翻译并行进行，拿到物理页号后再与组内各路物理 tag 比较，第二次访问命中同一个行，两份副本无从产生。现代处理器的 L1 大多采用这种“虚拟索引、物理 tag、index 不超出页内偏移”的安排，既保留并行查找的速度，又让别名退化为普通命中。把两种安排画成对照图，差别就看得很清楚。



图示：VIPT 别名问题与限制 index 位数的对照。左：32 KiB 直接映射 cache 的 index 取 bit 14..6，0x1040 与 0x2040 的 index 分别是 65 与 129，同一物理行 0x35040 被存进两个槽，经 0x1040 写入后槽 65 是新值、槽 129 仍是旧值。右：限制 index 加 offset 不超出页内偏移 12 bit (如 32 KiB、8 路组相联，index 为 bit 11..6)，两个别名的 index 都来自页内偏移，必然映到同一组，再由物理 tag 确认身份，双副本无从产生。

专题：cache 与 TLB 是两套不同的近路 cache 缓存数据或指令内容，TLB 缓存虚拟页到物理页的地址转换。二者经常同时出现在一次 LOAD 路径上，但回答的问题不同：TLB 问“这个虚拟地址能否翻译成哪个物理地址”；cache 问“这个物理位置的内容是否已经在近处”。把二者混在一起，会导致读者无法理解为什么 TLB miss 不等于 cache miss，页错误也不等于总线错误。



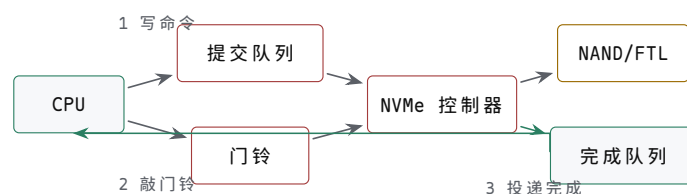
图示：TLB miss 发生在地址转换阶段，cache miss 发生在数据回填阶段。二者都可能让 LOAD 变慢，但失败位置不同。

事件	真实含义	后续动作
TLB hit + cache hit	地址转换和数据都在近处	LOAD 快速返回
TLB miss + 页表命中	转换不在 TLB，但页表有效	硬件或软件页表遍历后填 TLB
页不存在	页表没有有效映射	进入页错误异常，可能由 OS 分配或从磁盘读页
cache miss	地址有效，但数据不在当前 cache	从下一级 cache 或 DRAM 回填 cache line
权限错误	转换存在但访问类型不允许	报保护异常，不能提交数据

专题：SSD 和硬盘不是慢内存，而是块设备 硬盘和 SSD 都不能像 SRAM 那样把地址线接上就读一个字节。CPU 要通过控制器提交命令：读哪个逻辑块、读多长、放到哪个主存缓冲区、完成后写哪个 completion。硬盘的内部约束来自机械寻道和旋转；SSD 的内部约束来自 NAND flash 页读写、块擦除、FTL 映射、磨损均衡、垃圾回收和掉电保护。它们对 CPU 暴露的共同形态，是队列化块设备。

对象	内部结构重点	对系统可见的边界
机械硬盘	盘片、磁头、磁道、扇区、缓存	随机访问受寻道和旋转影响，顺序访问更友好
SATA SSD	NAND flash、FTL、通道、擦除块	写放大、垃圾回收、坏块管理、掉电保护
NVMe SSD	PCIe、多队列、doorbell、completion queue	并发队列深度高，CPU 通过 MMIO 和 DMA 协作
SD/eMMC	控制器隐藏 flash 细节，命令集较简单	嵌入式启动和存储，延迟波动明显

理解 SSD 时要避免两个误解。第一，SSD 没有机械寻道，不等于每次访问都一样快；内部垃圾回收、擦除、映射表缓存和热控都会让尾延迟上升。第二，逻辑块地址连续，不等于物理 NAND 页连续；FTL 会把逻辑写重映射到新位置，再在后台回收旧块。因此块设备的正确抽象不是“巨大数组”，而是“命令队列 + DMA 缓冲区 + completion + 错误恢复”。



图示：NVMe 的基本形态：CPU 写队列和门铃，设备 DMA 读写主存，completion 才说明请求完成。

专题：读硬件结构时要区分数据路径和控制路径 CPU、cache、DRAM、SSD、网卡和显卡都可以用同一个提问框架阅读：数据从哪里到哪里，控制由谁发起，完成由谁报告，错误由谁保存。低质量描述常把这些混成“设备把数据给 CPU”。更好的描

述应写出两条路径：数据可能经 DMA 在设备和主存之间移动；控制通常由 CPU 写 MMIO、设备写 completion、CPU 读状态或处理中断。

路径	典型方向	必须说明
控制路径	CPU 到 MMIO/doorbell, 设备到状态/中断	命令何时被接收, 完成如何通知
数据路径	设备 DMA 与主存缓冲区之间	缓冲区地址、长度、cache 可见性、权限
错误路径	设备或互连到错误码/异常	超时、校验、越界、权限和重试策略
恢复路径	驱动或固件到 reset/retry/rebuild	哪些状态保留, 哪些队列丢弃或重放

章末练习

任务	要求	预期结果
SRAM 读写周期	画出地址、CS、OE、WE、数据线和采样窗口	能指出谁驱动数据线, 谁采样
地址映射	设计 64 KiB ROM/RAM/GPIO/定时器映射	任意地址最多一个片选有效
等待状态	为慢 ROM 和慢 I/O 设计 READY/WAIT 规则	等待期间地址和方向稳定, 有超时策略
MMIO 寄存器	设计 GPIO 的 DIR/OUT/IN/STAT 语义	外部输入经过同步, 状态清除规则明确
中断顺序	写出设备 ready 到 CPU 服务程序返回的 trace	不丢事件, 不重复清旧事件
DMA 顺序	写出描述符、缓冲区、completion 和 cache 可见性边界	CPU 和设备所有权不重叠
valid/ready	用 valid/ready 描述一次读请求和读响应	能说明背压和 completion 的作用
cache 行	用 tag/index/offset 解释一次 cache 命中和一次 miss	能说明 cache line、valid、dirty 和替换边界
MMIO 属性	说明为什么设备寄存器通常不能按普通 cacheable 内存处理	能指出副作用、顺序和可见性要求
未映射访问	规定未映射地址的超时、错误或固定返回值	CPU 不会永久等待未知目标
存储层次图	画出寄存器、L1/L2/L3、DRAM、SSD/硬盘的数据粒度和完成语义	能区分 cache line、页、块和 completion
TLB/cache 区分	写出一次 LOAD 中 TLB hit/miss、页错误、cache miss 的不同路径	不把地址转换错误和数据回填混为一谈

块设备队列	为 NVMe 或简化 SSD 写出 submission、doorbell、DMA、completion 顺序	能说明 CPU 写门铃不等于 I/O 完成
DRAM 时序计算	给定 tRCD、tCL、时钟周期和突发长度，估算一次随机读时延，并与行命中情形对比	三段时延拆分正确，量级为几十 ns
UART 帧分析	给定 RX 波形和标称波特率，数出起始位、数据位、停止位，还原字节并核对波特率误差	帧结构和误差判断都有依据
组相联查找	用本章的四次访问序列，分别在直接映射和 2 路组相联下走一遍	能指出哪几次由 miss 变 hit 及原因
描述符环顺序	写出 CPU 提交两个描述符到设备完成的 SW/HW 指针、doorbell 与完成写回顺序	doorbell 不当作完成，写回可见性明确
bank 交错拍数	4 个 bank 按 cache line 轮转交错，单 bank 一次访问占用 4 拍，bank 间每拍可启动一个新请求；计算连续读 8 条相邻 cache line 的总拍数，并与单 bank 串行对比	交错 11 拍、单 bank 32 拍，能写出每行的启动与完成拍
替换策略	2 路组相联、LRU，某组初始为空，依次访问 A、B、A、C、A、B；逐步写出命中/失效和组内容，并与同 index 直接映射对比	2 路共 3 次 miss，直接映射 6 次全 miss
预取演算	顺序读 16 条 cache line，每次 miss 延迟 30 拍、每行处理 8 拍；计算无预取与 next-line 预取的总拍数，并求能完全隐藏预取的最小处理节拍	608 拍对 488 拍，处理节拍不小于 30 才能完全隐藏
I2C 事务序列	写出主机从 7 位地址 0x50 的从机寄存器 0x10 读 2 字节的完整序列：START、地址字节、读写位、ACK/NACK、repeated START、STOP	0xA0/0xA1 正确，最后一字节 NACK，repeated START 位置正确
仲裁对照	用一句话分别说明集中式仲裁器、菊花链、轮询计数的线数、优先级和公平性，并指出片上互连中仲裁的位置	三种做法的取舍清楚，仲裁做进交换节点
MESI 走查	两核初始均为 I，按“A 读 X、B 读 X、A 写 X、B 读 X”写出每步两核状态和互连消息	每步最新值位置唯一，失效与写回/转发正确

参考解答与要点

任务	参考解答	必须保留的要点
SRAM 周期	读时地址和片选稳定，OE 有效、WE 无效，由 SRAM 驱动数据；写时 OE 无效，窗口满足建立/保持由发起者驱动数据，WE 在地址和数据稳定后有效。	输出使能互斥，写时间。

地址映射	例如低半区 ROM，高半区中一块 RAM，最高 4 KiB 为 I/O；用高位地址产生互斥片选，未映射区返回错误或超时。	片选互斥和未映射行为必须写入规格。
等待状态	慢目标在 ready 之前保持事务未完成，控制器保持地址、片选和方向；超过最大等待则返回错误。	等待状态是硬件事务延长。
MMIO	GPIO 输出写入锁存器，输入来自同步后的外部引脚，状态位说明 ready/error/irq，清除规则避免新旧事件混淆。	MMIO 有副作用，不是普通变量。
中断	设备锁存事件并请求中断，中断控制器投递，CPU 进入入口，服务程序读状态、处理数据、按设备规则清事件，再发送 EOI 或恢复中断。	清除顺序决定是否丢事件。
DMA	CPU 准备描述符和缓冲区并保证对设备可见，设备搬运数据并投递 completion，CPU 在 completion 后按一致性规则读取缓冲区。	描述符可见、数据可见和缓冲区所有权必须分开。
valid/ready	只有 valid 和 ready 同时为 1 才传输；ready 为 0 时发送方保持 payload；读请求和读响应可隔多个周期。	背压不等于错误，completion 才是完成标志。
未映射访问	未映射区域可以返回固定值、产生错误或等待到超时，但规则必须固定且可观察；不能让总线悬空成为随机值。	错误路径也是硬件规格。
cache 行	先算 $\text{offset}=\log_2(\text{行大小})$ 、 $\text{index}=\log_2(\text{行数})$ 、tag= 剩余位；命中要求槽 valid 且 tag 相等；miss 时取整行填入并更新 tag/valid，dirty 位决定替换时是否要先写回下一级。	三个字段的分工和 valid/dirty 语义不能丢。
MMIO 属性	设备寄存器有副作用：写是命令、读可能清状态；若被 cache 缓冲或合并，设备可能永远看不到命令，CPU 也可能一直读到旧状态，所以设备内存要映射为不可缓存、不可合并、顺序保持。	副作用、顺序、可见性三条都要答到。
存储层次图	寄存器（字，当拍完成）、L1/L2/L3（cache line，命中即返回）、DRAM（行/页，控制器调度与刷新）、SSD/硬盘（块，I/O 请求和 completion）；越往下粒度越大，完成语义越依赖显式通知而不是当拍返回。	粒度与完成语义要对应，不能只画容量和速度。
TLB/cache 区分	TLB miss 发生在地址转换阶段：硬件查页表，缺页则走页错误异常；cache miss 发生在拿到物理地址之后：向下一级回填整行，CPU 只是停顿或乱序等待。页错误是控制流改道，cache miss 只是数据慢。	转换错误与数据回填是两条独立路径。

块设备队列	CPU 把描述符写进提交队列，写 doorbell 通知设备；设备 DMA 取走描述符、搬完数据后写完成队列，再（可选）发中断。doorbell 只表示“有新请求”，不代表 I/O 完成，完成以完成队列项为准。	submission 与 completion 分离，doorbell 不等于完成。
DRAM 时序	一次需要开行的随机读约为 t_{RCD} （开行）加 t_{CL} （列出数）加突发传输，常见 DDR3/DDR4 器件合计约 30~40 ns 量级；行命中可省去 t_{RCD} ，行冲突还要先 precharge。	时延由行、列、突发三段组成，不是固定常数。
UART 帧	先找起始位下降沿，再按位时间中点逐位采样：1 起始位、8 数据位（低位在前）、1 停止位；还原字节后核对实测位时间与标称波特率的偏差，双方每一侧误差一般不超过约 2%，收发合计留有约 5% 的理论上限。	位边界、采样点和误差都要核对。
组相联	index 选中整个组，组内两路 tag 并行与地址 tag 比较，任一路相等且 $V=1$ 即 hit；两个热地址各占一路后不再互相驱逐；miss 时按替换位选一路驱逐。	并行比较、mux 和替换位缺一不可。
描述符环	CPU 只推进 SW 指针，设备只推进 HW 指针，末尾回绕；doorbell 只提示有新描述符，不代表完成；完成以设备写回状态位或 completion 项为准，回收前确认写回可见。	指针分离、doorbell 非完成、写回可见性。
bank 交错	第 1..4 拍依次启动第 1..4 行（各占一个 bank），第 i 行在第 i 拍启动、第 $i+3$ 拍完成；第 5 行在第 5 拍回到 bank 1 时前一次访问刚结束，无需等待。第 8 行在第 11 拍完成，共 11 拍。单 bank 串行需 $8 \times 4 = 32$ 拍。	加速来自启动流水化，单 bank 本身没有变快。
替换策略	2 路 LRU：A miss 得 {A,-}；B miss 得 {A,B}；A hit（B 成为最久未用）；C miss 驱逐 B 得 {A,C}；A hit；B miss 驱逐 C 得 {A,B}，共 3 次 miss。直接映射下同 index 只有一个槽，每次访问都驱逐上一行，6 次全 miss。	LRU 驱逐“最久未用”，不是“最早进入”。
预取演算	无预取：每行 miss 30 拍加处理 8 拍， $16 \times 38 = 608$ 拍。next-line 预取：第 1 行第 30 拍就绪，处理第 i 行的 8 拍只能隐藏预取的一部分，第 i 行数据在第 $30i$ 拍就绪，稳态每行 30 拍，总 $30 \times 16 + 8 = 488$ 拍。处理节拍 $T \geq 30$ 时预取被完全隐藏，总拍数降为 $30 + 16T$ 。	预取隐藏延迟但不消除首次 miss；处理太慢时仍受 miss 间隔限制。
I2C 事务	START $\rightarrow$ 0xA0（0x50 左移一位、读写位 0 表示写） $\rightarrow$ 从机 ACK $\rightarrow$ 0x10（寄存器号） $\rightarrow$ ACK $\rightarrow$ repeated START $\rightarrow$ 0xA1（读写位 1 表示读） $\rightarrow$ ACK $\rightarrow$ 读第 1 字节 $\rightarrow$ 主机 ACK $\rightarrow$ 读第 2 字节 $\rightarrow$ 主机 NACK $\rightarrow$ STOP。	先写寄存器地址，repeated START 转读；最后字节 NACK 通知从机结束。

仲裁对照	集中式：每主设备一对请求/授权线，仲裁器并行比较，最快但线数为 $2N$ 且有单点；菊花链：授权逐级传递，线数与设备数无关，但优先级由位置固定、传播慢；轮询计数： $\lceil \log_2 N \rceil$ 根扫描线，从上次授权点继续则轮转公平，但平均扫描表最慢。片上互连把仲裁做进每个交换节点的端口。	三者在线数、延迟、公平性之间取舍；仲裁没有消失。
MESI 走查	步骤 1：A 由 I 变 S，B 保持 I（读入共享副本）；步骤 2：B 由 I 变 S，A 保持 S；步骤 3：A 广播失效，B 由 S 变 I，A 由 S 变 M，内存过期；步骤 4：B 读 miss，A 写回或转发后由 M 降为 S，B 得 S，内存恢复最新。	每步最新值唯一：S 期间在内存，M 期间在 A 的 cache。

经典芯片、启动与平台演进

前面已经把最小 CPU 和整机事务分开讲清楚。本章再回到真实芯片：6502、Z80、8051、8086、80386、80486、Pentium 和现代 SoC 不只是历史名称，而是硬件边界逐步扩展的样本。读这些芯片时，不应只背年份、位宽或指令集，而要问：状态保存在哪里，地址怎样形成，外部事务怎样发起，异常和等待怎样改变下一步，哪些功能被逐步搬进芯片内部。

芯片	本章观察重点	对整机理解的贡献
6502/Z80	8-bit CPU、外部地址/数据总线、控制周期	看见 CPU、存储器和 I/O 如何靠板级总线协作
8051	CPU、片内存储、I/O、定时器和中断集成	看见单片机把一部分整机控制收进芯片
8086/8088	16-bit x86、20-bit 地址、复用总线和预取队列	看见早期微处理器如何在引脚和地址空间之间取舍
80386	32-bit、保护模式、分页和异常机制	看见 CPU 如何承担系统级地址和权限约定
486/Pentium	片上 cache、流水线、浮点和预测方向	看见低时延来自片上集成和执行结构重组
现代 SoC	CPU、cache、内存控制器、I/O、安全启动同片集成	看见整机边界继续向片内移动

本章采用经典系统教材常用的标注方式：不把 CPU、总线、cache、I/O 当作孤立名词，而把它们写成可追踪的事务。阅读任何一颗芯片时，都应同时回答五个问题：状态保存在哪里，组合路径经过哪些部件，控制信号在什么时候有效，外部事务由谁驱动和谁采样，异常或等待如何改变下一步。



演进主线不是“位宽越来越大”这么简单，而是地址形成、等待隐藏、异常诊断和外设事务逐步进入芯片内部。

图示：经典芯片观察线索。每一代都把前一章的事务边界移动、扩宽或缓存。

一、早期微处理器：从 6502、Z80 到 8086

在 8086 之前，8-bit 微处理器已经把“CPU 加外部总线”这台戏完整演过一遍。6502、Z80 和 8051 各自回答了一个整机问题：寄存器太少怎么办，上下文切换和 DRAM 刷新如何省掉总线事务，整机部件能不能收进同一颗芯片。

6502 用极少的寄存器和零页换取简单的数据通路 6502 的编程模型只有累加器 A、变址寄存器 X 和 Y、栈指针 SP、程序计数器 PC 和标志寄存器 P，没有一组可自由选用的通用寄存器；整颗芯片约 3500–4500 个晶体管，数据通路围绕累加器组织，没有乘除法指令。寄存器这样少，工作数据就必须经常放在存储器里，6502

的对策是把地址空间最低的第 0 页 (0x0000-0x00FF) 当作“伪寄存器堆”使用：零页寻址的指令只有 2 个字节，3 个时钟拍完成一次读写，比访问任意 16-bit 地址更短更快。栈则被硬件固定在第 1 页 (0x0100-0x01FF)，SP 因此只需保存 8-bit 页内偏移。数组处理依靠 (zp),Y 间接变址：从零页的一对字节取出 16-bit 基地址，再加上 Y 作为元素偏移，一次读用 5 拍，相加后若跨页再加 1 拍。

指令形式	拍数	对应的总线动作
LDA 立即数	2	取操作码，再把操作数字节直接读入 A
LDA 零页	3	取操作码，取零页地址，从第 0 页读数据
LDA 绝对地址	4	取操作码，取 16-bit 地址的两个字节，再读数据
LDA (zp),Y	5+1	取操作码，从零页读基地址两字节，加 Y 后读数据，跨页加 1 拍
STA (zp),Y	6	取操作码，从零页读基地址两字节，加 Y 形成地址后写入，写比读多一拍
JMP 绝对地址	3	取操作码，取目标地址两个字节并装入 PC

每条指令的拍数固定而且很小，总线在每个时钟拍上完成一个确定动作，因此 6502 程序可以手工逐拍 trace（执行轨迹）：哪一拍取指、哪一拍读零页、哪一拍形成地址，全能数清楚。6502 的便宜正来自这种简单：没有乘除法，没有多通用寄存器，数据通路只围绕累加器，芯片面积和售价才可能压到最低。

6502 的总线周期是每拍一次的确定事务 拍数表能手工 trace，前提是一条更硬的硬件事实：6502 的地址总线在每个时钟拍上都有有一次确定动作——取操作码、取操作数、读零页、读数据或写数据，连内部运算不需要访问存储器的拍，也会做一次不改变结果的读。总线没有空闲拍，这是 6502 与后来把一条指令拆成若干段、段内还有空闲等待的设计最大的差别。一拍之内再分两相：时钟输入被整形为 ϕ_1 、 ϕ_2 两相不交叠时钟， ϕ_1 期间 CPU 把新地址和读写方向送上总线， ϕ_2 期间传输数据——读周期由存储器驱动数据线，CPU 在 ϕ_2 结束前采样；写周期由 CPU 驱动数据线，存储器在 ϕ_2 窗口内写入。

“存储器只要跟上每拍一事的速度”这句话可以算出来。以 1 MHz 主频为例，一拍 1000 ns， ϕ_2 高电平约占一半即 500 ns，再扣除数据建立时间约 100 ns，留给存储器的访问窗口约 400 ns 量级；当年的 2114 SRAM（约 250-450 ns）和 2716 EPROM（约 450 ns）正好在这个量级，不需要任何等待机制。而且访问速率均匀可预期——每拍一次、一拍不多一拍不少——显示电路甚至可以固定借用 ϕ_1 半拍读显存而不与 CPU 冲突，Apple II 一类机器正是这样共享存储器的。真正跟不上窗口的慢器件用 RDY 引脚解决：在 ϕ_1 期间把 RDY 拉低，CPU 就把当前这一拍延长，地址和读写控制保持不变，直到 RDY 释放。要注意 NMOS 6502 的 RDY 只对读周期有效，写周期照样按时完成；想让写周期也能暂停，得靠后来 65C02 的改进。

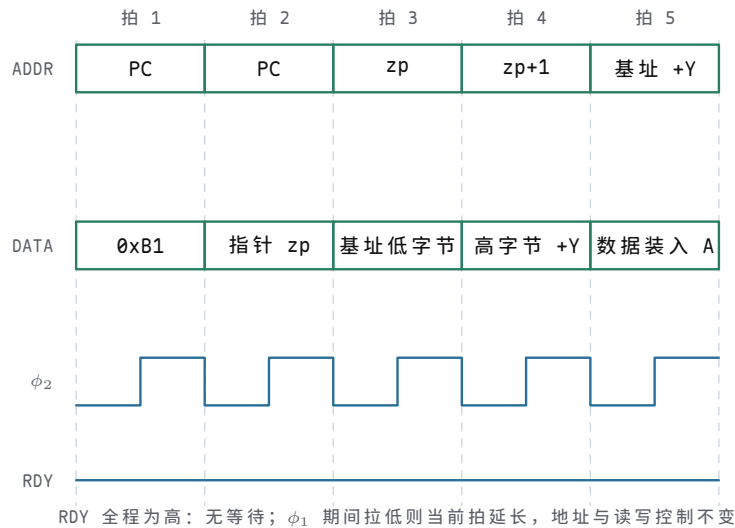
以前一张表的 LDA (zp),Y 为例，把 5 拍逐拍摊开，就能看到“每拍一事”的确切含义：

拍	地址总线	读/写	本拍完成的动作
1	PC	读	取操作码 0xB1，PC 加一
2	PC	读	取零页指针 zp，PC 加一
3	zp（第 0 页）	读	读出基地址低字节
4	zp+1（第 0 页）	读	读出基地址高字节，同时低字节与 Y 相加
5	基址 +Y	读	读出数据装入 A；若加 Y 无进位，指令到此结束

若低字节加 Y 产生进位，第 5 拍读到的是高字节尚未修正的旧地址数据，是一

次无效读；第 6 拍用修正后的高字节重读，共 6 拍——这就是上一张表里“5+1”的来源。把无效读也留在总线上而不是干脆停一拍，正说明 6502 的控制逻辑按“每拍必有总线动作”组织：多一个确定的读，比加一套判断和停顿的控制便宜。

把这五拍画成时序图，地址与数据的配合就更直观：每拍地址先送上总线，数据随后在 ϕ_2 窗口内传输，RDY 全程保持高电平，总线按每拍一次的节奏推进。



图示：LDA (zp), Y 的五拍总线时序：五拍全为读周期，每拍地址先送上总线，数据在 ϕ_2 窗口内由存储器驱动，CPU 在 ϕ_2 结束前采样。第 4 拍读基址高字节的同时，低字节与 Y 相加形成最终地址；RDY 全程为高，故没有任何等待拍。

280 用双寄存器组和取指刷新换取系统效率 280 在软件上与 8080 兼容，并在此之上做了几处扩展。最显眼的是寄存器组：主寄存器组 AF、BC、DE、HL 之外还有整套备用组 AF'、BC'、DE'、HL'，EX AF, AF' 与 EXX 各用一条指令即可完成现场交换。中断响应时不必把寄存器逐个压栈再逐个恢复，现场切换不发生任何存储器访问。第二个扩展与 DRAM 刷新有关：R 寄存器保存刷新行地址，在每个 M1 取指周期的后半段——此时指令字节已经读入、地址总线正好空闲——自动把 R 输出到地址总线上完成一次刷新，随后自动加一；刷新被“借”进取指周期，外部不再需要单独的刷新周期。此外，I 寄存器配合中断模式 2 提供向量页地址，IX、IY 变址寄存器扩展了寻址方式。

280 机制	硬件做法	换取的收益
主/备寄存器组	AF/BC/DE/HL 与 AF'/BC'/DE'/HL'，EX AF, AF' 交换 AF 一对，EXX 交换 BC/DE/HL 三对	中断现场切换不访问存储器
R 寄存器	M1 取指后半段输出刷新地址，随后自动加一	DRAM 刷新不占额外总线周期
I 寄存器与中断模式 2	I 提供向量页地址，设备提供页内偏移	中断入口可以成表组织
IX/IY 变址寄存器	变址基址加偏移形成有效地址	数据结构和栈帧访问更直接

280 的教学点有两条。第一，寄存器越多，上下文切换越少访问内存：一组备用寄存器换来的是中断响应里整段存储器往返的消失。第二，刷新是“借用已有事务”的设计：DRAM 刷新本来要占用专门的总线时间，280 把它塞进每个取指周期里必然出现的空闲半段，系统里就再也看不见它。

280 的指令时序按 M 周期和 T 状态两层计数 6502 用一层计数就够了：一条

指令几拍，每拍一次总线动作。Z80 用两层：一条指令先拆成若干机器周期（M 周期），每个 M 周期再含 3~6 个 T 状态。M1 永远是取指周期，固定 4 个 T 状态：T1 把 PC 送上地址总线，T2 用 MREQ、RD 读回操作码，T3、T4 把总线让给 R 寄存器做 DRAM 刷新，CPU 内部同时译码——上一段讲的“借取指周期刷新”，就发生在这两个 T 状态里。存储器读、写周期各 3 个 T 状态；I/O 读写周期固定插入一个等待，共 4 个 T 状态；慢器件还可以用 WAIT 引脚在 T2 与 T3 之间继续插入 Tw。

用 LD A,(HL) 走一遍：它含 2 个 M 周期、共 7 个 T 状态——M1 取指 4T，M2 按 HL 指出的地址读存储器 3T。LD (HL),A 同样是 7T，差别只在 M2 的方向：CPU 在 T2 驱动数据线、WR 有效，存储器在写窗口内采样。按 4 MHz 主频折算，一个 T 状态 250 ns，7T 共 1.75 μ s。

T 状态	所属 M 周期	总线动作（以 LD A,(HL) 为例）
T1	M1	PC 送上地址总线
T2	M1	MREQ、RD 有效，读回操作码
T3	M1	输出 R 寄存器刷新地址，CPU 内部开始译码
T4	M1	刷新收尾，总线不传送指令数据
T5	M2	HL 送上地址总线
T6	M2	MREQ、RD 有效，存储器把数据放上线
T7	M2	CPU 采样数据装入 A，指令结束

把这 7 个 T 状态按时间轴排开，M 周期与 T 状态的两层关系、以及哪些 T 状态真正在传送数据，一眼就能看清。



带底纹的 T2、T6 是总线真正传送指令或数据的时段；T3、T4 让给了刷新和内部译码

图示：LD A,(HL) 的 M 周期结构：M1 取指固定 4 个 T 状态，其中 T3、T4 把地址总线让给 R 寄存器做 DRAM 刷新，CPU 内部同时译码；M2 存储器读 3 个 T 状态。7 个 T 状态里只有 T2、T6 真正在总线上传送指令或数据；按 4 MHz 折算每 T 250 ns，共 1.75 μ s。

与 6502 对照时要注意数法不同。Z80 这 7 个 T 状态里，真正在传送指令或数据的只有 T2 和 T6 两段，T3、T4 让给了刷新和内部译码；6502 的 5 拍则每拍都在传数据。两种数法没有对错，但比较芯片时不能拿“7”和“5”直接相减：Z80 把刷新和内部事务显式编进时序，6502 把“每拍必做总线动作”做到底，省掉了这些状态。读 Z80 手册的时序图，要同时带 M 周期和 T 状态两把尺子；读 6502 手册，一把就够。

8051 把一部分整机收进同一颗芯片 8051 是单片机：CPU、4~8 KiB ROM/EPROM、128~256 B 片内 RAM、4 个 8-bit 端口、2~3 个定时器、UART 和中断控制器全部在同一颗芯片内。片内 RAM 分区使用：最低 32 字节是 4 组工作寄存器 R0~R7，由程序状态字 PSW 的 RS0、RS1 两位选组，一条改选组的指令就完成一次快速上下文切换；0x20~0x2F 的 16 字节是位寻址区，其中 128 个 bit 可以单独寻址、置位和清零，标志位不必占用整个字节；其余部分是通用区。片内外设寄存器称为特殊功能寄存器（SFR），占用 0x80~0xFF 的直接地址。四个端口采用准双向结构：引脚锁存器先写入 1，该引脚才能作为输入被读取。

8051 的教学点直接连着第五章：把 ROM、RAM、地址译码、MMIO、定时器、串口和中断控制器收进一颗芯片之后，“总线”变成了片内连线，板级事务缩成片内信号，但语义没有改变——写 SFR 仍然是写设备寄存器，读端口仍然是 MMIO 读，定时器溢出仍然是一个中断事件。整机边界在这里第一次明显移动，而读者已经学过的所有约定原样保留。

8086 是理解早期 x86 体系的重要起点。它是 16-bit 微处理器，拥有 16-bit 通用寄存器和 16-bit 数据通路特征，同时通过 20-bit 地址能力访问最多 1 MiB 物理地址空间。它的历史意义不只在指令集，还在于展示了 CPU 芯片如何通过外部总线、地址锁存、控制信号和存储器/外设共同构成一台机器。

8086 的寄存器组包括 AX、BX、CX、DX、SP、BP、SI、DI，以及 CS、DS、SS、ES 等段寄存器。段寄存器和偏移地址共同形成物理地址：

```
physical_address = segment * 16 + offset
```

这个规则反映了一个时代的硬件取舍：内部仍以 16-bit 编程模型为主，但外部需要访问超过 64 KiB 的地址空间，于是通过段基址左移和偏移相加得到 20-bit 地址。分段不是高级抽象，而是地址形成硬件的一部分。

8086 的外部总线也值得注意。地址线 and 数据线存在复用，某些引脚在总线周期的不同阶段承担不同含义。硬件需要地址锁存器在地址有效阶段保存地址，随后这些引脚可以用于数据传输。一次总线访问不是“读内存”三个字，而是分阶段发生的事务：输出地址、锁存地址、给出读写控制、等待存储器或 I/O 响应、传输数据、结束总线周期。

8086 观察点	硬件含义
16-bit 寄存器	数据运算和编程模型以 16-bit 为核心
20-bit 地址形成	通过段寄存器和偏移访问 1 MiB 物理空间
地址/数据复用总线	芯片引脚有限，外部需要锁存和分阶段传输
存储器与 I/O 周期	访问目标由控制信号和地址空间规则区分
BIU/EU 分工	取指队列和执行部件可部分重叠工作

8086 的典型板级连接还需要外部支持器件。地址/数据复用引脚在 T1 阶段输出地址，在后续阶段承载数据，因此常用地址锁存器保存低位地址；时钟、复位和 ready 同步可由时钟发生器组织；最大模式下，总线控制信号还可由总线控制器从状态信号译出。读数据手册时，不应只看“8086 是 16-bit CPU”，还要看它把哪些系统责任交给了板级芯片。

信号或器件	硬件作用	关键问题
ALE	地址锁存允许，提示外部锁存低位地址	锁存器是否在地址有效窗口保存地址
AD0-AD15	复用地址/数据线	哪个阶段由 CPU 驱动，哪个阶段由存储器或 I/O 驱动
A16-A19/S3-S6	高位地址与状态复用	地址是否在访问开始阶段被正确保存
RD/WR	读写控制	目标芯片输出使能和写使能是否互斥
M/IO	区分存储器周期和 I/O 周期	地址译码是否把事务送到正确目标
READY	外部目标请求插入等待状态	慢存储器是否能可靠延长总线周期
INTR/NMI	可屏蔽和不可屏蔽中断入口	请求是否同步，响应和清除顺序是否明确
8284/8288	时钟/复位/ready 或总线控制辅助	外部控制信号是否满足时序和极性要求

读总线周期要按 T 状态逐段读 上一张信号表说明了每个引脚负责什么，但还不够：同一个总线周期里，同一组引脚在不同时间段做不同的事。8086 把一次总线访问切成 T1、T2、T3、T4 四个状态，必要时在 T3 与 T4 之间插入等待状态 Tw。

读手册的正确方式是逐状态确认：这一拍谁在驱动地址/数据引脚，哪个控制信号有效，外部芯片应在哪个边沿锁存或采样。

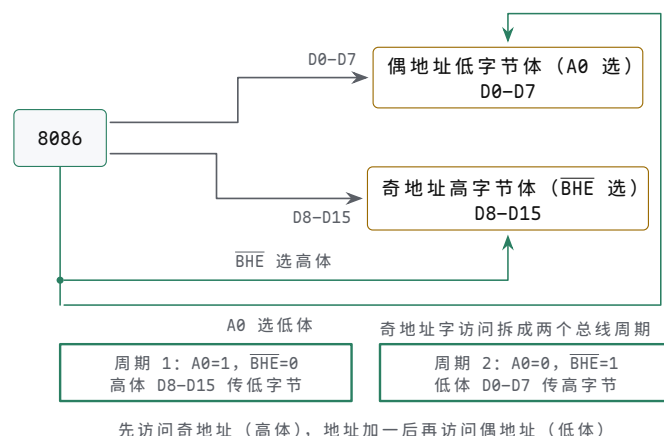
T 状态	地址/数据引脚	控制信号	外部动作
T1	CPU 在 AD0-AD15 和 A16-A19/S3-S6 上输出地址	ALE 拉高	地址锁存器在 ALE 下降沿保存地址
T2	地址从复用脚撤出，引脚准备转入数据方向	RD（读）或 WR（写）有效	数据收发器按读/写确定传输方向
T3	读周期中由目标芯片驱动数据线	控制保持，READY 为低则插入 Tw	存储器或 I/O 把数据放到总线上，等待状态期间地址和控制保持不变
T4	CPU 采样读入的数据，或结束写周期	控制信号撤销	目标释放数据线，总线周期结束

写周期与读周期骨架相同，差别在数据方向：CPU 从 T2 起亲自驱动数据线，并保持到写窗口结束，目标芯片在写控制有效的窗口内采样数据。“读总线周期”读到最后，就是把每个 T 状态里谁驱动什么、谁采样什么写清楚；有了这张表，上一张信号表里的每个信号才有了时间轴上的位置。

8086 的存储体选择是理解 16-bit 总线的关键 8086 有 16-bit 数据总线，但外部存储器常按 8-bit 芯片组织成偶地址低字节存储体和奇地址高字节存储体。地址最低位 A0 与高字节使能 $\overline{\text{BHE}}$ 共同决定哪些字节参与本次传输。这样，16-bit 字访问在偶地址上可以一次完成；若字从奇地址开始，通常需要拆成两个总线周期。

A0	$\overline{\text{BHE}}$	参与字节	典型含义
0	0	D0-D7 与 D8-D15	偶地址 16-bit 字访问
0	1	D0-D7	偶地址低字节访问
1	0	D8-D15	奇地址字节访问
1	1	无有效字节	正常访问中不应作为有效传输

这张表能解释为什么“16-bit CPU”不等于每次总线访问都简单搬 16 bit。存储芯片、地址译码器和数据收发器必须理解字节车道。若把两个 8-bit 存储体简单并在一起而不处理 A0 和 $\overline{\text{BHE}}$ ，字节写入会破坏相邻数据，奇地址字访问也无法正确完成。两个存储体与 A0、 $\overline{\text{BHE}}$ 的连接，以及奇地址字的两个总线周期，如图所示。



图示：8086 的字节存储体：偶地址字节在低体，接 D0-D7，由 A0=0 选中；奇地址字节在高体，接 D8-D15，由 $\overline{\text{BHE}}$ =0 选中。偶地址字访问两体同时选中，一个总线周期完成；奇地址字访问拆成两个周期：先 A0=1、 $\overline{\text{BHE}}$ =0 经高体传字的低字节，地址加一后 A0=0、 $\overline{\text{BHE}}$ =1 经低体传字的高字节。

最小模式、最大模式和外部控制器 8086 可以在最小模式下直接输出较多总线控制信号，也可以在最大模式下把状态信息交给外部总线控制器生成控制信号。前者适合单处理器小系统，后者便于接入更复杂的总线环境。这个差别说明，CPU 引脚不是固定地“表示某个概念”，它们会随系统模式承担不同职责。

系统形态	典型外部对象	学习重点
最小模式	地址锁存器、数据收发器、ROM/RAM、简单 I/O	CPU 直接给出读写、方向、使能等控制
最大模式	8288 总线控制器、总线仲裁、协处理或多主设备	状态信号被外部控制器译成总线命令
8088 小系统	8-bit 外部数据总线、较低成本板级存储器	编程模型相近，外部总线带宽和周期不同

一个 8086 存储器读周期可以按 T 状态走读。T1 中，CPU 把地址放到复用总线并拉出 ALE，外部锁存器保存地址；T2 中，CPU 改变总线方向并给出读控制；T3/Tw 中，存储器或 I/O 设备把数据放到数据线上，若 READY 不满足，CPU 插入等待状态；T4 中，CPU 采样数据并结束周期。这个过程把第一章的三态总线、第二章的时钟边界和第三章的控制 trace（执行轨迹）合到一起：总线上的每一段时间都必须有唯一驱动者和明确采样者。

8086 memory read sketch:

T1: address valid, ALE latches address
 T2: RD active, bus turns around for target data
 T3: target drives data, READY may insert wait states
 T4: CPU samples data, control deasserts



复用总线不是一根线同时表示所有含义，而是在同一个读周期内按 T 状态依次承担地址、方向、数据和完成边界。

图示：8086 存储器读周期。地址锁存、总线转向、等待和采样必须分阶段观察。

8086 写周期、中断响应和总线让出也要按事务读 读周期只说明目标驱动数据线；写周期则相反，CPU 在合适阶段驱动数据线，外部存储器或 I/O 设备在写控制有效窗口内采样。若数据收发器方向或写使能时序错误，写入可能丢失，也可能造成 CPU 和外设同时驱动总线。8086 的中断响应周期又是另一类事务：CPU 不访问普通内存数据，而是响应中断控制器，取得中断类型或进入规定入口。

事务	总线动作	观察重点
存储器写	地址先被锁存，CPU 驱动数据，WR 有效，目标采样	数据方向、字节存储体和写窗口
I/O 写	地址或端口号选择外设寄存器，CPU 驱动命令或数据	M/I/O 或 I/O 周期控制、端口译码
中断响应	CPU 接受 INTR 后执行响应周期，中断控制器给出类型或向量信息	请求保持、优先级、响应和屏蔽状态
DMA 让总线	DMA 请求总线，CPU 完成当前边界后让出，总线主控权转移	HOLD/HLDA 类握手、三态释放、仲裁

8086 memory write sketch:

```

T1: address valid, ALE latches address
T2: CPU drives write data, WR becomes active
T3: target samples data, READY may insert wait states
T4: WR deasserts, bus returns to idle or next cycle

```

这组事务可以直接指导早期微机板级排错。若读 ROM 正常、写 RAM 失败，应重点查数据收发器方向、WR 极性、字节存储体选择和 SRAM 写入时间；若外设中断不进入，应同时查外设请求、8259 屏蔽位、CPU 中断允许状态和中断响应周期；若 DMA 后内存内容异常，应查总线让出时 CPU 是否真正三态、DMA 地址计数是否正确、目标存储器是否满足时序。

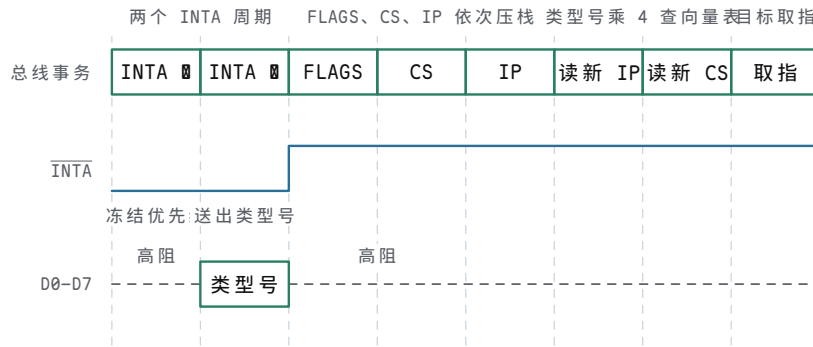
一次 INTR 从请求到进入服务程序要走完整条硬件链 前面把中断响应当作“另一类总线事务”提了一句，这里把链条走完。8086 的可屏蔽中断从一根 INTR 引脚开始：外设请求经 8259A 中断控制器判优、查屏蔽后拉高 INTR；CPU 在每条指令结束的边界采样 INTR，FLAGS 中的 IF 位为 0 就继续顺序执行，为 1 才响应。响应不是直接跳走，而是先跑两个背靠背的 INTA 总线周期：第一个 INTA 通知 8259A 冻结优先级状态，第二个 INTA 由 8259A 把 8-bit 中断类型号驱动到数据总线低 8 位，CPU 读入。最大模式下，CPU 还会在两个 INTA 之间保持总线锁定，防止仲裁器在响应中途把总线让给别的主设备。

拿到类型号之后的动作全由硬件完成：类型号乘 4 得到中断向量表内偏移—向量表固定在物理地址 `0x00000-0x003FF`，共 256 项、每项 4 字节（偏移 2 字节加段基址 2 字节），恰好 1 KiB。CPU 先把 FLAGS 压栈并清 IF、TF，再依次压栈 CS、IP，然后从表项读出新 IP 和新 CS，转向服务程序第一条指令；IRET 逆序弹出 IP、CS、FLAGS，现场复原。整条链的开销也能数出来：2 个 INTA 总线周期、3 次压栈写、2 次向量表读，共 7 个总线周期，每个至少 4 个 T 状态，合计 28 个 T 状态往上一手册把一次 INTR 的完整响应记为 61 个时钟拍，多出来的部分是 CPU 内部的微码步。

步骤	硬件动作	关键问题
1	外设拉高请求，8259A 判优、查屏蔽后拉高 INTR	请求要一直保持到被响应
2	CPU 在指令边界采样 INTR，IF 为 0 则不理睬	采样点只在指令之间，不在指令中间
3	第一个 INTA 总线周期：8259A 冻结优先级状态	期间不响应新的同级请求
4	第二个 INTA 总线周期：8259A 送出 8-bit 类型号	类型号由控制器给出，不是 CPU 推算的
5	FLAGS 压栈，清 IF、TF，CS、IP 依次压栈	清 IF 后同级 INTR 被屏蔽，防止嵌套失控
6	类型号乘 4，从 <code>0x00000</code> 起读新 IP、新 CS	表项错位 1 字节就会跳到错误入口
7	从新 CS:IP 取指，进入服务程序	预取队列作废重填，计入响应代价

IF 是 INTR 的总开关：CLI 清 IF，STI 置 IF，且 STI 的效果要推迟一条指令才生效—这样 STI 紧跟 IRET 的收尾序列不会在中途再被打断。NMI 则完全不受 IF 控制，不经过 INTA 周期，类型号由 CPU 内部固定为 2，边沿触发，留给掉电告警、存储器奇偶错这类“不许被软件关掉”的事件。把三者并排看就清楚：INTR 是可商量的外部请求，NMI 是不可拒绝的硬件告警，软件 INT 是程序主动调用同一套向量机制—它们共用同一张向量表和同一套压栈序列，差别只在入口从哪里来。

把这条链上的总线事务按时间排开，两个 INTA 周期、压栈写和查表读的先后次序、以及各步数据的来源，就更清楚。



图示：INTR 响应的总线事务序列：两个背靠背的 INTA 周期——第一个通知 8259A 冻结优先级状态，第二个由 8259A 把 8-bit 类型号驱动到 D0-D7，CPU 读入；随后 FLAGS、CS、IP 依次压栈，再按类型号乘 4 从 0x00000 起的向量表读出新 IP、新 CS。连同目标取指共 7 个总线周期、28 个 T 状态以上。

8086 内部常被划分为总线接口单元和执行单元。总线接口单元负责取指、形成物理地址、访问外部总线，并维护预取队列；执行单元负责译码和执行指令。这个分工说明，即使没有现代深流水，CPU 内部也已经在尝试把“取指访问外部世界”和“执行内部运算”分开，以减少外部存储访问延迟对执行的影响。

从本书前面建立的硬件模型看，8086 可以视为一个更复杂的同步系统：PC 在 x86 语境中体现为指令指针和代码段组合；地址形成部件把段和偏移送入加法路径；总线接口把地址和控制信号变成外部事务；执行部件译码指令并驱动寄存器、ALU 和标志位。8086 不是脱离前面章节的新对象，而是前述部件在真实芯片中的组合。

8086 还说明“低时延”在早期处理器中已经是系统问题。预取队列试图在执行部件工作时提前从外部总线取回后续指令，减少执行部件等待取指的时间。这个机制不等于现代流水线和分支预测，但思想相通：把慢的外部访问与内部执行尽量重叠。若分支改变控制流，预取队列中已经取得的顺序指令可能失效；若外部总线慢于指令消费速度，执行部件仍会等待。这里可以看见第一章快慢路径表的早期版本。

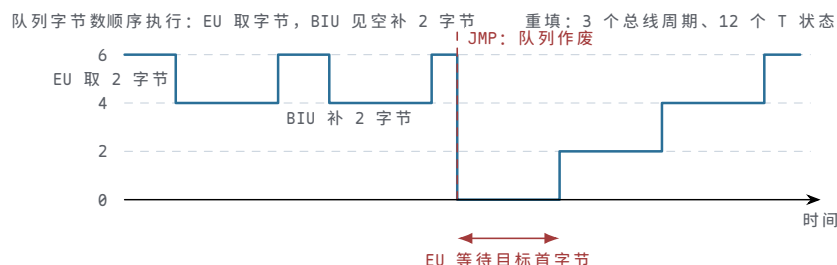
预取队列把取指和执行第一次重叠起来 8086 内部的总线接口单元（BIU）和执行单元（EU）各管一摊：EU 译码执行，BIU 管段地址加法、外部总线事务，还维护一个 6 字节的指令预取队列。规则很朴素：EU 从队列头取指令字节；只要队列空出至少 2 个字节且总线空闲，BIU 就主动发起一次 16-bit 取指总线周期，把顺序的后续两个字节补进队列尾。顺序执行时，取指被藏进 EU 的执行时间里——EU 执行一条多拍指令（比如乘法或带存储器操作数的指令）期间，BIU 已经把它后面的指令取好，EU 拿下一字节几乎不等待。这就是“执行与取指重叠”，是后来流水线最直接的雏形：没有预测，没有译码并行，只是把“去外面取指令”这件慢事和“在里面算”这件快事在时间轴上错开。

代价集中在控制流改变时。JMP、CALL、RET、中断都会使队列里已预取的顺序字节作废：BIU 清空队列，从目标地址重新预取，EU 在第一个目标字节到手之前只能等待。这笔账可以算：队列 6 字节，一次对齐的 16-bit 取指总线周期补 2 字节、占 4 个 T 状态，把队列补满要 3 个总线周期即 12 个 T 状态；按 PC/XT 的 4.77 MHz 折算约 2.5 μ s，比不少顺序指令本身还长。所以 8086 热路径上紧挨着的跳转有真实的硬件成本，这不是软件风格问题，而是队列重填时间。

观察点	顺序执行	跳转之后
队列内容	始终是当前 IP 之后的顺序字节	全部作废，从目标地址重填
BIU 总线动作	总线空闲即补 2 字节，队列维持接近满	先取目标首字节，再连续预取补满
EU 等待	几乎不等待，取指藏在执行中	等到第一个目标字节到手才能继续
重填代价	无	6 字节约 3 个总线周期、12 个 T 状态

8088 把同一机制缩到 4 字节队列、每次只预取 1 字节，因为它的外部数据总线只有 8 bit；这也意味着 8088 的队列更容易见底，同样主频下更怕跳转。这条 6 字节队列留下的教训一直有效：取指带宽、译码带宽和执行带宽要配平，任何一端长期供不上，另一端就会空等。第四章讲的流水线和分支预测，正是把这个队列做的事继续加深、加预测、加冲刷管理。

把队列中的字节数画成随时间变化的水位线，顺序执行时的补充节奏与跳转后的重填代价可以直接对照。



图示：8086 预取队列的水位变化：顺序执行时 EU 从队头取字节，队列空出至少 2 字节且总线空闲，BIU 就发起一次 16-bit 取指补进 2 字节，水位维持在接近满；跳转到来时已预取的顺序字节全部作废，从目标地址重填 6 字节要 3 个总线周期、12 个 T 状态（4.77 MHz 下约 2.5 μ s），EU 在第一个目标字节到手之前只能等待。

围绕 8086/8088 的经典外围芯片同样值得提前认识。8255 可编程并行接口把若干引脚组织成可配置 I/O 口；8253/8254 定时器提供周期性计数和中断来源；8259 中断控制器把多个外设请求汇总给 CPU；8237 DMA 控制器让设备和内存之间进行批量搬运；8250/16450/16550 UART 把串行线上的位流转换为 CPU 可读写的寄存器和 FIFO。它们共同说明，整机不是一颗 CPU，而是一组围绕地址、数据、控制和完成事件协作的芯片。

经典芯片	解决的问题	与本章主题的连接
8255 PPI	把并行引脚配置成输入、输出或握手端口	GPIO 和设备寄存器的早期形态
8253/8254 PIT	用计数器产生周期、延时和中断节拍	定时器寄存器与完成事件
8259 PIC	汇总、屏蔽和优先级排序中断请求	中断控制器和 CPU 入口
8237 DMA	在设备和内存之间搬运数据	总线主设备、仲裁和 cache 可见性
8250/16550 UART	串行收发、状态位和 FIFO	设备状态、数据寄存器和中断通知

8088/8086 最小系统可以按模块逐步实现 若要把 8086 类系统搭成现实电路，不应从“跑 DOS”开始，而应从最小可运行系统开始。8088 常用于教学，因为外部数据总线为 8 bit，存储器接口更容易搭建；8086 则保留 16-bit 外部数据总线，需要同时处理奇偶字节存储体。无论选择哪一种，系统都要包括时钟/复位、地址锁存、数据收发、ROM、RAM、I/O 译码和至少一个可观察外设。

模块	典型器件或做法	第一轮检查
时钟/复位	8284 或等价时钟复位电路	复位保持期间写控制无效，释放后第一拍地址可预测
地址锁存	74LS373/8282 锁存 AD 线上低位地址	ALE 有效窗口内保存地址，数据阶段地址不丢失

数据收发	74LS245/8286 控制数据方向	读周期外设驱动，写周期 CPU 驱动，不发生总线争用
启动 ROM	EPROM/Flash 映射到复位入口	复位入口读出跳转或初始化代码
SRAM	静态 RAM 与字节使能/写使能连接	写一个地址后再读回同一数据，邻近字节不变
I/O 译码	74LS138、比较器或 PAL/GAL	8255、8253、8259、UART 等端口互不重叠
可观察外设	LED 锁存器、按键、串口或逻辑分析仪探针	CPU 写端口后能看到确定副作用

这样的系统可以分三步 bring-up。第一步只让 CPU 从 ROM 取指，确认复位入口、地址锁存和读周期成立；第二步加入 SRAM，执行一个写后读回的小程序，确认写周期、数据方向和等待状态成立；第三步加入 8255 或简单 LED 锁存器，确认 I/O 周期、端口译码和外设副作用成立。每一步都应记录地址线、ALE、读写控制、数据总线方向和目标片选。若第一步没有通过，后续软件调试没有意义。

等待状态发生器是慢器件和 CPU 之间的约定 8086/8088 的 READY 信号把固定周期总线变成可延长事务。慢 ROM、慢 SRAM、外设端口或板级译码较深时，目标可以通过 READY 让 CPU 插入等待状态。等待状态不是“让程序慢一点”的软件概念，而是总线周期中的硬件暂停：地址、控制和方向必须保持可解释，目标在满足访问时间后再允许 CPU 采样或结束写周期。

对象	READY/等待的作用	关键问题
慢 ROM	延长取指或读常量周期	等待期间地址和片选是否保持稳定
慢 SRAM	延长数据读写窗口	写周期中数据是否覆盖完整写入窗口
I/O 外设	给外设状态机足够时间返回数据	外设未 ready 时 CPU 是否错误采样浮空总线
DMA/仲裁	让总线所有权变化落在安全边界	CPU 是否在让总线前完成当前事务

一个简单等待状态发生器可以由地址译码、访问类型和计数器组成：快 SRAM 区域不插等待，慢 ROM 区域插入一个等待，外设区域插入两个等待或等待外设 ACK。正式规格应说明等待的最大长度和超时行为。否则一个坏外设可能永久拉住 CPU，使整机看起来像死机。

PC/XT 到 PC/AT 是早期整机平台案例 8088、8086、80286 这些处理器进入实际计算机后，通常不会单独出现。以 PC/XT、PC/AT 风格的平台为例，CPU 周围有启动 ROM、DRAM、DMA 控制器、中断控制器、定时器、键盘控制、串口、并口、显示适配器和扩展总线。它们共同定义了“软件看到的机器”。因此，理解这类平台时，不能只问 CPU 支持哪些指令，还要问端口地址怎样分配、中断线怎样汇总、DMA 通道怎样仲裁、启动 ROM 映射在哪里、扩展卡如何把自己的寄存器和 ROM 挂进系统。

平台对象	硬件职责	预期行为
BIOS ROM	复位后提供第一段固件，初始化基本设备	复位入口命中 ROM，高地址别名或跳转正确
8259A PIC	汇总外设中断并向 CPU 投递	屏蔽位、优先级、EOI 和中断向量正确
8253/8254 PIT	提供周期节拍和计数事件	计数输入、模式字、输出和中断请求正确

8237A DMA	在设备和内存之间搬运块数据	通道、页寄存器、地址计数和终端计数正确
键盘控制器	接收键盘扫描码，也常参与复位/门控控制	输入缓冲、输出缓冲和命令应答顺序正确
UART/并口	提供串行或并行外设连接	状态位、FIFO 或握手线能解释数据何时可收发
扩展总线	允许插卡响应 I/O、内存窗口或中断线	片选互斥、总线方向、中断线和 DMA 请求不冲突

这个案例把本章许多概念放到同一台机器里。BIOS ROM 是启动约定；8259 是中断入口；8254 是定时事件；8237 是 DMA 总线主设备；UART 是状态寄存器与数据寄存器；扩展总线是地址译码、三态和仲裁的板级版本。若一台 PC/AT 风格机器无法启动，应按硬件路径排查：复位后 CPU 是否访问 BIOS 入口，ROM 是否驱动数据总线，时钟和 READY 是否满足，随后才检查键盘、显示、磁盘或操作系统。

端口地址表是平台规格的一部分 早期 x86 平台大量使用独立 I/O 空间。设备寄存器不一定映射在普通内存地址中，而是通过端口号和 I/O 周期访问。平台必须写清楚哪些端口归哪个设备，哪些中断线归哪个设备，哪些 DMA 通道归哪个设备。否则，即使 CPU 指令正确，两个设备也可能响应同一个端口，或一个中断请求被错误解释成另一个设备事件。

规格项	说明内容	典型错误
I/O 端口	控制、状态、数据寄存器的端口号和读写语义	端口译码过宽，多个设备同时响应
中断线	哪个设备接到哪条 IRQ，是否可屏蔽和共享	设备已请求但 PIC 屏蔽，或 EOI 顺序错误
DMA 通道	通道号、方向、地址和长度寄存器	8-bit/16-bit 通道混用或页寄存器错误
内存窗口	显存、Option ROM、缓冲区分或 MMIO 区域	RAM、ROM 和设备窗口重叠
启动顺序	固件先初始化哪些控制器，再枚举哪些设备	设备尚未复位稳定就被访问

把端口地址表写成规格，有助于理解“平台兼容性”。处理器可能更换为更快的 80286、80386 或 80486，但许多 I/O 端口、中断和启动约定仍然保留，使旧软件能够找到键盘、定时器、串口和磁盘控制器。硬件的发展不是每一代从零开始，而是在保留平台约定的同时，把部分功能移入更高集成度的芯片组。

80286 是 8086 与 80386 之间承前启后的过渡 80286 在实模式下就是一颗更快的 8086：同样的段基址左移 4 位加偏移，已有软件原样运行。它的新东西在保护模式里：地址线扩到 24 位，物理空间从 1 MiB 扩到 16 MiB (2^{24} B)；段寄存器不再是段基址，而是“选择子”——高 13 位是描述符表内索引，TI 位选 GDT 还是 LDT，低 2 位是请求特权级。描述符表的每个表项 8 字节，装着 24 位段基址、16 位段限长和访问权字节；CPU 访问存储器时先按选择子查出描述符，再用描述符里的基址加偏移得到物理地址，限长和访问权同时参与检查。GDT、LDT、IDT 三张表的基址和限长分别放在 GDTR、LDTR、IDTR 里，用 LGDT、LIDT 装载——这套“表在内存、指针在 CPU”的约定被 80386 原样继承，本章后面会反复见到。

两个边界数字值得记住。选择子的索引是 13 位，一张描述符表最多 $2^{13} = 8192$ 项，每项 8 字节，GDT 最大 64 KiB；描述符的限长是 16 位，一个段最大仍是 64 KiB——286 虽然摸得到 16 MiB 物理内存，大对象还是得切成段来管理。

286 新机制	硬件做法	留下的问题
---------	------	-------

24 位地址线	物理空间 16 MiB	段内仍是 16 位偏移，大对象要切段
选择子	段寄存器指向描述符，不再直接是基址	实模式软件把段寄存器当基址用，两种解释不兼容
描述符	8 字节：24 位基址、16 位限长、访问权	限长 16 位，段最大 64 KiB
GDTR、LDTR、IDTR	各存表基址加限长，LGDT/LIDT 装载	约定在此确立，表项格式到 386 还要扩
保护模式入口	MSW 的 PE 位置 1 即进入	LMSW 不能把 PE 清回，退不出
无分页	地址转换只有段这一层	按页换出的虚存要等 80386

286 保护模式用得少，原因正是表末两行。一是回程困难：286 没有提供把 PE 位清回去的手段，从保护模式回实模式只能靠硬件复位—PC/AT 的取巧做法是让键盘控制器拉复位线，BIOS 按事先约定的关机码恢复现场，慢而且别扭。二是没有分页：保护模式解决了越界检查和特权级，却没解决按页换出，而 DOS 生态的软件全按实模式写死，真正用 286 保护模式的只有 OS/2 1.x、XENIX 等少数系统。但它的历史位置不能跳过：GDTR/IDTR 的装载方式、选择子加描述符这层间接、“段寄存器不再是地址而是句柄”的观念转变，都是 80386 及其后所有 x86 保护模式的直接先例。下一节读 386 时，描述符表原样保留，只是基址扩到 32 位、访问权更细，再叠上分页这一层。

二、80386：32-bit、保护模式与地址转换

80386 把 x86 推入 32-bit 时代。它扩大了通用寄存器宽度，引入 EAX、EBX、ECX、EDX、ESP、EBP、ESI、EDI 等 32-bit 形式；地址和数据处理能力显著增强，并提供保护模式、分页支持和更完整的内存保护机制。相较 8086，80386 展示的不只是“位宽变大”，而是 CPU 开始承担更复杂的系统级硬件职责。

80386 的一个关键变化是 32-bit 线性地址。程序形成的有效地址先经过分段机制得到线性地址；在启用分页时，线性地址再经过页表转换为物理地址。可以用简化链路表示：

```
effective address -> segmentation -> linear address
linear address    -> paging          -> physical address
```

这条链路说明，地址不再只是“寄存器加偏移后送到总线”。CPU 内部必须有地址形成、权限检查、页表访问、异常触发等硬件机制。若页表项不存在、权限不允许、访问越界，CPU 不是简单得到一个错误数据，而是产生异常事件，控制流转入规定入口。

保护模式把“谁能访问什么”变成硬件可检查的约定。段描述符、特权级、页表权限等机制，使得不同程序或系统组件不能随意访问全部内存或执行全部指令。这些机制后来成为现代操作系统隔离、虚拟内存和系统调用的硬件基础。对本书来说，重点不是展开操作系统，而是看见硬件层面的扩展：CPU 不只执行 ALU 运算，还参与地址翻译、权限判断和异常入口选择。

80386 观察点	硬件含义
32-bit 通用寄存器	数据通路和编程模型扩展到 32-bit
32-bit 线性地址	地址空间显著扩大，地址形成更复杂
保护模式	CPU 硬件检查权限、界限和特权级
分页机制	线性地址可经页表映射到物理地址
异常机制	地址或权限错误进入硬件规定的控制入口

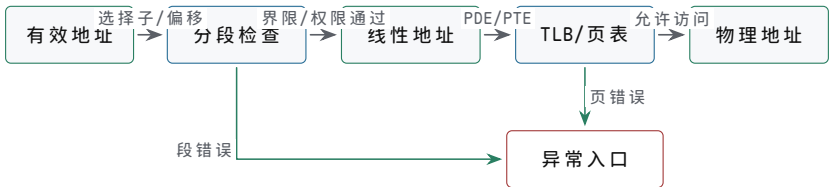
80386 的外部接口同样要按事务理解。32-bit 数据路径并不意味着每次访问都

天然完整 32-bit 字。处理器需要用地地址低位和字节使能说明本次访问哪些字节有效；外部总线、存储器阵列和 cache 控制器必须据此选择正确字节、处理未对齐访问或拆分周期。若只把 80386 理解成“寄存器变成 32 位”，就会漏掉字节使能、总线周期、异常和地址转换这些整机边界。

80386 对象	硬件含义	常见调试点
地址线与数据线	指出事务目标并传输 32-bit 数据片段	地址对齐、字节选择和目标译码是否一致
字节使能	表明本周期哪些字节有效	字节/半字/字访问不会破坏相邻字节
总线状态	区分读、写、取指、确认等周期语义	外部控制器是否按状态产生正确控制线
分页部件	查页目录和页表，形成物理地址	缺页、权限和存在位错误是否触发异常
TLB 思想	缓存近期地址转换结果	命中降低转换时延，失效和权限改变要刷新

分页可以用一次内存加载来观察。指令先形成有效地址，分段得到线性地址；分页开启时，线性地址被拆成页目录索引、页表索引和页内偏移。若转换结果已在 TLB 中，地址转换可以很快完成；若未命中，就要访问内存中的页表结构。页表项还带有存在位、读写权限、用户/内核权限、访问位和脏位等硬件标志。任何一步失败，这次访问不应返回任意数据，而应进入异常处理路径。

```
80386-style address walk:
effective address
-> segmentation checks
-> linear address
-> TLB lookup or page-table walk
-> permission checks
-> physical address or exception
```



80386 的一次内存加载在真正访问 cache 或外部总线前，已经可能因为段、页或权限检查失败而进入异常路径。

图示：80386 地址转换链路。地址转换和权限检查是访存事务的一部分，不是软件注释。

用一个线性地址拆开分页硬件 以线性地址 0x00403ABC 为例，80386 的 4 KiB 页式分页可把它拆成页目录索引、页表索引和页内偏移。高 10 bit 选择页目录项，中间 10 bit 选择页表项，低 12 bit 是页内偏移。该地址的页目录索引为 1，页表索引为 3，页内偏移为 0xABC。若 CR3 指向的页目录中第 1 项不存在，访问在真正读数据前就会变成异常；若页表项存在但权限不允许，也不能继续把物理地址送到 cache 或外部总线。

字段	示例值	硬件用途
页目录索引	1	选择页目录中的一个 PDE
页表索引	3	选择对应页表中的一个 PTE
页内偏移	0xABC	与物理页框基址相加形成最终地址

P/RW/US 等位 来自 PDE/PTE 判断存在性、读写权限和用户/特权访问

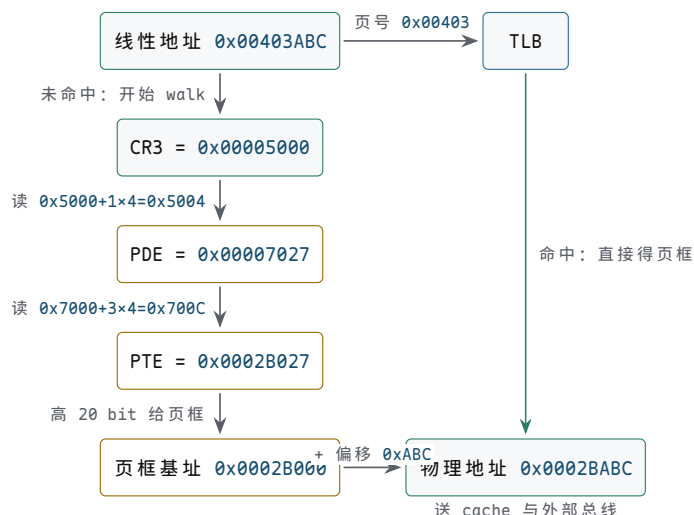
把这次拆页走成一次完整的两级 walk 上面的例子只做了字段拆分，还没有真正读页表。现在把 `0x00403ABC` 从头到尾走完，假设 CR3 保存的页目录基址是 `0x00005000`（页目录本身 4 KiB 对齐，低 12 bit 恒为 0，不参与拼接）。第一步，用页目录索引 1 算出 PDE 的地址： $0x00005000 + 1 \times 4 = 0x00005004$ ，分页部件在这里做一次 4 字节内存读，取回 PDE。设取回的 PDE 为 `0x00007027`，其高 20 bit 给出页表基址 `0x00007000`，低 12 bit 是标志位 `0x027`（P=1、RW=1、US=1、A=1）。第二步，用页表索引 3 算出 PTE 的地址： $0x00007000 + 3 \times 4 = 0x0000700C$ ，再做一次 4 字节内存读，取回 PTE。设取回的 PTE 为 `0x0002B027`，其高 20 bit 给出物理页框基址 `0x0002B000`，标志位同样允许本次访问。第三步，把页内偏移 `0xABC` 拼上： $0x0002B000 + 0xABC = 0x0002BABC$ ，这才是最终送到 cache 和外部总线的物理地址。

CR3	= <code>0x00005000</code>	(page directory base)
PDE address	= CR3 + 1×4	= <code>0x00005004</code>
PDE	= <code>0x00007027</code>	(table base <code>0x00007000</code> , flags <code>0x027</code>)
PTE address	= <code>0x00007000</code> + 3×4	= <code>0x0000700C</code>
PTE	= <code>0x0002B027</code>	(frame base <code>0x0002B000</code> , flags <code>0x027</code>)
physical addr	= <code>0x0002B000</code> + <code>0xABC</code>	= <code>0x0002BABC</code>

这次 walk 的代价值得单独看一眼：在读到目标数据之前，分页部件已经先做了两次内存读（一次读 PDE，一次读 PTE）。若每次访存都这样走，内存流量和地址时延都会成倍增加。TLB 解决的正是这个问题：近期用过的“线性页号到物理页框”转换缓存在片内，命中时两次页表读全部省掉，只剩目标数据那一次访问。权限位和存在位也要随转换结果一起缓存，否则 TLB 命中路径会绕过检查；反过来，软件修改页表后必须刷新对应的 TLB 项，否则硬件会继续按旧映射放行访问。这也是前一章讨论 cache 时见过的“缓存必然带来失效问题”在地址转换路径上的重演。

walk 步骤	地址算术	关键问题
读 PDE	$0x00005000 + 1 \times 4 = 0x00005004$	PDE 的 P 位是否为 1，页表是否存在
读 PTE	$0x00007000 + 3 \times 4 = 0x0000700C$	PTE 的 P/RW/US 是否允许本次访问
拼物理地址	$0x0002B000 + 0xABC = 0x0002BABC$	页框基址加偏移，不再查表
TLB 命中	不访问页表，直接得到页框	两次页表读全部省掉，只读目标数据

把这次 walk 画成指针链，两次页表读与 TLB 旁路的位置就一目了然。



图示：80386 两级页表 walk 的指针链（线性地址 0x00403ABC：页目录索引 1、页表索引 3、页内偏移 0xABC）。CR3 给出页目录基址 0x5000，读 0x5004 取回 PDE=0x00007027；按页表基址 0x7000 读 0x700C 取回 PTE=0x0002B027；页框 0x0002B000 加偏移得物理地址 0x0002BABC。右侧 TLB 命中时整条链被旁路，两次页表读全部省掉。

控制寄存器和描述符表是系统约定的入口 80386 的保护模式不能只理解为“地址变大”。处理器必须知道段描述符在哪里、异常入口在哪里、页目录在哪里、当前是否启用保护和分页。这些信息由控制寄存器、GDTR/IDTR 等状态保存。它们不是普通程序变量，而是 CPU 执行内存访问、异常和特权检查时自动使用的硬件约定。

对象	保存内容	硬件使用场景
CR0	保护模式、分页等控制位的一部分	决定地址和权限检查的总体模式
CR3	页目录基址	TLB 未命中或刷新后查页表
GDTR	全局描述符表位置和界限	装载段选择子、检查段属性
IDTR	中断/异常描述符表位置和界限	异常、中断、陷入入口选择
EFLAGS	条件码、方向、中断允许等状态	分支、算术结果和中断接收边界

从实模式进入保护模式是一段硬件约定切换 80386 复位后并不会自动处于完整分页保护环境。系统固件或早期启动代码必须先准备描述符表，再装载 GDTR，设置 CR0 中的保护模式相关位，并通过跳转刷新取指路径和段状态。分页还需要准备页目录、页表，把 CR3 指向页目录，再打开分页控制位。每一步都是硬件状态切换，顺序错误会导致取指、数据访问或异常入口无法解释。

阶段	硬件动作	错误后果
准备 GDT/IDT	在内存中放置段描述符和异常入口描述符	段装载或异常入口失败
装载 GDTR/IDTR	CPU 记录描述符表基址和界限	后续选择子无法被正确解释
设置 CR0	开启保护模式相关控制	旧的取指和段状态需要刷新
远跳转/重装段	让 CS/DS/SS 等进入新描述符语义	继续按旧语义执行会失控
准备页表并设置 CR3	给分页硬件页目录入口	TLB 未命中时无法 walk
开启分页	线性地址开始经页表转换	映射缺失会立即异常

描述符和异常错误码把失败原因带进硬件入口 保护模式中的段描述符不只是“基址和长度”。它还包含类型、存在位、特权级、可读写/可执行等属性。分页项也不只是物理地址，还包含存在、读写、用户/特权、访问和脏等位。异常发生时，CPU 需要把足够信息交给异常入口，使系统能够判断是不存在、权限错误、越界，还是其他硬件条件失败。

对象	关键字段	硬件问题
段描述符	base、limit、type、DPL、present	这个选择子是否存在、能否以当前特权使用
页表项	page frame、present、RW、US、accessed、dirty	这个线性页是否存在、能否按本次方式访问
异常入口	selector、offset、gate type、DPL	失败后 CPU 应进入哪段处理代码
错误码	选择子或页错误原因位	处理程序如何定位失败对象和原因

一次页错误不是“内存读失败”这么简单。CPU 已经形成了线性地址，检查 PDE/PTE 或权限时发现不能继续，于是放弃普通写回路径，保存足够的异常信息，并从 IDT 中选择页错误入口。若处理器允许一部分状态已经提交、一部分状态未提交，系统就难以恢复。因此异常路径必须与指令提交边界配合：错误指令不应像成功指令一样修改目标寄存器或设备状态。

页错误诊断要同时看地址、访问类型和权限位 80386 把页错误从“总线读不到数据”提升为可诊断的硬件事件。CPU 会保存导致故障的线性地址，并通过异常错误码给出若干原因位。学习阶段不必记住所有后续扩展，但必须掌握三个基本问题：页是否存在，访问是读还是写，访问来自用户态还是特权态。只有把这三项与 PDE/PTE 中的 present、RW、US 等位对照，才能判断是缺页、写只读页、用户态访问内核页，还是页表本身没有建立。

观察对象	含义	排查方向
故障线性地址	哪个线性地址无法完成转换或权限检查	页目录索引、页表索引和页内偏移是否落在预期区域
P 位	故障由不存在页还是保护违规引起	缺页处理、页表初始化或权限设置
W/R 位	本次访问是写还是读导致故障	写只读页、写保护代码段、copy-on-write 类机制
U/S 位	本次访问来自用户级还是特权级	用户程序越权访问内核映射或段/页权限不一致
IDT 入口	页错误应跳到哪个处理程序	异常入口描述符、栈切换和处理代码是否有效

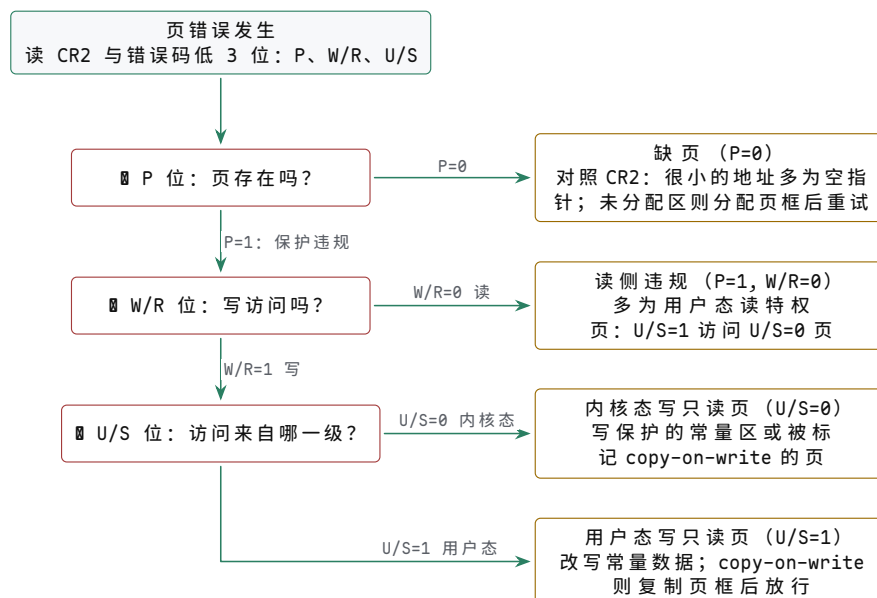
这套诊断方法可以反过来指导最小保护模式实验。先建立一段恒等映射，让取指、栈和数据访问都可成功；再故意取消某个页表项的 present 位，确认访问该页会进入页错误入口；随后把某页设置为只读，执行写访问，确认错误码能区分权限失败和不存在页。若异常处理程序本身所在页没有映射，CPU 可能在处理一个错误时再次出错，最终表现为无法恢复的故障。因此保护模式 bring-up 必须先保证 GDT、IDT、栈、异常代码和页表所在内存可访问。

一次页错误可以把三个原因位逐项读完 看一个完整案例。某次内存访问触发页错误，CPU 把故障线性地址保存在 CR2：CR2 = 0x00DEAD00；异常入口处的错误码为 0x00000003。先拆 CR2：0x00DEAD00 >> 12 = 0x00DEA，说明故障发生在线性页 0x00DEA 内偏移 0xD00 处。再拆错误码低 3 bit：bit0 (P)=1，bit1 (W/R)=1，bit2 (U/S)=0。逐项解释。P=1 表示这次故障不是“页不存在”，而是保护违规：PDE/PTE

链存在，CPU 是在权限检查上拦下了这次访问。W/R=1 表示触发故障的是一次写访问。U/S=0 表示访问发生时 CPU 处于特权级，即内核态代码。三项合起来，结论只有一句：**内核态写只读页**。典型场景是内核错误地写了一个按只读映射的页，例如写保护的常量区或被标记为 copy-on-write 的页。这里有一个历史细节值得知道：80386 上特权级访问不受 RW 位约束，80486 起 CR0 增加 WP 位，置位后特权态写只读页同样触发页错误；不置 WP 时这类错误可能悄悄写成数据损坏，置 WP 之后它才变成可诊断的页错误。

同样的读法可以做两个练习。第一，错误码 0x00000004：P=0、W/R=0、U/S=1，即用户态的一次读访问命中了不存在的页。此时看 CR2：若它是个很小的地址（例如 0x00000018），最常见的解释是用户程序解引用了空指针或野指针；若它落在合法但尚未分配的堆、栈区域，则是典型的按需分配缺页，处理程序应分配页框、建立映射后重新执行该指令。第二，错误码 0x00000007：P=1、W/R=1、U/S=1，即用户态写一个已存在但只读的用户页，典型解释是程序试图改写常量数据或共享库的只读映射；若系统正在使用 copy-on-write，这反而是预期中的缺页，处理程序复制页框后放行即可。判断顺序永远相同：先看 P 区分“不存在”与“保护违规”，再看 W/R 确定访问类型，然后用 U/S 确定访问来源，最后拿 CR2 对照页表定位具体对象。

把这条判断顺序画成决策树，三个原因位与四种结论的对应关系可以按图直接走一遍。



图示：页错误诊断决策树：先看 P 位区分缺页与保护违规，缺页时拿 CR2 对照预期区域；保护违规再看 W/R 位确定读或写，最后用 U/S 位确定访问来源。左列是固定的判断次序，右列是四种结论与各自的处理方向。

错误码	三个原因位	判断与排查方向
0x00000003	P=1、W/R=1、U/S=0	特权态写只读页；检查该页映射属性和内核写路径
0x00000004	P=0、W/R=0、U/S=1	用户态读不存在页；空指针、野指针或按需分配缺页
0x00000007	P=1、W/R=1、U/S=1	用户态写只读用户页；常量区误写或 copy-on-write

80386 总线的字节使能与 8086 一脉相承 80386 对外有 32-bit 数据总线。地址线通常给出 32-bit 对齐字的高位地址，字节使能信号指出本次传输中哪些 8-bit 字节有效。这样，一个字节写不会破坏同一 32-bit 数据片段中的其他字节；

一个未对齐的 32-bit 写可能需要拆成多个传输，或由硬件和总线控制器处理额外周期。

访问类型	有效字节车道	关键问题
字节访问	一个 BE 低有效	相邻字节不能被写改
半字访问	两个相邻 BE 低有效	地址最低位和字节序必须一致
对齐字访问	四个 BE 低有效	一个总线数据片段可完成传输
未对齐字访问	跨越两个对齐片段	需要拆分周期或触发规定异常

从时延角度看，80386 到 80486/Pentium 的演进可以读成“把常用慢路径搬近”。TLB 避免每次访存都走页表；片上 cache 避免每次命中数据都走外部总线；流水线把长执行路径切成多个边界；分支预测减少控制流改变带来的等待。硬件并没有消灭延迟，而是用缓存、预测、并行和更细边界，让常见情况少走远路，让少见情况通过异常、回填或冲刷流水线处理。

从 8086 到 80386，可以看到 CPU 和整机边界的变化。8086 主要展示了早期微处理器如何通过外部总线接入存储器和 I/O；80386 则展示了处理器如何把内存管理和保护机制纳入芯片内部。后来的处理器继续把 cache、TLB、流水线、分支预测、乱序执行、多核互连等机制纳入芯片，但基本问题没有改变：地址从哪里来，权限如何检查，数据经哪条路径返回，状态在哪个边界提交。

80486 和 Pentium 可以作为 80386 之后的两个观察窗口。80486 把片上 cache、流水线和浮点方向的集成推到更显著的位置，使常用指令和数据更容易走近路径；Pentium 进一步展示双流水线、分支预测和更复杂控制对吞吐率的影响。它们并没有改变“寄存器到组合逻辑到寄存器”的基本模型，而是复制、切分和调度这些路径，让常见工作更少等待远处存储器和长控制路径。

阶段	新增硬件重点	与前文概念的连接
80386	32-bit 地址、分页、保护和异常	地址路径和权限判断进入 CPU 内部关键结构
80486	片上 cache、流水线、浮点集成	常见数据靠近核心，长组合路径被边界切分
Pentium	双流水线、预测和更复杂发射控制	多条候选执行路径需要统一调度和恢复
现代核心	乱序、重命名、TLB、多级 cache、多核互连	快慢路径、状态提交和一致性成为核心问题

三、整机启动与经典芯片后的演进

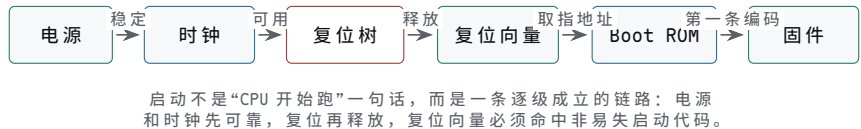
通电或复位后，CPU、主存控制器、互连和设备控制器都必须进入规定初始状态。PC 通常被置到固定启动地址，那里连接 ROM、片上存储或其他启动介质。启动路径的第一件事，是让硬件从可预测位置开始取编码。

整机启动不仅是 CPU 复位。时钟源要稳定，复位树要按顺序释放，主存控制器要完成训练或初始化，互连要进入可转发请求的状态，必要设备要处于安全默认值。若这些硬件条件没有满足，即使 CPU 本身能取指，也可能在第一次访问主存或设备时无法继续推进。

从复位向量走到第一条有效指令 启动可以写成一条硬件 trace。上电后，电源管理电路等待电压进入允许范围；时钟源或晶振稳定后，复位电路保持 CPU 和关键外设安全状态；复位释放时，PC 或取指地址逻辑装入固定复位向量；地址译码把这个地址指向 ROM、Flash 或片上启动存储；取指路径读回第一条编码；控制器译码并开始执行初始化序列。任何一步没有定义，整机都会表现为“CPU 不启动”，但根因可能不在 CPU。

启动阶段	硬件动作	预期行为
------	------	------

电源爬升	电压达到允许范围，去耦提供瞬态电流	欠压复位未提前释放
时钟稳定	晶振、PLL 或外部时钟进入稳定范围	时钟频率和占空比满足规格
复位保持	CPU、控制器、设备进入安全默认状态	危险输出关闭，写使能无效
复位释放	PC 装入复位向量或启动地址	第一拍地址可预测
取启动码	ROM/Flash/片上存储响应该地址	数据总线返回有效编码
初始化	配置栈、RAM、时钟、设备和中断	每项配置有完成或错误状态



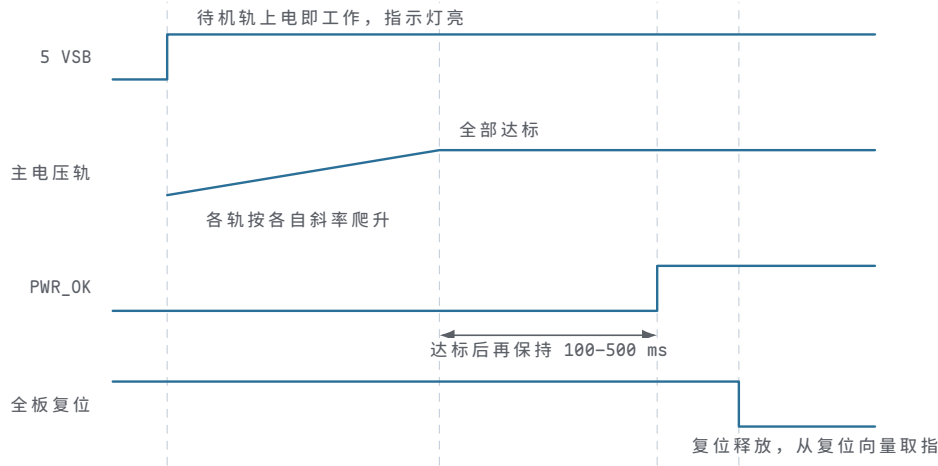
图示：整机启动链路。第一条有效指令依赖电源、时钟、复位、地址映射和启动存储共同成立。

主板电源时序决定了复位何时可以释放 上图链路的前两级——电源和时钟——值得再放慢看一遍。按下电源键后，ATX 电源的各电压轨（+3.3 V、+5 V、+12 V 以及主板上的二级稳压输出）并不是瞬间到位：输出电容要充电，稳压反馈环路要收敛，不同轨的爬升斜率也不相同。在电压进入允许误差范围之前，逻辑门的输出没有定义，此时若释放复位，CPU 和控制器可能在半稳定状态下取指、写寄存器，结果不可预测。因此电源内部设有 PWR_OK（power good）信号：它在各主要电压轨稳定达标之后再保持一小段时间才有效（ATX 规范给出的典型窗口是 100–500 ms），电压跌落时则提前失效，给整机留出收尾的余量。主板复位电路把 PWR_OK 当作释放条件之一：PWR_OK 无效就保持全板复位，有效之后再按时钟节拍有序释放复位树。这就解释了为什么“插电能亮”不等于“电压稳定”：待机指示灯只要 5 VSB 待机轨工作就能点亮，它在上电瞬间即可工作，而主电压轨爬升和 PWR_OK 确认是几十到几百毫秒量级的过程，两者相差多个数量级。

冷启动与热启动的差别也在这里。冷启动从完全断电开始，必须走完整个电压爬升、时钟稳定和 PWR_OK 等待，DRAM 内容不可靠，内存自检和初始化不能省略；热启动（复位键或软件复位）时各电压轨早已稳定，时钟不需要重新等待，复位电路只产生一个短复位脉冲，固件可以通过约定标志跳过耗时的内存测试。对调试而言，区分两者很有用：只在冷启动出现的问题，多半与电源爬升、时钟起振或复位释放顺序有关；冷热启动都出现的问题，才更可能在逻辑本身。

时序阶段	硬件事实	关键问题
待机	5 VSB 待机轨供电，点亮指示灯、等待开机事件	指示灯亮只说明待机轨在工作
电压爬升	各电压轨按各自斜率上升，输出电容充电	未达标前逻辑输出没有定义
PWR_OK 确认	电压达标后再保持约 100–500 ms 才有效	提前释放复位会让 CPU 在半稳定状态取指
复位释放	复位电路以 PWR_OK 为条件，按时钟释放复位树	复位向量处必须已有可靠时钟和存储响应
热启动	电压轨保持稳定，仅产生短复位脉冲	可跳过内存测试，但复位时序仍要完整

把上表画成时间轴，几条信号线之间的先后约束就一目了然：主电压轨先爬升达标，PWR_OK 延迟确认，复位最后才释放。



图示：主板电源时序：待机轨 5 VSB 上电即工作，主电压轨按各自斜率爬升，全部达标之后 PWR_OK 再保持 100-500 ms 才有效，复位电路以 PWR_OK 为释放条件、按时钟节拍释放复位树。指示灯亮只说明待机轨在工作，它与主电压轨稳定相差几十到几百毫秒——这就是“插电能亮”不等于“电压稳定”的时序根源。

复位入口体现地址空间顶端和底端的不同设计 不同处理器的复位入口不相同，但背后的硬件问题一致：复位后 CPU 必须从非易失存储中的确定位置取得第一条可执行编码。6502 从固定向量中取入口地址，Z80/8051 常从 0 地址开始执行，8086 从 1 MiB 空间顶端附近开始取指，80386 及后续 x86 则把初始取指放到更高物理地址附近，以便系统固件映射在地址空间顶端。

处理器	典型复位入口	硬件含义
6502	从 <code>0xFFFFC/0xFFFFD</code> 取 16-bit 入口	顶端向量表必须由 ROM 响应
Z80/8051	从 <code>0x0000</code> 开始执行	低地址必须映射启动代码
8086	物理地址 <code>0xFFFF0</code> 附近	1 MiB 顶端 ROM 保存跳转或启动片段
80386 及后续 x86	高物理地址顶端附近	固件映射到高地址，早期代码再跳转到初始化区域
现代 SoC	片上 Boot ROM 或可配置启动介质	先验证/选择介质，再初始化 DRAM 和外设

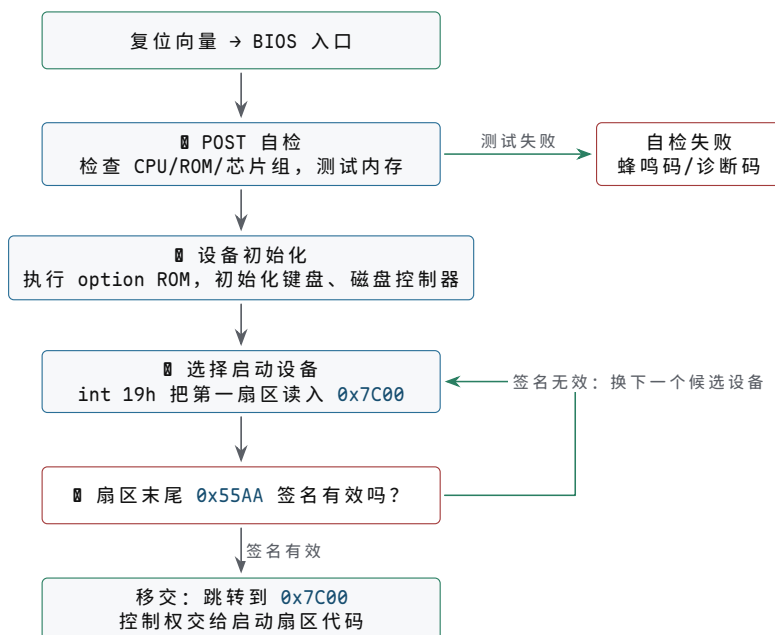
在 8086 类系统中，复位后从规定的高地址入口开始取指，外部 ROM 必须映射到这个入口；在许多微控制器中，复位向量可能位于片内 Flash 的固定地址，前几个字保存初始栈指针和入口地址；在现代 SoC 中，片上 Boot ROM 可能先验证启动介质、配置内存控制器，再把控制权交给后续固件。形式不同，问题相同：复位向量指向哪里，谁在该地址响应，第一批指令执行前哪些硬件已经可靠。

BIOS 把固件启动分成自检、初始化、选设备和移交四步 复位向量把 CPU 送进 BIOS 之后，固件并不是立刻去“找操作系统”，而是按四个阶段把整机带到可交接状态。第一阶段是 POST (power-on self-test, 加电自检)：BIOS 先检查 CPU、ROM 校验和与芯片组基本寄存器，再初始化并测试内存——早期 PC 对 DRAM 做逐段读写测试，容量越大耗时越长，这正是热启动要跳过它的原因；测试失败时用蜂鸣码或诊断口代码报告，因为此时连显示都未必可用。第二阶段是设备初始化：先点亮显示（视频 BIOS 常以 option ROM 形式挂在 `0xC0000` 附近），再初始化键盘控制器、磁盘控制器等；BIOS 会扫描规定的地址区间，找到带签名头的 option ROM 就把它初始化入口执行一遍。第三阶段是启动设备选择：按约定的启动顺序（软盘、硬盘、光盘等）逐个尝试，int 19h 把候选设备的第一个扇区（512 字节）读到固定物理地址 `0x0000:0x7C00`。第四阶段是移交：扇区已在内存，检查其末尾的 `0x55AA` 签名，签名有效则跳转到 `0x7C00`，控制权自此属于启动扇区代码；签名无效则换下一个候选设备。

在移交之前，BIOS 还扮演“早期操作环境”的角色：它以中断向量的形式提供服务，`int 10h` 操作文本显示，`int 13h` 读写磁盘扇区，`int 16h` 读键盘。启动扇区和早期引导代码没有任何驱动可用，正是靠这些实模式中断服务完成显示和读盘；进入保护模式后实模式中断向量表失效，操作系统才必须换成自己的驱动。这套约定后来被 UEFI 整体重写：分区表从 MBR 换成带 GUID 的 GPT，不再依赖 `0x7C00` 装载约定和 `0x55AA` 签名；固件功能组织成可扩展的驱动与应用模型，启动装载程序是固件直接装载执行的 `.efi` 文件；设置界面从字符画面换成图形界面，启动项保存在 NVRAM 变量中由启动管理器选择。载体全换了，四阶段的骨架仍在：自检并初始化硬件、准备服务环境、按顺序选择启动源、把控制权交给下一阶段。

阶段	硬件动作	预期结果
POST 自检	检查 CPU/ROM/芯片组，初始化并测试内存	失败有蜂鸣码或诊断码，不进入下一阶段
设备初始化	执行视频等 option ROM，初始化键盘、磁盘控制器	显示可用，设备寄存器进入约定默认态
选择启动设备	按启动顺序逐个把第一扇区读到 <code>0x7C00</code> ，检查 <code>0x55AA</code> 签名	第一个签名有效的设备被选中
移交控制权	签名有效则跳转到 <code>0x7C00</code>	CPU 从启动扇区第一条编码开始执行

把四个阶段连同失败分支画成流程图，签名检查后“换下一个候选设备”的回退路径也一并可见。



图示：BIOS 启动的四阶段流程：POST 失败时以蜂鸣码或诊断口代码报告，不进入后续阶段；选设备阶段按约定的启动顺序逐个尝试，签名无效就回退到下一个候选设备；只有签名有效的扇区才能得到 CPU 控制权。

整机实验可以从 ROM、RAM 和两个 MMIO 寄存器开始 一个可搭建的最小整机不需要复杂操作系统。可先规定 8-bit 或 16-bit 地址空间：低地址接 ROM，中间接 RAM，高地址映射 LED 输出寄存器和按键输入寄存器。CPU 或教学控制器复位后从 ROM 取指，执行一段程序：读取按键状态，若按下则把 LED 寄存器置 1，否则置 0。这个系统已经包含整机硬件的核心对象：取指、数据访存、地址译码、MMIO、副作用、外部输入同步和可观察输出。

```

minimal whole-machine program:
loop:
    r1 = MMIO[GPI0_IN]
  
```

```

    if r1 == 0 goto off
    MMIO[GPIO_OUT] = 1
    goto loop
off:
    MMIO[GPIO_OUT] = 0
    goto loop

```

这段程序的硬件检查不应只看 LED 是否跟随按钮。应同时确认 ROM 地址、RAM 地址和 MMIO 地址不会被同时片选；按钮已经同步后才被读；写 GPIO_OUT 只改变输出锁存器，不误写 RAM；分支改变的是 PC 下一值；循环运行时没有总线争用。若以后把 LED 换成电机、继电器或功率负载，还必须增加驱动级、续流保护和安全停机链路，不能把 MMIO 输出锁存器直接当成功率开关。

最小整机的检查应分成三层 第一层检查硬件静态条件：电源、复位、时钟、片选互斥和三态方向。第二层检查单条指令：ADD、MOV、JZ 等每条指令都能在单步下看到正确的控制线和提交边界。第三层检查小程序：ROM 中循环程序能够读按键、分支、写 LED，并能在异常输入下保持安全状态。

层次	具体检查	失败时优先排查
静态硬件	电源纹波、复位释放、时钟边沿、片选互斥	供电、复位树、译码极性
单指令	PC、IR、控制字、ALUOut、MDR、写回寄存器	控制器、寄存器堆、ALU、存储器时序
小程序	按键输入同步、条件分支、LED 输出锁存	MMIO 地址、分支目标、外设副作用
错误路径	未映射地址、慢设备等待、外设 error 位	超时、错误响应、异常入口

在实验报告中，应记录每个检查点的“预期值、实测值、观察工具和结论”。例如 PC 第 0 拍应为 0x0000，ROM 片选应有效，IR 应锁存第一条 MOV；执行写 GPIO_OUT 的指令时，RAM 片选应无效，LED 锁存器写使能应只出现一个有效窗口。这样的记录比“程序运行成功”更接近工程实践。

可采购器件清单要匹配教学目标 若目标是搭一台低速可观察整机，器件选择应服务于可测性，而不是追求最高频率。可以使用 74HC/74HCT 系列计数器、锁存器、三态缓冲器、加法器、比较器、EEPROM、SRAM 和 LED/按键模块；也可以用 CPLD/FPGA 把控制器和数据通路放进可编程逻辑，再把 ROM/RAM/MMIO 作为独立模块观察。

对象	可选器件	选择理由
PC/计数	74HC161/163 或 FPGA 寄存器	可清零、可装载、可单步观察
IR/锁存	74HC574/273	时钟沿保存指令或中间状态
ALU 原型	74HC283、异或门、比较器或 FPGA 逻辑	可先实现加法、零检测和简单逻辑
总线隔离	74HC245/244	控制数据方向，避免多源驱动
ROM	AT28C 系列 EEPROM 或片内 ROM	保存启动程序和指令编码
RAM	低速异步 SRAM 或 FPGA block RAM	接口直观，便于单步读写
I/O	LED 锁存器、按键同步器、UART 模块	形成可观察输入输出和串口调试

搭建顺序应从电源、时钟、复位、单个寄存器和单条总线开始。只有确认三态控制不会冲突、复位后所有写使能无效、时钟边沿干净，才应接入 ROM、RAM 和 I/O。

若直接把所有模块连接后调试，任何错误都可能表现为“程序不跑”，排查成本会急剧上升。

整机 bring-up 要记录症状、信号和假设 真实硬件第一次上电时，常见问题不是“CPU 坏了”，而是复位、时钟、片选、总线方向、等待状态或启动 ROM 映射有误。把症状、相关信号和当前假设逐项记录下来，才能形成一条可逐步核对的排查链条，而不是笼统地说“检查硬件”。

症状	优先观察信号	可能原因
PC 不变化	复位、时钟、PC 写使能	复位未释放、时钟不稳定、控制器未进入取指
ROM 无输出	地址线、ROM 片选、输出使能	复位向量未映射到 ROM，译码极性错误
数据总线异常发热	CPU/存储器输出使能、方向控制	两个器件同时驱动总线
加载读错值	地址、字节使能、读响应、MDR	地址形成错误或存储器时序不满足
LED 乱闪	MMIO 片选、写使能、外设锁存时钟	地址译码过宽或写周期毛刺
中断重复进入	设备状态、中断控制器 ACK、CPU 入口	清除顺序错误或事件源未确认

经典微处理器的发展可以用若干硬件剖面概括。下表不是处理器史年表，而是说明每一代芯片把哪些硬件问题推到读者面前。

芯片或系列	典型特征	对硬件学习的意义
4004	4-bit 微处理器	微处理器把控制器、寄存器和算术部件集成到单芯片形态
6502/Z80	经典 8-bit 微处理器	外部总线、存储器、I/O 和多周期控制构成早期微机基本形态
8051	经典单片机	CPU、片内存储、I/O、定时器和中断控制展示片上整机控制
8080/8085	8-bit 微处理器	外部总线、地址空间、存储器和 I/O 开始形成通用微机结构
8086/8088	16-bit x86 起点	段：偏移地址、指令预取、复用总线和外部锁存器体现早期整机取舍
80286	保护模式前驱	分段保护和特权机制开始成为处理器硬件的一部分
80386	32-bit x86	32-bit 寄存器、线性地址、分页和异常机制把 CPU 推向系统级硬件
80486/Pentium	片上 cache 与更深执行结构	流水线、cache、浮点单元和更复杂控制逻辑成为性能核心

8051 的意义在于把“小整机”收进一颗芯片。它让读者看到，CPU 旁边可以直接布置片内 RAM、ROM、I/O 口、定时器、串口和中断控制。外部引脚不再只是地址/数据总线，也可能直接是可编程 I/O。对硬件学习而言，这说明整机边界可以移动：同样是“写设备寄存器改变输出”，在早期微机上可能通过板级地址译码访问外设，在单片机上则可能是写特殊功能寄存器改变某个引脚锁存器。

80486 的意义在于把片上 cache 和更清楚的流水化执行推到学习视野中。片上 cache 缩短了高频取指和数据访问路径，使常见访问不必每次走外部总线；流水线把取指、译码、执行等工作分段，使不同指令可以重叠推进。它同时提醒读者，性能不是单靠提高门速度，把常见数据放近和把长路径切段同样关键。

Pentium 的意义在于展示更强的并行执行方向。双流水线、分支预测和更复杂的发射控制，使处理器尝试在同一时间推进多条合适的指令。此时硬件问题不再只是“一个 ALU 算得多快”，还包括指令之间是否有依赖、分支方向是否预测正确、两条流水线是否争用资源、错误路径如何清除、异常如何保持精确边界。这些机制为现代乱序执行、寄存器重命名和提交队列铺路。

80486 把 cache、FPU 和流水线收进芯片 80386 时代，cache 是主板上的独立芯片加标签存储器，浮点运算是另一颗 80387 协处理器，两者都通过板级总线与 CPU 通信。80486 把三样东西收进片内：8 KiB 统一 cache（指令和数据共用，采用写穿策略）、浮点单元和一条 5 级流水线。配套的还有总线接口的改变：486 引入突发总线周期，一次 cache 行填充用 4 拍连续传完 16 B，只有第一拍需要给出地址，后面三拍不再重复。收益很直接：cache 命中时取指和读数不再走板级总线，常见指令的吞吐接近每拍一条；代价同样明确：片内 cache 要处理一致性，写穿策略持续占用外部总线带宽，流水线的段间控制、数据相关和分支处理也比多周期实现复杂得多。

Pentium 用双发射和分支预测重组执行结构 Pentium 的性能提升不再主要来自时钟频率，而是来自执行结构的重组。它有 U、V 两条整数流水线，满足配对规则的简单指令可以在同一拍内双发射；分支目标缓冲（BTB）预测分支方向，猜错才冲刷流水线；指令 cache 和数据 cache 分开，各 8 KiB，取指和读数不再争同一个端口；外部数据总线扩到 64 bit，一次总线事务搬运的数据翻倍。Pentium Pro 又往前走了一步：x86 指令先被译成微操作，由乱序执行核心按数据就绪顺序执行，寄存器重命名消除假依赖，最后按程序顺序提交，从而在乱序执行下保持精确异常。执行可以和提交分离，是这一代留下的主线：第四章讲的执行/提交区分，在这里成为真实芯片的基本组织方式。

386/486 主板把 CPU、内存和外设分成平台层次 80386、80486 之后，整机越来越依赖主板芯片组。CPU 不再直接以简单板级总线面对所有外设，而是通过内存控制器、cache 控制器、总线桥和 I/O 控制器访问不同速度、不同协议的目标。早期 PC 主板可以粗略理解为：CPU 与高速本地总线连接，芯片组负责 DRAM、cache、ISA/EISA/VLB/PCI 等扩展路径，低速外设再挂到更慢的 I/O 控制器。后来的“北桥/南桥”说法，就是把高速内存/图形路径和低速 I/O 路径分开组织。

平台层次	负责对象	与本章概念的关系
CPU 本地总线	取指、数据访问、cache miss、写回	地址、数据、字节使能、等待和总线错误
内存/cache 控制	DRAM 行列访问、二级 cache、写缓冲	命中/未命中、回填、写回和时序预算
高速扩展路径	图形、磁盘控制、网络或总线桥	DMA、总线主设备、仲裁和中断
低速 I/O 控制	串口、并口、键盘、软驱、RTC 等	端口 I/O、状态寄存器、完成事件
固件层	BIOS/Option ROM、设备初始化和启动选择	复位向量、枚举、内存检测和启动介质

这个分层解释了为什么整机性能不能只看 CPU 主频。若 cache 未命中要走慢 DRAM，若 PCI 设备 DMA 被总线仲裁阻塞，若中断控制器或桥片配置错误，CPU 再快也会等待。另一方面，把 cache、内存控制器、图形控制、PCIe 根复合体等功能逐步移近 CPU，也正是现代平台降低时延的方向：常用路径减少芯片间往返，慢速外设通过桥接和队列异步处理。

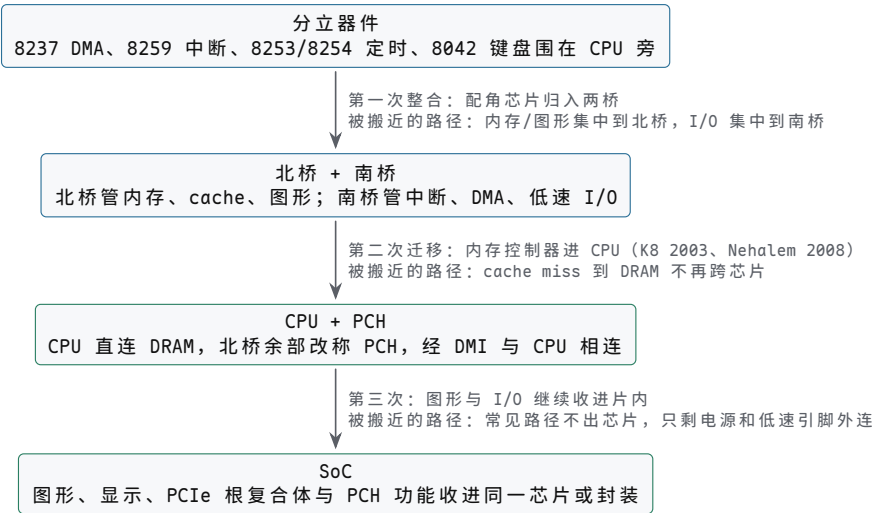
芯片组演进是把慢路径一段段搬进更快的边界 上表的平台分层不是一开始就存在的。最早的 PC 主板上，这些职责是各自独立的分立器件：8237 负责 DMA，8259 负责中断，8253/8254 负责定时，8042 负责键盘，再算上时钟发生器和总线缓冲，一

颗 CPU 旁边要围一圈“配角芯片”。第一次整合是南北桥结构：北桥接管内存控制器、cache 控制和图形接口（后来的 AGP），直接坐在 CPU 前端总线上；南桥接管中断、DMA、定时器和 IDE、USB、ISA/LPC 等低速 I/O，两桥之间用专用链路相连。第二次迁移是内存控制器进 CPU：从 AMD K8（2003）和 Intel Nehalem（2008）开始，CPU 直连 DRAM，北桥名存实亡，剩余部分改叫 PCH（平台控制中心），通过 DMI 与 CPU 相连。第三次是 SoC：图形、显示、PCIe 根复合体乃至 PCH 的功能继续收进同一颗芯片或同一封装。

每一次迁移，移动的都是某条具体路径的时延。分立器件时代，CPU 读一次内存要穿过主板走线和总线缓冲；南北桥时代，访存路径变成“CPU 经前端总线到北桥再到 DRAM”，多一次芯片间往返，引脚和板级走线的时延直接加在每次 cache miss 上；内存控制器进 CPU 后，这一段跨芯片往返消失，cache miss 到 DRAM 的路径不再离开芯片——这是收益最大的一次迁移，因为取指和读数是整机里最频繁的路径。中断和 DMA 则一直留在南侧：它们本来就慢，继续经南桥或 PCH 转发，换来的是 CPU 引脚和主板面积的节省。判断一代平台时，只要问“这次被搬近的是哪条路径”，就能看出它为什么变快、又付出了什么代价。

平台形态	集成对象	被缩短的路径
分立器件 北桥 + 南桥	8237/8259/8253 等各管一摊 北桥管内存/图形，南桥管低速 I/O	无片内捷径，全部走板级总线 内存路径集中到北桥，I/O 路径集中到南桥
PCH	内存控制器进 CPU，南桥余部改称 PCH	cache miss 到 DRAM 不再跨芯片，I/O 经 DMI 转发
SoC	图形、PCIe 根复合体、外设全部片内	常见路径不出芯片，只剩电源和低速引脚外连

把三次迁移按顺序画成演进链，每一次被搬近的具体路径直接标在迁移箭头上。



图示：芯片组演进链：每一次迁移搬近的都是某条具体路径——先是配角芯片归入南北两桥，再是内存控制器进 CPU、让 cache miss 到 DRAM 不再跨芯片（收益最大的一次，因为取指和读数是整机里最频繁的路径），最后图形、PCIe 根复合体和 PCH 功能收进 SoC。中断和 DMA 这类本来就慢的路径一直留在南侧。

从 PCI 到 PCIe，设备仍然是事务发起者和响应者。PCI/PCIe 这类现代扩展互连比早期 ISA 总线复杂得多，但本章的分析方法仍然适用。设备可以暴露配置空间、MMIO BAR、DMA 能力和中断机制；CPU 或固件先枚举设备，分配地址窗口和中断资源；驱动程序再通过 MMIO 配置设备，通过 DMA 描述符让设备读写内存，并通过 MSI/MSI-X 或传统中断接收完成事件。

PCI/PCIe 对象 硬件含义	关键问题
------------------	------

配置空间	设备身份、能力、BAR 和控制位	枚举是否能读到设备，BAR 是否分配到不冲突地址
BAR/MMIO	设备寄存器窗口映射到物理地址	访问属性是否不可缓存或按设备规则排序
DMA	设备作为总线发起者读写主存	地址权限、IOMMU、cache 一致性和 completion 顺序
MSI/MSI-X	设备通过内存写形式投递中断消息	消息地址/数据配置是否正确，是否会丢事件
错误报告	链路、权限、超时或设备内部错误	错误能否被记录、上报和恢复

这一段对学习现代硬件很重要：PCIe 不再是多块板卡共享一排并行地址线的简单总线，但它仍然要解决同样的问题。请求从哪里来，目标地址是谁，数据何时有效，完成如何返回，错误如何上报，中断如何投递，DMA 写入何时对 CPU 可见。只要把这些问题标出来，复杂协议就不会变成无法理解的名词堆。

SoC 把北桥、南桥和许多外设继续收进芯片 现代 SoC 的趋势是把 CPU 核、cache、内存控制器、GPU/NPU、视频编解码、PCIe/USB/SATA、网卡、显示控制、GPIO、定时器和安全启动逻辑放进一颗或少数几颗芯片。这样做的直接收益是缩短常见路径、降低引脚数量、减少板级功耗和提高集成度；代价是片上互连、一致性、时钟/电源域、调试和安全启动变得更复杂。

SoC 结构	低时延来源	新增边界
片上 L2/L3 cache	多个核心共享近处数据	一致性状态、替换、QoS 和带宽竞争
集成内存控制器	CPU 到 DRAM 控制路径更短	DRAM 训练、刷新、ECC 和功耗状态
片上互连	核、DMA、GPU、外设通过网络通信	仲裁、优先级、背压和死锁避免
集成外设	GPIO、UART、SPI、I2C、USB 等近距离访问	MMIO 属性、中断路由和复位/时钟门控
安全启动	Boot ROM 验证后续固件	密钥、签名、回滚保护和调试锁定

因此，“现代芯片为什么时延低”可以更完整地回答：不是所有路径都低时延，而是常见路径被集成到片上、被 cache/TLB 缓存、被预取和预测提前准备、被流水线和乱序执行并行化、被 DMA 和队列卸载；不常见路径则通过 cache miss、TLB miss、DRAM 排队、设备背压、错误报告和中断恢复来处理。低时延来自体系结构、微架构、互连、存储层级和平台集成共同作用。

从 8086、80386 向后看，处理器继续沿几条方向扩展。第一，内部执行路径更深，流水线把取指、译码、执行、访存、写回切成多个阶段。第二，存储层级更复杂，片上 cache 和 TLB 成为地址访问性能的关键。第三，控制逻辑更复杂，分支预测、乱序执行和异常恢复要求硬件保存更多内部状态。第四，多核和片上互连让“总线事务”扩展成核间一致性和缓存一致性协议。第五，外设从简单寄存器发展到 PCI、PCIe、USB、SATA 等分层协议，但本质仍然是地址、数据、控制、握手、完成和错误状态。

演进方向	保持不变的硬件问题
流水线	状态边界如何切分，错误路径如何清除
cache/TLB	地址如何命中、缺失、替换和回填
保护与异常	何时拒绝访问，如何进入规定处理入口
DMA 与设备	谁发起事务，谁完成事务，完成如何通知 CPU

到这里，本书的硬件链路闭合：电和器件形成可控开关，开关形成门，门形成组合逻辑，反馈和时钟形成状态，寄存器和 ALU 形成数据通路，控制器组织动作，CPU 执行编码，主存、cache、互连、设备和启动路径把它扩展成整机。8086 和 80386 的意义在于提供了两个清晰历史剖面：一个展示早期微处理器如何连接总线和地址空间，一个展示处理器如何承担更复杂的地址转换、保护和异常机制。

四、从目标文件、固件到可启动机器

前面几章已经解释了 CPU 如何执行编码，主存和设备如何响应地址，经典芯片如何定义平台边界。还差一条常被硬件教材略过、但对系统理解非常关键的链路：程序怎样从目标文件进入内存，符号地址怎样变成机器地址，启动固件怎样把硬件带到可执行状态，异常表和页表怎样让失败路径可恢复。参考经典系统教材的读法，这一节不展开编译器细节，而是把链接、装载和启动都还原成硬件地址约定。

目标文件里的符号最终要变成地址 汇编器或编译器产生的目标文件通常还不知道所有最终地址。函数名、全局变量名、外部符号和跳转目标先以符号或重定位项表示。链接器把多个目标文件和库合并，决定代码段、只读数据段、数据段、BSS 段等放到什么地址，并把需要修补的位置改成最终地址或相对偏移。

对象	系统含义	硬件落点
代码段	机器指令编码	取指路径从这些地址读取 opcode
只读数据	常量、跳转表、字符串	普通加载读取，权限通常禁止写
数据段	已初始化全局变量	启动时从镜像复制到 RAM
BSS	零初始化全局变量	启动时清零，镜像中不必保存全量零字节
重定位项	需要链接器或装载器修补的地址位置	改写指令立即数字段、地址表或指针字

这说明“函数地址”不是抽象标签。取指硬件需要 PC 中的数值地址；CALL 需要目标地址或相对偏移；全局变量加载需要数据地址；中断向量需要入口地址。链接器和装载器的工作，就是把这些名字和段布局变成 CPU 能取指、加载、存储和跳转的地址。

```
source idea:
    call uart_putc
    load global_counter

after link/load:
    CALL 0x00102480
    MOV R1, [0x00203010]
```

装载器建立的是第一张可执行地址地图 在有操作系统的机器上，装载器把可执行文件映射进进程地址空间，建立代码、数据、堆、栈和共享库区域；在裸机或固件场景中，启动代码负责把镜像从 ROM/Flash 复制或映射到合适位置，清 BSS，设置栈指针，初始化全局数据，然后跳到入口函数。两者规模不同，但硬件问题一致：哪些地址可取指，哪些地址可读写，栈在哪里，异常入口在哪里，设备寄存器在哪里。

启动/装载动作	硬件意义	失败表现
---------	------	------

设置栈指针	CALL/RET、局部变量和异常入口有可用内存	第一次函数调用或中断即破坏内存
复制数据段	RAM 中全局变量得到初始值	程序读到未初始化数据
清 BSS	零初始化对象符合语言和固件约定	状态机、计数器、指针初值随机
建立向量表	中断和异常有入口地址	外部事件或错误无法处理
初始化时钟/总线	后续 RAM、设备、cache 路径满足时序	访问慢器件超时或读写失败
跳入口	PC 改到主程序第一条指令	程序真正进入可维护代码路径

裸机启动常见伪代码如下。它表面像软件初始化，实质上每一步都在建立后续硬件事务的前提。

```
reset_handler:
    set SP to top_of_stack
    copy .data from flash image to SRAM
    zero .bss in SRAM
    initialize clocks and memory controller
    install vector table base
    initialize essential MMIO devices
    jump main
```

异常向量表是硬件失败路径的地址表 只要系统会处理中断、页错误、非法指令或设备事件，就需要某种入口表。早期处理器可能用固定地址或固定向量；保护模式处理器使用更复杂的描述符表；微控制器常在启动区域放置向量表。表的形式不同，但作用相同：当硬件事件发生时，CPU 能把事件编号转成处理程序入口，并保存足够状态以便恢复或诊断。

向量项	保存内容	硬件使用场景
复位向量	第一条启动代码地址或跳转指令	上电或复位后装入 PC
非法指令向量	无法译码或不允许执行时的入口	opcode 不合法或特权不允许
页错误/总线错误向量	地址转换或外部事务失败入口	加载/存储/取指不能完成
外设中断向量	设备完成、错误或数据到达入口	中断控制器投递事件
系统调用入口	用户代码请求特权服务入口	权限切换和受控进入内核或监控程序

向量表本身也必须位于可访问地址。若页表或地址译码没有覆盖异常入口，CPU 在处理第一个错误时会遇到第二个错误；若栈没有准备好，保存 PC/Flags 或错误码时会破坏未知内存；若中断处理程序所在代码被错误标成不可执行，事件到达后系统会在入口处再次失败。启动顺序必须先保证最小异常路径可靠，再打开复杂设备和中断。

页表把装载结果提升为可保护的运行时地图 链接器决定镜像中的相对布局，装载器把段放进内存，页表则把运行时地址空间变成可检查的约定。代码页可以标为只读和可执行，数据页可读写但不可执行，内核页禁止用户访问，设备 MMIO 页设置为不可缓存或强顺序。这样，同一条加载/存储/取指路径在真正访问 cache 或总线前，会先经过地址转换和权限检查。

区域	典型页属性	硬件效果
代码	只读、可执行	对代码页的写操作触发保护异常
只读数据 普通数据/堆/栈	只读、不可执行或按平台规定 可读写、通常不可执行	防止常量被误写或执行数据 数据写入允许，取指可被阻止
内核映射 MMIO 窗口	特权访问，用户禁止 不可缓存、设备顺序属性	用户态越权访问触发异常 读写按设备事务执行，不被普通 cache 吞掉

这张表把前几章内容合并起来：第三章的 bit 和地址解释，第四章的 PC 与加载/存储，第五章的 cache/MMIO 属性，第六章的 80386 分页和异常，在页表属性中汇合。现代系统的许多安全和可靠性机制，本质上就是让硬件在每次取指和访存前检查这些约定。

从固件到操作系统是平台所有权转移 固件启动阶段拥有硬件控制权：它配置时钟、内存控制器、基本 I/O、启动介质和必要安全检查。操作系统接管后，会重新建立页表、中断控制、设备驱动和调度机制。这个交接不是一句“加载内核”，而是一段平台所有权转移：谁拥有中断表，谁拥有页表，谁管理内存，谁接管设备，谁定义错误处理策略。

阶段	固件职责	交给后续系统的约定
早期启动 内存初始化	电源、时钟、复位、最小取指路径 DRAM 控制器、cache/TLB 初始策略	CPU 能从可信入口执行 可用内存范围和保留区域
设备发现	基本总线枚举、启动介质选择	设备地址窗口、中断和 DMA 能力
镜像验证/加载	读取、校验、解压或复制内核/程序	入口地址、参数和内存布局
控制权转移	设置寄存器、栈、页表或模式后 跳转	后续系统接管异常、中断 和设备

在微控制器上，固件和“应用”可能就是同一个镜像；在 PC 或服务器上，固件、引导加载器、内核和用户程序分层明显；在 SoC 上，还可能有 Boot ROM、一级引导、可信固件、安全监控和普通操作系统。层数不同，但每一层都要把可执行地址、栈、异常入口、内存属性和设备所有权交代清楚。

安全启动把第一条指令也纳入信任链 现代 SoC 常把 Boot ROM 固化在芯片内部，复位后先执行不可修改的 ROM 代码。Boot ROM 读取下一阶段固件，验证签名、版本和配置，再决定是否跳转。这样做不是密码学装饰，而是硬件启动约定的一部分：如果第一段可变固件不可信，后续页表、设备配置、密钥和操作系统都可能被篡改。

安全启动对象	硬件/固件动作	失败处理
Boot ROM	从固定复位入口执行，不可现场 改写	提供根信任和最小验证代 码
公钥/哈希 镜像签名	存在熔丝、ROM 或安全存储中 覆盖代码、配置和版本信息	用于验证下一阶段镜像 验证失败则停止、降级或 进入恢复模式
回滚保护	记录允许的最小版本	防止加载旧漏洞固件

调试锁定 限制 JTAG、串口下载或内存读出 防止启动后被外部接口改写控制流

这也解释了为什么“硬件从零到整机”不能只讲电路能跑。整机一旦能取指、访存和访问设备，就必须回答谁能改代码、谁能访问内存、谁能发起 DMA、谁能接收中断、启动镜像是否可信、错误路径是否可恢复。经典芯片提供了这些问题的历史剖面，现代平台则把它们推向更复杂的安全和系统集成边界。

案例：一个裸机镜像从链接地址跑到 main 假设一块教学板把 Flash 映射到 `0x00000000`，SRAM 映射到 `0x20000000`，GPIO 映射到 `0x40000000`。链接脚本规定向量表和代码放在 Flash，已初始化数据运行在 SRAM，但初始内容存放在 Flash 镜像中，BSS 在 SRAM 中清零，栈顶放在 SRAM 高地址。这个布局看起来是软件文件，实际上直接决定复位后每一次取指、加载和存储的地址。

```
memory map:
0x00000000-0x0003FFFF Flash: vectors, code, const, data image
0x20000000-0x20007FFF SRAM: data, bss, stack
0x40000000-0x40000FFF GPIO MMIO
```

镜像区域	链接/装载含义	硬件事务
向量表	复位入口和异常入口地址	复位时取入口，异常时取向量
<code>.text</code>	指令编码放在 Flash	PC 取指访问 Flash 或映射后的代码区
<code>.rodata</code> <code>.data</code> 镜像	常量和只读表 初始值存放在 Flash	加载读取，通常禁止写 启动代码复制到 SRAM 运行地址
<code>.bss</code>	运行时应为 0 的对象	启动代码对 SRAM 执行写操作清零
栈	函数调用和异常保存上下文	SP 指向 SRAM 可写区域

复位后第一段代码不应依赖尚未初始化的全局变量，也不应调用需要完整栈和运行时的复杂函数。它要先建立最小可运行边界。

```
reset trace:
PC = vector_table.reset_handler
SP = _stack_top
copy bytes from _sidata in Flash to _sdata in SRAM
zero bytes from _sbss to _ebss
configure clocks and essential wait states
set vector table base if architecture supports it
call main
```

启动动作	不能省略的原因	常见错误
设置 SP	后续 CALL、PUSH、异常保存需要栈区	SP 指到 Flash 或未映射区
复制 <code>.data</code>	全局初值要出现在 RAM 运行地址	变量初值全错或仍为 Flash 内容
清 <code>.bss</code>	语言和固件约定零初始化	状态机初值随机，指针非零
配置等待状态	Flash、SRAM 或外设满足时序	高频下取指或访问偶发失败
设置向量表	中断和异常能进入正确入口	第一场中断跳到错误地址

跳 <code>main</code>	进入应用主逻辑	入口地址或模式错误导致程序失控
---------------------	---------	-----------------

案例：保护模式最小启动清单 以 80386 风格机器为例，若要从实模式进入带分页的保护环境，必须先建立一组硬件可读的表和寄存器状态。这个过程常被写成启动代码技巧，但本质是把 CPU 的地址解释约定从旧模式切换到新模式。

步骤	硬件目的	预期行为
建立 GDT	给代码段、数据段、栈段描述符	描述符 base/limit/type/present 正确
建立 IDT	给异常和中断入口描述符	页错误、非法指令等入口 可达
加载 GDTR/IDTR	CPU 知道描述符表位置	表所在内存可读，界限正 确
打开保护模式位	改变段选择子解释方式	远跳转后 CS 使用新描述 符
重装段寄存器	DS/SS/ES 等进入新约定	数据和栈访问按新描述符 解释
建立页目录/页表	线性地址可转换到物理地址	取指、栈、异常代码均被 映射
加载 CR3 并开启分页	CPU 开始查页表和 TLB	缺失映射会触发页错误而 非随机访问

最小保护模式 bring-up 应先保守映射。把当前代码、栈、页表、GDT、IDT 和 VGA/串口调试 MMIO 映射好，再打开分页；确认能打印或点亮调试输出后，再逐步收紧权限。若一开始就建立复杂高半区映射、多个页大小、用户/内核分离和设备属性，任何一个地址错误都可能导致连续异常，难以定位。

```
protected-mode checklist:
GDT covers code/data/stack selectors
IDT covers at least fault handlers
stack is writable after mode switch
current instruction stream is mapped
page tables themselves are mapped
debug output MMIO is mapped uncached
fault handlers do not fault recursively
```

案例：从异常号定位启动失败 启动阶段最可怕的现象是“黑屏”或“无输出”。更有效的做法是把失败分层。若能进入非法指令异常，说明取指至少到了某条不合法编码；若能进入页错误，说明保护模式和 IDT 至少部分工作；若连续进入双重错误或复位，说明异常处理路径本身可能失败；若完全没有总线活动，可能是复位、时钟或复位向量映射问题。

现象	可能边界	优先观察
无取指总线活动	复位、时钟、复位向量、ROM 片选	示波器看时钟/复位/地址线
取指后立即异常	编码、模式、段描述符或权限错误	IR/PC、异常号、错误码
页错误循环	页表缺映射、权限错误、异常入口未映射	故障地址、PDE/PTE、CR3
中断后死机	栈、IDT、EOI 或清中断顺序错误	SP、向量号、中断控制器状态

DMA 后数据错	cache 可见性、IOMMU、缓冲区所有权	描述符、completion、cache 维护记录
----------	------------------------	---------------------------

这张表把整本书的排查方法统一起来：不要先猜“CPU 坏了”，先问硬件行为停在哪个边界。电源和时钟是第一章的边界，触发器和状态是第二章的边界，ALU 和控制线是第三章的边界，PC/IR/寄存器是第四章的边界，主存/总线/MMIO/DMA 是第五章的边界，描述符、页表、固件和平台是第六章的边界。

案例：把教学板扩展成一台小系统 最后给一个可以落地的整机扩展路线。第一阶段只跑 ROM 中的 LED 循环；第二阶段加入 SRAM 和栈，能调用函数；第三阶段加入定时器中断，能周期性切换状态；第四阶段加入简化 DMA，把一段内存复制到另一段；第五阶段加入保护或至少地址错误检测，让未映射访问进入错误入口。每一阶段都对应书中一个硬件层次。

阶段	新增硬件/固件	预期行为
ROM LED	复位向量、ROM、GPIO MMIO	上电后固定节奏改写 LED 寄存器
SRAM 栈	SRAM 译码、SP 初始化、CALL/RET	函数调用和局部变量不破坏全局数据
定时器中断	定时器、向量表、中断清除	周期事件不丢失，主循环可继续运行
简化 DMA	描述符、源/目标地址、completion	DMA 完成后 CPU 读到正确内存内容
错误入口	未映射检测、总线超时或简化异常	错误地址进入可观察处理程序

这条路线比“一步到位跑操作系统”更适合硬件书。读者每完成一层，都能指出新增了哪些地址、哪些控制线、哪些状态边界、哪些错误路径。等这些边界稳定后，再看 8086、80386 或现代 SoC，就不是背芯片史，而是在识别同一组问题如何被真实平台放大和工程化。

综合项目：写一份最小整机规格 本章最后可以把全书内容收束成一份最小整机规格。规格不需要一开始很复杂，但必须让别人能按它搭出同一台机器，并能判断每个访问是成功、等待还是错误。下面给出一个建议模板，读者可以用分立逻辑、FPGA、微控制器外设模拟或软件仿真来实现。

规格项目	必须写清楚	预期行为
地址空间	ROM、RAM、MMIO、未映射区范围	任意地址最多一个目标响应
复位行为	PC 初值、SP 初值、控制线安全值	上电第一条取指可重复观察
指令集	编码、寄存器字段、立即数扩展和标志规则	每条指令能写成控制 trace
内存访问	字节序、对齐、字节使能、等待和错误	加载/存储不破坏相邻字节
设备寄存器	每个位的读写、副作用和清除规则	GPIO/定时器/UART 状态可验证
中断/异常	向量入口、保存状态、清除和返回规则	外部事件和错误能进入可诊断路径
DMA/队列	描述符格式、所有权、completion 和 cache 规则	CPU 与设备不同时拥有缓冲区

规格写完后，建议按下面的顺序实现，而不是同时接上所有模块。

里程碑	要实现的最小能力	预期行为
M1	复位后从 ROM 取指，PC 顺序前进	逻辑分析仪能看到连续取指地址
M2	ADD/SUB/JNZ 正确执行计数循环	寄存器值和分支方向符合 trace
M3	SRAM 加载/存储和栈可用	函数调用能返回，局部变量不乱
M4	GPIO MMIO 可读写	LED/按键通过地址事务控制
M5	定时器中断可进入和返回	中断不破坏主循环寄存器和栈
M6	简化 DMA 或块复制控制器可搬数据	completion 后内存内容正确
M7	未映射访问进入错误入口	错误地址和原因可观察

每个里程碑都应保留一个最小测试程序。不要只保留最终大程序，因为最终程序一旦失败，无法判断是取指、译码、ALU、存储器、MMIO、中断还是 DMA 出错。小测试程序就是硬件书里的单元测试。

```
M2 counter loop:
    MOV R1, 0
    MOV R2, 10
loop:
    ADD R1, 1
    SUB R2, 1
    JNZ R2, loop
    MOV [GPIO_OUT], R1
```

测试失败现象	优先怀疑	观察信号
PC 不变	复位、时钟、PC 写使能或取指等待	PC、reset、clock、ready
循环次数错	立即数扩展、SUB、Zero 标志或 JNZ	R2、Zero、pc_sel、branch_target
GPIO 不变	地址译码、MMIO 写使能、字节使能	address、io_cs、mem_write、write_data
函数返回错	SP、CALL/RET 返回地址、栈 RAM	SP、栈内容、PC、writeback
中断后错乱	保存/恢复寄存器、EOI、清状态顺序	vector、SP、saved PC、irq status
DMA 数据错	描述符、地址权限、cache 可见性、completion	desc、buffer、owner、done

这个综合项目的目的，是让读者把书中的概念变成可执行机器：第一章的电平和输出驱动，第二章的状态边界，第三章的 ALU 和控制字，第四章的 CPU trace，第五章的地址事务和 DMA，第六章的启动、异常和平台约定。能写出并验证这份规格，才算真正把“硬件从零到整机”连起来。

综合项目的实验报告模板 为了避免实验结果只停留在“好像能跑”，每个里程碑都应留下实验报告。报告不需要长篇叙事，但必须保存足够的观察记录，让另一个人能复现同样结论。建议每个实验至少记录以下字段：

报告字段	要记录的内容	为什么重要
------	--------	-------

硬件版本	原理图版本、器件型号、时钟频率、电源电压	不同器件和频率会改变时序结论
固件版本	ROM 镜像、链接地址、入口地址、编译选项	同一硬件跑不同镜像可能表现不同
地址地图	ROM/RAM/MMIO/未映射区域范围	后续访问错误可直接对照
测试程序	最小汇编或机器码列表	能定位具体指令和控制 trace
观察信号	PC、IR、控制字、地址、数据、ready、写使能	证明结果来自正确硬件路径
预期结果	预期寄存器、内存、设备和异常状态	避免把偶然现象当成通过
失败记录	失败现象、假设、修改和复测结果	保留完整调试记录

例如 M3“SRAM 栈可用”的实验报告应包含：复位后 SP 的值，CALL 前后的 PC，栈上保存的返回地址，函数内部局部变量地址，RET 后 PC 是否回到调用点，R0 或指定返回值寄存器是否正确。若只写“函数调用成功”，后续出现异常时无法判断是栈、返回地址、寄存器保存还是取指路径出了问题。

```
M3 trace example:
reset PC = 0x00000000
initial SP = 0x20008000
CALL writes return address 0x00000124 to [SP-4]
callee uses [SP-8] for local variable
RET loads PC = 0x00000124
R0 = expected return value
```

这类报告会把学习从“看懂章节”推进到“能维护机器”。硬件工程的核心不是一次点亮 LED，而是每次改动后仍能解释：哪个状态边界改变了，哪个事务完成了，哪个错误路径被覆盖了，哪些观察结果足以支持结论。

专题：从 8086 到 SoC 的“同一问题不同边界” 8086、80386、Pentium、微控制器和现代 SoC 看起来差别很大，但平台工程反复面对同一组问题：第一条指令从哪里来，地址如何到达目标，慢设备如何等待，错误如何上报，外设如何通知 CPU，DMA 与 cache 如何保持可见，调试器如何观察状态。演进改变的是边界位置，而不是问题本身。

平台问题	早期芯片形态	现代 SoC 形态
复位入口	外部 ROM 响应固定复位向量	片内 Boot ROM、熔丝、启动介质选择和安全检查
地址译码	板级译码器产生片选	片上互连、地址防火墙、IOMMU 和外设桥
等待状态	READY/WAIT 拉长总线周期	valid/ready、AXI response、QoS 和超时
中断	外部中断控制器或简单向量	GIC/APIC、优先级、亲和性、虚拟化中断
DMA	简单外设总线主控	多主设备、cache 一致性、SMMU/IOMMU 权限
调试	逻辑分析仪看总线引脚	JTAG/SWD、trace macrocell、性能计数器和固件日志

这张对照能帮助读者读芯片手册。看到 AXI、APB、GIC、IOMMU、Boot ROM、secure monitor 时，不应把它们当成孤立名词，而应问：它们分别把早期平台的哪一条裸露信号或板级功能搬进了芯片内部？搬进去之后，哪些行为变得更快、更安全，哪

些调试信息变得更难直接观察？

专题：BIOS、Boot ROM 和 loader 的最小移交清单 启动链路不是“跳到下一段代码”这么简单。每一级启动代码都必须向下一级交出可用的执行环境：当前特权级、栈、内存可用范围、异常入口、时钟、串口或日志设备、镜像位置和校验结果。缺少任何一项，下一阶段可能运行一段时间后才以难以定位的方式失败。

移交项目	必须说明的内容	失败现象
入口地址	下一阶段第一条指令位置和执行状态	跳入错误模式或错误对齐地址
栈	栈顶、增长方向、可用范围	CALL/异常保存破坏数据或 ROM 区
内存地图	RAM、ROM、MMIO、保留区、安全区	loader 覆盖设备寄存器或固件数据
异常向量	向量基址、异常栈、默认处理器	早期错误无法诊断，直接死机
时钟/电源	CPU、总线、外设时钟是否打开	访问外设永久等待或超时
日志通道	UART、半主机、内存日志或调试口	失败时没有可观察的输出
镜像验证	长度、哈希、签名、加载地址	执行损坏或不可信代码

安全启动只是把这张移交清单加上信任约束：Boot ROM 信任芯片内固化公钥或熔丝状态，验证下一阶段签名，再把控制权交给经过验证的镜像。若验证失败，平台不能继续执行任意代码，而应进入恢复、锁定或错误报告路径。这样读者能把“安全启动”理解为启动链路中的状态机和逐级验证过程，而不是抽象口号。

案例：一次 int 13h 磁盘读取的流程 实模式下的 PC 固件不只负责启动，还向上层软件提供一组设备服务。磁盘读写服务挂在中断号 0x13 上：调用者把功能号和参数放进寄存器，执行 int 0x13，BIOS 中的磁盘服务例程接管 CPU，完成与磁盘控制器的全部交互，再把结果按约定交还。以经典的 CHS 读扇区为例，调用约定如下表。

寄存器	调用时放入的值	补充约定
AH	功能号，0x02 表示读扇区	返回时变为状态码，0 表示成功
AL	要读的扇区数	一次调用不宜越过当前磁道剩余扇区
CH	柱面号低 8 位	10-bit 柱面号的高 2 位挤在 CL 高 2 位
CL	低 6 位为扇区号	扇区从 1 开始编号，没有 0 号扇区
DH	磁头号	双面软盘取 0 或 1
DL	驱动器号	0x00 为第一张软盘，0x80 为第一块硬盘
ES:BX	缓冲区段地址与偏移	必须落在 1 MiB 实模式空间内，软盘 DMA 时不可跨越 64 KiB 边界

一次完整的调用写成汇编是这样的。

```
; read one sector from the first hard disk, CHS = (0, 0, 1)
; buffer at ES:BX = 0x0000:0x7E00
mov ah, 0x02      ; function: read sectors
```

```

mov al, 1          ; sector count
mov ch, 0          ; cylinder low 8 bits
mov cl, 1          ; sector 1, cylinder high bits 0
mov dh, 0          ; head 0
mov dl, 0x80       ; first hard disk
mov bx, 0x7E00     ; buffer offset, ES already 0
int 0x13
jc disk_error      ; CF=1 means failure, AH holds status code

```

`int` 指令把标志和返回地址压栈、按向量表跳到 BIOS 例程之后，剩下的工作由固件完成：它把 CHS 地址翻译成磁盘控制器的命令序列，对软盘借助 DMA 控制器把 512 字节直接搬进内存，对硬盘按当时平台的能力用 PIO 或总线主控方式传输；传送结束后，BIOS 检查控制器状态寄存器，把结果写成“进位标志 CF 为 0 表示成功、为 1 表示失败且 AH 保存状态码”的约定，再用 `iret` 返回调用者。对调用者而言，整件事只表现为一条中断指令和一段被填好的缓冲区：控制器端口地址、DMA 通道编程、出错重试全部藏在 ROM 里。

这正是“固件提供设备驱动”的早期形态：硬件细节被封装在一组寄存器约定接口后面，boot 扇区代码不必知道磁盘控制器长什么样也能读盘。代价同样明显：BIOS 例程工作在实模式，缓冲区被限制在 1 MiB 以下的常规内存，传输方式以当时的 DMA 和 PIO 为上限，接口能力跟不上后来的大容量硬盘和高速控制器。操作系统接管平台后，会用自带的驱动替换这层固件服务——内核直接编程磁盘控制器，从 IDE、AHCI 一路演进到 NVMe，用自己的缓冲区和队列调度 I/O；`int 0x13` 只在启动早期、内核驱动就绪之前承担读盘任务。这条替换关系把“从固件到操作系统的所有权转移”具体到了每一个设备服务入口。

专题：平台 bring-up 日志怎样定位到具体层 bring-up 的第一原则是每一步只引入一个新假设，并记录足够的观察结果。若一开始就运行完整系统，失败时无法知道是时钟、复位、内存、cache、异常、串口、设备树、驱动还是文件系统出错。更稳妥的办法是从复位向量开始逐层扩大可用机器。

阶段	建议日志	证明的边界
复位	PC、模式、第一条指令字	ROM 映射和取指可用
串口	时钟源、波特率、发送寄存器状态	MMIO、外设时钟和基本输出可用
SRAM	写读 walking bit、地址线测试	片内 RAM 和数据访问可用
DRAM	初始化参数、训练结果、块读写校验	外部内存控制器和板级连线可用
异常	故障地址、异常号、返回 PC	向量表和异常栈可用
cache/MMU	页表摘要、属性、TLB flush 记录	地址转换和一致性边界可控
中断	控制器配置、pending/active 状态	异步事件链路可用

每条日志都应能回答“这条记录来自哪里”。例如串口能打印字符，不只证明 C 函数被调用，还证明 CPU 能取指、写 MMIO、外设时钟打开、引脚复用正确、波特率近似正确。若后续打开 cache 后打印乱码，就可以把变化集中到 cache 属性、写缓冲、MMIO ordering 或时钟切换，而不是重新怀疑所有层。

专题：把经典芯片放进同一张学习路线 本章不要求记住每颗芯片的全部引脚，而要求形成可迁移的读法。读 6502 时看复位向量、零页和简单总线；读 8086 时看段：偏移、复用总线和字节存储体；读 80386 时看保护模式、分页和异常；读 Pentium 时看流水线、cache 和总线协议；读 SoC 时看片上互连、启动 ROM、DMA、一致性和安全边界。

阅读对象	优先抓住的问题	可迁移能力
6502/Z80/8051	复位、寄存器、地址空间、简单外设	从最少状态读懂机器执行
8086	分段、外部总线、READY、板级译码	把 CPU 周期映射到平台信号
80386	特权级、描述符、分页、异常	理解操作系统为什么需要硬件保护
80486/Pentium	cache、流水线、总线事务	解释性能和一致性不再只是 ISA 问题
现代 SoC	Boot ROM、片上互连、DMA、安全、调试	把整机看成多主设备协作系统

这样安排后，经典芯片不是怀旧材料，而是复杂系统的分层样本。每前进一步都在回答同一件事：更多性能、更多外设、更多安全约束进入系统后，哪些状态必须被显式保存，哪些事务必须被确认完成，哪些错误必须变成可诊断信息。

专题：RISC-V 与 x86 的两种指令集风格 本章读过的 8086 到 Pentium 都属于 x86 家族；把它和 RISC-V 放在一起对照，能看清指令集设计的两条路线。RISC-V 是定长的 load/store 架构：基本指令一律 32-bit 宽，只有加载和存储指令访问内存，运算指令的源和结果都在寄存器中，通用寄存器有 x1-x31 共 31 个（x0 恒为 0）。x86-64 则是变长的寄存器-内存混合架构：指令长度从 1 字节到 15 字节不等，一条运算指令可以直接拿内存操作数参与计算，通用寄存器只有 16 个。同一段数组求和循环，两种风格写出来差别很直观。

```
# RISC-V (RV64): a0 = base, a1 = n, sum in a2
    mv    a2, zero           # sum = 0
    mv    t0, zero          # i = 0
loop: bge t0, a1, done       # if i >= n, exit
    slli  t1, t0, 3          # offset = i * 8
    add   t1, a0, t1         # address of a[i]
    ld    t2, 0(t1)         # load a[i]
    add   a2, a2, t2         # sum += a[i]
    addi  t0, t0, 1          # i++
    j     loop
done:
```

```
; x86-64 (Intel syntax): rdi = base, rsi = n, sum in rax
    xor   rax, rax           ; sum = 0
    xor   rcx, rcx           ; i = 0
loop: cmp  rcx, rsi
    jge   done              ; if i >= n, exit
    add   rax, [rdi + rcx*8] ; sum += a[i], memory operand
    inc   rcx               ; i++
    jmp   loop
done:
```

风格差别首先体现在译码器上。RISC-V 译码器拿到 4 字节就能确定操作码、寄存器字段和立即数位置，取指边界固定，多条指令可以成组并行译码；x86 译码器要先扫描前缀、确定指令边界，才知道操作码和操作数在哪里，变长边界识别本身就是一长串组合逻辑，现代 x86 靠预译码标记和微操作 cache 把这笔开销藏起来——Pentium Pro 以来“先译成微操作再乱序执行”的结构，很大程度上就是为了消化变长指令。其次体现在编译器上：load/store 架构把“算”和“访存”拆成两条指令，编译器要把加载尽量提前发射来隐藏内存等待，寄存器多则溢出到栈上的次数少；x86 的内存操作数把加载和加法折叠在一条指令里，代码更短、寄存器压力更小，但这

条指令在片内仍要被拆成加载微操作和加法微操作，只是拆分者从编译器换成了硬件。最后是编码密度与译码功耗的取舍：定长指令浪费一些编码空间，换来简单的取指和译码路径；变长指令节省空间和取指带宽，换来更复杂的译码前端。两种风格都能做出高性能实现，差别在于复杂度放在译码硬件里，还是放在编译器调度里。

专题：ARM 把条件执行和移位操作数带进指令 经典的 32-bit ARM 指令集（ARM 状态）走的是 x86 与 RISC-V 之间的另一条路：几乎每条数据处理指令都可以带一个条件码后缀，第二个源操作数可以先经过桶形移位器再参与运算。条件码占指令编码最高的 4 bit，取指和译码路径不变，执行时先检查程序状态寄存器的标志：条件不成立，这条指令直接变成空操作，不写结果、不改标志。短小的条件赋值因此不必跳来跳去，`moveq`、`addne` 这样的指令让短分支在流水线里原地消失。

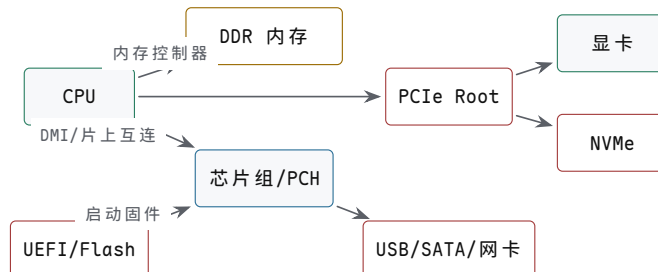
```
@ ARM: if (r1 == r2) r0 = 1 else r0 = r0 + 1
    cmp    r1, r2
    moveq  r0, #1           @ executes only when Z = 1
    addne  r0, r0, #1       @ executes only when Z = 0

@ ARM: r0 = r1 + (r2 << 2) in one instruction
    add    r0, r1, r2, lsl #2
```

第一个例子里，条件成立与否都会顺序取完这三条指令，没有分支方向要预测，短 `if` 的代价从“可能猜错分支、冲刷流水线”降为“多占一拍执行槽”；第二个例子里，桶形移位器放在 ALU 第二操作数的入口前，`lsl #2` 不占额外指令，数组下标乘 4、乘 8 这类地址计算被折进单条运算指令。

和 x86-64 对照，能看出设计取向的异同。x86-64 用 `cmov` 实现条件赋值，思路与条件执行相同——都是避免短分支——但只有少数指令形式支持，其余场合要靠编译器权衡“分支可能猜错”还是“两条路径都算一遍”；x86 的运算指令不带移位操作数，但复杂寻址模式 `[base + index*scale + disp]` 把“下标乘 1/2/4/8、加基址、加偏移”折进了访存指令，与 ARM 的移位操作数解决的是同一类地址计算问题，只是一个挂在加载/存储的地址端，一个挂在 ALU 的数据端。值得注意的是，ARM 自己的 64-bit 架构（AArch64）放弃了几乎全集的条件执行，只保留条件选择等少数形式：每条指令都带条件检查，执行控制更复杂，条件码字段还占去本可用于扩大寄存器编号或操作类型的编码位。这些取舍说明，指令集里没有免费的特性，每一项便利都要在别处付出代价。

专题：主板不是插槽集合，而是平台约定 主板把 CPU、内存、固件、电源、时钟、PCIe 设备、USB/SATA/NVMe、显卡、网卡和低速管理控制器连成一个可启动平台。它的核心职责不是“提供很多接口”，而是保证复位后每个设备处于可枚举、可供电、可配置、可报告错误的状态。CPU 能跑程序只是平台的一部分；内存训练、PCIe 枚举、电源时序、时钟分配、固件配置和热管理同样决定整机是否可靠。



图示：PC 主板的关键不是“很多线”，而是启动、枚举、供电、时钟和错误报告共同构成平台约定。

主板对象	负责的问题	典型失败表现
供电 VRM	把电源转换成 CPU/GPU/内存所需电压，并按时序上电	复位不释放、负载变化时掉压、机器重启

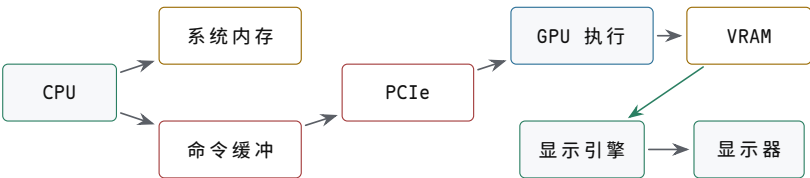
时钟源	给 CPU、内存、PCIe、USB 等提供参考时钟	链路训练失败、设备枚举不稳定
固件 Flash	保存 UEFI/BIOS、微码、平台配置	无法取第一阶段代码或配置错误
DDR 通道	连接内存颗粒或 DIMM，完成训练和 ECC	内存训练失败、随机数据错误
PCIe Root	枚举显卡、NVMe、网卡和扩展设备	设备消失、降速、AER 错误
低速管理	Super I/O、EC、传感器、风扇、RTC	键盘、风扇、温度或睡眠唤醒异常

专题：PCIe 是分组互连，不是并行总线升级版 早期外部总线常暴露地址线、数据线、读写控制和 READY；PCIe 则把事务封装成包，在点对点串行链路上传输。CPU 或芯片组通过 root complex 发起配置、内存读写和消息事务；设备通过 BAR 暴露 MMIO 窗口，通过 MSI/MSI-X 发送中断，通过 DMA 访问主存。PCIe 的关键边界包括链路训练、配置空间、BAR 映射、事务层包、完成包和错误报告。

PCIe 概念	含义	本书对应概念
配置空间	枚举设备、读取 ID、配置 BAR 和能力	地址地图和平台发现
BAR	把设备寄存器映射到 CPU 地址空间	MMIO 副作用和访问属性
Memory Read/Write TLP	设备或 root 发起的内存事务包	总线事务和 completion
MSI/MSI-X	设备通过写内存样式消息触发中断	中断不一定是独立引脚
AER	高级错误报告	错误路径可诊断

把 PCIe 讲清楚后，显卡、NVMe、网卡都可以归入同一模型：先由平台枚举设备并分配地址窗口；驱动把命令写入队列或 MMIO；设备通过 DMA 读写主存；完成后写 completion 或发 MSI；错误通过状态寄存器、completion 状态或 AER 上报。这样读者不会把“插在主板上的设备”理解成 CPU 的附属品，而会把它看成能主动访问内存的总线主设备。

专题：显卡的核心不是显示器接口，而是并行处理器加显存系统 显卡既是显示设备，也是大规模并行计算设备。GPU 内部有许多执行单元、调度器、寄存器文件、L1/L2 cache、纹理/采样单元、光栅化和显示引擎；外部或封装上有高带宽显存，例如 GDDR 或 HBM。CPU 不会逐像素把画面写到显示器，而是准备命令缓冲区、资源描述、纹理和顶点数据，GPU 读取这些对象并在显存中生成帧缓冲，显示引擎再按扫描时序把帧送到显示接口。



图示：GPU 数据路径：CPU 提交命令，GPU 经 PCIe/内存读取资源，在显存中生成帧，显示引擎独立扫描输出。

显卡对象	硬件含义	常见误解
命令缓冲	CPU 写入、GPU 异步消费的工作队列	不是 CPU 直接调用 GPU 函数

VRAM	GPU 近处的大带宽存储系统	不是普通 DRAM 的别名，带宽和访问模式不同
着色/计算单元	大量相似线程并行执行	不是少数超快 CPU 核心
纹理/cache 单元	优化二维或三维局部访问和采样	不等于通用 cache 的简单复制
显示引擎	按固定时序读取帧缓冲并输出信号	渲染完成和显示扫描是两个边界

GPU 的性能问题也不同于 CPU。CPU 常追求低时延和强单线程控制流；GPU 常追求吞吐，用大量并行线程隐藏显存等待。分支发散、访存不合并、显存带宽不足、CPU/GPU 同步过多，都会让 GPU 性能掉下来。理解显卡时应问：命令何时提交，资源何时对 GPU 可见，GPU 何时完成，帧缓冲何时可显示，CPU 是否过早读回结果。

专题：整机案例：从打开程序到读取文件和显示图像 一个真实用户动作会穿过多种硬件。用户打开程序，操作系统可能从 SSD 读取可执行文件和页面；CPU 触发块设备队列；NVMe 控制器 DMA 数据到主存；页表和 cache 让 CPU 取指执行；程序提交图形命令；GPU 读取资源、渲染到 VRAM；显示引擎扫描帧缓冲。这里没有哪个部件可以单独解释全程，必须把 CPU、内存、SSD、PCIe、GPU、显示和中断连成 trace。

阶段	硬件动作	完成标志
缺页/读文件	OS 提交 NVMe 读请求，设备 DMA 到主存	块 I/O completion 和页状态有效
执行代码	CPU 取指、TLB/cache 查找、分支和写回	指令提交，异常路径为空
准备图形资源	CPU 写命令缓冲和资源描述符	命令队列对 GPU 可见
GPU 渲染	GPU 读取资源，执行 shader，写 VRAM	fence 或 completion 表示渲染完成
显示扫描	显示引擎按刷新时序读取帧缓冲	vblank、翻页或显示控制器状态

这个案例可以作为读完整本书后的纸面项目。要求不是实现操作系统和驱动，而是把每一步的机器行为写清楚：谁拥有缓冲区，谁发起 DMA，哪个地址空间有效，cache/TLB 在哪里参与，哪个 completion 才说明完成，哪个错误会阻止下一阶段。

专题：用仿真器观察硬件行为 纸面规格之外，QEMU 这类全系统仿真器给了读者另一双眼睛。它把目标机器的指令逐块翻译成宿主机能执行的操作：目标机的寄存器是宿主机内存里的数据结构，目标机的物理地址空间是一张查找表，读 RAM 变成读宿主机上的数组，写 MMIO 地址则变成对一段模拟设备代码的调用，由它更新设备状态、必要时触发模拟中断。于是“一台 PC”可以只是宿主机上的一个进程，但它照样从复位向量取指、跑 BIOS、读磁盘镜像。

仿真器在教学上的价值，恰恰是把真实硬件里看不到的状态变得可见。接上调试器（QEMU 内置 monitor 或 GDB 调试桩）后，读者可以从复位后的第一条指令开始单步，看 BIOS 怎样建立段描述符和中断向量，看 boot 扇区怎样用 `int 0x13` 读盘；可以在任意时刻打印全部寄存器和任意一段物理内存，改一个字节再继续运行，直接观察“数据段复制错了”会长成什么样；还可以故意让代码访问未映射地址，看仿真器报告总线错误或目标机异常，而不必担心烧坏任何东西。这类实验把本章反复使用的 trace 方法从纸面搬到了可运行的机器上。

但仿真结果不能全部当成硬件结论。第一，仿真器按功能正确性建模，不按时间建模：一次访存在真实 DRAM 上要等几十纳秒，在仿真器里和一次寄存器加法几乎一样快，cache miss、等待状态、DRAM 刷新和总线仲裁的时序开销都不存在，性能结论无从谈起。第二，电气层面的问题整个不可见：复位时序、电源爬升、信号完整

性、三态冲突、虚焊和接触不良，这些真实 bring-up 中最常见的故障在仿真器里没有对应物。第三，设备模型是简化的，作者没有建模的寄存器行为和时序敏感的错误路径，仿真器可能永远表现为“工作正常”。所以仿真器和真实硬件是互补关系：仿真器适合回答地址地图、启动顺序、异常路径这类“逻辑对不对”的问题；时序裕量、电气可靠性这类“电路稳不稳”的问题，仍要回到真实板级平台，用示波器和日志回答。

专题：纸面项目应替代不可执行的大实验 如果没有硬件平台，仍然可以做高质量项目。项目形式应从“搭出来能跑”改成“规格写得可检查”。例如给出一台简化 PC：一个 CPU、两级 cache、DDR 控制器、PCIe root、NVMe、GPU、USB 控制器和固件 ROM。读者要写地址地图、启动顺序、设备枚举、一次文件读取、一次 GPU 命令提交、一次中断处理和一个错误路径。这样的纸面项目比空泛地说“了解主板”更有效。

纸面项目	必须交付	合格标准
简化 PC 地址地图	DRAM、固件、MMIO、PCIe BAR、未映射区	任意地址有明确目标或错误
NVMe 读路径	submission、doorbell、DMA buffer、completion、中断	CPU 不把门铃返回当成 I/O 完成
GPU 命令路径	命令缓冲、资源可见性、VRAM、fence、显示扫描	渲染完成和显示完成分开
主板启动路径	电源、时钟、复位、固件、内存训练、PCIe 枚举	每一步有失败后可诊断状态
错误注入	TLB miss、cache miss、PCIe 错误、SSD 超时、GPU hang	错误归属到具体层，不混成“程序卡住”

章末练习

任务	要求	学习目标
8086 地址形成	用段寄存器和偏移计算一个物理地址	能说明 20-bit 地址来自硬件加法路径
复用总线	写出 8086 读周期中地址、ALE、数据方向和 READY 的阶段	能指出何时锁存地址、何时采样数据
字节存储体	解释 A0 与 $\overline{\text{BHE}}$ 如何选择低/高字节	能处理奇地址字访问
80386 分页	把一个 32-bit 线性地址拆成目录索引、页表索引和页内偏移	能说明异常发生在真正访存前
保护模式入口	列出 GDTR/IDTR、CR0、CR3 和跳转刷新各自作用	能说明状态切换顺序
复位向量	比较 6502、Z80/8051、8086、80386 后续 x86 的复位入口	能说明第一条指令必须由非易失存储响应
平台分层	说明 CPU 本地总线、内存/cache 控制、高速扩展、低速 I/O 和固件层职责	能说明等待来自整个平台
SoC 演进	说明把内存控制器、I/O 和安全启动搬进片内的收益与代价	能指出片上互连和调试复杂度
主板结构	画出 CPU、DDR、PCH、UEFI Flash、PCIe、NVMe、GPU 的连接关系	能说明供电、时钟、复位和枚举不是 CPU 内部问题

PCIe 设备	说明配置空间、BAR、TLP、MSI 和 AER 各自解决什么问题	能把显卡/NVMe/网卡统一成总线主设备
显卡路径	写出 CPU 提交命令、GPU 读资源、VRAM 写帧、显示扫描的 trace	能区分渲染完成、DMA 完成和显示完成
6502 寻址与拍数	用零页寻址和 (zp),Y 间接变址写数组求和循环并逐拍计数	理解拍数就是总线事务序列
Z80 交换与刷新	说明 EXX 怎样加速中断响应、R 寄存器怎样把刷新藏进取指	寄存器组交换 = 不访存的上下文切换
486/Pentium 集成	列出 80486 和 Pentium 各自搬进片内的部件及对应收益	集成把等待从板级移到片内
6502 五拍 trace	写出 (zp),Y 间接变址读一个字节的 5 个时钟拍：每拍地址总线上是什么、CPU 内部在做什么	拍数能还原成总线事务序列
Z80 M/T 计数	数出 LD A,(HL) 和 EXX 各占几个机器周期、几个 T 状态，并指出 DRAM 刷新藏在哪一拍	M1 后半拍兼做刷新，不另占总线事务
页表 walk 演算	给定 CR3、两级页表内容和线性地址 0x00403ABC，写出 walk 每一步读到的地址和最终物理地址	一次转换是两次表项读取加一次数据访问
页错误诊断	给定故障地址、错误码和 PDE/PTE 权限位，判断是缺页、写只读页还是用户态越权	三问：存在否、读或写、谁发起
BIOS 四阶段	把 PC 从复位到交出 boot 扇区控制权的过程拆成四个阶段，写出每阶段完成的边界	每阶段有明确的完成标志和失败现象
RISC-V/x86 对照	用同一段求和循环说明定长 load/store 与变长内存操作数在译码和编译器上的差别	复杂度放在译码前端还是编译器调度里

参考解答与要点提示

任务	参考解答	必须掌握的要点
8086 地址形成	物理地址由 <code>segment*16+offset</code> 得到，内部编程模型保持 16-bit，外部地址能力扩展到 20-bit。	分段是地址形成硬件。
复用总线	T1 输出地址并用 ALE 锁存；T2 改变总线方向并给出读控制；T3/Tw 等待目标驱动数据且 READY 可延长；T4 采样并结束。	复用引脚必须分阶段解释。
字节存储体	偶地址低字节由 A0 选择，奇地址高字节由 BHE 参与选择；奇地址字访问可能拆成两个周期。	位宽不等于任意地址一次完成。
80386 分页	4 KiB 页下，高 10 bit 选 PDE，中 10 bit 选 PTE，低 12 bit 是页内偏移；PDE/PTE 不存在或权限不允许时触发异常。	地址转换也是硬件路径。

保护模式入口	进入保护模式前要准备描述符表、装载 GDTR/IDTR、设置 CR0、刷新取指路径；分页还需准备页目录/页表、装载 CR3、打开分页位。	状态切换顺序错误会让取指和异常入口失效。
复位向量	6502 从高端向量取入口，Z80/8051 常从 0 地址开始，8086 从 0xFFFF0 附近，后续 x86 从更高物理地址附近，SoC 可从 Boot ROM 进入。	启动入口是硬件规格。
平台分层	CPU 本地总线处理核心事务，内存/cache 控制处理 DRAM 和回填，高速扩展处理图形/磁盘/网络，低速 I/O 处理串口/键盘/RTC，固件负责初始化和启动选择。	主频之外还有平台等待。
SoC 演进	片上集成缩短常见路径、减少引脚和功耗，但引入片上互连、一致性、时钟/电源域、安全启动和调试复杂度。	低时延来自边界移动，不是所有路径变快。
主板结构	内存控制器在 CPU 内，CPU 直连 DDR 内存和 PCIe Root（显卡、NVMe 等高速设备）；PCH 经 DMI 与 CPU 相连，挂低速 I/O（串口、键盘、RTC）与固件 Flash；固件上电先执行并完成初始化。	平台分层是把不同速度的路径分开组织。
PCIe 设备	固件读配置空间枚举设备并分配 BAR 地址窗口，驱动经 MMIO 配置队列，设备用 DMA 搬数据，MSI/MSI-X 投递完成中断，AER 报告错误。	枚举-配置-DMA-完成是统一模型。
显卡路径	CPU 写命令环并按 doorbell，GPU 读资源和命令渲染，结果写 VRAM，显示控制器按扫描时序读出；渲染完成、DMA 完成、显示完成是三件不同的事。	三种完成语义不能混。
6502 寻址与拍数	LDA 零页 3 拍，(zp),Y 间接变址 5 拍（跨页 6 拍）；求和循环每轮用 (zp),Y 读一个元素，接 INY、BNE，拍数可手工核对。	简单数据通路让逐拍 trace 可行。
Z80 交换与刷新	EX AF,AF' 交换 AF 一对，EXX 一条指令交换 BC、DE、HL 三对，省去压栈；M1 取指后半段把 R 输出到地址总线完成 DRAM 刷新。	交换和借用都是减少总线事务。
486/Pentium 集成	486 搬入 cache、FPU、流水线；Pentium 搬入双发射、BTB 预测、分离 cache 和 64-bit 数据总线。	每一代都在移动等待的位置。
6502 五拍 trace	拍 1 取 opcode，拍 2 取零页操作数，拍 3、拍 4 从零页相邻两字节读出 16-bit 指针的低、高字节，拍 5 用指针加 Y 的有效地址读数据；低字节加法跨页时多 1 拍修正高字节，共 6 拍。	5 拍 = 2 拍取指、2 拍取指针、1 拍读数。
Z80 M/T 计数	LD A,(HL) 为 2 个机器周期共 7 个 T 状态（M1 取指 4T，存储器读 3T）；EXX 只有 M1，共 4 个 T 状态。刷新藏在每个 M1 的 T3、T4：指令字节已入 CPU，地址总线让给 R 寄存器输出刷新行地址。	T 状态计数能直接核对数据手册的时序图。

页表 walk 演算	以 CR3=0x00010000、PDE 给出页表基址 0x00020000、PTE 给出页框基址 0x00030000 为例：地址拆成目录索引 1、页表索引 3、偏移 0xABC；先读 PDE（地址 0x00010004），再读 PTE（地址 0x0002000C），最终物理地址为 0x00030000+0xABC=0x00030ABC。	TLB 未命中时，一次访问背后是一串总线读。
页错误诊断	错误码 P 位为 0 说明对应 PDE 或 PTE 的 present 为 0，是缺页，可分配页后重试；P 为 1 且 W/R 位为 1、而 PTE 的 RW 为 0，是向只读页写入；P 为 1、U/S 位为 1 且 PTE 的 US 为 0，是用户态访问特权页，应按策略拒绝或终止。故障线性地址由 CR2 给出。	错误码、CR2 和表项权限位三者要对照看。
BIOS 四阶段	阶段一，复位释放后 CPU 从地址空间顶端的复位向量取指，跳入 BIOS ROM；阶段二，POST 初始化内存、芯片组和基本外设，建立中断向量表与 BIOS 数据区；阶段三，BIOS 按启动顺序把启动介质第一个扇区（512 字节）读到 0x0000:0x7C00；阶段四，检查扇区末尾 0x55AA 签名，有效则远跳转到 0x7C00，交出控制权。	每阶段交出确定状态，下一段才有约定可依。
RISC-V/x86 对照	RISC-V 定长 32-bit、load/store、31 个自由寄存器：译码边界固定，循环中加载与加法各一条指令，编译器调度加载以隐藏等待；x86-64 变长 1-15 字节、16 个寄存器、运算可带内存操作数：同一循环指令更少更短，但译码器要先确定指令边界，内存操作数在片内被拆成加载微操作和加法微操作。	ISA 风格决定复杂度由译码前端还是编译器调度承担。

动手搭一台 8-bit 教学计算机

前面六章把每个部件都单独讲过：逻辑门、触发器、加法器、寄存器堆、控制字、总线、存储器和调试方法。本章只做一件事：把这些部件按一张确定的总图接成一台真的能执行程序的 8-bit 教学计算机。机器很小——8-bit 数据总线、16 字节地址空间、11 条指令（含 NOP）——但小是刻意的：小到每个控制信号都能用 74HC 器件数出来，小到任何故障都能用 LED 和单步时钟定位。这样的机器在教学文献里常被称为 SAP-1 类结构，它的价值不在性能，而在于“没有一处不可观察”。

先把整机规格定下来，后面每一节都只是这些规格的实现。

对象	规格	实现要点
数据总线	8-bit，三态共享	任何时刻只有一个驱动器
地址空间	16 字节（4-bit 地址）	程序和数据同处一块 RAM
寄存器	A、B、IR、MAR、OUT、PC、Flags	74HC273/173，带装载使能
ALU	加法和减法	74HC283 加法器，减法是取反加一
标志	Zero、Carry	供条件跳转使用
控制	微码 ROM 译码	opcode 加 T 状态索引微码
时钟	自动（555 振荡）和单步（按钮）	调试时逐拍推进

指令集只有 11 条（含 NOP），高 4 位是 opcode，低 4 位是地址或立即数。每条指令占一个字节，程序最长 16 条（剩余字节放数据）。这套编码与本章微码 ROM 的内容一一对应，后面 § 四会逐条展开微操作。

指令	opcode	含义	用到的主要路径
NOP	0x0	空操作	只占周期
LDA addr	0x1	$A \leftarrow \text{RAM}[\text{addr}]$	MAR、RAM 读、A 装载
ADD addr	0x2	$A \leftarrow A + \text{RAM}[\text{addr}]$	B 装载、ALU、标志
SUB addr	0x3	$A \leftarrow A - \text{RAM}[\text{addr}]$	B 取反加一、ALU
STA addr	0x4	$\text{RAM}[\text{addr}] \leftarrow A$	MAR、RAM 写
LDI imm	0x5	$A \leftarrow \text{imm}$	总线低 4 位、A 装载
JMP addr	0x6	$\text{PC} \leftarrow \text{addr}$	PC 装载
JC addr	0x7	Carry=1 时 $\text{PC} \leftarrow \text{addr}$	标志、PC 装载
JZ addr	0x8	Zero=1 时 $\text{PC} \leftarrow \text{addr}$	标志、PC 装载
OUT	0xE	$\text{OUT} \leftarrow A$	输出寄存器
HLT	0xF	停机	时钟停止

读本章的正确方式不是先看器件清单，而是先问：每条指令的每一个 T 状态，总线是谁在驱动、谁在装载、哪根控制线有效。能回答这个问题，面包板就只是这张表的物理拷贝。

核心思想

教学计算机的全部秘密在” 每一步可观察”：总线值用 LED 显示，时钟可以单步，控制线可以逐根核对。搭建顺序和调试顺序都围绕这个原则组织。

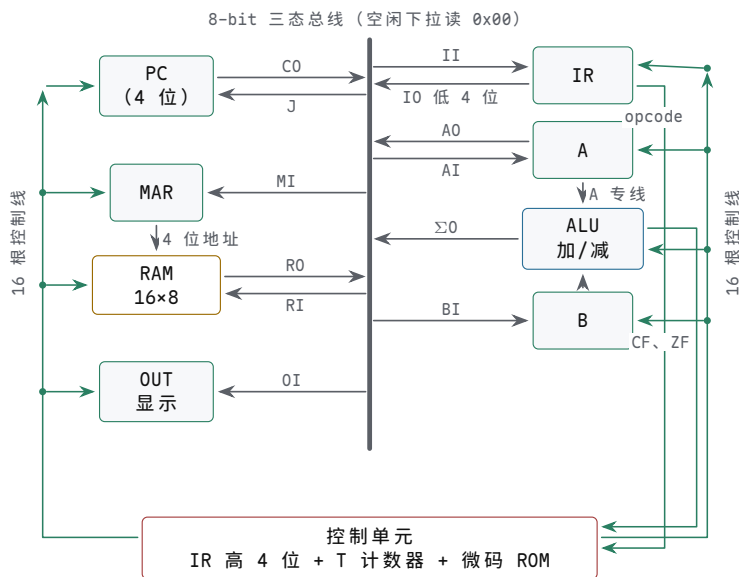
一、总体架构：总线、时钟与复位

整台机器在物理上只有三类东西：一条 8-bit 三态总线、挂在总线上的若干个功能模块，以及让一切按拍行动的时钟与复位。本节不谈任何一条指令，只回答三个搭建问题：模块在面包板上怎么摆，时钟脉冲从哪里来，上电以后机器怎样进入一个已知状态。这三个问题在原理图上几乎不占地方，在面包板上却决定了后面每一步调试是顺利推进还是原地打转。

本节按搭建的先后次序组织：先定总线这条中轴和它的接线约定，再解决时钟的两个来源，然后做复位链，最后把“每次上电先查什么”固定成一套顺序。次序本身就是设计的一部分：先被依赖的部件先查，每一步都建立在前一步已经正常的前提下。

面包板布局按总线居中组织 整机里流量最大的结构是总线：取指时 PC 的值经它去 MAR，取回的字节经它去 IR，运算时数据经它在 RAM、A、B 与 ALU 之间流动，停机前最后一个动作还是经它把 A 的值送进 OUT。布局的第一原则因此不是省面包板，而是让总线这条中轴最短、最直、最容易测量。具体做法是把 8 根数据线排在面包板阵列的中央：可以用一整块面包板专走总线，各功能模块分列上下两侧；也可以把几块面包板纵向拼成长条，让总线沿中央沟槽两侧贯通到底。模块大体按数据流向排位：PC、MAR、RAM 居一侧，A、B、ALU 居另一侧，IR 与控制单元靠一端，OUT 与显示排靠另一端。这样每个模块到总线的距离相近，飞线短而且彼此平行，出了故障能顺着总线一根一根查，而不是一团交叉的杜邦线里猜。

先把整机的连接关系画成一张总图：总线居中，模块分列两侧，控制单元居下。后面各节的每个模块，都只是这张总图的局部放大。



图示：整机总框图。中央竖条是 8-bit 三态总线；左列挂 PC、MAR→RAM、OUT，右列挂 IR、A、ALU、B，控制单元居下。实线是数据路径：五个总线驱动来源各标在出口处（C0、R0、A0、I0、Σ0），装载类控制线标在入口一侧；虚线是控制与状态线，16 根控制线经左右两条干线分到各模块。A、B 到 ALU 走不经总线的专线，IR 高 4 位的 opcode 专线直下控制单元，CF、ZF 沿虚线回报给条件跳转。任何一拍的“谁在说、谁在听”，都应能在这张图上逐根指出来。

以常见的 830 孔面包板估量，整机大约要三到四块：一块专走总线与 LED 排，一块放 A、B 与 ALU，一块放 RAM、MAR 与 OUT，控制单元与微码 ROM 单独占一块或半块。时钟与复位电路最小，放在面板开关手边即可。宁可多留空排也不要挤：模块之间留出的空隙，后面插测试钩、挪飞线、加扩展时都会用到。

8 根线之间还有一个位序约定：全机统一，例如 D7 在上、D0 在下，每个模块按同一方向插接。位序接反不会烧坏任何东西，但字节会被镜像，0x01 变成 0x80——这种故障不冒烟、不发热，只表现为数值诡异，排查起来格外费时，值得在插第一根线之前就用标签把方向定下来。顺手再给导线分个颜色：红色电源、黑色地、黄色总线、蓝色控制线，后面任何一个模块都可以凭颜色先猜出每根线的身份，猜错了再量，也比逐根追线快得多。

电源轨也要先规划。面包板两侧的长条电源轨常常在板子中间断开，上下两段之间要用跳线跨接，全机所有板块的轨还要连成同一个 5V 和同一个地——共地不是可选项，所有三态约定、所有电平判断都以同一参考地为准。每块面包板的电源入口先放一只 $10\mu\text{F}$ 电容，后面每插一片芯片再补一只 100nF 。电源走线的优先级高于信号线：先把轨连通、测过电压，再插第一片芯片。

每往总线上挂一个模块，都按同一组三问接线：它的数据口接总线哪几位，它的装载由哪根控制线在什么沿触发，它的输出由哪根控制线使能。三个问题都有确定答案，模块才算真正挂上了总线；有一个含糊，就先别插。A、B、IR、RAM、ALU 的这三问答案各不相同，§ 二、§ 三会逐个给出，但问法是全机统一的。

总线是共享资源，共享的接线约定只有一条：任何时刻至多一个驱动者。全机能驱动总线的来源只有五个，各对应一根输出类控制线：C0（PC 输出）、R0（RAM 输出）、A0（A 输出）、I0（IR 低 4 位输出）、 $\Sigma 0$ （ALU 输出）。微码保证同一拍这五根线至多一根有效，§ 四会逐拍列出；但只靠微码正确还不够，每个驱动者的输出都必须经过三态门：输出使能由对应控制线驱动，控制线一撤，输出立即回到高阻，把总线让出来。74HC173 自带三态输出，74HC273 则要外接 74HC244 之类的三态缓冲，§ 二会给出两种接法。

把“谁能说、说什么”先列成一张表，接线时照表核对：

驱动来源	控制线	有效时总线上出现的内容
PC（低 4 位）	C0	当前指令地址，总线高 4 位无人驱动
RAM	R0	MAR 选中单元的字节
A 寄存器	A0	累加器当前值
IR 低 4 位	I0	指令的地址或立即数字段，高 4 位无人驱动
ALU	$\Sigma 0$	加法器当前结果（A+B 或 A-B）

约定的另一半是“没轮到你说话时必须沉默”。上电瞬间所有输出类控制线还没有意义，因此每个三态使能的默认电平都必须是无效——这要靠接线保证，不能指望程序小心驾驶。还有一个容易漏掉的角落：五个来源都不驱动时，总线本身悬空，LED 可能半亮，挂在总线上的 CMOS 输入长期停在中间电平还会额外耗电发热。给 8 根总线各接一只 $100\text{k}\Omega$ 下拉电阻，空闲拍总线稳定读出 `0x00`。这既给高阻节点一个默认归宿，也顺手送了一个调试提示：某拍本该有驱动者，总线却读 `0x00`，先查那根输出控制线到场了没有。

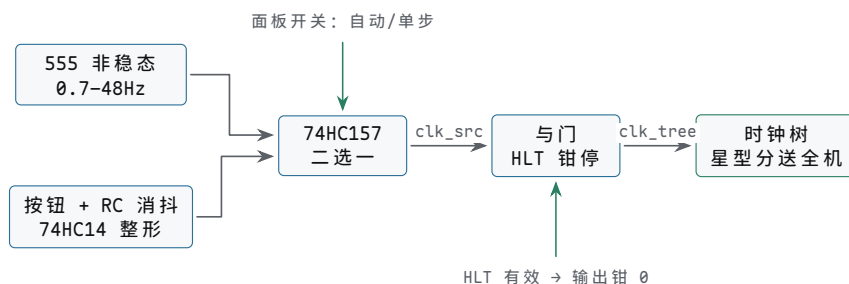
总线正中央还值得留一整排位置给 8 只 LED：每根数据线经一只限流电阻（ 330Ω 到 $1\text{k}\Omega$ ，按亮度调整）接一只 LED，常年点亮。这排灯占的地方不小，换来的是整机唯一的实时读数：单步执行时，每一拍总线上是誰的值、是不是预期的值，低头就能看到。后面调试的大多数问题——该 R0 的拍没人驱动、取指取回了错字节、ALU 结果没上总线——第一反应都是看这排灯。本章开头说教学机的目标是“没有一处不可观察”，总线 LED 排就是这条原则最大的一笔投资：一排面包板位置，换全机最繁忙结构的全程可见。

两只推挽输出同时驱动同一根线、且一高一低时，电流只受两级导通电阻限制：轻则逻辑电平被夹在中间、芯片发热，重则损坏输出级。争用不会自动报错，它表现为“偶尔算错”“某片芯片发烫”这类离病根很远的怪现象。所以“同一时刻至多一个驱动者”这条约定要用接线来保证——不用的三态使能固定在无效电平——而不是只靠微码写得对。

时钟分自动和单步两路 全机所有触发器共用一棵时钟树，树根只有一个。但运行和调试对时钟的要求正好相反：运行时希望它自己往前走，调试时希望它一步一停、每步都可检查。教学机的解决办法是做两路时钟源头，用一个开关选一路：自动档由 555 定时器产生连续方波，单步档由一个经过消抖的按钮产生人工脉冲，两

路经选择器会合后再送往时钟树。

两路时钟从振荡源到时钟树一共四级，先画出结构再逐件展开。



图示：时钟两路。自动挡 555 非稳态输出 0.7-48Hz 连续方波，单步档按钮经 RC 消抖和 74HC14 施密特整形给出干净脉冲；74HC157 按面板开关二选一，选中一路再经与门—HLT 有效时输出被钳在低电平，时钟树冻结，机器停在当前拍。时钟树从与门输出星型分到全机所有触发器的时钟脚，不菊花链。

路径	核心器件	输出特征	适用场合
自动挡	555 非稳态，R1、R2、C 定时	0.7-48Hz 连续方波 (C=10 μ F)	连续运行程序
单步档	按钮、RC 消抖、74HC14 整形	按一下一个干净脉冲	逐拍调试
选择器	74HC157 一路二选一	面板开关选源，HLT 可钳停	两路会合进时钟树

自动挡用 555 的非稳态接法：电阻 R1 接在 VCC 与放电脚之间，R2 接在放电脚与阈值脚之间，电容 C 从触发脚到地，输出脚直接出方波，频率 $f \approx 1.44 / ((R1 + 2R2) \cdot C)$ 。取 R1=1k Ω 、R2 用一只 100k Ω 电位器串联 1k Ω 固定电阻、C=10 μ F：电位器拧到最小 (R2 $\approx$ 1k Ω) 时 $f \approx$ 48Hz，拧到最大 (R2 $\approx$ 101k Ω) 时 $f \approx$ 0.7Hz——一只旋钮覆盖 1 到 50Hz，常用调试区间都在里面；把 C 换成 1 μ F，整个范围上移十倍。占空比为 $(R1 + R2) / (R1 + 2R2)$ ：低速端接近 1/2，高速端约 2/3。这个不对称对本机无碍：最窄的半拍也是毫秒量级，而 74HC 器件要求的最小时钟脉宽只有几十纳秒，余量有几十万倍。

为什么不干脆跑快些？面包板加人眼给时钟频率划了上限：LED 每拍都要看得清，排查时每拍都要停得住，1 到 10Hz 是观察最舒服的区间。教学机追求的不是快，而是慢到每一步都可观察；想验证整机在更高频率下是否仍然正确，等程序全部调通以后再把电位器拧上去也不迟。

单步档只有一个按钮：按一下，机器走一拍。麻烦在于机械触点会在几毫秒内反复通断，把触点信号直接当时钟用，“按一下”会被计数器看成好几拍。消抖用 RC 加施密特整形：按钮一端接地，另一端经 10k Ω 电阻上拉到 VCC，同一节点再经 1 μ F 电容到地——时间常数 10ms，足以盖过典型的抖动窗口；该节点送 74HC14 施密特反相器整形，输出才是干净的单步脉冲。“按一下、只跳变一次”这条性质，后面每一步调试都要依赖，所以消抖不是可选项。

两路时钟经一片 74HC157 (四路二选一，只用其中一路) 选择，选择端接面板上的拨动开关；选中一路再与停机逻辑会合—HLT 有效时把时钟树输入钳在低电平，机器停在当前拍。用伪码写就是：

```

clk_src = sel ? clk_555 : clk_step    -- 面板开关选一路
clk_tree = halt ? 0 : clk_src        -- HLT 有效时钳住时钟树
  
```

切换选择开关的瞬间可能切出窄毛刺，所以约定只在停机或按住复位按钮时换档。时钟树从选择器输出这一点出发，星型分到各模块的时钟脚，不要菊花链式地一个模块串一个模块——每根时钟线末端的边沿质量，决定那个模块的状态边界是否真实存在。时钟模块自身的检查靠眼睛：自动挡拧到 1Hz 左右，总线 LED 应约每

秒整体变一次；单步档按一下，LED 变一次且只变一次。按一下变了两次，查消抖；自动档完全不动，先量 555 的输出脚。

两路时钟的分工要养成一个硬习惯：任何程序第一次上板，一律从单步开始。逐拍走完第一条指令——看 T0 拍 PC 的值上总线、T1 拍指令字节进 IR、T2 拍以后各条控制线按预期起落——确认无误后再拨到自动档连续运行。自动档是“跑”，单步档是“看”；没看过的程序直接跑，出了错连错误发生在第几拍都不知道，只能从头再来。调试的全部增量信息都来自单步，自动档只负责最后确认。

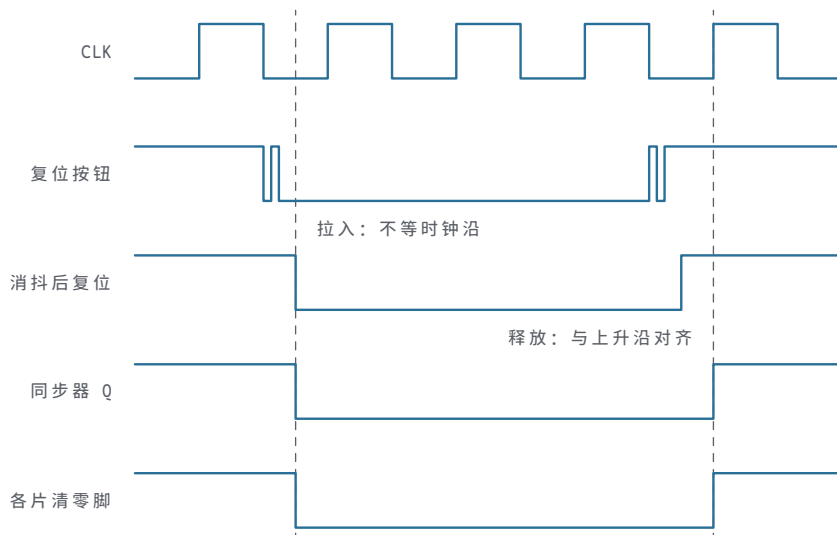
经典教材讲数据通路时，时钟和复位总是作为第一批元件画进图里，而不是最后补上的配角：没有确定的时钟边界就没有“状态”，没有确定的复位就没有“初态”。本章的两路时钟加复位链，不过是把这两条经典要求做进面包板的电源轨之间。

复位把 PC、T 状态和控制状态清到已知值 上电瞬间，每一片触发器的输出都是随机的：PC 可能是 0 也可能是 13，T 状态计数器可能停在 T5，IR 里是一个说不上名字的字节。不复位就直接运行，第一条被执行的“指令”来自随机地址、从随机 T 状态进入微码，同一块板子上电十次可能有十种开局。程序的第一条性质不是正确而是可复现，可复现的前提是每次上电都从同一个已知状态出发。复位链的任务，就是把这个前提实实在在地做出来。

必须清掉的状态只有三类。一是 PC：程序计数器清 0，第一条指令永远从地址 0 取。二是 T 状态计数器：清 0 后机器从 T0 开始，而 T0、T1 是公共取指拍，微码 ROM 里所有 opcode 对应的 T0 行都填同一个字 **CO+MI**，T1 行都填 **RO+II+CE** (§ 四会展开)——所以 IR 的随机初值无害：T1 拍 II 有效，RAM[0] 的字节会把 IR 整体覆盖。三是 Flags：程序并不保证第一次 JC/JZ 之前一定执行过 ADD 或 SUB，标志寄存器必须接复位，否则开头几次条件跳转读的是随机值。A、B、OUT 的内容不影响执行顺序，程序第一条 LDA 或 LDI 自然会给 A 一个确定值；教学机上把它们也接进复位，只是为了让上电后的 LED 读数好看，不是必须。

复位信号由复位按钮经 RC 消抖与施密特整形产生，链路与单步时钟相同。使用时遵守“异步拉入、同步释放”。按下按钮：复位直接进各片的异步清零脚（74HC273 的 $\overline{CLR}$ 、74HC161 的 $\overline{CLR}$ 等），不等时钟沿、立即生效——上电过程中时钟还不干净，复位不能反过来依赖时钟。松开按钮：释放沿先经过一级触发器与时钟对齐，再撤掉各片的清零。具体做法是把消抖后的复位信号接一片 74HC74 的 D 端、系统时钟接它的时钟脚，用 Q 去驱动各片的清零脚，全机就在同一个时钟沿之后一起离开复位，不会出现一半模块先走、一半还停着的局面。释放后第一个上升沿执行的就是 T0：**CO+MI** 把 PC=0 送进 MAR。机器的第一拍从此永远相同，这是后面所有调试能成立的地基。

把从按下到松开的全过程画成时序，拉入与释放的不对称直接可见。



图示：复位链时序：异步拉入、同步释放。按钮按下（电平有弹跳）后，消抖复位接在 74HC74 的异步清零端，Q 与各片清零脚立即变低，不等待时钟沿；按钮松开后，消抖复位早已接在 74HC74 的 D 端，Q 要等下一个时钟上升沿才变高，各片在同一沿之后一起离开复位，第一个装载沿执行的就是 T0 的 CO+MI。

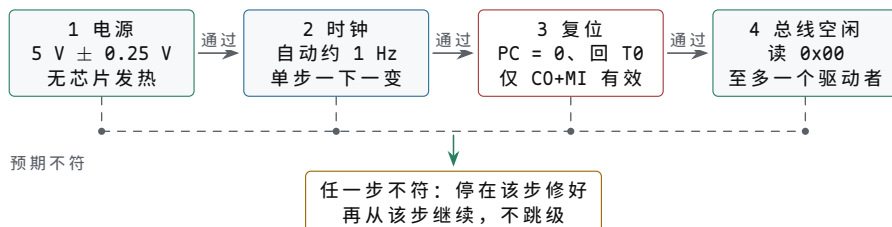
复位按钮不只在电上电时用。调试中途程序走乱了、想换一个初始条件、或者只是想再看一遍开头几拍，按一下复位就回到已知起点：PC 归 0、T 归 0、标志清零，而 RAM 的内容不受复位影响——程序还在，随时可以重新单步。复位因此是面板上使用频率最高的键，它的地位相当于调试器里的“回到开头”，把它装在顺手的位置，比把显示排摆得好看更实际。

正式产品里常用 RC 加电压监控做成上电自动复位，让机器一得电就自己进入已知状态。教学机用一个手动按钮已经足够：上电后先按一下复位，效果相同。要知道的只是差别——自动复位保证的是“没有人按也确定”，手动按钮保证的是“按过就确定”，调试场景里后者反而更方便，因为复位时机由你掌握。

“程序很短，不复位多半也跑得对”是一种错觉。随机状态意味着十次里可能九次碰巧正常，第十次 PC 从 7 起跳、T 计数器从 T3 进入微码，程序行为完全变样，而排查者很容易误判为板子接错了。复位链是整机最便宜的确定性来源：一个按钮、两只电阻、一只电容，再加一片施密特反相器。

上电检查顺序先于任何程序 机器接完、或每次改动接线之后，都不要急着把程序拨进 RAM。上电检查有一套固定顺序，每一步都有明确的预期现象，前一步不符合就不做下一步。顺序按被依赖的程度排：电源不依赖任何东西；时钟只依赖电源；复位的效果要靠时钟才能观察；总线空闲态则依赖前三者再加上全部三态使能。

把这套顺序画成流程图，四步的依赖与各自的出口一并标出：



图示：上电检查流程（与正文顺序表一致）。顺序按依赖排：电源不依赖任何东西，时钟只依赖电源，复位的现象要靠时钟观察，总线空闲态依赖前三者加全部三态使能。每步先写下预期再通电：符合才前进；任一步不符就停在该步，修好后从该步继续——换一片芯片、挪一组飞线这类局部改动，至少回到被改动模块所属的那一步重查。

步骤	检查内容	预期现象	异常先查何处
----	------	------	--------

1 电源	各片 VCC/GND、去耦电容	每片 VCC 脚对地 5V±0.25V，无芯片发热	电源轨断点、极性接反
2 时钟	自动档约 1Hz，再试单步档	总线 LED 每秒约变一次；按一下只变一次	555 外围 R/C、消抖电容
3 复位	按住复位按钮再松开	PC 显示 0、回到 T0、仅 C0 与 MI 有效	复位链路、各片清零脚
4 总线	复位后逐拍单步空走	空闲拍读 0x00，每拍至多一个驱动器	三态使能默认电平、控制线接反

第一步查电源与去耦：每片 74HC 的电源脚到位，每片旁边一只 100nF 去耦电容，每块面包板的电源轨入口再加一只 10μF。第二步查时钟，现象上一节已经写过。第三步按住复位按钮：PC 的 4 位 LED 应全灭，T 状态显示应回到 T0，控制线上只剩 C0 与 MI 有效，其余全灭；松开后再按一次，现象应完全重复。第四步看总线空闲态：复位释放后、程序装入前逐拍单步，凡是没有输出控制线有效的拍，总线 LED 都应因下拉电阻而读 0x00；任何两片芯片微热，都说明某处有两个驱动器在同时说话。

如果只是局部改动——换了一片 244、挪了一组飞线——不必每次都从第一步重来，但至少要从被改动模块所属的那一步查起：动过电源轨就回第一步，动过时钟或复位就回第二、三步，动过任何三态使能就回到第四步逐拍空走。四步全部符合预期，机器才具备装入程序的条件。这个顺序看起来慢，其实是最快路径：程序运行不对时，故障一定在四步之一或程序本身，先把四步重走一遍，就把排查范围切掉一半。后面各节搭每个模块时沿用同一思想的展开：先静态、再单步、后程序。

最后补一条纪律：每一步的预期现象要先写在纸上，再通电。先写预期再看现象，是防止“看着差不多就放过”的唯一办法——人眼有一种替电路找借口的天赋，灯大致在闪就当时钟正常，字大致在变就当总线正常。纸上的预期是通电前写下的，不会被现场说服；实测与它不符就停下来查，这一步省掉的时间，最后都会在排查里加倍还回去。

核心思想

总线居中、两路时钟、确定复位、固定上电顺序，这四件事的共同目的只有一个：让“机器此刻在干什么”永远是一个可以当场回答的问题。教学机的性能无关紧要，可观察性就是它的全部价值。

二、寄存器与 ALU 模块

本节把数据通路的核心立在板上：两个通用寄存器（A、B）、两个特殊寄存器（IR、OUT）、一台只做加减法的 ALU，以及随 ALU 而生的两个标志。这些模块的公共结构只有一句话：触发器保存值，三态门决定何时对总线说话，时钟沿决定何时把总线上的值听进来。本节把这句话做成具体电路；§ 三的存储器和 § 四的控制单元，不过是同一句话的更多实例。

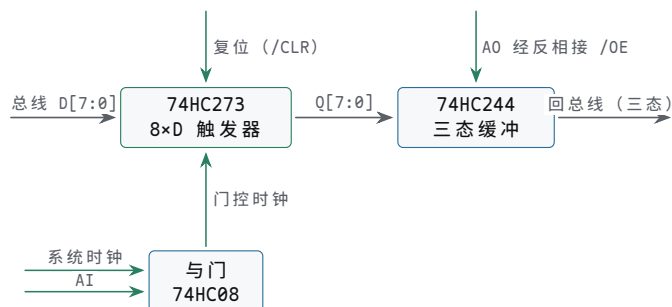
寄存器模块 = 触发器 + 三态输出 一个挂在总线上的寄存器要做三件事：平时保持自己的 8 位值；装载控制线有效的那拍，在时钟沿把总线上的值锁进来；轮到输出时把自己的值驱动到总线上，其余时刻保持沉默。三件事对应三种器件资源：一组 D 触发器、一条受控的装载通路、一排三态输出门。74 系列里现成的组合有两档：74HC273 一片集成 8 个 D 触发器，带公共时钟和异步清零，但输出直出不带三态；74HC173 一片集成 4 个 D 触发器，自带装载使能和三态输出，两片拼成一个 8 位寄存器。

	74HC273	74HC173
集成规模	一片 8 位，正好一个寄存器	一片 4 位，两片拼 8 位
装载使能	无，需门控时钟	有（低有效），时钟直连

输出形式	直出，上总线需外加三态缓冲	自带三态（低有效使能），可直挂总线
异步清零	有（ $\overline{CLR}$ ，低有效）	有（MR，高有效）

用 273 的接法：8 个 D 端接总线， $\overline{CLR}$ 接复位线。273 没有装载使能脚，“AI 有效才装载”用门控时钟实现——系统时钟与 AI 进一片 74HC08 与门，输出接 273 的时钟脚：AI=0 时时钟被挡住，寄存器保持；AI=1 时的那个上升沿，总线值进入寄存器。门控时钟能安全使用有一个前提：控制线必须在时钟低电平期间就已经稳定。本机的 T 状态在时钟下降沿推进（§ 四），AI 在装载上升沿到来前半拍就已就位，与门输出因此只有一个干净的装载沿，不会切出多余边沿。输出侧补一片 74HC244 三态缓冲：输入接 273 的 Q，输出接总线；注意 244 的输出使能是低有效，A0 是高有效，中间要加一级 74HC14 反相。A0 一撤，244 回到高阻，A 寄存器对总线沉默。

这段接法画成图就是“一进一出两条路、一根门控时钟”。



听 = 时钟沿（沿触发）；说 = 组合（电平使能）

图示：寄存器模块（273 方案）。左侧总线值进 D 口，右侧 Q 口经 74HC244 三态缓冲回总线。装载是沿触发的：系统时钟与 AI 相与产生门控时钟，只有 AI=1 的拍才出现装载沿；输出是组合的：A0 经反相打开 244 的使能，整拍驱动，A0 一撤立即高阻。173 方案把装载使能和三态都收进片内，时钟直连，但信号语义与此图完全相同。

用 173 的接法可以省掉外挂缓冲：两片分管低 4 位与高 4 位，D 端接总线，Q 端直接挂总线。173 的装载使能和输出使能同样都是低有效，各加一级反相再接 AI、A0；时钟则直接接系统时钟，不再需要门控时钟——这是 173 比 273 干净的地方，代价是芯片从一片变两片再加反相器。两种接法在教学机里都常见，本章后续一律只说“AI 有效沿装载、A0 有效驱动总线”，具体器件任选一种即可。无论选哪种，每个寄存器的 Q 端都配一排 LED：寄存器存的是什么，应当和总线值一样随时可读。两种方案的核心接线可以各浓缩成三行，插线时对照：

```
-- 273 方案（以 A 寄存器为例）
D[7:0] <- 总线；CLK <- 系统时钟 AND AI；/CLR <- 复位
Q[7:0] -> 74HC244 -> 总线；244 的 /OE <- NOT A0
-- 173 方案（两片拼 8 位）
D[7:0] <- 总线；装载使能 <- NOT AI；CLK <- 系统时钟
Q[7:0] -> 总线；输出使能 <- NOT A0
```

第二章把“用普通与门切时钟”列为禁令，针对的是有同步时序预算的设计：时钟一快，门控引入的偏斜和使能毛刺就会被当成真实时钟沿，建立与保持无从保证。这台教学机里门控时钟却是标准做法，差别不在原则而在条件：只有低速、控制字提前半拍稳定时门控时钟才安全——本机时钟慢到毫秒量级，T 状态在下降沿推进（§ 四），控制线在装载上升沿前半拍就已就位，与门输出不可能切出多余边沿，第二章担心的那条失效路径在这里不成立。条件变了结论就变——若哪天把这台机器移植到同步时序严格的器件上，第一件事就是把门控时钟改回时钟使能。

A、B 两个寄存器比一般模块多一捆线：除总线接口外，A 的 Q 还有 8 根专用线直连 ALU 的 A 输入，B 的 Q 同样直连 ALU 的 B 输入，不走总线、不经三态、常年有效。原因在 ALU 是组合逻辑：它要求两个操作数在同一拍里稳定在场，而总线一拍只能传一个值——若操作数也走总线，一拍之内先送 A 再送 B，结果还没地方回来。专用线的代价是 16 根飞线，换来加法那拍的极简格局：A、B 站好，ALU 出结果， $\Sigma 0$ 把结果送上总线，AI 把它收回 A。另外，B 寄存器根本不给总线配输出：没有任何指令需要把 B 单独送上总线，它的输出三态可以直接省掉，273 方案里因此少接一片 244。B 的值从哪来？仍然从总线来：ADD、SUB 的第一个执行拍用 R0+BI 把 RAM 里的操作数抄进 B，专用线再把这份稳定的 B 呈给 ALU。装载走总线、输出走专线，两个方向各用各的路，这正是 A、B 与一般模块的差别所在。

A 在全机寄存器里活得最累：LDA、LDI 从总线装它，ADD、SUB 的结果写回它，STA 要它说、OUT 要它说、JC/JZ 的判断间接来自它。它既是累加器又是事实上的通用暂存，驱动总线的次数全机最多。所以 A 模块的接线要格外扎实：AI、A0 两根控制线、总线接口、ALU 专用线三路交汇，任何一处虚接都会在最频繁的通路上发作。查整机怪故障时，A 模块永远值得第一个怀疑。

寄存器模块的插线顺序也固定：先插芯片、接电源与去耦，量过 VCC；再接 D 口到总线的 8 根线；然后接 Q 端到 LED 排；最后才接控制线——装载、输出使能、清零。控制线最后接的理由是：没接控制线时模块应当完全安静，不装载也不驱动，此时总线其他部分的调试不受它干扰。每接好一组就停下来核对一次，比一次全接完再回头找错快得多。

寄存器模块不进程序就能检查。静态：用跳线在一排空排上摆出已知字节（如 0x5A），手动给一个装载沿，Q 端 LED 应显示同一字节。单步：撤掉装载使能再按几拍，值应保持不变；让输出使能有效，总线 LED 应重现该字节。存、装、说三个动作分别正确，这片寄存器才算搭好，才可以往总线上挂。

IR 和 OUT 是两类特殊寄存器 IR 也是 8 位寄存器，装载与 A 完全一样：II 有效沿把总线上的指令字节锁进来。特殊在输出分两半、各走各路。高 4 位是 opcode：不经总线，专用线直连微码 ROM 的地址高位（§ 四），常年有效——译码逻辑要随时看得到“当前是哪条指令”。低 4 位是地址或立即数字段：经三态上总线，由 IO 控制，服务于两条路径：把地址字段送 MAR（IO+MI，见于 LDA、ADD、STA 等带地址的指令），或把立即数送 A（IO+AI，即 LDI 的装载拍）。用接线列表写下来就是：

IR.D[7:0]	<- 总线	-- II 有效沿锁存
IR.Q[7:4]	-> 微码 ROM 地址	-- 直连，不经三态
IR.Q[3:0]	-> 三态 -> 总线	-- 仅 IO 有效时驱动低 4 位

接法上的要点是：低 4 位的三态缓冲只接 Q0-Q3，高 4 位绝不允许跟着上总线。用两片 173 拼 IR 时，低 4 位那片的输出使能接 IO，高 4 位那片的输出使能直接固定在无效电平——从接线上删掉高 4 位驱动总线的的能力，比指望微码永远不犯错可靠。C0 与 IO 都只驱动总线低 4 位，高 4 位悬空：MAR 本就只锁存低 4 位，不受影响；LDI 拍 AI 锁存整个字节时，高 4 位正是 § 一那排下拉电阻读出的 0，于是 A 收到的是干净的 0x0 拼接立即数。一个前面看似多余的布局决定，到这里显出用途。

高 4 位走专用线而不是总线，也是同一套时刻关系决定的。译码逻辑在每个 T 状态都要看到 opcode：T2 起的每一拍，微码 ROM 的地址都是 opcode 与 T 的拼接（§ 四展开），opcode 必须常年在场。若让 opcode 也经总线送译码，就得专门安排一拍把 IR 高 4 位驱动出去，而那一拍总线往往正被取指或运算占用。寄存器输出分两路、各走各的，译码与数据传送就互不排队。

OUT 是另一种特殊：只进不出。OI 有效沿从总线锁存——OUT 指令的装载拍是 A0+OI，A 说、OUT 听；Q 端直驱显示，8 只 LED 一排，或经译码驱动数码管（接法 § 三展开），输出侧没有任何通往总线的三态门。输出设备不该驱动总线，理由有三层。功能上，没有任何指令需要把 OUT 的值读回来，配了输出门也是浪费。电

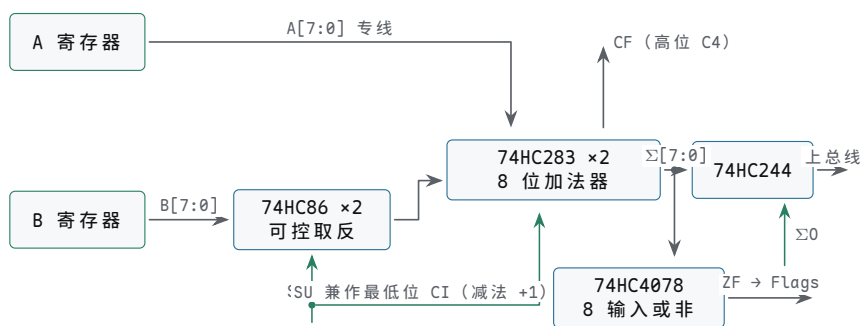
气上，总线上每多一个驱动者，就多一根可能插反的使能线、多一处争用隐患。原则上，“只进不出”应当由接线来保证：即使微码写错、杜邦线插错排，OUT 在物理上也不可能与别人抢总线。把约定做进硬件而不是写进纪律，是这台小机器反复使用的思路——§ 一的三态使能默认无效、本段 IR 高 4 位不上总线、OUT 不配输出门，是同一条思路的三次应用。

OUT 还是整台机器的脸面。总线 LED 显示的是“此刻正在流动”的值，随拍就变；OUT 显示的是“程序想让人看到”的值，装载一次就一直亮着。机器执行 HLT 之后，时钟停了、总线静了，最后留在面板上的就是 OUT 的那一排灯——程序的对错，最终由它宣布。所以 OUT 的显示排值得放在面板最显眼的位置，朝向观看者，而不是藏在模块背后。

ALU = 74HC283 加法器加一路取反 本机 ALU 只做两种运算： $A+B$ 与 $A-B$ 。两种运算共用一条加法路径，差别全在 B 进入加法器之前怎样处理。依据是补码恒等式 $A-B=A+\bar{B}+1$ ，其中 $\bar{B}$ 表示 B 的逐位取反。于是 ALU 的全部家当是：两片 74HC283 拼成 8 位加法器，两片 74HC86 拼成 8 位可控取反器，再加几门标志逻辑。这正是第三章“一条加法路径加输入处理”层次链的末端：门组成全加器，全加器连成 283，283 配上可控取反就是加减法 ALU。

两片 283 级联：低位片的进位输出接高位片的进位输入，高位片的进位输出就是全 8 位的 CF 来源。A 输入 8 位直连 A 寄存器的 Q，即上一段的专用线。B 输入不直连，先过 86：每片 86 含 4 个异或门，每个门一端接 B 的对应位，另一端 8 个门并在一起接 SU 控制线。异或的性质 $X \oplus 0 = X$ 、 $X \oplus 1 = \bar{X}$ 正好构成可控取反：SU=0 时 B 直通，SU=1 时 B 逐位取反。同一根 SU 还接到最低位加法器的进位输入：加法时 CI=0，结果是 $A+B$ ；减法时 CI=1，结果是 $A+\bar{B}+1$ ，即 $A-B$ 。一根控制线同时完成取反与加一，这是补码减法最省器件的形态。ALU 的结果经一片 74HC244 三态缓冲挂总线，使能经反相接 $\Sigma 0$ 。

这套组合画成结构图，就是一条加法路径加一路可控取反。



图示：ALU 结构。A、B 经专线常年到场：A 直连两片级联 74HC283 的 A 输入，B 先过两片 74HC86 可控取反——SU=0 直通、SU=1 逐位取反，同一根 SU 分叉接最低位进位输入 CI，取反加一一步完成，两片 283 之间的片间进位（低位 C4 接高位 CI）在框内完成。结果经 74HC244 三态上总线，使能由 $\Sigma 0$ 反相驱动；CF 取自高位片进位输出，ZF 由 74HC4078 对 8 个和位取或非得到。两个标志此刻还是组合信号，要等 FI 那拍才锁存。

接线之前先手工演算两组数，确认这套组合真的等于减法。算 7-5：00000111 +11111010+1，和为 100000010—8 位结果是 00000010 即 2，进位输出为 1。再算 5-7：00000101+11111000+1，和为 011111110，8 位结果 11111110 正是 -2 的补码，进位输出为 0。两组数合在一起说明：减法时进位输出为 1 表示没有借位，即 $A \geq B$ ；为 0 表示 $A < B$ 、结果是补码负数。所以 JC 接在 SUB 后面时，语义是“A 大于等于 B 则跳”，这个约定 § 五写程序时会反复用到。加法的进位输出就是普通溢出，例如 200+100：11001000+01100100，结果 00101100 即 44，CF=1。

两个标志都从加法器输出组合产生。CF 最直接：高位 283 的进位输出引出即是。ZF 要求“结果全 0”：8 个和位送进一片 74HC4078（8 输入或/或非），取它的或非输出——任何一位为 1 输出即 0，全为 0 时输出 1，这正是 ZF。要注意此刻两个标志都还是组合信号：它们只对当前这一拍的 A、B、SU 有效，下一拍 A 或 B 一变，标志跟着变。组合标志没法直接给跳转用，因为 JC/JZ 的判断拍在加法拍之后，中间还隔着若干拍——标志必须先被记住。

顺带有两个接线细节。其一，244 之前、加法器输出之后是一个值得利用的观察点：在这里分一排 LED，就能看到“当下 A、B 与 SU 的函数值”—A、B 的专用线常年有效，这组组合结果也就常年有效，即使 $\Sigma 0$ 没让结果上总线，低头也能读出它。调试 ALU 时看的正是这组灯。其二，SU 一根线要分叉到 9 个终点：8 个异或门加最低位进位输入。74HC 输出的扇出能力带这 9 个负载毫无压力，但走线要短而整齐，因为 SU 一切换，整条 ALU 的函数就变了。

这条 ALU 不进整机就能先做静态检查，它本质上就是第三章那个 4 位加减法实验台加宽到 8 位：A、B 各用 8 只拨码开关代替寄存器直接摆值，SU 用一只开关手控，结果与 CF、ZF 各接 LED。加法摆几组、减法摆几组，其中包括 7-5、5-7、200+100 这三组已经手工算过的数，LED 读数与手算一一对上，ALU 这片电路才算可信，A、B 寄存器的专用线这时再接过来。若等到装进整机才发现减法不对，就分不清是 ALU 错、专用线错还是微码错。

进位链的速度也不用操心：两片 283 级联，进位从最低位传到最高位经过两级片内进位加一次片间传递，总延迟在几十纳秒量级；本机一拍是几十毫秒，进位有充足时间走完，不需要先行进位之类的加速结构。慢，在这里不是缺陷而是设计目标的一部分。

“减法就是加上负数”是经典教材反复强调的一点：有了补码，数据通路里只需要一条加法路径，减法、比较、条件跳全都复用它。本机的 ALU 把这一点做到了极简——一片加法器、一路可控取反、一根兼作加一的 SU 线，再没有别的运算硬件。

标志寄存器在加法完成那拍装载 记住标志的办法是在它还新鲜的那拍锁存。Flags 是一片小寄存器：两个 D 触发器（一片 74HC74 正好，或从 74HC273 借两位），D 端分别接组合的 CF 与 ZF，装载时钟由 FI 控制，与 A、B 相同的门控时钟结构。微码只在 ADD、SUB 的求和拍让 FI 有效——与 $\Sigma 0$ 、AI 同一拍，这一拍的上升沿同时做两件事：A 锁存和或差，Flags 锁存这一对 CF、ZF。此后 FI 一直保持无效，锁存值原样保持，直到下一次加法或减法把它们刷新。

清零接法随器件而定：从 273 借两位时，那片 273 的 $\overline{CLR}$ 直接接复位线；用 74HC74 时，两个触发器各自的异步清零脚并起来接复位线。两种接法的效果相同——复位期间 CF、ZF 都为 0，与 § 一“标志必须接复位”的要求一一对应。Flags 的 Q 端同样各接一只 LED：锁存的 CF、ZF 是条件跳转的依据，应当像寄存器值一样常年可见。

跳转指令读的是锁存值：JZ 的判断拍看锁存的 ZF 是否为 1，JC 看锁存的 CF。因为读的是寄存器输出而不是组合输出，判断拍总线上走什么、ALU 输入变成什么，都与这次判断无关——这正是要锁存的原因。由此也看得清微码的一条硬约定：全机只有 ADD、SUB 的求和拍有 FI，LDA、LDI、OUT 等一概不刷新标志。程序里“先 ADD、隔几条指令再 JZ”是安全的，“先 LDA 再 JZ”判断的是更早一次运算留下的标志。复位时 Flags 清零（§ 一），保证第一次 ADD/SUB 之前 JC/JZ 读到的是确定值。

“运算顺便刷标志”还塑造了本机的程序风格。循环计数不必单独比较：把循环变量放在 RAM 里，每轮用 SUB 减一，减到零的那次 ZF 自然置起，紧跟一条 JZ 就跳出循环——一条 SUB 同时完成算数和判断，这正是把标志锁存在求和拍带来的便利。§ 五的完整程序会用到这个惯用法。

FI 这一拍也可以单步核对。让机器停在 ADD 的求和拍： $\Sigma 0$ 应有效、总线应显示和、AI 与 FI 应同时有效；按一下单步，A 的 LED 更新为和，标志的 LED 同步更新。再按几拍让总线和 ALU 输入随意变化，标志 LED 应纹丝不动——动的只有下一次 ADD 或 SUB 的求和拍。这两段观察合起来，锁存的意义就不是定义而是亲眼所见。

模块按依赖关系从简单到复杂检查：先 ALU、后标志——标志锁存的只是 ALU 的组合输出，ALU 不对，标志锁得再准也是错的。每个模块内部则统一按“先静态、再

单步、后程序”三步走：静态检查不上时钟，用跳线摆输入、看组合输出，它不过，后面两步都没有意义；单步检查用按钮时钟逐拍推进，核对每一拍的装载与输出，它不过，程序层面的任何异常都无法定位；程序检查才让模块在整机里跑指令。各模块的检查点汇总如下：

模块	静态检查	单步检查	程序检查
A/B 寄存器	总线摆 0x5A，手动装载沿后 Q 显示 0x5A	使能有效拍装载、无效拍保持；A0 拍总线重现该值	LDI/LDA 后 A 的 LED 为预期值
IR	装载后 Q 直通显示，高低 4 位分路正确	II 拍锁存；IO 拍仅低 4 位上总线	取指派后 IR 与 RAM[0] 一致
OUT	OI 装载后显示保持不闪	A0+OI 拍把 A 的值抄到显示	OUT 指令后显示当前 A
ALU	A/B 摆定值，SU 切换看和、差与 CF/ZF	ΣO 拍结果上总线，其余拍高阻	ADD/SUB 后 A 与标志同步更新
Flags	翻动 ALU 输入，锁存值不随动	FI 拍锁存新值，此后保持不变	ADD 后接 JC/JZ，走向与手工预算一致

至此数据通路的一半已经立在板上：寄存器能存能说，ALU 能算，标志能记。§ 三给它配上记忆—MAR 与 RAM—和脸面，§ 四再给它配上发号施令的控制单元。

三、内存与输出模块

本节做两件事：把“存 16 个字节”做成具体电路，把“把结果显示给人看”做成具体电路。内存模块是全机最讲究边界的一节——它既要在运行时受控制单元精确驱动，又要在装程序时把整个界面让给人的手指；显示模块则相反，它只看不碰，是整机里唯一没有回路的部分。下面按 RAM 本体、地址锁存、输出显示、编程模式的顺序展开。

本节用到的整机信号先集中登记，读到后文任何一根都可以回来查：

信号	方向	在本节的作用
MI	入	MAR 的装载使能，拍末把总线低 4 位锁存为地址
RI	入	RAM 写请求，经门控成为 189 的写窗口
RO	入	RAM 读请求，经反相打开 240 的三态输出
OI	入	OUT 寄存器的装载使能，拍末锁存总线值
总线 D0-D7	双向	地址、数据、显示值都经它中转
时钟、复位	入	节拍与清零，§ 一的约定原样沿用

RAM 模块的两种实现路线 16 字节内存用两片 74HC189 拼成。74HC189 是一片 16 字乘 4 位的静态 RAM：4 根地址线选中 16 个字之一，4 路数据输入、4 路数据输出，控制线只有两根——片选 S（低有效）和写使能 W（低有效）。读的条件是 S 有效且 W 无效，输出脚驱动读出值；写的条件是 S 有效且 W 有效，输入脚的值进入选中字。两片并排放置，一片接总线低 4 位、一片接总线高 4 位，地址线并联，就得到 16 乘 8 的内存。位宽扩展在这里没有任何译码技巧，纯粹是每片管一半位，与第五章讲的存储器位宽扩展是同一种做法，只是规模小到能放在手掌上。

接线之前，先把 189 的引脚与整机信号——对应列成表。照表核对比对着原理图找线可靠得多，尤其是两片并联的模块，任何一根跨片的错接都会表现为“高 4 位或低 4 位整体出错”。

引脚	名称	方向	在整机中的接向
----	----	----	---------

A0-A3	地址输入	入	MAR 的 Q0-Q3, 两片并联
D1-D4	数据输入	入	数据总线对应的 4 位
S	片选, 低有效	入	接地, 恒选中
W	写使能, 低有效	入	RI 与时钟低半拍相与后取反, 两片并联
01-04	数据输出, 反相 三态	出	74HC240 的输入
Vcc、GND	电源	-	5 V, 每片电源脚旁并 0.1 微法去耦电容

两片的分工按总线位次划：一片的 D、0 接总线 D0-D3，另一片接 D4-D7，两片之间没有任何信号交叉。先焊低 4 位那片、单独测通，再焊高 4 位那片，是最省心的顺序。去耦电容要贴着每片的电源脚放：RAM 在读写切换瞬间的电流突变全靠它就地吸收，省掉这一步，后面会出现“时而对时而错”的软故障，是所有故障里最难定位的一类。

时序余量也值得算一次。5 V 下 74HC189 的读取延迟约三四十纳秒，74HC240 的打开延迟十几纳秒，合起来从 R0 有效到总线数据稳定不超过一百纳秒；而本机时钟最激进的挡位也不过 48Hz，一拍是毫秒级——最快挡约 20 毫秒。一百纳秒对 20 毫秒，余量约五个数量级，所以这台机器不需要任何等待状态——读周期里“地址稳定、打开 R0、等访问时间、拍末采样”这四个动作，前三个在一拍的开头瞬间就完成了，第五章讨论的那种等存储器的无奈，在这里一次都不会出现。

顺手把读数据手册的方法在这片芯片上练一遍。存储器手册里真正要读的参数只有三类：读路径的延迟、写路径的窗口、各信号的先后约束。把 74HC189 的典型量级列在这里（确切数值以手边器件的手册为准），并对照本机给出结论：

手册参数	典型量级	对本机的含义
地址到输出的读取延迟	几十纳秒	毫秒级一拍（最快挡约 20 毫秒），余量约五个数量级
写脉冲最小宽度	二十纳秒上下	低半拍开窗，窗口宽十毫秒上下
数据先于写沿的建立时间	二十纳秒上下	数据在窗口打开前已稳定半拍
写沿后数据的保持时间	接近零	窗口先关、数据后撤，天然满足

看手册的顺序因此是：先找这几行参数，再回看自己的时钟周期，只要周期比参数大三四个数量级，时序这一章就可以合上书了。这台机器的全部存储器时序分析到此为止——剩下的都是逻辑问题。

用 74HC189 有一个必须处理的细节：它的数据输出是反相的——读出来的是存储值按位取反的结果（同系列里同相输出的型号是 74HC219，不如 189 常见）。还原的方法不是加一片普通反相器，而是加一片 74HC240。240 是八反相三态缓冲器，两个四位组各有独立的输出使能，把它们并接后由 R0 控制（240 的使能低有效，R0 经一级反相后接入）。这一片器件同时完成两件事：把反相再反相、还原出原值，以及在 R0 无效时让输出呈高阻。“RAM 输出经三态上总线”这条整机约定就藏在这一片 240 里：R0 有效，RAM 选中的字出现在总线上；R0 无效，240 的输出端从总线上断开，RAM 对总线彻底隐身，别的驱动器感觉不到它的存在。

240 的八路缓冲在本模块里刚好用满：两个四位组的使能端（1G、2G，低有效）并接后由 R0 反相驱动，八路输入接两片 189 的反相输出，八路输出上总线。把这片缓冲器的信号也列成表，接线时与上一张引脚表对照使用：

240 引脚	接向	说明
1G、2G	R0 经反相后并接	R0 有效时八路同时打开

输入 8 路	两片 189 的 01-04	收到的是存储值的反相
输出 8 路	数据总线 D0-D7	再反相一次，原值上总线

两级反相串联、极性归零，这个结构值得多看一眼：189 的反相输出本是器件的脾气，240 既治好了脾气又提供了三态边界，一片钱办两件事。若采购到同相输出的 74HC219，结构不变，只把 240 换成同相的 74HC244 即可，其余接线一根不动。

两片 189 的写使能 W 并接，来源是控制线 RI；读路径的开关是 R0。写使能与输出使能的分工因此非常清楚：RI 管“把总线上的值写进 MAR 选中的字”，R0 管“把 MAR 选中的字送到总线上”，两者经过不同的使能路径作用于同一组地址，互不相干。另有一个时序细节值得照做：让写窗口只占时钟的低半拍，即 W 在“RI 有效且 CLK 为低”的区间内打开。这样写窗口在拍末上升沿到来时已经关闭，而 MAR 的地址要在边沿之后、经过触发器延迟才会变化，写窗口关闭永远先于地址变化，不会把数据顺手写进下一个地址。这个细节的代价不过是一级门，换来的是写操作对边沿时刻完全不敏感。

信号	接向	有效条件	作用
地址 A3-A0	MAR 的 Q3-Q0	常接	选择 16 个字之一
数据输入 D	数据总线	常接	写窗口内由总线驱动者供数
片选 S	接地	恒有效	读写区分交给 W 与 240
写使能 W	RI 与时钟低半拍相与后取反	RI 有效的低半拍	总线值写入选中字
反相输出 0	74HC240 输入	读出期间驱动	值为存储内容的反相
240 输出使能	R0 反相后接入	R0 有效	原值上总线；无效则高阻

第二条路线解决的是另一个问题：程序从哪里来。189 只是存储体，把程序装进去还需要一套输入手段。成本最低的做法是 DIP 开关加一只按钮：4 只开关拨地址，8 只开关拨数据，按钮发出写脉冲。这条路慢，装一个字节要十几秒，但每一个比特都经过手指，地址、数据、写脉冲三个概念被迫分开操作——这恰恰是教学想要的慢。编程电路与运行电路在 189 的三组引脚上交接：地址来源用一片 74HC157（四路二选一选择器）在 MAR 与地址开关之间切换；数据开关经一片 74HC244 送上总线，只在编程模式下使能，189 的数据输入照旧取自总线，无需切换；写脉冲由按钮经 74HC14 施密特整形后直接驱动 189 的 W，绕开 RI。切换的完整次序在本节最后一个主题给出，那里的核心约束只有一句：先停机，再切换。

路线	器件	关键问题
RAM 本体	74HC189 两片 + 74HC240 一片	输出反相要靠 240 还原；RI 与 R0 永不同拍
编程输入	DIP 开关 12 只、按钮、74HC157、74HC244、74HC14	成本低、每字节十几秒；切换前必须先停机，避免与运行电路争用

也有人会问能不能干脆用一片大 SRAM，比如 6116（2K 乘 8）。能，而且读写逻辑与 189 几乎一样，还免去了反相输出的麻烦；代价是 11 根地址线里有 7 根要固定接地或接高，走线反而比两片 189 更琐碎，并且“16 字节的小内存”这个教学尺度被一片 2K 的芯片遮住了一数组大一千倍，能看见的仍然只有 16 个字。189 的价值恰在它的合身：地址脚、字数的规格与整机一一对应，没有一根要假装不存在的线。三条路线里没有对错，只有“想让学习者看见什么”的差别。

把第二条路线再推到极端，还存在一种连 189 都省掉的历史方案：纯开关存储器。七十年代的 Altair 8800 前面板就是一排开关加一排灯，靠手拨开关输入引导

程序；再往前，早期计算机的操作员直接在控制台上拨地址和数据。那条路对 16 字节的小机器不是不能用——16 组各 8 只开关加 16 片八输入与门译码，就是一块“硬接线 ROM”——但 128 只开关的体积和 16 片译码器的接线量，换来的只是省掉两片 RAM，教学上是壮举、工程上是倒退。本书把它列为见识而非建议：知道存储器可以原始到就是一排开关，才能理解 189 那 16 个字是多么划算的文明。

DIP 开关路线的真正价值因此不在省钱，而在节奏。用编程器烧 EEPROM 是“整批交付”，错了一个字节要整批重来；DIP 开关是“逐字节对话”，每按一次按钮都能立刻读回核对。对第一次把程序送进机器的人来说，这种一个字节一次心跳的节奏，本身就是存储器语义的最好教具。

本节涉及的器件汇总如下，备料时照表清点即可；门器件一栏按需备，控制线的反相与门控都从这里出：

器件	数量	用途
74HC189	2	16 乘 8 RAM 本体
74HC240	1	反相还原加三态上总线
74HC173	1	MAR
74HC377	1	OUT 寄存器
74HC157	1	编程与运行的地址来源切换
74HC244	1	编程数据开关上总线
74HC14	1	按钮整形，兼作控制线反相
DIP 开关	12 位	编程地址 4 位加数据 8 位
按钮	1	编程写脉冲
LED 加 330 欧电阻	8 套	二进制显示
10 千欧电阻	12 只	开关的上拉或下拉
0.1 微法电容	4 只	各片电源去耦

MAR 是 4 位地址锁存器 MAR（存储器地址寄存器）只有 4 位，一片 74HC173 正好装下。它必须是锁存器而不是一段导线，原因在于地址要比总线活得久。取指时 PC 只在 T0 这一拍驱动总线，T1 总线就要还给 RAM 传送指令；如果 189 的地址直接取自总线，T1 总线内容一换，地址就丢了，指令会从一个错误的位置读出来。MAR 在 T0 拍末把地址锁存下来，此后无论总线怎么翻，189 的地址引脚始终稳定，直到下一次 MI 有效的拍末才更新。接法上，173 的两个装载使能 E1、E2 并接，经反相后由 MI 驱动；两个输出使能 OE1、OE2 直接接地——MAR 到 189 是一条点对点的专线，不经过共享总线，输出可以常开，不需要三态；清除端接整机复位，保证上电后地址是已知值，而不是一片随机花样。注意 173 的清除端 MR 是高有效，而整机复位链按低有效设计（§ 一里 273 的 $\overline{CLR}$ 、161 的 $\overline{CLR}$ 都是低有效），中间要加一级反相再接复位线，极性接反，这片 MAR 就会常年被钳在清零状态。

整个内存模块的连接关系先画成一张图：MAR 锁地址，157 选地址来源，两片 189 拼字宽，240 管读出边界。

场合	地址来源	有效控制线	数据方向
取指 T1	MAR (T0 自 PC 锁存)	R0 (另有 II、CE)	RAM 到 IR
LDA/ADD/SUB 的 T3	MAR (T2 自 IR 锁存)	R0 (另有 AI 或 BI)	RAM 到 A 或 B
STA 的 T3	MAR (T2 自 IR 锁存)	RI (另有 AO)	A 到 RAM
其余各拍	不变	无	RAM 不访问总线

三态是不是真的好，用一只电阻就能量出来。R0 无效时，把 240 的某个输出脚经 1 千欧电阻先后接到 5 V 和地各量一次：真正高阻的脚会被电阻先后拉成 1 和 0；如果电平固执不变，说明还有别的驱动器挂在上面——最常见的嫌疑人是 240 的使能接反、某片 189 的 W 被误接地、或相邻模块的某根输出使能恒有效。这个“电阻拉扯法”对全机每一条三态路径都适用，是 § 一总线检查的标准动作之一。

写拍的完整时序走一遍更有体感。以 STA 的 T3 为例，拍首上升沿一过，控制字 A0+RI 稳定：A 寄存器的输出缓冲打开，总线被驱动为 A 的值；MAR 里是上拍锁存的目标地址，早已稳定；时钟转入低半拍后，189 的 W 被拉低，写窗口打开，总线值流进选中字；拍末上升沿到来，时钟转高，W 先随门控电平关闭，控制字随后翻成下一拍的内容，A 的输出缓冲关闭，总线释放。整个写操作里，数据在写窗口打开前已经稳定了半拍，窗口关闭后地址数据才开始变动——建立与保持两条边界都靠“低半拍开窗”这一招天然满足。

STA 的 T3 一拍之内（时间从左到右）：

拍首上升沿	控制字 A0+RI 稳定
高半拍	A 驱动总线，MAR 地址早已稳定，W 保持高
转入低半拍	W 拉低，写窗口开，总线值写入 RAM[MAR]
拍末上升沿	时钟转高，W 先关，控制字再翻，总线释放

值得注意，MAR 在一条指令的生命里常常要被装两次。取指 T0 装一次（来自 PC），执行期 T2 可能再装一次（来自 IR 的地址字段）——LDA、ADD、SUB、STA 这四条访存指令都是如此。两次装载共用同一个 MI、同一条总线路径，不发生任何冲突，因为它们的用途首尾相接：第一次为了把指令取回来，第二次为了按指令里的地址去取数据。T2 的 IO+MI 与 T0 的 CO+MI 在微码表里是两行不同的控制字，驱动器不同、锁存者相同，这正是“总线轮值、寄存器复用”的标准画面。

最后回答一个容易被跳过的问题：MAR 为什么只需要 4 位。地址空间是 16 字节，4 位二进制正好数完 0 到 15，多一位浪费、少一位不够。这个“刚好”是整机规格联动的结果：PC 是 4 位、指令的地址字段是 4 位、189 的地址脚是 4 根，四处必须一致，任何一处扩到 5 位都会强迫其余三处跟着动。也正因如此，这台机器连第五章讲的地址译码器都不需要——16 个字就是全部存储，选中就是全部译码。将来第一轮扩展（比如 32 字节内存）要动的正是这一组联动的 4 位，§ 五会回到这个话题。

OUT 显示把寄存器值变成人可读结果 OUT 寄存器是全机唯一的“只进不出”状态：它在 OUT 指令的 T2 拍末锁存总线值（那一拍 A0 与 OI 同时有效，A 驱动总线，OUT 收数），此后一直驱动显示电路，直到下一条 OUT 指令执行才更新。显示的内容因此是“最近一次 OUT 时 A 的值”，而不是 A 的实时值——这个区别调试时要想清楚：程序继续跑，A 早已变了，数码管上留着的仍是 OUT 那一刻的快照。器件用一片 74HC377（八位 D 触发器，带低有效装载使能）正好：使能端接反相后的 OI，输出常开接显示。377 没有清除端也不要紧，上电显示一个随机值，第一条 OUT 执行后自然被覆盖。

显示的第一档方案是 8 只 LED 各串一只 330 欧电阻，直接挂在 377 的 8 根输出上，亮为 1、灭为 0。LED 有极性：长脚是正极，接 377 的输出端，短脚经电阻接地，接反了不亮也不坏，调过来就是。二进制读数需要心算，但它最忠实——每一位

都对应一根真实的导线，没有译码器可以怀疑。第二档方案是十六进制数码管：两位显示 00 到 FF，需要能把 4 位译成包括 A-F 在内的 16 种段码的驱动器，DM9368 这类十六进制译码驱动一体的芯片两片即可；也可以用一片小 EEPROM 自制译码——把 4 位二进制当地址、7 段码当内容烧进去，这是把微码“查表”的思想在显示端再用一次。这里有一个常见坑要点名：74HC47、74HC48 是 BCD 译码器，输入 1010 到 1111 时输出的不是 A-F，而是没有定义的段组合；拿它做十六进制显示，数值超过 9 就开始出怪字。至于十进制显示，需要先做二进制到 BCD 的转换，对本机得不偿失，不做。

限流电阻的取值是算出来的，不是抄来的。红色 LED 的正向压降约 2 V，74HC377 输出高电平带载后约 4.5 V，目标电流取 5 mA——足够亮，又不逼近 HC 器件单脚 25 mA 的输出上限——电阻值就是 $(4.5 - 2) / 0.005 = 500$ 欧，标准件里取 470 欧或 330 欧都成立：330 欧对应电流约 7.6 mA，亮度余量更足，这也是本书一律写 330 欧的原因。数码管的每一段就是一只 LED，同样的算法再算一遍，七段加小数点共八只限流电阻，一只都省不掉；省掉的代价是段亮度随字形变化——显示 1 时两段分一份电流显得亮，显示 8 时七段分显得暗。

选 EEPROM 自制译码这条路的，烧片时直接照抄下面这张十六进制字形表。段码按 g、f、e、d、c、b、a 的顺序排成 7 位，1 表示该段点亮，字形就是常见数码管上的 0 到 F：

数字	段码	数字	段码	数字	段码
0	0111111	6	1011111	C	0111001
1	0000110	7	0000111	D	1011110
2	1011011	8	1111111	E	1111001
3	1001111	9	1101111	F	1110001
4	1100110	A	1110111		
5	1101101	B	1111100		

表里的 B 和 D 用小写字形 (b、d) 是数码管的通行做法：大写 B 与 8、大写 D 与 0 在七段上无法区分，小写字形虽然委屈了字母，却保住了可读性。烧写时地址就是 4 位输入值，内容就是对应段码，16 行写完即收工——译码在这里不是逻辑设计，而是又一次填表。

最重要的一条边界是：显示只是观察，不能影响总线。OUT 的输出去向只有显示负载，整机里不存在任何一根“OUT 输出回总线”的控制线，所以显示部分无论怎么接——LED、数码管、示波器探头，乃至接错一个脚——都不可能改变总线上的任何电平。这条单方向边界值得在接线完成后当场核实：用万用表通断档量 OUT 的每个输出脚，它们应当只连到显示负载，不连总线。

数码管驱动还要做一个小小的架构选择：静态还是扫描。静态驱动是每位数管一套译码、常亮；扫描驱动是多位共用一套译码、轮流快速点亮，靠视觉暂留看起来同时亮。商用设备爱用扫描，因为省线省芯片；本机必须用静态，理由就在调试场景里——单步时钟和 HLT 停机时，扫描的驱动时钟也停了，数码管会只剩一位亮着甚至全灭，而静态显示在任何状态下都忠实保持 OUT 寄存器的内容。显示电路的存在是为了让机器“随时可看”，一个在停机时罢工的显示器，等于在最需要观察的时刻闭了眼。

上电瞬间 OUT 寄存器里是随机值，显示一片乱码，属正常现象——377 没有清除端，第一条 OUT 指令执行后乱码自然被正确的值覆盖。若执意让显示也从已知值出发，把 377 换成两片 74HC173（输出使能接地、装载使能接反相后的 OI、清除端接整机复位）即可，代价是多占一片的位置；多数搭建者的选择是不在意这片刻的乱码，因为程序的第一条显示指令之前，显示内容本来就没有约定。

OI 的时序与别的装载使能没有两样：OUT 指令的 T2，A0 让 A 驱动总线，OI 让 377 在拍末上升沿把总线值锁存。同一拍里 377 既不驱动总线也不影响任何别的模块，所以 OUT 是全部指令里最安静的一条——总线给它送数据，它什么也不用还。

摆放 OUT 指令的位置是编程风格问题，先在这里立个规矩：凡是想让操作者看到的中间结果或最终结果，显式地用一条 OUT 送出来；凡是要停机的地方，HLT 之前先放一条 OUT，否则停机后显示的是上一次 OUT 的旧值，结果等于没显示。§ 五的两个程序都按这个规矩写，读程序时可以对照。

显示方案	器件	能显示什么	关键问题
LED 8 只	330 欧电阻 8 只	二进制 8 位	最忠实，读数要心算
十六进制数码管两位	DM9368 两片	00-FF	芯片较老，注意共阴共阳与限流
EEPROM 自制译码	28C16 一片加数码管	任意自定义段码	查表思想的又一次复用
74HC47 两片	BCD 译码	只能正确显示 0-9	输入 A-F 出怪段，不适合十六进制

编程模式把程序逐字节拨进 RAM 编程模式要解决的本质问题，是让人手临时取代控制单元，在不与运行电路冲突的前提下操作 189。模式开关（一只拨动开关，以下称 PROG）拨到编程位时做三件事。第一，切断时钟：T 状态计数器停在原地，全部控制线随之静止，RI、RO 都为无效—RAM 既不被读也不被写，总线上没有任何运行中的驱动者。第二，74HC157 的选择端切到 DIP 地址开关一侧，189 的地址来源从 MAR 换成手指。第三，使能数据开关侧的 74HC244 和写按钮的通路：8 只数据开关经 244 驱动总线，按钮脉冲经 74HC14 整形后接 189 的 W。此后每装一个字节都是同一组动作：拨好 4 位地址、拨好 8 位数据、按一下写按钮。按钮抖动对这种写法其实无害—重复写同一个地址同一个值，结果与写一次相同；真正要紧的是次序，先把开关全部拨到位再按写，松手之后才允许动开关，否则在地址切换的瞬间，数据可能被写进别的字。

开关的电平方向要先定死再动手。本机的约定是开关拨到 ON 一侧时对应位为 1，实现上有两种接法：开关串在 5 V 与信号线之间、信号线经 10 千欧下拉电阻接地，断开为 0、闭合为 1，拨上去就是 1，最合直觉；或者开关一端接地、信号线经 10 千欧上拉，电平正好反过来。两种都对，但 12 只开关必须用同一种接法，并且在面板上用贴纸标明哪边是 1、哪只是最高位。高位低位标反是编程模式的头号人为故障：把 D0 当成 D7，写进去的字节正好按位倒序，程序的表现完全无法理喻，而电路没有任何毛病。

74HC157 的选择逻辑一行就能写完：选择端接 PROG 模式信号，运行位时 4 路输出取自 A 组—MAR 的 Q0-Q3；编程位时取自 B 组—4 只地址开关。157 的使能端接地常开，输出直送两片 189 的地址脚。MAR 的输出照常挂在 A 组输入上，不需要断开—选择器本身就隔离了两路来源，这正是用选择器而不用跳线的理由。写按钮一侧，74HC14 加 RC（10 千欧与 0.1 微法，时间常数 1 ms）把按钮的机械抖动滤成干净的一次跳变，输出接两片 189 的 W；编程模式下 RI 恒无效，运行电路对 W 没有任何影响。

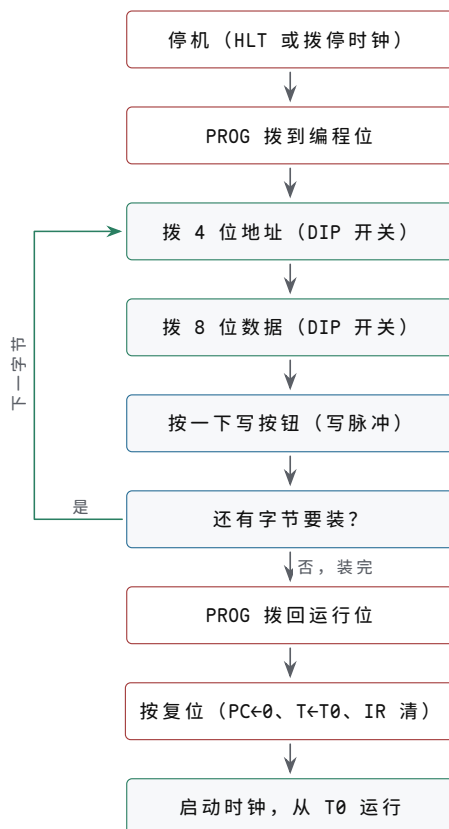
把两种模式下 189 各引脚的信号来源对照列出来，切换电路的设计就只剩照表施工：

信号	运行模式来源	编程模式来源
189 地址 A3-A0	MAR（经 74HC157 A 组）	4 只地址开关（经 157 B 组）
189 数据输入	数据总线（写拍由 A0 驱动）	数据总线（8 只数据开关经 244 驱动）
189 写使能 W	RI 与时钟低半拍相与后取反	写按钮经 74HC14 整形

240 输出使能	R0 反相	保持无效，RAM 不上总线
总线其它驱动者	各模块按控制字轮值	全部静默（时钟已停）
T 状态计数器	逐拍计数	冻结

模式切换本身要不要担心毛刺？不太要。切换发生在时钟停止的状态下，全机没有任何边沿在走，157 换源、244 开关、按钮通断，都是静态电平，此消彼长；毛刺之所以可怕，是因为它会冒充时钟边沿触发装载，而此刻没有装载对象醒着。唯一真正有破坏力的动作是在运行中把 PROG 拨过去——时钟被拦腰截断的那一拍可能窄到反常，触发器的行为不再可预期。所以纪律只有一条：先停机，后切换；先切回，后复位，再启动。

整个流程先画成一张图，再按步展开。



图示：编程模式流程。先停机再切换：PROG 拨到编程位后时钟冻结、74HC157 切到 DIP 地址、数据开关经 244 上总线、写按钮接管 189 的 W；每个字节都是“拨地址、拨数据、按写按钮”三步的循环。装完切回运行位后，复位不能省——它把 PC、T、IR 清到已知值，程序才从地址 0 的 T0 起跑。

完整流程七步：

1. 停机：程序自然跑到 HLT，或直接把时钟开关拨到停
2. 切模式：PROG 拨到编程位（时钟断、157 切到 DIP、按钮通路开）
3. 设地址：4 只地址开关拨出目标地址（0x0 到 0xF）
4. 设数据：8 只数据开关拨出要写入的字节
5. 写入：按一下写按钮，松开；回到第 3 步装下一字节
6. 切回：全部字节装完，PROG 拨回运行位
7. 复位运行：按复位（PC 清零、T 计数器回 T0、IR 清空），再启动时钟

第 7 步的复位不能省。编程期间 T 状态计数器和 PC 停在任意值，IR 里是上次停机时的指令残影；不复位就启动时钟，机器会从某个未知的 T 状态、某个未知的 PC 接着走，第一条执行的指令几乎必然不是程序的起点，后面的行为也就没有讨论意义。复位把 PC、T 计数器、IR 清到已知值，等于把机器放回一条确定的出发线上，程序从地址 0 开始、从 T0 开始，一拍一拍都可以对照 § 四的微操作表推演。

下面是一段 5 字节的写入示例。程序功能是把立即数 6 与存在 0x4 单元的 7 相加，显示结果后停机：

次序	地址开关	数据开关	字节	含义
1	0000	0101 0110	0x56	LDI 0x6: $A \leftarrow 6$
2	0001	0010 0100	0x24	ADD 0x4: $A \leftarrow A + \text{RAM}[0x4]$
3	0010	1110 0000	0xE0	OUT: 显示 A
4	0011	1111 0000	0xF0	HLT: 停机
5	0100	0000 0111	0x07	数据: 加数 7

5 个字节拨完、切回运行、复位、启动时钟，预期结果是显示 0000 1101，即 0x0D，十进制 13：LDI 把 6 装入 A，ADD 从 0x4 单元取出 7 相加，OUT 把 13 锁存进显示，HLT 停住时钟。如果显示的不是这个值，先别急着怀疑运行电路——编程模式下最容易错的是人：拨错的开关、漏按的按钮、忘了复位。逐字节核对与逐拍追踪的完整流程在 § 五展开。

有人会用一种正当的疑惑看待这套手工流程：既然程序固定，为什么不把它烧进 ROM，省去每次上电拨开关？答案是整机规格里“程序与数据同处一块 RAM”这半句——STA 指令要写内存，累加、乘法循环都要靠可写的单元保存中间结果，ROM 撑不起这些程序。编程模式不是权宜之计，是这台机器唯一合理的装程序方式。时间预算也算一笔：熟练后每字节约十秒，装满 16 字节三分钟，再加一遍读回核对，总共不超过十分钟；§ 五的两个程序连代码带数据都装在这 16 字节里，一次拨全。

把常见的人为错误集中在这里，装程序时对照自查。一是地址开关高低位拨反，字节按位倒序写入；二是数据开关拨错一位，程序能跑但结果错，最难怀疑；三是拨完忘记按按钮，该单元仍是旧值或上电随机值；四是按钮按了两次本无妨，但两次之间动了开关，数据进了相邻单元；五是切回运行位后忘了复位，机器从随机的 T 状态和 PC 起跑。这五种错里没有一种是电路故障——先用 DIP 开关装程序的另一重收获正在这里：把人会犯什么错，在最小规模上全部经历一遍。

读回核对的方法也说清。PROG 位下时钟是停的，控制单元帮不上忙，但核对只需要回答一个问题：这个地址里现在是什么。把地址开关拨到待核对的字节，再临时给 240 的使能一个有效电平——用一只带引线的按钮把 R0 反相后的节点接地即可——该字节就会经 240 出现在总线上，由 § 一的总线显示 LED 读出，核对完松开。这个动作本质上是手工重演了一遍 LDA 的 T3：地址给好、R0 打开、存储值上总线，只是每一步都来自手指。把 16 个地址逐个读一遍，与想写入的清单逐字节比对，比运行出错后再回头猜要省一个数量级的时间。

本节收尾给一份检查清单，按从上到下的顺序做，任何一项不符都先停下来排因，再往下走。

检查点	方法	预期结果
189 输出极性	预存 0xA5，R0 有效时量总线	总线读得 0xA5，而非 0x5A
R0 无效的三态	R0 无效时量 240 输出脚	高阻，总线电平由别人决定
MAR 锁存	单步到 T1，总线内容已换	189 地址脚仍保持 T0 锁存的地址
写窗口边界	执行 STA 后读相邻单元	只有目标地址被改写
OUT 显示	LDA 一个已知值后执行 OUT	LED 逐位等于该值
位序方向	预存 0x01，读出时看总线 D0	是最低位亮，而非最高位
编程读回	PROG 下逐地址读出 16 字节	与写入清单逐字节一致
切换纪律	运行中误拨 PROG 后复位重跑	复位后行为恢复正常

5 字节程序

按上表装入后复位运行

显示 0x0D 后停机

至此，机器有了记忆也有了脸面：RAM 能存能取，MAR 把地址看住，OUT 把结果留下，编程模式让人能把程序一个字节一个字节地请进内存。这一节反复出现的词是边界——RI 与 RO 的互斥边界、三态的高阻边界、显示的单向边界、模式切换的先后边界。下一节轮到控制单元登场，它的全部工作，就是按一张表逐拍打开和关上这一节装好的每一扇门。

四、控制单元：指令译码与微码

控制单元是全机的“读表机器”。它不再引入新的数据通路部件，只回答一个问题：当前这一拍，16 根控制线各自是有效还是无效。答案来自一张 128 行乘 16 位的表，行地址由两块状态拼成——IR 里的 opcode 说明“现在执行的是哪条指令”，T 状态计数器说明“现在走到这条指令的第几拍”。本节先把这三块部件（IR、T 计数器、微码 ROM）的接法讲清，再给出取指公共拍与全部 11 条指令的微操作表，最后讨论这张表烧进 EEPROM 和搭成门阵列两种实现的取舍。

把控制单元的输入输出先登记成表——它的接口之窄，本身就是微码方案的优点：

方向	信号	说明
入	时钟、复位	节拍推进与清零，§ 一的约定
入	总线 D0-D7	仅 T1 供 IR 装入指令
入	CF、ZF	仅 JC、JZ 的 J 选通使用
出	16 根控制线	全机所有模块的使能与方式

控制单元 = IR + T 状态计数器 + 微码 ROM IR 在 T1 拍末从总线装入指令字节（II 有效），此后整条指令的执行期间它保持不变——这是它与 A、B 这些“数据寄存器”的本质区别：数据寄存器装的是会被加工的值，IR 装的是要管好几拍的命令。本机用两片 74HC173 拼成 8 位 IR，两片的分工不是随意的高低位，而是按去向划的：高片存 opcode（IR7-IR4），输出使能接地常开，4 根输出线直接送往微码 ROM 的地址脚；低片存地址或立即数字段（IR3-IR0），输出使能由 IO 控制，IO 有效时这 4 位经三态上总线低 4 位。两片共用同一个时钟与装载使能（E1、E2 并接，经反相接 II），清除端接整机复位——与 MAR 一样，MR 是高有效，需经一级反相接低有效的复位线。

这个分工里藏着一条重要的接线纪律：opcode 永远不经过总线。IR 高片到 ROM 地址脚是一条点对点专线，就像 MAR 到 RAM 的那条一样——凡是“控制信息”，在本机里都走专线，总线只运“数据”。好处不止是不争用：取指完成的同一瞬间，opcode 已经摆在 ROM 地址脚上，T2 的控制字不经过任何中转就开始译出。按 § 一的总线约定，总线高 4 位在无人驱动时读为 0，所以 IO 有效时总线呈现的是“高 4 位为 0、低 4 位是指令字段”，LDI 装入 A 的自然是零扩展后的 4 位立即数，JMP 装给 PC 的也正好是 4 位地址，不多不少。

两片 173 的接法对照如下，与 § 三 MAR 那片 173 是同一片型、三种用法：

引脚	IR 高片（opcode）	IR 低片（地址/立即数）
D0-D3	总线 D4-D7	总线 D0-D3
CP	整机时钟	整机时钟
E1、E2	并接，经反相接 II	并接，经反相接 II
OE1、OE2	接地，常开	并接，经反相接 IO
MR	整机复位	整机复位
Q0-Q3	微码 ROM 地址高 4 位	总线低 4 位

注意两片唯一的差别在输出使能：高片常开、低片受控。这个差别在排错时是个趁手的判据——如果 opcode 出现“时有时无”，先查是不是把高片的 OE 也接到了 IO

上；如果 I0 拍总线低 4 位不动，先查低片的 OE 是不是忘了接。

T 状态计数器是一片 74HC161，只用低 3 位输出 Q2、Q1、Q0——它们就是拍号的三位二进制编码，000 是 T0，111 是 T7。计数使能 CEP、CET 接高电平，时钟脚经一级反相接入，每个时钟下降沿加一——T 状态在拍中推进，新控制字到下一个上升沿前半拍就已稳定，装载沿采到的因此是毫不含糊的电平。回绕用同步装载实现：74HC11 的一个三输入与门译出 111，接 161 的同步装载使能 PE（低有效），并行数据输入 D3-D0 全部接地；计数到 T7 的那一拍，装载条件成立，拍末下降沿把 000 装进去，下一拍回到 T0。第 4 位输出 Q3 悬空不用，它永远不会变成 1。

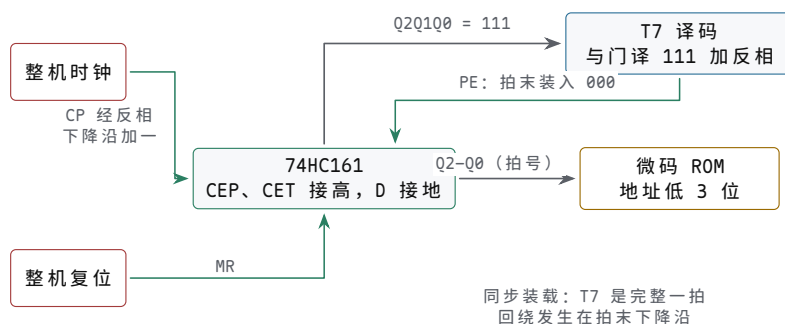
把八个拍与计数器状态、回绕动作的对应列出来，单步调试时对照 Q2-Q0 就知道机器走到哪一拍：

拍	Q2 Q1 Q0	ROM 地址低 3 位	拍末发生什么
T0	0 0 0	000	计数加一
T1	0 0 1	001	计数加一
T2	0 1 0	010	计数加一
T3	0 1 1	011	计数加一
T4	1 0 0	100	计数加一
T5	1 0 1	101	计数加一
T6	1 1 0	110	计数加一
T7	1 1 1	111	PE 有效，装入 000 回 T0

161 的其余引脚——交代：CP 经一级反相接整机时钟，于是计数在时钟下降沿发生；CEP、CET 并接高电平；PE 接 74HC11 译出的 T7（低有效，故与门输出要反相一次——用 74HC11 加 74HC04 的一门，或干脆用一片 74HC10 三输入与非门，省掉反相器）；D0-D3 接地；MR 接整机复位；Q3 与进位输出 TC 悬空。整片 161 在本机里扮演的角色等价于一个“八状态的模 8 计数器”，只是用同步装载拼出来的模 8，比专用模 8 器件更容易买到。

这里必须强调用同步装载而不用异步清零的理由。若把译出的 T7 接去 161 的异步清零端 MR，T7 这个状态只存在译码延迟加清零延迟那么几十纳秒，随即被抹掉——T7 行的控制字来不及稳定，依赖 T7 的扩展指令就全部出错；更糟的是这个窄脉冲本身是个毛刺源。同步装载让 T7 是和别的拍一样完整的一拍，回绕发生在拍末边沿，干干净净。161 的 MR 另有正经用途：接整机复位，保证上电和每次复位后第一拍是 T0。

把这片 161 和它的回绕译码画成结构图，引脚去向一并标出：

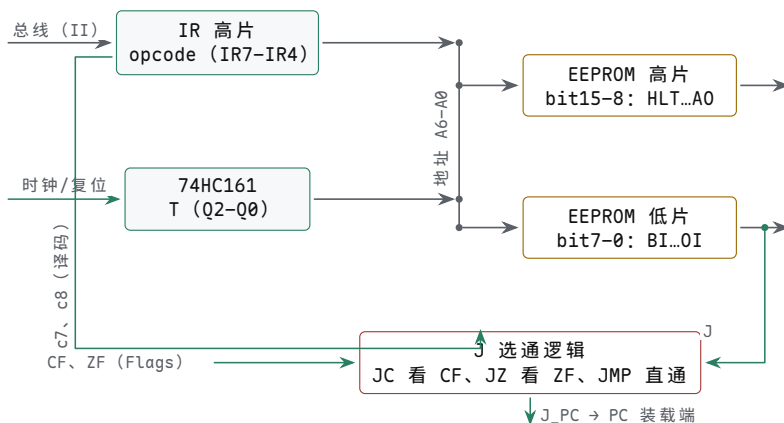


图示：T 状态计数器结构。74HC161 的 CEP、CET 接高电平，时钟经一级反相接 CP——每个下降沿加一，新拍号到装载沿前半拍已经稳定；Q2-Q0 直接作微码 ROM 地址的低 3 位。回绕用同步装载：三输入与门译出 111（T7），反相后接 PE，并行输入 D3-D0 全接地，T7 拍末的下降沿把 000 装入，下一拍回到 T0——T7 因此是和别的拍一样完整的一拍，不像接异步清零那样只剩几十纳秒的窄脉冲。MR 另有正经用途：接整机复位，保证上电和每次复位后第一拍是 T0。

微码 ROM 是这张表的物理载体。表有 128 行，行地址 7 位：高 4 位接 IR 的 opcode，低 3 位接 T 计数器的 Q2-Q0，即 ROM 地址 = opcode 乘 8 加 T。每条指令因此占据连续 8 行：第 0、1 行是取指，第 2-7 行是执行拍，本机最长的指令只

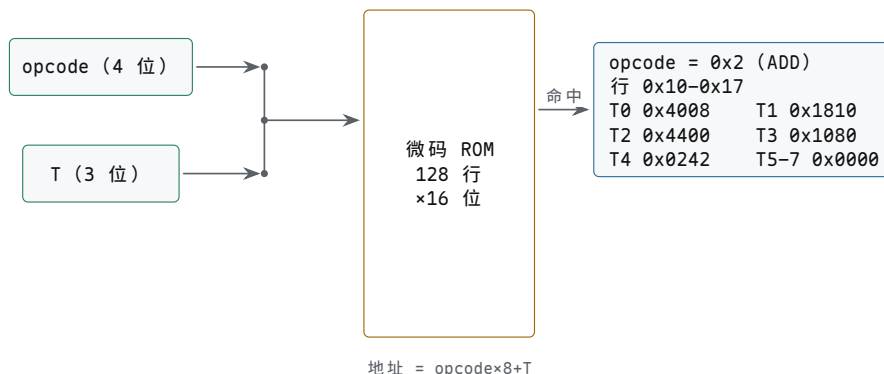
用到第 4 行。每行 16 位，正好对应 16 根控制线，用两片 8 位 EEPROM (AT28C64、28C16 均可) 并联地址实现：两片共用全部 7 根地址线，高片输出控制字的高 8 位 (HLT 到 A0)，低片输出低 8 位 (BI 到 0I)，16 根控制线一次出齐，一根不浪费。EEPROM 的片选与输出使能都接地常开——控制字必须随时有效，控制单元没有休息的权利；用不到的高位地址脚全部接地，28C64 有 8K 行，本机只用其中 128 行，余下的空间留给扩展。

三块部件的接线关系画成图，就是一台查表机器。



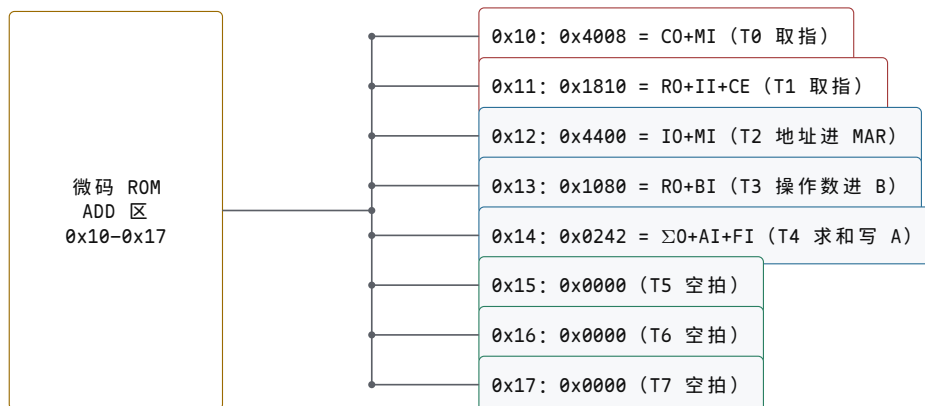
图示：控制单元。IR 高片的 opcode (4 位) 与 74HC161 的 T 状态 (Q2-Q0, 3 位) 拼成 7 位地址，并送到两片 EEPROM，每片出 8 位，合起来正好 16 根控制线；地址 = opcode×8+T，每条指令占连续 8 行。J 是唯一的例外：它出 ROM 后还要过一级选通——c7、c8 由 opcode 译出，JC 时看 CF、JZ 时看 ZF、JMP 时直通——选通是纯组合逻辑，控制字整拍稳定，拍末采到的是确定电平。

地址怎么拼、表怎么查，再单画一张示意。



图示：微码查找。7 位地址由高 4 位 opcode 与低 3 位 T 拼成 (地址 = opcode×8+T)，每条指令占据 ROM 中连续 8 行：第 0、1 行是所有指令相同的取指 (C0+MI、R0+II+CE)，第 2 行起按指令分化，用不满的行填 0x0000。图中展开 ADD (opcode 0x2) 的 8 行：T2 送操作数地址、T3 操作数进 B、T4 求和写 A 并记标志，其余三行空转。

把命中区的 8 行再逐行摊开，行地址、控制字、动作三者一一对应：



图示：ADD (opcode 0x2) 在微码 ROM 中的 8 行逐行展开，行地址 = opcode 乘 8 加 T。T0、T1 是 16 个 opcode 完全相同的公共取指 (0x4008 = C0+MI、0x1810 = R0+II+CE)；T2 把指令里的地址字段送进 MAR，T3 把 RAM 读出的操作数装进 B，T4 打开 ALU 三态输出，A 收 A+B 并锁存标志；T5-T7 三行控制字全 0，总线空闲、无装载，等回到 T0 取下一条。

有读者会注意到一个看似浪费的结构：取指两行在每个 opcode 下重复存了 16 份，32 行内容两两相同。为什么不把取指单独译出来、让 ROM 只存执行拍？因为省不下什么——单独译取指需要额外的“T0、T1 一律放行”逻辑，等于在 ROM 之外再造一小块硬连线译码，而 ROM 里的行本来就是免费的：128 行用满是本机的规格，空着也不会变快。把全部规则留在一张表里，换来的是核对时只需面对一个对象。这个“宁可在表里重复、也不在表外加逻辑”的取向，与第三章对微码控制的评价一致：复杂度进了内容，结构就退到最简。

控制线的根数也在这里交代清楚。16 根不是算出来的理论下限，而是数据通路里每一扇“门”各配一根的结果：八个寄存器或部件的装载使能 (MI、II、AI、BI、OI、FI、J、RI)，五路总线输出使能 (C0、R0、I0、A0、Σ0)，一个 ALU 修饰 (SU)，一个计数使能 (CE)，一个停机 (HLT)。少一根，某扇门就只能常开或常关；多一根，也只是表里多一位恒 0。将来扩展指令时要做的第一件事，就是回头数这张门清单——加寻址方式可能要加输出使能，加状态寄存器可能要加装载使能，两片 ROM 的 16 位就是这时开始不够用的。

控制线在控制字里的位序是任意定的，但一旦定下就是全机的约定，烧表、接线、排错都按它来。本机的位序如下：

位	控制线	有效时的动作	所在片
bit15	HLT	切断时钟源，全机冻结	高片
bit14	MI	拍末 MAR ← 总线低 4 位	高片
bit13	RI	写窗口内 RAM ← 总线	高片
bit12	R0	RAM 选中字经 240 上总线	高片
bit11	II	拍末 IR ← 总线	高片
bit10	I0	IR 低 4 位上总线低 4 位	高片
bit9	AI	拍末 A ← 总线	高片
bit8	A0	A 上总线	高片
bit7	BI	拍末 B ← 总线	低片
bit6	Σ0	ALU 结果上总线	低片
bit5	SU	ALU 改做减法 (B 取反加一)	低片
bit4	CE	拍末 PC ← PC + 1	低片
bit3	C0	PC 上总线低 4 位	低片
bit2	J	拍末 PC ← 总线低 4 位	低片
bit1	FI	拍末 Flags ← ALU 的 ZF、CF	低片
bit0	OI	拍末 OUT ← 总线	低片

全机的节拍规则只有一条：所有装载类动作——名字以 I 结尾的控制线，外加 CE、J、FI——都在时钟上升沿同时发生。T 状态计数器在时钟下降沿推进 (161 时钟脚经一级反相)，此后 ROM 地址随之改变，新控制字经过触发器延迟加 ROM 读取延迟后稳定，到下一个上升沿前半拍就已就位。输出类控制线 (C0、R0、I0、A0、Σ0) 是电平有效的整拍驱动，RI 是拍内低半拍的一段写窗口。控制字在拍内稳定、寄存器在上升沿装载——这十个字就是多周期控制器的全部节奏。

取指的两拍对所有指令相同 取指为什么必须是公共前奏，想一想 T0 开始时机器知道什么就明白了：IR 里还是上一条指令的残值，PC 指着下一条指令的地址，而“下一条指令是什么”要取回来才知道。在不知道指令是什么的前提下，唯一正确的动作就是把 PC 指的字节取回来——所以 T0、T1 两拍的控制字对全部 16 个 opcode 完全相同，微码表里每个 opcode 的第 0、1 行内容一模一样。这不是浪费，是“取指必须先于译码”这个因果关系的直接抄写。

T0 的控制字是 C0+MI：PC 经三态缓冲驱动总线低 4 位，拍末 MAR 把这 4 位

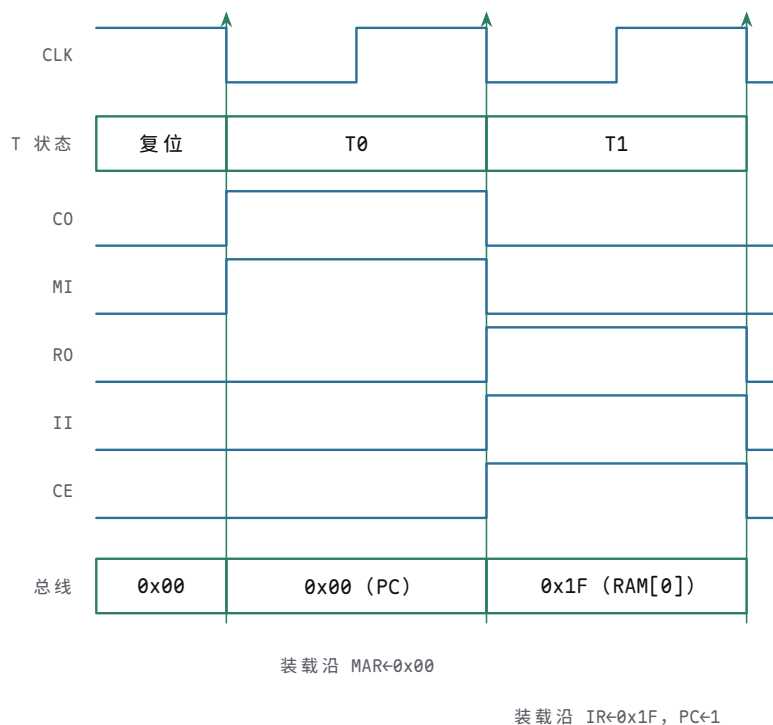
锁存一下下一条指令的地址就此交给内存。T1 的控制字是 $R0+II+CE$ ：RAM 把 MAR 选中的字（即指令字节）经 240 驱动上总线，拍末同时发生两件互不相干的事——IR 锁存总线值，指令到手；PC 在 CE 作用下加一，为再下一次取指备好地址。CE 与 II 同拍是安全的：PC 计数和 IR 装载共用同一个边沿，但一个改的是 PC 内部状态，一个改的是 IR，互不影响；而此时 PC 的输出缓冲是关的，总线上唯一的驱动者是 RAM，CO 与 R0 从不在同一拍出现。

先把这两拍的数据流画成通路图，谁驱动总线、谁在拍末装载，可以与波形逐拍对照：



图示：取指两拍的数据流（对照本节波形图）。T0：CO 打开 PC 的三态缓冲，地址经总线低 4 位送出，拍末 MI 把它锁进 MAR——此后地址走 MAR 到 RAM 的点对点专线，不再依赖总线；T1：R0 让 RAM 把 MAR 选中的指令字节经 240 驱动上总线，拍末 II 锁进 IR，同拍 CE 让 PC 内部加一。总线一拍只服务一对收发，所以取指至少两拍；两拍的驱动者分别是 CO 与 R0，从不在同一拍出现。

把这两拍画成波形，沿与电平的关系一眼可见。



图示：取指两拍（复位后的第一条指令，对照 § 五程序一： $RAM[0]=0x1F$ ）。T0：CO 让 PC 驱动总线，装载沿（上升沿）MI 把地址 $0x00$ 锁进 MAR；T1：R0 让 $RAM[0]$ 上总线，装载沿 II 把它锁进 IR、CE 让 PC 加一。三根参考虚线是 T 状态翻转的下降沿：T 计数器在下降沿推进，新控制字到装载沿前半拍已经稳定，所有装载都在拍内上升沿发生。这两拍对全部 16 个 opcode 完全相同。

T 拍	控制字	有效控制线	总线驱动者	拍末动作
T0	0x4008	CO、MI	PC	$MAR \leftarrow PC$

T1 0x1810 RO、II、CE RAM IR ← RAM[MAR], PC ← PC+1

控制字读不顺的时候，把它拆回位再数一遍是最快的自检。0x4008 展开是 0100 0000 0000 1000: bit14 为 1 是 MI, bit3 为 1 是 CO, 合起来 CO+MI; 0x1810 展开是 0001 1000 0001 0000: bit12 为 RO、bit11 为 II、bit4 为 CE。任何一行控制字都经得起这样拆——拆出来的控制线集合与微操作表不一致时，错的要么是位序表，要么是你的加法，而位序表只有一张，加法可以重算。

T1 拍末是个值得停下来看一秒的时刻：IR 刚换上新指令，ROM 地址高 4 位跟着换新，从 T2 起控制字开始按指令分化——取指的公共性到此为止。还要说一句执行拍用不满的指令：本机所有指令固定走满 T0 到 T7 八拍，T4 或 T2 就做完了的指令，余下的拍控制字全 0，总线空闲、无装载，机器空转到 T7 再回 T0。固定八拍换来的是控制单元里没有任何“提前结束”逻辑——想省这些空拍，需要加一套按 opcode 让 T 计数器提前回零的预置电路，电路不难，但每多一种拍数，控制字表就多一列要核对的例外。本机选择不省，把简单留给核对。

两个相关的问题一并回答。其一，取指为什么不能并成一拍：T0 做的是“PC 经总线进 MAR”，T1 做的是“RAM 经总线进 IR”，两件事都依赖总线，而总线一拍只能服务一对收发——把两拍并成一拍，等于要求总线在同一拍先送地址、再送指令，这是两个总线周期，不是一个。单端口内存加单总线的结构，注定取指至少两拍。其二，复位之后机器的第一动作长什么样：复位把 PC 清为 0、T 计数器清为 T0、IR 清为 0x00，ROM 地址是 opcode 0 的第 0 行，控制字正是 CO+MI——机器从地址 0 的取指开始，而 IR 里的 NOP 残值保证了即使有什么使能残存，执行的也只是空操作。设计的每个已知初值都指向同一个出发线。

取指是两拍里最容易核对的一段，建议第一次上电就把它核熟。单步时钟，T0 时看总线 LED 应显示 PC 的值（第一次是 0x0），拍末 MAR 的四个输出脚应变成同一个值（用表笔或给 MAR 也挂一组 LED 都行，调试期值得挂）；T1 时总线应显示 RAM 该单元的内容，拍末 IR 的高 4 位 LED 应变成该字节的 opcode。三个量、两个沿，全对，说明 PC、MAR、RAM、IR、T 计数器、ROM 取指行这一整条链是通的；任何一环不对，都停在这两拍里排，不要往后走。

11 条指令的逐条微操作表 下面这张全表是本章的心脏。读表约定：每格列出该拍有效的控制线，“—”表示该拍控制字为 0x0000，无动作；所有装载类动作在该拍末尾的上升沿完成。JC、JZ 两行里的 J 带星号，含义随后说明。

指令	T2	T3	T4	T5-T7
NOP	—	—	—	—
LDA	IO+MI	RO+AI	—	—
ADD	IO+MI	RO+BI	Σ0+AI+FI	—
SUB	IO+MI	RO+BI	SU+Σ0+AI+FI	—
STA	IO+MI	AO+RI	—	—
LDI	IO+AI	—	—	—
JMP	IO+J	—	—	—
JC	IO+J*	—	—	—
JZ	IO+J*	—	—	—
OUT	AO+OI	—	—	—
HLT	HLT	—	—	—

逐条看一遍这张表在说什么。NOP 取指后空转六拍，唯一的作用是占时间。LDA 用 T2 把指令里的地址字段送进 MAR，T3 把该地址的字读进 A。ADD 比 LDA 多一拍：操作数先进 B (T3)，T4 才打开 ALU 的三态输出，把 A+B 送回 A，同拍 FI 把新的 ZF、CF 记进 Flags——记住标志的办法是在它还新鲜的那拍就把它锁存。SUB 与 ADD 同构，T4 多一根 SU: B 经 74HC86 取反、进位输入置 1，加法器算出的就

是 A-B。STA 是唯一写内存的指令，T3 的 A0+RI 让 A 驱动总线、RAM 在低半拍收数。LDI 是最快的装载，IR 低 4 位直接进 A，连 MAR 都不惊动。三条跳转里，JMP 无条件把指令字段装进 PC；JC、JZ 外形与 JMP 相同，差别就在那个星号上。OUT 把 A 抄给显示。HLT 在 T2 拉起 HLT 线， ϕ 一的时钟与门随即关闭，T 计数器冻结，全机停在原地——退出停机的唯一办法是复位：复位清 T 计数器和 IR，HLT 线随之撤销，时钟恢复。

把每条指令的执行要领浓缩成一栏，与上表对照着记：

指令	执行期要领
NOP	取指之后空转六拍，只占时间
LDA	先送地址 (T2)，再读数进 A (T3)
ADD	操作数先进 B (T3)，再整加写 A 并记标志 (T4)
SUB	同 ADD，T4 加 SU：B 取反加一即减
STA	唯一写内存：A0 与 RI 同拍 (T3)
LDI	最快的装载：IR 低 4 位直接进 A (T2)
JMP	无条件：IR 低 4 位进 PC (T2)
JC	同 JMP 的外形，CF 为 1 才装载
JZ	同 JMP 的外形，ZF 为 1 才装载
OUT	把 A 抄给显示，总线只送不还 (T2)
HLT	关掉时钟源，全机冻结，唯复位可退出 (T2)

星号的含义是条件装载。微码 ROM 在 JC、JZ 的 T2 行照常输出 J 请求，但这根线到 PC 的装载端之前要过一级标志选通：仅当对应标志成立时才放行。用 IR 高 4 位译出 c7（当前指令是 JC）和 c8（当前指令是 JZ）两条线，选通逻辑如下：

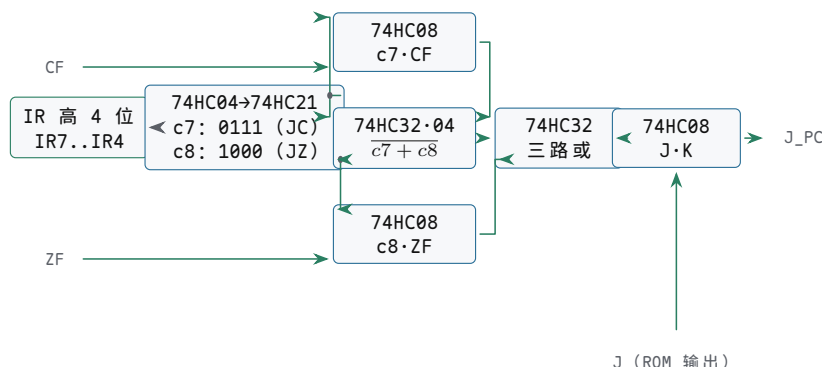
```

c7 = (opcode == 0111)      ; IR7..IR4 = 0x7, 由 74HC21 译出
c8 = (opcode == 1000)      ; IR7..IR4 = 0x8, 同上
J_PC = J AND ( (c7 AND CF) OR (c8 AND ZF) OR NOT(c7 OR c8) )

```

逐项检查这个式子：JMP (0x6) 时 c7、c8 都为 0，第三项为 1，J 原样放行，无条件跳；JC 时 c7 为 1，式子退化成 J AND CF，进位标志说了算；JZ 同理退化成 J AND ZF；其它指令的 T2 行里 J 本来就是 0，选通无影响。器件账：反相器用 74HC04，两个四输入译码用一片 74HC21，尾部的与或用 74HC08 和 74HC32 各一片，合计四片，全部接在 ROM 的 J 输出脚与 PC 装载端之间。选通是纯组合逻辑，控制字整拍稳定，标志早在前面的 ADD 或 SUB 里就锁存好了，拍末边沿采样到的 J 是毫不含糊的电平。

把这片选通网络画成门级图，四片 74HC 的分工如下。



图示：J 的条件选通逻辑。IR 高 4 位经 74HC04 按位取反、74HC21 四输入译码，分出 c7 (opcode=0111, JC) 与 c8 (opcode=1000, JZ) 两条指令线；74HC08 把 c7、c8 分别与 CF、ZF 相与，74HC32 加 74HC04 给出第三项 (c7、c8 都为 0，对应 JMP)；三路经 74HC32 相或后再与 J 相与，得到送进 PC 装载端的 J_PC。JMP 走第三项无条件放行，JC、JZ 由各自标志裁决，其余指令的 T2 行 J 本就为 0，选通无影响。

条件不成立时的“顺序执行”值得拆开看一眼，因为它其实是免费的：JC 的 T1

里 CE 已经让 PC 加过一，PC 指向的就是下一条指令；T2 的 J 被标志拦下，PC 保持不动，回绕后的 T0 照常把这个已加一的值送去 MAR——不跳转不需要任何专门动作，它只是“装载没有发生”。跳转目标的语义也借这里定死：装进 PC 的是指令低 4 位，一个绝对地址，所以本机的跳转只能落在 0 到 15 这 16 个字节里，没有相对跳转、没有偏移量——程序多大，跳转范围就多大，这就是 4 位 PC 的世界观。

最后交代空闲拍的总线。T5 到 T7 的控制字全 0，没有任何驱动者占用总线，总线电平由 § 一的总线下拉电阻给出确定值——读作 0x00。这个约定不是装饰：没有它，空闲拍浮空的总线会在 0 与 1 之间漂，虽然无人装载、逻辑无害，但总线 LED 会无规律地闪烁，单步调试时平白添一层噪声。让所有无人使用的观测量都有确定读数，是这台机器“每一步可观察”的最后一块拼图。

把微操作表逐列翻译成 16 位控制字，再按“地址 = opcode 乘 8 加 T”排列，就是要烧进两片 EEPROM 的全部内容。128 行里非零的只有下面这些——外加每个 opcode 都相同的两行取指——其余一律 0x0000，包括未使用的 opcode 0x9 到 0xD：它们的执行拍全空，行为与 NOP 无异，程序里误用了不会破坏状态，但也不会有什么作为。

ROM 地址	控制字	含义
op*8+0	0x4008	T0 取指：C0+MI，16 个 opcode 相同
op*8+1	0x1810	T1 取指：R0+II+CE，16 个 opcode 相同
0x0A	0x4400	LDA T2: I0+MI
0x0B	0x1200	LDA T3: R0+AI
0x12	0x4400	ADD T2: I0+MI
0x13	0x1080	ADD T3: R0+BI
0x14	0x0242	ADD T4: $\Sigma 0$ +AI+FI
0x1A	0x4400	SUB T2: I0+MI
0x1B	0x1080	SUB T3: R0+BI
0x1C	0x0262	SUB T4: $SU+\Sigma 0$ +AI+FI
0x22	0x4400	STA T2: I0+MI
0x23	0x2100	STA T3: A0+RI
0x2A	0x0600	LDI T2: I0+AI
0x32	0x0404	JMP T2: I0+J
0x3A	0x0404	JC T2: I0+J, J 经 CF 选通
0x42	0x0404	JZ T2: I0+J, J 经 ZF 选通
0x72	0x0101	OUT T2: A0+OI
0x7A	0x8000	HLT T2: HLT

两表的一致性要逐列核对，方法很机械：从微操作表的每一格出发，按位序表把控制线翻成位、加出控制字，再到 ROM 表里按 opcode 乘 8 加 T 找到那一行，两者必须相同。以 ADD 的 T4 为例： $\Sigma 0$ 是 bit6、AI 是 bit9、FI 是 bit1，控制字为 $0x0040+0x0200+0x0002 = 0x0242$ ；ADD 的 opcode 是 0x2，地址为 $0x2*8+4 = 0x14$ ——ROM 表第 0x14 行正是 0x0242。其余 17 个非零行照此全部核一遍，这是烧片前不可省略的一步，错一位就是一条指令的隐疾。

最后把 ADD 的完整六拍走一遍，作为两张表的联合读法（设 $A=0x06$ ， $RAM[0x4]=0x07$ ，该指令存在地址 0x1；清单是纯文本，把控制线 $\Sigma 0$ 写作 S0）：

T0	C0+MI	总线 $\leftarrow PC(0x1)$	拍末 MAR $\leftarrow 0x1$
T1	R0+II+CE	总线 $\leftarrow RAM[0x1]=0x24$	拍末 IR $\leftarrow 0x24$, PC $\leftarrow 0x2$
T2	I0+MI	总线 $\leftarrow IR$ 低4位=0x4	拍末 MAR $\leftarrow 0x4$
T3	R0+BI	总线 $\leftarrow RAM[0x4]=0x07$	拍末 B $\leftarrow 0x07$
T4	S0+AI+FI	总线 $\leftarrow A+B=0x0D$	拍末 A $\leftarrow 0x0D$, Flags $\leftarrow \{ZF=0, CF=0\}$
T5..T7	--	总线空闲，无装载，等回绕到 T0 取下一条	

微码 ROM 的两种实现与门阵列对照 EEPROM 路线的全部制造工作就是上面两

张表：算好 128 个 16 位控制字，高字节烧进高片、低字节烧进低片，插上板子，控制单元就完工。它的脾气是“规则写在内容里”——改指令行为就是改表重烧：想让 LDA 顺便清标志，把 0x0B 行加上 FI 位即可；想添一条新指令，把空着的 opcode 0x9 的 8 行填上即可，面包板上一根线不动。烧写用通用编程器最省事；按 § 三编程模式的思路用开关加按钮自编程也行，EEPROM 在 5 V 下的字节写入时序与 RAM 相仿，只是每字节要留足器件手册要求的写入时间，典型是几毫秒。手工填 128 行容易出错，用一段几行的脚本生成更稳妥：

```
W = [0x0000] * 128
for op in range(16):
    W[op*8+0] = 0x4008      # T0: CO+MI (所有指令相同)
    W[op*8+1] = 0x1810      # T1: RO+II+CE
def put(op, t, word): W[op*8+t] = word
put(0x1,2,0x4400); put(0x1,3,0x1200)          # LDA
put(0x2,2,0x4400); put(0x2,3,0x1080); put(0x2,4,0x0242) # ADD
put(0x3,2,0x4400); put(0x3,3,0x1080); put(0x3,4,0x0262) # SUB
put(0x4,2,0x4400); put(0x4,3,0x2100)          # STA
put(0x5,2,0x0600)                             # LDI
put(0x6,2,0x0404)                             # JMP
put(0x7,2,0x0404); put(0x8,2,0x0404)          # JC/JZ (J 经标志选通)
put(0xE,2,0x0101)                             # OUT
put(0xF,2,0x8000)                             # HLT
# 高字节写高片，低字节写低片；未置的行保持 0x0000
```

烧完不等于烧对，上板之前先把 ROM 内容读回来核一遍：编程器都有读出校验功能，把两片各 128 行读出，与源表逐字节比对，全对再插。跳过这一步直接上板的后果，是把“烧写错误”混进“接线错误”里一起排——两类故障在面包板上的症状一模一样，而区分它们只需要一次读回。养成“凡可写必先读回”的习惯，在 § 三的 DIP 开关和这里的 EEPROM 上是同一条纪律。

条件跳转在 EEPROM 路线上还有第二种做法，值得一提：把 CF、ZF 直接接成 ROM 的第 8、9 根地址线，表从 128 行扩成 512 行，JC、JZ 的 T2 行按标志的四种组合各写一份——成立的位置写 0x0404，不成立的位置写 0x0000。这样 J 不再需要片外选通，四片门省掉了，代价是表大四倍、烧写时要多一层小心。这是“存储空间换组合逻辑”的典型交易，28C64 的容量完全付得起。

门阵列路线把同一张表用组合逻辑“算”出来：一片 74HC154 (4 线到 16 线译码器) 把 opcode 译成 16 条指令线，一片 74HC138 (3 线到 8 线译码器) 把 T 译成 8 条拍线，每根控制线就是若干“指令线乘拍线”的或。两片的输出都是低有效，实际搭的时候用与非的视角更顺手；下面的式子按高有效写出逻辑关系：

```
MI = T0 + T2*(LDA+ADD+SUB+STA)
RO = T1 + T3*(LDA+ADD+SUB)
II = T1
CE = T1
CO = T0
IO = T2*(LDA+ADD+SUB+STA+LDI+JMP+JC+JZ)
AI = T2*LDI + T3*LDA + T4*(ADD+SUB)
BI = T3*(ADD+SUB)
SO = T4*(ADD+SUB)          ; SO 即正文所说的加法器结果输出使能
SU = T4*SUB
FI = T4*(ADD+SUB)
AO = T2*OUT + T3*STA
RI = T3*STA
J = T2*(JMP + JC*CF + JZ*ZF)
OI = T2*OUT
HLT = T2*opF              ; opF 即 HLT 指令的译码线
```

注意 J 那一行：条件跳转的标志选通在门阵列里被天然吸收进译码式，不再需

要 EEPROM 方案那级片外选通——这是门阵列少有的扳回一分的地方。其余各行读法相同：每根控制线追溯下去，不过是几条指令线、几条拍线、最多两根标志的组合。代价在器件数量和可改性：译码两片加与或非门约十片，总共十二片上下，占用整整一块面包板；更实质的代价是改一条指令的行为就要动接线，改上三五次之后，那团线的正确性就只剩当事人敢打包票。

维度	EEPROM 微码	74HC 门阵列
规则写在哪儿	ROM 的内容，即数据	门的接线，即结构
改一条指令	重烧几行，硬件不动	改接线
器件数	两片，加选通门四片	十二片上下
控制字延迟	触发器加 ROM 读取，约两百纳秒	触发器加几级门，几十纳秒
扩展条件跳转	加选通门，或把标志接成地址线扩表	直接写进译码式
调试方式	把 ROM 内容读回，与表逐行比对	逐级量门输出，对照译码式
适合阶段	指令集还在调整期	指令集冻结、想省掉 ROM 时

两条路线与第三章、第四章的对照是同一个故事的实物版。第三章比较过硬连线、状态机与微码三种控制器组织方式，第四章在最小 CPU 上演示过同一张状态表的两种实现：一边是“状态触发器加次态逻辑加输出译码”，一边是“每行一条微指令的控制存储器”。本节的 EEPROM 与门阵列正是那一组取舍落在面包板上的样子——两边每一拍输出的控制线必须一根不差，差别只在规则存放在内容里还是接线里。务实的建议是用 EEPROM 把机器先跑通，表错了重烧就是；等指令集稳定，再把门阵列版当作一次“把存储器译码展开成逻辑”的练习，两边对照着量同一根控制线，是理解那两章取舍表最直接的方式。速度差异在这里不成议题：两百纳秒对几十纳秒，在百赫兹量级的教学时钟面前都是四个数量级的余量。

两条路线各有自己的高发故障，排法也不同。EEPROM 路线的高发故障是位序类：高低两片插反（控制字高低字节对调，症状是全机行为荒诞但隐约像样）、位序接反（每片内部 D0 到 D7 颠倒，症状是某几类控制线集体错位）、地址线序错（opcode 与 T 的拼接顺序颠倒，取指能对、执行全错）。这类故障的共同点是“错得整齐”，把 ROM 内容读回与表比对一次即现形。门阵列路线的高发故障是接线类：某根拍线漏接（对应拍所有动作消失）、154 与 138 的低有效输出被当成高有效用（所有动作集体反相）、某级与门输入错线（单根控制线行为异常）。排法是顺一根控制线的译码式逐级量，式子在手，最多三四级就到底。

收尾仍是检查清单，按序做：

检查点	方法	预期结果
复位状态	按复位后量 T 计数器与 IR	T=0, IR=0x00, 控制线全无效
取指两拍	单步任意指令的 T0、T1	T0 为 C0+MI, T1 为 R0+II+CE
opcode 专线	单步到 T2, 量 ROM 地址高 4 位	等于刚取回字节的 opcode
回绕干净	连续单步 16 拍看 Q2-Q0	T0 到 T7 循环两遍，无跳态
地址拼接	装入 ADD 后单步到 T4, 量 ROM 地址脚	地址 0x14, 控制字 0x0242
互斥边界	逐拍列出总线驱动器	任何一拍只有一个驱动器

条件跳转	CF 为 0、1 各执行一次 JC	前者顺序执行，后者跳 到目标
空闲拍	单步到 T5-T7，看总线 LED	总线读 0x00，无驱动器
停机与退出	执行 HLT 后量时钟，再按复位	时钟先冻结，复位后从 T0 重跑
两表一致	微操作表逐格译成控制字	与 ROM 表逐行相同

至此控制单元也立起来了：一张表、两块状态、十六根线，机器从此会自己按拍发号施令。下一节把前两节的部件和这一节的时序合在一起，用两个完整程序做一次从头到尾的演练。

五、编程、调试与扩展

前四节把机器搭了起来：总线、时钟、寄存器、ALU、RAM 和控制单元各自通过了各自的检查。本节做最后一件事——让这台机器真的跑程序。先给两个完整程序的逐字节编码与逐拍执行记录，再讲三类最典型的搭机故障，然后讨论这台小机器能往哪些方向扩，最后给一张整机上电后的检查清单。两个程序都不长，但它们是整台机器的总检查：编码表覆盖数据通路与指令格式，逐拍记录覆盖微码与时序，输出序列覆盖所有模块的协同。读本节时手边放一张纸：本节的编码表在左，§ 四的微码表在右，每读一条指令，先把它的 8 拍在纸上走一遍，再与正文对照。

程序一：累加循环的完整机器码与逐拍 trace 程序一的任务是从 1 加到 4，结果为 10。一个累加循环需要三样东西：累加和、计数器、结束判断。先做一个容易忽视的决定：累加和 sum 与计数器 i 都必须住在 RAM 里，而不能住在 A 里。原因是循环中途要用 A 做比较——把 i 减去上限——A 的旧值会被覆盖，凡是要跨指令保存的量都必须写回 RAM。这是单累加器机器的第一课：A 是工作台，RAM 才是仓库。第二个决定是被 16 字节的预算逼出来的。若把初始化也写进程序（两条 LDI 加两条 STA 共 4 字节），再加常数 1、上限 4、i、sum 四个数据字节，总共 20 字节，装不下。解法是把初值交给编程模式：§ 三讲过，拨码可以把每个字节预先写好，i 拨成 1、sum 拨成 0，程序从地址 0 起只保留循环体本身。这也解释了为什么程序一里一条 LDI 都用不上：LDI 是为运行中赋初值准备的，而这里的初值在上电前就已经在 RAM 里了。程序与数据同处一块 RAM，改一个数据字节就改变了程序行为——本章习题会让读者亲手验证这一点。布局上沿用本章约定：程序从地址 0 向上长，数据从地址 15 向下长，中间是空闲带；程序一旦超长就会撞上数据，这正是真实系统里代码段与数据段的最小模型。循环结构采用“先加、再显示、后判断”：先把 i 加进 sum，用 OUT 显示部分和；再把 i 与上限 4 比较。比较不需要专门的指令：SUB 13 计算 i 减 4，结果为零时 ALU 把 ZF 置 1，紧跟的 JZ 11 读到这个标志就跳到 HLT。判断没通过时才真正计数：i 加 1、写回、JMP 0 回循环头。跳转目标 11 是数着编码表数出来的：数据区从 12 开始，HLT 放在 11。完整编码如下，高 4 位 opcode、低 4 位地址或立即数，与章首的 ISA 表一一对应。

地址	内容	助记符	注释
0	0x1F	LDA 15	$A \leftarrow \text{sum}$ ；循环从这里开始
1	0x2E	ADD 14	$A \leftarrow \text{sum} + i$
2	0x4F	STA 15	$\text{sum} \leftarrow \text{sum} + i$ ，写回 RAM
3	0xE0	OUT	显示当前部分和
4	0x1E	LDA 14	$A \leftarrow i$
5	0x3D	SUB 13	$A \leftarrow i - 4$ ；i 到顶时结果为 0，ZF=1
6	0x8B	JZ 11	ZF=1 即跳到 11 号停机
7	0x1E	LDA 14	没跳走： $A \leftarrow i$ （刚才的差已无用了）
8	0x2C	ADD 12	$A \leftarrow i + 1$
9	0x4E	STA 14	$i \leftarrow i + 1$
10	0x60	JMP 0	回循环头

11	0xF0	HLT	停机，时钟冻结
12	0x01	常数 1	计数增量
13	0x04	常数 4	累加上限
14	0x01	变量 i	编程模式预置 1，运行中被改写
15	0x00	变量 sum	编程模式预置 0，运行中被改写

以地址 6 为例读一遍编码：JZ 的 opcode 是 0x8，目标 11 即 0xB，合成 0x8B，拨码时这个字节是二进制 10001011。把 16 个字节拨进机器的顺序是：上电、切到编程模式、从地址 0 到 15 逐字节操作——先拨地址开关，再拨数据开关，按一次写按钮；全部拨完切回运行模式，按复位，再启动时钟。拨码时最容易犯的错是地址或数据拨串了一位，所以拨完把 16 个字节通读一遍再运行——读一遍只要一分钟，能省下半天的追查。四圈循环的关键数据演化如下。” $A \leftarrow i - 4$ ” 一行是 SUB 13 的结果，ZF 由它决定；注意 ZF=1 全程只在第四圈出现一次。

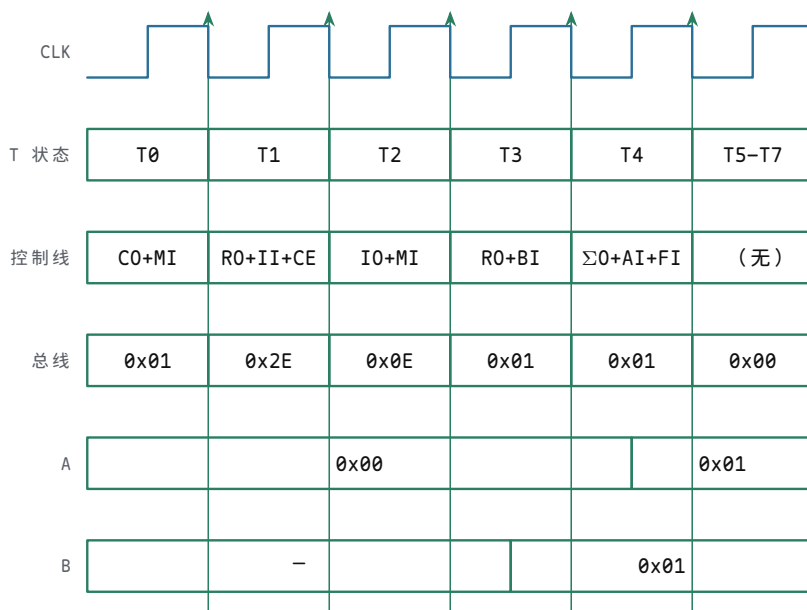
圈数	i (入圈)	sum (加后)	OUT 显示	$A \leftarrow i - 4$	ZF	去向
1	1	1	1	0xFD (-3)	0	继续，i 变 2
2	2	3	3	0xFE (-2)	0	继续，i 变 3
3	3	6	6	0xFF (-1)	0	继续，i 变 4
4	4	10	10	0x00	1	JZ 11，停机

OUT 依次显示 1、3、6、10——部分和按升序出现，最后一个显示值就是答案 10。把 OUT 放在循环体内而不是循环外，是为了让每一次累加都可见：只看最终结果，就无法区分“算对了”与“胡乱跳到一个恰好等于 10 的值”。第四圈还有一个细节值得记下：SUB 13 算出 0x00 的同时 CF=1，因为 4 加 4 的取反再加 1 等于 256，向第 9 位进了位。JZ 只看 ZF 不看 CF，所以这个进位无害。顺带一提，本程序里 i 到 4 时 CF 与 ZF 同时置 1，把判断换成 JC 恰好与 JZ 在同一点跳出，行为看不出差别；两条指令的语义差别要在 i 可能越过上限、CF 与 ZF 不再同置时才显现——选条件跳转前，先看清楚标志的定义。再看时间维度。复位后 T 计数器从 T0 起走；本机的 T 计数器没有提前清零逻辑，T0 到 T7 走满一圈才回绕，所以每条指令恰好占 8 拍。指令提前执行完，剩余的拍就是空拍：所有控制线无效、总线高阻。第一条指令 LDA 15 的完整 8 拍记录如下（复位后 PC=0；A 的旧值无关紧要，T3 会被覆盖）。

拍	有效控制线	PC	IR	总线	A	说明
T0	CO+MI	0	旧值	0x00	旧值	PC 把地址 0 送上总线，时钟沿 $MAR \leftarrow 0$ ，取指开始
T1	RO+II+CE	1	0x1F	0x1F	旧值	RAM[0] 出到总线， $IR \leftarrow 0x1F$ ，PC 同时加 1
T2	IO+MI	1	0x1F	0x0F	旧值	IR 低 4 位送总线， $MAR \leftarrow 15$ ：操作数在 RAM[15]
T3	RO+AI	1	0x1F	0x00	0x00	RAM[15] (sum 的初值 0) 出到总线， $A \leftarrow 0$
T4	(无)	1	0x1F	高阻	0x00	LDA 已完成，空拍
T5	(无)	1	0x1F	高阻	0x00	空拍
T6	(无)	1	0x1F	高阻	0x00	空拍
T7	(无)	1	0x1F	高阻	0x00	空拍；下一拍回绕到 T0，开始取下一条

第 9 拍起进入第二条指令 ADD 14：T0 送地址 1，T1 取回 0x2E，T2 把操作数地址 14 送 MAR，T3 是 RO+BI——RAM[14]（此时 i=1）装入 B，T4 是 $\Sigma 0+AI+FI$ ——A

得到 $A+B=1$ ，同时锁存 ZF 与 CF。把这两拍与 § 四的微码表对照，编码、微码、时序三者就闭合了。把 ADD 14 的关键五拍画成波形（第一圈： $A=0x00$ ， $i=RAM[14]=0x01$ ），装载沿与总线值的对齐关系一眼可见。



装载沿 $B \leftarrow 0x01$

装载沿 $A \leftarrow 0x01$ ，FI 记标志

图示：程序一第二条指令 ADD 14 的 T0-T4（第一圈： $A=0x00$ ， $i=RAM[14]=0x01$ ）。T0、T1 是公共取指：总线依次是 PC 的 0x01 和指令字节 0x2E；T2 总线是 IR 低 4 位 0x0E，拍末 $MAR \leftarrow 14$ ；T3 总线是 $RAM[14]$ 的 0x01，拍末 $B \leftarrow 0x01$ ；T4 $\Sigma 0$ 把 $A+B=0x01$ 放上总线，拍末 AI 收回 A、FI 锁存标志。参考虚线是 T 状态翻转的下降沿：B 在 T3 的装载沿（拍内上升沿）更新，A 在 T4 的装载沿更新，控制字与总线值在装载沿前半拍已经稳定。

读 trace 表还要记住一个事实：空拍里总线高阻，LED 显示的是下拉后的 0x00，那不算任何模块在驱动总线。判断“谁在驱动”，只看有效控制线里的“出”使能。最后算一笔时间账。每条指令固定 8 拍，程序一共执行 41 条指令：前三圈各 11 条，第四圈在 JZ 处跳走、只执行 8 条（LDA、ADD、STA、OUT、LDA、SUB、JZ 之后直达 HLT）。逐条统计如下。

指令	执行次数	拍数小计
LDA	11（前三圈各 3，末圈 2）	88
ADD	7（前三圈各 2，末圈 1）	56
STA	7（前三圈各 2，末圈 1）	56
OUT	4	32
SUB	4	32
JZ	4	32
JMP	3	24
HLT	1	8
合计	41	328

328 拍的含义很具体：1 Hz 的单步节奏下要五分半钟才能跑完，10 Hz 的 555 自动时钟下约 33 秒。想亲眼看清每一拍，就把时钟切到单步按 328 次——这个数听起来大，按起来其实不慢，而且每一次都能在 LED 上核对总线值。

上电之前，先在纸上把逐圈表自己走一遍：从 $i=1$ 、 $sum=0$ 起，每圈写下 A、i、sum、ZF 四个值，直到 JZ 跳走。纸上结果与机器显示一致，说明人与机器理解的是同一个程序；不一致，先怀疑纸上的自己——机器只是忠实执行你拨进去的字节。桌面演练能查出两类错误：拨错的字节，和想错的循环，两类错误在上电前区分不开。程序一还有一个天生的边界：8 位机器的累加和最大 255。把上限从 4 改成 9（从

1 加到 9 得 45) 依然安全, 再往上就要先估算结果是否溢出——能显示的最大答案由数据宽度决定, 这一点程序二同样躲不开。顺带一提, JC 在本章两个程序里都没有出场: 它适合“溢出即停”这类保护性判断, 留给读者在改编程序时试用。

程序二：用循环加法做 3×4 乘法 ISA 里没有乘法指令, 而乘法并没有消失——它变成了“计数加法”: 3×4 就是把 3 自加 4 次。只有加法器的机器上这是唯一办法; 后来有乘法指令的处理器, 在电路或微码里做的本质上也是这件事的加速版。程序二与程序一结构同构: 循环体仍是“加—显示—减—判断”, 只是被加的量换成存在数据区的被乘数 3, 判断对象换成递减到 0 的计数器。数据区安排四个字节: 地址 12 放常数 1 (递减步长), 地址 13 放被乘数 3, 地址 14 是计数器 n、预置 4, 地址 15 是积 p、预置 0。与程序一不同, 这次把判断放在圈尾: 先把 3 加进 p, 再让 n 减 1, 减到 0 就停。为什么放圈尾? 这套 ISA 没有“与 0 比较”的指令, 要测 n 是否为 0, 只能借递减那一下的 ZF——把判断紧跟在 SUB 之后, 减法就身兼“计数”与“测零”两职, 省掉一条专门的比较指令。圈尾判断还有一个关键细节: SUB 12 与 JZ 9 之间隔着一行 STA 14。JZ 读的是标志寄存器, 而 STA 不做任何 ALU 操作、FI 无效, ZF 仍是 SUB 算出的结果——标志不会被“路过”的指令破坏, 这一点必须在 § 2 装载标志的电路里就已经保证。完整编码如下。

地址	内容	助记符	注释
0	0x1F	LDA 15	$A \leftarrow p$; 循环从这里开始
1	0x2D	ADD 13	$A \leftarrow p + 3$
2	0x4F	STA 15	$p \leftarrow p + 3$
3	0xE0	OUT	显示部分积
4	0x1E	LDA 14	$A \leftarrow n$
5	0x3C	SUB 12	$A \leftarrow n - 1$; 结果为 0 时 ZF=1
6	0x4E	STA 14	$n \leftarrow n - 1$; 不影响标志
7	0x89	JZ 9	n 减到 0 就跳到 9 号停机
8	0x60	JMP 0	否则回循环头
9	0xF0	HLT	停机
10	0x00	NOP	填充字节, 正常流程到不了
11	0x00	NOP	填充字节, 正常流程到不了
12	0x01	常数 1	递减步长
13	0x03	被乘数 3	要算几乘几, 改这里与计数器
14	0x04	计数器 n	编程模式预置 4, 运行中被改写
15	0x00	积 p	编程模式预置 0, 运行中被改写

程序对不对, 用循环不变量一句话就能说清: 每次到达地址 0 时, 恒有 $p + 3n = 12$ 。进入循环时 $p=0$ 、 $n=4$, $0+3 \times 4=12$, 不变量成立。每转一圈, p 增加 3、n 减少 1, $p + 3n$ 的值不变, 不变量保持。退出循环时 $n=0$, 于是 $p=12$, 正是 3×4 。不变量把“算多少次”与“每次加多少”锁在一起, 是计数加法类程序通用的论证方法。四圈的数据演化如下。

圈数	n (入圈)	p (加后)	OUT 显示	n - 1	ZF	去向
1	4	3	3	3	0	继续
2	3	6	6	2	0	继续
3	2	9	9	1	0	继续
4	1	12	12	0	1	JZ 9, 停机

OUT 依次显示 3、6、9、12, 共转 4 圈——放在循环里的 OUT 等于把乘法表的一行打了出来; 若只要答案, 把 OUT 挪到 HLT 前即可, 但那就少了一次观察计数过程的机会。拍数同样好算: 前三圈各 9 条指令, 末圈在 JZ 处跳走、连 HLT 共 9 条, 合计 36 条, 乘每条 8 拍得 288 拍。把“乘法 = 计数加法”再推进一步: 除

法就是计数减法—反复减去除数、数减了几次；乘方是嵌套的计数加法。这台 16 字节的小机器装不下那些程序，但道理完全一样：没有某条指令，不等于没有那种运算，只是要多花几条指令、几拍时间。想算 5×6 也不必改程序：把地址 13 拨成 5、地址 14 拨成 6，机器就算另一道题—乘法与程序无关，只与数据有关。

程序二还有两个细节值得多说一句。一是被乘数为什么放在数据区，而不是用 LDI 内嵌进程序：内嵌也能算，但那样每换一道题就要改程序字节；被乘数放数据区，同一份程序对任何乘法题都成立。二是 10、11 两个字节为什么填 NOP：编程模式下每个字节都要拨出确定值，NOP (0x00) 是最安全的填充。正常流程不会到达它们；万一调试时乱按时钟把 PC 送过了 9 号，NOP 至少不会随手把 RAM 写花——当然，PC 继续滑进 12 号之后执行的就是“数据当指令”，所以填充只是对齐手段，不是保险。三是乘积的位宽：p 同样只有 8 位， $3 \times 4 = 12$ 很安全， $16 \times 16 = 256$ 就溢出了。挑乘法题要按结果位宽挑，超过 255 的乘法需要两个字节存积，那是下一台机器的程序。

故障排查案例三则 程序跑不起来时，故障几乎总在三类地方：总线、控制线、复位。下面三则案例按“症状、定位、根因、修复”各讲一遍；它们也是本节末尾整机检查清单里最容易栽跟头的三项。

案例一：总线争用。 **症状** 某一拍总线 LED 显示一个谁也不认识的值——既非 RAM 内容也非任何寄存器内容，个别 LED 亮度发暗；整机电流明显偏大，摸相关芯片微微发热。**定位** 把时钟停在出事的拍，列出该拍微码里全部“出”使能 (C0、R0、A0、Σ0、I0)，用逻辑笔逐根点：任何一拍只能有一根为高；发现两根同时为高，就找到了争用的双方。**根因** 使能互斥被破坏：微码 ROM 某一行多烧了一位“出”使能，或者两根使能线在面包板上插错了孔，或者某片三态输出的使能端极性接反、输出常开。**修复** 改微码或改接线；此后把“任何时刻总线只有一个驱动器”当作硬约定——每改动一根使能线，就把 11 条指令的全部微码行重新核一遍，上电先单步空走 8 拍，看总线有无意驱动。

案例二：RI、RO 接反。 **症状** 读变写、写变读：LDA 之后 A 拿到的是总线上的随机值；更糟的是 RAM 被意外改写——执行完 STA 15，15 号单元没变、别的单元却变了，程序越跑越乱，因为代码自己在被啃。**定位** 拨入一条 LDA 15 加 HLT，单步到 T3，用两根逻辑笔同时点 RI 与 RO：这一拍应当 RO=1、RI=0，若正好相反就实锤了。另有一个旁证：跑完任何程序后，把几个没写过的 RAM 单元倒出来看，若内容与拨入时不同，说明写使能不在该有效的时候有效过。**根因** 微码 ROM 到 RAM 的两根控制线在排线中相邻，最容易整对调；也可能是烧 ROM 时两行控制字填串了。**修复** 断电，对调两根线（或重烧 ROM），再做一次全指令抽查：每条指令跑完，核对相关 RAM 单元未被意外改写。顺手把 RI 的时序也核一遍：RI 必须在 MAR 地址稳定之后才有效，且只在需要写的那一拍有效。

案例三：复位未清 T 状态。 **症状** 按复位后 PC 确实回到 0，但第一条指令执行得莫名其妙——像是从半中间开始，少了一个取指拍，或多出一个不该有的写 RAM 动作；每次上电犯的错还不一样。**定位** 复位后连续单步，盯住有效控制线：第一拍必须是 C0+MI。若看到的第一拍是 I0+MI 或别的组合，说明 T 计数器没有从 T0 起走。**根因** 复位线只接了 PC 的清零端，没接 T 计数器 (74HC161) 的清零端；T 停在复位前的那一拍——比如 T3——取指序列整个错位：该取指时在执行，该执行时在取指。**修复** 复位必须覆盖所有时序状态：PC、T 计数器、IR、标志寄存器同时清零；A、B、OUT 这些数据通路寄存器可以保留，反而便于上电观察。改完后的检查：复位后单步 8 拍，控制线序列必须逐拍与微码表一致，且每次上电结果都一样。

三类故障的速查对照如下，排查时先对症状、再用首选手段确认，不要盲目换芯片。

故障类型	典型症状	首选定位手段
总线争用	某拍总线显示谁也不认识的中间值，电流偏大	停在该拍，逐根点全部“出”使能

控制线接反	读变写、写变读，RAM 被意外改写	单步到相关拍，同时点 RI 与 R0
复位不全	复位后第一拍不是 CO+MI，每次上电错得不一样	复位后连续单步，核对控制线序列

三则案例对应三条搭机纪律：使能互斥逐拍核、相邻控制线双人核对、复位覆盖全部时序状态。它们也有共同点：定位手段都是“单步加逻辑笔”，没有一件工具超出实验盒的范围。机器小到没有一处不可观察，是它作为教具最大的本事。

三则案例之外还有一条排查总原则：先怀疑最后动过的地方，一次只改一处，改完立刻重跑最小复现程序。同时改两处，等于亲手毁掉对照组。这几条听起来像软件调试，因为调试的本质不分软硬件：控制变量、最小复现、对照实验。另有一类不在三则之列的隐性问题值得点名——接触不良：面包板孔内的簧片会疲劳，症状是“轻碰板子，程序行为就变”，单步和逻辑笔都抓不住它。预防胜于定位：走线横平竖直、导线插到底、不用过长的飞线，能躲开一大半这类问题。

扩展方向：更大、更快、更多指令 这台机器故意做得很小，但每个方向都留着扩的口子。下表按“要动哪些模块”把四条最常见的扩展路径排出来，随后逐条细说。

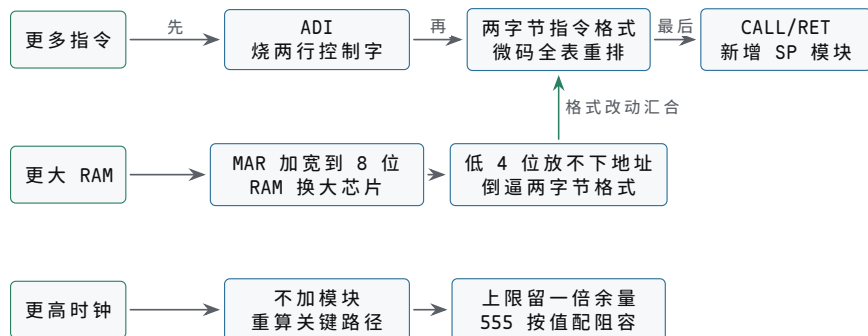
扩展方向	要动的模块	主要代价
RAM 扩到 256 字节	MAR 加宽到 8 位、RAM 芯片、编程模式拨码	低 4 位放不下地址，须改两字节指令格式，微码全部重排
加立即数指令 (ADI)	只动微码 ROM：T2 发 IO+BI，T3 发 $\Sigma 0 + AI + FI$	最便宜的扩展：不动数据通路，只烧两行控制字
加 CALL/RET 与栈	新增 SP 计数器模块、RAM 划出栈区、微码序列拉长	最贵的扩展：PC 要新增经总线进出 RAM 的路径
提高时钟频率	不加模块，重算关键路径：总线 $\rightarrow$ ALU $\rightarrow$ A 建立时间	面包板引线电容压低实际上限，频率须留一倍余量
微码 ROM 换成 RAM	微码存储改静态 RAM，另加上电加载电路	得到可写控制存储；加载电路本身成为新的故障源

扩 RAM 是最诱人的方向，也是连锁反应最多的：地址从 4 位变 8 位，MAR 要加一片寄存器，RAM 要换大芯片，而最大的改动在指令格式——一条指令一个字节，低 4 位最多指到 15。解决办法是学真实机器：指令占两个字节，第一字节 opcode、第二字节地址，取指之后多插一拍把地址字节读进来；于是每条指令的微码序列都要重排，本章所有编码表作废。这也解释了为什么本章把地址空间钉死在 16 字节：小，才能让每张表完整印在一页里。加指令则展示了微码结构的甜头。以立即数加法 ADI ($A \leftarrow A + \text{立即数}$) 为例：数据通路什么也不用加——IR 低 4 位本来就能上总线 (IO)，B 本来就能装载 (BI)，T2 发 IO+BI、T3 发 $\Sigma 0 + AI + FI$ ，两行控制字烧进 ROM 就完了。硬连线控制里加一条指令要重画译码逻辑，微码控制里只是往表里多写两行——“微码便宜、硬线贵”，这是值得亲手烧一次 ROM 的理由。CALL 与 RET 是另一个极端：它们需要栈，而栈需要新硬件。栈指针 SP 是一个能加一减一的计数器模块；CALL 要把 PC 的当前值经总线写进 RAM[SP]、SP 减一、再把目标地址装进 PC，RET 反过来把 RAM[SP] 读回 PC。每条都要 6 到 8 个执行拍，PC 还得新增“把值送上总线”之外的路径——在 16 字节的机器上做栈得不偿失，但它标出了下一台机器的方向。提高时钟不需要加任何模块，只需要算账。本机最长一拍是 ADD 的 T4：操作数在 T3 已装进 B，T4 这一拍要走过 74HC283 的行波进位 (8 位逐级传递)、总线传播和 A 寄存器的建立时间，全部加起来取倒数，就是时钟频率的理论上限。面包板引线长、分布电容大，实际上限比器件手册算出来的低得多，所以预算规则是：算出上限再留一倍余量，555 的阻容按这个值配。最后一个方向是把微码 ROM 换成 RAM，得到可写控制存储：上电时由加载电路把微码灌进静态 RAM，之后运行时也能

改指令定义——今天 OUT 是显示，明天可以改成“显示并响蜂鸣器”。代价是加载电路本身成为新的故障源，而且微码错了，机器会以最难调试的方式出错：每条指令都不对。四条路径没有先后，但有一条共同纪律：每扩一处，先回章首把整机规格表被改动的行改掉，再重做本节末尾的整机检查。

有读者会问中断和进位链：它们其实已经藏在表里。进位链藏在“提高时钟”那一行——74HC283 的行波进位正是关键路径的主角，换超前进位器件能直接抬高频率上限。中断则藏在 CALL/RET 那一行：中断的本质是“硬件替你执行一次 CALL”，同样要保存 PC 现场，同样需要栈；没有栈的机器谈中断，就像没有 RAM 的机器谈子程序。所以扩展的顺序不是随机的：先有栈，才谈得上子程序；再有富余的地址空间，才谈得上中断向量。

把四条路径按目标画成决策图，每条路径内部的先后就写在链上：



图示：扩展路径决策（对应正文扩展方向表）。三个目标各有走法：加指令按成本递增排——ADI 只烧两行控制字，最便宜；两字节指令格式要重排全表微码；CALL/RET 与栈要新增 SP 计数器模块，最贵。扩 RAM 连锁最多：MAR 加宽、RAM 换大之外，低 4 位放不下地址，同样倒逼两字节格式。提高时钟不加模块，只重算“总线到 ALU 到 A 建立时间”这条关键路径，理论上限再留一倍余量。所以扩展的顺序不是随机的：先有栈才谈得上子程序，再有富余的地址空间才谈得上中断向量。

整机上电检查清单 整机第一次上电，不要直接跑程序。按“静态、单模块、单指令、程序”四级往上走，每一级都把故障隔离在尽可能小的范围：静态不过谈不上模块，模块不过谈不上指令，指令不过谈不上程序。四级顺序本质上也是二分法：每通过一级，故障可能被锁定的范围就缩小一半以上。

层级	检查内容	预期现象
静态（不上时钟）	目检焊点与走线；量电源与地之间的电压；复位有效时看 PC、T、IR、标志是否全为已知值；全部控制线应为无效，总线应为高阻	总线 LED 全灭（空闲读 0x00）；整机电流在几十毫安量级；无芯片发热
单模块	单步下逐一激励各模块：PC 计数与装载、A/B/IR 装载、RAM 单字节读写、ALU 加减与 ZF/CF、OUT 显示	各模块孤立动作与本章 § 二、§ 三的分模块检查结果一致
单指令	拨入 LDA 15 与 HLT，单步走完两圈 8 拍；再抽验 ADD、STA、JZ、OUT 各一条	每拍有效控制线与 § 四微码表逐拍一致；任何一拍总线只有一个驱动者
程序	拨入程序一、程序二的完整 16 字节，自动时钟跑到底	OUT 依次显示 1、3、6、10（程序一）与 3、6、9、12（程序二）；HLT 后时钟冻结、状态稳定可读

任何一级不过，就停在那一级修好再往上走——跨级调试是最常见的浪费时间方式。这张清单值得打印出来贴在面包板旁边，每次上电按顺序走一遍。它还有一条使用规则：每次改动接线或重烧微码之后，无论改动多小，都从静态级重新走起——改动之后直奔程序级，是最常见的跳级错误。两个程序的输出序列 1、3、6、10 与

3、6、9、12 是这台整机的最终签名：它们对，说明从 555 的 RC 到微码 ROM 的每一位都对。到这里，从第一章的第一个开关到本章的一台可编程计算机，整条链路闭合。

专题：整机联调日志——从一次真实 bring-up 看排错顺序 下面这份日志来自一次真实的整机 bring-up：各模块在分测阶段都单独过了一遍，第一次连成整机跑程序，从上午十点到傍晚六点，一共踩了六个坑。日志按时间顺序原样保留，因为顺序本身就是方法：每一步只记四件事——现象、第一怀疑、测量点、结论与改动。读它的重点不在六个坑本身，你的板子会踩出属于它自己的坑；重点在每一步的节奏：先量哪里、一次改几处、改完退回到哪一级。

步骤	现象	第一怀疑	测量点	结论与改动
■ 上电静态检查	尚未通电：蜂鸣档量电源轨，5 V 与 GND 直通	有跳线插错孔，或某片芯片插反	逐段拔跳线缩小范围；目检全部芯片缺口方向	一根跳线把电源轨与地轨短在相邻孔；另有一片 74HC273 缺口朝反。挪线、掉头；上电后整机电流几十毫安，无芯片发热，逐片去耦电容在位
■ 时钟不起振	555 输出脚 LED 常亮不闪，预期 1~50Hz 连续方波	555 损坏，或芯片插反	LED 逐点看：输出脚半亮不灭，阈值脚也半亮——半亮说明在快速开关，只是肉眼跟不上	定时电容错一档：10 μ F 插成 0.01 μ F，频率高了三个数量级，“不起振”其实是“快得看不见”。换回 10 μ F，电位器拧到 1Hz 起振
■ 复位后 PC 不归零	按复位按钮，PC 不归零，反而开始计数；一松手，PC、T、IR 全被清掉——相位整个反了	PC 那片计数器的清零脚虚接	复位链末级输出脚的常态电平：低有效清零端平时应为高，实测平时为低	按钮消抖级的上下拉接反，复位链整体反相：平时等于按住复位，按下等于松手。改正后按一下全机归零，松手从 T0 起跑
■ 取指取回 0xFF	复位后单步：T0 总线 LED 全亮（应显示 PC 的 0x0），T1 取回 0xFF，机器随即停在 HLT	程序字节拨错，或微码 ROM 取指行烧错	逐拍看总线：T0 拍就不是一个驱动者该有的读数；再量 74HC240 的使能脚——悬空，被器件读成有效	R0 反相器到 240 使能脚一根线漏插，RAM 全程驱动总线，T0 与 PC 争用，两边输出级对抗把总线顶成全亮。补线后 T0 显示 0x0，T1 取回程序第一字节

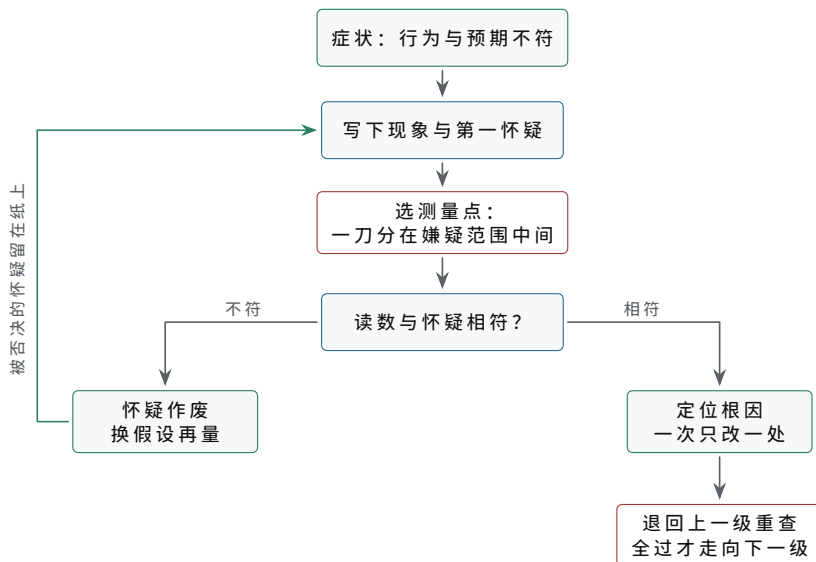
▣ ADD 结果错 1	拨入 § 三的 5 字节程序：LDI 6、ADD 0x4 (存 7)，应显示 0x0D，实测 0x0E，恰好大 1；SUB 却正确	微码 ROM 里 ADD 的 T4 行烧错一位	加法器输出那排常驻观察 LED：A=6、B=7、SU=0 时已显示 0x0E，错在 ALU 内部，与微码无关；再量最低位 283 的 CI 脚：悬空	SU 的第九个分叉漏接：8 个异或门都接了，进位输入 CI 没接，悬空被读成 1，加法变 A+B+1；减法 CI 本来就要 1，故碰巧正确。补线后显示 0x0D
▣ 程序跑通	拨入程序一全部 16 字节，自动时钟跑到底：OUT 依次显示 1、3、6、10，HLT 后时钟冻结、读数稳定	无须怀疑，只须确认	先单步走完第一条指令的 8 拍，逐拍对微码表；再拨自动档	签名序列 1、3、6、10 全部对上。这一步不改任何线，只做回归：从静态级把四级检查重走一遍，全部通过

排错动作可以写成一个循环，贴在面包板旁边比任何口诀都顶用：

while 行为与预期不符：

1. 停在当前层级，写下现象与第一怀疑
2. 选一个测量点，先量再改
3. 一次只改一处，改完退回上一级重查
4. 本级全部通过，才允许走向下一级

把这个循环画成决策树，“怀疑提出假设、测量负责否决”的走法就固定下来：



图示：故障排查决策树（对应联调日志六步的走法）。症状写下后提出第一怀疑，怀疑的职责是指定第一个测量点——选在与各嫌疑都无关、一刀分在范围中间的位置量；读数与怀疑不符，怀疑就地作废、换假设再量；相符才动手，一次只改一处；改完退回上一级重查，本级全过才允许走向下一级。日志里六个坑平均四十分一个，靠的就是这条看起来慢的纪律。

第一条纪律是一次只改一处，理由是纯因果的：一次改两处，机器好了，你不知道是哪处起的作用；更常见的是机器没好，而你从此分不清“原本的故障”和“新引入的故障”。第 ▣ 步最能说明问题：取指取回 0xFF 的时候，手边至少有三个诱人的改动——重拨程序、重烧微码、补 240 那根线。若是把前两个“顺手”做掉再补线，机器也能跑通，但这份日志就此失去结论，下次遇到同样的症状还得从头再来。忍住不改、先量使能脚，才把故障钉在那根漏插的线上。所以每次动手之前先问一句：这

次改动针对哪一条测量？答不上来，就说明还没到改的时候，先回去量。

第二条纪律是改完退回上一级。本章的四级检查顺序——静态、单模块、单指令、程序——把整机切成四层，任何一处改动之后，从被改动的那一层重走，而不是从刚才停下的地方继续。第 Ⅱ 步换完电容，不是直接再跑程序，而是退回单模块级，把时钟的两路输出重新核一遍；第 Ⅲ 步补完线，退回单指令级重核 T0、T1 两拍，而不是直接拨自动档。退回的代价是几分钟，跳级的代价是故障在上一层复活时，你得再花一整轮从头定位。六个坑平均四十分钟一个，靠的就是这条看起来慢的纪律；在面包板上，一天之内从通电到跑通，已经算是一场顺利的 bring-up。

第三条要说的是“第一怀疑”的用法。六个坑里，第一怀疑只命中了一次，这不是记录者笨拙，而是硬件故障的常态：第一怀疑来自模式匹配，而模式匹配在跨模块的故障面前经常指错方向。它的真正用途不是命中，而是决定第一个测量点——怀疑 555 坏了，于是先量输出脚；量出来是半亮，与怀疑不符，怀疑就地作废，测量领着你走向电容。怀疑可以被推翻，测量不会说谎；日志里的每一行，都是“怀疑提出假设、测量负责否决”的一次循环。把怀疑写下来还有一层好处：被否决的怀疑留在纸上，两个小时后你就不会把它当成新想法再试一遍。

四条记录之外，表里还有一条暗线值得挑明：测量点的选法。好的测量点要一刀剁在故障可能范围的中间——怀疑程序拨错还是微码烧错，就量一个与两者都无关的点：240 的使能脚。它正常，ROM 与程序的嫌疑同时洗清；它不正常，两边都不用再拆。第 Ⅳ 步的测量点尤其典型：加法器输出那排常驻 LED 把“微码错”与“ALU 错”一刀分开，一次读数否决一半可能。选测量点的本领，说到底二分法的本领：每一次测量都把剩下的可能砍掉一半，六次二分足以从六十四个嫌疑里揪出一个——而一块面包板上的嫌疑，通常凑不齐这个数。

每一条“结论与改动”栏的末尾还有一个新读数：1Hz 起振、T0 显示 0x0、OUT 显示 0x0D。这不是行文习惯，是纪律的最后一环：改动之后必须有一个能证明改动生效的测量，否则这次改动不算完成。修机器最常见的自欺，是改完线看一眼“好像好了”就往下走，三天后故障换一副面孔回来，没人记得它上次是怎么“好”的。新读数把每次修复钉死在一个可复查的状态上：它既是对本次改动的确认，也是下一次退回上一级时的起跑线。

最后一层是记录本身。bring-up 到第三个小时，人会开始原地转圈：这根线量过没有？这组读数是改动前还是改动后的？纸面上的日志把记忆问题变成查表问题，也把一下午的六个坑变成下次搭建的六条经验——第二次见到“总线全亮”，你会直接去量 240 的使能脚，而不是再怀疑一遍程序。日志的形式不拘：纸本、纯文本、手机照片都行，要紧的只有三点——四栏齐全、按时间排列、每次改动都配一个新读数。满足这三点，铅笔头和键盘没有区别。这台机器造好之后，真正陪你走向下一台机器的，不是面包板，是这本日志。

专题：面包板常见的十类硬件故障 把多次搭建中出过的故障归拢起来——自己的、同学的、论坛上看来的一会得到一个有点扫兴的结论：绝大多数故障与电路原理无关，与手有关。原理图上的错误在搭建之前就被思考过滤了大半，而手的错误没有任何东西过滤：插错的孔、拿错的电阻、漏接的线、看反的缺口，它们按概率均匀分布在每一次插拔里。所以排故障的人不该先问“哪个原理我没懂”，而该先问“哪类手的错误我还没排除”。

下表按发生频率排出十类，每类给三栏：典型症状、最快定位法、一行实测经验。症状与故障不是一一对应的——“时好时坏”可能是接触不良，也可能是悬空脚——所以表里最值钱的是定位法那一栏：它回答“怎样用一次测量把这一类排除掉”。熟悉这张表的意义不在背诵，而在把第二次见到的故障压缩到五分钟内解决：第一次见是故障，第二次见应该只是核对。

故障类别	典型症状	最快定位法	实测经验
接触不良	时好时坏，晃动板子或按压芯片时现象跟着变	分区轻晃、镊子逐个压芯片，找到敏感区再逐孔重插	面包板插孔有寿命，“软故障”查到最后，八成是它

芯片插反	上电即发热，该芯片无输出	立即断电，摸温度找发烫的片，目检缺口朝向	全板缺口统一朝向的纪律，能防掉其中大半
电源轨半板未接	一半模块正常，另一半整体不工作	不量电源入口，逐片量芯片的 VCC 脚	长面包板的电源轨在中段断开，两半要分别跨接
未用输入悬空	输出随机翻转，手靠近时读数会变	对照手册把每片没用到的输入脚数一遍	悬空脚是天线，全部按功能接高或接低，一个不留
三态使能互斥遗漏	总线停在中间电平，LED 全亮，整机电流变大	逐拍核对有效控制线，“出”使能必须只有一个	争用几分钟后输出级会微热，趁热摸能找到当事片
消抖不足	按一次按钮，机器走好几拍	看 T 计数器在一次按键里的跳变次数	触点抖动几毫秒，RC 取 10ms 才盖得住
控制线交叉接反	功能成对错位：该读时写、该装 A 装了 B	把可疑的两根对调验证；平时逐根对照控制线清单	同色跳线并排最容易拿错，按功能分色省一半误会
下拉电阻遗漏	空闲拍总线读随机值，而不是 0x00	让所有驱动者无效，读总线空闲值	100k Ω 一排 8 只，少一只，那一位就漂
LED 限流电阻错档	显示过暗、刺眼，或干脆烧灯	量单路电流，与 330 Ω 档的计算值对照	33 Ω 与 330 Ω 只差一环色环，亮度差十倍
去耦电容忘记接	低速正常，提高时钟后偶发出错	故障只在提速后出现时，先数电容再查别的	0.1 μ F 贴着电源脚放，电流突变全靠它就地吸收

这张表的顺序值得多说一句：按频率排，是为了让排查从概率最高的地方开始。遇到“机器不工作”，先做的三件事永远在表的前三行——捏一捏芯片、摸一摸温度、量一量各片的 VCC 脚——全程不过两分钟，却能拦下一大半的故障。排在后面的几类各有鲜明的指纹：全亮是争用、多拍是消抖、成对错位是线序、空闲随机是下拉、过暗烧灯是限流、提速才错是去耦，指纹对上了就直接用对应的定位法，不必从头扫表。真正要警惕的是那些指纹不鲜明的症状，它们多半属于第一行。

最后要说的是：这张表是攒出来的，不是背出来的。每解决一个故障，就把它登记成一行——类别、症状、怎么找到的——攒过二十行，你会得到一张属于自己的表，顺序和这张未必相同：手的习惯不同，故障的分布就不同，常用长跳线的人，“接触不良”那一行会特别长。别人的表给的是起步的顺序，自己的表给的才是定位的直觉。等到发现自己的表连续半年没有新增一行，那不是故障消失了，是这张表毕业了——可以换 PCB 的表开始攒了。

这十类故障在 PCB 的世界里换了一副面孔，但一类不少地都在：接触不良变成虚焊与冷焊点，芯片插反变成丝印方向看错，半板未接变成电源平面分割不当；悬空输入还是悬空输入，三态争用还是总线争用，消抖照样要消抖，去耦只会比面包板更讲究。差别只在定位工具：面包板上用镊子和手指，PCB 上用放大镜和示波器；而“先电源、后时钟、再信号，先静态、后动态”的顺序不换。把这张表在面包板上练熟，等于提前攒下了 PCB 时代的定位直觉——面包板因此是最便宜的故障训练场，这里每一类故障的学费，不过一根跳线。

专题：把教学机的规格写成一页纸 本章开头有一张整机规格表，扩展主题里又反复要求“改动之前先回章首改规格表”。现在把这件事做到底：把整台机器的全部约定压缩进一页纸，一行一条，每条都能用一次测量核对。规格不是散文，是可核对的清单——“时钟要快”不是规格，“0.7-48Hz”才是；“内存够用”不是规格，“16 字节”

才是。下面这张表就是这台机器的一页规格书：它的每一行在本章正文里都有整整一段展开，反过来说，第七章的全部内容都可以读作这十三行的注脚。

项目	一句话规格
总线	8-bit 三态共享，任何时刻只有一个驱动器；各线 100kΩ 下拉，空闲拍读 0x00
地址空间	16 字节，4 位地址；程序与数据同处一块 RAM，不需要译码器
寄存器	A、B、IR、OUT 为 8 位；MAR、PC 为 4 位；Flags 存 CF、ZF 两位
ALU	只做 A+B 与 A-B；减法是 B 取反加一，SU 一根线兼管取反与进位输入
ISA	11 条指令（含 NOP），每条一字节，高 4 位 opcode、低 4 位地址或立即数：NOP、LDA、ADD、SUB、STA、LDI、JMP、JC、JZ、OUT、HLT
T 状态	每条指令固定走满 T0-T7 八拍；T0、T1 公共取指，用剩的拍控制字全 0
控制线	16 根：HLT、MI、RI、RO、II、IO、AI、AO、BI、Σ0、SU、CE、C0、J、FI、OI；“出”使能互斥，“入”使能不限
控制实现	微码 ROM 共 128 行乘 16 位，地址为 opcode 乘 8 加 T；JC、JZ 的 J 经 CF、ZF 选通
复位行为	异步拉入、同步释放；清 PC、T、IR、Flags；RAM 内容不受复位影响
时钟	自动档 0.7-48Hz（555 振荡，C=10μF）与单步档两路，面板开关选择；HLT 有效时钳停时钟；调试区间 1-10Hz
编程模式	PROG 开关切到 DIP 拨码：4 位地址、8 位数据、写按钮，逐字节装入；先停机后切换，切回复位再启动
输出	OUT 寄存器锁存 A，8 只 LED 二进制直读；停机后读数保持稳定
自检签名	程序一输出 1、3、6、10，程序二输出 3、6、9、12；序列对，则整机对

写这页纸的方法只有三条：一行只写一件事；每件事都给出数值或名字；凡给不出数值的行，就是设计还没做完的行。“时钟”那一行若写不出 0.7-48Hz，说明 555 的阻容账还没算；“复位行为”若写不出清哪几个寄存器，说明复位链还没画完。规格先于搭建存在，它的漏洞在写的那一刻就会暴露，而不是等到调试的第三个小时——这是写规格最实际的回报。还有一条隐含的要求：不许写形容词。“快”“够”“稳定”这类词在规格里没有位置，它们各自对应的数值才有位置。

写规格与读规格，是同一种功夫的两个方向。读一片陌生芯片的数据手册，本质上是读别人机器的一页规格书：先找位宽、地址空间、控制极性、时序窗口这几行，找到就能上手，找不到就知道该补什么实验。反过来，能把自己的机器写成一页纸的人，读手册时自然会按同样的栏目对号——栏目是通用的，只是别人的机器比你这台大几个数量级。日后改动这台机器时，顺序也由这页纸定死：先改纸上对应的那一行，再动面包板上的线，最后按四级检查回归。规格是设计者留给后来的自己的约定：它不在乎你隔了多久回来，只要求每一条都还经得起测量。

这种“读规格”的功夫，本章其实已经练过一次。§ 三读 74HC189 数据手册时，满页参数里只取了三行：读路径的延迟、写路径的窗口、各信号的先后约束——因为对本机而言，手册的其余各行都能用“一拍是毫秒级”这一句话豁免。会写规格的人，才知道手册里哪些行是要紧的：手册不过是芯片厂写给你的规格书，它的一页纸版本，就藏在那三行里。写与读在这里是同一件事的两端：写的一方负责把约定压缩成可核对的行，读的一方负责把行还原成可动手的约定。

这页纸的第三种用法是当改动的靶子：任何扩展念头，写在纸面上就是划掉某一行、写上另一行。想把 RAM 扩到 256 字节，“地址空间”那一行从 16 改成 256，紧接着会发现 ISA、T 状态、编程模式三行都跟着失效——连锁反应在纸面上先走一遍，比在线路上走出来便宜得多。规格页越短，这种推演越彻底，这正是一页纸的价值：它让“改一处牵动几处”这件事，在动手之前就能数清楚。

这页纸的第一读者，是三个月后的你自己。到那时，面包板上的每一根线你都不再记得，而这页纸还在：想改机器，先读它找回全局；机器出了新故障，先读它确认哪条约定被违反了。它也是对外的全部说明：别人想复刻这台机器，这页纸告诉他造什么，本章五节告诉他怎么造——规格回答“是什么”，正文回答“怎么做”，两者缺一，复刻都要靠猜。一页纸装不下的细节，本章有的是地方放；但全局的骨架，永远只配占一页。

专题：从这台机器走向自己的设计 § 五前面列出了几条扩展路径，却没有回答先走哪一条。答案不在机器里，在你身上：扩展是为了学东西，不同的学习目标对应不同的顺序。把决策画成树，根节点只有一个问题——这台机器接下来要教你什么？向下分三个杈：指令集、时间轴、软件。三个杈没有高下，只有“这一次想要什么”的区别；选错了也不要紧，每条路都回得来。

学习目标	推荐扩展顺序	预估工作量	这条路教会你什么
学 ISA 设计	先加 ADI（只烧两行微码），再改两字节指令格式（微码全表重排），最后 CALL/RET 与栈（新增 SP 模块）	一个晚上、一个周末、一周以上，逐档倍增	一条指令的成本分布在哪里：改表便宜、改格式贵、加通路最贵
学总线时序	先拧高时钟、重算关键路径，再换更快的存储或加等待状态，最后用示波器核对建立与保持时间	几个晚上，外加一台示波器	频率上限是算出来的，不是试出来的；余量要留给分布参数
学系统编程	先加串口输出（结果不再靠数 LED），再扩 RAM 到 256 字节，最后写一个 monitor 让机器自己装程序	数周，三条路里最重	硬件的尽头是软件：让程序代替拨码开关伺候程序

三条路不互斥，但不要并行。这台机器的全部调试手段——四级检查、一页规格、逐拍单步——都建立在“一次只改一处”的前提上；同时扩两处，故障签名立刻失去意义，你会在自己最熟悉的机器上，体会陌生机器的调试难度。正确的节奏是：选一个目标，走完它的一整条顺序，回归检查、更新规格页，然后才允许想下一个。每走完一轮，机器变大一点，而它的可观察性不许减少——这是扩展的及格线，也是这台机器作为实验平台的本分。

工作量那一栏也值得看一眼：三条路的量级差着不止一档，这不是编排失误，而是三个学科本身的分量。ISA 的第一课便宜，是因为微码把“改指令”变成了“改表”；时序课贵一些，因为它要求添置仪器、学会测量；系统编程课最贵，因为它实际上是两门课——先扩硬件，再写软件。按自己能投入的时间选路，比按兴趣硬扛更能走到终点。

回到根节点那个问题，它有两种常见的错误答法，值得提前识破。一是“先加最酷的”：中断、串口、流水线，哪个名词响亮先上哪个，结果每台机器都半途而废；二是“先加最大的”：RAM、指令、时钟一次全改，第一周就淹没在自己制造的故障里。两种答法的共同毛病，是把扩展当成了收藏而不是学习。正确的答法只有一个句式：“我想弄懂 X，所以先改 Y。”这个句子填不出来，就说明还没选好路——留在原地再玩一个星期这台机器，比贸然开工更有收获。

所以，把第七章这台机器当作什么，决定了它还能给你什么。当作一个完成品，它的使命在上电那一刻就结束了；当作一个最小可改动的实验平台，它才刚开始工作。它够小，小到任何改动都能在一页规格上找到对应行；它够完整，完整到 ISA、

时序、软件三条路都走得通；它够透明，透明到每次改动是对是错，总线 LED 当晚就告诉你。第一台机器是照着本章搭的；第二台机器不必是从划掉那页规格书的某一行、写上你自己的一行开始，那就是你的设计的开头。

章末练习

任务	要求	学习目标
总线互斥检查	写出 T0、T1 两拍全部有效的”出”使能与”入”使能，说明为什么不构成争用；再指出若微码把 R0 与 Σ0 同拍置 1 会发生什么	总线驱动者唯一
时钟两路	说明自动时钟（555）与单步按钮之间切换的时机要求，以及按钮为什么必须消抖——不消抖会发生什么	一拍不多一拍不少
微码读法	微码 ROM 地址为 {opcode, T}：查 ADD (opcode 0x2) 在 T3、T4 两拍的有效控制线，并写出这两个 ROM 单元的地址（二进制）	会按 opcode 拼 T 查表
累加程序改编	只改程序一数据区的一个字节，把”从 1 加到 4”改成”从 1 加到 2”（结果为 3）：指出地址与新值，并写出新的输出序列	程序与数据同 RAM
故障反推	症状：复位后单步，第一拍有效控制线是 I0+MI 而不是 CO+MI，之后执行全错。推断故障点并给出修复	复位覆盖全部时序状态
扩展设计	把 RAM 扩到 32 字节（5 位地址）：列出至少三处必须同步修改的地方，并说明哪一处改动最大	地址位宽牵动全局

参考解答与要点

任务	参考解答	必须掌握的要点
总线互斥检查	T0 的”出”使能只有 CO（PC 驱动总线），”入”使能只有 MI；T1 的”出”使能只有 R0，”入”使能是 II 与 CE——CE 让 PC 内部计数，并不驱动总线。两拍都只有一个驱动者，故无争用。若 R0 与 Σ0 同拍为 1，RAM 与 ALU 同时驱动总线，线上电平由两边输出级对抗决定，LED 显示既非 RAM 值也非 ALU 值的中间态，总线电流猛增，时间一长会损坏输出级	”出”使能必须互斥，”入”使能不限
时钟两路	切换要在时钟低电平期间完成，切换后先单步几拍确认再继续。机械触点抖动持续数毫秒，不消抖则按一次按钮产生多个时钟沿，T 计数器连跳数拍、微操作序列被跳步，而且这种错误无法稳定复现。用 RS 触发器或施密特触发器整形，保证一次按键恰好给出一个完整时钟周期	单步的价值在于拍数确定

微码读法	ADD 的 T3: $R0+BI$, 把 RAM[addr] 经总线装入 B, ROM 地址为 0010011 (高 4 位 opcode 0010, 低 3 位 T 011); T4: $\Sigma 0+AI+FI$, $A \leftarrow A+B$ 并锁存 ZF、CF, ROM 地址为 0010100。两行其余控制线全为 0	微码 ROM 是”指令 × 拍”的查找表
累加程序改编	把地址 13 的上限常数由 0x04 改为 0x02, 其余 15 字节原样。循环在 i=2 时 SUB 13 结果为 0、ZF=1 而退出, OUT 依次显示 1、3, 最终和为 3。数据字节改了, 程序行为就改了	循环边界是数据不是代码
故障反推	T 计数器没接复位: PC 被清到 0, T 却停在复位前的 T2 (I0+MI 正是访存类指令 T2 的组合), 第一条指令从 T2 开始错位执行, 取指序列整个乱掉。修复: 把复位信号同时接到 T 计数器 (74HC161) 的清零端; 改后复位再单步, 控制线序列必须从 C0+MI 起逐拍与微码表一致	复位要清 PC、T、IR、标志
扩展设计	至少三处: MAR 由 4 位加宽到 5 位; RAM 换成 32×8, 编程模式拨码覆盖第 5 位; 指令格式: 低 4 位放不下 5 位地址—要么改两字节指令 (取指后加一拍读地址字节), 要么约定程序只用低 16 字节、高 16 字节纯作数据。改动最大的是: 微码 T 序列与本章全部编码表都要跟着重排	地址位宽是全局约定, 牵一发动全身

工程边界：电源、时钟与信号

本部分导读

逻辑正确不等于机器可靠。电压会跌、时钟会抖、信号会变形，这些问题不出现在真值表里，却决定真机是否工作。本部分把电源分配、时钟分配和信号完整性写成可计算、可测量的工程问题。

电源、时钟与信号完整性

前面七章默认了两件事：电源永远是稳定的电压，时钟永远是干净的方波。真实机器里这两条都要靠工程手段挣出来。电压会因为瞬态电流而跌落，时钟会因为走线长度不一而偏斜，信号边沿会在走线上来回反射。这些问题不写在真值表里，却决定真值表能不能成立。本章把这三类问题从”经验之谈”变成可计算、可测量的对象：每个现象先给物理原因，再给数量级估算，最后给测量和排查方法。

对象	失效时的样子	本章回答的问题
电源分配	瞬态电流让电压跌落，逻辑误翻转	去耦网络怎样按时间尺度分层
时钟分配	各触发器时钟沿不齐，时序预算作废	偏斜和抖动从哪里来、怎么限
信号完整性	边沿振铃、过冲、回沟、串扰	什么时候必须把走线当传输线

核心思想

电源、时钟和信号共享同一个物理视角：它们都是沿导体传播的电磁过程，寄生电阻、电容和电感在低速时可以忽略，在边沿变快、电流变大时必然现身。本章的每条规则都能还原成这三个寄生参数的数量级比较。

一、电源分配：稳压、去耦与电源完整性

本节从”电源”这两个字背后的一整串工程问题开始。数据手册把供电写成一行参数： $VCC = 5V \pm 5\%$ 。真实板子上，这一行要靠三类手段兑现：稳压器把未经稳压的输入变成标称电压，去耦网络对付稳压器来不及响应的快变化，电源与地平面的几何形状决定噪声在板内怎样传播。三者缺一，逻辑完全正确的电路也会偶发失误。

本节按时间尺度从慢到快组织：稳压器管 ms 级以上的平均供电；瞬态电流与电压跌落说明为什么稳压器天然管不了 ns 级事件；去耦电容按时间尺度分层，接管从 ns 到 ms 的全部空档；目标阻抗把这些层次统一成一条随频率变化的阻抗曲线；最后，地平面与回流路径回答”电流回哪里去”这个同样关键的问题。

线性稳压与开关稳压的取舍 两类稳压器的差别在一个物理事实上：多余的那部分电压去了哪里。线性稳压器（78xx 系列、各类 LDO）内部是一只工作在线性区的调整管，输入高出输出的电压直接降在管子上，以热的形式耗散。效率因此几乎只由电压比决定： $\eta \approx V_{out}/V_{in}$ 。5V 降到 3.3V，效率上限就是 $3.3/5 = 66\%$ ，与器件好坏无关——这是物理限制，不是工艺限制。开关稳压器（LM2596 一类 buck）的做法完全不同：调整管只有全开、全关两个状态，占空比决定平均输出，电感在每个开关周期里储能再放能，把能量一批批搬到输出端。理想开关不耗能，实际效率约 75%~90%，损失主要在开关过程、电感电阻和续流二极管上。

给 5V 转 3.3V、1A 这一路算一笔账。用 LDO：输入功率 5W，输出 3.3W，差值 $(5 - 3.3)V \times 1A = 1.7W$ 全部变成热。1.7W 对 T0-220 封装不是小数目：无散热片时结到环境热阻约 $50\text{ }^{\circ}\text{C/W}$ ，温升约 $85\text{ }^{\circ}\text{C}$ ，室温稍高结温就逼近上限，散热片省不掉。换 LM2596 按效率 80% 估：输入只需 $3.3/0.8 \approx 4.1W$ ，损耗约 0.8W，不到线性方案的一半。压差越大差距越悬殊：输入换成 12V，线性方案效率只剩 $3.3/12 \approx 28\%$ ，开关方案仍在 80% 附近——同一负载，发热相差五倍以上。

代价在纹波和外围器件。LDO 的输出接近一条直线，纹波以 μV 到 mV 计，适合给模拟前端和噪声敏感电路供电。开关稳压器的输出叠加着开关频率的锯齿：LM2596 工作在约 150kHz，输出纹波典型几十 mV ，还混有开关瞬间的尖峰；给数字逻辑供电通常无妨，给 ADC 参考电压或射频电路供电就要追加滤波。外围器件也更多：buck 需要电感、续流二极管（或同步开关管）和输入输出电容，电感的体积和成本常常超过芯片本身。7805 还要求输入至少高出输出约 2V——这个最小压差称为 dropout，5V 输出就至少要 7V 输入；现代 LDO 可低至零点几 V，“低压差”三个字正是这么来的。选型因此是一道算术题而不是口味题：压差小、电流小、噪声敏感，用 LDO；压差大、电流大、效率优先，用开关。

开关与线性还可以串联使用，这正是经典组合“开关预稳压加 LDO 后级滤波”的理由。LDO 的反馈环路不仅稳压，也抑制输入纹波：数据手册的电源抑制比（PSRR）在低频段常有 60dB，输入 100mV 的纹波到输出只剩 0.1mV；但 PSRR 随频率升高而下降，150kHz 的开关纹波还剩多少抑制，必须查曲线而不是猜。正确用法是让 buck 把 12V 高效降到 4V 上下，再由 LDO 滤到干净的 3.3V——效率由开关挣，干净由 LDO 挣。直接拿开关输出给噪声敏感电路供电，等于只算了这笔账的一半。

精确一点，线性稳压器的效率应写成 $\eta = V_{\text{out}}I_{\text{out}}/(V_{\text{in}}(I_{\text{out}} + I_Q))$ ， I_Q 是器件自身的静态电流，7805 约几 mA。重载时 I_Q 可忽略，效率就是电压比；轻载时 I_Q 占比上升，效率比电压比还低。开关稳压器轻载时也要为每个开关周期付出固定损耗。所以“效率 80%”是额定负载附近的数字，空载时两类稳压器都不好看——给电池供电的常开电路选型时，数据手册的静态电流一栏比效率曲线更关键。

测量上，两类稳压器一眼可分。量输入、输出电流：线性稳压器的输入电流几乎等于输出电流（只差几 mA 静态电流）；开关稳压器的输入电流按功率折算明显更小，12V 转 5V/1A 的 buck 输入电流只有约 0.5A。器件烫手时，先算 $(V_{\text{in}} - V_{\text{out}}) \times I_{\text{out}}$ 是否超出封装散热能力；纹波异常时，示波器 AC 耦合看输出：近直线是线性，规则的 kHz 锯齿是开关，锯齿幅度超标多半是输出电容老化失效。

关键问题	线性稳压（7805 / LDO）	开关稳压（LM2596 buck）
多余电压的去向	降在调整管上变成热	开关搬运，理想不耗能
效率（5V 转 3.3V/1A）	66%，浪费 1.7W	约 80%，浪费约 0.8W
输出纹波	μV – mV 级，接近直线	几十 mV ，含 150kHz 锯齿
必需外围器件	输入输出电容各一只	电感、二极管、输入输出电容
输入压差要求	7805 约 2V，LDO 可更低	输入范围宽，大压差也能工作
适用场合	小电流、噪声敏感、压差小	大电流、压差大、效率优先

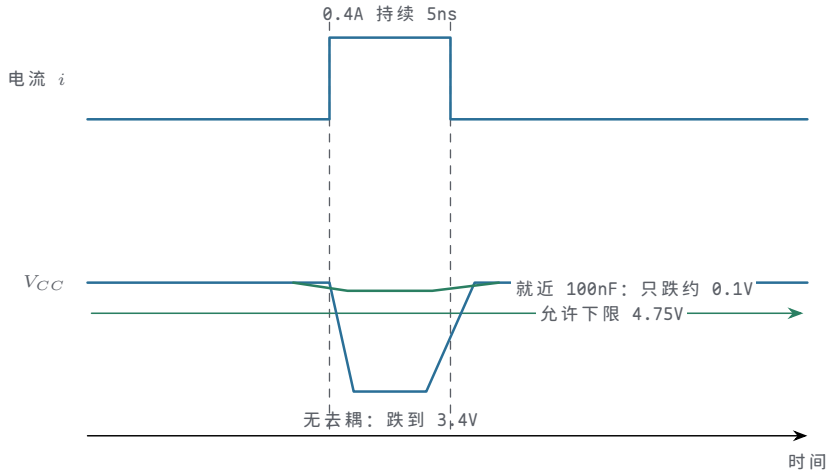
瞬态电流让电压跌落 芯片不是按平均值耗电的。CMOS 门只在翻转瞬间从电源取电流：输出由低充高，电流给负载电容充电；由高放低，电荷泄到地。一组门同时翻转——32 位总线换数、计数器多位同翻、时钟沿后整片组合逻辑一起动作——尖峰叠加成一个 ns 级电流脉冲。教学规模就容易达到 0.4A 在 5ns 内出现又消失的量级（16 路输出各取 25mA 即 0.4A）；大型芯片的同时开关电流以 A/ns 计。

电流尖峰本身不可怕，可怕的是它流过寄生电感。任何一段导体都有电感，经验法则：每 mm 走线约 1nH，2cm 电源走线就是约 20nH。电感的本性是反对电流变化，感应电压 $v = L \cdot di/dt$ 。代入： $di/dt = 0.4\text{A}/5\text{ns} = 0.08\text{A/ns}$ ， $v = 20\text{nH} \times 0.08\text{A/ns} = 1.6\text{V}$ 。尖峰出现的几 ns 内，芯片实际看到的电压从 5V 跌到 3.4V——而 $5\text{V} \pm 5\%$ 只允许 $\pm 0.25\text{V}$ ，这一次跌落是允许值的六倍多。后果不是“信号差一点”这种模拟意义上的劣化，而是门延迟突变、存储节点电荷平衡被破坏，触发器采错或保持不住，表现为与数据模式相关的偶发错误。

指望稳压器纠正这种跌落，是错配了时间尺度。稳压器靠反馈环路工作：采样输出、比较基准、调整管子，环路带宽折算成响应时间是 μs 级——LDO 几 μs ，开关稳

压器还受开关周期限制，150kHz 一个周期 6.7μs。电压跌落发生在 5ns 里，比稳压器的反应快三个数量级，稳压器连“出事”都来不及知道。ns 级事件的供电只能靠储能就在芯片身边的元件：电容。电容与芯片之间的路径电感决定最初几 ns 的电压缺口，容值决定之后能撑多久。

把电流尖峰与电压跌落画在同一时间轴上，因果的对齐直接可见。



图示：瞬态电流与电压跌落在同一时间轴上的对齐。0.4A 持续 5ns 的电流尖峰流过约 20nH 的电源路径电感，感应出 1.6V 缺口：无去耦时芯片看到的 V_{CC} 从 5V 跌到 3.4V，远低于 4.75V 的允许下限；贴近电源脚的 100nF 在尖峰期间就地供电，同一事件下只剩约 0.1V 的浅坑。稳压器的 μs 级响应在这个时间尺度上帮不上忙。

电容要出多少货，可以算出来。0.4A 持续 5ns，电荷 $Q = I \times t = 2\text{ nC}$ 。允许电压因此下降 0.1V，需要 $C = Q/\Delta V = 20\text{ nF}$ ——一只 100nF 在忽略寄生时绰绰有余。但前提写在上一段：电容到电源脚的路径电感必须远小于 20nH，否则最初几 ns 仍是 1V 量级的缺口。这就是“就近”排在“够大”前面的原因：贴着电源脚的 100nF，远胜板子另一端的 10μF。

大型处理器把同一个方程推到极端：核心电压 1V 上下、瞬时电流变化几十 A、边沿 ps 级，di/dt 以几十 A/ns 计——哪怕 1nH，也是几十 mV 起步的坑。处理办法仍是同一套时间尺度分工，只是层数更多：板上电容、封装电容、片上电容各管一段，电源引脚数以百计。芯片封装内部也要算这笔账：每个电源脚自带几 nH 键合线电感，多个输出缓冲同时翻转时，片内电源轨和地轨跟着晃动，称为同时开关噪声 (SSN)，§ 四会从信号串扰的角度再看它一次。教学机的 0.4A/5ns 与处理器的几十 A/ns 相差两三个数量级，方程是同一个。

排查时把示波器探头直接搭在芯片电源脚与就近地之间，地线越短越好——长地线自己就是电感，§ 五会专门讲。用单次触发捕捉总线翻转瞬间的电源波形。错误指纹也值得记住：跑固定数据死、换一组数据就过，这种与数据模式相关的偶发错误，第一嫌疑就是电压跌落。

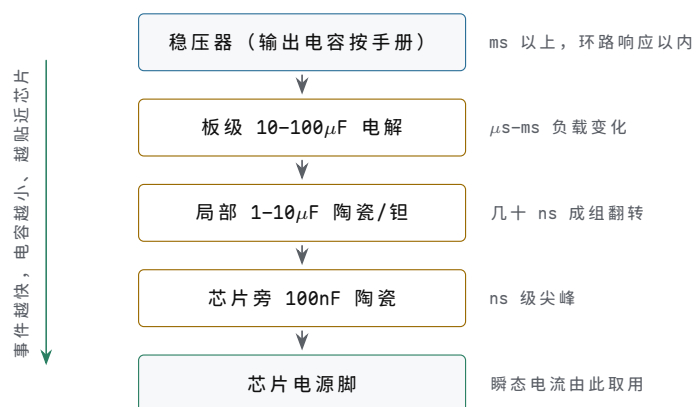
去耦电容按时间尺度分层 没有一只电容能覆盖全部时间尺度，原因仍是寄生：每只实际电容都串联着引脚与内部结构的电感 (ESL) 和电阻 (ESR)。大容量电解、钽电容体积大、ESL 大，对快事件相当于不在场；小容值的陶瓷电容 ESL 小、响应快，但储能少、撑不久。去耦网络因此按时间尺度分层，每层只管自己来得及管的事：

层级	典型容值与介质	服务的时间尺度	位置规则
芯片旁	100nF 陶瓷	ns 级尖峰	贴电源脚，路径几 mm 内
局部	1-10μF 陶瓷/钽	几十 ns 的成组翻转	每个功能模块一只
板级	10-100μF 电解	μs-ms 负载变化	电源入口，撑到稳压器响应

稳压器输出 数据手册指定 环路响应以内 紧贴稳压器，关系环路稳定

距离规则逐层放宽的理由：层级越高，事件的 di/dt 越小，路径电感惹出的 $v = L \cdot di/dt$ 越小，允许的距离就越大。ns 级尖峰只给几 mm 的往返路径，100nF 必须贴在电源脚旁； μs 级事件的 di/dt 小了三个数量级，10 μF 放在 cm 级距离照样有效；板级电解甚至可以只管电源入口。容值规则逐层加大约百倍的理由：每层既要自己存货够用，又要能在下一层耗尽之前接力。100nF 在 0.1V 压降下放完的储能是 10nC，约接五次 2nC 尖峰；尖峰成群出现时，由旁边的 1-10 μF 给它补仓，依此类推，直到稳压器环路接管最慢的部分。

把这张表竖起来画，去耦网络的分层结构就更直观：越靠近芯片，服务的事件越快、电容越小、贴得越近。



图示：去耦网络按时间尺度分层。储能自上而下逐层减小、响应逐层变快：每层只管自己来得及管的事件，耗尽之前由下一层补仓，最慢的部分留给稳压器环路。

缺层的后果各不相同，指纹却是同一张：与数据模式相关的偶发错误。只有 100 μF 没有 100nF，快事件没人接，电源波形上全是窄尖峰；只有 100nF 没有 μF 级，成群翻转时 100nF 瞬间耗尽，电压缓跌几 μs ；100nF 装了却离电源脚 5cm，路径电感把它与快事件隔开，等于没装——手工板上最常见的一种。分层规则浓缩成三句话：小电容给快事件，大电容给慢事件；越小的电容越贴近芯片；相邻层级容值相差约百倍，让每层都接得住下一层。

手工板与面包板上的做法是对这张表的直译：每片芯片的电源脚旁一只 100nF 陶瓷，引脚剪到最短、跨在电源与地之间；每块面包板的电源入口一只 10 μF ；整板进线处一只 47-100 μF 电解。这套规矩从 TTL 时代沿用至今，器件换了几代，时间尺度的物理没有变。最常见的错误是把电容插在离芯片两排孔之外”意思一下”——一对 ns 级事件，那几 mm 的孔距就是电感，电容约等于没插。

去耦是否到位，可以用对比实验量出来。示波器探头搭在某片芯片的电源脚上，先记录当前纹波；再在同一个电源脚上并一只 100nF，纹波中的窄尖峰应明显变矮；把这只电容挪到几 cm 外，尖峰又涨回来——同一只电容，距离不同，效果立见。这个实验值得在面包板上亲手做一遍：它把”就近”两个字从规则变成亲眼所见。

PDN 目标阻抗：把电压波动写成欧姆问题 前面把时间轴切成几段、每段配一只电容，工程上还需要一个把所有段落统一起来的设计目标。办法是换到频率轴：把”电压不许波动超过 ΔV ”与”负载电流阶跃最大 ΔI ”相除，得到一条对全频率成立的阻抗上限——电源分配网络（PDN）的目标阻抗 $Z = \Delta V / \Delta I$ 。5V $\pm 5\%$ 意味着 $\Delta V = 0.25V$ ，瞬态 1A 意味着 $\Delta I = 1A$ ，目标阻抗 0.25 Ω （250m Ω ）。窗口一收紧数字立刻变小： $\pm 0.5\%$ （25mV）对 1A，目标就是 25m Ω ；3.3V $\pm 5\%$ （165mV）对 5A 瞬态，约 33m Ω ；1V 核电压、几十 A 瞬态的处理器，目标阻抗以 m Ω 计。电源设计的全部手段，都是让 PDN 阻抗曲线在整个频率轴上位于这条目标线以下。

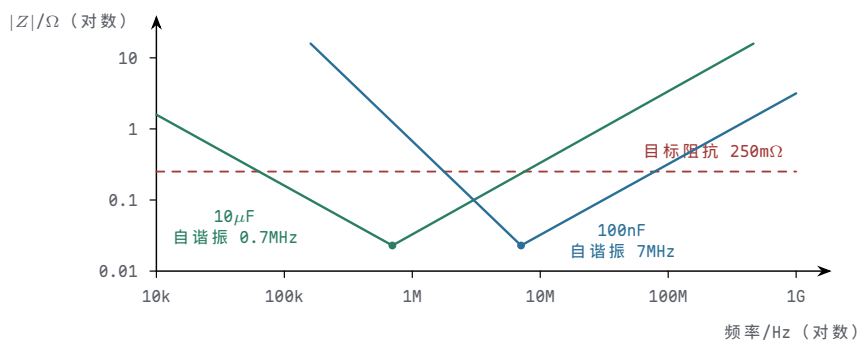
”足够多、足够近、足够多种容值”这条口诀，在阻抗曲线上看得最清楚。单只电容的阻抗随频率呈 V 形：低频端是 $1/(2\pi fC)$ ，随频率升高而下降；高频端被

ESL 接管, $2\pi fL$ 随频率升高而上升; 谷底是自谐振频率 $1/(2\pi\sqrt{LC})$ 。给 100nF 配 5nH ESL, 自谐振约 7MHz , 高于此频率它是一只电感; $10\mu\text{F}$ 配同样 5nH , 自谐振约 0.7MHz 。两条 V 形错开正好接力: $10\mu\text{F}$ 管 MHz 以下, 100nF 管 10MHz 附近——这就是“足够多种容值”。同值电容并联 N 只, 阻抗近似除以 N ——这就是“足够多”。每只电容到芯片的路径电感直接抬高 V 形右半边, 距离翻倍、高频段阻抗跟着翻倍——这就是“足够近”。更高频段 (百 MHz 以上) 分立电容无能为力, 靠电源-地平面对的平面电容和片上电容收尾; 教学机到不了这个频段, 规则却是同一条。

目标阻抗可以直接用来“点兵”。仍取 $5\text{V} \pm 5\%$ 、瞬态 1A , 目标 0.25Ω 。看 10MHz 这一点: 一只 100nF 配 5nH ESL, 已过自谐振, 阻抗约 $2\pi fL = 2\pi \times 10^7 \times 5 \times 10^{-9} \approx 0.31\Omega$, 单只不达标; 4 只并联, 阻抗约 0.08Ω , 达标。同一算法对每个关心的频率点重复一遍, 电容的数量与搭配就定了一去耦设计至此从经验口诀变成算术。

目标阻抗也把测量变成明确的事: 测出 PDN 阻抗随频率的曲线, 凡是冒过目标线的频段, 就是偶发错误的嫌疑频段。排查顺序: 先算目标阻抗, 再看曲线在哪个频率冒头, 代入自谐振公式, 就能锁定该补哪一层。

把 $10\mu\text{F}$ 与 100nF 两条 V 形曲线连同目标线画在同一张图上, 这层“接力”关系就一目了然。



图示: PDN 阻抗随频率的分布 (双对数示意)。单只电容的阻抗呈 V 形: 低频端按 $1/(2\pi fC)$ 下降, 自谐振点触底, 高频端被 ESL 接管按 $2\pi fL$ 上升。两种容值的 V 形错开接力, 设计目标是让合成阻抗在全频段压在设计目标线以下。

地平面与回流路径 去耦回答电流从哪里来, 同样重要却更常被忽略的是: 电流回哪里去。每个信号都携带一个回流—驱动器推出电流, 沿信号线到达接收端, 再沿参考面流回驱动器的地。直流回流走电阻最小的路径, 在整个地平面上铺开; 高频回流走阻抗最小的路径, 而高频下阻抗以电感为主, 电感最小意味着环路面积最小——于是高频回流贴着信号线正下方的地平面走。这是电磁场自己选的路: 信号线与正下方的回流构成一对紧耦合导体, 环路面积被限制在“走线长度乘介质厚度”的窄条里, 介质厚度只有零点几 mm 。

一旦在地平面上开缝, 这条窄条就被撕开。信号线跨越分割缝时, 正下方没有铜, 回流只能绕到最近的桥接点再绕回来, 环路面积从 mm^2 级涨到 cm^2 级。环路电感大致随面积增长, 几十 nH 很平常; 再代入本节开头那个 $0.4\text{A}/5\text{ns}$ 的尖峰, 又是 1V 量级的噪声——只是这次它出现在地参考上, 称为地弹: 所有以地为参考的信号跟着一起晃。大环路还是高效天线, § 四讲电磁兼容时还会回到这一笔。

分割不是原罪, 跨缝才是。合理的分割把噪声悬殊的电路隔开: 继电器、电机驱动与逻辑电路分两地, 唯一连接做在电源入口的单点; 模拟小信号区与数字区分地, 唯一连接做在 ADC 正下方。分割成立的判据只有一条: 没有任何信号线跨越分割缝, 所有跨区信号都经过桥接点同行。信号密集交错、说不清哪根线会跨缝的板子, 宁可保留完整地平面、靠器件分区摆放来隔离, 也不要开一条逼着回流绕路的缝。

没有完整地平面时的折中方案, 同样是把回流“贴着信号走”人工维持出来。双面板可以织网格地: 顶层地线横走、底层地线纵走, 过孔在交叉点相连; 关键信号 (时钟、复位) 旁边再有意识地并行走一根地线。面包板连网格也没有, 那就把时钟线与其回流地线绞在一起, 把环路面积拧到最小。低速时绕路代价小, 高速时绕路

直接报废时序与波形；但回流不是自动存在的，是被设计出来的——这一点高低速相同。

完整地平面还有第二个身份：它是信号走线特性阻抗的参考面。走线阻抗由导线与参考面的几何关系决定，参考面连续，阻抗才连续；回流贴着信号走与阻抗连续，是同一块铜的两个作用。§ 三讲传输线时会发现，阻抗突变处的反射与回流绕路处的噪声，追到底常常是同一条地缝。地平面的完整性因此同时是电源完整性与信号完整性两章内容的共同前提。

排查接地问题的手法有两个。一是查图：沿每根高速信号走线看正下方的地平面是否连续，跨缝处就是第一嫌疑人。二是量地弹：示波器探头的信号端与地线分别接同一地网络上的两个点，测到的就是两点间的电压差，即回流惹出的噪声；偶发错误若与某几根信号的活动强相关，先量这些信号的回流路径。

现象	先怀疑什么	检查点
负载加重就复位	板级储能不足、稳压器限流	电源入口有无 μs 级缓跌
错误与数据模式相关	100nF 缺失或离芯片太远	芯片电源脚上的窄尖峰
某模块一动作全板出错	回流绕路引起地弹	该模块信号是否跨越地缝
输出有规则 kHz 锯齿	开关纹波，判断是否超标	AC 耦合测峰峰值对照容忍度
LDO 烫手	$(V_{in} - V_{out}) \times I_{out}$ 热耗	散热片与封装热阻

这张排查表的使用方式是从指纹倒推元件：先确认错误与数据模式、负载、某模块动作中的哪一项相关，再按时间尺度对号入座——ns 级尖峰找 100nF 的位置， μs 级缓跌找板级储能与稳压器，地与信号一起晃找回流路径。电源问题极少需要换芯片，答案几乎都在电容、距离与路径这三件事上。

核心理想

电源完整性可浓缩为一条主线：稳压器负责 ms 级以上的平均电压，去耦网络负责让 PDN 阻抗在所有频率下低于 $\Delta V/\Delta I$ ，地平面的完整性决定回流是否绕路。按时间尺度分工、按频率轴核算，是电源设计的全部方法。

二、时钟树：分配、偏斜与抖动

第二章建立时序预算时，默认每个触发器在同一个瞬间看到时钟沿。物理上这个瞬间不存在：时钟从源出发，沿走线传播，经缓冲器放大，到达各触发器的时刻必然有先有后——这是**偏斜** (skew)，确定性的、由结构决定的时间差。即使结构完全对称，沿的位置还会被电源噪声、串扰和器件噪声推着随机游移——这是**抖动** (jitter)，统计性的、由噪声决定的时间差。

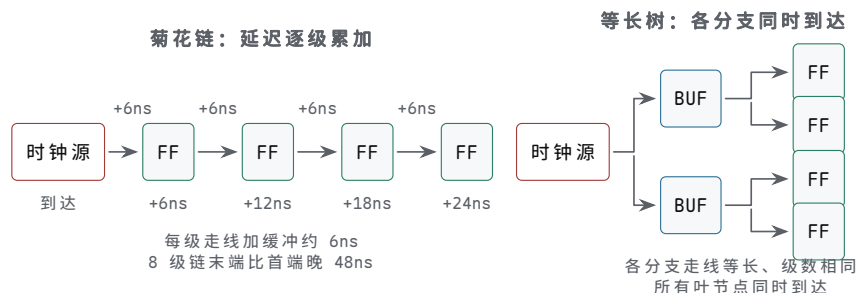
把两者分清有一个实际理由：修理手法互不通用。偏斜靠改结构（等长、对称、补齐级数），抖动靠改噪声（去耦、屏蔽、换更好的源），锁定问题靠改环路（带宽、参考、配置次序）。故障定位的第一步永远是先问：这个时间差是确定的，还是随机的？本节按“先结构后统计”组织：先看分配结构怎样把偏斜收小，再把偏斜与抖动写回时序预算，最后看管时钟源头的两个部件：锁相环与晶体振荡器。

时钟分配用等长的树 让所有触发器的时钟沿对齐，等价于让“时钟源到每个触发器”的每条路径延迟相等。延迟来自两部分：走线传播延迟与缓冲器延迟。FR4 板材上信号传播速度约 15cm/ns（光速除以介电常数的平方根， $\sqrt{4}=2$ ），1cm 走线约 67ps；一级缓冲器几 ns。两种极端结构的对照最能看清问题。菊花链：时钟先到第一个触发器区，再串到下一个，像击鼓传花——8 级链、每级走线加缓冲 6ns，末端比首端晚 48ns，而 50MHz 的周期一共只有 20ns。等长树：从根分权，每级扇出经

缓冲器驱动下一级，各分支走线等长、缓冲级数相同，所有叶节点的沿在结构上同时到达。

把数字代进树里。某设计用两级缓冲，每级 5ns。只要两个分支的缓冲级数相同， $2 \times 5\text{ns}$ 在两边相消，偏斜只剩走线差：分支长度差 10cm， $10\text{cm} \div 15\text{cm/ns} \approx 0.67\text{ns}$ ，这就是两个叶节点之间的偏斜。再看级数不相等的情形：一个分支两级、另一个三级，仅缓冲延迟就差出 5ns，加上走线差共 5.67ns——级数匹配比长度匹配敏感近十倍。负载也要匹配：同样的长度下，负载电容差 50pF，缓冲延迟可再差 1-2ns。工程做法因此是三部曲：树先对称，级数对齐，长度再用蛇形走线补齐。芯片内部的时钟树综合（CTS）做的是同一件事，只是分支成千上万，由工具在版图上逐级插缓冲、绕等长，H 形分权的几何保证从中心到每个角落距离相等。

两种结构并排画出，延迟的累加与抵消一眼可辨。



图示：时钟分配的两种结构。菊花链中每级走线与缓冲延迟逐级累加，末端沿越来越晚；等长树让每条“根到叶”路径的走线长度与缓冲级数相同，缓冲延迟在分支间相消，所有触发器在结构上同时看到时钟沿。

树形还有第二个理由：驱动能力。一个时钟源要送到几十上百个触发器，负载电容累加起来是几十到几百 pF，单级门驱动这样的负载，延迟从 5ns 涨到几十 ns，边沿也变缓——缓边沿本身又制造新的偏斜与保持问题。逐级缓冲让每个缓冲器只面对本级扇出的小负载，延迟可预测、边沿保持陡峭。板级实现里，专用时钟缓冲芯片（一路进、多路等相位出）就是一棵预制好的小树。

树形分配的代价是功耗。时钟网络是全芯片活动率最高的网络：每个周期无条件全翻转，负载电容又是各网络中最大的一类，现代处理器里时钟树可占动态功耗的三到四成。这正是低功耗设计给时钟树配门控的原因——而门控必须由时钟树工具统一插入并重新平衡，手工在某一枝上加门，偏斜立刻失控。功耗与偏斜在这里短兵相接：省功耗的手段制造偏斜，治偏斜的手段费功耗，工程上是同一个权衡的两面。

菊花链并非一无是处：移位寄存器、扫描链这类数据本身就逐级流动的结构，时钟顺着数据方向走反而有利——下一级时钟更晚，恰好帮助保持约束。但它服务的是“顺序”而非“同时”；凡要求沿对齐的地方，答案都是树。

测量偏斜用双通道示波器：两路探头分别搭在两个触发器的时钟脚，直接读沿的时间差。注意探头必须同型号、同长度、同衰减比——探头差异本身会冒充偏斜。低速教学机上 1ns 以下的偏斜，很容易被测量装置自己的不对称掩盖，这也是一条测量教训。

偏斜可以在面包板上亲手制造。搭一个两位移位寄存器：第一级的时钟直接取自时钟源，第二级的时钟故意绕一条远路，或串两级反相器充当延迟。缓慢送入已知图样，观察第二级输出：第二级时钟晚到得足够多时，同一个时钟沿会被两级先后采到，数据连跳两位——保持违例的实物版。把绕路拆掉、两级等长，故障消失。这个几分钟的实验比任何公式都更能让人记住：“时钟沿必须同时到达”不是修辞，是可以被违反、也必然会出事的物理约定。

关键问题	菊花链	等长树（逐级缓冲）
沿到达时刻	逐级递晚，末端晚 N 段延迟	结构上同时到达
偏斜量级	8 级链约 48ns，超周期	长度差 10cm 约 0.67ns
驱动能力	首级驱动全部负载	每级缓冲只驱动本级扇出

适用结构 移位、扫描等顺序型数据 一切要求沿对齐的同步系统
流

偏斜吃掉时序预算 第二章的预算例：周期 20ns (50MHz)， $t_{CQ_max} = 3ns$ ，组合逻辑最坏 9ns， $t_{SU} = 2ns$ ，无偏斜时占用 14ns，余量 6ns。现在把偏斜按方向分两次代入，结论会分裂成两条。

建立检查怕数据来得晚。若目标触发器的时钟比源早到 1ns，有效周期只剩 19ns：14ns 需求对 19ns 供给，余量降到 5ns。同一偏斜换个方向——目标时钟晚到 1ns——有效周期变成 21ns，余量升到 7ns。对建立而言，“目的钟晚到”是礼物。

保持检查怕数据走得早。目标触发器采样沿之后，旧数据必须再坚持 $t_H = 0.5ns$ 。若目标时钟晚到 1ns，坚持时间被延长成 $0.5 + 1 = 1.5ns$ ；而数据最早在 $t_{CQ_min} = 1.5ns$ 后就可能更新——1.5 对 1.5，余量为零，站在违例边缘。偏斜到 2ns，要求 2.5ns 对供给 1.5ns，保持违例 1ns——这时把周期拉长到 100ns 也无济于事，因为保持约束与周期无关。刚才对建立的“礼物”，在保持上是债务：同一偏斜方向相反时，两种后果互换。这就是“偏斜吃掉预算”的准确含义：它不只是少了几 ns，而是同时从两个方向威胁两条约束。

检查	不利偏斜方向	代入数字	结果
建立	目标时钟早到 1ns	有效周期 19ns 对需求 14ns	余量 5ns
保持	目标时钟晚到 1ns	供给 1.5ns 对要求 1.5ns	余量 0，边缘
保持	目标时钟晚到 2ns	供给 1.5ns 对要求 2.5ns	违例 1ns

修复手段也分两条。建立问题可以降频换时间，因为建立余量随周期增长；保持问题与频率无关，只能在数据路径补最小延迟（插缓冲器、绕一段线）或修时钟树本身。高级设计里还有“有用偏斜”（useful skew）：故意让关键路径的目的钟晚到一点，向保持还富裕的路径借时间——这是把双刃剑，借来的每一分都要在保持端偿还。排查看错误指纹：随频率改变的错是建立方向，不随频率改变的错先查保持。

偏斜还有一个隐蔽来源：时钟门控。用与门把时钟“关掉”以省电或实现使能，门本身的几 ns 延迟只出现在被门控的分支上，等于在树的一侧凭空加了一级缓冲——偏斜立刻超标。第二章的结论在这里仍然成立：教学机上不要门控时钟，用时钟使能（CE）让触发器自持；门控时钟是带完整时钟树工具链的设计里才允许的手法。

把第二章那张预算表补全：占用项依次是 t_{CQ_max} 3ns、组合 9ns、 t_{SU} 2ns、偏斜 1ns、抖动窗口 0.7ns，合计 15.7ns；周期 20ns，余量 4.3ns。偏斜与抖动单独看都不致命，写进同一行才知道预算还剩多少——这就是时序预算必须把所有项列全的原因。

抖动是沿位置的随机游移 偏斜是结构的账，结构定了它就不变；抖动是噪声的账，每个沿都在理想位置附近游移。按观察窗口分三类：**周期抖动**，单个周期长度对理想周期的偏离；**周期间抖动**，相邻两个周期长度之差；**长期抖动**，沿位置对理想位置的累积偏离（TIE）。来源在教学机上就能找齐三个：电源噪声——门延迟对电压敏感，量级上 1% 的电压变化带来 1% 级的延迟变化，100mV 电源波动（5V 的 2%）扫过 5ns 的缓冲链，沿位置就被推着走 100ps 量级；串扰——邻近信号翻转通过容性、感性耦合在时钟线上叠加台阶，§ 四展开；振荡器与缓冲链自身的热噪声与闪烁噪声，这是再干净的板子也去不掉的底。

随机抖动的统计描述用高斯模型：沿位置以理想位置为中心，标准差 σ 即均方根（RMS）抖动。“峰值抖动”对高斯分布没有绝对上限，只能按可容忍的失误率取倍数： $\pm 3\sigma$ 覆盖 99.73%， $\pm 7\sigma$ 才把失误率压到 10^{-12} 量级。一只 RMS 50ps 的时钟按 $\pm 7\sigma$ 取窗口，沿位置的不确定范围是 $\pm 350ps$ ，峰峰 0.7ns。写进预算的做法很直接：建立检查用最坏周期——标称周期减去抖动窗口。与上一节的 1ns 偏斜合并：20ns 周期、14ns 需求，余量从纸面的 6ns 被偏斜和抖动合计吃掉 1.7ns，剩 4.3ns。确定性抖动（占空比失真、与数据码型相关的分量）不是高斯的，按峰峰值与随机分量

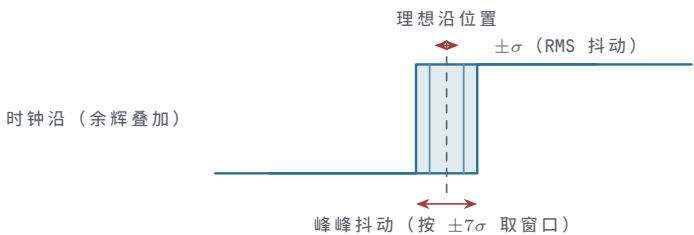
相加：总抖动 = DJ + $n \times$ RJ。多级缓冲链上，各级随机抖动按平方和开根累积，所以时钟链每多一级，抖动底就厚一分。

三种抖动各有主管的场合。周期抖动直接决定最坏周期，是同步时序预算的常客；周期间抖动刻画相邻周期的突变，PLL 与延迟锁定环（DLL）能否跟得上，看它；长期抖动决定串行链路和多周期同步的取样窗口，通信系统先问它。预算时选错种类等于白算：给 50MHz 同步机用长期抖动指标，或给串行链路只验周期抖动，都是把噪声看漏了一半。

测量抖动最直观的是示波器余辉（persistence）模式：让波形持续叠加几十秒，时钟沿”涂抹”出的宽度就是抖动的峰峰直观值；要求定量时，用时间间隔误差（TIE）直方图读出 RMS。抖动与 ξ 一的电源噪声是同一枚硬币的两面，测量时可以直接验证：两路探头一路看电源轨纹波、一路看时钟沿的散开，翻转密集、纹波增大时沿的散开跟着变宽——电源噪声调制门延迟的链条就此闭环。反过来，给时钟缓冲器就近补一只 100nF，沿的散开会明显收窄。这是少数能用一块面包板直接演示的”噪声变抖动”实验。

类型	定义	主要来源	预算写法
周期抖动	单周期长度对理想值的偏离	电源噪声、缓冲链噪声	用最坏周期代入建立检查
周期间抖动	相邻两周期长度之差	电源尖峰、串扰事件	限制 PLL 跟踪假设
长期抖动 (TIE)	沿位置对理想位置的累积偏离	振荡器漂移、低频噪声	与偏斜并列扣除余量

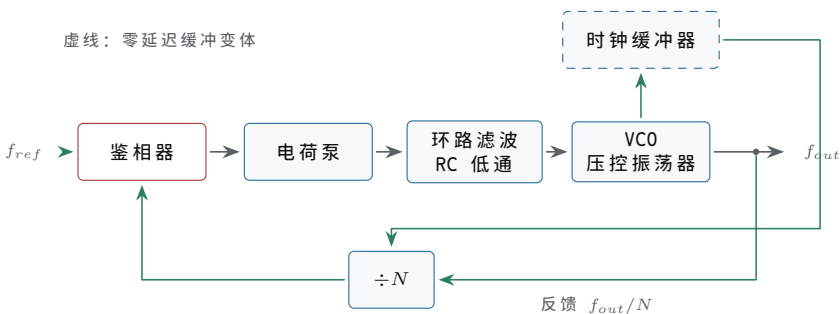
把同一个时钟沿用余辉模式叠加几十秒，沿位置的游移就涂抹成一条可见的带，带的宽度与统计指标直接对应。



图示：抖动的直接图像：示波器余辉模式下，同一个时钟沿在理想位置两侧涂抹成一条带。带的散布宽度对应 RMS 抖动 σ ，时序预算按 $\pm 7\sigma$ 的峰峰窗口扣除。

锁相环：倍频、去偏斜和滤波 PLL 的基本结构五环相扣：鉴相器比较参考沿与反馈沿的先后，输出与相位差成正比的误差信号；电荷泵把误差换成电流脉冲；环路滤波器（RC 低通）把脉冲积成平缓的控制电压；压控振荡器（VCO）按控制电压改变频率；分频器（ $\div N$ ）把输出降回参考频率附近，送回鉴相器。环路锁定时反馈沿与参考沿对齐， $f_{out} = N \times f_{ref}$ 。

把五环画成环路图，反馈取点与变体的改接处如下。



图示：锁相环基本环路（实线）与零延迟缓冲变体（虚线）。主环五环相扣：鉴相器比较参考沿与反馈沿，电荷泵把相位差换成电流脉冲，RC 环路滤波器积分平缓的控制电压，VCO 按它改变频率，输出经 $\div N$ 送回鉴相器，锁定时 $f_{out} = N \times f_{ref}$ 。变体把时钟缓冲器串在 VCO 之后、反馈改从缓冲器输出取：环路锁定强迫缓冲后的沿对齐参考沿，缓冲器那几 ns 延迟被相位比较自动吸收，这就是零延迟缓冲。

用途一，倍频。片外 12MHz 晶体，片内乘 8 得 96MHz。为什么不直接买 96MHz 振荡器？因为 96MHz 方波在 PCB 上的分配本身就是偏斜、反射与辐射的麻烦（§ 三、§ 四）；把高频限制在芯片内部、板级只走 12MHz，是高速系统的普遍取舍。用途二，去偏斜。把时钟缓冲器放进反馈环：鉴相器比较的不是 VCO 输出，而是缓冲器输出，环路锁定强迫“缓冲后的沿”对齐参考沿，缓冲器那几 ns 延迟被相位比较自动吸收——这叫零延迟缓冲。板级多片芯片要共用严格对齐的时钟时，专用 zero-delay buffer 芯片就是这个原理。用途三，滤波。环路带宽（例如 1MHz）以下的参考抖动，环路跟随；带宽以上的快抖动，环路来不及反应，输出由 VCO 自身的短期稳定度决定——参考上的 10MHz 抖动分量就这样被滤掉。给抖动的参考“洗干净”，靠的就是把带宽设低；代价是锁定变慢，上电后要等几十到几百 μs ，复位电路必须等 PLL 的 locked 信号才能释放。

倍频不是没有代价：相位噪声随倍频比恶化。频率乘 N 的同时，相位扰动也乘 N ，相位噪声谱整体上移 $20 \log_{10} N$ dB——乘 8 就是约 18dB。参考晶体在偏离载波 1kHz 处的相位噪声若是 -120dBc/Hz，倍频到 96MHz 后同一偏移处约 -102dBc/Hz。高倍频系统对参考质量的要求因此不降反升：PLL 放大频率的同时，也放大了参考的相位起伏。这与“调整器”的定位一致——它传递并放大参考的特性，而不是发明新的特性。

微控制器把这套结构做成了寄存器配置：外部 8MHz 晶体经 PLL 乘 9 得 72MHz 一类的选项，是启动代码里最平常的几行。配置次序有讲究：先起振、等稳定、再切时钟源、最后等 PLL 锁定——任何一步抢跑，CPU 就跑在一个还没站稳的时钟上。工业设计里还会给 PLL 配失锁检测：锁定丢失时自动切回安全时钟，免得一个时钟毛刺把整机带进未知状态。

定位要准确：PLL 不是时钟发生器，而是时钟调整器。振荡本身由 VCO 完成，VCO 自由运行时频率随电压、温度漂移；PLL 做的是把这只不听话的振荡器锁到外部参考上，让输出继承参考的准确度。参考才是准确度的来源，环路只是调整机构。用 PLL 的第一问永远是：参考从哪里来，它本身有多准。

晶振与振荡器的选择 参考源的常规选项有两个。晶体（quartz crystal）是一块石英薄片，靠压电效应在机械谐振频率上振动，本身不能起振，需要配振荡电路：教科书结构是皮尔斯（Pierce）——一只无缓冲反相器（74HCU04，或 MCU 片内的同等电路）跨接晶体，晶体两端各经一只负载电容到地，再并一只 $M\Omega$ 级反馈电阻把反相器偏置在线性区。独立振荡器模块（有源晶振）把晶体、放大与缓冲封在一个四脚金属壳里：VCC、GND、输出，通电即出方波，即插即用。

晶体的坑集中在负载电容。数据手册给的是负载电容规格 CL ，常见 18pF，它等于两只外接电容的串联值加走线杂散： $CL = C_1 C_2 / (C_1 + C_2) + C_{stray}$ 。取 $C_1 = C_2 = 27pF$ 、杂散按 5pF 估： $27 \times 27 / 54 = 13.5pF$ ，加 5pF 得 18.5pF，与规格吻合。配不准不会停振，但频率被拉动几十 ppm：32.768kHz 钟表晶体偏 20ppm，一天走差 $86400 \times 20 \times 10^{-6} \approx 1.7s$ 。起振也有讲究：晶体 Q 值高，幅度要 ms 级才能建立，上电后最初几 ms 时钟不可用，复位电路必须等它。布局上晶体要紧贴芯片、下方不走线——晶体走线是全板最敏感的高阻节点，串扰直接变成频率抖动。

两类常见晶体的差异也值得知道：MHz 级晶体多为 AT 切石英片，等效电阻几十 Ω ，起振快；32.768kHz 钟表晶体是音叉结构，等效电阻几十 $k\Omega$ ，驱动电平必须极低，过驱反而缩短寿命甚至停振。给 RTC 电路配晶体时，串联一只限流电阻、负载电容严格按规格，是“能振”与“振得准”的分界线。

数据手册里三个相近的词要分开：准确度指出厂时（25 °C、额定负载下）频率与标称值的偏差；稳定度指温度、电压变化范围内频率的漂移；老化指谐振频率随时间的缓慢单向漂移，典型每年几 ppm。一只“ $\pm 20ppm$ ”的普通晶体通常是三项的总和；温补晶振（TCXO）用温度补偿电路把稳定度一项收到 $\pm 1ppm$ 量级，代价是价格与功耗。选型时先问哪一项伤害应用：实时钟怕老化与温漂，通信接口怕总偏差，按

需买项，不必为用不到的指标付钱。

关键问题	晶体 + 片内振荡电路	独立振荡器模块
外围与调试	需皮尔斯电路与负载电容匹配	通电即用，无外围
启动时间	ms 级，幅度缓慢建立	ms 级，含内部稳定时间
频率稳定度	$\pm 20\sim 50\text{ppm}$ ，受匹配影响	同等级晶体，出厂已校准
成本与引脚	器件便宜，占两个引脚	单价高，体积大，即插即用
适用场合	批量产品、成本敏感设计	面包板、调试期、小批量

低速教学机的实用建议：调试期直接用振荡器模块或 555—教学机不与外部设备对表，ppm 无所谓，555 的 RC 漂移（百分之几）完全够用。什么时候必须上晶体？当时钟要和外部世界对齐：UART 串口要求收发两端波特率总误差控制在约 $\pm 2\%$ 以内，RC 振荡器已贴近边缘，晶体 $\pm 20\text{ppm}$ 则富余几个数量级。判据只有一句：只跟自己走的机器，频率差不多就行；要跟别的设备对表的机器，频率必须是有约定的 ppm。

振荡器模块也不是插上就万事大吉：它自己是耗电与时钟驱动的大户，电源脚同样需要 100nF 去耦；带 OE 脚的模块注意使能电平，悬空可能停在半导通状态；输出驱动长线时同样受 $\S$ 三反射问题约束，必要时串一只几十 Ω 的源端电阻。时钟源越“成品化”，越容易让人忘记它仍是一个模拟与数字混合的电路。

排查晶体问题的顺序：不起振，先查负载电容是否配错（过大难振、过小频偏），再查是否误用了带缓冲的反相器—74HC04 不行，要 74HCU04 这类无缓冲型，缓冲级的额外相移会破坏起振条件—最后查晶体本身是否受摔受损。频率用频率计或示波器直读；频率对而系统偶发错，查振幅与驱动电平是否不足。

核心思想

时钟分配的核心是管理两类时间差：偏斜是确定的、由结构决定，用对称的树把它收小；抖动是随机的、由噪声决定，用统计窗口把它计入预算。锁相环与晶振管的是源头质量：一个负责把振荡调整到与参考对齐，一个负责提供值得对齐的参考。

三、信号完整性：传输线、反射与端接

本节的出发点只有一句话：边沿不是瞬间到达的。一个边沿从驱动器出发，要以有限速度沿走线推进，途中每遇到一次阻抗变化，就分出一部分波反射回去。本节按问题出现的次序组织：先定“多长的走线算长”，再看长线上的行波由什么参数刻画，然后演算反射怎样形成振铃，最后给出端接的三种标准做法，以及过冲、回沟对输入端的压力与处置。全节只用三个量：传播速度、特性阻抗、反射系数；每一条结论都能还原成这三者的四则运算。

短线判据的定量版 电信号在导线里推进的载体是电磁场，速度由介质决定： $v = c/\sqrt{\epsilon_r}$ 。真空中 $c \approx 30\text{cm/ns}$ ；FR4 板材的相对介电常数约 4.4，内层带状线的速度就是 $30/\sqrt{4.4} \approx 14.3\text{cm/ns}$ ；表层微带线一半贴着空气，有效介电常数降到 3 上下，速度约 $16\sim 17\text{cm/ns}$ 。工程估算统一取 15cm/ns ，即每厘米约 67ps ：一根 10cm 走线，边沿从一头到另一头要约 0.7ns 。这个“慢”是否要紧，取决于和边沿时间的比。

判据比的不是绝对长度，而是线延迟与边沿时间的比值。边沿沿线推进时，反射波与入射波在途中叠加；若线延迟远小于边沿时间，反射还没来得及与入射分离，就被淹没在边沿的斜坡里，线上各点几乎一起动—这是“短”线。经验分界线取单向延迟等于边沿时间的六分之一：超过它，反射波在边沿尚未走完时已独立成形，过冲与振铃开始可以分辨，必须按传输线处理。写成算式：临界长度 $l_c = t_r \cdot v/6$ ， t_r 为边沿时间。六分之一不是物理定律，是“反射刚开始能被看见”的工程刻度，取四分之一或八分之一的主张都有，结论差不到一倍。

代入数字。74HC 在 5V 下边沿约 5–8ns, $l_c = 5\text{ns} \times 15\text{cm/ns}/6 \approx 12.5\text{cm}$, 慢边沿对应 20cm—面包板上十几厘米的飞线大多仍在短线区, 这正是前面七章能用集总模型讲下来的资格。换成边沿 1ns 的高速 CMOS, 临界长度只剩 2.5cm; 边沿 0.3ns 的现代 FPGA 输出, 5mm 就算长线。同一块板子上, “这根线短不短”没有统一答案, 要一根一根拿器件的边沿去比。

短线区与长线区是两套模型。短线区用集总模型: 整条线当作一个节点, 寄生参数并成一个对地电容 (10cm 走线约 5–15pF), 驱动器只是多带了一个负载—前面各章默认的就是这个世界。长线区必须用分布模型: 线上每一点的电压电流可以不同, 要用单位长度的电感 L' 与电容 C' 刻画, “波”的概念登场。

逻辑系列	典型边沿	临界长度 $t_r v/6$	10cm 走线怎么算
74HC (5V)	5–8ns	12–20cm	短线, 集总模型足够
74AC、74AHC	2–3ns	5–7.5cm	临界附近, 按长线稳妥
现代 FPGA、处理器 IO	0.1–1ns	2.5mm–2.5cm	长线, 必须按传输线设计

不知道边沿多快时有两条路。查数据手册的上升/下降时间一项 (74HC 手册给的是 50pF 负载下的典型值); 或用示波器直接量, 注意示波器加探头的等效带宽要满足 $0.35/t_r$ —量 2ns 的边沿, 测量链至少要有 175MHz, 即 200MHz 这一档。既没有手册也没有示波器时, 按“线长接近临界值就当长线”设计: 为长线做的端接用在短线上无害, 反过来则直接是故障, 宁保守勿侥幸。

特性阻抗不是电阻 边沿刚进入走线时, 并不知道末端接了什么—它只看到沿途每一小段要充的电和要建立的磁场。设线每厘米电容 C' 、传播速度 v , 波前每推进 1cm 用时 $1/v$, 并把这一厘米的电容充到 V , 需电荷 $Q = C'V$, 于是波前从驱动器抽走的电流 $I = Q/t = C'vV$ 。电压与电流之比 $V/I = 1/(C'v)$ 只由线本身决定, 定义为特性阻抗 Z_0 ; 用 $v = 1/\sqrt{L'C'}$ 代入, 就得到常见的形式 $Z_0 = \sqrt{L'/C'}$ 。它的物理含义一句话讲尽: 行波看到的瞬时负载。波在途中的一切行为—分压、反射、吸收—都由这个比值决定, 与末端还有多远无关。

代入典型值。一条 50Ω 微带线, $C' \approx 1.3\text{pF/cm}$, $L' \approx 3.3\text{nH/cm}$ 。验算: $Z_0 = \sqrt{3.3\text{nH}/1.3\text{pF}} \approx \sqrt{2538} \approx 50\Omega$; $v = 1/\sqrt{L'C'} = 1/\sqrt{3.3\text{nH} \times 1.3\text{pF}} \approx 65\text{ps/cm}$, 即约 15.3cm/ns, 与上一段的取值自洽。3.3V 边沿进入这样的线, 波前瞬时电流 $I = 3.3/50 \approx 66\text{mA}$ —哪怕末端开路、稳态电流为零, 驱动器在边沿期间也要供出这 66mA。这就是“传输线负载”与“电阻负载”最直观的差别: 稳态开路, 瞬态却是 50Ω。

50Ω 不是物理常数, 而是历史折中的沉淀。同轴电缆时代, 空气介质同轴衰减最小的点在约 77Ω, 功率容量最大的点在约 30Ω, 50Ω 是两者的折中, 随射频与仪器工业成为标准; 视频与有线电视继承了低损耗那一端, 75Ω 沿用至今。PCB 沿用 50Ω 还有现实理由: 现代 CMOS 驱动器输出阻抗十欧上下, 串一只三十几欧的电阻就能配到 50Ω; 定得更高驱动器够不着, 定得更低静态功耗又太大。双绞线约 100Ω、高速串行链路常用 85–100Ω 差分, 都是同一类折中的不同选择。

三个“无关”值得逐一确认。与线长无关: L' 、 C' 都是每单位长度的参数, 比值里长度被约掉。与频率无关: 无损近似下两者都不含频率, Z_0 在很宽频段里是常数—它不是电阻, 却表现得像一个电阻, 这正是能拿电阻做端接的根据。与绝对粗细无关: 把横截面按同一比例放大, L' 与 C' 的比不变, Z_0 不变—它只由几何比例 (线宽对介质厚度之比) 与介电常数决定。但若只加宽线、不动介质, C' 增、 L' 减, Z_0 按平方根下降: 50Ω 的线线宽加倍, 阻抗大致减半。电源线加宽总是好事, 信号线加宽之前要先算这笔数。

用万用表电阻档量不出 50Ω: 欧姆表量的是导体的直流电阻, 一条铜走线只有毫欧级。特性阻抗只对变化中的边沿“显形”, 稳态下它不存在。反过来也别走

到另一极端，把传输线想成神秘元件：它就是一段处处一样的介质，全部学问只有 L' 、 C' 两个参数和它们的比。

测量上，没有仪器也能估 Z_0 ：用一只电容表量一段已知长度走线对参考面的总电容，除以长度得 C' ，再由 $Z_0 = 1/(C'v)$ 、取 $v \approx 15\text{cm/ns}$ 反推。例：30cm 走线量得 40pF， $C' = 1.33\text{pF/cm}$ ， $Z_0 = 1/(1.33\text{pF/cm} \times 15\text{cm/ns}) \approx 50\Omega$ 。专业仪器是时域反射计 (TDR)：向走线发一个已知快边沿，从回波形状直接读出沿线每处的阻抗剖面，连接器、过孔、线宽突变在图上各成一个台阶——原理就是本节三个公式的反向使用。

反射系数与振铃演算 行波到达阻抗突变处，波带来的电压电流比是 Z_0 ，新阻抗要求的却是另一个比值，矛盾只能靠再产生一个波调和：一部分透射，一部分反射。由边界上电压连续、电流连续两个条件，解出反射电压与入射电压之比——反射系数 $\Gamma = (Z_L - Z_0)/(Z_L + Z_0)$ 。三个特例值得记住： $Z_L = Z_0$ 时 $\Gamma = 0$ ，波被完整吸收，这叫匹配；末端开路 $Z_L \rightarrow \infty$ 时 $\Gamma = +1$ ，反射与入射同号，末端电压跳到入射波的两倍；末端短接 $Z_L = 0$ 时 $\Gamma = -1$ ，反射反号，末端电压归零、电流加倍。

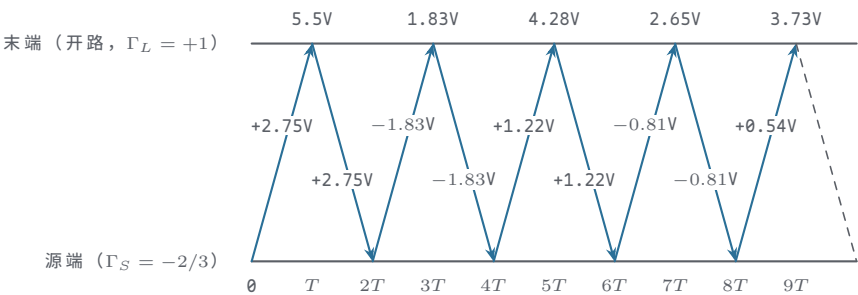
反射波回到源端，源端同样是一处阻抗突变，按 $\Gamma_S = (Z_S - Z_0)/(Z_S + Z_0)$ 再反射一次。CMOS 驱动器的输出阻抗通常小于 Z_0 ， Γ_S 为负：回波到源端被反号弹回，再向末端走去，到末端又全反射……波就在两端之间来回弹，每弹一圈，幅度打一个 $|\Gamma_S\Gamma_L|$ 的折扣。振铃不是神秘现象，它就是这场来回弹的直接记录。

完整演算一遍。3.3V 系统、50Ω 走线、源阻 10Ω、末端开路，记单向延迟为 T 。入射波幅度由源阻与线阻抗分压： $3.3 \times 50/(50 + 10) = 2.75\text{V}$ 。 $t = T$ 时波到末端， $\Gamma_L = +1$ ，末端电压从 0 跳到 $2.75 + 2.75 = 5.5\text{V}$ ——稳态 3.3V 的 1.67 倍；若源阻为零，入射即满幅 3.3V，末端将冲到 6.6V，接近电源电压的两倍，这就是“开路全反射，过冲接近两倍”的准确含义。回波 2.75V 返到源端， $\Gamma_S = (10 - 50)/(10 + 50) = -2/3$ ，弹回 -1.83V； $t = 3T$ 时它到达末端并再次全反射，末端电压 $5.5 - 1.83 - 1.83 = 1.83\text{V}$ 。如此往复，每往返一圈幅度乘 2/3，逐拍收敛到 3.3V：

时刻	到达末端的波	末端电压	现象
$t = T$	入射 2.75V，末端全反射再加 2.75V	5.5V	最大过冲
$t = 3T$	源端弹回 -1.83V，末端再反射	1.83V	最深回沟
$t = 5T$	+1.22V 到达并翻倍	4.28V	二次过冲
$t = 7T$	-0.81V 到达并翻倍	2.65V	二次回沟
$t = 9T$	+0.54V 到达并翻倍	3.73V	向 3.3V 收敛

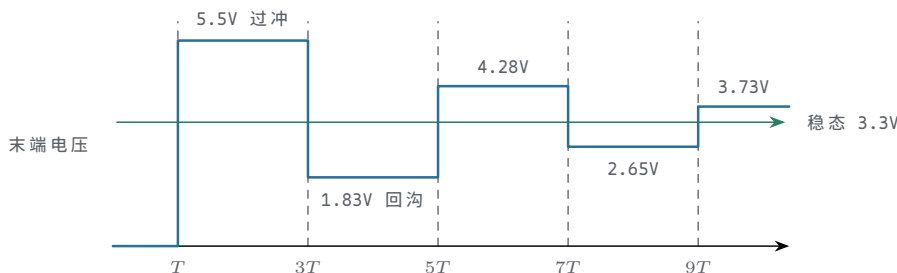
两点解读。其一，末端电压每 $2T$ 跳一次，一个完整的“峰-谷-峰”历时 $4T$ ：15cm 走线 $T \approx 1\text{ns}$ ，振铃约 250MHz——线越短振铃越高频，低速示波器看不见，看不见不等于不存在。其二，每圈衰减因子 $|\Gamma_S\Gamma_L| = 2/3$ 由两端不匹配程度的乘积决定；源阻若只有 1Ω（强驱动器）， Γ_S 约为 -0.96，振铃要拖很多圈才平息。把这套逐拍递推画成“时间-位置”斜线图（弹跳图），就是分析任意端接组合的通用工具：每个交点按当地的 Γ 分裂，照抄上面的四则运算即可。

把上面这组数画成弹跳图，每一次反射都与表中的逐拍演算对应。



图示：弹跳图：横轴时间、纵轴位置，斜线是行波在源端与末端之间的来回，每跨一次耗时 T 。入射 $+2.75\text{V}$ 在 $t = T$ 到达开路末端， $\Gamma_L = +1$ 全反射，末端跳到 5.5V ；回波在源端按 $\Gamma_S = -2/3$ 反号弹回，每往返一圈幅度乘 $2/3$ 。上轨标出的各时刻末端电压 5.5 、 1.83 、 4.28 、 2.65 、 3.73V 与上表逐项一致，逐拍收敛到稳态 3.3V 。

把这张逐拍表画成末端电压波形，振铃的形状与收敛过程直接可见。



图示： 3.3V 、 50Ω 走线、源阻 10Ω 、末端开路时的末端电压逐拍演算。入射 2.75V 在 $t = T$ 到达末端并全反射成 5.5V ，回波在源端被负反射后逐拍抵消，每往返一圈幅度乘 $2/3$ ，最终收敛到稳态 3.3V 。

测量上反过来用这套表：在末端量出振铃周期 $4T$ ，就能反推走线的单向延迟与等效长度——这是没有 TDR 时的土法测长。若末端改短接，同一工具给出源端波形：先是 2.75V 的台阶， $2T$ 后回落，逐拍趋向 0 ；稳态就是源阻对地短路， 10Ω 源阻下电流 330mA ——这也是拿短接输出脚来“试波形”为什么危险的原因。

端接的三种做法 端接的总原则只有一句：要么让反射不发生，要么让它只发生一次并被吸收。前者在末端做匹配，令 $\Gamma_L = 0$ ；后者在源端做匹配，令 $\Gamma_S = 0$ ，故意放一次全反射过去再把它接住。三条标准路线对应不同的拓扑与功耗预算。

源端串联端接：在驱动器出口串一只电阻，使“驱动器内阻加串联电阻”等于 Z_0 ——内阻 15Ω 就串 35Ω （E24 标准件取 36Ω ，合计约 51Ω ）。入射波因此只有半幅，沿线走到开路末端全反射翻倍，恰好一次到达满幅；回波回到源端时被匹配吸收， $\Gamma_S \approx 0$ ，故事就此结束，稳态功耗为零。代价藏在波形里：线的中途各点看到的是“先半幅、回波经过再到满幅”的两段台阶，只有末端那一点的边沿干净。所以它只适合点对点、负载集中在末端——时钟线、片选线的首选；负载沿线分布时，中途的接收器会在阈值附近看到一个停顿平台，这不能接受。顺带一记： 74HC 输出级导通电阻典型几十欧，本身就近似一只天然的串联端接电阻，面包板上 74HC 的振铃常比理论值温和——这是运气，不是可以依赖的设计。

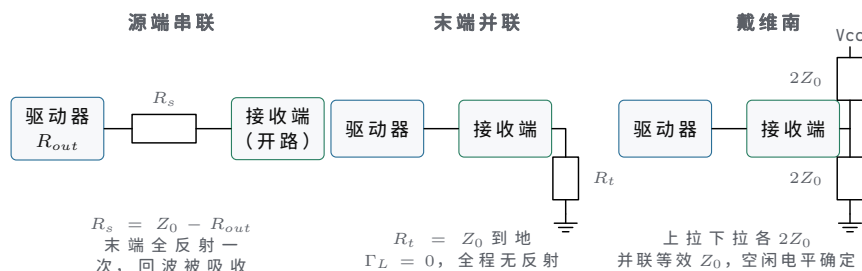
末端并联端接：在线的最远端接一只 $R_L = Z_0$ 的电阻到地， $\Gamma_L = 0$ ，入射波一次到达即满幅，全程无反射，沿线每一点看到同一个干净边沿——多负载沿线分布的总线只能走这条路。代价是功耗： 3.3V 驱动 50Ω ，高电平期间每线恒流 66mA ，八位总线全高就是半安培级，驱动器与电源都要按这个电流设计。这是拿功耗换干净的典型交易。

戴维南端接用两只电阻：上拉、下拉各取 $2Z_0$ （ 50Ω 线各用 100Ω ），并联等效恰好 50Ω ，中点电压 1.65V 顺手给总线一个确定的空闲电平——需要空闲态有定义的总线用它，静态功耗比单电阻小些但仍可观。AC 端接把端接电阻串一只 $100\text{--}200\text{pF}$ 电容再到地：直流被隔断，静态功耗近零；边沿到来的几纳秒内电容近似短路，电路呈现 Z_0 。代价是每次翻转要给这只电容充放电，且效果对边沿快慢略有依赖——它适合连续翻转的时钟，不适合偶尔跳一下的控制线。

端接方式	接法	反射行为	静态功耗	适用场合
源端串联	出口串 $R = Z_0 - Z_S$	末端全反射一次后被吸收	零	点对点，负载在末端
末端并联	末端 $R = Z_0$ 到地	无反射	每线数十 mA	多负载总线末端
戴维南	上拉下拉各 $2Z_0$	无反射，空闲电平确定	中等	需定义空闲态的总线

AC 端接	Z_0 串 100-200pF 到地	边沿期间近似无反射	近零	连续时钟， 功耗敏感
-------	-------------------------	-----------	----	---------------

三种做法画成电路对照，电阻的取值依据与等效阻抗如下。



图示：三种端接的拓扑。源端串联故意放一次全反射过去，再在源端把它吸收，适合点对点；末端并联与戴维南让反射根本不发生，适合多负载总线，代价是静态功耗。

多负载的布线拓扑与端接绑定。菊花链让一条线依次经过各负载、在末端一次端接，中途负载只引出短桩（桩本身要短于临界长度，否则桩也成传输线），是总线的现实选择；DDR 地址线的 fly-by 结构就是它的工程化版本。星形从驱动器直接分出 N 条支路，驱动器看到的阻抗变成 Z_0/N ，分叉口本身又是一次阻抗突变一支路多于两三条就很难收拾，只能每条支路各自源端串联、按点对点逐条处理。端接电阻的摆放也有纪律：源端电阻贴驱动器出口，末端电阻贴最后一个负载之后，远离位置的端接等于没接。先按负载分布选拓扑，再按拓扑选端接，顺序不能反。

过冲与回沟对输入端的压力 上面演算出的 5.5V 不是纸上数字，它真实出现在 3.3V 系统的输入脚上。每个 CMOS 输入脚都带两只钳位二极管：一只阳极在脚、阴极在 V_{cc} ，脚电压超过 V_{cc} 约 0.5-0.7V 时导通；另一只反向到地，脚电压低于约 -0.5V 时导通。它们的本职是泄放静电，不是处理日常波形。上升沿的 5.5V 过冲会打开上管；下降沿的镜像过程把脚电压冲到 -2.2V（入射 -2.75V 全反射），打开下管。

保护管导通后的电流由线阻抗限制： $(5.5 - 3.3 - 0.7)/50 \approx 30\text{mA}$ ，每个边沿一次。手册通常只给这类电流 $\pm 20\text{mA}$ 上下的定额——偶尔超一点不会立刻坏，但每个时钟沿都打它一次，是按年计的可靠性消耗。更大的过冲还有急性病：触发芯片内部寄生的晶闸管结构（闩锁，latch-up）， V_{cc} 与地之间形成低阻通路，芯片数秒内发烫，只有彻底断电才能解除，反复几次即永久损伤。闩锁是过冲问题里最不能讨价还价的一种。

回沟的危害在时钟线上最尖锐。数据线上的回沟发生在建立时间之前很久，采样时早已平息，往往无害；时钟线不同——接收触发器只认阈值穿越。上面 10Ω 源阻的例子，回沟停在 1.83V，离 1.65V 的典型阈值只剩约 0.2V 裕量；源阻若只有 5Ω （强驱动）， $\Gamma_s \approx -0.82$ ，回沟探到约 1.1V，直接跌穿阈值，随后 $t = 5T$ 的回升再向上穿越一次——一次边沿被数成先下后上两次触发，状态机多走一拍。这类故障表现为“计数偶尔加二”“程序莫名跳步”，离病根极远，排查时极易误判为软件问题。

处置按三条思路排：消灭反射（源端串联或末端端接，反射不存在，回沟无从谈起）；把线变短（放慢边沿或缩短走线，回到短线判据之内——1MHz 的总线没有理由用 2ns 边沿的器件）；吸收残余（不计速度的信号如复位线，可在输入端前加 RC：串联 $1\text{k}\Omega$ 、对地 100nF 量级，把毛刺整体滤掉）。

手段	作用机理	代价	适用场合
源端串联	末端一次到满幅，无回沟	中途点见台阶	点对点时钟、片选
末端并联、AC	反射根本不发生	功耗或元件	总线、多负载
放慢边沿	走线相对边沿重新变短	速度上限下降	低速总线首选

缩短走线 RC 吸收	延迟降到临界值以下 毛刺被积分滤除	布局受限 边沿变缓	一切场合 复位、低速控制线
---------------	----------------------	--------------	------------------

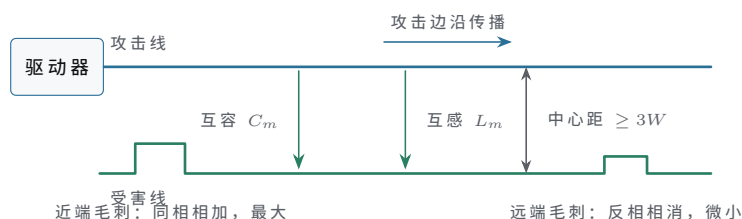
测量时记住一件事：传输线上不同位置的波形不同，驱动脚看到的是入射波与回波的中途叠加，接收脚看到的才是判决依据——探头必须点在接收脚上。判读两条红线：过冲顶点不许碰 $V_{cc}+0.5V$ ，回沟谷底不许碰阈值区。两条都碰了还不端接，剩下的就不是会不会坏的问题，而是什么时候坏的问题。

四、串扰、回流与电磁兼容入门

§ 三管的是一根线上的波形；本节管线线之间、芯片内外、板子内外的相互作用。串扰、回流绕路、同时开关噪声、差分与辐射，看似五件事，物理上只有一件：变化的电场与磁场不认走线这个边界。本节按作用范围由近及远排开，每条规则仍然先给物理图像，再给数量级，最后给排查方法。

串扰来自平行长走线 两条平行走线共享同一段介质，攻击线上的变化有两条路渗进受害线。容性耦合走电场：攻击线电压变化，经线间互容向受害线注入电流 $I = C_m dV/dt$ ，注入电流在受害线上向两端各分一半。感性耦合走磁场：攻击线电流变化，互感在受害线上感应电压，驱动起一个与攻击电流反方向的环流。两种耦合有方向性：朝驱动器方向（近端）去的两半同相相加，朝末端方向（远端）去的两半反相相消——近端串扰因此通常最大，带状线里远端甚至接近抵消干净；微带线因两种耦合的传播速度略有差异，远端会留一点残余。量串扰，探头先放在受害线的近端。

把两条平行走线的耦合路径画出来，近端与远端毛刺的方向性就清楚了。



图示：平行走线间的两条耦合路径：互容注入电流、互感感应环流。朝近端去的两半同相相加，近端毛刺最大；朝远端去的两半反相相消，远端只剩残余。中心距拉开到三倍线宽（3W 规则），可把耦合压到两成上下。

耦合强度随间距下降的速度比直觉快。走线下方有参考面时，耦合系数近似按 $1/(1+(s/h)^2)$ 衰减， s 为两线净间距， h 为走线到参考面的介质厚度。取常见情形 h 约等于线宽 W ：净距为零时耦合记作 100%，净距一个线宽约 50%，净距两个线宽约 20%。3W 规则——相邻走线中心距不小于三倍线宽，即净距不小于两倍线宽——正是把串扰按到两成上下的那条经验线。它不花一个元件，只花布线面积；继续拉到 5W、10W 收效递减，不如把省下的面积留给回流。注意 3W 的前提是下方有完整参考面把场约束住：没有参考面，公式里的 h 失去意义，拉开多远都不够。

平行长度也有自己的刻度。近端串扰随平行长度增长，直到平行走线的往返延迟追平边沿时间后饱和，饱和长度 $l_s = t_r \cdot v/2$ ：6ns 边沿对应约 45cm。面包板上 10cm 的并行走线远没到顶，所以教学机上缩短平行段立竿见影；反过来，边沿 0.3ns 时饱和长度只剩约 2.3cm，高速板上几厘米的并行早已到顶——“能容忍多长的并行”，同样要拿边沿时间去比。

受害线旁边最不能住的三类邻居：时钟、复位、片选（写使能同理）。时钟边沿快且永不停歇，串扰以时钟周期重复出现，最容易准时混进数据的采样窗口；复位线上一个几纳秒的毛刺就可能复位全机，而毛刺窄到普通示波器抓不住，故障现象（莫名重启）离原因极远；片选或写使能上的假脉冲意味着假读写——RAM 被改写、外设被误触发，破坏是永久性的。

攻击源	危险之处	布线对策
-----	------	------

时钟	边沿快、周期性重复，易混入采样窗	与数据线保持 3W，必要时地线隔离
复位	窄毛刺即可复位全机，事后无痕	远离快信号、走线短、输入端加 RC
片选、写使能	假选通等于假读写，数据被破坏	与数据线同等管理，注意端接

必须紧邻时，在攻击线与受害线之间布一条接地隔离线，把电场截到地，耦合明显削弱。但隔离线两端必须就近接地、长则沿途多点接地；悬空或只接一头的隔离线自己就是一根转发耦合的天线，比没有更糟。排查串扰有三个可动的旋钮：毛刺幅度随平行长度增长、随间距加大下降、随攻击边沿加快而上升——让机器跑起来，把受害线驱动成固定电平，逐个动这三个变量，符合规律即可确认。若噪声与“同拍翻转的路数”相关而非与某条邻线相关，那是后面要讲的 SSN，两者治法不同，先分清再动手。

感性耦合可以用变压器理解：攻击线是初级，受害线是次级，各一匝，磁路是两者之间共享的那部分磁通——并行越长、间距越近，共享磁通越多，互感越大；容性耦合则是一只跨接在两线之间的小电容。两只“寄生元件”的值都由几何决定，这正是 3W 规则背后那张等效电路。测量时两种耦合还能分开辨认：近端毛刺的宽度等于并行段的往返时间，远端毛刺的宽度等于攻击边沿的时间——用这两个“指纹”，在示波器上就能读出耦合走的是哪条路。

总线场景要把串扰再算重一点。8 位总线并行走线，中间那根线两侧各有一个攻击者，串扰两份相加——总线里最惨的总是中间位。最坏的激励是邻位全部同拍同向翻转、受害位保持不动：两侧的耦合电流同相叠加，等效攻击强度翻倍。评估一条总线的串扰，要按这个最坏图案算，而不是按平均翻转率算。

教学机上没有参考面， h 的约束不存在，串扰按最坏情形估：两根并排 10cm 的杜邦线，互容约 5pF 量级。攻击线 5V/6ns 的边沿注入电流 $I = C_m dV/dt \approx 5\text{pF} \times 5\text{V}/6\text{ns} \approx 4\text{mA}$ ，受害线上两端各分一半：受害线被 50Ω 驱动时，2mA 产生的毛刺约 0.1V，尚在噪声容限之内；受害线若是悬空的输入脚，25pC 的电荷注入几 pF 的节点电容，毛刺就是伏特级，足以误触发。这就是面包板上“输入脚不许悬空”的另一半理由：上拉、下拉电阻不仅给逻辑一个默认值，也是给串扰电流一条低阻旁路。

排线与带状电缆则是把回流影子做成实物：工程排线把地线按“地—信—信—地”或更密的“地—信—地”间隔排布，让每根信号旁边都有自己的回流通道，环路面积与串扰同步受控。一根 26 芯排线里插 5~6 根地线，牺牲的是几根信号位，换来的是整束线的可靠；全插信号的排线在高速下几乎一定出错。面包板上把时钟线两侧各插一根地线，是同一手段的零成本版本。

回流路径决定环路面积 “电流从地流回去”这句话在高速下要改写。直流与低频时，回流在整个地平面上铺开，走电阻最小的路径；高频时回流走电感最小的路径——恰好是信号走线正下方参考面上的那条窄带，像信号投在参考面上的影子。物理原因很朴素：贴着信号流，环路面积最小、环的自感最小，电流总是拣能量最省的路。推论很重要：高频下的“地”不是一整片等位体，每条信号线脚下都拖着自己专属的回流影子；影子走不通的地方，信号一定出问题。

回流被迫绕路有三个经典场景。其一，换层：信号打过孔换到另一面，若上下参考面之间没有就近的地过孔或层间通路，回流要绕到远处才能跟上一换层的信号孔旁边配一只“伴随地孔”，就是为回流修的便桥。其二，跨分割：参考面被开槽、分区，信号线跨过槽缝，回流被迫绕到槽的端头，环路面积从“线到面的距离”量级暴涨到“槽长”量级——地平面分割的代价不在地本身，在于逼着回流画大圈。其三，连接器与排针：几十根信号的回流若只能挤过一两根公用地针，公共段就成了所有环路的共享部分，一根线上的变化经共享段电感耦合进所有线。高速连接器按信号与地 2:1 到 4:1 插地针，面包板上把地线分散插满排，都是同一个道理。

环路面积是串扰、辐射、敏感度三者共同的根源：一个环的电感大体随面积增长（1cm 见方的小环约 10nH 量级），这个电感既决定它向外辐射多少，也决定它从外

界场里捡起多少、与相邻环互感多少。给共享回流算一笔数：公共段电感 5nH ，8 路信号各 20mA 、边沿 5ns 同时经过， $V = L \text{d}I/\text{d}t = 5\text{nH} \times 160\text{mA}/5\text{ns} = 0.16\text{V}$ ——所有经过这段的信号，低电平被整体垫高 0.16V 。环路面积省下的每一平方毫米，都是三份收益。

查回流路径不需要仪器，只需要一个习惯：对每根关键信号问一句“它的回流从哪条路回来”，再顺着答案走一遍。答不上来的、要绕远的、要跟别人挤的，就是隐患。跨分割、换层无伴随地孔、连接器地针不足——带着这三个答案去审一块板，能抓住大半信号完整性隐患，成本只是几分钟的追问。

同时开关噪声 SSN 前面讲的都是线对线的骚扰；同一片芯片内部的输出也会联手骚扰自己。8 位或 16 位总线同拍翻转时，每路输出都要给各自的负载电容充放电，瞬态电流全部经过同一对（或同一小段）电源脚与地脚。封装引脚、键合丝、板上过孔串起来有几 nH ，电流突变在这点电感上产生 $V = L \text{d}I/\text{d}t$ ：放电电流涌向地脚，芯片内部的“地”被垫高（地弹，ground bounce）；充电电流从电源脚灌入，内部 V_{CC} 被拉低。芯片内外看到的电源在边沿瞬间塌了一块，塌多少与同时翻转的路数成正比。

代入数字。8 路输出、每路负载 50pF 、摆幅 5V 、边沿 5ns ：每路瞬态电流 $I = C \text{d}V/\text{d}t = 50\text{pF} \times 5\text{V}/5\text{ns} = 50\text{mA}$ ，8 路合计 400mA ，在 5ns 内全部进出同一个地脚。地脚回路电感取 5nH （一只引脚加过孔的量级），地弹幅度 $V = 5\text{nH} \times 400\text{mA}/5\text{ns} = 0.4\text{V}$ ；16 路就是 0.8V 。后果有两层：这一拍没翻转的输出，其低电平被垫高 $0.4\text{--}0.8\text{V}$ ——74HC 在 5V 下低电平噪声容限约 1.2V ，被吃掉大半；更隐蔽的是芯片自己的时钟脚、复位脚若正经过阈值附近，一次总线大翻转就能造出一个假沿。SSN 的典型症状是“数据总线大动作时，不相干的地方出错”。早期双列直插封装把电源脚放在对角、引线又长，SSN 尤其严重；现代封装把电源与地脚成对放在芯片近旁、多脚并联，正是被这道题逼出来的形状。

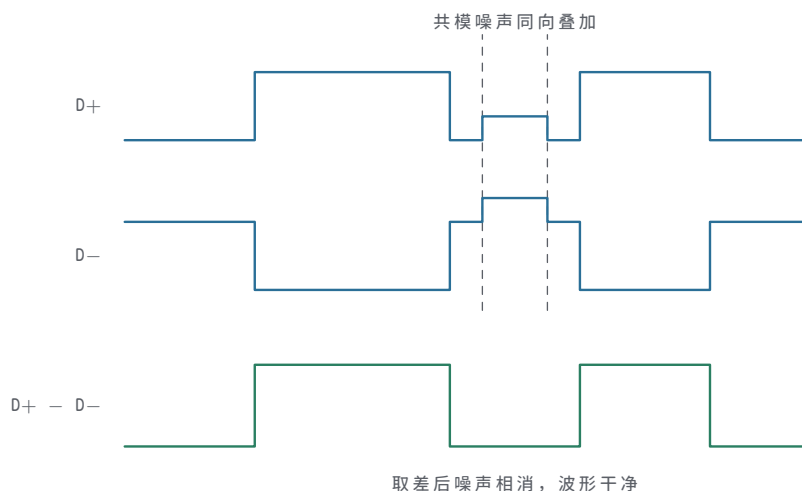
缓解 SSN 的每条手段都对应公式里的一个因子：

手段	改变的因子	效果量级
错开翻转	同时翻转路数 N	8 拆成两拍各 4，噪声减半
增加电源、地脚	回路电感 L （并联）	脚位加倍， L 近似减半
减小负载电容	每路电流 $C \text{d}V/\text{d}t$	C 减半则电流减半
放慢边沿、调低驱动	$\text{d}t$	边沿加倍，噪声减半
就近去耦	电流路径长度	瞬态电流不经过引脚电感

执行这张表在各层各有对应动作。芯片厂在现代 FPGA 的高速 Bank 旁按厂家给定的信号地比配脚、片上集成大量去耦；板级设计者把总线收发器分组使能、把同拍翻转的路数拆开，电源入口去耦按 § 一的分层做足；写状态机的人也有份：让同拍变化的输出位数最小的编码，是免费的 SSN 抑制。测量用短接地针的探头（长地线会引入假信号，见 § 五），直接量芯片一只“静止”输出的低电平：总线翻转那一拍它被垫起多少，就是地弹。鉴别 SSN 与普通串扰靠翻转路数可变：同一组线上 1 路翻转与 8 路翻转各跑一次，噪声幅度与路数成正比的是 SSN，与某条特定邻线的有无强相关的才是串扰。先分清病因，再选上面表里的手段。

差分信号为什么抗干扰 差分把一根信号线换成紧挨着的一对：驱动器输出等大反相的两半各走一条，接收端只放大两者之差。设两条线各收到同样的干扰 n ——远处来的场、参考面的晃动、电源噪声——单端信号里 n 原样叠加在信号上；差分里接收端算 $(s+n) - (-s+n) = 2s$ ，干扰在减法里相消，这就是共模抑制。相消的前提是“同向等量”：干扰源离两条线越等距、两条线经历越一致，残留越少——不对称只有 1% 量级时，残留就是干扰的 1%，即 40dB 的抑制。双绞线把两条线拧在一起、板上把对内两根紧耦合等长布线，都是在维护这个前提。

把共模噪声的叠加与取差相消画成波形，这个过程一眼可见。



图示：共模抑制的过程：噪声 n 同向叠加到 $D+$ 与 $D-$ 上，接收端只放大两者之差， $(s+n) - (-s+n) = 2s$ ，噪声在减法中相消。相消的前提是两条线对称——干扰到两条线的耦合越相等，残留越少。

回流路径上差分同样占便宜。单端信号的回流必须走参考面，环路面积由“线到面”的环决定；紧耦合差分对里，一条线的回流大部分由另一条线承担——去程与回程在对内部闭合，环路面积缩到对内间距量级，对外辐射和捡拾外界干扰的能力同步下降。这就是差分对参考面的依赖远低于单端的原因；不对称的残余仍以共模电流形式存在，所以参考面依然要，只是要求可以放宽。

小摆幅是第三个红利。LVDS 在 100Ω 差分端接上只摆 $\pm 350\text{mV}$ ，驱动电流约 3.5mA 且大小恒定、只换方向， dI/dt 天然温和；接收端有几十到一百 mV 的差值即可可靠判定。摆幅小、电流稳、环路面积小，合起来就是低功耗加低辐射。凡是要出板、出箱、速率上台阶的信号都选了这条路：USB 的 $D+$ 与 $D-$ 是 90Ω 差分，PCIe 每个 lane 一对 $85\text{--}100\Omega$ ，LVDS、SATA、以太网同理——不是单端完全不能用，而是到了那个速率，单端的串扰、辐射与回流问题一起长大，差分是更省的方案。

差分的全部好处押在“对称”二字上，布线纪律因此很具体：对内等长—— 1mm 的长度差就是约 7ps 的相位偏移，直接把一部分差分信号转成共模，Gbps 级链路要计较到 0.1mm 量级；对内等距、同层、过孔成对；出板之后用双绞或屏蔽对。教学机上没有真正的差分链路，但有一条可以直接搬走的直觉：让去的路和回的路贴在一起、经历相同，干扰就同来同去——单端世界里“信号贴着回流走”那条规则，与差分本是同一个物理思想。

EMC 入门：辐射来自大环路与**快边沿** 电磁兼容（EMC）问的是最后一个尺度的问题：板子作为整体，向外辐射多少、能忍受多少。一个电流环就是一圈小天线：环尺寸远小于波长时，辐射场强 $E \propto I \cdot A \cdot f^2/r$ ——正比于电流、环路面积、频率的平方，随距离反比下降。频率平方是最无情的因子：频率加倍，场强四倍、功率密度十六倍。板的辐射因此由两个旋钮控制——环路面积与频率成分；本节前面四段其实都在拧这两个旋钮，发射与敏感还遵守互易：同一个大环，既善于发射也善于接收。

数字信号的高频含量由边沿而非时钟频率决定：拐点频率 $f_k \approx 0.5/t_r$ ，高于它的谐波迅速衰减。74HC 边沿约 6ns ， $f_k \approx 83\text{MHz}$ ； 1ns 边沿对应 500MHz ； 0.1ns 对应 5GHz 。“总线只跑 1MHz ”与高频无关是错觉——边沿决定高频含量，时钟频率只决定能量在谱线上怎么排。另一个尺度是天线效率：导体长度接近四分之一波长才是有效天线， 10cm 走线对应 $\lambda/4$ 的频率约 750MHz ，所以 83MHz 的成分在 10cm 走线上辐射效率其实很低——这是运气，不是设计。

减缓手段对着两个旋钮加一个盖子。减小环路面积：完整地平面、回流贴信号走、连接器多插地针——本节前三段的内容在 EMC 语境下原样重读一遍，就是发射控制。减缓边沿：用满足速度要求的最慢器件，源端串电阻、调低驱动强度，把 f_k 从几百 MHz 拉回几十 MHz ，辐射按平方退潮；这条几乎不花钱，只花“不用用不上的速度”的自觉。屏蔽与接地是盖子：金属机壳接保护地，线缆进出机壳处加滤波或共模扼流圈，机壳孔缝远小于波长（经验线取 $\lambda/20$ ， 1GHz 对应约 1.5cm ）。屏蔽应当最后

上场：前两条做到位的板子，常常不需要它。排查辐射超标用的是近场探头：一只小磁环探头贴着板面扫，哪里场强异常哪里就有大环或快边沿——它量到的正是 $I \cdot A \cdot f^2$ 的分布图。

EMC 还有辐射之外的另一半：传导。30MHz 以下的干扰主要沿导线进——电源线、信号线、地线都是通道，治法是滤波与去耦：电源入口的 π 型滤波、线缆出口的共模扼流圈（差模电流在磁环里磁场相消、畅行无阻，共模电流磁场相加、遇到高阻），把噪声拦在设备边界上。30MHz 以上导线越来越像天线，辐射接过主导权，治法回到环路面积与边沿速度。两者分界并不绝对，但“低频沿线走、高频向空发”这张地图，足够指导排查的先后：先看线，再看环。

低速教学机为什么天然友好？把三个因子逐项代入就明白：时钟 1-10Hz、有效翻转率极低；负载电流 mA 级；74HC 虽有约 80MHz 的边沿成分，但几到十几厘米的走线相对波长太短，辐射效率很低。 I 、 A 、 f 三项里两项半在低位，面包板不做任何 EMC 措施也不会骚扰谁——它不是超越了规律，而是无意中符合了规律：电流小、环小、边沿相对线长而言慢。

辐射因子	面包板教学机	高速板
时钟与翻转率	1-10Hz，极低	数十至数百 MHz
边沿时间 t_r	5-8ns (74HC)	0.1-1ns
拐点频率 $0.5/t_r$	约 60-100MHz	0.5-5GHz
走线长度与天线效率	数厘米，远低于 $\lambda/4$	可达数十厘米
综合判断	天然低风险	必须按本节手段逐项设计

但同样的布线习惯搬到 50MHz 时钟、0.5ns 边沿的板子上，辐射会按平方规律讨回来。教学机真正可以带走的遗产是那套检查顺序：先问环多大，再问边沿多快，最后问回流从哪走——三个问题在任何速率下都成立，这正是本章把信号完整性放在工程边界来谈的用意：它不是高速板才有的新问题，而是低速下被宽限、高速下必还清的老约定。

五、测量与工程实践

前面四节把电源、时钟、信号完整性各自的机理和算法讲完了，这一节换一个身份：从设计者换成测量者。测量不是中立的旁观——仪器本身是被测电路的一部分：探头会改变电路，带宽会改写波形，地线会凭空造出振铃。本节先把两件主力仪器的能力边界算清楚，再讲电源纹波和眼图这两类专项测量，然后用三则调试案例把前四节的规则串起来，最后回答一个经常被问的问题：面包板上这些“宽松”的经验，到了高速世界还剩什么。

一个贯穿本节的观点：每次测量都是两个系统的相遇——被测电路是一个，仪器与探头是另一个。波形上的每个特征，理论上都要先回答“这是谁产生的”，才有资格回答“这说明什么”。前四节给的算法正是回答这两个问题的工具：它们既能算电路，也能算仪器；本节反复出现的“自检”“换地针复测”“改采样率复测”，本质上都是在把两个系统的贡献分离开。

示波器带宽与探头决定你能看见什么 示波器的前端是一台低通滤波器：超过带宽的频率分量被衰减，到达屏幕时波形已经变形。物理图景很清楚——方波可以分解成基波加一串奇次谐波，边沿越陡，承担边沿的谐波次数越高；示波器带宽决定了哪些谐波能活着到达屏幕。工程上用一个经验式把带宽换算成时间：仪器自身可分辨的上升时间 $t_r \approx 0.35/f_{BW}$ 。100MHz 示波器的自身上升时间约 3.5ns，200MHz 约 1.75ns，1GHz 约 0.35ns。

屏幕上显示的上升时间是信号与仪器两者的合成，按方和根相加： $t_{disp} \approx \sqrt{t_{sig}^2 + t_{scope}^2}$ 。拿 100MHz 示波器看一个 1ns 边沿： $\sqrt{1^2 + 3.5^2} \approx 3.6\text{ns}$ ——屏幕上是一条约 3.6ns 的圆角斜坡，原本可能存在的过冲和振铃（它们由更高的谐波构成）被

滤得干干净净。换句话说，你看到的不是信号，而是信号经过示波器之后剩下的部分；边沿上最快的那些细节，恰好最先被仪器扔掉。由此得一条选择规则：要“如实看”一个边沿，示波器上升时间应快于信号 3 倍以上（即带宽约 $3 \times 0.35/t_{sig}$ ），此时显示误差约 5%；只要求 10% 误差，仪器上升时间也要快于信号一半左右。

0.35 这个系数值得拆开推一遍，因为它示范了本章的方法论。把示波器前端近似成一阶 RC 低通：阶跃响应按 $1 - e^{-t/\tau}$ 爬升，从 10% 到 90% 的上升时间为 $\tau \ln 9 \approx 2.2\tau$ ；同一电路的 -3dB 带宽为 $f_{BW} = 1/(2\pi\tau)$ 。两式相乘消去 τ ： $t_r \cdot f_{BW} = 2.2/(2\pi) \approx 0.35$ ——“带宽”这个频域指标被翻译成了“上升时间”这个时域指标。真实示波器不止一个极点，系数会略偏离 0.35，但做数量级估算完全够用；遇到任何只给带宽不给时域指标的仪器，都可以先乘一遍 0.35 再说。

换一个等价的频域看法：10MHz 方波由 10MHz 基波加 30MHz、50MHz、70MHz……的奇次谐波叠加而成，边沿越陡，承担边沿细节的谐波次数越高。经验上要看到“像方波”的形状至少得保留 5 次谐波，要看清 1ns 级边沿，谐波要收到 GHz 门口。所以 100MHz 示波器看 10MHz 方波只是“够用”：显示出来的圆滑，一半是仪器给的，心里要有数。

还要注意示波器与探头的带宽是各自独立的低通，系统带宽同样按方和根合成：100MHz 示波器配 100MHz 探头，系统带宽只剩约 $100/\sqrt{2} \approx 70$ MHz。买仪器时探头标称带宽要留出一档余量，原因就在这里。

把前面的算法串成工作流程：拿到一个测量任务，先问被测信号最快的边沿是多少——查数据手册的输出跳变时间，而不是看时钟频率，决定带宽需求的是边沿；再由 t_{sig} 反推系统带宽下限，核示波器与探头合成后的系统带宽够不够；不够就别测时域形状，改测能量与频谱，或者干脆承认“这台仪器只能看趋势”。最常见的错误是拿时钟频率当带宽依据：1MHz 的时钟照样可以有 5ns 的边沿，而看住 5ns 边沿需要的带宽是 1MHz 基波的一百多倍。

比带宽更隐蔽的是探头。10x 无源探头标配的鳄鱼夹地线长约 10cm，这段导线加上探头尖端回地夹所围的环路，电感量级约 100–300nH（直导线每厘米约 10nH 上下）。探头输入电容约 10pF，与这段电感构成一个 LC 回路，谐振频率 $f = 1/(2\pi\sqrt{LC})$ ；代入 150nH 与 10pF，得约 130MHz——正好落在快边沿的谐波密集区。于是只要被测边沿足够快，探头回路自己就被激励起来，屏幕上出现一百多 MHz 的衰减振铃；这振铃不是电路的，是测量装置的。判别办法很便宜：把长地线换成套在探针尖上的短弹簧地针（环路电感降到 10nH 量级），重新测同一点——振铃若明显缩小甚至消失，之前看到的就是测量伪影；振铃若原样不动，才轮到怀疑电路本身。”同一信号、长地线与短地针各测一次，两张波形并排存下来”是每个实验室都该亲手做一次的实验，比任何说教都有说服力。

10x 探头相对 1x 探头的价值也在这里。1x 探头就是一根屏蔽线，输入电容高达 50–150pF，带宽往往只有十几 MHz，挂到 74HC 的输出脚上就是一个可观的负载，边沿被它自己拖慢。10x 探头内置 9M Ω 电阻，与示波器的 1M Ω 输入阻抗组成分压：输入阻抗提高到 10M Ω ，等效输入电容降到 10pF 上下，带宽可达数百 MHz；代价是信号被衰减 10 倍，示波器底噪相对变大，毫伏级小信号不好测。测数字边沿、时钟、串扰毛刺时一律用 10x；只有测电源纹波的精细幅度、带宽无所谓时，才考虑 1x。

用 10x 探头前有一道常被跳过的手续：补偿。探头电缆的分布电容与示波器输入电容，会和那 9M Ω 电阻形成随频率变化的分压误差，探头柄上的小可调电容就是用来把它调平的。做法是把探头接到示波器面板的 1kHz 校准方波上：波形平顶，补偿正好；前沿冲起尖角，过补偿；前沿圆钝，欠补偿。跳过这一步，幅度读数可能差出一成，快边沿的形状更是直接失真——很多人测到的“振铃”，源头其实是一颗没调好的补偿电容。

即使是补好的 10x 探头，负载效应也要先估再测。10pF 在 100MHz 下的容抗为 $1/(2\pi fC) \approx 160\Omega$ ——对一个 50 Ω 系统的节点来说，探头一搭上去，等于并联了一个同量级的负载，被测波形当场就变了；1x 探头的 100pF 在同一频率下只有约 16 Ω ，近乎短路。估算顺序应该是：先按被测点的阻抗与信号最高频率算出探头引入的负载，

再决定测不测、怎么测。测高速节点时有源探头（输入电容 1pF 上下）存在的意义正在于此；本书的面板信号源阻抗高、边沿慢，10x 无源探头已经足够宽绰。

还有一个安全层面的常识：示波器的地夹接机壳保护地。测量不以机壳地为参考的电路（开关电源原边、市电相关节点）时，地夹一夹上去就是短路——轻则跳闸烧探头，重则伤人。有人为此剪断示波器电源线的地脚让整机“浮地”，这是把危险从电路搬到机壳上，坚决不可取；正确做法是差分探头、隔离变压器，或者用两通道分别测两点再做数学相减。本书全部实验都是低压直流共地系统，不会遇到这个问题，但第一天就该知道这条线划在哪。

带宽与探头之外，显示模式也在做选择。单次触发把一次瞬态定格，适合抓跌落与毛刺；长余辉把成千上万次扫描叠在一起，适合看抖动与偶发异常；平均模式把多次触发的波形逐点平均，随机噪声被压下去、周期性细节浮出来，但它对触发之间会漂移的事件同样无情——平均十次，偶发毛刺就永远消失了。选模式之前先想清楚要看的是“典型”还是“最坏”：两者都重要，但没有任何一个模式同时擅长。

探头方式	输入电容 / 带宽 量级	典型用途	常见误用
1x 探头	50-150pF / 十几 MHz	毫伏级小信号、低 频模拟	拿它看快边沿，负载电 容把边沿拖慢还怪电路
10x + 长地 线	约 10pF / 数百 MHz	日常数字信号粗 看	长地线环路自己振铃， 把伪影当电路振铃
10x + 弹簧 地针	约 10pF，环路电 感最小	边沿、振铃、纹波 的定量测量	地针没顶在最近的地点 上，环路仍然偏大

逻辑分析仪与时序分辨率 逻辑分析仪与示波器是两种哲学：示波器把少量通道的电压波形尽量测准，逻辑分析仪把几十根线在每个采样点上只判成 0 或 1，丢掉全部模拟细节，换来通道数和存储深度。代价写在采样率里：100MSa/s 意味着 10ns 才采一个点，比 10ns 窄的毛刺可能从两个采样点之间溜走，或者只在某一个采样点上闪现成一根孤立的细线，真假难辨。不少机型带毛刺检测（把采样点之间的跳变也记入），但可捕获的最窄脉宽仍由专门的毛刺时钟决定；读规格书时找“最小可捕获脉宽”这一项，它比宣传的最大采样率更接近真相。

采样还带来一个与示波器共有的陷阱：混叠。采样定理要求采样率高于信号最高变化频率的两倍，否则高频成分被折叠成低频假象——用 50MSa/s 去采一个 48MHz 的时钟，屏幕上会显示一个 2MHz 的“信号”，而电路里根本不存在这样的东西。判别办法是改采样率再测一次：真信号的频率不随采样率动，混叠产物会跟着跑。所以定时模式的采样率一般取被测最快信号的 4-10 倍，既躲混叠，也留足看清先后关系的余量。

逻辑分析仪的另一半价值在触发与存储。触发条件可以写成码型（“地址为 0xF0 且写使能有效”）、序列（“先看到 A 再看到 B”）或毛刺；仪器在环形缓冲里不停地采，条件命中时把命中点前后各一段定格——偶发故障因此第一次变得可以反复研究。存储深度与采样率互相抢资源：观察窗口 = 深度 ÷ 采样率，1M 点深度配 100MSa/s 只有 10ms 窗口；想看更长的时间跨度就得降采样率，这正是“看得清”与“看得久”的经典交换，买机器和设参数时是同一笔账。新机型的协议解码（UART、I²C、SPI 等）把逐拍的 0/1 直接翻译成帧，调串行总线效率倍增；但要记住解码建立在采样判决之上，模拟层的问题——缓慢边沿、擦着阈值的毛刺——会被它悄悄抹平，协议层怎么也说不通时，仍要回到示波器看模拟波形。

接线也有纪律：每路信号的参考地要就近接，探头飞线越短越好——逻辑分析仪探头同样受本章全部寄生参数支配，几十根飞线捆成一束，本身就是一个串扰丛生的结构。阈值电平要按被测逻辑家族设定：5V CMOS 与 3.3V LVTTTL 的判决点不同，设错了，0 和 1 的边界本身就是错的，后面全部结论跟着作废。

举个实战例子把触发写具体：怀疑第七章的总线在某拍被两个源同时驱动，就设码型触发——“时钟上升沿且 C0=1 且 R0=1”，命中一次就抓到一次争用的瞬间；再

设序列触发”LDA 的 T3 之后紧跟 STA 的 T2”，可以核对指令切换的边界拍。逻辑分析仪与示波器还能联合作战：很多机型有触发输出端子，条件命中时吐出一个脉冲，把它接进示波器的外部触发，示波器就在”逻辑分析仪认定的那个时刻”展开模拟波形——时序定位与波形细节同时对齐，偶发故障的最后一层隐身术就此解除。

由此有分工：查多信号之间的相对时序——8 位总线、十几根控制线、地址有效到数据有效的先后——用逻辑分析仪；查单根线上的波形细节——过冲多高、振铃什么频率、毛刺多宽——用示波器。逻辑分析仪还有一种与被测电路对齐的用法：状态模式下用电路自己的时钟作采样时钟，每个时钟沿记录一次全部通道，得到一张逐拍的状态表，与第七章的微码表逐行对照最方便。这等于把”仪器视图”对齐到”设计视图”：设计是按时钟拍的，状态模式看到的就是时钟拍的。

教学实验里怎么选：调第七章的微码序列、核对总线争用、追一条控制线哪一拍被意外置位，逻辑分析仪一次看全 8 根总线加全部控制线，是主力；查电源纹波、时钟振铃、复位毛刺，通道只要两三个但波形细节就是全部信息，用示波器。一个粗糙的判别口诀：要问”哪一拍、谁先谁后”，用逻辑分析仪；要问”多高、多快、什么形状”，用示波器。

抓到的数据别浪费：逻辑分析仪的记录可以导出成逐拍表格，与微码仿真器的 trace 逐行对拍——差异出现的第一行，就是硬件与设计的第一处分歧。这一步把调试从”盯着屏幕找不同”变成可重复的对照实验，也是第七章逐拍 trace 方法论在仪器上的延伸。

维度	示波器	逻辑分析仪
通道数量级	2-4 个	16-34 个起步
看到的東西	电压随时间的连续形状	每个采样点上的 0/1 判决
分辨率由什么定	带宽与上升时间	采样率与最小可捕获脉宽
典型任务	振铃、过冲、纹波、边沿时间	多线相对时序、协议、逐拍状态
本书实验里的用法	测纹波、查毛刺、验时钟质量	核对微码序列、抓总线争用

电源纹波怎么测 电源测量的第一个敌人是直流本身：5V 上叠着 50mV 纹波，DC 耦合下整条波形被顶在屏幕上方，纹波只占垂直方向的百分之一，什么都看不清。把通道切到 AC 耦合，信号路径里串入一个隔直电容，直流被挡住，只剩变化部分，于是可以用 10-20mV/div 的档位把纹波展开。第二个敌人是拾取：开关电源、时钟线、继电器都在周围辐射，探头地线围成的环路就是一副小天线。纹波测量要用最短的地——弹簧地针直接点在去耦电容两端，而不是夹着一根长地线去够远处的电源脚；示波器若有 20MHz 带宽限制，打开它，挡住与电源无关的高频拾取。

探测点的选择同样要先想清楚：纹波要测在负载端——目标芯片电源脚旁的去耦电容两端，那才是芯片真实看到的电源；测在稳压器输出端，读到的是供电路径的起点，中间走线的压降与跌落全被漏掉。排查时可以沿供电路径逐点测量：从稳压器输出、过板级电容、到芯片引脚，跌落深度在哪里突变，路径阻抗就藏在哪里。

测到波形之后要会分类，因为两类波形指向两类病根。稳态纹波是周期性的：线性电源之后是工频整流留下的 100Hz 锯齿，开关电源之后是开关频率（几十 kHz 到几 MHz）的三角波或尖刺，幅度基本不随负载突变。瞬态跌落是事件性的：负载阶跃（电机启动、继电器吸合、一片芯片几十个输出同时翻转）之后电压下陷再回升，持续几 μs 到几百 μs ，要用单次触发或长余辉去抓。稳态纹波大，问题多在稳压器与滤波；瞬态跌落深，问题多在去耦储能不足或供电路径阻抗大——第一节的目标阻抗算法管的就是后者。

两类波形都能再拆细、再演算。线性稳压器输出的稳态纹波，主要是整流滤波后残余的 100Hz 波动穿过稳压环路：LDO 的电源抑制比在低频有 60-70dB，但随频率升高迅速变差，到几百 kHz 往往只剩 20-30dB，输入上的高频波动会被大量放行——指望 LDO 挡住开关预稳压器的全部噪声，是不读曲线的想当然。开关电源的纹

波由两部分组成：电感电流纹波在输出电容 ESR 上产生的三角波（50mΩ 的 ESR 配 200mA 峰峰值纹波电流，就是 10mV 峰峰值），加上开关管动作瞬间的振铃尖刺——后者能量小但频谱很高，正是 20MHz 带宽限制要挡住的东西。瞬态跌落同样先估再测：100mA 的负载阶跃全靠一颗 10μF 本地电容支撑 10μs，跌落为 $\Delta V = I\Delta t/C = 0.1 \times 10/10 = 0.1V$ ；够不够、该由哪一层补，看跌落深度与持续时间的比例就明白。测到的波形因此不是一堆“噪声”，而是一条可以反推 PDN 在哪个时间尺度上缺电容的诊断曲线——这与第一节按时间尺度分层的思路首尾相接。

读数也有讲究：纹波按峰峰值读，不按有效值——决定逻辑是否误翻转的是最坏的一瞬间，不是平均能量。测量时探头方向、地的接点、带宽限制状态都要写进记录，因为换任何一项读数都会变；两次测量差出 20%，往往不是电源变了，而是测法变了。有条件的实验室还可以把示波器的 FFT 打开，或直接上频谱仪：稳态纹波在频谱上是一根根谱线，谱线位置直接指认来源——100Hz 是整流残余，开关频率及其谐波是电源本身，与时钟频率重合的那根则要怀疑耦合而不是供电。配一个近场小环探头在板子上扫一遍，辐射最强的位置往往就是回流路径最拧巴的位置，这是把第四节 EMC 机理变成定位手段的用法。

最常见的误判，是把探头环路拾到的磁场感应当成电源噪声。判别方法叫双点同地：把探头尖端与地针同时点在同一个地焊盘上，理论上应显示一条平线；若此时屏幕上仍有“纹波”，那它是环路拾取的干扰，不是电源的。先做这个自检再下结论，是电源测量的纪律；整个流程可以写成一张短清单，每次照做：

纹波测量流程

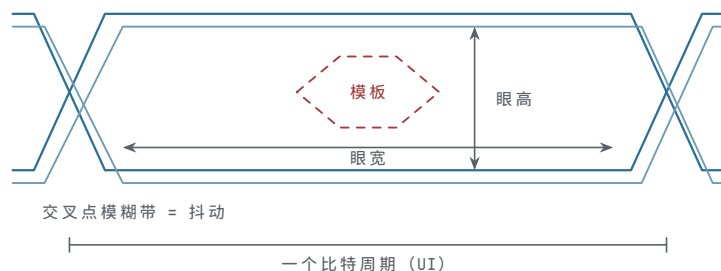
1. 10x 探头换弹簧地针，点在目标芯片的去耦电容两端
2. AC 耦合，10-20mV/div，打开 20MHz 带宽限制
3. 双点同地自检：尖与地针同点一处，平线才算环境干净
4. 看稳态纹波：频率、峰峰值，与开关频率对照
5. 单次触发抓瞬态：用负载动作信号做外触发
6. 记录：波形 + 探头接法，缺一不可

眼图：把时序裕量画成一只眼睛 让示波器用远高于比特率的采样率连续采集一段数据波形，再按一个比特周期（UI，单位间隔）把波形切成段、全部叠到同一张图上：所有 0→1、1→0、0→0、1→1 的轨迹交织在一起，中央留出一块空白，整张图形像一只睁开的眼睛，这就是眼图。眼睛的张开程度把两个抽象裕量变成一眼可见的形状：垂直方向张开多高，是判决时刻的噪声裕量；水平方向张开多宽，是抖动留给采样点的时序裕量；交叉点那团模糊带的宽度，就是抖动的直接读数。高速链路的标准干脆用模板（mask）说话：在眼睛中央画一个禁止进入的多边形，发射端在所有码型下都不许让波形碰它；接收端依此设计判决门限与采样时刻。

本书的面板用不上眼图：时钟周期以 μs 计，裕量以百 ns 计，眼睛永远全开，测它没有信息量。但要理解千兆位链路为什么人人谈眼图：1Gbps 的 UI 只有 1ns，5Gbps 只剩 200ps，抖动和噪声各吃掉一大块之后，眼睛只剩一条缝；还能不能可靠判决，先看眼图，再看误码率。眼图的本质是“把统计学画成一张图”：它叠的是成千上万个周期，显示的是最坏情况出现的频率，而不只是一次偶然的波形——这正是单次触发波形给不出的视角。

读眼图先认部位：中央开口的高度叫眼高，对应噪声裕量；开口宽度叫眼宽，对应时序裕量；上下两条“眼皮”的厚度，是幅度噪声与反射的堆积；交叉点的横向模糊带是抖动。规范做法是统计判决时刻处 0 电平与 1 电平各自分布的交叠程度——交叠越深，误码越近。抖动本身也要分两类看：确定性抖动有界（来自码型相关、串扰、占空比失真），随机抖动无界、服从高斯分布（来自热噪声与时钟源的相位噪声）。把误码率对采样时刻画成曲线，就是著名的“浴缸”：中间一段误码率低到测不出，两端沿随机抖动的高斯尾部陡升；两壁间距再扣掉规范要求的置信水平，才是链路真正能用的时序裕量。眼图是浴缸曲线的直观版——前者给人看，后者给标准算。

把各部位标在一张示意眼图上，对照着读更直观。



图示：眼图的形成与读法：把数据波形按一个 UI 切段叠加，所有跳变轨迹交织出中央开口。眼高对应判决时刻的噪声裕量，眼宽对应采样点的时序裕量，交叉点的横向模糊带是抖动的直接读数；红色六边形是规范模板，任何轨迹都不得进入。

眼图部位	对应物理量	典型恶化来源
眼高（开口高度）	判决时刻的噪声裕量	反射、串扰、信道衰减、电源噪声
眼宽（开口宽度）	采样点的时序裕量	抖动、码间干扰、偏斜
眼皮厚度	幅度噪声与多重反射的堆积	端接不良、电源波动
交叉点模糊带	抖动总量的直接读数	时钟源相位噪声、串扰、码型相关
交叉比（交叉点电平）	判决门限的对称程度	上下拉驱动不对称、阈值偏移

主流高速标准都把模板测试写成明文：USB、PCIe、DDR 各自规定一张禁止进入的模板和配套测试夹具，发射端在最坏码型、最坏温度下都不许让波形碰模板边界。模板的意义，是把“链路能不能工作”这个系统问题，拆成发射端、信道、接收端三方可以各自独立验证的指标——这是工程上处理复杂系统的标准手法，与软件里的接口规格同源。

面包板上其实可以做一个概念演示：用函数发生器出伪随机码流，串一级 RC 把带宽故意压窄，示波器用码流的同步时钟触发并开长余辉——眼睛会肉眼可见地闭上，RC 越大闭得越紧。这个演示不能量化任何裕量，但能让你亲手把“带宽不足 → 眼图闭合 → 判决变难”这条因果链走一遍；将来在高速链路的规格书里再遇到眼图模板，看到的就不再是黑话，而是当年那只被 RC 慢慢捏合的眼睛。

把眼图与本章其他内容连起来看：眼高被串扰与电源噪声啃——那是第四、第一节的事；眼宽被抖动与偏斜啃——那是第二节的事；眼皮的形状由反射决定——那是第三节的事。一张眼图，就是全章算法的一次联合阅卷。

调试案例三则 以下三则案例分别对应第一、二、四节的机理，按“症状、定位、根因、修复”各讲一遍；排查手段都不超出本节讲过的仪器纪律。

案例一：大电流外设动作时 MCU 随机复位。 **症状**系统平时稳定，每当继电器吸合（或电机启动、电磁阀动作）的瞬间，MCU 有相当概率复位；外设不动作时连跑几天不出错。**定位**示波器 AC 耦合、弹簧地针，探在 MCU 电源脚旁的去耦电容两端，用外设动作信号作外触发：在继电器吸合时刻抓到一次几十 μs 的电压深坑，谷底跌破复位芯片的阈值；坑深与吸合电流大小正相关。**根因**继电器线圈几百 mA 的吸合电流与 MCU 共用一段细长电源走线，深坑的主因是走线电阻与储能不足：吸合电流流过走线电阻直接产生压降，板级储能电容离外设又远，来不及补给。走线寄生电感的 $L di/dt$ 项贡献其实有限——电感到几百 nH、电流在 μs 内变化几百 mA 时，约 0.1V 量级；伏级的感性反冲只在触点断开瞬间的回路里才会出现。用第一节的语言说：在这次瞬态的时间尺度上，电源分配网络没有就近的电荷库。**修复**外设电源入口处就近并 100–470 μF 电解电容作本地储能；外设与 MCU 的供电从电源出口起星形分开走线，不再共用细线段；MCU 一侧维持原有 100nF 加 10 μF 的本地去耦。改后同一触发下重测，深坑应缩到复位阈值之上一个安全距离。

案例二：同一片上两寄存器偶尔传错数据。 **症状**某条寄存器到寄存器的传输绝大多数时候正确，偶发错一个值；温度升高或换一批芯片后出错率变化；把时钟频率

降下来就不出错——看起来像建立时间被吃掉了。**定位**逻辑分析仪状态模式逐拍记录数据通路，把出错的拍挑出来，发现错的是“新值没被采到、采到的还是旧值”；再用示波器两通道分别点发射端与接收端的时钟脚，测得两个时钟沿之间有约 0.3ns 的固定差距，方向是接收端早到。**根因**时钟树两条分支长度差约 5cm（FR4 上信号传播约 60–70ps/cm，5cm 即 0.3ns 量级），其中一支又多经过一级缓冲器，偏斜方向不利；第二节的建立时间预算按“理想零偏斜”留的裕量，实际被偏斜、抖动、温度漂移分掉，最坏组合下为负。**修复**重布时钟分支使两支等长，拉平缓冲器级数；来不及改板，就在该路径上减一级组合逻辑或换更快的器件，把数据路径延迟收回预算内。长期办法是把偏斜当作预算表的正式一项，不再假设为零；改后用高温、满速做长时间回归。

案例三：复位线上的时钟毛刺造成偶发复位。**症状**系统偶发复位，但没人碰复位按钮、看门狗也没到期；出错与程序行为无关，却在时钟负载最重的外设工作时更频繁。**定位**示波器两通道同时看时钟线与复位线：复位线上每逢某些时钟沿就出现一个窄毛刺，宽几 ns 到十几 ns，与时钟沿严格对齐、极性同向；毛刺幅度只在复位输入阈值附近徘徊，所以只有一部分毛刺触发复位——“偶发”由此而来。**根因**复位线与时钟线在板上长距离平行走线，按第四节的串扰机理，容性耦合与时钟沿的磁场变化都在邻线上感应电压；平行越长、间距越近，毛刺越大。复位输入又是高阻、无滤波的敏感端，几 ns 的毛刺也认账。**修复**能改板：复位线与时钟线拉开间距（至少 3 倍线宽量级）、缩短平行段，必要时中间隔一根接地护线。不能改板：复位输入端加 RC 滤波（如 10kΩ 加 100nF，时间常数 1ms，远大于毛刺宽度）再经施密特触发器整形，几 ns 的毛刺被积分掉。改后复测：毛刺仍在，但进不了阈值，复位不再误发。

案例	关键机理	首选定位手段	修复方向
外设动作致复位	瞬态电流在无本地储能的路径上压出深坑	AC 耦合 + 外触发，抓电压深坑	就近储能 + 星形供电
寄存器偶发传错	时钟偏斜吃掉建立时间裕量	状态模式挑错拍 + 两通道测时钟沿	等长分支 + 压缩数据路径延迟
复位线误触发	平行走线串扰出窄毛刺	两通道看毛刺与时钟沿的对齐	拉开间距 + RC 滤波整形

三则案例的共同结构值得再看一眼：症状都是“偶发”，定位都靠“把偶发变成必现”——外触发让深坑每次都被抓到，状态模式让错拍可以被挑出，双通道对齐让毛刺与时钟沿的因果关系无可抵赖。偶发故障不是不可复现，只是复现条件藏得深；仪器纪律的全部意义，就是把藏着的条件挖出来。把三则案例的共通动作抽出来，就是一张排查清单，顺序比内容更重要——先让故障变得可重复，再谈根因：

偶发故障排查顺序

1. 定义“偶发”：与什么事件相关（外设动作/温度/批次/上电时长）
2. 造触发：用相关事件做示波器外触发，或逻辑分析仪码型触发
3. 固定对照：一次只改一个变量，保留未出错时的原始记录
4. 先排测量伪影：换短地针、改采样率，确认真假再下结论
5. 定位到时间与位置：哪一拍、哪个引脚、沿什么路径耦合
6. 根因归类：电源/时钟/串扰，回到本章对应的预算表
7. 修复后回归：最坏条件（高温、满速、大负载）长时间复测

三则之外还有两类常客值得点名。一是接触不良：面包板簧片疲劳、焊点虚焊，症状是“拍一拍板子行为就变”，任何触发都抓不住它，只能靠逐区按压和补焊排查。二是静电与门锁：干燥环境里人体带的静电打入输入脚，轻的让芯片内部状态翻转，重的触发寄生可控硅结构、电源电流猛增——症状同样是“偶发”，且与操作员走近的顺序相关。这两类不归本章任何一张预算表管，但它们共享同一条方法论：先找相关性，再造复现，最后才谈机理。

最后补一条贯穿所有案例的习惯：记测量日志。每次测量写下日期、仪器设置、探头接法、触发条件与结论，波形截图编号存档——“测不到”也要记，它是排除法的一部分。调试偶发故障常常横跨几天，记忆会骗人，日志不会；而且日志强制你把每次测量的前提写清楚，很多“互相矛盾”的测量结果，一对日志就发现是接法不同。

测量日志条目示例

2026-05-11 纹波复测(案例一改板后)

设置：CH1 AC耦合 20mV/div, 20MHz带宽限制, 10x探头

接法：弹簧地针点在 MCU 电源脚旁 C7 两端(负载端)

触发：继电器吸合信号外触发，单次捕获

结果：跌落 120mV 持续 40μs, 谷底 4.88V

对照：改板前跌落 320mV, 谷底 4.68V

结论：谷底已高于复位阈值 4.6V 有余量，转入长时间回归

低速实验与高速世界的关系 本书全部实验生活在一个安全区里：面包板上走线几厘米、时钟周期以 μs 计、74HC 边沿几 ns 到十几 ns。把数字摆开：5cm 走线的传播延迟约 0.3ns，只是边沿时间的几十分之一，集总模型稳稳成立；瞬态电流以 mA 到百 mA 计，100nF 加 10μF 的两级去耦足够宽绰；探头接法随意一点，波形也大差不差。安全区里的规则是“建议”，违反了大不出丑。同一批规则到 100MHz 以上全部变成硬约束：周期 10ns、边沿 1ns 上下时，5cm 走线延迟已达边沿时间的三分之一，走线必须按传输线端接；0.3ns 偏斜占周期 3%，预算表必须逐项列出；瞬态电流以 A 计、以 ns 计，去耦要按目标阻抗设计；探头与治具不讲究，测量本身就是噪声源。但请注意：机理一条没变，变的只是数量级把寄生参数从“可忽略”推到了“必须算”。所以从低速到高速的迁移不是换一批公式，而是换一套检查的严格程度——下表把本书经验逐条对应过去，左边每条你都在面包板上见过，右边每条都是 100MHz 之后必须做的功课。

主题	本书面包板上的经验形态	100MHz+ 世界的对应约束
走线长度	几 cm 飞线，延迟可忽略	延迟与边沿可比，按传输线处理并端接
边沿速度	74HC 边沿几 ns，慢而安全	边沿 1ns 以内，谐波到 GHz，EMC 成问题
地	面包板地排栅加星形，大致等电位	完整地平面是硬前提，回流贴着信号走
去耦	每片 100nF 加每区 10μF，按惯例放	按目标阻抗分层设计，仿真与实测闭环
时序	裕量以百 ns 计，闭眼都够	预算逐项列表，偏斜与抖动各占一行
测量	长地线也测得出大概	探头即电路的一部分，治具与校准先行

表里的每一行都能还原成一组数字对照。面包板：1MHz 时钟、周期 1μs、边沿约 5-10ns、5cm 走线延迟约 0.3ns——延迟只占边沿的 3%、周期的 0.03%，寄生参数全线失守不了。100MHz 数字板：周期 10ns、边沿约 1ns，同样 5cm 走线的 0.3ns 已占边沿的三分之一、周期的 3%；谐波伸到 GHz，任何一段没端接的走线都是潜在天线，任何一块被割开的地平面都让回流绕路。同样的走线、同样的接法，变的只是时钟与边沿，结论就从“怎么都行”翻成“步步要算”——这正是信号完整性作为一门学科存在的理由。

真拿到一块高速任务，检查顺序可以沿用本书的习惯，只是每一项都要量化：先问边沿多快（决定带宽、端接与 EMC 的一切），再问走线多长（决定要不要传输线模型），接着查回流路径（地平面完整吗、跨分割了吗），然后查去耦分层（各时间尺度的电容都在位吗），最后查时序预算（偏斜与抖动逐项列出了吗）。五问下来，大

多数低速经验会原样通过；通不过的那几项，就是要按本章算法重新计算的地方。

什么时候该离开面包板、上正式 PCB？判据可以直接从本章读出：时钟频率上到 10MHz 量级、边沿要求快于几 ns、电源电流上到安培级、或者出现毫伏级模拟小信号——满足任何一条，面包板的寄生参数就从“宽绰”变成“捣乱”。离开之前，先在面包板上把好习惯养出来；下面这张清单每条都对应本章一处算法，到了 PCB 上只会更严格：

面包板阶段就要养成的测量与设计习惯

1. 探头常备弹簧地针，长地线只用于粗看
2. 任何振铃先换地针复测，再怀疑电路
3. 电源测量先做双点同地自检，再读数
4. 时序问题先写预算表，再上仪器
5. 偶发故障先造触发条件，再谈根因
6. 每次测量记日志，“测不到”也记

这套迁移眼光也是本书通往工程实践的桥：你在面包板上养成的习惯——先问物理原因、再估数量级、最后测量复核——正是高速设计每天在做的事，只是那里的估算必须更准、测量必须更讲究。仪器教会人的最后一件事是谦卑：屏幕上每个波形，都是电路与测量系统共同的作品；分清哪一部分是谁的，是硬件工程师的基本功。

章末练习

任务	要求	学习目标
LDO 功耗演算	9V 输入、5V 输出的 LDO 带 200mA 负载：求 LDO 自身功耗与效率；输入改为 12V 后两者各变多少？无散热片的 T0-220 封装热阻约 50°C/W，哪种输入下会过热	压差乘电流就是发热
去耦分层选择	某芯片输出翻转时在 10ns 内需要 50mA 瞬态电流，电源脚允许跌落 50mV：由 $\Delta Q = C\Delta V$ 求旁路电容下限，并说明它属于哪一层去耦、必须放在哪里	瞬态由最近一层供给
偏斜重算预算	时钟周期 10ns，两寄存器间数据路径最大延迟 6ns、最小延迟 1ns，建立时间 2ns、保持时间 1ns：偏斜为 0 时建立与保持裕量各多少？若接收端时钟晚到 1.5ns，哪个裕量变好、哪个被击穿	保持违例不能靠降频挽救
反射系数计算	50Ω 传输线末端分别开路、短路、接 50Ω、接 150Ω：求四种情形的反射系数，入射阶跃 1V 时反射波各是多少伏、末端电压各是多少	$\Gamma = (Z_L - Z_0)/(Z_L + Z_0)$
串扰间距规则	两条 5cm 平行走线紧挨（间距 0.6mm），时钟沿在邻线上感应出 0.8V 毛刺；按耦合随间距近似反比下降估算，把毛刺降到 0.1V 以下至少要拉开到多少？不能改间距时给出两条替代措施	间距、平行长度、护线
示波器带宽选择	要如实观察 74HC 输出约 5ns 的上升沿，要求显示的上升时间误差不超过 10%：示波器带宽至少多少？手头 100MHz 机型不够，差多少	$t_r \approx 0.35/f_{BW}$ 与方和根合成

参考解答与要点

任务	参考解答	必须掌握的要点
LDO 功耗演算	$P = (9 - 5) \text{ V} \times 0.2 \text{ A} = 0.8 \text{ W}$ ，效率 $= 5/9 \approx 56\%$ ；12V 输入时 $P = 7 \times 0.2 = 1.4 \text{ W}$ ，效率降到 $5/12 \approx 42\%$ 。 无散热片 T0-220 按 50°C/W ：0.8W 温升 约 40°C ，室温下勉强可用；1.4W 温升约 70°C ，叠加环境温度后越过结温红线，必 须加散热片或改用开关电源	LDO 效率约等于 V_{out}/V_{in} ，压差全部 变成热
去耦分层选择	$C \geq I\Delta t/\Delta V = 50 \text{ mA} \times 10 \text{ ns}/50 \text{ mV} = 10 \text{ nF}$ ， 取标准 100nF 陶瓷留一个数量级裕量。 10ns 时间尺度的电荷只能由最近一层供 给：它属于芯片旁去耦层，必须贴在电源 脚边、回路总长压到毫米级；局部 1-10 μF 与板级 10-100 μF 管更慢、更大 的瞬态，救不了这一拍	$\Delta Q = C\Delta V$ ；时间 尺度决定由哪一 层供
偏斜重算预算	偏斜为 0：建立裕量 $= 10 - 6 - 2 = 2 \text{ ns}$ ， 保持裕量 $= 1 - 1 = 0 \text{ ns}$ ，刚好贴边。接收 端晚到 1.5ns：建立裕量 $= 10 + 1.5 - 6 - 2 = 3.5 \text{ ns}$ ，变好；保持裕量 $= 1 - 1.5 - 1 = -1.5 \text{ ns}$ ，被击穿—发射端下 一拍的新数据在接收端采样沿之前就到 了：保持裕量 -1.5 ns 的含义，正是新数 据比保持要求提前 1.5ns 变化。注意保 持检查与时钟周期无关，降频救不回来， 只能改数据路径延迟或时钟偏斜	偏斜对建立与保 持的作用方向相 反；保持违例与周 期无关
反射系数计算	$\Gamma = (Z_L - Z_0)/(Z_L + Z_0)$ 。开路 $\Gamma = +1$ ：反射 $+1\text{V}$ ，末端 2V；短路 $\Gamma = -1$ ：反射 -1V ， 末端 0V；接 50Ω 时 $\Gamma = 0$ ：无反射，末 端 1V；接 150Ω 时 $\Gamma = 100/200 = +0.5$ ： 反射 $+0.5\text{V}$ ，末端 1.5V	阻抗连续处无反 射； Γ 的符号决定 回波极性
串扰间距规则	反比估算：间距需扩大 $0.8/0.1 = 8$ 倍， $0.6 \text{ mm} \times 8 \approx 4.8 \text{ mm}$ ，取 5mm 以上。替代措 施：两线之间插一根接地护线吸收大部分 耦合；缩短平行段长度（耦合与平行长度 近似正比）；还可在受害线输入端加 RC 滤波、或给时钟线源端串电阻减缓边沿	串扰随间距快速 下降、随平行长度 正比增长
示波器带宽选 择	要求 $t_{disp} = \sqrt{t^2 + t_{scope}^2} \leq 1.1 \times 5 \text{ ns}$ ，解得 $t_{scope} \leq 5\sqrt{1.21 - 1} \approx 2.3 \text{ ns}$ ，带宽 $\geq 0.35/2.3 \text{ ns} \approx 150 \text{ MHz}$ 。100MHz 机型自身 上升时间 3.5ns，显示值 $\sqrt{25 + 12.25} \approx 6.1 \text{ ns}$ ，误差约 22%，不够； 需要 150-200MHz 档位	10% 误差要求仪 器上升时间快于 信号一半左右

描述与度量

本部分导读

硬件描述语言不是编程语言，而是画电路的另一种笔。本部分先学会把 Verilog 读回成寄存器、组合逻辑和状态机，再学会用数字回答“这台机器有多快、瓶颈在哪里”。

RTL 与 Verilog 阅读入门

前面八章里有两种描述硬件的方式：门级电路图和时序波形。工程世界里还有第三种：硬件描述语言。Verilog 看起来有点像 C，但它不是程序——每一行描述的都是真实存在的触发器、组合门和连线，而不是顺序执行的指令。把 Verilog 当程序读，是最常见的入门错误；把 Verilog 读成“用文字画的电路图”，才是正确姿势。本章的目标很窄：让已经理解硬件结构的读者，能把常见 Verilog 写法翻译回自己熟悉的硬件对象。

Verilog 写法	对应的硬件	误读为程序时的危险
<code>assign /</code> <code>always_comb</code>	组合逻辑网络	以为有执行顺序
<code>always_ff @(po</code> <code>sedge clk)</code>	一组触发器	以为“每一步都执行一遍”
非阻塞赋值 <code><=</code>	触发器输入端	以为立即生效
<code>case</code> 不写满	可能推断出锁存器	以为“没写就不变”是免费的

核心理念

读 Verilog 的三问：这一行产生了什么硬件？这个信号是组合路径还是触发器？综合器在哪些写法下会推断出我不想要的东西？能回答这三问，就能读懂绝大多数工程 RTL。

一、从电路图到 RTL：两种描述同一个硬件

本节的任务只有一个：建立代码行与电路元素的逐项对应。读完本节，看到任何一个 module，都应该能立刻说出它的引脚在哪里、内部连线是哪几根、每个门对应哪一行。这是一切后续阅读的基础——后面的时序写法、testbench、综合边界，全部建立在这份对应表之上。

module 是带端口的电路框图 先给结论：`module` 不是程序里的函数或类，它是一张有名字的电路图。`module` 与 `endmodule` 之间的全部文字，描述的是这张图里有哪些元件、哪些线连到图的边缘。图的边缘——这张电路图与外界交换信号的位置——由端口表给出：`input` 是进图的引脚，只能被外界驱动；`output` 是出图的引脚，必须由图内某个元件驱动。方向声明的本质是约定驱动权：一根线在同一时刻只能有一个驱动者，这和第一章总线规则是同一条纪律。

图内部的中间节点用 `wire`（线网）声明。它对应面包板上两个芯片引脚之间那根独立的跳线：没有自己的值，值永远来自驱动它的那一端。另一类声明 `reg`（SystemVerilog 中建议写成 `logic`）表示“可以被 `always` 块驱动的信号”——这个名字是历史遗留的误导。它叫“寄存器”，但不一定是寄存器：信号是不是触发器，取决于驱动它的 `always` 块是时序块还是组合块。组合块驱动的 `logic` 综合出来依然是一团门，一个触发器都没有。第三节写时序块时会把这件事讲透，这里先记住：声明的是“驱动资格”，不是“存储行为”。

逐项对应关系可以列成一张表。这张表是本节的索引，值得记熟：

Verilog 元素	电路图上的对象	关键问题
------------	---------	------

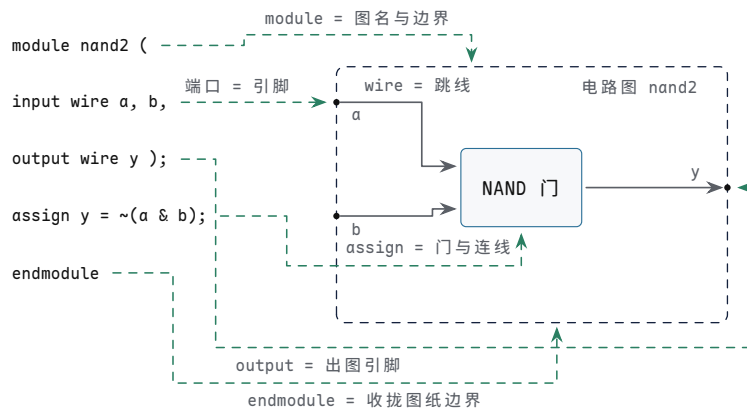
<code>module/endmodule</code>	一张电路图的图名与边界	这张图对外暴露哪些引脚
<code>input</code>	进图的引脚，只能被外界驱动	谁驱动它；断线时默认值是什么
<code>output</code>	出图的引脚，默认是线网型	图内哪个元件驱动它
<code>wire</code>	图内一根连线（或一束线）	两端各连什么；是否有且仅有一个驱动者
<code>reg/logic</code>	可被 <code>always</code> 块驱动的点	驱动它的是时序块还是组合块
<code>parameter</code>	图纸上的标注常数，不参与连接	改标注会不会改变接线方式

最小的完整例子是一个两输入 NAND，对应 74HC00 里四路门中的一路：

```
module nand2 (  
    input wire a,    // 引脚：进图  
    input wire b,    // 引脚：进图  
    output wire y     // 引脚：出图  
);  
    assign y = ~(a & b); // 一个 NAND 门，输出焊在 y 上  
endmodule
```

逐行读回硬件：第一行给出图名 `nand2`；端口表画出三根连到图边缘的线，方向标得一清二楚；块内唯一的 `assign` 放一个 NAND 门，两个输入端分别焊在 `a`、`b` 上，输出端焊在 `y` 上；`endmodule` 收拢图纸边界。五行文字就是一张完整电路图，没有一句“会被执行的代码”。

上面这段逐行解读可以直接画成对照图：左边是 `nand2` 的五行代码，右边是它描述的电路图，虚线把每行代码连到它对应的图元。



图示：五行代码与电路图元的逐项对应（虚线为对应关系，不是电路连接）：`module/endmodule` 圈出图纸边界，端口表给出进出图的三根引脚，`wire` 是引脚到门之间的跳线，`assign` 把一只 NAND 门接在 `a`、`b` 与 `y` 之间。读任何 `module` 都用这个顺序：先看引脚，再数跳线，最后对每个门对号入座。

端口与驱动的关系可以浓缩成一张速查表，读任何 `module` 时拿来对照：

对象	谁来驱动	误用的后果
<code>input</code>	图外，经端口送入	图内给它赋值：方向冲突
<code>output</code> （线网型）	图内唯一一个驱动者	两处驱动：输出对短路
<code>output reg/logic</code>	图内某一个 <code>always</code> 块	多个块驱动同一信号：多驱动错误
<code>inout</code>	双向，靠三态结构让出	让出不及时：总线争用，见第五章

内部 `wire`图内一处 `assign` 或实例
输出无驱动：悬空，电平不确
定

由此得到一个读 module 的固定顺序：先看端口表，弄清这张图与外界交换哪些信号、各进各出；再看内部声明，数出图里有几根中间跳线、几个可被块驱动的点；最后逐行看驱动关系，把每个门、每根线对号入座。端口表是图纸的图例，先读图例再看图——这个顺序到第四节读大设计时会省大量回头路。

边界上说三件事。第一，module 之间不能嵌套定义，一个 module 是一张独立的图；图与图发生关系的唯一方式是实例化，见本节末尾。第二，端口方向一旦声明就固定了驱动关系，试图在图内驱动 `input`，或在两个地方同时驱动同一根 `output`，都是连线冲突，与面包板上把两个输出脚对短路同罪。第三，SystemVerilog 的 `logic` 可以同时接受 `assign` 驱动或单个 `always` 块驱动，能替换绝大多数 `wire/reg`；本书沿用两种写法并存的工程习惯，读到时只需问“谁驱动它”，不必纠结声明用的是哪个关键字。

顺带认识一种老写法。早期的 Verilog 把端口表只写成一串名字，方向和宽度在块内另行声明：

```
module nand2_old (a, b, y); // 端口表只列名字
    input a; // 方向与宽度在块内声明
    input b;
    output y;

    assign y = ~(a & b);
endmodule
```

两种写法描述的是同一张图，硬件没有任何差别；老写法只是把“图边缘有哪些引脚”拆成两处说。新代码一律用前面那种在端口表内直接声明的 ANSI 式写法，读旧代码时认出这是同一件事的两种记法即可。

最后看 `parameter`。它不是信号，不占任何一根线，是图纸上的标注常数——像原理图空白处写的“本板按 8 位总线装配”。标注参与描述，改动标注就换了一张不同尺寸的图纸：

```
module and_n #(
    parameter WIDTH = 8 // 图纸标注：默认 8 位宽
) (
    input wire [WIDTH-1:0] a,
    input wire [WIDTH-1:0] b,
    output wire [WIDTH-1:0] y
);
    assign y = a & b; // WIDTH 个并排的 AND 门
endmodule
```

`WIDTH` 决定了这张图里画几个门、每束线有几根；实例化时可以用 `#(.WIDTH(16))` 改标注，得到一张 16 位的图。参数化让一张图纸服务多种规格，省的是画图遍数——每一组参数值仍然综合出一份独立的硬件，这一点与实例化的边界一致。

assign 是连线，不是赋值 先讲机制。`assign y = ~(a & b);` 在程序语言里读作“把某个值算出来存进 y”，在 RTL 里这句话的含义完全不同：综合器看到它，就把右边表达式对应的组合门网络画出来，把网络的输出端直接焊到左边那根线上。焊完之后没有“执行”这回事——`a` 或 `b` 一变化，经过门的传播延迟，`y` 跟着变。所以它的正式名字叫连续赋值（continuous assignment）：“连续”指的是连接不间断，不是“不停地计算”。

连续赋值有一个程序语言里没有的性质：顺序无关。看下面三行：

```
assign carry = a & b;
assign sum = a ^ b;
```



```
assign overflow = sum & carry;
```

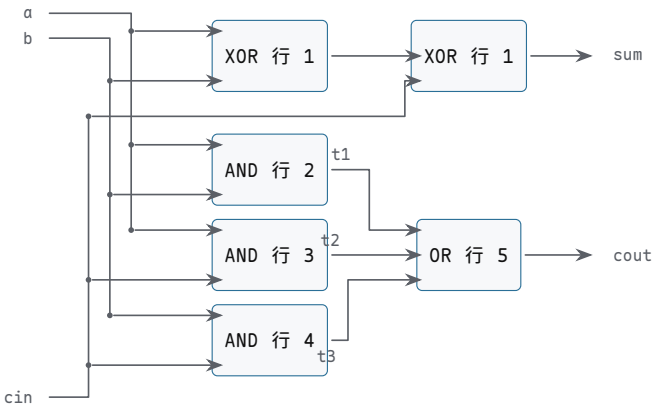
三行任意调换次序，综合出来的电路一模一样。这不是特例，而是连续赋值的普遍性质：每一行各自描述一条独立的焊接关系，焊接的先后不影响焊完的板子。仿真器也维持这个性质——任何源信号变化都会触发与之相关的 assign 重新求值（事件驱动），而不是按行号轮流执行。所以读 RTL 时，遇到一组 assign 的正确读法是“这一片有哪些门、各自连到哪”，而不是“先算什么、后算什么”。

这是 Verilog 与程序的第一处分歧，也是把 Verilog 当程序读的读者最先摔倒的地方。程序里“后面的语句看到前面语句的结果”；RTL 里所有 assign 同时成立，像一张图上所有门同时通电。第一章的安全启动控制板通电瞬间，所有门一起开始工作，没有哪个门“先运行”——assign 描述的正是这种并行存在。

再看一个略大的网络，把第三章的完整加法器写成门级 assign：

```
// 一位全加器：五行 assign 对应五个门，书写次序任意
assign sum = a ^ b ^ cin;
assign t1 = a & b;
assign t2 = a & cin;
assign t3 = b & cin;
assign cout = t1 | t2 | t3;
```

把这五行画成门级电路，每一行对应一到两只门，t1-t3 三根中间跳线也有了实体：



图示：一位全加器的门级电路，行号标在门内：行 1 的两只 XOR 产生 sum，行 2-4 的三只 AND 与行 5 的 OR 产生 cout。左侧竖线是输入分发干线，圆点为连接点，十字交叉处不相连。五行任意调换书写次序，综合出的仍是这张图。

t1、t2、t3 是图内三根中间跳线：不给名字、把 cout 写成一个表达式，综合器画出的网络相同；命名的价值在于可读性和调试测点——这与第一章给 safe_chain_n 这类中间节点起名的理由完全一样。把 cout 那行挪到最前面，板子不变；网络上每一点的电平只由此刻的 a、b、cin 决定。两种读法的分野可以归纳成一张表：

程序式误读	硬件正读
“先算 sum，再算 t1……”	五个门同时通电，没有先后
“t1 里存着一次运算的结果”	t1 是一根线，电平随输入即时变化
“调换两行，结果会不一样”	焊接次序不影响焊完的板子
“cout 用到 t1，所以它后执行”	cout 的门接在 t1 线上，只有传播延迟

常见误区：“assign 把结果存下来了，后面直接用。”没有任何东西被存下来。assign 的左边是一根线，线上此刻的电平就是右边门网络此刻的输出。想”

存”，就得画出存储元件——第二章的锁存器、触发器——那对应 `always` 块和时钟，是后面两节的内容。

边界上只有一条：`assign` 只能驱动线网型信号，且每根线网只能有一处 `assign` 驱动（多驱动是另一回事，需要三态总线那种特殊结构，见第五章）。想在一个地方根据条件写多条“赋值”，用 `always_comb`，那是第二节的话题。

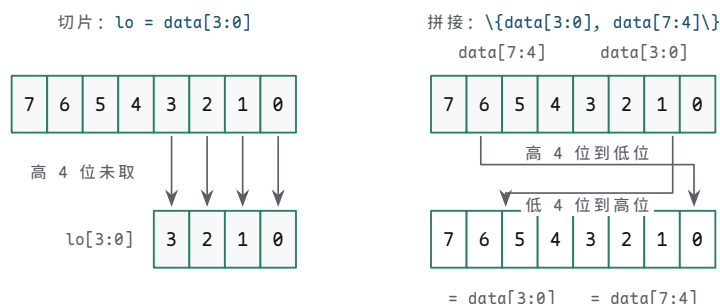
位宽、拼接与切片 机制层面，每根信号都有固定宽度，对应一束并排的导线。`wire [7:0] data;` 声明八根线，编号从 7 到 0，最高位写在左边——与第三章的 `bit` 编号约定一致，`data[7]` 是最高位。切片 `data[3:0]` 是从线束里抽出低四根；拼接用一对花括号把几束线并成一束更宽的；常数写成 `8'hFF` 这种“位宽-进制-值”三段式，位宽写在最前面。

```
wire [7:0] data;
wire [3:0] lo;
wire [7:0] swapped;

assign lo      = data[3:0];           // 抽出低 4 根线
assign swapped = {data[3:0], data[7:4]}; // 高低半字节换位
```

拼接和切片不消耗任何门——它们只是排线方式。这一点必须和运算分清：`swapped` 在硅片上的成本是零个门、八根重新排列的连线，而 `data + 1` 的成本是一整条加法器进位链。读到拼接时，脑子里应该出现“几束线换了排法”，而不是“发生了什么计算”。花括号里写在前面的信号占高位，就像把几扎电线从左到右排好：`data[7:4]` 写在后面，所以换到低位。次序写反，线束就接反——和第七章总线位序接反是同一类事故，不冒烟、不发热，只表现为数值诡异。

把这两行画成位图就更清楚了：切片是抽线，拼接是重排，两处都没有门的影子。



图示：左：切片 `data[3:0]` 从八根线里抽出低四根，高四位不参与。右：拼接 `\{data[3:0], data[7:4]\}` 把低半字节换到高位、高半字节换到低位，`swapped` 的每一位仍来自同一束线。两种操作都只是重新排线——图上没有一只门，硬件成本为零。

位宽不匹配时，Verilog 的处理规则是机械的：表达式先按参与运算的最大位宽对齐，短的一侧高位补 0；结果赋给左侧时，左侧更宽就继续补 0，左侧更窄就直接丢弃高位。这套规则没有任何“智能”，只有约定。它导致的经典事故有两种：两个 8 位数相加，和需要 9 位，左边只给 8 位，进位被静默丢弃；1 位标志与 8 位数据比较，前者先被补成 8 位再比，比较对象与直觉不符。统计工程实践中 RTL 低级 bug 的来源，位宽类问题常年排在前列，远比“逻辑写反”常见——因为每一行单独看都像是写的对。

两段有病的代码，病都在位宽：

```
wire [7:0] a, b;
wire [7:0] sum_bad;
wire [8:0] sum_good;
wire      flag;
wire [7:0] threshold;
```

```
wire        too_low;

assign sum_bad  = a + b;           // 病：第 9 位进位被静默丢弃
assign sum_good = {1'b0, a} + {1'b0, b}; // 先扩到 9 位再加，进位有处可去
assign too_low  = (threshold < flag); // 病：flag 补成 8 位再比，阈值形同虚设
```

`sum_bad` 仿真时大部分输入都对，只在和溢出时出错——最折磨人的一类 bug。`too_low` 更隐蔽：`flag` 只有 0 和 1 两种取值，比较退化成了“`threshold` 是否为 0”，阈值设定完全失效。两种病的共同点是编译不报错、每行都“像是对的”，只有逐位推算才能发现。

写法	含义	硬件成本
<code>wire [7:0] d</code>	8 根并排的线	8 根连线
<code>d[3:0]</code>	抽出低 4 根	0 个门
<code>d[3:0]</code> 与 <code>d[7:4]</code> 拼接	两束并成一束，次序即排位	0 个门
<code>4'b0</code> 与 <code>d[3:0]</code> 拼 接	高位补 0 扩展到 8 位	0 个门，补位接固定电平
<code>d + 8'd1</code>	8 位加法	一条 8 位进位链

写 RTL 的两条纪律由此而来。第一，常数永远写全位宽，`8'd0` 而不是裸 0，让每一行的宽度意图可见。第二，加法结果先想进位去向：要么左边多给一位，要么把左边写成 `carry` 与 `sum` 的拼接，让进位进入有名信号。第三章给 ALU 算标志时就是这么处理的，这里只是把同一件事写成文字。

读别人的代码时，位宽是头号排查对象，按三个检查点过一遍：每个运算符左右两侧的宽度各是多少；每个常数有没有写明位宽；每处拼接的次序与预期的高位排位是否一致。三问都答得出，位宽类 bug 基本无处藏身；答不出，就先停下来把宽度逐个标注在草稿上，再继续往下读。

案例：安全启动控制板的并排对照 第一章用一片 74HC04、一片 74HC00、一片 74HC32 搭过三级报警优先级电路：`fire` 优先级最高，`fault` 次之，`info` 最低，任一时只点亮当前最高优先级那盏灯，高优先级请求以反相形式出现在低优先级的乘积项里，从逻辑上把低优先级输出强制为 0。现在把同一张电路图写成 Verilog：

```
module alarm_priority (
    input wire fire,
    input wire fault,
    input wire info,
    output wire fire_lamp,
    output wire fault_lamp,
    output wire info_lamp,
    output wire any_alarm
);
    wire fire_n;    // fire 的反相，U1 门 1
    wire any_hi;    // fire OR fault, U3 门 1
    wire any_hi_n;  // any_hi 的反相，U1 门 3

    assign fire_n      = ~fire;
    assign fire_lamp    = fire;           // 直连，不经门
    assign fault_lamp   = fault & fire_n; // U2 门 1 加 U1 门 2
    assign any_hi       = fire | fault;
    assign any_hi_n     = ~any_hi;
    assign info_lamp    = info & any_hi_n; // U2 门 2 加 U1 门 4
    assign any_alarm    = any_hi | info;   // 校验输出，U3 门 2
endmodule
```

代码行与芯片门的对应是一一可数的：

代码行	第一章对应资源	检查点
<code>fire_lamp = fire</code>	直连，不经门	最高优先级路径最短
<code>fire_n = ~fire</code>	U1 (74HC04) 门 1	反相信号参与屏蔽项
<code>fault_lamp = fault & fire_n</code>	U2 (74HC00) 门 1 加 U1 门 2	NAND 加反相实现 AND
<code>any_hi = fire fault</code>	U3 (74HC32) 门 1	高优先级汇总
<code>any_hi_n = ~any_hi</code>	U1 门 3	供 <code>info</code> 屏蔽项使用
<code>info_lamp = info & any_hi_n</code>	U2 门 2 加 U1 门 4	德摩根改写省去三输入 AND
<code>any_alarm = any_hi info</code>	U3 门 2	两条独立路径算同一事实

并排看有两点值得停留。第一，结构完全对应：三片芯片、八个门、若干跳线，变成二十行文字，每个门都能在代码里点到名，`any_alarm` 这条刻意多算的校验路径也原样保留——两条独立路径算同一事实的调试思想，在 RTL 里同样成立。第二，控制精度发生了变化：第一章能精确说出“`fire` 到 `fault_lamp` 三级门”，因为门是亲手挑的；RTL 把“用哪几个门”的决定权交给了综合器和工艺库，文字里只剩逻辑关系。放弃对门数的精确控制，换取写法的紧凑和可维护——这就是 RTL 相对原理图的定位。想保住精确门数也有办法：门级实例化写法，下一个专题就讲。

还有一些原理图上的要素在 RTL 里根本没有对应行：电源脚、去耦电容、未用门的输入处理、电平余量。它们并没有消失，而是移交给了工艺库、后端工具和板级设计——模块实例化成芯片之后，这些约定仍然成立，只是不再由 RTL 文字表达。读 RTL 时心里要留着这张移交清单，尤其在把 FPGA 引脚接到真实报警灯的时候：驱动电流、上拉下拉这类电气约定，永远要去原理图和数据手册里查，代码里找不到。

反方向的练习同样重要：拿到一段 RTL，把它读回电路图。方法是对每一行 `assign` 问三个问题——右边表达式是什么门网络，左边线网连到哪里，线网还有没有其他驱动者。三个问题答完，`alarm_priority` 就重新变回八个门、三根内部跳线、一条直连。再用第一章的真值表复核：`fire=1` 的四行里 `fire_lamp` 恒为 1 且另两灯恒为 0，`fault` 只在 `fire=0` 时才能点亮自己的灯——代码与真值表逐行相符，这段 RTL 才算真正读完。

层次化实例化 机制：实例化是把一张画好的电路图整个放进另一张电路图。写 `alarm_priority u_alarm (...)` 时，`alarm_priority` 是图名，`u_alarm` 是这一次放置的元件编号——就是原理图上的 U1、U2。同一张图可以放很多次，每一次放置都是一份独立的硬件拷贝，像同一型号的芯片买了好几片，各自通电、互不干扰。

端口连接推荐点名法 `.端口(信号)`：

```
module panel_top (
    input wire    fire,
    input wire    fault,
    input wire    info,
    output wire [2:0] lamps,
    output wire    buzzer
);
    wire any_alarm_w; // 子图输出到蜂鸣器之间的跳线

    alarm_priority u_alarm (
        .fire      (fire),           // 子图 fire 脚接本图 fire 线
        .fault     (fault),
        .info      (info),
        .fire_lamp  (lamps[2]),      // 三盏灯捆成一束线
        .fault_lamp (lamps[1]),
        .info_lamp  (lamps[0]),
        .any_alarm  (any_alarm_w)
    );
endmodule
```



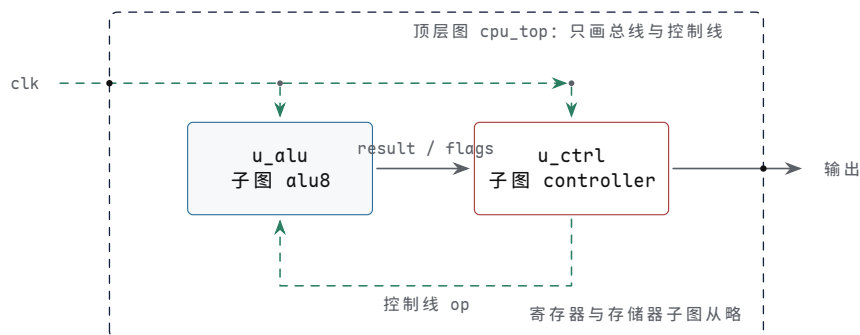
```
);

    assign buzzer = any_alarm_w; // 蜂鸣器由校验输出驱动
endmodule
```

`.fire(fire)` 读作“把子图的 `fire` 引脚焊到本图的 `fire` 线上”。点名法里端口次序随意，名字对应即可；三盏灯拼成 `lamps[2:0]` 一束线，子图的三个单位输出分别焊到束中各根——拼接在端口连接处同样只是排线。还有一种按端口表顺序连的位置法，端口一增删就错位，工程代码一律用点名法。漏连的端口默认悬空，和真实芯片引脚悬空一样危险：第一章讲过输入悬空的电平漂移，综合器对悬空端口通常只给警告，警告必须逐条看完。

大设计按“小图进中图、中图进整图”自底向上组织。第四章的最小 CPU 就是这样分图的：ALU 一张、寄存器一张、控制器一张、存储器一张，顶层图只画它们之间的总线和控制线。Verilog 的 module 层次就是这套图纸体系的文字版。读别人写的大设计，正确顺序也是先找顶层 module 的实例化列表，弄清有哪几张子图、子图之间连了哪些线，再逐张进去看细节——和拿到一台新设备先看整机框图、再拆电路板是同一个动作。

把这套“小图进中图、中图进整图”画出来，module 层次就是图套图：顶层图纸的边界圈住若干张已完成的子图，顶层自己只画子图之间的连线。



图示：module 层次就是图纸层次：顶层图 `cpu_top` 的虚线边界内放着两张子图——`u_alu` 是 `alu8` 的一次放置，`u_ctrl` 是控制器图的一次放置；同一张图可放置多次，每次都是一份独立硬件。子图之间只有两类线：实线的数据通路（`result/flags`）与虚线的控制线（`op`、`clk`），穿过边界的圆点是顶层图的端口。寄存器与存储器两子图从略。

边界上有一句必须说的话：实例化产生的是真实的第二份硬件，不是“调用”。三个实例就是三倍面积、三倍功耗。软件里“抽个函数复用代码”的直觉在这里要改：RTL 复用模块，省的是画图和验证的工夫，不是硅片。想要“一份硬件分时复用”，那是状态机加数据通路的课题，第四章已经见过——控制器让同一台 ALU 在不同周期干不同的活，那才是硬件意义上的“子程序”。

实例化还有一层用途，正好兑现上一个专题的承诺：门级实例化。把基本门各自写成 module，再全部用实例化拼装，就能让门数精确到个位，恢复第一章原理图的控制精度：

```
module not1 (input wire a, output wire y);
    assign y = ~a;
endmodule

module or2 (input wire a, input wire b, output wire y);
    assign y = a | b;
endmodule

// 三级报警的门级版本：与第一章芯片分配逐门对应
module alarm_priority_gates (
    input wire fire,
    input wire fault,
    input wire info,
    output wire fire_lamp,
```

```

output wire fault_lamp,
output wire info_lamp,
output wire any_alarm
);
wire fire_n, m1, any_hi, any_hi_n, m2;

not1 u_inv1 (.a(fire),          .y(fire_n));      // U1 门 1
nand2 u_nand1 (.a(fault),      .b(fire_n), .y(m1)); // U2 门 1
not1 u_inv2 (.a(m1),          .y(fault_lamp));    // U1 门 2
or2 u_or1 (.a(fire),          .b(fault), .y(any_hi)); // U3 门 1
not1 u_inv3 (.a(any_hi),      .y(any_hi_n));     // U1 门 3
nand2 u_nand2 (.a(info),      .b(any_hi_n), .y(m2)); // U2 门 2
not1 u_inv4 (.a(m2),          .y(info_lamp));    // U1 门 4
or2 u_or2 (.a(any_hi),      .b(info), .y(any_alarm)); // U3 门 2

assign fire_lamp = fire; // 直连
endmodule

```

每个实例对应第一章芯片分配表里的一个门，实例名 `u_nand1` 之类就是元件编号，`m1`、`m2` 这些中间线名也原样保留。这种写法叫结构级描述：文字里没有任何逻辑表达式，只有元件和连线，读起来像一份网表。它的用途有二：一是教学上证明“module 层次就是图纸层次”到底到了什么程度；二是某些对实现有精确要求的场合（手工摆放的关键路径、复用经过验证的硬核），工程师真的会写到这一层。代价同样明显：逻辑意图被拆散到几十个实例里，“这段代码想干什么”要拼完整张图才能回答。所以日常 RTL 停在 `assign` 和 `always` 块这一层，把门级留给需要精确控制的地方——抽象层次的选择本身就是一种工程判断。

收拢本节，五份对应已经齐备：`module` 是带端口的电路框图，端口表是图边缘的引脚；`assign` 是连线不是赋值，次序无关是它区别于程序的第一处；位宽、拼接、切片说的都是线束的排法，大多数低级 bug 藏在这里；实例化是把一张图放进另一张图，实例名就是元件编号。这套对应不依赖任何工具，纸笔即可验证——这正是“用文字画电路”的含义：每一行都能读回具体硬件，读不回去的行，多半也没写对。

下一节进入组合逻辑的第二种写法。当分支和中间量多到 `assign` 一行写不下时，`always_comb` 登场——但请记住，它只是换了一种描述手段，焊出来的仍然是没有记忆的网网络。

二、组合逻辑的 Verilog 写法

组合逻辑有两种文字写法：上一节的 `assign`，和本节的过程块 `always_comb`。本节依次回答五个问题：两者怎么分工；每个运算符背后是什么电路、多大代价；`case` 和 `if-else` 链综合出的选择结构有何差别；分支没写满时综合器会补出什么；以及怎样把组合块写成一眼能读回硬件的文字。

assign 与 always_comb 的分工 机制上，两者完全等价：都描述组合网络，输出都直接连到对应信号上，输入一变输出就跟着变。区别只在描述手段。`assign` 一行写一条连接，右边是一个表达式；`always_comb` 的一个块里可以写多行、引入临时变量、使用 `if/case` 分支，综合器读完整个块，按数据依赖把最终结果展开成门网络。写法不同，焊出来的东西是同一类。

同一个 2 选 1 选择器，两种写法并排看：

```

assign y = sel ? b : a;

always_comb begin
    if (sel) y = b;
    else    y = a;
end

```

综合结果完全相同：每比特一个二选一选择器，`sel` 接选择端。既然如此，分工

按可读性来：一行能写完的表达式用 `assign`；分支多、中间量多、有临时代号的逻辑用 `always_comb`。第三章 ALU 那种“候选结果并排算好、再按 `op` 挑一个”的结构，用 `always_comb` 写远比嵌套三元运算符清楚，本节末尾会给出完整例子。

`always_comb` 里最容易误读的还是顺序。块内语句从上到下书写，这只是描述顺序：对同一个信号，后面的赋值覆盖前面的，最后一笔生效；对不同信号，赋值之间没有先后，展开成并行存在的门。仿真器按书写顺序执行块内语句（阻塞赋值），配合 `always_comb` 自带的完整敏感表——块内读到的任何信号一变，整块立即重算——最终稳态结果与综合出的门网络一致。两套视角殊途同归，但硬件图景只有一个：块里没有“步骤”，只有连接。把 `always_comb` 读成“每当输入变化就执行一遍的小程序”，就又把 RTL 读回程序了。

临时变量是 `always_comb` 相对 `assign` 多出来的主要表达手段。把上一节的三级报警改写成一块：

```
always_comb begin
    // 临时变量是描述中的代号，综合后是中间跳线
    fire_n      = ~fire;
    any_hi      = fire | fault;
    any_hi_n    = ~any_hi;

    fire_lamp   = fire;
    fault_lamp  = fault & fire_n;
    info_lamp   = info & any_hi_n;
    any_alarm   = any_hi | info;
end
```

`fire_n`、`any_hi`、`any_hi_n` 在块内先写后用，读起来有先后，但这只是给人看的书写次序——综合器按数据依赖展开后，与上一节七行 `assign` 的版本是同一批门、同一批跳线。块内赋值用阻塞赋值 `=`，它的语义是“代号立即更新，后面的语句看到新值”；综合之后没有任何元件对应这个“立即”，它只是描述规则。正是为了守住“组合块用阻塞、时序块用非阻塞”这条纪律，第四节的移位寄存器反例才有判据。

分工可以浓缩成一张速查表：

场景	推荐写法
单行表达式：一根线、一个门网络	<code>assign</code>
多分支选择、需要中间代号	<code>always_comb</code>
模块之间穿线、端口直连	<code>assign</code>
需要 <code>if/case</code> 表达译码或优先权	<code>always_comb</code>

核心思想

`assign` 与 `always_comb` 的选择标准是表达力，不是功能：表达式一行写得清，用 `assign`；需要分支和中间量，用 `always_comb`。两者综合出的都是无记忆的组合网络——判断依据永远是“输入变，输出经传播延迟后跟着变”，没有时钟、没有保持。

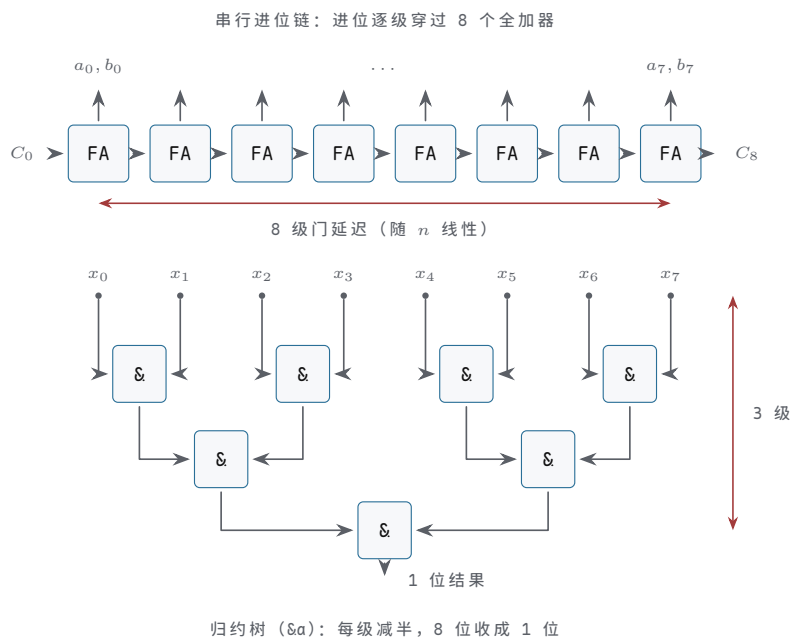
运算符与硬件的对应表 每个运算符背后都是一类具体电路，面积和延迟的量级各不相同。写 RTL 的人心里要有这张表，写完一行就能报出代价：

运算符	综合出的硬件	面积与延迟量级（ n 位宽）
<code>a + b</code>	串行进位加法器	n 个全加器；延迟沿进位链约 n 级门

$a - b$	加补器：加法器加 B 反相、Cin=1	与加法器同级，多 n 个可控反相
$a == b$	逐位 XNOR 加与归约树	n 个 XNOR，约 $\log n$ 级门延迟
$a < b$	复用减法路径，看借位或符号位	与减法器同级
$sel ? x : y$	每比特一个 2 选 1 mux	n 个 mux，1 级门延迟
逐位 $\&$ 、 $ $	n 个并行 AND/OR 门	n 个门，1 级门延迟
归约 $\&a$	与归约树： n 位收成 1 位	$n-1$ 个 AND，约 $\log n$ 级
归约异或	XOR 归约树，即奇偶校验	$n-1$ 个 XOR，约 $\log n$ 级
$\sim a$	n 个反相器	n 个门，1 级门延迟
$a << 1$ (常数移位)	重新排线，空位接固定电平	0 个门

三组区分值得单独说。第一组是逐位与归约： $a \& b$ 中两个 n 位输入对应位各自过门，输出仍是 n 位；归约 $\&a$ 只有一个操作数，把 n 位收成 1 位，硬件是一棵树。写法只差一个操作数的位置，硬件从“ n 个门并排”变成“一棵 n 输入的树”，读代码时先看操作数个数再下结论。第二组是算术与比较：减法不单独造电路，沿用第三章的补码方案，B 反相加最低位进位 1，与加法共用进位链；比较 $a < b$ 通常就复用这条减法路径，看最高位借位或符号位——所以比较器不便宜，它背后藏着一整条加法器。第三组是移位：移常数位只是排线，零成本；移变量位才是桶形移位器，面积随位宽平方增长，不可混为一谈。

链与树的形状差别画出来，比量级公式更直观。



图示：同样 8 位，两种形状的延迟量级一眼可分。上：串行进位链，进位必须逐级穿过 8 个全加器，延迟随位宽 n 线性增长，是数据通路上最贵的一级。下：归约树，每级把输入减半，8 位收成 1 位只要 3 级，延迟随 $\log_2 n$ 增长。比较与归约便宜、算术加法贵，差别就在这两种形状上。

量级的意义在于预算。一条 32 位加法链是几十级门的延迟，一个 32 位相等比较只要六级左右；写 `cnt == 32'd50000` 比写 `cnt >= 32'd50000` 便宜得多。第八章算时序预算时会再回到这张表；现在先养成习惯：每写一个运算符，报得出它是什么电路、什么量级。

把预算练成直觉，最快的办法是给熟悉的模块估一遍价。以第七章教学计算机的 8 位数据通路为例：

数据通路上的操作	对应硬件	量级估计
----------	------	------

<code>A + B</code> (加法指令)	8 位串行进位链	约 8 级进位延迟
<code>A == 8'h00</code> (零标志)	8 个 XNOR 加 3 级与树	约 4 级门延迟
<code>sel ? alu_out : mem_out</code>	8 个 2 选 1 mux	1 级门延迟
操作码译码选控制信号	4 位译码加 mux 阵列	2 到 3 级门延迟

对照第八章的时钟周期核算：这一条链路上最贵的是加法器的进位链，所以它往往决定最小时钟周期；译码和 mux 加起来不过几级门，通常不是瓶颈。能在写代码之前说出“这条路径上最贵的是哪个运算符”，就已经具备了一半时序直觉；剩下的一半在触发器和时钟偏斜里，第三、四节接着讲。

case 语句与优先结构 `case` 和 `if-else` 链都可能综合成多路选择结构，但形状不同，表达的意图也不同。`case` 表达“按选择信号的值挑一个”：综合器对选择信号译码，译码结果控制一组并行的选择路径，各分支地位平等，数据到输出的路径深度相同。`if-else` 链表达“先看第一个条件，不成立再看下一个”：综合成一串级联的 2 选 1 mux，越靠前的条件控制越靠近输出的 mux—路径最短；最后一个条件的数据要穿过整条链。四个分支，两种写法：

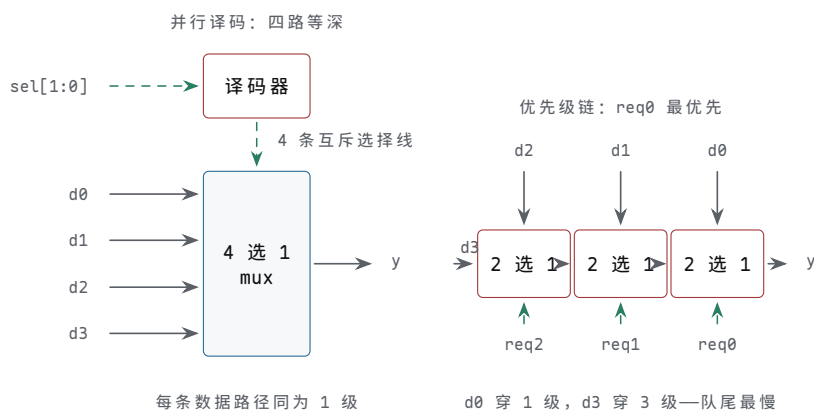
```
// 写法一：并行译码，四路地位平等
always_comb begin
    case (sel)
        2'b00: y = d0;
        2'b01: y = d1;
        2'b10: y = d2;
        default: y = d3;
    endcase
end

// 写法二：优先级链，req0 最优先
always_comb begin
    if (req0) y = d0;
    else if (req1) y = d1;
    else if (req2) y = d2;
    else y = d3;
end
```

硬件差别对照如下：

	并行译码 (case)	优先级链 (if-else)
选择结构	译码器加并行 mux	级联 2 选 1 mux
各分支路径	等深	逐级加深，链尾最深
分支间关系	互斥，由译码保证	靠前者屏蔽靠后者
天然适用	按编码选路：操作码、地址段	按优先权裁决：中断、报警

两种综合结果画成结构图，“等深”与“逐级加深”的差别一眼可分：



图示：左：`case` 综合成译码器加并行 mux，`sel` 译出四条互斥选择线，`d0` 到 `d3` 到达输出的路径同为一级，地位平等。右：`if-else` 链综合成级联 2 选 1，`req0` 控制的那级紧贴输出，`d0` 只穿一级，链尾的 `d3` 要穿三级—优先级不是注释，是物理上的路径长度。

第一章的三级报警就是优先级链的天然例子：`fire` 一来，`fault` 和 `info` 的输出被逻辑本身关断——用 `if-else` 链写，`fire` 放第一级，硬件自动获得“最高优先级路径最短”的性质，与第一章“越重要的信号越快”的结论吻合。反过来，译码场景写 `if-else` 链则是自找麻烦：第四章指令译码按 `opcode` 选控制信号，十六种编码地位平等，用 `case` 既平行又清楚。

SystemVerilog 还提供两个意图声明。`unique case` 声明各分支互斥且覆盖完备，综合器可放心按并行译码优化，若实际输入打破声明（分支重叠或遗漏），仿真会报告；`priority case` 声明按书写顺序带优先级、允许未列出的组合不覆盖，综合器按链式结构处理。意图声明不是装饰，是让工具替你检查约定的写法；但它只影响检查与优化方向，不改变你写下的逻辑关系本身。

优先级链有一个直接的量化后果：链越长，队尾越慢。四级 `if-else` 的最后一个分支，数据要穿过三级串联的 2 选 1 mux；而把三级报警写成 `if-else` 链，`fire` 的路径是零级 mux——直接就是输出：

```
// 三级报警的 if-else 版：硬件与第一章逐门对应
always_comb begin
    if (fire) begin
        fire_lamp = 1'b1;
        fault_lamp = 1'b0;
        info_lamp = 1'b0;
    end else if (fault) begin
        fire_lamp = 1'b0;
        fault_lamp = 1'b1;
        info_lamp = 1'b0;
    end else if (info) begin
        fire_lamp = 1'b0;
        fault_lamp = 1'b0;
        info_lamp = 1'b1;
    end else begin
        fire_lamp = 1'b0;
        fault_lamp = 1'b0;
        info_lamp = 1'b0;
    end
end
```

综合器把它展开后，`fault_lamp` 只在“无 `fire` 且有 `fault`”时为 1——`fire` 的反相出现在 `fault` 的使能项里，与第一章“高优先级以反相形式出现在低优先级乘积项中”的结构逐字对应。分支里的常量 `1'b0` 不是“关灯动作”，是把对应输出接到固定电平；三个输出在每条路径上都有确定值，这段代码不会推断锁存器，下一专题的默认值纪律它天然满足。

顺带提一下 `casez`：它允许分支项里写 `?` 作为“这位不比较”的通配，适合译码时只关心部分位的场合——例如“高三位是 `101` 就选中，低位任意”。通配位在硬件上是译码矩阵里被省去的比较，写得好能省门；但多个带通配的分支可能同时匹配，`casez` 按书写顺序取第一个匹配项，这就把优先级悄悄引了回来，读 `casez` 必须像读 `if-else` 链一样留意次序。入门阶段建议先用朴素的 `case` 写全编码，确认需要通配再换 `casez`。

默认赋值与锁存器推断 机制：组合块的输出必须在所有输入组合下都有确定驱动。如果某条执行路径走到底也没给某个输出赋值，综合器面对的规格就成了“这种情况下输出保持原值”——保持需要存储，于是推断出一个电平敏感的锁存器。这是第二章讲过的那种靠反馈维持状态的元件，只是这一次不是你画的，是工具替你补的，补完往往只在综合日志里留一行警告。

```
// 危险写法：sel 为 2'b11 时 y 没有新值，推断出锁存器
always_comb begin
    case (sel)
        2'b00: y = d0;
        2'b01: y = d1;
        2'b10: y = d2;
    endcase
end
```

“没写就不变”在程序里天经地义，在组合块里是危险信号：一段电平敏感的存储悄悄混进了你以为纯组合的路径。锁存器在使能期间对输入直通，毛刺跟着穿过去；它把时序分析从“周期边界”变成“电平窗口”，工具检查困难；测试设备对锁存器的可控性也远差于触发器。第二章花了一整节把锁存器升级成边沿触发器，工程 RTL 对意外锁存器的态度就是那一次升级的延续：除非故意，绝不引入。

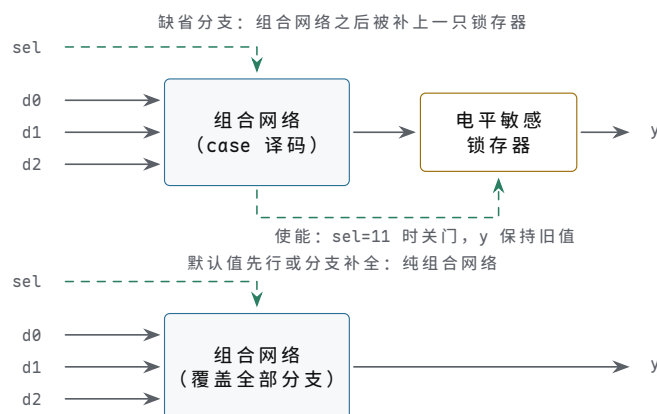
消除方法有两种，工程上首选第一种。方法一：块首先给所有输出写默认值，再用分支覆盖需要的项——默认值保证任何路径都有确定驱动，锁存器无从产生。方法二：把分支补全，给 `case` 加 `default`、给 `if` 补 `else` 兜底。两种写法如下：

```
// 方法一：默认值先行，分支只写例外（推荐）
always_comb begin
    y = d3;           // 默认值
    case (sel)
        2'b00: y = d0;
        2'b01: y = d1;
        2'b10: y = d2;
        default: y = d3;
    endcase
end

// 方法二：分支补全，每条路都有明确驱动
always_comb begin
    if (sel == 2'b00)    y = d0;
    else if (sel == 2'b01) y = d1;
    else if (sel == 2'b10) y = d2;
    else                y = d3;
end
```

方法一在长代码里优势明显：输出有十个时，默认值集中写在块首，一眼可查“每个输出的归宿是什么”；方法二把兜底散在各分支末尾，输出一多就容易漏一个——而漏一个就是一只锁存器。多输出的组合块几乎总是选方法一。

把坏写法与修复写法分别综合成硬件摆在一起，差别就直观了：前者的输出路径上多了一只你从未画过的存储元件。



图示：同一个 `always_comb` 的两种综合结果。上：`sel=11` 时没有任何分支给 `y` 新值，综合器补出一只电平敏感锁存器——使能关门期间 `y` 由存储维持，透明窗口与毛刺穿透随之混入。下：默认值先行（或分支补全）后，所有输入组合都有确定驱动，输出直接来自组合网络，一个存储元件也没有。

多输出的例子更能说明问题。下面这个块有三个输出，`valid` 分支里漏写了 `err`：

```
// err 在 valid=1 的路径上没有新值：一只锁存器
always_comb begin
    if (valid) begin
        data_out = data_in;
        ready    = 1'b1;
    end else begin
        data_out = 8'h00;
        ready    = 1'b0;
        err      = 1'b0;
    end
end

// 修好：三个输出的默认值先行，分支只写例外
always_comb begin
    data_out = 8'h00;
    ready    = 1'b0;
    err      = 1'b0;
    if (valid) begin
        data_out = data_in;
        ready    = 1'b1;
    end
end
```

坏版本里 `err` 在 `valid=1` 期间保持旧值——一只锁存器只挂在三个输出之一上，其余两个仍是纯组合。这种“半组合半存储”的混合体最欺软怕硬：模块在多数测试里表现正常，只有特定输入序列才让 `err` 显出记忆。好版本里三个默认值排成一列，每个输出的归宿一目了然，分支只描述“什么情况下偏离默认”。

写完任何 `always_comb`，按这张清单过一遍：

检查点	预期结果
每个输出在块首有默认值，或每条路径都被覆盖	任一输入组合下输出都有确定驱动
<code>case</code> 有 <code>default</code> 或分支枚举完备	任何输入组合都有分支接住
综合日志无 <code>inferred latch</code> 字样	工具与你对硬件的想象一致
块内读到的信号都会触发重算	<code>always_comb</code> 敏感表自动覆盖

常见误区：“综合器帮我补了锁存器，功能仿真也对，就这样用。”仿真对是因为激励恰好没踩中电平窗口里的毛刺。意外锁存器的故障模式是温度、电压、批次相关的间歇性错误，最难排查的一类。综合日志里出现 `inferred latch` 字样，应当作错误处理，回去补默认值。

把组合块写成可读的硬件文字 可读性的标准只有一条：每一行都能立刻读回具体硬件。三条习惯服务于这个标准。第一，输入默认、逐项覆盖：块首给默认值，下面的 `case` 只写例外项，读者先看“通常是什么”，再看“什么时候不是”。第二，避免嵌套三元：`sel ? (s2 ? a : b) : (s3 ? c : d)` 是一团看不清形状的 mux 树，同样的硬件用 `case` 写出来，结构一目了然。第三，宽信号先切片命名：把 `data[7:4]` 起名为 `hi_nibble` 再使用，读者看到的是有意义的线名，而不是一堆位号。

按这三条习惯，把第三章的 ALU 写成 `always_comb` 版本。功能与第三章一致：加、减、按位与、按位或、按位异或，外加零标志：

```
module alu8 (
    input wire [7:0] a,
    input wire [7:0] b,
    input wire [2:0] op,
    output reg [7:0] result,
    output wire      zero
);
    always_comb begin
        case (op)
            3'b000: result = a + b;    // ADD: 8 位加法器
            3'b001: result = a - b;    // SUB: 加补器
            3'b010: result = a & b;    // AND: 8 个与门
            3'b011: result = a | b;    // OR : 8 个或门
            3'b100: result = a ^ b;    // XOR: 8 个异或门
            default: result = 8'h00;   // 兜底：无锁存器
        endcase
    end

    assign zero = (result == 8'h00);   // 8 个 XNOR 加与树
endmodule
```

逐行读回硬件。五种运算各是一群候选电路，全部并排存在、同时在工作：`a + b` 一条 8 位进位链，`a - b` 沿用第三章的加补方案，三个按位运算各 8 个门。`case` 对 `op` 译码，控制一组 8 比特宽的 5 选 1 mux，从五群候选里挑出 `result`——这正是第三章“候选结果并排算好再选择”的结构，只是从门级图纸换成了文字。`default` 保证完备，这个块不可能推断出锁存器。`result` 声明为 `reg`，只因为驱动它的是 `always` 块；它身上没有一位触发器，纯粹是组合网络的输出端。`zero` 单独用 `assign` 写，因为“一行说得清”，分工习惯在此兑现。

最后说一句意图与实现的关系。第三章讲过 ADD 与 SUB 应复用同一条加法链；这里写成 `a + b` 和 `a - b` 两行，综合器通常能识别并共享加法器，但不保证。需要严格复用时，就得像第三章那样显式写出 `b_to_adder` 和 `carry_in`，把复用变成文字里看得见的结构。这是 RTL 的普遍处境：写法表达意图，综合器决定实现；意图越接近结构，实现越接近预期。下一节进入时序逻辑，触发器登场之后，这条原则只会更重要。

把“严格复用”写出来看看。下面的版本只用一条加法链，ADD 与 SUB 共享，与第三章 74HC283 加 74HC86 的实验台同构：

```
module alu8_shared (
    input wire [7:0] a,
    input wire [7:0] b,
```



```

input wire [2:0] op,
output reg [7:0] result,
output wire      zero,
output wire      carry
);
reg [7:0] b_to_adder; // B 通路上的可控反相
reg      carry_in;    // 最低位进位
wire [8:0] sum9;      // 9 位：和加进位，各得其所

always_comb begin
    // 默认值先行：加法，B 原样通过，进位 0
    b_to_adder = b;
    carry_in    = 1'b0;
    if (op == 3'b001) begin // SUB: B 取反加一
        b_to_adder = ~b;
        carry_in    = 1'b1;
    end
end

assign sum9 = {1'b0, a} + {1'b0, b_to_adder} + {8'b0, carry_in};

always_comb begin
    case (op)
        3'b000, 3'b001: result = sum9[7:0]; // ADD/SUB 共用一条链
        3'b010:          result = a & b;
        3'b011:          result = a | b;
        3'b100:          result = a ^ b;
        default:          result = 8'h00;
    endcase
end

assign carry = sum9[8];
assign zero  = (result == 8'h00);
endmodule

```

逐行读回硬件。`b_to_adder` 前面的块是 8 个可控反相器——`op` 控制每比特过原值还是反值，正是第三章 B 输入那排 XOR 门；`carry_in` 由同一个 `op` 派生，对应 `sub` 接最低位 `Cin`。`sum9` 左边主动写成 9 位并用拼接扩位，位宽纪律在此兑现：进位不再被静默丢弃，而是进入有名信号 `carry`。`case` 把 `sum9[7:0]` 的两个分支合并书写，文字层面就声明了共享——综合器不必再猜测。三个块各有分工：B 通路准备、加法链、结果选择，对应第三章实验台上的三片芯片。`b_to_adder` 与 `carry_in` 声明为 `reg` 并由 `always` 块驱动，但它们没有保持任何旧值的路径——默认值先行保证了两端都是纯组合。

这个版本多写了十几行，换来的是面积的确信：一条加法链而不是两条。选择写哪个版本，取决于你在乎什么——在乎写法简短，用前一个；在乎实现确定，用这一个。两种写法都没有对错，区别只在“把多少决定权留给综合器”。本章反复说的话在此又响一遍：Verilog 不是程序，是用文字画电路——你画到哪一层，工具就接手到哪一层。

三、时序逻辑的 Verilog 写法

前两节写的都是组合逻辑：给定输入，输出立即确定，电路本身没有记忆。第二章已经说明，记忆来自触发器——一组 D 触发器共用一个时钟，每个上升沿把各自 D 端的值抄进 Q 端，沿与沿之间 Q 保持不变。时序逻辑的 Verilog 写法因此只描述一件事：存在哪些触发器，每个时钟沿它们的 D 端接的是什么。本节把第二章见过的基本时序件——寄存器、带复位的触发器、计数器、移位寄存器、状态机——逐一写成 `always_ff` 模板，并且每写一个模板就把它读回硬件。读完本节，看到

`always_ff @(posedge clk)` 应当形成条件反射：这里有一排触发器，块内每个 `<=` 的左端是一只触发器的 Q 端，右端表达式是它 D 端前面的组合网络。

`always_ff` 的标准模板

先讲机制。D 触发器有三个端口：时钟、D、Q。它只在时钟上升沿动作一次，把此刻 D 的值抄进 Q；D 在沿与沿之间怎么变化，Q 一概不理。Verilog 用敏感表声明“什么事件会让这块硬件动作”，于是时钟是敏感表里唯一的内容。一个寄存器的最小模板如下：

```
module dff (
    input logic clk,
    input logic d,
    output logic q
);
    // 一只 D 触发器：上升沿把 d 采进 q
    always_ff @(posedge clk) begin
        q <= d;
    end
endmodule
```

这三行读回硬件没有歧义：`posedge clk` 是触发器的时钟端，`q` 是 Q 端，`d` 通过非阻塞赋值接到 D 端。用本章开头的三问套一遍：这一行产生了什么硬件——一只触发器；这个信号是组合路径还是触发器——`q` 是触发器输出，`d` 是组合路径的终点；综合器会推断什么多余的东西——不会，模板里每个符号都有明确的硬件归宿。三问都答得出，模板才算读懂。

一个 `always_ff` 块可以给多个信号赋值，得到的是一排共用同一时钟的触发器：

```
// 8 位寄存器：八只触发器共用一个时钟
logic [7:0] reg8;

always_ff @(posedge clk) begin
    reg8 <= data_in;
end
```

第二章的 8 位寄存器就是这个模样的八份并排。位宽写在声明里，赋值语句一行不变——文字的简法是 Verilog 的长处，但读的时候要记得展开：这一行是八只触发器，不是一只。

为什么敏感表不写 `d`？因为 D 端变化不会让触发器动作，它只是改变“下一次沿到来时将被采样的值”。若把敏感表写成 `@(posedge clk or d)`，等于声明“`d` 的任何变化也是一次提交事件”——真实世界里不存在这种触发器，综合器要么报错，要么推断出一个并不想要的结构。这里藏着一条读 RTL 的基本功：敏感表不是“块内读到了哪些信号”的清单，而是“哪些边沿构成提交边界”的声明。块内用到却不在敏感表里的信号，正是穿过组合网络送到 D 端的数据输入；区分时钟与数据，先看敏感表，再看赋值方向。

`always_ff` 是 SystemVerilog 的关键字，比 Verilog-2001 的 `always @(posedge clk)` 多一层自我声明：它明确告诉综合器“我意图描述触发器”。若块内写法推不出触发器——例如同一个信号在多个块里被赋值——综合器会给出警告，而不是默默猜测。读旧代码遇到 `always @(posedge clk)`，硬件含义完全相同，只是少了这层声明带来的检查。

边界只有一条，但必须记住：一个信号只能在一个 `always_ff` 块里被赋值。一只触发器的 D 端只有一个驱动源；同一个信号出现在两个块的 `<=` 左端，等于把两个输出短接到同一根导线上，综合器会报多驱动错误。这也是“Verilog 不是程序”的一条硬理由——程序里两个函数可以先后写同一个变量，硬件里两个驱动源不能同时推一根导线。

非阻塞赋值的语义：先读旧值，同时提交

机制仍然是第二章那句话：时钟沿到来时，所有触发器在同一瞬间翻转，采样的都是沿之前的旧值。非阻塞赋值 `<=` 就是这句话的文字形式。一个 `always_ff` 块内所有 `<=` 的右端，都用沿到来之前的值计算；全部右端算完之后，左端才同时更新。读 `<=` 的正确姿势不是“执行这一句”，而是“给这只触发器的 D 端接线”。

用交换两个寄存器来验证这套语义：

```
always_ff @(posedge clk) begin
    a <= b;    // A 的 D 端接 B 的 Q
    b <= a;    // B 的 D 端接 A 的 Q
end
```

两个右端读的都是旧值，于是一个沿之后 `a` 与 `b` 互换。硬件上这是两只触发器交叉对接：A 的 D 接 B 的 Q，B 的 D 接 A 的 Q。两行代码谁先谁后不影响结果，因为接线没有先后——这与第二章“时钟沿同时翻转”的图景严格对应，非阻塞赋值就是把“同时”写进语言的方式。逐拍看更清楚：

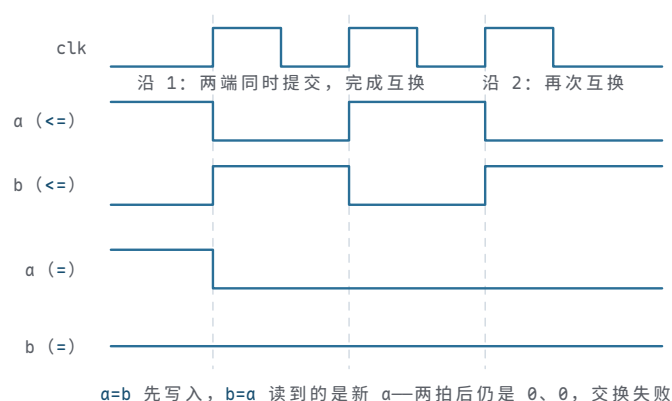
时刻	a	b	说明
沿之前	1	0	两个右端各自采样旧值
沿之后	0	1	两端同时提交，完成互换
再一个沿后	1	0	再次互换，回到原样

同一场合换成阻塞赋值 `=`，语义立刻改变。`=` 先更新左端，后面的语句读到的是新值：

```
always_ff @(posedge clk) begin
    a = b;    // a 立刻变成 b 的值
    b = a;    // 读到的是新 a，即 b 的旧值
end
```

结果 `a` 与 `b` 都变成 `b` 的旧值，交换失败；硬件上相当于两只触发器的 D 端都接到了 B 的 Q，A 的旧值被丢弃。更麻烦的是，`=` 让结果依赖书写顺序——两行一调换，推断出的接线就跟着变。仿真行为随书写顺序漂移，正是把 Verilog 当程序读要付的第一笔学费。

把两种写法画到同一条时间轴上（沿前值均为 `a=1、b=0`），“同时提交”与“立即生效”就成了两种波形形状：



图示：沿前值 `a=1、b=0`。上两行：`<=` 的右端都读沿前旧值，沿 1 之后 `a、b` 互换，沿 2 再换回——硬件是两只触发器 D 端交叉互连。下两行：`=` 立即生效，`a=b` 先把 `a` 写成 0，`b=a` 读到的是刚写入的新 `a`，`b` 停在 0——等效于两只触发器的 D 端同接 `b` 的 Q，交换被同化。

“同时提交”还有一个对全书都重要的推论：`<=` 的左端在本沿之后、下一沿之前保持不变，于是组合逻辑在沿与沿之间读到的是稳定电平。第二章分析建立时间、保持时间和时钟偏斜时，预设了“采样沿前后各有一段安静期”；在非阻塞赋值的世界里，这个预设自动成立。这就是同步设计把 `<=` 定为标准的根本原因，也是本节要

记的规则：时序块里只写 `<=`。§ 四还会用移位寄存器的正反两例，把这条规则在综合层面再算一遍账。

复位的两种写法

第二章讲复位时分过工：拉入要异步，保证任何时刻都能进入安全状态；释放要同步，让所有触发器在同一拍恢复。Verilog 的两种复位写法正好对应这两个选择，差别只在敏感表。

异步复位把复位信号本身当作一个提交事件，所以它进敏感表：

```
// 异步复位：rst_n 的下降沿不等时钟，立即清零
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        q <= '0;    // 复位沿到来，Q 直接归 0
    else
        q <= d;    // 时钟沿到来，正常采样
end
```

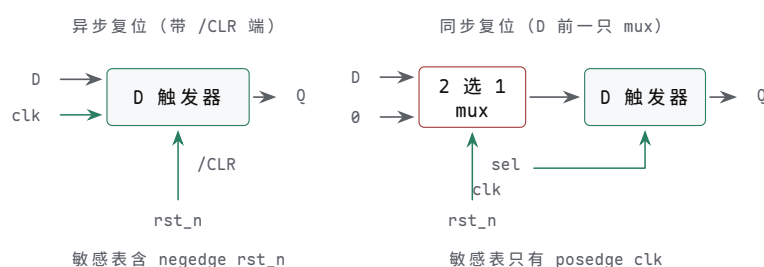
读回硬件：这是一只带异步清零端的触发器，第二章的 74HC74 就有这样一只 `/CLR` 引脚。`rst_n` 的下降沿不经过时钟，直接把 `Q` 拉到 0；`rst_n` 为高时，每个时钟沿正常采样 `D`。敏感表里出现两个边沿，是“异步复位”在文字上的签名。名字里的 `_n` 后缀与低电平有效的 `negedge` 也是一对约定：看到 `rst_n`，就知道复位是拉低生效。

同步复位不进敏感表，它只是 `D` 端网络的一个输入：

```
// 同步复位：是否清零由时钟沿裁决
always_ff @(posedge clk) begin
    if (!rst_n)
        q <= '0;    // 沿到来且复位有效，采 0
    else
        q <= d;    // 沿到来且复位无效，采 d
end
```

代码只差敏感表里的几个字符，硬件却完全不同：`rst_n` 不再接触发器的清零端，而是变成 `D` 端前面一只二选一 mux 的选择端——复位有效时 mux 选 0，否则选 `d`。复位何时生效完全由时钟沿决定，与时钟域外的世界没有直接通道。

两种写法读回的硬件并排画出，差别只在复位信号接到哪里。



图示：两种复位写法读回的硬件。左：异步复位把 `rst_n` 接到触发器的异步清零端（74HC74 的 `/CLR`），下降沿不等时钟、立即清零，敏感表里因此出现 `negedge rst_n`。右：同步复位的 `rst_n` 只控制 `D` 端前面的二选一 mux，复位有效时 mux 选 0，是否清零由时钟沿裁决，敏感表里只有 `posedge clk`。

什么时候用哪种？对照第二章“异步拉入、同步释放”的约定：来自芯片外部的复位按钮是异步事件，入口级用异步写法，保证任何时刻都能把状态机拉进安全状态；释放路径经过两级触发器同步之后，内部各模块看到的已是同步信号，其后的大量寄存器用同步写法即可。第二章那只复位同步小电路，写成 Verilog 是两行触发器：

```
// 复位释放同步器：异步拉入、同步释放
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
```

```

    rst_ff1 <= 1'b0;
    rst_ff2 <= 1'b0;
end else begin
    rst_ff1 <= 1'b1;      // 释放时机由时钟裁决
    rst_ff2 <= rst_ff1;
end
end
// 内部模块使用同步后的 rst_sync_n = rst_ff2

```

外部 `rst_n` 拉低时，两级触发器立刻清零，复位“进得去”；外部释放后，`rst_ff1`、`rst_ff2` 在两个连续时钟沿逐级变高，内部所有寄存器看到的是同一个沿上翻来的释放信号，复位“出得来”而且出得整齐。这正是第二章“异步拉入、同步释放”的模板化形态。

同步写法还有两个实际好处：综合器可以把复位逻辑并进数据路径，省掉专用的全局复位网络；复位路径与普通数据路径一样参与建立时间检查，时序分析更简单。FPGA 和 ASIC 的标准单元库通常两种复位端的触发器都备有，选型由系统复位策略决定，不是语法偏好。下表把两种写法并排对照：

复位写法	读回的硬件	生效时刻	典型场合
异步复位	触发器的异步清零端（如 74HC74 的 <code>/CLR</code> ）	<code>rst_n</code> 边沿立即生效，不等时钟	上电、急停、外部复位入口
同步复位	D 端前面的二选一 mux	下一个时钟沿	内部已同步的复位域

最后一个细节：不是每个寄存器都要复位。第二章的原则是“控制状态机必须复位，数据通路视情况而定”。写进 Verilog 就是少写一个 `if`：不复位的寄存器，模板里干脆不出现 `rst_n`，综合器就少布一根复位线，触发器也可能换成无复位端的更小单元。复位是硬件需求，不是代码洁癖。

使能端与计数器、移位寄存器模板

第二章把计数器定义为“寄存器加一组合逻辑的反馈”。使能端是插在反馈路径上的一只 mux：使能有效时 D 接 `q+1`，无效时 D 接 `q` 自身。写成 Verilog 只有三行：

```

// 标准计数器：en 有效才加一，否则保持
always_ff @(posedge clk) begin
    if (en)
        q <= q + 1'b1;
end

```

注意这里没有 `else` 分支。时序块里“没写”不等于“没变”，而是“D 端接回 Q 的旧值”——触发器天然会保持，保持不需要任何代码。这是时序块与组合块最关键的差别；§ 四讲锁存器推断时，同一句“没写”在组合块里会惹出完全不同的麻烦。使能端也是同步设计里“减速”的正道：想让计数器每秒才加一，就给它一个每秒出现一次的使能脉冲，而不是去动时钟——第二章交通灯的 1Hz 分频使能就是这个思路；时钟本身应保持完整，第八章还会从时钟树的角度重讲这条纪律。

工程里的计数器通常有三个控制：同步清零、装载、使能。三者的优先级写在 `if` 链里，越靠前优先级越高：

```

// 带清零、装载、使能的计数器：优先级 clr > load > en
always_ff @(posedge clk) begin
    if (clr)
        q <= '0;           // 清零最优先
    else if (load)
        q <= load_val;     // 其次装载
    else if (en)

```



```

    q <= q + 1'b1;      // 再次计数
    // 三者都无效：保持
end

```

读回硬件：D 端是一条三级优先 mux 链，clr 控制的那一级离触发器最近、优先级最高。if 链的书写顺序就是 mux 的物理顺序——调整条件顺序，等于在电路里挪动 mux 的位置。第二章交通灯控制器里那个“装载 30/5/2 秒、减到 0 发 timer_done”的定时器，把加一换成减一、再配一个比较器，就是这个模板的直接变形：

```

// 交通灯定时器：装载秒数，1Hz 使能下递减，到 0 发脉冲
always_ff @(posedge clk) begin
    if (load)
        timer <= load_val;
    else if (tick_1hz && (timer != 0))
        timer <= timer - 1'b1;
end
assign timer_done = (timer == 0);

```

移位寄存器同样四行。以第二章和第六章反复出现的 74HC595 式串入并出为例：

```

// 74HC595 式移位寄存器：en 有效时左移一格
always_ff @(posedge clk) begin
    if (en)
        q <= {q[6:0], sin};
end

```

拼接表达式读作连线：第 i 只触发器的 D 接第 $i-1$ 只的 Q，第 0 只的 D 接 sin。八只触发器排成一列，每个沿所有 bit 同时挪一格——“同时”依然来自 <= 的旧值语义，第 i 只采到的是第 $i-1$ 只的旧值，串行输入一个沿只能前进一级。若误用 =，第一句赋出的新值立刻被下一句读到，一个沿里输入 bit 直达末尾，三级链条退化成一级直通；这笔账 § 四开头会细算。595 芯片还有第二级存储寄存器——把移位结果一次性送到输出引脚的那一排——写成 Verilog 是另一个时钟下的整块复制：

```

// 595 的完整模型：移位级 + 存储级，两个时钟各管一排
always_ff @(posedge srclk) begin
    if (sr_en)
        shift <= {shift[6:0], sin};
end

always_ff @(posedge rclk) begin
    q_out <= shift;    // 存储级：移位结果整体亮相
end

```

两个 always_ff 块、两个时钟、两排触发器，与芯片手册里 SRCLK 和 RCLK 两只引脚——对应。再读 595 的数据手册时看到这两只时钟，就能直接想起这两段代码。

两段式状态机模板

第二章给状态机画的结构图是三块：状态寄存器、下一状态组合逻辑、输出组合逻辑。两段式模板把这个结构原样搬进 Verilog：一段 always_ff 只做一件事——沿到来时 state <= next_state；一段 always_comb 根据当前状态和输入算出 next_state 与全部输出。分工固定之后，读任何状态机都有了固定路线：case 语句对照转移表，输出赋值对照输出表，而 always_ff 段永远只有三行，没有值得多看的内容。模板的价值，就在于让无聊的段落真正无聊。

现在把第二章的交通灯 FSM 完整写成模板。需求复述一遍：东西绿 30 秒、东西黄 5 秒、全红 2 秒、南北绿 30 秒、南北黄 5 秒、全红 2 秒，循环往复；计时

交给一个独立的带装载递减计数器，它减到 0 时给出单周期脉冲 `timer_done`，这是状态机唯一的转移条件；灯色只由当前状态决定，是 Moore 型输出；复位入口选全红。先声明状态编码与灯色，并写状态寄存器段：

```
typedef enum logic [2:0] {
    EW_GREEN, EW_YELLOW, ALL_RED_EW,
    NS_GREEN, NS_YELLOW, ALL_RED_NS
} state_t;

typedef enum logic [1:0] {RED, YELLOW, GREEN} light_t;

state_t state, next_state;
light_t ew_light, ns_light;

// 第一段：状态寄存器。沿到来时提交下一状态
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        state <= ALL_RED_EW;    // 上电先全红
    else
        state <= next_state;
end
```

第二段是下一状态逻辑，它就是第二章转移表的逐行翻译：

```
// 第二段（上）：下一状态逻辑，纯组合
always_comb begin
    next_state = state;    // 默认保持：timer_done=0 时不动
    case (state)
        EW_GREEN:    if (timer_done) next_state = EW_YELLOW;
        EW_YELLOW:   if (timer_done) next_state = ALL_RED_EW;
        ALL_RED_EW:  if (timer_done) next_state = NS_GREEN;
        NS_GREEN:    if (timer_done) next_state = NS_YELLOW;
        NS_YELLOW:   if (timer_done) next_state = ALL_RED_NS;
        ALL_RED_NS:  if (timer_done) next_state = EW_GREEN;
        default:     next_state = ALL_RED_EW;    // 非法编码回全红
    endcase
end
```

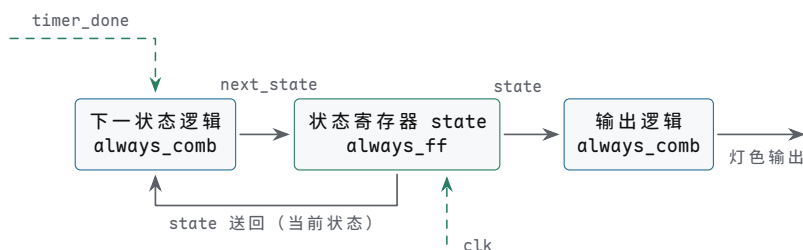
输出段对应第二章的输出表。它可以并入上一段，这里分开写，以强调它回答的是另一个问题——“这个状态点亮哪盏灯”：

```
// 第二段（下）：输出逻辑，Moore 型，只由状态决定
always_comb begin
    ew_light = RED;        // 先给默认：双向全红
    ns_light = RED;
    case (state)
        EW_GREEN:    ew_light = GREEN;
        EW_YELLOW:   ew_light = YELLOW;
        NS_GREEN:    ns_light = GREEN;
        NS_YELLOW:   ns_light = YELLOW;
        default:     ;      // 两个全红态保持默认
    endcase
end
```

逐段读回硬件。`state` 是 3 只触发器——六状态二进制编码，与第二章的编码表一致；若改用独热方案，把 `typedef` 换成 6 位向量加六个单 bit 常量，就是 6 只触发器。`next_state` 的组合网络是一只以 `state` 为选择端的 8 输入 mux：注意每个分支若不满足 `timer_done`，mux 的输出就是 `state` 自身——“保持”是 mux 把旧值接回 D 端，不是免费的静止。`timer_done` 只出现在 `if` 条件里，它是 mux 链上

的辅助选择信号。输出段是两个译码器：输入 `state`，输出两组灯色；进 `case` 前先给默认值、再按需覆盖，保证每种状态下每根输出线都有确定电平，不给锁存器留任何机会（§ 四细讲这条纪律）。

三段代码与硬件的对应可以直接画成结构图——它与第二章那张三框结构图是同一张图：



图示：两段式模板的硬件形状：左框 (`always_comb`) 按 `state` 与 `timer_done` 算出 `next_state`，中框 (`always_ff`) 在 `clk` 沿把 `next_state` 提交进状态寄存器，右框 (`always_comb`) 把 `state` 译成 `ew_light`、`ns_light` 两组灯色。`state` 经下方通道送回左框——“保持”就是 mux 把旧值接回 D 端。`timer_done` 是这台状态机唯一的转移条件，计数器本体在模块之外。

用一条逐拍轨迹把三段连起来看。假设 `state` 当前是 `EW_GREEN`，定时器还在计数，`timer_done=0`：组合段的默认行生效，算出 `next_state = state`，时钟沿把 `state` 原样写回——状态机原地不动，灯色不变。若干秒后定时器减到 0，`timer_done=1` 的那一拍，组合段算出 `next_state = EW_YELLOW`，沿到来时状态寄存器提交，下一拍起输出译码器看到新状态，东西向由绿转黄。每个周期状态机恰好前进一步或原地不动，这正是第二章“状态只在时钟边界提交”的代码形态：

时钟沿	<code>timer_done</code> (沿前)	<code>state</code> (沿后)	灯色变化
第 n 沿	0	<code>EW_GREEN</code> (保持)	不变
第 $n+1$ 沿	1	<code>EW_YELLOW</code> (提交)	沿后东西转黄
第 $n+2$ 沿	0	<code>EW_YELLOW</code> (保持)	不变

还有两个边界要点，都是第二章强调过的工程细节在代码里的形状。其一，复位状态是 `ALL_RED_EW` 而不是某个绿灯态——上电后所有方向先看到红灯。其二，`default` 分支把 3 位编码里两个未使用的编码导向全红；删掉这两处，代码更短，路口更危险，而综合器不会替你保留安全余量。最后注意定时器不在状态机里：第二章的分工是状态机只回答“现在在哪个阶段、下一个阶段是什么”，计数交给带装载的递减计数器，两者靠 `timer_done`、装载值、启动三条线相连。这个边界在 Verilog 里体现为两个模块各写各的——状态机模板里一个计数语句都不出现。

两段式不是唯一写法。把状态寄存器与下一状态逻辑挤在同一段里的“一段式”也能综合，但状态跳转和输出译码混在一起，读起来要自己在脑子里重新分块；在两段之外再加一段输出寄存器的“三段式”，用一拍延迟换输出无毛刺，适合驱动外部危险负载。对初学者，两段式与第二章结构图的对应最整齐：一段文字对应图上一个方框，读代码就是读图。

五个模板与它们读回的硬件，汇总成一张对照表，作为本节的收尾：

模板	读回的硬件	敏感表的形状
最小寄存器 异步复位	一只 D 触发器 带 <code>/CLR</code> 端的触发器 (74HC74 式)	只有 <code>posedge clk</code> 时钟沿加复位沿
同步复位寄存器 计数器	触发器加 D 端 mux 触发器排加一器加使能 mux	只有 <code>posedge clk</code> 只有 <code>posedge clk</code>
移位寄存器	触发器排首尾相接	只有 <code>posedge clk</code>

两段式状态机	状态寄存器加两片组合网络	只有 <code>posedge clk</code> (或加复位沿)
--------	--------------	-------------------------------------

右列值得多看一眼：除了异步复位，所有模板的敏感表都只有 `posedge clk`。同步设计的文字签名就是这么朴素——一个时钟沿统领全部提交。敏感表里多出任何别的东西，都要像对待异步复位那样追问一句：这个边沿对应哪只真实引脚？

本节五个模板共用同一条读法：敏感表指出提交边界，`<=` 的两端指出触发器与 D 端网络，`if` 链指出 mux 的优先级。把这三样读出来，任何时序 RTL 都能翻回第二章的触发器图。Verilog 不是程序，是用文字画电路；时序部分画的，是“沿”与“沿之间”。

四、从 Verilog 读回硬件：综合边界

前三节解决“怎么写”，本节解决“写完之后综合器怎么想”。综合器不是编译器：编译器把语句翻译成顺序执行的指令流，综合器把文字翻译成门与触发器的网表，而且只接受语言的一个子集。读工程 RTL 必须知道这条边界的走向——哪些写法读回确定的硬件，哪些写法会被推断成意料之外的元件，哪些写法根本不属于 RTL。

本节按五类问题清点这条边界：赋值符号的差别、“没写全”的代价、语言的仿真专属区、位宽与符号的隐性约定、编译期复制结构的 `generate`。每一类的读法都一样：先问综合器看到这段文字会画什么，再问那是不是自己想要的硬件。

阻塞与非阻塞的正反两例

§ 三立过规矩：时序块里只写 `<=`。本专题用一对正反例把这条规矩算到综合层面，先看它为什么对，再看违反它损失什么。

正例：三级移位寄存器，非阻塞赋值。

```
// 正例：三级移位寄存器，每个 <= 都采旧值
always_ff @(posedge clk) begin
    s1 <= in;
    s2 <= s1;
    s3 <= s2;
end
```

读回硬件：三只触发器首尾相接，`in` 进第一级的 D，`s1` 的 Q 进第二级的 D，`s2` 的 Q 进第三级的 D。每个沿所有右端同时采旧值，输入 bit 每拍恰好前进一级。设三级的初值都是 0，逐拍推演：

沿序号	<code>in</code> (沿前)	<code>s1</code> (沿后)	<code>s2</code> (沿后)	<code>s3</code> (沿后)
第 1 沿	1	1	0	0
第 2 沿	0	0	1	0
第 3 沿	1	1	0	1

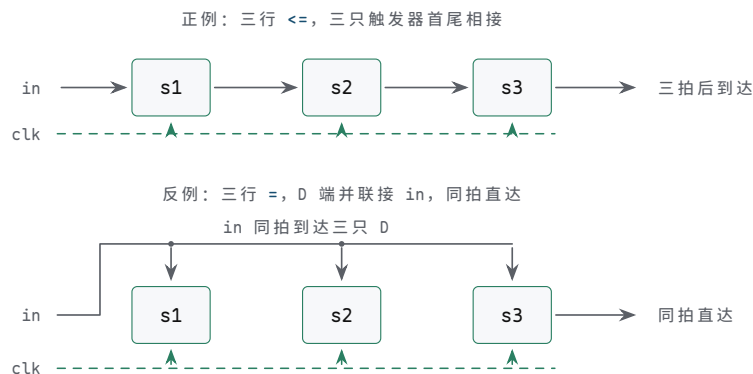
“每拍前进一级”不是文字游戏，是硬件结构决定的：第 i 级的 D 端物理上接着第 $i-1$ 级的 Q，沿到来时第 $i-1$ 级的新值还没来得及穿过时钟到输出的延迟，第 i 级采到的必然是旧值。第二章讲时钟到输出延迟与保持时间时画的沿口竞态，正是这套语义的物理底板。仿真器兑现“旧值语义”的方式也值得一提：它先把块内所有 `<=` 的右端按当前值算好、暂存，再把左端一起更新——计算有先后，生效同时刻，与硬件的沿口提交一一对应。

反例：同样三行，换成阻塞赋值。

```
// 反例：= 立即生效，新值沿书写顺序连跳三级
always_ff @(posedge clk) begin
    s1 = in;
    s2 = s1;
    s3 = s2;
end
```

逐句追踪：第一句 `s1 = in` 立即生效，`s1` 当场变成 `in` 的新值；第二句读到的 `s1` 已是新值，`s2` 跟着变成 `in`；第三句同理——同一个沿里，`in` 的值沿书写顺序连跳三级，直达 `s3`。硬件读法：三只触发器的 D 端全接到了 `in`，三级移位退化成三只并联的单级采样器。书写顺序在这里成了硬件的一部分：三行换个排列，推断出的接线就换一副面孔。

两种写法的接线差别，画成图就是两副完全不同的面孔：



图示：同一个 `always_ff` 块的两种硬件读法。上：三行 `<=` 都采沿前旧值，综合出首尾相接的三级移位链，输入 bit 每拍恰好前进一级，三拍后到达 `s3`。下：三行 `=` 立即生效，等效于三只触发器的 D 端并联接 `in`——同拍直达 `s3`，三级移位退化成一级采样。写法差三个字符，硬件差两级触发器。

这种错误最阴险的地方在于，它可能通过粗略的功能检查。若测试只看“输入最终出现在输出”，反例一样能做到——错的只是拍数：本该三拍后到达，实际一拍直达。而拍数恰恰是硬件语义的一部分：第二章给状态机画对齐波形时逐周期核对控制线，核对的就是这种东西。写法差一个字符，硬件差两级触发器；仿真里差一个字符，行为差两级流水。

那么组合块为什么反其道而行，用 `=`？因为 `always_comb` 描述的是导线网络，电平必须立即传播才能正确建模级联的门。§ 二讲过组合块可以引入临时变量分步计算——先算中间量、再算输出——与“与门输出接或门输入”——对应；若改用 `<=`，等于声明“中间量要等某个沿才生效”，既不符合导线的事实，也会让分支完备性检查错乱。组合块里本来就没有“沿之前的旧值”这个概念，`<=` 的旧值语义无从谈起。

时序块里确实需要中间量时，工程写法是把两类赋值分开用：临时变量用 `=` 在块内当场算清，真正进触发器的信号用 `<=` 提交。临时变量综合后是 D 端组合网络的一部分，不进任何触发器——只在块内当场消费的变量没有独立的硬件对应物，它只是表达式的一种写法。

记忆规则一句话：时序用 `<=`，组合用 `=`。物理依据也是一句话：`<=` 对应“触发器在沿处同时提交”，`=` 对应“导线上的电平立即传播”。两个赋值符号不是两种执行速度，而是两类硬件。

反例也值得画一张逐拍表。同样的初值、同样的输入序列，阻塞版本的三级在每个沿后全部等于当时的 `in`：

沿序号	<code>in</code> (沿前)	<code>s1</code> (沿后)	<code>s2</code> (沿后)	<code>s3</code> (沿后)
第 1 沿	1	1	1	1
第 2 沿	0	0	0	0
第 3 沿	1	1	1	1

两张表放在一起，正反例的差别一眼可见：正例的 `s3` 是 `in` 延迟三拍的副本，反例的 `s3` 就是 `in` 本身——一个是移位寄存器，一个是一根导线带三只并联触发器。写代码时二者只差三个字符，读代码时若扫一眼赋值符号就翻页，这个差别就漏过去了。所以读时序块的第一眼，永远先落在赋值符号上。

还有一种更隐蔽的混用：同一个时序块里，一部分信号用 `<=`、另一部分信号用 `=`。仿真器按各自规则照单处理，结果往往“大体像对的”——错位只出现在混用处附近的拍序上。更麻烦的是仿真与综合对书写顺序的敏感度不完全一致：仿真严格按顺序执行阻塞赋值，而综合器推断组合前级时并不承诺维持这份顺序语义，于是仿真通过、硬件出错，成为最难追查的一类缺陷。工程评审把“时序块里出现 `=`”列为必查项，查的不是风格，是这块硬件里可能多了一根直通导线、少了一级流水。

早年资料里还有一种说法——“非阻塞赋值是并行执行”。它并不准确：仿真器照样一句一句地算，只是左端的生效时刻被推迟到整个块算完之后。把 `<=` 理解成“延迟到沿尾生效的赋值”也不算错，但都不如直接读成“D 端接线”来得干净：接线没有执行，只有连接。

锁存器推断的产生与消除

§ 二已经看过推断的产生：组合块里某条路径没给输出赋值，综合器面对“保持原值”的规格，只好搬出第二章那只电平敏感的锁存器来填空。本专题把视角挪到综合边界上，回答三个相关的问题：时序块为什么天然免疫，综合警告怎么读，除了默认值和补全分支还有什么手段。

先回答免疫问题。时序块里“没写”是常态：§ 三的计数器只在 `en` 有效时加一，其余分支一字不写，语义是“保持旧值”——而保持是触发器的本能，D 端经一只 `mux` 接回 `Q` 即可，不需要任何新元件。同一个“没写”，在时序块里是免费的本能，在组合块里却要多造一只锁存器，差别全在信号背后有没有记忆元件。对照如下：

块类型	某分支没给信号赋值	综合结果	处置
<code>always_ff</code>	保持旧值，D 端经 <code>mux</code> 接回 <code>Q</code>	触发器加使能，通常正是本意	无需处理
<code>always_comb</code>	语义要求保持，组合网络却无记忆	推断锁存器，几乎从非本意	默认值、补全、 <code>unique</code> 三选一

所以锁存器推断有一个可靠的判别式：它只可能发生在 `always_comb`（或电平敏感的 `always`）里，永远不可能发生在 `always_ff @(posedge clk)` 里。读综合报告时若看到锁存器警告指向一个自认是时序逻辑的块，那一定是敏感表写成了电平敏感，或者块内混进了组合馈通——先查敏感表，再查赋值符号。

综合警告怎么读？典型的锁存器警告会给出信号名与位置，含义是“综合器为这个信号推断了一只锁存器”。读它抓三个要素：哪个信号、哪个块、哪条路径漏了赋值。这类警告不是可以随手跳过的噪音——每一条都对应真实硬件里多出来的一只透明锁存器，带着第二章分析过的全部麻烦：使能窗口内输入直通、毛刺穿透、时序分析从周期边界变成电平窗口。工程上的纪律是：锁存器警告数必须为零，或者每一处都有写下来的理由。

消除手段，§ 二给了两种——块首写默认值、分支补全——这里补上第三种：用 `unique` 把分支性质变成工具可检查的声明。

```
// unique: 声明分支互斥且覆盖所有取值
always_comb begin
    unique case (sel)
        2'b00: y = d0;
        2'b01: y = d1;
        2'b10: y = d2;
        2'b11: y = d3;
    endcase
end
```

`unique case` 向综合器声明两件事：各分支互斥，且覆盖所有可能取值。声明属实，综合器按并行译码放手优化，遇到遗漏会警告而不是静默推断锁存器；声明不属实——例如实际出现了没列出的编码——仿真器会在运行期报错。它把“我已经检查过分支完备性”这句口头承诺，变成仿真与综合都能核对的约定。若确实需要优先级语

义、分支可能重叠，对应的关键字是 `priority`：它声明“按书写顺序取第一个匹配”，同样禁止锁存器推断，但保留 `if-else` 链的次序含义。

最后留一道边界题：有没有故意要锁存器的场合？有一某些超低功耗或特殊时序设计会显式使用电平敏感存储，但那属于高级技法，会用专门的写法与评审流程把它标记出来。对本书读者，规则不变：组合块的输出必须在每条路径上有确定值，锁存器警告数为零。

分支不全也不止 `case` 缺 `default` 一种形状，`if` 缺 `else` 同样漏风：

```
// if 缺 else: sel=0 时 y 保持，推断锁存器
always_comb begin
    if (sel)
        y = a;
end
```

读法与 `case` 缺口的读法一致：顺着每条执行路径问一句“y 在这条路上被写了吗”，只要有一条路的答案是否定，锁存器就进门。形状越朴素越容易被漏看——三行的块在评审里往往比三十行的块更危险。

把推断出的锁存器与触发器并排看，差别回到第二章的透明窗口：触发器只在沿上采样，沿与沿之间输出是安静的；锁存器在整个使能电平期间透明，输入毛刺会原样穿出。时序分析也随之分裂：触发器路径按周期边界算，锁存器路径要按电平窗口算，而工具的默认检查流程大多按前者设计——这就是“意外锁存器”常常同时带来功能风险与验证盲区的原因。

顺带划清一个名词边界：本节说的是“推断”——工具替你补出来的锁存器。若设计真的需要电平敏感存储，应当显式实例化并写明意图，让它在评审与报告里以本来面目出现，而不是作为某段组合代码的副产品悄悄混入。另外，不同综合器的警告措辞并不统一，有的写推断锁存器，有的写某信号可能被保持；读陌生工具的报告时，先把它的锁存器措辞认下来，再谈清零纪律。

不可综合写法

Verilog 语言先于综合工具而存在，它的全集是写给仿真器的；RTL 只是全集中“能读回门与触发器”的子集。判断一个写法是否在子集里，只需问一句：它对应哪种真实存在的硬件结构？以下几类常见写法通不过这一问，逐一看原因。

延时语句 `#10`。它的含义是“仿真时间前进 10 个单位”。真实电路里没有“等待 10 个时间单位”的元件；延迟由门延迟与布线寄生决定——那是第八章的话题——不由代码决定。混进 RTL 的延时语句，综合器有的报错、有的悄悄忽略。忽略比报错更危险：仿真里存在的延迟在硬件里凭空消失，仿真与实物就此分叉，而工具一言不发。

初始化块 `initial`。它描述“仿真开始那一瞬间的值”。ASIC 的触发器上电取值不定，`initial` 没有硬件对应；多数 FPGA 的触发器带配置上电初值机制，这些平台的综合器因此接受 `initial`，把它变成上电初值——但这是平台特性，不是语言承诺。要跨平台移植的代码把初始化交给复位（§ 三的两种写法），不依赖 `initial`。读 FPGA 专用代码时见到 `initial`，要明白它读回的是“配置完成时刻的触发器初值”，而不是任何运转中的硬件。

打印系统任务。`\protect \TU\textdollar display` 往仿真终端写一行文本，`\protect \TU\textdollar monitor` 跟踪信号变化并自动打印，`\protect \TU\textdollar fatal` 报错并终止仿真。硬件里没有打印机，也没有终端，这些调用没有任何门级对应物。§ 五会看到它们在 `testbench` 里大显身手——那里才是它们的合法住所。

实数变量 `real`。浮点变量没有位宽，综合器不知道给它分配几只触发器。硬件里的“小数”是定点数：第三章的 `Q` 格式，用一个整数加上约定的二进制小数点位置来表示，所有运算仍是整数运算。`real` 只可能出现在仿真侧，或编译期常量折叠的表达式里。

那么，为什么 `testbench` 可以尽情使用它们？因为 `testbench` 没有综合义务。

它是仿真世界里的虚拟实验台：产生时钟、驱动激励、观察响应、打印报告，这一切只发生在仿真器内部，永远不需要变成硅片或查找表。同一语言、两个读者—RTL 写给综合器，testbench 写给仿真器。工程目录通常把 rtl/ 与 tb/ 分开放置，读代码时先看文件在哪个目录，就知道该换上哪种读法。反过来的判别同样好用：在声称要综合的文件里看到 `\protect \TU\textdollar display`，基本可以断定作者没打算把它做成硬件。

还要区分第三类文件：仿真模型。存储器厂商提供的模型、晶振的行为级描述，里面满是延时与 `initial`—它们同样不综合，用途是在仿真里扮演真实器件，好让你的 RTL 有逼真的对手。它们介于 RTL 与 testbench 之间：不比 testbench 更接近硅片，也不比 RTL 更远离。下表汇总这几类写法的边界：

写法	硬件里不存在的东西	综合器反应	合法出现场合
#10 延时	“等待 10 个时间单位”的元件	报错或悄悄忽略	testbench
<code>initial</code>	ASIC 触发器的确定上电值	多数报错；FPGA 流程序常收作上电初值	testbench、FPGA 专用 RTL
<code>\protect \TU\textdollar display</code> 一族	打印机与终端	忽略或报错	testbench
<code>real</code>	无位宽的浮点存储	报错	testbench、常量表达式

把这一专题收拢成一句话：仿真器执行的是语言，综合器接受的是结构。凡是不能在脑子里展开成门、触发器与连线的写法，都不属于 RTL—它要么属于 testbench，要么属于仿真模型，要么属于还没人发明出来的硬件。

阅读顺序也因此确定：拿到一份陌生代码，先做分拣—哪些是 RTL，哪些是测试与模型—再对 RTL 部分动用本章的读法。把 testbench 的 `initial` 当成设计缺陷去改，与把 RTL 的锁存器警告当成风格问题去忍，是同一种错位的两个方向。

signed 与位宽陷阱

第三章讲过：同一组 bit，按无符号读和按补码读是两个数。§ 2 的运算符表默认了“读数方式已经明确”，本专题处理这张表底下的暗礁。Verilog 的运算符对两种读法共用一套符号，歧义全靠声明消解—综合出来的是补零移位器还是补符号移位器、比较器按哪种数轴比大小，全看声明。三个要点分开说。

第一，`>>` 与 `>>>`。逻辑右移 `>>` 高位补 0；算术右移 `>>>` 高位补符号位—但 `>>>` 只在操作数声明为 `signed` 时才真正是算术移位，对无符号操作数写 `>>>` 仍然补 0。移位器补什么由声明决定，不由符号的长相决定：

```
logic      [7:0] u = 8'b1100_0000;    // 无符号：192
logic signed [7:0] s = 8'b1100_0000;    // 补码：-64

u >> 2    // 0011_0000：高位补 0，得 48
u >>> 2    // 0011_0000：无符号，仍补 0
s >>> 2    // 1111_0000：补符号位，得 -16
```

第二，`\protect \TU\textdollar signed` 与比较时的扩展规则。`\protect \TU\textdollar signed(x)` 强制把 `x` 按补码解释。比较运算的规则是：两个操作数都是 `signed`，先符号扩展到同宽，再按补码比较；只要有一个是 `unsigned`，两个都按无符号处理。最后这条是著名的反直觉规则，直接给踩坑演算：

```
logic      [3:0] a = 4'b1100;    // 无符号：读作 12
logic signed [3:0] b = 4'b0111;    // signed：读作 +7

if (a < b)           // a 无符号，b 被当成无符号
```

```
// 实际比较 12 < 7, 不成立

if ($signed(a) < b) // 两边同按补码: -4 < 7, 成立
```

逐行算: `a` 是 `4'b1100`, 按无符号读作 12; `b` 声明了 `signed`, `4'b0111` 读作 +7。第一个比较里 `a` 是无符号, 于是 `b` 也被当成无符号, 实际比较是 $12 < 7$, 不成立。若设计本意是 `a` 存放补码 -4, 必须写成 `\protect \TU\textdollar signed(a) < b`, 两边同按补码, $-4 < 7$ 成立。少写一个 `\protect \TU\textdollar signed`, 比较结果整个翻转——而仿真不会报任何错, 综合器也不会, 因为对工具来说两种读法都“合法”。符号与位宽是阅读 RTL 时必须亲手核对的隐性约定, 工具默认你清楚自己在写什么。

第三, 位宽与字面量。表达式的运算位宽按参与者的最大宽度进行, 赋值时再截断或扩展到左端宽度。§ 三计数器模板里写 `q + 1'b1` 而不写 `q + 1`, 原因就在此: 不带位宽的十进制字面量是 32 位有符号数, `q + 1` 会先把 `q` 扩展到 32 位再相加, 赋回 4 位的 `q` 时虽然会截断, 但中间过程一旦接进更宽的表达式, 差异就会露头:

```
logic [3:0] q = 4'd15;
logic [3:0] n;
logic [7:0] wide;

n    = q + 1'b1;    // 语境宽 4 位: 和为 1_0000, 进位被截断, n = 0
wide = q + 1'b1;    // 语境宽 8 位: q 先零扩展, wide = 16
```

同一笔 `q + 1'b1`, 赋给 4 位的 `n` 得 0, 赋给 8 位的 `wide` 得 16, 差别全在左端的接收宽度。语境宽度规则要记全: 表达式的运算位宽按语境内所有参与者的最大宽度取定, 赋值左端也算参与者; 算完再按左端宽度截断或扩展。所谓“进位丢弃”, 不是加法器没算出第 5 位, 而是接收端只有 4 位、进位在赋回时被截掉——想留住进位, 接收端必须够宽。Harris & Harris 的纪律值得照抄: 字面量永远带位宽与基数 (`4'b1100`、`8'd30`), 算术表达式每一段的宽度都要心里有数。三个陷阱的共同根源是一个: Verilog 不会替你做“这组 bit 是什么数、有多宽”的决定, 它把决定权连同出错的自由一起交给写法。读别人的 RTL, 见到算术运算先核对每个操作数的 `signed` 声明与位宽, 再谈逻辑对错。

符号扩展与零扩展的方向也要分清。把窄信号赋给宽信号时, `signed` 窄信号做符号扩展——高位复制符号位; `unsigned` 窄信号做零扩展。一次赋值里, 读数方式与扩展方式都由声明牵着走:

```
logic signed [3:0] s4 = 4'b1100;    // 补码: -4
logic          [3:0] u4 = 4'b1100;    // 无符号: 12
logic          [7:0] w1, w2;

w1 = s4;    // 符号扩展: 1111_1100, 按无符号读是 252
w2 = u4;    // 零扩展: 0000_1100, 仍是 12
```

`w1` 的 252 往往让第一次遇到的人愣住: 赋进去的是 -4, 读出来怎么成了 252? 答案是一切都按规则发生——`w1` 是无符号的 8 位向量, -4 的补码按无符号读正是 252。位没有丢, 读数方式变了。若接收端也声明 `signed`, 读回 -4, 这次扩展就“透明”了。可见扩展本身从不出错, 出错的是两端的读数方式不一致。

比较器的硬件也因此分两路。无符号比较沿 § 二说的减法路径看最高位借位; 有符号比较同样做减法, 判据却换成“差的符号位连同溢出一起判定”——第三章补码减法判正负的那套逻辑。两种比较器面积相近, 但接错声明, 等于把数轴的原点挪了位置: 无符号的 `4'b1100` 在数轴右端 (12), 有符号的它紧挨原点左侧 (-4)。同一个数轴、两个原点, 是比较陷阱的几何图像。

还有一处与位宽有关的阅读细节: 拼接与切片永远指名道姓。拼接结果的宽度是各段宽度之和, 切片抽出的宽度是下标区间——它们不随语境伸缩, 因此是最安全的

位宽操作。真正需要提防的是隐式部分：运算扩展、赋值截断、比较对齐，这三处都按规则默默进行；规则本身不复杂，复杂的是读代码的人要记得它们存在。

工程代码里常见的防御写法也值得认得：比较前把两个操作数显式调成同一宽度同一符号，或都用 `\protect \TU\textdollar signed` 包起来；计数比较能用 `==` 就不用 `>=`——§ 二算过账，等值比较比大小比较便宜；字面量一律带位宽。这些写法不改变硬件，改变的是“读者需要猜的东西”的数量。

generate 与参数化

第四章最小 CPU 的数据通路、第七章面包板教学机的总线，都是“同一结构复制 N 份”的产物：8 只触发器排成寄存器，8 个全加器排成并行加法器。§ 一讲过实例化是把画好的图放进另一张图——那是手动复制；本专题的 generate 是自动复制。先把最重要的话说在前面：generate 是编译期的复制，不是运行时的循环。它在综合开始之前就被展开成 N 份独立硬件，电路上没有任何“循环”在运行。

`parameter` 与 `localparam` 把尺寸做成参数：

```
module shift_reg #(
    parameter int WIDTH = 8          // 实例化时可覆盖
) (
    input logic          clk,
    input logic          en,
    input logic          sin,
    output logic [WIDTH-1:0] q
);
    localparam int LAST = WIDTH - 1; // 派生尺寸，内部常量

    always_ff @(posedge clk) begin
        if (en)
            q <= {q[LAST-1:0], sin};
    end
endmodule
```

`parameter` 在实例化时可被覆盖，同一个模块描述因此能生成 8 位、16 位、32 位的移位寄存器；`localparam` 是模块内部常量，用来给派生尺寸命名，外部改不到。第三章的加法器位宽、第四章的数据通路宽度、第六章各芯片的地址线数，都是 `parameter` 的天然用例——同一个设计，改一个数字就换一种规模，硬件随文字参数重新生成。

状态机的编码也可以用同一组机制收回纸面。§ 三的交通灯模板用 `typedef enum`，把具体编码交给综合器选择；若设计要求编码确定——例如按第二章的格雷方案减少翻转时的多位同时变化——就用 `localparam logic [2:0] EW_GREEN = 3'b000`；这样的常量逐个指定。许多综合器还提供状态编码属性（写在声明旁的专用属性标注），作用相同：把“用哪种编码”从综合器的自由裁量变成设计者的显式决定。属性本身不改变仿真语义，只是递给综合器的便签；读代码时把它当作有工具效力的注释即可。

`generate for` 复制结构。以 N 组独立使能的寄存器堆为例：

```
genvar i;
generate
    for (i = 0; i < DEPTH; i = i + 1) begin : gen_regs
        // 每一份复制都是一组独立的触发器
        always_ff @(posedge clk) begin
            if (we[i])
                regs[i] <= wdata;
        end
    end
endgenerate
```

读回硬件：综合后得到 `DEPTH` 组完全相同的触发器排，每组一个使能端，与把

单组代码手工抄 DEPTH 遍一字不差。begin 后的标号 gen_regs 不是装饰：它给每份复制命名，层次路径 gen_regs[3] 会出现在仿真波形和综合报告里，用来定位第 3 份——没有标号，工具只能给自动生成一串无意义的编号。

流水线与脉动阵列这类“N 级相同结构串联”的形态同理，只是级间连线改用数组下标表达——第 i 级的输出接第 $i+1$ 级的输入：

```
genvar k;
generate
  for (k = 0; k < STAGES; k = k + 1) begin : gen_stage
    always_ff @(posedge clk) begin
      if (k == 0)                // k 是编译期常量
        pipe[k] <= sample_in;
      else
        pipe[k] <= pipe[k-1]; // 第 k 级接第 k-1 级
    end
  end
endgenerate
```

展开之后是 STAGES 级触发器排首尾相接，与 § 三的移位寄存器同构，区别只在每级的内容从单 bit 换成一个 8 位样本。generate 不创造新硬件，它只是把重复了一遍又一遍的硬件收拢成一段参数化文字。同族的 generate if 按参数裁剪结构，本专题末尾会给出完整例子。

边界同样由“编译期”三个字决定：generate 的循环界与条件必须是编译期常量——parameter、localparam、genvar——不能是信号。硬件的份数在上板之前就定死了，运行时不能“多生成一份”。运行时要做的是“选择”而不是“生成”：从 N 份里选一份用 mux，那是 § 二组合逻辑的工作，不是 generate 的工作。把这两件事分清，读参数化模块就不会把 for 想成程序循环——它只是原理图复制粘贴的文字形式。

参数化的最后一环在实例化一侧。§ 一说过实例化是放元件；带参数的元件在放置时给出本次的规格——宽度、深度、级数——同一模块名在不同位置可以放出不同规模的硬件。读层次化设计时，模块定义处的 parameter 默认值只是“不传参数时的规格”，真实尺寸以实例化处为准；追踪一个信号的位宽，要一路追到最上层的参数传递，而不是停在模块声明那一行。

把按参数裁剪写成代码，就是 generate if：

```
generate
  if (WIDTH > 1) begin : gen_wide
    // 多位版本：进位逐级串联成链
    assign c_out = c[WIDTH];
  end else begin : gen_bit
    // 单位版本：没有进位链，直接相连
    assign c_out = c_in;
  end
endgenerate
```

条件里只能出现编译期常量，于是裁剪在综合开始前一锤定音：被选中的分支展开成真实电路，被裁掉的分支连一个门都不会剩下——留下的电路里找不到任何“判断”的痕迹。读带 generate if 的模块，顺序是先确定本次实例的参数值，再决定哪一份电路真正存在；另一份只是文字，不是硬件。这与运行时选择形成最后一组对照：if 语句选的是数据走向，generate if 选的是电路有无，一个发生在每一个时钟周期，一个发生在按下综合按钮的那一刻。

本节五类问题合在一起，就是“从 Verilog 读回硬件”的最后一道工序：先确认赋值符号属于哪类硬件，再确认每个分支都有确定电平，再确认写法属于可综合子集，再核对位宽与符号的隐性约定，最后把 generate 在脑子里展开成 N 份真实结构。过完这五关，文字与电路之间再无灰色地带——每一段 RTL 都能说出它是什么硬件、在哪里、有几份。

五、testbench 与仿真阅读

前四节做的是同一件事：把 Verilog 写法读回硬件。本节换一个场景——仿真器。仿真器里没有电压与电流，只有一个按时间推进的事件队列；被测的 RTL 在队列里被逐事件推演，而推动这一过程的环境叫 testbench。testbench 本身不是硬件，也永远不会变成硬件：它是一台“用文字搭出来的测试台”，负责产生时钟与复位、施加激励、观察输出并报告结果。分清“哪些代码要变成电路、哪些代码只在仿真器里工作”，是读懂一切工程仿真的前提。

testbench 是激励环境，不是被综合的硬件 先看机制。仿真器启动时需要一个顶层模块，这个模块没有端口——它对外的接口就是仿真器本身。testbench 正是这个无端口 module：内部用 `logic` 与 `wire` 声明一组信号，实例化被测模块（工程上常称 DUT），把这些信号接到 DUT 的端口上；然后用 `initial` 与 `always` 块按时间驱动 DUT 的输入，用系统任务观察 DUT 的输出：`\protect \TU\textdollar display` 在调用时刻打印一行，`\protect \TU\textdollar monitor` 挂上之后每当所列信号变化就自动打印，`\protect \TU\textdollar fatal` 打印错误并以失败姿态终止仿真（本节“激励与自检的写法”一段细讲）。整个结构可以读成面包板上那套动作的翻版：实例化 DUT 是把芯片插上面包板，驱动输入是拨动开关，观察输出是看 LED。

写法上，一个最小 testbench 骨架只有五个零件：无端口 module、信号声明、DUT 实例化、时钟激励、测试流程。

```
module tb_xxx;                                // 无端口：仿真器从这里启动
  logic      clk, rst_n;                      // 激励：将要接到 DUT 输入端
  wire [7:0] y;                               // 观察：接 DUT 输出端

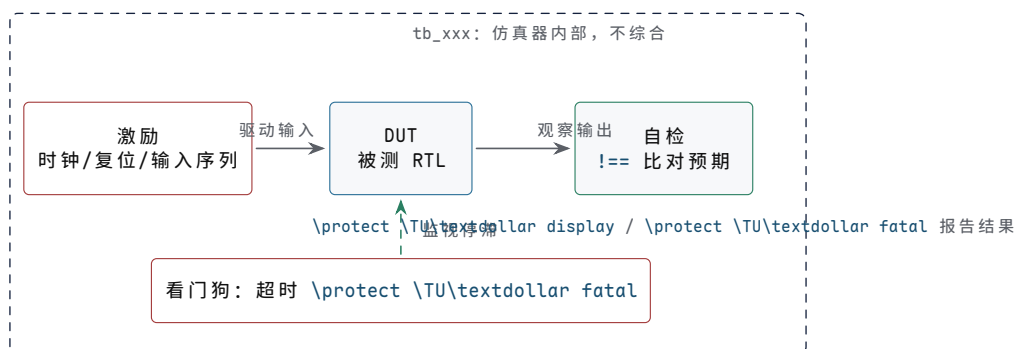
  xxx dut (                                   // DUT：被测模块，可综合的 RTL
    .clk(clk), .rst_n(rst_n), .y(y)
  );

  always #5 clk = ~clk;                       // 时钟激励，下一小节细讲

  initial begin                               // 测试流程：复位、激励、自检
    rst_n = 1'b0;
    #20 rst_n = 1'b1;
    // ... 施加激励、与预期比对 ...
    $finish;
  end
endmodule
```

边界只有一条判据：这段代码要不要变成芯片上的电路。DUT 以及被它实例化的一切，必须遵守前四节的全部综合纪律，每一行都要能读回触发器、组合门与连线；testbench 里的一切则不受约束——`initial`、`#` 延时、`\protect \TU\textdollar display`、文件读写全部合法，因为它们只活在仿真器里，综合器永远看不到。工程上 testbench 还有一项特权：层次化引用，例如 `dut.u_ram.mem[0]` 可以直接伸进 DUT 内部读写某个存储单元。这种写法不产生任何硬件，只是仿真器提供的观察与干预通道，本节后面的案例会用它给整机预置程序。

把这台虚拟实验台画成挂接图，四个部件的分工就是 testbench 的全部结构：



图示: testbench 的四个部件: 激励发生器产生时钟与复位、按时间驱动输入序列; DUT 是边界内唯一要综合的部分, 遵守前四节全部纪律; 自检把输出与预期逐项比对 (`!=` 让 X 也判失败); 看门狗在程序陷入死循环时强制作废本次仿真。四个部件连同虚线边界, 只存在于仿真器里。

时钟与复位的产生 机制: 仿真时间是离散的。骨架里那一行 `always #5` 让 `clk` 每过 5 个时间单位取反一次: 0 时刻为 0, 第 5 单位变 1, 第 10 单位变 0, 再第 15 单位变 1——周期因而是 10 个时间单位, 这是全仿真界通行的约定: # 后面的数字是半周期。文件开头的编译指令 `'timescale 1ns/1ns` 把“单位”定为纳秒, 这条时钟就是周期 10 ns、频率 100 MHz 的方波。这个 `always` 块在硬件里的对应物是第七章的 555 振荡器: 一个不依赖别人、自己永远走下去的激励源。

复位的写法同样对应面包板上的复位按钮: 上电即按下, 按住若干拍, 再松开。

```
initial begin
    rst_n = 1'b0;           // 上电即复位
    repeat (5) @(negedge clk); // 按住 5 拍
    rst_n = 1'b1;           // 在时钟低电平期间释放
end
```

拉低若干拍再释放, 是为了让 DUT 里每个触发器都至少经历一次复位、进入已知初态——这正是第七章“复位覆盖全部时序状态”那一课的仿真版: 不复位, 寄存器初值全是 X, 波形从第一拍起就不可读。选在低电平期间释放是个小习惯: 让释放动作远离时钟沿, 避免与 DUT 的时序块争抢同一时刻。

边界: # 延时与 `repeat` 等待只在 testbench 里合法。RTL 中不存在“等 5 ns”的电路——§ 四讲不可综合写法时已经说过, 综合器面对延时语句要么报错要么悄悄忽略, 两者都危险。真实硬件的时钟来自晶振或 555, 复位来自 RC 电路或按钮; 仿真里的这两三行是它们的替身, 而不是对它们的描述。

激励与自检的写法 机制: 激励是按时间施加到 DUT 输入端的值序列; 自检是让 testbench 自己把实际输出与预期值逐项比对, 只在最后报告结论。人眼读波形只能算抽查——波形越长, 漏看越多; 能自动比对的测试才谈得上可重复。三个系统任务的分工前面已经登记: `\protect \TU\textdollar display` 报进展, `\protect \TU\textdollar monitor` 追变化, `\protect \TU\textdollar fatal` 报错退出, 脚本凭退出状态判失败。

写法: 把激励与预期都预置成数组, 逐项驱动、逐项比对。以测一个 8 位加法器 (端口 `a`、`b`、`y`) 为例:

```
logic [15:0] stim [0:3];      // {a, b} 激励对
logic [7:0] gold [0:3];      // 预期和
integer i, errs;
initial begin
    stim[0] = {8'd3, 8'd4};    gold[0] = 8'd7;
    stim[1] = {8'd9, 8'd9};    gold[1] = 8'd18;
    stim[2] = {8'd200, 8'd100}; gold[2] = 8'd44; // 溢出回绕
    stim[3] = {8'd0, 8'd0};    gold[3] = 8'd0;
    errs = 0;
    for (i = 0; i < 4; i = i + 1) begin
```

```
{a, b} = stim[i]; #10;    // 驱动后等组合路径稳定
if (y !== gold[i]) begin
    $display("FAIL: i=%0d y=%0d expect=%0d", i, y, gold[i]);
    errs = errs + 1;
end
end
if (errs == 0) $display("ALL PASS");
else          $fatal(1, "%0d errors", errs);
$finish;
end
```

注意比对用 **!==**（区分 X 的严格不等）：输出哪怕是 X，也会被正确地判成失败。若用普通的 **!=**，含 X 的比较结果是“不确定”，错误就从指缝里漏过去了。

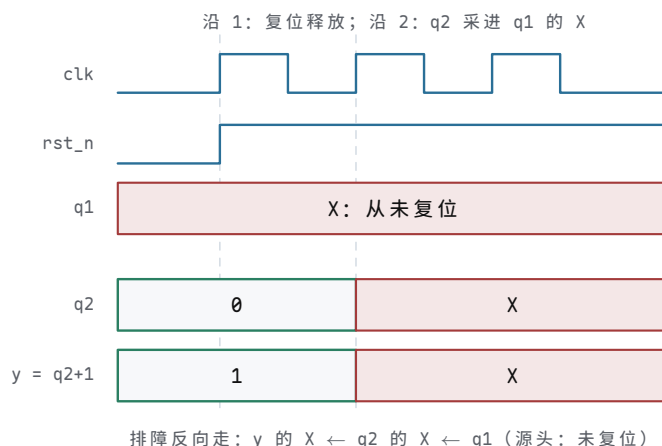
规模再大一些的工程用 golden 参考：同一模块写两份，一份是行为级的“会说话的规格”——怎么好懂怎么写，一份是 § 一到 § 四风格的 RTL——怎么可综合怎么写；同一份激励同时喂给两者，逐拍比对输出。此时预期值不再由人手工计算，而是由行为模型现场给出，RTL 每改一版就自动重测一轮。边界要记牢：自检的质量不会超过预期值的质量——预期算错，全部 PASS 也是假象；预期表的第一读者永远是人。

波形怎么读 波形窗口是三样东西的并排：纵轴是信号名，横轴是仿真时间，中间是每个信号的取值轨道。单比特信号画高低线，多比特总线画成首尾尖起的块、块内标注当前值。读波形先认四种特殊显示：

显示	含义	常见根源
X（通常红色）	未知态：仿真器不知道该值是 0 还是 1	寄存器没复位、存储器没初始化、多驱动冲突
Z（通常蓝色细线）	高阻：此刻没有任何驱动者	三态门全部关闭，如第七章空拍时的总线
总线值突然变 X	两个驱动者给出相反电平	总线争用：同一拍开了两个“出”使能
时钟沿上的瞬时细线	零延迟模型中的事件竞争	同一时刻多个块读写同一信号

最有用的排障手法是顺着 X 往回走：找到第一个变 X 的信号，看它的驱动源；驱动源若也是 X，就再往上追一层。链条的尽头几乎总是同一个地方——一个没接复位的触发器，或一块没写初值的存储器。这与第七章上电检查的第一条互为镜像：真机上“复位后全部寄存器应为已知值”，仿真里“复位释放后波形不应再出现 X”。Z 的读法则反过来：看到 Z 先别慌，问“此刻谁该驱动”。答案若本来就是“没人”——例如第七章空拍时的总线——Z 是正常；答案若该有人驱动，那就是使能链断在了某一级。

把“顺着 X 往回走”画成波形：q1 没接复位，它的 X 在一个沿之后传染给 q2，再顺着组合路径染到输出 y：



图示: X 沿数据路径的传播链: q1 从未复位, 恒为 X; 沿 2 这个 X 被采进 q2, 组合输出 y 随即跟着变 X。排障沿链反向走—先看到 y 变 X, 追到 q2, 再追到源头 q1: 一只没接复位的触发器。rst_n 对 q2 有效、对 q1 缺席, 正是本图假设的接线; 给 q1 补上复位, 整条链在第一拍之后就不再有 X。

案例：把第七章教学计算机写进仿真 现在把全部零件合起来。目标：用 § 一到 § 四的写法，把第七章那台 8-bit 总线机器写成 RTL；再用本节的写法给它配 testbench——把第七章程序一（从 1 加到 4）预置进 RAM，让仿真自己跑到 HLT，并自动核对 OUT 依次显示 1、3、6、10。

机器的核心是控制单元，而第七章 § 四已经给出了它的全部内容：一张以 {opcode, T} 为地址的微码表。RTL 化几乎是逐行誊写：每个 T 状态一个分支，每根有效控制线一个赋值。数据通路沿用第七章的总线结构——任一时刻至多一个驱动器，空拍时总线高阻。先写存储器：

```
module sap1_ram (                                // 16 字节：程序与数据同住
    input logic      clk,
    input logic      we,          // 即控制线 RI
    input logic [3:0] addr,       // 即 MAR 的输出
    input logic [7:0] din,
    output logic [7:0] dout
);
    logic [7:0] mem [0:15];

    always_ff @(posedge clk) begin // 写：时钟沿、RI 有效时
        if (we) mem[addr] <= din;
    end

    always_comb begin              // 读：组合读出，相当于常开的 R0 三态门
        dout = mem[addr];
    end
endmodule
```

注意 RAM 没有复位端：复位清的是机器状态，不是程序——面包板上拨进去的字节，也不会被复位按钮抹掉。整机如下，控制线与第七章 § 四逐根同名：

```
module sap1_machine (
    input logic      clk,
    input logic      rst_n,
    output logic [7:0] out,      // OUT 寄存器，接显示排
    output logic      halted    // HLT 后时钟冻结
);
    logic [7:0] a, b, ir, bus;
    logic [3:0] pc, mar;
    logic [2:0] t;              // T 状态计数器：T0..T7 循环
    logic      zf, cf;
    logic [7:0] ram_dout;
```



```

logic [3:0] op;
logic [8:0] alu9;                                // 第 9 位就是进位

// “出”使能（谁驱动总线）与“入”使能（谁在这一沿装载）
logic co, ro, ao, io, so;
logic mi, ii, ce, ai, bi, oi, ri, pi, fi, hlt;

assign op = ir[7:4];

// 总线：任一时刻至多一个驱动器；无人驱动时为高阻（第七章的空拍）
assign bus = co ? {4'h0, pc} :
            ro ? ram_dout :
            ao ? a :
            io ? {4'h0, ir[3:0]} :
            so ? alu9[7:0] : 8'hzz;

// ALU：减法是取反加一（第三章）
always_comb begin
    if (op == 4'h3) alu9 = {1'b0, a} + {1'b0, ~b} + 9'd1;
    else            alu9 = {1'b0, a} + {1'b0, b};
end

// 微码译码：第七章微码 ROM 的 RTL 抄本，行地址是 {op, t}
always_comb begin
    co = 0; ro = 0; ao = 0; io = 0; so = 0;
    mi = 0; ii = 0; ce = 0; ai = 0; bi = 0;
    oi = 0; ri = 0; pi = 0; fi = 0; hlt = 0;
    case (t)
        3'd0: begin co = 1; mi = 1; end // T0: MAR <- PC
        3'd1: begin ro = 1; ii = 1; ce = 1; end // T1: IR <- RAM[MAR], PC <- PC+1
        3'd2: case (op) // T2: 地址场 / 立即数 / 跳转
            4'h1, 4'h2, 4'h3, 4'h4: begin io = 1; mi = 1; end
            4'h5: begin io = 1; ai = 1; end // LDI: A <- 立即数
            4'h6: begin io = 1; pi = 1; end // JMP: PC <- 地址场
            4'h7: begin io = 1; pi = cf; end // JC: cf=1 才跳
            4'h8: begin io = 1; pi = zf; end // JZ: zf=1 才跳
            default: ;
        endcase
        3'd3: case (op) // T3: 访存与装载
            4'h1: begin ro = 1; ai = 1; end // LDA: A <- RAM[MAR]
            4'h2, 4'h3: begin ro = 1; bi = 1; end // ADD/SUB: B <- RAM[MAR]
            4'h4: begin ao = 1; ri = 1; end // STA: RAM[MAR] <- A
            4'hE: begin ao = 1; oi = 1; end // OUT: OUT <- A
            4'hF: hlt = 1; // HLT: 冻结
            default: ;
        endcase
        3'd4: case (op) // T4: 写回 A 与标志
            4'h2, 4'h3: begin so = 1; ai = 1; fi = 1; end
            default: ;
        endcase
        default: ; // T5..T7: 空拍，总线高阻
    endcase
end

// 全部寄存器共用一个时钟沿；“入”使能决定谁在这一沿装载
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        pc <= 0; mar <= 0; ir <= 0; a <= 0; b <= 0;
        out <= 0; t <= 0; zf <= 0; cf <= 0; halted <= 0;
    end else if (!halted) begin

```

```

    t <= t + 3'd1;
    if (mi) mar <= bus[3:0];
    if (ii) ir  <= bus;
    if (ce) pc  <= pc + 4'd1;
    if (pi) pc  <= bus[3:0];
    if (ai) a   <= bus;
    if (bi) b   <= bus;
    if (oi) out <= bus;
    if (fi) begin zf <= (alu9[7:0] == 8'h00); cf <= alu9[8]; end
    if (hlt) halted <= 1'b1;
end
end

sap1_ram u_ram (
    .clk(clk), .we(ri), .addr(mar), .din(bus), .dout(ram_dout)
);
endmodule

```

对照第七章的逐拍表读这段代码：T0 的 **co+mi** 就是 **CO+MI**，T1 的 **ro+ii+ce** 就是 **RO+II+CE**，ADD 的 T3、T4 正是 **RO+BI** 与 **ΣO+AI+FI**。微码 ROM 在面包板上是一片 EEPROM，在这里是一个组合逻辑块——同一张真值表的两种硬件实现，行为完全一致。**ce** 与 **pi** 永远不同时有有效，所以 PC 的两个装载条件互斥；这份互斥不是语法保证的，是微码表保证的，与第七章“出使能互斥”完全同构。

testbench 负责剩下的一切：时钟、复位、预置程序（对应编程模式拨码）、自检，外加一只看门狗。

```

`timescale 1ns/1ns
module tb_sap1;
    logic      clk    = 1'b0;
    logic      rst_n  = 1'b0;
    wire [7:0] out;
    wire      halted;

    sap1_machine dut (           // 被测整机
        .clk(clk), .rst_n(rst_n), .out(out), .halted(halted)
    );

    always #5 clk = ~clk;        // 时钟：周期 10 ns

    initial begin                // 复位：拉低 5 拍后释放
        repeat (5) @(negedge clk);
        rst_n = 1'b1;
    end

    initial begin                // 预置程序：第七章程序一逐字节抄录
        dut.u_ram.mem[ 0] = 8'h1F; // LDA 15   A <- sum
        dut.u_ram.mem[ 1] = 8'h2E; // ADD 14   A <- sum + i
        dut.u_ram.mem[ 2] = 8'h4F; // STA 15   sum 写回
        dut.u_ram.mem[ 3] = 8'hE0; // OUT      显示部分和
        dut.u_ram.mem[ 4] = 8'h1E; // LDA 14   A <- i
        dut.u_ram.mem[ 5] = 8'h3D; // SUB 13   i - 4, 判顶
        dut.u_ram.mem[ 6] = 8'h8B; // JZ 11    到顶即停机
        dut.u_ram.mem[ 7] = 8'h1E; // LDA 14
        dut.u_ram.mem[ 8] = 8'h2C; // ADD 12   i + 1
        dut.u_ram.mem[ 9] = 8'h4E; // STA 14   i 写回
        dut.u_ram.mem[10] = 8'h60; // JMP 0    回循环头
        dut.u_ram.mem[11] = 8'hF0; // HLT
        dut.u_ram.mem[12] = 8'h01; // 常数 1
        dut.u_ram.mem[13] = 8'h04; // 上限 4
    end
endmodule

```

```

    dut.u_ram.mem[14] = 8'h01;    // i 初值 1
    dut.u_ram.mem[15] = 8'h00;    // sum 初值 0
end

// 自检：OUT 每装入一个新值，就与预期序列对一项
logic [7:0] expect [0:3];
logic [7:0] out_d;
integer seen;
initial begin
    expect[0] = 8'd1; expect[1] = 8'd3;
    expect[2] = 8'd6; expect[3] = 8'd10;
    out_d = 8'h00;    // 复位后 OUT 的已知初值
    seen = 0;
end

always @(negedge clk) begin    // 在时钟低电平期间检查，避开沿上竞争
    if (rst_n && (out != out_d)) begin
        out_d = out;
        if (seen > 3)
            $fatal(1, "FAIL: extra OUT write %0d", out);
        else if (out != expect[seen])
            $fatal(1, "FAIL: step %0d expect %0d got %0d", seen, expect[seen], out);
        else
            $display("[%0t ns] OUT = %0d PASS step %0d", $time, out, seen);
        seen = seen + 1;
    end
end

initial begin    // 收尾：HLT 后核对输出次数
    wait (halted);
    repeat (2) @(negedge clk);
    if (seen == 4) $display("TEST PASS: OUT sequence 1 3 6 10, %0d checks", seen);
    else
        $fatal(1, "FAIL: halted after %0d outputs", seen);
    $finish;
end

initial begin    // 看门狗：程序死循环时强制作废本次仿真
    #20000;
    $fatal(1, "FAIL: timeout, machine never halted");
end
endmodule

```

这份 testbench 把前面几个写法一次用全：时钟与复位的产生、层次化预置（`dut.u_ram.mem` 就是编程模式的拨码）、数组化的预期值、`\protect \TU\textdollar fatal` 与 `\protect \TU\textdollar display` 自检；看门狗保证微码若有错、程序陷入死循环时，仿真不会无限空转。在 1 ns 时标、复位拉低 5 拍的这份激励下，仿真输出为：

```

[330 ns] OUT = 1 PASS step 0
[1210 ns] OUT = 3 PASS step 1
[2090 ns] OUT = 6 PASS step 2
[2970 ns] OUT = 10 PASS step 3
TEST PASS: OUT sequence 1 3 6 10, 4 checks

```

四次 OUT 与第七章的逐圈表逐项吻合；机器执行 41 条指令后在 HLT 的 T3 冻结，从复位释放到冻结共 324 拍。面包板上要按三百余次单步才能看完的过程，仿真器里只是一瞬间——但两者跑的是同一张微码表、同一份程序、同一个输出序列。仿真不是另一台机器，是同一台机器的理想化影子。

仿真与真实硬件的差别 仿真通过之后，最容易产生的错觉是“硬件也就这样了”。三处差别必须记牢。第一是零延迟的理想时序：RTL 仿真里组合逻辑在零时间内完成，所有触发器在同一个时钟沿同时更新；第八章讲的传播延迟、建立与保持时间、时钟偏斜，在这幅画面里根本不存在——它们要换成标注了真实延时的门级仿真才看得见，或者上板以后由示波器告诉你。第二是电气问题不可见：仿真世界里信号只有 0、1、X、Z 四种取值，没有电压与电流；电源跌落、串扰、短路发热、毛刺携带的能量，统统无法在逻辑仿真里现身。第三是 X 的语义边界：X 的意思是“仿真器不知道”，而不是某种物理电压。真实芯片上电后，每一位总是确定的 0 或 1（或正在翻转的亚稳态），只是你无法预知；仿真却可能因若干巧合，让这份“不知道”从未传播到输出——错误被 X 的乐观藏了起来，仿真通过而硬件照样出错。

仿真里的世界	真实硬件	补课章节
组合逻辑零延迟，沿上同时更新	传播延迟、建立与保持时间、时钟偏斜	第八章
信号只有 0/1/X/Z	电压、电流、串扰、发热	第一章、第八章
X 表示“不知道”，可以不传播	每位总是确定的 0 或 1（或亚稳态）	第二章、第八章
时钟与复位几行代码就有	晶振、复位电路要单独设计	第七章、第八章

所以仿真通过的正确定位是：“可以上板”的必要条件，而不是充分条件。它保证的是逻辑意图被正确地翻译成了 RTL；时序是否收敛、电气是否健康，是另外两关，分别在时序分析与第八章的测量里。把这三关的顺序搞反——先上板、出了怪事再回头怀疑逻辑——是调试时间最大的浪费。

章末练习

任务	要求	学习目标
module 读图	阅读 <pre>module m (input logic clk, en, rst , input logic [7:0] d, output logic [7:0] q); always_ff @(posedge clk) begin if (rst) q <= 8'h00; else if (en) q <= d; end endmodule</pre> 在纸上画出它对应的框图：标明每个元件的种类、位宽与全部连线，并指出哪部分电路对应 if (rst)、哪部分对应 if (en)	module 逐行读回硬件
位宽陷阱	<pre>logic [3:0] a, b; logic c; assign c = (a + b) > 4'd15;</pre> 想检测 4 位加法的进位。说明 c 的实际行为与原因，并给出两种正确写法	位宽由语境决定
锁存器消除	<pre>always_comb begin if (sel) y = a; end</pre> d 会推断出什么硬件？为什么会这样？给出两种消除写法，并说明两者综合出的电路	组合块每条路径都赋值
非阻塞交换	复位后 $a = 8'd1$ 、 $b = 8'd2$ 。块 <pre>always_ff @(posedge clk) begin a <= b; b <= a; end</pre> 运行两拍后 a、b 各是多少？若把 <= 全换成 = 呢？两种写法各自对应什么电路	非阻塞 = 先读旧值再同时提交

两段式 FSM 改写	第二章交通灯 FSM 曾把状态转移与灯输出全写在一个 <code>always_ff</code> 里（一段式）。参照本章 § 三的两段式模板，写出改写后两个块的骨架（注明每块的类型与内容），并说明灯输出现在属于哪类电路	状态寄存器与次态逻辑分家
testbench 自检写法	为本章 § 五的 <code>sap1_ram</code> 写自检 testbench 的核心部分：向地址 7 写入 <code>8'hA5</code> 再读回，比对不符时用 <code>\protect \TU\textdollar fatal</code> 报告，要求不依赖人眼读波形。列出生成时钟、驱动写读、自检三段各自的写法	预期 + 比对 + 报错才算自检

参考解答与要点

任务	参考解答	必须掌握的要点
module 读图	框图：8 个 D 触发器并联成 8 位寄存器，时钟端共接 <code>clk</code> ；每个触发器的 D 输入前串两级 2 选 1 选择器——最靠近 D 端、数据最后经过的一级由先写的 <code>rst</code> 控制， <code>rst=1</code> 时选常数 0；靠外的一级由 <code>en</code> 控制， <code>en=0</code> 时把 <code>q</code> 接回自身（保持）， <code>en=1</code> 时放行 <code>d</code> 。全部动作同步于时钟沿，无锁存器、无异步逻辑；端口 4 入 1 出，无隐藏状态	<code>always_ff</code> 里每层 <code>if</code> 都是 D 端前的一级 mux，先写的条件优先级最高，控制最靠近数据输入端（最后选择）的那一级
位宽陷阱	<code>c</code> 恒为 0。比较 <code>></code> 的两端位宽取语境最大值：左端 <code>a+b</code> 为 4 位、右端 <code>4'd15</code> 为 4 位，于是加法按 4 位计算，第 5 位进位在比较前已被丢弃，和最大 15，永远不大于 15。改正一： <code>assign {c, s} = a + b;</code> ，5 位拼接作接收端，加法按 5 位算，进位自然进 <code>c</code> ；改正二：先把加数显式扩展再比， <code>({1'b0, a} + {1'b0, b}) > 5'd15</code>	表达式位宽取语境最大；进位只在接收端够宽时才存在
锁存器消除	<code>sel=0</code> 的路径上 <code>y</code> 没有被赋值，”保持旧值”在组合逻辑里没有免费实现，综合器只好推断出一级锁存器： <code>sel</code> 成了它的使能端。消除法一：块首加默认赋值 <code>y = 1'b0;</code> ，让后面的 <code>if</code> 覆盖它；法二：补全分支 <code>else y = 1'b0;</code> 。两种写法综合出的电路相同：一个由 <code>sel</code> 选择的 2 选 1 mux	锁存器来自”某条路径没赋值”，与 <code>always_comb</code> 本身无关
非阻塞交换	非阻塞版：右端全部读旧值、时钟沿同时提交——一拍后 <code>a=2, b=1</code> ，两拍后 <code>a=1, b=2</code> ，每拍交换一次；对应电路是两只触发器 D 端交叉互连。阻塞版 <code>a = b; b = a;</code> 顺序生效： <code>a</code> 先得 2， <code>b</code> 再取到新的 <code>a</code> 也是 2——一拍后 <code>a=b=2</code> ，两拍后仍是 2、2；对应电路是两只触发器的 D 端同接 <code>b</code> ，交换被同化	<code><=</code> 的语义就是触发器的并行装载；时序块里的交换必须用非阻塞

两段式 FSM 改写	骨架: <code>always_ff @(posedge clk or negedge rst_n) if (!rst_n) state <= S0; else state <= next_state;;</code> <code>always_comb</code> 里块首先给 <code>next_state</code> 与各路灯输出默认赋值, 再 <code>case (state)</code> 逐状态计算次态与输出。灯输出现在是纯组合电路 (对状态的译码), 与状态寄存器分家、不再占用触发器; 一段式里它们是被时钟沿寄存的	时序块只装状态寄存器; 次态与输出全部组合
testbench 自检写法	时钟段沿用 § 5 骨架: <code>always #5 clk = ~{clk};</code> 。驱动段写在 <code>initial</code> 里: <code>addr = 4'd7; din = 8'hA5; we = 1'b1;</code> , 等一个上升沿后 <code>we = 1'b0;</code> 。自检段: 再隔一拍读回, <code>if (dout != 8'hA5) \protect \TU\textdollar fatal(1, "RAM FAIL");</code> 否则 <code>\protect \TU\textdollar display("PASS"); \protect \TU\textdollar finish;</code> 。激励按拍推进, 预期值 <code>8'hA5</code> 写进比对语句, 严格不等 <code>!=</code> 让 X 也判失败	预期值 + 逐项比对 <code>+\protect \TU\textdollar fatal</code> 三件套; 眼看波形不算自检

性能分析与定量方法

前面九章都在回答” 机器怎么工作”，本章回答” 机器有多快”。快不是一句” 主频高” 能概括的：同一块芯片跑不同程序可以差几十倍，同一个程序换一台机器不一定等比例变快。性能分析有一套固定工具：先分清时延和吞吐，再把总时间拆成可计算的因子，用 Amdahl 定律估计优化的上限，用 roofline 判断瓶颈在算力还是带宽，最后用测量代替猜测。本章把这套工具全部走成数值例子。

工具	回答的问题	常见误用
时延/吞吐拆分	用户等多久 vs 系统每秒做多少	用吞吐高掩盖时延长
总时间因子	指令数、CPI、周期各占多少	只优化占比小的因子
Amdahl 定律	优化一部分能换来多少整体加速	忘记未优化部分的拖累
roofline	瓶颈在算力还是在带宽	拿峰值算力当可达性能

核心思想

性能分析的铁律是” 先测量，再优化，最后验证”。一切没有测量支持的优化都是猜测，一切不回测的优化都可能让系统更慢。

一、时延、吞吐与利用率

本节先把两个最基本的量分开：时延回答” 一件事要等多久”，吞吐回答” 系统每秒能完成多少件事”。两者的原料相同——都是完成一件事所花的时间——却不是同一个量，在许多设计里甚至朝相反的方向走。本节先给定义与单位，用四种组合说明两个量可以自由搭配；再给出利用率接近饱和时时延发散的定量规律；然后重访第四章的流水线，看它到底改善了哪一个量；最后用第五章的教学总线和第七章的教学机各算一笔完整的账。

时延与吞吐是两个不同的量 时延（latency）是一个任务从发起到完成所经过的时间，单位是” 秒/条”——秒每指令、秒每请求、秒每次访问，回答” 用户等多久”。吞吐（throughput）是单位时间内完成的任务数，单位是” 条/秒”——指令每秒、请求每秒、字节每秒，回答” 系统每秒做多少”。一个是单个任务的视角，一个是系统管理者的视角。经典教材的说法是：在意响应时间的用户该买更快的处理器，在意吞吐的管理员该买更多的处理器。

量	定义	典型单位	回答的问题
时延	一个任务从发起到完成的时间	秒/条、秒/次、ns	用户等多久
吞吐	单位时间完成的任务数	条/秒、MB/s	系统每秒做多少

两个量的关系只在一个特殊情形下才简单：系统完全串行、无重叠、无排队时，吞吐恰好等于时延的倒数。一旦允许重叠——流水线、并行单元、批量传输——吞吐就可以脱离时延单独增长。把一件事切成 n 段流水执行，单件事的时延不变，吞吐最多放大 n 倍；把系统原样复制 n 份并排，吞吐再放大 n 倍，单件事的时延还是不变。这组不对称有物理根源：时延的下限由物理决定，信号要走完这段距离、电平要穿过这些逻辑门，堆资源降不动；吞吐的上限主要由资源决定，加并行度就能买。所以” 更快” 在工程上必须拆开问：是单件更快，还是每秒更多。

四种组合因此全都存在，各配一个真实例子：

组合	真实例子	为什么能这样搭配
快时延、高吞吐	乱序超标量服务器 CPU	深流水加多发射：单条指令纳秒级完成，每秒退出上百亿条
快时延、低吞吐	同城专人直送快递	一单直达、半小时送到，但一人一次只能带一单，一天送不了几单
慢时延、高吞吐	远洋集装箱货轮	跨洋要一个月，一船装两万个箱子，折算每天的吞吐相当可观
慢时延、低吞吐	邮政平信	一封信走上好几天，一次也只送这一封

后三行在硬件里各有对应。中断控制器处理单个事件的时延很短，但一次只处理一个事件，事件率一高就得排队，是”快时延、低吞吐”；磁带冷备份定位一盘带要几十秒，持续写入却是每秒几百兆字节、一夜能备份一个机房，是”慢时延、高吞吐”；机械电传打字机每个字符都要上百毫秒、每秒也只有几个字符，是两头都不占。判断一个系统属于哪一格，必须把两个量分别测出来——一个数字定不了位置。也要提醒一句：给”专人直送”加人手能提高每天的单量，却缩短不了任何一单的路程，这就是用并行度买吞吐、买不来时延的日常版本。

把这张表摆成坐标系，四格各居其位——两个量可以自由搭配，一格也不空：



图示：时延与吞吐的四象限：纵轴时延上快下慢，横轴吞吐左低右高。四格各有真实居民——快而少（专人直送）、快而多（超标量 CPU）、慢而多（集装箱货轮）、慢而少（邮政平信）。判断一个系统属于哪一格，两个量必须分别测出来，一个数字定不了位置。

广告为什么只报一个数。原因之一是一个数字好传播，”每秒 10 亿次”远比”在什么负载下、时延多少、吞吐多少”好记；原因之二是厂商总是挑有利的那一个报：内存条把带宽印在包装上、把 CL 时延藏进规格书深处，云服务报每秒请求数、不报第 99 百分位时延，手机芯片报峰值算力、不报持续负载下的实测值。读完本节的读者应养成一个反射：听到任何性能数字，先问它是时延还是吞吐，再问另一个是多少、在什么负载下量的。

还有一点必须补在定义之后：时延本身也不是一个数。同一台机器上，同类请求的时延会波动——第五章已经给出过来源：cache 命中与否、DRAM 行命中与否、队列里排没排队。所以谈时延要谈分布，至少给平均值加一个百分位。平均值会撒谎的例子随手就能造：100 次访问里 99 次各花 20 ns、1 次花了 2.02 μ s，平均值是整齐的 $4000/100 = 40$ ns，看上去一切正常，可那一次请求比平均值慢了 50 倍。云服务把第 99 百分位时延写进服务等级、而不是只写平均值，就是因为用户记住的是最慢的那一次，不是平均的那一次。吞吐没有这个问题：它本来就是总量对时间的平均，一个数基本够用——这又是两个量性格不同的一处表现。

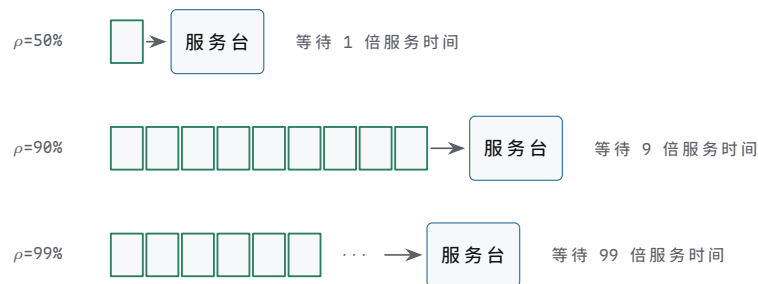
利用率接近 1 时延会发散 总线仲裁器、内存控制器、中断处理程序，抽象以后是同一个东西：一个服务台。请求以随机节奏到达，服务台以固定能力逐个处理。定义利用率 $\rho = \text{到达率} / \text{服务率}$ ，即服务台有多忙。 $\rho < 1$ 时系统跟得上， $\rho \geq 1$ 时队列无限变长，这是常识。不常识的部分是：即使 ρ 离 1 还有距离，时延也已经开始非线性膨胀，而且比直觉快得多。

排队论中最简单的 M/M/1 模型（泊松到达、指数服务时间、单服务台）给出定量结论，本书只引用、不推导：

平均排队等待 = 服务时间 $\times \rho / (1 - \rho)$
 平均停留时间 = 服务时间 $\times 1 / (1 - \rho)$ （等待 + 服务本身）

利用率 ρ	平均排队等待	平均停留时间	直观画面
50%	1 倍服务时间	2 倍服务时间	偶尔要等，等得不久
90%	9 倍服务时间	10 倍服务时间	大部分时间前面有队
99%	99 倍服务时间	100 倍服务时间	服务本身只占停留的零头

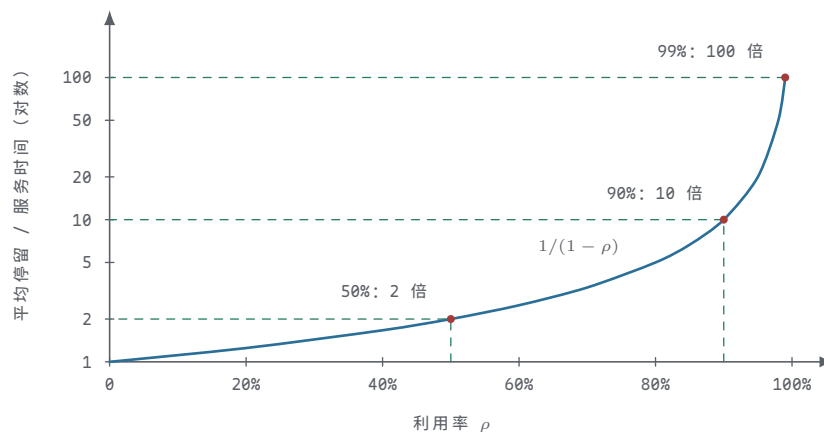
把三档利用率画成服务台前的队伍，表里的 1、9、99 倍就成了队伍的长度：



图示：同一个服务台在三档利用率下的平均队列（小方块为排队请求，对应上表的平均排队等待，不含服务本身）：50% 时平均 1 个在等，90% 时 9 个，99% 时 99 个。服务能力从一半用到九成，队伍却长了 8 倍——空闲时省下的能力存不起来，突发一到就只能排队消化。

从 50% 到 90%，服务能力多用了不到一倍，平均等待却放大 9 倍；从 90% 到 99%，再挤出一成能力，等待又放大 11 倍。代入具体数字感受一次：设服务时间为 10 ns（一次行命中 DRAM 访问的量级），到达率从服务率的一半提到九成，平均排队等待就从 10 ns 涨到 90 ns，平均停留从 20 ns 涨到 100 ns——服务本身一纳秒都没变慢，慢的全是排队。直觉解释只有一句：服务台空闲时省下的能力存不起来，空转的十分钟不能预支给下一波突发用；而到达是随机的，突发必然出现。利用率越接近 1，系统越没有余量吸收突发，队伍一旦排起来要很久才消化得掉，平均等待于是按 $1/(1 - \rho)$ 发散。

把这张表画成曲线，发散的陡峭就成了形状而不是形容词：



图示：M/M/1 模型下平均停留时间随利用率按 $1/(1-\rho)$ 发散，纵轴取对数。利用率 50%、90%、99% 对应 2、10、100 倍服务时间；曲线越往右越陡，末段几乎直立—服务本身没变，涨的全是排队。

工程含义可以写成一条纪律：任何共享服务台都要保持余量，稳态利用率压在五到七成，不要拿峰值能力去对标平均负载。三条具体建议，分别对应本书已经见过的三处硬件。

总线：仲裁与带宽按峰值负载的一倍半预留。第五章 DMA 周期窃取的例子就是这种余量思想—软盘搬运只占总线约 6% 的时间，其余 94% 留给 CPU 取指，CPU 的访存时延才不会被搬运拖长。改用块模式时搬运本身更快，却总线利用率逼近饱和，CPU 取指的排队时延立刻沿上面的曲线上行—第五章说块模式要与周期窃取权衡，定量的理由就是这张表。

内存控制器：它的“服务时间”本身不固定—行命中快、行冲突慢。负载没有变、行命中率下降，等价于服务时间变长、 ρ 上升，排队会突然恶化；有些机器“负载没怎么涨、响应突然变慢”，常见根因就在这里。工程上有两个做法：监控请求队列深度，它比带宽百分比更早报警；用写缓冲吸收突发写、给读请求更高优先级，因为读的后面有一个正在停顿的 CPU 在等，写通常没有。

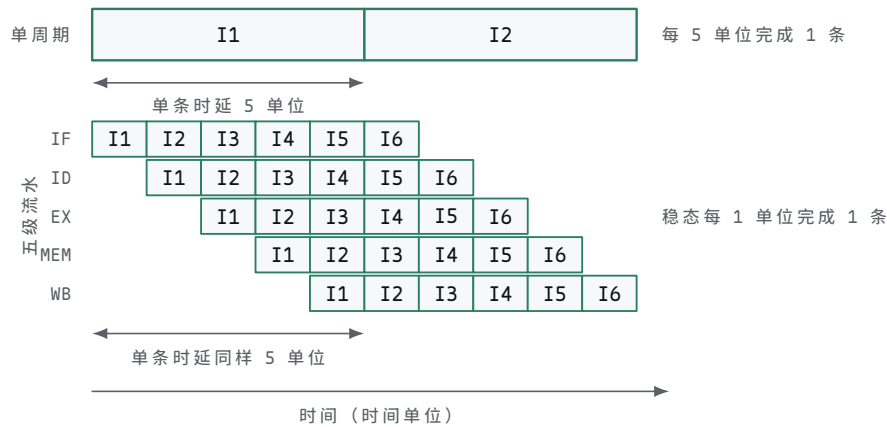
中断：中断到达是典型的随机事件，处理速率必须明显高于峰值到达率，经验上至少留出一倍余量，否则一次中断风暴就把响应时延推上发散曲线。处理程序只做最必要的事—读状态、记标志，重活推迟到后台批量处理；网卡的中断合并（一次中断处理一批包）本质上是把到达率除以批大小，直接降低 ρ ，是同一规律的应用。

最后把“保持余量”这句话说透：余量买的不是平均速度，而是时延的可预期性。 ρ 从 0.7 提到 0.9，吞吐只多了不到三成，平均等待却从 2.3 倍服务时间涨到 9 倍—把三成的能力留作余量，等价于把 $1/(1-\rho)$ 的值压在 10 以内。对终端用户来说，“每次都在 100 ns 内返回”往往比“平均 60 ns、偶尔 1 ms”值钱得多：前者来自余量，后者来自把利用率拉满。批处理是光谱的另一极：把到达攒成一批再服务，等效于服务率翻倍、 ρ 减半，代价是攒批本身增加时延—吞吐与时延的交换在这里又出现了一次，只是换成了排队的语言。也要声明适用范围：真实到达不总是泊松过程，服务时间也不总是指数分布，M/M/1 的具体数字会随之变化；但“利用率逼近 1 则时延发散”这个单调性结论，对几乎所有到达与服务分布都成立，这正是它能成为工程纪律的原因。

流水线提高吞吐不改变时延 第四章用同一个 6 条指令的小程序把三种组织算过一遍：每段组合路径 1 个时间单位时，单周期 30 单位、多周期 24 单位、理想流水 11 单位。当时的重点是“流水为什么快”；本节换一个问法：流水线里一条指令自己的时延是多少？答案是不变—从取指到写回依然 5 级、5 个单位，与多周期机的加载指令相同，与单周期机也相同。变快的不是任何一条指令，而是“每拍完成一条”的稳态节奏。把两个量分列出来：

组织	单条指令时延	6 条程序总时间	稳态吞吐
单周期	5 单位	30 单位	0.20 条/单位
多周期	3 至 5 单位	24 单位	0.25 条/单位
理想流水	5 单位	11 单位	短程序 $6/11 \approx 0.55$ ，长程序趋近 1

把单周期与五级流水画到同一条时间轴上，“时延不变、吞吐改变”就直接可见：



图示：单周期与五级流水执行同一指令流的对照。两种组织里，一条指令从取指到写回都要 5 个时间单位；不同的是完成节奏——单周期每 5 个单位退出 1 条，流水灌满后每 1 个单位退出 1 条。流水线买到的是吞吐，不是单条时延。图中画的是无冒险的理想情形，6 条指令 10 个单位走完；上表“理想流水 11 单位”另含 1 拍 load-use 停顿，比图示多出的 1 个单位正是它。

流水的吞吐在短程序上还没灌满，程序越长越接近每单位 1 条；但无论灌没灌满，单条时延始终是 5 个单位。写成公式：

$$\begin{aligned} \text{单条指令时延} &\approx \text{级数} \times \text{级延迟} \\ \text{稳态吞吐} &\approx 1 / \text{级延迟} \quad (\text{无冒险的理想情形}) \end{aligned}$$

两个式子里只有一个含级数。把 5 级改成 10 级、级延迟减半：单条时延 $10 \times 0.5 = 5$ ，原地不动；吞吐从每单位 1 条升到 2 条。这就是“加深流水”真正买到的东西——不是单条更快，而是每秒退出的指令更多。广告里的“20 级流水线”从不承诺你的某一条 LOAD 更快，它承诺的是吞吐。

真实机器里深流水甚至让单条更慢。每级之间要插流水寄存器，设其开销为 0.05 个单位：10 级流水的级延迟是 $0.5 + 0.05$ ，单条时延变成 $10 \times 0.55 = 5.5$ ，比 5 级还多出一成；吞吐也不是理想的 2，而是 $1/0.55 \approx 1.82$ 。级数越深，寄存器开销占比越大，边际收益越薄，这是流水线深度不能无限加的第一个原因。第二个原因第四章已经见过：级数越深，分支猜错时要冲刷的预取指令越多，有效 CPI 回升，吞掉一部分理论吞吐——下一节会把这个代价写进公式。商用 CPU 的流水深度在本世纪初曾一路冲到 30 级上下，随后又退回十几级，正是这两笔账共同裁决的结果：深度换来的吞吐增量，盖不过开销与冲刷带来的损失。

公式里还有一个容易读漏的下标：吞吐 $\approx 1/\text{级延迟}$ ，其中的级延迟取的是最慢的一级。同步流水的所有级共用同一个时钟，周期必须容纳最差的一级——设五级的真实延迟分别是 1.0、1.0、1.2、1.0、1.0 个单位，周期只能按 1.2 走，其余四级各浪费 0.2。此时单条时延是 $5 \times 1.2 = 6.0$ ，比五级延迟之和 5.2 还多；吞吐是 $1/1.2 \approx 0.83$ ，而不是按平均延迟算出来的 1。不平衡同时伤害两个量：时延被级数乘上最慢级，吞吐被最慢级单独封顶。这就是第四章强调阶段平衡的原因，也是真实 CPU 把访存这种“天然慢”的环节拆得更细、或干脆允许它独占多拍（多级流水化 cache）的原因——把最慢的一级切小，比把任何快一级做得更快都值钱。

案例：教学总线的一次访问时延 第五章把一次总线读事务拆成地址、等待、数据三段，并用时序预算决定要不要插入等待状态。现在给这台教学总线定一组数值：时钟周期 10 ns，一次读事务固定 4 拍——第 1 拍驱动地址与片选，第 2、3 拍是等待状态（慢存储器的访问时间经 READY/WAIT 机制插入），第 4 拍发起者采样数据。一次读的时延从地址生效算到数据被采样，连续读的吞吐上限按“每个事务独占 4 拍、互不重叠”计算：

$$\begin{aligned} \text{读时延} &= 4 \text{ 拍} \times 10 \text{ ns} = 40 \text{ ns} \\ \text{连续读吞吐上限} &= 1 \text{ 次} / 40 \text{ ns} = 2.5 \times 10^7 \text{ 次/秒} = 25 \text{ M 次/秒} \\ \text{按 32-bit 折算} &= 25 \text{ M 次/秒} \times 4 \text{ B} = 100 \text{ MB/s} \end{aligned}$$

这组数字是同一台总线的两张面孔，用两个改动可以分别验证它们谁动、谁不动：

访问形态	单次时延	连续吞吐	32-bit 带宽
基本事务（4 拍）	40 ns	25M 次/秒	100 MB/s
多插 1 拍等待（5 拍）	50 ns	20M 次/秒	80 MB/s
4 字突发（4+3 拍）	首字 40 ns	平均 17.5 ns/字	约 229 MB/s
长突发极限	首字 40 ns	10 ns/字	400 MB/s

多插一拍等待，时延涨 25%、吞吐跌 20%，两个量同向恶化，因为它们共享“事务总长”这个分母——这是“慢”的通常情形。突发传输则是另一个方向的操作：首字时延一纳秒没省，平均吞吐却翻到接近 4 倍。第五章 DRAM 的行缓冲突发、cache 的整行填充，用的全是这一招：把每次访问的固定开销摊到一串数据上。摊薄固定开销提高的是吞吐，受益的也只是第 2 个字起的时延；第 1 个字永远要等满 40 ns——首字时延由存储器的访问时间决定，和后面排多少数据无关。

写事务值得顺带一提：写可以把数据先丢进写缓冲就返回，时延被缓冲藏住，只有缓冲写满时才暴露真实事务时延——第五章的写缓冲正是利用“写后面没有人在等”这一点。最后补一条工程边界：等待拍不能无限插下去，第五章强调过事务要有最大等待长度与超时返回，否则一个不存在的目标会把整机永久挂在第一拍上——时延分析的前提是时延有限。

教学总线还有一个故意不做功能值得点名：地址与数据的解耦。现代高性能总线允许事务拆分——地址拍发出后总线让给别人，数据准备好后再占用总线送回——等待的时间不再独占总线，吞吐上限从“1/事务总长”逼近“1/地址拍”。代价是每笔事务要带标签、目标要能乱序完成，控制器复杂度远超教学总线的 READY/WAIT。本书的总线停在教学模型是对的；但将来读真实总线的规格书时要记得，“4 拍事务、25M 次/秒”描述的是总线的一种用法，不是总线的天花板。

案例：第七章教学机的时延与吞吐 第七章的面板计算机是时延与吞吐的极端样本。它的时钟自动档约 1 Hz——面包板加人眼给频率划了上限，LED 每拍都要看得清；它的 T 状态计数器是模 8 的，没有提前回绕逻辑，每条指令固定走满 T0 到 T7 八拍。于是：

单条指令时延 = 8 拍 × 1 秒 = 8 秒/条
吞吐 = 1 条 / 8 秒 = 0.125 条/秒

第七章的数灯程序共执行 41 条指令，其中 HLT 在 T3 一拍即冻结、只占 4 拍，其余 40 条各 8 拍，共 $40 \times 8 + 4 = 324$ 拍，在这台机器上要跑 324 秒，五分半钟。对照组：一颗 5 GHz、平均 IPC 为 2 的真实 CPU 完成同样 41 条指令约需 21 个周期、4 纳秒。吞吐相差将近 11 个数量级，时延相差 9 个数量级还多——本书最小与最快的两台机器之间，隔着整个处理器工业几十年的优化史。

这台串行、无重叠的教学机也顺手验证了本节开头的命题：无重叠时吞吐恰是时延的倒数， $1/8 = 0.125$ 。若把它的时钟从 1 Hz 提到 1 MHz（电路完全撑得住，只是人眼再也跟不上），时延变为 8 微秒、吞吐变为 12.5 万条/秒——两个量按同一倍数变化。要打破这种同进同退，只有引入重叠：流水、突发、并行，全是第七章之后各章的主题。

其实第七章已经把这个旋钮做进了硬件：时钟分自动与单步两路，自动档还能调速。往上调，时延与吞吐按比例变好，LED 先是看不清、再是只剩残影，最后只剩调试器才知道机器在干什么——可观察性与性能的交换是连续的，不是二选一。教学机把这个连续旋钮摆在面板上，本身就是对性能工程最直白的一课：所有的快都有代价，区别只在以什么形式支付。

教学机慢不是失败，它的设计目标本来就写在别处。慢到 1 Hz，是为了让每一

拍可观察：总线值、控制线、T 状态都能用 LED 逐拍核对；固定 8 拍，是为了让微码表保持“地址 = opcode 乘 8 加 T”这一条规律，省掉按指令提前结束的例外逻辑；单步按钮则把时延直接交到人手里，按一下才走一拍。时延与吞吐被刻意牺牲，换来的是“没有一处不可观察”。真实 CPU 的方向正相反：把每拍做短、把多条指令的拍重叠起来、把访存等待藏起来——分别对应第四章的流水线、第五章的 cache 与预取，以及本书未展开的乱序与多发射。读任何性能数字都要连同设计目标一起读：在教学机的指标表上，可观察性排在快慢前面。

本节一句话：时延与吞吐分开问、分开测、分开优化；听到一个性能数字，先问它是哪一个、在什么负载下量的。下一节把视角从“一件事”切到“一段程序”，给出总时间的三因子分解，并把中间那个因子 CPI 拆到可以逐项计算的程度。

二、CPI 与指令级性能

上一节讨论的是“一件事”的两个量；本节把视角切到“一段程序”，回答整机性能最基本的问题：跑完这个程序要多久。第四章给出过答案的骨架——程序时间等于指令数乘 CPI 再乘时钟周期；本节把它完整展开：三个因子各自由什么决定、优化时怎样说清动的是哪一个，然后把中间那个因子 CPI 拆到可以逐项计算——按指令类型加权、加上存储与分支的失效代价——最后讨论“用哪个程序测”这个绕不开的问题，并用两个设计点的完整对比收束。

总时间三因子的完整展开 第四章给出的基本乘法是本章一切计算的出发点：

$$\text{程序时间} = \text{指令数} \times \text{CPI} \times \text{时钟周期} = \text{指令数} \times \text{CPI} / \text{主频}$$

三个因子连乘，意味着每个因子都是总时间的一个完整维度：任何一个翻一倍，总时间翻一倍；任何一个（假想着）降到零，总时间降到零。同样重要的是，三个因子各有各的决定者，几乎没有哪个手段能只动其中一个而不惊动另外两个：

因子	由什么决定	典型优化手段	常见的反向移动
指令数	算法、编译器、ISA 的表达能力	更好的算法与编译优化、更高效的指令	单条指令变重，CPI 或周期变长
CPI	微结构（流水、cache、旁路、预测）与程序行为	加 cache、改进预测、乱序执行	控制变复杂，面积、功耗、验证成本上升
周期	最长组合路径、工艺、电压与温度	切短关键路径、加深流水、先进制程	级间寄存器开销增加，CPI 回升

逐行看这张表。指令数一行说的是“程序要机器干多少活”：同一个 C 程序，关优化和开优化编译出的动态指令数可以差好几倍；同一源码编译到两套 ISA 上，指令数也不同——所以比较指令数必须固定编译器与 ISA，否则比的是软件不是机器。注意这里的指令数一律指动态执行的条数：循环体里的一条加法执行几百万次就计几百万次，静态代码里写了多少行不作数。CPI 一行说的是“机器干每条活平均花几拍”，它一半由微结构决定（流水线多深、cache 多大、预测多准），另一半由程序行为决定（访存多不多、分支好不好猜）——后半正是本节后文要逐项算的。周期一行说的是“一拍有多长”，由最慢的那条组合路径封顶：第四章的单周期机就是反面教材，为了容纳加载指令的五段串行路径，每条指令都得按 5 个单位的周期走。

把三因子画成一条乘法链会更直观：每个因子挂着自己的决定者，乘积才是程序时间。



图示：程序时间三因子连乘：指令数由算法、编译器与 ISA 决定，CPI 由微结构与程序行为共同决定，时钟周期被最长组合路径封顶。数值取自本节的完整演算： 2×10^9 条 $\times 1.5 \times 2$ ns = 6 s。任何优化手段都要先说清动的是哪个因子、另外两个动不动。

“优化必须说清动的是哪个因子”不是形式主义，因为三个因子互相牵制，压下一个常常顶起另一个。两个经典例子。其一是 ISA 之争：复杂指令用一条指令干更多事，指令数降了，每条却更重、CPI 更高；精简指令把单条做轻，CPI 接近 1，指令数又升回去——净效果只有算完乘积才知道，这正是上世纪 RISC 之争最终靠实测而不是靠口号裁决的原因。其二是深流水：级数加深把周期切短，分支冲刷却更贵、流水寄存器更多，CPI 回升，上一节已经算过寄存器开销那笔账。再给一个假想的评审案例：某编译器优化宣称“动态指令数减少 30%”，但新指令序列让平均 CPI 从 1.0 涨到 1.5——乘积 $0.7 \times 1.5 = 1.05$ ，总时间不降反升 5%。单看任何一个因子的“改善”都不构成优化，这是本章第一条纪律；任何一个优化提案，评审时只问一句：另外两个因子动不动、动多少，乘积降不降。

三个因子凑齐，先完整算一遍再往下拆：某程序动态执行 2×10^9 条指令，机器平均 CPI 为 1.5，主频 500 MHz（周期 2 ns）：

$$\text{程序时间} = 2 \times 10^9 \text{ 条} \times 1.5 \times 2 \text{ ns} = 6 \times 10^9 \text{ ns} = 6 \text{ s}$$

6 秒这个结果拆得毫无悬念： $2 \times 10^9 \times 1.5 = 3 \times 10^9$ 拍，每拍 2 ns。有价值的是反过来的问法——要把它跑进 2 秒，有三条独立的路：指令数降到三分之一（换算法或编译器）、CPI 降到 0.5（换微结构或换程序形态）、周期降到 0.67 ns（换工艺或重切关键路径）。三条路的难度天差地别，定量框架的作用就是先告诉你每条路要降多少，再去评估哪条走得通。再给一个编译器动指令数的实感：同一程序开优化后动态指令数从 2×10^9 降到 1.2×10^9 ，若 CPI 不变，6 秒直接变成 3.6 秒——编译器一分钱硬件没动，只动了第一个因子。这也是对比机器时必须固定编译选项的原因：你测的是机器，不是编译器。

关于周期这个因子还要补一段史实：2005 年前后，商用 CPU 的主频在几 GHz 处撞上了功耗墙——频率再往上，功耗与散热先撑不住，“周期逐年减半”的时代就此结束。性能增长的主战场被迫转向另外两个因子：用更宽的微结构压 CPI，用更多的核分摊指令数。本章后面谈多核的 Amdahl 极限、谈功耗约束下的设计取舍，根子都在这一段历史上。

CPI 按指令类型加权 CPI 的全称是每条指令的平均周期数，“平均”必须落实到具体程序上：同一台 CPU 跑不同程序，指令混合不同，平均 CPI 就不同。计算方法是按指令类型加权——每类指令的动态占比乘上该类自己的 CPI，再求和。权重用动态条数（执行了多少次），不用静态条数（程序里写了多少行）。

取一个有代表性的整数程序混合，逐类给出该类 CPI，加权求和：

指令类型	动态占比	该类 CPI	对平均 CPI 的贡献
ALU 运算	40%	1	$0.40 \times 1 = 0.40$
加载	20%	2	$0.20 \times 2 = 0.40$
存储	10%	2	$0.10 \times 2 = 0.20$
分支	20%	1.5	$0.20 \times 1.5 = 0.30$
乘除	10%	8	$0.10 \times 8 = 0.80$
合计	100%	—	平均 CPI = 2.1

$$\begin{aligned} \text{CPI} &= 0.40 \times 1 + 0.20 \times 2 + 0.10 \times 2 + 0.20 \times 1.5 + 0.10 \times 8 \\ &= 0.40 + 0.40 + 0.20 + 0.30 + 0.80 \\ &= 2.1 \end{aligned}$$

这张表里最值得停一秒的是最后一行：乘除指令只占条数的 10%，却贡献了 $0.8/2.1 \approx 38\%$ 的执行时间。占比小、单类慢，照样能主导平均值——这是“先弄清时间花在哪”的定量版本，也是下一节 Amdahl 定律的主题。在这个混合上比较两个

候选优化：把乘除单元加快一倍（该类 CPI 从 8 降到 4），省下 $0.4/2.1 \approx 19\%$ 的总时间；把占条数最多的 ALU 加快一倍（CPI 从 1 降到 0.5），只省 $0.2/2.1 \approx 10\%$ 。条数最多不等于最该优化，加权表把这句话变成了算术。

反过来，换一个几乎没有乘除的程序（比如纯控制流代码），把其余四类按比例归一，同一台机器的平均 CPI 立刻降回 1.4 上下。CPI 是“机器乘程序”的性质，从来不是硬件常数——第四章章末练习里“CPI 是动态混合的平均”那句话，到这里有了完整算法。工程上还要知道权重从哪来：动态占比靠指令级模拟器、插桩或性能计数器统计得到，这正是第五节测量工具要回答的第一类问题。

权重的“动态”二字值得用反例再强调一次。某程序静态代码 1000 条指令，乘除只占 5 条（0.5%），单看静态混合会以为乘除无关紧要；可这 5 条全部在最内层循环里，动态占比 10%——正是上表那个混合。拿静态占比加权会算出完全不同的 CPI，进而错判优化方向。静态条数回答“程序写了什么”，动态条数回答“机器跑了什么”，性能计算只认后者。

有效 CPI 要加上 memory 与分支代价 上一张表的“该类 CPI”还不是全部真相：它默认每次访存都命中、每个分支都猜对。真实机器要把存储系统和分支预测的失效率折算进去，得到有效 CPI：

$$\begin{aligned} \text{有效 CPI} &= \text{基础 CPI} \\ &+ \text{每条指令访存次数} \times \text{miss 率} \times \text{miss 代价} \\ &+ \text{每条指令分支数} \times \text{误测率} \times \text{冲刷代价} \end{aligned}$$

形式上与第四章的 cache 演算一脉相承，只是把分支项一并写进来。给一组完整数值：基础 CPI 取 1.0（理想流水、全命中、全猜对）；每条指令平均访存 1 次（取指与数据访问合并，按此简化；第四章按 1.2 次算过更细的版本），cache miss 率 2%，每次 miss 罚 50 拍；分支占指令流的 25%，预测错误率 20%——折合每条指令摊到 5% 次误测——每次误测冲刷 2 拍。

$$\begin{aligned} \text{存储项} &= 1 \times 0.02 \times 50 &&= 1.0 \\ \text{分支项} &= 0.25 \times 0.20 \times 2 &&= 0.1 \\ \text{有效 CPI} &= 1.0 + 1.0 + 0.1 &&= 2.1 \\ \text{有效 IPC} &= 1 / 2.1 &&\approx 0.48 \end{aligned}$$

拆开看占比：基础执行只占 48%，存储系统吃掉 48%，分支吃掉 5%。一台“IPC 设计值 1”的机器实测 0.48——IPC 曲线总比理想低，原因就在这里：理想值假设失效率为零，而每个失效事件都带一个大乘数。98% 的命中率听上去接近完美，可 50 拍的 miss 代价让那 2% 变成整整 1.0 个 CPI。敏感性也算一笔：把 miss 率从 2% 压到 1%，有效 CPI 从 2.1 降到 1.6，程序快 31%——命中率小数点后面每一位都明码标价，这就是第五章用一整章讲存储层次的性能理由。分支项同理：深流水机的冲刷代价不是 2 拍而是 4、5 拍，同样的误测率立刻把分支项放大两三倍，机器对预测器的依赖随流水线深度一起加深。超标量也逃不开同一公式：理想 IPC 为 4 的机器，失效项按每条指令折算后可能只剩 1.5——基础越宽，失效项越显眼。工程读法两条：其一，降失效率比继续堆基础宽度便宜，因为大头在失效项；其二，报性能必须连失效率一起报——只说 IPC 0.48，不说 miss 率和误测率，既没解释机器为什么是这个速度，也没给下一步指出该动哪一项。

把这三项按占比画成一条带，”一半去了哪”比公式更直观：



图示：有效 CPI 2.1 按来源切成三段：基础执行与存储代价各占 48%，分支占 5%。设计值 IPC 1.0 的机器实测约 0.48，差值几乎全由失效项贡献——98% 命中率的存储系统，吃掉的周期与执行本身一样多。优化预算该拨给哪一项，从条带上直接读。

把三项的占比与出处列成表，本书前后每一章的性能话题都能在其中找到自己的格子：

项	数值	占有效 CPI	对应的硬件机制（章节）
基础执行	1.0	48%	数据通路与理想流水（第四章）
存储代价	1.0	48%	cache、存储层次与行调度（第五章）
分支代价	0.1	5%	分支预测与冲刷控制（第四章）

这张表还有两种用法。设计早期没有机器可测，就用它做纸面估算：替候选参数各算一遍有效 CPI，谁的乘积小选谁，把昂贵的仿真留给入围方案。机器造出来以后，表里的失效率不再靠假设——miss 率、误测率都能用性能计数器直接读出，那是第五节的主题；读出来以后代回本表，就知道下一阶段的优化预算该拨给哪一项。

基准程序的思想 CPI 依赖程序，“这台机器有多快”于是绕不开一个前置问题：用哪个程序测？单一 kernel 有两个经典陷阱。一是代表性：一个矩阵乘测不出分支预测的好坏，一个整数排序压不出浮点单元；任何单一程序都只覆盖机器能力的一个切片。二是可被定向优化：编译器厂商认出 benchmark 的热循环，专门为它生成特殊代码，分数涨了，普通程序没有变快一度量一旦变成目标，就不再是好的度量。

SPEC 类基准套件的思想就是正面回应这两个陷阱：不用单一 kernel，而选一组真实的、有公开源码的程序，覆盖整数与浮点、计算密集与存储密集，按用途分成若干组；每个程序固定输入、单独计时，报告相对参照机的比值；最后把全部程序的比值聚合成总分。整套路数里，“用哪个程序测”与“怎么加权”同样重要：程序选偏了，加权再公道也没用；加权选错了，真实的分项结果也会被平均数扭曲。

聚合为什么用几何平均而不是算术平均，一个小例子就能看清。设机器乙相对机器甲在两个程序上的加速比分别是 4 与 0.25（一个快四倍，一个慢四倍）：

算术平均 = $(4 + 0.25) / 2 = 2.125$	（看似乙大胜）
几何平均 = $\text{sqrt}(4 \times 0.25) = 1$	（实际是平手）

算术平均被那个 4 主导；几何平均对“快多少”与“慢多少”一视同仁，快四倍与慢四倍恰好抵消。几何平均还有两个实用性质：比值换一台参照机重算，各机器的名次不变（算术平均会随参照机换人）；连乘结构的加速比与连乘结构的优化过程相容，分步改进可以直接相乘再开方。代价也要说清：几何平均描述的是“相对表现的典型水平”，不预测任何一个程序的真实运行时间；规范的做法是永远同时给出每个程序的分项，几何平均只当索引。测试纪律也是套件的一部分：固定输入数据、公开编译选项、禁止针对个别程序的特判——缺了这三条，平均分再漂亮也只是广告。

基准套件自己也会过时，这是容易被忽略的一点。程序会随时代失去代表性：十几年前的热点程序放到今天，可能整个被 cache 装下，存储子系统毫无压力；编译器会逐代学会套件里的惯用写法，分数的水分逐年增加；负载形态本身也在迁移，从单核整数程序到并行服务再到 AI 推理。所以严肃的套件都定期换血、严格带版本号——引用一个分数而不带版本与分项，等于引用一个没有时间地点的测量值。本书的立场是：套件用来训练判断力，不是用来背分数。

套件的内部结构同样值得看一眼。常见做法是按整数与浮点分成两组，因为两类负载几乎不共享瓶颈：整数程序考分支预测与指针追踪，浮点程序考运算管线与存储带宽。有的套件还区分 speed 与 rate 两种测法——前者测一个程序单独跑多快，是时延视角；后者测同时跑若干份时的总完成量，是吞吐视角——一套基准把上一节分开的两个量都照顾到了。读分时先弄清看到的是哪一组、哪一种测法：整数 speed 领先、浮点 rate 落后并不矛盾，它只是说明瓶颈长在不同程序上。这也回答了“为什么干脆只报一个总分”：总分是给采购的一行摘要，分组与分项才是给工程师的

地图。

案例：两个设计点的完整对比 把本节全部工具用在一次设计裁决上。同一套 ISA 有两个实现：设计 A 走高频路线，2 GHz、平均 CPI 1.2，靠深流水把周期做短；设计 B 走宽发射路线，1.5 GHz、平均 CPI 0.9，每拍干得更多但频率上不去。先跑同一个程序，动态指令数 10^9 条：

设计	主频	CPI	IPC	每秒指令数	程序时间
A	2 GHz	1.2	0.83	$2 \times 0.83 \approx 1.67$ (G 条/秒)	$1.2/2 = 0.60$ s
B	1.5 GHz	0.9	1.11	$1.5 \times 1.11 \approx 1.67$ (G 条/秒)	$0.9/1.5 = 0.60$ s

精确打平。每秒指令数等于主频乘 IPC：A 用频率补 IPC，B 用 IPC 补频率，乘积相同，程序时间自然相同——“高频低 IPC”与“低频高 IPC”在这个程序上分不出胜负，这正是单看主频或单看 IPC 都会误判的原因。

换一个程序立刻分出胜负。程序乙分支密集：每 4 条指令 1 个分支，误测率 10%，A 的深流水每次误测冲刷 8 拍，B 只冲刷 2 拍：

A：有效 CPI = $1.2 + 0.25 \times 0.10 \times 8 = 1.40$ 时间 = $1.40/2 = 0.70$ s
 B：有效 CPI = $0.9 + 0.25 \times 0.10 \times 2 = 0.95$ 时间 = $0.95/1.5 = 0.63$ s

分支预测一进场，浅而宽的 B 赢了约 10%。再换程序丙：无分支、全命中的规则数值循环，两台机器的有效 CPI 都退化为 1.0，频率说了算——A 用 0.50 s，B 用 0.67 s，A 反赢 33%。三个程序三种结果：打平、B 赢、A 赢。谁赢取决于程序，而程序通过 CPI 这一列进入乘积。换成采购语言就是：挑机器之前先弄清自己的程序长什么样——分支密集选预测好、冲刷便宜的，规则计算选频率高的；厂商的综合分可以当初筛，不能当裁决，原因上一个主题已经说透。

还有一个读表细节：程序甲上 0.60 s 对 0.60 s 是这组参数下的精确结果，不是设计哲学的必然——CPI 只差 0.05，结论就会翻转。两个设计点离得越近，越要回到失效率的分项数据上做判断：谁的 miss 率更低、谁的误测冲刷更便宜，这些小数点后一位的差别，比主频整数位的差别更能决定胜负。只比总分，等于在噪声里掷硬币。

这张对比表的使用前提也要写明：两台机器同一套 ISA、跑同一份二进制，指令数这一列才相同，裁决才能只交给 CPI 与周期两列。一旦连编译器或 ISA 也变了，指令数列必须进表重算——设 B 平台的编译器把同一程序编出 1.1×10^9 条指令，程序甲的时间就变成 $1.1 \times 0.9/1.5 = 0.66$ s，原本的打平立刻反转成 A 赢。三因子缺一个都算不完：读任何性能对比，先检查表里有没有指令数、CPI、周期这三列，缺列的对比不是简化，是隐瞒。

本节收束为一条纪律：主频不是性能，IPC 不是性能，指令数也不是性能，只有“指令数 $\times$ CPI $\times$ 周期”的乘积是性能。下一节回答它的对偶问题：已经知道时间花在哪了，优化其中一部分，整体能快多少——Amdahl 定律给出上限。

三、Amdahl 定律与加速极限

本节回答“优化值不值得做、先做哪一个”。直觉的回答是“把最慢的部件换快”，但整机的快慢取决于时间花在哪里，而不取决于哪个部件孤立地看最慢。Amdahl 定律把这件事写成一条公式：只需要两个输入——被优化部分原来占总时间的比例、这部分能快多少倍——就能算出整机快多少。它是性能分析里被引用最多、也被误用最多的一条公式：只报局部加速比、不报占比，是最常见的半截话；只记得“加速有上限”、忘了上限怎么算，是第二常见的半截话。

本节的做法和前几章一致：先把公式完整推导出来，不给它留任何神秘感；再用一张上限表把“占比决定一切”摆成数字；然后把多核扩展、Gustafson 的对照视角和一次三方案排序各算一遍。所有数值都可以手算复核，公式里没有任何近似。

Amdahl 公式的推导 设机器跑某程序的总时间为 T 。用测量手段（本章第五节的性能计数器，或第五章的总线监视）把 T 分成两份：打算优化的部分原来占 p ($0 \leq p \leq 1$)，其余部分占 $1 - p$ 。现在让前一部分快 s 倍 ($s > 1$)，后一部分原封不动。推导只有三步：

优化前： $T = p \cdot T + (1 - p) \cdot T$ (两份时间相加)
优化后： $T' = p \cdot T / s + (1 - p) \cdot T$ (优化部分快 s 倍，其余不变)
加速比： $S = T / T' = 1 / ((1 - p) + p / s)$

第三步把 T 约掉，剩下的式子里只有 p 和 s 两个数，这就是 Amdahl 定律： $S = 1 / ((1 - p) + p / s)$ 。它没有引入任何近似，唯一的假设是“优化不影响其余部分”——被优化的部分单独计时，没碰的部分原样计时。这个假设后面还要反复回来检查。

两个输入缺一不可。只说“这个优化让内存访问快 2 倍”而不说它占多少时间，等于什么都没讲：内存访问占 60% 时这是整机 1.43 倍，占 5% 时只有约 1.03 倍，同一个“快 2 倍”差出十几倍的整体效果。反过来只说“这部分占 80%”同样不够，还知道能快几倍。占比和局部加速比是同一个乘积里的两个因子，缺了任何一个，另一个再大也定不了积。

同一个“局部快 2 倍”，占比不同，整体完全不同：
 $p = 60\%: S = 1 / (0.4 + 0.6 / 2) = 1 / 0.700 = 1.43$
 $p = 25\%: S = 1 / (0.75 + 0.25 / 2) = 1 / 0.875 = 1.14$
 $p = 5\%: S = 1 / (0.95 + 0.05 / 2) = 1 / 0.975 = 1.026$

再给一个完整演算。某部分占 40%，优化后快 2 倍： $T' = 0.4T / 2 + 0.6T = 0.8T$ ， $S = 1 / 0.8 = 1.25$ 。局部快 2 倍只换来整机快 1.25 倍，这个“打折”不是工程损耗，是算术必然：优化后那 40% 缩成了 20%，省下的 20% 就是全部收益，而其余 80% 的时间一分不少。比例看不懂时，把 T 换成具体毫秒再算一遍就清楚了：

设 $T = 200 \text{ ms}$ ，被优化部分占 40% 即 80 ms，其余 120 ms
优化后： $80 / 2 + 120 = 160 \text{ ms}$
加速比： $200 / 160 = 1.25$ (与比例算法一致)

第四章按拍数逐条核算指令时间时，读者已经见过同样的思想：总时间永远被所有部分分完，加快任何一部分，收益都以它占的份额为限。本节只是把“拍数”换成“时间份额”，把同一思想写成通用公式。也正因如此， p 和 s 的来源必须分别说清楚：

输入	含义	从哪里来
占比 p	被优化部分原来占总时间的份额	只能测量：profiler、性能计数器、总线监视
局部加速 s	该部分优化前后的时间比	微基准实测，或按部件参数推算
整体加速 S	由前两者唯一决定： $S = 1 / ((1 - p) + p / s)$ ，不是第三个独立输入	

把推导浓缩成一句使用说明：先测 p ，再估 s ，最后才谈整体。实践中这条公式的误用也集中在这两个输入上，对照如下：

常见说法	问题出在哪	正确问法
“新部件快 10 倍，值得上”	只有 s ，没有 p	这部分原来占总时间的多少
“这段代码占了 80% 的时间”	只有 p ，没有 s	它能被加快多少倍
“整机应该能快 3 倍”	把 s 当成了 S	代入 $1 / ((1 - p) + p / s)$ 再报数

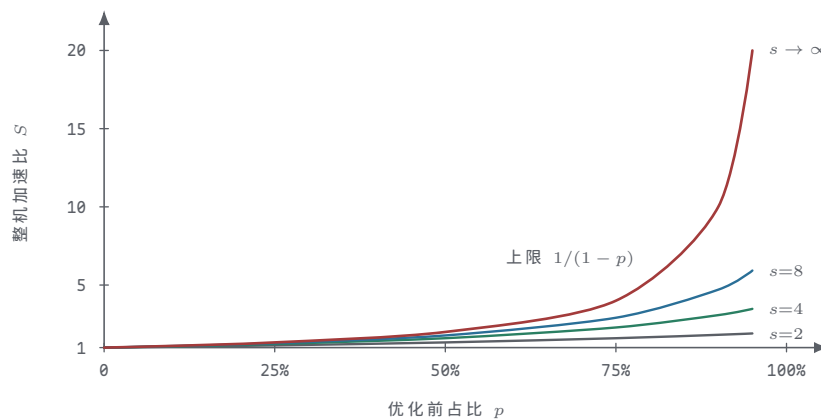
”优化后这部分只占 5% 拿优化后的占比当 p p 必须取优化前的份额了”

末行是最隐蔽的一处误用：公式里的 p 是”优化前”的份额。优化见效之后这部分已经缩水，拿缩水后的占比再代一次公式，等于把同一笔收益算了第二遍。所有迭代式优化（做一轮、测一轮、再做一轮）都必须回到新一轮测量重新取 p ，不能沿用上一次的数字。

加速上限表 把 p 取六档、 s 取四档代入公式，逐个算出来，就是下面这张表。它是性能工程里最值得背下来的一张表：背下来之后，任何”某部件快了几倍”的宣传都能在十秒内翻译成整机收益。

占比 p	局部快 2 倍	局部快 4 倍	局部快 8 倍	无限加速上限
10%	1.05	1.08	1.10	1.11
25%	1.14	1.23	1.28	1.33
50%	1.33	1.60	1.78	2.00
75%	1.60	2.29	2.91	4.00
90%	1.82	3.08	4.71	10.00
95%	1.90	3.48	5.93	20.00

把这张表的四列画成曲线，”占比决定上限”就从数字变成了形状：



图示：Amdahl 定律的四条加速曲线。 $s \rightarrow \infty$ 那条就是上限 $1/(1-p)$ ， $p = 0.9$ 时也只到 10；其余三条被它压在下面。读法： s 决定曲线弯成什么形状， p 决定曲线能爬到多高一占比才是主变量。

最后一列是 $s \rightarrow \infty$ 的极限，公式退化成一个更简单的式子：

$s \rightarrow \infty$ 时 $p/s \rightarrow 0$ ，分母只剩 $(1-p)$ ：

$p = 50\%$: 上限 = $1/0.5 = 2.00$

$p = 90\%$: 上限 = $1/0.1 = 10.00$

$p = 95\%$: 上限 = $1/0.05 = 20.00$

”无限加速”的含义是把这部分优化到不花时间—瞬时完成。即使做到这一步，整机也只快 $1/(1-p)$ 倍：占一半的部分优化到无穷快，整机快 2 倍；占九成的部分优化到无穷快，整机也才快 10 倍。上限被未优化的 $1-p$ 钉死，与被优化部分究竟多快毫无关系。这就是”Amdahl 上限”的准确含义：它不是工程上暂时够不着的理想，而是数学上的天花板。

读这张表要竖着读，也要横着读。竖着看：同一列里占比每涨一档，收益跳一档—占比是收益的主变量，想拿大加速先找大占比。横着看：同一行里 s 从 2 提到 4 收益明显，从 8 提到无穷反而所剩无几。把 $p = 75\%$ 那一行单独展开，收益递减看得更清楚：

局部加速 s	整体加速	比上一档多拿	关键问题
----------	------	--------	------

2	1.60	—	第一笔投入最划算
4	2.29	0.69	仍然值得
8	2.91	0.62	开始递减
16	3.37	0.46	投入翻倍，增量缩水
32	3.66	0.29	接近天花板
∞	4.00	0.34	理论极限，与 32 倍 只差 0.34

工程含义很直接：局部加速比做到一个量级之后，就该回头检查占比，而不是继续死磕同一个部件的 s 。把 s 从 8 做到 16 往往要花一代工艺，把同样的精力转去扩大优化的覆盖面——让更多代码路径受益于这次优化——常常能多拿一整档。第七章的教学机是这张表最直白的注脚：它每条访存都要等一个完整总线周期，访存在总时间里占大头，任何“把 ALU 做快”的改法都撞在表的前几行；反过来，第五章给主存加一级 cache、让大多数访存不再等总线，才是对它真正有效的那一行。

上限表还有一个用法：快速否决。发布会和宣传页上的数字，都可以代入公式反推，看占比是否对得上：

宣传：“新浮点单元快 10 倍，整机快 3 倍”

反推占比： $3 = 1/(1 - p + p/10) \rightarrow 1/3 = 1 - 0.9p \rightarrow p = 0.74$

含义：浮点要占到总时间的 74%，这个宣传才成立

若 profile 实测浮点只有 40%： $S = 1/(0.6 + 0.4/10) = 1/0.64 = 1.56$

结论：3 倍不成立，真实收益 1.56 倍

反推出的 74% 与实测的 40% 差了近一倍，宣传里的“整机快 3 倍”只能在某个浮点极密集的特例程序上成立。这类反推不需要任何仪器，一张上限表加一条公式就够，是读评测报告时最省钱的过滤器。

上限表还可以反过来用：已知整机加速和局部加速，反推占比是否说得通。某团队报告“cache 改造让访存快 4 倍，整机快了 1.5 倍”，反推一下：

$1.5 = 1/(1 - p + p/4) \rightarrow 2/3 = 1 - 0.75p \rightarrow p = 0.444$

含义：访存原来要占 44.4%，这组数字才互相说通

核对：该程序的访存占比实测约 70%

若 $p = 0.70$ ： $S = 1/(0.3 + 0.7/4) = 1/0.475 = 2.11$ ，不是 1.5

对不上本身就是信息：要么“快 4 倍”只是微基准里的理想值，真实程序没有兑现；要么改造影响了其他部分（例如 cache 结构变了之后取指行为也跟着变了），违反了“其余部分不动”的假设。无论哪一种，下一步都是回到测量，而不是在公式里调数字。

多核扩展的阿姆达尔极限 多核是“局部加速”最引人注目的特例：把程序的可并行部分摊到 n 个核上，相当于这部分 $s = n$ ；而串行部分——启动、收尾、归约、临界区、I/O——一个核也帮不上，原样留在 $1 - p$ 里。设串行部分占 5%，可并行部分占 95%，代入公式：

$S(n) = 1 / (0.05 + 0.95/n)$

$n = 64$ ： $S = 1/(0.05 + 0.95/64) = 1/0.0648 = 15.42$

$n \rightarrow \infty$ ： $S = 1/0.05 = 20$

核数 n	并行部分耗时	总耗时	整体加速
2	0.4750	0.5250	1.90
4	0.2375	0.2875	3.48
8	0.1188	0.1688	5.93
16	0.0594	0.1094	9.14
32	0.0297	0.0797	12.55

64	0.0148	0.0648	15.42
128	0.0074	0.0574	17.41
256	0.0037	0.0537	18.62
∞	0	0.0500	20.00

64 个核只换来约 15.4 倍加速——不是直觉里的 64 倍，离上限 20 倍也还差一截。这张表和上一张上限表是同一族曲线：95% 占比那一行，局部加速取 2、4、8 时恰好就是 2 核、4 核、8 核的整机加速，核数只是 s 的另一种写法。

核数翻倍不等于速度翻倍，而且差距越拉越大：16 核加到 32 核，整机快 $12.55/9.14 \approx 1.37$ 倍；64 核加到 128 核，只快 $17.41/15.42 \approx 1.13$ 倍；128 核加到 256 核，只剩 $18.62/17.41 \approx 1.07$ 倍。曲线弯下来的原因在分母：核数越多，总耗时越接近串行部分的 0.05，新增的核全部砸在已经只剩零头的并行部分上。串行部分像地板一样等在那里，核数再多也只是无限贴近它，永远穿不过去。换算成采购语言更刺眼：

核数	整体加速	每核效率（加速/核数）	关键问题
16	9.14	0.57	效率尚可
32	12.55	0.39	效率明显下滑
64	15.42	0.24	四分之三的核在陪跑

所以多核工程的真实收益不来自“加核”，而来自把串行部分也改小：全局锁拆成分段锁，归约改成树形，启动和收尾改成与计算流水交叠，串行 I/O 改成批量异步。效果可以直接算出来——同样 64 核，串行占比从 5% 改到 1%，加速从 15.42 跳到 39.26：

串行占比	64 核加速	无限核上限	关键问题
10%	8.77	10	核多无用武之地
5%	15.42	20	服务器常见量级
2%	28.32	50	需要专门改串行段
1%	39.26	100	动串行一档，胜过加核四倍

末行那句话值得再算一遍：串行 5% 时从 64 核加到 256 核，核数翻两番，加速只从 15.42 涨到 18.62；串行从 5% 改到 1%，加速从 15.42 涨到 39.26。并行的真实收益来自把串行部分也变小，这不是口号，是第四列对第一列的斜率。还要提醒一句：5% 的串行占比已经算乐观——真实并行程序还有负载不均、通信和同步开销，它们的效果等价于把串行占比再抬高几档，表里的数字只会更差。

“把串行部分改小”也不是一句口号，它对应一组具体手段，每种手段削的都是串行占比的不同来源：

串行来源	典型场景	缩小手段
启动与收尾	主线程分发任务、回收结果	分发与计算交叠，收尾流水化
全局锁	所有核排队进同一个临界区	分段锁、无锁结构、线程私有副本
归约	各核结果最后串行汇总	树形归约，深度 $\log n$ 代替 n
串行 I/O	计算前一次读入、算完一次写出	预取、双缓冲、批量异步
负载不均	最快的核等最慢的核	动态调度、任务切细

负载不均值得单独说一句：它不在公式里出现，效果却等价于串行占比升高——64 个核里 63 个做完了等最后 1 个，等待期间等价于 63 个核被串行化。所以表里的”任务切细”不是锦上添花：任务粒度越细，调度器越容易把各核的完工时间拉齐，等价的串行尾巴越短。

串行占比本身也不是猜的，可以测：同一程序分别在 1、2、4、8 核上测总时间，算出各档加速 $S(n)$ ，理论上 $1/S(n) = f + (1-f)/n$ (f 为串行份额)，把 $1/S$ 对 $1/n$ 描点应近似一条直线——截距是 f ，斜率是 $1-f$ 。若点明显跑到直线上方，说明存在随核数增长的额外开销（通信、竞争），串行份额本身在随核数上升。这是公式之外、实测之内才能看到的東西，也是下一节的测量方法要回答的问题。

串行份额的实测拟合：

$$1/S(n) = f + (1-f)/n$$

以 $1/S$ 对 $1/n$ 描点：截距 = f ，斜率 = $1-f$

点上弯(高于直线) = 有随核数增长的额外开销

第七章的教学机是这条曲线的另一个端点：它整机只有一个执行流，串行占比 100%，加核对它毫无意义——它能讲清楚并行，恰恰是因为自己一点也不并行。

Gustafson 的对照视角 Amdahl 的假设是”问题规模固定”：同一道题，核越多越好。Gustafson 换了一个假设：机器变大了，题也跟着变大——加核不是为了把同一道题算得更快，而是为了在同样的时间里算更细的网格、更大的模型、更多的粒子。这时串行部分占墙上时间的比例 f 不随规模增长（启动还是那几秒），并行部分却随核数线性放大。推导同样只有几步：

n 核机器上，墙上时间归一化为 1：串行部分占 f ，并行部分占 $1-f$

同一问题搬到单核：串行部分仍是 f

并行部分一个核要做 n 份，为 $n*(1-f)$

单核总时间 = $f + n*(1-f) = n - (n-1)*f$ <- 放大规模下的加速比

代入 $f = 0.05$, $n = 64$: $64 - 63*0.05 = 60.85$

同样 5% 的串行占比、同样 64 个核，Amdahl 算出 15.42，Gustafson 算出 60.85，差了将近四倍。逐档对照：

核数 n	Amdahl (规模固定)	Gustafson (规模放大)	关键问题
2	1.90	1.95	核少时两者接近
8	5.93	7.65	分歧开始显现
16	9.14	15.25	几乎差一倍
32	12.55	30.45	差出一倍以上
64	15.42	60.85	假设差异完全显形
∞	20.00	∞	一个有顶，一个没顶

两个公式都没有错，分歧只在”什么保持不变”。Amdahl 固定问题规模，是悲观的：规模不变时串行占比原地不动，核越多越撞上它。Gustafson 固定墙上时间、放大问题规模，是乐观的：规模放大后并行部分变大，串行占比被稀释——原本占 5% 的启动开销，在放大一千倍的问题里可能只占 0.005%。历史背景也有意思：Amdahl 定律 1967 年提出时的语境是为单处理器辩护，论证并行机前途有限；Gustafson 在 1988 年用上千个处理器的机器按放大规模实测，拿到接近线性的加速，说明那场争论的双方各抓住了问题的一半。

适用场合按”用户等的是什么”来分：交互程序、实时控制、固定的夜间批处理，题就这么大，用户等的是同一道题快多少，用 Amdahl 估计才诚实；超算中心、气象预报、AI 训练这类”机器多大题就多大”的负载，预算批的是算力规模，用 Gustafson 更贴近真实采购逻辑。反过来也有题不会变大的场合：编译一次工程、

应答一次数据库查询，没人因为机器大了就把查询写复杂十倍，这些负载永远服从 Amdahl。成熟的做法是两个都算：Amdahl 给出固定规模的下限，Gustafson 给出放大规模的上限，真实收益位于两者之间，由“题到底会不会跟着变大”决定靠近哪一端——买机器之前先回答这个问题，比争论哪个公式“对”有用得多。

两个公式还可以用同一个具体场景各走一遍，看它们到底在算什么。同样 5% 的串行占比、同样 64 个核、同样 60 分钟，两个场景的 60 分钟不是一回事：

场景一(Amdahl, 题固定): 单核跑完要 60 分钟, 其中串行 3 分钟
 搬上 64 核: $3 + 57/64 = 3.89$ 分钟
 加速 = $60/3.89 = 15.42$
 场景二(Gustafson, 题随机器放大): 64 核跑 60 分钟, 串行段占 3 分钟
 这 60 分钟里并行段干了 $57 \times 64 = 3648$ 核·分钟的活
 同一道题搬回单核: $3 + 3648 = 3651$ 分钟
 加速 = $3651/60 = 60.85$

场景一的 57 分钟是总工作量，场景二的 57 分钟是墙上时间（背后是 64 份并行工作量）。把两个场景的数字混着用——拿场景二的占比代场景一的公式——是围绕 Gustafson 最常见的计算错误；分清“题固定”还是“时间固定”，是两个公式共同的前提检查。

案例：三个候选优化点的排序 把本节全部工具用在一个完整问题上。某程序 profile 一轮，时间分布为：内存访问 60%，整数计算 30%，分支代价 10%。手头有三个候选方案，每个只动一处：方案 A 做 cache 优化，让内存访问快 2 倍；方案 B 重做 ALU，让整数计算快 3 倍；方案 C 加分支预测，消除 80% 的分支代价。哪个先做？不许凭感觉，逐个代入公式：

方案 A: 内存占 60%，快 2 倍
 $T' = 0.6/2 + 0.4 = 0.70$ $S = 1/0.70 = 1.43$
 方案 B: 整数占 30%，快 3 倍
 $T' = 0.3/3 + 0.7 = 0.80$ $S = 1/0.80 = 1.25$
 方案 C: 分支占 10%，消除 80% 代价 = 这部分只剩 20%，即快 5 倍
 $T' = 0.1/5 + 0.9 = 0.92$ $S = 1/0.92 = 1.09$

方案	作用部分（占比）	局部加速	整体加速	排序
A: cache 优化	内存（60%）	2	1.43	1
B: 重做 ALU	整数（30%）	3	1.25	2
C: 分支预测	分支（10%）	5	1.09	3

排序结果 $A > B > C$ ，恰好与局部加速比的直觉排序相反：B 的局部 3 倍比 A 的 2 倍漂亮，C 的“消除 80%”听起来最猛，但收益是占比与局部效果的乘积，两个因子谁小谁卡谁——60% 配上平平的 2 倍，照样赢过 10% 配上惊人的 5 倍。方案 C 还藏着本案例唯一的翻译陷阱：“消除 80% 代价”不是 $p = 0.8$ ，而是分支那 10% 里消掉 80%，即这部分时间缩到原来的五分之一；把自然语言翻译成 p 和 s 时用错档位，是 Amdahl 计算里最常见的错误。

这张表的输入全部来自 profile: 60、30、10 是测出来的，不是看代码猜的。凭直觉很多人会先做 B，因为“ALU 快 3 倍”听起来最像硬件进步；数字说先做 A。这也是章首“先测量，再优化”的具体含义：没有占比这个输入，公式只剩一半，而一半公式给出的排序往往是错的。分支代价在第四章的流水线里见过实体——误预测的冲刷拍；第五章的 cache 则是方案 A 的实体。占比测量告诉工程师该把晶体管花在哪一章的技术上。排序对测量误差有多敏感，也可以当场检查：

内存占比实测	方案 A 整体加速	方案 B（不变）	预期结果
50%（低估 10 点）	1.33	1.25	A 仍第一

60% (基准)	1.43	1.25	A 第一
70% (高估 10 点)	1.54	1.25	A 第一

占比误差正负十个点之内排序不变，这个结论是稳健的；若三个方案的整体加速本来就只差一两个点，才需要回去提高测量精度。最后检查公式的边界：三个方案互斥的假设是“动一处不影响其余两处”。若 cache 优化同时改变了分支行为（例如预取挤掉了代码行），就要重新测量、重新代入，不能把三张单子各算一遍再相加——Amdahl 是线性拆分，交叉影响不在公式里。还要记住上限表的告诫：即使方案 A 做到内存访问无穷快，整机也只有 $1/0.4 = 2.5$ 倍；若业务需要 4 倍，单靠优化内存永远不够，必须同时动 B 和 C 让 $1-p$ 缩小。排序做完不是终点：做完 A 之后内存时间减半，新的时间分布约成 43/43/14，下一轮要重新 profile、重新排序，优化的顺序可能因此改变。把后两轮也算完，看这个迭代实际怎么走：

第二轮(做完 A 之后，总时间已从 100 缩到 70)：

新分布：内存 30、整数 30、分支 10

方案 B: $30/3 + 40 = 50$ 本轮加速 = $70/50 = 1.40$

方案 C: $10/5 + 60 = 62$ 本轮加速 = $70/62 = 1.13$

仍选 B；做完 B 后分布：内存 30、整数 10、分支 10(共 50)

第三轮只剩 C 可做：

方案 C: $10/5 + 40 = 42$ 本轮加速 = $50/42 = 1.19$

累计：100 → 70 → 50 → 42，整机共快 $100/42 = 2.38$ 倍

三个数字各有一处值得停一下。第一，方案 B 的本轮加速从第一轮 1.25 涨到 1.40：A 把总时间做小之后，整数计算的相对占比变大了一——同一个方案的价值随工程进程变化，这正是每轮必须重测占比的原因。第二，累计 2.38 倍正好等于三轮加速的连乘 $1.43 \times 1.40 \times 1.19$ ，因为三个方案作用在互不重叠的部分上；若方案之间有交叉影响，连乘关系就不成立，必须回到合并后的分布重算。第三，2.38 倍离“内存无限快”的 2.5 倍上限已经很近：这个程序再往下做，每一轮的可选方案会越来越少、单轮收益越来越小，优化的尽头是被各部分 $1-p$ 轮流接力的天花板。

本节收束成一句话：Amdahl 定律不预测哪个优化最有效，它只负责把测得的占比和局部加速翻译成整体数字；翻译没有弹性，弹性全在输入里——所以性能工作的第一步永远是测量，而不是动手改。

四、带宽、roofline 与瓶颈定位

上一节把“时间”拆成份额，这一节把“字节”摆上秤。现代机器的算力和带宽是两个独立的物理量：算力由片上晶体管和时钟决定，带宽由引脚、走线和 DRAM 工艺决定，两者的增长速度从来不一样。性能分析必须回答的第二个问题因此是：这个程序的瓶颈在算力侧，还是在带宽侧？方向判断错了，优化就是把钱花在错误的屋顶上。本节给三件工具：Little 定律的直觉版，用来算“要多少在途请求才喂得满带宽”；内存墙的历史曲线，用来说明这个问题为什么一年比一年尖锐；roofline 模型，用一条拐线把任意 kernel 归类到算力瓶颈或带宽瓶颈。最后回到工程实质：大多数性能工作，说到底是在管理数据移动。

Little 定律的直觉版 排队论里的 Little 定律说：系统内平均在途数等于平均吞吐率乘以平均时延。它不假设任何排队规则，稳态下恒成立。用到内存系统上，形式特别直白：要维持某个带宽，必须随时有足够多的字节“在路上”——在途字节数等于带宽乘以时延。直觉画面是水管：水速一定时，管子越长，管中存的水越多；想维持出水量不变，加长水管就必须同时多灌水。

要喂满带宽，需要维持在途的字节数 = 带宽 * 时延

= $100 \text{ GB/s} * 100 \text{ ns}$

= $10^{11} \text{ B/s} * 10^{-7} \text{ s}$

= $10^4 \text{ B} = 10 \text{ KB}$

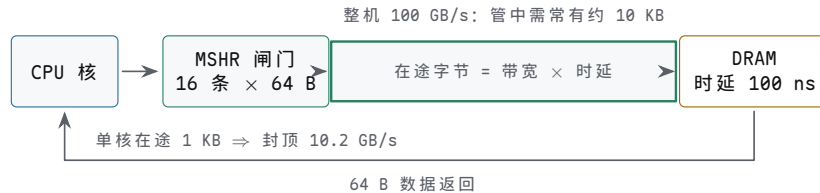
按 64 B 一条 cache 行折算：10000 / 64 = 156 个在途请求

结论：这台机器必须随时有约 156 行访存“在路上”，

在途数低于此值，带宽就吃不满

这个数字立刻解释了一个常见困惑：为什么单个核跑不出整机带宽。一个核同时在途的 miss 是有限的——主流乱序核的 miss 处理资源（MSHR）大约在十几条的量级，取 16 条、每条 64 B，在途预算就是 1 KB； $1\text{ KB}/100\text{ ns} \approx 10.2\text{ GB/s}$ ，这就是单核带宽的封顶。整机 100 GB/s 需要约 10 个这样的核同时发出访存，或者靠预取把单核的在途数撑大。

把这段算术画成水管，闸门宽度与管中存水的关系就一眼可见：



图示：Little 定律的直观版：时延 100 ns 的水管里要维持 100 GB/s，线上必须随时有约 10 KB 字节在途。单核的 MSHR 只有 16 条、在途预算 1 KB，于是单核带宽被闸门封顶在 $1\text{ KB}/100\text{ ns} \approx 10.2\text{ GB/s}$ ——水管再粗，闸门只开这么宽。

在途字节数	折合在途行	带宽上限 (100 ns 时延)	典型来源
256 B	4	2.6 GB/s	保守的顺序小核
512 B	8	5.1 GB/s	较小的 MSHR 配置
1 KB	16	10.2 GB/s	主流乱序核的 MSHR 量级
2 KB	32	20.5 GB/s	加强的 miss 处理
4 KB	64	41.0 GB/s	核加硬件预取缓冲合计
10 KB	160	102.4 GB/s	喂满整机带宽所需总量

这张表把乱序执行、MSHR、硬件预取三件事的统一本质说穿了：维持在途数。乱序让一条 miss 后面的独立指令继续走，走着走着往往又触发新的 miss，于是一个 miss 变成多个 miss 同时在途；MSHR 是在途请求的登记表，容量直接划定单核带宽上限；预取更彻底，把将来才会发生的 miss 提前变成现在的在途。第五章算预取提前量时见过同一个公式：miss 时延 200 拍、每 32 拍消化一行，提前量取 $200 \times \frac{1}{32} \approx 6.25$ 行、向上取 7 行——那正是“在途数等于时延乘消耗速率”的 Little 定律写法。第七章的教学机则是另一个极端：它一次只发起一笔访存，发起到完成是一个完整总线周期，期间没有任何别的请求在飞，在途数恒为 1，于是吞吐被时延直接钉死为 $1/\text{时延}$ 。它慢得坦白，也把“为什么在途数重要”衬得最清楚。

使用这条定律前先核对单位：带宽是“字节每秒”，时延是“秒”，乘积是“字节”——在途数是一个库存量，不是速率。任何“带宽乘时延”算出来不是字节的，都是单位代错了。

再把它翻译成一个日常类比。收银台每小时接待 60 位顾客（吞吐），每位顾客在店里平均待 6 分钟即 0.1 小时（时延），店里平均就有 $60 \times 0.1 = 6$ 位顾客（在途数）。想让店里人少，要么收银更快（提高吞吐、缩短停留），要么确实少来人——没有哪种办法能绕过这个乘积。内存系统同理：时延由 DRAM 物理决定，带宽目标由整机决定，唯一可调的旋钮就是在途数。

公式还可以反着用。已知单核在途预算 4 KB、DRAM 时延 100 ns，单核带宽上限是多少？ $4\text{ KB}/100\text{ ns} \approx 41\text{ GB/s}$ 。若工艺改进把时延降到 50 ns，同样 4 KB 的在途预算立刻值 82 GB/s——时延减半，同样的在途数能喂两倍的带宽。这正是各代内存一边提带宽、一边压时延的原因：两个参数通过同一个乘积耦合在一起。

反向演算：在途预算 4 KB

时延 100 ns：带宽上限 = $4096\text{ B} / 100\text{ ns} = 41\text{ GB/s}$

时延 50 ns：带宽上限 = $4096\text{ B} / 50\text{ ns} = 82\text{ GB/s}$

结论：在途预算固定时，带宽上限与时延成反比

把三种机器摆在一起，”在途数”这条线索就完整了：

机器	同时在途的访存	吞吐与时延的关系	关键问题
第七章教学机	恒为 1 笔	吞吐 = 1/ 时延	每个总线周期只做一笔，慢得坦白
简单顺序核	几笔	吞吐 $\approx$ 在途数 / 时延	MSHR 规模直接划定上限
乱序核加预取	几十到上百笔	同上，但在途数被刻意撑大	窗口、预取都在为在途数服务

从教学机到现代核，访存子系统的全部复杂化都可以读成一句话：在改变不了 DRAM 时延的前提下，把同时在途的请求数从 1 撑到上百。

内存墙：算力与带宽的历史分叉 为什么要费这么大劲维持在途数？因为算力和带宽的差距是几十年复利滚出来的。教科书反复引用的量级估计是：处理器算力大约每年增长 50%，DRAM 带宽（以及时延）大约每年改善 7%。这两个数字是说明斜率差的量级，不是某一年的精确统计，但它们的复利后果必须算出来看：

经过年数	算力倍数（年增 50%）	带宽倍数（年增 7%）	算力/带宽差距
0	1	1	1
5	7.6	1.40	约 5.4
10	57.7	1.97	约 29
15	437.9	2.76	约 159
20	3325	3.87	约 859

差距本身也是一条复利曲线：每年乘以 $1.5/1.07 \approx 1.4$ ，十年约 29 倍，二十年约 859 倍。”内存墙”不是某一年突然砌起来的墙，而是两条指数曲线的分叉口。它的后果链条在整机结构里随处可见：时延差拉大，cache 就从一级加到三级，容量越给越大；带宽差拉大，预取器、乱序窗口、MSHR 就越堆越多。这套机制是遮羞布，不是解药——它把墙的代价藏起来，藏的条件是程序有局部性。

遮羞布盖不住的负载一直存在：没有复用的流式负载直接撞墙，视频处理把帧数据过一遍就走，AI 推理把权重流一遍就走，大数据扫描把表读一遍就走；工作集一旦超过 cache 容量，miss 率回升，墙重新露头；预取本身消耗带宽，猜错的预取是纯浪费，第五章已经算过这笔得失。按局部性把负载分类，墙的表现一目了然：

负载类型	局部性	墙的表现	有效手段
稠密数值（分块矩阵）	高，复用充分	被 cache 遮住	分块、向量化
指针追踪（链表、图）	差，访问不可预测	时延墙全暴露	布局重排、池化分配
流式扫描（视频、日志）	空间有，时间无	带宽墙全暴露	预取、压缩、减少趟数
随机小消息（网络服务）	几乎无	双墙齐撞	批量、合并、缓存热集

这张表也预告了下一节的结论：对局部性差的负载，优化手段几乎全部围绕”少移动数据”展开，因为墙的那一侧没有更快的零件可买——DRAM 的 7% 不会因为你的程序着急而变成 50%。

这条历史曲线要读对两处。第一，50% 和 7% 是斜率不是现状：它们描述”差距

为什么越拉越大”，不描述“今天差多少倍”——今天的绝对差距取决于各代产品的具体参数，但斜率差决定了这个差距明年一定更大。第二，7% 指的主要是 DRAM 核心的访问时延与单位引脚带宽：行激活、预充电这些物理动作受电容充放电约束，不是堆晶体管就能解决的，这正是 DRAM 与逻辑工艺走势分叉的根源。

复利演算(年增 50% vs 年增 7%)：

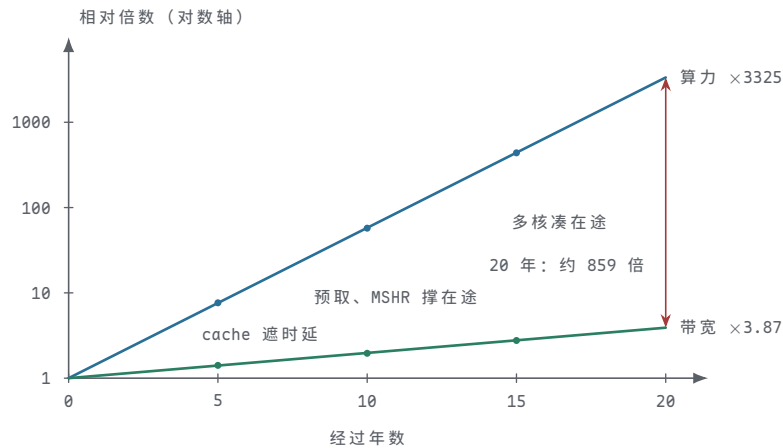
算力： $1.5^{10} = 57.7$ 带宽： $1.07^{10} = 1.97$

差距本身的年增长率： $1.5/1.07 = 1.40$

10 年： $1.40^{10} \approx 29$ 20 年： $(1.5/1.07)^{20} \approx 859$

读法：每过 5 年，同一程序里“等内存”的时间份额再上一档

把这两行复利画成半对数曲线，“分叉口”就成了两条直线的夹角：



图示：内存墙的历史分叉（半对数坐标）：算力年增 50%、DRAM 带宽年增 7%，两条指数曲线在对数轴上都是直线，斜率差就是分叉角。20 年后算力领先约 859 倍；楔形区里的多级 cache、预取与 MSHR、多核，是整机绕墙而行的遮羞手段——墙本身还在原地，且每年长高一点。

墙在整机设计上留下的痕迹，按出现顺序排大致是：先加 cache（遮时延），再加预取（遮带宽），再加深乱序窗口和 MSHR（撑在途数），再堆多核（用核数凑在途数）。每一步在前面两节的公式里都有对应的位置：

设计手段	遮的是哪一面墙	在本节公式里的位置
多级 cache	时延差	把有效时延从 100 ns 压向几 ns
硬件预取	带宽差	把将来的 miss 变成现在的在途数
乱序窗口、MSHR	在途数不足	直接抬升单核的在途预算
多核	单核在途数有限	用核数乘以每核在途数

注意这张表里没有一行是“让 DRAM 本身变快”——整机的全部聪明都花在绕过墙上，墙本身还在原地，且每年长高一点。

也要公平地说，工业界并非没有正面攻过墙，只是攻法都带限定条件：

修补手段	做法	改变了什么、没改变什么
宽接口存储（HBM 等）	把存储堆叠到逻辑芯片近旁，引脚以千计	带宽密度抬一个量级；容量与成本受限，拐点位置变了，墙还在
加大片上缓存	把更大的工作集留在片内	遮羞布更厚；无局部性的负载照旧撞墙
近存计算	把部分运算搬进存储侧	省掉来回搬运；适用面窄，编程模型未定型

这些手段的共同点是抬高局部的带宽密度，而不是抹平斜率差：逻辑与存储的复利曲线没有因此合拢，分叉仍在继续。

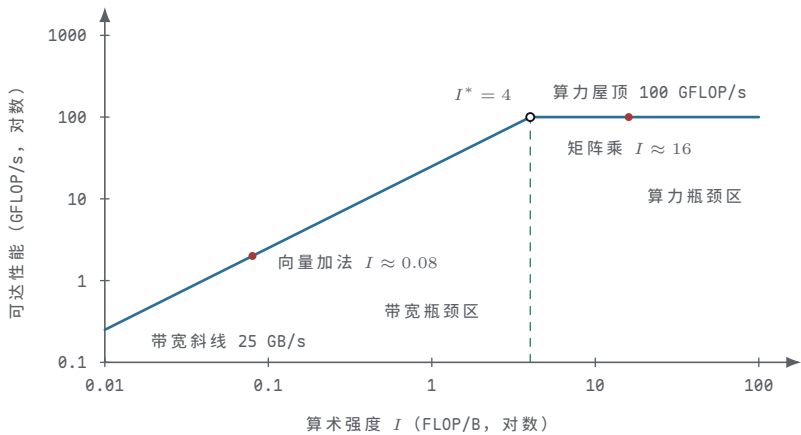
roofline 模型 roofline 模型把前面的讨论收成一条拐线。先给定义：算术强度 (arithmetic intensity) I 等于程序完成的浮点操作数除以它从 DRAM 搬运的字节数，单位 FLOP/B，这是程序的属性；可达性能等于峰值算力与“带宽乘算术强度”两者的较小者，这是机器给的上限；两者的交点叫拐点，拐点强度 I^* 等于峰值算力除以带宽，这是机器的属性。

可达性能(FLOP/s) = min(峰值算力, 内存带宽 * 算术强度)
拐点强度 $I^* = \text{峰值算力} / \text{内存带宽}$
 $I < I^*$: 带宽瓶颈(拐线左侧), 性能 = 带宽 * I , 随强度线性上升
 $I > I^*$: 算力瓶颈(拐线右侧), 性能 = 峰值算力, 平顶

给一台具体机器：峰值算力 100 GFLOP/s, 内存带宽 25 GB/s, 拐点 $I^* = 100/25 = 4$ FLOP/B。取两个 kernel：甲的算术强度 0.25 FLOP/B，乙的算术强度 16 FLOP/B。甲：带宽允许 $25 \times 0.25 = 6.25$ GFLOP/s，小于峰值 100，可达性能 6.25 GFLOP/s，带宽瓶颈，峰值只用到 6.25%；乙：带宽允许 $25 \times 16 = 400$ GFLOP/s，超过峰值，可达性能 100 GFLOP/s，算力瓶颈，峰值用满。

kernel	算术强度	带宽允许上限	峰值算力	可达性能	瓶颈
甲	0.25 FLOP/B	6.25 GFLOP/s	100 GFLOP/s	6.25 GFLOP/s	带宽
乙	16 FLOP/B	400 GFLOP/s	100 GFLOP/s	100 GFLOP/s	算力

把这台机器画成 roofline 图，两个 kernel 落在哪一侧、离屋顶多远，一眼可读：



图示：roofline 模型（双对数坐标）。斜线是带宽屋顶，水平线是算力屋顶，交点即拐点 $I^* = 4$ FLOP/B。向量加法 $I \approx 0.08$ 落在带宽瓶颈区，可达约 2 GFLOP/s；分块矩阵乘 $I \approx 16$ 落在算力瓶颈区，用满峰值。

左右两侧的优化顺序完全不同。左侧的 kernel 缺的是字节：先动数据—布局、分块、压缩、减少遍历趟数—把 I 往右推；右侧的 kernel 缺的是运算吞吐：这时才轮到动指令—向量化、FMA、多核—把峰值用起来。在左侧做向量化的结果，是把等待带宽的时间花得更快；在右侧做数据布局的收益，早就拿完了。roofline 最常见的误用是拿峰值算力评估左侧 kernel：“机器标称 100 GFLOP/s，我的程序怎么只有 6？”——因为程序在拐线左侧，峰值与它无关。章首表格里“拿峰值算力当可达性能”那条误用，指的就是这件事。

屋顶还会随机器移动：换一台机器，拐点位置就变了。三台机器的对照可以直接算：

机器	峰值算力	带宽	拐点强度	甲 (0.25)	乙 (16)
----	------	----	------	----------	--------

A	100 GFLOP/s	25 GB/s	4 FLOP/B	6.25, 带宽	100, 算力
B	200 GFLOP/s	25 GB/s	8 FLOP/B	6.25, 带宽	200, 算力
C	100 GFLOP/s	50 GB/s	2 FLOP/B	12.5, 带宽	100, 算力

甲在机器 B 上毫无变化—算力翻倍与它无关；在机器 C 上直接翻倍—带宽才是它的屋顶。乙正好相反。一句话总结采购逻辑：左侧 kernel 为带宽付费，右侧 kernel 为算力付费，升级机器之前先定位 kernel 在拐线的哪一侧。

算力屋顶其实不止一条：标量峰值、向量峰值、单核峰值、多核峰值各是一条水平线，而带宽屋顶通常只有一条。把算力屋顶抬高（加向量单元、加核），拐点就向右移，原先在右侧的 kernel 可能被划到左侧。看一个具体例子：给机器 A 再加三个核，峰值算力变 400 GFLOP/s，带宽仍是 25 GB/s，拐点从 4 移到 16 FLOP/B。

配置	峰值算力	带宽	拐点强度	乙 ($I=16$)	丙 ($I=10$)
单核	100 GFLOP/s	25 GB/s	4 FLOP/B	100, 算力	100, 算力
四核	400 GFLOP/s	25 GB/s	16 FLOP/B	400, 恰在拐 点	250, 带宽

kernel 丙在单核机器上是算力瓶颈，换到四核机器上反而成了带宽瓶颈：可达性能从 100 涨到 250，但离 400 的新峰值还差一截，而且再怎么优化指令也摸不到 400—带宽 25 GB/s 乘以强度 10 只允许 250。升级机器把瓶颈也升级了：算力侧的投入越多，剩下的 kernel 越集中到带宽一侧，这正是内存墙在 roofline 图上的样子。

另一个反例同样值得记住：向量点积 $c += a[i]*b[i]$ ，每对元素 2 次浮点操作、读 8 字节，算术强度 0.25 FLOP/B，与矩阵规模无关。给这种 kernel 做向量化，可达性能一格不涨—它在拐线左侧，上限是 $25 \times 0.25 = 6.25$ GFLOP/s，向量指令只让“等待”执行得更快。它的唯一出路是改变算法本身：把点积融合进一个更大的、有复用的计算里，人为抬高算术强度。

再补一个有复用但复用不足的中间态例子：五点 stencil（图像模糊、流体网格里到处都是）。每个输出点读 4 个邻居加自身、写回 1 个点，约 6 次浮点操作；若完全无复用，每点读 5 个 float、写 1 个，共 24 B，强度 $6/24 = 0.25$ FLOP/B；cache 利用行间复用后，平均每个输出点只需从 DRAM 新读约 1 个元素再加写回，约 8 B，强度升到约 0.75 FLOP/B。在机器 A 上可达 $25 \times 0.75 \approx 18.8$ GFLOP/s—比点积好三倍，仍然在拐线左侧。

五点 stencil 在机器 A 上：

无复用：6 FLOP / 24 B = 0.25 FLOP/B → 6.25 GFLOP/s

行间复用：6 FLOP / 8 B = 0.75 FLOP/B → 18.8 GFLOP/s

结论：cache 帮了三倍，瓶颈仍是带宽 ($0.75 < 4$)

点积、stencil、分块矩阵乘恰好排成算术强度的三个台阶：0.25（无复用）、0.75（行间复用）、16（块内复用）。三个台阶的名字都一样——“用了 cache”——效果差出两个数量级；问“cache 有没有用”之前，先问复用发生在哪一行、哪一块。

最后交代 roofline 模型的使用边界：它假设 kernel 处于稳态、算术强度按 DRAM 流量计算、忽略时延对短 kernel 的影响。启动阶段的冷 cache、毫秒级的小 kernel、时延受限的指针追踪，都不在这条拐线的适用范围里；那些场合要回到 Little 定律和上一节的时间拆分，一项一项算。

案例：矩阵乘法两个版本的算术强度 同一数学、两种写法，是演示 roofline 的经典案例。 $n \times n$ 单精度矩阵乘 $C = A \times B$ ，数学上是 $2n^3$ 次浮点操作，这一点两个版本完全相同；不同的是数据从 DRAM 走几趟，也就是算术强度。

朴素版(i-j-k 顺序, float):

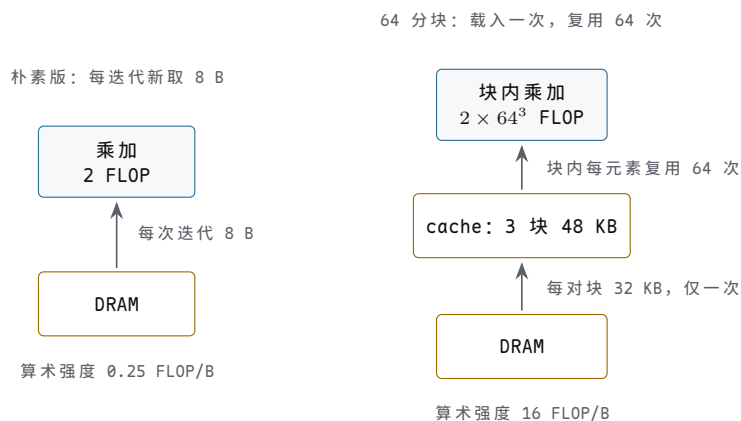
```
for (i) for (j) for (k)
    c[i][j] += a[i][k] * b[k][j];
```

内层每迭代：2 FLOP；若无 cache 复用，读 a、b 各 4 B，共 8 B
 算术强度 = 2 FLOP / 8 B = 0.25 FLOP/B

分块版(64 x 64 子块驻留 cache):
 for (ii) for (jj) for (kk) // 步长 64
 三个 64x64 子块驻留 cache
 for (块内 i) for (块内 j) for (块内 k)
 c[i][j] += a[i][k] * b[k][j];
 每对分块: $2 \times 64^3 = 524288$ FLOP
 DRAM 读入 $2 \times 64 \times 64 \times 4 \text{ B} = 32768 \text{ B} = 32 \text{ KB}$
 算术强度 = $524288 / 32768 = 16 \text{ FLOP/B}$

朴素版的强度 0.25 FLOP/B 与矩阵规模无关：矩阵再大，每次乘法加法还是要等新的字节，它永远停在拐线左侧，在机器 A 上只拿 6.25 GFLOP/s。分块版的强度 16 FLOP/B 来自复用：一个 64×64 的块载入一次，参与 64 次乘加，字节被反复使用；三个块的工作集 $3 \times 16 \text{ KB} = 48 \text{ KB}$ ，驻留 cache 毫无压力。注意一个整齐的关系：分块边长是 64，强度恰好从 0.25 提到 16，提升倍数就是分块边长——每维复用次数直接折算成算术强度。

把两个版本的数据流画出来，0.25 与 16 的差距就是字节走几趟的差距：



图示：同一段矩阵乘的两种数据流。左：朴素版内层每次乘加都从 DRAM 新取两个元素，2 FLOP 配 8 B，强度 0.25 FLOP/B，与矩阵规模无关。右： 64×64 分块后三个块共 48 KB 驻留 cache，每对块从 DRAM 读入 32 KB 只有一次，块内完成 2×64^3 次乘加——字节被反复使用，强度升到 16 FLOP/B。优化没有改动任何一次浮点运算，只改了字节走的路程。

版本	DRAM 流量	算术强度	机器 A 可达	峰值利用	瓶颈
朴素三重循环	每迭代 8 B	0.25 FLOP/B	6.25 GFLOP/s	6.25%	带宽
64 分块	每 524288 FLOP 仅 32 KB	16 FLOP/B	100 GFLOP/s	100%	算力

同一段数学，位于屋顶两侧。这次优化没有改动任何一个浮点运算，只改了字节走的路程——这就是第五章局部性原理在性能侧的兑现：cache 的价值不是“让访存变快”，而是让算术强度变大，把 kernel 从拐线左侧搬到右侧。工程次序也因此固定：先做分块把数据侧的问题解决，再谈向量化和多核把算力侧的峰值用满；顺序反了，就是把左侧 kernel 的等待做得更快。

64 这个数字不是魔法，它来自一个通式。设分块边长为 B ，每对分块做 $2B^3$ 次浮点操作，从 DRAM 读入两个 $B \times B$ 块共 $8B^2$ 字节，强度就是 $B/4$ ；代价是三个分块的工作集 $12B^2$ 字节必须驻留 cache。把几档 B 摆出来：

分块边长 B 的通式(单精度):
 每对分块 FLOP = $2B^3$
 DRAM 流量 = $2 * B^2 * 4 = 8B^2$ 字节

算术强度 $I = 2B^3 / 8B^2 = B/4$
 工作集 = 3 个分块 = $12B^2$ 字节
 $B = 32$: $I = 8$, 工作集 12 KB
 $B = 64$: $I = 16$, 工作集 48 KB
 $B = 128$: $I = 32$, 工作集 192 KB
 约束: 工作集必须驻留 cache, 否则复用落空, 强度退回左侧

强度随 B 线性上涨, 工作集随 B^2 上涨, 选 B 就是在这两条曲线之间找最大可行点: B 取到工作集刚好能驻留的那一档为止, 再大一档, 块本身开始被挤出 cache, 复用落空, 强度不但不涨反而退回左侧。这与第五章“cache 容量决定工作集上限”是同一条约束的两种说法。

还有一个容易混淆的优化要分清: 不调分块、只调换三重循环的次序 (例如 i-k-j), 能让内层循环改为沿行顺序访问, 空间局部性改善、冲突 miss 减少, 朴素版的 0.25 FLOP/B 会变得更“扎实”; 但换序不产生跨迭代的批量复用, 强度的量级不变, kernel 仍在拐线左侧。换序是把 0.25 做扎实, 分块才是把 0.25 变成 16—前者修访问模式, 后者改数据流量, 两件不同的事。最后从测量侧看一眼: 两个版本的浮点操作数完全一样, 性能计数器里不同的只是 cache miss 次数和 DRAM 流量; 下一节的测量工具会把这个差别直接摆成数字。

数据移动成本表 矩阵乘法的例子不是特例, 它背后是整机存储结构的统一图景。把全书零散出现过的访问时延收进一张表, 按数量级记—这些是每个硬件工程师该能脱口而出的预算数:

层级	典型时延	相对寄存器	比上一级慢	一句话
寄存器	约 0.5 ns	1 倍	—	几乎不计价, 但容量以百计
L1 cache	约 1 ns	2 倍	2 倍	容量以万字节计
L2 cache	约 5 ns	10 倍	5 倍	容量以十万字节计
DRAM	约 100 ns	200 倍	20 倍	容量以十亿字节计
SSD	约 100 μ s	200000 倍	1000 倍	第一级“出板”的存储
网络	约 1 ms	2000000 倍	10 倍	第一级“出机器”的存储

读这张表要看相邻级之间的断崖: 不是百分之几十的差别, 而是 2 倍到 1000 倍的跳变, DRAM 到 SSD 那一级最狠, 整整三个数量级—所以一次意外的缺页换页至今仍是事故级事件。能耗图景与时延同构: 教科书的能耗表里, 把一个字送到片外再取回的代价, 同样比一次片上运算高出几个数量级。时延和能耗两条曲线指向同一个结论: “优化等于把数据推进”, 这是大多数性能工作的实质。寄存器分配把热变量推进寄存器, 分块把热数据推进 L1 和 L2, 预取把将来的访问提前推进 cache, 压缩用更少字节把同样的信息推过带宽, 批量 I/O 把一千次小读换成一次大读—手段各异, 方向全部相同: 沿这张表向上走。

沿这张表向上走的具体手段, 按目标层级各归其位:

目标层级	把数据推进的手段	关键问题
寄存器	循环展开、寄存器分配、减少溢出	编译器与手工安排的分工
L1/L2	分块、布局重排、结构体拆分	工作集是否小于容量
DRAM	预取、压缩、减少遍历趟数	在途数是否喂满带宽
SSD	顺序批量代替随机小读	是否避免了同步等待
网络	合并消息、缓存热集、异步流水	是否把等待藏进计算

上面那张时延成本表的日常用法是心算预算。 $1\mu\text{s}$ 的预算里允许约 10 次 DRAM 访问、约 1000 次 L1 访问、约 2000 次寄存器操作；一个要求 1 ms 内应答的在线服务，满打满算约一万次 DRAM 访问，任何一次 SSD 读就直接吃掉预算的十分之一。性能评审时先问“这 1 ms 里各级访问各有多少次”，比先问“算法复杂度多少”更接近瓶颈。第七章的教学机在这张表里的位置也值得一看：它没有 cache，每次访存都按总线周期全额付费，等于所有访问都停在 DRAM 那一行。它能用 74HC 器件搭出来，是因为慢对它不是缺陷而是教学目标；现代机把同样的访问藏进表的前三行，才有了本节全部问题的起点。

数量级还差一个直觉。把寄存器的一次访问当成 1 秒，其余层级按同一比例放大，这张表立刻变成日常经验：

层级	真实时延	人类尺度	直观画面
寄存器	约 0.5 ns	1 秒	抬手就到
L1 cache	约 1 ns	2 秒	伸手拿桌上的东西
L2 cache	约 5 ns	10 秒	起身去书架
DRAM	约 100 ns	3 分半	下楼去仓库取货
SSD	约 100 μs	约 2.3 天	等一趟跨省快递
网络	约 1 ms	约 23 天	等一班跨洋海运

记住“3 分半”和“23 天”这两个数，很多设计决策就不用再讨论：为取一个值等 3 分半，当然要把顺路能拿的都拿上（cache 行一次取 64 B 而不是 4 B）；等 23 天的请求，当然不能在原地空等（网络请求必须异步、批量）。前四行由硬件自动管理—cache、预取器、MMU 替软件做决定；后两行的决策必须由软件显式做出—I/O 调度、批量、异步、流水。数量级的断崖划出了软硬件分工的边界：断崖之上，同步等待可以接受；断崖之下，同步等待就是事故。

预算分配演算把分工再具体化。一个 1 ms 应答的在线服务，一种可行的分配是：

1 ms 在线服务预算的一种分配：
 DRAM 访问 2000 次：200 μs
 L2 级访问 10000 次：50 μs
 留给计算与其余开销：750 μs
 红线一：任何一次 SSD 读（约 100 μs ）吃掉预算的十分之一
 红线二：任何一次跨机同步调用（约 1 ms）直接超支

这张预算表也解释了为什么高性能服务的架构评审总围绕“热数据是否在内存里、热路径是否不出本机”展开——不是洁癖，是数量级不允许。

本节三件工具是一条链：Little 定律告诉你带宽要用在途数去喂，内存墙告诉你为什么必须喂，roofline 告诉你喂饱之后瓶颈还在不在带宽。与上一节合在一起，性能分析的一半工具已经齐备：Amdahl 管时间怎么分，roofline 管字节怎么走；剩下一半是测量，由下一节完成。

五、性能测量与案例

前四节回答的是“应该有多快”：第一节把时延和吞吐分成两个量，第二节把总时间拆成指令数、CPI、时钟周期三个因子，第三节用 Amdahl 定律给出优化的上限，第四节用 roofline 区分算力瓶颈和带宽瓶颈。公式给出的是预期；机器实际跑多快，只能测量。本节回答“实际有多快、测到的数字可不可信”。

测量不是“跑一遍掐个表”。同样是“测一下”，墙钟计时、性能计数器、采样分析看到的是三个不同的世界；测到的数字里混着冷启动、随机噪声和混淆变量；连“两组数字能不能比”都有一份要先过的清单。本节先讲测量的层次，再讲误差控制，然后讲计数器的读法，再用三个案例把完整流程各走一遍，最后给出性能报告的固定格式。

测量的三个层次 测量手段按“能看到什么”分三层，三层回答三个不同的问题，谁也不能替代谁。

第一层是墙钟计时 (wall-clock time): 在程序起点和终点各读一次时钟求差，得到第一节定义的时延。它回答“用户等了多久”，这是性能的最终定义——所有其他测量都只是为它找解释。

墙钟计时的代码只有三行，任何语言都写得出：

```
t0 = now();           /* 读单调时钟 */
work();
t1 = now();
elapsed = t1 - t0;    /* 墙钟时延，含全部系统噪声 */
```

墙钟的优点是定义简单、无处可藏；缺点是它把这段时间里系统发生的全部事情都计了进去：操作系统把 CPU 调度给别的进程、中断处理、定时器滴答、主频按需升降、邻居程序把 cache 挤掉，全部算在读数里。所以墙钟的精度上限不在时钟本身——现代系统的时钟分辨率早已到纳秒级——而在系统噪声：同一程序连测十次，读数差出 10% 是常态。

第二层是性能计数器 (performance counter): CPU 内部一组可编程的硬件寄存器，先选定事件类型（退休一条指令、一次 cache miss、一次分支误测），硬件此后每检测到一次该事件就加一。它回答“硬件发生了什么”，精确到单个事件；被测进程不在 CPU 上时，它的事件本来就不发生，所以读数几乎不受调度和后台负载污染。

计数器的成本在两头：一是必须先知道该看哪个事件——几十个事件名摆在面前，没有假设就无从下手；二是计数器的物理数量有限，典型同时可编程 4 到 8 个，想看的事件多于这个数就得分几轮测，工具自动多路复用读数还会带上“本轮未全程计数”的标记。

第三层是采样分析 (sampling profiler): 以固定频率打断程序，记录当时的程序计数器和调用栈，事后按函数归并样本。它回答“时间花在哪段代码”：占样本 70% 的函数就是热点，优化从那里开始。

采样是统计测量，分辨率由频率和总时长决定。典型频率每秒 99 次：跑 10 秒的程序约采 990 个样本，一个占时 12% 的函数期望分到约 119 个样本，计数本身的统计起伏就有 $\pm\sqrt{119} \approx \pm 11$ 个，即占比估计自带约 $\pm 1\%$ 的不确定。总共只跑 2 ms 的函数，平均摊不到四分之一一个样本——采样找不到它，不等于它不慢，而是这个方法看不见它。

层次	回答的问题	精度	成本与局限
墙钟计时	用户等了多久	时钟分辨率高， 读数含全部系统 噪声	最简单；说不清时间 花在哪里
性能计数器	硬件发生了多少事件	精确到单个事件	要先选对事件；同时 可测事件数有限
采样分析	时间花在哪段代码	由采样频率与总 时长决定	短函数、罕见路径会 漏；不解释原因

三层的分工是从外到内：先用墙钟确认“确实慢了、慢了多少”，再用采样定位“慢在哪个函数”，最后用计数器回答“为什么慢”。顺序反过来就会走弯路：只报墙钟数字，改进无从下手；一上来就读计数器，可能在只占 2% 时间的代码上数了半天事件。

三层其实相连：计数器计到指定次数可以溢出触发中断，采样器借此把“哪条指令在 miss”也记录下来——事件采样是计数器的精度加采样的位置信息。第七章的教学机是天然的对照组：它没有性能计数器，测一段程序只有秒表和手工数指令两个手段，墙钟加“知道执行了多少条指令”就是全部测量学。计数器不是更高级的玩具，而是把第四章的 CPI 公式做成了可以直接读数的硬件。

墙钟计时还有一个实现细节：读哪只钟。日历时钟（墙上的时间）可能被校时跳变，测时间间隔要用单调时钟（单调递增、不受校时影响）；在第七章的教学机上没有操作系统，直接数时钟周期反而更本质——墙钟的“钟”从来就是周期计数，只是被系统调用封装成了时间单位。

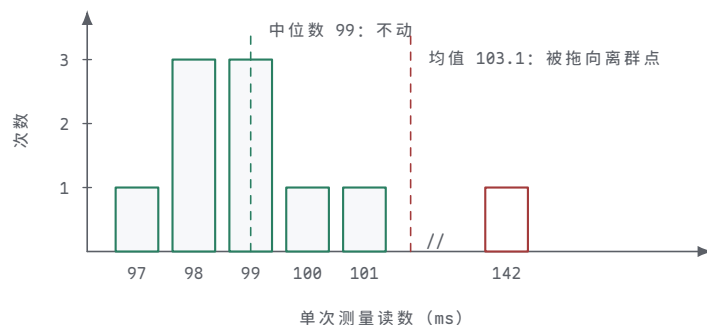
三层各有趁手的工具：墙钟一层是 `time` 命令和语言自带的计时函数；计数器一层是 `perf stat`、VTune 这类直接读硬件寄存器的工具；采样一层是 `perf record`、`gprof`。选工具先看要回答的问题属于哪一层，再看精度够不够用——拿 `time` 去定位热点，和拿采样器去报单次时延，是用错了层次。

消除测量误差 测量的目标量是“这段代码在稳定状态下花多少时间”，而任何一次真实测量都混着三类与代码无关的成分。误差控制不是把数字测得更“准”，而是把这三类成分分别从读数里剥掉或固定住。

第一类是冷启动。第一次运行时，I-cache、D-cache、TLB、分支预测器全是空的，输入文件可能还躺在磁盘页缓存之外，CPU 主频可能还没升上来。某函数稳态 9.4 ms、首次冷跑 41 ms，4 倍多的差距一点不稀奇。冷启动本身是有意义的量——用户第一次使用就是冷的一但它和稳态是两个量，必须分开报告：要么先跑若干次预热再计时，要么明确给出“冷”“热”两组数字，混着报等于什么都没报。

第二类是随机噪声。调度、中断、后台进程让每次读数略有不同，读数序列常常双峰：大部分时候安静，偶尔被狠狠干扰一次。看一组十次读数 (ms)：98、99、101、97、142、98、99、100、98、99。均值 103.1，中位数 99，最小值 97——均值被 142 那一个离群点拖高了 4%，而代码本身什么都没变。

把这十次读数画成直方图，均值如何被拖离群体就一目了然：



图示：十次读数的分布（横轴在 101 之后断轴）：九次聚在 97–101 ms，一次离群到 142 ms。中位数 99 不受离群点影响；均值 103.1 被它拖高 4%，偏离了九次读数所在的群体。所以这组读数要报告中位数加离散程度；在“机器本身能跑多快”的问题上，最小值反而是受噪声污染最少的估计。

对这种序列，取均值是最差的汇总方式：既被离群值拖着走，又掩盖双峰。正确做法是测几十次、报告分布：中位数加离散程度；在“这台机器本身能跑多快”的问题上，最小值反而是受噪声污染最少的估计——没有任何机制能让程序“虚假地变快”，却有无数机制让它变慢。次数的底线是中位数要稳定：从 30 次起步，加到读数不再明显移动为止。

第三类是混淆变量：两次比较之间改了不止一个东西。最典型的翻车现场是“优化后快了 30%”，复查发现输入从 10^6 个元素换成了 10^5 个——时延当然变了，和代码无关。固定做法是一句老生常谈但最容易被违反的话：一次实验只改一个因子。

还要注意系统误差，它不像随机噪声那样能靠多次测量摊掉：地址空间布局随机化让每次运行的 cache 冲突略有不同，链接顺序变化能让同一源码的实测差出几个百分点，主频调节策略（省电还是性能）更是全程一致的偏移。这类误差只能靠固定环境和明确声明来控制，靠平均消不掉。

一个具体例子：同一份源码、同一台机器，仅改变链接顺序，热循环与某个大数据结构的 cache 组映射关系就变了，实测差出 5%——不是代码变快或变慢，是布局运气变了。对这类误差的诚实写法是报告分布并声明布局条件，而不是挑一个最好看的数字。

把三类误差倒过来看，就得到测量计划的固定写法：动手之前先写假设（“这个改动应把 cache miss 率从 56% 降到 12%”），再写要测的事件与次数，再对照清单

固定环境，最后才运行程序。计划写在测量之前；”想到什么测什么”，本身就是误差最大的测量方法。

”这组数字能不能比”，是引用任何对比数字之前要先问的问题。下面这张清单建议逐条过：

检查点	关键问题	不满足时的后果
同一输入	两次运行的数据规模和内容是否相同	时延随输入变化，比较失去意义
同一环境	机器、主频策略、后台负载是否一致	环境噪声可以大于被测差异
同一构建	编译器版本与优化选项是否一致	差异来自编译器而非代码改动
冷热状态	两次是否都预热，或都明确测冷启动	冷热混测可差出数倍
样本与分布	次数是否足够，报中位数还是均值	单次读数可能只是离群点
单位与定义	报的是时延、吞吐还是加速比	不同量纲的数字不能互比

工程含义：性能评审里的大量争论——“我的版本快 20%”——不可能，我这里还慢了——最后常常发现一边测的是冷启动、一边测的是稳态，或者一边开了 -O2、一边没开。把测量约定写在数字前面，数字才有意义；清单上任何一条答不上来，结论先打回重测，不丢人。

性能计数器能回答什么 计数器把第四章的公式变成可以直接读数的量。第四章给出总时间 = 指令数 × CPI × 时钟周期，并把有效 CPI 拆成基础值加访存、分支等各项代价；cycles 和 instructions 两个事件正好对应公式的前两个因子，其余事件对应各项代价的分子与分母。

最常用的四个比值：IPC (instructions per cycle, instructions/cycles, 即 CPI 的倒数；四发射机器的理论上限是 4, 实测 0.83 意味着近八成发射槽在空转)；cache miss 率 (cache-misses/cache-references, 第五章的命中率就是 1 减去它)；分支误测率 (branch-misses/branch-instructions, 每次误测都要按第四章说的清空错误路径，付出十几拍)；TLB miss 率 (dTLB-load-misses/dTLB-loads, 每次失效意味着额外的页表遍历访存)。

Linux 下一条 perf stat 就能报出这组数字，输出形态如下（数字为示意）：

```
$ perf stat ./prog
 6,291,456 cycles
 5,242,880 instructions          # 0.83 insn per cycle
 2,097,152 cache-references
  41,943 cache-misses           # 2.0% of all cache refs
1,048,576 branch-instructions
 31,457 branch-misses           # 3.0% of all branches
```

读这组数字就是按第四章的有效 CPI 公式做分解。cache 一项：每条指令平均 0.4 次数据访存，miss 率 2%，miss 代价 50 拍，贡献 $0.4 \times 0.02 \times 50 = 0.40$ 拍/指令。分支一项：分支占指令两成，误测率 3%，每次罚 15 拍，贡献 $0.2 \times 0.03 \times 15 = 0.09$ 拍/指令。基础 CPI 若为 1.0，有效 CPI 就是约 1.49——分解做完，该查哪一侧已经写在算术里。

IPC 与 CPI 互为倒数，只是同一件事的两种说法：perf 习惯报 IPC，第四章的公式用 CPI，换算只要取倒数——IPC 0.83 就是 CPI $1/0.83 \approx 1.20$ 。报告里用哪个都可以，但一份报告内部要统一，免得读者把 0.83 和 1.20 当成两台机器的数字。

使用计数器还有两个实务提醒。一是读硬件计数器需要权限，容器和虚拟机里事件可能被屏蔽或换成虚拟值，正式测量前先用一个行为已知的小程序标定：比如空转的时钟周期数应约等于时长乘主频，对不上就说明读数通道有问题。二是事件定

义因微架构而异，Intel、AMD、ARM 上同名事件的统计边界可能不同，报告里要写清机器型号，否则别人无法复核。

perf 事件名	含义	典型比值与判断
<code>cycles、instructions</code>	总拍数与退休指令数	IPC 远低于发射宽度，停顿多，继续查 miss 与误测
<code>cache-references、cache-misses</code>	末级 cache 访问与失效	miss 率超过几个百分点，访存是瓶颈候选
<code>branch-instructions、branch-misses</code>	分支总数与误测次数	分支占两成、罚 15 拍时，误测率每 1% 抬高 CPI 约 0.03
<code>dTLB-loads、dTLB-load-misses</code>	数据 TLB 访问与失效	失效偏高，查大页与跨页访问模式
<code>L1-dcache-load-misses</code>	L1 数据 cache 读失效	与末级 miss 对照，定位失效发生在哪一级
<code>L1-icache-load-misses</code>	指令 cache 失效	热代码 footprint 超过 L1i 的信号（见案例三）

计数器的正确读法是先假设、后验证，而不是把事件全测一遍看哪个大。完整流程是：先根据现象提出一个可证伪的假设（“这段代码慢是因为 cache miss 多”）；再选一组能区分假设成立与否的事件（`cache-references` 与 `cache-misses`）；最后看读数支持还是推翻。假设被推翻同样是收获：实测 miss 率只有 0.3%，”cache 瓶颈”这条线就此划掉，转查分支误测或前端取指。没有假设的直接读数，面对几十个事件名只会越看越糊涂——计数器回答”是不是”和”有多少”，假设才决定该问哪个问题。

四个比值还天然排成一个排查顺序：先看 IPC 低不低，确认确实在停顿；再看停顿来自哪一侧——cache miss 率高查数据局部性，分支误测率高查代码形态，TLB miss 率高查页面布局，I-cache miss 高查代码体积。下面三个案例各按这个顺序走一遍。

案例一：矩阵转置的 cache 行为 任务：把 1024×1024 的双精度矩阵 A 转置写入 B。每个元素 8 字节，整矩阵 8 MiB。机器参数沿用第五章的设定：cache line 64 B（恰好装 8 个 double），L1 数据 cache 32 KiB（512 条 line），L2 256 KiB，写策略 write-allocate。两个朴素版本只差循环顺序，第三个版本做 8×8 分块：

```
/* 版本甲：i 外层 j 内层。A 按行读（连续），B 按列写（跳） */
for (i = 0; i < 1024; i++)
    for (j = 0; j < 1024; j++)
        B[j][i] = A[i][j];

/* 版本乙：j 外层 i 内层。A 按列读（跳），B 按行写（连续） */
for (j = 0; j < 1024; j++)
    for (i = 0; i < 1024; i++)
        B[j][i] = A[i][j];

/* 版本丙：8x8 分块，块内完成小转置 */
for (ii = 0; ii < 1024; ii += 8)
    for (jj = 0; jj < 1024; jj += 8)
        for (i = ii; i < ii + 8; i++)
            for (j = jj; j < jj + 8; j++)
                B[j][i] = A[i][j];
```

先按第五章的局部性原理算理论值。版本甲：A 按行连续读，一条 line 装 8 个元素，前 7 次访问命中同一条 line，读侧 miss 率 $1/8 = 12.5\%$ ，全部是不得不付的强制失效；B 按列写，相邻两次写地址相差一整行 8 KiB，每次写都访问一条不

同的 line，一列 1024 个元素没写完，开头的 line 早被挤出 32 KiB 的 L1，写侧 miss 率接近 100%。版本乙把好坏对调：B 连续写 12.5%，A 跳跃读接近 100%。版本丙的每个 8×8 块内，A 的 8 行各贡献一条被用满的 line，写入 B 的 8 条 line 同样被用满，两侧都回到 12.5% 的强制失效下限。

理论 miss 数可以直接算：读侧、写侧各 1,048,576 次访问；连续方向 miss $1,048,576/8 = 131,072$ 次，跳跃方向约 1,048,576 次。预热 3 次、各测 30 次取中位数，实测与理论吻合（教学测量，示意数据）：

版本	A 侧 miss 率	B 侧 miss 率	总 miss 数	墙钟时间
甲（A 顺 B 跳）	12.5%	≈100%	1,179,648	28.3 ms
乙（A 跳 B 顺）	≈100%	12.5%	1,179,648	31.1 ms
丙（ 8×8 分块）	12.5%	12.5%	262,144	9.4 ms

第一点值得展开的是那 8 倍：miss 率 12.5% 对 100% 不是“快一点”的程度差别，而是一条 line 里装了几个元素的结构性差别。空间局部性按 line 计、不按字节计：顺着 line 走，一条 line 服务 8 次访问；跨着 line 跳，一条 line 只服务 1 次。第五章讲的“取回一整行”，在这里变成了可以精确预测的 miss 率。

第二点是甲、乙总 miss 数完全相同（都是 $131,072 + 1,048,576$ ），时间却是 28.3 ms 对 31.1 ms。这不是噪声：读 miss 直接挡住等数据的指令，写 miss 可以被写缓冲吸收一部分，同一个 miss 数，读侧的代价更大。调换循环顺序不是免费的优化，要算清哪一侧更怕 miss。

第三点是步长 8 KiB 恰好是 2 的幂：按列访问在组映射下集中命中少数几个组，即便容量还剩，冲突失效也把 miss 率顶在接近 100%。这也是高性能矩阵代码常给行加 padding、避开 2 幂行距的原因。

TLB 也在同一场景里承压：矩阵占 $8 \text{ MiB}/4 \text{ KiB} = 2048$ 个页，按列走时连续访问以 8 KiB 为步长跨页，一列没走完 dTLB 已经换了好几轮，dTLB-load-misses 随 cache miss 一起上涨。计数器之间会互相印证——这正是“先假设后验证”里假设越来越具体的形态。

跳跃写的代价里还藏着 write-allocate 的一份：写 miss 要先把整行取进 cache 再改，B 侧的实际内存流量是“取一遍加写回一遍”的两倍。版本乙把写侧变连续后，写合并和写缓冲才发挥得了作用。转置这个例子里，第五章的每一条 cache 规则都至少出场一次。

硬件预取器在同一场景里的表现也值得一提：对连续流（版本甲的 A 侧、版本乙的 B 侧），预取器能把一部分强制失效的时延提前隐藏，实测时间略好于纯 miss 数推算；对 8 KiB 步长的跳跃流，多数预取器直接放弃。这解释了为什么跳跃方向的代价往往比“miss 数乘 miss 时延”的估算更硬——它连预取这根拐杖都用不上。

回看这个案例该测哪些事件，清单几乎是自己写出来的：L1-dcache-load-misses 验证读侧，cache-misses（含写失效）看总量，dTLB-load-misses 看跨页，再加 cycles、instructions 换算 CPI——五条事件足够判定“局部性假设”成立与否。事件清单之所以短，是因为假设先写清楚了。

朴素转置的读写两侧方向天然相反，单层循环顺序怎么换都只能照顾一边；分块改的是数据粒度而不是访问次数。块多大合适？把块尺寸扫一遍就清楚了（其余条件同前）：

块尺寸	B 侧复用窗口	总 miss 数	墙钟时间
8×8	8 条 B line 加 1 条 A line	262,144	9.4 ms
64×64	64 条加 8 条	262,144	9.3 ms
256×256	256 条加 32 条	262,144	9.4 ms

512 × 512	512 条加 64 条， 超过 L1	$\approx 4.4 \times 10^5$	13.0 ms
1024 × 1024 (不分块)	1024 条加 128 条	1,179,648	28.3 ms

窗口的意思是：对 B 的一条 line 写完 8 个元素，要横跨一整轮块内循环，期间触碰的 line 数就是“块高条 B line 加块宽除以 8 条 A line”。窗口小于 512 条 L1 容量时，每个块都贴着强制失效下限 $2 \times 131,072 = 262,144$ ；窗口超过容量，line 在复用之前被挤出，miss 率回升；块大到等于不分块，就退回版本甲。分块参数不是调出来的魔法数，是用 cache 容量算出来的。

这正是第四节 roofline 中“cache 友好版”矩阵算法的同一招：不减少访问次数，改变访问顺序让每条 line 被用满。也请注意时间与 miss 数并非严格正比（miss 数 4.5 倍，时间 3.0 倍）——乱序执行和写缓冲会掩盖一部分停顿，精确的时间模型留给机器实测，计数器负责把方向指对。

案例二：循环展开换来的 CPI 任务：对 1,048,576 个 double 求和。机器 2.0 GHz、四发射，加法延迟 1 拍，L1 命中的 load 可流水供应。基础版循环体每元素 5 条指令：load、add、下标加一、比较、条件跳转——前两条是“正事”，后三条是循环开销。

先算指令数因子。基础版： $5 \times 1,048,576 = 5,242,880$ 条。展开 4 次后每 4 个元素 11 条指令（4 load、4 add，下标、比较、跳转各 1），共 $262,144 \times 11 = 2,883,584$ 条，省了 45.0%——省的正是被摊薄的循环开销。分支数同步从 1,048,576 降到 262,144。

再算 CPI 因子，这一步的关键是用独立累加器 sum0 到 sum3 分别累加、出循环再相加。原因在依赖链：基础版的 sum 每次 add 都等上一次 add，链长等于元素个数，四发射机器被这条链钉在每元素约 1.2 拍；四条独立累加链让加法流水填满，CPI 降到 0.70。

```
sum0 = sum1 = sum2 = sum3 = 0;
for (i = 0; i < N; i += 4) {
    sum0 += a[i];      /* 四条累加链互相独立 */
    sum1 += a[i+1];
    sum2 += a[i+2];
    sum3 += a[i+3];
}
sum = sum0 + sum1 + sum2 + sum3;
```

把展开倍数扫一遍，两个因子的此消彼长就全在表里（同机实测，预热后取 30 次中位数）：

展开倍数	指令数	分支数	CPI	墙钟时间
1	5,242,880	1,048,576	1.20	3.15 ms
2	3,670,016	524,288	0.95	1.74 ms
4	2,883,584	262,144	0.70	1.01 ms
8	2,490,368	131,072	0.75	0.93 ms
16	$\approx 2,523,000$	65,536	0.90	1.14 ms

演算核对：基础版 $5,242,880 \times 1.20 = 6,291,456$ 拍，除以 2.0 GHz 得 3.15 ms；展开 4 次 $2,883,584 \times 0.70 \approx 2,018,509$ 拍得 1.01 ms。对基础版的总加速约 3.1 倍，其中指令数贡献 1.82 倍（ $5,242,880/2,883,584$ ）、CPI 贡献 1.71 倍（ $1.20/0.70$ ），两个因子相乘、方向一致——这不多见，案例三会给反例。

分支一项值得说清：循环分支几乎 100% 可预测，展开前后误测率都低到可以忽略，收益不在误测率，而在每元素少付 2.25 条开销指令和依赖链变短。看优化收益时，先分清它作用在公式的哪个因子上。

展开 16 次开始倒贴：理论上每 16 个元素 35 条指令、共 $65,536 \times 35 = 2,293,760$

条，但 16 个累加器加 16 个 load 目标超出 16 个通用寄存器，编译器把中间值溢写到栈上，实测指令数回升约一成，且溢写引入新依赖，CPI 回到 0.90，时间反弹到 1.14 ms。8 次的 0.93 ms 是这台机器上的甜点：8 个累加器加 8 个 load 目标，恰好顶到寄存器堆容量的边缘。

收益边界由两个物理量写死：寄存器数量和代码体积。累加器与 load 目标各占一个寄存器，展开倍数越过容量就开始用 load/store 溢写还债；代码体积按倍数乘循环体大小增长，涨过 I-cache 容量就是案例三的剧情。还有一点：结论不可随输入类型外推，整数加法与双精度加法的延迟不同、链的限制不同，换数据类型要重测——这正是控制变量那一节的要求。

两个相关问题顺手说清。其一，现代编译器在 -O2 下本来就会做适度展开和调度，手写展开前要先看编译器已经做到了哪一步——“编译器自动展开”与“手写四累加器”的差，才是手写这一笔的真实收益，否则容易把编译器的功劳算在自己名下。其二，展开常与软件流水混淆：展开是复制循环体、摊薄开销，软件流水是把不同迭代的阶段错开重叠；两者都打破依赖，但流水不改变指令总数，展开改变。

展开还是向量化的前奏：四条独立累加链的下一步，往往是一条指令同时算四路的 SIMD。从这个意义上说，先把依赖链打断、再谈更宽的执行单元，顺序不能反——链不断，多宽的部件都在等上一次的 sum。

案例三：一次错误的“优化” 某协议栈里有一个 24 字节的小循环——逐字节查表更新 CRC——调用非常频繁，采样分析也确认它是热点。优化者决定把它内联到全部约 40 个调用点，顺手把循环完全展开。推导无懈可击：省掉每次调用的 call/ret、参数搬运和栈帧调整，动态指令数确实降了 8%，从 1.00×10^8 条降到 9.2×10^7 条。

版本	指令数	L1i miss	CPI	墙钟时间
内联前	1.00×10^8	0.2×10^6	1.10	55.0 ms
内联加全展开	0.92×10^8	3.1×10^6	1.50	69.0 ms

慢了 25.5%。演算核对：内联前 $1.00 \times 10^8 \times 1.10 = 1.10 \times 10^8$ 拍，55.0 ms；内联后 $0.92 \times 10^8 \times 1.50 = 1.38 \times 10^8$ 拍，69.0 ms。指令数降了 8%，CPI 却从 1.10 涨到 1.50，总账由盈转亏。

CPI 为什么涨：内联加完全展开后，主循环的指令 footprint 从 24 KiB 涨到 40 KiB，超过 32 KiB 的 L1 I-cache，每一轮主循环都要从 L2 重新取一部分指令。L1-icache-load-misses 从 0.2×10^6 涨到 3.1×10^6 ，每次 miss 从 L2 取回约付 10 拍，新增 2.9×10^6 次就是约 2.9×10^7 拍——比省下的 8×10^6 条指令折合的全部收益还多。第五章讲 D-cache 的那套局部性逻辑对 I-cache 一字不差地成立：取指也是访存，代码也有 footprint。

```
$ perf stat -e cycles,instructions,L1-icache-load-misses ./before
110,034,512 cycles    100,012,440 instructions    198,744 L1i
$ perf stat -e cycles,instructions,L1-icache-load-misses ./after
138,120,977 cycles    92,011,305 instructions    3,104,882 L1i
```

教训有两条。其一，“优化”是个测量命题，不是逻辑命题：推导再漂亮也只是假设，必须按本节的流程回测——先测基线，只改一个因子，再测，对照核对清单逐条过。其二，假设要选对因子：如果预先的假设是“省指令数就是省时间”，instructions 下降 8% 会漂亮地“确认”假设；只有同时读 cycles 和前端取指事件，才会发现假设漏掉了 I-cache 这个变量。指令数和 CPI 是两个独立的因子，只算其中一个的优化，连方向都不能保证。

同类事故还有别的形态：头文件里的大宏在每个调用点展开、模板实例化让同一份逻辑复制几十份、过度循环展开——机制相同，都是热路径的代码 footprint 悄悄涨过 I-cache。正确做法是给代码体积设预算：改动前后各测一次 L1-icache-load-misses，热路径 footprint 不许越过 I-cache 容量这条线；真要内联，只内联最热的一两个调用点，而不是全部四十个。

把这次失败按本节的流程复盘一遍，每一步都对应得上：采样确认热点（第三层）→ 提出假设”内联省指令就是省时间”→ 只改代码结构这一个因子（控制变量做对了）→ 回测墙钟发现变慢（第一层）→ 用计数器定位到 L1i（第二层）→ 假设被推翻，撤回改动。流程本身没有失败，失败的是没走流程就合入——这正是”先假设后验证”最值钱的场景。

工程上的默认动作因此只有一条：任何以”快”为目标的改动，合入前必须附回测数据，且按报告模板的五段写全。没有回测数据的”优化”一律按未优化处理——这条规矩看起来慢，实际省掉的是案例三这种来回。

性能报告模板 测量做到最后，要变成别人能复核的文字。一份最低限度可信的性能报告固定五段：环境、方法、数据、分析、结论。

环境段写机器与主频、cache 层次参数、编译器与选项、输入规模——换一台机器时，读者据此判断哪些数字会跟着变。方法段写测了什么事件、冷热如何处理、测了多少次、报的是中位数还是均值——这一段决定数字能不能和别人的比。数据段放原始测量表，不加评论。分析段做比值演算，明说每个假设被支持还是被推翻。结论段只写可执行的判断，不写感想。

各段都有典型错误：环境段漏写主频调节策略；方法段只写”测了几次取平均”；数据段只给加速比不给原始值；分析段把假设成立当成理所当然，不给演算；结论段写”明显更快”这种无法执行的句子。方法段的合格写法可以直接照抄这个句式：”预热 3 次后连续测量 30 次，表中数值为中位数；两次运行之间仅修改循环顺序，编译选项固定为 -O2，机器无其他负载。”

数据段不加评论是有道理的：原始数字是报告中唯一不随作者立场改变的部分，评论写进分析段，读者才能把”你测到了什么”和”你怎么解释”分开复核。两者混在一起的报告，复核者要先把它们剥开才能检查。

结论段的合格句式是”在什么条件下、采用什么版本、预期多少收益”，例如：”在 32 KiB L1d、line 64 B 的机器上，矩阵转置采用 8×8 分块版，预期比朴素版快约 3 倍。”条件写在前面，是因为条件一变结论就可能失效——这正是环境段存在的意义。

按案例一填写的一份完整样例：

段落	写什么	样例（按案例一填写）
环境	机器、cache、编译选项、输入	2.0 GHz 四发射；L1d 32 KiB、line 64 B、write-allocate；-O2；1024 × 1024 double
方法	测什么、冷热、次数、统计量	perf stat 测 cache-misses 等；预热 3 次后测 30 次，报中位数
数据	原始测量表	三版总 miss 数 1,179,648 / 1,179,648 / 262,144；时间 28.3 / 31.1 / 9.4 ms
分析	比值演算与假设检验	假设”跳跃方向 miss 率接近 100%”，实测与 1/8、8/8 的理论值吻合，假设成立；分块回到强制失效下限
结论	可执行判断	采用 8×8 分块版；单层调换循环顺序只改好坏归属，不改总 miss 数

报告还有一个跨版本的作用：同一台机器、同一份方法，性能数字才能按版本追踪，回归才有人认。模板五段一段不省，后来者才能在不重测的情况下，判断你的结论在什么条件下成立。

回到本章开头那句话：先测量，再优化，最后验证。三个案例是同一句话的三次应用——案例一把”局部性好”从形容词变成 8 倍的 miss 率差；案例二说明收益要按因子拆开算，指令数和 CPI 会朝相反方向走；案例三说明不回测的”优化”连方向都不能保证。会算公式、会读计数器、会写报告，这一章的工具才算齐备。

章末练习

任务	要求	学习目标
时延/吞吐区分	某服务单核处理一个请求的时延为 2 ms，请求一到就处理、无排队。求单核吞吐（请求/秒）；增加一个相同的核并行后，吞吐与单请求时延各变为多少；再举出一个“吞吐翻倍而时延不变”的硬件机制	分清两个量纲，知道各自何时独立变化
加权 CPI	某程序指令构成为 ALU 50%、load 20%、store 10%、branch 20%，四类 CPI 分别为 1、2、1、3。求平均 CPI；分支预测改进使 branch 类 CPI 降为 2 后，求新的平均 CPI 与总时间变化百分比（指令数与主频不变）	平均 CPI 按占比加权，总时间同比缩放
Amdahl 排序	某程序中浮点运算占 40% 时间、cache 访问占 30%、分支占 20%；三个候选优化分别把对应部分加速 4 倍、2 倍、1.5 倍，且作用互不重叠。用 Amdahl 定律分别计算整体加速比并给出实施排序	占比决定优先级，倍数不等于收益
roofline 落点	某机器峰值 64 GFLOPS、内存带宽 16 GB/s。计算拐点算术强度；kernel 甲算术强度 0.5 FLOP/B、kernel 乙 8 FLOP/B，分别判断位于哪一侧并给出可达性能	可达性能取峰值与强度乘带宽的较小者
Little 定律在途数	DDR 读时延 80 ns，每个请求返回一条 64 B 的 cache line。要维持 12.8 GB/s 读带宽，求请求率（请求/秒）与所需平均在途请求数；若时延降到 40 ns，在途数如何变化	$L = \lambda W$ ，带宽靠并发支撑
测量误差识别	同事报告“优化把函数从 10.2 ms 降到 9.8 ms，快 4%”：两次各测一次，第一次含冷启动，两台机器后台负载未知。指出至少三个使结论不可靠的因素，并给出能得到可信结论的测法	先核对测量约定，再比较数字

参考解答与要点

任务	参考解答	必须掌握的要点
时延/吞吐区分	单核串行：吞吐 = $1/0.002\text{ s} = 500$ 请求/秒，单请求时延 2 ms。增加一核后两核并行，理想吞吐约 1000 请求/秒，单请求时延仍为 2 ms——时延由单个请求的处理路径决定，与并行度无关。”吞吐翻倍而时延不变”的硬件例子：第五章的多 bank 交错与总线流水化，单次访问时延不变，单位时间完成数成倍增加	时延与吞吐是两个独立的量；提吞吐不等于降时延

加权 CPI	平均 CPI $= 0.5 \times 1 + 0.2 \times 2 + 0.1 \times 1 + 0.2 \times 3 = 0.5 + 0.4 + 0.1 + 0.6 = 1.6。$ branch 类 CPI 降为 2 后：$0.5 + 0.4 + 0.1 + 0.2 \times 2 = 1.4。$ 总时间与平均 CPI 成正比，变为 $1.4/1.6 = 0.875$，即缩短 12.5%	权重是指令占比；改进只作用在对应那一项上
Amdahl 排序	$S = 1/(1 - f + f/s)$。浮点：$1/(0.6 + 0.4/4) = 1/0.7 \approx 1.43$；cache：$1/(0.7 + 0.3/2) = 1/0.85 \approx 1.18$；分支：$1/(0.8 + 0.2/1.5) \approx 1/0.933 \approx 1.07$。排序：浮点 > cache > 分支。分支优化 1.5 倍听着不小，但占比 20% 使整体收益只剩约 7%	比较候选优化先看占比 f ，再看倍数 s
roofline 落点	拐点 = 峰值算力 / 带宽 = $64/16 = 4$ FLOP/B。甲强度 $0.5 < 4$，位于带宽侧：可达 $= 0.5 \times 16 = 8$ GFLOPS，只有峰值的八分之一；乙强度 $8 > 4$，位于算力侧：可达 $= \min(64, 8 \times 16) = 64$ GFLOPS，用满峰值	先算拐点再比强度；强度不够时加算力无用
Little 定律在途数	请求率 $\lambda = 12.8 \text{ GB/s} \div 64 \text{ B} = 2 \times 10^8$ 请求/秒；$L = \lambda W = 2 \times 10^8 \times 80 \times 10^{-9} = 16$ 个在途请求。时延降到 40 ns 则 $L = 8$：带宽目标不变时，时延减半，所需并发减半；反之时延降不下来，只能靠增加在途数（更多 MSHR、预取、多 bank）补	W 用秒、 λ 用每秒，量纲先配平再代入
测量误差识别	不可靠因素（任答其三）：两次各测一次，无法区分真实差异与随机噪声；第一次含冷启动而第二次未说明，冷热混测；两台机器后台负载未知，环境不一致；且 4% 的差异本身就在典型系统噪声幅度之内。可信测法：同一台机器或完整声明环境；预热后各测不少于 30 次；报中位数与分布而非单次值；两次之间只改“是否优化”这一个因子	核对清单：输入、环境、构建、冷热、样本、量纲

外设与加速

本部分导读

CPU 再快也要等外设：磁盘按块搬运，网络按帧到达，显示按帧扫描。这些设备各有物理脾气，但接入方式却高度一致—队列、描述符、DMA 和完成通知。本部分先把四类外设的硬件特性讲透，再解释 GPU 怎样把计算也变成一种队列化事务，最后把程序的出生过程—编译、链接、装载—还原成硬件地址约定。

外设与队列：SSD、硬盘、网卡

主存、总线和 DMA 已经讲完，本章看设备本身。磁盘按块搬运，网络按帧到达，它们的速度都比内存慢几个数量级，而且慢的方式完全不同：闪存能读快写慢还有寿命问题，机械盘怕寻道，网络怕突发。但它们的接入方式惊人地一致——操作系统准备好队列和描述符，设备通过 DMA 搬运，完成后再通知。理解外设，一半在理解介质，一半在理解这套队列机制怎样把慢介质藏在高吞吐后面。

设备	介质的脾气	队列机制要解决什么
SSD	读快写慢、擦除单位大、有寿命	写放大、FTL、磨损均衡
机械硬盘	寻道和旋转是毫秒级	调度合并、NCQ、顺序化
网卡	帧突发、线速不可退	描述符环、中断聚合、多队列

核心思想

外设性能的本质是”让介质按其擅长的方式工作，让队列吸收其不擅长的部分”。读外设文档时，先问介质的物理限制是什么，再看队列机制怎样弥补。

一、块设备与文件系统的硬件接口

本节回答一个朴素的问题：一次写文件，到了设备那端究竟发生了什么。答案分两半。一半是编址：块设备从不按字节寻址，主机看到的是一组从 0 起编的扇区，这套编址从 CHS 几何参数一路演化到 LBA；另一半是缓存：从 CPU 到介质隔着页缓存、设备缓存两层 RAM，”写完了”在哪一层被宣布，直接决定断电后数据还在不在。本节先划清块设备与字节设备的界线，再讲扇区与 LBA 的编址史，然后算文件系统块与设备扇区对齐与否的代价，接着清点读写缓存的层级，最后把一次 fsync 的耗时拆开算细。这一节只讲主机看到的接口约定，介质本身的脾气——闪存的擦除、磁盘的寻道——留给后面两节。

块设备与字节设备的分界 操作系统把外设粗分成两类。字节设备 (character device) 按流读写：串口每来一个中断交出一个字节，键盘把按键编码逐个上交，终端把输出字节按顺序显示。流没有”位置”可言：读过的字节不会再来，写出去的字节收不回，”回到第 37 个字节”这种请求在流上不成立。块设备 (block device) 按固定大小的整块读写：磁盘、SSD、eMMC 都以块为最小传输单位，每块有编号，可按号随机读、随机写、反复改写。前者像自来水管，后者像一墙编了号的储物柜；/dev 里设备文件类型位的 b 与 c，标的就是这个分界。

分界不是软件随意划的，它几乎由介质单方面决定。存储介质普遍成块，因为定位与寻址的代价按次数收、不按字节收：机械盘的磁头移一次、闪存的页选通一次、磁带走带定位一次，都是固定开销，摊到越多字节上越划算；介质于是把字节捆成块、给块编号，让一次定位服务一整块。终端和串口相反：它们的”介质”是线路另一端的人或进程，数据以事件节奏零散到达，没有可寻址的静止形态，天然成流。设备分类表因此大体是一张介质决定论的清单：

类别	代表设备	读写单位	为什么是这样
块设备	机械盘、SSD、eMMC、U 盘	整块，按号寻址，可改写	介质按块组织，定位代价按次收

字节设备	串口、键盘、终端、 打印机	字节流，无地址， 不可回看	数据以事件节奏到达， 无静止形态
网络设备	网卡	帧（分组），突发到 达	既非流也非块，第四 节专门讲
特例	内存条	按地址随机访问	它就是”地址”本身， 不需块这层抽象

两处边界值得留神。其一，两种接口可以互相伪装，只是代价不对等：块设备装成流很容易，`dd` 按字节计数拷盘，靠的是软件把流切成块、逐块下发；字节设备装成块则要在流上模拟随机访问，理论上可行，工程上没人做。磁带是骑在边界上的活例子：它按记录块寻址、算块设备，却只支持顺序定位，跳读远处的块以十秒计，于是当不了文件系统的载体，只配做归档——“可随机定位”才是块设备配享文件系统的入场券。其二，“可改写”对块设备是逻辑承诺、不是物理事实：主机以为自己在原地改写了 37 号块，SSD 的 FTL 完全可能把新数据写到别处、只改一张映射表。这是下一节的主题，这里先记住结论：块设备承诺的是“按号寻址的整块读写”，不承诺任何物理位置。

还要防止一个误读：流设备也有缓冲，缓冲不改变流的本质。串口的 UART 自带 16 字节 FIFO，终端有行缓冲，管道在内核里有 64 KiB 环形缓冲——它们吸收的是速率抖动，不是引入地址：缓冲里的字节依然先到先走、读完即弃，没有任何“回到第 37 字节”的办法。判断一个设备是流还是块，不问它有没有缓冲，只问它的数据有没有编号、能不能按编号回头取。

扇区、块与 LBA 块设备的最小寻址单位叫扇区（sector），称呼来自机械盘：盘片上每个磁道被切成若干弧段，一段就是一个扇区。早期接口把物理几何直接暴露给软件——第六章 `int 13h` 的调用约定就是标本：柱面号、磁头号、扇区号分装三个寄存器，柱面 10 位、磁头 8 位、扇区 6 位且从 1 起编。这套 CHS（cylinder-head-sector）编址把容量上限锁死在寄存器位宽里：

```
CHS 上限 = 1024 柱面 x 256 磁头 x 63 扇区 x 512 字节
           = 8,455,716,864 字节，约 8.4 GB
DOS 兼容约定把磁头数限到 255，上限收缩到约 7.8 GiB
```

更早还有一道更矮的墙：BIOS 与 IDE 寄存器各自的几何位宽取交集，最窄处拼出的上限更小：

接口	柱面位数	磁头位数	扇区位数
BIOS <code>int 13h</code>	10	8	6（从 1 起编）
IDE/ATA	16	4	8
两者交集（实际可用）	10	4	6

交集那行就是 $1024 \times 16 \times 63 \times 512$ 字节 ≈ 504 MiB 这道墙的出处，1990 年代中期的盘纷纷撞线。解法不是继续加位宽，而是换掉抽象本身：LBA（logical block addressing）把盘看成从 0 起编的一维扇区数组，几何换算藏进盘内控制器。`int 13h` 扩展（`AH=0x42`）改用磁盘地址包传 64 位 LBA；ATA 协议的 LBA28 给 28 位地址，上限 $2^{28} \times 512$ 字节 = 128 GiB；LBA48 给 48 位，上限 2^{57} 字节 = 128 PiB，容量焦虑就此退场。CHS 从此退化为兼容字段：现代盘报出的柱面磁头数是编出来哄老软件的，与盘片真实布局毫无关系。第六章讲固件把硬件细节封装在寄存器约定后面，LBA 是同一招用在编址上一把“盘面几何”换成“线性数组”，接口从此与介质解耦。

扇区本身多大，三十年里的答案是 512 字节；2010 年前后 Advanced Format 把物理扇区扩到 4 KiB：同样的盘片面积能放下更多数据（每扇区的同步头、间隙与纠错码占比随扇区变大而变小），纠错能力也更强。麻烦在操作系统生态仍按 512 字节发命令，过渡期于是生出两种盘：

盘类型	逻辑/物理扇区	部分写的代价
512n	512 B / 512 B	无翻译，老盘直接写
512e	512 B / 4 KiB	固件翻译；写不满一个物理扇区时盘内读-改-写
4Kn	4 KiB / 4 KiB	无翻译；只认 512 B 的老软件直接出错

512e 引出了”原子单位”这个关键概念。块设备的读写以逻辑扇区为原子单位：一次写要么整个扇区成功、要么整个扇区失败，不存在”半个新扇区加半个旧扇区”的可见中间态（断电恰好发生在两个扇区之间是另一回事，靠文件系统日志兜底）。主机写一个 512 字节逻辑扇区、物理扇区却是 4 KiB 时，那 4 KiB 里有八分之七是必须保留的老数据，盘内只能读-改-写：先把物理扇区整页读进盘内缓存，替换其中 512 字节，再整页写回。一次逻辑写放大成”一次读加一次写”；在 7200 rpm 的机械盘上若错过当圈的写窗口，还要多等一整圈 $60/7200 \approx 8.3$ ms。读-改-写 (read-modify-write) 这个词后面还会反复出现：文件系统未对齐写、RAID 的部分条带写、闪存的页内改写，全是同一机制在不同层级的化身。

还要记住 LBA 的编号单位是逻辑扇区、不是字节：同一个 LBA 2048，在 512e 盘上是字节偏移 1 MiB，在 4Kn 盘上是 8 MiB。分区表、引导代码、dd 的 skip 计数，全都按逻辑扇区计—换盘或换镜像格式时，这是最容易算错的一位。

文件系统块与设备扇区 文件系统在块设备之上又定了一级自己的单位：文件系统块，Linux 桌面与服务器最常见的取值是 4 KiB。文件的分配、空闲位图、日志全按这个粒度登记，一次写文件最终化成若干个 4 KiB 块下发到块层。两层单位同名不同级：设备扇区是硬件的原子单位，文件系统块是软件的分配单位。两者对齐则相安无事，错开则每次写都在交学费。

对齐是道算术题。设物理扇区 4 KiB，分区从第 63 个 512 字节扇区开始—MBR 时代按”柱面对齐”留下的经典起点： $63 \times 512 = 32\,256$ 字节，除以 4096 得 7.875，不是整数。于是分区里每个 4 KiB 文件系统块都横跨两个物理扇区：写这样一个块要动两个物理扇区，在 512e 盘上每个被部分覆盖的物理扇区都要读-改-写；一次文件系统写拆成两笔、每笔先读后写，随机写吞吐近乎腰斩，顺序写也因多出的盘内操作而打折。闪存上还有第二重错位：擦除块以几百 KiB 计（下一节细讲），分区起点与擦除块边界错开，垃圾回收每轮都要多搬一批页。

解决办法笨拙而有效：对齐到一个足够大的公倍数。 $1\text{ MiB} = 2^{20}$ 字节，是 512 字节扇区的 2048 倍、4 KiB 扇区的 256 倍，也能整除常见 RAID 条带 (64/128/256 KiB) 和多数闪存擦除块 (256 KiB–1 MiB)。2000 年代末起，主流分区工具把默认起点从 63 号扇区改到 2048 号扇区（整 1 MiB），”分区对齐 1 MiB”从此成为惯例。三种起点的对照一目了然：

分区起点	起点字节	整除 4 KiB?	后果
63 号 512 B 扇区	32 256 B	否 (7.875)	每个文件系统块横跨两个物理扇区
64 号 512 B 扇区	32 768 B	能 (8)	对齐扇区，但不对齐 1 MiB
2048 号 512 B 扇区	1 MiB	能 (256)	扇区、条带、擦除块全部对齐

检查办法只剩一句算术：分区起始 LBA 乘扇区大小，能同时被 4096、条带大小与擦除块大小整除，才算对齐；fdisk -l 与 lsblk 都能直接读出起点，装完系统第一件事就该验这道题。

读写缓存的层级 从 CPU 到介质，数据要经过两层 RAM，每一层都可能”代收”一次写：

层级	位置与容量级	读命中时延量级	断电后
页缓存	主存, GiB 级	约 100 ns	丢失
设备缓存	盘内 DRAM, MiB-GiB 级	数十 μ s	无掉电保护则丢失
介质	闪存或盘片, 百 GiB-TiB 级	$10^2 \mu$ s-10 ms	保留

页缓存 (page cache) 是操作系统拿空闲主存开的代收点：写系统调用把数据拷进页缓存即返回，真正的写盘由回写线程按几十秒级的节奏异步完成。第五章讲的 DMA 与描述符环就在回写这一刻登场：内核把脏页排进块层队列、写成描述符，磁盘控制器按描述符用 DMA 搬数据，搬完写回完成状态，CPU 全程不经手字节。读的路径对称：命中页缓存的读是百纳秒级，设备连被惊动都没有；未命中才排进队列、走一遍 DMA，操作系统还会顺手预读 (readahead)，把顺序后继的块一并读进页缓存。这台机器上“读文件”与“读盘”是两回事，日常十有八九是前者。三个时延层级 10^2 ns、 $10^2 \mu$ s、10 ms 排成 $1:10^3:10^5$ 的阶梯，正是第五章存储层次向外设方向的延伸。

设备缓存是盘内的一小块 DRAM (机械盘几十到几百 MiB, SSD 可达 GiB 级)，职责有二：读预取，把顺序后继扇区提前读进来；写暂存，把主机刚写来的数据先存下。SSD 的盘内缓存还有一份差事：下一节讲的 FTL 映射表就放在里面。写暂存的策略由 ATA 的 WCE (write cache enable) 一类的开关控制。开，对应 write-back 语义：数据一进盘内缓存，控制器就向上报完成，随后择机写介质——报完成那一刻数据还不在介质上，断电即丢，除非盘内有掉电保护。关，对应 write-through 语义：报完成之前数据必须已在介质上，每笔写慢一个介质写时延，换来“完成即安全”。主机侧有同名概念：页缓存本身就是操作系统的 write-back，`O_DIRECT` 与 `O_SYNC` 是绕过或收紧它的两个阀门。于是“写命中哪一层才算安全”有了明确答案：数据到达非易失的去处——介质本身，或带掉电保护、保证能把缓存清空的盘内 RAM——这次写才算数；停在任何一层易失 RAM 上的“完成”，都只是承诺。

这个答案解释了企业盘与消费盘的一处规格差异。企业级 SSD 板上留一组储能电容——第八章讲过去耦电容按时间尺度分层，这组电容接管其中最慢的一档：市电消失——保证突然断电时盘内缓存能全部写进闪存。有了它，盘可以放心用 write-back 报完成，既快又安全；没有它的消费盘，write-back 的“完成”在断电瞬间可能兑不了现。

两层缓存的性格也不同。页缓存面向工作集：热文件反复命中，命中与否取决于访问的局部性；设备缓存面向流：机械盘的预取对顺序读几乎百发百中，对随机读基本帮不上忙，于是盘“顺序快、随机慢”的脾气在缓存层又被放大了一次。

sync/fsync 的硬件含义 页缓存异步回写意味着 `write` 返回时，数据多半还躺在主存里。要让数据真正到达介质，必须有人把每一层都走完，这正是 `fsync` 的语义。一次 `fsync(fd)` 做三件事：把该文件在页缓存里的脏页排进块层队列，等它们全部写完；向设备发 FLUSH CACHE 命令，要求盘把缓存里所有暂存写清到介质；等设备确认 FLUSH 完成，系统调用才返回。`sync` 是同一件事的全盘版：清所有脏页、刷所有设备的缓存。FLUSH 在不同协议里名字不同，角色是同一个：

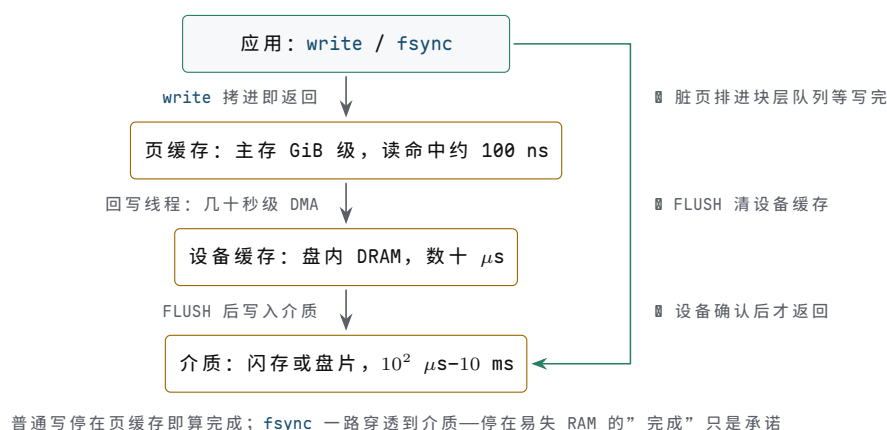
协议	命令名	语义
ATA/SATA	FLUSH CACHE (EXT)	把盘内写缓存全部写入介质
SCSI/SAS	SYNCHRONIZE CACHE	同上，可指定 LBA 区间
NVMe	Flush	把指定命名空间的易失缓存写清

注意 FLUSH 是发给设备的命令，走设备自己的队列：在第五章的描述符环里，它与读写命令一样占一个描述符，只是语义从“搬数据”变成“把已搬的数据做实”。

`fsync` 有多贵，拆开算。极端情形先看 7200 rpm 机械盘：脏页在盘上随机分布，每一笔都要定位—平均寻道约 4 ms，平均旋转等待半圈 $60/7200/2 \approx 4.17$ ms，4 KiB 传输约 27 μ s—单笔约 8-12 ms，FLUSH 再等最后一笔真正写入磁性介质。单线程每提交一个事务就 `fsync` 一次，吞吐上限被钉在 $1/10$ ms = 100 次/秒量级。换无掉电保护的 SATA SSD：系统调用与文件系统日志处理约 10 μ s，命令下发加 DMA 搬运 4 KiB 约 10 μ s，FLUSH 要等一次 TLC 页编程、约 1 ms（为什么这么慢，下一节讲），合计仍是毫秒级，上限约 10^3 次/秒。带掉电保护的盘可以把 FLUSH 截停在缓存层—电容兜底，缓存就算数—`fsync` 收缩到几十微秒，上限抬高两个量级。一次 `fsync` 的时间构成列表如下：

fsync 时间构成（无掉电保护的 SATA SSD，量级）	
系统调用 + 文件系统日志写页缓存	约 10 μ s
命令下发 + DMA 搬运 4 KiB	约 10 μ s
FLUSH：等 TLC 页编程写入介质	约 1 ms
合计	毫秒级，上限约 10^3 次/秒

把这条层级链与 `fsync` 的走法画在一起，普通写与同步写的分岔点一目了然：



图示：块设备访问的三层结构与 `fsync` 的穿透路径。写系统调用把数据拷进页缓存即返回，回写线程按几十秒级的节奏异步写盘，盘内还有一层设备缓存代收；`fsync` 则把每一层都走完：脏页排进块层队列等写完、发 FLUSH 命令清设备缓存、等设备确认才返回。数据到达非易失的去处才算数—停在任何一层易失 RAM 上的“完成”，都只是承诺。

数据库对 `fsync` 延迟敏感，原因全在这笔明细里。事务日志（WAL）的正确性规则是“日志先到介质、数据后到介质”，每条提交记录都要过 `fsync` 这一关，提交时延里 `fsync` 常占九成以上；它还是串行瓶颈，单个连接的提交速率就是 `fsync` 速率的倒数。工程对策全在这笔明细里做文章。组提交把一批事务攒进同一次 `fsync`：10 ms 内攒 200 个事务，吞吐就是 $200/10$ ms = 2×10^4 事务/秒—单个事务多等几毫秒，总量放大两百倍，第十章“用时延换吞吐”的交换，这里是最贵的一例。换带掉电保护的盘，直接砍掉明细里最贵的那一项。放宽同步频率（如 `innodb_flush_log_at_trx_commit=2`，每秒刷一次）则是拿一秒的丢失窗口换两个量级。理解 `fsync` 的硬件含义之后，每个调优选项都能换算成“哪一层 RAM 被信任、丢失窗口有多大”，而不是背参数口诀。

还有一层容易漏算：文件系统自己的日志。ext4 默认 `data=ordered` 模式下，`fsync` 不只刷目标文件的数据—若这次写伴随元数据变更（扩块、改大小），还要先等日志提交写入介质，再刷数据区，一次 `fsync` 可能触发的不止一次 FLUSH。数据库干脆自己管理日志、把数据文件用 `O_DIRECT` 打开，连页缓存这层也省掉，只留设备缓存一层要刷—层级少一层，耗时构成就少一项不确定性。

小结一句：块设备这一层的全部复杂度，都来自“线性数组”这个接口约定与“缓存代收”这个性能手段的拉锯—编址演进让数组更大、更抽象，缓存层级让“完成”更便宜也更危险，`fsync` 则是把两者重新对齐的那道闸门。

二、SSD 与闪存的硬件特性

第一节把块设备当成“按号寻址的线性数组”，那是接口视角。本节打开 SSD 的盖子看介质：NAND 闪存的读、写、擦三个操作各有各的粒度，谁也不迁就谁——读按页、写按页、擦按块，改写不许原地进行。为了让线性数组的承诺在这种介质上成立，SSD 内部跑着一整套地址翻译、垃圾回收与磨损均衡软件，盘里那颗控制器的算力超过第七章的整机。本节先讲 NAND 的三个粒度与寿命限制，再讲 FTL 怎样用映射表藏起介质的怪脾气，然后算写放大这笔最要紧的账，接着讲磨损均衡与 TBW 标称，最后解释 TRIM 凭什么把用旧的盘救回来。

NAND 闪存的页写块擦 NAND 闪存的存储单元是浮栅（或电荷俘获）晶体管：绝缘层里困住的电子改变阈值电压，读出时给控制栅加参考电压、看沟道导不导通，就知道这一位是 0 还是 1。一个单元存几位，决定同一片硅的容量、速度与寿命：

类型	每单元位数	阈值档位	P/E 循环量级	典型去处
SLC	1	2	约 10^5 次	盘内缓存、企业特殊款
MLC	2	4	约 3×10^3 – 10^4 次	早期主流，现多见于工业盘
TLC	3	8	约 10^3 – 3×10^3 次	当前消费与数据中心主流
QLC	4	16	数百次	大容量、读多写少

位数越多，阈值电压要分清的档位越多：TLC 是 $2^3 = 8$ 档，QLC 是 16 档。写入时把电压逐档调准的时间越长，读出时分辨档位的余量越小，单元能经受的擦写次数也越少——容量、速度、寿命在同一条曲线上取舍，没有白捡的密度。闪存还有 NOR 一派：支持按字节随机读，用来存固件，但密度低、成本高；U 盘、SD 卡、eMMC、SSD 全是 NAND 一派——牺牲随机字节访问换密度，再用控制器把粒度差藏起来。本章说的“闪存”默认指 NAND。

怪脾气集中在三个操作的粒度上，而且粒度由物理直接给定：

操作	粒度	时延量级	物理原因
读	页，4–16 KiB	25–100 μ s	一页共享字线，整页同时选通
编程（写）	页，只能 1 写成 0	数百 μ s 到约 1 ms	逐档校准阈值电压，TLC 调 8 档
擦除	块，256 KiB–4 MiB	数 ms	高压加在整个块的衬底上，按块放电子

编程只能把 1 写成 0，想让任何一位回到 1 必须擦除；擦除高压施加在整个块共用的衬底上，单独擦一页等于给每页配隔离与高压通路，芯片面积与可靠性都不答应——块做得越大，摊到每字节上的外围电路越少，擦除块一路长到数 MiB 正是这条经济学。三条合起来就是“页写块擦”：写以页为单位，一页在两次擦除之间只能写一次；擦以块为单位，一块几百页同生共死。”改写”这个词在闪存里因此失去含义：把一页从 A 改成 B 不能原地进行，原页里的 0 变不回 1。唯一的出路是把 B 写到另一页、把旧页标记为无效，攒够一整块无效页后一起擦除。闪存天生是一种“只许追加、不许原地改”的介质。与主存对照数量级：读一页约 50 μ s，是 DRAM 读的约 500 倍；写一页约 1 ms，是 DRAM 写的约一万倍——SSD“读快写慢”的物理出处就在这里。

寿命限制同样来自物理。每次擦除的高压都磨损浮栅外的隧穿氧化层，磨到电子关不住，块就报废。注意寿命的单位是每块：一块 2 MiB 的块按 1000 次 P/E 计，一生承担 2 GiB 写入；整盘寿命是全部块寿命之和。这个“按块计”给后面的磨损均衡出了题目：谁让擦除集中到少数块上，谁就先杀死整块盘。另有两次要脾气

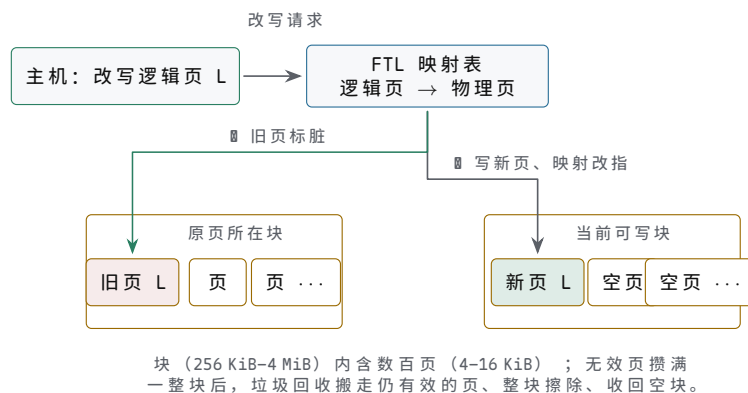
记在这里：同一块被读得太多会扰动相邻页（读干扰），数据放久了电荷会漏（保持力）——FTL 因此要定期巡检、搬动刷新，这是盘内又一份主机看不见的日常工作。

FTL：闪存翻译层 主机要的接口是“按 LBA 原地改写的线性数组”，介质给的能力是“只许追加、按块擦除”，中间的鸿沟由 FTL (flash translation layer, 闪存翻译层) 填平。FTL 是跑在盘内控制器上的固件，日常工作四件事，每件对应介质的一条限制：

职责	对应的介质限制	做法
地址映射	不许原地改写	写新页、改映射、旧页标脏
空页供给	写前必须有已擦空块	预留空闲块池，随写随补
垃圾回收	擦按块、块内混有有效页	搬走有效页，再整块擦除
掉电恢复	映射表放在易失 DRAM	定期写回闪存，凭日志重建

第一件是映射表。FTL 维护从逻辑页号（LBA 按 4 KiB 折算）到物理页号的映射；主机写某个逻辑页，FTL 从不原地改，而是把数据写进当前可写块的下一个空页，把映射指向新页、旧页标脏。”改写 = 写新页 + 旧页标脏”，原地改写就这样被翻译成追加写。映射的粒度也有讲究：页级映射最灵活、表也最大；块级映射表小，但块内页偏移被锁死，改一页要动整块，现代盘几乎清一色选页级。表的体积可以算：256 GiB 的盘按 4 KiB 分页，共 $2^{38}/2^{12} = 2^{26} \approx 6.7 \times 10^7$ 个逻辑页，每条映射 4 字节，整表 2^{28} 字节 = 256 MiB ——恰好是“每 GiB 容量配 1 MiB 表”的比例。这就是 SSD 要配 DRAM 的首要原因，256 GiB 的盘配 256 MiB 上下；无 DRAM 的廉价盘把映射挤在小块 SRAM 和闪存里，或借主机内存（NVMe 的 HMB 机制）暂存，随机访问映射本身成了新的瓶颈。

把介质的页块格局与 FTL 的翻译过程画在一起，“改写”在盘内的真实形态就清楚了：



图示：NAND 闪存的页写块擦与 FTL 地址映射。主机以为自己原地改写了逻辑页 L；盘内实际是两步：FTL 把数据写进当前可写块的下一个空页、映射改指新页，旧页标脏等待回收。无效页攒满一整块后，垃圾回收搬走仍有效的页、整块擦除、收回空块——追加写加周期性回收，就是 SSD 在只许追加的介质上提供原地改写接口的全部机制。

第二件是空页供给：追加写需要源源不断的已擦空块，FTL 平时预留若干，写完再补。第三件因此必然登场——垃圾回收 (garbage collection, GC)：无效页攒多了，挑一个无效页占比高的块，把仍有效的页读出、搬进新块，再整块擦除回收。读、搬、擦全在盘内悄悄进行，主机只看到 LBA 数组一切如常，只是时延偶尔抖一下。第四件是掉电恢复：映射表在 DRAM 里，FTL 要定期把表与变更日志写回闪存，断电后凭日志重建，否则一次意外掉电就是一块砖。

四件事合起来看，SSD 里的控制器名不副实：它远不止“控制”，那是一颗多核 ARM 级处理器，跑着带映射管理、GC、磨损均衡、坏块替换、掉电重建的完整软件，配的 DRAM 比第七章整台教学机的主存大几个数量级。买一块 SSD 等于买回一台专用小计算机，它的全部工作是让 NAND 假装成一块可以原地改写的磁盘。下一节会看到，机械盘的控制器同样不闲，只是忙的方向相反——不是躲开介质的限制，而是迎合介质的转动。

写放大的计算 GC 搬运的有效页本身是额外的闪存写入：主机没让它们发生，FTL 为了让介质继续可用不得不写。写放大 (write amplification, WA) 把这笔开销量化：

$$WA = \text{实际写入闪存的总量} / \text{主机写入的总量}$$

$WA = 1$ 是理想值：主机写多少，介质写多少，GC 零搬运。顺序写满一块再整块改写的负载就接近它——改写让整块的页同时失效，回收时没有有效页可搬，擦掉即可。WA 大于 1 的部分，全是 GC 搬家的运费。

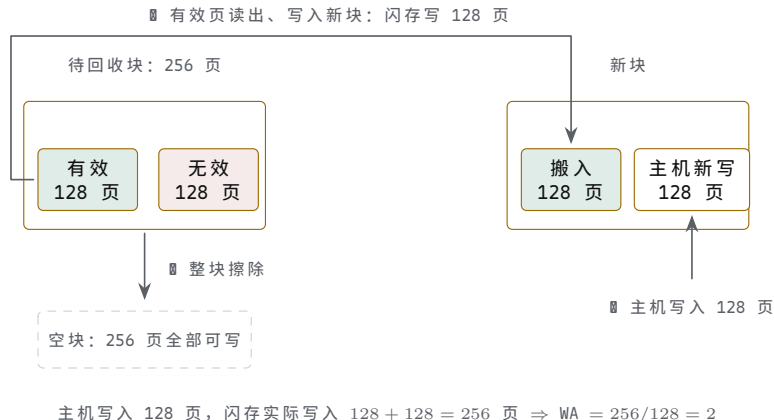
具体算一次。设块含 256 页，某块一半 (128 页) 有效、一半已无效。回收它：128 个有效页读出并写入新块，闪存写入 128 页；擦掉旧块得 256 个空页，其中 128 页被搬来的数据占用，剩 128 页供主机写新数据。等主机写满这 128 页盘点：主机写入 128 页，闪存实际写入 $128 + 128 = 256$ 页， $WA = 256/128 = 2$ 。有效页占一半，写放大恰为 2 倍。一般化只剩一步：块内有效页比例为 v 时，回收一块要搬 vN 页、腾出 $(1-v)N$ 页给主机：

$$WA = (vN + (1-v)N) / ((1-v)N) = 1 / (1 - v)$$

v 每涨一点，代价不是线性涨，是倒数式地涨：

有效页比例 v	WA	直观画面
0	1	无页可搬，擦掉即可（顺序改写的尽头）
0.25	约 1.33	搬 1 页换 3 页空位
0.5	2	搬一页换一页，写一倍送一倍
0.9	10	接近写满的盘做随机写

把 $v = 0.5$ 那一行画成过程图，“搬一页换一页”的运费就看得见了：



图示：一次 GC 的完整过程（块含 256 页、一半有效）。回收：128 个有效页读出并写入新块，这是主机没有要求的闪存写入；旧块整块擦除，换回 256 个空页，其中 128 页被搬来的数据占用，剩 128 页供主机写新数据。等主机写满盘点：主机写 128 页，介质写 256 页，WA 恰为 2——有效页占一半，写一倍送一倍。

随机写为什么放大更严重，看 v 的下场就知道。顺序写把失效集中到少数块上：文件被顺序覆盖，整块一起变脏， v 趋于 0、WA 趋于 1。均匀随机写把失效撒到所有块上：每个块都攒一点无效页，又都剩一堆有效页，长期稳态下各块的 v 收敛到同一高位——盘越满， v 越高。剩余空间只有一成的盘，各块平均 $v \approx 0.9$ ，按上式 WA 逼近 10：主机每写 1 GiB，介质实际承受约 10 GiB。GC 挑块因此遵循贪心原则：优先回收无效页最多的块，同样换回一个空块， $v = 0.1$ 的块只搬一成， $v = 0.9$ 的块要搬九成。GC 的时机也影响体验：空闲时做的后台 GC 主机无感，写路径上被空块耗尽逼出来的同步 GC 则直接拖慢当次写——同一笔写放大，前者藏在空闲里，后者变成时延尖峰。同一个“随机写”，在第十章只是时延问题，在闪存这里同时是寿命问题：放大几倍，P/E 预算就被多吃几倍。工程上的缓冲垫是预留空间 (overprovisioning)：

盘内实装闪存多于标称容量，消费盘多留约 7%，企业盘留到 28% 上下，差额直接压低 v 的有效值，把稳态 WA 降回个位数。消费 TLC 盘还有一招 SLC 缓存：拿一部分闪存按 SLC 模式用，突发写先进缓存、空闲时再腾进 TLC，缓存打穿后的“缓外速度”才是介质的真速度。

磨损均衡与寿命 写放大吃掉的是寿命预算，而 P/E 次数按块计：写入分布不均，少数块先被擦穿，整盘提前报废。磨损均衡 (wear leveling) 的任务是把擦除次数在所有块之间摊平。动态磨损均衡只盯活跃数据：每次分配空块时挑擦除次数最少的，新写入自然流向年轻块。它对冷数据无效——一批文件写进去三年不动，占着的块擦除次数恒为零，剩下的块继续挨打。静态磨损均衡连冷数据也搬：定期把长期不改写的数据挪到擦除次数多的块里“养老”，把年轻块换出来轮换。消费盘多做动态加有限的静态，企业盘两者做足；注意均衡搬运同样计入写放大，寿命的公平本身是花钱买的。坏块管理走的是同一条备用通道：出厂坏块与使用中长出的坏块都由备用块池顶替，顶替一次，可用块与预留空间就少一分。

盘厂把寿命预期做成一个标称值：TBW (total bytes written)，承诺“累计主机写入达到此数之前，盘在规格内工作”。它按主机写入计、不按闪存实际写入计——写放大是盘厂自己的成本，TBW 是给用户的承诺。拿 TBW 反推每日预算是一道除法。例：256 GB 消费盘，TBW 150 TB，每天写 40 GB：

$$\text{寿命} = 150 \text{ TB} / (40 \text{ GB/天}) = 150000 / 40 = 3750 \text{ 天, 约 } 10.3 \text{ 年}$$

约十年，超过多数机器的服役期——前提写在算式背后：WA 低、负载里没有集中的擦写热点。同一张标称，不同的每日写入量对应完全不同的寿命预期：

每天写入量	预算天数	折合年限
20 GB	7500 天	约 20.5 年
40 GB	3750 天	约 10.3 年
100 GB	1500 天	约 4.1 年
500 GB	300 天	约 0.8 年

同一道题换个角度更有意思：150 TB 相当于把 256 GB 的盘写满约 $150000/256 \approx 586$ 次，而 TLC 的 P/E 预算还有 1000-3000 次——标称值留出的安全边，正是给写放大与均衡搬运预备的。若负载让 WA 长期停在 3，同样的主机写入量下，闪存实际承受约 $586 \times 3 \approx 1760$ 次全盘擦写，落在 TLC 预算中段。这也让“TBW 相同的两块盘，体质可能差几倍”有了定量含义：标称一样，WA 不同，介质实际寿命就不同。

规格书里的配套指标 DWPD (drive writes per day，保修期内每天可把整盘写几遍) 与 TBW 是同一笔账的两种记法：上例每天 40 GB 合 $40/256 \approx 0.16$ DWPD，五年保修累计约 $40/256 \times 365 \times 5 \approx 285$ 次全盘写入，仍在 586 次标称之内。读寿命规格时先分清它在报哪一个，再换算成自己的每日写入量，比直接比数字可靠。温度是另一个隐形变量：电荷保持力随温度升高而下降，消费盘的保持力标称大致是“断电后 30 摄氏度存放一年”；FTL 的定期巡检刷新正是为这条曲线兜底，刷新本身也耗 P/E，又一笔主机看不见的磨损。

TRIM 与稳态性能 最后一个机制同时解释“盘为什么越用越慢”和“为什么能慢得不那么狠”。文件系统删除文件时只改自己的元数据，从不通知盘——机械盘时代这是美德，老数据原地留着，误删还能恢复。对 SSD 这却意味着两个世界脱节：文件早删了，FTL 还当那些页是有效数据，GC 照样把它们当宝贝搬走，写放大白白上涨。盘越用越满， v 越高，WA 沿 $1/(1-v)$ 爬升，随机写越来越慢——“盘满变慢”的全部机理就在此。

TRIM 是补上脱节的命令：文件系统删文件、释放区间时，顺手给盘发一条 TRIM (ATA 里是 DATA SET MANAGEMENT 命令的 TRIM 属性，NVMe 里是 Dataset Management 的 deallocate)，告知这段 LBA 不再使用。FTL 收到后直接把这些页标记无效，下次 GC 它们与从没写过一样，不必再搬。TRIM 不改变任何物理规律，

只改变 FTL 的信息量： v 从此按真实值算，而不是按“文件系统有史以来写过的所有页”算。顺带一提，被 TRIM 的页读回什么由盘决定（不少盘返回全零），指望删除后还能恢复文件的软件从此失业。下发时机有两派：挂载选项 `discard` 每次删除即时下发，简单但每次删除多一笔命令开销；`fstrim` 定时任务攒一批区间一起发，是多数发行版的默认。

把出厂态与稳态摆在一起，差距一眼见底：

状态	空块来源	WA 量级	4 KiB 随机写 IOPS 量级
出厂态（新盘）	全部块空闲	1	数万
稳态，有 TRIM 与预留	GC 回收	2-3	万级
稳态，无 TRIM 且盘满	GC 反复搬运	5-10	千级，伴随时延尖峰

新盘所有块皆空，写请求直接编程空页， $WA = 1$ ；持续随机写把盘推入稳态后，每次写都可能触发 GC，IOPS 按 WA 的倒数缩水，时延还伴随 GC 尖峰——主机在队列那头看到的现象、它对本章第五节尾延迟的含义，留到那时再算。企业盘规格书分报“出厂态”与“稳态”两组随机写数字，正因为它们本就是两个量；消费盘多半只报前者，读规格书时这是第一个要问的问题。要把整块盘拉回出厂态，手段是全盘擦除：ATA 的 `SECURITY ERASE`、NVMe 的 `Format/Sanitize`，或对所有已用区间一次性 TRIM——映射表清空、块全部回收， WA 回到 1；二手交易前的数据清除与性能恢复，是同一条命令的两份用途。

稳态性能还有队列一侧的条件，用第十章的 Little 定律随手可算：4 KiB 随机读时延约 $80\ \mu\text{s}$ ，队列深度 1 时吞吐上限是 $1/80\ \mu\text{s} = 12500\ \text{IOPS}$ ，约 50 MB/s；想吃到 NVMe 标称的 50 万 IOPS，需要随时有 $5 \times 10^5 \times 80\ \mu\text{s} = 40$ 个请求在途。这就是深队列存在的理由，也是“单线程跑不满 SSD”的正解——不是盘虚标，是队列深度不够。维护一块盘的长期性能，说到底三件事：留足空闲容量压低 v 、打开 TRIM 让 FTL 知情、别把随机写当顺序写使。三条都不花钱，花的只是对介质脾气的理解。

最后把介质与队列接起来。SSD 的全部怪脾气——写慢、擦除、GC、磨损——主机都看不见细节，只看见两件事：吞吐随负载形状变化，时延分布长出尾巴。第十章的忠告在这里照用：先测量，再优化。同一块盘，顺序与随机、空盘与满盘、队列深度 1 与 32，是好几台不同的机器，规格书上的任何一个数字都只在它标注的条件下成立。下一节轮到机械盘：它的脾气比闪存直白得多，毫秒级的寻道与旋转藏不住，队列调度也由此有了用武之地。

三、机械硬盘与访问调度

机械盘是主存之外仍在大量服役的机电设备：存储介质是磁，定位机构是音圈电机加主轴电机。它的全部性能特征可以归结为一个事实——盘片上只有两个运动部件：摆动的磁臂和旋转的盘片，两者的时间常数都是毫秒级；而读通道、盘上缓存、接口电路这些电的部分都在纳秒到微秒级。机械盘几十年的性能工程，本质上是一部“怎样躲开这两件机械运动”的历史。本节先看几何结构怎样决定访问时间的构成，再看操作系统与盘固件怎样用排队把机械运动的开销摊薄。

磁道、柱面与扇区 盘片是铝合金或玻璃基板，两面涂磁性材料，每面配一个磁头；所有磁头固定在同一根磁臂上同步摆动，盘片由主轴电机带着恒速旋转。磁头停在某个半径上不动、盘片转过一周，磁头在盘面上扫出的圆环叫磁道；一张盘面上有几十万条同心磁道。所有盘面上同一半径的磁道集合叫柱面——这些圆环在空间里恰好叠成一个圆柱面。寻道完成后整组磁头同时就位，所以连续数据按柱面组织：在同一柱面内换面读不用再挪磁臂，只需电子开关换一个磁头。磁道沿圆周切成小段，每段是一个扇区：传统容量 512 字节，近年的 Advanced Format 把物理扇区改为 4096 字节，同时以 512e 模式向旧软件模拟 512 字节扇区。

先建立结构上的数量级直觉。一块 7200rpm 的 4TB 桌面盘通常用三张碟片、六个磁头，单碟约 1.3TB；每张盘面刻下几十万条磁道，道间距以十纳米计；磁头的飞行高度只有几纳米，比烟尘颗粒还小几个数量级——这就是盘体必须密封、怕震、一粒灰就可能划伤盘面的原因。机械盘的全部精密性都花在“让磁头以几纳米高度贴着盘面飞、又永远不许碰上去”这一件事上。

磁记录方式本身也换过几代。纵向磁记录（LMR）把磁畴平铺在盘面内，面密度很快撞上极限；垂直磁记录（PMR）把磁畴竖起来写，配合更窄的读磁头，面密度提高了一个量级，今天的 CMR 盘都是 PMR。再往后，写磁头的宽度做不下去了——磁场不能无限聚焦——SMR 干脆利用“写宽读窄”的差值让磁道重叠，这是本节后段的主题；HAMR 与 MAMR 则用激光或微波辅助写入更小的晶粒，仍在爬坡。磁密度的每一步都同时改写容量、速度与“介质的脾气”，这正是本章要一类设备一节分开讲的原因。

结构要素	物理实质	典型数量级
盘片 platter	铝合金或玻璃基板，双面涂磁，恒速旋转	7200rpm 桌面盘 1-4 张碟
磁头 head	每面一个，固定在磁臂上由音圈电机驱动	飞行高度仅数纳米
磁道 track	磁头不动、盘转一周扫出的圆环	每面几十万条，道间距数十纳米
柱面 cylinder	所有盘面同半径磁道的集合	柱面内换面不再寻道
扇区 sector	磁道沿圆周的最小读写单位	512 字节或 4096 字节

区域位记录（ZBR）决定了一块盘的容量与速度分布。早期磁盘每条磁道的扇区数相同，容量被最内圈锁死：外圈周长是内圈两倍多，却写同样多的位，外圈的线密度被白白浪费。区域位记录把盘面按半径分成十几个区，区内每条磁道的扇区数固定，越靠外的区每道扇区越多。算一笔几何账：数据区外半径约 43mm、内半径约 20mm，周长比约 $43/20 \approx 2.15$ ，恒定线密度下外圈每条磁道能容纳的扇区数约为内圈的 2 倍。转速恒定，单位时间从磁头下流过的字节数正比于每道扇区数，于是外圈的顺序读速约为内圈两倍：同一块盘，外圈约 200MB/s，扫到内圈只剩约 100MB/s。LBA 编号从外圈排起，“分区越靠前越快”这条老经验，物理根源就在 ZBR。

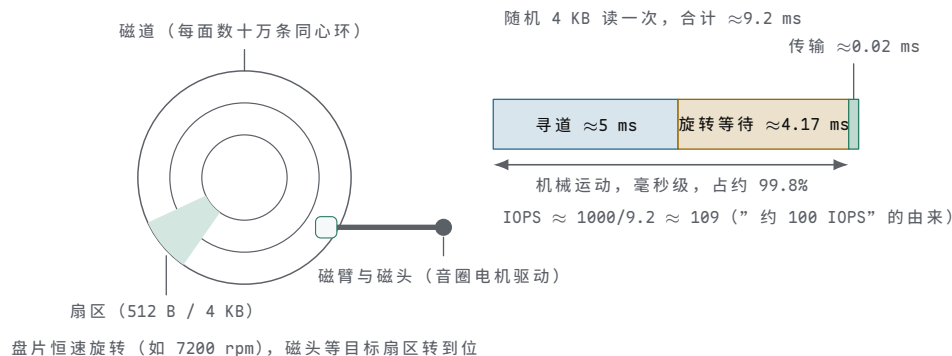
4K 物理扇区带来一条连带约定：分区与文件系统必须按 4K 对齐。若文件系统的 4K 块错开一个 512 字节逻辑扇区，每一次块写都会横跨两个物理扇区，盘内要先读出两个 4K、合并、再写回——一次写放大成两次读加两次写。512e 模式把这件事藏进固件，性能账却藏不住；4Kn 盘干脆只暴露 4K 扇区，要求软件直接按物理现实对齐。

扇区的传统地址是三元组（柱面，磁头，扇区）——这三个数曾经是货真价实的几何坐标：柱面号指挥磁臂停到哪条半径，磁头号选中哪张盘面，扇区号等盘片转到第几段。第六章 int 13h 的寄存器约定 CH、CL、DH 就按这三个量命名；BIOS 上限 1024 柱面、255 磁头、63 扇区乘 512 字节，拼出约 8GB 的寻址上限，正是几何时代的遗产。ZBR 与坏道重映射普及之后，盘内真实布局对外不再透明，CHS 退化成一组虚构参数，BIOS 与 ATA 各自做一层“假几何”翻译；LBA 随后把寻址拍平成从 0 开始的扇区序号，盘固件收到 LBA 后自己换算（区，道，扇区）。磁盘地址从几何坐标变成逻辑序号，是外设把物理复杂性藏进固件的典型一例，也是本书反复出现的主题：约定一旦立下，硬件在约定后面换了几轮实现，约定本身仍在服役。

寻道与旋转延迟 一次扇区访问的时间拆成三段：寻道（seek），磁臂移到目标磁道并安定下来；旋转等待（rotational latency），等目标扇区转到磁头下方；传输，数据经读通道、盘上缓存、接口由 DMA 搬进内存——最后这段走的就是第五章描

述符与 DMA 的通道，微秒级。前两段是机械运动，毫秒级。随机小请求的总耗时几乎全在前两段，这是本节的中心事实。

把三段耗时按真实比例画在一条时间条上，与盘片的结构对照着看：



图示：机械盘的结构与一次随机 4 KB 读的时间构成。左图俯视盘片：同心圆环是磁道，圆周上的一小段是扇区，磁臂把磁头送到目标半径后，等目标扇区转到磁头下方。右图把三段耗时按真实比例排开：寻道约 5 ms、旋转等待约 4.17 ms 都是毫秒级的机械运动，传输约 0.02 ms 是微秒级的电过程——随机小请求的时间几乎全部花在前两段，“一块机械盘约 100 IOPS”由此而来。

寻道本身还可再拆：磁臂加速、巡航、减速、在目标磁道上安定到可读写的精度。短寻道花不完加速段，长寻道大部分时间在巡航，所以“平均寻道时间”必须注明工作负载：厂商标称的 4-10ms 通常指随机访问下的平均；相邻磁道间的一步只需不到 1ms，从盘片最外到最内的全程寻道则要 15-20ms。后文电梯调度能省时间，靠的正是把随机寻道换成道间寻道。

阶段	物理动作	典型耗时与决定因素
寻道	磁臂移动并安定	随机平均 4-10ms；道间 <1ms；全程 15-20ms
旋转等待	等目标扇区转到位	平均半圈：7200rpm 约 4.17ms，15000rpm 约 2ms
传输	读通道、缓存、DMA 进内存	4KB 约 0.02ms，由介质读速决定

旋转等待的演算：7200rpm 即每秒 120 转，一转 $60000/7200 \approx 8.33$ ms；目标扇区等概率出现在圆周任意位置，平均等半圈，即约 4.17ms。这个数由转速唯一决定，软件动不了它；15000rpm 的企业盘把它减半到 2ms，代价是更小的盘片、更小的容量与更高的功耗。调度能做的只有重排——让每次定位更接近“顺路”，而不是消灭等待本身。

随机 4KB 读的总账：取随机平均寻道 5ms（4-10ms 区间的中值），加旋转等待 4.17ms，加传输 0.02ms，合计约 9.2ms，常说“约 10ms”。于是单盘纯随机 4KB 读每秒最多完成约 $1000/9.2 \approx 109$ 次，即约 100 IOPS；乘 4KB，吞吐约 0.45MB/s。“一块机械盘约 100 IOPS”这条经验值就是这么来的：IOPS 不是接口参数，而是机械时间常数的倒数。工程含义很直接：凡把机械盘当随机小请求设备用的设计——数据库直接随机写裸盘、把虚拟机镜像打散在机械盘上——容量规划都要从每盘约 100 IOPS 起步，而不是从标称的 200MB/s 起步。

短行程（short-stroking）是用容量换定位时间的经典手段：只使用外圈一小部分柱面，寻道范围被限制在最窄的一弧，平均寻道从 5ms 压到 1-2ms，旋转等待不变，单次随机访问降到约 6ms，IOPS 提到约 170；外圈同时又是 ZBR 里最快的一区，顺序吞吐也最高。代价是只用盘容量的一小角。它说明一个事实：机械盘的容量、时延、IOPS 三者可以互相兑换，兑换率由机械时间常数决定。

这个换算直接进容量规划。一个需要 5000 随机 IOPS 的应用，用机械盘做 RAID0 需要约 50 块盘（每盘约 100 IOPS），哪怕总容量只需要其中十块盘的量；同一负载换 SATA SSD（随机读数万 IOPS），一块就够。存储选型先定 IOPS 指标、

再定容量指标，顺序弄反是机械盘时代最常见的预算事故之一——盘买够了容量，性能却只有预算值的几十分之一。第十章的 Amdahl 定律在这里换成一句大白话：整个阵列的随机吞吐，等于盘数乘以每盘那约 100 次。

顺序与随机的吞吐差距 顺序读时磁头定位一次，之后盘片每转一圈交出一条完整磁道，接口与缓存全速运行：外圈约 200MB/s，整盘平均约 150–160MB/s。顺序吞吐由位密度与转速决定，是“电”的速度；随机吞吐由寻道与旋转决定，是“机械”的速度。同一块盘有两个速度，差多少完全取决于访问模式。接口在这里不是瓶颈：SATA 6Gb/s 按 8b/10b 编码折算有效带宽约 600MB/s（第八章讨论过这类编码开销的来源），介质供数最多 200MB/s，线路一直空着一多半；是 SSD 的出现才把 SATA 真正打满。

三笔演算把差距钉死。吞吐比： $200 \div 0.45 \approx 450$ 倍，超过两个数量级。任务时间：顺序读 1GB 约 5 秒；随机读同样 1GB 是 $2^{30}/2^{12} = 262144$ 次 4KB 读，乘 9.2ms 约 2410 秒——5 秒对 40 分钟，读的是同一份数据。整盘扫描：4TB 盘以平均 160MB/s 顺序扫一遍约 $4 \times 10^{12} / (1.6 \times 10^8) = 25000$ 秒，约 7 小时——阵列重建与备份窗口的时长下限就是这么估的。

这三个数不必靠相信，一台机器几条命令就能复现。fio 的顺序读、队列深 1 的随机读、队列深 32 的随机读，恰好对应本节的三个吞吐档位；iodepth 就是在途请求数，与第十章 Little 定律里的在途数是同一个量。

```
# same 7200rpm disk, three access patterns (Linux, fio)
fio --name=seq --rw=read --bs=1M --size=4G --filename=/dev/sdX
#    => about 200 MB/s on outer tracks
fio --name=rand --rw=randread --bs=4k --size=4G --iodepth=1 ...
#    => about 100 IOPS, i.e. about 0.4 MB/s
fio --name=randq --rw=randread --bs=4k --size=4G --iodepth=32 ...
#    => NCQ reorders in the drive: a few hundred IOPS
```

访问模式	每次定位开销	7200rpm 盘的吞吐
顺序读	一次寻道，之后免定位	外圈约 200MB/s，平均约 160MB/s
随机 4KB，队列深 1	每次都寻道加旋转	约 100 IOPS，约 0.45MB/s
随机 4KB，排队重排后	短寻道加部分旋转	数百 IOPS，约 1–1.6MB/s

物理根源是惯性。磁臂和盘片有质量，加速、减速、安定都要毫秒；而磁性状态的读出快得多——磁头一旦就位，读通道以每纳秒若干位的速度采样流过的弧段。”机械盘怕随机”不是介质慢，是定位慢。由此得到全部工程推论的共同方向：把随机访问变成顺序访问。日志结构文件系统先写日志、数据库顺序追加 redo、文件系统按分配组聚集数据，以及下一段的调度与合并，都是这同一件事在不同层的化身。

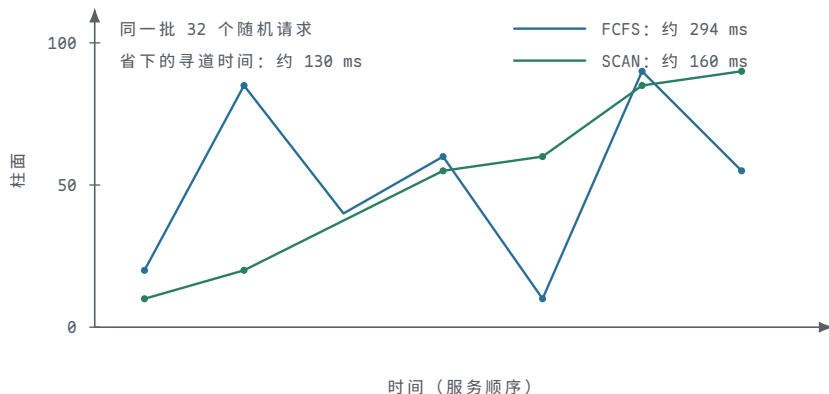
电梯调度与合并 块层队列给了软件选择权：队列里同时压着 32 个请求时，先服务谁是自由的。SCAN（电梯算法）把请求按目标柱面排序，磁臂从一端扫到另一端、沿途服务，到头反向，像电梯不在无人楼层停靠；LOOK 不到物理端点就提前折返，C-LOOK 只单向服务、到头快速回扫。排序换掉的是”随机平均寻道 5ms”这个前提：排序后相邻请求大多位于相邻柱面，寻道从毫秒级的全程动作退化成不到 1ms 的道间一步。

与排序同时发生的是合并：LBA 相邻的请求在块层拼成一个更大的请求。8 个相邻 4KB 读合成一次 32KB 读，一次定位全部到手；排序吃的是磁道维度上的重排，合并吃的是”LBA 相邻本就大概率在同一条磁道”。请求在队列里停留的时间因此有了双重价值——既等来了排序的依据，也等来了可合并的邻居。

收益演算：32 个随机 4KB 请求按到来顺序（FCFS）服务，约 $32 \times 9.2 \approx 294\text{ms}$ 。电梯排序后，磁臂一趟扫过请求分布的柱面区间，每请求摊到短寻道加一次旋转等

待，按 $0.8 + 4.17 + 0.02 \approx 5\text{ms}$ 粗算，约 160ms ；LBA 相邻的请求再被合并，总时间更短。注意这个收益的前提：队列里得有东西可排。队列深度为 1 时没有重排空间，调度无能为力。这正是第十章 Little 定律的另一面——在途请求数既是吞吐的原料，也是调度的原料；评测随机性能必须把队列深度写进条件，QD1 的约 100 IOPS 与 QD32 的数百 IOPS 是同一块盘的两种真实，谁也不是“作弊”。

把同一批请求的两种服务顺序画成磁臂轨迹，排序省下的是什么就有了形状：



图示：同一批随机请求的两种磁臂轨迹（示意其中七个请求的位置）。FCFS 按到达先后服务，磁臂在柱面间来回甩动，32 个请求全程约 $32 \times 9.2 \approx 294\text{ ms}$ ；SCAN 按柱面排序后单趟扫过，每请求只摊到短寻道加一次旋转等待，全程约 160 ms 。两条轨迹之间省下的约 130 ms ，全是寻道时间。

调度策略	服务顺序	关键问题
FCFS	按到达先后	无重排，随机负载下磁臂来回甩
SATF	最短寻道优先	吞吐最好，远端请求可能饿死
SCAN / LOOK	按柱面排序，到头折返	兼顾公平，折返点附近等待偏长
C-LOOK	单向扫描，到头回扫	等待更均匀，回扫本身空跑一趟
mq-deadline	排序加截止时间，读优先于写	读期限远短于写期限，防止饿死
none	不调度	SSD 无机械定位，排序是净开销

读优先于写有应用层的原因：读大多是同步等待，应用线程停在原地；写走页缓存 write-back，应用写完就走。把写无限推迟却不行——脏页终究要回写，推迟过久会把抖动挪到更糟的时刻，所以调度器给写也设期限，只是宽得多。调度器也不是越聪明越好：机械定位消失之后，排序本身成了净开销，介质换成 SSD，内核调度器就该退场。软件结构跟着介质的物理常数走，这是外设栈的普遍规律。

期限的具体值可以读出来：Linux 的 mq-deadline 给读请求默认约 500ms 、写请求约 5s 的期限，到期未服务的请求优先发出，饿死被时间上限封死。调度器与队列预算也可以直接观察：

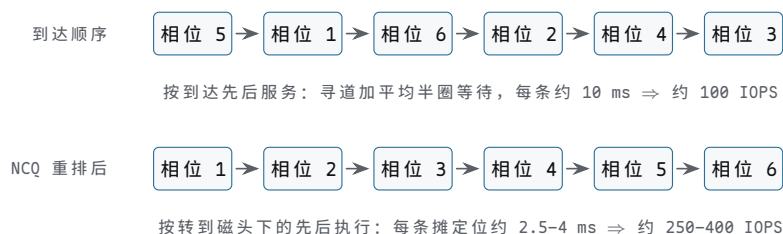
```
$ cat /sys/block/sda/queue/scheduler
none [mq-deadline] kyber bfq      # current: mq-deadline
$ cat /sys/block/sda/queue/nr_requests
128                               # block layer queue depth budget
```

块层队列预算 (nr_requests) 决定最多能攒多少请求供排序与合并：太小，重排空间不足；太大，单请求的排队时延上升。同一个旋钮，在机械盘上和 NVMe 上的合适应答完全不同——第五节会再回到这个数。

测量的另一半在 `iostat` 里：`r_await` 与 `w_await` 把每次读写的平均总时延以毫秒报出，机械盘随机负载下看到 5-10ms，正是寻道加旋转的直接读数；`await` 突然涨到几十毫秒，多半意味着队列在某一层堵住或盘内在做重映射。第十章“用测量代替猜测”的原则在外设上格外省钱：磁盘几乎从不说谎，它的毫秒数摆在那里，与本节的时间构成一项一项核对即可。

NCQ 与 SMR 操作系统的排序发生在驱动里，驱动不知道盘此刻磁头停在哪、盘片转到什么角度；盘固件知道。SATA 的原生命令队列（NCQ）允许主机一次提交最多 32 条命令，盘内按“当前柱面加旋转相位”重排：同一柱面的多个请求按扇区转到磁头下的先后执行，旋转等待从平均半圈降到更小。同样是 32 个随机 4KB 请求，盘内重排后每条摊到的定位时间约 2.5-4ms，IOPS 从约 100 提到约 250-400。这正解释了“一次随机读约 10ms”与“标称数百 IOPS”为何同时成立：前者是单请求的物理时延，后者是排队重排后的吞吐；时延没变，在途数变了，按第十章 Little 定律，吞吐随之改变。AHCI 只有一条 32 深的队列；更深的队列模型—NVMe 的 64K 条队列、每条 64K 深—为 SSD 准备，本章第五节展开。对机械盘，32 个在途请求提供的重排空间已把大部分收益吃掉。

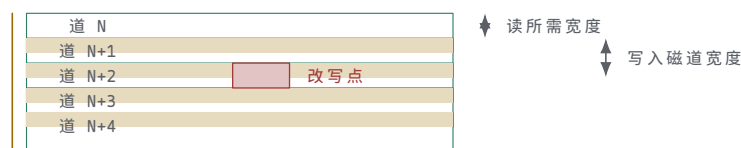
重排做的事用六个请求示意便清楚（实际队列里是 32 个）：



图示：同一批随机请求的两种服务顺序（示意六个，实际是 32 个在途）。按到达先后服务，相邻请求相位乱跳，每条都要付一次寻道加平均半圈的旋转等待；盘内固件知道磁头位置与盘片角度，按“当前柱面加旋转相位”重排后，同一柱面的请求按转到磁头下的先后执行，旋转等待被摊薄，每条定位约 2.5-4 ms。时延没变，在途数变了一—IOPS 从约 100 提到约 250-400。

SMR（叠瓦式磁记录） 从另一头榨容量。写入磁道天生比读取所需宽度宽，叠瓦布局让后一条磁道盖住前一条的边缘，像屋顶瓦片一样部分重叠，磁道密度因此提高，单盘容量便宜约两成。代价全在写的方向：写第 N 道会顺带覆盖第 $N+1$ 道的重叠区，于是一组磁道构成一条带（band），典型 256MB，带内只能从带首顺序追加；要改写带中间一个扇区，必须把它之后的内容读出、合并、整条带重写。极端情况下一次 4KB 随机写牵连 256MB 的重写，写放大约六万五千倍；盘内固件当然不会每次真这么做，它用下一段的缓存机制把账延后，但账不会消失。

叠瓦的结构与改写的牵连画在一起看，“便宜两成”的代价就具体了：



一条带（band，典型 256 MB）：带内只能从带首顺序追加

改写带中一个扇区：其后各道读出、合并，整条带重写—极端时 4 KB 写牵连 256 MB

图示：SMR 叠瓦磁道的结构（深色条带是被后一道盖住的重叠区）。写入磁道天生比读取所需的宽度宽，叠瓦布局让后一道盖住前一道边缘，磁道密度提高，单盘容量便宜约两成。代价在写的方向：写第 N 道会顺带覆盖第 $N+1$ 道的重叠区，于是一组磁道构成一条带，带内只能从带首顺序追加；要改写带中间一个扇区，必须把它之后的内容读出、合并、整条带重写—极端情况一次 4 KB 随机写牵连 256 MB 重写，写放大约六万五千倍。

盘管式 SMR (DM-SMR) 用一小片 CMR 缓存区先收下随机写，空闲时再整理回叠瓦区：轻负载看不出差别，持续随机写打穿缓存之后，稳态写吞吐可跌到个位数 MB/s，单次写延迟出现秒级尖峰；NAS 阵列重建时 SMR 盘超时掉盘的案例，根源即此。主机管理型（HM-SMR）索性要求主机按带分段、带内顺序写，把约束摆上接口，换取

确定性——这与 NVMe 分区命名空间（ZNS）是同一思想：介质把脾气讲清楚，软件按脾气写，双方都轻松。

特征	CMR 传统盘	SMR 叠瓦盘
磁道布局 随机写稳态	磁道分开，可原位改写 约 100–数百 IOPS	磁道重叠，带内只能顺序追加 缓存打穿后可跌到个位数 MB/s
缓存特征 典型定位 阵列重建 确认途径	常见 64–256MB 通用、NAS、监控 按标称 IOPS 估算时间 厂商型号规格页	同容量点常配异常大的缓存 归档、冷数据、顺序写为主 持续负载下可能超时掉盘 查厂商公布的 SMR 型号清单

SMR 值得单独成题，还因为一段行业插曲：2020 年前后，厂商曾把 DM-SMR 盘不加说明地混进 NAS 与桌面产品线，用户在做阵列重建与持续随机写时遭遇数小时卡顿乃至掉盘，追查之下才发现同一型号存在 CMR 与 SMR 两个版本。事件以三家大厂公开 SMR 型号清单收场。教训写在规格页的边角：同一名称的盘，介质组织方式可以不同，而介质组织方式决定的是写的语义，不只是速度。读规格书时，CMR 还是 SMR 这一栏与容量、转速同等重要——它回答的是“这块盘允许什么样的访问模式”。

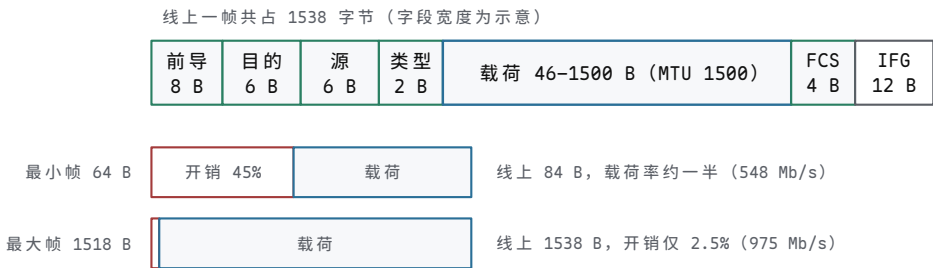
选购识别并不靠猜：三家大厂在 2020 年后都公布了 SMR 型号清单；同系列里缓存异常大的容量点值得警惕；商品页出现“归档”“冷数据”定位词时按 SMR 理解。用途涉及随机写、数据库、RAID 重建的，明确选 CMR。SMR 不是“坏盘”，它把“只能顺序写”这条介质的脾气换成容量与价格；买错场景才是问题。本节结尾回看章首那句：让介质按其擅长的方式工作——对机械盘，“擅长”二字几乎就是“顺序”的同义词。

四、网卡与网络队列

网卡面对的约束与磁盘相反：磁盘是“你求它快”，网络是“它不管你收不收”。链路上的帧按线速到达，不能暂停、不可减速；对端何时发、发多快，本机说了不算。网卡的全部队列设计都在回答同一个问题：帧已经在线上了，CPU 还没准备好，怎么办。本节沿帧的生命周期走一遍：线上怎么数帧，帧进内存走哪条环，CPU 以什么节奏被叫醒，多核怎样分流，以及数据怎样少搬一次是一次。

帧、MTU 与线速 以太帧在线上的完整样子：先是 7 字节前导码加 1 字节帧起始符（SFD），让接收方的时钟恢复电路锁定比特边界；随后 6 字节目的 MAC、6 字节源 MAC、2 字节类型字段（EtherType），指明载荷交给谁——IPv4、IPv6 还是 ARP；载荷 46–1500 字节，不足 46 字节必须填充；最后 4 字节 FCS，对整帧做 CRC32 校验，错帧在网卡层面直接丢弃，不进内存。帧与帧之间还有 12 字节的帧间隙（IFG），是链路上的静默间隔。MTU 1500 指的是载荷上限；线上一帧实为 14 字节头加 1500 字节载荷加 4 字节 FCS，共 1518 字节，再算上前导与帧间隙，每帧占线 1538 字节。

把这一帧的字节布局画出来，固定开销与载荷的比例就有了形状：



图示：以太帧在线上的字节布局：前导与 SFD 8 字节用于时钟锁定，MAC 地址与类型字段 14 字节，载荷最多 1500 字节，FCS 4 字节校验，帧间还有 12 字节静默。每帧 38 字节的固定开销与帧长无关：64 字节小帧把它摊成 45% 的负担，1518 字节大帧只付 2.5%——压力测试用小帧考 pps、吞吐测试用大帧考带宽，原因就在这张图上。

字段	字节数	作用
前导码 + SFD	8	时钟恢复与帧定界，属于物理层，不进内存
目的 MAC	6	网卡按它做第一层过滤，不符则直接丢弃
源 MAC	6	标识发送方，交换机按它学习转发表
类型 EtherType	2	指明载荷协议：IPv4、IPv6、ARP 等
载荷 payload	46–1500	MTU 所指的部分，不足 46 字节要填充
FCS	4	CRC32 校验，错帧在网卡即丢弃
帧间隙 IFG	12	帧间静默，链路占用但无内容

线速的换算从这里开始。1GbE 的线上比特率是 10^9 bit/s——第八章讲眼图时给过这个速率的直观：1Gbps 下每个比特只有 1ns 的判决窗口。帧越小，每秒能过的帧越多，因为每帧都要付 38 字节的固定开销（前导 8、帧间隙 12、头 14、FCS 4）。两个端点情形：最小帧 64 字节，线上占 $64 + 8 + 12 = 84$ 字节即 672 bit，1GbE 每秒过 $10^9 / 672 \approx 1.49 \times 10^6$ 帧，约 1.49 Mpps；最大帧 1518 字节，线上占 1538 字节即 12304 bit，每秒约 $10^9 / 12304 \approx 81274$ 帧。10GbE 把这两个数都乘 10：约 14.88 Mpps 与约 81.3 万 pps。

帧长（字节）	线上占用（字节）	1GbE（pps）	10GbE（pps）
64（最小帧）	84	约 149 万	约 1488 万
512	532	约 23.5 万	约 235 万
1518（最大帧）	1538	81274	约 81.3 万

有效载荷效率值得单独算：小帧线速下，每秒载荷只有 $1.49 \times 10^6 \times 46 \times 8 \approx 548$ Mb/s，约线速的一半；大帧下 $81274 \times 1500 \times 8 \approx 975$ Mb/s，接近满载。所以“线速”与“pps”是两个独立的指标：线速（bit/s）是物理层能力，pps 度量每帧固定成本的总量——网卡 DMA 一次、描述符消耗一个、协议栈走一遍，都与帧长无关。压力测试用 64 字节小帧考 pps，吞吐测试用 1518 字节大帧考带宽，两个数字各回答一个问题；选型时先量业务的平均帧长，再决定该为哪个指标付钱。

帧格式也不是只有这一种长相。802.1Q 在源 MAC 与类型字段之间插入 4 字节标签，携带 VLAN 编号，帧长上限随之变成 1522 字节；QinQ、MPLS 再往外套。每多一层封装，MTU 的有效载荷就被咬一口：隧道出口看到的 1500 字节帧里，真正的数据可能只剩 1400 余字节——VPN 与隧道环境里“MTU 要调小”这条运维经验的来源，就是各层封装头的总长度。

最小帧 64 字节也有来历。早期共享总线式以太网靠冲突检测（CSMA/CD）工作：发送方必须发够长的时间，才能让冲突信号从总线最远端传回来，否则冲突发生而发送方浑然不觉。按 10Mb/s 时代的线缆长度与传播时延算，这个窗口定为 512 比特时，即 64 字节——最短合法帧由此而来。交换式全双工以太网早已没有冲突，最小帧却作为约定的一部分留了下来，今天我们数 pps 时仍在为它付费：每帧 38 字节的固定开销，一半来自这段历史。

MTU 本身也可以调。数据中心内部常用 9000 字节载荷的巨型帧（jumbo frame）：线上每帧约 9038 字节，1GbE 每秒帧数降到约 1.4 万，每帧固定成本被摊到六倍的载荷上，pps 与中断、协议栈开销同步下降约六倍。前提是路径上每一跳的 MTU 都得一致放大，否则分片甚至静默丢包会把收益全部吃回去——巨型帧是全网约定，不是单机参数。

“网卡不能喊停”严格说有一个例外：以太网流控允许接收方发 PAUSE 帧，请对端暂停发送指定时长。它的本意是防丢包，实际效果是把缓冲区不足的问题传染给上游——一台主机收不动，整条链路上的无辜流量一起被压。现代数据中心普遍关闭全局 PAUSE，丢包问题留给更深的环、更快的消费和拥塞协议去解决；存储网络使用的按类暂停（PFC）是另一套权衡，以队头阻塞为代价换无损。这个例外恰好坐

实本节的出发点：默认情况下，到达速率不由接收方控制，缓冲深度与消费速率必须自己兜底。

接收环与发送环 网卡与内存之间走的就是第五章的描述符环，机制原封不动：环是内存里的描述符数组，CPU 推进软件指针、设备推进硬件指针，doorbell 只提示“指针动了”，完成以设备写回的状态位为准。网卡的特殊性在于，收发两个方向的环各有一条不对称的约定：接收环要预投，发送环要回收。

RX 环是预投的。收帧不可预约—对端随时发，网卡不能喊停—所以驱动必须提前把一批空缓冲区的地址铺满接收环。帧到达后，网卡 MAC 校验 FCS，由 DMA 把帧直接写进下一个空缓冲区，写回描述符状态（帧长、校验结果、RSS 队列号），再按中断策略通知 CPU。驱动取走数据、把缓冲区交给协议栈之后，必须立刻补投新的空缓冲区：预投是义务，不是优化。环上可用缓冲区一旦耗尽，再到的帧没有地方放，只能丢弃—帧本身没有错，是没人接。

TX 环是回收的。发送由软件发起：驱动把待发帧的描述符挂上发送环、敲 doorbell；网卡按描述符 gather 数据、送上线路，发完写回完成状态；驱动在中断或轮询里回收描述符并释放缓冲区。回收不及时，环被挂满，新帧无处提交，协议栈只能排队等待，背压沿协议栈向上传导。两个方向的环因此有不同的“满”的含义：RX 环空是丢包，TX 环满是阻塞。

发送环的回收节奏同样可调：每发完一帧就中断一次太奢侈，驱动通常让网卡攒一批完成再通知，或干脆在下次发送与定时轮询时顺手回收。回收延迟直接占用环深——一批未回收的描述符，与一串未补投的空缓冲区，是同一个问题的两面：环的有效深度，等于标称深度减去滞留在“已完成未回收”状态里的那部分。

环深与丢包的关系可以直接算。RX 环的深度必须吸收“最坏通知延迟”内到达的全部帧：1GbE 小帧每秒约 149 万帧，若中断聚合加调度把最坏通知延迟拖到 200μs，期间到达 $1.49 \times 10^6 \times 200 \times 10^{-6} \approx 298$ 帧—环深 64 必然丢，512 才够；10GbE 同样的窗口要容纳约 2980 帧，常见取 4096。丢包在以太网层面不报错、不重传，只体现为网卡计数器里的数字，最后由 TCP 重传表现为“网速偶尔抖一下”。第五章“缓冲区何时归 CPU、何时归设备”的所有权问题，在这里直接变成丢包与否的分界线。

```
$ ethtool -g eth0      # show ring depths: RX 512, TX 512
$ ethtool -S eth0 | grep -E 'rx_missed|rx_no_buffer'
    rx_missed_errors: 1382      # frames arrived with no buffer
$ ethtool -G eth0 rx 4096      # deepen RX ring (driver permitting)
```

丢包计数是这条链上最诚实的仪表。应用看到的是吞吐波动与重传，协议栈看到的是乱序，只有网卡计数器直接说出“哪一环没接住”：rx_no_buffer 指预投断供，rx_missed 指环深或处理速度不够。排障的顺序因此固定：先看计数器分辨丢在哪一层，再回头调环深、中断参数或核数—本节后面三个话题，正好各管其中一环。

中断聚合 每帧—中断在小帧线速下根本不成立。1.49 Mpps 意味着每 0.67μs 一次中断；一次完整的中断处理—保存现场、读状态、递交协议栈、恢复现场—按保守的 3μs 计，单核每秒最多服务约 33 万次。需要的算力是 $1.49 \times 10^6 \times 3\mu s \approx 4.5$ 颗核的全部时间，而且全花在中断进出上，协议栈和应用一行代码都轮不到：越忙越处理不完，吞吐反而崩，这就是中断风暴（receive livelock）。0.67μs 这个帧间隔值得记住：它就是“软件为每帧固定成本付费”的时钟节拍。

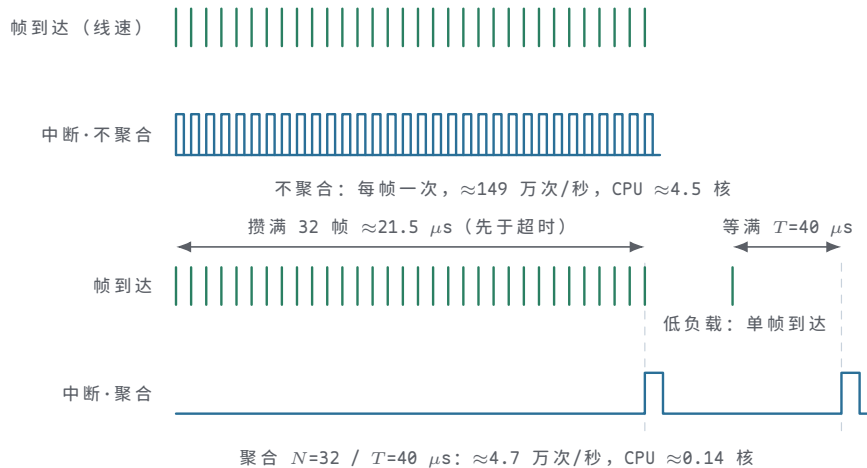
NAPI 把“中断驱动”改成“中断提示加轮询收尾”：第一帧触发中断后关闭该队列的中断，驱动进入轮询，在预算内尽量多地收完积压的帧，收干了再重新开中断—忙时纯轮询，闲时纯中断，两种状态自动切换。网卡硬件侧的中断合并（interrupt moderation）提供两个阈值：攒够 N 帧发一次中断，或距上次中断满 T μs 发一次，先到为准；低负载时靠超时保证单帧不被无限积压。

补一个实现细节：NAPI 的轮询收包运行在软中断（softirq）上下文，协议栈的 IP、TCP 处理大多也在其中。top 里看到的 si 一项的占比，就是这条收包路径的 CPU 开销；它高到挤压应用，说明帧处理本身—而不只是中断进出—已经成为瓶颈，

下一步手段是多队列分流，见下一段。

权衡可以量化。设合并参数 $N=32$ 、 $T=40\mu\text{s}$ ：小帧线速下 32 帧只需 $32 \times 0.672 \approx 21.5\mu\text{s}$ ，先于超时触发，中断率从约 149 万次/秒降到 $1.49 \times 10^6 / 32 \approx 4.7$ 万次/秒，CPU 开销 $4.7 \times 10^4 \times 3\mu\text{s} \approx 0.14$ 颗核——从中断风暴的 4.5 颗核降到约七分之一颗。代价是延迟：一帧平均多等约半批的时间，十微秒量级。吞吐型业务把 N 、 T 调大；低延迟业务（行情、内存数据库）调小甚至关闭合并，改用绑核加 busy-poll——拿一颗核 100% 的占用，换确定的几微秒时延。两个方向都对，区别只在业务把哪一头算作成本。

把两种策略各画一条时序，这笔交换的形状如下：



图示：中断聚合的时序对比。上方不聚合：每帧一次中断，1.49 Mpps 折合每秒约 149 万次中断进出，约 4.5 颗核的算力全花在保存与恢复现场上。下方聚合：攒满 32 帧（线速下约 $21.5 \mu\text{s}$ ，先于超时）或等满 $T=40 \mu\text{s}$ ——先到为准——才发一次中断，中断率降到约 4.7 万次/秒，CPU 开销约 0.14 核；代价是一帧平均多等约半个聚合窗口。

参数	机制	偏向吞吐	偏向时延
帧数阈值 N	攒够 N 帧发一次中断	调大（如 32-64）	调小或置 1
超时 T	距上次中断满 $T \mu\text{s}$ 即发	调大（几十 μs ）	调小或置 0
轮询预算	NAPI 每次最多收多少帧	调大	适中，防占核过久
busy-poll	应用自旋收帧，跳过中断	浪费核	时延最低且稳

观察手段是现成的：每秒读一次 `/proc/interrupts` 里各网卡队列的中断计数，与流量对比就知道合并参数是否合身——中断率贴近 pps，说明合并未生效；中断率过低而时延投诉上升，说明 N 、 T 过了头。ethtool -c 与 -C 查询、调整这些参数。调网卡的第一步通常不是换硬件，而是把中断率、环深、pps 三个数摆在一起看；三者相符，瓶颈才轮到协议栈和应用。

```
$ grep eth0 /proc/interrupts
28: 1204431      0      0      0 IO-APIC 28-fastio eth0-rx-0
29:      0 1188204      0      0 IO-APIC 29-fastio eth0-rx-1
30:      0      0 1210022      0 IO-APIC 30-fastio eth0-rx-2
31:      0      0      0 1195330 IO-APIC 31-fastio eth0-rx-3
# per-queue counters spread over 4 CPUs: RSS and affinity agree
```

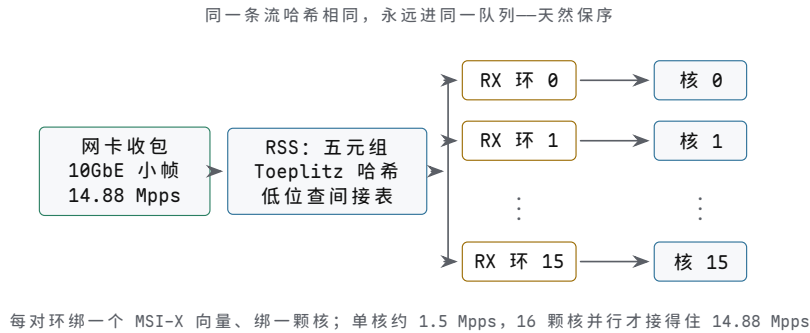
读法：每个队列的中断计数应大致均匀地散在各核上；挤在一列说明 RSS 或绑核没有配对，整行全为 0 的队列说明中断亲和漏配。这个文件也是下一话题的入口：多队列不是插上就有，是用对才有。

多队列与 RSS 单队列的天花板先算清楚：一个环只能由一颗核服务——保序与 cache 局部性都要求如此——单核跑协议栈每秒约 1-2M 帧已是优化后的水平；10GbE 小帧是 14.88 Mpps，差一个数量级。单队列网卡插上万兆链路，瓶颈不在线，在环。用第十章 Little 定律的读法：收帧路径的吞吐等于并行消费者数乘单消费者速率，

在途帧排得再多，消费者只有一颗核。

多队列网卡把 RX/TX 环做成很多对，常见 16-128 对，每对绑一个 MSI-X 中断向量、绑一颗核。接收侧的分流由硬件完成：RSS 对每帧的五元组（源 IP、目的 IP、源端口、目的端口、协议号）计算 Toeplitz 哈希，取哈希低位查间接表，把帧定向到对应队列。同一条流哈希相同，永远进同一队列，天然保序；不同的流散到不同核，并行处理。发送侧对称：软件按流或按 CPU 选发送队列，各核各挂各的环，提交路径无锁。

分流的全过程画成一张图，就是哈希定向加按核绑环：



图示：RSS 接收分流：网卡对每帧的五元组算 Toeplitz 哈希，取低位查间接表，把帧定向到对应 RX 环；每对环绑一个 MSI-X 中断向量、绑一颗核，各核独立处理自己环上的帧，提交路径无锁。同一条流哈希相同、永远进同一队列，保序由哈希天然保证；并行度由队列对数决定—10GbE 小帧 14.88 Mpps 对单核约 1.5 Mpps，16 队列绑 16 核才接得住。

配置按一遍数：10GbE 小帧 14.88 Mpps，单核约 1.5 Mpps，需要约 10 颗核起步，留余量配 16 队列、16 个 MSI-X 向量、绑 16 颗核。三个配置必须一起看：队列数、间接表、中断亲和—一队列开了 32 条而中断全绑在核 0，等于单队列；irqbalance 之类的自动均衡在万兆以上往往帮倒忙，绑核要按队列手工固定。

同样的算法推到更高速率：25GbE 小帧约 37.2 Mpps，40GbE 约 59.5 Mpps，100GbE 约 148.8 Mpps—单核约 1.5 Mpps 的处理率面前，需要的核数按比例涨到约 25、40、100 颗，已经没有机器装得下。速率每翻一番，”每帧固定成本”问题就尖锐一分：100GbE 时代还按”中断加协议栈逐帧处理”的模型工作是不成立的，内核旁路与硬件卸载从可选项变成必选项，这正是零拷贝一段要收尾的方向。

组件	作用	配置不当的典型症状
队列对	收发各一条环，独立 DMA 与中断	队列少于核数，部分核闲置
MSI-X 向量	每队列一个中断号	向量不足，多队列共享中断互拖
RSS 间接表 中断亲和	哈希值到队列的映射 把向量绑到指定核	映射不均，各核负载歪斜 全绑核 0，等于退回单队列

RSS 也有边界。它只认五元组：GRE、VxLAN 等隧道的外层包头都相同，所有隧道流量挤进一条队列，较新的网卡支持按内层字段哈希来解决。单条大象流永远只能跑一颗核的速度，这是保序的代价，只能靠多开几条流来绕。与 RSS 同族的卸载还有 TSO 与 GRO：发送侧由网卡把大块数据切成 MTU 帧，接收侧把同流相邻帧拼成大块再上交—它们不改队列结构，改的是”每帧固定成本”里的帧数，与巨型帧是同一思想的软硬件两种实现。

零拷贝的基本思想 先算拷贝一次的成本。传统收包路径上，网卡 DMA 把帧写进内核缓冲区，这一步不占 CPU；协议栈处理后再由 copy_to_user 把数据复制一份到应用缓冲区—这一次是 CPU 逐字节的复制，读一遍、写一遍。10Gb/s 的流量约 1.25GB/s，一次拷贝产生 2.5GB/s 的内存流量；收发各拷一次就是 5GB/s，约占一条 DDR4-3200 通道（25.6GB/s）可用带宽的两成，还没算 cache 被冲刷的间

接代价。用第十章 `roofline` 的话说：每次无谓的拷贝都在消耗全机共享的带宽屋顶，留给应用的屋顶相应变矮。

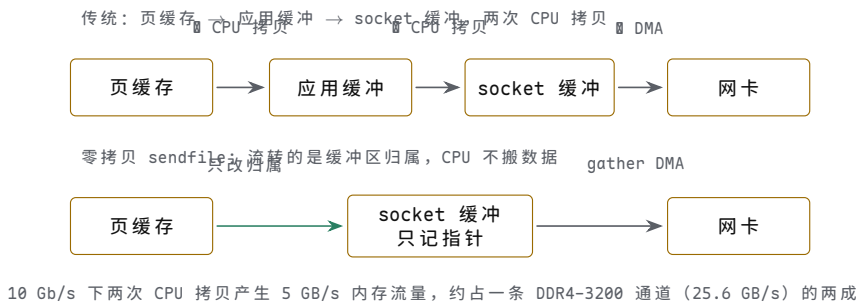
零拷贝的思路是让数据在内存里只躺一份，流转的是”这块缓冲区归谁”。`sendfile` 让文件页缓存直接对接 `socket`，配合支持 `scatter-gather` 的网卡，连”协议头加数据”的拼接都由网卡 `gather DMA` 完成，CPU 一个字节不搬；`splice` 在两个文件描述符之间搬运页指针；`mmap` 让应用直接读写内核页，省掉复制但引入页错误与同步的成本；`io_uring` 的固定缓冲区与 `AF_XDP` 更进一步，把缓冲区的所有权约定直接开放给用户态。注意”零拷贝”省的从来不是 `DMA` 那一次—设备搬运本来就绕不开—省的是内核与用户态之间那一次 `CPU` 复制。

```
#include <sys/sendfile.h>

/* stream a file to a socket without copying through user space */
ssize_t sendfile(int out_fd, int in_fd, off_t *offset, size_t count);

/* kernel path: page cache pages -> socket buffer -> NIC gather DMA;
   file contents never pass through a CPU register */
```

两条路径并排画出来，零拷贝省掉的是哪一段就一目了然：



图示：发送一个文件的两种路径。传统路径上数据两次经过 CPU：页缓存拷进应用缓冲、应用缓冲再拷进 socket 缓冲，每次都是读一遍写一遍，10 Gb/s 的流量就此产生 5 GB/s 的内存流量。`sendfile` 让页缓存直接对接 socket，缓冲区只改归属、内容原地不动，网卡用 `gather DMA` 把协议头与数据拼接发出，CPU 一个字节不搬—省的是内核与用户态之间那次复制，设备 `DMA` 那一步本来就有。

机制	省掉的是哪次拷贝	适用场景
<code>sendfile</code>	文件页缓存到用户态的往返	静态文件服务、CDN
<code>splice</code>	任意两个 fd 之间的搬运	代理、转发管道
<code>mmap</code>	读方向的复制	随机访问大文件
<code>io_uring</code> 固定缓冲	每次 I/O 的缓冲区注册与复制	高频异步 I/O
<code>AF_XDP</code> / <code>DPDK</code>	整个内核协议栈路径	极端 pps 的专用应用

工程含义与边界。零拷贝的本质是所有权转移代替内容复制，正是第五章”缓冲区何时归 CPU、何时归设备”那条问题在协议栈内部的延长线。它不是免费午餐：页共享带来生命周期管理与写时复制的复杂度，数据量小时固定开销可能盖过节省；它适用的条件是大块数据、高带宽、CPU 比带宽更紧张—这正是存储服务器、CDN、反向代理人手一份 `sendfile` 的原因。从本章视角看，零拷贝与中断聚合、RSS 是同一族手段：网络栈的性能工作，大半是在给”每帧固定成本”与”每字节搬运成本”这两项开销找出路。

旁路也不是没有代价。把协议栈整个绕开，意味着 `TCP` 的拥塞控制、分片、校验和、防火墙一并让位，应用要么自己实现，要么声明不需要；内核协议栈占的 CPU，换来的正是这些不免费的功能。工程上成熟的做法是分层：极端 pps 的入口（行情网关、负载均衡）用旁路或 `AF_XDP`，业务主体仍走内核栈加零拷贝。第五章那句提醒在这里同样成立：约定一旦绕开，原来由内核代管的所有权、可见性与错误处理，全部换成应用自己的功课。

五、外设队列综合案例与尾延迟

前四节分别看了三类设备的介质与各自的队列结构，本节把它们汇合到一个问题上：队列机制怎样决定外设实际表现出来的性能。先深入一个样本—NVMe 的队列模型，把第五章描述符环的全部细节在一块真实设备上走一遍；再逐拍跟踪一次读请求从 `read()` 到数据可用的完整路径；然后回答平均时延回答不了的问题：P99 长尾从哪里来；接着算队列深度这笔吞吐与时延的交换账；最后用两则调试案例和一张三类设备通用的检查表收束全章。

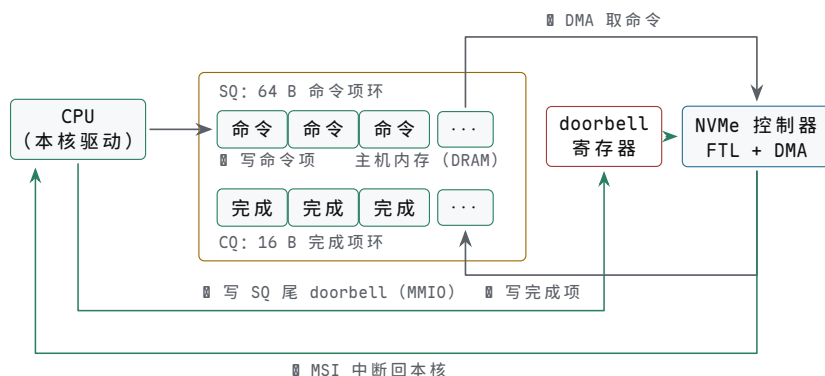
NVMe 的队列模型 第五章讲描述符环时给过一般结构：环不动、指针动，doorbell 只通知、不携带数据，完成与否以设备写回的状态为准。NVMe 把这套结构定型为规范，并针对多核做了两处关键改动。第一处改动是队列成对：NVMe 的基本单元是一对队列—提交队列 SQ (submission queue) 与完成队列 CQ (completion queue)。SQ 项是 64 字节的命令：操作码（读、写、flush）、起始 LBA、块数，以及 PRP 数据指针列表，逐页给出数据在主机内存里的物理地址；CQ 项是 16 字节的完成记录：命令 ID、状态码，以及一个相位位—队列回绕一圈相位翻一次，主机据此区分新旧完成项，不必比较指针。提交与完成分成两个环，各走各的指针：设备消费 SQ 不必等主机回收 SQ，完成信息也不挤在命令项里写回。这是比第五章通用环更彻底的分工。

一次提交的完整序列有七步，逐步对应第五章的每一条约定：

1. 驱动把 64 B 命令项写入本核 SQ 的尾槽
2. 屏障：保证命令项先于通知对设备可见
3. 写 SQ 尾 doorbell（一次 MMIO 写，内容只是新尾指针）
4. 设备看到指针差，DMA 取走新命令项
5. 设备执行：查 FTL、读 NAND，按 PRP 地址把数据 DMA 进内存
6. 设备写 16 B CQ 完成项，相位位翻转
7. 设备发 MSI-X 中断；驱动读 CQ、按命令 ID 找回请求并回收，最后写 CQ 头 doorbell，把完成槽位还给设备

第 3 步的 doorbell 仍然不携带数据：设备靠首尾指针的差补读新命令，偶尔错过一次 doorbell 并无妨碍；第 6、7 步的次序也不许交换：先写完成项、再发中断，主机在中断处理里读到的一定是已经写好的 CQ 项。第五章“描述符先可见、再通知”和“完成以写回状态为准”这两条规则，在这里原样成立，只是被写进了设备规范，由硬件而不是驱动员的细心来保证。

把这条链路画成图，主机内存、doorbell 寄存器与设备之间的分工如下：



图示：NVMe 的提交-完成链路。驱动把 64 B 命令项写进本核 SQ 尾槽 (1)，屏障之后写 SQ 尾 doorbell (3)——doorbell 只携带新尾指针；设备按指针差 DMA 取走命令 (4)，执行后把 16 B 完成项写进 CQ (6)，此时数据已按 PRP 地址 DMA 进入内存，最后以 MSI-X 中断通知本核 (7)。SQ 与 CQ 各自独立、每核一对，提交路径上没有需要上锁的共享状态。

第二处改动是多队列。规范允许最多 65535 对 I/O 队列，每对深度上限 65535；真实盘实现的对数远小于此（几十到几千），但已足够给每个 CPU 核配一对。每核一对的用意是免锁：各核写自己的 SQ、收自己的 CQ，尾指针是核私有的，提交路

径上没有任何共享状态需要上锁。这不是锁实现得好，而是根本没有共享——第十章谈多核时说过，把共享拆开比把锁做快更值钱，NVMe 的多队列是这句话在外设接口上的直接应用。配合 MSI-X 的多向量中断，完成也能按队列分发回本核处理，提交与完成两条路径都不出核。

对照 AHCI，就能看出这两处改动解决了什么。AHCI 整套控制器只有一张命令表，32 个槽位（NCQ 深度 32），一个完成中断源：所有核的提交都抢同一张表的锁，完成中断也集中在一处。在 SATA 的 600 MB/s、约十万 IOPS 的上限下，这张表不是瓶颈；PCIe SSD 把带宽抬高一个数量级、把 IOPS 抬高两个数量级，提交路径上的共享状态先成为瓶颈。NVMe 的多队列不是锦上添花，而是把 AHCI 那张唯一的表拆开之后的必然形态。

维度	AHCI (SATA 时代)	NVMe (PCIe 时代)
队列数	整控制器一张命令表	规范上限 65535 对 SQ/CQ
每队深度	32 (NCQ 槽位)	上限 65535，常用 32 至 1024
提交路径	共享表，多核提交要抢锁	每核一对，核私有指针，免锁
完成通知	单一中断源	MSI-X 多向量，可按核分发
典型上限	600 MB/s、约十万 IOPS	数 GB/s、几十万到百万 IOPS

深度上限 65535 也不意味着要用满。队列深度的用途是维持在途请求数，第十章的 Little 定律给出该维持多少；用满上限买到的是排队而不是吞吐——本节第四主题会把这笔账算到具体数字。

案例：一次 NVMe 读的完整 trace 把一个进程执行 `read(fd, buf, 4096)`、且数据不在页缓存的全过程按时序列出。设备取一块典型的 PCIe 3.0 x4 NVMe SSD，请求是 4 KB 随机读。

步	动作	耗时量级	执行者
1	系统调用陷入内核，检查参数与文件描述符	约 1 μ s	CPU
2	VFS 查页缓存：未命中，分配空闲页框	微秒级	CPU
3	文件系统把文件偏移翻译成设备 LBA	微秒级	CPU
4	块层生成请求，调度器尝试与相邻请求合并	微秒级	CPU
5	驱动把请求编成 64 B 命令项，写入本核 SQ 尾槽	微秒级	CPU
6	屏障之后写 SQ 尾 doorbell (MMIO 写)	百纳秒级	CPU
7	设备 DMA 取走命令项	微秒级	设备
8	查 FTL 映射，读 NAND 页，ECC 校验	约 80 μ s	介质
9	数据按 PRP 地址 DMA 写入第 2 步的页框	约 1 μ s	设备
10	设备写 16 B CQ 完成项，相位位翻转	微秒级	设备
11	设备发 MSI-X，中断控制器转成 CPU 中断	微秒级	设备

12	中断处理读 CQ，找回请求，标记完成，唤醒进程	微秒级	CPU
13	页缓存数据拷入用户缓冲， <code>read</code> 返回 4096	微秒级	CPU

总时延约 100 μs ，其中第 8 步介质本身占去八成，软件与队列合计约两成——这就是 NVMe 时代的比例：协议和驱动的开销已被压到介质时延的零头，继续优化软件路径收益有限，优化介质的访问形态（对齐、合并、足够的在途深度）才是主战场。这张表也是排障地图：慢在哪个量级，就查哪一列——微秒级的步骤出问题多半是驱动或内核配置，毫秒级的问题才轮到介质与垃圾回收。

第 6 步值得停一下：先写命令项、再写 `doorbell`，中间隔一道屏障，这正是第五章“描述符先可见、再通知”的约定；顺序反了，设备可能读到半写入的命令项。第 11 步也值得一提：MSI 不是一根专用电平线，而是设备向中断控制器的特定地址做一次写事务。第八章讨论过沿导线传播的信号要付出走线与时序的代价；MSI 干脆把中断变成数据流里的一次写，省掉专用引脚，还能在写数据里携带向量号，MSI-X 于是能按队列把中断分发到不同核——中断从“一根线”演化成“一条消息”，和多队列是同一个设计方向。

同一件事在第六章的 BIOS 路径上长什么样，对照一下就清楚了：

维度	int 13h（第六章 BIOS 路径）	NVMe 路径
提交	参数装寄存器，执行 <code>int 0x13</code>	命令项写内存队列， <code>doorbell</code> 通知
地址等待	CHS 柱面/磁头/扇区 BIOS 轮询状态寄存器，独占 CPU	LBA 加 PRP 物理页列表 进程睡眠，完成由 MSI 唤醒
并发搬运	一次一个在途 软盘走 8237 DMA，硬盘 PIO 或总线主控	几十上百在途，多队列并行 设备直接 DMA 到页框

骨架是同一个：准备命令、通知设备、DMA 搬数、确认完成。`int 13h` 是这套骨架的同步简化版——提交即等待，全程没有队列，CPU 停在 BIOS 例程里轮询；NVMe 是它的并发完整版——提交与完成解耦，等待换成中断，在途请求从一个变成几百个。从 8237 的一个 DMA 通道到每核一对队列，外设接口四十年的演进主线，就是把“一次一单”改成“持续事务流”，而描述符与 DMA 这两个零件自始至终没有换过。

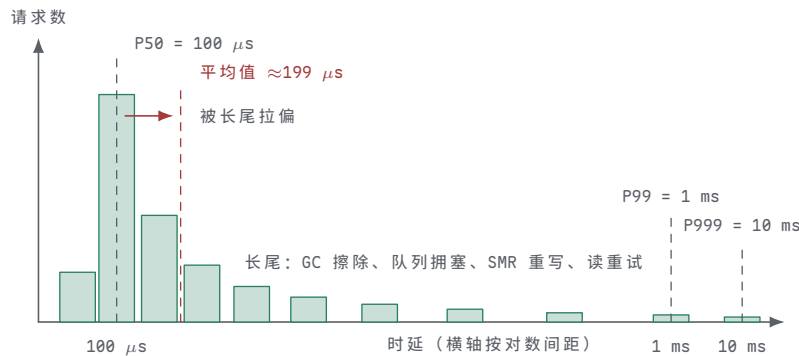
尾延迟：P99 从哪里来 前面反复出现的“典型时延 100 μs ”只是中位数附近的值。第十章已经立下规矩：时延是一个分布，平均值会撒谎。外设是把这条规矩演绎得最充分的地方——慢请求不是随机噪声，而是有明确物理来源的事件，每一类都对应本章前几节讲过的机制：

长尾来源	物理机制	典型幅度	触发条件
GC 擦除	写请求排在一次块擦除与有效页搬运后面	毫秒级	空闲块耗尽、盘接近写满
队列拥塞	深队列高负载下排队，第十章的 $1/(1-\rho)$	百微秒级	深度过大或负载接近饱和
SMR 重写	区域写满触发整段重写（第三节）	十毫秒级	随机写命中已写区域
读重试	阈值电压漂移，ECC 纠不动，换挡重读	数十微秒至毫秒	老盘、高温、久置数据
映射回读	FTL 映射在 DRAM 中未命中，回读 NAND 映射页	百微秒级	冷地址、大跨度跳跃

这些事件不常发生，所以中位数很好看；但一旦发生就是十倍、百倍的慢。消费级 NVMe SSD 上 4 KB 随机读的典型分布是：

百分位	时延	相对中位数	直观含义
P50	100 μ s	1 倍	典型请求
P99	1 ms	10 倍	每 100 个里约 1 个这么慢
P999	10 ms	100 倍	每 1000 个里约 1 个这么慢

把这张分布画成直方图，长尾的形状比数字更直接：



图示：消费级 NVMe SSD 4 KB 随机读的时延分布（横轴按对数间距）。主峰在 $P50 = 100 \mu\text{s}$ ；GC 擦除、队列拥塞、SMR 重写、读重试这类不常发生、一发生就是十倍百倍慢的事件，把分布拖出长尾： $P99 = 1 \text{ ms}$ ， $P999 = 10 \text{ ms}$ 。平均值被长尾向右拉偏，离中位数越来越远——所以外设时延必须带百分位一起报。

平均时延把这些全藏了起来。算一遍第十章的同款例子：100 个请求里 99 个花 100 μs 、1 个花 10 ms，平均值是 $(99 \times 100 + 10000)/100 = 199 \mu\text{s}$ ——平均看上去只比中位数差一倍，可那一个请求慢了 100 倍。若服务目标是“99% 的请求快于 200 μs ”，一份只报平均值 199 μs 的报告，会把一次彻底的违约藏进“接近达标”的表象里。用户记住的从来不是平均那次，而是最慢那次：页面偶尔卡住三秒，原因往往就是排在 P999 上的那个请求撞上了 GC 擦除或 SMR 重写。

工程含义有三条。其一，报外设时延必须带百分位：平均值至少配 P99，交互与实时负载再看 P999，只报平均值的测试报告不可信。其二，容量规划按 P99 反推负载上限：让 P99 保持在目标内的负载，通常只有平均时延开始明显变坏时的一半左右——这与第十章“稳态利用率压在五到七成”是同一条纪律在外设上的形式。其三，长尾不能靠平均优化消掉，只能按来源逐个治理：GC 尖峰靠预留空闲块和 TRIM，队列拥塞靠限深或限流，SMR 重写靠改变写入形态——这正是本节后面两个主题的内容。

队列深度与吞吐时延的权衡 第十章的 Little 定律 $L = \lambda W$ 用在外设上的形式是：在途请求数等于 IOPS 乘单请求时延。介质时延 W 由物理决定（4 KB 随机读约 100 μs ），吞吐目标 λ 由应用决定，队列深度是主机侧唯一的旋钮。两个方向都有代价。

深度太小，喂不饱设备。QD=1 时 $\lambda = 1/W = 10^4$ IOPS，按 4 KB 折算只有 40 MB/s——一块标称 3.5 GB/s 的盘只用了约 1%。深度加大，吞吐近似线性上升，直到撞上设备自身的并行度上限（NAND 通道数与控制器能力）。设某盘上限为 50 万 IOPS，拐点的位置由 $L = \lambda W = 5 \times 10^5 \times 10^{-4} = 50$ 唯一决定。越过拐点再加深度，吞吐不变，多出来的在途全部变成排队：QD=256 时平均时延约 $256/(5 \times 10^5) = 512 \mu\text{s}$ ，是介质时延的五倍多。

队列深度	可达 IOPS	平均时延	位置
1	1 万	100 μs	深度限制，带宽饿着
4	4 万	100 μs	深度限制

16	16 万	100 μ s	深度限制
32	32 万	100 μ s	接近拐点
64	50 万（封顶）	约 128 μ s	刚过拐点，开始排队
128	50 万	约 256 μ s	排队区，吞吐不再涨
256	50 万	约 512 μ s	排队区，纯买时延

这条曲线的形状与第十章的 roofline 是同一族：拐点左侧被深度（在途数）限制，右侧被设备能力封顶，拐点本身由 λW 决定。工程读法也只有一句：深度按 λW 取，再留一档余量；过了拐点一分钱吞吐也买不到，买到的全是时延。

主机侧的经验值：每队列深度取 32 至 128 是常见区间；`fio` 压测的惯例 `io depth=32` 正在典型拐点附近，这也是它成为默认值的原因。时延敏感的负载反而要浅：日志刷盘、同步写这类请求应走独立队列或浅队列，避免排在批量流量后面——用第十章的语言说，浅队列买的是时延的可预期性，不是平均速度。多队列改变的是这笔账的分母： n 个核各开一对队列，每对只承担 λ/n ，同样的总吞吐用更浅的每队深度就够，排队时延同步下降——NVMe 的多队列与深度取舍，是同一条定律的两个应用。

调试案例两则 两则案例的共同点是：症状都出现在吞吐或时延上，根因都在队列机制与介质特性的交界处，定位方法都是“先看队列计数，再看介质状态”。

案例一：队列深度 1 的吞吐瓶颈。症状：新服务器的 NVMe SSD 规格 3.5 GB/s，实测顺序读只有约 40 MB/s，4K 随机读约 1 万 IOPS，离规格差两个数量级。定位：`iostat -x` 显示设备利用率 100%，但队列长度 `aqu-sz` 恒为 1.00；压测用的是 `fio` 默认的同步 I/O，`iodepth=1`。根因：同步 I/O 一次只有一个请求在途，Little 定律直接封顶——4 KB/100 μ s = 40 MB/s。设备没有病，是提交方式只给它一单做一单。修复：改用 `libaio` 或 `io_uring`，`iodepth` 提到 32。同机复测：4K 随机读从 1 万 IOPS 升到约 32 万 IOPS，顺序读回到 3 GB/s 量级——吞吐涨了约 32 倍，硬件一分钱没动。教训：报外设性能必须先报队列深度，没有深度的 IOPS 数字没有意义；看到利用率 100% 先看队列长度——利用率满而队列空，瓶颈在提交侧，不在设备侧。

案例二：GC 风暴导致的周期性延迟尖峰。症状：持续随机写的服务，P50 写时延稳定在 200 μ s，但每隔几秒出现一批 10 至 50 ms 的慢写，P99 从 1 ms 恶化到 30 ms；写入速率越快，尖峰间隔越短。定位：时延直方图呈双峰；盘已写到接近满容量；SSD 健康信息里主机写入量远小于 NAND 实际写入量，写放大超过 3。根因：盘写满后没有现成空闲块，每次写都要先做一次 GC——读出整块、搬走有效页、擦除——写请求排在一次毫秒级的擦除后面；空闲块耗尽一批就尖峰一次，所以呈周期性，且写入越快耗尽越快。对照实验：对盘做安全擦除（或全量 TRIM）后重测，尖峰消失、P99 回落，确认根因在空闲块供给，不在队列深度或接口。修复：文件系统开启 `discard` 或定期 `fstrim`，让删除的文件及时变回设备可用的空闲块；预留空间提到 20% 以上；时延敏感的服务不把盘写满，容量水位线设在 70%。用闲置容量换空闲块的持续供给，本质上是给 GC 留余量——与第十章“服务台保持余量”是同一条纪律。

两则案例放在一起看，方法上的共性比结论本身更值得带走。两案的设备都是完好的，换硬件解决不了任何一案；两案的突破口都是计数器而不是猜测——队列长度、利用率、写放大、时延直方图，全是现成的可观测数据；两案的修复也都不动设备，一个改提交方式，一个改介质水位。外设问题的默认值应当是“机制问题”而不是“硬件坏了”：先把 Little 定律和介质的脾气代进去算一遍，算不通再怀疑硬件。

外设问题的通用读法 全章收束为一张检查表。面对任何外设的性能或异常问题，按三步走：先问介质特性——这类设备的物理限制是什么；再问队列机制——它用什么结构弥补介质的短板；最后测量验证——不看规格书看计数器，不看平均看百分位。三类设备放进同一张表：

设备	介质先问	队列再问	测量验证
----	------	------	------

SSD	空闲块剩多少，写放大多少	提交深度多少，谁在排队	<code>fio</code> 按百分位报时延， <code>iostat</code> 看 <code>aqu-sz</code>
机械盘	随机还是顺序，寻道占几成	调度器合并了吗，NCQ 用上了吗	<code>iostat</code> 看 <code>await</code> 与服务时间之差
网卡	先到 pps 顶还是带宽顶，帧多大	聚合参数多少，多队列吃满核了吗	<code>ethtool -S</code> 看丢包计数，按核看中断分布

三类设备共享同一张检查表不是巧合：第五章的描述符环与 DMA、第十章的 Little 定律与百分位纪律，在每一块外设上都是同一套语言。读外设问题的功力，说到底就是先认出介质那一栏，再走完两栏。

把三步走完整演练一次。某数据库机器报告“磁盘慢”：第一步问介质——这是一块 SATA SSD 还是 NVMe SSD、写到了几成满、随机写占多大比例，因为满盘加随机写正是 GC 尖峰的经典触发条件；第二步问队列——提交深度多少、走的是单队列还是多队列、同步写有没有和批量读混在同一对队列里，因为平均时延 $200\ \mu\text{s}$ 但 P99 到 $20\ \text{ms}$ 的形状，指向的是“排在慢事件后面”而不是“介质本身慢”；第三步测量验证——`fio` 单独复现负载并按百分位报时延，`iostat -x` 对照 `aqu-sz` 与利用率，再查 SSD 健康信息里的写放大与剩余备用块。三步走完，“磁盘慢”这句模糊报告就被翻译成一句可执行的话：盘已到 92% 容量、写放大 4.5，先把水位降到 70% 再谈别的。同一个流程搬到网卡上，只是第一栏换成 pps 与帧长、第三栏换成 `ethtool` 计数——方法不动，参数换设备。

章末练习

任务	要求	学习目标
扇区对齐	某 4K 扇区硬盘的分区从 63 号逻辑 (512 B) 扇区开始，文件系统按 4 KB 块写入。求分区起始字节偏移是否为 4 KB 的整数倍；说明每次文件系统块写要触及几个物理扇区、代价是什么，并给出修复做法	分区边界与物理扇区对齐
写放大计算	某 SSD 页 16 KB、每块 256 页，随机写稳态下每块平均 75% 为有效页。求写放大系数；主机累计写入 10 GB 时，NAND 实际被写入多少	写放大 = $1/(1 - \text{有效占比})$ 的来源
随机 IOPS 估算	7200 rpm 硬盘，平均寻道 4 ms，顺序传输率 150 MB/s。估算 4 KB 随机读的 IOPS 与等效带宽，并与顺序读带宽比较	毫秒级寻道加旋转决定随机 IOPS
中断聚合权衡	1 Gb/s 网卡满载 64 B 小帧（线速约 148.8 万 pps），每次中断处理约 $5\ \mu\text{s}$ 。求不聚合时的 CPU 开销；聚合到每 32 帧一次中断后的开销；说明聚合付出的代价	聚合用时延换 CPU 占用
NVMe trace 排序	把一次 NVMe 读的这些事件排成时间序：DMA 写数据到内存、写 SQ 尾 doorbell、页缓存未命中、设备写 CQ 完成项、MSI 中断、驱动写命令项到 SQ、唤醒进程	提交-执行-完成三段的次序

队列深度选择	某 NVMe SSD 4 KB 随机读时延 $90\ \mu\text{s}$ ，应用目标 40 万 IOPS。用 Little 定律求所需最小队列深度；若设备服务率上限 50 万 IOPS，求 QD=256 时的平均时延	$L = \lambda W$ ；过拐点只加排队
--------	---	--------------------------

参考解答与要点

任务	参考解答	必须掌握的要点
扇区对齐	起始偏移 = $63 \times 512 = 32256\ \text{B}$ ， $32256/4096 = 7.875$ ，不是 4 KB 的整数倍，分区未对齐。文件系统每个 4 KB 块都跨越两个物理 4 KB 扇区，写一块要先读出两个扇区、修改、再写回（读-改-写），写代价约翻倍；SSD 上同理跨页、放大 GC 负担。修复：分区起始对齐到 1 MiB (2048 个 512 B 扇区) 边界，现代分区工具默认如此	先算偏移再谈对齐；不对齐的代价是读-改-写
写放大计算	每回收一个块：块容量 $256 \times 16\ \text{KB} = 4\ \text{MB}$ ，其中有效 $4 \times 0.75 = 3\ \text{MB}$ 须先搬出，只换得 1 MB 空闲。稳态下主机每写 1 MB，NAND 要写 1 MB 数据加 3 MB 搬运共 4 MB，写放大 = $4 = 1/(1 - 0.75)$ 。主机累计写 10 GB，NAND 实际写入 $10 \times 4 = 40\ \text{GB}$	有效占比越高写放大越大；留空闲空间直接降放大
随机 IOPS 估算	7200 rpm 每转 $60/7200 \approx 8.33\ \text{ms}$ ，平均旋转等待半转 $4.17\ \text{ms}$ ；加平均寻道 $4\ \text{ms}$ ，4 KB 传输 $4096/(1.5 \times 10^8) \approx 0.03\ \text{ms}$ ，单次共约 $8.2\ \text{ms}$ 。IOPS $\approx 1/0.0082 \approx 122$ ；等效带宽 $122 \times 4\ \text{KB} \approx 0.49\ \text{MB/s}$ ，与顺序 150 MB/s 相差约 300 倍	随机 IOPS 由寻道加旋转写死；顺序与随机差两个数量级
中断聚合权衡	不聚合： $1.488 \times 10^6 \times 5\ \mu\text{s} \approx 7.4\ \text{CPU 秒/秒}$ ，约需 8 个核专职处理中断，系统实际早已丢包。聚合 32 帧一次：中断率 $1.488 \times 10^6/32 \approx 4.65 \times 10^4\ \text{次/秒}$ ，CPU 开销 $\approx 4.65 \times 10^4 \times 5\ \mu\text{s} \approx 0.23\ \text{秒/秒}$ ，约一个核的 23%。代价：帧要等够一批或等到聚合定时器超时，平均时延增加约半个聚合窗口（定时 $64\ \mu\text{s}$ 时约 $32\ \mu\text{s}$ ）	聚合把到达率除以批大小；吞吐与时延的交换
NVMe trace 排序	次序：页缓存未命中 → 驱动写命令项到 SQ → 写 SQ 尾 doorbell → 设备 DMA 写数据到内存 → 设备写 CQ 完成项 → MSI 中断 → 唤醒进程。两处次序是硬性约定：doorbell 必须在命令项写好之后（先可见、再通知），CQ 完成项必须在 DMA 数据之后（先数据、后完成），唤醒只能在中断处理确认完成之后	提交-执行-完成三段；每段先后都是可见性约定

队列深度选择 $L = \lambda W = 4 \times 10^5 \times 90 \times 10^{-6} = 36$ ，最小深度 36，工程上取 64 留一档余量。设备上限 50 万 IOPS 时已经饱和，QD=256 的平均时延 $\approx 256 / (5 \times 10^5) = 512 \mu\text{s}$ ：吞吐仍是 50 万 IOPS，多出的深度全部变成排队，时延约为 $90 \mu\text{s}$ 的 5.7 倍

GPU 与显示系统

CPU 为单条指令的时延优化，GPU 为海量数据的吞吐优化——这是理解 GPU 的全部出发点。为了吞吐，GPU 把芯片面积让给成排的算术单元，把单线程性能让给了同时运行的上万个线程；为了喂饱这些单元，它配了独立的显存和巨大的带宽。但 GPU 对系统程序员来说并不神秘：命令照样走队列，数据照样走 DMA，完成照样走 fence 和中断。本章把 GPU 从”显卡”还原成一台吞吐优先的并行处理器和它的显示子系统。

对象	与 CPU 的差别	本章回答的问题
执行模型	上万吨线程批量推进，不追单线程时延	SIMT 怎样组织并行
显存	独立 VRAM/HBM，带宽数倍于主存	数据怎样进出 GPU
命令路径	命令环 +DMA+fence，不逐条同步	CPU 与 GPU 怎样协作而不互等
显示子系统	按固定时序扫描帧缓冲	渲染完成为什么不等于显示完成

核心思想

GPU 的性能秘密不是”核多”，而是”用海量在途工作隐藏一切时延”。读 GPU 文档时，先问在途有多少线程、数据在谁的存储里、同步在哪个边界。

一、GPU 为什么长得不像 CPU

本节从一块硅片的预算说起。可用的晶体管总数大体固定，怎么花，决定了芯片擅长什么：CPU 把预算的大头给了控制与缓存，因为通用程序里单条指令等不起；GPU 把预算给了成排的算术单元，因为它要处理的工作天然是一百万份互不依赖的小任务。本节先看两种设计取向怎样分配面积，再从显示需求推出”批量小核”的结构必然性，然后拆开一个 SM 看内部各部件的分工，接着把 GPU 与 CPU 的规格逐项对照，最后回答一个常见疑问：GPU 为什么放着单线程不快不管。

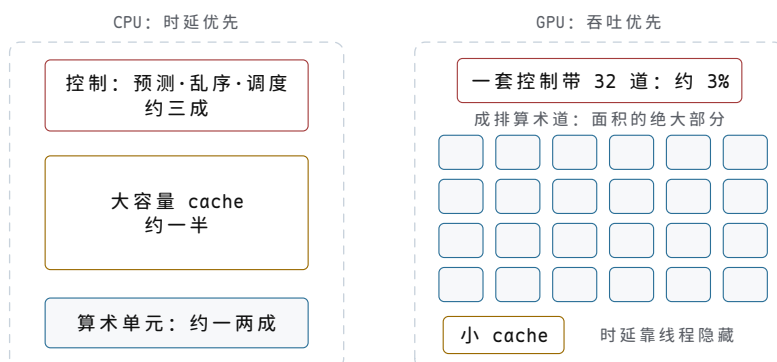
时延优先与吞吐优先的面积分配 芯片设计是一笔零和的账：功耗与面积的预算固定，多给控制逻辑一平方毫米，算术单元就少一排。两种处理器的分歧，从预算表的第一行就开始了。

CPU 的预算花在并不直接算数的地方。一个现代乱序核里，真正干活的执行单元——整数 ALU、浮点单元——只占核面积的一小部分，大约一两成；剩下的大部分给了三件事。分支预测器赌下一条指令在哪里，赌错了整条流水线清空重来；乱序调度维护几十到上百条在途指令的重排序缓冲、重命名寄存器与发射队列，只为让一条指令流不必等前面那条慢指令；大容量 cache 则把平均访存时延从几百拍压到几拍。这三件事有一个共同的客户：单线程。它们服务的不是”多算”，而是”这条指令流别停”。

GPU 把同一笔预算反过来花。一个 SM 里，取指、译码、发射只做一套，却同时驱动 32 条算术道：控制面积被 32 路摊薄，省下的预算再复制成几十上百个这样的 SM。分支预测很简陋，乱序完全没有，cache 也不大——在 GPU 的任务假设下，这些都是浪费：既然片上随时驻留着上万个互不依赖的线程，又何必为某一个线程的去向破费晶体管？

两种机器的超标量也值得一比。CPU 的超标量是一条指令流每拍发射多条——高端核做到 4 到 6 条——代价是每拍都要为候选指令独立检查相关性、争用端口；GPU 是一条指令带 32 份数据，相关性整组只检查一次。同样追求“一拍多条”，一个把复杂度花在“找”，一个花在“排”。设一套控制的面积与一条算术道相当，那么“一套控制带一条道”的机器，控制占去一半面积；“一套控制带 32 道”之后，控制占比降到 $1/(1+32) \approx 3\%$ ，同样的预算下算术道接近翻倍。这就是 GPU 的面积账，第二节会看到这笔账在执行时的形态。

把这笔预算账画成两块硅片的草图，两种押注一目了然：同一方硅片上预算是零和的，控制与缓存多占一分，算术道就少一排。



图示：同一笔面积预算的两种花法（比例为示意）。CPU 的大头给了控制与缓存：它们不直接算数，只为单条指令流不停顿；GPU 一套发射逻辑带 32 条算术道，控制占比摊薄到约 $1/(1+32) \approx 3\%$ ，省下的预算再复制成更多算术道。两种分法没有高下：一个押“这条指令流别停”，一个押“每秒总量最大”。

顺着这笔账问到底：既然要并行，为什么不把同样的预算堆成一百个乱序核？答案是汇率不划算。一个乱序核连同它的预测器、调度窗口与私有 cache，面积抵得上几十条算术道；同样一块硅片，堆乱序核得到一两百个在途线程，堆 SM 得到二十六万。并行度的汇率差了三个数量级，而 GPU 的工作假设——海量、同构、互不依赖——恰恰用不上乱序核买来的那些单线程能力。预算的每种花法，都是在赌自己的工作长成什么样；GPU 赌赢了图形，图形也塑造了 GPU。

预算去向	CPU（时延优先）	GPU（吞吐优先）
算术单元	核面积的小头，约一两成	面积的绝大部分，成排复制
控制逻辑	预测、乱序、重命名，占大头	一套发射带 32 道，能省则省
缓存	MB 到上百 MB，压低平均时延	每 SM 小容量，时延靠线程隐藏
优化对象	单个任务的完成时间	每秒完成的总工作量

要防止一个误读：这不是“先进与落后”的对照。GPU 核的简陋是刻意为之——在“工作可拆成海量小任务”的假设下，复杂的控制逻辑只增加功耗，不增加吞吐；反过来，把 GPU 的小核搬去跑通用程序，会因为单线程太慢而根本没法用。第十章把时延与吞吐拆成两个独立的量，这里看到的是这两个量在硅片上的形状：两种芯片各自把预算押给了其中一个。

这个分工也不是从来如此。1990 年代的“显卡”只是帧缓冲加固定功能管线，2000 年代可编程着色器让算术单元成片出现，再往后统一着色器架构把顶点与像素单元合并成通用的算术道，SIMT 的形态就此定型。芯片演化的每一步，都是把预算从固定功能挪给可编程算术——本节的面积账，在三十年里被反复执行。

P&H 讲 GPU 的那一节开门见山：GPU 是为吞吐优化的多处理器，它要回答的不是“一条指令多快”，而是“怎样让每秒数百亿次运算流过芯片”。本节后面

每一项结构差别，都是这句话的推论。

图形需求天然是海量并行 这种结构配得上什么工作？先看图形给出的数字。一帧 1080p 画面有 $1920 \times 1080 = 2,073,600$ 个像素，4K 画面有 $3840 \times 2160 = 8,294,400$ 个；按 60 帧每秒，GPU 每秒要着色的像素是 1.2 亿到 5 亿个。每个像素的着色——纹理采样、光照、混合——要几十到几百次运算，乘起来就是每秒 10^{10} 到 10^{12} 次运算的量级。顶点一侧同理：一个精细角色就有几万顶点，一屏上百个角色加场景，每帧顶点数以千万计，每个顶点做一次 4×4 矩阵变换约 32 次浮点运算，仅几何变换又是每秒 10^{10} 次上下的一笔。

比数量更重要的是形状：这些计算几乎互不依赖。第五十万个像素的颜色，不等第四十九万个算完；每个顶点的变换，不需要知道相邻顶点的结果。这种“海量、同构、互不依赖”的任务叫数据并行，口语里叫“令人愉快的并行”——愉快之处就在于并行度是白送的，不需要任何拆分技巧。实际负载只会更重：半透明叠加与过绘制让同一像素被重复着色，每帧的片元数往往是像素数的 2 到 4 倍，上面还是保守估计。

拿帧预算反推一次结构。60 帧每秒给每帧 16.7 ms。假设每像素只花 100 拍，把 1080p 一帧全部串行放在一条 2 GHz 的算术道上跑：

```
串行时间 = 2,073,600 像素 × 100 拍 / 2 GHz
           ≈ 104 ms, 是帧预算的 6 倍多
再乘两条：每像素实际远不止 100 拍，
           几何与光栅化还没算进去
结论：需要  $10^2$  到  $10^3$  条并行的算术道
```

帧预算还有第二层含义：平均值之外是稳定。固定节奏的影片只要帧率恒定就流畅，交互图形的 60 帧却怕抖动——某一帧多花了 5 ms，人立刻感到卡顿。这对结构的要求是：并行度不仅要够，还要为最坏的场景留余量，工程上常按峰值需求的一两倍配置算力。批量小核在这里又赢一次：加并行度远比提单线程速度便宜，余量因此给得起。

任务结构决定机器结构。工作是一百万个同构的小题时，配一百万道简装工序，远比配一个全能大师傅划算——大师傅再快，一次也只能做一道题。这就是“批量小核”的必然性：不是工程师偏爱核多，是需求的形状把结构逼成了这样。章首核心思想说 GPU 的秘密是“用海量在途工作隐藏一切时延”，那句话的前提就在这里：需求本身得先提供足够多的在途工作。

这段推理还给出一张可复用的判定单：一个工作适配 GPU 的三个条件——任务量以百万计、每份同构、份与份互不依赖。缺第一条，下发的开销收不回；缺第二条，分支分歧吃掉吞吐；缺第三条，同步吃掉一切。第五章讲 DMA 时说过它只擅长“整块、连续、大批”的搬运，这里看到的是同一哲学在计算上的版本：批量换效率。

GPU 的基本组成 一个 SM (streaming multiprocessor, AMD 阵营叫计算单元 CU) 的内部，是把第四章最小 CPU 的思路改了比例：部件还是那些部件，配比完全不同。最小 CPU 里一套控制带一条算术道；SM 里一套发射带 32 条道，这样的组再复制四份。

部件	规模量级 (每 SM)	职责
算术道	128 道 FP32, 分 4 组	执行数值运算，每组每拍消化一条 warp 指令
寄存器堆	65,536 个 32 位，共 256 KiB	全体驻留线程的上下文常驻其中
共享内存/L1 warp 调度器	几十到一百多 KiB 4 个	块内线程交换数据的近端存储 每拍各挑一组就绪 warp 发一条指令

寄存器堆值得单独讲，因为它是“换线程零代价”的物理基础。CPU 切换线程要把当前寄存器逐个保存到内存、再装入下一个线程的，一次切换几百拍起步，操作系统线程更是微秒级。GPU 的做法是让所有驻留线程的上下文永远留在寄存器堆里：每个 warp 的 32 个线程各占一段，“切换”只是下一拍换一组来执行，没有任何保存与恢复。代价就是那 $65,536 \times 4 \text{ B} = 256 \text{ KiB}$ ——比 SM 自己的 L1 还大。GPU 实际上把别的芯片给 cache 的预算，挪给了“驻留更多线程”；这与它的时延哲学一致：不去缩短一次访存的时延，而是让这段时延里永远有别的线程可算。调度器与寄存器堆怎么配合，第二节讲占用率时还要回来算细账。

和第四章的寄存器组对照最直观。最小 CPU 里 8 到 16 个寄存器就够用，因为一次只执行一条指令，状态随指令流动；SM 的 65,536 个寄存器按 32 线程一组切成 warp 上下文，全部静态摊在明面上。GPU 片上最大的单片存储不是 cache，是寄存器堆——它把“谁在现场”做成静态分配，而不是动态换人。

特殊功能单元与纹理单元是图形时代的遗产：正弦、开方、倒数这类超越函数用专用硬件算，比软件查表快得多；纹理单元自带插值与寻址模式。通用计算兴起后它们留了下来，只是换了服务的负载。这一节不展开，记住 SM 里不只有算术道即可。

总算力可以把数字直接乘出来。一张现代中高端卡有上百个 SM，每个 SM 每拍完成 128 个 FP32 乘加：

```
总算力 = SM 数 × 每拍乘加数 × 2 (乘加算两次运算) × 时钟
        = 128 × 128 × 2 × 1.5 GHz
        ≈ 49 TFLOPS
同代通用 CPU 约低一到两个数量级
```

这个数字单独看没有用处，配上带宽才完整——第十章 roofline 的拐点正是“峰值算力除以带宽”，带宽一侧第三节补齐，这里先记住算力的量级。

GPU 与 CPU 的规格对照 把两种芯片并排摆开，每一项差别都能回溯到前面的面积分配：

项目	桌面/服务器 CPU	现代独立显卡 GPU
在途线程数	核数乘超线程，16-128	上万，满配约 26 万
时钟	4-6 GHz	1.5-2.5 GHz
主存储带宽	DDR5 双通道约 77 GB/s	GDDR6 数百 GB/s，HBM 过 1 TB/s
cache 策略	大容量，缩短时延本身	小容量，靠线程数填满时延
单线程速度	基准	慢数倍
优化目标	单任务时延	全片总吞吐

逐项看。线程数：GPU 的在途数是 warp 数乘 32，按 $128 \times 64 \times 32 = 262,144$ 计，满配二十六万线程同时在片上；CPU 满打满算一百出头。注意这是“驻留”而不是“创建”：二十六万线程的上下文全在寄存器堆里，随时可以被调度器点名。

时钟：GPU 的算术阵列太宽，频率抬不上去。第八章讲电源时算过，动态功耗随频率与电压一起涨，频率抬一档、电压往往还要跟一档；把这笔账乘以上万条算术道，就知道 GPU 的时钟为什么停在 CPU 的一半上下。单条道因此更慢，但道数把差距赢回来几十倍。

带宽：DDR5 双通道按 $4800 \text{ MT/s} \times 8 \text{ B} \times 2 \approx 77 \text{ GB/s}$ ；GDDR6 用 384 位宽接口，按 $16 \text{ Gbps} \times 384 / 8 = 768 \text{ GB/s}$ ；HBM 把存储堆叠到处理器旁边，整卡过 1 TB/s。十倍的带宽差距不是锦上添花，是 GPU 结构的另一半——第三节专门讲这笔带宽从哪来、怎么花。

cache 策略的分歧最值得回味。CPU 用大 cache 把时延本身缩短：平均一次访存从几百拍变成几拍。GPU 的 cache 小，因为它不打算缩短时延，而打算用时间换：

访存的几百拍里，调度器换别的 warp 上道，时延被”填满”而不是被”缩短”。第十章说”用并行度买吞吐、买不来时延”，两种芯片把这句话各自实现了一遍，只是买的方向相反。

这张表的读法也因此明确：不要跨栏比较单核性能。两栏优化的本来就不是同一个量，跨栏比就像问”集装箱船和快艇哪个好”——答案取决于你要运什么。

为什么 GPU 不追求单线程快 把上面几行规格合起来，单线程慢是结构性的：时钟低一半，没有乱序和多发射，分支预测简陋，发射宽度还要被一个 warp 的 32 个线程共享。粗算差距：时钟差 2 到 3 倍；指令级并行再差数倍——乱序核一拍能退出多条，SIMT 单线程只能等整组共享的那一条发射。乘起来，同一段串行代码在 GPU 上慢 5 到 10 倍是正常区间。这个数字不必精确，记住方向就够：差距是结构性的，不随驱动或编译器消失。

这也不是缺陷，更不需要修复。GPU 的优化目标从来不是”这一个线程多久算完”，而是”每秒完成多少工作”。那能不能给 GPU 也装上乱序和大 cache？预算零和：装了就得砍算术道。乱序窗口换来的是单线程时延，砍掉的道失去的是总吞吐——对一块为吞吐而生的芯片，这是亏本交换。工程上没有”什么都要”的芯片，只有”押哪个量”的芯片。

也要补一句平衡：两种取向并非泾渭分明。近年的 GPU，cache 一代比一代大，线程调度一代比一代灵活；CPU 的向量单元也一代比一代宽。边界在移动，优先级没有变：GPU 添的每一项”聪明”都要先回答”它给吞吐带来多少”，CPU 添的每一点并行都要先回答”它拖不拖单线程”。看懂预算表，就看懂了这种互相借鉴却不互相变成的演化。

第十章的拆分在这里收尾：时延与吞吐是两个独立的量，改善一个不必改善另一个。GPU 把单线程时延整块让渡出去，换回二十六万在途线程的总吞吐。由此得到一个直接的工程推论：串行段、低并行度的代码应该留在 CPU 上，只有能拆出几万个任务的部分才值得下发；第十章的 Amdahl 定律同时提醒，串行段的长短决定整个混合系统的加速上限。CPU 与 GPU 的分工由此而来，本章第四节的命令路径，就是为这种分工设计的协作机制。

”GPU 核多，所以什么都能加速”是最常见的误读。拆不出几万个独立任务的工作上了 GPU 反而更慢：单线程本来就慢数倍，下发与回收本身还有一笔开销（第四节）。GPU 加速的前提是任务的形状，不是芯片的名气。

二、SIMD/SIMT 执行模型

第一节回答了”GPU 为什么长这样”，本节回答”这些算术道怎样被指令流驱动”。答案分两层：编程视角是一组标量线程各算各的，硬件视角是 32 条道共享一条指令流。本节先分清 SIMD 与 SIMT 这两个词，再看 warp 的调度怎样把访存时延藏起来——占用率是第一机制，然后算清两笔吞吐账：分支分歧与访存模式，最后把一个百万元素的向量加法从网格一路推演到算术道，作为全节的收束。

SIMD 与 SIMT 的区别 SIMD (single instruction, multiple data) 是一条指令同时操作多个数据：一条 AVX-512 加法取出两个 512 位向量寄存器，一拍完成 16 对 float 相加。并行度写在指令集里——向量的宽度是指令的一部分，从 SSE 的 128 位一路加到 512 位；程序员看到的是”一个线程，手里握着很宽的寄存器”，分支与循环仍是标量语义。SIMT (single instruction, multiple threads) 换了组织方式：程序写的是一组标量线程，每个线程有自己的寄存器、自己的数据、逻辑上自己的程序计数器；硬件把 32 个步调一致的线程绑成一组，共用一次取指与发射。并行度不写在指令里，写在程序结构里——线程号决定数据号。

维度	SIMD	SIMT
----	------	------

并行写在哪儿	指令里，宽度属于指令集	程序结构里，线程数属于启动配置
寄存器	一组宽向量寄存器	每线程一套标量寄存器
程序计数器	一个	逻辑上每线程一个，硬件每组一个
分支	向量内无分支，分支走标量路	组内分歧时各路轮流走（本节后文）
典型形态	SSE、AVX-512、NEON	CUDA/OpenCL 内核

两种模型能算同一件事，差别在“谁来暴露并行”。SIMD 要编译器或程序员把数据亲手装进向量，分支一多就装不满；SIMT 让程序员只管写一个线程的逻辑，宽度的利用交给硬件调度。后者在分支多、访存模式不规则的真实图形与计算代码里好写得多——这是 GPU 选 SIMT 的工程理由，不是理论上的必需。

GPU 的“线程”必须和操作系统线程分开称量。操作系统线程创建要分配栈与内核结构，10 到 100 微秒起步，切换一次也要微秒级；GPU 线程的“创建”只是调度器在寄存器堆里划出一段、指派一条算术道，切换是常驻换组，一拍完成，约合半纳秒。两端相差三到四个数量级，所以 GPU 程序一次启动敢铺几十万线程，服务器上的线程池却以几十计。第一节说寄存器堆是“换线程零代价”的物理基础，这里看到的是它的程序员视角：线程廉价到可以按数据量随手开，开线程这件事本身从预算表里消失了。

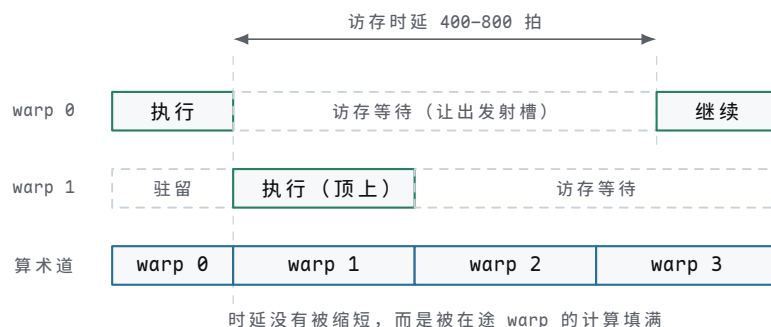
P&H 把 SIMT 称为硬件视角的 SIMD：对程序员它是一簇独立线程，对数据通路它就是向量。更早的血统是向量机——一条指令定义为对一整串数据的循环；GPU 相当于把这个循环摊开成 32 条并排的道。

warp/wavefront 的调度 SIMT 的执行单位叫 warp（NVIDIA，32 线程）或 wavefront（AMD，64 线程，新架构也可按 32 配置）。一个 warp 一个程序计数器：调度器选中它，就发射一条指令，组内 32 个线程的算术道同时执行各自的数据。取指、译码、发射只做一次，服务 32 个线程——第一节说的“控制面积摊薄”，指的就是这个结构。

关键规则是：warp 是调度的最小单位，组内共进退。只要有一个线程的操作数没准备好——典型情况是访存的数据还没回来——整组这条指令就不能发射，调度器把发射槽让给另一组就绪的 warp。实现上每个 warp 带着就绪标志，操作数齐了才置位；乱序核用重排序缓冲在一团指令里找就绪的那条，SM 用整组的就绪位在几十组 warp 里找就绪的那组——同一个问题，两种规模的答案。

让出的那几百拍是这笔买卖的核心。一次显存访问要 400 到 800 拍（1.5 GHz 下约合 270 到 530 纳秒，正是 GDDR 与 HBM 的真实量级），这段空窗不由发起访存的 warp 硬等，而由别的 warp 的计算填满。这就是 GPU 时延隐藏的第一机制：不缩短时延，用别人的工作盖住时延。一个 SM 四个调度器每拍最多发四条 warp 指令，即 128 个线程运算——第一节那 49 TFLOPS 就是这么一拍一拍垫出来的。

把这次让出与顶上画在时间轴上，“不缩短时延、用别人的工作盖住时延”就有了具体的形状。



图示：warp 调度怎样隐藏访存时延：warp 0 发起访存后整组让出发射槽，调度器立刻换上驻留待派的 warp 1（以及更多 warp），算术道一行始终有主；数百拍后数据返回，warp 0 重新就绪继续。空窗由别人的工作盖住——这就是“不缩短时延、填满时延”的时序形状。

这个思想本书已经见过一次。第十一章讲 NVMe 队列时算过：设备时延固定，吞吐靠在途请求数撑起来，是第十章 Little 定律的直接应用。GPU 把同一公式搬进处理器内部：访存时延固定，算术单元的利用率靠在途 warp 数撑起来。一个叫队列深度，一个叫驻留线程数，名字不同，机制是同一个。

顺带回答一个常见疑问：为什么不把 warp 定成 128 线程，把控制摊得更薄？因为分歧与合并访存的代价都随组变大：组越大，越难凑齐同方向的分支，越难凑出连续的一片地址，一次让出浪费的算术道也越多。32 是“摊薄控制”与“组内一致性”之间的平衡点；AMD 历史上用 64，新架构也退回可按 32 配置，原因相同。

占用率：在途线程数决定隐藏能力 调度器手里有多少 warp 可换，直接决定能盖住多长的时延。这个量的行话叫占用率（occupancy）：

占用率 = 本 SM 实际驻留的 warp 数 / 硬件上限
硬件上限的典型值是 64 warp/SM

需要多少才够，可以算。设一次访存时延 400 拍，每个线程平均算 100 拍就要发起下一次访存：

一个 warp 每 100 拍供给 100 拍工作，然后空等 400 拍
要算术道不停：至少需要 $400/100 = 4$ 组 warp 轮转
计入发射间隔与毛刺，工程上取 8 组以上才稳
占用率低于 $4/64 \approx 6\%$ 时，算术道大量空转

驻留上限本身由三样资源卡：寄存器堆、共享内存、warp 槽位，三者取最紧的那个。寄存器堆的账最直观：

每线程 40 个寄存器：一个 warp 占 $32 \times 40 = 1,280$ 个
 $65,536 / 1,280 = 51$ ，可驻留 51 组 warp
每线程 255 个寄存器：一个 warp 占 8,160 个
 $65,536 / 8,160 \approx 8$ ，只能驻留 8 组
占用率 $8/64 = 12.5\%$ ，400 拍的访存藏不干净

工程含义有两层。其一，内核少用寄存器不是洁癖，是在给时延隐藏能力腾地方：编译器报告每线程寄存器数时，报的其实是占用率的上限；为省寄存器把变量溢出到内存也是常见交换——多花一次访存，换更多在途 warp，只要换来的 warp 数盖得住新的访存就划算。其二，占用率不是唯一杠杆：每线程若能多排几条互相独立的指令（指令级并行），等待期间本组 warp 自己也有活干，4 组的下限就可以再低。第一节说“时延被填满而不是被缩短”，填满它的可以是更多的线程，也可以是每线程更多的独立指令——两者相乘，才是总隐藏能力。

共享内存卡上限时，账法相同。设 SM 给共享内存的配额是 100 KB，每个块申请 32 KB、含 8 组 warp：

$100 / 32 = 3$ ，只能驻留 3 块，共 24 组 warp
占用率 $24/64 = 37.5\%$

每块申请砍到 16 KB：驻留 6 块，共 48 组 warp

共享内存是块内线程的协作空间，用量越大单块越强，但驻留越少——又一次”线程便利”与”在途数量”的交换。它具体怎么用，第三节讲显存层次时展开。

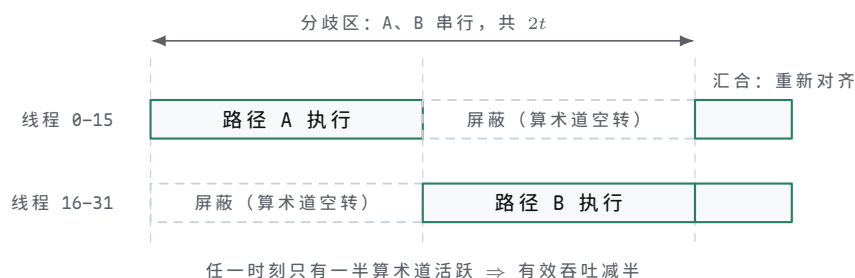
还要看清占用率与性能的关系不是线性的。从 4 组加到 16 组，算术道的利用率显著提升，因为 400 拍的空窗还没填满；从 48 组加到 64 组，空窗早已填满，多出来的 warp 只是排队。隐藏时延这件事有饱和点，调占用率的目标是”过拐点”，不是”拉满”——超过拐点之后，该看的就不是占用率，而是访存模式与分支，也就是接下来两笔账。

分支分歧的代价 warp 共享一个程序计数器，遇到分支怎么办？硬件的答案是：都走，轮流走。if-else 两边各有线程要走时，先执行 A 侧、把走 B 的线程屏蔽成不活跃，再执行 B 侧、屏蔽走 A 的，走完两边在汇合点重新对齐。被屏蔽线程的算术道空转，这段空转就是分歧的代价，行话叫分支分歧 (branch divergence)。

代价可以精确算。设 A、B 两侧各花 t 拍：

无分歧：32 线程同走一路，warp 花 t 拍，吞吐 $32/t$
 一半走 A 一半走 B：warp 花 $2t$ 拍，
 期间每侧只有 16 线程活跃
 有效吞吐 = $32/(2t) = 16/t$ ，直接减半

把这段过程画在时间轴上，哪一半线程在干活、哪一半在空等，一眼可见。



图示：分支分歧的时间账：warp 共享一个程序计数器，A、B 两侧只能轮流执行——走 A 时 B 侧 16 条线程被屏蔽，走 B 时 A 侧被屏蔽，汇合点之后才恢复整组同行。两段路径的耗时相加，期间算术道始终有一半在空转，有效吞吐因此减半。

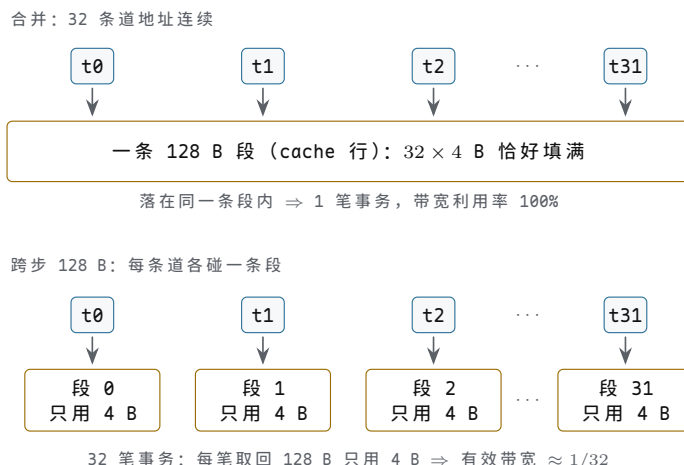
嵌套起来更狠：每多一层分歧，顺序执行的路径数最多翻一倍， d 层嵌套最坏 2^d 条路径挨个走；32 个线程完全散开时，warp 退化成 32 条单线程路径，吞吐只剩 $1/32$ 。较新的架构引入独立线程调度，允许组内更灵活地分裂与汇合，但”同组同方向最便宜”这条代价曲线没有变，只是毛刺少了。

工程含义与第五章 cache 友好遍历是同一类动作：按机器的偏好排数据。分支前把同方向的数据聚到同一个 warp（排序或分块），或者干脆把分支改写成无分支的掩码算术——两侧都算、按条件择一，编译器对短分支常自动做这个改写，长分支只能靠人。判断该不该改写的尺子就是上面那笔账：两侧都短时，掩码算术花 $2t$ 买无分歧；一侧明显更长时，先重排数据再分歧。

合并访存 warp 的一条 load 指令携带 32 个地址。内存子系统不逐地址处理，而是先看这 32 个地址落在几条 128 字节段（cache 行）里，每条段发一笔事务。最好的情况是连续线程访问连续地址：32 个线程各取 4 字节， $32 \times 4 = 128$ 字节恰好填满一条段，拼成一笔大事务，一次拿满，这叫合并访存 (coalesced access)。最坏的情况是跨步：每线程隔 32 个 float (128 字节) 取一个，32 个线程落在 32 条不同的段上：

连续：32 线程 $\times 4 \text{ B} = 128 \text{ B}$ ，一笔事务，利用率 100%
 跨步 128 B：32 笔事务，每笔只有 4 B 有用
 带宽利用率 = $4/128 = 3.125\% \approx 1/32$
 有效带宽 = $500 \text{ GB/s} \times 1/32 \approx 15.6 \text{ GB/s}$

把两种地址模式画成 warp 视角的对照图，”拼成一笔”与”散成 32 笔”的差别一眼可见（图中各画 4 条道代表 32 条）。



图示：合并访存与跨步访存的对照（各画 4 条道示意 32 条）。地址连续时，warp 的 32 个 4 B 请求恰好拼满一条 128 B 段，一笔事务人人有份；跨步 128 B 时，每条道落在不同的段上，32 笔事务每笔只取走 4 B 有用数据，有效带宽只剩 1/32。同一台显存，地址模式差出一个数量级还多。

15.6 GB/s，比 CPU 的主存还慢一截。三种典型模式并排看更清楚：

warp 内地址模式	每指令事务数	有效带宽 (500 GB/s 峰值)
连续，32 线程 × 4 B	1 笔 128 B	约 500 GB/s，全额
跨步 16 B	4 笔	约 125 GB/s，四分之一
跨步 128 B	32 笔	约 15.6 GB/s，三十二分之一

这就是 GPU 版的空间局部性。第五章讲 cache 行与突发传输时立过同一条规矩：按行取回，就用满整行。GPU 把这条规矩放大到 warp 粒度，惩罚也放大到 32 倍——因为它更靠带宽吃饭。工程推论随手可得：结构体数组 (AoS) 按字段访问天然跨步，改成数组结构 (SoA) 让同名字段连续存放，往往一处改动换回一个数量级的有效带宽；写操作同理会并，读写混合的核心里，每个 warp 的地址模式都要过一遍这杆秤。第三节算显存总账时，还会回到这条规矩。

推演：一次向量加法在 SIMT 下怎样跑 把本节全部机制装进一个最小例子走一遍： $N = 2^{20} = 1,048,576$ 个 float 的向量加法 $c[i] = a[i] + b[i]$ 。内核本身只有一行有效运算：每个线程读 $a[i]$ 、读 $b[i]$ 、相加、写 $c[i]$ 。启动配置取每块 256 线程，逐层映射到硬件。

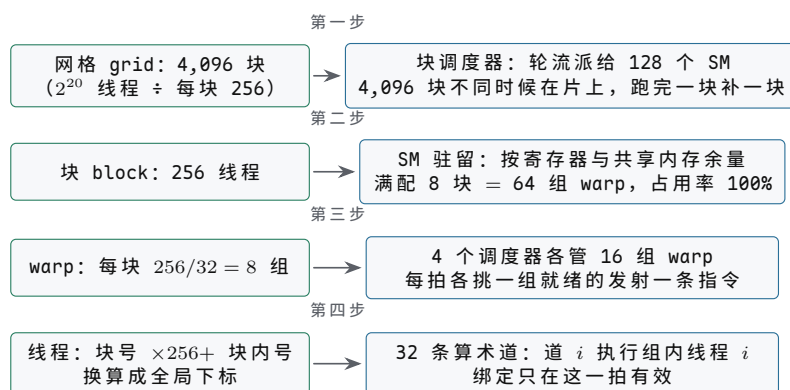
第一步，网格与块。总块数 $1,048,576 / 256 = 4,096$ ，每块 $256 / 32 = 8$ 组 warp。网格只描述工作量，不指定在哪跑； N 在这里恰好整除，不能整除时最后一组 warp 靠掩码收尾，与第二节分歧是同一套机制。

第二步，块到 SM。块调度器把 4,096 块轮流派给 128 个 SM；每个 SM 按寄存器与共享内存的余量决定驻留几块，假设满配 8 块，正好 $8 \times 8 = 64$ 组 warp，占用率 100%。4,096 块不会同时在片上，跑完一块补一块——这个队列行为和第十一章的命令队列是同一张脸。

第三步，warp 到调度槽。SM 内 4 个调度器各管 16 组 warp，每拍各挑一组就绪的发射一条指令。驻留的 64 组里，大多数在等访存回来，少数就绪——400 拍的空窗由它们轮转填满，这正是占用率演算里”4 组起步、8 组才稳”的现场。

第四步，线程到算术道。一条 warp 指令发射时，道 i 执行组内第 i 个线程；线程号经 块号 × 256 + 块内号 换算成全局下标，决定读写哪个元素。道与线程的绑定只在那一拍有效，下一拍发射槽换上另一组 warp 的 32 个线程。

四步映射画成一张对照图：左边是启动配置描述的软件层级，右边是承接它的硬件——



图示：一次向量加法启动的四级映射 ($N = 2^{20}$ 、每块 256 线程)。网格只描述工作量：4,096 块由块调度器轮流派给 128 个 SM；每个 SM 按资源余量驻留若干块，满配 8 块合 64 组 warp；SM 内 4 个调度器分管这些 warp，每拍各挑一组就绪的发射；一条 warp 指令发射时，道 i 执行组内第 i 个线程，道与线程的绑定只在这一拍有效。

第五步，访存合并。每个 warp 读 $a[]$ 的 32 个下标天然连续， $32 \times 4 = 128$ 字节一笔事务，上一节的结论直接兑现； $b[]$ 与写回 $c[]$ 同理。若下标换算里掺了跨步，带宽立刻按上一节的表折扣。

最后算两笔账，看这件事总共花多少时间。指令账：

每线程 2 load + 1 add + 1 store，共 4 类指令
warp 指令总数 = $1,048,576 / 32 \times 4 = 131,072$ 条
全卡发射能力 = $128 \text{ SM} \times 4 \text{ 调度器} = 512$ 条/拍
发完指令流 $\approx 131,072 / 512 = 256$ 拍 ≈ 0.17 微秒

数据账：

数据量 = 读 8 MiB + 写 4 MiB = 12 MiB ≈ 12.6 MB
1 TB/s 带宽下：12.6 MB / 1 TB/s ≈ 12.6 微秒

0.17 微秒对 12.6 微秒，指令发射比数据到达快了近两个数量级——这是一个纯带宽题。用第十章 roofline 的话说：算术强度是 $1/12 \approx 0.08$ 次运算每字节，机器拐点是 $49 \text{ TFLOPS} / 1 \text{ TB/s} \approx 49$ 次每字节， $0.08 \ll 49$ ，深深位于带宽区。这类内核的优化方向因此只有一个：访存模式与有效带宽，而不是算术。与第十一章 NVMe 一次读的 trace 对照着看更有意思：两边都是“队列下发、硬件自取、完成通知”的事务，区别在规模——SM 内部这十几微秒里的几百次 warp 切换对 CPU 完全不可见，CPU 只在 fence 处看到“完成”二字（第四节展开）。

收束全节：拿到一个 SIMT 内核，先问三件事——占用率够不够盖住访存时延，访存合不合并，分支分歧多不多。这三问对应本节的三笔账，也是一切 GPU 性能分析的入口。

三、显存系统：VRAM、HBM 与数据进出

显存是 GPU 版图里最显眼的分界线。CPU 的主存是所有设备共享的主场，GPU 却另立一座仓库：数据先要搬进显存，才能被上万个线程高速消费。本节回答四个问题：显存里到底装着什么；GDDR 与 HBM 动辄数百 GB/s 的带宽从哪里来；数据跨 PCIe 进出 GPU 的代价有多大；片上两级近端存储怎样把这笔代价再省回来。最后一个演算把全部数字收进第十章的 roofline：很多 GPU 程序的快慢，从头到尾不由算力决定。

VRAM 身兼三职 VRAM（显存）的第一职是帧缓冲：屏幕每个像素的颜色值躺在显存一块连续区域里，等显示控制器按固定节拍逐行取走——那是第四节的题目。一块 1920×1080 、每像素 4 字节的帧缓冲约 8.3 MB，双缓冲再翻一倍，对今天的显存容量只是零头。第二职是图形资源的仓库：纹理贴图、顶点缓冲、着色器编译后的指令。这一职的访问强度远大于帧缓冲——同一小片纹理在一帧里被成千上万个像素反复采样，命中频次以亿计。第三职是通用计算的数据区：CUDA 程序里的数组、矩

阵、中间结果都分配在显存里，内核启动后线程直接读写，计算密集时每个字节每毫秒都会被访问好几次。

用途	装的什么	访问特征
帧缓冲	待显示的像素颜色	显示控制器逐行顺序读，渲染引擎随机写
纹理/资源	贴图、顶点、着色器指令	读多写少，同一片数据被大量像素重复读
通用数据	数组、矩阵、中间结果	由内核决定，读写都可能极密集

三职共享同一块显存，也共享同一份带宽：显示扫描每帧固定取走约 0.5 GB/s（第四节会算），只是零头；纹理采样与通用计算才是真正的带宽大户，两者靠片上 cache 与调度错开峰值。显存容量也跟着三职一路膨胀：早期显卡几百 MB 就够放帧缓冲与纹理，今天一张计算卡配几十 GB，装的是模型参数与中间张量——第三职成了容量的主要推手。

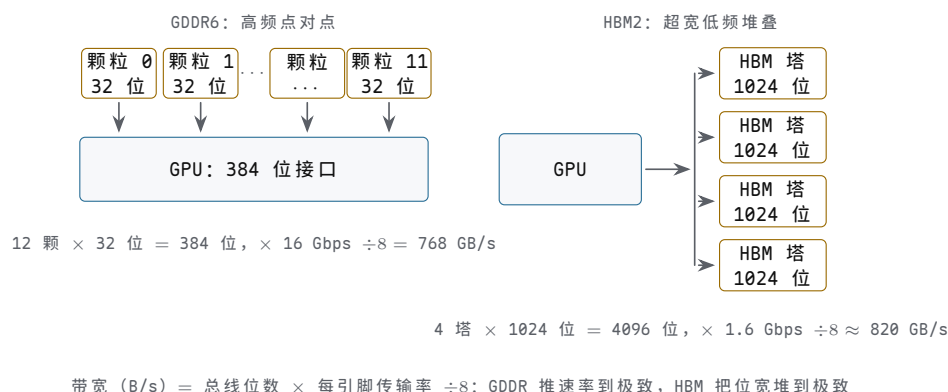
三职的访问局部性也各不相同，这解释了 GPU 存储设计的取舍。帧缓冲按行顺序被扫、按块随机被写，局部性最好；纹理采样有强烈的二维邻近性——相邻像素采样相邻纹素——所以 GPU 给纹理配了专用 cache 与专门的二维分块布局，把线性地址打乱成小块，让一次采样带回来的整行数据下一拍还能用；通用计算的局部性完全由程序员决定，写得好时比纹理还整齐，写得差时全是指针追逐，硬件帮不上忙。同一块显存上三种访问模式并存，是 GPU 存储子系统比 CPU 更难设计的第一个原因；第二个原因前面已经说过——它还要大一个数量级的带宽。

三职合在一处，”显存不够怎么办”就成了必答题。操作系统与驱动给的答案是换页：把暂时不用的部分挪回主存，用到时再搬回来，程序逻辑上不必修改。能跑，但性能是断崖。量级一摆就清楚：本节后面要算的 HBM 显存带宽约 820 GB/s，跨 PCIe 回主存只有约 30 GB/s，差 25 倍以上；时延同样差一个数量级，显存内部一次访问约几百纳秒，跨总线到主存以微秒计。更麻烦的是触发方式：换页由缺页异常驱动，内核运行到一半发现数据不在显存，整个网格停下来等搬运，一次缺页就把十万线程并行攒下的吞吐优势赔进去。工程结论因此很硬：显存容量是 GPU 程序的第一约束；放不下的数据集必须先分块，让每一块在显存里被反复用够再换下一块，而不是指望换页兜底。第四节讲命令路径时还会看到，连”什么时候搬”都是显式安排的，而不是等异常发生。

GDDR 与 HBM 的带宽来源 显存带宽不是某项聪明设计的产物，而是两个朴素物理量的乘积：带宽 (B/s) = 总线位数 × 每引脚传输率 ÷ 8。GDDR 与 HBM 的分歧，在于各自把乘积中的哪一个因子推到极致。GDDR 走”高频点对点”：每颗显存颗粒以 32 位接口直连 GPU，走线短、负载轻，单引脚跑到 16–24 Gbps——GDDR6 约 16 Gbps，GDDR6X 改用四电平 PAM4 编码，用两个比特压进一次电平跳变，单引脚推到 24 Gbps。十二颗颗粒围 GPU 一圈，拼出 384 位接口。这条路线的天花板在封装边缘：384 位意味着芯片四周引出近四百组数据线加配套的电源与地，位宽再想翻倍，封装周长不够用了；20 Gbps 以上的信号完整性也让每一代提速都更艰难。HBM 走另一条路：”超宽低频堆叠”。DRAM 芯片垂直堆成一塔，用硅穿孔 (TSV) 上下连通，整塔与 GPU 并排安放在同一块硅中介层上；单塔接口宽达 1024 位——是 GDDR 单颗的 32 倍——每引脚反而只要 1–2 Gbps。宽度是堆出来的：一塔 1024 位，四塔并排就是 4096 位。

```
GDDR6: 384 位 × 16 Gbps/引脚 / 8 = 768 GB/s
GDDR6X: 384 位 × 24 Gbps/引脚 / 8 = 1152 GB/s
HBM2: 1024 位/塔 × 4 塔 = 4096 位
      4096 位 × 1.6 Gbps/引脚 / 8 = 819.2 GB/s, 约 820 GB/s
对照: 双通道 DDR5-6400 主存 = 128 位 × 6.4 Gbps / 8 ≈ 102 GB/s
```

两条路线的结构并排画出来，”哪个因子被推到极致”一眼可辨：



图示: GDDR 与 HBM 求带宽的两条路线 (各画部分颗粒与塔示意)。GDDR 走高频点对点: 每颗颗粒 32 位直连 GPU, 单引脚 16 Gbps, 十二颗拼出 384 位、768 GB/s, 位宽再涨受封装周长限制。HBM 走超宽低频: DRAM 用硅穿孔堆成塔、与 GPU 同放一块硅中介层, 单塔 1024 位、每引脚只要 1.6 Gbps, 四塔 4096 位约 820 GB/s, 宽度是用堆叠与封装成本换的。同一道公式, 一个把速率推到极致, 一个把位宽堆到极致。

路线	每引脚速率	总位宽	带宽的物理出处
GDDR6/6X	16–24 Gbps	256–384 位	封装边缘高频点对点, 位宽受引脚数限制
HBM2	1–2 Gbps	1024 位/塔, 可堆 4–8 塔	硅中介层超宽接口, 宽度用堆叠换
DDR 主存	3–6.4 Gbps	64 位/通道	双通道 128 位, 带宽约 100 GB/s

三组数字并排, 结论一目了然: HBM2 的 820 GB/s 约是双通道 DDR5 主存的 8 倍, 而这 8 倍没有任何魔法—位宽之比 $4096/128 = 32$, 引脚速率之比约 $1/4$, 两者相乘正好是 8。带宽是物理堆出来的: 要宽度, 就得出面积、出硅中介层、出封装成本; 要速率, 就得在信号完整性上持续投入, 还要承受高频带来的功耗。HBM 每引脚速率低附带一个红利: 速率低则每比特搬运的能耗低, 同样带宽下 HBM 的显存功耗明显小于 GDDR—这正是功耗顶格的计算卡偏爱 HBM、成本敏感的消费卡坚持用 GDDR 的原因。选哪条路线是预算题, 不是高下题。HBM 的短板也要说全: 硅中介层与堆叠封装把成本抬高一截, 单塔容量受堆叠层数封顶, 所以 HBM 卡贵且容量档位少; GDDR 颗粒便宜、扩容容易, 代价是带宽与能效都停在更低的档位上。

两条路线各自的演进也沿着同一道公式走。HBM 从 HBM2 的 1.6–2 Gbps 每引脚提到 HBM3 的 6.4 Gbps, 位宽一塔没变, 带宽直接增到四倍, 单塔从约 205 GB/s 涨到 819 GB/s—速率这条轴上它一直在补课。GDDR 守不住位宽就守速率: GDDR6X 引入 PAM4 之后, 再往下走要靠更激进的编码与更短更平的走线, 每代提速换来的 Gbps 越来越少。把两家的路线图叠在一起看, 结论还是那个: 位宽必须用面积与封装换, 速率必须向信号完整性换, 带宽没有第三种来源。

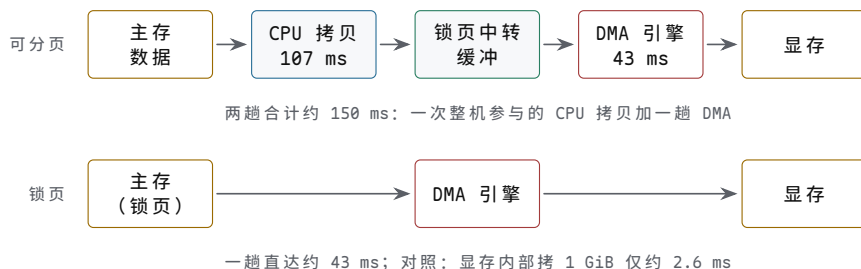
数据进出: PCIe DMA 与 pinned memory 独立显存的另一面, 是数据得先跨过 PCIe 才能进 GPU。搬运不靠 CPU 逐字拷贝—那会把一个核完全占死—而是由 GPU 板上的 DMA 引擎完成: 驱动把源地址、目的地址、长度写成描述符, 引擎自己读主存、写显存, 完成后写状态、发中断。这正是第五章描述符 DMA 的那张图, 只是执行体从磁盘控制器换成了 GPU。瓶颈在链路本身: PCIe 4.0 x16 每通道 16 GT/s, 共 16 通道, 128b/130b 编码, 理论带宽 $16 \times 16 \times 128/130 \div 8 \approx 31.5$ GB/s, 扣除 TLP 包开销实得约 25–30 GB/s; 上一代的 3.0 x16 则只有约 16 GB/s。对照上一题的 820 GB/s, 进出 GPU 的路比显存内部窄了约 30 倍—显存系统真正的窄门不在显存, 在总线。

有一个使用细节必须先交代: 普通可分页内存不能被 DMA 直接使用, 它的物理页随时可能被操作系统换走, DMA 搬到一半页面易主, 就是数据损坏。所以可分页数据要走两趟: CPU 先把它拷进一块锁页的中转缓冲, 再由 DMA 从缓冲搬走。锁页

内存 (pinned memory, 页被锁定不许换出) 则省掉中转, DMA 一步直达。搬 1 GiB 数据, 两条路径各花多少时间, 可以直接算:

可分页路径: CPU 拷 1 GiB 到中转缓冲 (单流约 10 GB/s, 约 107 ms)
 + DMA 搬 1 GiB (约 25 GB/s, 约 43 ms) ≈ 150 ms
 锁页路径: DMA 一步直达, $1 \text{ GiB} / 25 \text{ GB/s} \approx 43$ ms
 对照: 显存内部拷 1 GiB = 读 1 GiB + 写 1 GiB
 $2.15 \text{ GB} / 820 \text{ GB/s} \approx 2.6$ ms

两条路径并排画出来, 150 ms 与 43 ms 的差距就是两趟与一趟的差距:



图示: 搬 1 GiB 进显存的两条路径。可分页内存的物理页随时可能被换出, DMA 不能直接使用, 必须先由 CPU 拷进锁页中转缓冲 (约 107 ms) 再交给 DMA (约 43 ms), 两趟合计约 150 ms; 锁页内存省掉中转, DMA 一步直达约 43 ms。对照线: 同样的 1 GiB 在显存内部拷贝只需约 2.6 ms—跨总线比显存内部贵一个数量级以上。

三组数字的工程含义一层比一层硬。第一层: 跨总线搬运比显存内部搬运贵一个数量级以上, 43 ms 在 60 Hz 下接近三帧—渲染循环若每帧都要从主存拉 1 GiB 新数据, 帧率上限立刻被锁在 20 fps 出头, 与算力完全无关。第二层: 锁页把两趟并成一趟, 省掉的是一次整机参与的 CPU 拷贝, 所以大流量传输的第一条规矩就是缓冲一律锁页。第三层: 锁页的页退出主存调度, 锁得太多会拖垮整机的内存子系统, 惯例是开几块锁页缓冲循环复用, 而不是把全部数据区都锁死。还有第四层: 小拷贝比大拷贝更糟, 每次 DMA 提交有微秒级的固定开销, 若按 4 KB 一批拆着传 1 GiB, 光固定开销就叠到一秒量级—传输必须攒大块, 这与第十章“批量摊薄固定成本”是同一条算术。在此之上再叠一层调度: DMA 引擎与计算引擎各自独立, 拷贝队列与计算队列并行推进, 第 k 块数据在总线上搬运时第 $k-1$ 块正在算, 分块加异步, 把 30 倍差距藏进流水。

链路本身也在演进, 但节奏值得注意: PCIe 5.0 x16 把每通道速率翻倍到 32 GT/s, 理论带宽约 63 GB/s, 6.0 再翻一倍—每代翻一倍, 听起来很快; 可与显存侧对比, HBM 单塔约六年增到四倍, 总线始终在追赶, 而且越追相对差距越大。所以“数据进了显存就别出来”这条纪律, 每一代硬件都在加强而不是放松。工程上的对应做法也越来越统一: 训练框架把数据集常驻显存, 推理框架把权重锁在显存里, 跨卡交换走 GPU 之间的直连链路而不是绕回主存—都是同一个动机: PCIe 的 30 倍差距, 绕开一次是一次。

最后一层便利来自统一虚拟寻址 (UVA, unified virtual addressing): 主存锁页区与显存被映射进同一张 64 位虚拟地址空间, 指针的数值本身就标明了数据在哪一端, 驱动按地址区间路由访问。程序员的实惠是拷贝函数不必分别声明“主存指针”“显存指针”, 同一个指针两端通用; 硬件的实惠是地址翻译表只有一套。但务必记住, UVA 统一的是编址, 不是速度: 指针两端通用, 带宽两端照旧差 30 倍。

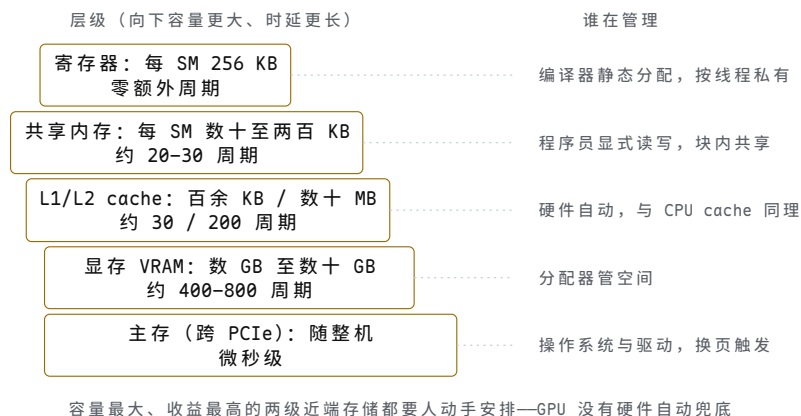
把整道题收拢成操作清单, 只有三条。其一, 算得久的数据一次传入、常驻显存, 传输成本按使用次数摊薄; 其二, 必须频繁进出的数据走锁页缓冲加异步 DMA, 与计算重叠, 把总线时间藏进显存时间里; 其三, 放不下的数据按块调度, 块的大小至少要让“算这块的时间”显著大于“搬这块的时间”—搬算比不足 1:10 的分块方案, 等于花钱买总线。这三条与算力无关, 却决定了多数真实程序的实际速度。

共享内存与寄存器: GPU 的两级近端存储 显存带宽再大, 离算术单元仍嫌远: 一次显存往返约四五百个时钟周期, 操作数若每个都现取现用, 再宽的带宽也被时延拖空。GPU 的答案是把两级近端存储摆得极近、给得极多。第一级是寄存器堆: 每个

SM 配 64K 个 32 位寄存器、共 256 KB，按线程静态瓜分——驻留 2048 个线程时，每线程分到 32 个；驻留 1024 个时，每线程 64 个。寄存器由编译器直接编进指令，访问不占额外周期，是真正意义上的零等待。第二级是共享内存（shared memory）：每个 SM 数十到两百 KB 的片上 SRAM，由同一线程块内的线程共同寻址，时延约二三十个周期，介于寄存器与显存之间。与这两级形成对照的是 L1/L2 cache：它们存在、也有用，但由硬件自动管理，容量与命中率都不由程序员直接安排。

层级	容量量级	时延量级	谁在管理
寄存器	每 SM 256 KB	零额外周期	编译器静态分配，按线程私有
共享内存	每 SM 数十至两百 KB	约 20–30 周期	程序员显式读写，块内共享
L1/L2 cache	每 SM 百余 KB / 整卡数十 MB	约 30 / 200 周期	硬件自动，与 CPU cache 同理
显存 VRAM	数 GB 至数十 GB	约 400–800 周期	分配器管空间，带宽见前文
主存（跨 PCIe）	随整机	微秒级	操作系统与驱动，换页触发

五个层级连同管理者排成一列，GPU 与 CPU 的差别全在右列：



图示：GPU 存储层级与各级管理者。寄存器与共享内存这两级近端存储由编译器与程序员显式安排：寄存器按线程静态瓜分，多用一个变量就可能挤掉一整组驻留线程；共享内存由程序员写代码搬运与复用。L1/L2 仍由硬件自动管理，再往外是显存与跨 PCIe 的主存。与 CPU 全由硬件 cache 自动平滑不同，GPU 把近端两级的控制权交给给人——做对时效率更高，做错时没有兜底。

与 CPU 的存储层次对照，最刺眼的差别在“谁在管理”那一列。CPU 的 cache 对程序员透明，写得规整的循环自然被硬件照顾；GPU 容量最大、收益最高的两级近端存储，都要人动手安排，而且各有代价。寄存器按线程配额：内核的变量一多就超额，编译器把超额部分溢出到显存，性能无声下滑；寄存器用量还反过来决定驻留线程数——每线程多用一倍寄存器，时延隐藏的兵力就少一半，这笔账第二节谈占用率时已算过。共享内存更彻底：矩阵乘法先由程序员写代码把子矩阵从显存搬进共享内存，块内线程围着这块数据复用几十上百次，再换下一块；搬多少、何时搬、谁读哪一块，全是显式的。可以说 GPU 把 cache 管理从硬件手里拿出来交给了程序员：控制权更大，做对时效率更高，做错时没有硬件自动兜底。这正是 GPU 优化文献里“分块（tiling）”出现的频率远高于任何指令技巧的原因——下一题的演算会给出它的定量回报。

给一组数字感受配额的刚性。一个 SM 的 64K 个寄存器，驻留 2048 线程、每线程 32 个寄存器时恰好用满： $2048 \times 32 = 65536$ ，一个不多一个不少。内核若被编译出 33 个寄存器，驻留就掉到 $65536/33 \approx 1986$ ，向下取整到调度粒度后实际少了一整个线程组——一个变量之差，占用率掉了一档。共享内存同样刚性：每块申请 48 KB、SM 上共有 100 KB 时，同驻的块数就是两块，第三块再怎么小也挤不进来。这些取

舍在 CPU 世界里由硬件 cache 的替换策略自动平滑，在 GPU 世界里则全部以整数台阶的形式出现在程序员面前。

带宽预算演算：向量操作为什么是带宽瓶颈 现在把本节全部数字收进第十章的 roofline。判定式只有一条：算术强度 I （每字节流量配多少 FLOP）与拐点 $I^* = \text{峰值算力} \div \text{带宽}$ 比较，小于拐点即带宽瓶颈。取一张有代表性的计算卡：峰值 fp32 算力 30 TFLOP/s，HBM 带宽 820 GB/s，拐点 $I^* = 30000/820 \approx 36.6$ FLOP/B。再看两个 kernel。向量加法 $C[i]=A[i]+B[i]$ ：每元素读 4 B、再读 4 B、写回 4 B，共 12 字节流量，只做 1 次加法， $I = 1/12 \approx 0.08$ FLOP/B；哪怕按最宽松的一字节数据计，每次运算配读、读、写三次访问，也只有 3 字节流量、 $I \approx 0.33$ ，离拐点仍差两个数量级。矩阵乘法 $C=A*B$ ： $n \times n$ 时运算 $2n^3$ 次、流量最少 $12n^2$ 字节， $I \approx n/6$ ； $n = 1024$ 时约 170 FLOP/B，远在拐点右侧。

拐点 $I^* = 30 \text{ TFLOP/s} / 820 \text{ GB/s} \approx 36.6 \text{ FLOP/B}$

向量加(fp32): $I = 1/12 \approx 0.08 \text{ FLOP/B}$

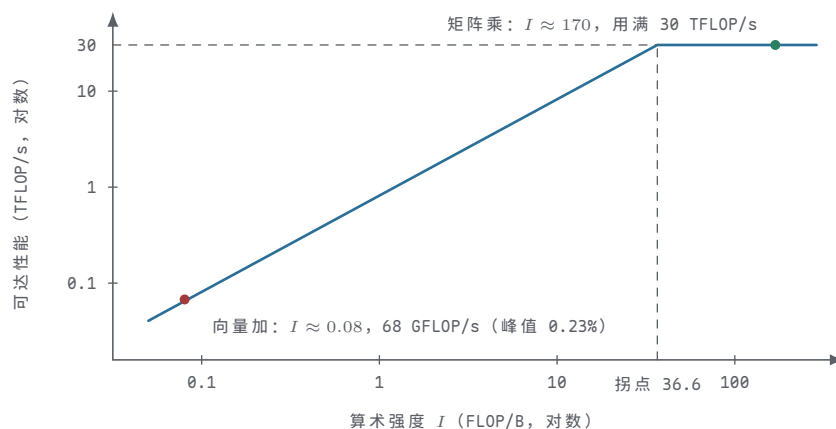
可达性能 = $820 \text{ GB/s} \times 1/12 \approx 68 \text{ GFLOP/s}$ ，约为峰值的 0.23%

矩阵乘($n=1024$): $I = 2n^3 / 12n^2 = n/6 \approx 170 \text{ FLOP/B}$

带宽允许 $820 \times 170 \approx 140 \text{ TFLOP/s} > \text{峰值 } 30$ ，算力瓶颈，峰值用满

kernel	算术强度	可达性能	瓶颈位置
向量加法	约 0.08 FLOP/B	约 68 GFLOP/s	带宽，峰值用不到 1%
矩阵乘法	约 170 FLOP/B	30 TFLOP/s	算力，峰值可以真正用满

把拐点与两个 kernel 的落点画在同一张图上：



图示：双对数坐标下的 roofline：斜段是带宽屋顶（可达性能 = 带宽 × 算术强度），平段是算力屋顶 30 TFLOP/s，拐点 $I^* = 30000/820 \approx 36.6$ FLOP/B。向量加的强度只有约 0.08 FLOP/B，被带宽屋顶压在 68 GFLOP/s—峰值的 0.23%；矩阵乘约 170 FLOP/B，远在拐点右侧，峰值真正用满。横轴每往左一格，同样的算力折扣再重一倍。

“向量操作是带宽瓶颈”至此有了定量说法：不是向量单元不够快，而是数据供给永远跟不上——一次加法要喂 12 个字节，喂得再快的显存也只能让算术单元空等。工程推论有三条。其一，逐元素的简单操作（向量加、缩放、激活函数）在 GPU 上天生用不满算力，优化方向是减少趟数：把连续几个逐元素操作融合成一个内核，中间结果留在寄存器里不写回显存，强度立刻按省下的趟数翻倍。其二，只有高强度操作（矩阵乘、卷积）值得为算力付费，AI 加速硬件围绕矩阵乘法展开，正是这条算术的产业化。其三，拐点随机器移动：换 HBM3、带宽 3 TB/s 的新卡，拐点降到约 10 FLOP/B，向量加法依旧深在左侧，一分算力红利也拿不到——第十章那句“左侧 kernel 为带宽付费”，在 GPU 上比在任何平台都极端。读规格书时也该养成对应的习惯：看到一张新卡，先查带宽与峰值算力算出拐点，再问自己手上的 kernel 强度多少，两个数就能判断这张卡对你有没有用。

把推论一演算到底，收益是肉眼可见的。两步连续的逐元素操作，比如先算 $D[i] = A[i] + B[i]$ 再算 $E[i] = D[i] * 2$ ，分两趟跑：流量是 $5 \times 4 = 20$ 字节（读 A、读 B、写 D、读 D、写 E），做 2 次运算， $I = 0.1$ FLOP/B。融合成一个内核后，D 留在寄存器里不落显存，流量只剩 $3 \times 4 = 12$ 字节， $I = 2/12 \approx 0.167$ ——强度提高 67%，可达性能同步提高 67%，而算术单元一行代码都没改。深度学习框架里的“算子融合”做的正是这件事：把激活、归一化、逐元素变换统统并进前一个高强度内核的尾巴上，让每一次显存读写服务的运算尽量多。在 roofline 左侧，省字节永远比加算力值钱——这是显存这一节最值得带走的一句话。

四、命令路径：命令环、fence 与显示扫描

GPU 与 CPU 的协作不靠函数调用，而靠一条与第十一章 NVMe 神似的命令路径：驱动把命令写进队列、写一下门铃，GPU 自己取命令、自己执行、自己报告完成。显示一侧另有一套不受渲染指挥的固定节拍。本节沿这条路径走一遍：命令环与 doorbell、fence 的完成语义、CPU 与 GPU 互不空等的三种同步模式，以及显示扫描的时序、撕裂与垂直同步。读完会发现 GPU 并不神秘：它就是第五章那台“执行描述符的设备”，只是描述符更肥、队列更深。

命令环缓冲与 doorbell 驱动提交一批 GPU 工作的最小单元是命令包：几十到几百字节，头部是操作码与长度，后面跟着参数——“把寄存器 X 设为 Y”“把显存 A 区拷到 B 区”“用这组参数启动一个内核”“绘制这组三角形”。命令包不直接写给 GPU，而是按序写进主存里的一块环缓冲（ring buffer）；写完一批，驱动做一次 MMIO 写，把新的尾指针送进 GPU 前端的 doorbell 寄存器。这一下就是全部通知：doorbell 不携带数据，只宣布“环上多了东西”。GPU 一侧，命令处理器比较头尾指针，发现差距后经 DMA 把新命令包取回片上，逐包解码、分派给对应引擎——渲染、拷贝、计算各有引擎，也各有各的环；每消化一段，它把读指针写回主存，驱动据此回收环上的槽位，环满时驱动只能停下来等 GPU 消费。

这条提交—执行—回收的环路画出来，与第五章、第十一章是同一副骨架：



图示：GPU 命令环的提交环路。驱动把命令包按序写进主存环缓冲（■），屏障之后一次 MMIO 写把新尾指针送进 doorbell（■）——doorbell 只宣布“环上多了东西”，不携带数据；GPU 前端比较头尾指针，经 DMA 把新命令取回片上解码、分派给渲染/拷贝/计算引擎（■）；每消化一段把读指针写回主存，驱动据此回收槽位（■）。环不动、指针动，通知可合并、丢失可自愈，与 NVMe 提交队列是同一副骨架。

把这段与第五章的描述符环并排，是同一张图的第三次出现：环不动、指针动；doorbell 只通知、不携带数据；设备按头尾差补读，偶尔错过一次通知也无妨。与第十一章 NVMe 提交队列的差别在细节而不在结构：NVMe 队列项定长 64 字节，GPU 命令包变长；NVMe 常按 CPU 核绑多对队列，GPU 按上下文与引擎分环。GPU 的特殊处只是规模：一批命令包可以上千，覆盖一帧的全部绘制，而 NVMe 一条命令就是一次读写。所以第十一章“先写命令项、再写 doorbell，中间隔一道屏障”的次序约定，在 GPU 这里逐字成立，理由也相同：通知必须在被通知的内容可见之后发出。

为什么要环加门铃，而不是每条命令就叫 GPU 一次？算一笔时延账：一次 MMIO 写加 GPU 响应的往返是微秒级，而一条绘图命令的执行可能只有几百纳秒——逐条握手等于让 GPU 每工作一份就空等十份。环把握手摊薄：驱动攒够一批再写一次门，门的成本被整批分摊；GPU 流水取命令，取下一批与执行当前批重叠。工程

推论：提交的开销与批次数成正比、与命令数几乎无关，所以 GPU 编程攒批攒得越狠越好——渲染 API 鼓励“一次录一大段命令缓冲再提交”，正是这条算术的直接应用。

把 GPU 命令环与第十一章的 NVMe 提交队列逐项对照，结构上的同构就看得更清楚——学过一个，另一个的机制几乎可以全部推导出来：

维度	NVMe 提交队列	GPU 命令环
队列项	定长 64 字节命令	变长命令包，几十到几百字节
通知方式	写 SQ 尾 doorbell	写 doorbell 寄存器，内容是新尾指针
取命令	设备经 DMA 主动取	前端经 DMA 主动取
回收依据	CQ 完成项 + 写 CQ 头 doorbell	GPU 写回读指针，驱动回收槽位
队列组织	按 CPU 核绑多对队列	按上下文与引擎分环

还有一层二级结构值得点名：应用先把命令录进“命令缓冲”这种可整块复用的对象，提交时再把整块挂到环上。环只是搬运通道，命令缓冲才是复用单位——帧里不变的绘制序列可以录一次、提交多次，驱动省掉重新编码的开销。这与第五章“描述符预先建好、传输时只改指针”是同一招：把构造的成本与提交的成本分开，贵的只做一次。

doorbell 还有两个容易忽略的性质，都来自“内容只是指针”这一件事。其一，它可以合并：驱动连续提交三批、写三次门，GPU 看到的只是尾指针停在最后的位置，一次取走全部三批——中间两次通知被自然吸收，不需要任何去重逻辑。其二，它可以丢失后自愈：万一某次门铃写因为种种原因没生效，下一次写仍会把尾指针推到位，GPU 按指针差补读，错过的那批命令照样被取走。通知不携带状态，状态全在环与指针里，这正是这套机制健壮与省事的共同来源。

驱动侧提交一批命令的序列：

1. 查环上剩余空间（尾指针与 GPU 写回的读指针之差）
2. 把命令包依次写入环缓冲（普通主存写，可被写合并）
3. 写屏障，保证命令包先于通知对 GPU 可见
4. 写 doorbell 寄存器（一次 MMIO 写，内容是新尾指针）
5. GPU 前端经 DMA 取走新命令，逐包解码、分派给引擎
6. GPU 推进读指针并写回主存，驱动回收环上槽位

fence 与完成语义 doorbell 写完只代表“命令已交出”，不代表“活已干完”。完成语义由 fence 提供：一批命令的末尾挂一条 fence 命令，引擎执行到它，就把一块约定的内存位置写成预定值，可选地再触发一次中断。CPU 或后续命令流读这个位置，就知道“第 N 批到此全部完成”。注意 fence 的粒度：它不是每条命令一个完成项，而是一道“到此为止”的水线。与第十一章 NVMe 完成队列对照：NVMe 给每条命令回一项 CQ 完成项，GPU 用一次内存写确认整批。粒度差别的理由很现实——GPU 一批命令动辄上千，逐条完成通知本身就是一笔可观的 PCIe 流量，fence 把完成语义压缩成一个字。中断也按同样思路省：一批一中断，而不是一条一中断，否则光中断处理就把 CPU 占满。

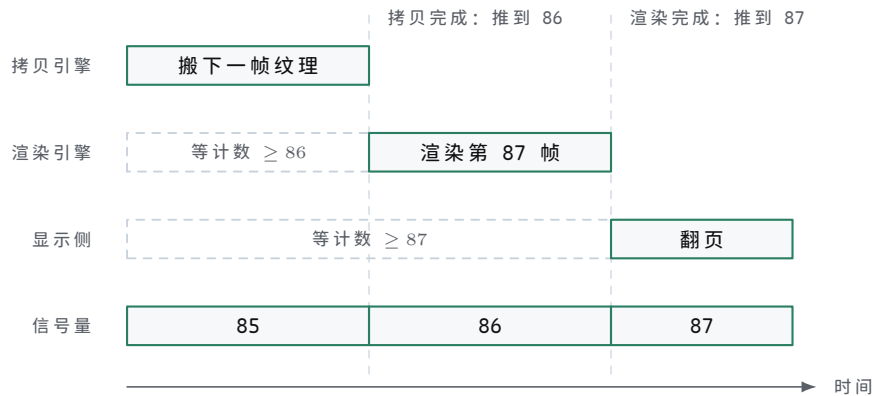
布尔 fence 只有“未完成/已完成”两态，用完必须重置才能复用，表达力很快见底：想表达“第 87 批完成、第 88 批还差拷贝引擎”就捉襟见肘。时间线信号量（timeline semaphore）把两态换成一个单调递增的 64 位计数器：每个完成点把它推到更大的值，等待方声明“等到计数不小于 K”。单调递增让旧值永不复现，等待条件因此没有歧义；K 可以是一帧、一批、一个流水级，同一个对象上还能叠加任意多个等待者与推进者。多引擎协作于是有了统一语言：渲染引擎推到 87 之后，显示侧只等“ ≥ 87 ”，拷贝引擎继续往 88 推，谁也不必知道别人的存在。Vulkan 与 D3D12 的现代同步全部时间线化，布尔 fence 只留在兼容性角落里。

布尔 fence: 值只有 0/1 两态; 等待”为 1”; 复用前须重置
 时间线信号量: 值单调递增; 等待”达到 K”; K = 帧号/批号/流水级
 对照 NVMe: CQ 完成项 = 每条命令回一条消息
 fence = 一次内存写确认”到此全部完成”

fence 还有一个不显眼但天天在用的角色：资源回收的依据。命令环上的槽位、这一帧用到的常量缓冲、中间纹理，都要等对应 fence 到位才能安全复用——驱动并不”等”它，只是定期查询，把已过水线的资源放回空闲池。完成语义在 GPU 世界里于是分成两层：给程序员的那层是”这批活干完了”，给资源管理器的那层是”这块内存可以再次投入使用了”。

举一个多引擎协作的完整例子，把两种语义都用上。第 87 帧的工作分三段：拷贝引擎先把下一帧的纹理搬进显存，完成后把时间线信号量推到 86；渲染引擎提交时声明”等到 ≥ 86 再开工”，干完把同一个信号量推到 87；显示侧提交翻页请求时声明”等到 ≥ 87 ”。三段命令分属三个引擎的三条环，谁也不等谁地并发推进，次序全靠计数器表达；CPU 从头到尾没有参与同步，它只是在提交第 90 帧前查了一眼计数器，确认 87 号之前的资源都可以回收。这就是时间线信号量比布尔 fence 灵活的根源：布尔 fence 表达一个点，时间线表达整条进度轴。

把第 87 帧这个例子画成时间线，”谁等谁”全写在计数器上：



图示：时间线信号量上的多引擎协作（第 87 帧）。拷贝引擎把下一帧纹理搬进显存，完成就把单调递增的计数从 85 推到 86；渲染引擎声明”等到 ≥ 86 再开工”，干完推到 87；显示侧等 ≥ 87 再翻页。三段命令分属三个引擎的三条环，并发推进、次序全靠计数器表达；CPU 不参与同步，只在回收资源前查一眼计数。单调递增让旧值永不复现，”等到不小于 K”没有歧义——布尔 fence 表达一个点，时间线表达整条进度轴。

CPU-GPU 同步的三种模式 知道”怎么查完成”之后，下一个问题是”查的时候 CPU 干什么”。三种模式对应三种答案。轮询等待：CPU 循环读 fence 所在的内存位置，直到值到位。发现完成的时延最低——只是一次 cache 行的读——代价是整个核被占死，还顺带把那行 cache 踢来踢去。它适合提交后马上要结果的短等待，比如读回一小段统计值。事件/中断：CPU 在 fence 上登记等待后睡眠，GPU 写 fence 后触发中断唤醒。CPU 让出来去干别的，代价是多一次中断处理与进程唤醒，时延增加数微秒到数十微秒，适合毫秒级以上的长等待。流水化：干脆不等——CPU 提交第 N 帧后不再碰第 N 帧的任何资源，转身开始录第 N+1 帧的命令；fence 只在回收资源时查询，从不作为等待点。

模式	CPU 的占用	完成发现的时延	适用场景
轮询等待	整核占死	最低，一次 cache 读	微秒级短等待、读回小结果
事件/中断	让出，可干别的	多一次中断与唤醒	毫秒级以上的长等待
流水化	从不等待	不查询，fence 只管回收	渲染主循环，吞吐优先

流水化值得再说一层，因为它是唯一随负载变好的模式。提交深度一帧：CPU 与

GPU 错开一整帧并行，吞吐拉满，输入到显示的时延多出一帧上下。深度两到三帧：GPU 永远有活干，偶发的慢帧被队列吸收，掉帧更平滑，时延再多一两帧——主机游戏常选这里，竞技游戏宁可锁深度一。深度不是免费的：它用内存（每级一套资源）与时延换吞吐。流水化也有一个天敌：读回。一旦 CPU 要拿 GPU 算出的结果做下一步决策——比如把计数器的值读回来决定后续提交多少活——就不得不排空整条流水，攒下的深度一次赔光；所以性能敏感的引擎把这类读回排到帧边界，或者干脆用上一帧的结果。

把深度的时延代价量化：60 Hz 下每级深度约添 16.7 ms 的输入到显示时延，深度三帧就是约 50 ms——手快的玩家对 30 ms 以上的差距已有明确体感，这是竞技游戏把深度压到一的全部理由。反过来，吞吐侧的账同样直接：深度为零（提交就等）时，GPU 在每次提交边界都要空转一个提交往返的微秒级空隙，一帧几十个边界就把帧预算啃掉一块。深度的选择于是是一个明确的工程旋钮：以毫秒为单位买帧间隔的平滑，买不买、买几毫秒，看应用对输入时延的容忍度。

这正引出本节的工程结论：最好的同步，是设计上不需要同步。帧缓冲、描述表、常量缓冲按帧轮换双份或三份，生产者和消费者各用各的份，fence 退化为回收水线而非等待点；队列深度、引擎数量，都是为“互不相等”服务的。反过来，凡是代码里出现“提交、立即等待、再提交”的段落，基本可以断定：那里丢掉的吞吐，比省下的内存值钱得多。

真实的等待策略通常是轮询与事件的混合体：先自旋几十微秒，等到了就赚回中断开销，等不到再登记睡眠——这与第五章中断合并、第十一章轮询模式（如高性能 NVMe 驱动 polling CQ）是同一种权衡：固定成本与等待时长之比决定谁划算。功耗维度也值得记一笔：轮询模式把一个核烧在空转上，台式机上换来最低的时延尚可接受，笔记本与手机上则直接影响续航；事件模式让出 CPU，核可以降频甚至进深度睡眠。同步模式的选择从来不只是时延题，也是功耗题。

显示扫描的固定时序 渲染引擎按命令干活，显示控制器按钟干活——两者是互相独立的硬件。显示控制器（scanout engine）不认命令环：它按像素时钟逐行从帧缓冲读像素，行内先发有效像素、再进入行消隐；帧内扫完有效行、再进入场消隐，循环往复，与 GPU 忙不忙毫无关系。这套时序是 CRT 时代的遗产：消隐期当年是电子束回扫的时间，数字接口把整张时序表原样保留下来，场消隐的空档如今还被用来夹带音频与辅助数据。以 1080p60 的标准时序为例：有效区 1920×1080 ，含消隐的总帧 2200×1125 ，像素时钟 148.5 MHz，每一个数都能算出物理含义：

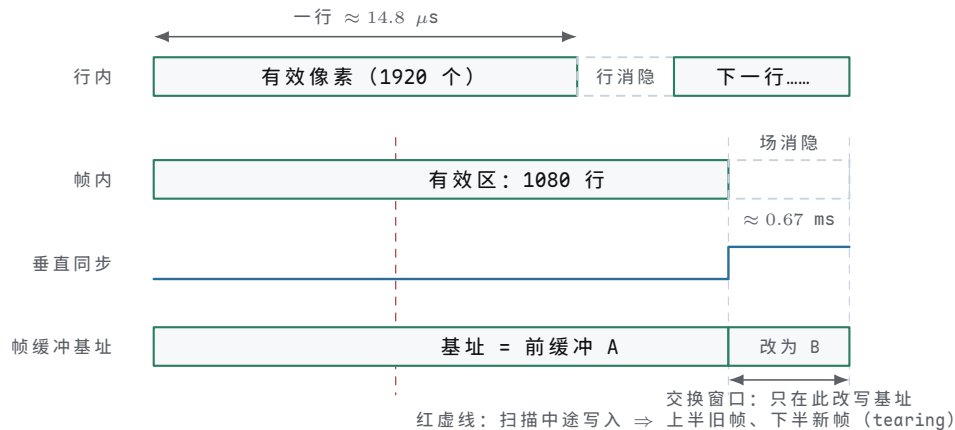
```
行时间 = 2200 像素 / 148.5 MHz ≈ 14.8 微秒
帧时间 = 1125 行 × 14.8 微秒 ≈ 16.67 ms (即 1/60 s)
行消隐 = 2200 - 1920 = 280 像素 ≈ 1.9 微秒
场消隐 = 1125 行 - 1080 行 = 45 行 × 14.8 微秒 ≈ 0.67 ms
扫描带宽 = 1920 × 1080 × 4 B × 60 ≈ 0.5 GB/s (4K60 约 2 GB/s)
```

三组数字各有工程后果。帧时间 16.67 ms 是渲染的预算线：一帧的全部工作必须装进这个窗口，装不进就掉帧。场消隐 0.67 ms 是每帧唯一允许“动手脚”的空档：切换帧缓冲、更新颜色查找表，都要在这个窗口内完成，错过就再等一帧。扫描带宽 0.5 GB/s 相对 820 GB/s 的显存带宽只是零头，所以显示扫描从不是带宽问题——它是节拍问题：渲染完成不等于显示完成，写进帧缓冲的内容要等扫描线走到那一行才真正可见，从“渲染完”到“用户看见”最多隔将近一帧。显示控制器的独立性还带来一个副产物：渲染引擎哪怕整个卡死，最后一帧画面照样被一遍遍扫出来，屏幕定格而非黑屏——多屏机器上每块屏各有一套控制器与像素时钟，各走各的节拍，一个窗口横跨两屏时，两屏的消隐期也未必对齐。

扫描的顺序本身也值得说细一点：每一帧从左上角逐像素向右、逐行向下，像读书一样扫完整屏，这个次序叫光栅扫描（raster scan），图形学里“光栅化”一词正由此来。历史上还有过隔行扫描的折中：1080i 把一帧拆成奇数行、偶数行两个场，各扫一次，用一半带宽换到接近逐行的观感——那是广播带宽紧张年代的产物，今天的接口带宽宽裕，逐行扫描已经一统天下。接口带宽若真不够，现代标准的答案是

显示流压缩 (DSC)：在显示控制器出口处把像素流做视觉无损的轻量压缩，接口带宽立省三分之二，解压则放在显示器内部——又是一次“用计算换带宽”，与本章开头 CPU/GPU 面积取舍是同一条思路。

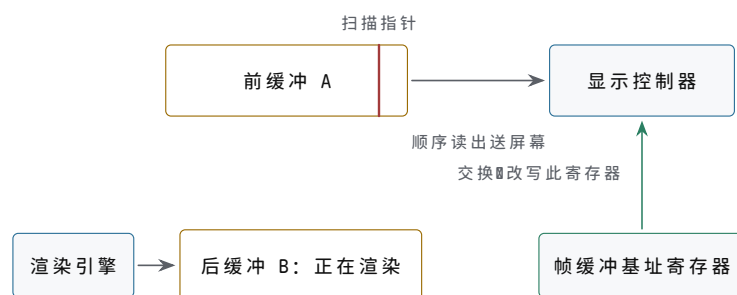
把行、场两个尺度的时序画在同一张图上，消隐窗口与交换点的位置关系就清楚了；图中也提前标出下一题要展开的 tearing。



图示：显示扫描的两层时序（示意，不按比例）：行内先有效像素再入行消隐，一行约 $14.8 \mu s$ ；帧内扫完 1080 个有效行入场消隐，垂直同步脉冲随之拉高。双缓冲的交换—改写帧缓冲基址寄存器—只允许发生在场消隐窗口内；若在扫描中途改写（红虚线），屏幕上半是旧帧、下半是新帧，这就是 tearing。

tearing 与双缓冲/垂直同步 单缓冲把渲染与扫描的矛盾直接摆上台面：渲染引擎写的就是显示控制器正在读的那块帧缓冲，新内容写进扫描线尚未走到的区域，这一屏就露出半张新图；画面快速横向运动时，上下两半错开一道水平缝——这就是 tearing。静态画面的撕裂不可见，运动越大缝越明显。双缓冲的解法是把读写分开：渲染写后缓冲，显示读前缓冲，交换只在场消隐那 $0.67 ms$ 内执行——交换本身只是改写显示控制器的帧缓冲基址寄存器，一次寄存器写，零拷贝。只要交换点严格避开扫描窗口，撕裂从机制上消失，这就是垂直同步 (vertical sync, vsync)：把缓冲交换锁在场消隐。

读写分开之后，交换的瞬间动作画出来不过是一次寄存器写：



交换只在场消隐那 $0.67 ms$ 内执行：一次寄存器写，零拷贝，A、B 角色互换

图示：双缓冲把渲染与扫描分开：渲染引擎写后缓冲 B，显示控制器经扫描指针读前缓冲 A，两块缓冲互不干扰。交换只是改写显示控制器的帧缓冲基址寄存器——一次寄存器写，零拷贝——且只在场消隐那 $0.67 ms$ 内执行，此后 A、B 角色互换。只要交换点严格避开扫描窗口，撕裂从机制上消失，这就是垂直同步；若扫描中途改写基址，屏幕上半是旧帧、下半是新帧，即 tearing。

锁住的代价立刻出现：渲染若错过窗口，后缓冲就得再等一整帧才能上前台。60 Hz 下渲染耗时 $17 ms$ ——只比预算多 $0.33 ms$ ——帧率不是降到 58，而是直接掉到 30，因为每一帧都被对齐到第二个消隐窗口；输入时延也随之最多添上 $16.7 ms$ 。三缓冲退一步：后缓冲备两份，渲染写完一份不必等交换就能写另一份，掉帧平滑了，内存与时延各添一份。可变刷新率 (VRR, FreeSync/G-Sync 一类) 干脆反转主从：显示器不再按固定节拍催帧，而是等 GPU 宣布“新帧就绪”才开始扫下一帧，撕裂与等待同时消除；代价是帧间隔不再均匀，低帧率时要靠倍帧补偿维持面板刷新，而

且整条链路—显卡、线缆、显示器—都得支持。还要补一句桌面环境的现状：窗口合成器默认按垂直同步的节拍合成，普通窗口程序其实一直活在等效 vsync 里，只有全屏独占或显式绕过合成的路径才有上面这些选择。

方案	撕裂	输入时延	代价与适用
无同步	有	最低	基准测试、对画质不敏感的场景
垂直同步	无	最多添一帧	帧率对齐节拍约数：17 ms 渲染掉到 30 fps
三缓冲	无	介于两者之间	多一份帧缓冲内存，掉帧更平滑
可变刷新率	无	接近无同步	需显示器支持；帧间隔不均，低帧率靠倍帧

把本节与前一节合看，GPU 系统的两条时钟就齐了：一条是“能跑多快跑多快”的命令环时钟，吞吐优先；一条是“一微秒也不商量”的扫描时钟，节拍优先。双缓冲与垂直同步正是两条时钟之间的翻译器：生产者随意快慢，消费者严格准时，交换只发生在双方都不看的瞬间。工程上选哪一档同步，本质是在帧率、时延、画质三者之间挑两个——机制层面，谁也给不出三全其美。

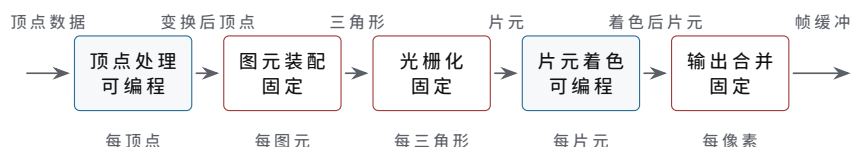
最后把垂直同步的帧率阶梯画清楚，它是“对齐”二字的直接后果。交换只能发生在消隐窗口，所以 60 Hz 下实际帧率只能取节拍的约数：16.7 ms 内完成是 60 fps，33.3 ms 是 30，50 ms 是 20，66.7 ms 是 15——帧耗时 17 ms 与 33 ms 在屏幕上得到同一个 30 fps，60 与 30 之间没有任何中间档，渲染稍慢一档，帧率就跳一档，体感上就是“忽然从 60 掉到 30”。VRR 把阶梯抹平：面板刷新率跟随渲染完成的时刻，17 ms 的帧就以约 59 Hz 显示，23 ms 就以约 43 Hz 显示，帧率第一次可以连续变化。代价也写在机制里：帧间隔忽长忽短，对帧间隔敏感的人会觉得“不匀”；低到面板下限（常见 48 Hz）以下时，驱动要靠低帧率补偿把每帧显示两三遍，把名义刷新率抬回面板能接受的范围。选择依旧回到那句：帧率、时延、画质，三选二。

五、从图形管线到通用计算

本节把前四节装回两种真实负载。先沿图形管线走一遍：一个三角形怎样从 API 调用变成屏幕上的像素，看哪些阶段至今仍是固定硬件、哪些交给了可编程 shader；再换到通用计算一侧，看 CUDA/OpenCL 的内核模型怎样映射到同一台机器，并把一次内核启动做成与第十一章 NVMe 读并排可比的 trace；最后总结 GPU 编程最常见的三个陷阱——它们破坏的是同一个根本：海量在途工作——并顺带看一眼 GPU 之外更专的加速器。

图形管线的固定与可编程部分 实时图形几十年只做一件事：把三维场景变成一帧二维像素，每秒至少 60 次。这件事天然分层。顶点处理：给每个顶点的坐标乘上模型、视图、投影三个矩阵，变换到屏幕空间，顺带算出顶点的颜色与纹理坐标。图元装配：把变换后的顶点按连接关系拼成三角形，裁掉落在视锥之外的部分。光栅化：对每个三角形枚举它覆盖哪些像素，在覆盖点上对顶点属性做插值，输出一批片元（fragment）——片元是“候选像素”，带着插值出的位置、颜色与纹理坐标。片元着色：对每个片元求最终颜色，查纹理、算光照。输出合并：把片元与帧缓冲里已有的像素做深度测试（近的挡住远的）、模板测试与颜色混合，活下来的才写进帧缓冲。从顶点到像素，数据形态换了四次，每一级的输出就是下一级的输入。

把这五级画成一条流水线，数据形态的接力与可编程的分界就看得更清楚。



蓝框：可编程（shader 在 SM 上按 SIMT 执行）；红框：固定硬件（算法几十年不变，专用电路）

图示：图形管线五阶段：数据形态沿顶点、三角形、片元、颜色一路变换，每一级的输出就是下一级的输入。可编程与固定的分界清清楚楚——顶点处理与片元着色交给 shader，在 SM 上按 SIMT 跑；装配、光栅化、输出合并的算法几十年不变，做成固定电路。级间全走先进先出队列，不需要全局调度者。

哪些可编程，一句话：需要”每个图元各不相同”的数学开放给 shader，永远同一件事的步骤做成固定硬件。顶点处理与片元着色是可编程的：游戏要的变换与光照千变万化，厂商把这两级交给程序员写 kernel——顶点 shader 与片元 shader，编译成与通用内核一样的指令，在 SM 上按 SIMT 方式跑。图元装配、光栅化、输出合并是固定的：裁剪就是那几个不等式，光栅化就是逐像素判”点落在三角形哪一侧”的边函数，深度测试就是一次比较加一次条件写——算法几十年不变，做成专用电路比可编程核快得多、省得多。裁剪留在固定硬件还有个隐蔽理由：它的输出不规则，一个三角形裁完可能剩零到几个多边形，不适合 SIMT 的整齐数组风格。

阶段	干什么	谁来做	并行粒度
顶点处理	矩阵变换、顶点属性计算	可编程（顶点 shader）	每顶点一个线程
图元装配与裁剪	拼三角形、视锥剔除	固定硬件	每图元
光栅化	枚举覆盖像素、属性插值	固定硬件	每三角形
片元着色	纹理采样、光照、最终颜色	可编程（片元 shader）	每片元一个线程
输出合并	深度与模板测试、混合写回	固定硬件	每像素

再算一笔负载账，看面积该怎么分。4K 帧缓冲 $3840 \times 2160 \approx 8.3 \times 10^6$ 像素，60 Hz 下每秒产出约 5×10^8 像素；画面里三角形层层遮挡，过绘制按保守 3 倍计，片元着色每秒要处理约 1.5×10^9 个片元。每片元做纹理过滤加几项光照按 100 次浮点操作估，就是每秒 1.5×10^{11} 次操作，还没算顶点与光栅化；CPU 单核每秒能做的浮点操作在 10^{10} 量级，连片元着色这一级都扛不住。这就是图形管线必须并行、且把并行做成流式的理由：片元数量比顶点大一到两个数量级，面积自然跟着数据量走——可编程的片元级占去芯片大部分，固定级做窄而深。

更妙的是这条管线不需要调度者。每级只盯着自己的输入队列：顶点处理完一个顶点就交给装配，光栅化每产生一个片元就喂给片元着色器；级间是先进先出队列，级内是成千上万个互不相同的顶点与片元。数据并行叠着流水并行，没有全局汇合，没有全局同步，每一级按自己的节奏铺满硬件——流式并行是图形的天然结构，也是后来通用计算直接继承的结构。

一次绘制命令的旅行 把”应用画一个三角形”从 API 调用到屏幕发光按时序走一遍。这是第四节的命令环与显示扫描在图形负载上的一次完整应用，这里只追数据流，同步原语的细节不再重复。

步	动作	执行者
1	应用把顶点数据（坐标、颜色、纹理坐标）放入显存可见的缓冲，调用绘制命令	CPU
2	驱动校验渲染状态，把绘制翻译成 GPU 命令包，写入命令环尾部	CPU

3	屏障之后写 doorbell 寄存器 (MMIO 写), 通知 GPU CPU 有新命令	
4	命令处理器 DMA 取走命令包, 解析出绘制, 配置管线状态	GPU 前端
5	顶点 fetch: DMA 按索引从显存读出全部顶点的属性	GPU 前端
6	顶点着色: 每个顶点一个线程, 在 SM 上跑顶点 shader	SM
7	图元装配与裁剪: 拼出三角形, 确认落在视锥之内	固定硬件
8	光栅化: 枚举三角形覆盖的像素, 插值属性, 产出一批片元	固定硬件
9	片元着色: 每个片元一个线程, 查纹理、算光照, 得到颜色	SM
10	输出合并: 深度测试、混合, 写帧缓冲	固定硬件
11	显示控制器按行扫描时序从帧缓冲读像素, 逐行送出	显示引擎
12	像素经显示接口抵达屏幕, 对应位置发光	显示器

这张表有三处值得停下来看。第一, 第 1 到第 3 步与第十一章 NVMe 的提交一模一样: 备好数据、写命令、doorbell——第五章“描述符先可见、再通知”的约定同样适用。第二, CPU 在第 3 步就结束了本次绘制的参与: 它不等待渲染完成, 转身去准备下一帧的命令; GPU 从第 4 步起自主跑完整个管线。命令环换来的正是这份解耦: CPU 与 GPU 各跑各的, 只在环的生产与消费处碰头。第三, 第 11 步与前面所有步骤异步: 显示扫描按自己的固定时钟走, 读的是“此刻完整的帧缓冲”, 渲染完成与显示完成之间隔着第四节讲的垂直同步与帧切换。一帧 16.7 ms 的预算里, CPU 备命令通常只占一二毫秒, 其余都是 GPU 的管线时间; 哪一级超载哪一级的队列先满, 帧率被最慢的一级锁死——管线吞吐由瓶颈级决定, 这与第十章的利用率分析是同一条规律。

CUDA/OpenCL 的内核模型 通用计算借用同一台机器, 只换一层软件抽象。内核(kernel)就是“在 N 个数据点上运行的同一个函数”: 程序员写处理一个元素的代码, 启动时给出总数, 硬件负责把 N 个实例铺开。CUDA 把一次启动组织成三层: 网格(grid)是这次启动的全部线程; 块(block)是网格切成的组, 块内线程可以经共享内存协作、可以块内同步; 线程是最小单位, 拿着自己的索引算自己那份。三层到硬件的映射很直接: 块整体调度到一个 SM 上常驻; 块内每 32 个线程编成一条 warp; warp 是 SM 真正发射指令的单位, 一条 lane 对应一个线程。OpenCL 的 NDRange、work-group、work-item 是同一张表的另一套名词。向量加法内核的骨架如下:

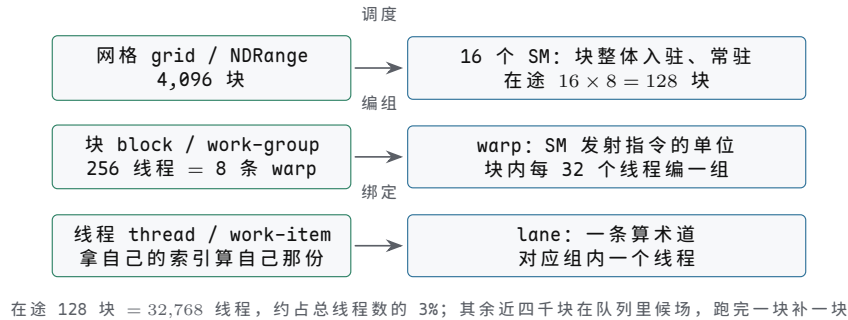
```
/* 向量加法: c[i] = a[i] + b[i], 共 N 个元素 */
__global__ void vec_add(const float *a, const float *b,
                        float *c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x; /* 全局索引 */
    if (i < N) /* 越界的线程直接退出 */
        c[i] = a[i] + b[i];
}

/* 主机侧: N = 2^20, 每块 256 线程 */
int threads = 256;
int blocks = (N + threads - 1) / threads; /* 4096 块 */
vec_add<<<blocks, threads>>>(d_a, d_b, d_c, N);
```

算一下这次启动的规模。 $N = 2^{20} = 1048576$, 每块 256 线程, 得 4096 块; 每块 $256/32 = 8$ 条 warp。若 GPU 有 16 个 SM、每 SM 可常驻 8 块, 则在途 128 块、

$128 \times 256 = 32768$ 个线程——本章开头说的“上万线程”就是这样来的：在途的只是第一梯队，约占总线程数的 3%，其余近四千块在队列里候场，某块跑完立即补位。内核里“越界即退出”那行判断，是网格与 N 不整除时的边界保护（本例恰好整除，但写成习惯）。再注意内核代码里没有循环、没有同步、没有“第几个元素归谁”的概念：并行度全部交给启动配置。这与第四章最小 CPU“一条指令流顺序走”是两种世界观：程序员交出了控制流，换回来的是上万份数据同时推进。

三层软件名词到三层硬件的对应画成对照，这次启动的规模也就摆齐了：



图示：CUDA 三层抽象到硬件的映射（OpenCL 名词列在斜线后）。网格是这次启动的全部线程，切成 4,096 块；块整体调度到一个 SM 上常驻，16 个 SM 各驻 8 块，在途 128 块合 32,768 个线程；块内每 32 个线程编成一条 warp，warp 才是 SM 真正发射指令的单位，一条 lane 对应一个线程。程序员只写处理一个元素的代码，并行度全部交给启动配置。

案例：一次内核启动的完整 trace 把上面这个 `vec_add` 从数据到结果走完。设 $N = 2^{20}$ ，GPU 接在 PCIe 3.0 x16 上（单向约 15.75 GB/s），显存带宽按 400 GB/s 计，输入 a 、 b 共 8 MiB，输出 c 共 4 MiB。

步	动作	耗时量级	执行者
1	把 a 、 b 钉在 pinned 缓冲（第五章：DMA 要求物理地址稳定）	一次性	CPU
2	发拷贝命令，DMA 把 8 MiB 经 PCIe 搬进显存	约 $530 \mu\text{s}$	GPU DMA
3	启动命令（网格配置、参数指针）写入命令环，写 doorbell	微秒级	CPU
4	GPU 取命令，把 4096 块分发到各 SM	微秒级	GPU 前端
5	内核执行：读 a 、 b 写 c ，显存流量共 12 MiB	约 $31 \mu\text{s}$	SM
6	全部块完成，GPU 把 fence 写到主存并发中断	微秒级	GPU
7	DMA 把 4 MiB 结果搬回主存	约 $270 \mu\text{s}$	GPU DMA
8	fence 就绪，CPU 读结果缓冲	微秒级	CPU

总时长约 $830 \mu\text{s}$ 。拆开看占比：两次 PCIe 拷贝约 $800 \mu\text{s}$ ，占九成半；内核本身 $31 \mu\text{s}$ ，只占 4%。根源是算术强度低得无可救药：全内核只做 2^{20} 次加法，流量 12 MiB，约 0.08 次操作每字节；按第十章 roofline，峰值算力 10 TFLOPS、带宽 400 GB/s 的机器，拐点在 $10^{13}/(4 \times 10^{11}) = 25$ 次操作每字节，这个内核差了近三百倍，注定钉在带宽屋顶上。工程结论有两条。其一，向量加法这种强度的负载不值得上 GPU：数据还没算完，搬运先花掉二十几倍的时间；值得上 GPU 的负载，必须让每个字节多做几十上百次操作。其二，真要用 GPU 就得批：数据常驻显存，一次搬运喂一串内核，把 PCIe 这笔固定开销摊到足够多的计算上——分块流水、异步重叠，都是在跟这九成半作战。

把这次 trace 与第十一章的 NVMe 读并排摆开，是同一个骨架穿了两套衣服：

维度	内核启动	NVMe 读（第十一章）
提交物	命令包：网格配置、参数指针	64 B 命令项：LBA 与 PRP 列表
队列	命令环（第四节）	SQ/CQ 队列对
通知	doorbell MMIO 写	doorbell MMIO 写
搬运	DMA 双向：输入进显存、结果回主存	DMA 单向：NAND 数据进页框
完成	fence 写主存加中断	CQ 完成项加 MSI
大头	PCIe 搬运，占九成半	介质读，占八成

骨架相同不是巧合：凡是“CPU 发单、设备干活”的接口，都要回答同样四个问题——命令放哪（内存队列）、怎么通知（doorbell）、数据谁搬（DMA）、完成怎么确认（完成项或 fence）。NVMe 与 GPU 驱动是这套约定的两个实例，学会一个，另一个的文档就只剩名词表要背。差别在优化的主战场：NVMe 的大头是介质，优化指向访问形态（对齐、合并、足够的在途深度）；GPU 内核的大头是搬运，优化指向批量化与数据常驻。

GPU 编程的三个常见陷阱 GPU 的宽容度很高：内核写糙了照样算出正确结果，性能却能差出百倍。最常见的三个陷阱看着互不相关，破坏的却是同一个根本——海量在途工作。

陷阱一：分支分歧。一条 warp 共享一条指令流，遇到各 lane 去向不同的分支，硬件只能串行：先放行走向 if 一侧的 lane、其余停等，再放行走向 else 一侧的 lane、其余停等。最坏情形是每条 lane 走不同的路：

```
/* 反例：按 lane 号散到 32 个分支，warp 退化成顺序执行 */
switch (threadIdx.x % 32) {
    case 0: c[i] = f0(a[i]); break;
    case 1: c[i] = f1(a[i]); break;
    /* ... 直到 case 31 */
}
```

后果直接按 SIMT 模型算：32 条路径串行，这条 warp 的吞吐退到峰值的 1/32，等于把 32 条 lane 的硬件用成 1 条。随机数据上的普通 if-else 也好不到哪去：约半数 lane 走 A、半数走 B，两条路径都得执行，耗时是 $t_A + t_B$ 的串行和，而不是两者之一。

陷阱二：未合并访存。一条 warp 的 32 次访存若地址连续，恰好拼成 128 字节，一次内存事务取回一整条缓存线，人人有份；若地址跨步跳跃，每个线程各碰一条线：

```
/* 反例：跨步 32，每条 lane 各要一条 128 B 的线 */
c[i] = a[i * 32] + b[i * 32];
```

原来一次事务办完的事变成 32 次，每次取 128 字节只用 4 字节，带宽效率 1/32：400 GB/s 的显存被用成 12.5 GB/s。内核从带宽受限变成事务受限，内存队列里堆满等数据的 warp——占用率再高，也藏不住被放大了 32 倍的时延。

陷阱三：CPU-GPU 过度同步。每次启动后立刻等 fence、立刻把结果拷回、立刻在 CPU 上检查再决定下一步：

```
/* 反例：1000 次小启动，每次都同步等结果 */
for (int k = 0; k < 1000; ++k) {
    vec_add(<<1, 256>>>)(d_a + k*256, d_b + k*256,
                        d_c + k*256, 256);
    gpu_synchronize(); /* 等 fence, GPU 排空 */
    check_on_cpu(d_c + k*256); /* 立刻读回 */
}
```

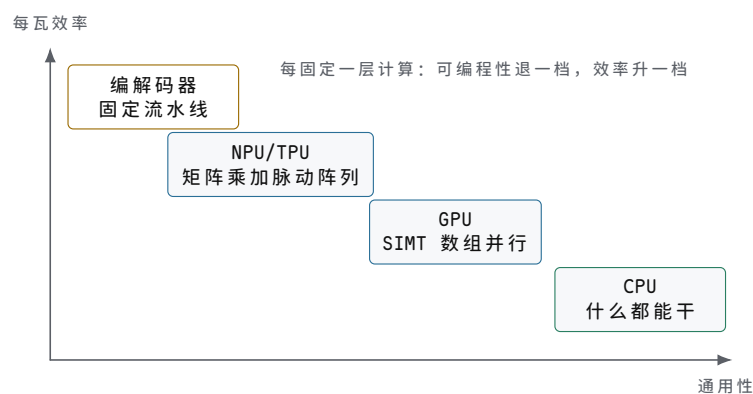

每次同步都把整条管线抽干：doorbell、执行、fence、中断，一趟往返按 $10\ \mu\text{s}$ 估，1000 次就是 10 ms；同样的数据量一次启动只需几十微秒，慢出去两个数量级。更要命的是同步期间 GPU 完全空闲：在途工作清零，时延没有任何东西可藏。三个陷阱的药方其实是同一句话：让分支收敛回整块整块地走，让访存连续回 128 字节一条线，让同步退到一批工作的末尾——都是在把”在途”二字养回来。

GPU 之外的加速器 GPU 相对 CPU 已经专了一级：放弃单线程时延，换吞吐。这条路再往前走是更专的芯片，规律始终同一条：通用性再降一档，效率再升一档。两个代表。其一是 NPU/TPU 一类的矩阵引擎。神经网络的核心操作是大矩阵乘加，TPU v1 干脆把 256×256 个乘加器排成脉动阵列 (systolic array)：数据从阵列一端泵入，在相邻单元间逐级传递，每拍完成 65536 次乘加，700 MHz 下约 4.6×10^{13} 次乘加每秒 (约 92 TOPS)。关键不在运算更快，而在搬运更少：数据在阵列内部的寄存器间流动，几百次乘加才需要一次对外访存，而访存正是能耗大头。其二是视频编解码器。H.264/HEVC 的帧间预测、变换、熵编码是几十年不变的固定流水线，做成专用电路后，手机 SoC 上一小块逻辑就能实时解 4K60 的视频，功耗以瓦计；同样的码流交给 CPU 软件解码，要动用多个大核，功耗高出一个数量级。

芯片	固定下来的计算	放弃的通用性	换来的效率
GPU	SIMT 数组并行、图形管线	单线程时延、乱序投机	每瓦吞吐高一个量级
NPU/TPU	矩阵乘加脉动阵列	只剩张量算子可表达	每瓦算力再升一档
视频编解码器	预测、变换、熵编码整条流水线	几乎不可编程	瓦级实时 4K

这张表从上往下看，就是加速器的一般规律：每固定一层计算，可编程性退一档，单位能耗能做的事上一个数量级。CPU 在表外最上头：什么都能干，什么都最贵。系统设计的问题从来不是”用不用加速器”，而是”负载里哪一段足够固定，固定到值得为它造一块电路”。这正是第十章定量方法给出的决策顺序：先测负载各段占多少时间，再决定把哪一段交给专用硬件。

把这张表摊成坐标，”通用性换效率”就是一条一眼可见的走向：



图示：通用性-效率坐标上的四类芯片。CPU 在最右下：什么都能干，什么都最贵；GPU 固定了 SIMT 数组并行与图形管线，放弃单线程时延，每瓦吞吐高一个量级；NPU/TPU 只剩张量算子可表达，数据在脉动阵列内部流动、几百次乘加才一次对外访存，每瓦算力再升一档；视频编解码器把整条预测、变换、熵编码流水线做成电路，几乎不可编程，换来瓦级实时 4K。系统设计的问题从来不是”用不用加速器”，而是负载里哪一段固定到值得为它造一块电路。

本节小结：图形管线把渲染拆成五级流水，可编程的顶点与片元两级交给 shader，其余三级因算法几十年不变而固化成电路；绘制命令与内核启动走同一条命令路径—写命令、doorbell、DMA、fence，与第十一章的 NVMe 共享同一副骨架；内核模型把并行度交给启动配置，程序员只管写好一个元素的计算；三个常见陷阱（分歧、未合并访存、过度同步）破坏的都是同一样东西—海量在途工作；GPU 之外，NPU 与视频编解码器把”通用性换效率”再推一档。至此本章的 GPU 已不再是”显

卡”，而是一台命令走队列、数据走 DMA、完成走 fence 的吞吐优先处理器，它的每个怪癖都能从这条设计取向里推出来。

章末练习

任务	要求	学习目标
时延与吞吐取向	同一晶体管预算两个方案：A 为 1 个 4 发射乱序核、5 GHz，每拍完成 4 次浮点加；B 为 128 条简单 lane、1.5 GHz，每 lane 每拍 1 次浮点加。对规则数组加法求两方案的峰值吞吐与比值；再说明指针追逐类代码该选哪个、为什么	时延取向与吞吐取向的面积分配
占用率演算	某 SM 有 65536 个 32 位寄存器，内核每线程用 32 个寄存器，每块 256 线程。求每 SM 最多常驻几块、合多少线程与 warp；若访存时延约 400 拍、每 warp 平均 8 拍发一条指令，估算藏住时延需要的 warp 数并判断该内核够不够；每线程改用 64 个寄存器时又如何	在途 warp 数决定时延隐藏
分歧代价	某 warp 执行“为正走 A、否则走 B”的分支，路径 A 约 200 拍、B 约 300 拍，数据随机使约半数 lane 走 A。求该 warp 的耗时，并与全部走 A 的理想情形相比；若 32 条 lane 各走不同分支，吞吐退到多少	串行执行两条路径，最坏 1/32
合并访存	某 warp 读 32 个连续 float，恰为 128 B 一次事务；改为跨步访问。求跨步 8 ($a[i*8]$) 与跨步 32 时一次 warp 访存各需几次 128 B 事务、带宽效率各为多少；峰值 400 GB/s 下跨步 32 的有效带宽是多少	连续地址拼事务，跨步拆事务
带宽预算	向量加法 $N = 2^{20}$ （输入 8 MiB、输出 4 MiB），PCIe 3.0 x16 单向 15.75 GB/s，显存带宽 400 GB/s。忽略启动开销，求拷入、内核、拷回各耗时、总时长与内核占比；给出一条降低搬运占比的工程做法	算术强度决定值不值得上 GPU
内核 trace 排序	把一次内核启动的这些事件排成时间序：写 doorbell、输入 DMA 入显存、fence 更新、启动命令写入命令环、内核在 SM 执行、结果 DMA 回主存、CPU 读结果；并指出哪几处先后是硬性约定	提交-执行-完成三段次序

参考解答与要点

任务	参考解答	必须掌握的要点
----	------	---------

时延与吞吐取向	A: $4 \times 5 \times 10^9 = 2 \times 10^{10}$ 次/秒; B: $128 \times 1.5 \times 10^9 = 1.92 \times 10^{11}$ 次/秒, 约为 A 的 9.6 倍。但 B 的单 lane 每拍只 1 次操作且时钟更低, 指针追逐这类串行依赖代码在 B 上只有一条 lane 干活, 时延比 A 差几十倍; 数组加法没有依赖, B 的吞吐直接兑现。取向由负载的依赖结构决定	吞吐方案赢在可并行负载, 串行负载仍归时延方案
占用率演算	每块用 $256 \times 32 = 8192$ 个寄存器, $65536/8192 = 8$, 最多 8 块 = 2048 线程 = 64 条 warp。每 warp 每 8 拍发一条, 400 拍时延内需 $400/8 = 50$ 条可发射 warp; $64 > 50$, 时延可藏住。每线程改用 64 个寄存器后每块占 16384 个, 只容 4 块 = 32 条 warp, $32 < 50$, 发射槽开始空转, SM 吞吐明显下滑。寄存器用量是占用率的第一约束	先算每块资源, 再算 warp 数, 最后对时延需求
分歧代价	warp 内两条路径都有 lane 走, 硬件串行执行 A 再 B, 耗时 $200 + 300 = 500$ 拍; 全部走 A 只需 200 拍, 分歧代价 $500/200 = 2.5$ 倍。32 条 lane 各走不同分支时 32 条路径全串行, warp 吞吐退到峰值的 $1/32$ 。分歧的代价由路径数决定, 不由走每条路径的 lane 数决定	按路径串行计费; 让同 warp 的 lane 走同一条路
合并访存	连续: 32 个 float 恰为 128 B, 1 次事务, 效率 1。跨步 8: lane 地址间隔 32 B, warp 覆盖 $32 \times 32 \text{ B} = 1024 \text{ B} = 8$ 条线, 8 次事务, 取 1024 B 只用 128 B, 效率 $1/8$ 。跨步 32: 间隔 128 B, 每 lane 各碰一条线, 32 次事务, 效率 $1/32$, 有效带宽 $400/32 = 12.5 \text{ GB/s}$ 。访存效率看”一次事务取回的字节里用掉几个”	事务数等于触及的线数; 跨步越大带宽越薄
带宽预算	拷入 $8 \times 2^{20}/(15.75 \times 10^9) \approx 533 \mu\text{s}$; 内核流量 12 MiB, $12 \times 2^{20}/(400 \times 10^9) \approx 31 \mu\text{s}$; 拷回 $4 \times 2^{20}/(15.75 \times 10^9) \approx 266 \mu\text{s}$ 。总约 830 μs , 内核占 $31/830 \approx 3.7\%$, 搬运占九成半。做法: 数据常驻显存、一次搬运后连发多个内核, 或把拷入、内核、拷回分块流水重叠; 算术强度这么低的负载, 最直接的答案是干脆留在 CPU 上	PCIe 是瓶颈; 摊薄搬运靠批量化与常驻
内核 trace 排序	次序: 输入 DMA 入显存 → 启动命令写入命令环 → 写 doorbell → 内核在 SM 执行 → fence 更新 → 结果 DMA 回主存 → CPU 读结果。三处硬性约定: doorbell 必须在命令可见之后 (先可见、再通知); fence 必须在内核完成之后; CPU 读结果必须在 fence 与回拷完成之后。输入拷贝本身也走”命令加 doorbell”, 此处抽象成一步	与 NVMe 同骨架: 提交-执行-完成, 次序即可见性

链接、装载和系统软件边界

硬件章节到这里已经把机器讲完了：从晶体管到 CPU，从内存到外设。但还有最后一段旅程没被硬件书讲清：源码写完以后，它是怎样变成内存里可执行的位模式的。目标文件里的符号是谁定址的，程序是怎样被装进内存的，第一条指令执行前发生了什么，运行中的程序又怎样请求操作系统替它做事。这一段常被归为”系统软件”，但它的每一步都是硬件地址约定：重定位是地址修补，装载是建立页表，系统调用是特殊的异常入口。本章把程序的出生过程写成一条可追踪的硬件链路。

阶段	做什么	对应的硬件动作
编译	源码变目标文件	生成指令编码和重定位记录
链接	合并目标文件、解析符号	把符号名变成确定地址
装载	把程序映像放进内存	建立页表、映射段
运行	请求内核服务	系统调用陷入异常入口

核心思想

链接和装载不是编译器内部魔法，而是”什么时候把哪个名字定成哪个地址”的时间表。读懂这张时间表，静态库、动态库、PLT、系统调用这些名词都会各就各位。

一、目标文件与符号

本节回答一个问题：编译器交出来的目标文件，里面到底装了什么。前面十二章讲的都是硬件怎样执行指令，从本节开始换一个视角：指令被执行之前，先作为字节躺在文件里。目标文件就是这批字节的容器，读懂它只需要一张”哪段字节是什么、给谁看”的地图。本节先读 ELF 头部的固定字段，分清三种目标文件类型；再看几个常见节各自的分工，特别是只占内存、不占文件的 `.bss`；然后看符号表怎样把名字映射到位置；接着用 `readelf` 与 `objdump` 把一个真实目标文件翻一遍；最后把一个最小 C 函数从源码逐字节解剖到符号表和重定位记录，把全节的概念串在同一条链上。

ELF 是目标文件的通用容器 x86-64 Linux 上，编译器每处理一个源文件就交出一个 ELF 目标文件，链接器拼出的可执行文件是 ELF，共享库也是 ELF。ELF (Executable and Linkable Format) 是这一整条流程的通用容器：同一套字节排布约定，既能让链接器读到”这里有一段代码、那里有一张符号表”，也能让内核装载机读到”把哪一段映射到哪个虚拟地址”——后者是第三节的事。

容器从固定格式的头部开始。前 16 字节是身份区：头 4 字节是魔数 `0x7F` 加字母 `E、L、F`，任何工具拿到文件先核对这个签名，对不上就立刻拒收；随后两个字节说明类别（32 位还是 64 位）和字节序（大端还是小端）——只凭这两个字段，解析程序就知道后面所有多字节字段该按什么宽度、什么顺序读。头部其余字段里最重要的几个列在下表。

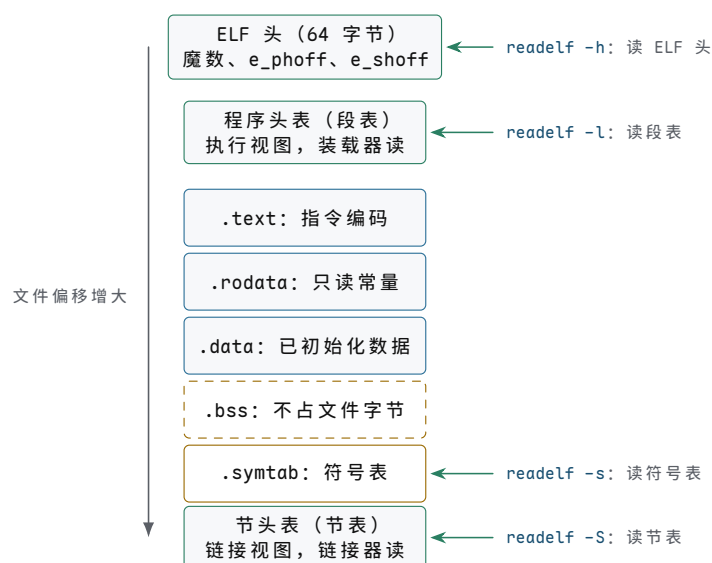
字段	内容	谁来读
<code>e_type</code>	文件类型： <code>ET_REL</code> / <code>ET_EXEC</code> / <code>ET_DYN</code>	一切工具先按它分流

<code>e_machine</code>	指令集编号, x86-64 为 62	链接器据此拒绝混链不同 ISA
<code>e_entry</code>	进程入口的虚拟地址	装载器; 可重定位文件恒为 0
<code>e_phoff</code>	程序头表 (段表) 的文件偏移	装载器
<code>e_shoff</code>	节头表 (节表) 的文件偏移	链接器与观察工具

三种类型对应程序出生过程的三个形态。可重定位文件 (`.o`) 是半成品: 代码和数据都在, 但地址一律未定, 凡涉及地址的字段全部留空, 另附一叠”这里以后要填”的重定位记录。可执行文件是成品: 每个节的虚拟地址已定, 入口字段指向第一条要执行的指令 (现代发行版常把可执行文件编成 ET_DYN 形态的 PIE, 把定址推迟到装载时, 第三节展开)。共享库 (`.so`) 介于两者之间: 代码数据齐全, 却故意不绑定固定装载地址, 好让多个进程各自把它映射到不同位置。三种形态共用一套头部与节表格式, 差别只在几个字段的取值和洞里填没填。

头部之后是两份”目录”, 给出同一批字节的两种看法。节表 (section header table) 逐项列出每个节的名字、类型、大小、文件偏移和属性标志, 这是链接视图: 链接器按节合并、按节定址。程序头表 (program header table, 常称段表) 把属性相同的若干节归并成段, 逐项给出段的虚拟地址、文件范围和读写执行权限, 这是执行视图: 装载器只看它, 根本不关心 `.text` 叫什么名字。可重定位文件通常只有节表——它离执行还早; 可执行文件两份都有, 装载时只用段表。同一份字节, 链接器和装载器各取所需, 这正是”通用容器”的含义。

把这份容器按文件偏移画成竖向排布, 头部、两份目录与各节的位置关系便一目了然; `readelf` 的每个常用选项, 恰好对应图中的一块:



图示: ELF 文件按字节偏移的排布 (自上而下偏移增大)。ELF 头中的 `e_phoff` 与 `e_shoff` 分别给出段表与节表的文件偏移; `.bss` 是 NOBITS 类型, 节表里只登记大小, 文件里一个字节也不占, 图中用虚线框表示。右侧标出 `readelf` 各选项读取的部分: 同一份字节, 链接器取节表与符号表, 装载器只取段表。

最后要戳破一层神秘感: ELF 没有任何魔法, 它只是按约定排布的字节。文件自己并不”知道”节表在哪, 是工具按 `e_shoff` 找过去的; 头部里某个偏移写错, 读出来的就是垃圾, 没有类型系统替你兜底。本书反复出现的主题在这里再次出现: 软与硬之间、工具与文件之间, 一切协作都靠约定, 不靠神秘力量。

常见节的分工 打开任意一个目标文件的节表, 总能见到几张熟面孔。它们的分工不是编译器的发明, 而是把”用途与权限相同”的字节归在一起: 用途决定放在哪, 权限决定内存页怎么保护。

`.text` 装指令字节, 属性是分配、只读、可执行; `.rodata` 装只读常量——字符串字面量、`const` 全局量、编译器为 `switch` 生成的跳转表, 属性是分配、只读; `.data`

装初值非零的全局变量和静态局部变量，文件里存的就是初值本身，属性是分配、可写；`.bss` 装未初始化或初值恰为零的全局量与静态量，属性同样是分配、可写，但它在文件里一个字节都不占。

节	装什么	占文件字节	内存权限
<code>.text</code>	指令编码	占	只读、可执行
<code>.rodata</code>	字符串、 <code>const</code> 常量、跳转表	占	只读
<code>.data</code>	初值非零的全局/静态变量	占，内容即初值	可读写
<code>.bss</code>	未初始化或初值为零的变量	不占	可读写，装载时清零

`.bss` 省空间的机制一句话就能说清：它的节头类型是 NOBITS，节表里只有名字、大小和对齐要求，没有对应的文件范围；装载器读到它时不复制任何字节，直接在内存里划出大小相等的一片并填零。这与装载器的既有动作正好合拍——内核给新映射提供的本来就是填零的页（第五章的页帧分配、第三节的装载过程都会再用到），所以 `.bss` 的清零几乎不花额外代价。

省多少可以算出来。假设程序里有一个 1 MiB 的未初始化静态数组：若放进 `.data`，目标文件要多存 1 MiB 内容，而这 1 MiB 全是零字节，运载的信息量却是零；放进 `.bss`，文件只多一行几十字节的节头记录，内存里照样得到 1 MiB 全零空间。文件的职责是保存非零信息；全零内容用“长度加填零”几个字就说完了。C 的规则也顺着这个机制设计：显式初始化为零的变量同样进 `.bss`——`int x = 0;` 与 `int x;` 同等待遇，只有初值非零才需要 `.data` 里的真实字节。

节的属性标志不是注释，它们会一路传到硬件。链接器按属性把节归并成段，段头里的读写执行标志最终被装载器转成页表项的权限位：于是 `.text` 所在页不可写、`.data` 所在页不可执行。这些保护不是操作系统额外的好意，而是目标文件里早就写明的属性，经链接、装载两步原样传给 MMU。第五章问页表项权限位的来历，答案有一半就在这里。

符号表是名字到位置的映射 节回答了“字节是什么”，还没回答“名字在哪”。`main.c` 里调用 `add`，编译器生成 `main.o` 时并不知道 `add` 的地址——它可能根本不在本文件里。目标文件的处理办法是：把每个名字连同它的状态登记进符号表 `.symtab`，名字与位置的对应关系全部走这张表。

符号表每个条目记录：名字（字符串存在 `.strtab`，条目里存偏移）、值（`st_value`：可重定位文件里是节内偏移，可执行文件里是虚拟地址）、大小、类型（函数 FUNC、数据 OBJECT）、绑定属性，以及“定义在哪个节”。最后一项是定义与引用的分界线：纯引用的条目这一栏是 UND（未定义），意思是“这个名字本文件要用，定义在别处”；有定义的条目这一栏是本文件某个节的编号。

绑定属性分三档。LOCAL 是文件内私有：`static` 修饰的函数和变量都是 LOCAL，不参与跨文件解析，别的文件有同名符号也互不干扰。GLOBAL 是默认档：普通函数与全局变量，在全程序范围内参与解析。WEAK 是可以商量档：弱定义遇到同名 GLOBAL 强定义时自动让位，没有强定义时才用它——库常用弱符号提供可替换的默认实现；弱引用更宽容，找不到定义也不报错，运行时取到 0，由程序自己判空。

类别	怎么产生	链接器怎么处理
LOCAL	<code>static</code> 函数/变量	不进全局解析，只供本文件与调试器
GLOBAL 定义	普通函数、全局变量	全程序只允许一个强定义
GLOBAL 引用 (UND)	调用或访问外部名字	必须在某个输入文件里找到定义

WEAK 定义 `__attribute__((weak))` 有强定义则让位，否则用它

链接器两个经典报错都来自这张表。未定义引用：扫完所有输入文件，某个 UND 条目始终没找到对应的 GLOBAL 定义，于是报告 `undefined reference to 'foo'`——这不是玄学，是符号表一次查询失败的回执。多重定义：同一名字出现两个 GLOBAL 强定义，链接器无法裁决哪个算数，只能拒绝。在 C 头文件里误定义非 `static`、非 `inline` 的变量，被多个源文件包含后各产生一个强定义，正是这个错误的头号来源。

把符号表看成“接口的物理形态”很有用：目标文件对外提供什么（GLOBAL 定义）、向外索取什么（UND 引用），表上一目了然。第二节的符号解析、第三节的动态链接，处理的始终是这三个问题：名字是谁、值是多少、定义在不在场。

用 `readelf/objdump` 观察真实目标文件 这些结构不必靠想象，工具可以直接翻开看。常用的四个用法：`readelf -h` 读头部，核对类型、机器、入口和两份目录的位置；`readelf -S` 列节表，看每个节的类型、偏移、大小和属性标志；`readelf -s` 列符号表；`objdump -d -M intel` 反汇编 `.text`，换 `-r` 则看重定位记录。拿一个最小源文件完整试一遍：

```
/* min.c */
int counter = 42;          /* 初值非零：进 .data    */
const char msg[] = "hi";   /* 只读常量：进 .rodata */
int stash;                 /* 未初始化：进 .bss    */

int bump(int x)            /* 函数体：进 .text     */
{
    counter += x;          /* 两条指令引用 counter */
    return counter;
}
```

用 `gcc -c -O1 -o min.o min.c` 编译，然后逐个窗口看进去。第一扇窗是头部，注意类型、机器、入口和两份目录的位置：

```
$ readelf -h min.o
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  Type:                                   REL (Relocatable file)
  Machine:                                Advanced Micro Devices X86-64
  Entry point address:                   0x0
  Start of program headers:              0 (bytes into file)
  Start of section headers:              472 (bytes into file)
  Size of this header:                   64 (bytes)
  Number of program headers:              0
  Number of section headers:              9
  Section header string table index:      8
```

魔数之后，`02` 表示 64 位、`01` 表示小端，与本机一致；类型 REL 说明这是半成品，所以入口地址是 0、程序头表个数也是 0——离执行还早，段表还没必要存在。第二扇窗是节表：

```
$ readelf -S min.o
There are 9 section headers, starting at offset 0x1d8:

Section Headers:
```

[Nr]	Name	Type	Address	Off	Size	ES	Flg	Lk	Inf	Al
[0]		NULL	0000000000000000	000000	000000	00		0	0	0
[1]	.text	PROGBITS	0000000000000000	000040	00000d	00	AX	0	0	1
[2]	.rela.text	RELA	0000000000000000	0001a8	000030	18	I	6	1	8
[3]	.data	PROGBITS	0000000000000000	000050	000004	00	WA	0	0	4
[4]	.bss	NOBITS	0000000000000000	000054	000004	00	WA	0	0	4
[5]	.rodata	PROGBITS	0000000000000000	000054	000003	00	A	0	0	1
[6]	.symtab	SYMTAB	0000000000000000	000058	0000f0	18		7	6	8
[7]	.strtab	STRTAB	0000000000000000	000148	00001e	00		0	0	1
[8]	.shstrtab	STRTAB	0000000000000000	000166	00003f	00		0	0	1

Flags: W (write), A (alloc), X (execute), I (info)

三处细节值得停下来看。第一，所有节的 Address 一栏全是 0——地址还不存在，这正是“半成品”最直观的迹象。第二，.bss 的文件偏移 (0x54) 与 .rodata 相同：NOBITS 不占文件字节，所以下一节从同一偏移接着排，而它的 4 字节大小照样登记在案，装载时按这个数划出内存。第三，.text 只有 13 字节——一个函数的全部代码就这么多，里面还留着两个待填的洞。第三扇窗是符号表：

```
$ readelf -s min.o
Symbol table '.symtab' contains 10 entries:
  Num:      Value              Size Type      Bind      Vis      Ndx Name
   0: 0000000000000000          0 NOTYPE   LOCAL    DEFAULT  UND
   1: 0000000000000000          0 FILE     LOCAL    DEFAULT  ABS min.c
   2: 0000000000000000          0 SECTION LOCAL    DEFAULT    1
   3: 0000000000000000          0 SECTION LOCAL    DEFAULT    3
   4: 0000000000000000          0 SECTION LOCAL    DEFAULT    4
   5: 0000000000000000          0 SECTION LOCAL    DEFAULT    5
   6: 0000000000000000        13 FUNC      GLOBAL   DEFAULT    1 bump
   7: 0000000000000000          4 OBJECT    GLOBAL   DEFAULT    3 counter
   8: 0000000000000000          4 OBJECT    GLOBAL   DEFAULT    4 stash
   9: 0000000000000000          3 OBJECT    GLOBAL   DEFAULT    5 msg
```

源码里的四个名字各就各位：bump 是 13 字节的 FUNC，定义在 1 号节 (.text)；counter 是 4 字节 OBJECT，在 3 号节 (.data)；stash 在 4 号节 (.bss)；msg 是 3 字节（连结尾的 '\TU\textbackslash 0'），在 5 号节 (.rodata)。Value 一栏全是 0：它们目前都只是节内偏移，三个变量恰好都位于各自节的起点。最后一扇窗看代码本身和那两个洞：

```
$ objdump -d -M intel min.o
Disassembly of section .text:

0000000000000000 <bump>:
   0: 01 3d 00 00 00 00    add  DWORD PTR [rip+0x0],edi    # 6
   6: 8b 05 00 00 00 00    mov  eax,DWORD PTR [rip+0x0]    # c
   c: c3                  ret

$ objdump -r min.o
RELOCATION RECORDS FOR [.text]:
OFFSET                TYPE                VALUE
0000000000000002  R_X86_64_PC32      counter-0x4
0000000000000008  R_X86_64_PC32      counter-0x4
```

两条指令都带 [rip+0x0] 形式的 4 字节位移字段，内容暂时是 0——这就是洞。重定位记录说清了两个洞的位置（偏移 2 和 8，各在操作码之后）、填法（R_X86_64_PC32，PC 相对 32 位）和目标（符号 counter，加数 -4）。注意 counter 就在本文件定义，照样要留洞：它在 .data，代码在 .text，两节的距离要等布局后才知道，怎么填正是第二节的主题。

一个最小函数的完整解剖 收尾前做一次最彻底的走查：把一个最小函数从源码

到字节、到符号、到重定位记录逐项对一遍。源码只有一行：

```
int add(int a, int b) { return a + b; }
```

gcc -O0 生成的汇编如下 (Intel 语法, 小写寄存器):

```
add:
    push    rbp
    mov     rbp, rsp
    mov     DWORD PTR [rbp-4], edi
    mov     DWORD PTR [rbp-8], esi
    mov     edx, DWORD PTR [rbp-4]
    mov     eax, DWORD PTR [rbp-8]
    add     eax, edx
    pop     rbp
    ret
```

这段代码汇编成 20 个字节。逐字节解剖如下, 每一条指令的编码都能用第四章的字段格式读出来:

偏移	字节	指令	编码要点
0x00	55	push rbp	单字节, 寄存器号 5 编在操作码低 3 位 (0x50+5)
0x01	48 89 e5	mov rbp, rsp	48 是 REX.W 前缀; ModRM=0xE5: mod=11 寄存器直寻, reg=100 是 rsp, rm=101 是 rbp
0x04	89 7d fc	mov [rbp-4], edi	ModRM=0x7D: mod=01 带 8 位位移, disp8=0xFC 即 -4, reg=111 是 edi
0x07	89 75 f8	mov [rbp-8], esi	同一格式, reg=110 是 esi, disp8=0xF8 即 -8
0x0a	8b 55 fc	mov edx, [rbp-4]	操作码 8B 方向相反: 内存到寄存器, reg=010 是 edx
0x0d	8b 45 f8	mov eax, [rbp-8]	reg=000 是 eax, 位移仍是 -8
0x10	01 d0	add eax, edx	ModRM=0xD0: rm=000 (eax) 是目的, reg=010 (edx) 是源
0x12	5d	pop rbp	单字节, 0x58+5
0x13	c3	ret	弹栈顶作为返回地址

每一列都能和第四章对上。ModRM 字节的 mod、reg、rm 三字段正是第四章“寻址方式加寄存器号”的同一套结构; `[rbp-4]` 里的 -4 就是编码格式中的 disp8 位移字段。参数与返回值的位置则来自调用约定——第四章“调用约定把寄存器和栈的责任写成明确约定”的 x86-64 版本: 头两个整型参数依次在 edi、esi, 返回值在 eax。还可以看到 -O0 的字面意思: 参数先原样存入栈、用时再取回, 20 个字节里真正做加法的只有 01 d0 两个字节。若开 -O1, 整个函数收缩成 8d 04 37 c3: lea eax, [rdi+rsi] 借用“基址加变址”的求和路径直接算出 a+b, 外加一条 ret——地址形成硬件被借来做算术, 这是第四章 SIB 求和器的又一次登场。

再看符号与重定位两栏的对应。符号表里这个函数产生一个条目: 名字 add, 类型 FUNC, 绑定 GLOBAL, 大小 20, Ndx 指向 .text, Value 为 0——函数正好位于本文件 .text 的起点, 最终虚拟地址要等链接器分配, 此刻只能记节内偏移。重定位记录一栏则是空的: 这个函数的每个操作数不是寄存器就是 rbp 相对的栈槽, 位移 -4、-8 在编译时就是定数, 整条指令流不引用任何名字, 自然没有要填的洞。这给出一条判据: 重定位记录只在指令里出现名字——外部函数、全局变量——时才产生。把函数改成带外部调用的版本, 记录立刻现身:

```
extern int twice(int);

int add_twice(int a, int b) { return twice(a + b); }
```

-01 生成的代码与字节如下（分号后是注释）：

```
add_twice:
    sub    rsp, 8           ; 48 83 ec 08  栈对齐到 16 字节
    lea    edi, [rdi+rsi]   ; 8d 3c 37    a+b 送入第一个参数
    call   twice            ; e8 00 00 00 00  <- 洞在这里
    add    rsp, 8           ; 48 83 c4 08
    ret                                ; c3
```

`call` 的 4 字节相对偏移在编译时未知——`twice` 在别的文件里，它的地址此刻不存在。汇编器把字段清零，并登记一条重定位记录：

```
$ objdump -r add_twice.o
RELOCATION RECORDS FOR [.text]:
OFFSET          TYPE          VALUE
0000000000000008 R_X86_64_PC32      twice-0x4
```

读法：`.text` 偏移 `0x8` 处（`e8` 操作码之后那 4 个字节）要按 PC 相对 32 位规则、用符号 `twice` 的地址修补，加数 -4。三个要素——改哪里、按什么规则、用谁的值——全部登记在案，但演算还做不了：S 尚未存在。填洞需要符号的值，而符号的值来自链接器的布局，这就是第二节的开场。

二、链接：重定位与地址形成

上一节的结论是：目标文件是半成品——字节齐备，地址未定，凡涉及名字的字段都留着洞，洞的位置和填法记在重定位记录里。本节看链接器怎样把若干半成品拼成一张完整的地址地图。先总览它做的三件事；再逐字段读重定位记录，把第一节那条 `call` 记录的修补演算做完；然后比较绝对重定位与 PC 相对重定位各自的适用场合；接着拆开静态库“按需求出”的抽取机制，弄清链接顺序为什么重要；最后回到第四章，说明链接器填的正是指令编码里的立即数与地址字段——链接不是编译的尾声，而是 ISA 地址形成的最后一步。

链接器做三件事 把链接器看成一台只做三件简单事的机器，它的行为就不再神秘。第一件，合并同类节：把各输入文件的 `.text` 依次接成输出的 `.text`，`.data`、`.rodata`、`.bss` 同样处理，再按读写执行属性把节归并成段。第二件，符号解析与地址分配：给输出的每个节定下虚拟地址，于是每个符号的值从“某节内偏移”换算成“节基址加偏移”的具体数；同时把各文件的符号表汇成一张全局表，为每个 UND 引用找到唯一的 GLOBAL 定义。第三件，重定位：按重定位记录逐条改写输出文件里的字节——填 `call` 的相对偏移、填全局变量的位移、填数据区的指针初值。

步骤	输入	产出	改文件字节吗
合并同类节 符号解析与定址	各 <code>.o</code> 的同名节 全部符号表	输出的节与段布局 全局符号表、各节基址	拼接拷贝 不改，只算数
重定位	重定位记录加符号值	填好的指令与数据	改，逐条修补

三件事里前两件是簿记，第三件才真正动字节；但改哪里、改多少，完全由前两件的结果决定。这就是“链接等于把多个半成品拼成一张完整地址地图”的准确含义：地图由第二件事算出，由第三件事誊进字节。次序也不能乱：只有合并与定址

完成，符号才有值；符号有了值，重定位的算术才能做。所以第一节那两个经典报错都发生在第二件事——名字查不到或重复定义，地图就画不出来，根本轮不到第三件事动手。

把三件事放到一对最小输入上看一遍数值。设链接器只收到 `main.o` 与 `add.o`：合并把 `main.o` 的 `.text(0x20 字节)` 铺在输出 `.text` 的基址 `0x401120` 处，`add.o` 的 `.text(20 字节, 即 0x14)` 紧随其后，从 `0x401140` 开始；数据节同样依次排布、各按对齐要求取整。定址随即给出符号值：`main = 0x401120`, `add = 0x401140`, `main.o` 符号表里那条 UND 的 `add` 就此有了定义；重定位拿着这两个值去填 `main.o` 里那条 `call` 留下的洞——具体演算正是下一个专题的内容。

工程上，布局不是只能听默认值。链接脚本可以指定每个节的位置、入口符号和段的对齐方式：第七章的裸机镜像把 `.text` 钉在 ROM 起始地址、把入口钉在复位向量，用的就是这套机制的手工模式；Linux 的默认链接脚本则把 `.text` 安排在 `0x400000` 附近的惯例地址。同一台机器，自动挡与手动挡并存；学完本节和第三节，可以回头把第七章那个链接脚本逐行再读一遍。

重定位记录怎么读 第一节末尾留下一条记录：`offset 0x8, type R_X86_64_PC32, symbol twice, addend -4`。现在把它读透。一条重定位记录回答三个问题：位置 (`offset`)——改输出文件里的哪几个字节；类型 (`type`)——按哪条公式算；符号与加数 (`symbol`、`addend`)——用谁的地址、再做什么修正。x86-64 的 ELF 用 RELA 格式，加数显式存在记录里，被修补字段的旧内容直接丢弃不看。

顺带一提，32 位 x86 的 ELF 用的是 REL 格式：记录里没有加数字段，加数藏在被修补字段的旧内容里，链接器要先读出旧值再代入计算。RELA 把加数显式化，字段旧内容一律作废，逻辑更直白。这也解释了观察重定位时的一个现象：在 RELA 文件里，洞里的旧字节是什么都无所谓，汇编器统一清零——上一节目标文件里那些 `00 00 00 00` 的位移字段，正是这个约定的样子。

最常用的两种类型对应两条公式。绝对 32 位重定位 (`R_X86_64_32`)：往字段写入 `S+A` 的低 32 位，`S` 是符号的最终地址，`A` 是加数——填的是地址本身。PC 相对 32 位重定位 (`R_X86_64_PC32`)：写入 `S+A-P`，`P` 是被修补字段自己的最终地址——填的是字段到目标的距离。下面把一条 `call` 的修补从头算到尾。设 `main.o` 里有一处调用 `add` (定义在另一个文件)，用 `objdump -dr` 可以把指令和记录放在一起看：

```
$ objdump -dr -M intel main.o
Disassembly of section .text:

0000000000000000 <main>:
...
10: e8 00 00 00 00      call    15 <main+0x15>
                        11: R_X86_64_PC32 add-0x4
...

```

链接器做完合并与定址后：`main` 所在 `.text` 块的基址定为 `0x401120`，`add` 定在 `0x401140`。于是三个数全部就位：被修补字段的地址 `P = 0x401120 + 0x11 = 0x401131`；符号值 `S = 0x401140`；加数 `A = -4`。代入公式，并站在 CPU 执行语义一侧核对结果：

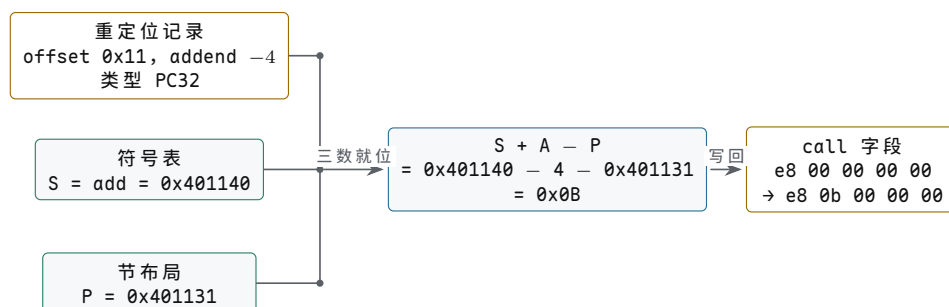
```
写入值 = S + A - P
        = 0x401140 + (-4) - 0x401131
        = 0x401140 - 0x401135
        = 0x0000000B

核对 (CPU 执行 call 的语义)：
目标 = 下一条指令地址 + rel32
      = (0x401131 + 4) + 0xB
      = 0x401135 + 0xB
      = 0x401140 = add    对上了

```

修补后，该处字节从 `e8 00 00 00 00` 变成 `e8 0b 00 00 00`—`0xB` 按小端排布，低字节在前，这正是链接器必须通晓 ISA 字节序与字段位置的原因。加数为什么是 `-4` 也值得说透：CPU 求 `call` 目标时的基准是下一条指令的地址，即字段末尾之后；公式里的 `P` 却是字段的开头，两者恰好差 4 个字节，汇编器把这 4 字节折进加数里补齐。整个演算没有一个新数：`S` 来自符号表，`P` 来自节布局，`A` 来自记录本身，公式来自 ISA 对 `call` 语义的定义。四者各就各位，填洞就是一次减法。

把这条演算画成修补流水线，四个数各就各位的动作就是四步：



图示：重定位修补流水线（对应正文 `main.o` 调用 `add` 的演算）。链接器读出记录的位置、类型与加数，从符号表取 `S`、从节布局取 `P`，按类型公式代入：`PC32` 写入 `S+A-P = 0x401140 - 4 - 0x401131 = 0x0B`；结果按小端排布写回字段，`e8 00 00 00 00` 变成 `e8 0b 00 00 00`。`RELA` 格式下字段旧内容直接丢弃，整个修补就是一次减法加一次写字节。

绝对重定位与 PC 相对重定位 两种类型为什么并存？因为它们服务的字段性质不同。`PC` 相对重定位填距离，适合代码内部的控制转移和就近访问的数据：`call`、`jmp`、条件转移，以及 `x86-64` 的 `RIP` 相对数据寻址，字段 32 位，够到前后各 2 GiB。绝对重定位填地址本身，适合数据区里的指针：函数指针数组、`switch` 的跳转表、`char *p = msg`；这类全局指针的初值，以及 `movabs` 的 64 位立即数字段——这些场合程序要的是值，不是距离。

差别要到镜像搬动时才显形。绝对字段把装载地址烧进了字节：镜像每换一个装载位置，所有绝对字段都得按新基址重填一遍——这正是第三节要讲的装载时重定位，也是 `ASLR` 每次随机布局都必须付的代价。`PC` 相对字段只记距离：整段代码连同它引用的目标一起搬，距离不变，字节原样可用，一处都不用改。

看一个绝对重定位的具体形状。`switch` 语句分支密集时，编译器常在 `.rodata` 里生成跳转表：每个表项是 8 字节的绝对地址，指向某个分支的代码。目标文件里这些表项先是零，旁边一一对应 `R_X86_64_64` 记录；链接时按公式 `S+A` 填入各分支标签的最终地址。一旦镜像换了装载基址，整张表每一项都要加上同一个差值——表有多大，重填的工作量就有多大，绝对字段的代价在此现出原形。

条件转移则是 `PC` 相对的纯血用户：`x86-64` 的 `jz`、`jne` 只有 `rel8` 与 `rel32` 两种偏移形式，编码里根本没有放绝对目标的位置，想跳远只能先跳到附近再做间接转移。控制流指令因此天生位置无关，代码段整体搬移时，从来不必为它们操心。

这就是位置无关代码（`PIC`）的雏形。共享库想被多个进程映射到各自不同的虚拟地址、又共享同一批物理代码页，前提是 `.text` 里几乎没有绝对字段：代码内部的转移天然是 `PC` 相对的，剩下绕不开的绝对访问——调外部函数、取外部数据——全部改道两张间接表（`GOT` 与 `PLT`），需要按装载地址修补的绝对字段集中收进这两张表里，`.text` 保持干净。编译共享库时加 `-fPIC` 的本质，就是把绝对重定位从代码节里赶出去；两张表怎么工作，是第三节的内容。

类型	写入内容	典型场合	镜像整体搬动后
<code>R_X86_64_32 / 64</code>	<code>S+A</code> ，地址本身	数据指针、跳转表、 <code>movabs</code>	所有绝对字段要重填

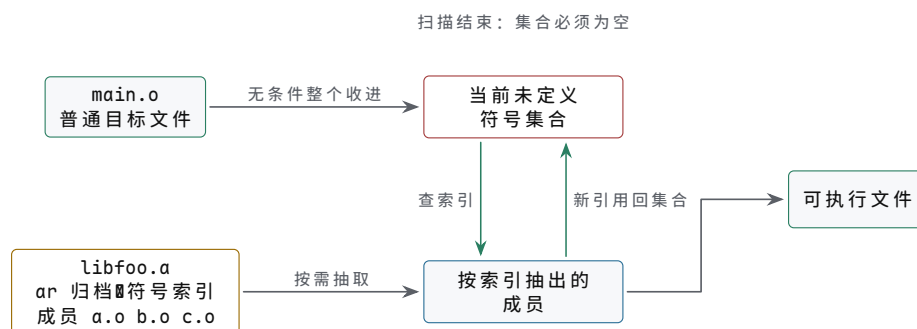
R_X86_64_PC32 S+A-P, 与字段的距离 `call/jmp`、RIP 相 字节原样可用
对访存

回头看历史能加深理解：x86-64 把 RIP 相对寻址设为默认的数据寻址方式，正是操作系统全面转向 PIC 之后 ISA 做出的配合。第四章讲寻址方式时给过这一模式的硬件结构，现在能看到它另一半成因在软件侧：链接器和装载器喜欢距离，讨厌地址本身。ISA 与系统软件在这里是互相塑造的。

静态库是按需求出的目标文件包 静态库 `libfoo.a` 不是新的文件格式：用 `ar` 把一批 `.o` 捆成一个归档，再附一张符号索引（`ar` 的 `s` 选项，旧系统里由 `ranlib` 生成），索引记录哪个名字在哪个成员里，仅此而已。库成员仍是普普通通的可重定位文件，链接器对它们的处理也和对普通 `.o` 一样——只多一条抽取规则。

抽取规则是理解静态库的全部关键。链接器从左到右扫描命令行上的输入，沿途维护一个“当前未定义符号”集合：扫到普通 `.o`，无条件整个收进，它定义的符号进全局表，它引用的符号进未定义集合；扫到库，则翻看索引，只把能消解当前未定义符号的成员抽出来——被抽出的成员自己又可能带来新引用，于是对同一个库要反复查，直到不再有新成员被抽出。扫描结束时，未定义集合必须为空，否则报错。

把这条抽取规则画成数据流，“用多少、取多少”就是中间那个集合的进出平衡：



图示：静态库的按需抽取。链接器从左到右扫描输入：普通目标文件无条件整个收进，它引用的符号进未定义集合；扫到库时翻符号索引，只把能消解当前未定义符号的成员抽出；被抽成员若带来新引用，就对同一个库再查一遍，直到没有新成员。扫描结束时集合必须为空，否则报未定义引用——库因此必须排在命令行上使用它的文件之后。

顺序因此有了意义：库必须出现在使用它的文件之后。`gcc main.o -lfoo` 里，扫到库时 `main.o` 的引用已在未定义集合里等着，成员被正常抽出；`gcc -lfoo main.o` 里，扫到库时集合还是空的，一个成员也不抽；等 `main.o` 把引用放进来，库已经扫过，链接器不会回头。

连锁抽取值得再看一眼：成员甲被抽出后又引用了成员乙定义的符号，乙也得跟着出库，所以库内部的依赖会被引用链自动拉齐。真正的麻烦在库与库互相引用：`liba.a` 需要 `libb.a`，`libb.a` 又回头需要 `liba.a`，单向扫描无论谁先谁后都可能漏抽。`ld` 为此准备了 `--start-group` 与 `--end-group`：把互相循环的库圈进一组，链接器在组内反复扫描，直到没有新成员被抽出。日常遇到的顺序问题多半不是循环依赖，而是单纯把库摆错了位置。

```

$ gcc -c main.c helper.c
$ ar rcs libhelper.a helper.o
$ gcc main.o -L. -lhelper -o app      # 正确：库在使用者之后
$ gcc -L. -lhelper main.o -o app     # 经典错误：库被白白扫过
/usr/bin/ld: main.o: in function 'main':
main.c:(.text+0x1f): undefined reference to 'helper'
collect2: error: ld returned 1 exit status
  
```

第一节那两个经典错误，在这里能看到完整成因。未定义引用：扫描结束而未定义集合非空——可能忘了给库，可能顺序错了，也可能库里确实没有这个名字；三重原因共用一个症状，排查时就按这三条挨个问。多重定义：两个输入各交出一个同

名强定义，链接器无法裁决。边界情形也值得一记：两个弱定义不报错，链接器任取一个；一强一弱取强——这条规则正是库用弱符号提供可替换默认实现的依据。

“用多少、取多少”的抽取粒度带来两个直接的工程后果。其一，只引用库里的一个函数就只抽一个成员，所以 `libc.a` 这类库把每个函数单独成文，静态链接出的可执行文件才不会把整个库背在身上。其二，抽取沿引用链传递：抽出的成员又引用别的成员会连锁抽取，所以被依赖的底层库（如 `-lm`）要放在命令行更靠后的位置。第三节会看到，动态库换了一套哲学：整个库先映射进内存再说，抽取问题让位给绑定问题。

与第四章 ISA 寻址方式的关系 现在把镜头拉回第四章。那里讲指令编码时反复出现两类字段：立即数字段和地址/偏移字段——教学 ISA 里 `JZ +3`、`JMP -5` 的偏移，x86-64 里 `call` 的 `rel32`、RIP 相对寻址的 `disp32`、`movabs` 的 `imm64`。当时这些字段的值都由汇编器顺手算出；本章能看到它们的另一半来历：凡字段里出现名字，值就要等链接器。链接器填的正是第四章编码格式里那些 `imm` 与地址字段，一分不多，一分不少。

两处约定遥相呼应。第四章的教学 ISA 规定“跳转目标等于本条地址加 1 加偏移”，x86-64 规定“目标等于下一条指令地址加 `rel32`”——同一个“相对下一条指令”的约定，一个定长一个变长而已。本节的公式 $S+A-P$ 就是这个约定的代数形式：式中每个量都对应编码格式里一个具体的字段位置。学会读重定位记录，等于把第四章的指令编码又读了一遍，只是这次手里拿的是链接器的视角。

还有一个细节能巩固这层对应。第四章给教学 ISA 定编码时，把立即数与偏移一律向低位对齐，理由是扩展逻辑单一；链接器面对的正是同一批低位字段——`rel32` 从指令的固定位开始占 4 字节，链接器只改写这 4 字节，绝不碰操作码与 `ModRM`。字段的位置由 ISA 定死，字段的值由链接器算定，双方各守一半约定，拼起来才是一条完整的指令。

更完整地说，地址形成是一条跨越四个时刻的流水线，每个时刻各填各的字段：

时刻	谁定值	定什么
编译时	编译器/汇编器	字段的位置与格式、寄存器与寻址方式、重定位记录
链接时	链接器	静态符号的地址： <code>call</code> 偏移、全局量位移、数据指针初值
装载时	装载器/动态链接器	运行前才定的地址：库基址、GOT 条目（第三节）
运行时	CPU 硬件	有效地址求和：基址 + 变址 × 比例 + 位移

链接器坐在这条流水线的倒数第二站：它不管语法规义（那是编译前端的事），也不生成指令模板（那是编译后端的事），只管“名字到地址”的最后定值。说链接是 ISA 地址形成的最后一步，准确含义就在于此——第四章数据通路里那个“基址加变址乘比例加位移”的求和器，它的位移输入常常就是链接器在磁盘上写下的那个数。裸机世界同样成立：第七章的镜像没有操作系统，链接脚本把节钉在物理地址上，重定位照算不误，`S` 与 `P` 只是换成了手工指定的值。从教学板到 Linux，这套机制是同一件事的两种装法。

读懂这一层，链接错误就不再神秘：它是地址形成流水线倒数第二站的报错——某个名字走到这一站还没有地址，或者有两个地址。下一节进入最后一站：装载器把这张填好的地图铺进内存。

三、装载与动态链接

上一节结束时，磁盘上已经躺着一个地址全部定好的可执行文件：代码在 `0x401000` 附近，数据在 `0x403dd8` 附近，入口点写在文件头里。但它此刻还只是一串字节，离“正在运行的进程”还差最后一步，这一步由内核完成：`execve` 把文件变成进

程，动态链接器把库接进来。本节回答两件事：内核装载时做了什么，以及为什么其中相当大的一部分被推迟到“第一次用到”的那一刻。

先立一个总观点：装载不是搬运。直觉上“把程序装进内存”像一次大块复制——把文件读进 RAM，再跳过去执行；第七章的面包板机把程序逐字节拨进 RAM，就是这种字面意义的装载。现代内核的做法不同：它只登记映射——“这段虚拟地址对应那个文件的这段字节”——真正的字节要等程序第一次访问那一页时，才由缺页异常从磁盘取回。把**装载理解成建立映射而不是复制字节**，按需调页、写时复制、共享库就都是同一条逻辑的推论。

本节仍以 x86-64 Linux 上的 ELF 为对象，沿用第四章的异常骨架、第六章的分页硬件（CR3、页表项、U/S 位）与页错误诊断法；结尾回到第七章的裸机对照，看看没有这条边界时一切是什么样子。

execve 之后内核做的事 `execve` 是少数“成功后不返回”的系统调用：成功时原进程的地址空间被整个换掉，等它“返回”到用户态时，运行的已经是另一个程序。内核按固定顺序做四件事：读 ELF 头部与段表、按段表建立映射、准备栈、跳到入口。下面把它做完一遍。

第一件事，读头部。内核核对 ELF 魔数与机器类型，确认文件类型（EXEC 或 DYN），取出入口地址和程序头表（段表）。段表里的每个 `LOAD` 段回答一个问题：文件的哪一段字节，要以什么权限，出现在虚拟地址的哪一段。用 `readelf` 看一个典型小程序（节选 `LOAD` 段；数值为模拟，格式与真实输出一致）：

```
$ readelf -l hello
Elf file type is EXEC (Executable file)
Entry point 0x401040
There are 11 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz             MemSiz             Flags  Align
LOAD             0x0000000000000000 0x0000000000400000 0x0000000000400000
                 0x00000000000004f0 0x00000000000004f0 R       0x1000
LOAD             0x0000000000000100 0x0000000000401000 0x0000000000401000
                 0x0000000000000195 0x0000000000000195 R E     0x1000
LOAD             0x0000000000000200 0x0000000000402000 0x0000000000402000
                 0x0000000000000110 0x0000000000000110 R       0x1000
LOAD             0x00000000000002dd8 0x0000000000403dd8 0x0000000000403dd8
                 0x0000000000000248 0x0000000000000250 RW      0x1000
```

第二件事，按段表建映射。把四个 `LOAD` 段逐段算账。第一段从文件偏移 0 映到 `0x400000`，ELF 头和程序头表都在其中。第二段是代码：文件区间 `[0x1000, 0x1195)` 映到 `[0x401000, 0x401195)`，`0x195` 即 405 字节，不足一页；内核按整页映射，页内超出段长的字节来自文件同页的对齐填充。入口 `0x401040` 就在这段内偏移 `0x40` 处。第三段是只读数据。第四段是可写数据：`MemSiz` 比 `FileSiz` 多出 8 字节，多出的部分就是 `.bss`——文件里不占字节，内存里必须是零；内核映完文件部分后把这 8 字节清零，若 `.bss` 大到跨页，整页挂匿名零页。

段	文件偏移 → 虚拟地址	字节数（文件 → 内存）	权限与内容
1	<code>0x0000→0x400000</code>	<code>0x4f0→0x4f0</code>	只读：ELF 头、段表、 <code>.interp</code>
2	<code>0x1000→0x401000</code>	<code>0x195→0x195</code>	读/执行： <code>.text</code> ，入口在其中
3	<code>0x2000→0x402000</code>	<code>0x110→0x110</code>	只读： <code>.rodata</code>

4 0x2dd8→0x403dd8 0x248→0x250 读写：.data 与 .bss，多出 8 字节清零

每个 LOAD 段还满足一条同余：虚拟地址与文件偏移模页长同余——0x403dd8 与 0x2dd8 模 0x1000 都余 0xdd8。这不是巧合：只有页内偏移一致，一个文件页才能原样映到一个虚拟页上，内核不必做任何搬移对齐。上一节的链接器在定址时就按这条约束摆段：链接不仅决定“放在哪个地址”，还决定“按页怎样对齐”。

映射在内核里的形态是一张映射表（vm_area_struct 的集合），每条记录由段表折算而来。注意这张表是登记簿而非内容：此时程序的一个字节都还没进内存。

字段	含义	本例第四段的值
起止地址	虚拟地址区间	[0x403dd8, 0x404028)
权限	读/写/执行	读、写
来源	文件加偏移，或匿名	hello 偏移 0x2dd8
补齐规则	段尾不足一页如何处理	超出 FileSiz 的部分清零

第三件事，准备栈。内核把命令行参数和环境变量的字符串复制到新地址空间的栈顶，再向下摆辅助向量、envp 和 argv 指针数组，最后把 rsp 指向 argc：

```
high  strings: "./hello\0" "world\0" "PATH=/usr/bin\0" ...
      alignment padding; rsp ends 16-byte aligned
      AT_NULL
      auxv pairs: AT_PAGESZ=4096, AT_ENTRY=0x401040,
                  AT_PHDR=0x400040, ...
      envp: NULL, ptr -> envp strings ...
      argv: NULL, ptr -> "world", ptr -> "./hello"
low   rsp -> argc = 2
```

辅助向量（auxiliary vector）是内核留给用户态的小抄：AT_PAGESZ 告诉 libc 页长是 4096；AT_PHDR 告诉动态链接器段表映在哪里——0x400040 的来历很直接，ELF 头长 64 字节（0x40），程序头表紧跟其后，第一段又从 0x400000 开始映射，所以表在 0x400040；AT_ENTRY 就是文件头里的入口 0x401040。argc/argv/envp 的摆法则与第四章的栈约定一脉相承：指针数组以 NULL 结尾，字符串本体在栈顶，rsp 保持 16 字节对齐。

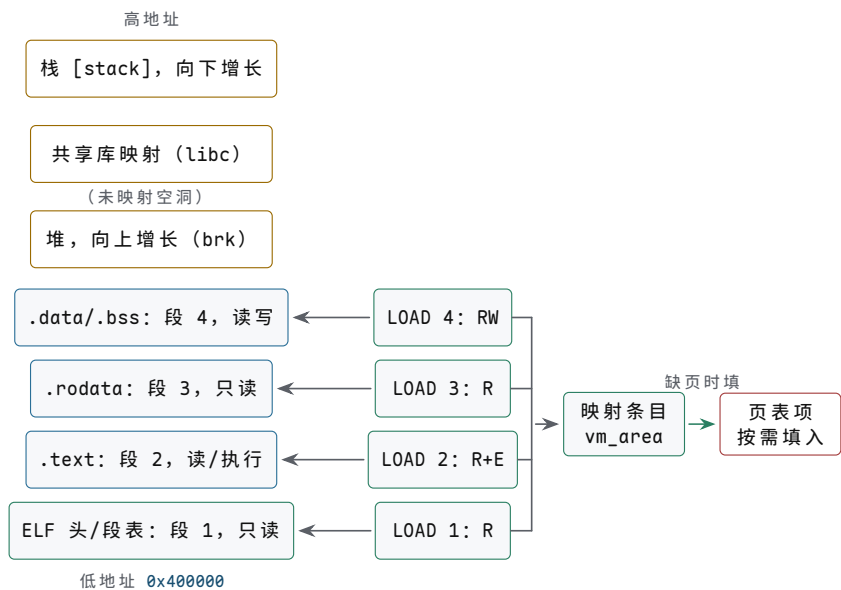
第四件事，跳到入口。静态链接程序的 rip 直接设为入口地址；动态链接程序则先映射 .interp 段指定的动态链接器 /lib64/ld-linux-x86-64.so.2，rip 指向它的入口，由它装载共享库、做完装载时重定位，最后才跳到 0x401040——本节第四、五个主题就发生在它身上。

装载完成后，这套映射从用户态可以直接核对：/proc 把它原样摆出来。逐项对照段表，能看到每个 LOAD 段变成一行，以及段表里没有的 libc 和栈：

```
$ cat /proc/742/maps
00400000-00401000 r--p 00000000 08:01 131073      /home/user/hello
00401000-00402000 r-xp 00001000 08:01 131073      /home/user/hello
00402000-00403000 r--p 00002000 08:01 131073      /home/user/hello
00403000-00404000 r--p 00002000 08:01 131073      /home/user/hello
00404000-00405000 rw-p 00003000 08:01 131073      /home/user/hello
7f9a3c090000-7f9a3c22c000 r-xp 00000000 08:01 262146  /lib/x86_64-linux-gnu/libc.
so.6
7fffffffde000-7fffffffef000 rw-p 00000000 00:00 0      [stack]
```

第四段的兑现方式值得多看一眼：它跨两页，第一页 0x403000 映的是文件页 0x2000——同一个文件页也映在第三段的 0x402000，两处只读映射共享同一个物理页；这页里混着 .rodata 的尾巴和 .data 的头，所以先按只读挂上，写入时再由写时复制拆开。GOT 所在的 0x404000 才是可写页。映射表把“一页一个权限、一页

一个来源”表达得清清楚楚。
把这套映射从低地址到高地址画成一张布局图，段表到页表的折算链放在右侧：



图示：进程地址空间从低地址到高地址的布局（对应正文 `hello` 例）：主程序四个 `LOAD` 段从 `0x400000` 起依次排开，堆紧随其后向上增长，共享库映在 `0x7f...` 区间，栈在最高处向下增长，各区之间隔着未映射的空洞。右侧是同一张地图的折算链：装载时内核把 `LOAD` 段折算成映射条目登记下来，页表项则等某页第一次被访问、触发缺页时才填入——装载只建映射，不读字节。

顺带把四件事摆成一张表，作为本主题的总览：

步骤	内核动作	可观察处
读头部	核对魔数与类型，取入口地址与段表	<code>readelf -h</code> 、 <code>readelf -l</code>
建映射	按 <code>LOAD</code> 段登记映射，不读字节	<code>/proc/PID/maps</code>
准备栈	复制字符串，摆 <code>argc/argv/envp/auxv</code>	调试器里查看栈顶
跳入口	<code>rip</code> 指向动态链接器或入口地址	<code>strace</code> 中 <code>execve</code> 后的 <code>mmap</code>

还有两张表容易混淆，要分清：段表（程序头表）是给装载器看的，节表是给链接器和调试器看的。`strip` 把节表整个剥掉，程序照样运行——装载只需要段表；反过来，段表若损坏，内核连映射都建不起来。此外 `execve` 有一类“不换”的东西：文件描述符、进程号、工作目录跨 `exec` 保留，地址空间全换而它们不换——`shell` 的重定向正是靠“`fork` 之后、`exec` 之前先换描述符”实现的。

工程含义只有一句：`execve` 本身很快，因为它几乎不搬数据。等 `rip` 指向 `0x401040` 的第一次取指触发缺页，程序的第一页才真正进内存。把装载从搬运改成建映射，是本节后面全部机制的地基。

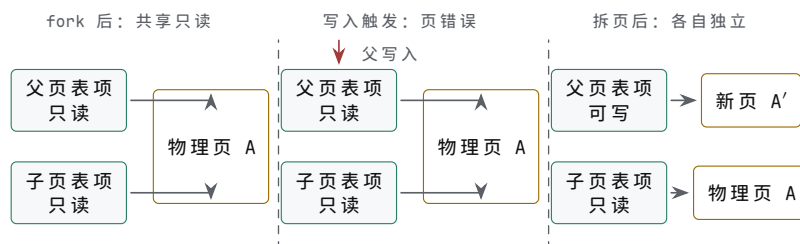
补一个反例帮助记忆：`execve` 失败时（文件不存在、格式不对、权限不足）正常返回 `-1`，原进程的地址空间分毫不动——四件事一件都没做。“成功后不返回”因此也可以反着读：只要它返回了，就一定没成功。

把装载想成“把程序读进内存”是最常见的误判。按这个模型，`fork` 之后 `exec` 前要复制全部分页、程序越大启动越慢、共享库无从谈起——每一条都与观察相反。修正办法是把单位想清楚：内核登记的对象是映射条目和页表项，不是字节；字节只在被访问到时才流动。

写时复制与按需调页 `fork` 的语义是复制整个地址空间，而几乎每个 `shell` 命令的出生都是 `fork` 紧跟 `exec`—若真复制，这将是全系统最昂贵的动作之一。两个机制把它变便宜，它们共用同一个驱动器：第六章的页错误硬件。

写时复制 (`copy-on-write`)：`fork` 时内核不复制任何页，只把父进程所有私有可写页在双方页表里都改成只读，父子页表指向同一批物理页。此后任何一方写入都会触发页错误—用第六章的三问来读：页存在 ($P=1$)、是写访问、权限不允许写—内核据此知道这不是真错误，而是该复制了：分配一个新页，把旧页内容抄过去，写入方的页表项指向新页并恢复可写；若某页只剩一个引用方，直接改回可写，连复制都省。读访问永远不触发复制。

把 `fork` 前后的页表画成三格，写时复制的全部动作就在其中：



图示：写时复制的三幕。`fork` 后：父子页表项指向同一批物理页，双方都标只读，复制量为零。写入触发：任何一方写共享页都会撞上只读权限，页错误硬件把这次写截住，内核据此知道该拆页了。拆页后：内核分配新页、抄好内容，写入方的页表项改指新页并恢复可写，另一方仍指旧页—复制只发生在被写的那一页上。

算一笔账：父进程有 5120 个可写私有页，共 $5120 \times 4 \text{ KiB} = 20 \text{ MiB}$ ；立即复制要搬 20 MiB 并改 5120 条页表项，写时复制只改 5120 条页表项。若子进程在 `exec` 前只写 200 页，实际复制 $200 \times 4 \text{ KiB} = 800 \text{ KiB}$ ，约为 20 MiB 的 4%；而 `fork` 后立刻 `execve` 的场合，子进程写的页屈指可数，复制量几乎为零。

历史上这条路走了三段：早期 Unix 的 `fork` 真的逐页复制，代价随地址空间线性增长；BSD 曾引入 `vfork` 作为权宜—子进程借用父进程的地址空间直到 `exec`，父进程全程冻结，语义脆弱；写时复制普及之后，`vfork` 这类绕道基本退场，`fork` 回到“语义最贵、实现最便宜”的状态。

按需调页 (`demand paging`) 是同一思路在装载一侧的应用：上一主题说装载只登记映射，映射的兑现靠缺页。CPU 访问一个存在位为 0 的页，触发页错误，CR2 保存故障地址；内核查映射表，发现该地址属于某文件的某段，就分配物理页、从磁盘读入对应文件页、填好页表项，再重新执行被中断的那条指令。对程序完全透明：它只看到访问成功，只是这一次慢了若干数量级。

再算一笔账：一个 200 MiB 映像共 $200 \text{ MiB} \div 4 \text{ KiB} = 51200$ 页的程序，启动到 `main` 实际触及约 500 页，即 $500 \times 4096 = 2048000$ 字节，约 2 MiB—整个映像的 1%。这就是大程序启动快的原因：启动成本与实际触及的页数成正比，与文件总大小几乎无关。没碰的页不读，几百 MiB 的程序也能在一秒内开窗。

缺页的三种结局值得并排看，因为它们是同一条检查链的三个出口：

结局	内核的发现与动作	程序看到什么
文件页	地址在某文件映射内，从文件读入该页	访问成功，仅这一次慢
匿名零页	地址在 <code>.bss</code> 、堆或栈映射内，挂零页	访问成功，读到 0
非法访问	地址不属于任何映射	收到 SIGSEGV（第四节）

按需调页还有一个对偶动作：换出。物理页紧张时，内核把久未访问的匿名页写到交换区，把对应页表项标为不存在；下次访问触发缺页，再把它读回来。于是“页错误”成了统一的内存入口—文件页、零页、交换页都从它进来，虚拟内存因此可以大于物理内存。每个物理页上的引用计数则回答另一个问题：这页被几方共享，写时复制要不要真拆。

方案	复制发生在何时	复制多少	页表改动
立即复制	fork 当场	全部可写页	新建整套
写时复制	第一次写某页	仅被写的页	双方先改只读
按需调页	第一次访问某页	仅被访问的页	缺页时填一条

缺页成本可以按页的来源分档，这也是 `/usr/bin/time -v` 里两类缺页计数的含义：

类型	页从哪来	典型量级
次要缺页 (minor)	页已在内存缓存，只需填页表项	亚微秒
主要缺页 (major)	从磁盘读入文件页	几十微秒 (SSD)
换入	从交换区读回	几十到几百微秒

```
$ /usr/bin/time -v ./hello
hello, world
...
Minor (reclaiming a frame) page faults: 152
Major (requiring I/O) page faults: 3
```

内核还会顺势多走一步：缺页读盘时通常顺带读入后续若干页（预读），因为程序访问文件页往往有明显的顺序性——用一次磁盘事务换掉后面的若干次缺页，这正是第五章“突发传输比单拍省事务”的同一思想。观察侧的对应概念是常驻集 (RSS)：进程实际占着的物理页数。一个 200 MiB 映像的程序启动后 RSS 只有几 MiB，这个差值就是按需调页最直白的观察。

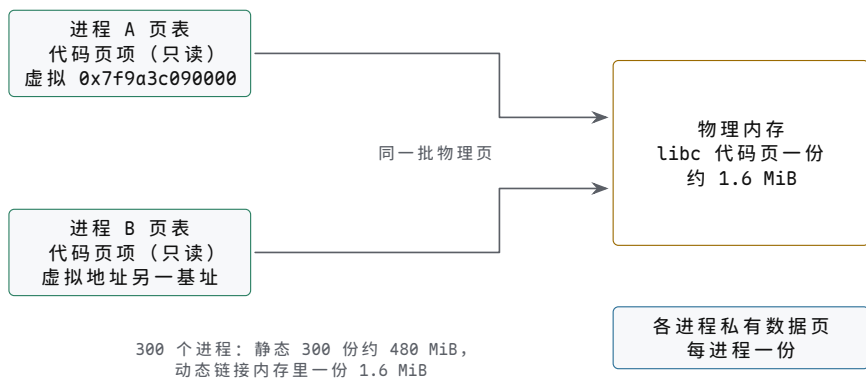
两个机制是同一骨架的两端：都把工作推迟到第一次真正需要时，都由页错误驱动。工程推论同样直接：缺页对程序不可见但真实耗时——磁盘取页约几十微秒，页仍在内存缓存则亚微秒——于是有了预热、内存锁定、大页这类“提前把页备好”的手段；一张 2 MiB 大页把 $2 \text{ MiB} \div 4 \text{ KiB} = 512$ 次缺页合并成 1 次，第十一章谈延迟与队列的视角在这里同样适用。

动态库的优势与代价 上一节的链接把全部代码装订进可执行文件，那是静态链接。动态链接把一部分装订推迟到装载时：可执行文件里只记录“我需要 `libc.so.6` 里的 `printf`”，真正的代码躺在系统里所有进程共享的库文件中。用 `readelf -d` 能看到这份“我需要谁”的清单（动态段，输出为模拟节选）：

```
$ readelf -d hello | head -4
Dynamic section at offset 0x2dd8 contains 25 entries:
  Tag              Type              Name/Value
  0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
```

收益一，共享代码页：`libc` 的代码段在全部进程间共享一份物理页。算一笔账：`libc` 代码段约 1.6 MiB（上面 `maps` 里的 `0x19c000` 字节），系统里 300 个进程都用它；静态链接意味着 300 份副本共约 480 MiB，动态链接意味着内存里一份 1.6 MiB（每进程另有少量私有数据页），差约 300 倍。共享成立的条件是代码页只读：谁也不写它，写时复制就不会把共享页拆成私有页。第五个主题会看到，这条约束反过来决定了位置无关代码的形状。

把“共享一份代码页”画成页表指向图，三百倍的差距就写在两组指针上：



图示：动态库的共享代码页。两个进程的页表各自把不同的虚拟地址映射到同一批 libc 物理页——代码段约 1.6 MiB，全系统只有一份；300 个进程时，静态链接要 300 份副本共约 480 MiB，动态链接只要一份（每进程另有少量私有数据页），差约 300 倍。共享成立的条件是页面只读：谁也不写它，写时复制就不会把共享页拆成私有页。

收益二，升级不重链：修复 libc 的一个缺陷，只需替换库文件本身，下次启动的程序自动用新版；静态链接则必须重新链接并重新分发每一个程序。安全补丁的分发成本因此从“重发所有程序”变成“重发一个库”。文件名里的“.6”是主版本号：它承诺二进制接口（ABI）不变，只有不兼容时才进位——libc 的主版本号已经二十多年停在 6，于是一个安全更新可以直接替换同名文件。

主版本号管不了的演化，由符号版本接着管：同一个 `printf` 在 libc 里带着版本标签 `GLIBC_2.2.5`，可执行文件记录的不仅是“要 `printf`”，还有“要哪个版本的 `printf`”。于是同一个 `libc.so.6` 可以在内部继续演化，老程序拿到的仍是当年那个语义的符号。

```
$ readelf -sW /lib/x86_64-linux-gnu/libc.so.6 | grep -w printf
1402: 000000000000645e0 137 FUNC GLOBAL DEFAULT 15 printf@GLIBC_2.2.5
```

还有一条更晚的定址时机值得一提：`dlopen/dlsym` 把“装载一个库”推迟到运行中的任意时刻，插件机制全靠它——浏览器加载扩展、文本编辑器加载语法插件，都是程序自己请动态链接器中途再接一个库进来。链接的三档时机（编译时、装载时、运行时）至此全部到齐。

代价同样明确。其一，装载时重定位：库内部对数据和函数的引用在基址确定之前是半成品，装载时要按“基址加偏移”把 GOT 等表项逐条填好，库越多表越长。其二，间接跳转：跨模块调用不能用直接 `call`，要经 PLT/GOT 间接一级，每次调用多一次内存间接。其三，依赖管理：“升级不重链”的另一面是“库变了程序没重编”，二进制接口兼容因此成为发行版甩不掉的长期负担。

方面	静态链接	动态链接
代码份数	每进程一份副本	全系统共享一份代码页
升级	重新链接并分发每个程序	替换库文件即可
启动前工作	无	映射库、按基址修补重定位表
调用开销	直接 <code>call</code>	PLT/GOT 间接一次
文件体积	大，自带全部代码	小，只记依赖名

顺带说明，静态链接并没有被淘汰：容器镜像和嵌入式固件反而偏爱它——一个自包含、零依赖的二进制，拷到任何机器上都能跑，这正是用磁盘体积换部署确定性。推到极限，第七章的裸机镜像就是“终极静态链接”：连操作系统都链在同一个映像里，没有任何运行期依赖可言。

PLT/GOT 的延迟绑定 动态链接的程序调用 `printf` 时，目标地址在链接时还不在本文档里。链接器的办法是两级间接：`call` 先到本模块内一小段固定桩（PLT 项），桩里是一条间接跳转，跳的目标写在 GOT 表项里；GOT 是可写数据，真实地址由装载时或运行时填入。这样代码段保持只读可共享，所有需要修补的地址集中在一小张表里。

为什么不干脆在装载时改写 `call` 指令的目标？因为那要写代码页：代码页一变脏，写时复制立刻把共享页拆成私有页，共享库和下一主题的 ASLR 一起失效。PLT/GOT 的全部存在理由就是八个字：代码不动，数据可改—代码段一个字节不动，所有可写的定址信息集中到数据段的一张表。

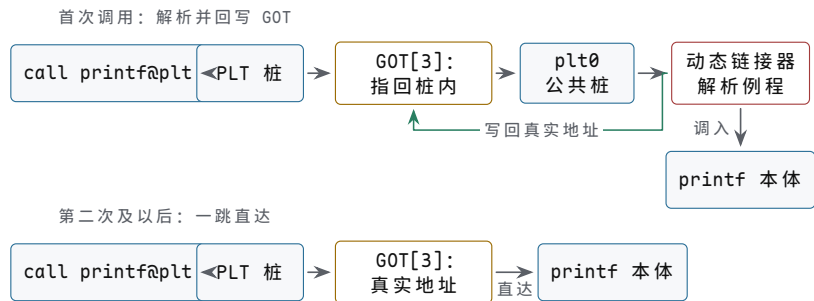
```
call printf@plt          ; call site in main

plt0:                    ; 0x401020
    push qword ptr [rip+0x2fe2] ; GOT[1] = link_map
    jmp  qword ptr [rip+0x2fe4] ; GOT[2] = _dl_runtime_resolve
printf@plt:               ; 0x401030
    jmp  qword ptr [rip+0x2fe2] ; GOT[3] at 0x404018
    push 0                  ; relocation index of printf
    jmp  plt0
```

把一次 `printf` 调用的两级跳走读一遍。第一级是静态的：`call printf@plt` 跳到 `0x401030`。第二级是动态的：桩里 `jmp qword ptr [rip+0x2fe2]` 是 `rip` 相对寻址，指令长 6 字节，下一条指令在 `0x401036`，所以表项地址是 `0x401036 + 0x2fe2 = 0x404018`，即 `GOT[3]`；`plt0` 的两条同理，分别指向 `GOT[1]` 与 `GOT[2]`，读者可自行核对 `0x401026 + 0x2fe2 = 0x404008`、`0x40102c + 0x2fe4 = 0x404010`。

第一次调用时，`GOT[3]` 里装的是 `0x401036`—桩里那条 `jmp` 的下一条指令自己，这次间接跳等于原地踏步；顺序执行 `push 0` (`printf` 的重定位下标) 和 `jmp plt0`，`plt0` 把 `link_map` 压栈后跳进动态链接器的解析例程。解析器按下标找到符号名 `printf`，在 `libc` 里查出真实地址（设为 `0x7f9a3c0f45e0`），写进 `GOT[3]`，再直接调进 `printf`。第二次调用只剩两条指令：`call` 进桩，`jmp [0x404018]` 直达 `0x7f9a3c0f45e0`，一次内存间接，没有任何解析。

把两次调用画成两条分开的路径，第一次的解析绕路与之后的一跳直达，差别一眼可见：



图示：PLT/GOT 两级间接的两条路径。首次调用时 GOT 表项指回桩内，流程经公共桩进入动态链接器的解析例程，查出真实地址后先写回 GOT 表项（虚线）再调入函数本体；此后每次调用只剩 `call` 进桩、桩内经 GOT 一次间接跳直达。代码段自始至终不被改写，可写的定址信息全部集中在 GOT 这张表里。

检查点	第一次调用	第二次及以后
GOT[3] 内容	0x401036（桩内下一条）	0x7f9a3c0f45e0（真实地址）
经过路径	桩 → plt0 → 解析器 → printf	桩 → printf
关键动作	查符号、写 GOT[3]	无
预期开销	一次符号查找，微秒级	一次间接跳，纳秒级

PLT 与 GOT 的分工可以列成一张对照：

对象	所在段	可写性	内容由谁何时填
PLT 桩	代码段	只读	链接器静态生成，运行期不变

GOT 表项	数据段	可写（见下）	动态链接器按基址与符号填
--------	-----	--------	--------------

GOT 的“可写”本身是一个攻击面：谁改写了 GOT[3]，谁就把下一次 `printf` 调用拐到自己选的地方。对策叫全 RELRO：装载时用 `LD_BIND_NOW` 把全部符号解析完，然后把 GOT 所在页改回只读——用启动时间换“运行期不可覆写”。这与上一个主题“代码页只读才能共享”是同一条纪律的延伸：凡是可被写的定址信息，要么推迟到必要时再写，要么写完立刻封死。

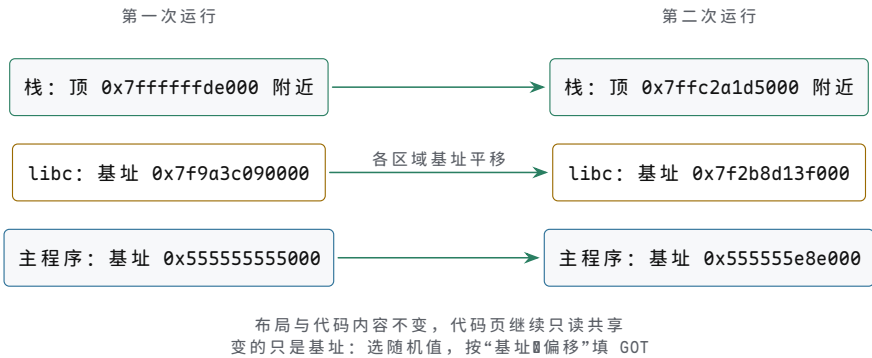
这种“第一次才解析”的做法叫懒绑定（lazy binding）。它的收益可以用数字说话：一个中型程序导入约 1200 个动态符号，典型启动路径只用其中约 150 个，懒绑定让启动时省掉约 1050 次符号查找——每次查找都要算哈希、比字符串、写表项。对短生命进程（脚本里一闪而过的每个命令）省得更多：大量导入函数整个生命期一次都没被调用。代价是第一次调用某个函数时有一次不可预测的解析延迟；实时系统可以设 `LD_BIND_NOW=1`，装载时把全部符号解析完，用启动时间换运行期的确定性。还要记住分工：桩和 GOT 的骨架、GOT 的初值，都是上一节的链接器在静态时就排好的局，动态链接器运行期只做“填表”这一件事。

ASLR 与装载时重定位 前面例子的地址全是固定的：代码在 `0x401000`，GOT 在 `0x404000`。固定地址对攻击者同样是固定的：栈溢出拿到 `rip` 控制权后，只要知道 `libc` 里 `system` 或某条指令片段的地址，就能拼接利用。地址空间布局随机化（ASLR）的对策是让每次运行的基址都不同：栈、堆、`mmap` 区和共享库各加一个随机偏移；编译成位置无关可执行文件（PIE）时，连主程序自己的基址也随机。同一个程序连跑两次：

<pre>\$./showaddr main = 0x555555555149 printf = 0x7f9a3c0f45e0 &buf = 0x7fffffffde2ac</pre>	<pre>\$./showaddr main = 0x5555555e8e149 printf = 0x7f2b8d1a35e0 &buf = 0x7ffc2a1d59ac</pre>
---	---

三个地址全变。换算成攻击者的处境：若某区域基址带 28 位随机位，一次猜中的概率是 $1/2^{28} = 1/268\,435\,456 \approx 3.7 \times 10^{-9}$ ；猜错通常意味着目标进程崩溃重来，而崩溃本身就on能暴露行踪。32 位时代有过反面教训：地址空间只有 4 GiB，随机位只有十几位，几千到几万次尝试量级就能爆破；64 位空间让随机位和猜测代价一起变大。ASLR 不是绝对防御——一次地址信息泄漏就能让它失效——但它把一大类利用的成本从“查一次表”抬高到“先泄漏、再猜、还不能错”。

把两次运行的地址空间并排画出，随机化的形状就是“布局不变、基址平移”：



图示：ASLR 两次运行对照（对应正文 `showaddr` 输出）。两次运行的区域顺序与相对布局完全相同，变的只是基址：主程序 `0x55555555000` 对 `0x5555555e000`，`libc` `0x7f9a3c090000` 对 `0x7f2b8d13f000`，栈顶 `0x7fffffffde000` 附近对 `0x7ffc2a1d5000` 附近。换基址不改代码一个字节——PIC 让模块内引用全是 `rip` 相对、跨模块引用全走 GOT，随机化的全部成本就是选基址加填那张可写的表。

区域	传统固定位置	ASLR 之后
----	--------	---------

主程序	0x400000 (-no-pie)	PIE 下随机基址
共享库与 mmap 区	固定在 0x7f... 某处	每次运行换基址
栈	0x7fffffff000 附近	栈顶加随机偏移
堆 (brk)	紧接 .bss 之后	起点加随机偏移

ASLR 便宜的前提是位置无关代码 (PIC)。若代码里写死绝对地址，换基址就要逐条修补代码；代码页一被写脏，写时复制立刻把共享页拆成私有页，“300 个进程共享一份 libc”就此作废。PIC 的做法是：模块内引用全部改成 rip 相对 (`leaq rdi, [rip+0x0e31]` 之类)，跨模块引用全部走 GOT。换基址因此只剩两步——随机选一个基址，按“基址加偏移”修补那张可写的 GOT——代码页一个字节都不动，继续只读、继续共享。

装载时重定位也正因为集中在一张表里而便宜：表项密度高、缓存友好。一个链接几十个库的 GUI 程序，启动前动态链接器要处理几万条 RELA 记录，每条不过是“基址加符号偏移，写入 GOT[n]”的一次内存写，总量毫秒级。相比之下，若允许对代码段做重定位 (老式的 text relocation)，每个进程都要先把代码页复制再逐条改写——共享、ASLR、启动速度同时受损，现代工具链因此默认拒绝生成代码重定位。

随机化可以被直接观察，也可以被关掉做对照。`setarch -R` 关闭 ASLR 后再跑同一个程序，两次运行的地址逐位相同——这正是“关掉随机化，世界就回到固定地址”的实验：

```
$ setarch -R ./showaddr      $ setarch -R ./showaddr
main = 0x555555555149        main = 0x555555555149
printf = 0x7ffff7d645e0      printf = 0x7ffff7d645e0
&buf = 0x7ffffffffffe2ac     &buf = 0x7ffffffffffe2ac
```

把四件事串成一条链：代码页只读，所以可共享；可共享，所以换基址不能改代码；不改代码，ASLR 的成本只剩选基址和填表。第七章的面包板机是这条链的反面：程序拨在哪个地址就运行在哪个地址，没有随机化、没有重定位、没有共享——不是做不到，而是那台机器上根本没有第二个程序需要防。

经典教材把定址时机分成三档：编译时 (静态链接，地址在链接后不再变)、装载时 (动态链接器映射库并修补重定位表)、运行时 (懒绑定在第一次调用时才解析符号)。本节给三档各配了一个可观察的数字：装载只建映射不读字节，所以 `execve` 快；懒绑定把上千次符号查找改成按需，所以启动快；PIC 把换基址缩成填一张 GOT，所以 ASLR 几乎免费。记住这张时机表，每个机制省的是哪一笔就都有数了。

四、系统调用与异常控制流

程序一旦开始执行，就运行在用户态：能算数、能读写自己的内存，但碰不了设备、改不了页表、造不出新进程——这些事必须请内核代办，请代办的门就是系统调用。本节把系统调用、上下文切换和信号串成一条线：它们都是第四章那张“受控的非顺序 PC 更新”表的实例，与第五章的硬件中断共用同一副骨架——保存现场、按号定向、服务、返回。

读完本节，“操作系统”在硬件眼里将只剩一句话：一个占着最高特权级、替所有进程保管设备的程序。这也是全书硬件视角的收尾——从晶体管到这条边界，每一层都是约定加检查。

syscall 指令触发的受控陷入 用户程序与内核之间有一条硬约定——系统调用 ABI：服务编号放 `rax`，参数依次放 `rdi`、`rsi`、`rdx`、`r10`、`r8`、`r9`，然后执行 `syscall` 指令，返回值放在 `rax`。它与第四章的普通调用约定几乎同形，只有一个反常：第

四个参数用 `r10` 而不是 `rcx`。原因马上会看到——`syscall` 指令本身要占用 `rcx`，参数只能绕道；这样设计的好处是包装层只需挪一个寄存器，其余位置原样沿用。参数至多六个，现实中系统调用的参数个数在设计时就被压在这条线内。

编号就是向量号。x86-64 Linux 的编号表由内核源码发布，一旦公布就永不回收、只增不改——已编译的用户程序把编号烧进了自己的代码，回收一个编号就等于弄瞎一批旧程序，这是 ABI 稳定性的含义。

```
; x86-64 Linux, arch/x86/entry/syscalls/syscall_64.tbl (excerpt)
0   common read
1   common write
2   common open
3   common close
9   common mmap
57  common fork
59  common execve
231 common exit_group
```

一条 `write(1, msg, 13)` 按 ABI 写出来是这样 (Intel 语法):

```
mov eax, 1          ; __NR_write
mov edi, 1          ; fd = stdout
lea rsi, [rip+msg]  ; buf
mov edx, 13         ; count
syscall             ; controlled trap into kernel
; rax = 13 on success, or -errno

msg:   db   "hello, world", 10
```

`syscall` 指令的硬件动作极少而精：把返回地址（下一条指令的 `rip`）装进 `rcx`，把 `rflags` 装进 `r11`，按 `IA32_FMASK` MSR 的掩码清掉 `rflags` 的指定位，特权级从 3 升为 0，最后把 `IA32_LSTAR` MSR 里的内核入口地址装进 `rip`。没有内存访问，没有查表——入口地址早在系统启动时就被内核写进了 MSR。

为什么参数全走寄存器，而不像普通调用那样允许栈传参？因为内核不能信任用户栈指针：`syscall` 那一刻 CPU 还在用户栈上，而这个栈指针本身就可能指向不可写的页。六个寄存器是“内核不用解引用任何用户指针就能拿到的全部输入”；参数个数因此被设计时压在这条线内——历史上 32 位 ABI 只有五个寄存器可用，第六个参数被迫走内存，至今还能在老代码里看到那处别扭。

把普通调用和系统调用并排，ABI 的同形与差异一眼可见：

方面	普通函数调用	系统调用
第四个参数	<code>rcx</code>	<code>r10</code> (<code>rcx</code> 被指令占用)
返回地址	由 <code>call</code> 压到栈上	装进 <code>rcx</code> 寄存器
特权级	不变	3→0，返回时 0→3
使用的栈	同一条用户栈	切到本进程内核栈
入口地址	写死在指令编码里	<code>IA32_LSTAR</code> MSR

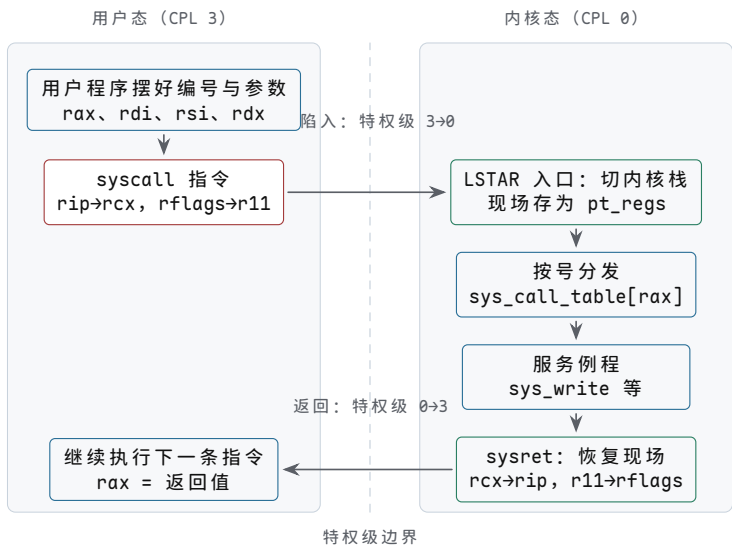
与老式的 `int 0x80` 对照最能看清这副骨架：`int` 要按第四章的“表基址 + 异常号 × 项宽”去内存里查 IDT，慢在一次内存访问，且向量表本身也要保护；`syscall` 把入口搬进 MSR，把“编号到服务”的分发表 `sys_call_table` 留给内核软件去查。但两者只是同一副骨架的两种实现：指令显式请求进入特权服务，硬件保存返回位置、切换权限和入口——第四章异常表里“系统调用”那一行，写的就是它。

骨架阶段	<code>syscall</code> 路径上的动作	对应前章的谁
保存现场	<code>rip</code> → <code>rcx</code> 、 <code>rflags</code> → <code>r11</code> 、切内核栈、建 <code>pt_regs</code>	第四章“保存故障 PC 和原因”

定向	硬件入口 = <code>IA32_LSTAR</code> ；内核再按 <code>sys_call_table[rax]</code> 分发	第四章向量表查表； <code>int 0x80</code> 走 IDT
服务	<code>sys_read/sys_write</code> 等操作设备与文件	第五章 MMIO/DMA 由内核代发
返回	<code>sysret: rcx→rip、r11→rflags</code> 、特权级 0→3	第五章中断返回恢复现场

这副“保存现场、按号定向、服务、返回”的骨架值得背下来，因为本节后面三个主题全是它的实例：上下文切换是“定时器中断加换一套现场”，信号是“返回用户态之前的检查点”，边界清单则是“哪些服务必须在这副骨架里完成”。

把一次系统调用的完整往返按时间自上而下画成两条泳道，特权级的两次切换就是图上两次越过边界的时刻：



图示：一次系统调用往返的泳道图：左为用户态（CPL 3），右为内核态（CPL 0），中间虚线是特权级边界，两次越界即 CPL 3→0 与 0→3 的切换点。`syscall` 指令把返回地址与标志装进 `rcx/r11`、把 `rip` 换成 MSR 中的内核入口；入口代码切到本进程内核栈，把全部通用寄存器存成 `pt_regs`，再按 `rax` 中的编号查 `sys_call_table` 分发；服务完成后 `sysret` 逆向恢复。内核全程不必解引用用户栈指针。

陷入与返回的上下文 `syscall` 指令只切换了最小状态，完整的上下文交接由内核入口代码完成：`swapgs` 换出内核数据基址；从本进程的 TSS 取出内核栈指针——每个进程都有一条独立的内核栈，典型大小 16 KiB，专门承接它陷入内核时的现场；然后把用户态的 `rsp`、`rcx`（用户 `rip`）、`r11`（用户 `rflags`）和全部通用寄存器按固定布局存成一张 `pt_regs`。此后内核 C 代码看到的“寄存器”，其实是这张内存表。返回时逆序恢复，`sysret` 用 `rcx/r11` 还原 `rip/rflags`，并把特权级降回 3。

```
; pt_regs on the per-process kernel stack, high to low address:
; ss, rsp(user), rflags(user), cs, rip(user)    <- saved by CPU
; orig_rax                                     <- syscall number
; rdi rsi rdx rcx rax r8 r9 r10 r11 rbx rbp
; r12 r13 r14 r15                             <- saved by entry code
```

为什么每进程一条内核栈？因为进程可能在内核里被切走：它服务到一半被定时器打断，现场得有个各自安放的地方。内核栈就是该进程“在内核里的工作台”，栈顶指针存在进程控制块里——下一个主题会看到，上下文切换保存的全部东西，最后都归结为这个指针。16 KiB 同时是一条纪律：内核态不能深递归，也不能在栈上放大数组，否则栈溢出踩穿的是自己的现场。

把一次 `read(0, buf, 4096)` 的完整往返走一遍。`strace` 里它只有一行：

```
$ strace -e read ./hello
```

```
read(0, "hello\n", 4096)           = 6
```

第一步，用户态摆好 ABI：`rdi=0`、`rsi=buf`、`rdx=4096`、`rax=0`，执行 `syscall`。到这一步为止，一切和普通函数调用的准备没有区别——区别从下一步开始。

第二步，陷入：特权级升为 0，`rip` 换成 `LSTAR` 入口，入口代码建 `pt_regs`，按 `rax=0` 查 `sys_call_table` 分发到 `sys_read`。注意返回地址此刻在 `rcx` 里、被存进 `pt_regs`，而不在用户栈上——内核从始至终不需要解引用用户栈指针。

第三步，服务：经虚拟文件层到终端驱动，键盘缓冲里已有 6 字节，复制到 `buf`——复制前内核先检查 `buf` 所在页可写，不可写就不复制，直接返回 `-14`。设备寄存器、DMA 描述符这些第五章的对象，只有在这一步、只有内核态才能碰。

第四步，返回：`rax=6`，`sysret` 回用户态，`libc` 包装看到非负值，直接把它作为返回值交出。返回值用 `rax` 不是巧合：编号进内核时用的就是 `rax`，一进一出，同一个寄存器完成交接，调用者连第二个寄存器都不用读。

```
before: rax=0 rdi=0 rsi=0x7fffffffde2a0 rdx=4096
trap:   rcx=rip(next) r11=rflags CPL 3->0 rip=LSTAR
return: rax=6
```

错误约定值得单独说清。内核用 `-errno` 表示失败：`rax` 落在 `[-4095, -1]` 区间即出错，例如 `-14==EFAULT` (`buf` 不可写)、`-9==EBADF` (描述符无效)。判断只需一次无符号比较：`-14` 的无符号形式是 `0xffffffffffffff2`，不小于 `-4095` 的无符号形式 `0xffffffffffffff01`，一条 `cmp` 就分出成败。`glibc` 的包装函数负责翻译：检出该区间就把 `errno` 置为 14、返回 `-1`——C 程序看到的是 `read` 返回 `-1` 且 `errno==EFAULT`，内核内部的 `-14` 被包装层吃掉。

为什么连 `buf` 都要检查？因为内核不能信任用户态递来的任何指针：它可能指向不可写的页，也可能故意指向内核地址。每个系统调用开头的参数检查，是内核自保的第一课，也是“边界”二字在服务一侧的具体形态。

常见错误码可以列成一张小表，下一章读 `strace` 输出时会反复遇到它们：

数值	名字	典型触发场景
-1	EPERM	操作不被允许，如向别人的进程发信号
-2	ENOENT	路径不存在， <code>open</code> 失败
-9	EBADF	描述符无效，如 <code>read</code> 一个已关闭的 <code>fd</code>
-11	EAGAIN	非阻塞读暂时无数据
-14	EFAULT	指针指向不可访问的页

`pt_regs` 里的 `orig_rax` 还藏着一条与信号的衔接：慢系统调用（比如等键盘的 `read`）若被信号打断，服务例程会返回一个内部码 `ERESTART`；处理函数若按 `SA_RESTART` 安装，内核就在 `sigreturn` 后用 `orig_rax` 里的编号把这次调用重新发起，用户态毫无察觉，否则 `libc` 把 `-EINTR` 交给程序，由程序自己决定要不要重试。系统调用能被“重启”，前提正是完整现场都在 `pt_regs` 里。

工程含义是一笔时间账：一次系统调用往返的典型开销在百纳秒级，比普通函数调用贵约两个数量级——贵在特权切换、`pt_regs` 保存、查表分发和参数检查。后世的优化几乎全部围绕“少进几次内核”：`stdio` 缓冲、`readv/writev` 批量提交、`mmap` 免复制、`vDSO` 和 `io_uring`，最后一个主题还会回到这张清单。

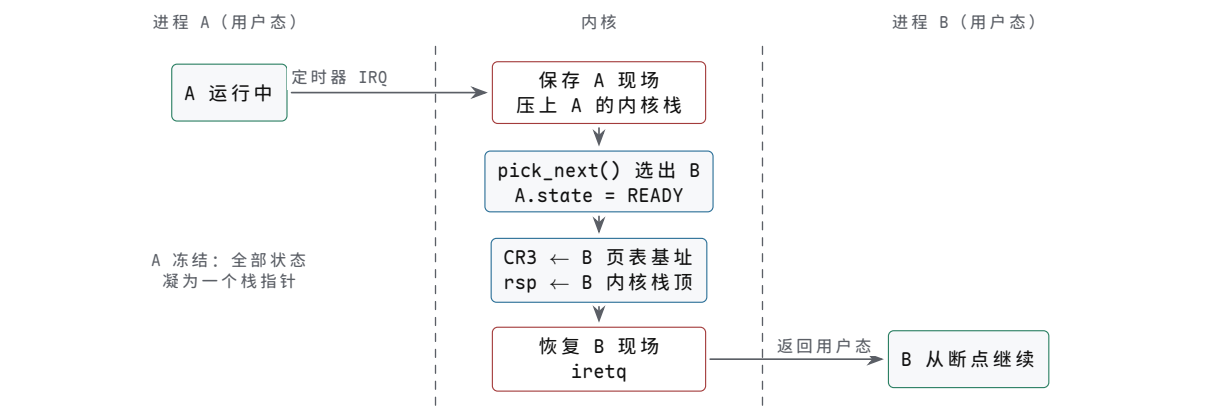
上下文切换的完整步骤 系统调用是程序主动进内核，定时器中断则是内核定期不请自来——走第五章那条链路：定时器拉起 `IRQ`，中断控制器判优，CPU 在指令边界接收，查向量表进服务程序。服务程序发现当前进程的时间片用完，就做一次上下文切换：把整套程序员可见状态换成另一个进程的。完整步骤可以写成一条 `trace`：

```
timer interrupt arrives while A runs
CPU:  push rip/cs/rflags/rsp/ss of A; enter ISR
ISR:  push A's general registers to A's kernel stack
```

```
A.state = READY
B = pick_next()                                ; scheduler decision
if (B.mm != A.mm) CR3 = B.pgd                  ; switch address space
rsp = B.saved_kernel_sp                        ; switch kernel stack
pop B's general registers
iretq                                          ; resume B at its saved rip
```

逐步核对。第一步保存 A：中断已经自动压好 `rip/cs/rflags/rsp/ss`，入口代码补齐通用寄存器—A 的全部可见状态此刻躺在它自己的内核栈上，进程控制块里只要存一个栈顶指针。第二步选下一个：调度器从就绪队列挑出 B——“换谁”是软件策略，“怎么换”才是这里的硬件约定，两者泾渭分明。第三步切地址空间：把 B 的页表基址写进 CR3（第六章：CR3 指向顶级页表）——从这一刻起，同一个虚拟地址 `0x401000` 指向的是 B 的代码；副作用随之而来，TLB 里 A 的翻译全部作废，除非页表项带 PCID 标记，B 恢复后的前几千次访存都会慢一拍。第四步恢复 B 的寄存器并 `iretq`—B 从自己的视角看，只是“上一次陷入返回得特别久”。

把这条 trace 画成泳道图，“换一套现场”就是三条道之间的一次往返：



图示：上下文切换泳道（对应正文 trace）。定时器中断在指令边界把 CPU 从 A 带进内核：A 的现场压上它自己的内核栈，调度器选出 B 后写入 B 的页表基址（CR3）、把栈指针切到 B 的内核栈，再弹出 B 的寄存器并 `iretq`—返回落进 B 的用户态，B 只觉得上次陷入返回得特别久。被切走的 A，全部状态归结为进程控制块里的一个栈指针，等下次被选中时原路恢复。

定时器不是唯一的切换入口：进程阻塞在 `read` 上等键盘、主动 `sched_yield` 让出、等待互斥锁，走的都是同一条“保存、选下一个、恢复”的路径。入口有多个，出口只有一个—沿目标进程的内核栈恢复现场并返回。

要素	保存在哪、切到哪里	检查点
通用寄存器与 <code>rip</code>	各自内核栈顶；进程控制块只存栈指针	恢复后 <code>rip</code> 指向被切走时的下一条
地址空间	CR3 写入新页表基址	同一虚拟地址从此属于新进程
内核栈	栈指针切到目标栈	返回路径沿目标栈展开

算一笔账。保存加恢复的寄存器约 20 个 64 位值，内存流量 $2 \times 20 \times 8 = 320$ 字节，纳秒级；真正的成本在 CR3 切换之后—TLB 和 cache 变冷，使一次进程切换的典型总成本在微秒级，约是一次系统调用往返的十倍。再看频率：设时间片 3 ms，一个核每秒至多切换约 330 次，每次 2 微秒，总开销约 660 微秒，占可用时间的千分之一不到—切换很便宜，而“看起来便宜”是因为时间片把它摊薄了。

有一个边角案例能检验这条 trace 是否真的成立：新进程从来没有“上次被切走”，它的第一次运行从哪来？答案是 `fork` 时内核手工搭出来的—按上一主题的格式伪造一份 `pt_regs` 和内核栈，把 `rip` 设成父进程 `fork` 返回后的下一条，把 `rax` 设成 0。于是子进程第一次被调度时，走的完全是“恢复现场并返回”的老路，只

是现场是造的。同一份代码在父子俩身上跑出两个返回值（父得 pid、子得 0），差别就写在这份手工现场里。

定时器中断接手调度，本身是一段历史进步：早期系统是协作式的，进程靠自觉定期让出，一个死循环就能拖死整机；抢占式调度把主动权收归内核——第五章那个“除了报时还会发起中断”的定时器，另一个身份就是操作系统的心跳。切换的频繁程度可以直接观察：`vmstat` 的 `cs` 列是每秒上下文切换次数，`in` 列是每秒中断次数，两者同量级，因为大部分切换正是中断带进来的。

```
$ vmstat 1 3
procs -----memory----- ---swap-- -----io----- -system-- -----cpu-----
 r  b    swpd    free    buff  cache   si   so    bi    bo    in   cs us  sy id wa st
 1  0        0 6123456 123456 2345678    0    0    10    20   210  512  5  3 92  0  0
 0  0        0 6123300 123456 2345680    0    0     0    12  198  487  4  2 94  0  0
```

TLB 作废的代价也在被硬件压缩：带 PCID 的 CPU 给每条 TLB 项打上地址空间标签，切 CR3 时不必清空 TLB，新旧进程的翻译按标签各归各——这正是第五章 cache 用 tag 区分归属的思想，在地址转换缓存上的重演。

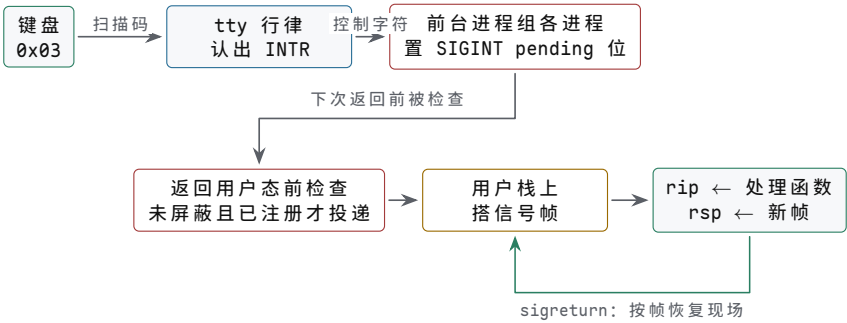
切换入口	触发原因	典型例子
被动切换	定时器中断，时间片用完	计算密集的循环被切走
主动切换	系统调用内阻塞，等资源	<code>read</code> 等键盘输入
让出切换	显式请求调度器换人	<code>sched_yield</code>

两个推论。其一，**进程切换就是把整套可见状态换一遍**：寄存器、地址空间、内核栈，一个不能少；而文件描述符表、信号处理表这类“属于进程但不属于寄存器”的状态挂在进程控制块上，换进程时自然跟着换，不需要逐条保存。其二，同进程的两个线程互切不用写 CR3——它们共享地址空间，TLB 不必作废——这正是线程比进程“轻”的硬件根源。

信号是软件级中断 中断让设备打断 CPU，信号让内核（或另一个进程）打断一个进程。两者同构：信号编号对应中断号，`sigaction` 安装的处理表对应 IDT，`sigprocmask` 对应中断屏蔽字，pending 位图对应中断挂起寄存器，`sigreturn` 对应 `iret`。差别只在发起者和投递点：中断由硬件在指令边界投递，信号由内核在“即将从内核返回用户态”这个固定检查点投递——每个进程从系统调用或中断返回前，都要过这一关。

Ctrl-C 是最好的一生案例，四步走完。第一步，按键：终端把 `0x03` 字节经第五章那条链路送进内核——设备锁存状态、拉起 IRQ、中断控制器判优、CPU 在指令边界接收、键盘服务程序读入扫描码。第二步，行律解释：tty 驱动发现 `0x03` 是控制字符 INTR，不把它放进输入缓冲，而是向前台进程组的每个进程置 SIGINT 的 pending 位。第三步，投递：目标进程下次返回用户态前，内核发现 SIGINT 未被屏蔽且注册过处理函数，就在用户栈上搭一帧信号帧（保存被打断时的 `rip`、`rsp`、`rflags` 和通用寄存器），把 `rip` 改为处理函数入口、`rsp` 指向新帧，然后才执行那次“返回”——返回去的地方已经是处理函数。第四步，收尾：处理函数执行完调用 `sigreturn`，内核按帧恢复现场，进程从被打断处继续；若进程从未注册处理函数，默认动作是终止——内核在投递点直接回收进程。Ctrl-C“杀掉”前台程序，走的就是这完整的四步。

把 Ctrl-C 的四步画成一条投递路径，“软件级中断”五个字就写在图上：



图示：信号投递路径（Ctrl-C 例）。按键字节 0x03 经键盘服务程序送进内核，tty 行律认出控制字符 INTR，不放进输入缓冲，而是向前台进程组每个进程置 SIGINT（2 号）的 pending 位；目标进程下次返回用户态前，内核检出该位未被屏蔽且处理函数已注册，就在用户栈上搭信号帧、把 rip 改为处理函数入口、rsp 指向新帧——那次“返回”落进处理函数；处理函数执行完 sigreturn，内核按帧恢复现场，进程从被打断处继续。

两个信号被故意排除在约定之外：SIGKILL（9）和 SIGSTOP（19）不可捕获、不可屏蔽、不可忽略。理由很硬：若失控进程能把所有信号都拦下，系统就没有回收它的最后手段——kill -9 之所以总是有效，靠的就是这条“保留一扇不可锁的门”的约定。

概念	硬件中断（第四、五章）	信号
编号	中断号/向量号	信号编号（SIGINT=2，SIGSEGV=11）
入口表	IDT：基址 + 号 × 项宽	每进程 sigaction 表
屏蔽	IF 位/中断控制器屏蔽位	sigprocmask 屏蔽字
挂起	IRR 挂起寄存器	pending 位图
返回	iret	sigreturn 系统调用
投递点	指令边界	内核返回用户态之前

常见信号的编号和默认动作可以各看一张表。编号表用 kill -l 就能列出来：

```
$ kill -l (excerpt)
1) SIGHUP    2) SIGINT    9) SIGKILL   11) SIGSEGV
13) SIGPIPE  14) SIGALRM   15) SIGTERM  18) SIGCONT
19) SIGSTOP  20) SIGTSTP
```

默认动作	代表信号	含义
终止	SIGINT、SIGTERM	内核直接回收进程
终止并转储	SIGSEGV、SIGILL	回收，另留下 core 文件供调试
忽略	SIGCHLD、SIGURG	默认什么也不做
停止	SIGSTOP、SIGTSTP	冻结进程，等 SIGCONT 唤醒

Ctrl-C 的四步只换了发起者，就能解释一整族终端控制：Ctrl-Z 发出的是 SIGTSTP，前台进程被“停止”而非终止——它的全部状态原样冻结，shell 把它挂进作业表；之后 fg 或 bg 让它收到 SIGCONT，从冻结处继续。这就是 shell 作业控制的全部硬件基础：同一套信号投递路径，不同的编号与默认动作。

还有两个工程细节补全这幅图。其一，标准信号的 pending 是位图，只能记“有”，记不了“几次”——处理函数运行期间再来的同号信号最多再记一位，这就是“信号不排队、可能合并丢失”的原因；需要计数的场合得用可排队的实时信号。其二，信号帧搭在用户栈上，若程序正因栈溢出而被 SIGSEGV 投递，用户栈已经不可用，处理函数需要一条事先用 sigaltstack 备好的备用栈——又一个“不能信任用户指针”的实例。

同构最漂亮的一处是段错误：第六章那台检查 P/W/U 三个原因位的页错误硬件，在内核判定“故障地址不属于任何映射”之后，并不直接杀死进程，而是向它投递 SIGSEGV；程序崩溃时打印的“Segmentation fault”，就是这枚信号走完上述投

递路径的结果。信号因此不需要任何新硬件——它全部用已有的异常返回路径搭出来，这正是“软件级中断”五个字的含义。

处理函数一侧还有一条对应的工程约束：信号可能在任意指令边界到来，处理函数里只能调用“异步信号安全”的函数，否则可能打断 `malloc` 到一半的链表——这与中断服务程序“快进快出、不碰半成品数据”的纪律同出一源。

最后注意一个归属问题：信号处理表和上一主题的页表一样，是每进程状态，挂在进程控制块上。上下文切换换走地址空间的同时，pending 位图和处理表也一起被换走——信号因此总是投递给“当前这个进程”的语义，而不会串到别的进程身上。

用户态与内核态的边界清单 本章反复出现的那条线，现在可以写到底：边界不是软件风俗，而是特权级检查。页表项里的 U/S 位决定哪些页用户态可碰——第六章页错误三问的第三问——检查由硬件在每次访存时完成，成本为零；`syscall/sysret` 是跨越特权级的唯一受控正门，中断门是另一扇，但钥匙在内核手里。判断一件事在哪一侧，只需问一句：它要不要碰特权状态？

操作	在哪侧	为什么
加法、比较、分支	用户态	只动通用寄存器，不碰特权资源
<code>strlen/memcpy</code> 等库函数	用户态	只读写本进程已映射的页
<code>malloc</code> 小对象	用户态	在已分配的堆区内管理空闲块；堆要长大才 <code>brk/mmap</code> 进内核
<code>read/write/open</code>	内核态	设备寄存器与 DMA 由内核独占（第五章）
<code>fork/execve/exit</code>	内核态	建/销地址空间要改页表与 CR3，属特权操作
<code>mmap/映射变更</code>	内核态	页表页标了 U/S=0，用户态写不了
<code>gettimeofday (vDSO)</code>	用户态	内核把只读时间页映给用户，读它不算越界

清单最后一行值得展开。`gettimeofday` 传统上是系统调用，现代内核把它和校准参数写进一页只读共享页，用户态直接读——一次跨越都省。这是边界工程的一般做法，按优先级排成三条：能不跨就不跨（vDSO）；必须跨就合并（stdio 缓冲、`readv` 批量、`io_uring` 一次提交一组）；非要高频跨越，就干脆修一座桥（共享页、门铃寄存器、用户态轮询）。

把跨越的代价并排摆出来，三条策略的动机就一目了然：

动作	典型量级	边界含义
普通函数调用	纳秒级	不跨边界
系统调用往返	百纳秒级	陷入加返回各一次
进程上下文切换	微秒级	陷入加更换整套可见状态
SSD 读 4 KiB 页	几十微秒	内核加设备加缺页服务

`strace -c` 能把一个程序一生跨越边界的次数统计出来——打印十次 `hello` 的程序，当输出重定向到文件或管道时（全缓冲），`write` 通常只出现一次（stdio 已把十次合并），其余是装载期的 `mmap` 和 `openat`。数一数这些行，就是把本节所有的账再对一遍：

```
$ strace -c ./hello
hello, world
% time      seconds  usecs/call   calls    errors syscall
-----
35.42      0.000017      17          1         0    execve
18.75      0.000009         1          9         0    mmap
16.67      0.000008         2          4         2    openat
12.50      0.000006         6          1         0    write
```

8.33	0.000004	1	3	1 read
4.17	0.000002	1	2	close
4.17	0.000002	2	1	arch_prctl

100.00	0.000048		21	3 total

边界同时是观察边界：`strace` 看不见函数调用，只能看见系统调用，因为陷入点是用户态唯一的“海关”。调试器、审计、沙箱全都挂在这同一个点上一拦住边界，就拦住了一个进程对外能做的全部事情；这也是为什么容器隔离的抓手是“过滤系统调用”，而不是“检查程序代码”。

边界的形态本身也是设计选择。宏内核把文件系统、驱动、协议栈都放在内核态，跨越成本是一次陷入；微内核把它们挪进用户态服务进程，跨越变成进程间消息——边界没有消失，只是挪了位置，而每次跨越的特权检查是物理事实，挪到哪里都要付。`time` 的输出把这笔开销记成两列：`user` 是用户态指令时间，`sys` 是内核替它干活的时间；`sys` 占比高的程序，多半就是边界跨越密集型。

```
$ time ./hello
hello, world
real    0m0.002s
user    0m0.001s
sys     0m0.001s
```

第七章的面板机是对照的另一极：程序直接拨进 RAM，每个 MMIO 地址都随手可碰，没有第二个特权级——那是“程序即内核”。多用户、多进程、要隔离要共享的系统必须有边界，而边界的全部成本，就是每次跨越时那一次受控陷入。

“库函数 = 系统调用”是最常见的混淆。`printf` 不是系统调用：它是用户态函数，把字符攒进缓冲区，只在缓冲满或换行刷新时才发起一次 `write`。用 `strace` 观察一个打印十次 `hello` 的程序：当输出重定向到文件或管道时（全缓冲），看不到十次 `write`——通常只有一次。判断依据不变：只有碰特权资源才需要陷入，拼字符串始终发生在用户态。

CSAPP 把异常控制流分成四类：中断、陷阱、故障、中止。本章的系统调用是陷阱（trap）——主动、同步、返回下一条；定时器中断是中断——被动、异步；缺页是故障——可修复、返回重试；段错误走到尽头是中止。一张表管住四种入口，剩下的全是各入口里的具体约定。

五、从源码到进程的完整案例

前四节把这条链拆成四段讲：目标文件是半成品，链接把名字定成地址，装载把映像请进内存，系统调用是运行中向内核求助的入口。本节把四段接回来，跟踪一个最小的 C 程序的一生——从三行源码，到进程退出、shell 打印出退出码为止。程序故意不用 `printf` 而直接调 `write`，这样它一生向内核提出的请求最少，看得最清楚：

```
#include <unistd.h>

int main(void)
{
    write(1, "hi\n", 3);
    return 42;
}
```


<code>execve</code>	内核建地址空间、映射段、把控制交给动态链接器	§ 三 装载；成功时永不返回
<code>brk/mmap</code>	动态链接器登记堆起点、申请工作区	§ 三 虚拟内存区与按需调页
<code>openat/read</code>	动态链接器查缓存、找到并打开 <code>libc.so.6</code>	§ 三 动态库的查找路径
<code>mmap/mprotect</code>	把 <code>libc</code> 各段映射进来、按段表修正权限	§ 三 段映射与页权限
<code>write</code>	你的 <code>main</code> : <code>rax</code> 为 1, 经 <code>syscall</code> 陷入	§ 四 异常入口与参数约定
<code>exit_group</code>	<code>exit</code> 的最后一站, 注销整个进程	§ 四 与本节的退出路径

这张表里藏着两个反直觉的事实。其一, `execve` 是唯一一个“成功就不回来”的系统调用: 行尾的 `= 0` 发生在一个崭新的地址空间里, 原来那个调用者 (shell 用 `fork` 复制出的副本) 已经被整体替换—`fork` 返回两次, `execve` 一次也不返回, 这对组合正是 shell 启动每条命令的标准动作。其二, `write` 之前那一串 `openat` 与 `mmap` 没有一条是你的代码发出的: 动态链接的可执行文件, 第一条用户态指令属于动态链接器 `ld.so`; 它跑完找库、映射、准备 GOT (§ 三) 之后, 才把控制交给 `_start`, 然后才轮到 `main`。

数字也都能闭合: `write` 返回 3, 即实际写出的字节数; `exit_group` 行尾是 `= ?`, 因为它根本不返回, 下一行 `+++ exited with 42 +++` 是 `strace` 自己补的旁白, 42 正是 `main` 的返回值。从 C 的 `return` 到 shell 的退出码, 整条链在一张表里走完。若改用 `-static` 静态链接重跑一遍, `execve` 与 `write` 之间几乎一片空白—那串调用全是动态链接为“共享库”三个字付的装载期成本, 也是 § 三说的装载时重定位代价里最显眼的部分。

读这张表还有几个顺手的工具用法。`strace -f` 会把 `fork` 出的子进程一并跟踪, shell 管道的每一环都能看到; `strace -c` 不逐行打印, 结束时给一张按耗时排序的系统调用统计表, 先找“哪类调用最多”再回头细看, 是性能排查的常规第一步; `strace -e trace=openat` 这类过滤开关只保留关心的调用, 用来对付输出以万行计的大程序。时间表是本章所有机制的公共观察窗: 装载、动态链接、系统调用、退出清理, 每一节的内容都能在这张表里找到对应的行。

程序退出时内核回收什么 `exit_group` (编号 231) 是进程的终点, 也是看清“进程是什么”的最好窗口。对用户来说, 进程是一个正在运行的程序; 对内核来说, 进程是一组挂在 `task_struct` 上的数据结构: `mm_struct` 描述地址空间 (页表加虚拟内存区链表), `files_struct` 是文件描述符表, 再加信号表、子进程链表、时钟统计。退出, 就是按固定顺序注销这组结构。

严格说, `exit_group` 之前还有一层分工要辨清: `exit` 是 C 库函数, 负责用户态的收尾—按注册的逆序跑 `atexit` 函数、冲洗 `stdio` 缓冲区; `_exit` 才是薄包装, 直接把控制交进内核。差别在缓冲上最明显: 若 `main` 里用的是 `printf` 且输出不含换行, 数据可能还躺在用户态缓冲里, 直接调 `_exit` 会把这些字节丢掉; `exit` 则先把它冲刷出去, 再走 `exit_group`。本节程序用 `write` 不用 `printf`, 也正是为了绕开这层缓冲, 让输出与系统调用一一对应。

地址空间最先退场: `mm_struct` 的引用计数归零后, 页表与虚拟内存区链表被释放, 匿名页和私有映射页回空闲链表; 共享映射 (比如 `libc` 的代码页) 只减一次引用计数, 别的进程还在用就一页不还—这正是 § 三“多个进程共享一份库代码”的镜像。文件描述符表随后: 每个表项对内核文件对象减一次计数, 计数归零才真正关闭; 这解释了 `fork` 之后管道的经典现象—父子各持一份写端, 必须两边都关闭, 读端才看得到 EOF。最后是亲属关系: 退出者的子进程被过继给 1 号进程, 保证每个“孤儿”将来都有人为它收尸。

但 `task_struct` 本身不能立刻删—退出码还没交到父进程手里。于是 `exit_group` 之后进程进入僵尸态: 只剩一条骨架记录 (退出码与资源统计), 不占地址空间,

永不再被调度，等父进程用 `wait` 一族来取。父进程拿到的是 32 位状态字，正常退出时退出码放在第 8 到 15 位：本例的 42 对应原始状态值 42 左移 8 位，即 10752（十六进制 `0x2a00`），宏 `WEXITSTATUS` 做的就是右移 8 位再取低 8 位。若父进程先死，过继来的子进程由 1 号进程负责收取，系统里不会留下无人认领的僵尸；真正会留僵尸的情形只有一种——父进程活着却忘了 `wait`，那是 `ps` 输出里状态列 `Z` 的来历。

”进程结束”因此不是一个硬件事件，而是内核数据结构的注销顺序；僵尸也不是故障，是”退出码必须可靠传给父进程”这条约定的必然代价。判断一个进程是否死透了，等价于问三个问题：地址空间还了吗，描述符表清空了吗，最后那条 `task_struct` 记录有人领走吗。

与第七章裸机路径的对照 把本节的托管路径与第七章的裸机路径并排摆，会看到同一件事——”程序出生”——在两个世界的两种做法。第七章的机器上，链接地址在构建时定死：复位向量的值就是程序的物理地址；装载器就是你自己——烧写器把镜像写进 ROM，或拨码开关逐字节拨进 RAM；复位后的第一条指令就是全部启动代码，没有 `crt`，更没有动态链接器；输出一个字符，是往 I/O 端口或内存映射的显存写一个字节，走第五章的 I/O 路径；”退出”是停机指令或原地跳转，没有内核，自然也没有谁来回收什么。本节的世界相反：链接时地址可以待定，装载由内核 `execve` 完成，`main` 之前已有 `ld.so` 和 `_start` 跑了一整圈，输出要经 `write` 请内核代劳，退出由内核逐项注销。

维度	裸机镜像（第七章）	托管进程（本章）
链接地址	构建时定死的固定地址	相对偏移，基址由内核装载时定
装载者入口	烧写器或拨码开关，即你自己复位向量直指你的第一条指令	内核 <code>execve</code> 加动态链接器入口指向 <code>_start</code> ， <code>main</code> 前还有一层
输出一个字符	直接写 I/O 端口或显存（第五章）	<code>write</code> 系统调用，内核驱动硬件
退出	停机或死循环，无人回收	<code>exit_group</code> ，内核逐项注销数据结构

两个世界的分界线只有一条：有没有内核替你干活。有内核，地址空间是借来的，文件是借来的，碰硬件要经 § 四的异常入口申请；作为交换，装载、符号收尾、设备驱动、退出清理全有人包办。没有内核，全部物理地址都归你，全部中断（第五章）也都冲你来，自由与负担同比例放大。第六章讲的启动链——固件、引导程序、内核接力——正是从前一个世界走进后一个世界的那座桥：引导程序活在裸机世界，它做的最后一件事是把内核请进内存；内核一旦接管，桥的这边就改按托管世界的规矩办。

经典教材把 `fork`、`execve`、`exit` 与信号一并归入”异常控制流”：它们和系统调用一样，都是控制从用户态到内核态的异常转移。本节的 `strace` 输出可以读成这种转移留下的时间表——每一行都是一次让渡与一次归还。把程序的一生读成一张系统调用时间表，是理解进程最不容易过时的方式。

章末练习

任务	要求	学习目标
----	----	------

ELF 节表读法	对 <code>gcc -c</code> 生成的 <code>hello.o</code> 执行 <code>readelf -S</code> , 找到 <code>.text</code> 、 <code>.rodata</code> 、 <code>.bss</code> 三节, 记下各自的类型与大小, 并解释 <code>.bss</code> 为什么在文件里一个字节也不占	分清 <code>PROGBITS</code> 与 <code>NOBITS</code>
重定位演算	一条 <code>call</code> 指令的起始地址为 <code>0x401120</code> , 其 32 位位移字段的地址 <code>P</code> 为 <code>0x401121</code> ; 目标符号地址 <code>S</code> 为 <code>0x401130</code> , 重定位类型 <code>R_X86_64_PC32</code> , 加数 <code>A</code> 为 <code>-4</code> 。算出写入该字段的值, 并用执行时的 <code>rip</code> 验证结果	重定位就是解一次 $S+A-P$
静态库顺序	<code>main.o</code> 用到 <code>libfoo.a</code> 里的函数 <code>foo</code> : 比较 <code>gcc main.o -lfoo</code> 与 <code>gcc -lfoo main.o</code> 两条命令, 指出哪一条会报未定义符号错误, 并解释原因	库按需求出, 顺序敏感
PLT/GOT 走读	程序先后两次调用 <code>write</code> : 分别写出两次调用经过 <code>PLT</code> 桩与 <code>GOT</code> 表项的路径差别, 并指出 <code>GOT</code> 表项在哪一步、被谁改写	第一次解析, 之后间跳直达
<code>syscall</code> 参数	按 x86-64 系统调用约定, 写出 <code>write(1, buf, 3)</code> 陷入内核前 <code>rax</code> 、 <code>rdi</code> 、 <code>rsi</code> 、 <code>rdx</code> 各放什么; 再说明第四个参数为什么用 <code>r10</code> 而不是 <code>rcx</code>	编号走 <code>rax</code> , 参数按固定寄存器序
编译链四阶段	手头只有一个 <code>hello.c</code> : 分别写出加 <code>-E</code> 、 <code>-S</code> 、 <code>-c</code> 与不加开关的四条 <code>gcc</code> 命令的产物文件名与性质, 并说明哪个产物还不能直接执行、缺哪一步	四把开关各停一站

参考解答与要点

任务	参考解答	必须掌握的要点
ELF 节表读法	<code>.text</code> 类型 <code>PROGBITS</code> , 是指令字节本身; <code>.rodata</code> 同为 <code>PROGBITS</code> , 本例装着字符串常量共 3 字节; <code>.bss</code> 类型 <code>NOBITS</code> , 只有大小声明、没有内容—未初始化数据按约定在装载时清零, 文件里存一串零纯属浪费, 所以它只占节表中的一个条目, 不占文件偏移	<code>NOBITS</code> 只声明大小, 装载时补零
重定位演算	写入值等于 $S+A-P$, 即 <code>0x401130</code> 减 4 再减 <code>0x401121</code> , 得 <code>0xb</code> , 按小端写入四字节, 依次为 <code>0b</code> 、 <code>00</code> 、 <code>00</code> 、 <code>00</code> 。执行时 CPU 取完这条 5 字节指令后 <code>rip</code> 为 <code>0x401125</code> (即 $P+4$), 跳转目标等于 <code>0x401125</code> 加 <code>0xb</code> , 得 <code>0x401130</code> , 正是 <code>S</code> , 演算闭合	PC 相对的基准是下一条指令

静态库顺序	<code>gcc -lfoo main.o</code> 报错。链接器从左向右扫描：处理 <code>-lfoo</code> 时未定义符号集合还是空的，库中任何成员都不会被取入；轮到 <code>main.o</code> 才引入未定义的 <code>foo</code> ，可库已经扫过，于是报 <code>undefined reference</code> 。正确写法是把库放在引用它的目标文件之后： <code>gcc main.o -lfoo</code> ——扫到库时 <code>foo</code> 正被需要，定义它的成员才被取出	引用在前、库在后
PLT/GOT 走读	第一次： <code>call</code> 进入 <code>write</code> 的 PLT 桩，桩首条指令经 GOT 表项间接跳转，而此时表项里存的是桩内下一条指令的地址——于是转回桩中，压入重定位序号，再跳公共解析桩；动态链接器算出 <code>write</code> 的真实地址、改写该 GOT 表项，最后才转入 <code>write</code> 本体。第二次：GOT 已是真实地址，桩里那一次间接跳转直达目标，解析路径被完全绕过。改写者是动态链接器的解析例程，时机是第一次调用期间	延迟绑定：第一次付费，以后一跳
<code>syscall</code> 参数	<code>rax</code> 放编号， <code>write</code> 是 1； <code>rdi</code> 放 1（文件描述符）， <code>rsi</code> 放 <code>buf</code> 的地址， <code>rdx</code> 放 3（长度）。 <code>syscall</code> 指令本身把返回地址存进 <code>rcx</code> 、把标志存进 <code>r11</code> ，这两个寄存器被硬件占用，所以内核约定第四个参数改用 <code>r10</code> ；完整顺序是 <code>rdi</code> 、 <code>rsi</code> 、 <code>rdx</code> 、 <code>r10</code> 、 <code>r8</code> 、 <code>r9</code> ，返回值由 <code>rax</code> 带回	<code>rcx</code> 与 <code>r11</code> 被指令占用
编译链四阶段	<code>-E</code> 得 <code>hello.i</code> ：宏与头文件展开后的纯 C 文本； <code>-S</code> 得 <code>hello.s</code> ：x86-64 汇编文本； <code>-c</code> 得 <code>hello.o</code> ：可重定位目标文件，机器码已生成但外部符号只有名字没有地址；不加开关得可执行文件（ <code>-o hello</code> 时名为 <code>hello</code> ，缺省 <code>a.out</code> ）。 <code>hello.o</code> 不能直接执行：缺链接这一步——拼入启动文件、解析 <code>write</code> 、填入口地址，三件事都在链接阶段完成	只有链接产物是给内核装载的

读完后的检查表

读完本书后，至少应该能回答下面这些问题：

主题	能力要求
比特	能说明为什么计算机用两个稳定区间表示 0 和 1，而不是追求连续电压的精确值。
逻辑门	能从开关路径解释 NOT、AND、OR、NAND 的行为，并理解为什么 NAND 可以构造其他门。
组合逻辑	能把真值表转成门电路，解释半加器、全加器、多路选择器和译码器分别解决什么问题。
时序逻辑	能说明锁存器、触发器、寄存器和时钟边界的关系，知道为什么组合逻辑不能保存历史。
状态机	能把状态、输入、转移条件和输出分开，并说明状态机怎样控制一组硬件动作。
ALU	能把加法、减法、按位运算、比较和标志位放在同一个计算部件里解释。
最小 CPU	能画出 PC、指令存储器、寄存器堆、ALU、数据存储器 and 控制器之间的数据流。
整机硬件	能说明主存、总线、MMIO、DMA、中断控制器和复位启动怎样把最小 CPU 扩展成整机。
平台事务	能把一次内存、MMIO、DMA 或中断访问写成发起者、目标、地址、握手、完成、错误和副作用的 trace（执行轨迹）。
存储层次	能区分寄存器、cache、TLB、DRAM、SSD/硬盘的访问粒度、时延、带宽、完成方式和错误路径。
主板与扩展	能说明供电、时钟、固件、DDR、PCIe、NVMe、USB/SATA、网卡和显卡如何组成平台，而不是把整机理解为 CPU 附件。
显卡与显示	能写出 CPU 提交命令、GPU 读取资源、VRAM 写帧、显示引擎扫描输出之间的异步边界。
经典芯片	能用同一套 trace 方法解释 8086 的复用总线、80386 的地址转换、PC 平台分层和现代 SoC 的片上集成。

最小整机毕业设计

如果要把本书内容真正实现在一块教学板、FPGA 工程或仿真模型上，最后不要只满足于“程序能跑”的结论，而要保留一组可复查的实验记录。下面这条路线把全书链路压缩成一个最小整机：复位后从 ROM 取指，能执行 ADD、MOV、JZ，能访问 SRAM，能通过 MMIO 读按键并写 LED，能用中断或轮询处理一个外部事件。每一步都应能单步观察，失败时能定位到具体环节。

阶段	预期行为	观察点
电源/时钟/复位	电平稳定、时钟边沿可用、复位期间危险输出关闭	示波器截图或仿真波形，复位释放前写使能无效
取指	PC 指向复位入口，ROM 被正确片选，IR 锁存第一条编码	PC、地址线、ROM CS、数据线、IR 写使能
单条算术	寄存器读、ALU、标志位和写回边界正确	ADD R3, R1, R2 的源值、ALUOut、目标寄存器写入

访存	地址形成、SRAM 读写、字节使能和等待状态正确	地址、读写控制、MDR、写数据、ready/wait
分支	Zero 或比较结果能选择下一 PC	顺序 PC 和分支目标各跑一次，错误路径不提交
MMIO	普通 RAM 和设备寄存器不会混淆，副作用可观察	GPIO 地址片选、LED 锁存器写窗口、按键同步后的输入值
外部事件	轮询或中断能把设备状态带回 CPU	状态位锁存、清除顺序、服务程序入口和返回
错误路径	未映射地址、慢设备或设备错误有固定行为	超时、错误位、异常入口或固定返回值，不允许总线悬空

实验报告应包含的材料

高质量硬件报告不靠形容词，而靠可复查的记录。至少应包含下面几类材料：

材料	内容	常见不足
地址地图	ROM、RAM、MMIO、未映射区和复位入口	只有文字说明，没有片选表达式或互斥证明
指令 trace	每条核心指令的控制线、数据路径和提交边界	只列汇编程序，不说明硬件何时写状态
时序波形	取指、SRAM 读、SRAM 写、MMIO 写、中断或 DMA 完成	只截最终 LED 状态，没有关键控制信号
故障注入	故意访问未映射地址、制造慢 ready、触发错误状态	只验证成功路径，失败路径不可解释
可见性规则	DMA 或设备事件中 CPU 与设备何时拥有缓冲区和状态位	completion 到达前后 cache/缓冲区规则不清楚
复位记录	复位期间和释放后前几拍的 PC、片选、写使能	复位后第一条指令来源不明

排错入口

调试硬件时，不要从“代码可能写错了”开始。先把症状定位到具体环节，再选择观察信号。

症状	优先观察	常见根因
复位后无取指	复位、时钟、PC、ROM 片选、数据线	复位未释放、复位入口未映射、ROM 输出使能错误
第一条指令错误	地址锁存、ROM 内容、字节序、IR 写使能	地址线接错、程序镜像偏移、取指采样过早
算术偶发错误	源寄存器、ALU 输入、标志位、写回 mux	控制字位错误、组合路径太长、写回目标错
取数读到旧值	地址、片选、OE/WE、ready、MDR	读采样早于数据有效，或片选命中错误目标
存数破坏相邻数据	字节使能、写使能宽度、地址低位	字节车道选择错误或写窗口过宽
LED 或设备乱动作	MMIO 片选、写使能、设备状态机输入	地址译码过宽，普通内存写误触发设备
中断丢失或重复	设备状态、IRQ、控制器屏蔽、EOI、清除位	清除顺序错误，新旧事件没有分离
DMA 后数据不对	描述符、缓冲区地址、cache 可见性、completion	描述符对设备不可见，CPU 过早重用缓冲区

SSD 读请求卡住	submission queue、doorbell、控制器状态、completion queue、中断	把门铃写返回误认为块 I/O 完成，或 completion 未被消费
显卡画面不更新	命令缓冲、资源可见性、VRAM、fence、显示扫描	渲染完成、帧缓冲翻页和显示扫描三个边界混淆
PCIe 设备消失	链路训练、配置空间、BAR、MSI/AER、电源状态	平台枚举或链路错误被误认为驱动逻辑错误

最终检查不以“能不能复述章节标题”为准，而以能不能说清楚机器的实际行为为准。

- 给一个信号节点，能说出它何时被拉高、何时被拉低、何时悬空、何时冲突。
- 给一个组合电路，能列真值表、最长路径、可能毛刺和后级采样边界。
- 给一个状态电路，能指出旧状态、下一状态、时钟沿、建立/保持约束和复位行为。
- 给一个算术结果，能分别按无符号和补码解释，并说明 Carry 与 Overflow 的差别。
- 给一条最小 CPU 指令，能写出控制信号 trace、数据通路 trace 和 PC 下一值。
- 给一次内存或设备访问，能写出地址译码、总线/互连事务、响应返回和完成状态。
- 给一次 DMA 或中断，能写出缓冲区所有权、可见性、状态清除和完成通知顺序。

如果这些内容都能写出来，说明读者已经把“电平 -> 门 -> 组合逻辑 -> 状态 -> 数据通路 -> 控制器 -> CPU -> 整机”的链路连成了一台机器。